

MScProject

Program, Test, Data and Result Listings

Tanatswa

21 August 2026

Purpose and Scope

This PDF is the readable program-listing submission for the MScProject on evidence-grounded table-to-text generation. Its source repository is available at github.com/tboysavage/MScProject. The document includes maintained programs, automated tests, reproducibility configuration, all 25 protected-holdout inputs, all 25 paired system/baseline outputs, and principal evaluation tables. The final manifest supplies file-level SHA-256 digests for the exact listed snapshot.

The listings exclude credentials, local environments, downloaded datasets, caches, transient run directories and duplicated frozen source snapshots. Jupyter notebooks are represented by their maintained Python companion programs because raw notebook JSON is not readable as a program listing.

Category	Files	Source lines
Programs	46	49,102
Tests	4	12,395
Configuration	14	–
Protected data	25	25 examples
Evaluation results	6	25 paired outputs plus tables

Listing Labels

P Program or executable research-support module.

T Automated test program.

C Reproducibility configuration.

D Protected-holdout data.

R Evaluation result.

M Machine-readable inclusion manifest.

Contents

1	Program Listings	4
2	Test Program Listings	548
3	Configuration Listings	685
4	Protected Data Listings	702
5	Evaluation Result Listings	799
6	Listing Manifest	818

Listings

1	P01: experiments/llm_only_pipeline/src/table2text_llm_only/_init_.py	4
2	P02: experiments/llm_only_pipeline/src/table2text_llm_only/backend.py	4
3	P03: experiments/llm_only_pipeline/src/table2text_llm_only/client.py	4
4	P04: experiments/llm_only_pipeline/src/table2text_llm_only/schemas.py	7
5	P05: experiments/llm_only_pipeline/src/table2text_llm_only/workflow.py	10
6	P06: submission/program_listings/build_program_listings.py	17
7	P07: submission/software_archive/build_software_submission.py	23
8	P08: submission/software_archive/examples/run_deterministic_demo.sh	27
9	P09: table2text_pydanticai/evaluation/notebooks/build_dissertation_visual_evaluations.py	27
10	P10: table2text_pydanticai/evaluation/notebooks/build_protected_holdout_notebook.py	36
11	P11: table2text_pydanticai/evaluation/notebooks/protected_holdout_full_system_evaluation.py	37
12	P12: table2text_pydanticai/evaluation/notebooks/task_aware_direct_baseline_evaluation.py	57
13	P13: table2text_pydanticai/evaluation/scripts/annotations/build_interactive_gpt56_annotation_artifacts.py	70
14	P14: table2text_pydanticai/evaluation/scripts/build_protected_holdout_results_bank.py	76
15	P15: table2text_pydanticai/evaluation/scripts/figures/build_evaluation_design_flow.py	89
16	P16: table2text_pydanticai/evaluation/scripts/figures/build_gpt56_sol_annotation_figure.py	93
17	P17: table2text_pydanticai/evaluation/scripts/reports/build_task_aware_complete_evaluation_dossier.py	97
18	P18: table2text_pydanticai/src/table2text/_init_.py	108
19	P19: table2text_pydanticai/src/table2text/_main_.py	108
20	P20: table2text_pydanticai/src/table2text/agents.py	109
21	P21: table2text_pydanticai/src/table2text/analytics.py	160
22	P22: table2text_pydanticai/src/table2text/audit.py	215
23	P23: table2text_pydanticai/src/table2text/capabilities.py	324
24	P24: table2text_pydanticai/src/table2text/cli.py	385
25	P25: table2text_pydanticai/src/table2text/config.py	387
26	P26: table2text_pydanticai/src/table2text/data.py	391
27	P27: table2text_pydanticai/src/table2text/evaluation/_init_.py	402
28	P28: table2text_pydanticai/src/table2text/evaluation/alignscore_client.py	403
29	P29: table2text_pydanticai/src/table2text/evaluation/cli.py	404
30	P30: table2text_pydanticai/src/table2text/evaluation/datasets.py	408
31	P31: table2text_pydanticai/src/table2text/evaluation/deepeval_metrics.py	418
32	P32: table2text_pydanticai/src/table2text/evaluation/diagnostics.py	423
33	P33: table2text_pydanticai/src/table2text/evaluation/external_factuality.py	425
34	P34: table2text_pydanticai/src/table2text/evaluation/generation.py	428
35	P35: table2text_pydanticai/src/table2text/evaluation/human_evaluation.py	437
36	P36: table2text_pydanticai/src/table2text/evaluation/llm_judge_annotations.py	439
37	P37: table2text_pydanticai/src/table2text/evaluation/models.py	443
38	P38: table2text_pydanticai/src/table2text/evaluation/notebook.py	447
39	P39: table2text_pydanticai/src/table2text/evaluation/reference_metrics.py	451
40	P40: table2text_pydanticai/src/table2text/evaluation/statistics.py	463
41	P41: table2text_pydanticai/src/table2text/evaluation_backends.py	466
42	P42: table2text_pydanticai/src/table2text/narrative.py	469
43	P43: table2text_pydanticai/src/table2text/schemas.py	473
44	P44: table2text_pydanticai/src/table2text/structure.py	488
45	P45: table2text_pydanticai/src/table2text/task_contracts.py	495
46	P46: table2text_pydanticai/src/table2text/workflow.py	500
47	T01: experiments/llm_only_pipeline/tests/test_workflow.py	548
48	T02: table2text_pydanticai/tests/test_reference_evaluation.py	549
49	T03: table2text_pydanticai/tests/test_semantic_event_pipeline.py	572
50	T04: table2text_pydanticai/tests/test_smoke.py	622
51	C01: experiments/llm_only_pipeline/.env.example	685
52	C02: experiments/llm_only_pipeline/config/variants.json	685
53	C03: experiments/llm_only_pipeline/pyproject.toml	685
54	C04: table2text_pydanticai/.env.example	686
55	C05: table2text_pydanticai/evaluation/config/datasets.json	687
56	C06: table2text_pydanticai/evaluation/config/metrics.json	694
57	C07: table2text_pydanticai/evaluation/config/metrics_reference_similarity.json	695
58	C08: table2text_pydanticai/evaluation/config/metrics_source_grounded.json	696
59	C09: table2text_pydanticai/evaluation/config/variants.json	696
60	C10: table2text_pydanticai/evaluation/config/variants_ablation.json	697

61	C11: table2text_pydanticai/evaluation/config/variants_raw_deepseek_v4_flash.json	698
62	C12: table2text_pydanticai/evaluation/config/variants_raw_deepseek_v4_pro.json	699
63	C13: table2text_pydanticai/evaluation/protected_holdout_full_system/config/protected_full_system_flash.json	699
64	C14: table2text_pydanticai/pyproject.toml	700
65	D01: submission/program_listings/build/generated/data/dart__dart-test-1791.json	702
66	D02: submission/program_listings/build/generated/data/dart__dart-test-1805.json	702
67	D03: submission/program_listings/build/generated/data/dart__dart-test-1828.json	703
68	D04: submission/program_listings/build/generated/data/dart__dart-test-2278.json	703
69	D05: submission/program_listings/build/generated/data/dart__dart-test-4597.json	704
70	D06: submission/program_listings/build/generated/data/e2e_nlg__e2e_nlg-test-1330.json	705
71	D07: submission/program_listings/build/generated/data/e2e_nlg__e2e_nlg-test-209.json	706
72	D08: submission/program_listings/build/generated/data/e2e_nlg__e2e_nlg-test-447.json	706
73	D09: submission/program_listings/build/generated/data/e2e_nlg__e2e_nlg-test-476.json	707
74	D10: submission/program_listings/build/generated/data/e2e_nlg__e2e_nlg-test-864.json	707
75	D11: submission/program_listings/build/generated/data/sportsett_basketball__5130.json	708
76	D12: submission/program_listings/build/generated/data/sportsett_basketball__5372.json	720
77	D13: submission/program_listings/build/generated/data/sportsett_basketball__5786.json	732
78	D14: submission/program_listings/build/generated/data/sportsett_basketball__5955.json	744
79	D15: submission/program_listings/build/generated/data/sportsett_basketball__6127.json	756
80	D16: submission/program_listings/build/generated/data/totto__totto-validation-1828.json	767
81	D17: submission/program_listings/build/generated/data/totto__totto-validation-4467.json	777
82	D18: submission/program_listings/build/generated/data/totto__totto-validation-6067.json	783
83	D19: submission/program_listings/build/generated/data/totto__totto-validation-839.json	786
84	D20: submission/program_listings/build/generated/data/totto__totto-validation-912.json	788
85	D21: submission/program_listings/build/generated/data/web_nlg__web_nlg_en-test-1209.json	796
86	D22: submission/program_listings/build/generated/data/web_nlg__web_nlg_en-test-1330.json	797
87	D23: submission/program_listings/build/generated/data/web_nlg__web_nlg_en-test-1466.json	797
88	D24: submission/program_listings/build/generated/data/web_nlg__web_nlg_en-test-859.json	797
89	D25: submission/program_listings/build/generated/data/web_nlg__web_nlg_en-test-864.json	798
90	R01: submission/program_listings/build/generated/protected_holdout_25_paired_outputs.json	799
91	R02: submission/program_listings/build/generated/results/protected_generation_summary.csv	806
92	R03: submission/program_listings/build/generated/results/automatic_metrics_overall.csv	811
93	R04: submission/program_listings/build/generated/results/automatic_metrics_by_dataset.csv	811
94	R05: submission/program_listings/build/generated/results/gpt56_annotation_summary.csv	812
95	R06: submission/program_listings/build/generated/results/gpt56_category_counts.csv	813
96	M01: Program-listing inclusion manifest	818

1 Program Listings

Maintained runtime, evaluation, notebook-companion and experimental programs.

Listing 1: P01: experiments/llm_only_pipeline/src/table2text_llm_only/__init__.py

```
1 """Isolated LLM-only multi-agent Table2Text experiment."""
2
3 from .backend import llm_only_multi_agent
4 from .workflow import LLMOnlyWorkflow
5
6 __all__ = ["LLMOnlyWorkflow", "llm_only_multi_agent"]
```

Listing 2: P02: experiments/llm_only_pipeline/src/table2text_llm_only/backend.py

```
1 from __future__ import annotations
2
3 import os
4 from pathlib import Path
5 from typing import Any
6
7 from table2text.evaluation.models import BenchmarkExample, VariantConfig
8
9 from .workflow import LLMOnlyWorkflow, write_result_artifact
10
11
12 def _environment_value(name: str) -> str | None:
13     value = os.getenv(name)
14     return value.strip() if value and value.strip() else None
15
16
17 def llm_only_multi_agent(
18     *,
19     example: BenchmarkExample,
20     variant: VariantConfig,
21     repetition: int,
22     seed: int,
23 ) -> dict[str, Any]:
24     """Run the LLM-only workflow through the main evaluator's callable interface."""
25     settings = dict(variant.settings_overrides)
26     workflow = LLMOnlyWorkflow.from_settings(settings)
27     result = workflow.run(example)
28
29     artifact_dir = Path(
30         str(
31             settings.get("llm_only_artifact_dir")
32             or _environment_value("T2T_LLM_ONLY_ARTIFACT_DIR")
33             or "artifacts/runs"
34         )
35     )
36     result = write_result_artifact(
37         result=result,
38         example=example,
39         variant_id=variant.variant_id,
40         repetition=repetition,
41         seed=seed,
42         artifact_dir=artifact_dir,
43     )
44
45     prompt_tokens = sum(trace.usage.prompt_tokens or 0 for trace in result.traces)
46     completion_tokens = sum(
47         trace.usage.completion_tokens or 0 for trace in result.traces
48     )
49     total_tokens = sum(trace.usage.total_tokens or 0 for trace in result.traces)
50     return {
51         "generated_text": result.generated_text,
52         "baseline_type": "llm_only_multi_agent",
53         "agent_count": len(result.traces),
54         "artifact_path": result.artifact_path,
55         "candidate_claim_count": len(result.candidate_claims.claims),
56         "accepted_claim_count": len(result.adjudicated_claims.accepted_claims),
57         "rejected_claim_count": len(result.adjudicated_claims.rejected_claims),
58         "audit_decision": result.final_audit.decision,
59         "audit_support_rate": result.final_audit.support_rate,
60         "repair_attempted": result.repaired_draft is not None,
61         "model": workflow.client.config.model,
62         "base_url": workflow.client.config.base_url,
63         "prompt_tokens": prompt_tokens or None,
64         "completion_tokens": completion_tokens or None,
65         "total_tokens": total_tokens or None,
66     }
```

Listing 3: P03: experiments/llm_only_pipeline/src/table2text_llm_only/client.py

```
1 from __future__ import annotations
```



```

2
3 import json
4 import os
5 import re
6 import time
7 from dataclasses import dataclass
8 from typing import Any
9
10 from table2text.config import load_env_files
11
12 from .schemas import AgentCallTrace, UsageSummary
13
14
15 def _first_env(*names: str) -> str | None:
16     for name in names:
17         value = os.getenv(name)
18         if value is not None and value.strip():
19             return value.strip()
20     return None
21
22
23 def _as_int(value: Any, default: int) -> int:
24     if value is None:
25         return default
26     return int(str(value).replace("_", ""))
27
28
29 def _as_float(value: Any, default: float) -> float:
30     if value is None:
31         return default
32     return float(str(value))
33
34
35 def _strip_json_fence(text: str) -> str:
36     stripped = text.strip()
37     fenced = re.match(r"^````(?:json)?s*(.*)s*````$", stripped, flags=re.DOTALL)
38     return fenced.group(1).strip() if fenced else stripped
39
40
41 def parse_json_object(text: str) -> dict[str, Any]:
42     stripped = _strip_json_fence(text)
43     try:
44         value = json.loads(stripped)
45     except json.JSONDecodeError:
46         start = stripped.find("{")
47         end = stripped.rfind("}")
48         if start < 0 or end <= start:
49             raise
50         value = json.loads(stripped[start : end + 1])
51     if not isinstance(value, dict):
52         raise ValueError("Expected the model to return a JSON object.")
53     return value
54
55
56 @dataclass(frozen=True)
57 class LLMOnlyClientConfig:
58     model: str = "deepseek-v4-flash"
59     base_url: str = "https://api.deepseek.com"
60     api_key_env: str = "DEEPSEEK_API_KEY"
61     max_output_tokens: int = 6000
62     temperature: float = 0.1
63     response_format_json: bool = True
64     json_repair_attempts: int = 1
65
66     @classmethod
67     def from_mapping(cls, values: dict[str, Any]) -> "LLMOnlyClientConfig":
68         load_env_files()
69         model = str(
70             values.get("llm_only_model")
71             or _first_env("T2T_LLM_ONLY_MODEL")
72             or cls.model
73         )
74         if model.startswith("deepseek:"):
75             model = model.split(":", 1)[1]
76         return cls(
77             model=model,
78             base_url=str(
79                 values.get("llm_only_base_url")
80                 or _first_env("T2T_LLM_ONLY_BASE_URL", "DEEPSEEK_BASE_URL")
81                 or cls.base_url
82             ),
83             api_key_env=str(
84                 values.get("llm_only_api_key_env")
85                 or _first_env("T2T_LLM_ONLY_API_KEY_ENV")
86                 or cls.api_key_env
87             ),
88             max_output_tokens=_as_int(
89                 values.get("llm_only_max_output_tokens")
90                 or _first_env("T2T_LLM_ONLY_MAX_OUTPUT_TOKENS"),
91                 cls.max_output_tokens,
92             ),

```

```

93         temperature=_as_float(
94             values.get("llm_only_temperature")
95             or _first_env("T2T_LLM_ONLY_TEMPERATURE"),
96             cls.temperature,
97         ),
98         response_format_json=_(
99             values.get("llm_only_response_format_json")
100             or _first_env("T2T_LLM_ONLY_RESPONSE_FORMAT_JSON")
101             or "true"
102         ).strip().lower()
103         in {"1", "true", "yes", "on"},
104         json_repair_attempts=_as_int(
105             values.get("llm_only_json_repair_attempts")
106             or _first_env("T2T_LLM_ONLY_JSON_REPAIR_ATTEMPTS"),
107             cls.json_repair_attempts,
108         ),
109     )
110
111
112 class LLMOnlyClient:
113     def __init__(self, config: LLMOnlyClientConfig):
114         self.config = config
115         api_key = _first_env(config.api_key_env)
116         if api_key is None:
117             raise RuntimeError(
118                 f"{config.api_key_env} is required to run the LLM-only workflow."
119             )
120         try:
121             from openai import OpenAI
122         except ImportError as exc:
123             raise RuntimeError(
124                 "The openai package is required to run the LLM-only workflow."
125             ) from exc
126         self._client = OpenAI(api_key=api_key, base_url=config.base_url)
127
128     def json_call(
129         self,
130         *,
131         stage: str,
132         system: str,
133         user: str,
134     ) -> tuple[dict[str, Any], AgentCallTrace]:
135         started = time.perf_counter()
136         text, usage = self._chat_json_mode(system=system, user=user)
137         try:
138             parsed = parse_json_object(text)
139         except json.JSONDecodeError as error:
140             if self.config.json_repair_attempts <= 0:
141                 raise RuntimeError(
142                     f"{stage} returned invalid JSON: {error}. "
143                     f"Response excerpt: {text[:1000]}"
144                 ) from error
145             repaired_text, repair_usage = self._chat_json_mode(
146                 system=(
147                     "You repair malformed JSON. Return only one valid JSON "
148                     "object. Do not add commentary, markdown fences, or fields "
149                     "that were not implied by the malformed JSON."
150                 ),
151                 user=(
152                     "The previous model response was intended to be JSON but "
153                     f"failed with this parser error: {error}\n\n"
154                     "Repair it into valid JSON. Preserve all complete content "
155                     "that can be safely recovered. If a string or array was "
156                     "truncated, close it conservatively rather than inventing "
157                     "missing details.\n\nMalformed response:\n"
158                     + text[:12000]
159                 ),
160             ),
161             try:
162                 parsed = parse_json_object(repaired_text)
163             except json.JSONDecodeError as repair_error:
164                 raise RuntimeError(
165                     f"{stage} returned invalid JSON, and repair failed: "
166                     f"{repair_error}. Original excerpt: {text[:1000]}"
167                 ) from repair_error
168             usage = UsageSummary(
169                 prompt_tokens=(usage.prompt_tokens or 0)
170                 + (repair_usage.prompt_tokens or 0)
171                 or None,
172                 completion_tokens=(usage.completion_tokens or 0)
173                 + (repair_usage.completion_tokens or 0)
174                 or None,
175                 total_tokens=(usage.total_tokens or 0)
176                 + (repair_usage.total_tokens or 0)
177                 or None,
178             )
179
180         trace = AgentCallTrace(
181             stage=stage,
182             model=self.config.model,
183             elapsed_seconds=time.perf_counter() - started,

```

```

184         usage=usage,
185     )
186     return parsed, trace
187
188 def _chat_json_mode(
189     self,
190     *,
191     system: str,
192     user: str,
193 ) -> tuple[str, UsageSummary]:
194     kwargs: dict[str, Any] = {}
195     if self.config.response_format_json:
196         kwargs["response_format"] = {"type": "json_object"}
197     response = self._client.chat.completions.create(
198         model=self.config.model,
199         messages=[
200             {"role": "system", "content": system},
201             {"role": "user", "content": user},
202         ],
203         temperature=self.config.temperature,
204         max_tokens=self.config.max_output_tokens,
205         **kwargs,
206     )
207     text = response.choices[0].message.content or ""
208     usage = getattr(response, "usage", None)
209     return (
210         text,
211         UsageSummary(
212             prompt_tokens=getattr(usage, "prompt_tokens", None),
213             completion_tokens=getattr(usage, "completion_tokens", None),
214             total_tokens=getattr(usage, "total_tokens", None),
215         ),
216     )

```

Listing 4: P04: experiments/llm_only_pipeline/src/table2text_llm_only/schemas.py

```

1  from __future__ import annotations
2
3  from typing import Literal
4
5  from pydantic import BaseModel, ConfigDict, Field, field_validator
6
7
8  class StrictModel(BaseModel):
9      model_config = ConfigDict(extra="forbid", validate_assignment=True)
10
11
12  def _string_list(value: object) -> list[str]:
13      if value is None:
14          return []
15      if isinstance(value, list | tuple | set):
16          return [str(item) for item in value if item is not None]
17      return [str(value)]
18
19
20  def _string_value(value: object) -> str:
21      if value is None:
22          return ""
23      if isinstance(value, list | tuple | set):
24          return "; ".join(str(item) for item in value if item is not None)
25      return str(value)
26
27
28  def _optional_int(value: object) -> int | None:
29      if value is None:
30          return None
31      if isinstance(value, int):
32          return value
33      if isinstance(value, float):
34          return int(value)
35      raw = str(value).strip()
36      if not raw or raw.casefold() in {"none", "null", "all", "n/a", "na"}:
37          return None
38      try:
39          return int(raw)
40      except ValueError:
41          return None
42
43
44  class UsageSummary(StrictModel):
45      prompt_tokens: int | None = None
46      completion_tokens: int | None = None
47      total_tokens: int | None = None
48
49
50  class AgentCallTrace(StrictModel):
51      stage: str
52      model: str
53      elapsed_seconds: float

```

```

54     usage: UsageSummary = Field(default_factory=UsageSummary)
55
56
57 class SourceInterpretation(StrictModel):
58     task_understanding: str
59     source_units: str
60     important_entities: list[str] = Field(default_factory=list)
61     important_fields: list[str] = Field(default_factory=list)
62     allowed_claim_types: list[str] = Field(default_factory=list)
63     risks: list[str] = Field(default_factory=list)
64
65     @field_validator("task_understanding", "source_units", mode="before")
66     @classmethod
67     def normalise_text(cls, value: object) -> str:
68         return _string_value(value)
69
70     @field_validator(
71         "important_entities",
72         "important_fields",
73         "allowed_claim_types",
74         "risks",
75         mode="before",
76     )
77     @classmethod
78     def normalise_lists(cls, value: object) -> list[str]:
79         return _string_list(value)
80
81
82 class LLMClaim(StrictModel):
83     claim_id: str
84     claim: str
85     claim_type: str = "source_fact"
86     source_refs: list[str] = Field(default_factory=list)
87     copied_values: list[str] = Field(default_factory=list)
88     calculation_or_reasoning: str = ""
89     confidence: float = Field(default=0.5, ge=0.0, le=1.0)
90
91     @field_validator("calculation_or_reasoning", mode="before")
92     @classmethod
93     def normalise_reasoning(cls, value: object) -> str:
94         if value is None:
95             return ""
96         return str(value)
97
98     @field_validator("source_refs", "copied_values", mode="before")
99     @classmethod
100     def normalise_lists(cls, value: object) -> list[str]:
101         return _string_list(value)
102
103     @field_validator("confidence", mode="before")
104     @classmethod
105     def normalise_confidence(cls, value: object) -> float:
106         if value is None:
107             return 0.5
108         if isinstance(value, int | float):
109             raw = float(value)
110             return raw / 100.0 if raw > 1.0 else raw
111         label = str(value).strip().casefold()
112         labels = {
113             "very high": 0.95,
114             "high": 0.9,
115             "medium": 0.65,
116             "moderate": 0.65,
117             "low": 0.35,
118             "very low": 0.2,
119         }
120         if label in labels:
121             return labels[label]
122         try:
123             raw = float(label.rstrip("%"))
124         except ValueError:
125             return 0.5
126         return raw / 100.0 if raw > 1.0 else raw
127
128
129 class ClaimSet(StrictModel):
130     claims: list[LLMClaim] = Field(default_factory=list)
131     analysis_notes: list[str] = Field(default_factory=list)
132
133     @field_validator("analysis_notes", mode="before")
134     @classmethod
135     def normalise_notes(cls, value: object) -> list[str]:
136         return _string_list(value)
137
138
139 class ClaimReview(StrictModel):
140     claim_id: str
141     status: Literal[
142         "supported",
143         "unsupported",
144         "overstated",

```

```

145         "wrong_number",
146         "wrong_entity",
147         "wrong_relation",
148         "needs_caveat",
149     ]
150     reason: str
151     corrected_claim: str | None = None
152
153
154 class ClaimCritique(StrictModel):
155     reviews: list[ClaimReview] = Field(default_factory=list)
156     global_warnings: list[str] = Field(default_factory=list)
157
158     @field_validator("global_warnings", mode="before")
159     @classmethod
160     def normalise_warnings(cls, value: object) -> list[str]:
161         return _string_list(value)
162
163
164 class RejectedClaim(StrictModel):
165     claim_id: str
166     claim: str
167     reason: str
168
169
170 class AdjudicatedClaims(StrictModel):
171     accepted_claims: list[LLMClaim] = Field(default_factory=list)
172     rejected_claims: list[RejectedClaim] = Field(default_factory=list)
173     warnings: list[str] = Field(default_factory=list)
174
175     @field_validator("warnings", mode="before")
176     @classmethod
177     def normalise_warnings(cls, value: object) -> list[str]:
178         return _string_list(value)
179
180
181 class WriterSentence(StrictModel):
182     text: str
183     claim_ids: list[str] = Field(default_factory=list)
184
185     @field_validator("claim_ids", mode="before")
186     @classmethod
187     def normalise_claim_ids(cls, value: object) -> list[str]:
188         return _string_list(value)
189
190
191 class WriterDraft(StrictModel):
192     title: str | None = None
193     sentences: list[WriterSentence] = Field(default_factory=list)
194
195
196 class AuditFinding(StrictModel):
197     sentence_index: int | None = None
198     severity: Literal["minor", "major", "critical"]
199     issue: str
200     suggested_action: Literal["delete", "weaken", "replace"]
201
202     @field_validator("sentence_index", mode="before")
203     @classmethod
204     def normalise_sentence_index(cls, value: object) -> int | None:
205         return _optional_int(value)
206
207     @field_validator("severity", mode="before")
208     @classmethod
209     def normalise_severity(cls, value: object) -> str:
210         raw = _string_value(value).strip().casefold()
211         if raw in {"blocking", "block", "blocked", "fatal", "severe", "critical"}:
212             return "critical"
213         if raw in {"major", "high", "serious"}:
214             return "major"
215         return "minor"
216
217     @field_validator("suggested_action", mode="before")
218     @classmethod
219     def normalise_suggested_action(cls, value: object) -> str:
220         raw = _string_value(value).strip().casefold()
221         if raw in {"delete", "remove", "drop"} or any(
222             phrase in raw for phrase in ("delete", "remove", "drop")
223         ):
224             return "delete"
225         if raw in {"weaken", "qualify", "caveat"} or any(
226             phrase in raw for phrase in ("weaken", "qualify", "caveat")
227         ):
228             return "weaken"
229         return "replace"
230
231
232 class OutputAudit(StrictModel):
233     decision: Literal["pass", "revise"]
234     support_rate: float = Field(ge=0.0, le=1.0)
235     findings: list[AuditFinding] = Field(default_factory=list)

```

```

236 notes: list[str] = Field(default_factory=list)
237
238 @field_validator("decision", mode="before")
239 @classmethod
240 def normalise_decision(cls, value: object) -> str:
241     raw = _string_value(value).strip().casefold()
242     return "pass" if raw == "pass" else "revise"
243
244 @field_validator("notes", mode="before")
245 @classmethod
246 def normalise_notes(cls, value: object) -> list[str]:
247     return _string_list(value)
248
249
250 class RepairedDraft(StrictModel):
251     sentences: list[WriterSentence] = Field(default_factory=list)
252     repair_notes: list[str] = Field(default_factory=list)
253
254     @field_validator("repair_notes", mode="before")
255     @classmethod
256     def normalise_repair_notes(cls, value: object) -> list[str]:
257         return _string_list(value)
258
259
260 class LLMOnlyResult(StrictModel):
261     generated_text: str
262     interpretation: SourceInterpretation
263     candidate_claims: ClaimSet
264     critique: ClaimCritique
265     adjudicated_claims: AdjudicatedClaims
266     writer_draft: WriterDraft
267     final_audit: OutputAudit
268     repaired_draft: RepairedDraft | None = None
269     traces: list[AgentCallTrace] = Field(default_factory=list)
270     artifact_path: str | None = None

```

Listing 5: P05: experiments/llm_only_pipeline/src/table2text_llm_only/workflow.py

```

1 from __future__ import annotations
2
3 import hashlib
4 import json
5 from pathlib import Path
6 from typing import Any
7
8 from table2text.evaluation.models import BenchmarkExample, OutputMode, TaskFamily
9
10 from .client import LLMOnlyClient, LLMOnlyClientConfig
11 from .schemas import (
12     AdjudicatedClaims,
13     ClaimCritique,
14     ClaimSet,
15     LLMClaim,
16     LLMOnlyResult,
17     OutputAudit,
18     RepairedDraft,
19     RejectedClaim,
20     SourceInterpretation,
21     WriterSentence,
22     WriterDraft,
23 )
24
25
26 def _compact_json(value: Any) -> str:
27     return json.dumps(value, ensure_ascii=False, sort_keys=True, separators=(",", ":"))
28
29
30 def _pretty_json(value: Any) -> str:
31     return json.dumps(value, ensure_ascii=False, sort_keys=True, indent=2)
32
33
34 def _truncate(text: str, max_characters: int) -> str:
35     if len(text) <= max_characters:
36         return text
37     return text[:max_characters] + "\n\n[Source truncated by LLM-only configuration.]"
38
39
40 def _final_text(draft: WriterDraft | RepairedDraft) -> str:
41     return " ".join(
42         sentence.text.strip()
43         for sentence in draft.sentences
44         if sentence.text.strip()
45     ).strip()
46
47
48 def _claim_payload(claims: list[LLMClaim]) -> list[dict[str, Any]]:
49     return [claim.model_dump(mode="json") for claim in claims]
50
51

```



```

52 SYSTEM_BASE = (
53     "You are part of an LLM-only multi-agent Table2Text experiment. "
54     "Use only the supplied source data and intermediate artifacts. "
55     "Human references, targets, summaries and evaluation answers are held out "
56     "and are not available to you. Return only valid JSON matching the requested "
57     "shape. Do not include markdown fences. Keep string values concise and keep "
58     "arrays short unless the requested schema explicitly asks for more."
59 )
60
61
62 class LLMOnlyWorkflow:
63     def __init__(
64         self,
65         *,
66         client: LLMOnlyClient,
67         max_source_characters: int = 100_000,
68         max_source_payload_characters: int = 5_000,
69         max_claims: int = 16,
70         repair_rounds: int = 1,
71     ):
72         self.client = client
73         self.max_source_characters = max_source_characters
74         self.max_source_payload_characters = max_source_payload_characters
75         self.max_claims = max_claims
76         self.repair_rounds = repair_rounds
77
78     @classmethod
79     def from_settings(cls, values: dict[str, Any]) -> "LLMOnlyWorkflow":
80         config = LLMOnlyClientConfig.from_mapping(values)
81         return cls(
82             client=LLMOnlyClient(config),
83             max_source_characters=int(
84                 values.get("llm_only_max_source_characters", 100_000)
85             ),
86             max_source_payload_characters=int(
87                 values.get("llm_only_max_source_payload_characters", 5_000)
88             ),
89             max_claims=int(values.get("llm_only_max_claims", 16)),
90             repair_rounds=int(values.get("llm_only_repair_rounds", 1)),
91         )
92
93     def _source_packet(self, example: BenchmarkExample) -> dict[str, Any]:
94         source_text = example.source_text.strip() or _pretty_json(example.source_payload)
95         source_payload = self._source_payload_for_packet(
96             example.source_payload,
97             source_text,
98         )
99         return {
100             "dataset_id": example.dataset_id,
101             "example_id": example.example_id,
102             "task_family": example.task_family.value,
103             "output_mode": example.output_mode.value,
104             "language": example.language,
105             "request": example.request,
106             "source_text": _truncate(source_text, self.max_source_characters),
107             "source_payload": source_payload,
108             "parent_table": example.parent_table,
109             "metadata": example.metadata,
110         }
111
112     def _source_payload_for_packet(self, source_payload: Any, source_text: str) -> Any:
113         payload_text = _compact_json(source_payload)
114         if len(payload_text) <= self.max_source_payload_characters:
115             return source_payload
116         if source_text.strip():
117             return {
118                 "omitted_from_prompt": True,
119                 "reason": (
120                     "source_payload was omitted to avoid duplicating the full "
121                     "source_text in every LLM-only agent prompt"
122                 ),
123                 "payload_characters": len(payload_text),
124             }
125         return {
126             "truncated_payload_json": payload_text[
127                 : self.max_source_payload_characters
128             ],
129             "payload_characters": len(payload_text),
130         }
131
132     def interpret_source(self, packet: dict[str, Any]) -> tuple[SourceInterpretation, Any]:
133         payload, trace = self.client.json_call(
134             stage="source_interpreter",
135             system=SYSTEM_BASE
136             + " You are the Source Interpreter Agent. Describe what the source "
137             "contains, what claims are allowed, and what risks could cause "
138             "hallucination. Do not produce final report claims. This is a compact "
139             "orientation artifact, not a field-by-field dump.",
140             user=(
141                 "Return JSON with keys: task_understanding, source_units, "
142                 "important_entities, important_fields, allowed_claim_types, risks. "

```

```

143         "Limits: important_entities <= 12, important_fields <= 20, "
144         "allowed_claim_types <= 10, risks <= 8. Each string should be "
145         "under 160 characters.\n\n"
146         "Source packet:\n"
147         + _pretty_json(packet)
148     ),
149 )
150 return SourceInterpretation.model_validate(payload), trace
151
152 def analyse_claims(
153     self,
154     packet: dict[str, Any],
155     interpretation: SourceInterpretation,
156 ) -> tuple[ClaimSet, Any]:
157     payload, trace = self.client.json_call(
158         stage="llm_analysis",
159         system=SYSTEM_BASE
160         + " You are the LLM Analysis Agent. Produce candidate claims directly "
161         "from the source. Every claim must cite source_refs and copied_values. "
162         "If arithmetic or comparison is needed, do it yourself and show it in "
163         "calculation_or_reasoning. Keep each claim concise.",
164         user=(
165             "Return JSON with keys: claims, analysis_notes. "
166             "claims must contain objects with claim_id, claim, claim_type, "
167             "source_refs, copied_values, calculation_or_reasoning, confidence. "
168             "source_refs and copied_values must be flat arrays of short "
169             "strings, never nested objects. Each claim should be one sentence. "
170             "confidence must be a number from 0.0 to 1.0, never a word such "
171             "as high or low. If no calculation is needed, set "
172             "calculation_or_reasoning to a short source-grounding note. "
173             f"Return at most {self.max_claims} claims. Prefer claims needed "
174             "for the requested output. Keep analysis_notes <= 5. Do not "
175             "return an empty claims list when the source contains visible "
176             "game, team, player, score, or table values.\n\n"
177             + self._claim_slot_instruction(packet)
178             + "\n\n"
179             "Task and source:\n"
180             + _pretty_json(packet)
181             + "\n\nSource interpretation:\n"
182             + interpretation.model_dump_json(indent=2)
183         ),
184     )
185     return ClaimSet.model_validate(payload), trace
186
187 def recover_claims(
188     self,
189     packet: dict[str, Any],
190     interpretation: SourceInterpretation,
191 ) -> tuple[ClaimSet, Any]:
192     payload, trace = self.client.json_call(
193         stage="llm_analysis_recovery",
194         system=SYSTEM_BASE
195         + " You are the Recovery Claim Analyst Agent. A previous analyst "
196         "returned zero claims from a non-empty source. Build a conservative "
197         "claim ledger from directly visible source values. This is still "
198         "LLM-only: do not use outside knowledge, references, or hidden "
199         "answers. Prefer simple copied facts over clever narrative.",
200         user=(
201             "Return JSON with keys: claims, analysis_notes. claims must "
202             "contain objects with claim_id, claim, claim_type, source_refs, "
203             "copied_values, calculation_or_reasoning, confidence. "
204             "source_refs and copied_values must be flat arrays of short "
205             "strings, never nested objects. Each claim should be one sentence. "
206             "When the source contains game/team/player fields, return at "
207             f"least 8 and at most {self.max_claims} claims. Only return zero "
208             "claims if the source is genuinely empty or unreadable.\n\n"
209             "For structured event sources, prefer directly checkable facts "
210             "when present: outcome, context, participant records, period or "
211             "stage progression, leading performances, participant totals, "
212             "contrasts, and subsequent-event context. Adapt these categories "
213             "to the source vocabulary instead of assuming a domain. "
214             "Avoid unsupported play-by-play runs or momentum claims.\n\n"
215             + self._claim_slot_instruction(packet)
216             + "\n\nTask and source:\n"
217             + _pretty_json(packet)
218             + "\n\nSource interpretation:\n"
219             + interpretation.model_dump_json(indent=2)
220         ),
221     )
222     return ClaimSet.model_validate(payload), trace
223
224 def critique_claims(
225     self,
226     packet: dict[str, Any],
227     claims: ClaimSet,
228 ) -> tuple[ClaimCritique, Any]:
229     payload, trace = self.client.json_call(
230         stage="claim_critic",
231         system=SYSTEM_BASE
232         + " You are the Claim Critic Agent. Independently check each claim "
233         "against the source. Be stricter with numbers, entities, highlighted "

```

```

234         "cells, triples, attributes, chronology and causal language.",
235         user=(
236             "Return JSON with keys: reviews, global_warnings. Each review "
237             "must contain claim_id, status, reason, corrected_claim. Use status "
238             "supported, unsupported, overstated, wrong_number, wrong_entity, "
239             "wrong_relation, or needs_caveat. Keep each reason to one concise "
240             "sentence.\n\nSource packet:\n"
241             + _pretty_json(packet)
242             + "\n\nCandidate claims:\n"
243             + claims.model_dump_json(indent=2)
244         ),
245     )
246     return ClaimCritique.model_validate(payload), trace
247
248 def adjudicate_claims(
249     self,
250     packet: dict[str, Any],
251     claims: ClaimSet,
252     critique: ClaimCritique,
253 ) -> tuple[AdjudicatedClaims, Any]:
254     payload, trace = self.client.json_call(
255         stage="claim_adjudicator",
256         system=SYSTEM_BASE
257         + " You are the Claim Adjudicator Agent. Create the final accepted "
258         + "claim ledger. Accept only claims that are supported or can be made "
259         + "supported by a conservative corrected_claim. Reject uncertain claims, "
260         + "but do not reject every claim from a non-empty source if some simple "
261         + "copied-value claims are visibly supported. If critique reviews are "
262         + "missing or empty, conservatively accept high-confidence claims with "
263         + "source_refs and copied_values instead of returning an empty ledger.",
264         user=(
265             "Return JSON with keys: accepted_claims, rejected_claims, warnings. "
266             "accepted_claims use the original claim schema. rejected_claims "
267             "contain claim_id, claim, reason. Every accepted claim must keep "
268             "non-empty source_refs and copied_values; reject claims that lack "
269             "evidence even if they look plausible. Do not invent new claims. "
270             "Keep warnings <= 8.\n\n"
271             "Task and source:\n"
272             + _pretty_json(packet)
273             + "\n\nCandidate claims:\n"
274             + claims.model_dump_json(indent=2)
275             + "\n\nCritique:\n"
276             + critique.model_dump_json(indent=2)
277         ),
278     )
279     return AdjudicatedClaims.model_validate(payload), trace
280
281 def enforce_accepted_claim_evidence(
282     self,
283     adjudicated: AdjudicatedClaims,
284 ) -> AdjudicatedClaims:
285     accepted: list[LLMClaim] = []
286     rejected = list(adjudicated.rejected_claims)
287     warnings = list(adjudicated.warnings)
288
289     for claim in adjudicated.accepted_claims:
290         if claim.source_refs and claim.copied_values:
291             accepted.append(claim)
292             continue
293         rejected.append(
294             RejectedClaim(
295                 claim_id=claim.claim_id,
296                 claim=claim.claim,
297                 reason=(
298                     "Accepted claim removed because it lacked source_refs "
299                     "or copied_values."
300                 ),
301             )
302         )
303
304     if len(accepted) != len(adjudicated.accepted_claims):
305         warnings.append(
306             "Accepted claims without explicit source_refs and copied_values "
307             "were moved to rejected_claims."
308         )
309
310     return AdjudicatedClaims(
311         accepted_claims=accepted,
312         rejected_claims=rejected,
313         warnings=warnings,
314     )
315
316 def fallback_adjudication(
317     self,
318     claims: ClaimSet,
319     critique: ClaimCritique,
320 ) -> AdjudicatedClaims:
321     accepted: list[LLMClaim] = []
322     rejected: list[RejectedClaim] = []
323     review_by_id = {review.claim_id: review for review in critique.reviews}
324

```

```

325     for claim in claims.claims:
326         review = review_by_id.get(claim.claim_id)
327         has_evidence = bool(claim.source_refs and claim.copied_values)
328         supported_by_review = (
329             review is not None
330             and review.status in {"supported", "needs_caveat"}
331             and has_evidence
332         )
333         safe_without_review = review is None and has_evidence and claim.confidence >= 0.8
334
335         if supported_by_review or safe_without_review:
336             accepted.append(claim)
337         else:
338             rejected.append(
339                 RejectedClaim(
340                     claim_id=claim.claim_id,
341                     claim=claim.claim,
342                     reason=(
343                         "Fallback adjudication did not find enough LLM "
344                         "evidence to accept this claim."
345                     ),
346                 )
347             )
348
349     if not accepted:
350         candidates = [
351             claim
352             for claim in claims.claims
353             if claim.source_refs and claim.copied_values
354         ]
355         if candidates:
356             best = max(candidates, key=lambda claim: claim.confidence)
357             accepted.append(best)
358             rejected = [
359                 item for item in rejected if item.claim_id != best.claim_id
360             ]
361
362     return AdjudicatedClaims(
363         accepted_claims=accepted,
364         rejected_claims=rejected,
365         warnings=[
366             "Fallback adjudication used cited candidate claims because the "
367             "adjudicator returned an empty ledger."
368         ],
369     )
370
371 def write_draft(
372     self,
373     packet: dict[str, Any],
374     adjudicated: AdjudicatedClaims,
375 ) -> tuple[WriterDraft, Any]:
376     payload, trace = self.client.json_call(
377         stage="writer",
378         system=SYSTEM_BASE
379         + " You are the Writer Agent. Write only from accepted_claims. "
380         + "You do not have permission to add facts from the raw source. "
381         + "Every sentence must list the claim_ids that support it.",
382         user=(
383             "Return JSON with keys: title, sentences. sentences must contain "
384             "text and claim_ids. The final style must match the task family, "
385             "output mode and language. "
386             + self._style_instruction(packet)
387             + "\n\nTask contract:\n"
388             + _pretty_json(
389                 {
390                     key: packet[key]
391                     for key in (
392                         "dataset_id",
393                         "task_family",
394                         "output_mode",
395                         "language",
396                         "request",
397                     )
398                 }
399             )
400             + "\n\nAccepted claims:\n"
401             + _pretty_json(_claim_payload(adjudicated.accepted_claims))
402             + "\n\nWarnings:\n"
403             + _pretty_json(adjudicated.warnings)
404         ),
405     )
406     return WriterDraft.model_validate(payload), trace
407
408 def fallback_draft(
409     self,
410     packet: dict[str, Any],
411     adjudicated: AdjudicatedClaims,
412 ) -> WriterDraft:
413     sentences = [
414         WriterSentence(text=claim.claim.rstrip(".") + ".", claim_ids=[claim.claim_id])
415         for claim in adjudicated.accepted_claims[:8]

```

```

416         if claim.claim.strip()
417     ]
418     task_family = TaskFamily(packet["task_family"])
419     title = (
420         "Event Report"
421         if task_family
422         in {TaskFamily.EVENT_REPORT, TaskFamily.CROSS_LINGUAL_EVENT_REPORT}
423         else None
424     )
425     return WriterDraft(title=title, sentences=sentences)
426
427 def audit_output(
428     self,
429     packet: dict[str, Any],
430     adjudicated: AdjudicatedClaims,
431     draft: WriterDraft | RepairedDraft,
432 ) -> tuple[OutputAudit, Any]:
433     payload, trace = self.client.json_call(
434         stage="output_auditor",
435         system=SYSTEM_BASE
436         + " You are the Output Auditor Agent. Check whether the draft uses "
437         "only accepted claims and whether each sentence is properly supported. "
438         "Also judge whether the draft covers the minimum required task slots. "
439         "This system never blocks output; severe factual problems should be "
440         "reported as revise so the repair agent can produce the safest "
441         "supported version.",
442         user=(
443             "Return JSON with keys: decision, support_rate, findings, notes. "
444             "findings contain sentence_index, severity, issue, suggested_action. "
445             "Use severity minor, major, or critical; critical means high "
446             "hallucination risk that should be repaired, not blocked. "
447             "Decision must be pass or revise, never block. For unsupported "
448             "wording or missing but available coverage, use decision revise "
449             "with suggested_action replace, delete, or weaken.\n\n"
450             "Required coverage:\n"
451             + self._coverage_instruction(packet)
452             + "\n\nTask and source:\n"
453             + _pretty_json(packet)
454             + "\n\nAccepted claims:\n"
455             + _pretty_json(_claim_payload(adjudicated.accepted_claims))
456             + "\n\nDraft:\n"
457             + draft.model_dump_json(indent=2)
458         ),
459     )
460     return OutputAudit.model_validate(payload), trace
461
462 def repair_output(
463     self,
464     packet: dict[str, Any],
465     adjudicated: AdjudicatedClaims,
466     draft: WriterDraft | RepairedDraft,
467     audit: OutputAudit,
468 ) -> tuple[RepairedDraft, Any]:
469     payload, trace = self.client.json_call(
470         stage="repair",
471         system=SYSTEM_BASE
472         + " You are the Repair Agent. Only delete, weaken, or replace text "
473         "using accepted claims. Do not add new claims from the raw source.",
474         user=(
475             "Return JSON with keys: sentences, repair_notes. sentences contain "
476             "text and claim_ids. You may add a sentence only when it is fully "
477             "supported by accepted claim_ids. Keep the output task-appropriate.\n\n"
478             "Required coverage:\n"
479             + self._coverage_instruction(packet)
480             + "\n\n"
481             "Task contract:\n"
482             + _pretty_json(
483                 {
484                     key: packet[key]
485                     for key in (
486                         "dataset_id",
487                         "task_family",
488                         "output_mode",
489                         "language",
490                         "request",
491                     )
492                 }
493             )
494             + "\n\nAccepted claims:\n"
495             + _pretty_json(_claim_payload(adjudicated.accepted_claims))
496             + "\n\nCurrent draft:\n"
497             + draft.model_dump_json(indent=2)
498             + "\n\nAudit:\n"
499             + audit.model_dump_json(indent=2)
500         ),
501     )
502     return RepairedDraft.model_validate(payload), trace
503
504 def run(self, example: BenchmarkExample) -> LLMOnlyResult:
505     packet = self._source_packet(example)
506     traces = []

```

```

507 interpretation, trace = self.interpret_source(packet)
508 traces.append(trace)
509 claims, trace = self.analyse_claims(packet, interpretation)
510 traces.append(trace)
511 if not claims.claims and self._has_visible_source_data(packet):
512     claims, trace = self.recover_claims(packet, interpretation)
513     traces.append(trace)
514 critique, trace = self.critique_claims(packet, claims)
515 traces.append(trace)
516 adjudicated, trace = self.adjudicate_claims(packet, claims, critique)
517 traces.append(trace)
518 if (
519     claims.claims
520     and not adjudicated.accepted_claims
521     and not adjudicated.rejected_claims
522 ):
523     adjudicated = self.fallback_adjudication(claims, critique)
524     adjudicated = self.enforce_accepted_claim_evidence(adjudicated)
525     draft, trace = self.write_draft(packet, adjudicated)
526     traces.append(trace)
527 if adjudicated.accepted_claims and not draft.sentences:
528     draft = self.fallback_draft(packet, adjudicated)
529     audit, trace = self.audit_output(packet, adjudicated, draft)
530     traces.append(trace)
531 repaired: RepairedDraft | None = None
532 final_text = _final_text(draft)
533 if audit.decision == "revise" and self.repair_rounds > 0:
534     repaired, trace = self.repair_output(packet, adjudicated, draft, audit)
535     traces.append(trace)
536     audit, trace = self.audit_output(packet, adjudicated, repaired)
537     traces.append(trace)
538     final_text = _final_text(repaired)
539
540 return LLMOnlyResult(
541     generated_text=final_text,
542     interpretation=interpretation,
543     candidate_claims=claims,
544     critique=critique,
545     adjudicated_claims=adjudicated,
546     writer_draft=draft,
547     final_audit=audit,
548     repaired_draft=repaired,
549     traces=traces,
550 )
551
552 def _style_instruction(self, packet: dict[str, Any]) -> str:
553     task_family = TaskFamily(packet["task_family"])
554     output_mode = OutputMode(packet["output_mode"])
555     if output_mode == OutputMode.ONE_SENTENCE:
556         return "Write exactly one sentence."
557     if output_mode in {OutputMode.SHORT_TEXT, OutputMode.DIRECT_ANSWER}:
558         return "Write one or two concise sentences with no heading."
559     if task_family in {TaskFamily.EVENT_REPORT, TaskFamily.CROSS_LINGUAL_EVENT_REPORT}:
560         return (
561             "Write a coherent event recap in 2 to 4 paragraphs. Lead with "
562             "the result, then cover score progression, leading performances "
563             "and team contrasts when those accepted claims are available."
564         )
565     return "Write a concise factual report."
566
567 def _has_visible_source_data(self, packet: dict[str, Any]) -> bool:
568     source_text = str(packet.get("source_text") or "").strip()
569     if source_text and source_text not in {"{}", "[]"}:
570         return True
571     source_payload = packet.get("source_payload")
572     return bool(source_payload)
573
574 def _claim_slot_instruction(self, packet: dict[str, Any]) -> str:
575     task_family = TaskFamily(packet["task_family"])
576     if task_family not in {
577         TaskFamily.EVENT_REPORT,
578         TaskFamily.CROSS_LINGUAL_EVENT_REPORT,
579     }:
580         return (
581             "Select claims that fully satisfy the requested task while "
582             "avoiding unrelated details."
583         )
584     return (
585         "For this event-report task, cover these claim slots when supported "
586         "by the source: event_result, event_context, record_context, "
587         "period_score_progression, turning_period_or_key_run, lead_scorer, "
588         "secondary_scorers, top_rebouncers_or_double_doubles, assist_leaders, "
589         "shooting_comparison, three_point_comparison, free_throw_comparison, "
590         "rebounding_comparison, defensive_stats, bench_or_supporting "
591         "contribution, and scope_note. It is better to include a concise "
592         "supported claim for each major slot than to over-focus on only "
593         "the final score and top scorer."
594     )

```

```

598 def _coverage_instruction(self, packet: dict[str, Any]) -> str:
599     task_family = TaskFamily(packet["task_family"])
600     if task_family not in {
601         TaskFamily.EVENT_REPORT,
602         TaskFamily.CROSS_LINGUAL_EVENT_REPORT,
603     }:
604         return "Cover the requested task exactly; do not require event-report sections."
605     return (
606         "A passable event recap should include the supported result, event "
607         "context, score progression or period-by-period development, leading "
608         "performances, and at least one team-level contrast. If accepted "
609         "claims contain these facts but the draft omits them, return revise."
610     )
611
612 def artifact_id(example: BenchmarkExample, variant_id: str, repetition: int, seed: int) -> str:
613     raw = ":".join(
614         [
615             example.dataset_id,
616             example.example_id,
617             variant_id,
618             str(repetition),
619             str(seed),
620             example.source_sha256,
621         ]
622     )
623     return hashlib.sha256(raw.encode("utf-8")).hexdigest()[:16]
624
625
626 def write_result_artifact(
627     *,
628     result: LLMOnlyResult,
629     example: BenchmarkExample,
630     variant_id: str,
631     repetition: int,
632     seed: int,
633     artifact_dir: Path,
634 ) -> LLMOnlyResult:
635     path = (
636         artifact_dir
637         / example.dataset_id
638         / f"{artifact_id(example, variant_id, repetition, seed)}.json"
639     )
640     path.parent.mkdir(parents=True, exist_ok=True)
641     result = result.model_copy(update={"artifact_path": str(path)})
642     path.write_text(result.model_dump_json(indent=2), encoding="utf-8")
643     return result
644

```

Listing 6: P06: submission/program_listings/build_program_listings.py

```

1 from __future__ import annotations
2
3 import argparse
4 import hashlib
5 import json
6 import os
7 import shutil
8 import subprocess
9 from collections import defaultdict
10 from pathlib import Path
11 from typing import Any, Iterable
12
13
14 SCRIPT_DIR = Path(__file__).resolve().parent
15 PROJECT_ROOT = SCRIPT_DIR.parents[1]
16 MAIN_PROJECT = PROJECT_ROOT / "table2text_pydanticai"
17 EXPERIMENT = PROJECT_ROOT / "experiments/llm_only_pipeline"
18 BUILD_DIR = SCRIPT_DIR / "build"
19 GENERATED_DIR = BUILD_DIR / "generated"
20 OUTPUT_PDF = SCRIPT_DIR / "Tanatswa Mapfumo Project Code Printout.pdf"
21 SOFTWARE_SUBMISSION = PROJECT_ROOT / "submission/software_archive"
22
23
24 DATA_SHARDS = [
25     MAIN_PROJECT
26     / "evaluation/protected_holdout_full_system/prepared/shards"
27     / filename
28     for filename in (
29         "e2e_nlg_e2e_nlg-test-1330.jsonl",
30         "e2e_nlg_e2e_nlg-test-209.jsonl",
31         "e2e_nlg_e2e_nlg-test-447.jsonl",
32         "e2e_nlg_e2e_nlg-test-476.jsonl",
33         "e2e_nlg_e2e_nlg-test-864.jsonl",
34         "totto_totto-validation-1828.jsonl",
35         "totto_totto-validation-4467.jsonl",
36         "totto_totto-validation-6067.jsonl",
37         "totto_totto-validation-839.jsonl",
38         "totto_totto-validation-912.jsonl",
39         "web_nlg_web_nlg_en-test-1209.jsonl",

```



```

40     "web_nlg__web_nlg_en-test-1330.jsonl",
41     "web_nlg__web_nlg_en-test-1466.jsonl",
42     "web_nlg__web_nlg_en-test-859.jsonl",
43     "web_nlg__web_nlg_en-test-864.jsonl",
44     "dart__dart-test-1791.jsonl",
45     "dart__dart-test-1805.jsonl",
46     "dart__dart-test-1828.jsonl",
47     "dart__dart-test-2278.jsonl",
48     "dart__dart-test-4597.jsonl",
49     "sportsett_basketball__5130.jsonl",
50     "sportsett_basketball__5372.jsonl",
51     "sportsett_basketball__5786.jsonl",
52     "sportsett_basketball__5955.jsonl",
53     "sportsett_basketball__6127.jsonl",
54 )
55 ]
56
57
58 RESULT_FILES = [
59     MAIN_PROJECT
60     / "evaluation/protected_holdout_full_system/results/protected_generation_summary.csv",
61     MAIN_PROJECT
62     / "evaluation/protected_holdout_baseline/results/automatic_metrics_overall.csv",
63     MAIN_PROJECT
64     / "evaluation/protected_holdout_baseline/results/automatic_metrics_by_dataset.csv",
65     MAIN_PROJECT
66     / "evaluation/protected_holdout_baseline/gpt56_judge/results/gpt56_annotation_summary.csv",
67     MAIN_PROJECT
68     / "evaluation/protected_holdout_baseline/gpt56_judge/results/gpt56_category_counts.csv",
69 ]
70
71
72 CONFIG_FILES = [
73     MAIN_PROJECT / "pyproject.toml",
74     MAIN_PROJECT / ".env.example",
75     MAIN_PROJECT / "evaluation/config/datasets.json",
76     MAIN_PROJECT / "evaluation/config/metrics.json",
77     MAIN_PROJECT / "evaluation/config/metrics_reference_similarity.json",
78     MAIN_PROJECT / "evaluation/config/metrics_source_grounded.json",
79     MAIN_PROJECT / "evaluation/config/variants.json",
80     MAIN_PROJECT / "evaluation/config/variants_ablation.json",
81     MAIN_PROJECT / "evaluation/config/variants_raw_deepseek_v4_flash.json",
82     MAIN_PROJECT / "evaluation/config/variants_raw_deepseek_v4_pro.json",
83     MAIN_PROJECT
84     / "evaluation/protected_holdout_full_system/config/protected_full_system_flash.json",
85     EXPERIMENT / "pyproject.toml",
86     EXPERIMENT / ".env.example",
87     EXPERIMENT / "config/variants.json",
88 ]
89
90
91 def unique_paths(paths: Iterable[Path]) -> list[Path]:
92     return sorted(set(paths), key=lambda path: path.relative_to(PROJECT_ROOT).as_posix())
93
94
95 def program_files() -> list[Path]:
96     paths: list[Path] = []
97     paths.extend((MAIN_PROJECT / "src").rglob("*.py"))
98     paths.extend((MAIN_PROJECT / "evaluation/scripts").rglob("*.py"))
99     paths.extend((MAIN_PROJECT / "evaluation/notebooks").glob("*.py"))
100    paths.extend((EXPERIMENT / "src").rglob("*.py"))
101    paths.append(SOFTWARE_SUBMISSION / "build_software_submission.py")
102    paths.append(SOFTWARE_SUBMISSION / "examples/run_deterministic_demo.sh")
103    paths.append(Path(__file__).resolve())
104    return unique_paths(paths)
105
106
107 def test_files() -> list[Path]:
108     return unique_paths(
109         [
110             *(MAIN_PROJECT / "tests").rglob("*.py"),
111             *(EXPERIMENT / "tests").rglob("*.py"),
112         ]
113     )
114
115
116 def sha256(path: Path) -> str:
117     return hashlib.sha256(path.read_bytes()).hexdigest()
118
119
120 def read_jsonl(path: Path) -> list[dict[str, Any]]:
121     records = []
122     for line in path.read_text(encoding="utf-8").splitlines():
123         if line.strip():
124             records.append(json.loads(line))
125     return records
126
127
128 def write_pretty_json(path: Path, payload: Any) -> None:
129     path.parent.mkdir(parents=True, exist_ok=True)
130     path.write_text(

```

```

131     json.dumps(payload, ensure_ascii=False, indent=2, sort_keys=True) + "\n",
132     encoding="utf-8",
133 )
134
135
136 def build_protected_data_listings() -> list[Path]:
137     output_paths = []
138     for path in DATA_SHARDS:
139         for record in read_jsonl(path):
140             payload = {
141                 "dataset_id": record["dataset_id"],
142                 "example_id": record["example_id"],
143                 "task_family": record["task_family"],
144                 "output_mode": record["output_mode"],
145                 "language": record["language"],
146                 "request": record["request"],
147                 "source_payload": record.get("source_payload"),
148                 "parent_table": record.get("parent_table"),
149                 "references": record.get("references", []),
150                 "source_sha256": record.get("source_sha256"),
151                 "reference_sha256": record.get("reference_sha256"),
152             }
153             safe_id = str(record["example_id"]).replace("/", "_")
154             output = GENERATED_DIR / "data" / f"{record['dataset_id']}__{safe_id}.json"
155             write_pretty_json(output, payload)
156             output_paths.append(output)
157     output_paths = unique_paths(output_paths)
158     if len(output_paths) != 25:
159         raise RuntimeError(
160             f"Expected 25 protected examples, found {len(output_paths)}"
161         )
162     return output_paths
163
164
165 def build_paired_output_listing() -> Path:
166     source = (
167         MAIN_PROJECT
168         / "evaluation/protected_holdout_baseline/comparison/full_system_and_baseline_sealed.jsonl"
169     )
170     grouped: dict[tuple[str, str], dict[str, Any]] = defaultdict(dict)
171     for record in read_jsonl(source):
172         key = (record["dataset_id"], str(record["example_id"]))
173         condition = "full_system" if record["variant_id"] == "full_system" else "baseline"
174         grouped[key][condition] = {
175             "generated_text": record["generated_text"],
176             "release_status": record.get("release_status"),
177             "writer_mode": record.get("writer_mode"),
178             "primary_evaluation_eligible": record.get("primary_evaluation_eligible"),
179         }
180         grouped[key]["dataset_id"] = record["dataset_id"]
181         grouped[key]["example_id"] = str(record["example_id"])
182         grouped[key]["task_family"] = record["task_family"]
183         grouped[key]["request"] = record["request"]
184         grouped[key]["references"] = record.get("references", [])
185
186     records = [grouped[key] for key in sorted(grouped)]
187     if len(records) != 25 or any(
188         "full_system" not in record or "baseline" not in record for record in records
189     ):
190         raise RuntimeError("The paired protected output file is incomplete.")
191     output = GENERATED_DIR / "protected_holdout_25_paired_outputs.json"
192     write_pretty_json(output, records)
193     return output
194
195
196 def copy_result_files() -> list[Path]:
197     output_paths = []
198     for source in RESULT_FILES:
199         destination = GENERATED_DIR / "results" / source.name
200         destination.parent.mkdir(parents=True, exist_ok=True)
201         shutil.copy2(source, destination)
202         output_paths.append(destination)
203     return output_paths
204
205
206 def latex_escape(value: str) -> str:
207     replacements = {
208         "\\": r"\textbackslash{}",
209         "&": r"&",
210         "%": r"%",
211         "$": r"$",
212         "#": r"#",
213         " ": r"\ ",
214         "{": r"\{ ",
215         "}": r"\} ",
216         "~": r"\textasciitilde{}",
217         "^": r"\textasciicircum{}",
218     }
219     return "".join(replacements.get(character, character) for character in value)
220
221

```

```

222 def listing_language(path: Path) -> str:
223     return {
224         ".py": "Python",
225         ".json": "json",
226         ".jsonl": "json",
227         ".sh": "bash",
228         ".csv": "",
229         ".toml": "",
230         ".example": "",
231     }.get(path.suffix, "")
232
233
234 def listing_command(identfier: str, caption: str, path: Path) -> str:
235     relative_path = Path(os.path.relpath(path, BUILD_DIR)).as_posix()
236     language = listing_language(path)
237     language_option = f"language={language}," if language else ""
238     return (
239         f"\\lstinputlisting[{{language_option}}"
240         f"caption={{\\latex_escape(identfier + ': ' + caption)}}},"
241         f"label={{\\lst:{identfier.lower()}}}"
242         f"{{{relative_path}}}"
243     )
244
245
246 def file_caption(path: Path) -> str:
247     relative = path.relative_to(PROJECT_ROOT).as_posix()
248     return relative
249
250
251 def build_manifest(
252     programs: list[Path],
253     tests: list[Path],
254     configs: list[Path],
255     data: list[Path],
256     results: list[Path],
257 ) -> Path:
258     records = []
259     for category, paths in (
260         ("program", programs),
261         ("test", tests),
262         ("configuration", configs),
263         ("data", data),
264         ("result", results),
265     ):
266         for path in paths:
267             records.append(
268                 {
269                     "category": category,
270                     "path": (
271                         path.relative_to(PROJECT_ROOT).as_posix()
272                         if path.is_relative_to(PROJECT_ROOT)
273                         else path.relative_to(SCRIPT_DIR).as_posix()
274                     ),
275                     "lines": len(path.read_text(encoding="utf-8").splitlines()),
276                     "sha256": sha256(path),
277                 }
278             )
279     output = GENERATED_DIR / "listing_manifest.json"
280     write_pretty_json(output, records)
281     return output
282
283
284 def render_tex(
285     programs: list[Path],
286     tests: list[Path],
287     configs: list[Path],
288     data: list[Path],
289     results: list[Path],
290     manifest: Path,
291 ) -> str:
292     sections: list[str] = []
293
294     def add_section(title: str, prefix: str, files: list[Path], description: str) -> None:
295         sections.extend(
296             [
297                 "\\clearpage",
298                 f"\\section{{{\\latex_escape(title)}}}",
299                 description,
300             ]
301         )
302         for index, path in enumerate(files, start=1):
303             identifier = f"{{prefix}}{index:02d}"
304             sections.append(listing_command(identifier, file_caption(path), path))
305
306     add_section(
307         "Program Listings",
308         "p",
309         programs,
310         "Maintained runtime, evaluation, notebook-companion and experimental programs.",
311     )
312     add_section(

```

```

313     "Test Program Listings",
314     "T",
315     tests,
316     "Automated test programs for the main system and the isolated LLM-only experiment.",
317 )
318 add_section(
319     "Configuration Listings",
320     "C",
321     configs,
322     "Reproducibility configuration. Environment examples contain placeholders only; no secrets are included.",
323 )
324 add_section(
325     "Protected Data Listings",
326     "D",
327     data,
328     "All 25 protected-holdout inputs, grouped across five datasets and "
329     "formatted as readable JSON. Where the benchmark stores both a "
330     "structured payload and a duplicate serialized source-text field, this "
331     "appendix displays the complete structured payload once.",
332 )
333 add_section(
334     "Evaluation Result Listings",
335     "R",
336     results,
337     "Paired Full System/Baseline outputs and the principal protected-holdout result tables.",
338 )
339 sections.extend(
340     [
341         "\\clearpage",
342         "\\section{Listing Manifest}",
343         "This machine-readable manifest records the category, path, line count and SHA-256 digest of every included
344         listing.",
345         listing_command("M01", "Program-listing inclusion manifest", manifest),
346     ]
347 )
348 total_source_lines = sum(
349     len(path.read_text(encoding="utf-8").splitlines()) for path in programs
350 )
351 total_test_lines = sum(
352     len(path.read_text(encoding="utf-8").splitlines()) for path in tests
353 )
354
355 return r"""
356 \documentclass[a4paper,10pt]{article}
357 \usepackage[top=16mm,bottom=17mm,left=16mm,right=14mm,headheight=14pt]{geometry}
358 \usepackage{fontspec}
359 \usepackage{xcolor}
360 \usepackage{listings}
361 \usepackage{hyperref}
362 \usepackage{fancyhdr}
363 \usepackage{longtable}
364 \usepackage{booktabs}
365 \usepackage{microtype}
366
367 \setmainfont{Times New Roman}
368 \setmonofont{Menlo}
369 \definecolor{codeblue}{HTML}{154A8A}
370 \definecolor{codegreen}{HTML}{176B3A}
371 \definecolor{codegray}{HTML}{9A2D25}
372 \definecolor{linegray}{HTML}{707070}
373 \definecolor{framegray}{HTML}{D5D9DE}
374 \definecolor{shade}{HTML}{F7F8FA}
375
376 \lstdefinlanguage{json}{
377     basicstyle=\ttfamily\fontsize{7.1}{8.2}\selectfont,
378     stringstyle=\color{codegreen},
379     commentstyle=\color{linegray},
380     morecomment=[s]{/*}{*/},
381     literate=
382     *{0}{{{color{codegray}0}}}{1}
383     {1}{{{color{codegray}1}}}{1}
384     {2}{{{color{codegray}2}}}{1}
385     {3}{{{color{codegray}3}}}{1}
386     {4}{{{color{codegray}4}}}{1}
387     {5}{{{color{codegray}5}}}{1}
388     {6}{{{color{codegray}6}}}{1}
389     {7}{{{color{codegray}7}}}{1}
390     {8}{{{color{codegray}8}}}{1}
391     {9}{{{color{codegray}9}}}{1}
392 }
393
394 \lstset{
395     basicstyle=\ttfamily\fontsize{7.1}{8.2}\selectfont,
396     keywordstyle=\bfseries\color{codeblue},
397     stringstyle=\color{codegreen},
398     commentstyle=\itshape\color{linegray},
399     identifierstyle=\color{black},
400     numbers=left,
401

```

```

402     numberstyle=\ttfamily\fontsize{5.8}{6.5}\selectfont\color{linegray},
403     numbersep=7pt,
404     frame=single,
405     rulecolor=\color{framegray},
406     backgroundcolor=\color{shade},
407     breaklines=true,
408     breakatwhitespace=false,
409     showstringspaces=false,
410     keepspaces=true,
411     columns=fullflexible,
412     tabsize=4,
413     captionpos=t,
414     aboveskip=1.2em,
415     belowskip=1.5em,
416     inputencoding=utf8,
417 }
418
419 \hypersetup{colorlinks=true,linkcolor=codeblue,urlcolor=codeblue,pdftitle={MScProject Program Listings}}
420 \pagestyle{fancy}
421 \fancyhf{}
422 \lhead{MScProject Program Listings}
423 \rhead{Evidence-grounded Table-to-Text Generation}
424 \cfoot{\thepage}
425 \setcounter{tocdepth}{1}
426
427 \title{MScProject\Program, Test, Data and Result Listings}
428 \author{Tanatswa}
429 \date{21 August 2026}
430
431 \begin{document}
432 \maketitle
433
434 \section*{Purpose and Scope}
435 This PDF is the readable program-listing submission for the MScProject on evidence-grounded table-to-text generation.
    Its source repository is available at \href{https://github.com/tboysavage/MScProject}{github.com/tboysavage/
    MScProject}. The document includes maintained programs, automated tests, reproducibility configuration, all 25
    protected-holdout inputs, all 25 paired system/baseline outputs, and principal evaluation tables. The final
    manifest supplies file-level SHA-256 digests for the exact listed snapshot.
436
437 The listings exclude credentials, local environments, downloaded datasets, caches, transient run directories and
    duplicated frozen source snapshots. Jupyter notebooks are represented by their maintained Python companion programs
    because raw notebook JSON is not readable as a program listing.
438
439 \begin{center}
440 \begin{tabular}{lrr}
441 \toprule
442 Category & Files & Source lines \\
443 \midrule
444 Programs & PROGRAM_COUNT & PROGRAM_LINES \\
445 Tests & TEST_COUNT & TEST_LINES \\
446 Configuration & CONFIG_COUNT & -- \\
447 Protected data & DATA_COUNT & 25 examples \\
448 Evaluation results & RESULT_COUNT & 25 paired outputs plus tables \\
449 \bottomrule
450 \end{tabular}
451 \end{center}
452
453 \section*{Listing Labels}
454 \begin{description}
455 \item[P] Program or executable research-support module.
456 \item[T] Automated test program.
457 \item[C] Reproducibility configuration.
458 \item[D] Protected-holdout data.
459 \item[R] Evaluation result.
460 \item[M] Machine-readable inclusion manifest.
461 \end{description}
462
463 \tableofcontents
464 \clearpage
465 \lstlistoflistings
466
467 SECTIONS
468
469 \end{document}
470 """#.replace("PROGRAM_COUNT", str(len(programs))).replace(
471     "PROGRAM_LINES", f"{total_source_lines:}"
472 ).replace("TEST_COUNT", str(len(tests))).replace(
473     "TEST_LINES", f"{total_test_lines:}"
474 ).replace("CONFIG_COUNT", str(len(configs))).replace(
475     "DATA_COUNT", str(len(data))
476 ).replace("RESULT_COUNT", str(len(results))).replace(
477     "SECTIONS", "\n\n".join(sections)
478 )
479
480
481 def validate_inputs(paths: Iterable[Path]) -> None:
482     missing = [path for path in paths if not path.is_file()]
483     if missing:
484         formatted = "\n".join(f"- {path}" for path in missing)
485         raise FileNotFoundError(f"Required listing inputs are missing:\n{formatted}")
486

```

```

487
488 def build_pdf(*, compile_pdf: bool = True) -> Path:
489     BUILD_DIR.mkdir(parents=True, exist_ok=True)
490     if GENERATED_DIR.exists():
491         shutil.rmtree(GENERATED_DIR)
492     GENERATED_DIR.mkdir(parents=True, exist_ok=True)
493
494     programs = program_files()
495     tests = test_files()
496     configs = unique_paths(CONFIG_FILES)
497     validate_inputs([*programs, *tests, *configs, *DATA_SHARDS, *RESULT_FILES])
498
499     data_files = build_protected_data_listings()
500     paired_outputs = build_paired_output_listing()
501     result_files = [paired_outputs, *copy_result_files()]
502     manifest = build_manifest(programs, tests, configs, data_files, result_files)
503
504     tex_path = BUILD_DIR / "program_listings.tex"
505     tex_path.write_text(
506         render_tex(programs, tests, configs, data_files, result_files, manifest),
507         encoding="utf-8",
508     )
509     print(f"Programs: {len(programs)}")
510     print(f"Tests: {len(tests)}")
511     print("Protected examples: 25")
512     print("Paired output examples: 25")
513     print(f"LaTeX: {tex_path}")
514
515     if not compile_pdf:
516         return tex_path
517
518     log_path = BUILD_DIR / "xelatex.log"
519     for pass_number in (1, 2, 3):
520         print(f"XeLaTeX pass {pass_number}/3...")
521         with log_path.open("a" if pass_number > 1 else "w", encoding="utf-8") as log:
522             result = subprocess.run(
523                 [
524                     "xelatex",
525                     "-interaction=nonstopmode",
526                     "-halt-on-error",
527                     tex_path.name,
528                 ],
529                 cwd=BUILD_DIR,
530                 stdout=log,
531                 stderr=subprocess.STDOUT,
532                 check=False,
533             )
534             if result.returncode:
535                 tail = "\n".join(log_path.read_text(encoding="utf-8").splitlines()[-30:])
536                 raise RuntimeError(f"XeLaTeX pass {pass_number} failed:\n{tail}")
537     shutil.copy2(BUILD_DIR / "program_listings.pdf", OUTPUT_PDF)
538     print(f"PDF: {OUTPUT_PDF}")
539     return OUTPUT_PDF
540
541
542 def main() -> None:
543     parser = argparse.ArgumentParser(description=__doc__)
544     parser.add_argument(
545         "--no-compile",
546         action="store_true",
547         help="Generate staged listings and LaTeX without invoking XeLaTeX.",
548     )
549     args = parser.parse_args()
550     build_pdf(compile_pdf=not args.no_compile)
551
552
553 if __name__ == "__main__":
554     main()

```

Listing 7: P07: submission/software_archive/build_software_submission.py

```

1  """Build the examiner-facing MScProject software submission archive."""
2
3  from __future__ import annotations
4
5  import hashlib
6  import json
7  import platform
8  import re
9  import shutil
10 import subprocess
11 import sys
12 import zipfile
13 from datetime import date
14 from pathlib import Path
15
16
17 SCRIPT_DIR = Path(__file__).resolve().parent
18 PROJECT_ROOT = SCRIPT_DIR.parents[1]

```

```

19 PACKAGE_DIR = PROJECT_ROOT / "table2text_pydanticai"
20 BUILD_DIR = SCRIPT_DIR / "build"
21 STAGE_PARENT = BUILD_DIR / "stage"
22 ARCHIVE_ROOT_NAME = "Tanatswa_Mapfumo_MScProject_Software_Submission"
23 STAGE_ROOT = STAGE_PARENT / ARCHIVE_ROOT_NAME
24 OUTPUT_ARCHIVE = SCRIPT_DIR / "Tanatswa_Mapfumo_Source_Code.zip"
25 OUTPUT_CHECKSUM = SCRIPT_DIR / f"{OUTPUT_ARCHIVE.name}.sha256"
26
27 SOURCE_EXCLUDED_PREFIXES = (
28     "submission/",
29     "table2text_pydanticai/evaluation/protected_holdout_full_system/"
30     "frozen_code_snapshot/",
31 )
32
33 SUBMISSION_DOCUMENTS = {
34     "ARCHIVE_README.md": "README.md",
35     "PROGRAM_DESCRIPTION.md": "PROGRAM_DESCRIPTION.md",
36     "INSTALLATION_AND_BUILD.md": "INSTALLATION_AND_BUILD.md",
37     "DEPENDENCIES.md": "DEPENDENCIES.md",
38     "VERIFICATION.md": "VERIFICATION.md",
39     "requirements-tested.txt": "requirements-tested.txt",
40 }
41
42
43 def sha256(path: Path) -> str:
44     digest = hashlib.sha256()
45     with path.open("rb") as handle:
46         for chunk in iter(lambda: handle.read(1024 * 1024), b''):
47             digest.update(chunk)
48     return digest.hexdigest()
49
50
51 def run(command: list[str], *, cwd: Path | None = None) -> None:
52     print("+", " ".join(command), flush=True)
53     subprocess.run(command, cwd=cwd, check=True)
54
55
56 def tracked_source_files() -> list[Path]:
57     completed = subprocess.run(
58         ["git", "ls-files", "-z"],
59         cwd=PROJECT_ROOT,
60         check=True,
61         capture_output=True,
62     )
63     relative_paths = [
64         Path(item.decode("utf-8"))
65         for item in completed.stdout.split(b"\0")
66         if item
67     ]
68     selected: list[Path] = []
69     for relative in relative_paths:
70         relative_text = relative.as_posix()
71         if relative_text == ".gitignore":
72             continue
73         if relative_text.startswith(SOURCE_EXCLUDED_PREFIXES):
74             continue
75         source = PROJECT_ROOT / relative
76         if source.is_file():
77             selected.append(relative)
78     return sorted(selected)
79
80
81 def copy_file(source: Path, destination: Path) -> None:
82     destination.parent.mkdir(parents=True, exist_ok=True)
83     shutil.copy2(source, destination)
84
85
86 def build_wheel() -> Path:
87     wheel_dir = BUILD_DIR / "wheel"
88     wheel_dir.mkdir(parents=True, exist_ok=True)
89     for existing in wheel_dir.glob("*.whl"):
90         existing.unlink()
91     run(
92         [
93             sys.executable,
94             "-m",
95             "pip",
96             "wheel",
97             "--no-deps",
98             "--wheel-dir",
99             str(wheel_dir),
100             str(PACKAGE_DIR),
101         ],
102         cwd=PROJECT_ROOT,
103     )
104     wheels = sorted(wheel_dir.glob("*.whl"))
105     if len(wheels) != 1:
106         raise RuntimeError(f"Expected one wheel, found {len(wheels)}")
107     return wheels[0]
108
109

```



```

110 def markdown_inline(text: str) -> str:
111     from xml.sax.saxutils import escape
112
113     escaped = escape(text.strip())
114     escaped = re.sub(
115         r"``([^\s]+)``",
116         r"<font name='Courier'>\1</font>",
117         escaped,
118     )
119     escaped = re.sub(r"\*\[^\s]+\*\*", r"<b>\1</b>", escaped)
120     return escaped
121
122
123 def build_program_description_pdf(output_path: Path) -> None:
124     from reportlab.lib import colors
125     from reportlab.lib.enums import TA_CENTER
126     from reportlab.lib.pagesizes import A4
127     from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
128     from reportlab.lib.units import mm
129     from reportlab.platypus import Paragraph, SimpleDocTemplate, Spacer
130
131     lines = (SCRIPT_DIR / "PROGRAM_DESCRIPTION.md").read_text(
132         encoding="utf-8"
133     ).splitlines()
134     styles = getSampleStyleSheet()
135     title_style = ParagraphStyle(
136         "SubmissionTitle",
137         parent=styles["Title"],
138         fontName="Helvetica-Bold",
139         fontSize=17,
140         leading=20,
141         alignment=TA_CENTER,
142         textColor=colors.HexColor("#17324D"),
143         spaceAfter=5,
144     )
145     heading_style = ParagraphStyle(
146         "SubmissionHeading",
147         parent=styles["Heading2"],
148         fontName="Helvetica-Bold",
149         fontSize=11.5,
150         leading=14,
151         textColor=colors.HexColor("#17324D"),
152         spaceBefore=4,
153         spaceAfter=2,
154     )
155     body_style = ParagraphStyle(
156         "SubmissionBody",
157         parent=styles["BodyText"],
158         fontName="Helvetica",
159         fontSize=10,
160         leading=12.3,
161         textColor=colors.HexColor("#202A33"),
162         spaceAfter=3,
163     )
164
165     story = []
166     paragraph: list[str] = []
167
168     def flush_paragraph() -> None:
169         if paragraph:
170             story.append(
171                 Paragraph(markdown_inline(" ".join(paragraph)), body_style)
172             )
173             paragraph.clear()
174
175     for line in lines:
176         stripped = line.strip()
177         if not stripped:
178             flush_paragraph()
179         elif stripped.startswith("# "):
180             flush_paragraph()
181             story.append(Paragraph(markdown_inline(stripped[2:]), title_style))
182         elif stripped.startswith("## "):
183             flush_paragraph()
184             story.append(Paragraph(markdown_inline(stripped[3:]), heading_style))
185         elif stripped.endswith(" "):
186             paragraph.append(stripped[:-2] + "<br/>")
187         else:
188             paragraph.append(stripped)
189     flush_paragraph()
190     story.append(Spacer(1, 1 * mm))
191
192     document = SimpleDocTemplate(
193         str(output_path),
194         pagesize=A4,
195         rightMargin=15 * mm,
196         leftMargin=15 * mm,
197         topMargin=13 * mm,
198         bottomMargin=13 * mm,
199         title="MScProject Program Description",
200         author="Tanatswa Mapfumo",

```

```

201     subject="Evidence-grounded table-to-text generation software",
202 )
203 document.build(story)
204
205
206 def stage_submission(wheel: Path, description_pdf: Path) -> None:
207     if STAGE_PARENT.exists():
208         shutil.rmtree(STAGE_PARENT)
209     STAGE_ROOT.mkdir(parents=True)
210
211     for source_name, destination_name in SUBMISSION_DOCUMENTS.items():
212         copy_file(SCRIPT_DIR / source_name, STAGE_ROOT / destination_name)
213     copy_file(description_pdf, STAGE_ROOT / "PROGRAM_DESCRIPTION.pdf")
214     copy_file(wheel, STAGE_ROOT / "executable" / wheel.name)
215
216     for relative in tracked_source_files():
217         copy_file(
218             PROJECT_ROOT / relative,
219             STAGE_ROOT / "source" / "MScProject" / relative,
220         )
221
222     for example in sorted((SCRIPT_DIR / "examples").iterdir()):
223         if example.is_file():
224             copy_file(example, STAGE_ROOT / "examples" / example.name)
225
226     listing_candidates = [
227         PROJECT_ROOT
228         / "submission/program_listings/Tanatswa Mapfumo Project Code Printout.pdf",
229         PROJECT_ROOT
230         / "submission/program_listings/MScProject_Program_Listings.pdf",
231     ]
232     listing_pdf = next((path for path in listing_candidates if path.is_file()), None)
233     if listing_pdf is not None:
234         copy_file(
235             listing_pdf,
236             STAGE_ROOT / "documentation" / listing_pdf.name,
237         )
238
239     commit = subprocess.run(
240         ["git", "rev-parse", "HEAD"],
241         cwd=PROJECT_ROOT,
242         check=True,
243         capture_output=True,
244         text=True,
245     ).stdout.strip()
246     version_text = (
247         "MScProject software submission\n"
248         "Package: table2text-pydanticai 0.1.0\n"
249         f"Git commit: {commit}\n"
250         f"Archive build date: {date.today().isoformat()}\n"
251         f"Builder Python: {platform.python_version()}\n"
252         f"Builder platform: {platform.platform()}\n"
253     )
254     (STAGE_ROOT / "VERSION.txt").write_text(version_text, encoding="utf-8")
255
256
257 def write_manifests() -> None:
258     payload_files = sorted(
259         path
260         for path in STAGE_ROOT.rglob("*")
261         if path.is_file()
262         and path.name not in {"SOFTWARE_MANIFEST.json", "CHECKSUMS.sha256"}
263     )
264     records = [
265         {
266             "path": path.relative_to(STAGE_ROOT).as_posix(),
267             "bytes": path.stat().st_size,
268             "sha256": sha256(path),
269         }
270         for path in payload_files
271     ]
272     manifest = {
273         "archive_root": ARCHIVE_ROOT_NAME,
274         "file_count": len(records),
275         "total_payload_bytes": sum(record["bytes"] for record in records),
276         "files": records,
277     }
278     (STAGE_ROOT / "SOFTWARE_MANIFEST.json").write_text(
279         json.dumps(manifest, indent=2) + "\n",
280         encoding="utf-8",
281     )
282     checksum_lines = [
283         f"{record['sha256']} {record['path']}" for record in records
284     ]
285     (STAGE_ROOT / "CHECKSUMS.sha256").write_text(
286         "\n".join(checksum_lines) + "\n",
287         encoding="utf-8",
288     )
289
290
291 def build_zip() -> None:

```

```

292 temporary_archive = BUILD_DIR / OUTPUT_ARCHIVE.name
293 if temporary_archive.exists():
294     temporary_archive.unlink()
295 with zipfile.ZipFile(
296     temporary_archive,
297     "w",
298     compression=zipfile.ZIP_DEFLATED,
299     compresslevel=9,
300 ) as archive:
301     for path in sorted(STAGE_ROOT.rglob("*")):
302         if path.is_file():
303             archive.write(
304                 path,
305                 (Path(ARCHIVE_ROOT_NAME) / path.relative_to(STAGE_ROOT)).as_posix(),
306             )
307 shutil.copy2(temporary_archive, OUTPUT_ARCHIVE)
308 OUTPUT_CHECKSUM.write_text(
309     f"{sha256(OUTPUT_ARCHIVE)} {OUTPUT_ARCHIVE.name}\n",
310     encoding="utf-8",
311 )
312
313
314 def main() -> None:
315     BUILD_DIR.mkdir(parents=True, exist_ok=True)
316     generated_dir = BUILD_DIR / "generated"
317     generated_dir.mkdir(parents=True, exist_ok=True)
318
319     print("Building Python wheel...", flush=True)
320     wheel = build_wheel()
321
322     print("Generating one-page program description...", flush=True)
323     description_pdf = generated_dir / "PROGRAM_DESCRIPTION.pdf"
324     build_program_description_pdf(description_pdf)
325
326     print("Staging maintained software and evidence...", flush=True)
327     stage_submission(wheel, description_pdf)
328     write_manifests()
329
330     print("Compressing submission archive...", flush=True)
331     build_zip()
332
333     print(f"Archive: {OUTPUT_ARCHIVE}")
334     print(f"SHA-256: {sha256(OUTPUT_ARCHIVE)}")
335     print(
336         "Archived files:",
337         len([path for path in STAGE_ROOT.rglob("*") if path.is_file()]),
338     )
339
340
341 if __name__ == "__main__":
342     main()

```

Listing 8: P08: submission/software_archive/examples/run_deterministic_demo.sh

```

1  #!/usr/bin/env bash
2  set -euo pipefail
3
4  SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
5  SUBMISSION_ROOT="$(cd "${SCRIPT_DIR}/.." && pwd)"
6
7  cd "${SUBMISSION_ROOT}"
8
9  table2text run examples/weather_sample.csv \
10 --request "Describe the dataset and report its strongest supported findings." \
11 --no-llm \
12 --output-dir demo_runs

```

Listing 9: P09: table2text_pydanticai/evaluation/notebooks/build_dissertation_visual_evaluations.py

```

1  """Build the dissertation visual-evaluation notebook.
2
3  This script writes a reproducible Jupyter notebook that turns the saved
4  evaluation artifacts into dissertation-ready figures. The focus is output
5  quality, factual support, reference alignment, model-strength comparisons,
6  and ablation evidence. Runtime and cost are intentionally not foregrounded.
7  """
8
9  from __future__ import annotations
10
11 import json
12 from pathlib import Path
13
14
15 PROJECT_DIR = Path(__file__).resolve().parents[2]
16 NOTEBOOK_PATH = PROJECT_DIR / "evaluation/notebooks/dissertation_visual_evaluations.ipynb"
17

```

```

18
19 def markdown(source: str) -> dict:
20     return {
21         "cell_type": "markdown",
22         "metadata": {},
23         "source": source.strip() + "\n",
24     }
25
26
27 def code(source: str) -> dict:
28     return {
29         "cell_type": "code",
30         "execution_count": None,
31         "metadata": {},
32         "outputs": [],
33         "source": source.strip() + "\n",
34     }
35
36
37 SETUP_CODE = r'''
38 from pathlib import Path
39 import json
40 import math
41 from collections import defaultdict
42
43 import numpy as np
44 import pandas as pd
45 import matplotlib.pyplot as plt
46 from IPython.display import Markdown, display
47
48 project_dir = Path("/Users/realgobs/Documents/MScproject/table2text_pydanticai")
49 fig_dir = project_dir / "evaluation/results/figures"
50 fig_dir.mkdir(parents=True, exist_ok=True)
51
52 main_generations_path = project_dir / "evaluation/generations/"
53     five_dataset_five_each_raw_generic_flash_20260805_181001_combined_generations.jsonl"
54 main_reference_metrics_path = project_dir / "evaluation/results/"
55     five_dataset_five_each_raw_generic_flash_20260805_181001_reference_metrics.jsonl"
56 main_source_metrics_path = project_dir / "evaluation/results/"
57     five_dataset_five_each_raw_generic_flash_20260805_181001_source_grounded_metrics.jsonl"
58
59 ablation_generations_path = project_dir / "evaluation/generations/"
60     ablation_sportsett_4934_20260805_021058_combined_generations.jsonl"
61 ablation_reference_metrics_path = project_dir / "evaluation/results/"
62     ablation_sportsett_4934_20260805_021058_reference_metrics.jsonl"
63
64 pro_generations_path = project_dir / "evaluation/generations/"
65     four_dataset_pro_comparison_20260812_215239_combined_generations.jsonl"
66 pro_reference_metrics_path = project_dir / "evaluation/results/"
67     four_dataset_pro_comparison_20260812_215239_reference_metrics_combined.jsonl"
68
69 plt.rcParams.update({
70     "figure.facecolor": "white",
71     "axes.facecolor": "white",
72     "axes.edgecolor": "#333333",
73     "axes.labelcolor": "#222222",
74     "xtick.color": "#222222",
75     "ytick.color": "#222222",
76     "font.size": 10,
77     "axes.titlesize": 14,
78     "axes.titleweight": "bold",
79     "axes.grid": True,
80     "grid.color": "#e8e8e8",
81     "grid.linewidth": 0.8,
82     "legend.frameon": False,
83 })
84
85 variant_labels = {
86     "full_system": "Full System",
87     "raw_generic_flash": "Raw Flash",
88     "raw_deepseek_v4_flash": "Raw Flash",
89     "full_system_pro": "Full System Pro",
90     "raw_generic_pro": "Raw Pro",
91     "no_insight_synthesis": "No Insight Synthesis",
92     "no_writer_quality_revision": "No Writer Quality",
93     "no_audit_repair_rounds": "No Audit Repair",
94 }
95
96 variant_colors = {
97     "full_system": "#0072B2",
98     "raw_generic_flash": "#D55E00",
99     "raw_deepseek_v4_flash": "#D55E00",
100     "full_system_pro": "#009E73",
101     "raw_generic_pro": "#CC79A7",
102     "no_insight_synthesis": "#E69F00",
103     "no_writer_quality_revision": "#6A3D9A",
104     "no_audit_repair_rounds": "#6B7280",
105 }
106
107 metric_labels = {
108     "bleu": "BLEU",

```

```

102     "chrF": "chrF",
103     "rougeL": "ROUGE-L",
104     "meteor": "METEOR",
105     "bertscore_f1": "BERTScore F1",
106     "ter": "TER",
107     "alignscore_base": "AlignScore",
108     "hhem_2_1_open_mean_support": "HHEM mean support",
109     "hhem_2_1_open_min_sentence_support": "HHEM min sentence",
110     "hhem_2_1_open_unsupported_sentence_rate": "Unsupported sentence rate",
111 }
112
113 core_reference_metrics = ["bleu", "chrF", "rougeL", "meteor", "bertscore_f1"]
114 semantic_content_metrics = ["chrF", "rougeL", "meteor", "bertscore_f1"]
115 source_metrics = [
116     "alignscore_base",
117     "hhem_2_1_open_mean_support",
118     "hhem_2_1_open_min_sentence_support",
119     "hhem_2_1_open_unsupported_sentence_rate",
120 ]
121
122 def read_jsonl(path):
123     path = Path(path)
124     if not path.exists():
125         print(f"Missing artifact: {path}")
126         return pd.DataFrame()
127     rows = []
128     with path.open("r", encoding="utf-8") as handle:
129         for line in handle:
130             line = line.strip()
131             if line:
132                 rows.append(json.loads(line))
133     return pd.DataFrame(rows)
134
135 def scored(frame, metrics=None):
136     if frame.empty:
137         return frame.copy()
138     out = frame[frame["status"].eq("scored")].copy()
139     if metrics is not None:
140         out = out[out["metric_name"].isin(metrics)]
141     return out
142
143 def metric_macro(frame, metrics=None, variants=None):
144     data = scored(frame, metrics)
145     if data.empty:
146         return pd.DataFrame()
147     macro = (
148         data.groupby(["variant_id", "metric_name"], as_index=False)
149         .agg(score=("score", "mean"), higher_is_better=("higher_is_better", "first"))
150     )
151     if variants is not None:
152         macro = macro[macro["variant_id"].isin(variants)]
153     return macro
154
155 def metric_pivot(frame, metrics=None):
156     data = scored(frame, metrics)
157     if data.empty:
158         return pd.DataFrame()
159     return data.pivot_table(
160         index=["dataset_id", "example_id", "metric_name"],
161         columns="variant_id",
162         values="score",
163         aggfunc="mean",
164     ).reset_index()
165
166 def direction_for_metric(frame, metric_name):
167     subset = frame[frame["metric_name"].eq(metric_name)]
168     if subset.empty:
169         return 1
170     return 1 if bool(subset["higher_is_better"].iloc[0]) else -1
171
172 def directional_gain(full_score, raw_score, higher_is_better=True):
173     if pd.isna(full_score) or pd.isna(raw_score):
174         return np.nan
175     return (full_score - raw_score) if higher_is_better else (raw_score - full_score)
176
177 def oriented_score(score, higher_is_better=True):
178     if pd.isna(score):
179         return np.nan
180     if higher_is_better:
181         return score
182     return 1 / (1 + max(score, 0))
183
184 def savefig(name):
185     path = fig_dir / name
186     plt.tight_layout()
187     plt.savefig(path, dpi=220, bbox_inches="tight")
188     plt.show()
189     print(f"Saved: {path}")
190
191 def word_count(text):
192     return len((text or "").split())

```

```

193
194 def reference_word_count(refs):
195     if isinstance(refs, list) and refs:
196         return float(np.mean([word_count(ref) for ref in refs]))
197     return np.nan
198
199 main_generations = read_jsonl(main_generations_path)
200 main_reference_metrics = read_jsonl(main_reference_metrics_path)
201 main_source_metrics = read_jsonl(main_source_metrics_path)
202 ablation_generations = read_jsonl(ablation_generations_path)
203 ablation_reference_metrics = read_jsonl(ablation_reference_metrics_path)
204 pro_generations = read_jsonl(pro_generations_path)
205 pro_reference_metrics = read_jsonl(pro_reference_metrics_path)
206
207 summary = pd.DataFrame([
208     {"artifact": "main_generations", "rows": len(main_generations)},
209     {"artifact": "main_reference_metrics", "rows": len(main_reference_metrics)},
210     {"artifact": "main_source_metrics", "rows": len(main_source_metrics)},
211     {"artifact": "ablation_generations", "rows": len(ablation_generations)},
212     {"artifact": "ablation_reference_metrics", "rows": len(ablation_reference_metrics)},
213     {"artifact": "pro_generations", "rows": len(pro_generations)},
214     {"artifact": "pro_reference_metrics", "rows": len(pro_reference_metrics)},
215 ])
216 display(summary)
217 '''
218
219
220 CELLS = [
221     markdown(
222         '''
223         # Dissertation Visual Evaluations
224
225         This notebook builds ten dissertation-ready evaluation visuals from the saved experiment artifacts.
226         It deliberately focuses on output quality rather than time or cost: reference alignment, semantic similarity,
227         source-grounded factuality, metric-family behaviour, ablation evidence, narration/coverage, and model-strength
228         effects.
229
230         The figures are saved to `evaluation/results/figures/`.
231         ''',
232     ),
233     code(SETUP_CODE),
234     markdown(
235         '''
236         ## 1. Main 25-Example Macro Reference Metrics
237
238         This line chart shows the headline comparison between the full architecture and the raw generic Flash baseline
239         across the shortlisted reference-alignment metrics, excluding TER because TER has the opposite interpretation.
240         ''',
241     ),
242     code(
243         r'''
244         macro = metric_macro(main_reference_metrics, core_reference_metrics, ["full_system", "raw_generic_flash"])
245         plot_data = macro.pivot(index="metric_name", columns="variant_id", values="score").reindex(core_reference_metrics)
246
247         fig, ax = plt.subplots(figsize=(10.8, 5.8))
248         x = np.arange(len(plot_data.index))
249         for variant in ["full_system", "raw_generic_flash"]:
250             values = plot_data[variant].values
251             ax.plot(
252                 x,
253                 values,
254                 marker="o",
255                 linewidth=2.8,
256                 markersize=8,
257                 label=variant_labels[variant],
258                 color=variant_colors[variant],
259             )
260             for xi, value in zip(x, values):
261                 ax.text(xi, value + 0.012, f"{value:.3f}", ha="center", va="bottom", fontsize=8, color=variant_colors[variant])
262
263         ax.set_title("Macro Reference Alignment: Full System vs Raw Flash")
264         ax.set_ylabel("Mean score, higher is better")
265         ax.set_xticks(x)
266         ax.set_xticklabels([metric_labels[m] for m in plot_data.index])
267         ax.set_ylim(0, max(1.0, np.nanmax(plot_data.values) * 1.15))
268         ax.margins(x=0.08)
269         ax.legend(loc="upper left")
270         savefig("01_main_macro_reference_metrics.png")
271         display(plot_data.rename(index=metric_labels, columns=variant_labels))
272         ''',
273     ),
274     markdown(
275         '''
276         ## 2. Main 25-Example TER Comparison
277
278         TER is kept separate because lower is better. The slope format makes the edit-distance reduction easier to read.
279
280         ''',
281     ),
282     code(
283         r'''

```

```

282 ter_macro = metric_macro(main_reference_metrics, ["ter"], ["full_system", "raw_generic_flash"])
283 ter_plot = ter_macro.pivot(index="metric_name", columns="variant_id", values="score")
284
285 fig, ax = plt.subplots(figsize=(7.5, 5.2))
286 variants = ["full_system", "raw_generic_flash"]
287 values = [float(ter_plot.loc["ter", variant]) for variant in variants]
288 ax.plot(
289     [0, 1],
290     values,
291     color="#334155",
292     linewidth=2.5,
293     marker="o",
294     markersize=9,
295 )
296 for xi, variant, value in zip([0, 1], variants, values):
297     ax.scatter([xi], [value], s=120, color=variant_colors[variant], zorder=3)
298     ax.text(xi, value + max(values) * 0.035, f"{value:.3f}", ha="center", fontsize=9)
299 if len(values) == 2:
300     ax.text(0.5, max(values) * 0.92, f"directional improvement: {values[1] - values[0]:.3f}", ha="center", color
301         ="#334155")
302 ax.set_title("TER: Lower Means Closer Surface Realisation")
303 ax.set_ylabel("Mean TER, lower is better")
304 ax.set_xticks([0, 1])
305 ax.set_xticklabels([variant_labels[v] for v in variants])
306 ax.set_ylim(0, max(values) * 1.25)
307 savefig("02_main_macro_ter.png")
308 display(ter_plot.rename(columns=variant_labels))
309
310 ),
311 markdown(
312     """
313     ## 3. Per-Dataset Architecture-Gain Heatmap
314
315     Positive values mean the full system beat the raw generic baseline after respecting each metric's direction.
316     This makes it easy to see where the architecture helps most and where more work is needed.
317     """
318 ),
319 code(
320     r'''
321     heat_metrics = ["bleu", "chrF", "rougeL", "meteor", "bertscore_f1", "ter"]
322     pivot = metric_pivot(main_reference_metrics, heat_metrics)
323     gain_rows = []
324     for _, row in pivot.iterrows():
325         metric = row["metric_name"]
326         if "full_system" not in row or "raw_generic_flash" not in row:
327             continue
328         higher = direction_for_metric(main_reference_metrics, metric) == 1
329         gain_rows.append({
330             "dataset_id": row["dataset_id"],
331             "metric_name": metric,
332             "directional_gain": directional_gain(row.get("full_system"), row.get("raw_generic_flash"), higher),
333         })
334     gain_df = pd.DataFrame(gain_rows)
335     heat = gain_df.pivot_table(index="dataset_id", columns="metric_name", values="directional_gain", aggfunc="mean").
336         reindex(columns=heat_metrics)
337
338     fig, ax = plt.subplots(figsize=(11, 5.8))
339     vmax = np.nanmax(np.abs(heat.values))
340     im = ax.imshow(heat.values, cmap="RdYlGn", vmin=-vmax, vmax=vmax, aspect="auto")
341     ax.set_title("Directional Full-System Gain By Dataset")
342     ax.set_xticks(np.arange(len(heat.columns)))
343     ax.set_xticklabels([metric_labels[m] for m in heat.columns], rotation=30, ha="right")
344     ax.set_yticks(np.arange(len(heat.index)))
345     ax.set_yticklabels(heat.index)
346     for i in range(heat.shape[0]):
347         for j in range(heat.shape[1]):
348             value = heat.iloc[i, j]
349             if pd.notna(value):
350                 ax.text(j, i, f"{value:+.3f}", ha="center", va="center", fontsize=8)
351     cbar = fig.colorbar(im, ax=ax, shrink=0.85)
352     cbar.set_label("Directional gain over raw baseline")
353     savefig("03_architecture_gain_heatmap.png")
354     display(heat)
355
356     ),
357     markdown(
358         """
359         ## 4. Per-Dataset Semantic And Content Dumbbell Chart
360
361         This chart focuses on chrF and BERTScore F1 because they tell a useful dissertation story:
362         chrF captures character-level content overlap, while BERTScore captures semantic similarity.
363         """
364     ),
365     code(
366         r'''
367         dumb_metrics = ["chrF", "bertscore_f1"]
368         dumb_data = (
369             scored(main_reference_metrics, dumb_metrics)
370             .groupby(["dataset_id", "metric_name", "variant_id"], as_index=False)["score"].mean()
371         )

```



```

371 fig, axes = plt.subplots(1, 2, figsize=(13, 5.8), sharey=True)
372 datasets = sorted(dumb_data["dataset_id"].unique())
373 for ax, metric in zip(axes, dumb_metrics):
374     subset = dumb_data[dumb_data["metric_name"].eq(metric)]
375     for yi, dataset in enumerate(datasets):
376         vals = subset[subset["dataset_id"].eq(dataset)].set_index("variant_id")["score"]
377         raw = vals.get("raw_generic_flash", np.nan)
378         full = vals.get("full_system", np.nan)
379         ax.plot([raw, full], [yi, yi], color="#94a3b8", linewidth=2)
380         ax.scatter(raw, yi, color=variant_colors["raw_generic_flash"], s=80, label=variant_labels["raw_generic_flash"])
381         if yi == 0 else ""
382         ax.scatter(full, yi, color=variant_colors["full_system"], s=80, label=variant_labels["full_system"]) if yi == 0
383         else ""
384         if pd.notna(raw) and pd.notna(full):
385             ax.text(max(raw, full) + 0.004, yi, f"{full-raw:+.3f}", va="center", fontsize=8)
386         ax.set_title(metric_labels[metric])
387         ax.set_xlabel("Score")
388         ax.set_yticks(np.arange(len(datasets)))
389         ax.set_yticklabels(datasets)
390         ax.set_xlim(max(0, np.nanmin(subset["score"]) - 0.05), min(1.02, np.nanmax(subset["score"]) + 0.06))
391 axes[0].legend(loc="lower right")
392 fig.suptitle("Semantic And Content Alignment By Dataset", fontsize=15, fontweight="bold")
393 savefig("04_semantic_content_dumbbell.png")
394 display(dumb_data.pivot_table(index=["dataset_id", "metric_name"], columns="variant_id", values="score"))
395 '''
396 ),
397 markdown(
398     """
399     ## 5. Metric-Family Profile
400
401     This line profile groups the metrics into dissertation-friendly families: lexical overlap, sequence/content
402     overlap, semantic similarity, and edit efficiency. TER is converted to a higher-is-better complement here only.
403     """
404 ),
405 code(
406     r'''
407 family_map = {
408     "bleu": "Lexical overlap",
409     "chrF": "Lexical overlap",
410     "rougeL": "Sequence/content overlap",
411     "meteor": "Sequence/content overlap",
412     "bertscore_f1": "Semantic similarity",
413     "ter": "Edit efficiency",
414 }
415 family_metrics = list(family_map)
416 family_rows = []
417 for _, row in scored(main_reference_metrics, family_metrics).iterrows():
418     family_rows.append({
419         "variant_id": row["variant_id"],
420         "metric_family_label": family_map[row["metric_name"]],
421         "oriented_score": oriented_score(row["score"], bool(row["higher_is_better"])),
422     })
423 family_df = pd.DataFrame(family_rows)
424 family_macro = (
425     family_df.groupby(["variant_id", "metric_family_label"], as_index=False)["oriented_score"].mean()
426 )
427 families = ["Lexical overlap", "Sequence/content overlap", "Semantic similarity", "Edit efficiency"]
428 plot_data = family_macro.pivot(index="metric_family_label", columns="variant_id", values="oriented_score").reindex(
429     families)
430
431 fig, ax = plt.subplots(figsize=(10.8, 5.8))
432 x = np.arange(len(families))
433 for variant in ["full_system", "raw_generic_flash"]:
434     values = plot_data[variant].values
435     ax.plot(
436         x,
437         values,
438         marker="o",
439         linewidth=2.8,
440         markersize=8,
441         label=variant_labels[variant],
442         color=variant_colors[variant],
443     )
444     for xi, value in zip(x, values):
445         ax.text(xi, value + 0.012, f"{value:.3f}", ha="center", va="bottom", fontsize=8, color=variant_colors[variant])
446 ax.set_title("Metric-Family Profile")
447 ax.set_ylabel("Oriented mean score, higher is better")
448 ax.set_xticks(x)
449 ax.set_xticklabels(families, rotation=20, ha="right")
450 ax.set_ylim(0, max(1.0, np.nanmax(plot_data.values) * 1.15))
451 ax.margins(x=0.08)
452 ax.legend(loc="upper left")
453 savefig("05_metric_family_profile.png")
454 display(plot_data.rename(columns=variant_labels))
455 '''
456 ),
457 markdown(
458     """
459     ## 6. Full-System Win Rate By Metric
460

```

```

459     This line chart asks a simple question: across individual examples, how often did the architecture beat the raw
460     generic baseline for each metric?
461     """
462     ),
463     code(
464         r'''
465 win_metrics = ["bleu", "chrF", "rougeL", "meteor", "bertscore_f1", "ter"]
466 pivot = metric_pivot(main_reference_metrics, win_metrics)
467 win_rows = []
468 for metric in win_metrics:
469     subset = pivot[pivot["metric_name"].eq(metric)].copy()
470     higher = direction_for_metric(main_reference_metrics, metric) == 1
471     wins = 0
472     ties = 0
473     comparable = 0
474     for _, row in subset.iterrows():
475         full = row.get("full_system")
476         raw = row.get("raw_generic_flash")
477         if pd.isna(full) or pd.isna(raw):
478             continue
479         comparable += 1
480         diff = full - raw if higher else raw - full
481         if abs(diff) < 1e-9:
482             ties += 1
483         elif diff > 0:
484             wins += 1
485 win_rows.append({
486     "metric_name": metric,
487     "win_rate": wins / comparable if comparable else np.nan,
488     "tie_rate": ties / comparable if comparable else np.nan,
489     "n": comparable,
490 })
491
492 win_df = pd.DataFrame(win_rows)
493 fig, ax = plt.subplots(figsize=(10.8, 5.8))
494 x = np.arange(len(win_df))
495 ax.plot(
496     x,
497     win_df["win_rate"],
498     color=variant_colors["full_system"],
499     marker="o",
500     linewidth=2.8,
501     markersize=8,
502 )
503 ax.fill_between(x, 0.5, win_df["win_rate"], where=win_df["win_rate"] >= 0.5, color="#9BD3EA", alpha=0.45, interpolate=
504 True)
505 for xi, value, n in zip(x, win_df["win_rate"], win_df["n"]):
506     ax.text(xi, value + 0.035, f"{value:.0%}\n(n={n})", ha="center", fontsize=8)
507 ax.axhline(0.5, color="#64748b", linestyle="--", linewidth=1.2)
508 ax.text(len(win_df) - 0.6, 0.52, "50% parity line", color="#64748b", fontsize=9)
509 ax.set_ylim(0, 1.05)
510 ax.set_title("How Often The Full System Beats Raw Flash")
511 ax.set_ylabel("Full-system win rate")
512 ax.set_xticks(x)
513 ax.set_xticklabels([metric_labels[m] for m in win_df["metric_name"]], rotation=20, ha="right")
514 savefig("06_full_system_win_rate_by_metric.png")
515 display(win_df)
516 '''
517     ),
518     markdown(
519         """
520         ## 7. Source-Grounded Factuality Diagnostics
521
522         This line chart compares outputs against the structured source rather than only the human reference.
523         AlignScore and HHEM support are higher-is-better; unsupported sentence rate is lower-is-better.
524         """
525     ),
526     code(
527         r'''
528 source_macro = metric_macro(main_source_metrics, source_metrics, ["full_system", "raw_generic_flash"])
529 plot_data = source_macro.pivot(index="metric_name", columns="variant_id", values="score").reindex(source_metrics)
530
531 fig, ax = plt.subplots(figsize=(11, 5.8))
532 x = np.arange(len(plot_data.index))
533 for variant in ["full_system", "raw_generic_flash"]:
534     values = plot_data[variant].values
535     ax.plot(
536         x,
537         values,
538         marker="o",
539         linewidth=2.8,
540         markersize=8,
541         label=variant_labels[variant],
542         color=variant_colors[variant],
543     )
544     for xi, value in zip(x, values):
545         ax.text(xi, value + 0.012, f"{value:.3f}", ha="center", va="bottom", fontsize=8, color=variant_colors[variant])
546 ax.set_title("Source-Grounded Factuality Diagnostics")
547 ax.set_ylabel("Mean score")
548 ax.set_xticks(x)
549 ax.set_xticklabels([metric_labels[m] for m in plot_data.index], rotation=25, ha="right")

```

```

549 ax.set_ylim(0, max(1.0, np.nanmax(plot_data.values) * 1.15))
550 ax.margins(x=0.08)
551 ax.legend(loc="upper left")
552 savefig("07_source_grounded_diagnostics.png")
553 display(plot_data.rename(index=metric_labels, columns=variant_labels))
554 '''
555 ),
556 markdown(
557     """
558     ## 8. SportSett 4934 Ablation Metric Impact
559
560     This line chart isolates components of the architecture on the same example. Positive values mean the full
561     system scored better than the ablated variant after metric direction is accounted for.
562     """
563 ),
564 code(
565     r'''
566     ab_metrics = ["chrfr", "rougel", "meteor", "bertscore_f1", "ter"]
567     ab_pivot = metric_pivot(ablation_reference_metrics, ab_metrics)
568     ab_variants = [
569         "raw_deepseek_v4_flash",
570         "no_insight_synthesis",
571         "no_writer_quality_revision",
572         "no_audit_repair_rounds",
573     ]
574     rows = []
575     for _, row in ab_pivot.iterrows():
576         metric = row["metric_name"]
577         higher = direction_for_metric(ablation_reference_metrics, metric) == 1
578         full = row.get("full_system")
579         for variant in ab_variants:
580             if variant in row and pd.notna(row.get(variant)):
581                 rows.append({
582                     "variant_id": variant,
583                     "metric_name": metric,
584                     "directional_full_gain": directional_gain(full, row.get(variant), higher),
585                 })
586     ab_gain = pd.DataFrame(rows)
587     ab_heat = ab_gain.pivot_table(index="variant_id", columns="metric_name", values="directional_full_gain", aggfunc="mean")
588     .reindex(ab_variants)
589
590     fig, ax = plt.subplots(figsize=(11.5, 5.8))
591     x = np.arange(len(ab_heat.columns))
592     for variant in ab_variants:
593         values = ab_heat.loc[variant].values
594         ax.plot(
595             x,
596             values,
597             marker="o",
598             linewidth=2.3,
599             markersize=7,
600             label=variant_labels.get(variant, variant),
601             color=variant_colors.get(variant),
602         )
603     ax.axhline(0, color="#334155", linewidth=1.2)
604     ax.set_title("Ablation Impact: Full-System Gain Over Each Variant")
605     ax.set_ylabel("Directional full-system gain")
606     ax.set_xticks(x)
607     ax.set_xticklabels([metric_labels[m] for m in ab_heat.columns], rotation=25, ha="right")
608     ax.legend(loc="best")
609     savefig("08_ablation_metric_impact.png")
610     display(ab_heat.rename(index=variant_labels, columns=metric_labels))
611     '''
612 ),
613 markdown(
614     """
615     ## 9. Output Coverage And Narrative Scope
616
617     This is not a speed chart. It shows, as a line profile, whether each system tends to under-produce, match,
618     or over-expand relative to the available human references. It is useful for discussing narration and coverage.
619     """
620 ),
621 code(
622     r'''
623     coverage = main_generations.copy()
624     coverage["generated_words"] = coverage["generated_text"].map(word_count)
625     coverage["reference_words"] = coverage["references"].map(reference_word_count)
626     coverage["output_reference_ratio"] = coverage["generated_words"] / coverage["reference_words"]
627
628     coverage_macro = (
629         coverage.groupby(["dataset_id", "variant_id"], as_index=False)
630         .agg(
631             generated_words=("generated_words", "mean"),
632             reference_words=("reference_words", "mean"),
633             output_reference_ratio=("output_reference_ratio", "mean"),
634         )
635     )
636
637     datasets = sorted(coverage_macro["dataset_id"].unique())
638     fig, axes = plt.subplots(1, 2, figsize=(14, 5.8), sharey=True)
639     for ax, column, title, x_label in [

```

```

639     (axes[0], "generated_words", "Mean Generated Length", "Words"),
640     (axes[1], "output_reference_ratio", "Output / Reference Length Ratio", "Ratio"),
641 ]:
642     x = np.arange(len(datasets))
643     for variant in ["full_system", "raw_generic_flash"]:
644         vals = (
645             coverage_macro[coverage_macro["variant_id"].eq(variant)]
646             .set_index("dataset_id")
647             .reindex(datasets)[column]
648         )
649         ax.plot(
650             x,
651             vals,
652             marker="o",
653             linewidth=2.6,
654             markersize=7,
655             label=variant_labels[variant],
656             color=variant_colors[variant],
657         )
658         if column == "output_reference_ratio":
659             ax.axhline(1.0, color="#64748b", linestyle="--", linewidth=1.2)
660             ax.text(len(datasets) - 1.25, 1.04, "reference length", color="#64748b", fontsize=9)
661         ax.set_title(title)
662         ax.set_ylabel(x_label)
663         ax.set_xticks(x)
664         ax.set_xticklabels(datasets, rotation=25, ha="right")
665     axes[0].legend(loc="lower right")
666     fig.suptitle("Narrative Scope And Coverage Profile", fontsize=15, fontweight="bold")
667     savefig("09_output_coverage_profile.png")
668     display(coverage_macro.pivot_table(index="dataset_id", columns="variant_id", values=["generated_words", "
        output_reference_ratio"]))
669     '''
670     ),
671     markdown(
672         """
673         ## 10. Model-Strength Comparison And ToTTo Subject-Linking Case
674
675         This line chart combines the Flash and Pro experiments. It separates two ideas: whether a stronger raw model
676         narrows the gap, and whether the architecture still protects against table-linking errors that a raw model can
        make.
677         """
678     ),
679     code(
680         r'''
681         main_4 = scored(main_reference_metrics, ["chr", "rougeL", "meteor", "bertscore_f1"])
682         main_4 = main_4[main_4["dataset_id"].isin(["e2e_nlg", "totto", "web_nlg", "dart"])]
683         main_4 = main_4[main_4["variant_id"].isin(["full_system", "raw_generic_flash"])]
684         pro_4 = scored(pro_reference_metrics, ["chr", "rougeL", "meteor", "bertscore_f1"])
685
686         combined = pd.concat([main_4, pro_4], ignore_index=True)
687         macro = (
688             combined.groupby(["variant_id", "metric_name"], as_index=False)["score"].mean()
689         )
690         plot = macro.pivot(index="metric_name", columns="variant_id", values="score").reindex(["chr", "rougeL", "meteor", "
            bertscore_f1"])
691         variant_order = ["full_system", "raw_generic_flash", "full_system_pro", "raw_generic_pro"]
692
693         fig, ax = plt.subplots(figsize=(12.5, 5.8))
694         x = np.arange(len(plot.index))
695         for variant in variant_order:
696             if variant not in plot:
697                 continue
698             ax.plot(
699                 x,
700                 plot[variant].values,
701                 marker="o",
702                 linewidth=2.4,
703                 markersize=7,
704                 label=variant_labels[variant],
705                 color=variant_colors[variant],
706             )
707         ax.set_title("Flash/Pro Model Strength Across Four Matched Datasets")
708         ax.set_ylabel("Mean score, higher is better")
709         ax.set_xticks(x)
710         ax.set_xticklabels([metric_labels[m] for m in plot.index])
711         ax.set_ylim(0, max(1.0, np.nanmax(plot.values) * 1.12))
712         ax.legend(ncols=2, loc="lower right")
713         savefig("10_model_strength_comparison.png")
714         display(plot.rename(index=metric_labels, columns=variant_labels))
715
716         totto = scored(combined, ["chr", "meteor", "bertscore_f1"])
717         totto = totto[
718             (totto["dataset_id"].eq("totto"))
719             & (totto["example_id"].astype(str).endswith("204"))
720         ]
721         totto_plot = totto.pivot_table(index="metric_name", columns="variant_id", values="score", aggfunc="mean").reindex(["
            chr", "meteor", "bertscore_f1"])
722
723         fig, ax = plt.subplots(figsize=(10, 5.4))
724         x = np.arange(len(totto_plot.index))
725         for variant in variant_order:

```

```

726     if variant in tutto_plot:
727         ax.plot(
728             x,
729             tutto_plot[variant].values,
730             marker="o",
731             linewidth=2.4,
732             markersize=7,
733             label=variant_labels[variant],
734             color=variant_colors[variant],
735         )
736 ax.set_title("ToTTo 204: Subject-Linking Stress Case")
737 ax.set_ylabel("Score")
738 ax.set_xticks(x)
739 ax.set_xticklabels([metric_labels[m] for m in tutto_plot.index])
740 upper = 1.0 if tutto_plot.empty else max(1.0, np.nanmax(tutto_plot.values) * 1.12)
741 ax.set_ylim(0, upper)
742 ax.legend(ncols=2, loc="lower right")
743 savefig("10b_tutto_subject_linking_case.png")
744
745 display(Markdown("""
746 **Qualitative note:** in this ToTTo example, the architecture links the highlighted percentage to Ma Ying-jeou,
747 while the raw Pro baseline can still attach the same value to Vincent Siew. This is a useful dissertation caveat:
748 stronger models improve fluency and sometimes overlap, but they do not remove the need for structure-aware grounding.
749 """))
750 display(tutto_plot.rename(index=metric_labels, columns=variant_labels))
751 '''
752 ),
753 markdown(
754     """
755     ## Figure Files
756
757     The notebook saves these dissertation-focused figures:
758
759     1. `01_main_macro_reference_metrics.png`
760     2. `02_main_macro_ter.png`
761     3. `03_architecture_gain_heatmap.png`
762     4. `04_semantic_content_dumbbell.png`
763     5. `05_metric_family_profile.png`
764     6. `06_full_system_win_rate_by_metric.png`
765     7. `07_source_grounded_diagnostics.png`
766     8. `08_ablation_metric_impact.png`
767     9. `09_output_coverage_profile.png`
768     10. `10_model_strength_comparison.png`
769
770     A supporting case-study figure is also saved as `10b_tutto_subject_linking_case.png`.
771     """)
772 ),
773 ]
774
775 def main() -> None:
776     NOTEBOOK_PATH.parent.mkdir(parents=True, exist_ok=True)
777     notebook = {
778         "cells": CELLS,
779         "metadata": {
780             "kernelspec": {
781                 "display_name": "Python 3",
782                 "language": "python",
783                 "name": "python3",
784             },
785             "language_info": {
786                 "name": "python",
787                 "pygments_lexer": "ipython3",
788             },
789         },
790         "nbformat": 4,
791         "nbformat_minor": 5,
792     }
793     NOTEBOOK_PATH.write_text(json.dumps(notebook, indent=2), encoding="utf-8")
794     print(f"Wrote {NOTEBOOK_PATH}")
795
796 if __name__ == "__main__":
797     main()

```

Listing 10: P10: table2text_pydtanticalai/evaluation/notebooks/build_protected_holdout_notebook.py

```

1  """Regenerate the protected-holdout notebook from its percent-format source."""
2
3  from __future__ import annotations
4
5  import ast
6  import json
7  import re
8  from pathlib import Path
9
10
11 SOURCE_PATH = Path(__file__).with_name(
12     "protected_holdout_full_system_evaluation.py")

```

```

13 )
14 TARGET_PATH = SOURCE_PATH.with_suffix(".ipynb")
15
16
17 def markdown_source(body: str) -> list[str]:
18     lines: list[str] = []
19     for line in body.splitlines(keepends=True):
20         if line.startswith("# "):
21             lines.append(line[2:])
22         elif line.startswith("#"):
23             lines.append(line[1:])
24         else:
25             lines.append(line)
26     return "".join(lines).splitlines(keepends=True)
27
28
29 def main() -> None:
30     source = SOURCE_PATH.read_text(encoding="utf-8")
31     parts = re.split(r"(?m)^# %%([^\n]*)\n", source)
32     cells: list[dict[str, object]] = []
33
34     for cell_number, index in enumerate(range(1, len(parts), 2), start=1):
35         marker = parts[index].strip()
36         body = parts[index + 1]
37         if "[markdown]" in marker:
38             cells.append(
39                 {
40                     "cell_type": "markdown",
41                     "metadata": {},
42                     "source": markdown_source(body),
43                 }
44             )
45             continue
46
47         compile(
48             body,
49             f"{SOURCE_PATH}:cell-{cell_number}",
50             "exec",
51             flags=ast.PyCF_ALLOW_TOP_LEVEL_AWAIT,
52         )
53         cells.append(
54             {
55                 "cell_type": "code",
56                 "execution_count": None,
57                 "metadata": {},
58                 "outputs": [],
59                 "source": body.splitlines(keepends=True),
60             }
61         )
62
63     notebook = {
64         "cells": cells,
65         "metadata": {
66             "kernelspec": {
67                 "display_name": "Python 3 (ipykernel)",
68                 "language": "python",
69                 "name": "python3",
70             },
71             "language_info": {
72                 "name": "python",
73                 "version": "3.11",
74                 "mimetype": "text/x-python",
75                 "codemirror_mode": {"name": "ipython", "version": 3},
76                 "pygments_lexer": "ipython3",
77                 "nbconvert_exporter": "python",
78                 "file_extension": ".py",
79             },
80         },
81         "nbformat": 4,
82         "nbformat_minor": 5,
83     }
84     TARGET_PATH.write_text(
85         json.dumps(notebook, indent=1, ensure_ascii=False) + "\n",
86         encoding="utf-8",
87     )
88     print(f"Wrote {TARGET_PATH} with {len(cells)} cells.")
89
90
91 if __name__ == "__main__":
92     main()

```

Listing 11: P11: table2text_pydanticai/evaluation/notebooks/protected_holdout_full_system_evaluation.py

```

1 # %% [markdown]
2 # # Protected 25-Example Full-System Evaluation
3 #
4 # This notebook creates and runs a frozen, previously unused holdout batch:
5 #

```

```

6 # - five examples from each of E2E, WebNLG, DART, ToTTo, and SportSett;
7 # - one Full-System generation per example;
8 # - DeepSeek V4 Flash for all six workflow roles;
9 # - exact task contracts from the prepared benchmark records;
10 # - held-out references physically replaced by a sentinel during generation;
11 # - per-example checkpoints, heartbeat logging, and resumable execution;
12 # - exact source/config/code hashes and complete workflow-artifact indexes;
13 # - sentence support, evidence/fact/insight, retry, audit, and token summaries.
14 #
15 # The selection is written once and never silently recomputed. The notebook
16 # also verifies the frozen implementation fingerprint before every generation.
17 # Reference-based scoring is a separate, disabled-by-default final phase that
18 # cannot run until all 25 generations have completed successfully.
19
20 # %%
21 import ast
22 import asyncio
23 import copy
24 import hashlib
25 import importlib.metadata
26 import json
27 import os
28 import platform
29 import random
30 import re
31 import shutil
32 import subprocess
33 import sys
34 import time
35 from collections import Counter, defaultdict
36 from dataclasses import asdict
37 from datetime import datetime, timezone
38 from pathlib import Path
39
40 import pandas as pd
41 from IPython.display import Markdown, display
42
43 from table2text.config import Settings
44 from table2text.evaluation import (
45     default_paths,
46     generate_reports_for_notebook,
47     load_project_env,
48     score_reference_metrics_for_notebook,
49 )
50 from table2text.evaluation.datasets import read_examples
51 from table2text.evaluation.generation import (
52     materialise_input,
53     read_generations,
54 )
55
56 # =====
57 # User configuration
58 # =====
59
60 PROJECT_DIR = Path("/Users/realgobs/Documents/MScproject/table2text_pydanticai")
61 EXPERIMENT_ID = "protected_holdout_full_system_flash_25"
62
63 DATASETS = [
64     "e2e_nlg",
65     "web_nlg",
66     "dart",
67     "totto",
68     "sportsett_basketball",
69 ]
70
71 EXAMPLES_PER_DATASET = 5
72 SELECTION_SEED = 20260820
73 GENERATION_SEED = 42
74 MODEL = "deepseek:deepseek-v4-flash"
75
76 # Leave as None to run all unfinished examples. Set to 5, for example, to
77 # process one interleaved block and resume later without changing selection.
78 MAX_CASES_THIS_SESSION = None
79
80 HEARTBEAT_SECONDS = 20
81 MAX_TOP_LEVEL_ATTEMPTS_PER_EXAMPLE = 2
82 CONTINUE_AFTER_FAILURE = True
83 PRINT_FINAL_OUTPUTS = True
84
85 # These declarations are copied into the protected-set manifest. Keep them
86 # True only if they are factually correct at the moment this notebook is run.
87 RESEARCHER_CONFIRMS_NO_PRIOR_MANUAL_INSPECTION = True
88 RESEARCHER_CONFIRMS_NO_DEVELOPMENT_CHANGES_AFTER_SELECTION = True
89
90 # References are unsealed only after all 25 generations succeed. These metric
91 # phases are deliberately opt-in and do not alter any workflow output.
92 RUN_POST_GENERATION_REFERENCE_METRICS = False
93 RUN_POST_GENERATION_SOURCE_METRICS = False
94
95 # =====

```



```

97 # Paths
98 # =====
99
100 PATHS = default_paths(PROJECT_DIR)
101 ARTIFACT_DIR = PROJECT_DIR / "evaluation" / "protected_holdout_full_system"
102 CONFIG_DIR = ARTIFACT_DIR / "config"
103 PREPARED_DIR = ARTIFACT_DIR / "prepared"
104 GENERATION_DIR = ARTIFACT_DIR / "generations"
105 SHARD_DIR = GENERATION_DIR / "shards"
106 ATTEMPT_RECORD_DIR = GENERATION_DIR / "attempt_records"
107 RUN_ROOT = GENERATION_DIR / "runs"
108 RESULT_DIR = ARTIFACT_DIR / "results"
109 SNAPSHOT_DIR = ARTIFACT_DIR / "frozen_code_snapshot"
110 MODEL_INPUT_AUDIT_DIR = ARTIFACT_DIR / "model_input_audit"
111
112 for directory in (
113     CONFIG_DIR,
114     PREPARED_DIR,
115     GENERATION_DIR,
116     SHARD_DIR,
117     ATTEMPT_RECORD_DIR,
118     RUN_ROOT,
119     RESULT_DIR,
120     SNAPSHOT_DIR,
121     MODEL_INPUT_AUDIT_DIR,
122 ):
123     directory.mkdir(parents=True, exist_ok=True)
124
125 VARIANT_PATH = CONFIG_DIR / "protected_full_system_flash.json"
126 FREEZE_MANIFEST_PATH = SNAPSHOT_DIR / "freeze_manifest.json"
127 SELECTION_MANIFEST_PATH = PREPARED_DIR / "protected_selection_manifest.json"
128 OPERATIONAL_EXAMPLES_PATH = PREPARED_DIR / "protected_operational_examples.jsonl"
129 EXCLUSION_MANIFEST_PATH = PREPARED_DIR / "historical_exclusions.json"
130 BATCH_MANIFEST_PATH = RESULT_DIR / "protected_batch_manifest.json"
131 PROGRESS_LOG_PATH = RESULT_DIR / "protected_progress.log"
132 ATTEMPT_LOG_PATH = RESULT_DIR / "generation_attempts.jsonl"
133 SEALED_GENERATIONS_PATH = GENERATION_DIR / "protected_full_system_generations_sealed.jsonl"
134 UNSEALED_GENERATIONS_PATH = GENERATION_DIR / "protected_full_system_generations_post_generation.jsonl"
135 SUMMARY_JSONL_PATH = RESULT_DIR / "protected_generation_summary.jsonl"
136 SUMMARY_CSV_PATH = RESULT_DIR / "protected_generation_summary.csv"
137 STAGE_USAGE_CSV_PATH = RESULT_DIR / "stage_token_usage.csv"
138 SUPPORT_MAP_JSONL_PATH = RESULT_DIR / "sentence_support_mappings.jsonl"
139 ARTIFACT_INDEX_PATH = RESULT_DIR / "run_artifact_indexes.jsonl"
140 EXECUTION_REPORT_PATH = RESULT_DIR / "PROTECTED_HOLDOUT_EXECUTION_REPORT.md"
141
142 # %% [markdown]
143 # ## Helpers
144 #
145 # Progress messages are written both to the cell output and to a persistent
146 # log. Atomic JSON writes prevent an interrupted kernel from leaving a partial
147 # manifest. Long generations emit a heartbeat every 20 seconds.
148
149 # %%
150 def utc_now():
151     return datetime.now(timezone.utc).isoformat()
152
153
154 def log(message):
155     line = f"[{datetime.now().strftime('%H:%M:%S')}] {message}"
156     print(line, flush=True)
157     with PROGRESS_LOG_PATH.open("a", encoding="utf-8") as handle:
158         handle.write(line + "\n")
159
160
161 def json_default(value):
162     if isinstance(value, Path):
163         return str(value)
164     if hasattr(value, "value"):
165         return value.value
166     raise TypeError(f"Cannot serialize {type(value).__name__}")
167
168
169 def write_json_atomic(path, payload):
170     path = Path(path)
171     path.parent.mkdir(parents=True, exist_ok=True)
172     temporary = path.with_suffix(path.suffix + ".tmp")
173     temporary.write_text(
174         json.dumps(
175             payload,
176             indent=2,
177             ensure_ascii=False,
178             default=json_default,
179         )
180         + "\n",
181         encoding="utf-8",
182     )
183     temporary.replace(path)
184
185
186 def write_jsonl_atomic(path, records):

```

```

188 path = Path(path)
189 path.parent.mkdir(parents=True, exist_ok=True)
190 temporary = path.with_suffix(path.suffix + ".tmp")
191 with temporary.open("w", encoding="utf-8") as handle:
192     for record in records:
193         if hasattr(record, "model_dump"):
194             record = record.model_dump(mode="json")
195             handle.write(
196                 json.dumps(record, ensure_ascii=False, default=json_default)
197                 + "\n"
198             )
199 temporary.replace(path)
200
201
202 def append_jsonl(path, record):
203     path = Path(path)
204     path.parent.mkdir(parents=True, exist_ok=True)
205     with path.open("a", encoding="utf-8") as handle:
206         handle.write(
207             json.dumps(record, ensure_ascii=False, default=json_default) + "\n"
208         )
209
210
211 def read_json(path, default=None):
212     path = Path(path)
213     if not path.exists():
214         return default
215     return json.loads(path.read_text(encoding="utf-8"))
216
217
218 def read_jsonl_objects(path):
219     path = Path(path)
220     if not path.exists():
221         return []
222     return [
223         json.loads(line)
224         for line in path.read_text(encoding="utf-8").splitlines()
225         if line.strip()
226     ]
227
228
229 def sha256_bytes(value):
230     return hashlib.sha256(value).hexdigest()
231
232
233 def sha256_file(path):
234     return sha256_bytes(Path(path).read_bytes())
235
236
237 def canonical_sha256(value):
238     payload = json.dumps(
239         value,
240         sort_keys=True,
241         separators=(",", ":"),
242         ensure_ascii=False,
243         default=json_default,
244     )
245     return sha256_bytes(payload.encode("utf-8"))
246
247
248 def safe_name(value):
249     return re.sub(r"^[A-Za-z0-9_.-]+", "_", str(value))
250
251
252 def relative_path(path):
253     path = Path(path).resolve()
254     try:
255         return str(path.relative_to(PROJECT_DIR.resolve()))
256     except ValueError:
257         return str(path)
258
259
260 def git_output(*arguments):
261     completed = subprocess.run(
262         ["git", *arguments],
263         cwd=PROJECT_DIR,
264         check=False,
265         text=True,
266         capture_output=True,
267     )
268     return completed.stdout.strip() if completed.returncode == 0 else None
269
270
271 def package_version(name):
272     try:
273         return importlib.metadata.version(name)
274     except importlib.metadata.PackageNotFoundError:
275         return None
276
277
278 async def with_heartbeat(awaitable, label, interval=HEARTBEAT_SECONDS):

```

```

279     started = time.perf_counter()
280     task = asyncio.create_task(awaitable)
281     while True:
282         try:
283             result = await asyncio.wait_for(asyncio.shield(task), timeout=interval)
284             elapsed = time.perf_counter() - started
285             log(f"{label}: completed after {elapsed:.1f}s")
286             return result
287         except asyncio.TimeoutError:
288             elapsed = time.perf_counter() - started
289             log(f"{label}: still running ({elapsed:.1f}s elapsed)")
290
291 def markdown_word_count(text):
292     without_markup = re.sub(r"<!--.*?-->", " ", text, flags=re.DOTALL)
293     without_markup = re.sub(r"^\s{1,6}\s+", "", without_markup, flags=re.MULTILINE)
294     return len(re.findall(r"\b[\\w'-]+\\b", without_markup, flags=re.UNICODE))
295
296
297 def prose_sentence_count(text):
298     lines = [
299         line.strip()
300         for line in text.splitlines()
301         if line.strip() and not line.lstrip().startswith(("#", "<!--"))
302     ]
303     prose = " ".join(lines)
304     return len(
305         [part for part in re.split(r"(?<=[.!?])\s+", prose) if part.strip()]
306     )
307
308
309 def nested_list(payload, *path):
310     current = payload
311     for key in path:
312         if not isinstance(current, dict):
313             return []
314         current = current.get(key)
315     return current if isinstance(current, list) else []
316
317
318 load_project_env(PROJECT_DIR)
319 log(f"Project: {PROJECT_DIR}")
320 log(f"Experiment: {EXPERIMENT_ID}")
321
322
323 # %% [markdown]
324 # ## 1. Freeze the Full-System configuration and implementation
325 #
326 # All environment-derived workflow settings are resolved once. The six model
327 # roles are then fixed to DeepSeek V4 Flash, and the complete settings payload
328 # is stored in the protected variant. A source-tree snapshot is used even if
329 # the Git worktree is dirty; the commit and dirty status are both disclosed.
330 # Every later run must match this fingerprint exactly.
331
332 # %%
333 MODEL_FIELDS = [
334     "data_understanding_model",
335     "orchestrator_model",
336     "evidence_model",
337     "verifier_model",
338     "writer_model",
339     "auditor_model",
340 ]
341
342
343 def build_frozen_variant():
344     resolved = asdict(Settings.from_env())
345     resolved["use_llm"] = True
346     for field_name in MODEL_FIELDS:
347         resolved[field_name] = MODEL
348
349     # These two are supplied per generation by the evaluation runner.
350     resolved.pop("output_dir", None)
351     resolved.pop("random_seed", None)
352
353     return {
354         "variants": [
355             {
356                 "variant_id": "full_system",
357                 "enabled": True,
358                 "backend": "table2text",
359                 "description": (
360                     "Frozen protected-holdout Full-System run with all six "
361                     "roles on DeepSeek V4 Flash."
362                 ),
363                 "settings_overrides": resolved,
364                 "task_contract_mode": "explicit",
365                 "request_override": None,
366                 "callable_path": None,
367                 "command": [],
368                 "precomputed_path": None,

```

```

370         "repetitions": 1,
371         "seeds": [GENERATION_SEED],
372     }
373 }
374 }
375
376
377 if not VARIANT_PATH.exists():
378     write_json_atomic(VARIANT_PATH, build_frozen_variant())
379     log(f"Created frozen variant: {VARIANT_PATH}")
380 else:
381     log(f"Reusing existing frozen variant: {VARIANT_PATH}")
382
383 frozen_variant_payload = read_json(VARIANT_PATH)
384 frozen_variant = frozen_variant_payload["variants"][0]
385 frozen_overrides = frozen_variant["settings_overrides"]
386
387 assert frozen_variant["variant_id"] == "full_system"
388 assert frozen_variant["task_contract_mode"] == "explicit"
389 assert frozen_variant["repetitions"] == 1
390 assert frozen_variant["seeds"] == [GENERATION_SEED]
391 assert all(frozen_overrides[field] == MODEL for field in MODEL_FIELDS)
392 assert frozen_overrides["use_llm"] is True
393
394
395 def implementation_files():
396     files = sorted((PROJECT_DIR / "src" / "table2text").rglob("*.py"))
397     files.extend(
398         path
399         for path in [
400             PROJECT_DIR / "pyproject.toml",
401             PROJECT_DIR / "evaluation" / "config" / "datasets.json",
402             PROJECT_DIR
403             / "evaluation"
404             / "notebooks"
405             / "protected_holdout_full_system_evaluation.py",
406         ]
407         if path.exists()
408     )
409     return sorted(set(path.resolve() for path in files))
410
411
412 def file_manifest(files):
413     return [
414         {
415             "path": relative_path(path),
416             "sha256": sha256_file(path),
417             "bytes": path.stat().st_size,
418         }
419         for path in files
420     ]
421
422
423 def implementation_fingerprint(files=None):
424     manifest = file_manifest(files or implementation_files())
425     return canonical_sha256(manifest), manifest
426
427
428 class AgentParameterVisitor(ast.NodeVisitor):
429     def __init__(self):
430         self.function_name = None
431         self.records = []
432
433     def visit_FunctionDef(self, node):
434         previous = self.function_name
435         self.function_name = node.name
436         self.generic_visit(node)
437         self.function_name = previous
438
439     visit_AsyncFunctionDef = visit_FunctionDef
440
441     def visit_Call(self, node):
442         function_name = getattr(node.func, "id", None)
443         if function_name == "agent_model_settings" and len(node.args) >= 2:
444             role_node = node.args[1]
445             role = role_node.value if isinstance(role_node, ast.Constant) else None
446             values = {"temperature": None, "max_tokens": None}
447             for keyword in node.keywords:
448                 if keyword.arg in values and isinstance(keyword.value, ast.Constant):
449                     values[keyword.arg] = keyword.value.value
450             self.records.append(
451                 {
452                     "builder": self.function_name,
453                     "role": role,
454                     "temperature": values["temperature"],
455                     "max_tokens": values["max_tokens"],
456                     "source_line": node.lineno,
457                 }
458             )
459             self.generic_visit(node)
460

```

```

461
462 def extract_agent_parameters():
463     path = PROJECT_DIR / "src" / "table2text" / "agents.py"
464     visitor = AgentParameterVisitor()
465     visitor.visit(ast.parse(path.read_text(encoding="utf-8")))
466     return visitor.records
467
468
469 current_implementation_sha256, current_file_manifest = implementation_fingerprint()
470 variant_sha256 = sha256_file(VARIANT_PATH)
471 prepared_examples_sha256 = sha256_file(PATHS["prepared_examples"])
472
473 if not FREEZE_MANIFEST_PATH.exists():
474     snapshot_file_root = SNAPSHOT_DIR / "files"
475     for item, source in zip(current_file_manifest, implementation_files()):
476         destination = snapshot_file_root / item["path"]
477         destination.parent.mkdir(parents=True, exist_ok=True)
478         shutil.copy2(source, destination)
479
480     freeze_manifest = {
481         "experiment_id": EXPERIMENT_ID,
482         "frozen_at": utc_now(),
483         "freeze_basis": "exact_source_tree_snapshot",
484         "git_commit": git_output("rev-parse", "HEAD"),
485         "git_branch": git_output("branch", "--show-current"),
486         "git_status_porcelain": (git_output("status", "--porcelain") or "").splitlines(),
487         "git_worktree_clean": not bool(git_output("status", "--porcelain")),
488         "implementation_sha256": current_implementation_sha256,
489         "implementation_files": current_file_manifest,
490         "variant_path": relative_path(VARIANT_PATH),
491         "variant_sha256": variant_sha256,
492         "prepared_examples_path": relative_path(PATHS["prepared_examples"]),
493         "prepared_examples_sha256": prepared_examples_sha256,
494         "models": {role.removesuffix("_model"): MODEL for role in MODEL_FIELDS},
495         "generation_seed": GENERATION_SEED,
496         "resolved_settings": frozen_overrides,
497         "agent_model_parameters": extract_agent_parameters(),
498         "provider_seed_forwarded_to_deepseek": False,
499         "deepseek_base_url": os.getenv("DEEPSEEK_BASE_URL", "provider default"),
500         "runtime": {
501             "python": sys.version,
502             "platform": platform.platform(),
503             "table2text_package_source": str(PROJECT_DIR / "src" / "table2text"),
504             "pydantic": package_version("pydantic"),
505             "pydantic_ai": package_version("pydantic-ai"),
506             "openai": package_version("openai"),
507             "pandas": package_version("pandas"),
508         },
509         "secrets_recorded": False,
510     }
511     write_json_atomic(FREEZE_MANIFEST_PATH, freeze_manifest)
512     log(f"Frozen implementation snapshot: {current_implementation_sha256}")
513 else:
514     freeze_manifest = read_json(FREEZE_MANIFEST_PATH)
515     log(f"Reusing frozen implementation: {freeze_manifest['implementation_sha256']}")
516
517
518 def assert_frozen_state():
519     current_sha256, _ = implementation_fingerprint()
520     errors = []
521     if current_sha256 != freeze_manifest["implementation_sha256"]:
522         errors.append(
523             "implementation fingerprint changed after protected selection"
524         )
525     if sha256_file(VARIANT_PATH) != freeze_manifest["variant_sha256"]:
526         errors.append("protected variant configuration changed")
527     if sha256_file(PATHS["prepared_examples"]) != freeze_manifest["prepared_examples_sha256"]:
528         errors.append("prepared benchmark file changed")
529     if errors:
530         raise RuntimeError("Protected-run freeze violation: " + "; ".join(errors))
531     return current_sha256
532
533
534 assert_frozen_state()
535
536 print("Freeze basis:", freeze_manifest["freeze_basis"])
537 print("Git commit:", freeze_manifest["git_commit"])
538 print("Git worktree clean:", freeze_manifest["git_worktree_clean"])
539 print("Implementation SHA-256:", freeze_manifest["implementation_sha256"])
540 print("Variant SHA-256:", freeze_manifest["variant_sha256"])
541 print("Models:", sorted(set(freeze_manifest["models"].values())))
542 display(pd.DataFrame(freeze_manifest["agent_model_parameters"]))
543
544
545 # %% [markdown]
546 # ## 2. Freeze the unseen 25-example selection
547 #
548 # Selection uses only dataset ID and example ID. Every generation record found
549 # elsewhere under `evaluation/**/generations/**` is excluded. Sources and
550 # references do not participate in selection. The five datasets are then
551 # interleaved, so a partial run remains balanced across task families.

```

```

552 #
553 # The operational examples replace the true references with a fixed sentinel
554 # before any generator call. True references are recovered only in the final,
555 # explicitly opt-in post-generation metric phase.
556
557 # %%
558 TARGET_DATASET_SET = set(DATASETS)
559 HELD_OUT_REFERENCE_SENTINEL = "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
560 REFERENCE_LIKE_METADATA_KEYS = {
561     "answer",
562     "answers",
563     "description",
564     "descriptions",
565     "gold",
566     "news_article",
567     "ref",
568     "reference",
569     "references",
570     "summary",
571     "summaries",
572     "target",
573     "targets",
574     "text",
575 }
576
577
578 def historical_generation_inventory():
579     identities = defaultdict(set)
580     files = []
581     evaluation_root = PROJECT_DIR / "evaluation"
582     for path in sorted(evaluation_root.rglob("*.jsonl")):
583         if path.is_relative_to(ARTIFACT_DIR):
584             continue
585         if "generations" not in path.parts:
586             continue
587
588         matched = 0
589         for item in read_jsonl_objects(path):
590             if "generated_text" not in item or "variant_id" not in item:
591                 continue
592             dataset_id = str(item.get("dataset_id", ""))
593             example_id = item.get("example_id")
594             if dataset_id in TARGET_DATASET_SET and example_id is not None:
595                 identities[dataset_id].add(str(example_id))
596                 matched += 1
597         if matched:
598             files.append(
599                 {
600                     "path": relative_path(path),
601                     "sha256": sha256_file(path),
602                     "matching_generation_rows": matched,
603                 }
604             )
605     return identities, files
606
607
608 all_examples = read_examples(PATHS["prepared_examples"])
609 example_lookup = {
610     (str(example.dataset_id), str(example.example_id)): example
611     for example in all_examples
612 }
613
614 if not SELECTION_MANIFEST_PATH.exists():
615     assert_frozen_state()
616     historical_ids, historical_files = historical_generation_inventory()
617     selected_by_dataset = {}
618
619     for dataset_id in DATASETS:
620         candidates = sorted(
621             [
622                 example
623                 for example in all_examples
624                 if example.dataset_id == dataset_id
625                 and str(example.example_id) not in historical_ids[dataset_id]
626             ],
627             key=lambda item: str(item.example_id),
628         )
629         if len(candidates) < EXAMPLES_PER_DATASET:
630             raise RuntimeError(
631                 f"{dataset_id} has only {len(candidates)} previously unused "
632                 f"prepared examples; {EXAMPLES_PER_DATASET} are required."
633             )
634         generator = random.Random(f"{SELECTION_SEED}:{dataset_id}")
635         selected_by_dataset[dataset_id] = sorted(
636             generator.sample(candidates, EXAMPLES_PER_DATASET),
637             key=lambda item: str(item.example_id),
638         )
639
640 # Round-robin order: one example from each dataset per block.
641 selected_examples = [
642     selected_by_dataset[dataset_id][position]

```

```

643     for position in range(EXAMPLES_PER_DATASET)
644     for dataset_id in DATASETS
645 ]
646 selection_timestamp = utc_now()
647
648 exclusion_manifest = {
649     "created_at": selection_timestamp,
650     "selection_rule": (
651         "Exclude every prior GenerationRecord found outside this protected "
652         "experiment; sample by identity only."
653     ),
654     "historical_generation_files": historical_files,
655     "excluded_example_ids": {
656         dataset_id: sorted(historical_ids[dataset_id])
657         for dataset_id in DATASETS
658     },
659 }
660 write_json_atomic(EXCLUSION_MANIFEST_PATH, exclusion_manifest)
661
662 selection_manifest = {
663     "experiment_id": EXPERIMENT_ID,
664     "selected_at": selection_timestamp,
665     "protected": True,
666     "selection_seed": SELECTION_SEED,
667     "selection_algorithm": (
668         "Dataset-stratified random sample after historical-generation "
669         "exclusion; source and reference fields were not used."
670     ),
671     "examples_per_dataset": EXAMPLES_PER_DATASET,
672     "dataset_order": DATASETS,
673     "run_order": [
674         {
675             "position": index,
676             "dataset_id": example.dataset_id,
677             "example_id": str(example.example_id),
678             "source_sha256": example.source_sha256,
679             "reference_sha256": example.reference_sha256,
680             "task_family": example.task_family.value,
681             "output_mode": example.output_mode.value,
682             "language": example.language,
683         }
684         for index, example in enumerate(selected_examples, start=1)
685     ],
686     "historical_overlap_count_at_selection": 0,
687     "researcher_declared_no_prior_manual_inspection": (
688         RESEARCHER_CONFIRMS_NO_PRIOR_MANUAL_INSPECTION
689     ),
690     "freeze_manifest_sha256": sha256_file(FREEZE_MANIFEST_PATH),
691 }
692 write_json_atomic(SELECTION_MANIFEST_PATH, selection_manifest)
693 log(f"Protected selection frozen at {selection_timestamp}")
694 else:
695     selection_manifest = read_json(SELECTION_MANIFEST_PATH)
696     selected_examples = []
697     for item in selection_manifest["run_order"]:
698         key = (item["dataset_id"], str(item["example_id"]))
699         if key not in example_lookup:
700             raise RuntimeError(f"Selected example disappeared from prepared data: {key}")
701         example = example_lookup[key]
702         if example.source_sha256 != item["source_sha256"]:
703             raise RuntimeError(f"Source hash changed for selected example: {key}")
704         if example.reference_sha256 != item["reference_sha256"]:
705             raise RuntimeError(f"Reference hash changed for selected example: {key}")
706         selected_examples.append(example)
707     log(f"Reusing protected selection from {selection_manifest['selected_at']}")
708
709 expected_total = len(DATASETS) * EXAMPLES_PER_DATASET
710 assert len(selected_examples) == expected_total
711 assert Counter(example.dataset_id for example in selected_examples) == Counter(
712     {dataset_id: EXAMPLES_PER_DATASET for dataset_id in DATASETS}
713 )
714
715
716 def sanitize_operational_metadata(metadata):
717     return {
718         key: value
719         for key, value in metadata.items()
720         if key.casefold() not in REFERENCE_LIKE_METADATA_KEYS
721         and "reference" not in key.casefold()
722         and not key.casefold().startswith("target")
723     }
724
725
726 operational_examples = [
727     example.model_copy(
728         update={
729             "references": [HELD_OUT_REFERENCE_SENTINEL],
730             "metadata": sanitize_operational_metadata(example.metadata),
731         }
732     )
733     for example in selected_examples

```



```

734 ]
735 write_jsonl_atomic(OPERATIONAL_EXAMPLES_PATH, operational_examples)
736
737
738 def audit_materialized_model_input(example):
739     path = materialise_input(example, MODEL_INPUT_AUDIT_DIR)
740     payload = json.loads(path.read_text(encoding="utf-8"))
741     serialized = json.dumps(payload, ensure_ascii=False)
742     forbidden_top_level = {
743         "references",
744         "reference_sha256",
745         "target",
746         "targets",
747         "summary",
748         "summaries",
749     }
750     present = sorted(forbidden_top_level & set(payload)) if isinstance(payload, dict) else []
751     if present:
752         raise RuntimeError(
753             f"Reference-like top-level fields reached model input for "
754             f"{example.dataset_id}/{example.example_id}: {present}"
755         )
756     if HELD_OUT_REFERENCE_SENTINEL in serialized:
757         raise RuntimeError(
758             f"Held-out reference sentinel reached model input for "
759             f"{example.dataset_id}/{example.example_id}."
760         )
761     return {
762         "dataset_id": example.dataset_id,
763         "example_id": str(example.example_id),
764         "materialized_input_path": relative_path(path),
765         "materialized_input_sha256": sha256_file(path),
766         "reference_fields_present": present,
767         "reference_sentinel_present": False,
768     }
769
770
771 model_input_audits = [
772     audit_materialized_model_input(example) for example in operational_examples
773 ]
774 write_json_atomic(
775     MODEL_INPUT_AUDIT_DIR / "model_input_isolation_manifest.json",
776     {
777         "created_at": utc_now(),
778         "reference_isolation": "pass",
779         "records": model_input_audits,
780     },
781 )
782
783 selected_table = pd.DataFrame(selection_manifest["run_order"])
784 display(
785     selected_table[
786         [
787             "position",
788             "dataset_id",
789             "example_id",
790             "task_family",
791             "output_mode",
792             "language",
793         ]
794     ]
795 )
796 print("Selected examples:", len(selected_examples))
797 print("Historical overlap:", selection_manifest["historical_overlap_count_at_selection"])
798 print("Reference isolation: PASS (sentinel absent from all materialized model inputs)")
799
800
801 # %% [markdown]
802 # ## 3. Initialize the experiment-level manifest
803 #
804 # This manifest is updated after every case. It records the frozen code and
805 # config, protected selection, model roles, task contracts, reference boundary,
806 # completion counts, and whether any implementation drift was detected.
807
808 # %%
809 def collect_final_records():
810     records = []
811     for item in selection_manifest["run_order"]:
812         slug = f"{safe_name(item['dataset_id'])}__{safe_name(item['example_id'])}"
813         path = SHARD_DIR / f"{slug}.jsonl"
814         if not path.exists():
815             continue
816         rows = read_generations(path)
817         if rows:
818             records.append(rows[-1])
819     write_jsonl_atomic(SEALED_GENERATIONS_PATH, records)
820     return records
821
822
823 def update_batch_manifest(status=None):
824     records = collect_final_records()

```

```

825     successful = [record for record in records if not record.error and record.generated_text]
826     failed = [record for record in records if record.error or not record.generated_text]
827     inferred_status = (
828         "complete"
829         if len(successful) == expected_total and not failed
830         else "running"
831         if records
832         else "selected"
833     )
834     prior = read_json(BATCH_MANIFEST_PATH, {})
835     payload = {
836         **prior,
837         "experiment": "Protected Holdout Validation",
838         "experiment_id": EXPERIMENT_ID,
839         "selection_date": selection_manifest["selected_at"],
840         "last_updated": utc_now(),
841         "status": status or inferred_status,
842         "protected_unseen": True,
843         "system_frozen_before_selection": True,
844         "freeze_basis": freeze_manifest["freeze_basis"],
845         "git_commit": freeze_manifest["git_commit"],
846         "git_worktree_clean_at_freeze": freeze_manifest["git_worktree_clean"],
847         "implementation_sha256": freeze_manifest["implementation_sha256"],
848         "configuration_path": relative_path(VARIANT_PATH),
849         "configuration_sha256": freeze_manifest["variant_sha256"],
850         "selection_manifest_sha256": sha256_file(SELECTION_MANIFEST_PATH),
851         "model_input_isolation_manifest_sha256": sha256_file(
852             MODEL_INPUT_AUDIT_DIR / "model_input_isolation_manifest.json"
853         ),
854         "model": "DeepSeek V4 Flash",
855         "models_by_role": freeze_manifest["models"],
856         "model_parameters": freeze_manifest["agent_model_parameters"],
857         "generation_seed": GENERATION_SEED,
858         "provider_seed_forwarded": False,
859         "examples": expected_total,
860         "datasets": {
861             dataset_id: EXAMPLES_PER_DATASET for dataset_id in DATASETS
862         },
863         "selected_examples": selection_manifest["run_order"],
864         "previous_manual_inspection_of_selected_examples": (
865             "none_declared"
866             if RESEARCHER_CONFIRMS_NO_PRIOR_MANUAL_INSPECTION
867             else "not_confirmed"
868         ),
869         "historical_generation_overlap_count": (
870             selection_manifest["historical_overlap_count_at_selection"]
871         ),
872         "development_changes_after_holdout_selection": (
873             "none_verified_by_fingerprint"
874             if RESEARCHER_CONFIRMS_NO_DEVELOPMENT_CHANGES_AFTER_SELECTION
875             else "not_confirmed"
876         ),
877         "implementation_drift_detected": False,
878         "human_references_available_to_generator": False,
879         "reference_isolation_method": {
880             "True references replaced by a sentinel in BenchmarkExample before "
881             "generation; materialized model inputs contain neither references nor "
882             "the sentinel."
883         },
884         "released_generations_per_example": 1,
885         "generation_configuration": relative_path(VARIANT_PATH),
886         "completion": {
887             "successful": len(successful),
888             "failed": len(failed),
889             "not_started": expected_total - len(records),
890         },
891         "sealed_generations_path": relative_path(SEALED_GENERATIONS_PATH),
892         "summary_path": relative_path(SUMMARY_JSONL_PATH),
893         "progress_log_path": relative_path(PROGRESS_LOG_PATH),
894         "true_references_unsealed_at": prior.get("true_references_unsealed_at"),
895     }
896     if payload["status"] == "complete" and "generation_completed_at" not in payload:
897         payload["generation_completed_at"] = utc_now()
898     if records and "generation_started_at" not in payload:
899         payload["generation_started_at"] = utc_now()
900     write_json_atomic(BATCH_MANIFEST_PATH, payload)
901     return payload
902
903
904 batch_manifest = update_batch_manifest()
905 display(pd.DataFrame([batch_manifest["completion"]]))
906
907
908 # %% [markdown]
909 # ## 4. Artifact and token extraction
910 #
911 # These functions summarize the workflow artifacts themselves. They do not
912 # infer missing information. Provider-reported token usage is parsed from the
913 # persisted trace, and monetary cost remains null unless directly returned.
914
915 # %%

```

```

916 USAGE_FIELD_PATTERN = {
917     "input_tokens": re.compile(r"\binput_tokens=(\d+)"),
918     "cache_read_tokens": re.compile(r"\bcache_read_tokens=(\d+)"),
919     "output_tokens": re.compile(r"\boutput_tokens=(\d+)"),
920     "requests": re.compile(r"\brequests=(\d+)"),
921 }
922
923
924 def parse_usage_string(value):
925     if not isinstance(value, str):
926         return None
927     result = {}
928     for name, pattern in USAGE_FIELD_PATTERN.items():
929         match = pattern.search(value)
930         result[name] = int(match.group(1)) if match else 0
931     if not any(result.values()):
932         return None
933     result["total_tokens"] = result["input_tokens"] + result["output_tokens"]
934     return result
935
936
937 def locate_pipeline_result(record):
938     if record.pipeline_result_path:
939         path = Path(record.pipeline_result_path)
940         if path.exists():
941             return path
942     if record.run_id:
943         matches = list(RUN_ROOT.rglob(f"{record.run_id}/pipeline_result.json"))
944         if len(matches) == 1:
945             return matches[0]
946     return None
947
948
949 def stage_usage_rows(record, run_directory):
950     trace_path = run_directory / "trace.jsonl"
951     rows = []
952     for index, event in enumerate(read_jsonl_objects(trace_path), start=1):
953         usage = parse_usage_string(event.get("details", {}).get("usage"))
954         if usage is None:
955             continue
956         rows.append(
957             {
958                 "dataset_id": record.dataset_id,
959                 "example_id": str(record.example_id),
960                 "run_id": record.run_id,
961                 "trace_event": index,
962                 "timestamp": event.get("timestamp"),
963                 "stage": event.get("stage"),
964                 "status": event.get("status"),
965                 **usage,
966             }
967         )
968     return rows
969
970
971 def run_artifact_index(record, run_directory):
972     files = []
973     for path in sorted(run_directory.rglob("*")):
974         if path.is_file():
975             files.append(
976                 {
977                     "path": relative_path(path),
978                     "sha256": sha256_file(path),
979                     "bytes": path.stat().st_size,
980                 }
981             )
982     return {
983         "dataset_id": record.dataset_id,
984         "example_id": str(record.example_id),
985         "run_id": record.run_id,
986         "run_directory": relative_path(run_directory),
987         "artifact_count": len(files),
988         "artifacts": files,
989         "index_sha256": canonical_sha256(files),
990     }
991
992
993 def trace_summary(run_directory):
994     events = read_jsonl_objects(run_directory / "trace.jsonl")
995     fallbacks = [event for event in events if event.get("status") == "fallback"]
996     retries = [event for event in events if ".retry." in str(event.get("stage", ""))]
997     non_success = [
998         event
999         for event in events
1000         if event.get("status") in {"fallback", "error", "failed"}
1001     ]
1002     return events, fallbacks, retries, non_success
1003
1004
1005 def extract_summary(record):
1006     pipeline_path = locate_pipeline_result(record)

```

```

1007     if pipeline_path is None:
1008         return {
1009             "dataset_id": record.dataset_id,
1010             "example_id": str(record.example_id),
1011             "run_id": record.run_id,
1012             "execution_outcome": "failed",
1013             "error": record.error or "pipeline_result.json not found",
1014             "exact_final_output": record.generated_text,
1015         }, [], [], None
1016
1017     result = read_json(pipeline_path)
1018     run_directory = pipeline_path.parent
1019     run_manifest = read_json(run_directory / "00_manifest.json", {})
1020     events, fallbacks, retries, non_success = trace_summary(run_directory)
1021     usage_rows = stage_usage_rows(record, run_directory)
1022     artifact_index = run_artifact_index(record, run_directory)
1023
1024     fact_candidates = nested_list(result, "fact_candidates", "candidates")
1025     writer_ready_facts = nested_list(result, "fact_ledger", "writer_ready_facts")
1026     rejected_facts = nested_list(result, "fact_ledger", "rejected_facts")
1027     evidence_items = nested_list(result, "evidence_ledger", "items")
1028     verified_insights = nested_list(result, "insight_ledger", "verified_insights")
1029     rejected_insights = nested_list(result, "insight_ledger", "rejected_insights")
1030     unverified_insights = nested_list(result, "insight_ledger", "unverified_insights")
1031     hypothesis_insights = nested_list(result, "insight_ledger", "hypothesis_only_insights")
1032     insight_candidate_payload = read_json(run_directory / "07_insight_candidates.json", {})
1033     insight_candidates = (
1034         insight_candidate_payload.get("candidates", [])
1035         if isinstance(insight_candidate_payload, dict)
1036         else []
1037     )
1038
1039     final_output = result.get("final_writer_output", {})
1040     raw_output = result.get("raw_writer_output", {})
1041     final_audit = result.get("final_audit", {})
1042     support = final_output.get("sentence_support", []) or []
1043     repair_rounds = result.get("repair_rounds_used", 0) or 0
1044     final_writer_mode = final_output.get("writer_mode") or record.writer_mode
1045     raw_writer_mode = raw_output.get("writer_mode")
1046     used_deterministic_fallback = "deterministic" in str(raw_writer_mode).casefold()
1047
1048     if repair_rounds > 0 or final_audit.get("applied_patches"):
1049         final_generation_path = "auditor_repaired"
1050     elif used_deterministic_fallback:
1051         final_generation_path = "deterministic_fallback"
1052     else:
1053         final_generation_path = "normal_llm_writer"
1054
1055     start_time = run_manifest.get("created_at")
1056     end_time = events[-1].get("timestamp") if events else None
1057     total_input_tokens = sum(row["input_tokens"] for row in usage_rows)
1058     total_output_tokens = sum(row["output_tokens"] for row in usage_rows)
1059     total_cache_tokens = sum(row["cache_read_tokens"] for row in usage_rows)
1060     total_requests = sum(row["requests"] for row in usage_rows)
1061
1062     final_report_path = run_directory / "final_report.md"
1063     exact_released_report = (
1064         final_report_path.read_text(encoding="utf-8")
1065         if final_report_path.exists()
1066         else record.generated_text
1067     )
1068     actual_input_path = None
1069     input_paths = run_manifest.get("input_paths", [])
1070     if input_paths:
1071         candidate = Path(input_paths[0])
1072         if candidate.exists():
1073             actual_input_path = candidate
1074     expected_input_audit = next(
1075         (
1076             item
1077             for item in model_input_audits
1078             if item["dataset_id"] == record.dataset_id
1079             and item["example_id"] == str(record.example_id)
1080         ),
1081         None,
1082     )
1083     actual_input_sha256 = (
1084         sha256_file(actual_input_path) if actual_input_path is not None else None
1085     )
1086     expected_input_sha256 = (
1087         expected_input_audit["materialized_input_sha256"]
1088         if expected_input_audit is not None
1089         else None
1090     )
1091
1092     summary = {
1093         "dataset_id": record.dataset_id,
1094         "example_id": str(record.example_id),
1095         "protected_unseen": True,
1096         "selection_timestamp": selection_manifest["selected_at"],
1097         "run_id": record.run_id,

```

```

1098 "generation_id": record.generation_id,
1099 "execution_outcome": "success" if not record.error else "failed",
1100 "error": record.error,
1101 "start_time": start_time,
1102 "end_time": end_time,
1103 "elapsed_seconds": record.elapsed_seconds,
1104 "task_family": record.task_family.value,
1105 "request": record.request,
1106 "output_mode": record.output_mode.value,
1107 "language": record.language,
1108 "task_contract": run_manifest.get("task_contract"),
1109 "report_genre": run_manifest.get("report_genre"),
1110 "communication_task": run_manifest.get("communication_task"),
1111 "focus_scope": run_manifest.get("focus_scope"),
1112 "models": run_manifest.get("models"),
1113 "generation_seed": record.seed,
1114 "configuration_sha256": freeze_manifest["variant_sha256"],
1115 "implementation_sha256": freeze_manifest["implementation_sha256"],
1116 "reference_available_to_generator": False,
1117 "source_sha256": next(
1118     item["source_sha256"]
1119     for item in selection_manifest["run_order"]
1120     if item["dataset_id"] == record.dataset_id
1121     and item["example_id"] == str(record.example_id)
1122 ),
1123 "reference_sha256": next(
1124     item["reference_sha256"]
1125     for item in selection_manifest["run_order"]
1126     if item["dataset_id"] == record.dataset_id
1127     and item["example_id"] == str(record.example_id)
1128 ),
1129 "input_paths": input_paths,
1130 "materialized_model_input_sha256": actual_input_sha256,
1131 "model_input_matches_reference_isolation_audit": (
1132     actual_input_sha256 is not None
1133     and actual_input_sha256 == expected_input_sha256
1134 ),
1135 "pipeline_result_path": relative_path(pipeline_path),
1136 "run_directory": relative_path(run_directory),
1137 "final_report_path": relative_path(final_report_path),
1138 "initial_writer_mode": raw_writer_mode,
1139 "final_writer_mode": final_writer_mode,
1140 "used_deterministic_fallback": used_deterministic_fallback,
1141 "final_generation_path": final_generation_path,
1142 "repair_rounds_used": repair_rounds,
1143 "audit_decision": final_audit.get("decision"),
1144 "release_status": result.get("release_status") or record.release_status,
1145 "approved_for_release": result.get("approved_for_release"),
1146 "primary_evaluation_eligible": result.get("primary_evaluation_eligible"),
1147 "native_support_rate": final_audit.get("support_rate"),
1148 "factual_sentence_count": final_audit.get("factual_sentence_count"),
1149 "supported_sentence_count": final_audit.get("supported_sentence_count"),
1150 "unsupported_factual_sentence_count": (
1151     (final_audit.get("factual_sentence_count") or 0)
1152     - (final_audit.get("supported_sentence_count") or 0)
1153 ),
1154 "sentence_support_mapping_count": len(support),
1155 "output_word_count": markdown_word_count(record.generated_text),
1156 "output_sentence_count": prose_sentence_count(record.generated_text),
1157 "evidence_item_count": len(evidence_items),
1158 "fact_candidate_count": len(fact_candidates),
1159 "verified_fact_count": len(writer_ready_facts),
1160 "rejected_fact_count": len(rejected_facts),
1161 "insight_candidate_count": len(insight_candidates),
1162 "verified_insight_count": len(verified_insights),
1163 "rejected_insight_count": len(rejected_insights),
1164 "unverified_insight_count": len(unverified_insights),
1165 "hypothesis_only_insight_count": len(hypothesis_insights),
1166 "top_level_attempt_count": sum(
1167     1
1168     for item in read_jsonl_objects(ATTEMPT_LOG_PATH)
1169     if item.get("event") == "started"
1170     and item.get("dataset_id") == record.dataset_id
1171     and item.get("example_id") == str(record.example_id)
1172 ),
1173 "trace_retry_event_count": len(retries),
1174 "fallback_event_count": len(fallbacks),
1175 "non_success_trace_event_count": len(non_success),
1176 "fallbacks": fallbacks,
1177 "trace_retries": retries,
1178 "provider_reported_input_tokens": total_input_tokens,
1179 "provider_reported_output_tokens": total_output_tokens,
1180 "provider_reported_total_tokens": total_input_tokens + total_output_tokens,
1181 "provider_reported_cache_read_tokens": total_cache_tokens,
1182 "provider_reported_requests": total_requests,
1183 "monetary_cost": record.estimated_cost_gbp,
1184 "monetary_cost_source": (
1185     "provider/evaluation record"
1186     if record.estimated_cost_gbp is not None
1187     else "not directly available"
1188 ),

```

```

1189     "artifact_count": artifact_index["artifact_count"],
1190     "artifact_index_sha256": artifact_index["index_sha256"],
1191     "exact_final_output": record.generated_text,
1192     "exact_released_report": exact_released_report,
1193 }
1194
1195 support_rows = [
1196     {
1197         "dataset_id": record.dataset_id,
1198         "example_id": str(record.example_id),
1199         "run_id": record.run_id,
1200         **mapping,
1201     }
1202     for mapping in support
1203 ]
1204 return summary, usage_rows, support_rows, artifact_index
1205
1206 def csv_safe_summary(summary):
1207     excluded = {
1208         "task_contract",
1209         "models",
1210         "input_paths",
1211         "fallbacks",
1212         "trace_retries",
1213         "exact_final_output",
1214         "exact_released_report",
1215     }
1216     return {key: value for key, value in summary.items() if key not in excluded}
1217
1218 def escape_markdown(value):
1219     if value is None:
1220         return ""
1221     return str(value).replace("|", "\\|").replace("\n", " ")
1222
1223 def rebuild_aggregate_artifacts():
1224     records = collect_final_records()
1225     summaries = []
1226     usage_rows = []
1227     support_rows = []
1228     artifact_indexes = []
1229     for record in records:
1230         summary, usage, support, artifact_index = extract_summary(record)
1231         summaries.append(summary)
1232         usage_rows.extend(usage)
1233         support_rows.extend(support)
1234         if artifact_index is not None:
1235             artifact_indexes.append(artifact_index)
1236
1237     write_jsonl_atomic(SUMMARY_JSONL_PATH, summaries)
1238     write_jsonl_atomic(SUPPORT_MAP_JSONL_PATH, support_rows)
1239     write_jsonl_atomic(ARTIFACT_INDEX_PATH, artifact_indexes)
1240     pd.DataFrame([csv_safe_summary(item) for item in summaries]).to_csv(
1241         SUMMARY_CSV_PATH,
1242         index=False,
1243     )
1244     pd.DataFrame(usage_rows).to_csv(STAGE_USAGE_CSV_PATH, index=False)
1245
1246 manifest = update_batch_manifest()
1247 lines = [
1248     "# Protected Holdout Execution Report",
1249     "",
1250     f"- Experiment: `{EXPERIMENT_ID}`",
1251     f"- Selection timestamp: `{selection_manifest['selected_at']}`",
1252     f"- Frozen implementation: `{freeze_manifest['implementation_sha256']}`",
1253     f"- Git commit: `{freeze_manifest['git_commit']}`",
1254     f"- Configuration: `{relative_path(VARIANT_PATH)}`",
1255     f"- Configuration SHA-256: `{freeze_manifest['variant_sha256']}`",
1256     f"- Model for all six roles: `{MODEL}`",
1257     f"- Status: `{manifest['status']}`",
1258     f"- Completion: `{manifest['completion']}`",
1259     "- References available during generation: `No`",
1260     "",
1261     "## Run Summary",
1262     "",
1263     "| Dataset | Example | Outcome | Path | Release | Support | Words | Tokens | Seconds |",
1264     "|---|---|---|---|---|---|---|---|---|",
1265 ]
1266 for item in summaries:
1267     lines.append(
1268         "| " + " | ".join(
1269             [
1270                 escape_markdown(item.get("dataset_id")),
1271                 escape_markdown(item.get("example_id")),
1272                 escape_markdown(item.get("execution_outcome")),
1273                 escape_markdown(item.get("final_generation_path")),
1274                 escape_markdown(item.get("release_status")),
1275                 escape_markdown(item.get("native_support_rate")),

```



```

1280         escape_markdown(item.get("output_word_count")),
1281         escape_markdown(item.get("provider_reported_total_tokens")),
1282         escape_markdown(
1283             round(item.get("elapsed_seconds") or 0.0, 1)
1284         ),
1285     ]
1286 )
1287 + " |"
1288 )
1289
1290 if usage_rows:
1291     usage_frame = pd.DataFrame(usage_rows)
1292     stage_totals = (
1293         usage_frame.groupby("stage", as_index=False)[
1294             ["input_tokens", "output_tokens", "total_tokens", "requests"]
1295         ]
1296         .sum()
1297         .sort_values("total_tokens", ascending=False)
1298     )
1299     lines.extend(
1300         [
1301             "",
1302             "## Provider-Reported Usage by Stage",
1303             "",
1304             "| Stage | Input tokens | Output tokens | Total tokens | Requests |",
1305             "|---|---|---|---|---|",
1306         ]
1307     )
1308     for row in stage_totals.to_dict(orient="records"):
1309         lines.append(
1310             f"| {escape_markdown(row['stage'])} | {row['input_tokens']} | "
1311             f"{row['output_tokens']} | {row['total_tokens']} | {row['requests']} |"
1312         )
1313
1314     lines.extend(
1315         [
1316             "",
1317             "## Artifact Locations",
1318             "",
1319             f"- Batch manifest: `{relative_path(BATCH_MANIFEST_PATH)}`",
1320             f"- Exact outputs and run summaries: `{relative_path(SUMMARY_JSONL_PATH)}`",
1321             f"- Sentence support: `{relative_path(SUPPORT_MAP_JSONL_PATH)}`",
1322             f"- Stage usage: `{relative_path(STAGE_USAGE_CSV_PATH)}`",
1323             f"- Run checksums: `{relative_path(ARTIFACT_INDEX_PATH)}`",
1324             f"- Progress log: `{relative_path(PROGRESS_LOG_PATH)}`",
1325             "",
1326         ]
1327     )
1328     EXECUTION_REPORT_PATH.write_text("\n".join(lines), encoding="utf-8")
1329     return pd.DataFrame([csv_safe_summary(item) for item in summaries])
1330
1331
1332 summary_frame = rebuild_aggregate_artifacts()
1333 if not summary_frame.empty:
1334     display(summary_frame)
1335
1336 # %% [markdown]
1337 # ## 5. Run or resume the protected generation batch
1338 #
1339 # Each example has an independent generation shard. Failed attempts are copied
1340 # to `attempt_records` before retry; successful outputs are never regenerated.
1341 # The exact implementation/config/input fingerprints are checked immediately
1342 # before every call.
1343
1344 # %%
1345 def operational_example_lookup():
1346     return {
1347         (str(example.dataset_id), str(example.example_id)): example
1348         for example in operational_examples
1349     }
1350
1351
1352 def attempt_count(dataset_id, example_id):
1353     return sum(
1354         1
1355         for item in read_jsonl_objects(ATTEMPT_LOG_PATH)
1356         if item.get("event") == "started"
1357         and item.get("dataset_id") == dataset_id
1358         and item.get("example_id") == str(example_id)
1359     )
1360
1361
1362 def existing_success(shard_path):
1363     if not shard_path.exists():
1364         return None
1365     rows = read_generations(shard_path)
1366     if not rows:
1367         return None
1368     row = rows[-1]
1369     return row if not row.error and bool(row.generated_text.strip()) else None
1370

```



```

1371
1372
1373 async def run_protected_batch():
1374     operational_lookup = operational_example_lookup()
1375     completed_before = {
1376         (record.dataset_id, str(record.example_id))
1377         for record in collect_final_records()
1378         if not record.error and record.generated_text.strip()
1379     }
1380     pending = [
1381         item
1382         for item in selection_manifest["run_order"]
1383         if (item["dataset_id"], str(item["example_id"])) not in completed_before
1384     ]
1385     if MAX_CASES_THIS_SESSION is not None:
1386         pending = pending[:MAX_CASES_THIS_SESSION]
1387
1388     log("=" * 80)
1389     log(
1390         f"Protected batch: {len(completed_before)}/{expected_total} already complete; "
1391         f"{len(pending)} scheduled this session."
1392     )
1393     if pending:
1394         manifest = read_json(BATCH_MANIFEST_PATH, {})
1395         if not manifest.get("generation_started_at"):
1396             manifest["generation_started_at"] = utc_now()
1397             manifest["status"] = "running"
1398             manifest["last_updated"] = utc_now()
1399             write_json_atomic(BATCH_MANIFEST_PATH, manifest)
1400
1401     for session_index, item in enumerate(pending, start=1):
1402         dataset_id = item["dataset_id"]
1403         example_id = str(item["example_id"])
1404         key = (dataset_id, example_id)
1405         example = operational_lookup[key]
1406         slug = f"{safe_name(dataset_id)}_{safe_name(example_id)}"
1407         example_path = PREPARED_DIR / "shards" / f"{slug}.jsonl"
1408         shard_path = SHARD_DIR / f"{slug}.jsonl"
1409         run_root = RUN_ROOT / slug
1410         example_path.parent.mkdir(parents=True, exist_ok=True)
1411         write_jsonl_atomic(example_path, [example])
1412
1413         success = existing_success(shard_path)
1414         if success is not None:
1415             log(f"SKIP {dataset_id}/{example_id}: successful shard already exists")
1416             continue
1417
1418         prior_attempts = attempt_count(dataset_id, example_id)
1419         remaining_attempts = max(
1420             0,
1421             MAX_TOP_LEVEL_ATTEMPTS_PER_EXAMPLE - prior_attempts,
1422         )
1423         if remaining_attempts == 0:
1424             log(f"EXHAUSTED {dataset_id}/{example_id}: no attempts remain")
1425             continue
1426
1427         log("-" * 80)
1428         log(
1429             f"CASE {session_index}/{len(pending)} this session; "
1430             f"protected position {item['position']}/{expected_total}: "
1431             f"{dataset_id}/{example_id}"
1432         )
1433
1434         case_succeeded = False
1435         for _ in range(remaining_attempts):
1436             assert_frozen_state()
1437             attempt_number = attempt_count(dataset_id, example_id) + 1
1438             attempt_started = utc_now()
1439             append_jsonl(
1440                 ATTEMPT_LOG_PATH,
1441                 {
1442                     "event": "started",
1443                     "timestamp": attempt_started,
1444                     "dataset_id": dataset_id,
1445                     "example_id": example_id,
1446                     "attempt": attempt_number,
1447                     "implementation_sha256": freeze_manifest["implementation_sha256"],
1448                     "configuration_sha256": freeze_manifest["variant_sha256"],
1449                 },
1450             )
1451
1452             if shard_path.exists():
1453                 archived = ATTEMPT_RECORD_DIR / f"{slug}__before_attempt_{attempt_number}.jsonl"
1454                 shutil.copy2(shard_path, archived)
1455                 shard_path.unlink()
1456
1457             try:
1458                 frame = await with_heartbeat(
1459                     generate_reports_for_notebook(
1460                         PROJECT_DIR,
1461                         examples_path=example_path,

```

```

1462         variants_path=VARIANT_PATH,
1463         output_path=shard_path,
1464         run_root=run_root,
1465         resume=False,
1466     ),
1467     label=(
1468         f"{dataset_id}/{example_id} attempt "
1469         f"{attempt_number}/{MAX_TOP_LEVEL_ATTEMPTS_PER_EXAMPLE}"
1470     ),
1471 )
1472 row = frame.iloc[-1].to_dict() if not frame.empty else {}
1473 error = row.get("error") or None
1474 generated_text = str(row.get("generated_text") or "")
1475 success = not error and bool(generated_text.strip())
1476 append_jsonl(
1477     ATTEMPT_LOG_PATH,
1478     {
1479         "event": "finished",
1480         "timestamp": utc_now(),
1481         "dataset_id": dataset_id,
1482         "example_id": example_id,
1483         "attempt": attempt_number,
1484         "success": success,
1485         "error": error,
1486         "run_id": row.get("run_id"),
1487         "pipeline_result_path": row.get("pipeline_result_path"),
1488         "elapsed_seconds": row.get("elapsed_seconds"),
1489     },
1490 )
1491 if shard_path.exists():
1492     shutil.copy2(
1493         shard_path,
1494         ATTEMPT_RECORD_DIR / f"{slug}__attempt_{attempt_number}.jsonl",
1495     )
1496
1497 rebuild_aggregate_artifacts()
1498 manifest = update_batch_manifest("running")
1499 log(
1500     f"{dataset_id}/{example_id}: success={success}, "
1501     f"release={row.get('release_status')}, "
1502     f"writer={row.get('writer_mode')}, error={error}"
1503 )
1504 log(f"Batch completion: {manifest['completion']}")
1505
1506 if PRINT_FINAL_OUTPUTS and generated_text:
1507     display(
1508         Markdown(
1509             f"### {dataset_id} / {example_id}\n\n{generated_text}"
1510         )
1511     )
1512 if success:
1513     case_succeeded = True
1514     break
1515 except Exception as exc:
1516     append_jsonl(
1517         ATTEMPT_LOG_PATH,
1518         {
1519             "event": "exception",
1520             "timestamp": utc_now(),
1521             "dataset_id": dataset_id,
1522             "example_id": example_id,
1523             "attempt": attempt_number,
1524             "success": False,
1525             "error": f"{type(exc).__name__}: {exc}",
1526         },
1527     )
1528 log(
1529     f"{dataset_id}/{example_id} attempt {attempt_number} raised "
1530     f"{type(exc).__name__}: {exc}"
1531 )
1532
1533 if not case_succeeded and not CONTINUE_AFTER_FAILURE:
1534     raise RuntimeError(f"Protected generation failed: {dataset_id}/{example_id}")
1535
1536 final_frame = rebuild_aggregate_artifacts()
1537 final_manifest = update_batch_manifest()
1538 assert_frozen_state()
1539 log("=" * 80)
1540 log(f"Session complete. Batch status: {final_manifest['status']}")
1541 log(f"Completion: {final_manifest['completion']}")
1542 log(f"Execution report: {EXECUTION_REPORT_PATH}")
1543 return final_frame
1544
1545
1546 protected_results = await run_protected_batch()
1547
1548 if not protected_results.empty:
1549     display(
1550         protected_results[
1551             [
1552                 "dataset_id",

```

```

1553         "example_id",
1554         "execution_outcome",
1555         "final_generation_path",
1556         "release_status",
1557         "native_support_rate",
1558         "output_word_count",
1559         "verified_fact_count",
1560         "verified_insight_count",
1561         "fallback_event_count",
1562         "provider_reported_total_tokens",
1563         "elapsed_seconds",
1564     ]
1565 ]
1566 )
1567
1568 # %% [markdown]
1569 # ## 6. Batch integrity checks
1570 #
1571 # This cell is safe to rerun at any point. It reports missing, failed, or
1572 # incomplete cases without regenerating anything.
1573
1574 # %%
1575 assert_frozen_state()
1576 records = collect_final_records()
1577 record_by_key = {
1578     (record.dataset_id, str(record.example_id)): record for record in records
1579 }
1580 integrity_rows = []
1581 for item in selection_manifest["run_order"]:
1582     key = (item["dataset_id"], str(item["example_id"]))
1583     record = record_by_key.get(key)
1584     pipeline_path = locate_pipeline_result(record) if record else None
1585     integrity_rows.append(
1586         {
1587             "position": item["position"],
1588             "dataset_id": item["dataset_id"],
1589             "example_id": item["example_id"],
1590             "record_present": record is not None,
1591             "generation_success": bool(
1592                 record and not record.error and record.generated_text.strip()
1593             ),
1594             "pipeline_result_present": bool(pipeline_path and pipeline_path.exists()),
1595             "reference_masked_in_record": bool(
1596                 record and record.references == [HELD_OUT_REFERENCE_SENTINEL]
1597             ),
1598             "model_input_isolation_hash_match": bool(
1599                 record
1600                 and pipeline_path
1601                 and extract_summary(record)[0].get(
1602                     "model_input_matches_reference_isolation_audit"
1603                 )
1604             ),
1605             "error": record.error if record else None,
1606         }
1607     )
1608 )
1609
1610 integrity_frame = pd.DataFrame(integrity_rows)
1611 display(integrity_frame)
1612 print("Successful:", int(integrity_frame["generation_success"].sum()), "/", expected_total)
1613 print("All references masked:", bool(integrity_frame["reference_masked_in_record"].all()))
1614 print("Implementation fingerprint unchanged: PASS")
1615
1616 # %% [markdown]
1617 # ## 7. Optional post-generation reference and source metrics
1618 #
1619 # This phase is blocked until all 25 protected generations are successful.
1620 # Only here are the true references joined back into a separate copy of the
1621 # GenerationRecords. The sealed generation file and all workflow artifacts are
1622 # left untouched.
1623
1624 # %%
1625 def unseal_generation_records_for_metrics():
1626     records = collect_final_records()
1627     successful = [record for record in records if not record.error and record.generated_text]
1628     if len(successful) != expected_total:
1629         raise RuntimeError(
1630             f"Cannot unseal references: {len(successful)}/{expected_total} "
1631             "protected generations are successful."
1632         )
1633     assert_frozen_state()
1634     true_lookup = {
1635         (example.dataset_id, str(example.example_id)): example
1636         for example in selected_examples
1637     }
1638     unsealed = [
1639         record.model_copy(
1640             update={
1641                 "references": true_lookup[
1642                     (record.dataset_id, str(record.example_id))
1643                 ]
1644             }
1645         )
1646     ]

```

```

1644         ].references
1645     }
1646 )
1647     for record in successful
1648 ]
1649 write_jsonl_atomic(UNSEALED_GENERATIONS_PATH, unsealed)
1650 manifest = read_json(BATCH_MANIFEST_PATH, {})
1651 if not manifest.get("true_references_unsealed_at"):
1652     manifest["true_references_unsealed_at"] = utc_now()
1653     manifest["true_references_unsealed_for"] = "post-generation evaluation only"
1654     manifest["unsealed_generations_path"] = relative_path(UNSEALED_GENERATIONS_PATH)
1655     write_json_atomic(BATCH_MANIFEST_PATH, manifest)
1656 return unsealed
1657
1658
1659 def build_metric_config(enabled_metrics, context, filename):
1660     payload = copy.deepcopy(read_json(PATHS["metric_config"]))
1661     payload["experiment_id"] = f"{EXPERIMENT_ID}_{context}"
1662     payload["prepared_examples_path"] = relative_path(OPERATIONAL_EXAMPLES_PATH)
1663     payload["generations_path"] = relative_path(UNSEALED_GENERATIONS_PATH)
1664     payload["result_directory"] = relative_path(RESULT_DIR)
1665     payload["baseline_variant"] = "full_system"
1666     payload["reference_metrics"]["enabled_metrics"] = enabled_metrics
1667     payload["reference_metrics"]["external_factuality_context"] = context
1668     payload["deepeval"]["enabled"] = False
1669     path = CONFIG_DIR / filename
1670     write_json_atomic(path, payload)
1671     return path
1672
1673
1674 reference_scores = pd.DataFrame()
1675 source_scores = pd.DataFrame()
1676
1677 if RUN_POST_GENERATION_REFERENCE_METRICS or RUN_POST_GENERATION_SOURCE_METRICS:
1678     unseal_generation_records_for_metrics()
1679
1680 if RUN_POST_GENERATION_REFERENCE_METRICS:
1681     reference_config_path = build_metric_config(
1682         [
1683             "bleu",
1684             "chrfr",
1685             "ter",
1686             "rouge1",
1687             "rouge2",
1688             "rougeL",
1689             "rougeLsum",
1690             "meteor",
1691             "bertscore",
1692             "parent",
1693         ],
1694         "references",
1695         "metrics_protected_reference.json",
1696     )
1697     log("Starting protected post-generation reference metrics")
1698     reference_scores = score_reference_metrics_for_notebook(
1699         PROJECT_DIR,
1700         generations_path=UNSEALED_GENERATIONS_PATH,
1701         metric_config_path=reference_config_path,
1702         output_path=RESULT_DIR / "protected_reference_metrics.jsonl",
1703         include_ineligible=True,
1704     )
1705     display(
1706         reference_scores.pivot_table(
1707             index="metric_name",
1708             values="score",
1709             aggfunc="mean",
1710         ).sort_index()
1711     )
1712
1713 if RUN_POST_GENERATION_SOURCE_METRICS:
1714     source_config_path = build_metric_config(
1715         ["hhem", "alignscore"],
1716         "source_text",
1717         "metrics_protected_source_grounded.json",
1718     )
1719     log("Starting protected post-generation source-grounded metrics")
1720     source_scores = score_reference_metrics_for_notebook(
1721         PROJECT_DIR,
1722         generations_path=UNSEALED_GENERATIONS_PATH,
1723         metric_config_path=source_config_path,
1724         output_path=RESULT_DIR / "protected_source_grounded_metrics.jsonl",
1725         include_ineligible=True,
1726     )
1727     display(
1728         source_scores.pivot_table(
1729             index="metric_name",
1730             values="score",
1731             aggfunc="mean",
1732         ).sort_index()
1733     )
1734

```

```

1735 print("Protected generation artifacts:", ARTIFACT_DIR)
1736 print("Batch manifest:", BATCH_MANIFEST_PATH)
1737 print("Execution report:", EXECUTION_REPORT_PATH)
1738 print("Exact outputs and summaries:", SUMMARY_JSONL_PATH)

```

Listing 12: P12: table2text_pydanticai/evaluation/notebooks/task_aware_direct_baseline_evaluation.py

```

1  # %% [markdown]
2  # # Task-Aware Direct Baseline Evaluation
3  #
4  # This notebook closes the main prompt-asymmetry threat in the 25-example
5  # evaluation. It compares three conditions on exactly the same stored cases:
6  #
7  # 1. `raw_generic_flash`: one DeepSeek V4 Flash call with a generic request;
8  # 2. `task_aware_direct_flash`: one DeepSeek V4 Flash call with the original
9  #    task request, task family, expected output form, language, and source;
10 # 3. `full_system`: the stored multi-agent workflow output.
11 #
12 # The notebook does not rerun Full or Raw Generic. Generation and judging
13 # are sharded by example, resumable, and accompanied by timestamped heartbeat
14 # messages. References are held out during generation.
15
16 # %%
17 import asyncio
18 import hashlib
19 import json
20 import os
21 import random
22 import re
23 import time
24 from collections import Counter
25 from datetime import datetime
26 from pathlib import Path
27
28 import pandas as pd
29 from IPython.display import Markdown, display
30
31 from table2text.evaluation import (
32     annotate_with_openai_judge_for_notebook,
33     default_paths,
34     generate_reports_for_notebook,
35     load_project_env,
36     score_reference_metrics_for_notebook,
37 )
38 from table2text.evaluation.datasets import read_examples, write_jsonl
39 from table2text.evaluation.generation import read_generations
40 from table2text.evaluation.backends import build_single_agent_prompt
41
42
43 # =====
44 # User configuration
45 # =====
46
47 PROJECT_DIR = Path("/Users/realgobs/Documents/MScproject/table2text_pydanticai")
48 EXPERIMENT_ID = "task_aware_direct_flash_25"
49
50 # None runs the canonical 25. Set to 1 or 5 only for a smoke test.
51 MAX_EXAMPLES = None
52
53 DEEPSEEK_MODEL = "deepseek-v4-flash"
54 MAX_SOURCE_CHARACTERS = 100_000
55 MAX_OUTPUT_TOKENS = 3_000
56 TEMPERATURE = 0.2
57 SEED = 42
58
59 RUN_TASK_AWARE_GENERATION = True
60 RUN_REFERENCE_METRICS = True
61 RUN_SOURCE_GROUNDED_METRICS = True
62
63 # These are opt-in because they consume API budget or create study materials.
64 RUN_GPT56_STRUCTURED_JUDGE = False
65 BUILD_BLINDED_HUMAN_PACKETS = False
66 RUN_STABILITY_EXPERIMENT = False
67
68 # When GPT judging is enabled, also fill the canonical 4975/full_system gap.
69 # The source/output stay unchanged; only the evaluator eligibility skip is
70 # bypassed, and that intervention is recorded in a sidecar note.
71 COMPLETE_MISSING_CANONICAL_GPT56 = True
72
73 GPT56_MODEL = "gpt-5.6-sol"
74 GPT56_REASONING_EFFORT = "high"
75
76 RESUME_GENERATION = True
77 RETRY_FAILED_GENERATIONS = True
78 GENERATION_ATTEMPTS = 2
79 RESUME_METRICS = True
80 RETRY_FAILED_JUDGE_ROWS = True
81
82 HEARTBEAT_SECONDS = 15

```

```

83 PRINT_GENERATED_OUTPUTS = True
84 INCLUDE_INELIGIBLE_STORED_OUTPUTS = True
85
86 # Optional stability study: one selected case per dataset, three runs/condition.
87 STABILITY_REPETITIONS = 3
88 STABILITY_SEEDS = [42, 43, 44]
89 STABILITY_INCLUDE_FULL_SYSTEM = True
90
91
92 # =====
93 # Canonical stored artifacts
94 # =====
95
96 PATHS = default_paths(PROJECT_DIR)
97 CANONICAL_GENERATIONS = (
98     PROJECT_DIR
99     / "evaluation/generations/"
100     "five_dataset_five_each_raw_generic_flash_20260805_181001_combined_generations.jsonl"
101 )
102 CANONICAL_REFERENCE_CONFIG = (
103     PROJECT_DIR
104     / "evaluation/config/"
105     "archive/"
106     "metrics_five_dataset_five_each_raw_generic_flash_20260805_181001_reference.json"
107 )
108 CANONICAL_SOURCE_CONFIG = (
109     PROJECT_DIR
110     / "evaluation/config/"
111     "archive/"
112     "metrics_five_dataset_five_each_raw_generic_flash_20260805_181001_source_grounded.json"
113 )
114 CANONICAL_SOURCE_METRICS = (
115     PROJECT_DIR
116     / "evaluation/results/"
117     "five_dataset_five_each_raw_generic_flash_20260805_181001_source_grounded_metrics.jsonl"
118 )
119 CANONICAL_GPT56_ANNOTATIONS = (
120     PROJECT_DIR / "evaluation/results/openai_structured_error_annotations.jsonl"
121 )
122
123 ARTIFACT_DIR = PROJECT_DIR / "evaluation" / "task_aware_direct_baseline"
124 CONFIG_DIR = ARTIFACT_DIR / "config"
125 PREPARED_DIR = ARTIFACT_DIR / "prepared"
126 GENERATION_DIR = ARTIFACT_DIR / "generations"
127 RESULT_DIR = ARTIFACT_DIR / "results"
128 JUDGE_DIR = RESULT_DIR / "judge_shards"
129 TASK_AWARE_JUDGE_DIR = JUDGE_DIR / "task_aware"
130 CANONICAL_GAP_JUDGE_DIR = JUDGE_DIR / "canonical_gap"
131 STABILITY_DIR = ARTIFACT_DIR / "stability"
132
133 for directory in (
134     CONFIG_DIR,
135     PREPARED_DIR,
136     GENERATION_DIR,
137     RESULT_DIR,
138     JUDGE_DIR,
139     TASK_AWARE_JUDGE_DIR,
140     CANONICAL_GAP_JUDGE_DIR,
141     STABILITY_DIR,
142 ):
143     directory.mkdir(parents=True, exist_ok=True)
144
145 PROGRESS_LOG = RESULT_DIR / f"{EXPERIMENT_ID}_progress.log"
146
147 # %% [markdown]
148 # ## Progress and persistence helpers
149 #
150 # Blocking model and metric calls run in a worker thread. The notebook event
151 # loop remains free to print a heartbeat every `HEARTBEAT_SECONDS`.
152
153 # %%
154 def log(message):
155     line = f"[{datetime.now().strftime('%H:%M:%S')}] {message}"
156     print(line, flush=True)
157     with PROGRESS_LOG.open("a", encoding="utf-8") as handle:
158         handle.write(line + "\n")
159
160
161 def write_json(path, payload):
162     path.parent.mkdir(parents=True, exist_ok=True)
163     path.write_text(json.dumps(payload, indent=2, ensure_ascii=False), encoding="utf-8")
164
165
166 def read_jsonl_objects(path):
167     path = Path(path)
168     if not path.exists():
169         return []
170     return [
171         json.loads(line)
172         for line in path.read_text(encoding="utf-8").splitlines()
173         if line.strip()
174     ]

```

```

174     ]
175
176
177 def safe_name(value):
178     return re.sub(r"^[A-Za-z0-9_.-]+", "_", str(value))
179
180
181 def key_for(item):
182     return (str(item.dataset_id), str(item.example_id))
183
184
185 def generation_key(item):
186     return (str(item.dataset_id), str(item.example_id), str(item.variant_id))
187
188
189 def file_fingerprint(*paths):
190     digest = hashlib.sha256()
191     for path in paths:
192         path = Path(path)
193         digest.update(str(path.resolve()).encode("utf-8"))
194         digest.update(path.read_bytes())
195     return digest.hexdigest()
196
197
198 async def with_heartbeat(awaitable, label, interval=HEARTBEAT_SECONDS):
199     started = time.perf_counter()
200     task = asyncio.create_task(awaitable)
201     while True:
202         try:
203             result = await asyncio.wait_for(asyncio.shield(task), timeout=interval)
204             log(f"{label}: complete after {time.perf_counter() - started:.1f}s")
205             return result
206         except asyncio.TimeoutError:
207             log(f"{label}: still running ({time.perf_counter() - started:.1f}s elapsed)")
208
209
210 async def run_blocking(func, *args, label, **kwargs):
211     return await with_heartbeat(
212         asyncio.to_thread(func, *args, **kwargs),
213         label,
214     )
215
216
217 def run_generation_blocking(
218     project_dir,
219     *,
220     examples_path,
221     variants_path,
222     output_path,
223     run_root,
224     resume,
225 ):
226     return asyncio.run(
227         generate_reports_for_notebook(
228             project_dir,
229             examples_path=examples_path,
230             variants_path=variants_path,
231             output_path=output_path,
232             run_root=run_root,
233             resume=resume,
234         )
235     )
236
237
238 def load_metric_frame(path):
239     rows = read_jsonl_objects(path)
240     return pd.DataFrame(rows)
241
242
243 def score_with_cache(
244     *,
245     generations_path,
246     config_path,
247     output_path,
248     include_ineligible,
249 ):
250     fingerprint_path = output_path.with_suffix(output_path.suffix + ".sha256")
251     current = file_fingerprint(generations_path, config_path)
252     if (
253         RESUME_METRICS
254         and output_path.exists()
255         and fingerprint_path.exists()
256         and fingerprint_path.read_text(encoding="utf-8").strip() == current
257     ):
258         return load_metric_frame(output_path)
259
260     frame = score_reference_metrics_for_notebook(
261         PROJECT_DIR,
262         generations_path=generations_path,
263         metric_config_path=config_path,
264         output_path=output_path,

```



```

265         include_ineligible=include_ineligible,
266     )
267     fingerprint_path.write_text(current + "\n", encoding="utf-8")
268     return frame
269
270
271 log(f"Notebook initialised. Progress log: {PROGRESS_LOG}")
272
273 # %% [markdown]
274 # ## 1. Preflight and exact 25-example selection
275
276 # %%
277 load_project_env(PROJECT_DIR)
278
279 required_paths = [
280     PATHS["prepared_examples"],
281     CANONICAL_GENERATIONS,
282     CANONICAL_REFERENCE_CONFIG,
283     CANONICAL_SOURCE_CONFIG,
284     CANONICAL_SOURCE_METRICS,
285 ]
286 missing_paths = [path for path in required_paths if not path.exists()]
287 if missing_paths:
288     raise FileNotFoundError(f"Missing required artifacts: {missing_paths}")
289
290 if RUN_TASK_AWARE_GENERATION and not os.getenv("DEEPSEEK_API_KEY"):
291     raise RuntimeError("DEEPSEEK_API_KEY is not available from the project environment.")
292
293 if RUN_GPT56_STRUCTURED_JUDGE and not (
294     os.getenv("OPENAI_API_KEY") or os.getenv("T2T_OPENAI_API_KEY")
295 ):
296     raise RuntimeError("OPENAI_API_KEY is required when GPT-5.6 judging is enabled.")
297
298 canonical_records = read_generations(CANONICAL_GENERATIONS)
299 full_records = [row for row in canonical_records if row.variant_id == "full_system"]
300 raw_records = [row for row in canonical_records if row.variant_id == "raw_generic_flash"]
301
302 full_keys = {key_for(row) for row in full_records}
303 raw_keys = {key_for(row) for row in raw_records}
304 if len(full_records) != 25 or len(raw_records) != 25 or full_keys != raw_keys:
305     raise ValueError(
306         "The canonical artifact is not the expected paired 25 Full + 25 Raw experiment."
307     )
308
309 selected_keys = sorted(full_keys)
310 if MAX_EXAMPLES is not None:
311     selected_keys = selected_keys[: int(MAX_EXAMPLES)]
312
313 all_examples = read_examples(PATHS["prepared_examples"])
314 example_by_key = {key_for(example): example for example in all_examples}
315 missing_examples = [key for key in selected_keys if key not in example_by_key]
316 if missing_examples:
317     raise ValueError(f"Prepared examples are missing canonical identities: {missing_examples}")
318
319 selected_examples = [example_by_key[key] for key in selected_keys]
320 selected_examples_path = PREPARED_DIR / f"{EXPERIMENT_ID}_examples.jsonl"
321 write_jsonl(selected_examples_path, selected_examples)
322
323 selection_frame = pd.DataFrame(
324     [
325         {
326             "dataset_id": example.dataset_id,
327             "example_id": example.example_id,
328             "task_family": getattr(example.task_family, "value", example.task_family),
329             "output_mode": getattr(example.output_mode, "value", example.output_mode),
330             "request": example.request,
331             "references": len(example.references),
332             "source_characters": len(example.source_text),
333         }
334         for example in selected_examples
335     ]
336 )
337
338 log(
339     f"Preflight passed: {len(selected_examples)} exactly matched cases across "
340     f"{selection_frame['dataset_id'].nunique()} datasets."
341 )
342 print("DeepSeek model:", DEEPSEEK_MODEL)
343 print("DeepSeek key available:", bool(os.getenv("DEEPSEEK_API_KEY")))
344 print("OpenAI key available:", bool(os.getenv("OPENAI_API_KEY") or os.getenv("T2T_OPENAI_API_KEY")))
345 display(selection_frame)
346
347 # %% [markdown]
348 # ## 2. Extract the existing same-item AlignScore/HHEM evidence
349 #
350 # This stage requires no model call. It extracts Full and Raw Generic values
351 # for the five researcher-adjudicated examples from the canonical 200-record
352 # source-grounded metrics artifact.
353
354 # %%
355 FOCUS_CASES = [

```

```

356     ("sportsett_basketball", "4934"),
357     ("totto", "totto-validation-204"),
358     ("e2e_nlg", "e2e_nlg-test-51"),
359     ("web_nlg", "web_nlg_en-test-51"),
360     ("dart", "dart-test-53"),
361 ]
362
363 existing_source_scores = load_metric_frame(CANONICAL_SOURCE_METRICS)
364 focus_mask = existing_source_scores.apply(
365     lambda row: (str(row["dataset_id"]), str(row["example_id"])) in FOCUS_CASES,
366     axis=1,
367 )
368 focus_source_scores = existing_source_scores[
369     focus_mask
370     & existing_source_scores["variant_id"].isin(["full_system", "raw_generic_flash"])
371     & existing_source_scores["status"].eq("scored")
372 ].copy()
373
374 focus_source_table = (
375     focus_source_scores.pivot_table(
376         index=["dataset_id", "example_id", "variant_id"],
377         columns="metric_name",
378         values="score",
379         aggfunc="first",
380     )
381     .reset_index()
382     .rename_axis(columns=None)
383 )
384
385 expected_focus_rows = len(FOCUS_CASES) * 2
386 if len(focus_source_table) != expected_focus_rows:
387     log(
388         f"WARNING: expected {expected_focus_rows} same-item source rows after pivot; "
389         f"found {len(focus_source_table)}."
390     )
391
392 focus_source_csv = RESULT_DIR / "selected_five_existing_full_raw_source_metrics.csv"
393 focus_source_table.to_csv(focus_source_csv, index=False)
394 log(f"Extracted existing same-item source metrics: {focus_source_csv}")
395 display(focus_source_table)
396
397 # %% [markdown]
398 # ## 3. Define and inspect the task-aware direct baseline
399 #
400 # `raw_baseline_prompt_style="structured"` uses one LLM call and includes the
401 # original request, task family, output mode, language, and source. It does not
402 # invoke any workflow agent, evidence ledger, verifier, writer pack, support
403 # map, auditor, or repair stage.
404
405 # %%
406 TASK_AWARE_VARIANT_ID = "task_aware_direct_flash"
407 task_aware_variant = {
408     "variant_id": TASK_AWARE_VARIANT_ID,
409     "enabled": True,
410     "backend": "callable",
411     "description": (
412         "One-call DeepSeek V4 Flash baseline with the original task and output contract."
413     ),
414     "settings_overrides": {
415         "raw_baseline_model": DEEPSEEK_MODEL,
416         "raw_baseline_prompt_style": "structured",
417         "raw_baseline_max_source_characters": MAX_SOURCE_CHARACTERS,
418         "raw_baseline_max_output_tokens": MAX_OUTPUT_TOKENS,
419         "raw_baseline_temperature": TEMPERATURE,
420     },
421     "callable_path": "table2text.evaluation_backends.single_agent_baseline",
422     "command": [],
423     "precomputed_path": None,
424     "repetitions": 1,
425     "seeds": [SEED],
426 }
427
428 task_aware_variants_path = CONFIG_DIR / f"variants_{EXPERIMENT_ID}.json"
429 write_json(task_aware_variants_path, {"variants": [task_aware_variant]})
430
431 preview_messages = build_single_agent_prompt(
432     selected_examples[0],
433     max_source_characters=MAX_SOURCE_CHARACTERS,
434     prompt_style="structured",
435 )
436 preview_without_source = preview_messages[1]["content"].split("Source data:", 1)[0]
437 print("SYSTEM PROMPT\n-----")
438 print(preview_messages[0]["content"])
439 print("\nUSER PROMPT CONTRACT PREVIEW\n-----")
440 print(preview_without_source + "Source data: [supplied in full, references excluded]")
441
442 # %% [markdown]
443 # ## 4. Generate the 25 task-aware direct outputs
444 #
445 # Each example has its own generation shard. Rerunning this cell resumes from
446 # completed shards. Failed calls can be retried without losing successful work.

```

```

447 # %%
448 task_aware_generations_path = GENERATION_DIR / f"{EXPERIMENT_ID}_generations.jsonl"
449 task_aware_run_root = GENERATION_DIR / f"{EXPERIMENT_ID}_runs"
450
451
452
453 def collect_task_aware_shards():
454     records = []
455     selected_key_set = set(selected_keys)
456     for shard in sorted((GENERATION_DIR / "shards").glob("*.jsonl")):
457         for record in read_generations(shard):
458             if (
459                 record.variant_id == TASK_AWARE_VARIANT_ID
460                 and key_for(record) in selected_key_set
461             ):
462                 records.append(record)
463     unique = {record.generation_id: record for record in records}
464     ordered = sorted(unique.values(), key=lambda row: (row.dataset_id, row.example_id))
465     write_jsonl(task_aware_generations_path, ordered)
466     return ordered
467
468
469 if RUN_TASK_AWARE_GENERATION:
470     (GENERATION_DIR / "shards").mkdir(parents=True, exist_ok=True)
471     for index, example in enumerate(selected_examples, start=1):
472         identity = f"{example.dataset_id}/{example.example_id}"
473         slug = f"{safe_name(example.dataset_id)}_{safe_name(example.example_id)}"
474         example_path = PREPARED_DIR / "shards" / f"{slug}.jsonl"
475         output_path = GENERATION_DIR / "shards" / f"{slug}.jsonl"
476         run_root = task_aware_run_root / slug
477         write_jsonl(example_path, [example])
478
479         log("=" * 76)
480         log(f"GENERATION {index}/{len(selected_examples)} starting: {identity}")
481
482         if output_path.exists() and RETRY_FAILED_GENERATIONS:
483             previous = read_generations(output_path)
484             if previous and any(row.error for row in previous):
485                 log(f"{identity}: removing failed shard before retry")
486                 output_path.unlink()
487
488         final_row = None
489         for attempt in range(1, GENERATION_ATTEMPTS + 1):
490             try:
491                 frame = await run_blocking(
492                     run_generation_blocking,
493                     PROJECT_DIR,
494                     examples_path=example_path,
495                     variants_path=task_aware_variants_path,
496                     output_path=output_path,
497                     run_root=run_root,
498                     resume=RESUME_GENERATION,
499                     label=f"{identity} generation attempt {attempt}/{GENERATION_ATTEMPTS}",
500                 )
501                 if frame.empty:
502                     raise RuntimeError("Generation returned no rows.")
503                 final_row = frame.iloc[-1]
504                 if not final_row.get("error"):
505                     break
506                 log(f"{identity}: recorded generation error: {final_row.get('error')}")
507             except Exception as exc:
508                 log(f"{identity}: {type(exc).__name__}: {exc}")
509
510             if attempt < GENERATION_ATTEMPTS:
511                 if output_path.exists():
512                     output_path.unlink()
513                 log(f"{identity}: retrying after 5 seconds")
514                 await asyncio.sleep(5)
515
516         collect_task_aware_shards()
517
518         if final_row is None:
519             log(f"{identity}: no generation row was produced")
520             continue
521
522         log(
523             f"GENERATION {index}/{len(selected_examples)} finished: {identity}; "
524             f"error={final_row.get('error')}; elapsed={final_row.get('elapsed_seconds')}s"
525         )
526         if PRINT_GENERATED_OUTPUTS:
527             generated = str(final_row.get("generated_text") or "").strip()
528             display(Markdown(f"### {identity} / task-aware direct\n\n{generated or '*No text*'")))
529     else:
530         log("Task-aware generation disabled; loading any existing shards.")
531
532 task_aware_records = collect_task_aware_shards()
533 task_aware_summary = pd.DataFrame(
534     [
535         {
536             "dataset_id": row.dataset_id,
537             "example_id": row.example_id,

```

```

538         "variant_id": row.variant_id,
539         "error": row.error,
540         "elapsed_seconds": row.elapsed_seconds,
541         "generated_characters": len(row.generated_text),
542     }
543     for row in task_aware_records
544 ]
545 )
546 display(task_aware_summary)
547
548 # %% [markdown]
549 # ## 5. Validate and construct the three-condition paired artifact
550
551 # %%
552 selected_key_set = set(selected_keys)
553 task_success = [
554     row
555     for row in task_aware_records
556     if key_for(row) in selected_key_set and row.error is None and row.generated_text.strip()
557 ]
558
559 task_success_keys = {key_for(row) for row in task_success}
560 missing_task_aware = sorted(selected_key_set - task_success_keys)
561 if missing_task_aware:
562     raise RuntimeError(
563         "Task-aware generation is incomplete. Rerun the generation cell. "
564         f"Missing/failed identities: {missing_task_aware}"
565     )
566
567 canonical_selected = [
568     row
569     for row in canonical_records
570     if key_for(row) in selected_key_set
571     and row.variant_id in {"full_system", "raw_generic_flash"}
572 ]
573 three_condition_records = canonical_selected + task_success
574 three_condition_records = sorted(
575     three_condition_records,
576     key=lambda row: (row.dataset_id, row.example_id, row.variant_id),
577 )
578
579 condition_counts = Counter(row.variant_id for row in three_condition_records)
580 expected_count = len(selected_keys)
581 for condition in ("full_system", "raw_generic_flash", TASK_AWARE_VARIANT_ID):
582     if condition_counts[condition] != expected_count:
583         raise ValueError(
584             f"Pairing failed for {condition}: expected {expected_count}, "
585             f"found {condition_counts[condition]}."
586         )
587
588 three_condition_generations_path = (
589     GENERATION_DIR / f"{EXPERIMENT_ID}_three_condition_generations.jsonl"
590 )
591 write_jsonl(three_condition_generations_path, three_condition_records)
592
593 pairing_table = pd.DataFrame(
594     [
595         {
596             "dataset_id": row.dataset_id,
597             "example_id": row.example_id,
598             "variant_id": row.variant_id,
599             "error": row.error,
600             "eligible": row.primary_evaluation_eligible,
601             "writer_mode": row.writer_mode,
602             "elapsed_seconds": row.elapsed_seconds,
603         }
604         for row in three_condition_records
605     ]
606 )
607
608 log(f"Three-condition artifact validated: {condition_counts}")
609 log(f"Saved: {three_condition_generations_path}")
610 display(pairing_table)
611
612 # %% [markdown]
613 # ## 6. Build matching reference and source-grounded metric configurations
614 #
615 # The focused reference set is BLEU, chrF, TER, ROUGE-L, METEOR and BERTScore
616 # F1, AlignScore and all three HHMM diagnostics are computed separately against
617 # the structured source, not against the references.
618
619 # %%
620 reference_config = json.loads(CANONICAL_REFERENCE_CONFIG.read_text(encoding="utf-8"))
621 reference_config["experiment_id"] = EXPERIMENT_ID
622 reference_config["prepared_examples_path"] = str(selected_examples_path)
623 reference_config["generations_path"] = str(three_condition_generations_path)
624 reference_config["result_directory"] = str(RESULT_DIR)
625 reference_config["baseline_variant"] = TASK_AWARE_VARIANT_ID
626 reference_config["reference_metrics"]["enabled_metrics"] = [
627     "bleu",
628     "chrF",

```

```

629     "ter",
630     "rougeL",
631     "meteor",
632     "bertscore",
633 ]
634 reference_config["deepeval"]["enabled"] = False
635
636 source_config = json.loads(CANONICAL_SOURCE_CONFIG.read_text(encoding="utf-8"))
637 source_config["experiment_id"] = f"{EXPERIMENT_ID}_source_grounded"
638 source_config["prepared_examples_path"] = str(selected_examples_path)
639 source_config["generations_path"] = str(three_condition_generations_path)
640 source_config["result_directory"] = str(RESULT_DIR)
641 source_config["baseline_variant"] = TASK_AWARE_VARIANT_ID
642 source_config["reference_metrics"]["enabled_metrics"] = ["hhem", "alignscore"]
643 source_config["reference_metrics"]["external_factuality_context"] = "source_text"
644 source_config["deepeval"]["enabled"] = False
645
646 reference_config_path = CONFIG_DIR / f"metrics_{EXPERIMENT_ID}_reference.json"
647 source_config_path = CONFIG_DIR / f"metrics_{EXPERIMENT_ID}_source_grounded.json"
648 write_json(reference_config_path, reference_config)
649 write_json(source_config_path, source_config)
650
651 print("Reference metrics:", reference_config["reference_metrics"]["enabled_metrics"])
652 print("Source metrics:", source_config["reference_metrics"]["enabled_metrics"])
653 print("Stored ineligible outputs included:", INCLUDE_INELIGIBLE_STORED_OUTPUTS)
654
655 # %% [markdown]
656 # ## 7. Score reference similarity
657
658 # %%
659 reference_metrics_path = RESULT_DIR / f"{EXPERIMENT_ID}_reference_metrics.jsonl"
660
661 if RUN_REFERENCE_METRICS:
662     log("Starting reference metrics for all three paired conditions")
663     reference_scores = await run_blocking(
664         score_with_cache,
665         generations_path=three_condition_generations_path,
666         config_path=reference_config_path,
667         output_path=reference_metrics_path,
668         include_ineligible=INCLUDE_INELIGIBLE_STORED_OUTPUTS,
669         label="Reference metrics",
670     )
671 else:
672     reference_scores = load_metric_frame(reference_metrics_path)
673
674 if reference_scores.empty:
675     log("No reference metric rows are available.")
676 else:
677     log(f"Reference metric rows: {len(reference_scores)}")
678     display(
679         reference_scores.groupby(["metric_name", "status"], dropna=False)
680         .size()
681         .rename("rows")
682         .reset_index()
683     )
684
685 # %% [markdown]
686 # ## 8. Score source-grounded AlignScore and HHEM
687
688 # %%
689 source_metrics_path = RESULT_DIR / f"{EXPERIMENT_ID}_source_grounded_metrics.jsonl"
690
691 if RUN_SOURCE_GROUNDED_METRICS:
692     log("Starting source-grounded AlignScore/HHEM for all three paired conditions")
693     source_scores = await run_blocking(
694         score_with_cache,
695         generations_path=three_condition_generations_path,
696         config_path=source_config_path,
697         output_path=source_metrics_path,
698         include_ineligible=INCLUDE_INELIGIBLE_STORED_OUTPUTS,
699         label="Source-grounded metrics",
700     )
701 else:
702     source_scores = load_metric_frame(source_metrics_path)
703
704 if source_scores.empty:
705     log("No source-grounded metric rows are available.")
706 else:
707     log(f"Source-grounded metric rows: {len(source_scores)}")
708     display(
709         source_scores.groupby(["metric_name", "status"], dropna=False)
710         .size()
711         .rename("rows")
712         .reset_index()
713     )
714
715 # %% [markdown]
716 # ## 9. Three-condition comparison tables
717
718 # %%
719 def scored_only(frame):

```

```

720     if frame.empty:
721         return frame.copy()
722     return frame[frame["status"].eq("scored") & frame["score"].notna()].copy()
723
724
725 def macro_table(frame):
726     scored = scored_only(frame)
727     if scored.empty:
728         return pd.DataFrame()
729     return (
730         scored.groupby(["variant_id", "metric_name"], as_index=False)["score"]
731         .mean()
732         .pivot(index="metric_name", columns="variant_id", values="score")
733         .reset_index()
734         .rename_axis(columns=None)
735     )
736
737
738 def dataset_table(frame):
739     scored = scored_only(frame)
740     if scored.empty:
741         return pd.DataFrame()
742     return (
743         scored.groupby(["dataset_id", "variant_id", "metric_name"], as_index=False)["score"]
744         .mean()
745         .pivot(
746             index=["dataset_id", "metric_name"],
747             columns="variant_id",
748             values="score",
749         )
750         .reset_index()
751         .rename_axis(columns=None)
752     )
753
754
755 reference_macro = macro_table(reference_scores)
756 source_macro = macro_table(source_scores)
757 reference_by_dataset = dataset_table(reference_scores)
758 source_by_dataset = dataset_table(source_scores)
759
760 print("REFERENCE METRICS – MACRO MEANS")
761 display(reference_macro)
762 print("REFERENCE METRICS – BY DATASET")
763 display(reference_by_dataset)
764 print("SOURCE-GROUNDED METRICS – MACRO MEANS")
765 display(source_macro)
766 print("SOURCE-GROUNDED METRICS – BY DATASET")
767 display(source_by_dataset)
768
769 reference_macro.to_csv(RESULT_DIR / f"{EXPERIMENT_ID}_reference_macro.csv", index=False)
770 source_macro.to_csv(RESULT_DIR / f"{EXPERIMENT_ID}_source_macro.csv", index=False)
771 reference_by_dataset.to_csv(
772     RESULT_DIR / f"{EXPERIMENT_ID}_reference_by_dataset.csv", index=False
773 )
774 source_by_dataset.to_csv(
775     RESULT_DIR / f"{EXPERIMENT_ID}_source_by_dataset.csv", index=False
776 )
777
778 # %%
779 # Direction-adjusted same-item wins. TER and unsupported-sentence rate are lower-is-better.
780 all_scores = pd.concat(
781     [
782         scored_only(reference_scores).assign(evaluation_family="reference"),
783         scored_only(source_scores).assign(evaluation_family="source"),
784     ],
785     ignore_index=True,
786 )
787
788 if not all_scores.empty:
789     all_scores["quality_score"] = all_scores.apply(
790         lambda row: row["score"] if bool(row["higher_is_better"]) else -row["score"],
791         axis=1,
792     )
793     paired = all_scores.pivot_table(
794         index=["evaluation_family", "dataset_id", "example_id", "metric_name"],
795         columns="variant_id",
796         values="quality_score",
797         aggfunc="first",
798     ).reset_index()
799
800     comparisons = []
801     for left, right, label in [
802         ("full_system", TASK_AWARE_VARIANT_ID, "Full vs Task-aware Direct"),
803         (TASK_AWARE_VARIANT_ID, "raw_generic_flash", "Task-aware Direct vs Raw Generic"),
804         ("full_system", "raw_generic_flash", "Full vs Raw Generic"),
805     ]:
806         available = paired.dropna(subset=[left, right]).copy()
807         delta = available[left] - available[right]
808         comparisons.append(
809             {
810                 "comparison": label,

```



```

811         "paired_metric_cases": len(available),
812         "left_wins": int((delta > 1e-12).sum()),
813         "ties": int(delta.abs().le(1e-12).sum()),
814         "right_wins": int((delta < -1e-12).sum()),
815     }
816 )
817 win_table = pd.DataFrame(comparisons)
818 display(win_table)
819 win_table.to_csv(RESULT_DIR / f"{EXPERIMENT_ID}_direction_adjusted_wins.csv", index=False)
820
821 if not source_scores.empty:
822     selected_three_source = source_scores[
823         source_scores.apply(
824             lambda row: (str(row["dataset_id"]), str(row["example_id"])) in FOCUS_CASES,
825             axis=1,
826         )
827         & source_scores["status"].eq("scored")
828     ].pivot_table(
829         index=["dataset_id", "example_id", "variant_id"],
830         columns="metric_name",
831         values="score",
832         aggfunc="first",
833     ).reset_index().rename_axis(columns=None)
834     selected_three_source.to_csv(
835         RESULT_DIR / "selected_five_three_condition_source_metrics.csv", index=False
836     )
837     print("SELECTED FIVE – ALL THREE CONDITIONS – SOURCE METRICS")
838     display(selected_three_source)
839
840 # %% [markdown]
841 # ## 10. Optional GPT-5.6 Sol structured error annotations
842 #
843 # This sends only the 25 new task-aware outputs. It supplies source + task + one
844 # output, excludes references/metrics/system identity, and saves one shard per
845 # case. Enable `RUN_GPT56_STRUCTURED_JUDGE` in the configuration cell only when
846 # API credit is available.
847
848 # %%
849 task_aware_judge_path = RESULT_DIR / f"{EXPERIMENT_ID}_gpt56_annotations.jsonl"
850 canonical_gap_judge_path = RESULT_DIR / "canonical_missing_gpt56_annotations.jsonl"
851
852
853 def collect_judge_shards(shard_directory, combined_path):
854     rows = []
855     for shard in sorted(Path(shard_directory).glob("*.jsonl")):
856         rows.extend(read_jsonl_objects(shard))
857     unique = {
858         (row["generation_id"], row["judge_model"], row["judge_repetition"]): row
859         for row in rows
860     }
861     ordered = sorted(
862         unique.values(),
863         key=lambda row: (row["dataset_id"], row["example_id"], row["variant_id"]),
864     )
865     write_jsonl(combined_path, ordered)
866     return ordered
867
868
869 if RUN_GPT56_STRUCTURED_JUDGE:
870     for index, record in enumerate(task_success, start=1):
871         identity = f"{record.dataset_id}/{record.example_id}"
872         slug = f"{safe_name(record.dataset_id)}_{safe_name(record.example_id)}"
873         judge_input = PREPARED_DIR / "judge_inputs" / f"{slug}.jsonl"
874         judge_output = TASK_AWARE_JUDGE_DIR / f"{slug}.jsonl"
875         write_jsonl(judge_input, [record])
876
877         if judge_output.exists() and RETRY_FAILED_JUDGE_ROWS:
878             prior = read_jsonl_objects(judge_output)
879             if prior and any(row.get("status") == "error" for row in prior):
880                 log(f"{identity}: removing failed GPT judge shard before retry")
881                 judge_output.unlink()
882
883         log(f"GPT-5.6 JUDGE {index}/{len(task_success)} starting: {identity}")
884     try:
885         frame = await run_blocking(
886             annotate_with_openai_judge_for_notebook,
887             PROJECT_DIR,
888             generations_path=judge_input,
889             output_path=judge_output,
890             judge_model=GPT56_MODEL,
891             judge_repetitions=1,
892             reasoning_effort=GPT56_REASONING_EFFORT,
893             max_source_characters=50_000,
894             max_output_tokens=2_500,
895             include_references=False,
896             include_system_identity=False,
897             include_metric_scores=False,
898             resume=True,
899             label=f"GPT-5.6 judge {identity}",
900         )
901     if frame.empty:

```



```

902         log(f"GPT-5.6 JUDGE {identity}: no row returned")
903     else:
904         row = frame.iloc[-1]
905         log(
906             f"GPT-5.6 JUDGE {identity}: status={row.get('status')}; "
907             f"errors={row.get('error_count')}; api_error={row.get('error')}"
908         )
909     except Exception as exc:
910         log(f"GPT-5.6 JUDGE {identity}: {type(exc).__name__}: {exc}")
911         collect_judge_shards(TASK_AWARE_JUDGE_DIR, task_aware_judge_path)
912 else:
913     log("GPT-5.6 structured judging disabled.")
914
915 task_aware_judge_rows = collect_judge_shards(
916     TASK_AWARE_JUDGE_DIR,
917     task_aware_judge_path,
918 )
919
920 canonical_judge_rows = read_jsonl_objects(CANONICAL_GPT56_ANNOTATIONS)
921 canonical_judged_ids = {row["generation_id"] for row in canonical_judge_rows}
922 missing_canonical_full = [
923     row
924     for row in full_records
925     if key_for(row) in selected_key_set
926     and row.generation_id not in canonical_judged_ids
927 ]
928
929 if RUN_GPT56_STRUCTURED_JUDGE and COMPLETE_MISSING_CANONICAL_GPT56:
930     if missing_canonical_full:
931         log(
932             "Canonical GPT-5.6 gap detected: "
933             + ", ".join(row.generation_id for row in missing_canonical_full)
934         )
935     for index, original_record in enumerate(missing_canonical_full, start=1):
936         # The original generation remains untouched. This evaluation-only copy
937         # bypasses the annotation runner's eligibility filter.
938         judge_record = original_record.model_copy(
939             update={"primary_evaluation_eligible": True}
940         )
941         identity = f"{judge_record.dataset_id}/{judge_record.example_id}/full_system"
942         slug = f"{safe_name(judge_record.dataset_id)}_{safe_name(judge_record.example_id)}"
943         judge_input = PREPARED_DIR / "judge_inputs" / f"canonical_gap_{slug}.jsonl"
944         judge_output = CANONICAL_GAP_JUDGE_DIR / f"{slug}.jsonl"
945         write_jsonl(judge_input, [judge_record])
946
947         if judge_output.exists() and RETRY_FAILED_JUDGE_ROWS:
948             prior = read_jsonl_objects(judge_output)
949             if prior and any(row.get("status") == "error" for row in prior):
950                 judge_output.unlink()
951
952         log(
953             f"GPT-5.6 CANONICAL GAP {index}/{len(missing_canonical_full)} "
954             f"starting: {identity}"
955         )
956     try:
957         frame = await run_blocking(
958             annotate_with_openai_judge_for_notebook,
959             PROJECT_DIR,
960             generations_path=judge_input,
961             output_path=judge_output,
962             judge_model=GPT56_MODEL,
963             judge_repetitions=1,
964             reasoning_effort=GPT56_REASONING_EFFORT,
965             max_source_characters=50_000,
966             max_output_tokens=2_500,
967             include_references=False,
968             include_system_identity=False,
969             include_metric_scores=False,
970             resume=True,
971             label=f"GPT-5.6 canonical gap {identity}",
972         )
973     if frame.empty:
974         log(f"GPT-5.6 CANONICAL GAP {identity}: no row returned")
975     else:
976         row = frame.iloc[-1]
977         log(
978             f"GPT-5.6 CANONICAL GAP {identity}: status={row.get('status')}; "
979             f"errors={row.get('error_count')}; api_error={row.get('error')}"
980         )
981     except Exception as exc:
982         log(f"GPT-5.6 CANONICAL GAP {identity}: {type(exc).__name__}: {exc}")
983
984 write_json(
985     RESULT_DIR / "canonical_gpt56_gap_provenance.json",
986     {
987         "reason": (
988             "The canonical annotation runner skipped generation records marked "
989             "primary_evaluation_eligible=false. The unchanged source and output "
990             "were judged through an evaluation-only copy with that gate enabled."
991         ),
992         "generation_ids": [row.generation_id for row in missing_canonical_full],
993     }
994 )

```

```

993     },
994 )
995
996 canonical_gap_judge_rows = collect_judge_shards(
997     CANONICAL_GAP_JUDGE_DIR,
998     canonical_gap_judge_path,
999 )
1000
1001 if task_aware_judge_rows:
1002     judge_frame = pd.DataFrame(task_aware_judge_rows)
1003     display(
1004         judge_frame.groupby(["variant_id", "status"], dropna=False)
1005         .agg(outputs=("generation_id", "count"), errors=("error_count", "sum"))
1006         .reset_index()
1007     )
1008
1009 combined_judge_path = RESULT_DIR / f"{EXPERIMENT_ID}_three_condition_gpt56_annotations.jsonl"
1010 combined_judge_rows = canonical_judge_rows + canonical_gap_judge_rows + task_aware_judge_rows
1011 combined_judge_rows = list(
1012     {
1013         (row["generation_id"], row["judge_model"], row["judge_repetition"]): row
1014         for row in combined_judge_rows
1015     }.values()
1016 )
1017 write_jsonl(combined_judge_path, combined_judge_rows)
1018 log(f"Combined canonical + task-aware GPT annotations: {combined_judge_path}")
1019
1020 # %% [markdown]
1021 # ## 11. Optional blinded independent-adjudication packets
1022 #
1023 # This only prepares materials. Do not recruit or collect new participant data
1024 # unless the work is covered by ethics approval or explicitly cleared by the
1025 # supervisor. System order is randomised and the mapping is stored separately.
1026
1027 # %%
1028 if BUILD_BLIDED_HUMAN_PACKETS:
1029     record_by_identity = {
1030         generation_key(row): row
1031         for row in three_condition_records
1032     }
1033     example_lookup = {key_for(example): example for example in selected_examples}
1034
1035     for comparison_name, condition_pair in {
1036         "full_vs_raw_generic": ("full_system", "raw_generic_flash"),
1037         "full_vs_task_aware": ("full_system", TASK_AWARE_VARIANT_ID),
1038     }.items():
1039         packet_rows = []
1040         private_rows = []
1041         rng = random.Random(SEED)
1042         for pair_index, case in enumerate(FOCUS_CASES, start=1):
1043             if case not in selected_key_set:
1044                 continue
1045             example = example_lookup[case]
1046             candidates = list(condition_pair)
1047             rng.shuffle(candidates)
1048             output_a = record_by_identity[(case[0], case[1], candidates[0])]
1049             output_b = record_by_identity[(case[0], case[1], candidates[1])]
1050             pair_id = f"PAIR_{pair_index:02d}"
1051             packet_rows.append(
1052                 {
1053                     "pair_id": pair_id,
1054                     "dataset_id": case[0],
1055                     "example_id": case[1],
1056                     "task_request": example.request,
1057                     "task_family": getattr(example.task_family, "value", example.task_family),
1058                     "output_mode": getattr(example.output_mode, "value", example.output_mode),
1059                     "source_text": example.source_text,
1060                     "output_a": output_a.generated_text,
1061                     "output_b": output_b.generated_text,
1062                     "preferred_output_a_b_or_tie": "",
1063                     "output_a_error_notes": "",
1064                     "output_b_error_notes": "",
1065                 }
1066             )
1067             private_rows.append(
1068                 {
1069                     "pair_id": pair_id,
1070                     "output_a_variant": candidates[0],
1071                     "output_b_variant": candidates[1],
1072                 }
1073             )
1074
1075     packet_path = RESULT_DIR / f"blind_packet_{comparison_name}.csv"
1076     key_path = RESULT_DIR / f"PRIVATE_blind_key_{comparison_name}.json"
1077     pd.DataFrame(packet_rows).to_csv(packet_path, index=False)
1078     write_json(key_path, private_rows)
1079     log(f"Blinded packet: {packet_path}")
1080     log(f"PRIVATE mapping: {key_path}")
1081 else:
1082     log("Blinded human packet generation disabled.")
1083

```

```

1084 # %% [markdown]
1085 # ## 12. Optional small repeated-generation stability experiment
1086 #
1087 # This is diagnostic, not part of the main 25-case result. It uses one selected
1088 # example per dataset and three seeds. Because it uses the current code and
1089 # current hosted model aliases, report it as a later stability check rather
1090 # than silently merging it with the historical main experiment.
1091
1092 # %%
1093 if RUN_STABILITY_EXPERIMENT:
1094     stability_cases = [case for case in FOCUS_CASES if case in selected_key_set]
1095     stability_examples = [example_by_key[case] for case in stability_cases]
1096     stability_records = []
1097
1098     stability_variants = [
1099         {
1100             **task_aware_variant,
1101             "variant_id": "stability_task_aware_direct_flash",
1102         }
1103     ]
1104     if STABILITY_INCLUDE_FULL_SYSTEM:
1105         stability_variants.append(
1106             {
1107                 "variant_id": "stability_full_system",
1108                 "enabled": True,
1109                 "backend": "table2text",
1110                 "description": "Current Full workflow used only for the stability diagnostic.",
1111                 "settings_overrides": {},
1112                 "callable_path": None,
1113                 "command": [],
1114                 "precomputed_path": None,
1115                 "repetitions": 1,
1116                 "seeds": [SEED],
1117             }
1118         )
1119
1120     total_calls = len(stability_examples) * len(stability_variants) * STABILITY_REPETITIONS
1121     call_number = 0
1122     for example in stability_examples:
1123         for base_variant in stability_variants:
1124             for repetition_index in range(STABILITY_REPETITIONS):
1125                 call_number += 1
1126                 seed_subset = STABILITY_SEEDS[: repetition_index + 1]
1127                 variant = {
1128                     **base_variant,
1129                     "repetitions": repetition_index + 1,
1130                     "seeds": seed_subset,
1131                 }
1132                 identity = (
1133                     f"{example.dataset_id}/{example.example_id}/"
1134                     f"{variant['variant_id']}/seed={seed_subset[-1]}"
1135                 )
1136                 slug = (
1137                     f"{safe_name(example.dataset_id)}_{safe_name(example.example_id)}_"
1138                     f"{safe_name(variant['variant_id'])}"
1139                 )
1140                 example_path = STABILITY_DIR / "prepared" / f"{slug}.jsonl"
1141                 variant_path = STABILITY_DIR / "config" / f"{slug}.json"
1142                 output_path = STABILITY_DIR / "generations" / f"{slug}.jsonl"
1143                 run_root = STABILITY_DIR / "runs" / slug
1144                 write_jsonl(example_path, [example])
1145                 write_json(variant_path, {"variants": [variant]})
1146                 log(f"STABILITY {call_number}/{total_calls}: {identity}")
1147                 await run_blocking(
1148                     run_generation_blocking,
1149                     PROJECT_DIR,
1150                     examples_path=example_path,
1151                     variants_path=variant_path,
1152                     output_path=output_path,
1153                     run_root=run_root,
1154                     resume=True,
1155                     label=f"Stability {identity}",
1156                 )
1157
1158     for path in sorted((STABILITY_DIR / "generations").glob("*.jsonl")):
1159         stability_records.extend(read_generations(path))
1160     stability_records = list(
1161         {record.generation_id: record for record in stability_records}.values()
1162     )
1163     stability_generations_path = STABILITY_DIR / "stability_generations.jsonl"
1164     write_jsonl(stability_generations_path, stability_records)
1165     log(f"Stability generations saved: {stability_generations_path}")
1166     display(
1167         pd.DataFrame(
1168             [
1169                 {
1170                     "dataset_id": row.dataset_id,
1171                     "example_id": row.example_id,
1172                     "variant_id": row.variant_id,
1173                     "seed": row.seed,
1174                     "error": row.error,

```

```

1175         "elapsed_seconds": row.elapsed_seconds,
1176     }
1177     for row in stability_records
1178     ]
1179 )
1180 )
1181 else:
1182     log("Repeated-generation stability experiment disabled.")
1183
1184 # %% [markdown]
1185 # ## 13. Final artifact manifest
1186
1187 # %%
1188 manifest = {
1189     "experiment_id": EXPERIMENT_ID,
1190     "created_or_updated_at": datetime.now().isoformat(),
1191     "model": DEEPSEEK_MODEL,
1192     "task_aware_prompt_style": "structured",
1193     "seed": SEED,
1194     "temperature": TEMPERATURE,
1195     "selected_examples": len(selected_keys),
1196     "condition_counts": dict(condition_counts),
1197     "include_ineligible_stored_outputs": INCLUDE_INELIGIBLE_STORED_OUTPUTS,
1198     "canonical_generations": str(CANONICAL_GENERATIONS),
1199     "selected_examples_path": str(selected_examples_path),
1200     "task_aware_generations": str(task_aware_generations_path),
1201     "three_condition_generations": str(three_condition_generations_path),
1202     "reference_metrics": str(reference_metrics_path),
1203     "source_grounded_metrics": str(source_metrics_path),
1204     "existing_selected_five_source_metrics": str(focus_source_csv),
1205     "task_aware_gpt56_annotations": str(task_aware_judge_path),
1206     "canonical_gap_gpt56_annotations": str(canonical_gap_judge_path),
1207     "progress_log": str(PROGRESS_LOG),
1208 }
1209 manifest_path = RESULT_DIR / f"{EXPERIMENT_ID}_manifest.json"
1210 write_json(manifest_path, manifest)
1211
1212 log("Evaluation notebook complete.")
1213 print(json.dumps(manifest, indent=2))

```

Listing 13: P13: table2text_pydanticai/evaluation/scripts/annotations/build_interactive_gpt56_annotation_artifacts.py

```

1  """Build transparent interactive GPT-5.6 Sol annotation artifacts.
2
3  This script does not call an API. The semantic judgements below were produced in
4  an interactive model session using the same source-only error taxonomy as the
5  project's OpenAI judge. Existing API-authenticated annotations are read but never
6  modified.
7  """
8
9  from __future__ import annotations
10
11  import csv
12  import json
13  from collections import Counter, defaultdict
14  from datetime import datetime, timezone
15  from pathlib import Path
16  from typing import Any
17
18  from table2text.evaluation.models import GenerationRecord, LLMJudgeAnnotationRecord
19
20
21  PROJECT_DIR = Path(__file__).resolve().parents[3]
22  EVALUATION_DIR = PROJECT_DIR / "evaluation"
23  RESULT_DIR = EVALUATION_DIR / "task_aware_direct_baseline" / "results"
24
25  TASK_AWARE_GENERATIONS = (
26      EVALUATION_DIR
27      / "task_aware_direct_baseline"
28      / "generations"
29      / "task_aware_direct_flash_25_generations.jsonl"
30  )
31  CANONICAL_GENERATIONS = (
32      EVALUATION_DIR
33      / "generations"
34      / "five_dataset_five_each_raw_generic_flash_20260805_181001_combined_generations.jsonl"
35  )
36  API_ANNOTATIONS = EVALUATION_DIR / "results" / "openai_structured_error_annotations.jsonl"
37
38  TASK_AWARE_OUTPUT = RESULT_DIR / "task_aware_direct_flash_25_interactive_gpt56_annotations.jsonl"
39  CANONICAL_GAP_OUTPUT = RESULT_DIR / "canonical_gap_interactive_gpt56_annotation.jsonl"
40  COMBINED_OUTPUT = RESULT_DIR / "gpt56_all_75_annotations_with_provenance.jsonl"
41  SUMMARY_CSV = RESULT_DIR / "gpt56_all_75_annotation_summary.csv"
42  PROVENANCE_OUTPUT = RESULT_DIR / "interactive_gpt56_annotation_provenance.json"
43  REPORT_OUTPUT = RESULT_DIR / "interactive_gpt56_annotation_report.md"
44
45  JUDGE_MODEL = "gpt-5.6-sol"
46
47

```

```

48 def error(span: str, category: str, explanation: str) -> dict[str, str]:
49     return {
50         "error_span": span,
51         "category": category,
52         "correction_or_explanation": explanation,
53     }
54
55
56 # Empty lists are deliberate scored judgements, not unreviewed records.
57 TASK_AWARE_ERRORS: dict[str, list[dict[str, str]]] = {
58     "dart_dart-test-204__task_aware_direct_flash_r0_s42": [],
59     "dart_dart-test-217__task_aware_direct_flash_r0_s42": [],
60     "dart_dart-test-244__task_aware_direct_flash_r0_s42": [],
61     "dart_dart-test-260__task_aware_direct_flash_r0_s42": [],
62     "dart_dart-test-53__task_aware_direct_flash_r0_s42": [],
63     "e2e_nlg_e2e_nlg-test-178__task_aware_direct_flash_r0_s42": [],
64     "e2e_nlg_e2e_nlg-test-51__task_aware_direct_flash_r0_s42": [],
65     "e2e_nlg_e2e_nlg-test-54__task_aware_direct_flash_r0_s42": [],
66     "e2e_nlg_e2e_nlg-test-61__task_aware_direct_flash_r0_s42": [],
67     "e2e_nlg_e2e_nlg-test-65__task_aware_direct_flash_r0_s42": [],
68     "sportsett_basketball_4934__task_aware_direct_flash_r0_s42": [],
69     "sportsett_basketball_4972__task_aware_direct_flash_r0_s42": [
70         error(
71             "but the Bucks could not overcome poor outside shooting.",
72             "CONTEXT",
73             "The source supports Milwaukee's 10-for-44 three-point shooting, but a box score alone does not establish
74             that this was a causal barrier that produced the loss.",
75         ),
76         error(
77             "then answered every Milwaukee push in the second half.",
78             "CONTEXT",
79             "The source provides quarter totals but no play-by-play sequence, so it cannot verify every Milwaukee push
80             or a corresponding Phoenix response.",
81         ),
82     ],
83     "sportsett_basketball_4975__task_aware_direct_flash_r0_s42": [
84         error(
85             "while forcing 20 Pistons turnovers.",
86             "CONTEXT",
87             "The source records 20 Detroit turnovers and 13 Milwaukee steals, but does not state that Milwaukee forced
88             every turnover.",
89         ),
90         error(
91             "but could not overcome its shooting struggles and turnovers.",
92             "CONTEXT",
93             "The box score supports the shooting and turnover figures, but it does not establish them as the causal
94             explanation for Detroit's defeat.",
95         ),
96         error(
97             "as only Griffin and Jackson scored in double figures",
98             "NUMBER",
99             "Andre Drummond also scored in double figures with exactly 10 points, so Griffin and Jackson were not the
100             only Detroit players to do so.",
101         ),
102         error(
103             "Detroit's miscues proved costly against a Bucks team that converted those turnovers into scoring chances.",
104             "CONTEXT",
105             "The source contains turnover totals but no points-off-turnovers or possession-level evidence showing that
106             Milwaukee converted those turnovers into scoring chances or that they proved causal.",
107         ),
108     ],
109     "sportsett_basketball_4982__task_aware_direct_flash_r0_s42": [
110         error(
111             "Friday night",
112             "NOT CHECKABLE",
113             "The source supplies the date and weekday but no start time or time-of-day information.",
114         ),
115         error(
116             "The Bucks led from the opening tip, building a 43-14 edge after the first quarter and never looking back.",
117             "CONTEXT",
118             "Quarter-end scores support the 43-14 first-quarter margin, but the source has no play-by-play evidence for
119             the opening tip or for a continuous lead throughout the game.",
120         ),
121         error(
122             "DeAndre' Bembry led all scorers with 19 points",
123             "CONTEXT",
124             "Bembry tied for the game-high 19 points with Milwaukee's Khris Middleton and Malcolm Brogdon rather than
125             leading alone.",
126         ),
127         error(
128             "Atlanta struggled against Milwaukee's pressure",
129             "CONTEXT",
130             "The source records 21 Atlanta turnovers but does not identify Milwaukee pressure as their cause.",
131         ),
132         error(
133             "helping Milwaukee maintain its large lead throughout the second half.",
134             "CONTEXT",
135             "The source does not attribute lead maintenance to those bench performances, and quarter-end totals do not
136             establish the continuous game state throughout the half.",
137         ),
138     ],
139 }

```

```

128     ),
129 ],
130 "sportsett_basketball__4986__task_aware_direct_flash__r0__s42": [
131     error(
132         "Monday night",
133         "NOT CHECKABLE",
134         "The source supplies the date and weekday but no start time or time-of-day information.",
135     )
136 ],
137 "totto__totto-validation-204__task_aware_direct_flash__r0__s42": [],
138 "totto__totto-validation-217__task_aware_direct_flash__r0__s42": [],
139 "totto__totto-validation-244__task_aware_direct_flash__r0__s42": [
140     error(
141         "represented Everett (39th Middlesex), and retired to run for State Treasurer.",
142         "TASK/FORMAT",
143         "The city/district and electoral-history details come from unhighlighted cells. The request required exactly one sentence about the highlighted cells and explicitly excluded unrelated cells.",
144     )
145 ],
146 "totto__totto-validation-260__task_aware_direct_flash__r0__s42": [],
147 "totto__totto-validation-712__task_aware_direct_flash__r0__s42": [],
148 "web_nlg__web_nlg_en-test-178__task_aware_direct_flash__r0__s42": [],
149 "web_nlg__web_nlg_en-test-51__task_aware_direct_flash__r0__s42": [],
150 "web_nlg__web_nlg_en-test-54__task_aware_direct_flash__r0__s42": [],
151 "web_nlg__web_nlg_en-test-61__task_aware_direct_flash__r0__s42": [],
152 "web_nlg__web_nlg_en-test-65__task_aware_direct_flash__r0__s42": [],
153 }
154
155 CANONICAL_GAP_ERRORS: dict[str, list[dict[str, str]]] = {
156     "sportsett_basketball__4975__full_system__r0__s42": [
157         error(
158             "Participant context records Milwaukee Bucks entered with 16 wins and 7 losses; Detroit Pistons entered with 13 wins and 9 losses.",
159             "CONTEXT",
160             "Those records each total the listed game number and therefore include this result. Milwaukee improved to 16-7 and Detroit fell to 13-9; they did not enter with those records.",
161         ),
162         error(
163             "The entire generated output is a sequence of evidence-ledger statements and rankings rather than a coherent multi-paragraph game report.",
164             "TASK/FORMAT",
165             "The request required a coherent game report that leads with the result and selects the most important performances and contrasts. The output is a mechanical inventory with repeated result statements and no paragraph-level narrative organization.",
166         ),
167     ]
168 }
169
170
171 def read_jsonl(path: Path) -> list[dict[str, Any]]:
172     return [
173         json.loads(line)
174         for line in path.read_text(encoding="utf-8").splitlines()
175         if line.strip()
176     ]
177
178
179 def write_jsonl(path: Path, rows: list[dict[str, Any]]) -> None:
180     path.parent.mkdir(parents=True, exist_ok=True)
181     path.write_text(
182         "".join(json.dumps(row, ensure_ascii=False) + "\n" for row in rows),
183         encoding="utf-8",
184     )
185
186
187 def annotation_record(
188     generation: GenerationRecord,
189     errors: list[dict[str, str]],
190 ) -> dict[str, Any]:
191     record = LLMJudgeAnnotationRecord.model_validate(
192         {
193             "generation_id": generation.generation_id,
194             "dataset_id": generation.dataset_id,
195             "example_id": generation.example_id,
196             "variant_id": generation.variant_id,
197             "repetition": generation.repetition,
198             "judge_model": JUDGE_MODEL,
199             "judge_repetition": 0,
200             "status": "scored",
201             "errors": errors,
202             "error_count": len(errors),
203             "duration_seconds": None,
204             "input_tokens": None,
205             "output_tokens": None,
206             "total_tokens": None,
207             "error": None,
208         }
209     )
210     return record.model_dump(mode="json")
211
212

```



```

213 def build_interactive_rows() -> tuple[list[dict[str, Any]], list[dict[str, Any]]:
214     task_generations = [
215         GenerationRecord.model_validate(row) for row in read_jsonl(TASK_AWARE_GENERATIONS)
216     ]
217     task_by_id = {row.generation_id: row for row in task_generations}
218     if set(task_by_id) != set(TASK_AWARE_ERRORS):
219         missing = sorted(set(task_by_id) - set(TASK_AWARE_ERRORS))
220         extra = sorted(set(TASK_AWARE_ERRORS) - set(task_by_id))
221         raise RuntimeError(f"Task-aware identity mismatch: missing={missing}; extra={extra}")
222
223     canonical_generations = [
224         GenerationRecord.model_validate(row) for row in read_jsonl(CANONICAL_GENERATIONS)
225     ]
226     canonical_by_id = {row.generation_id: row for row in canonical_generations}
227     if not set(CANONICAL_GAP_ERRORS).issubset(canonical_by_id):
228         missing = sorted(set(CANONICAL_GAP_ERRORS) - set(canonical_by_id))
229         raise RuntimeError(f"Canonical gap generations not found: {missing}")
230
231     task_rows = [
232         annotation_record(task_by_id[generation_id], TASK_AWARE_ERRORS[generation_id])
233         for generation_id in sorted(TASK_AWARE_ERRORS)
234     ]
235     gap_rows = [
236         annotation_record(canonical_by_id[generation_id], CANONICAL_GAP_ERRORS[generation_id])
237         for generation_id in sorted(CANONICAL_GAP_ERRORS)
238     ]
239     return task_rows, gap_rows
240
241
242 def provenance_row(row: dict[str, Any], execution_mode: str) -> dict[str, Any]:
243     return {
244         **row,
245         "execution_mode": execution_mode,
246         "api_authenticated": execution_mode == "openai_responses_api",
247     }
248
249
250 def summary_rows(rows: list[dict[str, Any]]) -> list[dict[str, Any]]:
251     grouped: dict[tuple[str, str, str], dict[str, Any]] = {}
252     for row in rows:
253         key = (row["execution_mode"], row["variant_id"], row["dataset_id"])
254         item = grouped.setdefault(
255             key,
256             {
257                 "execution_mode": key[0],
258                 "variant_id": key[1],
259                 "dataset_id": key[2],
260                 "outputs": 0,
261                 "outputs_with_errors": 0,
262                 "errors": 0,
263             },
264         )
265         item["outputs"] += 1
266         item["errors"] += row["error_count"]
267         item["outputs_with_errors"] += int(row["error_count"] > 0)
268     return [grouped[key] for key in sorted(grouped)]
269
270
271 def markdown_report(
272     task_rows: list[dict[str, Any]],
273     gap_rows: list[dict[str, Any]],
274     api_rows: list[dict[str, Any]],
275     combined_rows: list[dict[str, Any]],
276 ) -> str:
277     category_counts = Counter(
278         error_item["category"]
279         for row in combined_rows
280         for error_item in row.get("errors", [])
281     )
282     variant_counts = defaultdict(lambda: {"outputs": 0, "flagged": 0, "errors": 0})
283     for row in combined_rows:
284         item = variant_counts[row["variant_id"]]
285         item["outputs"] += 1
286         item["flagged"] += int(row["error_count"] > 0)
287         item["errors"] += row["error_count"]
288
289     lines = [
290         "# GPT-5.6 Sol Structured Error Annotation Results",
291         "",
292         "## Scope and provenance",
293         "",
294         f"- Judge label: `{JUDGE_MODEL}`",
295         f"- Existing API-authenticated rows retained unchanged: **{len(api_rows)}**",
296         f"- Interactive task-aware rows added: **{len(task_rows)}**",
297         f"- Interactive canonical-gap rows added: **{len(gap_rows)}**",
298         f"- Three-condition analysis rows: **{len(combined_rows)}**",
299         "- The interactive rows were produced without an OpenAI API call. Their model label follows the active model selection reported for the session, but that identity and the reasoning setting were not returned by an API response.",
300         "- Consequently, interactive rows must be reported separately from API-authenticated rows or accompanied by the execution-mode provenance field. They are not a silent replacement for a controlled API run.",

```



```

301     "- Judgements used the project taxonomy: NAME, NUMBER, WORD, CONTEXT, NOT CHECKABLE, OTHER, OMISSION and TASK/
FORMAT.",
302     "- The intended basis was source data + task request + one generated output. Human references and automatic
metric scores were not used as correctness criteria. Unlike the API runner, the surrounding interactive session was
not a formally blinded environment.",
303     "",
304     "## Three-condition totals",
305     "",
306     "| Variant | Outputs | Outputs flagged | Errors |",
307     "|---|---|---|---|",
308 ]
309 for variant_id in sorted(variant_counts):
310     item = variant_counts[variant_id]
311     lines.append(
312         f"| {variant_id} | {item['outputs']} | {item['flagged']} | {item['errors']} |"
313     )
314
315 lines.extend(
316     [
317         "",
318         "## Error categories across all 75 rows",
319         "",
320         "| Category | Count |",
321         "|---|---|",
322     ]
323 )
324 for category, count in sorted(category_counts.items()):
325     lines.append(f"| {category} | {count} |")
326
327 lines.extend(
328     [
329         "",
330         "## Interactive coverage table",
331         "",
332         "| Dataset | Example | Variant | Error count | Categories |",
333         "|---|---|---|---|---|",
334     ]
335 )
336 for row in sorted(
337     task_rows + gap_rows,
338     key=lambda item: (
339         item["dataset_id"],
340         item["example_id"],
341         item["variant_id"],
342     ),
343 ):
344     categories = ", ".join(item["category"] for item in row["errors"]) or "None"
345     lines.append(
346         f"| {row['dataset_id']} | {row['example_id']} | "
347         f"| {row['variant_id']} | {row['error_count']} | {categories} |"
348     )
349
350 lines.extend(
351     [
352         "",
353         "## Interactive annotations",
354         "",
355         "Rows not listed below were reviewed and assigned an empty error list.",
356         "",
357     ]
358 )
359 for row in task_rows + gap_rows:
360     if not row["errors"]:
361         continue
362     lines.extend(
363         [
364             f"### {row['dataset_id']} / {row['example_id']} / {row['variant_id']}",
365             "",
366         ]
367     )
368     for index, item in enumerate(row["errors"], start=1):
369         lines.extend(
370             [
371                 f"{index}. **{item['category']}**: \"{item['error_span']}\",",
372                 f"    - {item['correction_or_explanation']}",
373             ]
374         )
375     lines.append("")
376
377 lines.extend(
378     [
379         "## Interpretation boundary",
380         "",
381         "The combined file is convenient for descriptive analysis, but it contains mixed execution provenance: 49
API-authenticated annotations and 26 interactive annotations. Any dissertation table using all 75 rows should
disclose this split. Confirmatory claims about GPT-5.6 Sol as an API judge should use the 49 API-authenticated rows
unless the remaining cases are later rerun through the same controlled API procedure.",
382         "",
383         "## Artifacts",
384         "",
385         f"- {TASK_AWARE_OUTPUT.relative_to(PROJECT_DIR)}",

```

```

386         f"- \{CANONICAL_GAP_OUTPUT.relative_to(PROJECT_DIR)}`",
387         f"- \{COMBINED_OUTPUT.relative_to(PROJECT_DIR)}`",
388         f"- \{SUMMARY_CSV.relative_to(PROJECT_DIR)}`",
389         f"- \{PROVENANCE_OUTPUT.relative_to(PROJECT_DIR)}`",
390         """,
391     ]
392 )
393 return "\n".join(lines)
394
395
396 def main() -> None:
397     print("[1/5] Validating generation identities and interactive judgements...", flush=True)
398     task_rows, gap_rows = build_interactive_rows()
399     if len(task_rows) != 25 or len(gap_rows) != 1:
400         raise RuntimeError(
401             f"Expected 25 task-aware rows and one canonical gap; got {len(task_rows)} and {len(gap_rows)}"
402         )
403
404     print("[2/5] Writing separate interactive annotation artifacts...", flush=True)
405     write_jsonl(TASK_AWARE_OUTPUT, task_rows)
406     write_jsonl(CANONICAL_GAP_OUTPUT, gap_rows)
407
408     print("[3/5] Combining with existing API rows without modifying them...", flush=True)
409     api_rows = read_jsonl(API_ANNOTATIONS)
410     if len(api_rows) != 49:
411         raise RuntimeError(f"Expected 49 existing API rows; found {len(api_rows)}")
412     combined_rows = [
413         provenance_row(row, "openai_responses_api") for row in api_rows
414     ] + [
415         provenance_row(row, "interactive_session") for row in task_rows + gap_rows
416     ]
417     combined_rows.sort(
418         key=lambda row: (
419             row["dataset_id"],
420             row["example_id"],
421             row["variant_id"],
422             row["judge_repetition"],
423         )
424     )
425     identities = [row["generation_id"] for row in combined_rows]
426     if len(combined_rows) != 75 or len(set(identities)) != 75:
427         raise RuntimeError(
428             f"Combined artifact must contain 75 unique generation IDs; got rows={len(combined_rows)}, unique={len(set(
429                 identities))}"
430         )
431     counts = Counter(row["variant_id"] for row in combined_rows)
432     expected_counts = {
433         "full_system": 25,
434         "raw_generic_flash": 25,
435         "task_aware_direct_flash": 25,
436     }
437     if counts != expected_counts:
438         raise RuntimeError(f"Unexpected condition counts: {counts}")
439     write_jsonl(COMBINED_OUTPUT, combined_rows)
440
441     print("[4/5] Writing summary tables, provenance and report...", flush=True)
442     summary = summary_rows(combined_rows)
443     with SUMMARY_CSV.open("w", encoding="utf-8", newline="") as handle:
444         writer = csv.DictWriter(handle, fieldnames=list(summary[0]))
445         writer.writeheader()
446         writer.writerows(summary)
447
448     provenance = {
449         "created_at": datetime.now(timezone.utc).isoformat(),
450         "judge_model": JUDGE_MODEL,
451         "model_identity_basis": "User-reported active model selection; not independently returned by an API response.",
452         "reasoning_effort_basis": "High reasoning was reported for the active session; not independently returned by an API response.",
453         "execution_mode": "interactive_session",
454         "openai_api_call_made": False,
455         "api_authenticated": False,
456         "taxonomy_source": "src/table2text/evaluation/llm_judge_annotations.py",
457         "api_rows_preserved_unchanged": len(api_rows),
458         "interactive_task_aware_rows": len(task_rows),
459         "interactive_canonical_gap_rows": len(gap_rows),
460         "controlled_blinding": False,
461         "analysis_restriction": (
462             "Do not describe the interactive rows as API-authenticated. Stratify by execution_mode "
463             "or disclose the 49 API / 26 interactive split when reporting the combined artifact."
464         ),
465         "source_files": {
466             "task_aware_generations": str(TASK_AWARE_GENERATIONS),
467             "canonical_generations": str(CANONICAL_GENERATIONS),
468             "existing_api_annotations": str(API_ANNOTATIONS),
469         },
470     }
471     PROVENANCE_OUTPUT.write_text(
472         json.dumps(provenance, ensure_ascii=False, indent=2) + "\n",
473         encoding="utf-8",
474     )
475     REPORT_OUTPUT.write_text(

```

```

475     markdown_report(task_rows, gap_rows, api_rows, combined_rows),
476     encoding="utf-8",
477 )
478
479 print("[5/5] Complete.", flush=True)
480 print(f"Task-aware annotations: {TASK_AWARE_OUTPUT}")
481 print(f"Canonical gap: {CANONICAL_GAP_OUTPUT}")
482 print(f"Combined 75 rows: {COMBINED_OUTPUT}")
483 print(f"Report: {REPORT_OUTPUT}")
484 print(f"Condition counts: {dict(counts)}")
485 print(
486     "Interactive error counts: "
487     f"task-aware={sum(row['error_count'] for row in task_rows)}, "
488     f"canonical-gap={sum(row['error_count'] for row in gap_rows)}"
489 )
490
491
492 if __name__ == "__main__":
493     main()

```

Listing 14: P14: table2text_pydanticai/evaluation/scripts/build_protected_holdout_results_bank.py

```

1  """Build the protected 25-example evidence bank from sealed artifacts."""
2
3  from __future__ import annotations
4
5  import json
6  import re
7  from collections import Counter
8  from pathlib import Path
9  from typing import Any
10
11  import numpy as np
12  import pandas as pd
13
14
15  REPO_DIR = Path(__file__).resolve().parents[2]
16  PROJECT_DIR = REPO_DIR.parent
17  OUTPUT_PATH = (
18      PROJECT_DIR
19      / "docs/evaluation/PROTECTED_HOLDOUT_25_COMPLETE_RESULTS_BANK.md"
20  )
21
22  FULL_ROOT = REPO_DIR / "evaluation/protected_holdout_full_system"
23  BASELINE_ROOT = REPO_DIR / "evaluation/protected_holdout_baseline"
24
25  SELECTION_PATH = FULL_ROOT / "prepared/protected_selection_manifest.json"
26  FULL_MANIFEST_PATH = FULL_ROOT / "results/protected_batch_manifest.json"
27  FULL_CONFIG_PATH = FULL_ROOT / "config/protected_full_system_flash.json"
28  FULL_SUMMARY_PATH = FULL_ROOT / "results/protected_generation_summary.csv"
29  STAGE_USAGE_PATH = FULL_ROOT / "results/stage_token_usage.csv"
30
31  SEALED_PAIRED_PATH = (
32      BASELINE_ROOT / "comparison/full_system_and_baseline_sealed.jsonl"
33  )
34  METRIC_INPUT_PATH = (
35      BASELINE_ROOT / "comparison/full_system_and_baseline_for_metrics.jsonl"
36  )
37  BASELINE_GENERATIONS_PATH = (
38      BASELINE_ROOT / "generations/baseline_generations_sealed.jsonl"
39  )
40  REFERENCE_METRICS_PATH = (
41      BASELINE_ROOT / "results/reference_alignment_metrics.jsonl"
42  )
43  SOURCE_METRICS_PATH = (
44      BASELINE_ROOT / "results/source_grounded_metrics.jsonl"
45  )
46  JUDGE_ANNOTATIONS_PATH = (
47      BASELINE_ROOT / "gpt56_judge/results/gpt56_structured_annotations.jsonl"
48  )
49
50  CONDITION_LABELS = {
51      "full_system": "Full System",
52      "baseline": "Baseline",
53  }
54
55  LOWER_IS_BETTER = {
56      "ter",
57      "corpus_ter",
58      "hmem_2_1_open_unsupported_sentence_rate",
59  }
60
61  CORPUS_METRICS = {"corpus_bleu", "corpus_chrf", "corpus_ter"}
62
63  METRIC_NAMES = {
64      "alignscore_base": "AlignScore (base)",
65      "bertscore_f1": "BERTScore F1",
66      "bleu": "BLEU",

```

```

67     "chrF": "chrF",
68     "corpus_bleu": "Corpus BLEU",
69     "corpus_chrf": "Corpus chrF",
70     "corpus_ter": "Corpus TER",
71     "hhem_2_1_open_mean_support": "HHEM mean support",
72     "hhem_2_1_open_min_sentence_support": "HHEM minimum sentence support",
73     "hhem_2_1_open_unsupported_sentence_rate": (
74         "HHEM unsupported-sentence rate"
75     ),
76     "meteor": "METEOR",
77     "rouge1": "ROUGE-1",
78     "rouge2": "ROUGE-2",
79     "rougeL": "ROUGE-L",
80     "rougeLsum": "ROUGE-Lsum",
81     "ter": "TER",
82 }
83
84 PRIMARY_METRICS = {
85     "bertscore_f1",
86     "chrF",
87     "meteor",
88     "alignscore_base",
89     "hhem_2_1_open_mean_support",
90     "hhem_2_1_open_unsupported_sentence_rate",
91 }
92
93
94 def read_json(path: Path) -> Any:
95     return json.loads(path.read_text(encoding="utf-8"))
96
97
98 def read_jsonl(path: Path) -> list[dict[str, Any]]:
99     return [
100         json.loads(line)
101         for line in path.read_text(encoding="utf-8").splitlines()
102         if line.strip()
103     ]
104
105
106 def rel(path: Path) -> str:
107     return str(path.resolve().relative_to(PROJECT_DIR.resolve()))
108
109
110 def clean(value: Any) -> str:
111     if value is None or (isinstance(value, float) and np.isnan(value)):
112         return ""
113     if isinstance(value, (bool, np.bool_)):
114         return "Yes" if value else "No"
115     if isinstance(value, (np.floating, float)):
116         return f"{float(value):.4f}"
117     if isinstance(value, (np.integer, int)):
118         return f"{int(value)}"
119     return str(value).replace("|", "\\|").replace("\n", "<br>")
120
121
122 def table(headers: list[str], rows: list[list[Any]]) -> str:
123     lines = [
124         "|" + " | ".join(headers) + " |",
125         "|" + " | ".join("----" for _ in headers) + " |",
126     ]
127     lines.extend(
128         "|" + " | ".join(clean(value) for value in row) + " |"
129         for row in rows
130     )
131     return "\n".join(lines)
132
133
134 def code_block(value: str, language: str = "text") -> str:
135     return f"````{language}\n{value.rstrip()}\n````"
136
137
138 def word_count(value: str) -> int:
139     return len(re.findall(r"\S+", value))
140
141
142 def direction(metric_name: str) -> str:
143     return "Lower" if metric_name in LOWER_IS_BETTER else "Higher"
144
145
146 def adjusted_delta(metric_name: str, full_score: float, baseline_score: float) -> float:
147     if metric_name in LOWER_IS_BETTER:
148         return baseline_score - full_score
149     return full_score - baseline_score
150
151
152 def outcome(delta: float, tolerance: float = 1e-12) -> str:
153     if delta > tolerance:
154         return "Full System"
155     if delta < -tolerance:
156         return "Baseline"
157     return "Tie"

```

```

158
159
160 def bootstrap_interval(values: np.ndarray, seed: int = 42) -> tuple[float, float]:
161     rng = np.random.default_rng(seed)
162     indices = rng.integers(0, len(values), size=(10_000, len(values)))
163     means = values[indices].mean(axis=1)
164     return float(np.quantile(means, 0.025)), float(np.quantile(means, 0.975))
165
166
167 def annotation_text(annotation: dict[str, Any] | None) -> str:
168     if annotation is None:
169         return "No annotation record was found."
170     if not annotation["errors"]:
171         return "GPT-5.6 Sol reported no errors."
172     lines = []
173     for index, error in enumerate(annotation["errors"], start=1):
174         lines.extend(
175             [
176                 f"{index}. **{error['category']}**",
177                 f"    - Error span: {error['error_span']}",
178                 f"    - Correction or explanation: {error['correction_or_explanation']}",
179             ]
180         )
181     return "\n".join(lines)
182
183
184
185 def main() -> None:
186     required = [
187         SELECTION_PATH,
188         FULL_MANIFEST_PATH,
189         FULL_CONFIG_PATH,
190         FULL_SUMMARY_PATH,
191         STAGE_USAGE_PATH,
192         SEALED_PAISED_PATH,
193         METRIC_INPUT_PATH,
194         BASELINE_GENERATIONS_PATH,
195         REFERENCE_METRICS_PATH,
196         SOURCE_METRICS_PATH,
197         JUDGE_ANNOTATIONS_PATH,
198     ]
199     missing = [str(path) for path in required if not path.exists()]
200     if missing:
201         raise FileNotFoundError(
202             "Missing protected evaluation artifacts:\n" + "\n".join(missing)
203         )
204
205     selection = read_json(SELECTION_PATH)
206     full_manifest = read_json(FULL_MANIFEST_PATH)
207     full_summary = pd.read_csv(FULL_SUMMARY_PATH)
208     stage_usage = pd.read_csv(STAGE_USAGE_PATH)
209     sealed_records = read_jsonl(SEALED_PAISED_PATH)
210     metric_input_records = read_jsonl(METRIC_INPUT_PATH)
211     baseline_records = read_jsonl(BASELINE_GENERATIONS_PATH)
212     reference_metrics = pd.DataFrame(read_jsonl(REFERENCE_METRICS_PATH))
213     source_metrics = pd.DataFrame(read_jsonl(SOURCE_METRICS_PATH))
214     annotations = read_jsonl(JUDGE_ANNOTATIONS_PATH)
215
216     selection_order = [
217         (str(item["dataset_id"]), str(item["example_id"]))
218         for item in selection["run_order"]
219     ]
220     selected_keys = set(selection_order)
221
222     sealed_by_key = {
223         (row["dataset_id"], str(row["example_id"]), row["variant_id"]): row
224         for row in sealed_records
225     }
226     metric_input_by_key = {
227         (row["dataset_id"], str(row["example_id"]), row["variant_id"]): row
228         for row in metric_input_records
229     }
230     annotation_by_key = {
231         (row["dataset_id"], str(row["example_id"]), row["variant_id"]): row
232         for row in annotations
233     }
234     summary_by_key = {
235         (str(row.dataset_id), str(row.example_id)): row
236         for row in full_summary.itertuples(index=False)
237     }
238     selection_by_key = {
239         (str(item["dataset_id"]), str(item["example_id"])): item
240         for item in selection["run_order"]
241     }
242
243     assert len(selection_order) == 25
244     assert len(selected_keys) == 25
245     assert len(sealed_records) == 50
246     assert len(metric_input_records) == 50
247     assert len(baseline_records) == 25
248     assert len(annotations) == 50

```

```

249 assert all((key[0], key[1], "full_system") in sealed_by_key for key in selection_order)
250 assert all((key[0], key[1], "baseline") in sealed_by_key for key in selection_order)
251
252 all_metrics = pd.concat([reference_metrics, source_metrics], ignore_index=True)
253 scored_metrics = all_metrics[all_metrics["status"] == "scored"].copy()
254
255 individual_metrics = scored_metrics[
256     ~scored_metrics["metric_name"].isin(CORPUS_METRICS)
257 ].copy()
258
259 paired = (
260     individual_metrics.pivot_table(
261         index=["dataset_id", "example_id", "metric_name"],
262         columns="variant_id",
263         values="score",
264         aggfunc="first",
265     )
266     .dropna()
267     .reset_index()
268 )
269 paired["adjusted_delta"] = paired.apply(
270     lambda row: adjusted_delta(
271         row["metric_name"], row["full_system"], row["baseline"]
272     ),
273     axis=1,
274 )
275
276 macro_rows = []
277 for metric_name, group in paired.groupby("metric_name"):
278     deltas = group["adjusted_delta"].to_numpy(dtype=float)
279     low, high = bootstrap_interval(deltas)
280     full_score = float(group["full_system"].mean())
281     baseline_score = float(group["baseline"].mean())
282     delta = adjusted_delta(metric_name, full_score, baseline_score)
283     macro_rows.append(
284         {
285             "metric_name": metric_name,
286             "baseline": baseline_score,
287             "full_system": full_score,
288             "adjusted_delta": delta,
289             "ci_low": low,
290             "ci_high": high,
291             "full_wins": int((deltas > 1e-12).sum()),
292             "ties": int((np.abs(deltas) <= 1e-12).sum()),
293             "baseline_wins": int((deltas < -1e-12).sum()),
294             "preferred": outcome(delta),
295         }
296     )
297 macro = pd.DataFrame(macro_rows).sort_values("metric_name")
298
299 corpus = scored_metrics[scored_metrics["metric_name"].isin(CORPUS_METRICS)]
300 corpus_pivot = (
301     corpus.pivot_table(
302         index=["dataset_id", "metric_name"],
303         columns="variant_id",
304         values="score",
305         aggfunc="first",
306     )
307     .dropna()
308     .reset_index()
309 )
310 corpus_pivot["adjusted_delta"] = corpus_pivot.apply(
311     lambda row: adjusted_delta(
312         row["metric_name"], row["full_system"], row["baseline"]
313     ),
314     axis=1,
315 )
316
317 dataset_means = (
318     individual_metrics.groupby(
319         ["dataset_id", "metric_name", "variant_id"], as_index=False
320     )["score"]
321     .mean()
322     .pivot(
323         index=["dataset_id", "metric_name"],
324         columns="variant_id",
325         values="score",
326     )
327     .dropna()
328     .reset_index()
329 )
330 dataset_means["adjusted_delta"] = dataset_means.apply(
331     lambda row: adjusted_delta(
332         row["metric_name"], row["full_system"], row["baseline"]
333     ),
334     axis=1,
335 )
336 dataset_means["preferred"] = dataset_means["adjusted_delta"].map(outcome)
337
338 judge = pd.DataFrame(
339     [

```

```

340         {
341             **{key: value for key, value in row.items() if key != "errors"},
342             "condition": CONDITION_LABELS[row["variant_id"]],
343         }
344         for row in annotations
345     ]
346 )
347
348 judge_pairs = (
349     judge.pivot_table(
350         index=["dataset_id", "example_id"],
351         columns="condition",
352         values="error_count",
353         aggfunc="first",
354     )
355     .dropna()
356     .reset_index()
357 )
358 judge_pairs["comparison"] = np.where(
359     judge_pairs["Full System"] < judge_pairs["Baseline"],
360     "Full System fewer",
361     np.where(
362         judge_pairs["Full System"] > judge_pairs["Baseline"],
363         "Full System more",
364         "Equal",
365     ),
366 )
367
368 category_rows = []
369 for row in annotations:
370     counts = Counter(error["category"] for error in row["errors"])
371     for category in [
372         "NAME",
373         "NUMBER",
374         "WORD",
375         "CONTEXT",
376         "NOT CHECKABLE",
377         "OTHER",
378         "OMISSION",
379         "TASK/FORMAT",
380     ]:
381         category_rows.append(
382             {
383                 "dataset_id": row["dataset_id"],
384                 "example_id": str(row["example_id"]),
385                 "condition": CONDITION_LABELS[row["variant_id"]],
386                 "category": category,
387                 "count": counts.get(category, 0),
388             }
389         )
390 categories = pd.DataFrame(category_rows)
391
392 length_rows = []
393 for row in sealed_records:
394     length_rows.append(
395         {
396             "dataset_id": row["dataset_id"],
397             "example_id": str(row["example_id"]),
398             "condition": CONDITION_LABELS[row["variant_id"]],
399             "words": word_count(row["generated_text"]),
400         }
401     )
402 lengths = pd.DataFrame(length_rows)
403
404 metric_availability_rows = []
405 for (metric_name, status), group in all_metrics.groupby(["metric_name", "status"]):
406     metric_availability_rows.append(
407         [METRIC_NAMES.get(metric_name, metric_name), status, len(group)]
408     )
409
410 lines: list[str] = []
411 add = lines.append
412
413 add("# Protected Holdout 25: Complete Results Evidence Bank")
414 add("")
415 add(
416     "> Purpose: a result- and output-focused information bank from which a full "
417     "dissertation evaluation chapter can be written. It preserves the protected "
418     "experimental design, every measured score, every generated output, every "
419     "reference, the normalized source supplied for each case, and all GPT-5.6 Sol "
420     "structured annotations. It is not itself a polished chapter."
421 )
422 add("")
423 add("## 1. Evidence status")
424 add("")
425 add(
426     table(
427         ["Component", "Status", "Count", "Evidence"],
428         [
429             ["Protected examples", "Complete", 25, "5 datasets x 5 examples"],
430             ["Full System outputs", "Complete", 25, "One released output per example"],

```



```

431         ["Baseline outputs", "Complete", 25, "One direct generic-call output per example"],
432         ["Paired outputs", "Complete", 50, "25 matched Full System/Baseline pairs"],
433         [
434             "Reference metrics",
435             "Complete except PARENT",
436             len(reference_metrics),
437             "All configured observations retained, including unavailable statuses",
438         ],
439         [
440             "Source-grounded metrics",
441             "Complete",
442             len(source_metrics),
443             "AlignScore and three HHEM diagnostics for all 50 outputs",
444         ],
445         [
446             "GPT-5.6 Sol structured annotations",
447             "Complete",
448             len(annotations),
449             "One independent annotation per output",
450         ],
451         [
452             "Human annotations on this protected set",
453             "Not collected",
454             0,
455             "Do not describe GPT annotations as human gold labels",
456         ],
457     ],
458 )
459 )
460 add("")
461 add("### Core artifact paths")
462 add("")
463 for label, path in [
464     ("Protected selection", SELECTION_PATH),
465     ("Full System batch manifest", FULL_MANIFEST_PATH),
466     ("Full System run summary", FULL_SUMMARY_PATH),
467     ("Sealed paired generations", SEALED_PAISED_PATH),
468     ("Reference-unsealed metrics copy", METRIC_INPUT_PATH),
469     ("Reference metric observations", REFERENCE_METRICS_PATH),
470     ("Source-grounded metric observations", SOURCE_METRICS_PATH),
471     ("GPT-5.6 Sol annotations", JUDGE_ANNOTATIONS_PATH),
472 ]:
473     add(f"-- [{label}]{rel(path)}")
474 add("")
475
476 add("## 2. Headline evidence")
477 add("")
478 for item in [
479     f"Full System has the better direction-adjusted macro mean on all {len(macro)} scored per-example automatic
480 metrics.",
481     "Full System wins all 13 per-example metric families on DART and ToTTo.",
482     "Baseline wins 10 of 13 per-example metric families on E2E; both conditions receive zero GPT-reported errors
483 there.",
484     "SportSett splits six metric families to each condition with one tie.",
485     "GPT-5.6 Sol reports 16 errors for each condition; 18 Full System outputs and 14 Baseline outputs have zero
486 reported errors.",
487     "Full System native support rate is 1.0 for all 25 released outputs.",
488     "PARENT produced no numerical scores and is not evidence for either condition.",
489 ]:
490     add(f"-- {item}")
491 add("")
492
493 add("## 3. Experimental design and isolation")
494 add("")
495 add(
496     table(
497         ["Property", "Recorded value"],
498         [
499             ["Experiment", full_manifest.get("experiment")],
500             ["Selection timestamp", full_manifest.get("selection_date")],
501             ["Protected/unseen", full_manifest.get("protected_unseen")],
502             ["Examples", full_manifest.get("examples")],
503             ["Dataset allocation", json.dumps(full_manifest.get("datasets"), sort_keys=True)],
504             ["Historical overlap at selection", selection.get("historical_overlap_count_at_selection")],
505             ["Frozen Git commit", full_manifest.get("git_commit")],
506             ["Frozen implementation SHA-256", full_manifest.get("implementation_sha256")],
507             ["Full System configuration SHA-256", full_manifest.get("configuration_sha256")],
508             ["Generation seed", full_manifest.get("generation_seed")],
509             ["Provider seed forwarded", full_manifest.get("provider_seed_forwarded")],
510             ["Human references available during generation", "No"],
511             ["Generations per condition per example", 1],
512         ],
513     )
514 )
515 add("")
516 add("### Conditions")
517 add("")
518 add(
519     table(
520         ["Condition", "Model", "Input request", "Architecture", "Reference access"],
521         [

```

```

519         [
520             "Full System",
521             "DeepSeek V4 Flash for all six roles",
522             "Benchmark task-specific request and explicit task contract",
523             "Six-role workflow with evidence, verification, Writer, support mapping, and audit",
524             "None during generation",
525         ],
526         [
527             "Baseline",
528             "DeepSeek V4 Flash",
529             "Understand the supplied data and report its strongest supported findings.",
530             "One direct model call; no supplied task family or output-form metadata",
531             "None during generation",
532         ],
533     ],
534 )
535 )
536 add("")
537 add(
538     "The comparison deliberately tests whether the workflow can infer and enforce "
539     "the appropriate communication task better than a direct generic call using "
540     "the same model family. It does not isolate architecture from prompt "
541     "specificity. Any dissertation claim must describe the treatment as the "
542     "complete task-aware workflow, not as a pure agent-count ablation."
543 )
544 add("")
545 add("### Full System model-role configuration")
546 add("")
547 model_rows = []
548 for role, model in full_manifest.get("models_by_role", {}).items():
549     model_rows.append([role, model])
550 add(table(["Role", "Model"], model_rows))
551 add("")
552 parameter_rows = []
553 for item in full_manifest.get("model_parameters", []):
554     parameter_rows.append(
555         [item["role"], item["temperature"], item["max_tokens"], item["builder"]]
556     )
557 add(table(["Role", "Temperature", "Maximum tokens", "Builder"], parameter_rows))
558 add("")
559 add("### GPT-5.6 Sol annotation protocol")
560 add("")
561 add(
562     table(
563         ["Setting", "Value"],
564         [
565             ["Judge", "gpt-5.6-sol"],
566             ["Reasoning effort", "high"],
567             ["Calls", "50 independent single-output calls"],
568             ["Human references supplied", "No"],
569             ["Other condition output supplied", "No"],
570             ["Condition identity supplied", "No"],
571             ["Metric scores supplied", "No"],
572             ["Common benchmark task context supplied", "Yes"],
573         ],
574         [
575             "Taxonomy",
576             "NAME, NUMBER, WORD, CONTEXT, NOT CHECKABLE, OTHER, OMISSION, TASK/FORMAT",
577         ],
578     )
579 )
580 add("")
581
582 add("## 4. Metric framework")
583 add("")
584 add(
585     table(
586         ["Class", "Metrics", "What the class contributes", "Main limitation"],
587         [
588             [
589                 "Lexical/reference alignment",
590                 "BLEU, chrF, METEOR, ROUGE, TER",
591                 "Measures wording and content overlap with human references",
592                 "Penalises valid paraphrase and different report lengths",
593             ],
594             [
595                 "Semantic reference alignment",
596                 "BERTScore F1",
597                 "Measures contextual semantic similarity to references",
598                 "High semantic similarity does not guarantee exact factuality",
599             ],
600             [
601                 "Table-aware alignment",
602                 "PARENT",
603                 "Intended to combine reference and table entailment",
604                 "Unavailable in this run; no PARENT score can be claimed",
605             ],
606             [
607                 "Source-grounded diagnostics",
608                 "AlignScore, HHEM",
609                 "Tests support against normalized source rather than reference wording",
610             ],
611         ],
612     )
613 )

```

```

610         "Long structured inputs and model/context limits affect comparability",
611     ],
612     [
613         "Independent structured review",
614         "GPT-5.6 Sol error annotations",
615         "Produces span-level categories and explanations",
616         "An LLM judgement, not a human gold standard",
617     ],
618     [
619         "White-box workflow evidence",
620         "Support rate, support map, evidence/fact/insight counts, audit",
621         "Shows traceability and workflow behaviour",
622         "Available only for Full System and not an external quality score",
623     ],
624 ],
625 )
626 )
627 add("")
628 add(
629     "Recommended chapter emphasis: BERTScore, chrF, METEOR, AlignScore, the HHEM "
630     "support diagnostics, and GPT-5.6's category-level annotations. BLEU, ROUGE, "
631     "and TER remain useful secondary evidence. Corpus metrics should be labelled "
632     "as dataset-level. PARENT must be reported as unavailable."
633 )
634 add("")
635 add("### Metric availability")
636 add("")
637 add(table(["Metric", "Status", "Observations"], metric_availability_rows))
638 add("")
639 add(
640     "PARENT generated no numerical results: 30 observations were unavailable "
641     "because the optional KaijuML PARENT package was not installed, and 20 were "
642     "skipped because the corresponding adapter did not expose a PARENT-compatible "
643     "table. PARENT is therefore excluded from all comparative claims."
644 )
645 add("")
646
647 add("## 5. Overall automatic results")
648 add("")
649 add(
650     "The adjusted difference is defined so that a positive value always favours "
651     "Full System. For TER and HHEM unsupported-sentence rate, lower is better and "
652     "the subtraction is reversed. The confidence interval is a descriptive "
653     "10,000-resample paired bootstrap over the 25 examples; it should not be "
654     "treated as a substitute for a larger protected test set."
655 )
656 add("")
657 macro_table_rows = []
658 for row in macro.itertuples(index=False):
659     macro_table_rows.append(
660         [
661             METRIC_NAMES.get(row.metric_name, row.metric_name),
662             direction(row.metric_name),
663             row.baseline,
664             row.full_system,
665             row.adjusted_delta,
666             f"[{row.ci_low:.4f}, {row.ci_high:.4f}]",
667             f"{row.full_wins}/{row.ties}/{row.baseline_wins}",
668             row.preferred,
669         ]
670     )
671 add(
672     table(
673         [
674             "Metric",
675             "Better direction",
676             "Baseline",
677             "Full System",
678             "Adjusted difference",
679             "Paired bootstrap 95% CI",
680             "Full/Tie/Base wins",
681             "Macro preference",
682         ],
683         macro_table_rows,
684     )
685 )
686 add("")
687 add(
688     "The paired intervals exclude zero for AlignScore, BERTScore, BLEU, chrF, "
689     "HHEM minimum sentence support, METEOR, ROUGE-1, ROUGE-2, and TER. They cross "
690     "zero for HHEM mean support, HHEM unsupported-sentence rate, ROUGE-L, and "
691     "ROUGE-Lsum. This pattern supports an overall alignment improvement while "
692     "showing that not every grounding or sequence-overlap diagnostic is equally "
693     "stable across the 25 cases."
694 )
695 add("")
696 add("### Corpus-level metrics by dataset")
697 add("")
698 corpus_rows = []
699 for row in corpus_pivot.sort_values(["dataset_id", "metric_name"]).itertuples(index=False):
700     corpus_rows.append(

```

```

701         [
702             row.dataset_id,
703             METRIC_NAMES[row.metric_name],
704             direction(row.metric_name),
705             row.baseline,
706             row.full_system,
707             row.adjusted_delta,
708             outcome(row.adjusted_delta),
709         ]
710     )
711     add(
712         table(
713             [
714                 "Dataset",
715                 "Metric",
716                 "Better direction",
717                 "Baseline",
718                 "Full System",
719                 "Adjusted difference",
720                 "Preferred",
721             ],
722             corpus_rows,
723         )
724     )
725     add("")
726
727     add("## 6. Results by dataset")
728     add("")
729     for dataset_id in [
730         "e2e_nlg",
731         "web_nlg",
732         "dart",
733         "totto",
734         "sportsett_basketball",
735     ]:
736         add(f"### {dataset_id}")
737         add("")
738         rows = []
739         subset = dataset_means[dataset_means["dataset_id"] == dataset_id]
740         for row in subset.sort_values("metric_name").itertuples(index=False):
741             rows.append(
742                 [
743                     METRIC_NAMES.get(row.metric_name, row.metric_name),
744                     direction(row.metric_name),
745                     row.baseline,
746                     row.full_system,
747                     row.adjusted_delta,
748                     row.preferred,
749                 ]
750             )
751         add(
752             table(
753                 [
754                     "Metric",
755                     "Better direction",
756                     "Baseline",
757                     "Full System",
758                     "Adjusted difference",
759                     "Preferred",
760                 ],
761                 rows,
762             )
763         )
764         add("")
765
766     add("## 7. GPT-5.6 Sol structured error results")
767     add("")
768     judge_overall = []
769     for condition, group in judge.groupby("condition"):
770         judge_overall.append(
771             [
772                 condition,
773                 len(group),
774                 int(group["error_count"].sum()),
775                 float(group["error_count"].mean()),
776                 int((group["error_count"] == 0).sum()),
777                 int(group["total_tokens"].fillna(0).sum()),
778             ]
779         )
780     add(
781         table(
782             [
783                 "Condition",
784                 "Outputs",
785                 "Reported errors",
786                 "Mean errors/output",
787                 "Outputs with zero errors",
788                 "Judge tokens",
789             ],
790             judge_overall,
791         )

```

```

792 )
793 add("")
794 judge_dataset_rows = []
795 for (dataset_id, condition), group in judge.groupby(["dataset_id", "condition"]):
796     judge_dataset_rows.append(
797         [
798             dataset_id,
799             condition,
800             len(group),
801             int(group["error_count"].sum()),
802             float(group["error_count"].mean()),
803             int((group["error_count"] == 0).sum()),
804         ]
805     )
806 add(
807     table(
808         [
809             "Dataset",
810             "Condition",
811             "Outputs",
812             "Reported errors",
813             "Mean errors/output",
814             "Zero-error outputs",
815         ],
816         judge_dataset_rows,
817     )
818 )
819 add("")
820 category_pivot = (
821     categories.groupby(["category", "condition"])["count"]
822     .sum()
823     .unstack(fill_value=0)
824     .reset_index()
825 )
826 category_table_rows = []
827 for row in category_pivot.itertuples(index=False):
828     category_table_rows.append(
829         [row.category, getattr(row, "Baseline"), getattr(row, "_2", None)]
830     )
831 # Avoid pandas' tuple-name mangling for the column with a space.
832 category_table_rows = [
833     [
834         category,
835         int(categories[(categories["category"] == category) & (categories["condition"] == "Baseline")]["count"].sum()
836     ),
837         int(categories[(categories["category"] == category) & (categories["condition"] == "Full System")]["count"].sum()
838     ),
839     ]
840     for category in [
841         "NAME",
842         "NUMBER",
843         "WORD",
844         "CONTEXT",
845         "NOT CHECKABLE",
846         "OTHER",
847         "OMISSION",
848         "TASK/FORMAT",
849     ]
850 ]
851 add(table(["Error category", "Baseline", "Full System"], category_table_rows))
852 add("")
853 pair_counts = judge_pairs["comparison"].value_counts().to_dict()
854 add(
855     table(
856         ["Paired outcome", "Examples"],
857         [
858             ["Full System fewer GPT-reported errors", pair_counts.get("Full System fewer", 0)],
859             ["Equal error count", pair_counts.get("Equal", 0)],
860             ["Full System more GPT-reported errors", pair_counts.get("Full System more", 0)],
861         ],
862     )
863 )
864 add("")
865 add(
866     "The taxonomy clarifies the equal totals. Baseline's errors were dominated by "
867     "TASK/FORMAT violations (11), particularly generic responses that ignored "
868     "ToTTo's highlighted-cell restriction or SportSett's multi-paragraph form. "
869     "Full System reduced TASK/FORMAT errors to six but increased CONTEXT errors "
870     "from two to six, all concentrated in the event-report setting. No NAME, NOT "
871     "CHECKABLE, or OTHER errors were reported for either condition."
872 )
873 add("")
874 add(
875     "These annotations should be described as GPT-5.6 Sol judgements. They are "
876     "useful diagnostic evidence but cannot establish human precision or recall "
877     "until checked against independent human annotations. Some annotations are "
878     "strict, such as treating omission of a full birth date as a NUMBER error, so "
879     "category totals should be read with the included explanations."
880 )
881 add("")

```

```

881 add("## 8. Output length and qualitative form")
882 add("")
883 length_table_rows = []
884 for (dataset_id, condition), group in lengths.groupby(["dataset_id", "condition"]):
885     length_table_rows.append(
886         [
887             dataset_id,
888             condition,
889             float(group["words"].mean()),
890             float(group["words"].median()),
891             int(group["words"].min()),
892             int(group["words"].max()),
893         ]
894     )
895 add(
896     table(
897         ["Dataset", "Condition", "Mean words", "Median", "Minimum", "Maximum"],
898         length_table_rows,
899     )
900 )
901 add("")
902 add(
903     "Full System was much longer on SportSett (292.6 versus 159.4 mean words) but "
904     "far shorter on ToTTo (18.0 versus 62.4). This is not a uniform verbosity "
905     "effect. It reflects genre control: expansion for event reports and "
906     "compression for highlighted-cell descriptions. The ToTTo change aligns with "
907     "large metric and judge improvements. The SportSett expansion improves "
908     "coverage and several reference metrics but creates more opportunities for "
909     "contextual interpretation errors."
910 )
911 add("")
912
913 add("## 9. Full System provenance and execution evidence")
914 add("")
915 add(
916     table(
917         ["Recorded property", "Result"],
918         [
919             ["Successful protected executions", int((full_summary["execution_outcome"] == "success").sum())],
920             ["Primary-evaluation eligible", int(full_summary["primary_evaluation_eligible"].sum())],
921             ["Approved", int((full_summary["release_status"] == "approved").sum())],
922             [
923                 "Approved with warnings",
924                 int((full_summary["release_status"] == "approved_with_warnings").sum()),
925             ],
926             ["Mean native support rate", float(full_summary["native_support_rate"].mean())],
927             ["Unsupported factual sentences", int(full_summary["unsupported_factual_sentence_count"].sum())],
928             ["Normal LLM Writer path", int((full_summary["final_generation_path"] == "normal_llm_writer").sum())],
929             ["Auditor-repaired path", int((full_summary["final_generation_path"] == "auditor_repaired").sum())],
930             ["Deterministic fallback path", int((full_summary["final_generation_path"] == "deterministic_fallback").
931 sum())],
932             ["Evidence items", int(full_summary["evidence_item_count"].sum())],
933             ["Verified facts", int(full_summary["verified_fact_count"].sum())],
934             ["Rejected facts", int(full_summary["rejected_fact_count"].sum())],
935             ["Verified insights", int(full_summary["verified_insight_count"].sum())],
936             ["Rejected insights", int(full_summary["rejected_insight_count"].sum())],
937         ],
938     )
939 )
940 add("")
941 add(
942     "All 25 outputs passed the internal audit with complete mapped support, and no "
943     "invalid fact or evidence identifiers were recorded. This supports a claim of "
944     "successful provenance enforcement. It does not support a claim of zero "
945     "factual error, because the independent judge still found contextual and "
946     "task-level issues."
947 )
948 add("### Evidence, facts, and insights by dataset")
949 add("")
950 workflow_rows = []
951 for dataset_id, group in full_summary.groupby("dataset_id"):
952     workflow_rows.append(
953         [
954             dataset_id,
955             int(group["evidence_item_count"].sum()),
956             int(group["fact_candidate_count"].sum()),
957             int(group["verified_fact_count"].sum()),
958             int(group["rejected_fact_count"].sum()),
959             int(group["insight_candidate_count"].sum()),
960             int(group["verified_insight_count"].sum()),
961             int(group["rejected_insight_count"].sum()),
962         ]
963     )
964 add(
965     table(
966         [
967             "Dataset",
968             "Evidence items",
969             "Fact candidates",
970             "Verified facts",

```

```

971         "Rejected facts",
972         "Insight candidates",
973         "Verified insights",
974         "Rejected insights",
975     ],
976     workflow_rows,
977 )
978 )
979 add("")
980 add("### Operational footprint (secondary evidence)")
981 add("")
982 baseline_tokens = sum(int(row.get("total_tokens") or 0) for row in baseline_records)
983 baseline_seconds = sum(float(row.get("elapsed_seconds") or 0) for row in baseline_records)
984 add(
985     table(
986         ["Condition", "Provider-reported requests", "Provider-reported tokens", "Elapsed seconds"],
987         [
988             [
989                 "Full System",
990                 int(stage_usage["requests"].sum()),
991                 int(stage_usage["total_tokens"].sum()),
992                 float(full_summary["elapsed_seconds"].sum()),
993             ],
994             ["Baseline", 25, baseline_tokens, baseline_seconds],
995         ],
996     )
997 )
998 add("")
999 add(
1000     "Runtime and token use are retained for reproducibility, not treated as the "
1001     "main evaluation outcome. The conditions differ by orders of magnitude in "
1002     "computation, particularly on SportSett. This makes it important to present "
1003     "quality gains alongside the operational trade-off, without reducing the "
1004     "dissertation to a cost study."
1005 )
1006 add("")
1007 stage_rows = []
1008 for stage, group in stage_usage.groupby("stage"):
1009     stage_rows.append(
1010         [
1011             stage,
1012             int(group["requests"].sum()),
1013             int(group["input_tokens"].sum()),
1014             int(group["output_tokens"].sum()),
1015             int(group["total_tokens"].sum()),
1016         ]
1017     )
1018 stage_rows.sort(key=lambda row: row[-1], reverse=True)
1019 add(table(["Stage", "Requests", "Input tokens", "Output tokens", "Total tokens"], stage_rows))
1020 add("")
1021
1022 add("## 10. Evidence-use notes")
1023 add("")
1024 for item in [
1025     "Positive adjusted differences favour Full System; TER and unsupported-sentence rate are reversed because lower is better.",
1026     "The protected sample contains five examples per dataset and one generation per condition.",
1027     "The treatment is the complete task-aware workflow; the comparison does not isolate architecture from prompt specificity.",
1028     "GPT-5.6 Sol annotations are model judgements, not human gold labels.",
1029     "Native support rate is Full System-only provenance evidence, not a paired quality metric.",
1030     "PARENT was unavailable and must not be presented as a scored result.",
1031 ]:
1032     add(f"- {item}")
1033 add("")
1034
1035 add("## Appendix A. Complete case-level evidence")
1036 add("")
1037 add(
1038     "Each case below contains the normalized source presented to both conditions, "
1039     "the task-specific Full System request, the generic Baseline request, all human "
1040     "references (unsealed only after generation), both outputs, per-output metrics, "
1041     "GPT-5.6 Sol annotations, and Full System provenance statistics."
1042 )
1043 add("")
1044
1045 for case_number, key in enumerate(selection_order, start=1):
1046     dataset_id, example_id = key
1047     full = metric_input_by_key[(dataset_id, example_id, "full_system")]
1048     baseline = metric_input_by_key[(dataset_id, example_id, "baseline")]
1049     sealed_full = sealed_by_key[(dataset_id, example_id, "full_system")]
1050     sealed_baseline = sealed_by_key[(dataset_id, example_id, "baseline")]
1051     full_annotation = annotation_by_key.get((dataset_id, example_id, "full_system"))
1052     baseline_annotation = annotation_by_key.get((dataset_id, example_id, "baseline"))
1053     run = summary_by_key[key]
1054     selected = selection_by_key[key]
1055
1056     add(f"## A{case_number}. {dataset_id} / {example_id}")
1057     add("")
1058     add(
1059         table(

```



```

1060         ["Field", "Value"],
1061         [
1062             ["Dataset", dataset_id],
1063             ["Example ID", example_id],
1064             ["Task family", full["task_family"]],
1065             ["Output mode", full["output_mode"]],
1066             ["Language", full["language"]],
1067             ["Source SHA-256", selected["source_sha256"]],
1068             ["Reference SHA-256", selected["reference_sha256"]],
1069             ["Full System request", full["request"]],
1070             ["Baseline request", sealed_baseline["request"]],
1071         ],
1072     )
1073 )
1074 add("")
1075 add("### Normalized source supplied during generation")
1076 add("")
1077 source_language = "json" if full["source_text"].rstrip().startswith("{") else "text"
1078 add(code_block(full["source_text"], source_language))
1079 add("")
1080 add("### Human reference outputs")
1081 add("")
1082 for reference_number, reference in enumerate(full["references"], start=1):
1083     add(f"***Reference {reference_number}***")
1084     add("")
1085     add(code_block(reference, "text"))
1086     add("")
1087 add("### Full System output")
1088 add("")
1089 add(code_block(full["generated_text"], "markdown"))
1090 add("")
1091 add("### Baseline output")
1092 add("")
1093 add(code_block(baseline["generated_text"], "markdown"))
1094 add("")
1095
1096 case_metric_rows = []
1097 case_metrics = paired[
1098     (paired["dataset_id"] == dataset_id)
1099     & (paired["example_id"].astype(str) == example_id)
1100 ]
1101 for row in case_metrics.sort_values("metric_name").itertuples(index=False):
1102     case_metric_rows.append(
1103         [
1104             METRIC_NAMES.get(row.metric_name, row.metric_name),
1105             direction(row.metric_name),
1106             row.baseline,
1107             row.full_system,
1108             row.adjusted_delta,
1109             outcome(row.adjusted_delta),
1110         ]
1111     )
1112 add("### Automatic metrics")
1113 add("")
1114 add(
1115     table(
1116         [
1117             "Metric",
1118             "Better direction",
1119             "Baseline",
1120             "Full System",
1121             "Adjusted difference",
1122             "Preferred",
1123         ],
1124         case_metric_rows,
1125     )
1126 )
1127 add("")
1128 add("### GPT-5.6 Sol structured annotations")
1129 add("")
1130 add("***Full System***")
1131 add("")
1132 add(annotation_text(full_annotation))
1133 add("")
1134 add("***Baseline***")
1135 add("")
1136 add(annotation_text(baseline_annotation))
1137 add("")
1138 add("### Full System provenance and execution record")
1139 add("")
1140 add(
1141     table(
1142         ["Property", "Value"],
1143         [
1144             ["Run ID", run.run_id],
1145             ["Execution outcome", run.execution_outcome],
1146             ["Final generation path", run.final_generation_path],
1147             ["Final Writer mode", run.final_writer_mode],
1148             ["Release status", run.release_status],
1149             ["Audit decision", run.audit_decision],
1150             ["Repair rounds", run.repair_rounds_used],

```

```

1151         ["Native support rate", run.native_support_rate],
1152         ["Factual sentences", run.factual_sentence_count],
1153         ["Supported sentences", run.supported_sentence_count],
1154         ["Evidence items", run.evidence_item_count],
1155         ["Verified facts", run.verified_fact_count],
1156         ["Rejected facts", run.rejected_fact_count],
1157         ["Verified insights", run.verified_insight_count],
1158         ["Rejected insights", run.rejected_insight_count],
1159         ["Full System words", word_count(full["generated_text"])],
1160         ["Baseline words", word_count(baseline["generated_text"])],
1161         ["Full System elapsed seconds", sealed_full.get("elapsed_seconds")],
1162         ["Baseline elapsed seconds", sealed_baseline.get("elapsed_seconds")],
1163         ["Full System provider-reported tokens", run.provider_reported_total_tokens],
1164         ["Baseline provider-reported tokens", sealed_baseline.get("total_tokens")],
1165         ["Pipeline result", run.pipeline_result_path],
1166     ],
1167 )
1168 )
1169 add("")
1170
1171 add("# Appendix B. Complete GPT-5.6 Sol non-zero annotations")
1172 add("")
1173 add(
1174     "This appendix repeats every non-zero annotation in one place for taxonomy "
1175     "analysis. Zero-error records remain visible in each case section."
1176 )
1177 add("")
1178 for row in sorted(
1179     (item for item in annotations if item["error_count"] > 0),
1180     key=lambda item: (
1181         item["dataset_id"],
1182         str(item["example_id"]),
1183         item["variant_id"],
1184     ),
1185 ):
1186     add(
1187         f"## {row['dataset_id']} / {row['example_id']} / "
1188         f"{CONDITION_LABELS[row['variant_id']]}"
1189     )
1190     add("")
1191     add(annotation_text(row))
1192     add("")
1193
1194 add("# Appendix C. Reproducibility inventory")
1195 add("")
1196 inventory_rows = []
1197 for label, path in [
1198     ("Protected selection manifest", SELECTION_PATH),
1199     ("Full System batch manifest", FULL_MANIFEST_PATH),
1200     ("Full System configuration", FULL_CONFIG_PATH),
1201     ("Full System generation summary", FULL_SUMMARY_PATH),
1202     ("Full System stage usage", STAGE_USAGE_PATH),
1203     ("Sealed paired outputs", SEALED_PAIRED_PATH),
1204     ("Metrics input with references", METRIC_INPUT_PATH),
1205     ("Baseline sealed generations", BASELINE_GENERATIONS_PATH),
1206     ("Reference metrics", REFERENCE_METRICS_PATH),
1207     ("Source-grounded metrics", SOURCE_METRICS_PATH),
1208     ("GPT-5.6 Sol annotations", JUDGE_ANNOTATIONS_PATH),
1209 ]:
1210     inventory_rows.append([label, rel(path), path.stat().st_size])
1211 add(table(["Artifact", "Path", "Bytes"], inventory_rows))
1212 add("")
1213 add(
1214     "End of evidence bank. Numerical claims in a dissertation should be copied "
1215     "from the direction-aware tables in this document or recomputed from the "
1216     "linked JSONL artifacts."
1217 )
1218
1219 OUTPUT_PATH.write_text("\n".join(lines) + "\n", encoding="utf-8")
1220 print(f"Wrote {OUTPUT_PATH}")
1221 print(f"Lines: {len(lines):,}")
1222 print(f"Bytes: {OUTPUT_PATH.stat().st_size:,}")
1223
1224
1225 if __name__ == "__main__":
1226     main()

```

Listing 15: P15: table2text_pydanticai/evaluation/scripts/figures/build_evaluation_design_flow.py

```

1  """Build the dissertation evaluation-design flow diagram."""
2
3  from __future__ import annotations
4
5  import os
6  from pathlib import Path
7
8
9  PROJECT_DIR = Path(__file__).resolve().parents[3]
10 FIGURE_DIR = PROJECT_DIR / "evaluation/results/figures"

```

```

11
12
13 def main() -> None:
14     os.environ.setdefault(
15         "MPLCONFIGDIR",
16         str(PROJECT_DIR / ".matplotlib"),
17     )
18
19     import matplotlib.pyplot as plt
20     from matplotlib.patches import FancyArrowPatch, FancyBboxPatch, Rectangle
21
22     FIGURE_DIR.mkdir(parents=True, exist_ok=True)
23
24     colors = {
25         "ink": "#172033",
26         "muted": "#556070",
27         "line": "#768195",
28         "band": "#F7F8FA",
29         "source": "#EAF4FF",
30         "source_edge": "#2F6FDB",
31         "reference": "#ECFEFF",
32         "reference_edge": "#0891B2",
33         "workflow": "#E2F3EF",
34         "workflow_edge": "#009E73",
35         "raw": "#FFF2E8",
36         "raw_edge": "#D55E00",
37         "outputs": "#F1EDFF",
38         "outputs_edge": "#7C3AED",
39         "grounded": "#ECFDF5",
40         "grounded_edge": "#009E73",
41         "judge": "#FDF2F8",
42         "judge_edge": "#CC79A7",
43         "audit": "#FFFFFF",
44         "audit_edge": "#374151",
45     }
46
47     fig, ax = plt.subplots(figsize=(15.8, 8.8))
48     ax.set_xlim(0, 1)
49     ax.set_ylim(0, 1)
50     ax.axis("off")
51     fig.patch.set_facecolor("white")
52
53     def stage_label(x: float, text: str) -> None:
54         ax.text(
55             x,
56             0.855,
57             text,
58             ha="center",
59             va="center",
60             fontsize=10.5,
61             fontweight="bold",
62             color=colors["muted"],
63         )
64
65     def box(
66         xy: tuple[float, float],
67         width: float,
68         height: float,
69         text: str,
70         *,
71         face: str,
72         edge: str,
73         size: int = 11,
74         weight: str = "normal",
75         radius: float = 0.015,
76     ) -> tuple[float, float, float, float]:
77         patch = FancyBboxPatch(
78             xy,
79             width,
80             height,
81             boxstyle=f"round,pad=0.012,rounding_size={radius}",
82             linewidth=1.8,
83             edgecolor=edge,
84             facecolor=face,
85         )
86         ax.add_patch(patch)
87         ax.text(
88             xy[0] + width / 2,
89             xy[1] + height / 2,
90             text,
91             ha="center",
92             va="center",
93             fontsize=size,
94             fontweight=weight,
95             color=colors["ink"],
96             linespacing=1.35,
97         )
98         return (xy[0], xy[1], width, height)
99
100     def right(rect: tuple[float, float, float, float], y_frac: float = 0.5) -> tuple[float, float]:
101         return (rect[0] + rect[2], rect[1] + rect[3] * y_frac)

```

```

102
103 def left(rect: tuple[float, float, float, float], y_frac: float = 0.5) -> tuple[float, float]:
104     return (rect[0], rect[1] + rect[3] * y_frac)
105
106 def top(rect: tuple[float, float, float, float], x_frac: float = 0.5) -> tuple[float, float]:
107     return (rect[0] + rect[2] * x_frac, rect[1] + rect[3])
108
109 def bottom(rect: tuple[float, float, float, float], x_frac: float = 0.5) -> tuple[float, float]:
110     return (rect[0] + rect[2] * x_frac, rect[1])
111
112 def arrow(
113     start: tuple[float, float],
114     end: tuple[float, float],
115     *,
116     connectionstyle: str = "arc3,rad=0",
117     lw: float = 1.6,
118 ) -> None:
119     patch = FancyArrowPatch(
120         start,
121         end,
122         arrowstyle="->",
123         mutation_scale=14,
124         linewidth=lw,
125         color=colors["line"],
126         connectionstyle=connectionstyle,
127         shrinkA=6,
128         shrinkB=6,
129     )
130     ax.add_patch(patch)
131
132 def metric_tile(
133     xy: tuple[float, float],
134     width: float,
135     height: float,
136     title: str,
137     items: str,
138     *,
139     face: str,
140     edge: str,
141 ) -> tuple[float, float, float, float]:
142     rect = box(xy, width, height, "", face=face, edge=edge)
143     ax.text(
144         xy[0] + width / 2,
145         xy[1] + height * 0.73,
146         title,
147         ha="center",
148         va="center",
149         fontsize=8.8,
150         fontweight="bold",
151         color=colors["ink"],
152         linespacing=1.1,
153     )
154     ax.text(
155         xy[0] + width / 2,
156         xy[1] + height * 0.30,
157         items,
158         ha="center",
159         va="center",
160         fontsize=8.1,
161         color=colors["muted"],
162         linespacing=1.18,
163     )
164     return rect
165
166 ax.text(
167     0.5,
168     0.975,
169     "Evaluation Design: Workflow vs Raw Generic Baseline",
170     ha="center",
171     va="top",
172     fontsize=18.5,
173     fontweight="bold",
174     color=colors["ink"],
175 )
176
177 ax.text(
178     0.5,
179     0.925,
180     "Human references are isolated from generation, then reused only for evaluation.",
181     ha="center",
182     va="top",
183     fontsize=10.5,
184     color=colors["muted"],
185 )
186
187 for x0, width in [(0.04, 0.21), (0.29, 0.32), (0.64, 0.15), (0.82, 0.16)]:
188     ax.add_patch(
189         Rectangle(
190             (x0, 0.12),
191             width,
192             0.71,

```

```

193         facecolor=colors["band"],
194         edgecolor="#E5E7EB",
195         linewidth=1.0,
196         zorder=0,
197     )
198 )
199
200 stage_label(0.145, "Dataset setup")
201 stage_label(0.45, "Generation arms")
202 stage_label(0.72, "Pairing")
203 stage_label(0.895, "Evaluation")
204
205 source = box(
206     (0.07, 0.615),
207     0.15,
208     0.13,
209     "25 source\nexamples\n5 datasets x 5 cases",
210     face=colors["source"],
211     edge=colors["source_edge"],
212     size=11.5,
213     weight="bold",
214 )
215 box(
216     (0.07, 0.365),
217     0.15,
218     0.12,
219     "Reference\nisolation",
220     face=colors["reference"],
221     edge=colors["reference_edge"],
222     size=11.5,
223     weight="bold",
224 )
225 ax.text(
226     0.145,
227     0.315,
228     "Reference text is withheld from\nboth generation prompts.",
229     ha="center",
230     va="top",
231     fontsize=8.8,
232     color=colors["muted"],
233     linespacing=1.25,
234 )
235 arrow((0.145, 0.615), (0.145, 0.485))
236
237 workflow = box(
238     (0.31, 0.60),
239     0.26,
240     0.15,
241     "Full multi-agent system\nDeepSeek V4 Flash\nsix-role workflow",
242     face=colors["workflow"],
243     edge=colors["workflow_edge"],
244     size=11,
245     weight="bold",
246 )
247 audit = box(
248     (0.34, 0.465),
249     0.20,
250     0.08,
251     "Internal provenance,\nfact support and audit",
252     face=colors["audit"],
253     edge=colors["audit_edge"],
254     size=8.8,
255     weight="bold",
256     radius=0.012,
257 )
258 raw = box(
259     (0.31, 0.255),
260     0.26,
261     0.15,
262     "Raw generic baseline\nDeepSeek V4 Flash\nsingle LLM call",
263     face=colors["raw"],
264     edge=colors["raw_edge"],
265     size=11,
266     weight="bold",
267 )
268 arrow(right(source, 0.7), left(workflow, 0.55))
269 arrow(right(source, 0.3), left(raw, 0.55))
270 arrow((0.44, 0.60), top(audit, 0.5), connectionstyle="arc3,rad=0")
271
272 paired = box(
273     (0.655, 0.44),
274     0.12,
275     0.14,
276     "50 paired\noutputs",
277     face=colors["outputs"],
278     edge=colors["outputs_edge"],
279     size=12,
280     weight="bold",
281 )
282 arrow(right(workflow), left(paired, 0.72))
283 arrow(right(raw), left(paired, 0.28))

```

```

284     ref_eval = metric_tile(
285         (0.835, 0.64),
286         0.13,
287         0.13,
288         "Reference\nalignment",
289         "BERTScore\nchrF\nMETEOR",
290         face=colors["reference"],
291         edge=colors["reference_edge"],
292     )
293
294     source_eval = metric_tile(
295         (0.835, 0.455),
296         0.13,
297         0.13,
298         "Source-grounded\ndiagnostics",
299         "AlignScore\nHHEM",
300         face=colors["grounded"],
301         edge=colors["grounded_edge"],
302     )
303
304     judge_eval = metric_tile(
305         (0.835, 0.27),
306         0.13,
307         0.13,
308         "Independent\njudgement",
309         "GPT-5.6 Sol\nHuman study",
310         face=colors["judge"],
311         edge=colors["judge_edge"],
312     )
313
314     arrow(right(paired, 0.75), left(ref_eval))
315     arrow(right(paired, 0.50), left(source_eval))
316     arrow(right(paired, 0.25), left(judge_eval))
317
318     synthesis = box(
319         (0.835, 0.13),
320         0.13,
321         0.085,
322         "Synthesis\nsimilarity + grounding\n+ judgement",
323         face="#FFFFFF",
324         edge=colors["line"],
325         size=8.4,
326         weight="bold",
327     )
328
329     bus_x = 0.975
330     metric_centers = [right(ref_eval), right(source_eval), right(judge_eval)]
331     for x, y in metric_centers:
332         ax.plot([x, bus_x], [y, y], color=colors["line"], lw=1.2)
333     ax.plot(
334         [bus_x, bus_x],
335         [right(ref_eval)[1], top(synthesis)[1] + 0.015],
336         color=colors["line"],
337         lw=1.2,
338     )
339     arrow((bus_x, top(synthesis)[1] + 0.015), right(synthesis, 0.72), lw=1.2)
340
341     for suffix in ["png", "svg"]:
342         output_path = FIGURE_DIR / f"00_evaluation_design_flow.{suffix}"
343         fig.savefig(output_path, dpi=240, bbox_inches="tight")
344         print(f"Saved {output_path}")
345
346 if __name__ == "__main__":
347     main()

```

Listing 16: P16: table2text_pydanticai/evaluation/scripts/figures/build_gpt56_sol_annotation_figure.py

```

1  """Build a dissertation-ready GPT-5.6 Sol annotation figure."""
2
3  from __future__ import annotations
4
5  import json
6  import os
7  from collections import Counter, defaultdict
8  from pathlib import Path
9
10
11 PROJECT_DIR = Path(__file__).resolve().parents[3]
12 RESULTS_DIR = PROJECT_DIR / "evaluation/results"
13 FIGURE_DIR = RESULTS_DIR / "figures"
14 ANNOTATIONS_PATH = RESULTS_DIR / "openai_structured_error_annotations.jsonl"
15
16
17 VARIANT_LABELS = {
18     "full_system": "Full System",
19     "raw_generic_flash": "Raw Generic Flash",
20 }
21
22 VARIANT_COLORS = {
23     "full_system": "#0072B2",
24     "raw_generic_flash": "#D55E00",

```

```

25 }
26
27 CATEGORY_COLORS = {
28     "TASK/FORMAT": "#CC79A7",
29     "CONTEXT": "#E69F00",
30     "OMISSION": "#009E73",
31     "NUMBER": "#D55E00",
32     "NAME": "#56B4E9",
33     "WORD": "#7C3AED",
34     "NOT_CHECKABLE": "#6B7280",
35     "OTHER": "#374151",
36 }
37
38
39 def read_jsonl(path: Path) -> list[dict]:
40     rows: list[dict] = []
41     with path.open("r", encoding="utf-8") as handle:
42         for line in handle:
43             line = line.strip()
44             if line:
45                 rows.append(json.loads(line))
46     return rows
47
48
49 def category_counts(rows: list[dict]) -> dict[str, Counter]:
50     counts: dict[str, Counter] = defaultdict(Counter)
51     for row in rows:
52         variant_id = row.get("variant_id")
53         if variant_id not in VARIANT_LABELS:
54             continue
55         for error in row.get("errors") or []:
56             category = (error.get("category") or "OTHER").upper()
57             counts[variant_id][category] += 1
58     return counts
59
60
61 def main() -> None:
62     os.environ.setdefault("MPLCONFIGDIR", str(PROJECT_DIR / ".matplotlib"))
63
64     import matplotlib.pyplot as plt
65     from matplotlib.patches import FancyBboxPatch
66
67     FIGURE_DIR.mkdir(parents=True, exist_ok=True)
68
69     rows = [
70         row
71         for row in read_jsonl(ANNOTATIONS_PATH)
72         if row.get("status") == "scored"
73         and row.get("judge_model") == "gpt-5.6-sol"
74         and row.get("variant_id") in VARIANT_LABELS
75     ]
76     if not rows:
77         raise RuntimeError(f"No scored GPT-5.6 Sol annotations found in {ANNOTATIONS_PATH}")
78
79     by_variant: dict[str, list[dict]] = {
80         variant: [row for row in rows if row.get("variant_id") == variant]
81         for variant in VARIANT_LABELS
82     }
83     totals = {
84         variant: sum(int(row.get("error_count") or 0) for row in variant_rows)
85         for variant, variant_rows in by_variant.items()
86     }
87     outputs = {variant: len(variant_rows) for variant, variant_rows in by_variant.items()}
88     outputs_with_errors = {
89         variant: sum(1 for row in variant_rows if int(row.get("error_count") or 0) > 0)
90         for variant, variant_rows in by_variant.items()
91     }
92     means = {
93         variant: (totals[variant] / outputs[variant]) if outputs[variant] else 0
94         for variant in VARIANT_LABELS
95     }
96     category_by_variant = category_counts(rows)
97     category_order = [
98         category
99         for category in CATEGORY_COLORS
100         if any(category_by_variant[variant][category] for variant in VARIANT_LABELS)
101     ]
102
103     raw_total = totals.get("raw_generic_flash", 0)
104     full_total = totals.get("full_system", 0)
105     reduction = ((raw_total - full_total) / raw_total * 100) if raw_total else 0.0
106
107     plt.rcParams.update(
108         {
109             "figure.facecolor": "white",
110             "axes.facecolor": "white",
111             "axes.edgecolor": "#D1D5DB",
112             "axes.labelcolor": "#172033",
113             "xtick.color": "#172033",
114             "ytick.color": "#172033",
115             "font.size": 10,

```



```

116         "axes.titlesize": 13,
117         "axes.titleweight": "bold",
118         "axes.grid": True,
119         "grid.color": "#EEF2F7",
120         "grid.linewidth": 0.8,
121         "legend.frameon": False,
122     }
123 )
124
125 fig = plt.figure(figsize=(14.8, 8.4))
126 gs = fig.add_gridspec(
127     2,
128     2,
129     width_ratios=[0.95, 1.25],
130     height_ratios=[0.35, 0.65],
131     hspace=0.30,
132     wspace=0.22,
133 )
134
135 ax_title = fig.add_subplot(gs[0, :])
136 ax_title.axis("off")
137 ax_title.text(
138     0.0,
139     1.00,
140     "GPT-5.6 Sol Structured Error Annotations",
141     ha="left",
142     va="top",
143     fontsize=21,
144     fontweight="bold",
145     color="#172033",
146 )
147 ax_title.text(
148     0.0,
149     0.66,
150     (
151         "Cross-family judge: one output at a time, source data and task metadata only; "
152         "no references, competing outputs, automatic scores, or system identity."
153     ),
154     ha="left",
155     va="top",
156     fontsize=10.7,
157     color="#556070",
158 )
159
160 def card(x: float, title: str, value: str, note: str, color: str) -> None:
161     rect = FancyBboxPatch(
162         (x, 0.06),
163         0.225,
164         0.38,
165         boxstyle="round,pad=0.014,rounding_size=0.025",
166         linewidth=1.4,
167         edgecolor=color,
168         facecolor="#FFFFFF",
169         transform=ax_title.transAxes,
170     )
171     ax_title.add_patch(rect)
172     ax_title.text(
173         x + 0.018,
174         0.36,
175         title,
176         transform=ax_title.transAxes,
177         fontsize=9.5,
178         fontweight="bold",
179         color="#556070",
180         va="top",
181     )
182     ax_title.text(
183         x + 0.018,
184         0.235,
185         value,
186         transform=ax_title.transAxes,
187         fontsize=22,
188         fontweight="bold",
189         color=color,
190         va="top",
191     )
192     ax_title.text(
193         x + 0.018,
194         0.10,
195         note,
196         transform=ax_title.transAxes,
197         fontsize=8.9,
198         color="#556070",
199         va="bottom",
200     )
201
202 card(
203     0.00,
204     "Scored outputs",
205     str(sum(outputs.values())),
206     f"{outputs['full_system']} workflow + {outputs['raw_generic_flash']} raw",

```

```

207     "#374151",
208 )
209 card(
210     0.255,
211     "Total errors",
212     f"{full_total} vs {raw_total}",
213     "workflow vs raw generic",
214     "#7C3AED",
215 )
216 card(
217     0.51,
218     "Error reduction",
219     f"{reduction:.0f}%",
220     "relative to raw generic",
221     "#009E73",
222 )
223 card(
224     0.765,
225     "Workflow gap",
226     "1 missing",
227     "SportSett 4975 annotation absent",
228     "#D55E00",
229 )
230
231 axBars = fig.add_subplot(gs[1, 0])
232 variants = list(VARIANT_LABELS)
233 x = range(len(variants))
234 barWidth = 0.34
235 axBars.bar(
236     [i - barWidth / 2 for i in x],
237     [totals[variant] for variant in variants],
238     width=barWidth,
239     color=[VARIANT_COLORS[variant] for variant in variants],
240     label="Total errors",
241 )
242 axBars.bar(
243     [i + barWidth / 2 for i in x],
244     [outputs_with_errors[variant] for variant in variants],
245     width=barWidth,
246     color=["#9CC9E8", "#F4A582"],
247     label="Outputs with errors",
248 )
249 axBars.set_title("Judge-Detected Errors by Variant")
250 axBars.set_ylabel("Count")
251 axBars.set_xticks(list(x))
252 axBars.set_xticklabels([VARIANT_LABELS[variant] for variant in variants])
253 axBars.set_ylim(0, max(raw_total, full_total) + 4)
254 axBars.legend(loc="upper left")
255 axBars.spines[["top", "right"]].set_visible(False)
256 for i, variant in enumerate(variants):
257     axBars.text(
258         i - barWidth / 2,
259         totals[variant] + 0.35,
260         str(totals[variant]),
261         ha="center",
262         va="bottom",
263         fontsize=10,
264         fontweight="bold",
265         color=VARIANT_COLORS[variant],
266     )
267     axBars.text(
268         i + barWidth / 2,
269         outputs_with_errors[variant] + 0.35,
270         str(outputs_with_errors[variant]),
271         ha="center",
272         va="bottom",
273         fontsize=10,
274         fontweight="bold",
275         color="#556070",
276     )
277     axBars.text(
278         i,
279         -2.15,
280         f"mean {means[variant]:.3f}/output",
281         ha="center",
282         va="top",
283         fontsize=9,
284         color="#556070",
285         clip_on=False,
286     )
287
288 axStack = fig.add_subplot(gs[1, 1])
289 yPositions = [1, 0]
290 lefts = [0, 0]
291 for category in category_order:
292     values = [category_by_variant[variant][category] for variant in variants]
293     axStack.barh(
294         yPositions,
295         values,
296         left=lefts,
297         height=0.42,

```

```

298         color=CATEGORY_COLORS[category],
299         label=category,
300     )
301     for idx, value in enumerate(values):
302         if value:
303             ax_stack.text(
304                 lefts[idx] + value / 2,
305                 y_positions[idx],
306                 str(value),
307                 ha="center",
308                 va="center",
309                 fontsize=9.5,
310                 fontweight="bold",
311                 color="white" if category != "OMISSION" else "#172033",
312             )
313     lefts = [lefts[idx] + values[idx] for idx in range(len(values))]
314
315     ax_stack.set_title("Error Category Profile")
316     ax_stack.set_xlabel("Structured error count")
317     ax_stack.set_yticks(y_positions)
318     ax_stack.set_yticklabels([VARIANT_LABELS[variant] for variant in variants])
319     ax_stack.set_xlim(0, max(raw_total, full_total) + 2)
320     ax_stack.spines[["top", "right"]].set_visible(False)
321     ax_stack.legend(
322         loc="lower center",
323         bbox_to_anchor=(0.5, -0.28),
324         ncol=4,
325         fontsize=9,
326     )
327     ax_stack.text(
328         0,
329         -0.68,
330         (
331             "Reading: the workflow removed GPT-5.6-detected NUMBER and OMISSION errors in this "
332             "annotated sample; its remaining errors are concentrated in SportSett temporal context "
333             "and output-format realization."
334         ),
335         ha="left",
336         va="top",
337         fontsize=9.2,
338         color="#556070",
339         wrap=True,
340         transform=ax_stack.transData,
341     )
342
343     for suffix in ("png", "svg"):
344         output_path = FIGURE_DIR / f"11_gpt56_sol_structured_errors.{suffix}"
345         fig.savefig(output_path, dpi=240, bbox_inches="tight")
346         print(f"Saved {output_path}")
347
348
349 if __name__ == "__main__":
350     main()

```

Listing 17: P17: table2text_pydanticai/evaluation/scripts/reports/build_task_aware_complete_evaluation_dossier.py

```

1  """Build the complete three-condition task-aware evaluation dossier.
2
3  The generated Markdown is intentionally self-contained: it includes every
4  selected input, held-out reference, generated output, metric score and
5  structured error annotation from the completed 25-example experiment.
6  """
7
8  from __future__ import annotations
9
10 import csv
11 import hashlib
12 import json
13 import re
14 from collections import Counter, defaultdict
15 from datetime import datetime, timezone
16 from pathlib import Path
17 from typing import Any, Iterable
18
19 from table2text.evaluation.models import GenerationRecord
20
21
22 PROJECT_DIR = Path(__file__).resolve().parents[3]
23 EVALUATION_DIR = PROJECT_DIR / "evaluation"
24 EXPERIMENT_DIR = EVALUATION_DIR / "task_aware_direct_baseline"
25 RESULT_DIR = EXPERIMENT_DIR / "results"
26
27 GENERATIONS_PATH = (
28     EXPERIMENT_DIR
29     / "generations"
30     / "task_aware_direct_flash_25_three_condition_generations.jsonl"
31 )
32 PREPARED_EXAMPLES_PATH = (
33     EXPERIMENT_DIR

```

```

34     / "prepared"
35     / "task_aware_direct_flash_25_examples.jsonl"
36 )
37 REFERENCE_METRICS_PATH = RESULT_DIR / "task_aware_direct_flash_25_reference_metrics.jsonl"
38 SOURCE_METRICS_PATH = RESULT_DIR / "task_aware_direct_flash_25_source_grounded_metrics.jsonl"
39 ANNOTATIONS_PATH = RESULT_DIR / "gpt56_all_75_annotations_with_provenance.jsonl"
40 REFERENCE_MACRO_PATH = RESULT_DIR / "task_aware_direct_flash_25_reference_macro.csv"
41 SOURCE_MACRO_PATH = RESULT_DIR / "task_aware_direct_flash_25_source_macro.csv"
42 REFERENCE_BY_DATASET_PATH = RESULT_DIR / "task_aware_direct_flash_25_reference_by_dataset.csv"
43 SOURCE_BY_DATASET_PATH = RESULT_DIR / "task_aware_direct_flash_25_source_by_dataset.csv"
44 WINS_PATH = RESULT_DIR / "task_aware_direct_flash_25_direction_adjusted_wins.csv"
45 SELECTED_FIVE_PATH = RESULT_DIR / "selected_five_three_condition_source_metrics.csv"
46 EXPERIMENT_MANIFEST_PATH = RESULT_DIR / "task_aware_direct_flash_25_manifest.json"
47 ANNOTATION_PROVENANCE_PATH = RESULT_DIR / "interactive_gpt56_annotation_provenance.json"
48 PROGRESS_LOG_PATH = RESULT_DIR / "task_aware_direct_flash_25_progress.log"
49 VARIANT_CONFIG_PATH = (
50     EXPERIMENT_DIR / "config" / "variants_task_aware_direct_flash_25.json"
51 )
52 REFERENCE_CONFIG_PATH = (
53     EXPERIMENT_DIR / "config" / "metrics_task_aware_direct_flash_25_reference.json"
54 )
55 SOURCE_CONFIG_PATH = (
56     EXPERIMENT_DIR / "config" / "metrics_task_aware_direct_flash_25_source_grounded.json"
57 )
58 NOTEBOOK_PATH = EVALUATION_DIR / "notebooks" / "task_aware_direct_baseline_evaluation.ipynb"
59 NOTEBOOK_SOURCE_PATH = EVALUATION_DIR / "notebooks" / "task_aware_direct_baseline_evaluation.py"
60 ANNOTATION_BUILDER_PATH = (
61     EVALUATION_DIR
62     / "scripts"
63     / "annotations"
64     / "build_interactive_gpt56_annotation_artifacts.py"
65 )
66 DOSSIER_BUILDER_PATH = Path(__file__).resolve()
67
68 OUTPUT_PATH = EXPERIMENT_DIR / "COMPLETE_THREE_CONDITION_EVALUATION_DOSSIER.md"
69 OUTPUT_MANIFEST_PATH = RESULT_DIR / "complete_evaluation_dossier_manifest.json"
70
71 VARIANT_ORDER = ["full_system", "raw_generic_flash", "task_aware_direct_flash"]
72 VARIANT_LABELS = {
73     "full_system": "Full multi-agent system",
74     "raw_generic_flash": "Raw-generic direct Flash",
75     "task_aware_direct_flash": "Task-aware direct Flash",
76 }
77 DATASET_ORDER = ["dart", "e2e_nlg", "sportsett_basketball", "totto", "web_nlg"]
78
79 ITEM_REFERENCE_METRICS = [
80     "bleu",
81     "chrF",
82     "ter",
83     "rougeL",
84     "meteor",
85     "bertscore_f1",
86 ]
87 SOURCE_METRICS = [
88     "alignscore_base",
89     "hhem_2_1_open_mean_support",
90     "hhem_2_1_open_min_sentence_support",
91     "hhem_2_1_open_unsupported_sentence_rate",
92 ]
93 ALL_ITEM_METRICS = ITEM_REFERENCE_METRICS + SOURCE_METRICS
94
95 METRIC_LABELS = {
96     "bleu": "BLEU",
97     "chrF": "chrF",
98     "ter": "TER",
99     "rougeL": "ROUGE-L",
100     "meteor": "METEOR",
101     "bertscore_f1": "BERTScore F1",
102     "corpus_bleu": "Corpus BLEU",
103     "corpus_chrf": "Corpus chrF",
104     "corpus_ter": "Corpus TER",
105     "alignscore_base": "AlignScore",
106     "hhem_2_1_open_mean_support": "HHEM mean support",
107     "hhem_2_1_open_min_sentence_support": "HHEM minimum sentence support",
108     "hhem_2_1_open_unsupported_sentence_rate": "HHEM unsupported-sentence rate",
109 }
110
111
112 def read_jsonl(path: Path) -> list[dict[str, Any]]:
113     return [
114         json.loads(line)
115         for line in path.read_text(encoding="utf-8").splitlines()
116         if line.strip()
117     ]
118
119
120 def read_csv(path: Path) -> list[dict[str, str]]:
121     with path.open(encoding="utf-8", newline="") as handle:
122         return list(csv.DictReader(handle))
123
124

```

```

125 def sha256(path: Path) -> str:
126     digest = hashlib.sha256()
127     with path.open("rb") as handle:
128         for chunk in iter(lambda: handle.read(1024 * 1024), b''):
129             digest.update(chunk)
130     return digest.hexdigest()
131
132
133 def rel(path: Path) -> str:
134     return str(path.relative_to(PROJECT_DIR))
135
136
137 def safe_cell(value: Any) -> str:
138     if value is None or value == '':
139         return "-"
140     return str(value).replace("|", "\\|").replace("\n", "<br>")
141
142
143 def md_table(headers: list[str], rows: Iterable[Iterable[Any]]) -> list[str]:
144     lines = [
145         "|" + " | ".join(headers) + " |",
146         "|" + "|".join("----" for _ in headers) + "|",
147     ]
148     for row in rows:
149         lines.append("| " + " | ".join(safe_cell(item) for item in row) + " |")
150     return lines
151
152
153 def score(value: Any) -> str:
154     if value is None or value == '':
155         return "-"
156     number = float(value)
157     return f"{number:.9f}".rstrip("0").rstrip(".")
158
159
160 def seconds(value: Any) -> str:
161     if value is None or value == '':
162         return "-"
163     return f"{float(value):.3f}"
164
165
166 def fenced(text: str, language: str = "text") -> list[str]:
167     fence_length = max(3, max((len(match) for match in re.findall(r"`+", text)), default=0) + 1)
168     fence = "`" * fence_length
169     return [f"{fence}{language}", text.rstrip(), fence]
170
171
172 def formatted_source(source: str) -> tuple[str, str]:
173     stripped = source.strip()
174     if stripped.startswith(("{", "[")):
175         try:
176             payload = json.loads(stripped)
177             return json.dumps(payload, ensure_ascii=False, indent=2, sort_keys=False), "json"
178         except json.JSONDecodeError:
179             pass
180     return source, "text"
181
182
183 def csv_table(path: Path, *, score_columns: bool = True) -> list[str]:
184     rows = read_csv(path)
185     if not rows:
186         return ["*No rows.*"]
187     headers = list(rows[0])
188     values: list[list[str]] = []
189     for row in rows:
190         rendered: list[str] = []
191         for header in headers:
192             value = row[header]
193             if score_columns and header not in {
194                 "dataset_id",
195                 "metric_name",
196                 "comparison",
197                 "paired_metric_cases",
198                 "left_wins",
199                 "ties",
200                 "right_wins",
201             }:
202                 try:
203                     value = score(value)
204                 except ValueError:
205                     pass
206             rendered.append(value)
207         values.append(rendered)
208     return md_table(headers, values)
209
210
211 def generation_key(row: GenerationRecord) -> tuple[str, str, str]:
212     return row.dataset_id, row.example_id, row.variant_id
213
214
215 def case_key(row: GenerationRecord) -> tuple[str, str]:

```

```

216     return row.dataset_id, row.example_id
217
218
219 def direct_model(row: GenerationRecord) -> str:
220     metadata = row.metadata or {}
221     return str(metadata.get("model") or "-")
222
223
224 def token_value(row: GenerationRecord, field: str) -> Any:
225     metadata = row.metadata or {}
226     return metadata.get(field)
227
228
229 def word_count(text: str) -> int:
230     return len(re.findall(r"\b\w'+(?:[-']\w+)*\b", text))
231
232
233 def load_full_models(full_rows: list[GenerationRecord]) -> tuple[Counter[str], list[str]]:
234     signatures: Counter[str] = Counter()
235     missing: list[str] = []
236     for row in full_rows:
237         if not row.pipeline_result_path:
238             missing.append(row.generation_id)
239             continue
240         manifest = Path(row.pipeline_result_path).parent / "00_manifest.json"
241         if not manifest.exists():
242             missing.append(row.generation_id)
243             continue
244         payload = json.loads(manifest.read_text(encoding="utf-8"))
245         signature = json.dumps(payload.get("models", {}), sort_keys=True)
246         signatures[signature] += 1
247     return signatures, missing
248
249
250 def validate_population(
251     generations: list[GenerationRecord],
252     prepared_examples: list[dict[str, Any]],
253     reference_metrics: list[dict[str, Any]],
254     source_metrics: list[dict[str, Any]],
255     annotations: list[dict[str, Any]],
256 ) -> None:
257     expected_variant_counts = Counter({variant: 25 for variant in VARIANT_ORDER})
258     variant_counts = Counter(row.variant_id for row in generations)
259     if variant_counts != expected_variant_counts:
260         raise RuntimeError(f"Unexpected generation condition counts: {variant_counts}")
261     if len({row.generation_id for row in generations}) != 75:
262         raise RuntimeError("Generation IDs are not 75 unique values.")
263     dataset_counts = Counter(row.dataset_id for row in generations)
264     if dataset_counts != Counter({dataset: 15 for dataset in DATASET_ORDER}):
265         raise RuntimeError(f"Unexpected dataset counts: {dataset_counts}")
266
267     grouped: dict[tuple[str, str], list[GenerationRecord]] = defaultdict(list)
268     for row in generations:
269         grouped[case_key(row)].append(row)
270     if len(grouped) != 25 or any(len(rows) != 3 for rows in grouped.values()):
271         raise RuntimeError("Expected 25 cases with exactly three conditions each.")
272     for identity, rows in grouped.items():
273         anchor = rows[0]
274         for row in rows[1:]:
275             fields = [
276                 "source_text",
277                 "references",
278                 "task_family",
279                 "output_mode",
280                 "language",
281             ]
282             mismatched = [field for field in fields if getattr(row, field) != getattr(anchor, field)]
283             if mismatched:
284                 raise RuntimeError(f"Condition inputs differ for {identity}: {mismatched}")
285
286     by_variant = {row.variant_id: row for row in rows}
287     if by_variant["full_system"].request != by_variant["task_aware_direct_flash"].request:
288         raise RuntimeError(f"Full and task-aware requests differ for {identity}")
289
290     prepared_by_id = {
291         (row["dataset_id"], row["example_id"]): row for row in prepared_examples
292     }
293     if set(prepared_by_id) != set(grouped):
294         raise RuntimeError("Prepared-example and generation case identities differ.")
295     for identity, rows in grouped.items():
296         prepared = prepared_by_id[identity]
297         for row in rows:
298             if row.source_text != prepared["source_text"]:
299                 raise RuntimeError(f"Prepared and generated source text differ for {identity}")
300             if row.references != prepared["references"]:
301                 raise RuntimeError(f"Prepared and generated references differ for {identity}")
302     by_variant = {row.variant_id: row for row in rows}
303     if by_variant["full_system"].request != prepared["request"]:
304         raise RuntimeError(f"Prepared and Full-System requests differ for {identity}")
305
306     item_reference = [

```

```

307     row for row in reference_metrics if row["metric_name"] in ITEM_REFERENCE_METRICS
308 ]
309 if len(item_reference) != 75 * len(ITEM_REFERENCE_METRICS):
310     raise RuntimeError(f"Expected 450 item-level reference metrics; found {len(item_reference)}")
311 if len(source_metrics) != 75 * len(SOURCE_METRICS):
312     raise RuntimeError(f"Expected 300 source-grounded metrics; found {len(source_metrics)}")
313 if len(annotations) != 75:
314     raise RuntimeError(f"Expected 75 annotation rows; found {len(annotations)}")
315 if {row["generation_id"] for row in annotations} != {
316     row.generation_id for row in generations
317 }:
318     raise RuntimeError("Annotation and generation identity sets differ.")
319
320
321 def metric_lookup(
322     reference_metrics: list[dict[str, Any]],
323     source_metrics: list[dict[str, Any]],
324 ) -> dict[tuple[str, str], dict[str, Any]]:
325     lookup: dict[tuple[str, str], dict[str, Any]] = {}
326     for row in reference_metrics + source_metrics:
327         key = (row["generation_id"], row["metric_name"])
328         if key in lookup:
329             # Corpus rows are dataset aggregates attached to one generation ID;
330             # their names do not collide with item-level names.
331             raise RuntimeError(f"Duplicate metric key: {key}")
332         lookup[key] = row
333     return lookup
334
335
336 def generation_summary(generations: list[GenerationRecord]) -> list[str]:
337     lines: list[str] = []
338     for variant in VARIANT_ORDER:
339         rows = [row for row in generations if row.variant_id == variant]
340         lines.extend(
341             md_table(
342                 [
343                     "Condition",
344                     "Outputs",
345                     "Generation errors",
346                     "Total elapsed seconds",
347                     "Median output words",
348                     "Writer modes",
349                     "Release statuses",
350                 ],
351                 [
352                     [
353                         VARIANT_LABELS[variant],
354                         len(rows),
355                         sum(row.error is not None for row in rows),
356                         seconds(sum(float(row.elapsed_seconds or 0) for row in rows)),
357                         sorted(word_count(row.generated_text) for row in rows)[len(rows) // 2],
358                     ]
359                     if "Not applicable"
360                     if all(row.writer_mode is None for row in rows)
361                     else ", ".join(
362                         f"{key}: {value}"
363                         for key, value in sorted(
364                             Counter(str(row.writer_mode) for row in rows).items()
365                         )
366                     ),
367                 ],
368                 [
369                     "Not applicable"
370                     if all(row.release_status is None for row in rows)
371                     else ", ".join(
372                         f"{key}: {value}"
373                         for key, value in sorted(
374                             Counter(str(row.release_status) for row in rows).items()
375                         )
376                     ),
377                 ],
378             ),
379         )
380     ]
381     return [
382         "| Condition | Outputs | Generation errors | Total elapsed seconds | Median output words | Writer modes |"
383         "Release statuses |",
384         "|---|---:|---:|---:|---:|---|---|",
385         *lines,
386     ]
387
388
389 def annotation_aggregate(annotations: list[dict[str, Any]]) -> list[str]:
390     rows = []
391     for variant in VARIANT_ORDER:
392         selected = [row for row in annotations if row["variant_id"] == variant]
393         categories = Counter(
394             item["category"] for row in selected for item in row.get("errors", [])
395         )
396         execution = Counter(row["execution_mode"] for row in selected)

```



```

397     rows.append(
398         [
399             VARIANT_LABELS[variant],
400             len(selected),
401             sum(row["error_count"] > 0 for row in selected),
402             sum(row["error_count"] for row in selected),
403             ", ".join(f"{key}: {value}" for key, value in sorted(categories.items()))
404             or "None",
405             ", ".join(f"{key}: {value}" for key, value in sorted(execution.items())),
406         ]
407     )
408     return md_table(
409         ["Condition", "Outputs", "Flagged outputs", "Errors", "Categories", "Execution provenance"],
410         rows,
411     )
412
413
414 def per_example_metric_table(
415     case_rows: list[GenerationRecord],
416     metrics: dict[tuple[str, str], dict[str, Any]],
417 ) -> list[str]:
418     headers = ["Condition"] + [METRIC_LABELS[name] for name in ALL_ITEM_METRICS]
419     rows: list[list[str]] = []
420     by_variant = {row.variant_id: row for row in case_rows}
421     for variant in VARIANT_ORDER:
422         generation = by_variant[variant]
423         values = [VARIANT_LABELS[variant]]
424         for metric_name in ALL_ITEM_METRICS:
425             metric = metrics.get((generation.generation_id, metric_name))
426             values.append(score(metric["score"]) if metric else "-")
427         rows.append(values)
428     return md_table(headers, rows)
429
430
431 def output_metadata_table(row: GenerationRecord, full_model_label: str) -> list[str]:
432     metadata = row.metadata or {}
433     model = full_model_label if row.variant_id == "full_system" else direct_model(row)
434     return md_table(
435         ["Field", "Value"],
436         [
437             ["Generation ID", f"{row.generation_id}"],
438             ["Model", model],
439             ["Seed", row.seed],
440             ["Backend", row.backend],
441             ["Prompt style", metadata.get("prompt_style")],
442             ["Elapsed seconds", seconds(row.elapsed_seconds)],
443             ["Prompt tokens", token_value(row, "prompt_tokens")],
444             ["Completion tokens", token_value(row, "completion_tokens")],
445             ["Total tokens", token_value(row, "total_tokens")],
446             ["Output words", word_count(row.generated_text)],
447             ["Writer mode", row.writer_mode],
448             ["Release status", row.release_status],
449             ["Primary evaluation eligible", row.primary_evaluation_eligible],
450             ["Primary evaluation reason", row.primary_evaluation_reason],
451             ["Repair rounds", row.repair_rounds_used],
452             ["Audit support rate", score(row.audit_support_rate) if row.audit_support_rate is not None else None],
453             ["Mapped support sentences", row.mapped_support_sentence_count],
454             ["Support sentences", row.support_sentence_count],
455             ["Generation error", row.error],
456         ],
457     )
458
459
460 def annotation_section(annotation: dict[str, Any]) -> list[str]:
461     lines = [
462         f"- Judge model: `{annotation['judge_model']}`",
463         f"- Execution mode: `{annotation['execution_mode']}`",
464         f"- API authenticated: `{str(annotation['api_authenticated']).lower()}`",
465         f"- Status: `{annotation['status']}`",
466         f"- Error count: **{annotation['error_count']}**",
467     ]
468     if not annotation["errors"]:
469         lines.append("- Errors: none recorded.")
470     return lines
471     lines.append("- Errors:")
472     for index, item in enumerate(annotation["errors"], start=1):
473         lines.extend(
474             [
475                 f"    {index}. **{item['category']}** - \"{item['error_span']}\",
476                 f"        - {item['correction_or_explanation']}",
477             ]
478         )
479     return lines
480
481
482 def build_document() -> tuple[str, dict[str, Any]]:
483     generations = [
484         GenerationRecord.model_validate(row) for row in read_jsonl(GENERATIONS_PATH)
485     ]
486     prepared_examples = read_jsonl(PREPARED_EXAMPLES_PATH)
487     reference_metrics = read_jsonl(REFERENCE_METRICS_PATH)

```

```

488 source_metrics = read_jsonl(SOURCE_METRICS_PATH)
489 annotations = read_jsonl(ANNOTATIONS_PATH)
490 validate_population(
491     generations,
492     prepared_examples,
493     reference_metrics,
494     source_metrics,
495     annotations,
496 )
497
498 metrics = metric_lookup(reference_metrics, source_metrics)
499 annotation_by_id = {row["generation_id"]: row for row in annotations}
500 grouped: dict[tuple[str, str], list[GenerationRecord]] = defaultdict(list)
501 for row in generations:
502     grouped[case_key(row)].append(row)
503 prepared_by_id = {
504     (row["dataset_id"], row["example_id"]): row for row in prepared_examples
505 }
506
507 full_rows = [row for row in generations if row.variant_id == "full_system"]
508 full_model_signatures, missing_manifests = load_full_models(full_rows)
509 if missing_manifests:
510     raise RuntimeError(f"Missing Full-System manifests: {missing_manifests}")
511 if len(full_model_signatures) != 1:
512     raise RuntimeError(f"Full-System model settings were not constant: {full_model_signatures}")
513 full_models = json.loads(next(iter(full_model_signatures)))
514 full_model_label = ", ".join(
515     f"{{role}}={{model}}" for role, model in full_models.items()
516 )
517
518 artifact_paths = [
519     GENERATIONS_PATH,
520     PREPARED_EXAMPLES_PATH,
521     REFERENCE_METRICS_PATH,
522     SOURCE_METRICS_PATH,
523     ANNOTATIONS_PATH,
524     REFERENCE_MACRO_PATH,
525     SOURCE_MACRO_PATH,
526     REFERENCE_BY_DATASET_PATH,
527     SOURCE_BY_DATASET_PATH,
528     WINS_PATH,
529     SELECTED_FIVE_PATH,
530     EXPERIMENT_MANIFEST_PATH,
531     ANNOTATION_PROVENANCE_PATH,
532     PROGRESS_LOG_PATH,
533     VARIANT_CONFIG_PATH,
534     REFERENCE_CONFIG_PATH,
535     SOURCE_CONFIG_PATH,
536     NOTEBOOK_PATH,
537     NOTEBOOK_SOURCE_PATH,
538     ANNOTATION_BUILDER_PATH,
539     DOSSIER_BUILDER_PATH,
540 ]
541 artifact_hashes = {rel(path): sha256(path) for path in artifact_paths}
542
543 lines: list[str] = [
544     "# Complete Three-Condition Table-to-Text Evaluation Dossier",
545     "",
546     f"Generated from persisted artifacts on {datetime.now(timezone.utc).isoformat()}.",
547     "",
548     "This document is a self-contained results bank for the 25-example, five-dataset experiment comparing the
549     complete multi-agent workflow, a one-call raw-generic baseline and a one-call task-aware baseline. It includes
550     every selected source input, every condition-specific request, held-out human reference, exact generated output,
551     automatic metric score and structured error annotation.",
552     "",
553     "## 1. Experimental population",
554     "",
555 ]
556 lines.extend(
557     md_table(
558         ["Property", "Value"],
559         [
560             ["Datasets", "DART, E2E NLG, SportSett Basketball, ToTTo, WebNLG"],
561             ["Examples", "25 total; five per dataset"],
562             ["Conditions", "3"],
563             ["Generated outputs", "75"],
564             ["Seed", "42"],
565             ["Reference isolation", "Human references were held out from every generator"],
566             ["Request control", "Full and Task-aware Direct used the task-specific request; Raw Generic used one
567             generic request"],
568             ["Reference metric records", len(reference_metrics)],
569             ["Source-grounded metric records", len(source_metrics)],
570             ["Structured judge records", len(annotations)],
571         ],
572     ),
573 )
574 dataset_case_counts = Counter(dataset for dataset, _ in grouped)
575 task_counts = Counter(
576     str(rows[0].task_family.value) for rows in grouped.values()
577 )

```

```

575 output_counts = Counter(
576     str(rows[0].output_mode.value) for rows in grouped.values()
577 )
578 lines.extend(
579     [
580         """
581         ### 1.1 Dataset and task distribution",
582         """
583         *md_table(
584             ["Dataset", "Cases"],
585             [[dataset, dataset_case_counts[dataset]] for dataset in DATASET_ORDER],
586         ),
587         """
588         *md_table(
589             ["Task family", "Cases"],
590             [[key, value] for key, value in sorted(task_counts.items())],
591         ),
592         """
593         *md_table(
594             ["Output mode", "Cases"],
595             [[key, value] for key, value in sorted(output_counts.items())],
596         ),
597         """
598         ### 1.2 Conditions",
599         """
600         *md_table(
601             ["Variant ID", "Description", "Model configuration", "Generation path"],
602             [
603                 [
604                     "`full_system`",
605                     "Six-role multi-agent workflow with deterministic evidence infrastructure, verification, Writer
606 and Auditor.",
607                     full_model_label,
608                     "Source + request + prepared task contract pass through the complete workflow.",
609                 ],
610                 [
611                     "`raw_generic_flash`",
612                     "One direct DeepSeek call receiving a generic strongest-findings request and the source,
613 without task-family/output-form metadata.",
614                     "deepseek-v4-flash; temperature 0.2; maximum output 3,000 tokens",
615                     "Generic direct prompt",
616                 ],
617                 [
618                     "`task_aware_direct_flash`",
619                     "One direct DeepSeek call receiving the same source and request plus task family, output form
620 and language.",
621                     "deepseek-v4-flash; temperature 0.2; maximum output 3,000 tokens",
622                     "Structured direct prompt",
623                 ],
624             ],
625         ),
626         """
627         ### 2. Generator inputs and prompt contracts",
628         """
629         ### 2.1 Shared direct-baseline system prompt",
630         """
631         *fenced(
632             "You are a raw single-LLM data-to-text baseline. Generate the requested output directly from the
633 supplied source data. Use only the source data and the user request. Do not use outside knowledge. Do not invent
634 numbers, entities, chronology, causal explanations, or background. Do not mention hidden references, evaluation,
635 prompts, or uncertainty unless the source itself makes the requested output impossible."
636         ),
637         """
638         ### 2.2 Raw-generic user-prompt template",
639         """
640         *fenced(
641             "Request:\nUnderstand the supplied data and report its strongest supported findings.\n\nSource data:\n{
642 source}\n\nWrite the final answer only."
643         ),
644         """
645         ### 2.3 Task-aware direct user-prompt template",
646         """
647         *fenced(
648             "Task type: {task_family}\nExpected form: {output_mode}\nLanguage: {language}\n\nRequest:\n{example.
649 request}\n\nSource data:\n{source}\n\nWrite the final answer only."
650         ),
651         """
652         ### 2.4 Full-System operational input",
653         """
654         "The Full System received the same source data and task-specific request as Task-aware Direct together with
655 the prepared benchmark task metadata. It did not receive the human references. Unlike the direct conditions, this
656 is a workflow input rather than one monolithic LLM prompt: source interpretation, planning, evidence, verification,
657 writing and auditing are separate stages.",
658         """
659         "The Raw-vs-Task-aware comparison changes the complete communication contract: the request itself becomes
660 task-specific and the prompt adds task family, expected form and language. It should therefore be interpreted as a
661 task-contract ablation, not as an isolated test of metadata labels alone.",
662         """
663         ### 3. Evaluation measures",
664         """
665         *md_table(

```

```

653         ["Metric", "Family", "Orientation", "What is compared"],
654         [
655             ["BLEU / Corpus BLEU", "Lexical overlap", "Higher is better", "Output against held-out reference
text"],
656             ["chrF / Corpus chrF", "Character overlap", "Higher is better", "Output against held-out reference
text"],
657             ["TER / Corpus TER", "Edit distance", "Lower is better", "Output against held-out reference text"],
658             ["ROUGE-L", "Sequence overlap", "Higher is better", "Output against held-out reference text"],
659             ["METEOR", "Lexical-semantic alignment", "Higher is better", "Output against held-out reference
text"],
660             ["BERTScore F1", "Embedding similarity", "Higher is better", "Output against held-out reference
text"],
661             ["AlignScore", "Source-grounded alignment", "Higher is better", "Output against full structured
source"],
662             ["HHEM mean support", "Source-grounded support", "Higher is better", "Sentence support against full
source"],
663             ["HHEM minimum support", "Weakest-sentence support", "Higher is better", "Minimum sentence support
against full source"],
664             ["HHEM unsupported-sentence rate", "Unsupported-content diagnostic", "Lower is better", "Proportion
below the HHEM support threshold"],
665             ["GPT-5.6 Sol taxonomy", "Structured error annotation", "Fewer errors is better", "Source + task +
one output; no human reference as correctness criterion"],
666         ],
667     ),
668     """
669     "Corpus BLEU, corpus chrF and corpus TER are computed once per dataset-condition group of five examples.
They therefore appear in aggregate tables, not as independent per-example scores.",
670     """
671     """## 4. Generation outcomes",
672     """
673     *generation_summary(generations),
674     """
675     "All 75 generation records contain non-empty outputs and no generation-level error. Full-System release and
Writer-mode fields do not apply to the two direct baselines.",
676     """
677     """## 5. Aggregate reference-alignment metrics",
678     """
679     *csv_table(REFERENCE_MACRO_PATH),
680     """
681     """## 6. Aggregate source-grounded metrics",
682     """
683     *csv_table(SOURCE_MACRO_PATH),
684     """
685     """## 7. Direction-adjusted same-item metric wins",
686     """
687     "For TER and HHEM unsupported-sentence rate, lower values are treated as wins; all other included metrics
use higher values.",
688     """
689     *csv_table(WINS_PATH, score_columns=False),
690     """
691     """## 8. Structured error annotations",
692     """
693     *annotation_aggregate(annotations),
694     """
695     "The judge label is `gpt-5.6-sol`. The Full and raw-generic conditions contain 49 API-authenticated records
plus one interactive completion for the previously skipped Full-System SportSett 4975 output. All 25 task-aware
records were produced interactively without an API call. The combined artifact retains `execution_mode` and `
api_authenticated` fields so this split cannot be mistaken for a uniform API run.",
696     """
697     """## 8.1 Category totals",
698     """
699     ]
700 )
701 category_counts = Counter(
702     item["category"] for row in annotations for item in row.get("errors", [])
703 )
704 lines.extend(
705     md_table(
706         ["Category", "Count"],
707         [[category, count] for category, count in sorted(category_counts.items())],
708     )
709 )
710
711 lines.extend(
712     [
713         """
714         """## 9. Reference metrics by dataset",
715         """
716         *csv_table(REFERENCE_BY_DATASET_PATH),
717         """
718         """## 10. Source-grounded metrics by dataset",
719         """
720         *csv_table(SOURCE_BY_DATASET_PATH),
721         """
722         """## 11. Selected five-case source-grounded extraction",
723         """
724         "This is the notebook's dedicated exact-case extract for the five preselected diagnostic cases.",
725         """
726         *csv_table(SELECTED_FIVE_PATH),
727         """
728         """## 12. Aggregate observations",

```

```

729     """
730     "1. The Full System has the strongest overall reference-alignment macro scores: BLEU 0.3595, chrF 0.5972,
METEOR 0.5550, ROUGE-L 0.5599 and BERTScore F1 0.9246. Its macro TER is also lowest at 0.7161.",
731     "2. Supplying the complete task contract to the direct model substantially improves over the raw-generic
direct baseline: Task-aware Direct wins 172 of 265 paired metric cases, loses 46 and ties 47. This contrast
combines a task-specific request with task-family, output-form and language metadata.",
732     "3. The Full System still leads the task-aware direct condition in the paired analysis, with 110 wins, 78
losses and 77 ties, but the gap is much smaller than Full versus raw generic.",
733     "4. SportSett is the principal exception in reference alignment: task-aware direct has the highest
SportSett BERTScore, BLEU, METEOR and ROUGE-L. This indicates that explicit event-report metadata gives a strong
one-call model a major advantage on long-form game reports.",
734     "5. ToITo shows the largest architecture benefit. Full System materially exceeds both direct conditions
because highlighted-cell selection is a content-selection problem, not merely a fluency problem.",
735     "6. Source-grounded metrics disagree with each other. Full System has the highest macro AlignScore (0.6784),
while task-aware direct has the best HHEM mean support (0.5857), minimum support (0.5733) and unsupported-sentence
rate (0.2333). These models should be interpreted as separate diagnostics rather than combined into one factuality
score.",
736     "7. All three conditions receive extremely low HHEM/AlignScore values on SportSett despite mostly plausible
reports and source-checked judge findings. The long nested JSON source is a difficult input representation for
these local factuality models, so SportSett source scores should not be treated as direct hallucination rates.",
737     "8. Structured error annotations flag 10 Full-System errors, 19 raw-generic errors and 13 task-aware-direct
errors. Full System therefore retains the lowest annotation count, while task metadata removes a substantial share
of the raw baseline's task/format failures.",
738     "9. CONTEXT and TASK/FORMAT dominate the annotation taxonomy. Straightforward short-form datasets are
largely accurate; remaining weaknesses concentrate in chronology, causal narration, ranking language and scope
compliance.",
739     """
740     "## 13. Complete per-example records",
741     """
742     "Each record below contains the complete source, condition-specific requests, all three exact outputs, all
item-level metrics and all corresponding structured annotations. References are displayed for evaluation
transparency but were not supplied to any generator or used by the structured judge as its correctness criterion.",
743     """
744     ]
745 )
746
747 dataset_rank = {dataset: index for index, dataset in enumerate(DATASET_ORDER)}
748 ordered_cases = sorted(grouped, key=lambda key: (dataset_rank[key[0]], key[1]))
749 for case_index, identity in enumerate(ordered_cases, start=1):
750     dataset_id, example_id = identity
751     case_rows = grouped[identity]
752     by_variant = {row.variant_id: row for row in case_rows}
753     prepared = prepared_by_id[identity]
754     lines.extend(
755         [
756             f"# Case {case_index}: `{dataset_id}` / `{example_id}`",
757             """
758             "## Case metadata",
759             """
760             *md_table(
761                 ["Field", "Value"],
762                 [
763                     ["Dataset ID", dataset_id],
764                     ["Example ID", example_id],
765                     ["Task family", prepared["task_family"]],
766                     ["Output mode", prepared["output_mode"]],
767                     ["Language", prepared["language"]],
768                     ["Source characters", len(prepared["source_text"])],
769                     ["Reference count", len(prepared["references"])],
770                     ["Source SHA-256", prepared["source_sha256"]],
771                     ["Reference SHA-256", prepared["reference_sha256"]],
772                 ],
773             ),
774             """
775             "## Requests supplied by condition",
776             """
777             "### Full System and Task-aware Direct",
778             """
779             *fenced(by_variant["full_system"].request),
780             """
781             "### Raw Generic",
782             """
783             *fenced(by_variant["raw_generic_flash"].request),
784             """
785             "## Source text supplied to every condition",
786             """
787             "JSON source strings are pretty-printed below for readability; the parsed content is unchanged and its
original SHA-256 is recorded above.",
788             """
789             ]
790     )
791     source_text, source_language = formatted_source(prepared["source_text"])
792     lines.extend(fenced(source_text, source_language))
793
794     try:
795         parsed_source = json.loads(prepared["source_text"])
796     except (json.JSONDecodeError, TypeError):
797         parsed_source = None
798     if parsed_source != prepared["source_payload"]:
799         lines.extend(
800             [

```

```

801         """
802         """## Structured source payload",
803         """
804         *fenced(
805             json.dumps(prepared["source_payload"], ensure_ascii=False, indent=2),
806             "json",
807         ),
808     ]
809 )
810 else:
811     lines.extend(
812         [
813             """
814             "The parsed source text and structured source payload are identical for this case.",
815             """
816         ]
817     )
818 if prepared["parent_table"] is not None:
819     lines.extend(
820         [
821             """
822             """## Parent table representation",
823             """
824             *fenced(
825                 json.dumps(prepared["parent_table"], ensure_ascii=False, indent=2),
826                 "json",
827             ),
828         ]
829     )
830 lines.extend(
831     [
832         """
833         """## Prepared-example metadata",
834         """
835         *fenced(
836             json.dumps(prepared["metadata"], ensure_ascii=False, indent=2),
837             "json",
838         ),
839         """
840         """## Held-out human references",
841         """
842     ]
843 )
844 for reference_index, reference in enumerate(prepared["references"], start=1):
845     lines.extend(
846         [
847             f"""### Reference {reference_index}""",
848             """
849             *fenced(reference),
850             """
851         ]
852     )
853
854 lines.extend(["## Generated outputs", ""])
855 for variant in VARIANT_ORDER:
856     row = by_variant[variant]
857     lines.extend(
858         [
859             f"""### {VARIANT_LABELS[variant]}""",
860             """
861             *output_metadata_table(row, full_model_label),
862             """
863             """#### Exact generated text",
864             """
865             *fenced(row.generated_text),
866             """
867         ]
868     )
869
870 lines.extend(
871     [
872         """## Per-output metrics",
873         """
874         *per_example_metric_table(case_rows, metrics),
875         """
876         """Metric orientation: TER and HHEM unsupported-sentence rate are lower-is-better; every other column is
877         higher-is-better.",
878         """
879         """## Structured judge annotations",
880         """
881     ]
882 )
883 for variant in VARIANT_ORDER:
884     generation = by_variant[variant]
885     annotation = annotation_by_id[generation.generation_id]
886     lines.extend(
887         [
888             f"""### {VARIANT_LABELS[variant]}""",
889             """
890             *annotation_section(annotation),

```

```

891         """
892     ]
893 )
894
895 lines.extend(["---", ""])
896
897 lines.extend(
898     [
899         "# Artifact provenance",
900         """
901         The following SHA-256 hashes identify every persisted input used to build this dossier.",
902         """
903         *md_table(
904             ["Artifact", "SHA-256"],
905             [[path, digest] for path, digest in sorted(artifact_hashes.items())],
906         ),
907         """
908         The complete raw metric JSONL files remain authoritative for metric implementation details such as
909         sentence-level HHEM scores and durations. This document preserves every headline score while avoiding duplication
910         of those low-level diagnostic payloads.",
911         """
912     ]
913 )
914
915 manifest = {
916     "created_at": datetime.now(timezone.utc).isoformat(),
917     "output_path": str(OUTPUT_PATH),
918     "population": {
919         "datasets": 5,
920         "examples": 25,
921         "conditions": 3,
922         "generations": len(generations),
923         "reference_metric_records": len(reference_metrics),
924         "source_metric_records": len(source_metrics),
925         "annotation_records": len(annotations),
926     },
927     "condition_counts": dict(Counter(row.variant_id for row in generations)),
928     "dataset_counts": dict(Counter(row.dataset_id for row in generations)),
929     "full_system_models": full_models,
930     "artifact_sha256": artifact_hashes,
931 }
932
933 return "\n".join(lines), manifest
934
935
936 def main() -> None:
937     print("[1/4] Loading and validating generations, metrics and annotations...", flush=True)
938     document, manifest = build_document()
939     print("[2/4] Writing the self-contained Markdown dossier...", flush=True)
940     OUTPUT_PATH.write_text(document, encoding="utf-8")
941     print("[3/4] Writing the machine-readable dossier manifest...", flush=True)
942     OUTPUT_MANIFEST_PATH.write_text(
943         json.dumps(manifest, ensure_ascii=False, indent=2) + "\n",
944         encoding="utf-8",
945     )
946     print("[4/4] Complete.", flush=True)
947     print(f"Document: {OUTPUT_PATH}")
948     print(f"Manifest: {OUTPUT_MANIFEST_PATH}")
949     print(f"Document bytes: {OUTPUT_PATH.stat().st_size:,}")
950     print(f"Document lines: {len(document.splitlines()):,}")
951
952 if __name__ == "__main__":
953     main()

```

Listing 18: P18: table2text_pydanticai/src/table2text/__init__.py

```

1  """PydanticAI multi-agent Table2Text system."""
2
3  from .config import Settings
4  from .workflow import Table2TextWorkflow
5
6  __all__ = ["Settings", "Table2TextWorkflow"]
7
8  __version__ = "0.1.0"

```

Listing 19: P19: table2text_pydanticai/src/table2text/__main__.py

```

1  """Run the Table2Text command-line interface with ``python -m table2text``."""
2
3  from .cli import main
4
5
6  if __name__ == "__main__":
7      main()

```


Listing 20: P20: table2text_pydanticai/src/table2text/agents.py

```

1  """Build and configure the six PydanticAI agents used by the workflow."""
2
3  from __future__ import annotations
4
5  import re
6  from dataclasses import dataclass, field
7  from typing import Any
8
9  from pydantic_ai import Agent, ModelRetry, ModelSettings, RunContext
10 from pydantic_ai.models.ollama import OllamaModel
11 from pydantic_ai.output import NativeOutput, PromptedOutput
12 from pydantic_ai.providers.ollama import OllamaProvider
13
14 from .audit import (
15     CAUSAL_PATTERN,
16     EXPLANATORY_HYPOTHESIS_PATTERN,
17     FACTUAL_TITLE_PATTERN,
18     FIELD_LABEL_PATTERN,
19     FORECAST_PATTERN,
20     INTERNAL_CONTROL_PATTERN,
21     PREDICTIVE_PATTERN,
22     build_evidence_lookup,
23     content_requirement_errors,
24     extract_number_tokens,
25     fact_support_numbers,
26     flatten_numbers,
27     numbers_supported,
28     sentence_support_narrative_stats,
29     unsupported_backtick_entities,
30     validate_fact_candidates,
31     validate_repair_candidate,
32 )
33 from .config import Settings
34 from .capabilities import (
35     normalise_semantic_map,
36     normalise_event_evidence_queries,
37     validate_event_query_priorities,
38     validate_evidence_queries,
39     validate_semantic_map,
40 )
41 from .schemas import (
42     AnalysisRoute,
43     AuditDecision,
44     AuditMode,
45     AuditRepairProposal,
46     ClaimPermission,
47     ColumnMeaning,
48     ColumnRisk,
49     DataProfile,
50     DataUnderstanding,
51     EvidenceCapability,
52     ExecutionPlan,
53     FactCandidate,
54     FactCandidateSet,
55     FactLedger,
56     InsightCandidate,
57     InsightCandidateSet,
58     InsightContribution,
59     InsightLedger,
60     InsightObjective,
61     InsightRejection,
62     InsightType,
63     InsightVerificationFailure,
64     InsightVerificationResult,
65     InsightVerificationStatus,
66     InvestigationTask,
67     InputSemanticMap,
68     InputShape,
69     InterpretationLevel,
70     InputStructureProfile,
71     ReportComponent,
72     ReportGenre,
73     ReportPerspective,
74     ReportSelectionSource,
75     ReportSpecification,
76     SemanticRole,
77     Severity,
78     StructuralField,
79     SupportType,
80     TableUnderstanding,
81     TargetStatus,
82     ValidationStrategy,
83     VerificationResult,
84     VerifiedFact,
85     VerifiedInsight,
86     WriterAgentDraft,
87     WriterSectionDraft,
88     WriterSentenceDraft,
89 )

```

```

90
91 REPORT_QUALITY_DEFECT_PATTERN = re.compile(
92     r"\b("
93     r"report|sentence|section|wording|phrasing|selection|structure|"
94     r"coherence|redundan|repetit|overstat|understat|omit|unclear|"
95     r"unsupported|imprecise|paragraph|insight|genre|hypothesis|game report"
96     r")\b",
97     re.IGNORECASE,
98 )
99
100
101 def valid_quality_finding(
102     finding: str,
103 ) -> bool:
104     return bool(
105         REPORT_QUALITY_DEFECT_PATTERN.search(
106             finding
107         )
108     )
109
110
111 @dataclass
112 class AgentDependencies:
113     run_id: str
114     payload: dict[str, Any] = field(default_factory=dict)
115
116
117 def build_model(model_specification: str, settings: Settings) -> Any:
118     if model_specification.startswith("ollama:"):
119         model_name = model_specification.split(":", 1)[1]
120
121         return OllamaModel(
122             model_name,
123             provider=OllamaProvider(
124                 base_url=settings.ollama_base_url
125             ),
126         )
127
128     return model_specification
129
130
131 def _uses_openai_gpt5_defaults(model_specification: str) -> bool:
132     provider = ""
133     model_name = model_specification
134     if ":" in model_specification:
135         provider, model_name = model_specification.split(":", 1)
136
137     return (
138         provider == "openai"
139         or not provider
140     ) and model_name.startswith("gpt-5")
141
142
143 def agent_model_settings(
144     settings: Settings,
145     role: str,
146     *,
147     temperature: float,
148     max_tokens: int,
149 ) -> ModelSettings:
150     model_specification = settings.model_for(role)
151     kwargs: dict[str, Any] = {"max_tokens": max_tokens}
152     if not _uses_openai_gpt5_defaults(model_specification):
153         kwargs["temperature"] = temperature
154     return ModelSettings(**kwargs)
155
156
157 def output_schema(output_type: type, settings: Settings) -> Any:
158     if settings.structured_output_mode == "native":
159         return NativeOutput(output_type)
160
161     return PromptedOutput(output_type)
162
163
164 DATA_UNDERSTANDING_INSTRUCTIONS = """
165 You are the Data Understanding Agent in a Table2Text data-science system.
166
167 Combine:
168 - unit-of-observation reasoning;
169 - provisional data dictionary interpretation;
170 - quality and usability assessment;
171 - identification of methodological risks.
172
173 Use only the supplied deterministic profile, input-structure description and
174 sanitized structural field catalog. The catalog contains operational input
175 only; held-out reference text has already been removed.
176
177 Identify:
178 - constant and near-constant columns;
179 - suspicious zero or possible sentinel values;
180 - candidate identifiers;

```

```

181 - candidate time columns;
182 - possible target-proxy risks;
183 - fields that should not be used analytically.
184
185 Do not invent provenance, collection locations, units, scientific meanings,
186 diagnoses, interventions, or source metadata.
187
188 Preserve exact table and column names.
189
190 For structured input, create an `InputSemanticMap` that explains what the
191 record and its fields represent. Use only the broad controlled semantic roles
192 provided by the schema. Keep domain labels such as scoring, votes, revenue or
193 assists in the free-text `label` and `description`; do not invent a new role.
194
195 The semantic map is a broad analytical index for the supplied structure. Include
196 every event, participant and entity binding that may support a faithful report,
197 especially every aggregate participant-level measure and every substantive
198 nested-entity measure. Copy every `path_pattern` verbatim from the structural
199 catalog, including any `*` wildcards. A wildcard pattern binds the repeated
200 field family once: never expand it into concrete participant, entity, period or
201 record keys, and never bind the same catalog path twice.
202 Label wildcard bindings collectively, such as "Participant name" or "Entity
203 points"; never label a wildcard as one particular member such as home,
204 visitor, first or second.
205
206 For every event measure, assign `analytical_function` as a semantic judgement:
207 - `outcome` is the aggregate measure that determines the recorded event result;
208 - `outcome_component` is a substantive participant or entity measure that
209   contributes to comparison but is not itself the aggregate result;
210 - `performance` is a substantive recorded achievement or output;
211 - `participation` is duration, exposure or presence rather than
212   substantive performance;
213 - `context` is a measure whose purpose is descriptive context.
214 Do not treat playing time, duration or exposure as performance. This
215 classification interprets field meaning; it does not calculate a result.
216
217 Prioritise bindings in this order:
218 - event context, time, location and status;
219 - participant and nested-entity identifiers;
220 - participant-level event outcome measures;
221 - all participant-level outcome components, then other participant-level
222   measures that can support contrasts;
223 - all substantive entity-level measures, then participation measures.
224
225 For an event record, include report-critical roles before optional
226 administrative fields: human-readable participant identifiers, human-readable
227 nested entity identifiers when present, the aggregate outcome, event status,
228 participant measures and entity measures. Include enough date and location
229 fields to reconstruct the supplied context. Prefer human-readable names over
230 technical IDs or codes. Do not omit report-critical performance measures in
231 order to include technical record IDs, redundant name components, pre-event
232 records, nested status flags or administrative fields.
233
234 For event records, do not cap participant contrast measures or entity measures.
235 When a nested entity collection has numeric measures, bind each substantive
236 performance or outcome-component measure before optional participation
237 measures.
238
239 Omit low-value administrative fields and exhaustive period/component
240 measures. For a nested single-event record, top-level constancy is expected:
241 describe its scope once rather than creating a separate constant-column risk
242 for every event-context or container field.
243
244 Every semantic binding must use an exact table name and exact path pattern from
245 the supplied structural catalog. Bind the fields needed to identify context,
246 participants, nested entities, outcome measures and other salient measures.
247 When the same measure appears at aggregate and segment/sub-event levels, bind
248 the event outcome to the participant-level aggregate and keep the semantic
249 levels distinct. Do not substitute a period, phase or component value for an
250 event total.
251 Do not encode an analytical conclusion in a binding. Recommend `event_report`
252 when the sanitized structure represents one event with participants or
253 entities, even when the later reporting request may be generic.
254
255 Also recommend a complete communication contract in the semantic map when the
256 request and operational structure support it:
257 - one bounded event with participants and outcomes: `event_report`,
258   `multi_paragraph_report`, and `event_recap`;
259 - a table with an explicit highlighted region: `focused_table_description`,
260   `one_sentence`, and `highlighted_cells`;
261 - an attribute-oriented meaning representation: `attribute_verbalisation` and
262   `short_text`;
263 - subject-relation-object records: `triple_verbalisation` and `short_text`.
264 Use `dataset_overview` or `data_science_report` when no specialised
265 communication task is supported. Infer from field structure and the user
266 request, never from dataset IDs, benchmark labels or held-out references.
267 Leave a recommendation unset when it is genuinely ambiguous.
268
269 Semantic interpretation is not factual evidence. Do not write event results,
270 rankings or report prose in the semantic map.
271 Return structured output only.

```

```

272 """
273
274
275 def build_data_understanding_agent(settings: Settings) -> Agent:
276     agent = Agent(
277         build_model(
278             settings.model_for("data_understanding"),
279             settings,
280         ),
281         name="data_understanding_agent",
282         deps_type=AgentDependencies,
283         output_type=output_schema(
284             DataUnderstanding,
285             settings,
286         ),
287         instructions=DATA_UNDERSTANDING_INSTRUCTIONS,
288         model_settings=agent_model_settings(
289             settings,
290             "data_understanding",
291             temperature=0.0,
292             max_tokens=7_000,
293         ),
294         retries={"output": 2},
295     )
296
297 @agent.output_validator
298 def validate_output(
299     context: RunContext[AgentDependencies],
300     output: DataUnderstanding,
301 ) -> DataUnderstanding:
302     valid_tables = set(
303         context.deps.payload["table_names"]
304     )
305     valid_columns = {
306         table_name: set(columns)
307         for table_name, columns in context.deps.payload["columns"].items()
308     }
309
310     if (
311         output.profile_fingerprint
312         != context.deps.payload["fingerprint"]
313     ):
314         raise ModelRetry(
315             "Use the exact supplied profile_fingerprint."
316         )
317
318     for table in output.tables:
319         if table.table_name not in valid_tables:
320             raise ModelRetry(
321                 f"Unknown table: {table.table_name}"
322             )
323
324         for meaning in table.column_meanings:
325             if meaning.column_name not in valid_columns[table.table_name]:
326                 raise ModelRetry(
327                     f"Unknown column: {meaning.column_name}"
328                 )
329
330         for risk in table.column_risks:
331             if risk.column_name not in valid_columns[table.table_name]:
332                 raise ModelRetry(
333                     f"Unknown risk column: {risk.column_name}"
334                 )
335
336     catalog = [
337         StructuralField.model_validate(item)
338         for item in context.deps.payload.get(
339             "structural_catalog",
340             [],
341         )
342     ]
343     semantic_errors = validate_semantic_map(
344         normalise_semantic_map(output.semantic_map),
345         catalog,
346     )
347     if semantic_errors:
348         raise ModelRetry(
349             "Semantic-map validation failed:\n- " + "\n- ".join(semantic_errors[:12])
350         )
351     output = output.model_copy(
352         update={
353             "semantic_map": normalise_semantic_map(output.semantic_map)
354         }
355     )
356     if context.deps.payload.get("semantic_map_required") and (
357         output.semantic_map is None or not output.semantic_map.bindings
358     ):
359         raise ModelRetry(
360             "Structured operational input requires a non-empty semantic "
361             "map using exact catalog paths."
362         )

```

```

363         return output
364
365     return agent
366
367
368 ORCHESTRATOR_INSTRUCTIONS = """
369 You are the Orchestrator and Investigation Planner.
370
371 Create a frozen analytical plan before analytical results are observed.
372 Define bounded-insight objectives as questions before results are observed.
373 Objectives may use the request, report specification, table and column
374 structure, and planned tasks, but must not contain result values or predicted
375 conclusions.
376
377 The user wants a useful data-science report, not a dump of every statistic.
378
379 Rules:
380 - Use exact table and column names.
381 - Choose analyses relevant to the user's request.
382 - Normally create 2 to 8 tasks.
383 - Do not run prediction merely because a numeric column exists.
384 - A predictive target must be user-selected, metadata-confirmed, or explicitly
385   marked as an experimental candidate.
386 - Do not mark a target as user-selected unless the request names it.
387 - Use chronological validation when a usable time field exists.
388 - Forecasting requires a target, reliable time ordering, rolling evaluation,
389   and naive baselines.
390 - Causal work is feasibility-first.
391 - Include a report specification with a target length and word ceiling. Leave
392   finding and supporting-fact count limits unset; relevance, support,
393   non-duplication and the word ceiling govern report selection.
394 - Use only evidence capabilities listed as available in the supplied input.
395 - Never plan an event result, entity ranking, temporal change, milestone, or
396   comparison capability that is not available.
397 - Do not let one analytical route dominate a general dataset-understanding
398   report.
399 - Do not let one evidence subtype dominate a general dataset-understanding
400   report.
401 - For requests asking to understand a dataset, require dataset overview,
402   data quality, strongest relationships, limitations, and next steps.
403 - For general requests to report findings, cover overview, data quality, the
404   strongest relationships, and limitations/next steps unless the user narrowed
405   the scope.
406 - Prediction and forecasting remain optional and must not be added unless the
407   request or confirmed metadata supports them.
408 - Do not rewrite or replace the user's objective with a different objective.
409 - Negative and insufficiency findings are valid.
410 - For a generic request, honour the controller-selected genre derived from the
411   sanitized semantic map. A high-confidence single event should remain an
412   event report; do not turn it into a flat-table data-science report. Explicit
413   user instructions and experiment configuration still take priority. Genre
414   controls communication, never factual permission.
415 - Use neutral perspective by default. Subject-centred perspective may
416   prioritise verified facts about an explicitly named subject, but it must not
417   change numbers, claim permissions, or evaluative strength.
418 - A generic report may ask which findings jointly describe the strongest
419   structure, which contrast is strongest and non-redundant, whether variables
420   overlap substantially, which quality issue matters most, and what the reader
421   should remember.
422 - A sports report may ask which verified facts describe the result, salient
423   performances, team contrasts, and supported conventional milestones.
424 - An event report must request event_result, event_context,
425   participant_record_context, score_progression, event_sequence,
426   leading_performance, main_contrast and scope_limitations content slots only
427   when their required evidence or safety constraints are available. It must not
428   treat reference text as operational evidence, and sequence evidence must not
429   be inflated into unsupported causality, momentum or turning-point claims.
430 - When a semantic map is supplied, create generic evidence queries using only
431   semantic binding IDs. Use `retrieve` for context, `compare` for participant
432   measures, and `rank` for entity measures. Do not hard-code field aliases or
433   domain-specific extraction rules.
434 - Every field whose name ends in `_binding_id` or `_binding_ids` must contain
435   only exact `binding_id` strings from the supplied ID-only semantic binding
436   catalogue. Never put a path pattern, label or column name in those fields.
437 - Follow these generic query shapes:
438   * `event_context`: retrieve one or more event-level context/time/location
439     value IDs; no entity ID is required.
440   * `event_status`: retrieve one or more event-level status value IDs.
441   * `event_outcome`: compare exactly one participant-level outcome value ID and
442     set `entity_binding_id` to a participant-identifier ID.
443   * `entity_ranking`: rank exactly one entity-level measure value ID, set
444     `entity_binding_id` to an entity-identifier ID and optionally set
445     `group_binding_id` to a participant-identifier ID.
446   * `participant_comparison` or `event_contrast`: compare exactly one
447     participant-level measure value ID and set `entity_binding_id` to a
448     participant-identifier ID.
449 - Query only measures present in the semantic binding catalogue. The broader
450   structural catalog is not permission to reference an unbound measure.
451 - Query questions are pre-result analytical questions. Do not place observed
452   values, winners, rankings or conclusions in a query.
453

```

```

454 - For a supported event report, query event context, event status, the outcome
455 measure, all available substantive entity rankings, and all participant
456 contrasts that can support a faithful report.
457 - Treat the semantic binding's `analytical_function` as the content-priority
458 contract. Prefer `performance` and `outcome_component` for entity rankings.
459 Do not rank `participation` when substantive entity measures are available
460 unless the user's request explicitly asks about duration, exposure or
461 participation.
462 - For participant contrasts, prefer distinct `outcome_component` measures
463 before general performance or context measures. Relate components as
464 descriptive contrasts only; do not imply that they caused the result.
465 - Do not query the same measure/entity combination twice under different names
466 or repeat event context as a data-quality query.
467 - Use these evidence types exactly: event_outcome for outcome comparison;
468 event_context or event_status for context retrieval; entity_ranking for ranking;
469 entity_performance for entity performance; and participant_comparison or
470 event_contrast for participant comparisons.
471 - Do not place a result, statistic, double-double, hat-trick, dominance claim,
472 or other predicted conclusion in an insight objective.
473 - Set frozen=true.
474 """
475
476
477 def build_orchestrator_agent(settings: Settings) -> Agent:
478     agent = Agent(
479         build_model(
480             settings.model_for("orchestrator"),
481             settings,
482         ),
483         name="orchestrator_and_investigation_planner",
484         deps_type=AgentDependencies,
485         output_type=output_schema(
486             ExecutionPlan,
487             settings,
488         ),
489         instructions=ORCHESTRATOR_INSTRUCTIONS,
490         model_settings=agent_model_settings(
491             settings,
492             "orchestrator",
493             temperature=0.1,
494             max_tokens=8_000,
495         ),
496         retries={"output": 3},
497     )
498
499 @agent.output_validator
500 def validate_output(
501     context: RunContext[AgentDependencies],
502     output: ExecutionPlan,
503 ) -> ExecutionPlan:
504     valid_tables = set(
505         context.deps.payload["table_names"]
506     )
507     valid_columns = {
508         table_name: set(columns)
509         for table_name, columns in context.deps.payload["columns"].items()
510     }
511     allow_experimental = context.deps.payload["allow_experimental_targets"]
512     selected_report_genre = context.deps.payload.get("selected_report_genre")
513
514     if not output.frozen:
515         raise ModelRetry("Set frozen=true.")
516
517     user_request = context.deps.payload.get("user_request")
518     if user_request and output.objective.strip() != user_request.strip():
519         raise ModelRetry(
520             "Use the exact supplied user objective; do not substitute a new purpose."
521         )
522
523     if not output.tasks:
524         raise ModelRetry(
525             "Create at least one investigation task."
526         )
527
528     if (
529         selected_report_genre
530         and output.report_specification.genre.value != selected_report_genre
531     ):
532         raise ModelRetry(
533             f"Use the controller-selected report genre exactly: {selected_report_genre}."
534         )
535
536     seen: set[str] = set()
537     available_capabilities = {
538         EvidenceCapability(value)
539         for value in context.deps.payload.get(
540             "available_capabilities",
541             [],
542         )
543     }
544

```

```

545     for task in output.tasks:
546         if task.task_id in seen:
547             raise ModelRetry(
548                 f"Duplicate task ID: {task.task_id}"
549             )
550         seen.add(task.task_id)
551
552         if (
553             task.capability is not None
554             and task.capability not in available_capabilities
555         ):
556             raise ModelRetry(
557                 f"Task {task.task_id} selects unavailable capability "
558                 f"{task.capability.value}."
559             )
560
561         if task.table_name not in valid_tables:
562             raise ModelRetry(
563                 f"Unknown table: {task.table_name}"
564             )
565
566         referenced = [
567             *task.columns,
568             task.target_column,
569             task.time_column,
570             task.exposure_column,
571             task.outcome_column,
572             *task.confounder_columns,
573         ]
574
575         for column in [value for value in referenced if value]:
576             if column not in valid_columns[task.table_name]:
577                 raise ModelRetry(
578                     f"Unknown column `{column}` in `{task.table_name}`."
579                 )
580
581         if (
582             task.target_status == TargetStatus.EXPERIMENTAL_CANDIDATE
583             and not allow_experimental
584         ):
585             raise ModelRetry(
586                 "Experimental targets are disabled by configuration."
587             )
588
589         if (
590             task.route
591             in {
592                 AnalysisRoute.PREDICTIVE,
593                 AnalysisRoute.FORECASTING,
594             }
595             and not task.target_column
596         ):
597             raise ModelRetry(
598                 f"{task.task_id} requires a target column."
599             )
600
601         used_routes = {
602             task.route
603             for task in output.tasks
604         }
605
606         if not used_routes.issubset(
607             set(output.route_order)
608         ):
609             raise ModelRetry(
610                 "route_order must include every route used by the tasks."
611             )
612
613         if settings.enable_insight_synthesis and not output.insight_objectives:
614             raise ModelRetry(
615                 "Create a small set of frozen insight objectives as questions."
616             )
617
618         objective_ids: set[str] = set()
619         for objective in output.insight_objectives:
620             if not objective.objective_id.strip():
621                 raise ModelRetry("Insight objective IDs must not be empty.")
622
623             if objective.objective_id in objective_ids:
624                 raise ModelRetry(
625                     f"Duplicate insight objective ID: {objective.objective_id}"
626                 )
627             objective_ids.add(objective.objective_id)
628
629             if not objective.question.strip().endswith("?"):
630                 raise ModelRetry(
631                     "Insight objectives must be questions, not conclusions."
632                 )
633
634             if re.search(r"(?!\\w)\\d+(?:[.,]\\d+)?%", objective.question):
635                 raise ModelRetry(

```



```

636         "Insight objectives must not contain result values."
637     )
638
639     unknown_task_ids = (
640         set(objective.relevant_task_ids) - seen
641     )
642     if unknown_task_ids:
643         raise ModelRetry(
644             "Insight objectives reference unknown task IDs: "
645             f"{sorted(unknown_task_ids)}"
646         )
647
648     if (
649         output.report_specification.genre
650         in {
651             ReportGenre.EVENT_REPORT,
652             ReportGenre.SPORTS_GAME_REPORT,
653         }
654         and not context.deps.payload.get(
655             "event_genre_allowed",
656             False,
657         )
658     ):
659         raise ModelRetry(
660             "event_report requires an explicit request or experiment "
661             "configuration."
662         )
663
664     if (
665         user_request
666         and output.report_specification.genre
667         not in {
668             ReportGenre.EVENT_REPORT,
669             ReportGenre.SPORTS_GAME_REPORT,
670         }
671         and re.search(
672             r"\b(understand|overview|summariz[e]|describe|report findings|strongest findings)\b",
673             user_request,
674             re.IGNORECASE,
675         )
676     ):
677         required = set(output.report_specification.required_components)
678         expected = {
679             ReportComponent.DATASET_OVERVIEW,
680             ReportComponent.DATA_QUALITY,
681             ReportComponent.STRONGEST_RELATIONSHIPS,
682             ReportComponent.LIMITATIONS_NEXT_STEPS,
683         }
684         if not expected.issubset(required):
685             raise ModelRetry(
686                 "General dataset-understanding reports must require overview, "
687                 "data quality, strongest relationships, and limitations/next steps."
688             )
689
690     semantic_map_payload = context.deps.payload.get("semantic_map")
691     semantic_map = (
692         InputSemanticMap.model_validate(semantic_map_payload) if semantic_map_payload else None
693     )
694     structural_catalog = [
695         StructuralField.model_validate(item)
696         for item in context.deps.payload.get(
697             "structural_catalog",
698             [],
699         )
700     ]
701     if (
702         selected_report_genre
703         in {
704             ReportGenre.EVENT_REPORT.value,
705             ReportGenre.SPORTS_GAME_REPORT.value,
706         }
707         and semantic_map is not None
708         and semantic_map.bindings
709     ):
710         output = output.model_copy(
711             update={
712                 "evidence_queries": normalise_event_evidence_queries(
713                     queries=output.evidence_queries,
714                     semantic_map=semantic_map,
715                     tasks=output.tasks,
716                     available_capabilities=available_capabilities,
717                     request=user_request or "",
718                     structural_catalog=structural_catalog,
719                 )
720             }
721         )
722     query_errors = validate_evidence_queries(
723         output.evidence_queries,
724         semantic_map,
725         structural_catalog,
726         task_ids={task.task_id for task in output.tasks},

```

```

727         available=available_capabilities,
728         task_capabilities={
729             task.task_id: task.capability
730             for task in output.tasks
731         },
732     )
733     if selected_report_genre in {
734         ReportGenre.EVENT_REPORT.value,
735         ReportGenre.SPORTS_GAME_REPORT.value,
736     }:
737         query_errors.extend(
738             validate_event_query_priorities(
739                 output.evidence_queries,
740                 semantic_map,
741                 user_request or "",
742             )
743         )
744     if query_errors:
745         binding_guide = (
746             ", ".join(
747                 f"{binding.binding_id}={binding.label} "
748                 f"({binding.role.value}/{binding.level.value}/"
749                 f"{binding.analytical_function.value if binding.analytical_function else 'none'})"
750                 for binding in semantic_map.bindings
751             )
752             if semantic_map is not None
753             else "none"
754         )
755         raise ModelRetry(
756             "Evidence-query validation failed:\n- "
757             + "\n- ".join(query_errors[:12])
758             + "\nUse only these exact binding IDs: "
759             + binding_guide
760         )
761     if (
762         output.maximum_facts is not None
763         and len(output.evidence_queries) > output.maximum_facts
764     ):
765         raise ModelRetry(
766             "The number of evidence queries must not exceed the fact "
767             "budget because each query requires verifier review."
768         )
769
770     if (
771         selected_report_genre
772         in {
773             ReportGenre.EVENT_REPORT.value,
774             ReportGenre.SPORTS_GAME_REPORT.value,
775         }
776         and semantic_map is not None
777         and semantic_map.bindings
778     ):
779         query_signatures: set[
780             tuple[str, tuple[str, ...], str | None, str | None]
781         ] = set()
782         duplicate_query_ids: list[str] = []
783         for query in output.evidence_queries:
784             signature = (
785                 query.operation.value,
786                 tuple(query.value_binding_ids),
787                 query.entity_binding_id,
788                 query.group_binding_id,
789             )
790             if signature in query_signatures:
791                 duplicate_query_ids.append(query.query_id)
792                 query_signatures.add(signature)
793         if duplicate_query_ids:
794             raise ModelRetry(
795                 "Remove duplicate semantic queries for the same operation, "
796                 "measure and entity bindings: "
797                 + ", ".join(duplicate_query_ids)
798             )
799
800         binding_roles = {binding.role for binding in semantic_map.bindings}
801         query_evidence_types = {query.evidence_type for query in output.evidence_queries}
802         required_evidence_types: set[str] = set()
803         if (
804             SemanticRole.OUTCOME_MEASURE in binding_roles
805             and EvidenceCapability.EVENT_OUTCOME in available_capabilities
806         ):
807             required_evidence_types.add("event_outcome")
808         if binding_roles & {
809             SemanticRole.CONTEXT,
810             SemanticRole.TIME,
811             SemanticRole.LOCATION,
812         }:
813             required_evidence_types.add("event_context")
814         if SemanticRole.STATUS in binding_roles:
815             required_evidence_types.add("event_status")
816
817         missing_evidence_types = required_evidence_types - query_evidence_types

```

```

818         if missing_evidence_types:
819             raise ModelRetry(
820                 "The event plan is missing supported semantic evidence "
821                 "types: " + ", ".join(sorted(missing_evidence_types))
822             )
823
824         return output
825
826     return agent
827
828
829 EVIDENCE_INSTRUCTIONS = """
830 You are the Evidence Analyst Agent.
831
832 The analytical engine has already produced a rich Evidence Ledger containing
833 calculated values, practical interpretations, methodological limitations,
834 salience, and prohibited interpretations.
835
836 Create atomic verified-fact candidates for the verifier.
837
838 Rules:
839 - Every candidate must cite exact evidence IDs.
840 - Preserve calculated values exactly.
841 - Preserve negative and insufficiency findings.
842 - Do not convert the evidence into a final report.
843 - Do not create new statistics or domain explanations.
844 - Carry forward prohibited interpretations and material caveats.
845 - Exclude evidence marked eligible_for_writer=false.
846 - Do not make every candidate a headline; preserve recommended_use.
847 - Do not select facts only from the final evidence items or a single route.
848 - Preserve dataset overview facts, important quality facts, strong or moderate
849   relationships, negative modelling findings, and limitations.
850 - Do not promote small or weak effects to main findings when stronger unused
851   evidence is available.
852 - Do not copy internal prohibited interpretations into fact_summary.
853 - For generic semantic-query evidence, use the query's semantic label,
854   operation and structured metrics to propose the directly supported fact.
855   The executor intentionally does not author winner, ranking or comparison
856   sentences. Do not merely repeat "validated semantic query result".
857 - A compare result may be described only in the direction shown by its ordered
858   records. A rank result must preserve the supplied order, values and tie
859   annotations. Compose identities only from the supplied entity, group and
860   context values.
861 - For focused-table evidence, preserve direct highlighted role/value pairs,
862   supplied highlighted record groups, supplied highlighted-set contrasts, and
863   supplied focused record-style relations or list-page relations as
864   writer-eligible fact candidates. If the evidence supplies a highlighted
865   record-group summary, preserve it as a first-class candidate instead of
866   dropping later highlighted rows for brevity. Keep lower/higher contrast
867   wording scoped to the highlighted cells unless a cited table-wide rank fact
868   explicitly supports a broader claim.
869 """
870
871
872 def build_evidence_agent(settings: Settings) -> Agent:
873     agent = Agent(
874         build_model(
875             settings.model_for("evidence"),
876             settings,
877         ),
878         name="evidence_analyst_agent",
879         deps_type=AgentDependencies,
880         output_type=output_schema(
881             FactCandidateSet,
882             settings,
883         ),
884         instructions=EVIDENCE_INSTRUCTIONS,
885         model_settings=agent_model_settings(
886             settings,
887             "evidence",
888             temperature=0.0,
889             max_tokens=9_000,
890         ),
891         retries={"output": 3},
892     )
893
894     @agent.output_validator
895     def validate_output(
896         context: RunContext[AgentDependencies],
897         output: FactCandidateSet,
898     ) -> FactCandidateSet:
899         from .schemas import EvidenceLedger
900
901         ledger = EvidenceLedger.model_validate(
902             context.deps.payload["evidence_ledger"]
903         )
904
905         errors = validate_fact_candidates(
906             output,
907             ledger,
908         )
909 
```

```

909     valid_candidates: list[FactCandidate] = []
910     dropped_notes: list[str] = []
911     seen_candidate_ids: set[str] = set()
912     for candidate in output.candidates:
913         if candidate.candidate_id in seen_candidate_ids:
914             dropped_notes.append(
915                 f"Dropped duplicate fact candidate {candidate.candidate_id}."
916             )
917             continue
918         seen_candidate_ids.add(candidate.candidate_id)
919         candidate_errors = validate_fact_candidates(
920             FactCandidateSet(candidates=[candidate]),
921             ledger,
922         )
923         if candidate_errors:
924             dropped_notes.append(
925                 f"Dropped invalid fact candidate {candidate.candidate_id}: "
926                 + "; ".join(candidate_errors)
927             )
928             continue
929         valid_candidates.append(candidate)
930
931     if dropped_notes and valid_candidates:
932         output = output.model_copy(
933             update={
934                 "candidates": valid_candidates,
935                 "synthesis_notes": [
936                     *output.synthesis_notes,
937                     *dropped_notes,
938                 ],
939             }
940         )
941         errors = validate_fact_candidates(
942             output,
943             ledger,
944         )
945
946         if errors:
947             raise ModelRetry(
948                 "Fact candidate validation failed:\n- "
949                 + "\n- ".join(errors[:10])
950             )
951
952     return output
953
954 return agent
955
956 VERIFIER_INSTRUCTIONS = """
957 You are the Fact Verification Agent.
958
959 Verify each candidate against the cited evidence.
960
961 Review every candidate exactly once.
962
963 Reject:
964 - unsupported numbers;
965 - unsupported entities;
966 - direction or polarity changes;
967 - permissions absent from the evidence;
968 - facts derived from excluded evidence;
969 - predictive or forecast interpretations without validation;
970 - causal wording without a verified causal design.
971 - reversed ordering, winner/loser labels, ranking positions or comparison
972 direction relative to semantic-query metrics;
973 - event meanings not licensed by the query's semantic label and capability.
974
975 Judge every candidate independently.
976
977 A direct deterministic fact such as a row count, column count,
978 missing-value count, constant-field finding, correlation, or validated
979 group comparison can be fully valid even when it is simple or less
980 narratively interesting than another candidate.
981
982 Do not reject overview or data-quality facts merely to keep the ledger
983 concise. Reject them only when they are unsupported, numerically
984 inconsistent, semantically escalated, or methodologically invalid.
985
986 Do not rewrite candidates into final report prose.
987 """
988
989
990
991 def build_verifier_agent(settings: Settings) -> Agent:
992     agent = Agent(
993         build_model(
994             settings.model_for("verifier"),
995             settings,
996         ),
997         name="fact_verification_agent",
998         deps_type=AgentDependencies,
999         output_type=output_schema(

```

```

1000         VerificationResult,
1001         settings,
1002     ),
1003     instructions=VERIFIER_INSTRUCTIONS,
1004     model_settings=agent_model_settings(
1005         settings,
1006         "verifier",
1007         temperature=0.0,
1008         max_tokens=8_000,
1009     ),
1010     retries={"output": 3},
1011 )
1012
1013 @agent.output_validator
1014 def validate_output(
1015     context: RunContext[AgentDependencies],
1016     output: VerificationResult,
1017 ) -> VerificationResult:
1018     candidates = FactCandidateSet.model_validate(
1019         context.deps.payload["fact_candidates"]
1020     )
1021
1022     expected = {
1023         candidate.candidate_id
1024         for candidate in candidates.candidates
1025     }
1026     received = [
1027         review.candidate_id
1028         for review in output.reviews
1029     ]
1030
1031     if len(received) != len(set(received)):
1032         raise ModelRetry(
1033             "Duplicate fact reviews are not allowed."
1034         )
1035
1036     if set(received) != expected:
1037         raise ModelRetry(
1038             "Review every candidate exactly once. "
1039             f"Missing={sorted(expected - set(received))}; "
1040             f"extra={sorted(set(received) - expected)}"
1041         )
1042
1043     return output
1044
1045 return agent
1046
1047
1048 INSIGHT_SYNTHESIS_INSTRUCTIONS = """
1049 You are the Evidence Analyst performing a second bounded synthesis pass.
1050
1051 The first pass identified evidence-grounded facts. This pass relates verified
1052 facts into distinct, useful, evidence-constrained interpretations.
1053
1054 Definitions:
1055 - Finding: a directly supported observation.
1056 - Bounded insight: an interpretation formed by relating verified findings.
1057 - Analytical implication: why the related findings matter for interpretation
1058   or analysis, without proposing why the observed pattern exists.
1059 - Event synthesis: a supported relationship among an event outcome, context,
1060   rankings, performances or participant contrasts. It can be useful narrative
1061   synthesis without a deeper data-science implication.
1062 - Hypothesis: a plausible explanation requiring additional testing.
1063
1064 Rules:
1065 1. Use only supplied writer-ready verified facts and their referenced
1066   deterministic evidence.
1067 2. Never calculate a statistic.
1068 3. Never introduce a number absent from supplied support.
1069 4. Never introduce an entity absent from supplied support.
1070 5. Every candidate must cite source fact IDs.
1071 6. Every cited fact ID must contribute materially to the statement.
1072 7. A bounded insight normally requires at least the configured minimum number
1073   of source facts in analytical reports.
1074 8. Single-fact exceptions are permitted for anomaly, data-quality
1075   implication, a direct narrative summary supported by one compound fact, or
1076   event-report evidence that is already a structured outcome, context,
1077   status, ranking, performance, or participant contrast.
1078   Focused-table descriptions are also a single-fact exception when the
1079   highlighted cell evidence plus supplied table context expresses the
1080   requested relation.
1081   Structured-record verbalisation tasks are also a single-fact exception
1082   when the supplied attributes or triples contain the complete meaning
1083   representation to express.
1084 9. Do not merely paraphrase one fact, or several facts that repeat the same
1085   result, and label the restatement an insight.
1086 10. Do not turn correlation into causation.
1087     Use outcome_association, never outcome_driver, for descriptive evidence.
1088 11. Do not use drives, causes, explains, leads to, results in, or equivalent
1089     causal wording without explicit causal permission.
1090 12. Do not add domain knowledge from memory.

```

```

1091 13. Do not infer collection location, frequency, provenance, or measurement
1092 process.
1093 14. Do not claim that a variable is useless or universally redundant.
1094 15. Describe overlap as containing highly overlapping information in this
1095 dataset.
1096 16. Set `contribution` to `analytical_implication` for analytical reports,
1097 `event_synthesis` for event reports, and `descriptive_synthesis` for a
1098 concise dataset overview, focused table description, direct answer, or
1099 other short grounded verbalisation task.
1100 17. For `analytical_implication`, `why_it_matters` must add a concrete,
1101 evidence-bounded consequence for interpretation or analysis; it must not
1102 restate coefficients, effect labels, or the candidate statement.
1103 18. For `event_synthesis`, `why_it_matters` may be omitted. Relating supported
1104 rankings, performances, outcomes or participant contrasts is sufficient
1105 when it contributes to the event report and remains event-scoped.
1106 19. For focused-table evidence, a useful descriptive synthesis may infer the
1107 table relation expressed by the highlighted cell using only page/section
1108 titles, headers, row context, highlighted-cell markers and supplied source
1109 text. It may rewrite cell-context evidence into a natural proposition, but
1110 it must not use held-out references or outside knowledge. Set
1111 `interpretation_level` to `bounded_insight`, `contribution` to
1112 `descriptive_synthesis`, and `insight_type` to `narrative_summary` for this
1113 kind of candidate; do not label it as a direct `finding`.
1114 20. For focused-table tasks, row co-entities, headings and source text help
1115 identify the table relation. They are not automatically the grammatical
1116 subject of the output. Prefer the concise proposition conveyed by the
1117 highlighted cell when the local context supports it; use a conservative
1118 selected-cell description only when the relation remains ambiguous.
1119 21. When focused-table evidence includes span-aware logical row context,
1120 highlighted role/value pairs, placeholder roles, or page-title subject
1121 candidates, treat those structured fields as higher priority than raw
1122 adjacent-cell context. A page-title subject candidate may fill a missing
1123 role only when the supplied logical-row evidence and table context support
1124 that reading.
1125 22. For one-sentence focused-table tasks, centre the candidate on the
1126 highlighted role/value pair and the most specific supported primary subject
1127 candidate. Treat non-highlighted same-row values as context; include them
1128 only when they are needed to identify the highlighted relation. Do not
1129 combine a page-title subject candidate with row co-entities into a joint
1130 subject unless the supplied evidence explicitly represents a combined
1131 entity. Use supplied highlighted-measure comparisons when they support a
1132 concise outcome-like relation, but do not calculate new comparisons.
1133 When supplied highlighted-set contrasts relate multiple highlighted values
1134 under the same header, prefer scoped wording such as "among the highlighted
1135 rows/entities" and lower/higher language over table-wide highest/lowest
1136 wording unless a table-wide rank fact is explicitly cited.
1137 When supplied highlighted record groups preserve multiple highlighted rows,
1138 keep all grouped records that contribute to the focused proposition; do not
1139 drop later highlighted records simply to shorten the sentence.
1140 When supplied focused record-style relations pair a highlighted group or
1141 section label with a highlighted record-like value, prefer that natural
1142 proposition over wording about cell coordinates, headers, or "Total" rows.
1143 When supplied focused list-page relations connect a highlighted cell to a
1144 list page title, section, and column header, prefer that proposition over
1145 unrelated same-row details.
1146 23. For structured-record verbalisation tasks, infer only the natural
1147 sentence-level relation licensed by the supplied attributes or triples.
1148 Preserve every supplied entity, relation and value needed by the task, but
1149 do not introduce dataset profiling, data-quality, correlation, modelling or
1150 outside/domain facts.
1151 24. A possible reason why a pattern exists is a hypothesis, including claims
1152 that a pattern may reflect a dependency, data artifact, collection process,
1153 or unmeasured mechanism. Do not hide a hypothesis in `why_it_matters`, a
1154 limitation, or a recommendation.
1155 25. A hypothesis must be explicitly labelled as a hypothesis.
1156 26. A hypothesis must not be suitable for the main report unless hypotheses
1157 are explicitly allowed.
1158 27. Preserve deterministic qualitative strength labels. Do not relabel a
1159 strong association as moderate, or vice versa.
1160 28. For missingness, prefer the directly supported scope of the complete-case
1161 subset over an assumed bias mechanism. For duplicates, describe a possible
1162 influence only as a bounded methodological risk, never as a measured effect.
1163 29. Do not claim that data are complete, contain no missing values, or contain
1164 no duplicates unless the cited facts reference evidence that measured that
1165 exact data-quality property.
1166 30. Include limitations or alternative explanations where needed, but keep
1167 unverified explanations in explicitly labelled hypothesis candidates.
1168 31. Generate every distinct, report-relevant insight supported by the supplied
1169 facts. Do not target a fixed number. Stop when additional candidates would
1170 only duplicate existing synthesis or add weak, irrelevant material.
1171
1172 Return structured output only.
1173 """"
1174
1175
1176 INSIGHT_VERIFIER_INSTRUCTIONS = """"
1177 You are the Fact Verifier reviewing bounded insight candidates.
1178
1179 A candidate can contain correct individual facts while still making an
1180 unsupported interpretation. Review each candidate against only the supplied
1181 verified facts and deterministic evidence.

```



```

1182
1183 For each candidate decide verified, verified_with_caveat, hypothesis_only, or
1184 rejected.
1185
1186 For every record explicitly set:
1187 - `adds_bounded_synthesis`: true only when the statement relates findings and
1188   adds more than a direct finding restatement;
1189 - `analytical_implication_supported`: true only when `why_it_matters` is a
1190   concrete, evidence-bounded analytical implication rather than a paraphrase;
1191 - `contains_hypothesis`: true whenever the statement or analytical implication
1192   proposes a possible explanation that requires further testing.
1193
1194 A record may be verified or verified_with_caveat only when
1195 `adds_bounded_synthesis` is true and `contains_hypothesis` is false. For a
1196 data-science report, `analytical_implication_supported` must also be true. For
1197 an event report, a supported event synthesis does not require a separate
1198 analytical implication: rankings, performances, outcomes and participant
1199 contrasts can provide bounded narrative synthesis. In an event report, a
1200 single structured outcome, context, status, ranking, performance, or
1201 participant contrast can be report-worthy event synthesis when it fills the
1202 selected report contract. For focused-table evidence, a statement that relates
1203 the highlighted value to supplied page/section title, row context, headers and
1204 source text may be verified as descriptive synthesis even without a separate
1205 analytical implication. It is acceptable for this relation to be supported by
1206 one compound focused-table fact when that fact carries the highlighted value
1207 and its local table context. If the evidence contains span-aware logical
1208 role/value pairs or page-title subject candidates, evaluate the statement
1209 against those structured fields before raw adjacent-cell context. If the
1210 evidence contains a concise output focus, highlighted-measure comparison,
1211 highlighted-set contrast, focused record-style relation, or focused list-page
1212 relation, prefer the shortest supported statement centred on highlighted
1213 values and required subject context. A lower/higher contrast must remain
1214 scoped to the highlighted set unless a cited table-wide rank fact supports a
1215 broader highest/lowest claim. Do not require unhighlighted numeric row context
1216 in the final wording unless it is needed for disambiguation.
1217 For structured-record verbalisation evidence, a concise sentence expressing
1218 the supplied attributes or triples may be verified as descriptive synthesis
1219 when it preserves the supplied entities, relations and values and adds no
1220 outside information.
1221 Outside those event, focused-table and structured-record cases, a
1222 direct-finding restatement must be rejected. A candidate containing a possible
1223 explanation must be
1224 hypothesis_only or rejected.
1225
1226 Check that every cited fact exists and genuinely contributes; every number,
1227 table, column, group and entity is supported; the facts jointly support the
1228 statement; and the wording adds useful synthesis rather than renaming one
1229 fact. Match wording strength to evidence strength. Do not introduce causality,
1230 outside domain explanations, collection metadata, or generalisations beyond
1231 the analysed dataset. Preserve needed limitations and identify hypotheses
1232 explicitly. Exclude hypotheses from the main report unless explicitly allowed.
1233 Treat explanations involving possible dependencies, artifacts, collection
1234 processes, or unmeasured mechanisms as hypotheses, even when phrased as a next
1235 step. Do not call a deterministically strong relationship moderate.
1236 Assess salience against the request and selected report genre.
1237
1238 Do not approve a claim merely because it sounds plausible. Do not use outside
1239 knowledge or calculate new values. Preserve or safely weaken candidate
1240 wording; never strengthen it. Return one record for every candidate.
1241 """"
1242
1243
1244 HYPOTHESIS_LABEL_PATTERN = re.compile(
1245     r"\b(hypothesis|hypothesise|hypothesize|hypothesised|hypothesized)\b",
1246     re.IGNORECASE,
1247 )
1248
1249 UNIVERSAL_GENERALISATION_PATTERN = re.compile(
1250     r"\b(always|in general|universally|proves that|demonstrates that all)\b",
1251     re.IGNORECASE,
1252 )
1253
1254 MISSINGNESS_CLAIM_PATTERN = re.compile(
1255     r"\b(no|without|zero)\s+(?:missing(?:ness\s+(?:data|values?))?\b|"
1256     r"r"\bmissing(?:ness\s+(?:data|values?))\b|"
1257     r"r"\b(?:complete data|data (?:are|is|was|were) complete|completeness)\b",
1258     re.IGNORECASE,
1259 )
1260
1261 DUPLICATE_CLAIM_PATTERN = re.compile(
1262     r"\b(no|without|zero)\s+(?:exact\s+)?duplicates?\b|"
1263     r"r"\bduplicates?|deduplicat(?:e|ed|ion)\b",
1264     re.IGNORECASE,
1265 )
1266
1267 EVENT_REPORT_META_OMISSION_PATTERN = re.compile(
1268     r"\b(?:this\s+)?(?:report|summary)\b[^\.]{0,120}"
1269     r"r"\b(?:does\s+not|did\s+not|not|no)\b[^\.]{0,80}"
1270     r"r"\b(?:analy[sz]e[ds]?|include[ds]?|report(?:ed)?|cover(?:ed)?)\b|"
1271     r"r"\b(?:detailed\s+)?(?:event\s+)?(?:chronology|play[- ]by[- ]play|sequence)\b"
1272     r"r"[^\.]{0,120}\b(?:not|no)\b[^\.]{0,80}"

```



```

1273     r"\b(?:analy[sz]ed|included|reported|covered)\b",
1274     re.IGNORECASE,
1275 )
1276
1277 SINGLE_FACT_INSIGHT_TYPES = {
1278     InsightType.ANOMALY,
1279     InsightType.DATA_QUALITY_IMPLICATION,
1280     InsightType.NARRATIVE_SUMMARY,
1281 }
1282
1283 EVENT_INSIGHT_GENRES = {
1284     ReportGenre.EVENT_REPORT,
1285     ReportGenre.SPORTS_GAME_REPORT,
1286 }
1287
1288 STRONG_PERMISSIONS = {
1289     ClaimPermission.CAUSAL,
1290     ClaimPermission.PREDICTIVE,
1291     ClaimPermission.FORECAST,
1292 }
1293
1294
1295 def _normalise_statement(value: str) -> str:
1296     return re.sub(
1297         r"^[^a-z0-9]+",
1298         "",
1299         value.lower(),
1300     ).strip()
1301
1302
1303 def _statement_tokens(value: str) -> set[str]:
1304     return {
1305         token
1306         for token in _normalise_statement(value).split()
1307         if len(token) > 2
1308     }
1309
1310
1311 def _materially_duplicate_statements(
1312     left: str,
1313     right: str,
1314 ) -> bool:
1315     left_normalised = _normalise_statement(left)
1316     right_normalised = _normalise_statement(right)
1317
1318     if left_normalised == right_normalised:
1319         return True
1320
1321     left_tokens = _statement_tokens(left)
1322     right_tokens = _statement_tokens(right)
1323
1324     if not left_tokens or not right_tokens:
1325         return False
1326
1327     return (
1328         len(left_tokens & right_tokens)
1329         / len(left_tokens | right_tokens)
1330         >= 0.85
1331     )
1332
1333
1334 def _safe_fact_permissions(
1335     facts: list[VerifiedFact],
1336 ) -> set[ClaimPermission]:
1337     if not facts:
1338         return set()
1339
1340     permissions = {
1341         permission
1342         for fact in facts
1343         for permission in fact.claim_permissions
1344     }
1345
1346     for permission in STRONG_PERMISSIONS:
1347         if not all(
1348             permission in fact.claim_permissions
1349             for fact in facts
1350         ):
1351             permissions.discard(permission)
1352
1353     return permissions
1354
1355
1356 def _insight_support_numbers(
1357     facts: list[VerifiedFact],
1358     evidence_ledger: Any,
1359 ) -> list[float]:
1360     return [
1361         number
1362         for fact in facts
1363         for number in fact_support_numbers(

```

```

1364         fact,
1365         evidence_ledger,
1366     )
1367 ]
1368
1369
1370 def _schema_entities_for_evidence_ids(
1371     evidence_ids: set[str],
1372     evidence_ledger: Any,
1373 ) -> set[str]:
1374     lookup = build_evidence_lookup(evidence_ledger)
1375     return {
1376         entity
1377         for evidence_id in evidence_ids
1378         if evidence_id in lookup
1379         for entity in [
1380             *lookup[evidence_id].source_tables,
1381             *lookup[evidence_id].source_columns,
1382         ]
1383         if entity
1384     }
1385
1386
1387 def _entity_occurs(
1388     entity: str,
1389     statement: str,
1390 ) -> bool:
1391     return bool(
1392         entity
1393         and re.search(
1394             rf"(?<!\w){re.escape(entity)}(?:!\w)",
1395             statement,
1396             re.IGNORECASE,
1397         )
1398     )
1399
1400
1401 def _unsupported_insight_entities(
1402     *,
1403     statement: str,
1404     facts: list[VerifiedFact],
1405     evidence_ledger: Any,
1406 ) -> list[str]:
1407     cited_evidence_ids = {
1408         evidence_id
1409         for fact in facts
1410         for evidence_id in fact.evidence_ids
1411     }
1412     supported_entities = {
1413         entity
1414         for fact in facts
1415         for entity in fact.entities
1416         if entity
1417     }
1418     supported_entities.update(
1419         _schema_entities_for_evidence_ids(
1420             cited_evidence_ids,
1421             evidence_ledger,
1422         )
1423     )
1424
1425     all_schema_entities = _schema_entities_for_evidence_ids(
1426         {
1427             item.evidence_id
1428             for item in evidence_ledger.items
1429         },
1430         evidence_ledger,
1431     )
1432
1433     unsupported = {
1434         entity
1435         for entity in all_schema_entities
1436         if _entity_occurs(entity, statement)
1437         and entity not in supported_entities
1438     }
1439
1440     unsupported.update(
1441         unsupported_backtick_entities(
1442             statement,
1443             supported_entities,
1444         )
1445     )
1446
1447     for entity in re.findall(
1448         r"\b(?:Team|Player)\s+[A-Z] [\w'-]*(?:\s+[A-Z] [\w'-]*)?",
1449         statement,
1450     ):
1451         if entity not in supported_entities:
1452             unsupported.add(entity)
1453
1454     return sorted(unsupported)

```

```

1455
1456
1457 def _candidate_fact_lookup(
1458     fact_ledger: FactLedger,
1459 ) -> dict[str, VerifiedFact]:
1460     return {
1461         fact.fact_id: fact
1462         for fact in fact_ledger.writer_ready_facts
1463     }
1464
1465
1466 def _supports_data_quality_dimension(
1467     *,
1468     evidence_ids: set[str],
1469     evidence_lookup: dict[str, Any],
1470     dimension: str,
1471 ) -> bool:
1472     for evidence_id in evidence_ids:
1473         item = evidence_lookup.get(evidence_id)
1474         if item is None:
1475             continue
1476         if dimension == "missingness" and (
1477             item.capability == EvidenceCapability.MISSINGNESS
1478             or item.evidence_type == "missingness"
1479             or any(
1480                 key in item.metrics
1481                 for key in {"missing_count", "missing_rate"}
1482             )
1483         ):
1484             return True
1485         if dimension == "duplicates" and (
1486             item.capability == EvidenceCapability.DUPLICATES
1487             or item.evidence_type == "duplicate_rows"
1488             or any(
1489                 key in item.metrics
1490                 for key in {"duplicate_row_count", "duplicate_rate"}
1491             )
1492         ):
1493             return True
1494     return False
1495
1496
1497 def _candidate_has_reportworthy_evidence(
1498     *,
1499     candidate: InsightCandidate,
1500     fact_lookup: dict[str, VerifiedFact],
1501     evidence_lookup: dict[str, Any],
1502     report_genre: ReportGenre,
1503 ) -> bool:
1504     fact_evidence_ids = {
1505         evidence_id
1506         for fact_id in candidate.source_fact_ids
1507         if fact_id in fact_lookup
1508         for evidence_id in fact_lookup[fact_id].evidence_ids
1509     }
1510     fact_evidence_ids.update(candidate.source_evidence_ids)
1511     evidence_types = {
1512         evidence_lookup[evidence_id].evidence_type
1513         for evidence_id in fact_evidence_ids
1514         if evidence_id in evidence_lookup
1515     }
1516     capabilities = {
1517         evidence_lookup[evidence_id].capability
1518         for evidence_id in fact_evidence_ids
1519         if evidence_id in evidence_lookup
1520     }
1521
1522     if (
1523         EvidenceCapability.FOCUSED_TABLE_REGION in capabilities
1524         or "focused_table_region" in evidence_types
1525         or "focused_cell_context" in evidence_types
1526     ):
1527         return True
1528
1529     if report_genre not in EVENT_INSIGHT_GENRES:
1530         return False
1531
1532     return bool(
1533         evidence_types
1534         & {
1535             "event_context",
1536             "event_status",
1537             "participant_record_context",
1538             "score_progression",
1539             "event_outcome",
1540             "entity_ranking",
1541             "entity_performance",
1542             "event_sequence",
1543             "participant_comparison",
1544             "event_contrast",
1545         }

```

```

1546         or capabilities
1547     & {
1548         EvidenceCapability.EVENT_OUTCOME,
1549         EvidenceCapability.ENTITY_PERFORMANCE,
1550         EvidenceCapability.RANKING,
1551         EvidenceCapability.GROUP_COMPARISON,
1552     }
1553 )
1554
1555
1556 def _event_candidate_has_reportworthy_evidence(
1557     *,
1558     candidate: InsightCandidate,
1559     fact_lookup: dict[str, VerifiedFact],
1560     evidence_lookup: dict[str, Any],
1561     report_genre: ReportGenre,
1562 ) -> bool:
1563     return (
1564         report_genre in EVENT_INSIGHT_GENRES
1565         and _candidate_has_reportworthy_evidence(
1566             candidate=candidate,
1567             fact_lookup=fact_lookup,
1568             evidence_lookup=evidence_lookup,
1569             report_genre=report_genre,
1570         )
1571     )
1572
1573
1574 def validate_insight_candidates(
1575     candidate_set: InsightCandidateSet,
1576     fact_ledger: FactLedger,
1577     evidence_ledger: Any,
1578     settings: Settings,
1579     report_genre: ReportGenre = ReportGenre.DATA_SCIENCE_REPORT,
1580 ) -> list[str]:
1581     errors: list[str] = []
1582     fact_lookup = _candidate_fact_lookup(fact_ledger)
1583     evidence_lookup = build_evidence_lookup(evidence_ledger)
1584     seen_ids: set[str] = set()
1585     accepted_for_duplicate_check: list[InsightCandidate] = []
1586
1587     if (
1588         settings.max_insight_candidates is not None
1589         and len(candidate_set.candidates)
1590         > settings.max_insight_candidates
1591     ):
1592         errors.append(
1593             "Insight candidate count exceeds the configured limit of "
1594             f"{settings.max_insight_candidates}."
1595         )
1596
1597     for candidate in candidate_set.candidates:
1598         candidate_id = candidate.insight_id.strip()
1599
1600         if not candidate_id:
1601             errors.append("An insight candidate has an empty insight_id.")
1602             continue
1603
1604         if candidate_id in seen_ids:
1605             errors.append(f"Duplicate insight ID: {candidate_id}")
1606             continue
1607
1608         seen_ids.add(candidate_id)
1609
1610         if not candidate.statement.strip():
1611             errors.append(f"{candidate_id} has an empty statement.")
1612
1613         if not candidate.source_fact_ids:
1614             errors.append(f"{candidate_id} has no source fact IDs.")
1615             continue
1616
1617         if len(candidate.source_fact_ids) != len(
1618             set(candidate.source_fact_ids)
1619         ):
1620             errors.append(f"{candidate_id} repeats source fact IDs.")
1621
1622         unknown_fact_ids = [
1623             fact_id
1624             for fact_id in candidate.source_fact_ids
1625             if fact_id not in fact_lookup
1626         ]
1627         if unknown_fact_ids:
1628             errors.append(
1629                 f"{candidate_id} cites unknown fact IDs: "
1630                 f"{unknown_fact_ids}"
1631             )
1632             continue
1633
1634         facts = [
1635             fact_lookup[fact_id]
1636             for fact_id in candidate.source_fact_ids

```

```

1637     }
1638     fact_evidence_ids = {
1639         evidence_id
1640         for fact in facts
1641         for evidence_id in fact.evidence_ids
1642     }
1643
1644     unknown_evidence_ids = [
1645         evidence_id
1646         for evidence_id in candidate.source_evidence_ids
1647         if evidence_id not in evidence_lookup
1648     ]
1649     if unknown_evidence_ids:
1650         errors.append(
1651             f"{candidate_id} cites unknown evidence IDs: "
1652             f"{unknown_evidence_ids}"
1653         )
1654
1655     unlinked_evidence_ids = [
1656         evidence_id
1657         for evidence_id in candidate.source_evidence_ids
1658         if evidence_id not in fact_evidence_ids
1659     ]
1660     if unlinked_evidence_ids:
1661         errors.append(
1662             f"{candidate_id} cites evidence not referenced by its "
1663             f"source facts: {unlinked_evidence_ids}"
1664         )
1665
1666     analytical_why = (
1667         candidate.why_it_matters
1668         if report_genre == ReportGenre.DATA_SCIENCE_REPORT
1669         else None
1670     )
1671     writer_visible_text = " ".join(
1672         filter(
1673             None,
1674             [
1675                 candidate.statement,
1676                 analytical_why,
1677             ],
1678         )
1679     )
1680     if (
1681         report_genre in EVENT_INSIGHT_GENRES
1682         and EVENT_REPORT_META_OMISSION_PATTERN.search(
1683             writer_visible_text
1684         )
1685     ):
1686         errors.append(
1687             f"{candidate_id} describes report omissions rather than "
1688             f"supported event content."
1689         )
1690     if (
1691         report_genre == ReportGenre.DATA_SCIENCE_REPORT
1692         and candidate.interpretation_level
1693         == InterpretationLevel.BOUNDED_INSIGHT
1694         and not candidate.why_it_matters
1695     ):
1696         errors.append(
1697             f"{candidate_id} lacks an analytical implication required "
1698             f"for a data-science report."
1699         )
1700     if MISSINGNESS_CLAIM_PATTERN.search(
1701         writer_visible_text
1702     ) and not _supports_data_quality_dimension(
1703         evidence_ids=fact_evidence_ids,
1704         evidence_lookup=evidence_lookup,
1705         dimension="missingness",
1706     ):
1707         errors.append(
1708             f"{candidate_id} makes a missingness or completeness claim "
1709             f"without missingness evidence."
1710         )
1711     if DUPLICATE_CLAIM_PATTERN.search(
1712         writer_visible_text
1713     ) and not _supports_data_quality_dimension(
1714         evidence_ids=fact_evidence_ids,
1715         evidence_lookup=evidence_lookup,
1716         dimension="duplicates",
1717     ):
1718         errors.append(
1719             f"{candidate_id} makes a duplicate-data claim without "
1720             f"duplicate-row evidence."
1721         )
1722
1723     support_numbers = _insight_support_numbers(
1724         facts,
1725         evidence_ledger,
1726     )
1727     for field_name, field_text in {

```

```

1728         "statement": candidate.statement,
1729         "why_it_matters": analytical_why,
1730     }.items():
1731         if not field_text:
1732             continue
1733         if not numbers_supported(
1734             field_text,
1735             support_numbers,
1736         ):
1737             errors.append(
1738                 f"{candidate_id} {field_name} contains unsupported "
1739                 "numbers."
1740             )
1741
1742     unsupported_entities = _unsupported_insight_entities(
1743         statement=field_text,
1744         facts=facts,
1745         evidence_ledger=evidence_ledger,
1746     )
1747     if unsupported_entities:
1748         errors.append(
1749             f"{candidate_id} {field_name} contains unsupported table "
1750             f"or column entities: {unsupported_entities}"
1751         )
1752
1753     safe_permissions = _safe_fact_permissions(facts)
1754     if not set(candidate.claim_permissions).issubset(
1755         safe_permissions
1756     ):
1757         errors.append(
1758             f"{candidate_id} requests permissions stronger than its "
1759             "source facts."
1760         )
1761
1762     reportworthy_single_fact_synthesis = (
1763         _candidate_has_reportworthy_evidence(
1764             candidate=candidate,
1765             fact_lookup=fact_lookup,
1766             evidence_lookup=evidence_lookup,
1767             report_genre=report_genre,
1768         )
1769     )
1770
1771     if (
1772         candidate.interpretation_level
1773         == InterpretationLevel.BOUNDED_INSIGHT
1774         and len(set(candidate.source_fact_ids))
1775         < settings.min_facts_per_bounded_insight
1776         and candidate.insight_type
1777         not in SINGLE_FACT_INSIGHT_TYPES
1778         and not reportworthy_single_fact_synthesis
1779     ):
1780         errors.append(
1781             f"{candidate_id} is a single-fact pseudo-insight; "
1782             f"{settings.min_facts_per_bounded_insight} source facts "
1783             "are required."
1784         )
1785
1786     if (
1787         candidate.interpretation_level
1788         == InterpretationLevel.HYPOTHESIS
1789     ):
1790         if not HYPOTHESIS_LABEL_PATTERN.search(candidate.statement):
1791             errors.append(
1792                 f"{candidate_id} is a hypothesis but is not explicitly "
1793                 "labelled as one."
1794             )
1795
1796         if (
1797             candidate.suitable_for_main_report
1798             and not settings.allow_hypotheses_in_report
1799         ):
1800             errors.append(
1801                 f"{candidate_id} cannot be suitable for the main report "
1802                 "while hypotheses are disabled."
1803             )
1804
1805     if (
1806         candidate.interpretation_level
1807         == InterpretationLevel.BOUNDED_INSIGHT
1808         and (
1809             HYPOTHESIS_LABEL_PATTERN.search(writer_visible_text)
1810             or EXPLANATORY_HYPOTHESIS_PATTERN.search(
1811                 writer_visible_text
1812             )
1813         )
1814     ):
1815         errors.append(
1816             f"{candidate_id} contains an explanatory hypothesis but is "
1817             "classified as a bounded insight."
1818         )

```

```

1819
1820 finding_label_allowed_for_reportworthy_synthesis = (
1821     reportworthy_single_fact_synthesis
1822     and candidate.contribution
1823     in {
1824         InsightContribution.DESRIPTIVE_SYNTHESIS,
1825         InsightContribution.EVENT_SYNTHESIS,
1826     }
1827 )
1828 if (
1829     candidate.interpretation_level
1830     == InterpretationLevel.FINDING
1831     and not finding_label_allowed_for_reportworthy_synthesis
1832 ):
1833     errors.append(
1834         f"{candidate_id} is a finding, not a second-pass bounded "
1835         "insight or hypothesis."
1836     )
1837
1838 if (
1839     CAUSAL_PATTERN.search(writer_visible_text)
1840     and ClaimPermission.CAUSAL not in safe_permissions
1841 ):
1842     errors.append(
1843         f"{candidate_id} introduces unsupported causal wording."
1844     )
1845
1846 if (
1847     PREDICTIVE_PATTERN.search(writer_visible_text)
1848     and ClaimPermission.PREDICTIVE not in safe_permissions
1849 ):
1850     errors.append(
1851         f"{candidate_id} introduces unsupported predictive wording."
1852     )
1853
1854 if (
1855     FORECAST_PATTERN.search(writer_visible_text)
1856     and ClaimPermission.FORECAST not in safe_permissions
1857 ):
1858     errors.append(
1859         f"{candidate_id} introduces unsupported forecast wording."
1860     )
1861
1862 if UNIVERSAL_GENERALISATION_PATTERN.search(writer_visible_text):
1863     errors.append(
1864         f"{candidate_id} generalises beyond the analysed dataset."
1865     )
1866
1867 if (
1868     analytical_why
1869     and _materially_duplicate_statements(
1870         candidate.statement,
1871         analytical_why,
1872     )
1873 ):
1874     errors.append(
1875         f"{candidate_id} why_it_matters merely restates the insight."
1876     )
1877
1878 if analytical_why and any(
1879     _materially_duplicate_statements(
1880         analytical_why,
1881         fact_text,
1882     )
1883     for fact in facts
1884     for fact_text in [
1885         fact.fact_summary,
1886         *fact.allowed_interpretations,
1887     ]
1888     if fact_text.strip()
1889 ):
1890     errors.append(
1891         f"{candidate_id} why_it_matters merely restates a source "
1892         "finding."
1893     )
1894
1895 duplicate = next(
1896     (
1897         previous
1898         for previous in accepted_for_duplicate_check
1899         if _materially_duplicate_statements(
1900             previous.statement,
1901             candidate.statement,
1902         )
1903     )
1904     or (
1905         set(previous.source_fact_ids)
1906         == set(candidate.source_fact_ids)
1907         and previous.insight_type == candidate.insight_type
1908         and _materially_duplicate_statements(
1909             previous.supporting_summary,
1910             candidate.supporting_summary,

```



```

1910         )
1911     ),
1912     ),
1913     None,
1914 )
1915 if duplicate is not None:
1916     errors.append(
1917         f"{candidate_id} materially duplicates "
1918         f"{duplicate.insight_id}."
1919     )
1920 else:
1921     accepted_for_duplicate_check.append(candidate)
1922
1923 return errors
1924
1925
1926 def build_insight_synthesis_agent(
1927     settings: Settings,
1928 ) -> Agent:
1929     agent = Agent(
1930         build_model(
1931             settings.model_for("evidence"),
1932             settings,
1933         ),
1934         name="evidence_analyst_agent_second_pass_insight_synthesis",
1935         deps_type=AgentDependencies,
1936         output_type=output_schema(
1937             InsightCandidateSet,
1938             settings,
1939         ),
1940         instructions=INSIGHT_SYNTHESIS_INSTRUCTIONS,
1941         model_settings=agent_model_settings(
1942             settings,
1943             "evidence",
1944             temperature=0.0,
1945             max_tokens=8_000,
1946         ),
1947         retries={"output": 3},
1948     )
1949
1950     return agent
1951
1952
1953 def _verified_statement_is_safe(
1954     *,
1955     candidate: InsightCandidate,
1956     statement: str,
1957     source_fact_ids: list[str],
1958     source_evidence_ids: list[str],
1959     fact_ledger: FactLedger,
1960     evidence_ledger: Any,
1961     settings: Settings,
1962     report_genre: ReportGenre,
1963 ) -> bool:
1964     candidate_normalised = _normalise_statement(candidate.statement)
1965     statement_normalised = _normalise_statement(statement)
1966
1967     if not statement_normalised:
1968         return False
1969
1970     if (
1971         candidate_normalised not in statement_normalised
1972         and statement_normalised not in candidate_normalised
1973     ):
1974         return False
1975
1976     checked = candidate.model_copy(
1977         update={
1978             "statement": statement,
1979             "source_fact_ids": source_fact_ids,
1980             "source_evidence_ids": source_evidence_ids,
1981         }
1982     )
1983
1984     return not validate_insight_candidates(
1985         InsightCandidateSet(candidates=[checked]),
1986         fact_ledger,
1987         evidence_ledger,
1988         settings,
1989         report_genre,
1990     )
1991
1992
1993 def validate_insight_verification(
1994     verification: InsightVerificationResult,
1995     candidates: InsightCandidateSet,
1996     fact_ledger: FactLedger,
1997     evidence_ledger: Any,
1998     settings: Settings,
1999     report_genre: ReportGenre = ReportGenre.DATA_SCIENCE_REPORT,
2000 ) -> list[str]:

```

```

2001 errors: list[str] = []
2002 candidate_lookup = {
2003     candidate.insight_id: candidate
2004     for candidate in candidates.candidates
2005 }
2006 received_ids = [
2007     record.insight_id
2008     for record in verification.records
2009 ]
2010 expected_ids = set(candidate_lookup)
2011
2012 duplicate_ids = {
2013     insight_id
2014     for insight_id in received_ids
2015     if received_ids.count(insight_id) > 1
2016 }
2017 for insight_id in sorted(duplicate_ids):
2018     errors.append(
2019         f"{insight_id} has duplicate insight verification records."
2020     )
2021
2022 for insight_id in sorted(expected_ids - set(received_ids)):
2023     errors.append(
2024         f"{insight_id} was not reviewed by the insight verifier."
2025     )
2026 for insight_id in sorted(set(received_ids) - expected_ids):
2027     errors.append(
2028         f"Unexpected insight verification record: {insight_id}."
2029     )
2030
2031 for record in verification.records:
2032     candidate = candidate_lookup.get(record.insight_id)
2033     if candidate is None:
2034         continue
2035
2036     verified_status = record.status in {
2037         InsightVerificationStatus.VERIFIED,
2038         InsightVerificationStatus.VERIFIED_WITH_CAVEAT,
2039     }
2040     reportworthy = _candidate_has_reportworthy_evidence(
2041         candidate=candidate,
2042         fact_lookup=candidate_fact_lookup(fact_ledger),
2043         evidence_lookup=build_evidence_lookup(evidence_ledger),
2044         report_genre=report_genre,
2045     )
2046     if (
2047         verified_status
2048         and not record.adds_bounded_synthesis
2049         and not reportworthy
2050     ):
2051         errors.append(
2052             f"{record.insight_id} is a direct-finding restatement rather "
2053             f"than a bounded insight."
2054         )
2055     if (
2056         verified_status
2057         and report_genre == ReportGenre.DATA_SCIENCE_REPORT
2058         and not record.analytical_implication_supported
2059     ):
2060         errors.append(
2061             f"{record.insight_id} lacks a supported analytical implication."
2062         )
2063     if verified_status and record.contains_hypothesis:
2064         errors.append(
2065             f"{record.insight_id} contains a hypothesis and cannot be a "
2066             f"verified main insight."
2067         )
2068     if (
2069         record.status
2070         == InsightVerificationStatus.HYPOTHESIS_ONLY
2071         and not record.contains_hypothesis
2072     ):
2073         errors.append(
2074             f"{record.insight_id} is marked hypothesis_only without a "
2075             f"hypothesis."
2076         )
2077     if (
2078         candidate.interpretation_level
2079         == InterpretationLevel.HYPOTHESIS
2080         and verified_status
2081     ):
2082         errors.append(
2083             f"{record.insight_id} cannot verify a hypothesis as a bounded "
2084             f"main insight."
2085         )
2086
2087     source_fact_ids = (
2088         record.verified_source_fact_ids
2089         or candidate.source_fact_ids
2090     )
2091     source_evidence_ids = (

```

```

2092         record.verified_source_evidence_ids
2093         or candidate.source_evidence_ids
2094     )
2095
2096     if not set(source_fact_ids).issubset(
2097         set(candidate.source_fact_ids)
2098     ):
2099         errors.append(
2100             f"{record.insight_id} verifier introduced source fact IDs."
2101         )
2102
2103     candidate_evidence_ids = {
2104         evidence_id
2105         for fact in fact_ledger.writer_ready_facts
2106         if fact.fact_id in candidate.source_fact_ids
2107         for evidence_id in fact.evidence_ids
2108     }
2109     if not set(source_evidence_ids).issubset(candidate_evidence_ids):
2110         errors.append(
2111             f"{record.insight_id} verifier introduced source evidence IDs."
2112         )
2113
2114     if record.verified_statement and not _verified_statement_is_safe(
2115         candidate=candidate,
2116         statement=record.verified_statement,
2117         source_fact_ids=source_fact_ids,
2118         source_evidence_ids=source_evidence_ids,
2119         fact_ledger=fact_ledger,
2120         evidence_ledger=evidence_ledger,
2121         settings=settings,
2122         report_genre=report_genre,
2123     ):
2124         errors.append(
2125             f"{record.insight_id} verifier statement is unsupported or "
2126             "stronger than the candidate."
2127         )
2128
2129     return errors
2130
2131
2132 def build_insight_verifier_agent(
2133     settings: Settings,
2134 ) -> Agent:
2135     agent = Agent(
2136         build_model(
2137             settings.model_for("verifier"),
2138             settings,
2139         ),
2140         name="fact_verification_agent_second_pass_insight_verification",
2141         deps_type=AgentDependencies,
2142         output_type=output_schema(
2143             InsightVerificationResult,
2144             settings,
2145         ),
2146         instructions=INSIGHT_VERIFIER_INSTRUCTIONS,
2147         model_settings=agent_model_settings(
2148             settings,
2149             "verifier",
2150             temperature=0.0,
2151             max_tokens=8_000,
2152         ),
2153         retries={"output": 3},
2154     )
2155
2156     return agent
2157
2158
2159 def empty_insight_ledger(
2160     *,
2161     synthesis_enabled: bool,
2162     fallback_reason: str,
2163 ) -> InsightLedger:
2164     return InsightLedger(
2165         synthesis_enabled=synthesis_enabled,
2166         fallback_reason=fallback_reason,
2167     )
2168
2169
2170 def materialise_insight_ledger(
2171     *,
2172     candidates: InsightCandidateSet,
2173     verification: InsightVerificationResult,
2174     fact_ledger: FactLedger,
2175     evidence_ledger: Any,
2176     settings: Settings,
2177     report_genre: ReportGenre = ReportGenre.DATA_SCIENCE_REPORT,
2178 ) -> InsightLedger:
2179     candidate_errors = validate_insight_candidates(
2180         candidates,
2181         fact_ledger,
2182         evidence_ledger,

```

```

2183         settings,
2184         report_genre,
2185     )
2186     verification_errors = validate_insight_verification(
2187         verification,
2188         candidates,
2189         fact_ledger,
2190         evidence_ledger,
2191         settings,
2192         report_genre,
2193     )
2194     records = {
2195         record.insight_id: record
2196         for record in verification.records
2197     }
2198     evidence_lookup = {
2199         item.evidence_id: item
2200         for item in evidence_ledger.items
2201     }
2202     fact_lookup = _candidate_fact_lookup(fact_ledger)
2203     rejected: list[InsightRejection] = []
2204     unverified: list[InsightVerificationFailure] = []
2205     hypotheses: list[VerifiedInsight] = []
2206     eligible: list[tuple[int, InsightCandidate, VerifiedInsight]] = []
2207     candidate_ids = {
2208         candidate.insight_id
2209         for candidate in candidates.candidates
2210     }
2211     global_candidate_errors = [
2212         error
2213         for error in candidate_errors
2214         if not any(
2215             candidate_id in error
2216             for candidate_id in candidate_ids
2217         )
2218     ]
2219     for index, candidate in enumerate(candidates.candidates):
2220         candidate_reasons = [
2221             error
2222             for error in candidate_errors
2223             if candidate.insight_id in error
2224         ]
2225         candidate_reasons.extend(global_candidate_errors)
2226         record = records.get(candidate.insight_id)
2227
2228         if candidate_reasons:
2229             rejected.append(
2230                 InsightRejection(
2231                     insight_id=candidate.insight_id,
2232                     candidate=candidate,
2233                     reasons=list(dict.fromkeys(candidate_reasons)),
2234                 )
2235             )
2236             continue
2237
2238         if record is None:
2239             unverified.append(
2240                 InsightVerificationFailure(
2241                     insight_id=candidate.insight_id,
2242                     candidate=candidate,
2243                     reasons=[
2244                         "The verifier did not return a usable review for this "
2245                         "insight."
2246                     ],
2247                 )
2248             )
2249             continue
2250
2251         if record.status == InsightVerificationStatus.REJECTED:
2252             rejected.append(
2253                 InsightRejection(
2254                     insight_id=candidate.insight_id,
2255                     candidate=candidate,
2256                     reasons=record.verification_notes
2257                     or ["The verifier rejected this insight."],
2258                 )
2259             )
2260             continue
2261
2262         verification_reasons = [
2263             error
2264             for error in verification_errors
2265             if candidate.insight_id in error
2266         ]
2267
2268         source_fact_ids = list(
2269             dict.fromkeys(
2270                 (
2271                     record.verified_source_fact_ids
2272                     if record.verified_source_fact_ids
2273                     else candidate.source_fact_ids

```

```

2274         )
2275     )
2276 )
2277 source_evidence_ids = list(
2278     dict.fromkeys(
2279         (
2280             record.verified_source_evidence_ids
2281             if record.verified_source_evidence_ids
2282             else (
2283                 candidate.source_evidence_ids
2284                 or [
2285                     evidence_id
2286                     for fact_id in source_fact_ids
2287                     if fact_id in fact_lookup
2288                     for evidence_id in fact_lookup[
2289                         fact_id
2290                     ].evidence_ids
2291                 ]
2292             )
2293         )
2294     )
2295 )
2296 statement = (
2297     record.verified_statement
2298     if record.verified_statement
2299     else candidate.statement
2300 )
2301
2302 if (
2303     record.verified_statement
2304     and not _verified_statement_is_safe(
2305         candidate=candidate,
2306         statement=record.verified_statement,
2307         source_fact_ids=source_fact_ids,
2308         source_evidence_ids=source_evidence_ids,
2309         fact_ledger=fact_ledger,
2310         evidence_ledger=evidence_ledger,
2311         settings=settings,
2312         report_genre=report_genre,
2313     )
2314 ):
2315     verification_reasons.append(
2316         "The verifier statement was stronger than or unsupported by "
2317         "the candidate provenance."
2318     )
2319
2320 confidence = min(
2321     candidate.confidence,
2322     record.confidence,
2323 )
2324 salience = min(
2325     candidate.salience,
2326     record.salience,
2327 )
2328
2329 hypothesis_only = bool(
2330     candidate.interpretation_level
2331     == InterpretationLevel.HYPOTHESIS
2332     or record.status
2333     == InsightVerificationStatus.HYPOTHESIS_ONLY
2334 )
2335
2336 if verification_reasons:
2337     unverified.append(
2338         InsightVerificationFailure(
2339             insight_id=candidate.insight_id,
2340             candidate=candidate,
2341             reasons=list(dict.fromkeys(verification_reasons)),
2342         )
2343     )
2344     continue
2345
2346 contribution = (
2347     InsightContribution.EVENT_SYNTHESIS
2348     if report_genre in EVENT_INSIGHT_GENRES
2349     else (
2350         InsightContribution.DESCRPTIVE_SYNTHESIS
2351         if report_genre == ReportGenre.DATASET_OVERVIEW
2352         else InsightContribution.ANALYTICAL_IMPLICATION
2353     )
2354 )
2355 verified = VerifiedInsight(
2356     insight_id=candidate.insight_id,
2357     statement=statement,
2358     insight_type=candidate.insight_type,
2359     interpretation_level=(
2360         InterpretationLevel.HYPOTHESIS
2361         if hypothesis_only
2362         else InterpretationLevel.BOUNDED_INSIGHT
2363     ),
2364     contribution=contribution,

```

```

2365         source_fact_ids=source_fact_ids,
2366         source_evidence_ids=source_evidence_ids,
2367         source_capabilities=list(
2368             dict.fromkeys(
2369                 evidence_lookup[evidence_id].capability
2370                 for evidence_id in source_evidence_ids
2371                 if evidence_id in evidence_lookup
2372             )
2373         ),
2374         why_it_matters=(
2375             candidate.why_it_matters
2376             if contribution
2377             == InsightContribution.ANALYTICAL_IMPLICATION
2378             else None
2379         ),
2380         limitations=list(
2381             dict.fromkeys(
2382                 [
2383                     *candidate.limitations,
2384                     *record.limitations,
2385                 ]
2386             )
2387         ),
2388         claim_permissions=candidate.claim_permissions,
2389         confidence=confidence,
2390         salience=salience,
2391         verification_status=(
2392             InsightVerificationStatus.HYPOTHESIS_ONLY
2393             if hypothesis_only
2394             else record.status
2395         ),
2396     )
2397
2398     if hypothesis_only:
2399         if not HYPOTHESIS_LABEL_PATTERN.search(statement):
2400             rejected.append(
2401                 InsightRejection(
2402                     insight_id=candidate.insight_id,
2403                     candidate=candidate,
2404                     reasons=[
2405                         "A hypothesis-only statement must be explicitly "
2406                         "labelled as a hypothesis."
2407                     ],
2408                 )
2409             )
2410         else:
2411             hypotheses.append(verified)
2412             continue
2413
2414     if not candidate.suitable_for_main_report:
2415         rejected.append(
2416             InsightRejection(
2417                 insight_id=candidate.insight_id,
2418                 candidate=candidate,
2419                 reasons=["The candidate is not suitable for the main report."],
2420             )
2421         )
2422         continue
2423
2424     if confidence < settings.min_insight_confidence:
2425         rejected.append(
2426             InsightRejection(
2427                 insight_id=candidate.insight_id,
2428                 candidate=candidate,
2429                 reasons=[
2430                     "Confidence is below the configured main-insight "
2431                     "threshold."
2432                 ],
2433             )
2434         )
2435         continue
2436
2437     if salience < settings.min_insight_salience:
2438         rejected.append(
2439             InsightRejection(
2440                 insight_id=candidate.insight_id,
2441                 candidate=candidate,
2442                 reasons=[
2443                     "Salience is below the configured main-insight "
2444                     "threshold."
2445                 ],
2446             )
2447         )
2448         continue
2449
2450     eligible.append((index, candidate, verified))
2451
2452     eligible.sort(
2453         key=lambda item: (
2454             not item[1].suitable_for_main_report,
2455             -item[2].salience,

```

```

2456         -item[2].confidence,
2457         item[0],
2458     )
2459 )
2460 return InsightLedger(
2461     verified_insights=[
2462         verified
2463         for _, _, verified in eligible
2464     ],
2465     hypothesis_only_insights=hypotheses,
2466     rejected_insights=rejected,
2467     unverified_insights=unverified,
2468     verifier_notes=[
2469         *verification.verifier_notes,
2470         *[
2471             error
2472             for error in verification_errors
2473             if error not in {
2474                 reason
2475                 for rejection in rejected
2476                 for reason in rejection.reasons
2477             }
2478         ],
2479     ],
2480     synthesis_enabled=True,
2481 )
2482
2483 def writer_sentence_grounding_errors(
2484     *,
2485     sentence: WriterSentenceDraft,
2486     fact_lookup: dict[str, VerifiedFact],
2487     insight_lookup: dict[str, VerifiedInsight],
2488     sentence_label: str,
2489 ) -> list[str]:
2490     if sentence.support_type == SupportType.NON_FACTUAL:
2491         return []
2492
2493     expanded_fact_ids = list(
2494         dict.fromkeys(
2495             [
2496                 *sentence.fact_ids,
2497                 *[
2498                     fact_id
2499                     for insight_id in sentence.insight_ids
2500                     if insight_id in insight_lookup
2501                     for fact_id in insight_lookup[
2502                         insight_id
2503                     ].source_fact_ids
2504                 ],
2505             ],
2506         )
2507     )
2508
2509     supporting_facts = [
2510         fact_lookup[fact_id]
2511         for fact_id in expanded_fact_ids
2512         if fact_id in fact_lookup
2513     ]
2514
2515     if not supporting_facts:
2516         return []
2517
2518     errors: list[str] = []
2519     support_numbers = [
2520         number
2521         for fact in supporting_facts
2522         for number in flatten_numbers(
2523             fact.structured_values
2524         )
2525     ]
2526     support_numbers.extend(
2527         number
2528         for fact in supporting_facts
2529         for _, number in extract_number_tokens(
2530             fact.fact_summary
2531         )
2532     )
2533     for insight_id in sentence.insight_ids:
2534         insight = insight_lookup.get(insight_id)
2535         if insight is None:
2536             continue
2537         support_numbers.extend(
2538             number
2539             for _, number in extract_number_tokens(
2540                 insight.statement
2541             )
2542         )
2543     if not numbers_supported(
2544         sentence.text,
2545         support_numbers,
2546     ):

```



```

2547         errors.append(
2548             f"{sentence_label} contains a number unsupported by mapped facts "
2549             f"{expanded_fact_ids}."
2550         )
2551
2552     supported_entities = {
2553         entity
2554         for fact in supporting_facts
2555         for entity in fact.entities
2556     }
2557     unsupported_entities = unsupported_backtick_entities(
2558         sentence.text,
2559         supported_entities,
2560     )
2561     if unsupported_entities:
2562         errors.append(
2563             f"{sentence_label} contains unsupported entities "
2564             f"{unsupported_entities}; mapped fact IDs: "
2565             f"{expanded_fact_ids}."
2566         )
2567
2568     return errors
2569
2570
2571 def recover_missing_writer_insight_ids(
2572     draft: WriterAgentDraft,
2573     verified_insight_lookup: dict[str, VerifiedInsight],
2574 ) -> WriterAgentDraft:
2575     """Recover one unambiguous omitted insight ID from exact fact provenance."""
2576
2577     changed = False
2578     recovered_sections: list[WriterSectionDraft] = []
2579
2580     for section in draft.sections:
2581         recovered_sentences: list[WriterSentenceDraft] = []
2582
2583         for sentence in section.sentences:
2584             recovered = sentence
2585             if (
2586                 sentence.interpretation_level
2587                 == InterpretationLevel.BOUNDED_INSIGHT
2588                 and not sentence.insight_ids
2589                 and sentence.fact_ids
2590             ):
2591                 cited_fact_ids = set(sentence.fact_ids)
2592                 candidates = [
2593                     insight_id
2594                     for insight_id, insight
2595                     in verified_insight_lookup.items()
2596                     if set(insight.source_fact_ids)
2597                     == cited_fact_ids
2598                 ]
2599                 if len(candidates) == 1:
2600                     recovered = sentence.model_copy(
2601                         update={"insight_ids": candidates}
2602                     )
2603                     changed = True
2604
2605                 recovered_sentences.append(recovered)
2606
2607             recovered_sections.append(
2608                 section.model_copy(
2609                     update={"sentences": recovered_sentences}
2610                 )
2611             )
2612
2613     if not changed:
2614         return draft
2615
2616     return draft.model_copy(
2617         update={"sections": recovered_sections}
2618     )
2619
2620 WRITER_INSTRUCTIONS = """
2621 You are an expert data scientist and natural report writer.
2622
2623 Use the supplied verified evidence to produce a selective, coherent,
2624 reader-facing data-science report.
2625
2626 Write an insight-led report rather than a catalogue of unrelated statistics
2627 when verified bounded insights are available. The verified Insight Ledger is
2628 the only source of interpretive claims. Verified facts remain the source of
2629 direct findings and supporting details.
2630
2631 For each main analytical paragraph, state one verified bounded insight,
2632 support it with its verified facts, explain why it matters only when the
2633 verified insight carries an analytical implication, and integrate its
2634 limitation where needed. Event and descriptive synthesis can provide value by
2635 coherently relating supported facts without adding a deeper implication. Do
2636 not invent or strengthen an insight during writing.
2637

```

2638
 2639 Keep four roles distinct:
 2640 – a direct finding reports a verified observation or statistic;
 2641 – a bounded insight relates multiple findings into a dataset-scoped pattern;
 2642 – the analytical implication explains why that combined pattern matters for
 2643 interpretation or analysis, using only the insight's `why\_it\_matters`;
 2644 – a hypothesis proposes why the pattern exists and requires further testing.
 2645
 2646 Do not fill an analytical paragraph with a coefficient sentence followed by a
 2647 verbal restatement of the same coefficients. Use the verified analytical
 2648 implication. A possible dependency, artifact, collection process, or unmeasured
 2649 mechanism is a hypothesis even when introduced as something to investigate.
 2650
 2651 Do not turn association into causation, a group difference into an
 2652 explanation, overlapping variables into universal redundancy, a data-quality
 2653 risk into a confirmed error, or a game statistic into unsupported dominance.
 2654 Every bounded-insight sentence must cite its insight ID and retain relevant
 2655 source fact IDs. Every direct factual sentence must cite fact IDs. Do not use
 2656 rejected insights or hypothesis-only insights in the main report.
 2657
 2658 When hypotheses are disabled, do not write a hypothesis section. When they
 2659 are enabled, place them only in a separate "Questions for Further
 2660 Investigation" section, label each as a hypothesis or question, state what
 2661 additional analysis is needed, and never present it as a result.
 2662
 2663 Respect the selected genre, content slots and perspective. Respect a maximum
 2664 word count only when `maximum\_length\_words` is provided; otherwise treat
 2665 `target\_length\_words` as guidance rather than a hard ceiling.
 2666 A data-science
 2667 report uses bounded analytical prose. A dataset overview stays concise and
 2668 mainly finding-led. An event report communicates the verified result, leading
 2669 performances and major participant contrasts in conventional narrative form.
 2670 Fill each required slot only from evidence carrying its required capability.
 2671 Do not mention a slot when its evidence is unavailable. Do not invent
 2672 chronology, comeback leadership, dominance, milestones, audience, venue,
 2673 season context or historical significance. Neutral perspective is the
 2674 default; subject-centred perspective changes selection only, never facts.
 2675
 2676 For an event report, lead with the supported result when available, integrate
 2677 supported date, venue, participant record context, segment score progression
 2678 and status as context, then relate salient entity performances and
 2679 participant-level contrasts. For structured event reports, end with a short
 2680 event-scoped limitation. For reference-style event recaps, keep caveats
 2681 internal unless they are needed to avoid a misleading unsupported inference.
 2682 Do not discuss wrapper row counts, constant columns, missingness, correlation,
 2683 regression, statistical power, feature removal or predictive modelling unless
 2684 the user explicitly requested that analysis. A single event can still support
 2685 within-event comparison and ranking. When a visible limitation is required,
 2686 phrase it in event terms: the comparisons describe only the supplied event,
 2687 do not establish why the result occurred and do not support claims about
 2688 broader performance. Avoid generic boilerplate about "observed associations"
 2689 or "unadjusted group comparisons" in an event report.
 2690 If event-sequence evidence is present, you may mention that recorded sequence
 2691 or score-state information exists, but prefer concrete supported
 2692 score-changing facts over generic availability statements. Do not infer
 2693 unverified chronology, momentum or turning points. If actionable sequence
 2694 facts are present in `content\_requirements`, do not satisfy that slot by
 2695 saying sequence detail was not analysed; use the supported sequence facts or
 2696 omit unsupported commentary.
 2697
 2698 There is no fixed findings or insights quota. Cover required content first,
 2699 then use as many additional distinct, relevant verified facts and insights as
 2700 improve the report. If `maximum\_length\_words` is provided, stay within it; if
 2701 it is not provided, keep the report concise but do not omit strong supported
 2702 material merely to hit the target length. Do not pad the report, repeat
 2703 findings, or omit a stronger supported item merely to reach a particular
 2704 count.
 2705
 2706 When the Writer payload contains `content\_requirements`, treat it as a
 2707 controller-enforced coverage checklist. Use enough supported facts or verified
 2708 insights from each listed content unit to satisfy `minimum\_items`. If
 2709 `enforce\_minimum\_words` is true, write at least `minimum\_word\_count` useful
 2710 words. If an explicit `maximum\_length\_words` is provided, stay below it.
 2711 Expand by adding supported event context, secondary performances, participant
 2712 contrasts, and scoped limitations; do not expand by adding unsupported
 2713 explanation or filler.
 2714 For one-sentence, direct-answer and short-text verbalisation tasks, preserve
 2715 source surface forms that benchmark references are likely to depend on: keep
 2716 digits as digits, keep percentages and decimals unchanged, preserve compact
 2717 units, and do not spell out alphanumeric or hyphenated identifiers. For
 2718 example, keep `ALCO\_RS-3`, `12`, `17068.8` and `58.45%` rather than rewriting
 2719 them as words. Convert relation labels into concise natural predicates when
 2720 clear, such as `RANK = 11` -> `ranks 11th`, `TOTAL = 211.5` -> `total of
 2721 211.5`, and `cylinderCount = 12` -> `12 cylinders`.
 2722 When `realisation\_policy` is `natural\_reference\_style`, you may make harmless
 2723 surface normalisations that improve benchmark-style prose while preserving the
 2724 same supported entity or value, such as changing underscores between words to
 2725 spaces in an identifier. Never change digits, percentages, decimals, units,
 2726 entity identity, or relation meaning. When `realisation\_policy` is
 2727 `strict\_source\_surface`, keep source spelling and separators exactly.
 2728 When `realisation\_policy` is `concise\_table\_proposition`, produce the shortest

complete proposition supported by the focused cell or region; do not add headings, caveats, row-counts, or unrelated table context.

When the Writer payload contains `event\_report\_writing\_guidance`, use it as the event style contract. Start from the result, then build a readable recap from supported context, sequence/progression, leading performances and major participant contrasts. Select and combine the strongest distinct details instead of listing every ranking mechanically. Do not add dataset-quality, correlation, modelling or feature-selection discussion to an event report unless the user explicitly asks for that analysis.

When `event\_report\_writing\_guidance.realisation\_policy` is `event\_recap\_style`, prioritise reference-style event narration: concise opening result, natural progression, key performances, and compact team contrasts. Use only supported sequence/progression facts for chronology; avoid visible methodological caveats unless they prevent a misleading unsupported inference.

If the guidance or content requirements indicate `reference\_recap\_style`, write flowing reference-style event prose: do not use visible Markdown headings, do not add a generic scope/limitations paragraph, and keep caveats internal unless they are needed to avoid an unsupported inference. Use hidden sections only to organise the structured draft; the controller will render the visible text without headings.

For event-sequence units, prefer verified sequence insights for coherent narration, then cite additional sequence fact IDs only for score-changing steps not already covered by the insight. Do not expose internal role labels such as `lead\_change` or `late\_score\_change`; express the supplied scores and events naturally.

When `narrative\_requirements` are present, satisfy them with connected event prose: use verified insights or multi-fact synthesis, contrastive connectors such as "while" or "despite" where the support allows them, and an event-scoped caveat only when required by the supplied narrative requirements. Do not satisfy narration by inventing chronology, momentum or causes.

When the Writer payload contains `narrative\_plan`, follow its slot order and paragraph hints as the visible story plan. Cover higher-priority slots before secondary details, combine related facts into natural sentences, and use `low\_priority\_fact\_ids` only when they add distinct value after the result, sequence, leading performances and participant contrasts are covered.

For focused-table, structured-record verbalisation, direct-answer, one-sentence or short-text tasks, the report contract overrides normal report structure. Write only the requested answer form from the focused or supplied record facts and verified insights. If headings are not allowed, use the title and section fields only as hidden structure; they will not be rendered. Do not add dataset overview, data quality, missingness, correlation, modelling, generic limitations, or unrelated table facts unless the user explicitly asks for them.

When a verified focused-table descriptive insight is available, prefer its natural table relation over a mechanical description of the highlighted cell coordinates. Keep the sentence within the requested output form and cite the insight plus its source facts.

When structured-record evidence is available for an attribute or triple verbalisation task, express all and only those supplied records as fluent natural language. Prefer natural wording over a mechanical key/value dump, but do not add unsupported attributes, entities, relations or background facts.

For triple verbalisation, treat the supplied triples as the sentence schema: keep the source subject as the grammatical subject when natural, render each relation once, preserve relation order where possible, and use compact direct predicates. Do not add explanatory verbs, alternate measurements or background framing not present in the supplied triples. Preserve numeric punctuation exactly.

If the focused evidence or content requirements include a short-form selection policy, centre the sentence on highlighted role/value pairs and the most specific supported primary subject candidate. Treat non-highlighted same-row values as context. Omit them unless they are needed to identify the subject or relation. Do not combine that primary subject with row co-entities into a joint subject unless the supplied evidence explicitly represents a combined entity. Use supplied highlighted-measure comparisons for concise outcome-like wording when the table context supports it; do not calculate new comparisons. If the evidence supplies a highlighted-set contrast, prefer it for one-sentence lower/higher wording, and keep the wording scoped to the highlighted cells unless a cited table-wide rank fact supports a broader highest/lowest claim. If the evidence supplies highlighted record groups, preserve all grouped highlighted records that contribute to the focused answer, including repeated same-pattern rows. If the evidence supplies a focused record-style relation, prefer that relation over a mechanical description of highlighted cell labels, headers, or summary rows. If the evidence supplies a focused list-page relation, prefer it over venue, location, opponent, or other same-row details unless the user explicitly asks for those details.

You have freedom over:

- wording;
- structure;
- selection;
- synthesis;
- paragraph organisation;
- integration of verified analytical implications;
- consolidation of caveats.

You do not have freedom to invent:

- calculated values;
- table or column names;

```

2820 - categories;
2821 - dates;
2822 - locations;
2823 - provenance;
2824 - domain definitions;
2825 - causal explanations;
2826 - predictive performance;
2827 - forecast performance;
2828 - deployment claims.
2829
2830 Internal prohibited interpretations are private safety constraints.
2831 Never quote, enumerate, label, paraphrase as instructions, or expose them
2832 in the visible report.
2833
2834 Do not render internal evidence fields such as:
2835 - Finding:
2836 - Strength:
2837 - Important Note:
2838 - Interpretation Notes:
2839 - Recommended Use:
2840 - Methodological Strength:
2841 - User Relevance:
2842 - Salience:
2843 - Global Prohibited Interpretations
2844
2845 Translate effect labels and metrics into natural prose.
2846 Preserve their verified classification consistently. If mapped evidence calls
2847 an association strong, do not later call the same association moderate. Do not
2848 invent a qualitative strength label that differs from the supplied controlled
2849 strength label.
2850 Do not begin with generic boilerplate such as "This document summarizes",
2851 "This report provides", "Here's a breakdown", or "The goal is to provide".
2852
2853 For example, do not write:
2854
2855 "Strength: Large group difference; Standardized Difference: 1.00"
2856
2857 Write naturally:
2858
2859 "The groups differ substantially; the standardised mean difference is
2860 approximately 1.0."
2861
2862 The Data Understanding output is interpretive context, not an independent
2863 source of factual truth.
2864
2865 Every visible factual statement must be supported by the supplied verified
2866 facts.
2867
2868 Do not introduce a factual claim solely because it appears in:
2869 - dataset_summary;
2870 - unit_of_observation;
2871 - table summary;
2872 - column interpretation;
2873 - quality finding;
2874 - usability note.
2875
2876 In particular, do not state that observations are hourly, collected at a
2877 specific location, produced by a weather station, or gathered through a
2878 particular process unless a verified fact explicitly supports that statement.
2879
2880 Use neutral terms such as "rows", "records", "observations" or
2881 "timestamped observations" when a more specific unit is not verified.
2882
2883 Reader-facing next steps must come from supplied analytical recommendations,
2884 verified methodological facts or the explicit user request. Do not invent
2885 generic future modelling tasks. A recommendation to investigate whether a
2886 pattern reflects a dependency, artifact, collection process, or unmeasured
2887 mechanism is still a hypothesis and must follow the hypothesis policy.
2888
2889 When supported missingness facts are available, state clearly which observed
2890 subset the analysis describes. Do not assume missingness is non-random or has
2891 already biased an estimate. When supported duplicate facts are available,
2892 describe possible influence as an unmeasured methodological risk; do not claim
2893 that deduplication would change results unless that comparison was performed.
2894
2895 For a major group comparison, where supplied, explain:
2896 - the group means;
2897 - their absolute difference;
2898 - whether the difference is small, moderate, or large;
2899 - relevant group-size imbalance;
2900 - whether the comparison is adjusted or unadjusted.
2901
2902 Do not mechanically print all fields. Integrate the most useful context
2903 into natural prose.
2904
2905 Do not include every available fact.
2906 Prioritise the strongest, most relevant, and methodologically defensible
2907 findings.
2908
2909 A generic dataset-understanding report should normally include:
2910 - a concise dataset overview;

```

```

2911 - important data-quality findings;
2912 - the strongest observed relationships;
2913 - relevant methodological limitations;
2914 - grounded next analytical steps.
2915
2916 Small or weak effects should normally be omitted unless they materially
2917 qualify a stronger finding or the user requested completeness.
2918
2919 Deterministically recovered facts are direct representations of trusted
2920 calculated evidence. They are as grounded as LLM-verified facts and may be
2921 used normally, while their recovery method remains recorded internally.
2922
2923 When sufficient verified material exists, do not return only a heading and
2924 one or two factual sentences. Cover every required report component using
2925 the strongest available facts.
2926
2927 Prefer relationship diversity. When both are available, normally include a
2928 strong or moderate correlation and a large or moderate group comparison
2929 rather than several similar comparisons.
2930
2931 Do not use a small relationship merely to increase the number of findings.
2932
2933 Every factual sentence must be represented in the hidden sentence support
2934 map.
2935
2936 Return structured sections and sentences only.
2937 Do not return Markdown.
2938 Do not construct a separate support map.
2939 The controller will materialise Markdown and sentence support
2940 deterministically.
2941
2942 Each factual sentence must list its supporting fact IDs.
2943 List the fact IDs supporting the title in `title_fact_ids`. A factual title
2944 must not introduce an entity, value or result absent from those facts.
2945 Every backticked table or column name and every visible number must be
2946 supported by those same fact IDs. When a sentence combines facts, cite every
2947 fact needed for all of its named entities and values.
2948 Cite every fact and insight ID genuinely needed to support the title or
2949 sentence. Keep sections, sentences and support lists coherent within the report
2950 word ceiling. Never create placeholder IDs, ID ranges or exhaustive sequences
2951 of IDs.
2952 Non-factual transitions may be marked non_factual_transition and must not
2953 cite fact IDs.
2954 """
2955
2956
2957 def build_writer_agent(settings: Settings) -> Agent:
2958     agent = Agent(
2959         build_model(
2960             settings.model_for("writer"),
2961             settings,
2962         ),
2963         name="natural_data_science_writer_agent",
2964         deps_type=AgentDependencies,
2965         output_type=output_schema(
2966             WriterAgentDraft,
2967             settings,
2968         ),
2969         instructions=WRITER_INSTRUCTIONS,
2970         model_settings=agent_model_settings(
2971             settings,
2972             "writer",
2973             temperature=0.15,
2974             max_tokens=11_000,
2975         ),
2976         retries={"output": 3},
2977     )
2978
2979 @agent.output_validator
2980 def validate_output(
2981     context: RunContext[AgentDependencies],
2982     output: WriterAgentDraft,
2983 ) -> WriterAgentDraft:
2984     ledger = FactLedger.model_validate(
2985         context.deps.payload["fact_ledger"]
2986     )
2987
2988     fact_lookup = {
2989         fact.fact_id: fact
2990         for fact in ledger.writer_ready_facts
2991     }
2992     valid_fact_ids = {
2993         fact.fact_id
2994         for fact in ledger.writer_ready_facts
2995     }
2996     insight_ledger = InsightLedger.model_validate(
2997         context.deps.payload.get(
2998             "insight_ledger",
2999             {},
3000         )
3001     )

```

```

3002     verified_insight_lookup = {
3003         insight.insight_id: insight
3004         for insight in insight_ledger.verified_insights
3005     }
3006     hypothesis_insight_lookup = {
3007         insight.insight_id: insight
3008         for insight in insight_ledger.hypothesis_only_insights
3009     }
3010     all_insight_lookup = {
3011         **verified_insight_lookup,
3012         **hypothesis_insight_lookup,
3013     }
3014     output = recover_missing_writer_insight_ids(
3015         output,
3016         verified_insight_lookup,
3017     )
3018     valid_insight_ids = set(verified_insight_lookup)
3019     hypothesis_insight_ids = set(hypothesis_insight_lookup)
3020     all_insight_ids = set(all_insight_lookup)
3021     allow_hypotheses = bool(
3022         context.deps.payload.get(
3023             "allow_hypotheses_in_report",
3024             False,
3025         )
3026     )
3027     maximum_length_words = context.deps.payload.get(
3028         "maximum_length_words"
3029     )
3030     content_requirements = context.deps.payload.get(
3031         "writer_content_requirements"
3032     )
3033     short_form_without_headings = bool(
3034         isinstance(content_requirements, dict)
3035         and (
3036             content_requirements.get("allow_headings") is False
3037             or content_requirements.get("output_form")
3038             in {"one_sentence", "direct_answer", "short_text"}
3039         )
3040     )
3041
3042     errors: list[str] = []
3043
3044     draft_text_parts = (
3045         [
3046             sentence.text
3047             for section in output.sections
3048             for sentence in section.sentences
3049         ]
3050         if short_form_without_headings
3051         else [
3052             output.title,
3053             *[
3054                 part
3055                 for section in output.sections
3056                 for part in [
3057                     section.heading,
3058                     *[
3059                         sentence.text
3060                         for sentence in section.sentences
3061                     ],
3062                 ],
3063             ],
3064         ]
3065     )
3066     draft_word_count = len(
3067         re.findall(
3068             r"\b[\w'-]+\b",
3069             " ".join(draft_text_parts),
3070         )
3071     )
3072     if maximum_length_words is not None:
3073         if draft_word_count > int(maximum_length_words):
3074             errors.append(
3075                 f"The draft contains {draft_word_count} words and exceeds "
3076                 f"the {maximum_length_words}-word ceiling."
3077             )
3078
3079     unknown_title_fact_ids = set(output.title_fact_ids) - valid_fact_ids
3080     if unknown_title_fact_ids and not short_form_without_headings:
3081         errors.append(f"The title uses unknown fact IDs: {sorted(unknown_title_fact_ids)}")
3082     title_entities = {
3083         entity
3084         for fact in ledger.writer_ready_facts
3085         for entity in fact.entities
3086         if len(entity.strip()) >= 3
3087     }
3088     factual_title = bool(
3089         re.search(r"(?<!\w)\d+(?:[.,]\d+)?", output.title)
3090         or (
3091             FACTUAL_TITLE_PATTERN.search(output.title)
3092             and any(

```



```

3093         entity.casefold() in output.title.casefold()
3094         for entity in title_entities
3095     )
3096 )
3097 )
3098 if (
3099     factual_title
3100     and not output.title_fact_ids
3101     and not short_form_without_headings
3102 ):
3103     errors.append("A factual title must list supporting title_fact_ids.")
3104 if output.title_fact_ids and not unknown_title_fact_ids:
3105     supported_title_entities = {
3106         entity
3107         for fact_id in output.title_fact_ids
3108         for entity in fact_lookup[fact_id].entities
3109     }
3110     mentioned_title_entities = {
3111         entity
3112         for entity in title_entities
3113         if entity.casefold() in output.title.casefold()
3114     }
3115     unsupported_title_entities = (
3116         mentioned_title_entities - supported_title_entities
3117     )
3118     if (
3119         factual_title
3120         and unsupported_title_entities
3121         and not short_form_without_headings
3122     ):
3123         errors.append(
3124             "The title contains entities unsupported by its facts: "
3125             f"{sorted(unsupported_title_entities)}"
3126         )
3127     errors.extend(
3128         writer_sentence_grounding_errors(
3129             sentence=WriterSentenceDraft(
3130                 text=output.title,
3131                 fact_ids=output.title_fact_ids,
3132                 support_type=SupportType.DIRECT,
3133             ),
3134             fact_lookup=fact_lookup,
3135             insight_lookup={},
3136             sentence_label="Title",
3137         )
3138     )
3139
3140 if not output.sections:
3141     errors.append(
3142         "Return at least one report section."
3143     )
3144
3145 if short_form_without_headings and isinstance(content_requirements, dict):
3146     sentence_texts = [
3147         sentence.text
3148         for section in output.sections
3149         for sentence in section.sentences
3150         if sentence.text.strip()
3151     ]
3152     if (
3153         content_requirements.get("require_complete_sentence")
3154         and sentence_texts
3155         and sentence_texts[-1].strip()[-1] not in ".!?"
3156     ):
3157         errors.append(
3158             "The draft must provide a complete sentence for this "
3159             "output form."
3160         )
3161
3162 for section_index, section in enumerate(
3163     output.sections,
3164     start=1,
3165 ):
3166     if not section.sentences:
3167         errors.append(
3168             f"Section {section_index} contains no sentences."
3169         )
3170
3171     for sentence_index, sentence in enumerate(
3172         section.sentences,
3173         start=1,
3174     ):
3175         unknown = (
3176             set(sentence.fact_ids)
3177             - valid_fact_ids
3178         )
3179
3180         if unknown:
3181             errors.append(
3182                 "Sentence "
3183                 f"{section_index}.{sentence_index} "

```



```

3184         "uses unknown fact IDs: "
3185         f"{sorted(unknown)}"
3186     )
3187
3188     unknown_insights = (
3189         set(sentence.insight_ids)
3190         - all_insight_ids
3191     )
3192     if unknown_insights:
3193         errors.append(
3194             "Sentence "
3195             f"{section_index}.{sentence_index} "
3196             "uses unknown insight IDs: "
3197             f"{sorted(unknown_insights)}"
3198         )
3199
3200     if not unknown and not unknown_insights:
3201         errors.extend(
3202             writer_sentence_grounding_errors(
3203                 sentence=sentence,
3204                 fact_lookup=fact_lookup,
3205                 insight_lookup=all_insight_lookup,
3206                 sentence_label=(
3207                     "Sentence "
3208                     f"{section_index}.{sentence_index}"
3209                 ),
3210             )
3211         )
3212
3213     if (
3214         EXPLANATORY_HYPOTHESIS_PATTERN.search(
3215             sentence.text
3216         )
3217         and sentence.interpretation_level
3218         != InterpretationLevel.HYPOTHESIS
3219     ):
3220         errors.append(
3221             "Sentence "
3222             f"{section_index}.{sentence_index} presents a possible "
3223             "explanation without classifying it as a hypothesis."
3224         )
3225
3226     if (
3227         sentence.interpretation_level
3228         == InterpretationLevel.BOUNDED_INSIGHT
3229     ):
3230         if not sentence.insight_ids:
3231             errors.append(
3232                 "A bounded-insight sentence must cite a verified "
3233                 "insight ID."
3234             )
3235         elif not set(sentence.insight_ids).issubset(
3236             valid_insight_ids
3237         ):
3238             errors.append(
3239                 "A bounded-insight sentence may cite only verified "
3240                 "main insights."
3241             )
3242
3243     if (
3244         sentence.interpretation_level
3245         == InterpretationLevel.HYPOTHESIS
3246     ):
3247         if not allow_hypotheses:
3248             errors.append(
3249                 "Hypothesis sentences are disabled by configuration."
3250             )
3251         if not sentence.insight_ids or not set(
3252             sentence.insight_ids
3253         ).issubset(hypothesis_insight_ids):
3254             errors.append(
3255                 "A hypothesis sentence must cite a hypothesis-only "
3256                 "insight ID."
3257             )
3258         if section.heading.strip().lower() != (
3259             "questions for further investigation"
3260         ):
3261             errors.append(
3262                 "Hypotheses may appear only in the Questions for "
3263                 "Further Investigation section."
3264             )
3265         if not HYPOTHESIS_LABEL_PATTERN.search(
3266             sentence.text
3267         ) and not sentence.text.strip().endswith("?"):
3268             errors.append(
3269                 "A hypothesis sentence must be explicitly labelled "
3270                 "as a hypothesis."
3271             )
3272
3273     if (
3274         sentence.interpretation_level

```

```

3275         == InterpretationLevel.FINDING
3276         and sentence.insight_ids
3277     ):
3278         errors.append(
3279             "A direct finding must not be relabelled with insight IDs."
3280         )
3281
3282     if (
3283         sentence.support_type
3284         != SupportType.NON_FACTUAL
3285         and not sentence.fact_ids
3286         and not sentence.insight_ids
3287     ):
3288         errors.append(
3289             "Sentence "
3290             f"{section_index}.{sentence_index} "
3291             "is factual but has no supporting facts."
3292         )
3293
3294     if (
3295         sentence.support_type
3296         == SupportType.NON_FACTUAL
3297         and (
3298             sentence.fact_ids
3299             or sentence.insight_ids
3300         )
3301     ):
3302         errors.append(
3303             "A non-factual transition must not "
3304             "cite fact or insight IDs."
3305         )
3306
3307     if re.search(
3308         r"\[(?:CLM|FACT)_{d+}",
3309         sentence.text,
3310     ):
3311         errors.append(
3312             "Internal fact IDs must not appear "
3313             "in visible sentence text."
3314         )
3315
3316     if INTERNAL_CONTROL_PATTERN.search(
3317         sentence.text
3318     ):
3319         errors.append(
3320             "Visible sentence text exposes an "
3321             "internal control."
3322         )
3323
3324     if FIELD_LABEL_PATTERN.search(
3325         sentence.text
3326     ):
3327         errors.append(
3328             "Visible sentence text renders an "
3329             "internal evidence field."
3330         )
3331
3332     used_fact_ids = {
3333         *output.title_fact_ids,
3334         *[
3335             fact_id
3336             for section in output.sections
3337             for sentence in section.sentences
3338             for fact_id in sentence.fact_ids
3339         ],
3340     }
3341     used_insight_ids = {
3342         insight_id
3343         for section in output.sections
3344         for sentence in section.sentences
3345         for insight_id in sentence.insight_ids
3346     }
3347     errors.extend(
3348         content_requirement_errors(
3349             used_fact_ids=used_fact_ids,
3350             used_insight_ids=used_insight_ids,
3351             word_count=draft_word_count,
3352             requirements=content_requirements,
3353             narrative_stats=sentence_support_narrative_stats(
3354                 [
3355                     sentence
3356                     for section in output.sections
3357                     for sentence in section.sentences
3358                 ]
3359             ),
3360             include_word_count=True,
3361         )
3362     )
3363
3364     if errors:
3365         raise ModelRetry(

```

```

3366         "Writer draft validation failed:\n- "
3367         + "\n- ".join(errors)
3368     )
3369
3370     return output
3371
3372     return agent
3373
3374
3375 AUDITOR_INSTRUCTIONS = """
3376 You are the Factual Accuracy Auditor and Report Repair Agent.
3377
3378 The goal is to reduce residual hallucinations, not to demand that the
3379 writer copy deterministic templates.
3380
3381 You receive:
3382 - the raw or repaired report;
3383 - the hidden sentence support map;
3384 - verified facts;
3385 - full evidence;
3386 - deterministic profile support records;
3387 - deterministic audit findings;
3388 - methodological limitations;
3389 - optional trusted external facts.
3390
3391 Authority hierarchy:
3392 - The deterministic pre-audit is authoritative and cannot be erased.
3393 - Factual and interpretive authority, in order, is the Verified Fact Ledger,
3394 deterministic Evidence Ledger, deterministic profile support, verified
3395 Insight Ledger for exact bounded interpretations, properly scoped trusted
3396 external facts when allowed, and Data Understanding as non-authoritative
3397 context only.
3398 - Never validate one LLM-generated claim solely because another LLM output
3399 repeats it.
3400 - A claim that is exactly supported by deterministic profile data but missing
3401 from the sentence support map is a support-mapping defect. Do not call it a
3402 visible hallucination when the visible statement is correct.
3403 - Do not propose a visible rewrite when a deterministic hidden support-map
3404 patch fully resolves the problem.
3405 - The semantic Auditor may add supported annotations and repair candidates but
3406 must not erase deterministic findings.
3407
3408 A verified insight is an evidence-constrained interpretation, not permission
3409 to write any plausible related claim. Check that report wording is no stronger
3410 than the mapped insight. Do not call a supported bounded insight a
3411 hallucination merely because it is not a direct numeric fact.
3412
3413 Flag interpretations absent from the Insight Ledger, wording stronger than a
3414 verified insight, causal escalation, unsupported domain interpretation,
3415 unlabelled hypotheses, hypotheses presented as conclusions, and unsupported
3416 genre-specific narratives. Also flag qualitative strength wording that
3417 conflicts with the mapped deterministic strength label, such as calling the
3418 same verified association strong in one place and moderate in another. The
3419 Insight Ledger does not authorise new numbers, entities, recommendations,
3420 causality, generalisations or domain explanations. Do not use Data
3421 Understanding to validate an insight.
3422
3423 Perform two responsibilities.
3424
3425 A. Factual audit
3426 Detect:
3427 - incorrect numbers;
3428 - incorrect entities;
3429 - wrong direction or polarity;
3430 - unsupported synthesis;
3431 - causal overclaims;
3432 - predictive or deployment overclaims;
3433 - forecast overclaims;
3434 - unsupported metadata;
3435 - missing material caveats.
3436 - limitations that say supplied structure or evidence is unavailable/not
3437 captured when the evidence ledger shows it exists. In that case, repair to
3438 "not analysed in this report" or remove the limitation.
3439
3440 B. Targeted repair
3441 For each high-confidence repairable error:
3442 - produce several replacement candidates;
3443 - use only supplied facts and evidence;
3444 - favour factual support over style;
3445 - preserve useful meaning when possible;
3446 - use deletion only where no grounded replacement is suitable.
3447
3448 Do not rewrite unflagged portions of the report.
3449 Do not invent corrections.
3450 Do not use outside knowledge.
3451
3452 Also assess report quality separately:
3453 - request responsiveness;
3454 - finding selection;
3455 - coherence;
3456 - concision;

```

```

3457 - caveat integration;
3458 - data-science interpretation.
3459
3460 Quality weaknesses alone should normally be warnings, not factual blocks.
3461
3462 `quality_assessment.findings` must describe defects in the report's writing,
3463 selection, structure, interpretation or communication.
3464
3465 Valid examples:
3466 - The report overstates the implication of a constant field.
3467 - The report recommends duplicate removal without sufficient justification.
3468 - The report repeats closely related findings.
3469 - The report omits a required limitation.
3470
3471 Invalid examples:
3472 - The dataset contains duplicate rows.
3473 - Loud Cover is constant.
3474 - Pressure contains zero values.
3475 - Temperature and humidity are correlated.
3476
3477 Dataset observations belong in evidence or facts, not report-quality findings.
3478
3479 Apply these wording rules:
3480 - Do not accept hourly cadence unless regular spacing is verified.
3481 - Do not accept location or weather-station metadata unless verified.
3482 - Constant columns contain no observed variation for analyses that depend on
3483 variation; they are not universally worthless.
3484 - Suspicious zeros may represent missingness or measurement failure, but this
3485 must be validated before treating the interpretation as true.
3486 - Low missingness is not automatically harmless.
3487 - Duplicate rows should be reviewed before any decision to remove them.
3488 - Pearson correlation may not capture non-linear relationships and can be
3489 sensitive to influential observations.
3490 - Reader-facing next steps must be grounded in supplied recommendations,
3491 verified methodological facts or the explicit user request.
3492
3493 Internal-control leakage is a report-quality problem.
3494 Unsupported claims that group-size imbalance biases group means should be
3495 repaired to say unequal sizes can affect precision, stability, or
3496 representation unless the evidence explicitly supports bias language.
3497
3498 When a visible report contains:
3499 - Interpretation Notes;
3500 - Global Prohibited Interpretations;
3501 - Do not say...;
3502 - evidence-field labels;
3503
3504 propose a targeted natural-language repair or removal.
3505
3506 Do not classify this alone as a critical factual hallucination.
3507 Do not rewrite the complete report.
3508 """"
3509
3510
3511 def build_auditor_agent(settings: Settings) -> Agent:
3512     agent = Agent(
3513         build_model(
3514             settings.model_for("auditor"),
3515             settings,
3516         ),
3517         name="factual_accuracy_auditor_and_repair_agent",
3518         deps_type=AgentDependencies,
3519         output_type=output_schema(
3520             AuditRepairProposal,
3521             settings,
3522         ),
3523         instructions=AUDITOR_INSTRUCTIONS,
3524         model_settings=agent_model_settings(
3525             settings,
3526             "auditor",
3527             temperature=0.1,
3528             max_tokens=12_000,
3529         ),
3530         retries={"output": 3},
3531     )
3532
3533 @agent.output_validator
3534 def validate_output(
3535     context: RunContext[AgentDependencies],
3536     output: AuditRepairProposal,
3537 ) -> AuditRepairProposal:
3538     report_text = context.deps.payload["report_text"]
3539     valid_fact_ids = set(
3540         context.deps.payload["valid_fact_ids"]
3541     )
3542     valid_evidence_ids = set(
3543         context.deps.payload.get(
3544             "valid_evidence_ids",
3545             [],
3546         )
3547     )

```

```

3548     valid_profile_support_ids = set(
3549         context.deps.payload.get(
3550             "valid_profile_support_ids",
3551             [],
3552         )
3553     )
3554     valid_insight_ids = set(
3555         context.deps.payload.get(
3556             "valid_insight_ids",
3557             [],
3558         )
3559     )
3560     hypothesis_only_insight_ids = set(
3561         context.deps.payload.get(
3562             "hypothesis_only_insight_ids",
3563             [],
3564         )
3565     )
3566     all_insight_ids = (
3567         valid_insight_ids
3568         | hypothesis_only_insight_ids
3569     )
3570     insight_statements = dict(
3571         context.deps.payload.get(
3572             "insight_statements",
3573             {},
3574         )
3575     )
3576     sentence_insight_ids = {
3577         sentence: set(insight_ids)
3578         for sentence, insight_ids in context.deps.payload.get(
3579             "sentence_insight_ids",
3580             {},
3581         ).items()
3582     }
3583     allow_hypotheses = bool(
3584         context.deps.payload.get(
3585             "allow_hypotheses_in_report",
3586             False,
3587         )
3588     )
3589     repair_fact_ledger = (
3590         FactLedger.model_validate(
3591             context.deps.payload["fact_ledger"]
3592         )
3593         if "fact_ledger" in context.deps.payload
3594         else None
3595     )
3596     if "evidence_ledger" in context.deps.payload:
3597         from .schemas import EvidenceLedger
3598
3599         repair_evidence_ledger = EvidenceLedger.model_validate(
3600             context.deps.payload["evidence_ledger"]
3601         )
3602     else:
3603         repair_evidence_ledger = None
3604     repair_insight_ledger = InsightLedger.model_validate(
3605         context.deps.payload.get(
3606             "insight_ledger",
3607             {},
3608         )
3609     )
3610     deterministic_annotation_ids = set(
3611         context.deps.payload.get(
3612             "deterministic_annotation_ids",
3613             [],
3614         )
3615     )
3616     deterministic_serious_annotation_ids = set(
3617         context.deps.payload.get(
3618             "deterministic_serious_annotation_ids",
3619             [],
3620         )
3621     )
3622     deterministic_annotation_sentences = set(
3623         context.deps.payload.get(
3624             "deterministic_annotation_sentences",
3625             [],
3626         )
3627     )
3628
3629     annotation_ids = {
3630         annotation.annotation_id
3631         for annotation in output.annotations
3632     }
3633     all_annotation_ids = (
3634         annotation_ids
3635         | deterministic_annotation_ids
3636     )
3637     annotated_sentences = {
3638         annotation.sentence

```

```

3639         for annotation in output.annotations
3640     } | deterministic_annotation_sentences
3641
3642     for annotation in output.annotations:
3643         if annotation.sentence not in report_text:
3644             raise ModelRetry(
3645                 "Every annotated sentence must occur in the report."
3646             )
3647
3648         if (
3649             annotation.text_span
3650             and annotation.text_span not in annotation.sentence
3651         ):
3652             raise ModelRetry(
3653                 "Every text_span must occur in its sentence."
3654             )
3655
3656         unknown = set(annotation.fact_ids) - valid_fact_ids
3657
3658         if unknown:
3659             raise ModelRetry(
3660                 f"Unknown annotation fact IDs: {sorted(unknown)}"
3661             )
3662
3663         unknown_evidence = (
3664             set(annotation.evidence_ids)
3665             - valid_evidence_ids
3666         )
3667
3668         if unknown_evidence:
3669             raise ModelRetry(
3670                 "Unknown annotation evidence IDs: "
3671                 f"{sorted(unknown_evidence)}"
3672             )
3673
3674         unknown_profile_support = (
3675             set(annotation.profile_support_ids)
3676             - valid_profile_support_ids
3677         )
3678
3679         if unknown_profile_support:
3680             raise ModelRetry(
3681                 "Unknown annotation profile support IDs: "
3682                 f"{sorted(unknown_profile_support)}"
3683             )
3684
3685         unknown_insights = (
3686             set(annotation.insight_ids)
3687             - all_insight_ids
3688         )
3689         if unknown_insights:
3690             raise ModelRetry(
3691                 "Unknown annotation insight IDs: "
3692                 f"{sorted(unknown_insights)}"
3693             )
3694
3695         insight_subtype = annotation.subtype in {
3696             "unsupported_insight",
3697             "insight_exceeds_verified_wording",
3698             "insight_missing_source_support",
3699             "single_fact_relabelled_as_insight",
3700             "unlabelled_hypothesis",
3701             "hypothesis_presented_as_conclusion",
3702             "unsupported_causal_interpretation",
3703             "unsupported_domain_interpretation",
3704             "unsupported_sports_narrative",
3705             "genre_mismatch",
3706         }
3707         if (
3708             insight_subtype
3709             and not annotation.insight_ids
3710             and not sentence_insight_ids.get(annotation.sentence)
3711             and annotation.subtype
3712             not in {
3713                 "unsupported_insight",
3714                 "unlabelled_hypothesis",
3715                 "unsupported_sports_narrative",
3716                 "genre_mismatch",
3717             }
3718         ):
3719             raise ModelRetry(
3720                 "Insight-specific annotations must reference the mapped "
3721                 "insight where one exists."
3722             )
3723
3724     for repair in output.repairs:
3725         if repair.original_sentence not in report_text:
3726             raise ModelRetry(
3727                 "Every repair original_sentence must occur in the report."
3728             )
3729

```

```

3730         unknown_annotations = (
3731             set(repair.annotation_ids)
3732             - all_annotation_ids
3733         )
3734
3735     if unknown_annotations:
3736         raise ModelRetry(
3737             "Repair references unknown annotation IDs."
3738         )
3739
3740     if repair.original_sentence not in annotated_sentences:
3741         raise ModelRetry(
3742             "A repair may target only a sentence with an annotation."
3743         )
3744
3745     for candidate in repair.candidates:
3746         if (
3747             repair_fact_ledger is not None
3748             and repair_evidence_ledger is not None
3749         ):
3750             repair_errors = validate_repair_candidate(
3751                 candidate,
3752                 repair_fact_ledger,
3753                 repair_evidence_ledger,
3754                 repair_insight_ledger,
3755                 allow_hypotheses,
3756                 original_text=(
3757                     repair.original_sentence
3758                 ),
3759             )
3760             if repair_errors:
3761                 raise ModelRetry(
3762                     "Repair candidate validation failed: "
3763                     + "; ".join(repair_errors)
3764                 )
3765
3766             unknown = (
3767                 set(candidate.supporting_fact_ids)
3768                 - valid_fact_ids
3769             )
3770
3771             if unknown:
3772                 raise ModelRetry(
3773                     f"Unknown repair fact IDs: {sorted(unknown)}"
3774                 )
3775
3776             unknown_evidence = (
3777                 set(candidate.supporting_evidence_ids)
3778                 - valid_evidence_ids
3779             )
3780
3781             if unknown_evidence:
3782                 raise ModelRetry(
3783                     "Unknown repair evidence IDs: "
3784                     f"{sorted(unknown_evidence)}"
3785                 )
3786
3787             unknown_insights = (
3788                 set(candidate.supporting_insight_ids)
3789                 - all_insight_ids
3790             )
3791             if unknown_insights:
3792                 raise ModelRetry(
3793                     "Unknown repair insight IDs: "
3794                     f"{sorted(unknown_insights)}"
3795                 )
3796
3797             if (
3798                 set(candidate.supporting_insight_ids)
3799                 & hypothesis_only_insight_ids
3800                 and not allow_hypotheses
3801             ):
3802                 raise ModelRetry(
3803                     "Repairs may not introduce hypotheses while the "
3804                     "feature is disabled."
3805                 )
3806
3807             for insight_id in candidate.supporting_insight_ids:
3808                 insight_statement = insight_statements.get(
3809                     insight_id,
3810                     "",
3811                 )
3812                 replacement = candidate.replacement_text
3813                 if (
3814                     replacement
3815                     and insight_statement
3816                     and CAUSAL_PATTERN.search(replacement)
3817                     and not CAUSAL_PATTERN.search(insight_statement)
3818                 ):
3819                     raise ModelRetry(
3820                         "A repair candidate strengthens a verified insight "

```



```

3821         "with causal wording."
3822     )
3823
3824     if (
3825         replacement
3826         and UNIVERSAL_GENERALISATION_PATTERN.search(replacement)
3827         and not UNIVERSAL_GENERALISATION_PATTERN.search(
3828             insight_statement
3829         )
3830     ):
3831         raise ModelRetry(
3832             "A repair candidate generalises beyond its verified "
3833             "insight."
3834         )
3835
3836     if output.recommended_decision == AuditDecision.BLOCK:
3837         semantic_serious = any(
3838             annotation.severity
3839             in {
3840                 Severity.HIGH,
3841                 Severity.CRITICAL,
3842             }
3843             for annotation in output.annotations
3844         )
3845
3846     if (
3847         not semantic_serious
3848         and not deterministic_serious_annotation_ids
3849     ):
3850         raise ModelRetry(
3851             "recommended_decision=BLOCK requires at least one "
3852             "high or critical deterministic or semantic annotation."
3853         )
3854
3855     invalid_quality_findings = [
3856         finding
3857         for finding in output.quality_assessment.findings
3858         if not valid_quality_finding(finding)
3859     ]
3860
3861     if invalid_quality_findings:
3862         raise ModelRetry(
3863             "Quality findings must describe report defects, not plain "
3864             "dataset observations: "
3865             + "; ".join(invalid_quality_findings)
3866         )
3867
3868     return output
3869
3870     return agent
3871
3872 def fallback_understanding(
3873     profile: DataProfile,
3874 ) -> DataUnderstanding:
3875     tables: list[TableUnderstanding] = []
3876     supported_routes = {
3877         AnalysisRoute.DESCRPTIVE
3878     }
3879
3880     for table in profile.tables:
3881         numeric = [
3882             column
3883             for column in table.columns
3884             if column.semantic_type == "numeric"
3885             and not column.constant
3886         ]
3887         categorical = [
3888             column
3889             for column in table.columns
3890             if column.semantic_type == "categorical"
3891         ]
3892         datetime = [
3893             column
3894             for column in table.columns
3895             if column.semantic_type == "datetime"
3896         ]
3897
3898     if len(numeric) >= 2 or (numeric and categorical):
3899         supported_routes.add(
3900             AnalysisRoute.ASSOCIATION_COMPARISON
3901         )
3902
3903     if table.row_count >= 100 and numeric:
3904         supported_routes.add(
3905             AnalysisRoute.PREDICTIVE
3906         )
3907
3908     if table.row_count >= 40 and numeric and datetime:
3909         supported_routes.add(
3910             AnalysisRoute.FORECASTING
3911

```

```

3912     )
3913
3914     supported_routes.add(
3915         AnalysisRoute.CAUSAL_FEASIBILITY
3916     )
3917
3918     if table.candidate_keys:
3919         unit = (
3920             f"one row per unique `{table.candidate_keys[0]}` value"
3921         )
3922     else:
3923         unit = (
3924             f"one row per observed record in `{table.table_name}`"
3925         )
3926
3927     meanings = [
3928         ColumnMeaning(
3929             table_name=table.table_name,
3930             column_name=column.name,
3931             inferred_role=(
3932                 "candidate_identifier"
3933                 if column.candidate_key
3934                 else column.semantic_type
3935             ),
3936             interpretation=(
3937                 f"Observed `{column.name}` column with provisional "
3938                 f"`{column.semantic_type}` analytical role."
3939             ),
3940             evidence_basis=(
3941                 f"dtype={column.dtype}; unique={column.unique_count}; "
3942                 f"missing_rate={column.missing_rate:.2%}"
3943             ),
3944             confidence=(
3945                 0.95 if column.candidate_key else 0.70
3946             ),
3947             caveat=(
3948                 "The scientific or business meaning is not confirmed "
3949                 "by the data alone."
3950             ),
3951         )
3952         for column in table.columns
3953     ]
3954
3955     risks: list[ColumnRisk] = []
3956
3957     for column in table.columns:
3958         if column.constant:
3959             risks.append(
3960                 ColumnRisk(
3961                     table_name=table.table_name,
3962                     column_name=column.name,
3963                     risk_type="constant_column",
3964                     explanation=(
3965                         "The column has one observed non-missing value."
3966                     ),
3967                     analytical_consequence=(
3968                         "Exclude it from correlation, comparison, "
3969                         "prediction, and forecasting."
3970                     ),
3971                     confidence=1.0,
3972                 )
3973             )
3974
3975         if column.suspicious_zero_values:
3976             risks.append(
3977                 ColumnRisk(
3978                     table_name=table.table_name,
3979                     column_name=column.name,
3980                     risk_type="possible_sentinel_zero",
3981                     explanation=(
3982                         "Zero observations are separated from most of "
3983                         "the positive distribution."
3984                     ),
3985                     analytical_consequence=(
3986                         "Validate the zeros before relying on analyses "
3987                         "using this field."
3988                     ),
3989                     confidence=0.85,
3990                 )
3991             )
3992
3993     tables.append(
3994         TableUnderstanding(
3995             table_name=table.table_name,
3996             unit_of_observation=unit,
3997             summary=(
3998                 f"`{table.table_name}` has {table.row_count:,} rows "
3999                 f"and {table.column_count:,} columns."
4000             ),
4001             likely_keys=table.candidate_keys,
4002             column_meanings=meanings,

```

```

4003         column_risks=risks,
4004         quality_findings=table.warnings,
4005         usability_notes=[
4006             "Constant columns should not enter analytical models.",
4007             "Predictive targets require user, metadata, or explicit "
4008             "experimental confirmation.",
4009             "Temporal data should use chronological validation.",
4010             "Causal conclusions require a defensible identification design.",
4011         ],
4012     )
4013 )
4014
4015 return DataUnderstanding(
4016     profile_fingerprint=profile.fingerprint,
4017     dataset_summary=(
4018         f"The input contains {len(profile.tables)} profiled table(s). "
4019         "Semantic interpretations remain provisional without metadata."
4020     ),
4021     tables=tables,
4022     cross_table_notes=(
4023         [
4024             "Multiple tables were supplied. Joins are not assumed "
4025             "without explicit key evidence."
4026         ]
4027         if len(profile.tables) > 1
4028         else []
4029     ),
4030     supported_routes=sorted(
4031         supported_routes,
4032         key=lambda route: route.value,
4033     ),
4034     uncertain_routes=[],
4035     global_caveats=[
4036         "The data alone do not confirm provenance or domain semantics.",
4037         "Sampling, missingness, measurement, and temporal limitations "
4038         "may affect findings.",
4039     ],
4040 )
4041
4042
4043 def select_explicit_target(
4044     request: str,
4045     table: Any,
4046 ) -> str | None:
4047     for column in table.columns:
4048         if re.search(
4049             rf"\b{re.escape(column.name.lower())}\b",
4050             request.lower(),
4051         ):
4052             return column.name
4053
4054     return None
4055
4056
4057 def event_report_requested(
4058     request: str,
4059 ) -> bool:
4060     return bool(
4061         re.search(
4062             r"\b(write|create|produce|give|prepare)?\s*"
4063             r"(:a\s+)?(?:event|sports|game|match)\s+report\b|"
4064             r"\bwrite up (?:the|this) (?:event|game|match)\b",
4065             request,
4066             re.IGNORECASE,
4067         )
4068     )
4069
4070
4071 def sports_game_report_requested(
4072     request: str,
4073 ) -> bool:
4074     return event_report_requested(request)
4075
4076
4077 def profile_supports_sports_game_report(
4078     profile: DataProfile,
4079 ) -> bool:
4080     names = {
4081         column.name.lower()
4082         for table in profile.tables
4083         for column in table.columns
4084     }
4085     table_names = {
4086         table.table_name.lower()
4087         for table in profile.tables
4088     }
4089
4090     has_subject = any(
4091         token in name
4092         for name in names
4093         for token in {"team", "player", "opponent"}

```

```

4094     )
4095     has_result = any(
4096         token in name
4097         for name in names
4098         for token in {"score", "points", "winner", "result"}
4099     )
4100     game_named = any(
4101         token in name
4102         for name in table_names
4103         for token in {"game", "match", "boxscore", "box_score"}
4104     )
4105
4106     return has_subject and has_result and game_named
4107
4108 def fallback_insight_objectives(
4109     *,
4110     tasks: list[InvestigationTask],
4111     genre: ReportGenre,
4112     enabled: bool,
4113 ) -> list[InsightObjective]:
4114     if not enabled:
4115         return []
4116
4117     task_ids = [task.task_id for task in tasks]
4118
4119     if genre in {
4120         ReportGenre.EVENT_REPORT,
4121         ReportGenre.SPORTS_GAME_REPORT,
4122     }:
4123         questions = [
4124             (
4125                 "What verified facts best describe the result without "
4126                 "implying unsupported causality?",
4127                 [InsightType.NARRATIVE_SUMMARY],
4128             ),
4129             (
4130                 "Which verified performances are most salient to the game "
4131                 "report?",
4132                 [InsightType.DOMINANT_PATTERN, InsightType.CONTRAST],
4133             ),
4134             (
4135                 "Which verified team-level contrasts define the bounded game "
4136                 "narrative?",
4137                 [InsightType.CONTRAST, InsightType.NARRATIVE_SUMMARY],
4138             ),
4139             (
4140                 "Are any conventional milestones explicitly supported by the "
4141                 "verified facts?",
4142                 [InsightType.NARRATIVE_SUMMARY],
4143             ),
4144         ]
4145     else:
4146         questions = [
4147             (
4148                 "Which verified findings jointly describe the strongest "
4149                 "structure in the data?",
4150                 [InsightType.DOMINANT_PATTERN, InsightType.NARRATIVE_SUMMARY],
4151             ),
4152             (
4153                 "What is the strongest non-redundant verified contrast?",
4154                 [InsightType.CONTRAST],
4155             ),
4156             (
4157                 "Do any verified variables contain substantially overlapping "
4158                 "information in this dataset?",
4159                 [InsightType.REDUNDANCY, InsightType.OUTCOME_ASSOCIATION],
4160             ),
4161             (
4162                 "Which verified data-quality issue most affects interpretation?",
4163                 [InsightType.DATA_QUALITY_IMPLICATION, InsightType.ANOMALY],
4164             ),
4165             (
4166                 "What bounded message should the reader remember from the "
4167                 "verified findings?",
4168                 [InsightType.NARRATIVE_SUMMARY],
4169             ),
4170         ]
4171
4172     return [
4173         InsightObjective(
4174             objective_id=f"INSIGHT_OBJECTIVE_{index:03d}",
4175             question=question,
4176             preferred_insight_types=insight_types,
4177             relevant_task_ids=task_ids,
4178         )
4179         for index, (question, insight_types) in enumerate(
4180             questions,
4181             start=1,
4182         )
4183     ]
4184

```

```

4185
4186
4187 def fallback_execution_plan(
4188     request: str,
4189     profile: DataProfile,
4190     audit_mode: AuditMode,
4191     settings: Settings,
4192     *,
4193     input_structure: InputStructureProfile | None = None,
4194     available_capabilities: list[EvidenceCapability] | None = None,
4195     report_genre_override: ReportGenre | None = None,
4196 ) -> ExecutionPlan:
4197     tasks: list[InvestigationTask] = []
4198     available_capabilities = available_capabilities or []
4199     explicit_event_request = event_report_requested(request)
4200     explicit_data_science_request = bool(
4201         re.search(
4202             r"\b(data[- ]science report|statistical analysis)\b",
4203             request,
4204             re.IGNORECASE,
4205         )
4206     )
4207     structured_event = bool(
4208         input_structure is not None
4209         and input_structure.shape == InputShape.EVENT_RECORD
4210         and input_structure.confidence >= 0.7
4211     )
4212     report_genre = (
4213         ReportGenre.EVENT_REPORT
4214         if explicit_event_request
4215         else (
4216             ReportGenre.DATA_SCIENCE_REPORT
4217             if explicit_data_science_request
4218             else (
4219                 report_genre_override
4220                 if report_genre_override is not None
4221                 else (
4222                     ReportGenre.EVENT_REPORT
4223                     if structured_event
4224                     else (
4225                         ReportGenre.DATASET_OVERVIEW
4226                         if re.search(
4227                             r"\bdataset overview\b",
4228                             request,
4229                             re.IGNORECASE,
4230                         )
4231                         else ReportGenre.DATA_SCIENCE_REPORT
4232                     )
4233                 )
4234             )
4235         )
4236     )
4237
4238     if explicit_event_request or explicit_data_science_request:
4239         selection_source = ReportSelectionSource.EXPLICIT_USER_REQUEST
4240     elif report_genre_override is not None:
4241         selection_source = ReportSelectionSource.EXPERIMENT_CONFIGURATION
4242     elif structured_event:
4243         selection_source = ReportSelectionSource.STRUCTURED_INFERENCE
4244     else:
4245         selection_source = ReportSelectionSource.FALLBACK
4246
4247     if report_genre in {
4248         ReportGenre.EVENT_REPORT,
4249         ReportGenre.SPORTS_GAME_REPORT,
4250     } and profile.tables:
4251         event_table = profile.tables[0]
4252         event_task_specs = [
4253             (
4254                 EvidenceCapability.EVENT_OUTCOME,
4255                 AnalysisRoute.DESCRPTIVE,
4256                 "What is the verified event result, context, status and score progression?",
4257                 [
4258                     "event_outcome",
4259                     "event_context",
4260                     "event_status",
4261                     "participant_record_context",
4262                     "score_progression",
4263                 ],
4264             ),
4265             (
4266                 EvidenceCapability.ENTITY_PERFORMANCE,
4267                 AnalysisRoute.DESCRPTIVE,
4268                 "Which recorded entity performances are most salient?",
4269                 ["entity_performance"],
4270             ),
4271             (
4272                 EvidenceCapability.RANKING,
4273                 AnalysisRoute.DESCRPTIVE,
4274                 "Which entities lead the recorded performance rankings?",
4275                 ["entity_ranking"],

```

```

4276         ),
4277         (
4278             EvidenceCapability.GROUP_COMPARISON,
4279             AnalysisRoute.ASSOCIATION_COMPARISON,
4280             "What are the strongest participant-level contrasts?",
4281             ["participant_comparison"],
4282         ),
4283     ]
4284     for capability, route, question, evidence_types in event_task_specs:
4285         if capability not in available_capabilities:
4286             continue
4287         tasks.append(
4288             InvestigationTask(
4289                 task_id=f"TASK_{len(tasks) + 1:03d}",
4290                 question=question,
4291                 route=route,
4292                 priority=5 if capability == EvidenceCapability.EVENT_OUTCOME else 4,
4293                 table_name=event_table.table_name,
4294                 columns=[column.name for column in event_table.columns],
4295                 capability=capability,
4296                 input_fields=[column.name for column in event_table.columns],
4297                 entity_scope=input_structure.entity_levels if input_structure else [],
4298                 expected_evidence_types=evidence_types,
4299                 required_evidence=evidence_types,
4300                 claim_permissions=[
4301                     ClaimPermission.DESRIPTIVE,
4302                     ClaimPermission.COMPARATIVE,
4303                 ],
4304                 answerability_note=("Answerable from verified structured-event evidence."),
4305             )
4306         )
4307
4308     predictive_requested = bool(
4309         re.search(
4310             r"\b(predict|prediction|model|classify|estimate)\b",
4311             request,
4312             re.IGNORECASE,
4313         )
4314     )
4315     forecast_requested = bool(
4316         re.search(
4317             r"\b(forecast|future|time series|ahead)\b",
4318             request,
4319             re.IGNORECASE,
4320         )
4321     )
4322     causal_requested = bool(
4323         re.search(
4324             r"\b(cause|causal|effect|impact|intervention)\b",
4325             request,
4326             re.IGNORECASE,
4327         )
4328     )
4329
4330     for table in profile.tables:
4331         if report_genre in {
4332             ReportGenre.EVENT_REPORT,
4333             ReportGenre.SPORTS_GAME_REPORT,
4334         }:
4335             continue
4336
4337         tasks.append(
4338             InvestigationTask(
4339                 task_id=f"TASK_{len(tasks) + 1:03d}",
4340                 question=(
4341                     f"What are the structure, data quality, distributions, "
4342                     f"and analytically important fields in `{table.table_name}`?"
4343                 ),
4344                 route=AnalysisRoute.DESRIPTIVE,
4345                 priority=5,
4346                 table_name=table.table_name,
4347                 columns=[
4348                     column.name
4349                     for column in table.columns
4350                 ],
4351                 required_evidence=[
4352                     "dimensions",
4353                     "missingness",
4354                     "distribution diagnostics",
4355                     "constant columns",
4356                     "possible sentinel values",
4357                 ],
4358                 claim_permissions=[
4359                     ClaimPermission.DESRIPTIVE,
4360                     ClaimPermission.METHODOLOGICAL,
4361                     ClaimPermission.INSUFFICIENCY,
4362                 ],
4363                 answerability_note=(
4364                     "Directly answerable through deterministic profiling."
4365                 ),
4366             )

```

```

4367     )
4368
4369     numeric = [
4370         column.name
4371         for column in table.columns
4372         if column.semantic_type == "numeric"
4373         and not column.constant
4374     ]
4375     categorical = [
4376         column.name
4377         for column in table.columns
4378         if column.semantic_type == "categorical"
4379     ]
4380     datetime_columns = [
4381         column.name
4382         for column in table.columns
4383         if column.semantic_type == "datetime"
4384     ]
4385
4386     if len(numeric) >= 2 or (numeric and categorical):
4387         tasks.append(
4388             InvestigationTask(
4389                 task_id=f"TASK_{len(tasks) + 1:03d}",
4390                 question=(
4391                     f"Which substantively meaningful associations and group "
4392                     f"differences are present in `{table.table_name}`?"
4393                 ),
4394                 route=AnalysisRoute.ASSOCIATION_COMPARISON,
4395                 priority=4,
4396                 table_name=table.table_name,
4397                 columns=(numeric + categorical)[:20],
4398                 required_evidence=[
4399                     "effect magnitude",
4400                     "group counts",
4401                     "sampling method",
4402                     "association caveats",
4403                 ],
4404                 claim_permissions=[
4405                     ClaimPermission.ASSOCIATIONAL,
4406                     ClaimPermission.COMPARATIVE,
4407                     ClaimPermission.METHODOLOGICAL,
4408                     ClaimPermission.INSUFFICIENCY,
4409                 ],
4410                 answerability_note=(
4411                     "Answerable as observed association and comparison, not causation."
4412                 ),
4413             )
4414         )
4415
4416     target = select_explicit_target(
4417         request,
4418         table,
4419     )
4420
4421     if predictive_requested and target:
4422         tasks.append(
4423             InvestigationTask(
4424                 task_id=f"TASK_{len(tasks) + 1:03d}",
4425                 question=(
4426                     f"Can `{target}` be predicted better than a simple baseline "
4427                     f"after leakage screening?"
4428                 ),
4429                 route=AnalysisRoute.PREDICTIVE,
4430                 priority=3,
4431                 table_name=table.table_name,
4432                 target_column=target,
4433                 target_status=TargetStatus.USER_SELECTED,
4434                 prediction_definition=(
4435                     f"Estimate `{target}` using fields available in the supplied table."
4436                 ),
4437                 time_column=(
4438                     datetime_columns[0]
4439                     if datetime_columns
4440                     else None
4441                 ),
4442                 validation_strategy=(
4443                     ValidationStrategy.CHRONOLOGICAL_HOLDOUT
4444                     if datetime_columns
4445                     else ValidationStrategy.RANDOM_HOLDOUT
4446                 ),
4447                 required_evidence=[
4448                     "target confirmation",
4449                     "proxy leakage audit",
4450                     "feature exclusions",
4451                     "baseline comparison",
4452                     "holdout metrics",
4453                 ],
4454                 claim_permissions=[
4455                     ClaimPermission.PREDICTIVE,
4456                     ClaimPermission.METHODOLOGICAL,
4457                     ClaimPermission.INSUFFICIENCY,

```



```

4458         ],
4459         answerability_note=(
4460             "A positive result requires leakage-audited baseline improvement."
4461         ),
4462     )
4463 )
4464
4465 if forecast_requested and target and datetime_columns:
4466     tasks.append(
4467         InvestigationTask(
4468             task_id=f"TASK_{len(tasks) + 1:03d}",
4469             question=(
4470                 f"Can `{target}` be forecast using rolling temporal "
4471                 "evaluation and seasonal baselines?"
4472             ),
4473             route=AnalysisRoute.FORECASTING,
4474             priority=3,
4475             table_name=table.table_name,
4476             target_column=target,
4477             target_status=TargetStatus.USER_SELECTED,
4478             time_column=datetime_columns[0],
4479             validation_strategy=ValidationStrategy.ROLLING_ORIGIN,
4480             required_evidence=[
4481                 "time ordering",
4482                 "rolling folds",
4483                 "last-value baseline",
4484                 "seasonal-naive baselines",
4485                 "candidate-model metrics",
4486             ],
4487             claim_permissions=[
4488                 ClaimPermission.FORECAST,
4489                 ClaimPermission.METHODOLOGICAL,
4490                 ClaimPermission.INSUFFICIENCY,
4491             ],
4492             answerability_note=(
4493                 "A positive forecast claim requires consistent improvement "
4494                 "over the strongest relevant naive baseline."
4495             ),
4496         )
4497     )
4498
4499 if causal_requested:
4500     tasks.append(
4501         InvestigationTask(
4502             task_id=f"TASK_{len(tasks) + 1:03d}",
4503             question=(
4504                 f"Does `{table.table_name}` support a defensible "
4505                 "causal identification strategy?"
4506             ),
4507             route=AnalysisRoute.CAUSAL_FEASIBILITY,
4508             priority=2,
4509             table_name=table.table_name,
4510             required_evidence=[
4511                 "exposure",
4512                 "outcome",
4513                 "time ordering",
4514                 "confounders",
4515                 "identification design",
4516             ],
4517             claim_permissions=[
4518                 ClaimPermission.CAUSAL,
4519                 ClaimPermission.METHODOLOGICAL,
4520                 ClaimPermission.INSUFFICIENCY,
4521             ],
4522             answerability_note=(
4523                 "The likely output is a causal-feasibility or insufficiency finding."
4524             ),
4525         )
4526     )
4527
4528 route_order = [
4529     route
4530     for route in [
4531         AnalysisRoute.DESRIPTIVE,
4532         AnalysisRoute.ASSOCIATION_COMPARISON,
4533         AnalysisRoute.PREDICTIVE,
4534         AnalysisRoute.FORECASTING,
4535         AnalysisRoute.CAUSAL_FEASIBILITY,
4536     ]
4537     if any(task.route == route for task in tasks)
4538 ]
4539
4540 return ExecutionPlan(
4541     objective=request,
4542     tasks=tasks[:10],
4543     route_order=route_order,
4544     report_specification=ReportSpecification(
4545         report_purpose=request,
4546         genre=report_genre,
4547         perspective=ReportPerspective.NEUTRAL,
4548         communication_goal=(

```

```

4549         "Explain the verified result, leading performances and "
4550         "major team contrasts."
4551         if report_genre
4552         in {
4553             ReportGenre.EVENT_REPORT,
4554             ReportGenre.SPORTS_GAME_REPORT,
4555         }
4556         else (
4557             "Describe the table structure and strongest supported "
4558             "findings concisely."
4559             if report_genre == ReportGenre.DATASET_OVERVIEW
4560             else "Summarise the strongest supported findings."
4561         )
4562     ),
4563     target_length_words=(
4564         settings.writer_target_words
4565     ),
4566     maximum_length_words=(
4567         settings.writer_max_words
4568     ),
4569     maximum_main_findings=settings.writer_max_main_findings,
4570     maximum_supporting_facts=(
4571         settings.writer_supporting_fact_limit
4572     ),
4573     preferred_sections=(
4574         [
4575             "Event overview",
4576             "Score progression",
4577             "Key performances",
4578             "Participant contrasts",
4579             "Scope limitations",
4580         ]
4581         if report_genre
4582         in {
4583             ReportGenre.EVENT_REPORT,
4584             ReportGenre.SPORTS_GAME_REPORT,
4585         }
4586         else [
4587             "Overview and data quality",
4588             "Strongest observed relationships",
4589             "Modelling and validation",
4590             "Limitations and next steps",
4591         ]
4592     ),
4593     required_components=(
4594         []
4595         if report_genre
4596         in {
4597             ReportGenre.EVENT_REPORT,
4598             ReportGenre.SPORTS_GAME_REPORT,
4599         }
4600         else [
4601             ReportComponent.DATASET_OVERVIEW,
4602             ReportComponent.DATA_QUALITY,
4603             ReportComponent.STRONGEST_RELATIONSHIPS,
4604             ReportComponent.LIMITATIONS_NEXT_STEPS,
4605         ]
4606     ),
4607     required_content_slots=(
4608         [
4609             "event_result",
4610             "event_context",
4611             "participant_record_context",
4612             "event_status",
4613             "score_progression",
4614             "event_sequence",
4615             "leading_performance",
4616             "main_contrast",
4617             "scope_limitations",
4618         ]
4619         if report_genre
4620         in {
4621             ReportGenre.EVENT_REPORT,
4622             ReportGenre.SPORTS_GAME_REPORT,
4623         }
4624         else [
4625             "dataset_scope",
4626             "material_data_quality_issue",
4627             "strongest_analytical_finding",
4628             "limitation",
4629         ]
4630     ),
4631     optional_content_slots=(
4632         [
4633             "secondary_performance",
4634         ]
4635         if report_genre
4636         in {
4637             ReportGenre.EVENT_REPORT,
4638             ReportGenre.SPORTS_GAME_REPORT,
4639         }

```

```

4640         else []
4641     ),
4642     prohibited_claim_types=(
4643         [
4644             "unsupported_chronology",
4645             "unsupported_milestone",
4646             "unsupported_historical_significance",
4647             "unsupported_causality",
4648         ]
4649         if report_genre
4650         in {
4651             ReportGenre.EVENT_REPORT,
4652             ReportGenre.SPORTS_GAME_REPORT,
4653         }
4654         else ["unsupported_causality"]
4655     ),
4656     selection_source=selection_source,
4657     selection_confidence=(
4658         1.0
4659         if selection_source
4660         in {
4661             ReportSelectionSource.EXPLICIT_USER_REQUEST,
4662             ReportSelectionSource.EXPERIMENT_CONFIGURATION,
4663         }
4664         else 0.8
4665     ),
4666     include_negative_findings=(
4667         report_genre
4668         not in {
4669             ReportGenre.EVENT_REPORT,
4670             ReportGenre.SPORTS_GAME_REPORT,
4671         }
4672     ),
4673     include_methodological_details=(
4674         report_genre
4675         not in {
4676             ReportGenre.EVENT_REPORT,
4677             ReportGenre.SPORTS_GAME_REPORT,
4678         }
4679     ),
4680     prioritisation_rule=(
4681         "Prefer high-confidence, methodologically strong, user-relevant "
4682         "findings. Omit negligible effects and repetitive metadata."
4683     ),
4684 ),
4685 audit_mode=audit_mode,
4686 insight_objectives=fallback_insight_objectives(
4687     tasks=tasks[:10],
4688     genre=report_genre,
4689     enabled=settings.enable_insight_synthesis,
4690 ),
4691 available_capabilities=available_capabilities,
4692 selected_capabilities=[
4693     capability
4694     for capability in available_capabilities
4695     if (
4696         report_genre
4697         not in {
4698             ReportGenre.EVENT_REPORT,
4699             ReportGenre.SPORTS_GAME_REPORT,
4700         }
4701         or capability
4702         in {
4703             EvidenceCapability.DATASET_PROFILE,
4704             EvidenceCapability.EVENT_OUTCOME,
4705             EvidenceCapability.ENTITY_PERFORMANCE,
4706             EvidenceCapability.RANKING,
4707             EvidenceCapability.GROUP_COMPARISON,
4708         }
4709     )
4710 ],
4711 revision_limit=settings.max_revision_rounds,
4712 maximum_facts=None,
4713 frozen=True,
4714 rationale=(
4715     "The deterministic fallback plans descriptive and meaningful "
4716     "association work by default. Prediction, forecasting, and causal "
4717     "routes are added only when requested and sufficiently specified."
4718 ),
4719 )

```

Listing 21: P21: table2text_pydanticai/src/table2text/analytics.py

```

1  """Execute deterministic analytical tasks and produce typed evidence items."""
2
3  from __future__ import annotations
4
5  import math
6  import re

```

```

7 from itertools import combinations
8 from typing import Any
9
10 import numpy as np
11 import pandas as pd
12
13 from sklearn.compose import ColumnTransformer
14 from sklearn.dummy import DummyClassifier, DummyRegressor
15 from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
16 from sklearn.impute import SimpleImputer
17 from sklearn.linear_model import LinearRegression, LogisticRegression, Ridge
18 from sklearn.metrics import (
19     accuracy_score,
20     f1_score,
21     mean_absolute_error,
22     mean_squared_error,
23     r2_score,
24 )
25 from sklearn.model_selection import train_test_split
26 from sklearn.pipeline import Pipeline
27 from sklearn.preprocessing import OneHotEncoder, StandardScaler
28
29 from .capabilities import event_capability_evidence, semantic_query_evidence
30 from .config import Settings
31 from .data import DataBundle, classify_zero_risk, safe_hashable
32 from .schemas import (
33     AnalyticalFunction,
34     AnalysisRoute,
35     AnalyticalRecommendation,
36     ClaimPermission,
37     EvidenceCapability,
38     EvidenceItem,
39     EvidenceLedger,
40     ExecutionPlan,
41     InputSemanticMap,
42     InputShape,
43     InvestigationTask,
44     RecommendedUse,
45     ReportGenre,
46     SemanticLevel,
47     TargetStatus,
48     ValidationStrategy,
49     ZeroRisk,
50 )
51
52
53 def is_numeric_measure(series: pd.Series) -> bool:
54     return (
55         pd.api.types.is_numeric_dtype(series)
56         and not pd.api.types.is_bool_dtype(series)
57     )
58
59
60 def infer_evidence_capability(
61     route: AnalysisRoute,
62     metrics: dict[str, Any],
63 ) -> EvidenceCapability:
64     if route == AnalysisRoute.ASSOCIATION_COMPARISON:
65         if any(
66             key in metrics
67             for key in {"pearson_r", "spearman_r", "correlation"}
68         ):
69             return EvidenceCapability.ASSOCIATION
70         return EvidenceCapability.GROUP_COMPARISON
71
72     if route == AnalysisRoute.DESCRPTIVE:
73         if "duplicate_row_count" in metrics:
74             return EvidenceCapability.DUPLICATES
75         if "missing_count" in metrics or "missing_rate" in metrics:
76             return EvidenceCapability.MISSINGNESS
77         if "row_count" in metrics and "column_count" in metrics:
78             return EvidenceCapability.DATASET_PROFILE
79         return EvidenceCapability.DISTRIBUTION_SUMMARY
80
81     return EvidenceCapability.DATASET_PROFILE
82
83
84 class EvidenceBuilder:
85     def __init__(self, fingerprint: str):
86         self.fingerprint = fingerprint
87         self.items: list[EvidenceItem] = []
88         self.execution_notes: list[str] = []
89
90     def add(
91         self,
92         *,
93         route: AnalysisRoute,
94         task_ids: list[str],
95         finding: str,
96         metrics: dict[str, Any],
97         source_tables: list[str],

```

```

98     source_columns: list[str],
99     method: str,
100     practical_interpretation: str,
101     strength_label: str,
102     claim_permissions: list[ClaimPermission],
103     factual_confidence: float,
104     methodological_strength: float,
105     user_relevance: float,
106     salience: float,
107     recommended_use: RecommendedUse,
108     validation_strategy: ValidationStrategy = ValidationStrategy.NONE,
109     limitations: list[str] | None = None,
110     prohibited_interpretations: list[str] | None = None,
111     recommendations: list[AnalyticalRecommendation] | None = None,
112     eligible_for_writer: bool = True,
113     exclusion_reason: str | None = None,
114     capability: EvidenceCapability | None = None,
115     evidence_type: str | None = None,
116     source_paths: list[str] | None = None,
117     entity_scope: list[str] | None = None,
118     semantic_level: SemanticLevel = SemanticLevel.DATASET,
119     semantic_binding_ids: list[str] | None = None,
120     analytical_function: AnalyticalFunction | None = None,
121     query_id: str | None = None,
122 ) -> None:
123     evidence_id = f"EVD_{len(self.items) + 1:04d}"
124
125     item = EvidenceItem(
126         evidence_id=evidence_id,
127         route=route,
128         task_ids=task_ids,
129         capability=(
130             capability
131             or infer_evidence_capability(route, metrics)
132         ),
133         evidence_type=evidence_type or strength_label,
134         source_paths=source_paths or [],
135         entity_scope=entity_scope or [],
136         semantic_level=semantic_level,
137         semantic_binding_ids=semantic_binding_ids or [],
138         analytical_function=analytical_function,
139         query_id=query_id,
140         finding=finding,
141         metrics=metrics,
142         source_tables=source_tables,
143         source_columns=source_columns,
144         method=method,
145         validation_strategy=validation_strategy,
146         practical_interpretation=practical_interpretation,
147         strength_label=strength_label,
148         limitations=limitations or [],
149         prohibited_interpretations=prohibited_interpretations or [],
150         recommendations=recommendations or [],
151         claim_permissions=claim_permissions,
152         factual_confidence=factual_confidence,
153         methodological_strength=methodological_strength,
154         user_relevance=user_relevance,
155         salience=salience,
156         recommended_use=recommended_use,
157         eligible_for_writer=eligible_for_writer,
158         exclusion_reason=exclusion_reason,
159     )
160     item.metrics["priority_score"] = evidence_priority_score(item)
161     self.items.append(item)
162
163     def build(self) -> EvidenceLedger:
164         return EvidenceLedger(
165             fingerprint=self.fingerprint,
166             items=self.items,
167             execution_notes=self.execution_notes,
168         )
169
170     def tasks_for_route(
171         plan: ExecutionPlan,
172         route: AnalysisRoute,
173     ) -> list[InvestigationTask]:
174         return [task for task in plan.tasks if task.route == route]
175
176     def _cell_text(value: Any) -> str:
177         if value is None:
178             return ""
179         if isinstance(value, float) and math.isnan(value):
180             return ""
181         return str(value).strip()
182
183     def _cell_int(value: Any, fallback: int = 0) -> int:
184         if value is None:
185             return fallback
186

```

```

189     if isinstance(value, float) and math.isnan(value):
190         return fallback
191     try:
192         return int(value)
193     except (TypeError, ValueError):
194         return fallback
195
196
197 def _dedupe_nonempty(values: list[Any]) -> list[str]:
198     return list(
199         dict.fromkeys(
200             text
201             for value in values
202             if (text := _cell_text(value))
203         )
204     )
205
206
207 PLACEHOLDER_CELL_VALUES = {
208     "",
209     "-",
210     "--",
211     "-.",
212     "-.",
213     "n/a",
214     "na",
215     "none",
216     "null",
217     "unknown",
218     "not applicable",
219 }
220
221
222 FOCUSED_NUMBER_PATTERN = re.compile(
223     r"\d{1,3}(?:,\d{3})+(?:\.\d+)?%?|\d+(?:\.\d+)?%?"
224 )
225
226
227 def _is_placeholder_cell(value: Any) -> bool:
228     text = _cell_text(value)
229     return text.casefold() in PLACEHOLDER_CELL_VALUES
230
231
232 def _dedupe_meaningful(values: list[Any]) -> list[str]:
233     return list(
234         dict.fromkeys(
235             text
236             for value in values
237             if (text := _cell_text(value))
238             and not _is_placeholder_cell(text)
239         )
240     )
241
242
243 def _natural_join(values: list[str]) -> str:
244     if len(values) <= 1:
245         return "".join(values)
246     if len(values) == 2:
247         return f"{values[0]} and {values[1]}"
248     return ", ".join(values[:-1]) + f", and {values[-1]}"
249
250
251 def _contains_letter(value: str) -> bool:
252     return bool(re.search(r"[A-Za-z]", value))
253
254
255 def _looks_like_measure_value(value: str) -> bool:
256     text = value.strip()
257     return bool(
258         re.fullmatch(r"(?i)\d+(?:st|nd|rd|th)", text)
259         or text.endswith("%")
260         or re.fullmatch(
261             r"[\$£]? \d{1,3}(?:,\d{3})+(?:\.\d+)?|"
262             r"[\$£]? \d+(?:\.\d+)?",
263             text,
264         )
265     )
266
267
268 AGGREGATE_ROW_PATTERN = re.compile(
269     r"\b(total|subtotal|overall|sum|average|mean|median)\b",
270     re.IGNORECASE,
271 )
272
273
274 def _parse_numeric_cell_value(value: Any) -> float | None:
275     text = _cell_text(value)
276     if not text:
277         return None
278     cleaned = (
279         text.replace(",", "")

```

```

280         .replace("%", "")
281         .replace("$", "")
282         .replace("€", "")
283         .replace("£", "")
284         .strip()
285     )
286     if not re.fullmatch(r"-?\d+(?:\.\d+)?", cleaned):
287         return None
288     try:
289         return float(cleaned)
290     except ValueError:
291         return None
292
293
294 def _row_is_aggregate(row: pd.DataFrame) -> bool:
295     return any(
296         AGGREGATE_ROW_PATTERN.search(text)
297         for text in _row_text_values(row)
298     )
299
300
301 def _focused_numbers_in_text(value: Any) -> list[float]:
302     numbers: list[float] = []
303     for raw in FOCUSED_NUMBER_PATTERN.findall(_cell_text(value)):
304         cleaned = raw.rstrip("%").replace(",", "")
305         try:
306             number = float(cleaned)
307         except ValueError:
308             continue
309         if raw.endswith("%"):
310             number /= 100.0
311         if math.isfinite(number):
312             numbers.append(number)
313     return numbers
314
315
316 def _focused_support_numbers_from_context(
317     context: dict[str, Any],
318 ) -> list[float]:
319     values: list[Any] = [
320         *context.get("highlighted_values", []),
321         *context.get("row_context", []),
322         *context.get("raw_row_context", []),
323         *context.get("header_context", []),
324     ]
325     for key in [
326         "logical_row_context",
327         "highlighted_role_value_pairs",
328         "logical_row_placeholders",
329     ]:
330         for pair in context.get(key, []):
331             if isinstance(pair, dict):
332                 values.append(pair.get("value"))
333                 values.extend(pair.get("headers", []))
334
335     numbers: list[float] = []
336     for value in values:
337         numbers.extend(_focused_numbers_in_text(value))
338
339     return list(dict.fromkeys(numbers))
340
341
342 def _value_header_semantic_score(value: str, header: str) -> float:
343     value_text = value.casefold()
344     header_text = header.casefold()
345     score = 0.0
346
347     if value_text.endswith("%") and re.search(
348         r"\b(percent(?:age)?|share|rate|ratio)\b",
349         header_text,
350     ):
351         score += 6.0
352     if re.fullmatch(r"\d{1,3}(?:,\d{3})+(?:\.\d+)?|\d+(?:\.\d+)?", value_text):
353         if re.search(
354             r"\b(count|number|total|votes?|score|points?|goals?|amount|"
355             r"population|sales|revenue|value)\b",
356             header_text,
357         ):
358             score += 3.0
359     if re.search(r"\b\d{4}-\d{2}-\d{2}\b|\b\d{4}\b", value_text):
360         if re.search(r"\b(date|year|season|time|period)\b", header_text):
361             score += 3.0
362
363     return score
364
365
366 def _row_for_index(
367     frame: pd.DataFrame,
368     row_index: Any,
369 ) -> pd.DataFrame:
370     return frame[frame["row_index"] == row_index].sort_values(

```



```

371     "column_index"
372 )
373
374
375 def _row_text_values(row: pd.DataFrame) -> list[str]:
376     return [
377         text
378         for _, cell in row.iterrows()
379         if (text := _cell_text(cell.get("cell_value")))
380         and not _is_placeholder_cell(text)
381     ]
382
383
384 def _header_like_row_indices_before(
385     frame: pd.DataFrame,
386     selected_row_index: int,
387 ) -> list[int]:
388     indices: list[int] = []
389     rows_before = frame[
390         frame["row_index"] < selected_row_index
391     ]
392     if rows_before.empty:
393         return indices
394
395     first_row_index = int(rows_before["row_index"].min())
396     structural_header_started = False
397
398     for row_index, row in rows_before.groupby("row_index"):
399         row = row.sort_values("column_index")
400         texts = _row_text_values(row)
401         if not texts:
402             continue
403
404         explicit_header_count = int(
405             row.get("is_header", pd.Series(dtype=bool))
406             .fillna(False)
407             .astype(bool)
408             .sum()
409         )
410         has_measure_value = any(
411             _looks_like_measure_value(text)
412             for text in texts
413         )
414         explicit_structural_header = bool(
415             explicit_header_count > 0
416             and explicit_header_count / max(len(row), 1) >= 0.5
417         )
418         probable_first_header = bool(
419             int(row_index) == first_row_index
420             and not structural_header_started
421             and not has_measure_value
422             and sum(_contains_letter(text) for text in texts)
423             >= max(1, len(texts) // 2)
424         )
425         previous_header_text_count = (
426             len(_row_text_values(_row_for_index(frame, indices[-1])))
427             if indices
428             else 0
429         )
430         label_only_continuation = (
431             structural_header_started
432             and previous_header_text_count > 0
433             and len(texts) < previous_header_text_count
434             and all(_contains_letter(text) for text in texts)
435             and not has_measure_value
436         )
437         if (
438             explicit_structural_header
439             or probable_first_header
440             or label_only_continuation
441         ):
442             indices.append(int(row_index))
443             structural_header_started = True
444             continue
445
446         if structural_header_started:
447             break
448
449     return indices[-6:]
450
451
452 def _header_path_for_column(
453     frame: pd.DataFrame,
454     *,
455     selected_row_index: int,
456     column_index: int,
457 ) -> list[str]:
458     path: list[str] = []
459     for row_index in _header_like_row_indices_before(
460         frame,
461         selected_row_index,

```

```

462 ):
463     row = _row_for_index(frame, row_index)
464     for _, cell in row.iterrows():
465         text = _cell_text(cell.get("cell_value"))
466         if not text or _is_placeholder_cell(text):
467             continue
468         start_column = _cell_int(cell["column_index"])
469         end_column = _cell_int(
470             cell.get("column_end_index"),
471             start_column,
472         )
473         if start_column <= column_index <= end_column:
474             path.append(text)
475             break
476
477     return list(dict.fromkeys(path))
478
479 def _logical_row_context_for_highlighted_cells(
480     frame: pd.DataFrame,
481     highlighted: pd.DataFrame,
482     *,
483     page_title: str,
484 ) -> dict[str, Any]:
485     row_pairs: list[dict[str, Any]] = []
486     highlighted_pairs: list[dict[str, Any]] = []
487     placeholder_pairs: list[dict[str, Any]] = []
488     seen_pairs: set[tuple[str, str, int, bool]] = set()
489
490     for _, selected_cell in highlighted.iterrows():
491         selected_row_index = _cell_int(selected_cell["row_index"])
492         selected_column = _cell_int(selected_cell["column_index"])
493         row = _row_for_index(frame, selected_row_index)
494
495         for _, cell in row.iterrows():
496             column_index = _cell_int(cell["column_index"])
497             text = _cell_text(cell.get("cell_value"))
498             headers = _header_path_for_column(
499                 frame,
500                 selected_row_index=selected_row_index,
501                 column_index=column_index,
502             )
503             is_selected = bool(cell.get("is_highlighted", False)) or (
504                 column_index == selected_column
505                 and text == _cell_text(selected_cell.get("cell_value"))
506             )
507             is_placeholder = not text or _is_placeholder_cell(text)
508             if not headers and is_placeholder:
509                 continue
510
511             key = (
512                 " > ".join(headers),
513                 text,
514                 column_index,
515                 is_selected,
516             )
517             if key in seen_pairs:
518                 continue
519             seen_pairs.add(key)
520
521             pair = {
522                 "headers": headers,
523                 "value": text,
524                 "column_index": column_index,
525             }
526             if is_selected:
527                 highlighted_pairs.append(pair)
528             elif is_placeholder:
529                 placeholder_pairs.append(pair)
530             elif text:
531                 row_pairs.append(pair)
532
533     subject_candidates = []
534     highlighted_has_text_subject = any(
535         _contains_letter(text)
536         and not _looks_like_measure_value(text)
537         for text in (
538             _cell_text(cell.get("cell_value"))
539             for _, cell in highlighted.iterrows()
540         )
541     )
542
543     if page_title and not highlighted_has_text_subject:
544         for pair in placeholder_pairs:
545             headers = [
546                 str(header)
547                 for header in pair.get("headers", [])
548                 if str(header).strip()
549             ]
550             if not headers:
551                 continue
552             if not any(_contains_letter(header) for header in headers):

```

```

553         continue
554         subject_candidates.append(
555             {
556                 "value": page_title,
557                 "related_headers": headers,
558                 "basis": (
559                     "page title with placeholder value under this "
560                     "logical header path in the same row"
561                 ),
562             }
563         )
564     subject_candidates = sorted(
565         subject_candidates,
566         key=lambda candidate: len(candidate["related_headers"]),
567         reverse=True,
568     )
569
570     return {
571         "logical_row_context": row_pairs[:12],
572         "highlighted_role_value_pairs": highlighted_pairs,
573         "logical_row_placeholders": placeholder_pairs[:8],
574         "page_title_subject_candidates": subject_candidates[:4],
575     }
576
577 def _format_role_value_pairs(
578     pairs: list[dict[str, Any]],
579     *,
580     include_placeholders: bool = False,
581 ) -> list[str]:
582     rendered: list[str] = []
583     for pair in pairs:
584         headers = pair.get("headers") or []
585         header_text = " > ".join(str(header) for header in headers if header)
586         value = _cell_text(pair.get("value"))
587         if not header_text:
588             continue
589         if not value and not include_placeholders:
590             continue
591         rendered.append(f"{header_text} = {value or '[blank]'}")
592     return rendered
593
594
595 def _active_row_span_cells(
596     frame: pd.DataFrame,
597     row_index: int,
598 ) -> list[dict[str, Any]]:
599     active: list[dict[str, Any]] = []
600     for _, cell in frame.iterrows():
601         start_row = _cell_int(cell.get("row_index"))
602         span_height = _cell_int(cell.get("row_span"), 1)
603         if span_height <= 1:
604             continue
605         if not start_row < row_index < start_row + span_height:
606             continue
607         value = _cell_text(cell.get("cell_value"))
608         if not value or _is_placeholder_cell(value):
609             continue
610         active.append(
611             {
612                 "row_index": start_row,
613                 "raw_column_index": _cell_int(cell.get("raw_column_index")),
614                 "column_index": _cell_int(cell.get("column_index")),
615                 "column_end_index": _cell_int(
616                     cell.get("column_end_index"),
617                     _cell_int(cell.get("column_index")),
618                 ),
619                 "value": value,
620                 "is_highlighted": bool(cell.get("is_highlighted", False)),
621                 "is_inherited": True,
622             }
623         )
624     )
625
626     return sorted(active, key=lambda item: item["column_index"])
627
628
629 def _highlighted_row_is_span_context_only(
630     *,
631     frame: pd.DataFrame,
632     row_index: int,
633     highlighted_row_indices: set[int],
634 ) -> bool:
635     highlighted_cells = frame[
636         (frame["row_index"] == row_index)
637         & frame["is_highlighted"].fillna(False).astype(bool)
638     ]
639     if highlighted_cells.empty:
640         return False
641
642     has_descendant_highlight = False
643     for _, cell in highlighted_cells.iterrows():

```

```

644         span_height = _cell_int(cell.get("row_span"), 1)
645         if span_height <= 1:
646             return False
647         for descendant_row in range(row_index + 1, row_index + span_height):
648             if descendant_row in highlighted_row_indices:
649                 has_descendant_highlight = True
650
651     return has_descendant_highlight
652
653
654 def _field_role(field: dict[str, Any]) -> str:
655     headers = [
656         str(header).strip()
657         for header in field.get("headers", [])
658         if str(header).strip()
659     ]
660     if headers:
661         return headers[-1]
662     return f"column {field.get('column_index')}}"
663
664
665 def _record_field_lookup(
666     record: dict[str, Any],
667 ) -> dict[str, str]:
668     lookup: dict[str, str] = {}
669     for field in record.get("fields", []):
670         if not isinstance(field, dict):
671             continue
672         role = _field_role(field).casefold()
673         value = _cell_text(field.get("value"))
674         if role and value and role not in lookup:
675             lookup[role] = value
676     return lookup
677
678
679 def _value_for_role(
680     lookup: dict[str, str],
681     *role_names: str,
682 ) -> str | None:
683     for role_name in role_names:
684         role_key = role_name.casefold()
685         if role_key in lookup:
686             return lookup[role_key]
687     return None
688
689
690 def _non_list_page_subject(page_title: str) -> str:
691     title = page_title.strip()
692     if re.match(r"(?i)^\s*list of\b", title):
693         return ""
694     return title
695
696
697 def _record_group_clause(
698     record: dict[str, Any],
699     *,
700     subject: str,
701     include_subject: bool,
702 ) -> str | None:
703     lookup = _record_field_lookup(record)
704     tournament = _value_for_role(lookup, "tournament", "event", "competition")
705     game = _value_for_role(lookup, "game", "title")
706     place = _value_for_role(lookup, "place", "rank", "position", "result")
707
708     prefix = subject if include_subject and subject else ""
709     if place and game and tournament:
710         start = f"{prefix} placed" if prefix else "placed"
711         return f"{start} {place} in {game} at {tournament}"
712     if place and tournament:
713         start = f"{prefix} placed" if prefix else "placed"
714         return f"{start} {place} at {tournament}"
715
716     selected = [
717         field
718         for field in record.get("fields", [])
719         if isinstance(field, dict) and field.get("is_highlighted")
720     ]
721     if not selected:
722         return None
723     rendered = _format_role_value_pairs(selected)
724     if not rendered:
725         return None
726     start = f"{prefix} has " if prefix else ""
727     return start + "; ".join(rendered)
728
729
730 def _normalised_series_label(value: str) -> str:
731     text = re.sub(r"\s+#?\d+\s*$", "", value.strip())
732     return re.sub(r"\s+", " ", text).strip()
733
734

```

```

735 def _is_first_place(value: str) -> bool:
736     return bool(re.fullmatch(r"(?i)\s*(?:1st|first|1)\s*", value.strip()))
737
738
739 def _consecutive_repeated_record_summaries(
740     records: list[dict[str, Any]],
741     *,
742     subject: str,
743 ) -> tuple[list[str], set[int]]:
744     groups: dict[tuple[str, str, str], list[dict[str, Any]]] = {}
745     for record in records:
746         lookup = _record_field_lookup(record)
747         tournament = _value_for_role(
748             lookup,
749             "tournament",
750             "event",
751             "competition",
752         )
753         game = _value_for_role(lookup, "game", "title")
754         place = _value_for_role(lookup, "place", "rank", "position", "result")
755         if not tournament or not game or not place:
756             continue
757         series = _normalised_series_label(tournament)
758         if not series:
759             continue
760         groups.setdefault((series, game, place), []).append(record)
761
762     summaries: list[str] = []
763     consumed_rows: set[int] = set()
764     for (series, game, place), group_records in groups.items():
765         ordered = sorted(group_records, key=lambda item: item["row_index"])
766         if len(ordered) < 3:
767             continue
768         row_indices = [int(record["row_index"]) for record in ordered]
769         consecutive = all(
770             later == earlier + 1
771             for earlier, later in zip(row_indices, row_indices[1:])
772         )
773         if not consecutive:
774             continue
775
776         count_word = {
777             3: "three",
778             4: "four",
779             5: "five",
780         }.get(len(ordered), str(len(ordered)))
781         if _is_first_place(place):
782             verb = "won"
783             summary = (
784                 f"{subject + ' ' if subject else ''}{verb} "
785                 f"{count_word} times in a row in {game} "
786                 f"at {series}"
787             )
788         else:
789             summary = (
790                 f"{subject + ' ' if subject else ''}placed {place} "
791                 f"{count_word} times in a row in {game} "
792                 f"at {series}"
793             )
794         summaries.append(summary)
795         consumed_rows.update(row_indices)
796
797     return summaries, consumed_rows
798
799
800 def _join_record_clauses(clauses: list[str]) -> str:
801     cleaned = [clause.strip(" .") for clause in clauses if clause.strip()]
802     if not cleaned:
803         return ""
804     if len(cleaned) == 1:
805         return cleaned[0] + "."
806     if len(cleaned) == 2:
807         return f"{cleaned[0]} and {cleaned[1]}."
808     return ", ".join(cleaned[:-1]) + f", and {cleaned[-1]}."
809
810
811 def _highlighted_record_group_summary(
812     records: list[dict[str, Any]],
813     *,
814     page_title: str,
815 ) -> str | None:
816     if len(records) < 2:
817         return None
818
819     subject = _non_list_page_subject(page_title)
820     repeated_summaries, consumed_rows = (
821         _consecutive_repeated_record_summaries(records, subject=subject)
822     )
823     clauses: list[str] = []
824     subject_used = False
825     for record in records:

```

```

826         if int(record["row_index"]) in consumed_rows:
827             continue
828         clause = _record_group_clause(
829             record,
830             subject=subject,
831             include_subject=not subject_used,
832         )
833         if clause:
834             clauses.append(clause)
835             subject_used = subject_used or bool(subject)
836
837     for summary in repeated_summaries:
838         if subject and subject_used and summary.startswith(subject + " "):
839             summary = summary[len(subject) + 1 :]
840         clauses.append(summary)
841         subject_used = subject_used or bool(subject)
842
843     return _join_record_clauses(clauses)
844
845
846 def _highlighted_record_groups(
847     frame: pd.DataFrame,
848     highlighted: pd.DataFrame,
849     *,
850     page_title: str,
851 ) -> tuple[list[dict[str, Any]], str | None]:
852     highlighted_row_indices = {
853         _cell_int(row_index)
854         for row_index in highlighted["row_index"].tolist()
855     }
856     records: list[dict[str, Any]] = []
857
858     for row_index in sorted(highlighted_row_indices):
859         if _highlighted_row_is_span_context_only(
860             frame=frame,
861             row_index=row_index,
862             highlighted_row_indices=highlighted_row_indices,
863         ):
864             continue
865
866         merged: dict[int, dict[str, Any]] = {}
867         for inherited in _active_row_span_cells(frame, row_index):
868             merged[inherited["column_index"]] = inherited
869
870         row = _row_for_index(frame, row_index)
871         for _, cell in row.iterrows():
872             value = _cell_text(cell.get("cell_value"))
873             if not value or _is_placeholder_cell(value):
874                 continue
875             column_index = _cell_int(cell.get("column_index"))
876             merged[column_index] = {
877                 "row_index": row_index,
878                 "raw_column_index": _cell_int(cell.get("raw_column_index")),
879                 "column_index": column_index,
880                 "column_end_index": _cell_int(
881                     cell.get("column_end_index"),
882                     column_index,
883                 ),
884                 "value": value,
885                 "is_highlighted": bool(cell.get("is_highlighted", False)),
886                 "is_inherited": False,
887             }
888
889         fields: list[dict[str, Any]] = []
890         for column_index, cell in sorted(merged.items()):
891             headers = _header_path_for_column(
892                 frame,
893                 selected_row_index=row_index,
894                 column_index=column_index,
895             )
896             fields.append(
897                 {
898                     "headers": headers,
899                     "value": cell["value"],
900                     "column_index": column_index,
901                     "source_row_index": cell["row_index"],
902                     "is_highlighted": bool(cell["is_highlighted"]),
903                     "is_inherited": bool(cell["is_inherited"]),
904                 }
905             )
906
907     highlighted_fields = [
908         field for field in fields if field["is_highlighted"]
909     ]
910     if not highlighted_fields:
911         continue
912     records.append(
913         {
914             "row_index": row_index,
915             "fields": fields,
916             "highlighted_fields": highlighted_fields,

```

```

917     }
918 )
919
920 summary = _highlighted_record_group_summary(
921     records,
922     page_title=page_title,
923 )
924 return records, summary
925
926 def _same_measure_comparisons_for_highlighted_cells(
927     frame: pd.DataFrame,
928     highlighted: pd.DataFrame,
929 ) -> list[dict[str, Any]]:
930     comparisons: list[dict[str, Any]] = []
931
932     for _, selected_cell in highlighted.iterrows():
933         selected_value = _cell_text(selected_cell.get("cell_value"))
934         selected_number = _parse_numeric_cell_value(selected_value)
935         if selected_number is None:
936             continue
937
938         selected_row_index = _cell_int(selected_cell["row_index"])
939         selected_column = _cell_int(selected_cell["column_index"])
940         selected_headers = _header_path_for_column(
941             frame,
942             selected_row_index=selected_row_index,
943             column_index=selected_column,
944         )
945         comparable_values: list[dict[str, Any]] = []
946         aggregate_values: list[dict[str, Any]] = []
947
948         for _, cell in frame.iterrows():
949             if bool(cell.get("is_header", False)):
950                 continue
951             text = _cell_text(cell.get("cell_value"))
952             if not text or _is_placeholder_cell(text):
953                 continue
954             number = _parse_numeric_cell_value(text)
955             if number is None:
956                 continue
957
958             start_column = _cell_int(cell["column_index"])
959             end_column = _cell_int(
960                 cell.get("column_end_index"),
961                 start_column,
962             )
963             if not start_column <= selected_column <= end_column:
964                 continue
965
966             row = _row_for_index(frame, cell["row_index"])
967             row_context = [
968                 row_text
969                 for row_text in _row_text_values(row)
970                 if row_text != text
971             ][0:6]
972             entry = {
973                 "value": text,
974                 "numeric_value": number,
975                 "row_index": _cell_int(cell["row_index"]),
976                 "row_context": row_context,
977                 "is_highlighted": bool(cell.get("is_highlighted", False)),
978             }
979             if _row_is_aggregate(row):
980                 aggregate_values.append(entry)
981             else:
982                 comparable_values.append(entry)
983
984         if not comparable_values:
985             continue
986
987         sorted_values = sorted(
988             comparable_values,
989             key=lambda item: item["numeric_value"],
990             reverse=True,
991         )
992         selected_match = next(
993             (
994                 item
995                 for item in sorted_values
996                 if item["is_highlighted"]
997                 and math.isclose(
998                     item["numeric_value"],
999                     selected_number,
1000                     rel_tol=1e-12,
1001                     abs_tol=1e-12,
1002                 )
1003             ),
1004             None,
1005         )
1006         if selected_match is None:

```



```

1008         continue
1009
1010     selected_rank = sorted_values.index(selected_match) + 1
1011     is_percentage = selected_value.endswith("%") or any(
1012         re.search(r"\b(percent(?:age)?|share|rate)\b", header, re.I)
1013         for header in selected_headers
1014     )
1015     comparisons.append(
1016         {
1017             "highlighted_value": selected_value,
1018             "headers": selected_headers,
1019             "numeric_value": selected_number,
1020             "comparable_value_count": len(sorted_values),
1021             "rank_descending": selected_rank,
1022             "is_highest_comparable_value": selected_rank == 1,
1023             "is_percentage_or_share": is_percentage,
1024             "is_majority_percentage": (
1025                 is_percentage and selected_number > 50.0
1026             ),
1027             "comparable_values": sorted_values[:8],
1028             "excluded_aggregate_values": aggregate_values[:4],
1029         }
1030     )
1031
1032     return comparisons
1033
1034
1035 def _lower_initial_label(value: str) -> str:
1036     value = value.strip()
1037     if not value:
1038         return value
1039     if len(value) > 1 and value[:2].isupper():
1040         return value
1041     return value[:1].lower() + value[1:]
1042
1043
1044 def _pluralised_relation_label(value: str) -> str:
1045     text = _lower_initial_label(value).strip()
1046     if not text:
1047         return "highlighted rows"
1048     if "/" in text:
1049         text = text.split("/ ")[-1].strip()
1050     if text.endswith("y") and text[-2:-1].lower() not in {
1051         "a",
1052         "e",
1053         "i",
1054         "o",
1055         "u",
1056     }:
1057         return text[:-1] + "ies"
1058     if text.endswith("s"):
1059         return text
1060     return text + "s"
1061
1062
1063 def _strip_parenthetical_suffix(value: str) -> str:
1064     text = value.strip()
1065     while True:
1066         updated = re.sub(r"\s*\([^\)]*\)\s*$", "", text).strip()
1067         if updated == text or not updated:
1068             return text
1069         text = updated
1070
1071
1072 def _possessive(value: str) -> str:
1073     text = value.strip()
1074     if not text:
1075         return text
1076     return text + (" " if text.endswith("s") else "'s")
1077
1078
1079 def _split_final_year_label(value: str) -> tuple[str, str | None]:
1080     text = value.strip()
1081     match = re.search(r"((\d-{4})(?:[-]\d{2,4})?)\s*$", text)
1082     year = match.group(1) if match else None
1083     if match:
1084         text = text[: match.start()].strip()
1085     compact = _strip_parenthetical_suffix(text)
1086     return compact or text, year
1087
1088
1089 def _looks_like_record_value(value: str) -> bool:
1090     return bool(re.fullmatch(r"\d+\s-*[-]\s*\d+(?:\s-*[-]\s*\d+)?", value.strip()))
1091
1092
1093 def _indefinite_article(phrase: str) -> str:
1094     return "an" if re.match(r"(?i)[aeiou]", phrase.strip()) else "a"
1095
1096
1097 def _simple_plural_forms(noun: str) -> set[str]:
1098     text = noun.strip().casefold()

```

```

1099     if not text:
1100         return set()
1101
1102     forms = {text}
1103     if text.endswith("y") and text[-2:-1] not in {"a", "e", "i", "o", "u"}:
1104         forms.add(text[:-1] + "ies")
1105     elif text.endswith(("s", "x", "z", "ch", "sh")):
1106         forms.add(text + "es")
1107     else:
1108         forms.add(text + "s")
1109     return forms
1110
1111
1112 def _simple_singularise_final_word(word: str) -> str:
1113     match = re.fullmatch(r"([A-Za-z][A-Za-z-]*)" + r"([A-Za-z-]*)", word.strip())
1114     if not match:
1115         return word
1116
1117     stem, suffix = match.groups()
1118     lower = stem.casefold()
1119     if lower.endswith("ies") and len(stem) > 3:
1120         stem = stem[:-3] + "y"
1121     elif lower.endswith("es") and re.search(r"(?i)(s|x|z|ch|sh)es$", stem):
1122         stem = stem[:-2]
1123     elif lower.endswith("s") and not lower.endswith(("ss", "us", "is")):
1124         stem = stem[:-1]
1125
1126     return stem + suffix
1127
1128
1129 def _singularise_final_word(phrase: str) -> str:
1130     words = phrase.strip().split()
1131     if not words:
1132         return ""
1133     words[-1] = _simple_singularise_final_word(words[-1])
1134     return " ".join(words)
1135
1136
1137 def _trim_list_domain_part(phrase: str) -> str:
1138     text = phrase.strip(" .,:;")
1139     text = re.sub(r"(?i)^\s*(?:the|a|an)\s+", "", text).strip()
1140     text = re.split(
1141         r"(?i)\s+\b(?:by|in|from|with|for|during|after|before)\b\s+",
1142         text,
1143         maxsplit=1,
1144     )[0].strip(" .,:;")
1145     return text
1146
1147
1148 def _list_domain_parts(listed_domain: str) -> list[str]:
1149     text = re.sub(r"\s+", " ", listed_domain).strip(" .,:;")
1150     parts = [
1151         _trim_list_domain_part(part)
1152         for part in re.split(r"(?i)\s+(?:and|or)\s+|[,;/]+", text)
1153     ]
1154     return [part for part in parts if part]
1155
1156
1157 def _head_noun_from_label(label: str) -> str:
1158     words = re.findall(r"[A-Za-z][A-Za-z-]*", label)
1159     if not words:
1160         return ""
1161     return _simple_singularise_final_word(words[-1]).casefold()
1162
1163
1164 def _contains_head_noun(phrase: str, head_noun: str) -> bool:
1165     forms = _simple_plural_forms(head_noun)
1166     if not forms:
1167         return False
1168     pattern = r"\b(?:" + "|".join(re.escape(form) for form in forms) + r")\b"
1169     return bool(re.search(pattern, phrase, re.IGNORECASE))
1170
1171
1172 def _has_informative_modifier(phrase: str, head_noun: str) -> bool:
1173     words = re.findall(r"[A-Za-z][A-Za-z-]*", phrase)
1174     if not words:
1175         return False
1176     head_forms = _simple_plural_forms(head_noun)
1177     modifiers = [
1178         word
1179         for word in words
1180         if word.casefold() not in head_forms
1181     ]
1182     return bool(modifiers)
1183
1184
1185 def _list_member_category_phrase(
1186     *,
1187     listed_domain: str,
1188     column_label: str,
1189 ) -> str | None:

```

```

1190 head_noun = _head_noun_from_label(column_label)
1191 if not head_noun:
1192     return None
1193
1194 candidate_parts = [
1195     part
1196     for part in _list_domain_parts(listed_domain)
1197     if _contains_head_noun(part, head_noun)
1198 ]
1199 if not candidate_parts:
1200     return None
1201
1202 # The last matching conjunct often carries the most specific member type:
1203 # e.g. "closed-circuit events and pay-per-view events" -> "pay-per-view event".
1204 candidate = _singularise_final_word(candidate_parts[-1])
1205 if not _has_informative_modifier(candidate, head_noun):
1206     return None
1207
1208 return _lower_initial_label(candidate)
1209
1210
1211 def _best_subject_for_measure_cell(
1212     frame: pd.DataFrame,
1213     *,
1214     row_index: int,
1215     measure_column_index: int,
1216 ) -> dict[str, Any] | None:
1217     row = _row_for_index(frame, row_index)
1218     candidates: list[tuple[int, int, int, dict[str, Any]]] = []
1219
1220     for _, cell in row.iterrows():
1221         value = _cell_text(cell.get("cell_value"))
1222         if (
1223             not value
1224             or _is_placeholder_cell(value)
1225             or _looks_like_measure_value(value)
1226             or not _contains_letter(value)
1227         ):
1228             continue
1229
1230         column_index = _cell_int(cell["column_index"])
1231         headers = _header_path_for_column(
1232             frame,
1233             selected_row_index=row_index,
1234             column_index=column_index,
1235         )
1236         is_highlighted = bool(cell.get("is_highlighted", False))
1237         side_rank = 0 if column_index < measure_column_index else 1
1238         distance = abs(column_index - measure_column_index)
1239         candidates.append(
1240             (
1241                 0 if is_highlighted else 1,
1242                 side_rank,
1243                 distance,
1244                 {
1245                     "value": value,
1246                     "headers": headers,
1247                     "column_index": column_index,
1248                     "is_highlighted": is_highlighted,
1249                 },
1250             )
1251         )
1252
1253     if not candidates:
1254         return None
1255
1256     return sorted(candidates, key=lambda item: item[:3])[0][3]
1257
1258
1259 def _highlighted_set_contrasts(
1260     frame: pd.DataFrame,
1261     highlighted: pd.DataFrame,
1262 ) -> list[dict[str, Any]]:
1263     grouped: dict[tuple[str, ...], list[dict[str, Any]]] = {}
1264
1265     for _, cell in highlighted.iterrows():
1266         highlighted_value = _cell_text(cell.get("cell_value"))
1267         numeric_value = _parse_numeric_cell_value(highlighted_value)
1268         if numeric_value is None:
1269             continue
1270
1271         row_index = _cell_int(cell["row_index"])
1272         column_index = _cell_int(cell["column_index"])
1273         headers = _header_path_for_column(
1274             frame,
1275             selected_row_index=row_index,
1276             column_index=column_index,
1277         )
1278         if not headers:
1279             continue
1280

```

```

1281     subject = _best_subject_for_measure_cell(
1282         frame,
1283         row_index=row_index,
1284         measure_column_index=column_index,
1285     )
1286     grouped.setdefault(tuple(headers), []).append(
1287         {
1288             "row_index": row_index,
1289             "column_index": column_index,
1290             "highlighted_value": highlighted_value,
1291             "numeric_value": numeric_value,
1292             "headers": headers,
1293             "subject": subject,
1294             "is_percentage_or_share": (
1295                 highlighted_value.endswith("%")
1296                 or any(
1297                     re.search(
1298                         r"\b(percent(?:age)?|share|rate)\b",
1299                         header,
1300                         re.I,
1301                     )
1302                     for header in headers
1303                 )
1304             ),
1305         }
1306     )
1307
1308     contrasts: list[dict[str, Any]] = []
1309     for headers, entries in grouped.items():
1310         distinct_entries = [
1311             entry
1312             for index, entry in enumerate(entries)
1313             if not any(
1314                 earlier["row_index"] == entry["row_index"]
1315                 and earlier["column_index"] == entry["column_index"]
1316                 for earlier in entries[:index]
1317             )
1318         ]
1319         if len(distinct_entries) < 2:
1320             continue
1321
1322         sorted_entries = sorted(
1323             distinct_entries,
1324             key=lambda entry: entry["numeric_value"],
1325         )
1326         lower = sorted_entries[0]
1327         higher = sorted_entries[-1]
1328         if math.isclose(
1329             lower["numeric_value"],
1330             higher["numeric_value"],
1331             rel_tol=1e-12,
1332             abs_tol=1e-12,
1333         ):
1334             continue
1335
1336         subject_headers = [
1337             header
1338             for entry in distinct_entries
1339             if isinstance(entry.get("subject"), dict)
1340             for header in entry["subject"].get("headers", [])
1341             if str(header).strip()
1342         ]
1343         subject_label = (
1344             " / ".join(dict.fromkeys(subject_headers))
1345             if subject_headers
1346             else "row"
1347         )
1348         subject_group_label = _pluralised_relation_label(subject_label)
1349         measure_label = _lower_initial_label(" / ".join(headers))
1350         lower_subject = (
1351             lower["subject"].get("value")
1352             if isinstance(lower.get("subject"), dict)
1353             else None
1354         )
1355         higher_subject = (
1356             higher["subject"].get("value")
1357             if isinstance(higher.get("subject"), dict)
1358             else None
1359         )
1360
1361         if lower_subject and higher_subject:
1362             summary = (
1363                 f"Among the highlighted {subject_group_label}, "
1364                 f"{lower_subject} had the lower {measure_label} at "
1365                 f"{lower['highlighted_value']}, while {higher_subject} "
1366                 f"had the higher value at {higher['highlighted_value']}."
1367             )
1368         else:
1369             summary = (
1370                 f"Among the highlighted values for {measure_label}, "
1371                 f"{lower['highlighted_value']} was lower than "

```

```

1372         f"{higher['highlighted_value']}."
1373     )
1374
1375     contrasts.append(
1376         {
1377             "scope": "highlighted_values_only",
1378             "measure_headers": list(headers),
1379             "subject_header": subject_label,
1380             "lower": {
1381                 "subject": lower_subject,
1382                 "value": lower["highlighted_value"],
1383                 "numeric_value": lower["numeric_value"],
1384                 "row_index": lower["row_index"],
1385             },
1386             "higher": {
1387                 "subject": higher_subject,
1388                 "value": higher["highlighted_value"],
1389                 "numeric_value": higher["numeric_value"],
1390                 "row_index": higher["row_index"],
1391             },
1392             "highlighted_value_count": len(distinct_entries),
1393             "contrast_summary": summary,
1394         }
1395     )
1396
1397     return contrasts
1398
1399
1400 def _focused_record_style_relation(
1401     *,
1402     highlighted_role_value_pairs: list[dict[str, Any]],
1403     page_title: str,
1404     section_title: str,
1405 ) -> dict[str, Any] | None:
1406     text_pairs = [
1407         pair
1408         for pair in highlighted_role_value_pairs
1409         if isinstance(pair, dict)
1410         and _contains_letter(_cell_text(pair.get("value")))
1411         and not _looks_like_measure_value(_cell_text(pair.get("value")))
1412     ]
1413     value_pairs = [
1414         pair
1415         for pair in highlighted_role_value_pairs
1416         if isinstance(pair, dict)
1417         and (value := _cell_text(pair.get("value")))
1418         and not _contains_letter(value)
1419         and (
1420             _looks_like_measure_value(value)
1421             or _looks_like_record_value(value)
1422         )
1423     ]
1424
1425     section_phrase = _lower_initial_label(section_title)
1426     page_subject = _strip_parenthetical_suffix(page_title)
1427
1428     for value_pair in value_pairs:
1429         value = _cell_text(value_pair.get("value"))
1430         headers = [
1431             str(header).strip()
1432             for header in value_pair.get("headers", [])
1433             if str(header).strip()
1434         ]
1435         if not headers:
1436             continue
1437
1438         group_pair = next(
1439             (
1440                 pair
1441                 for pair in text_pairs
1442                 if _cell_text(pair.get("value")) in headers
1443             ),
1444             None,
1445         )
1446         if group_pair is None and len(text_pairs) == 1:
1447             group_pair = text_pairs[0]
1448         if group_pair is None:
1449             continue
1450
1451         group_label = _cell_text(group_pair.get("value"))
1452         metric_headers = [
1453             header
1454             for header in headers
1455             if header != group_label
1456         ]
1457         if not metric_headers:
1458             continue
1459
1460         metric_label = _lower_initial_label(metric_headers[0])
1461         compact_group, year = _split_final_year_label(group_label)
1462         year_phrase = f" in {year}" if year else ""

```

```

1463
1464     record_like = (
1465         _looks_like_record_value(value)
1466         or "record" in section_phrase.casefold()
1467         or metric_label.casefold() == "overall"
1468     )
1469     if metric_label.casefold() == "overall" and record_like:
1470         metric_phrase = "overall record"
1471     elif record_like and "record" not in metric_label.casefold():
1472         metric_phrase = f"{metric_label} record"
1473     else:
1474         metric_phrase = f"{metric_label} value"
1475
1476     article = (
1477         "an"
1478         if re.match(r"(?i)[aeiou]", metric_phrase)
1479         else "a"
1480     )
1481
1482     if page_subject and section_phrase:
1483         summary = (
1484             f"{_possessive(page_subject)} {section_phrase} shows "
1485             f"{compact_group}{year_phrase} with {article} "
1486             f"{metric_phrase} of {value}."
1487         )
1488     elif section_phrase:
1489         summary = (
1490             f"The {section_phrase} shows {compact_group}{year_phrase} "
1491             f"with {article} {metric_phrase} of {value}."
1492         )
1493     else:
1494         summary = (
1495             f"{compact_group}{year_phrase} has {article} "
1496             f"{metric_phrase} of {value}."
1497         )
1498
1499     return {
1500         "scope": "focused_record_relation",
1501         "page_subject": page_subject or None,
1502         "section_title": section_title or None,
1503         "group_label": group_label,
1504         "compact_group_label": compact_group,
1505         "year": year,
1506         "metric_headers": metric_headers,
1507         "value": value,
1508         "relation_summary": summary,
1509     }
1510
1511     return None
1512
1513 def _focused_list_page_relation(
1514     *,
1515     highlighted_role_value_pairs: list[dict[str, Any]],
1516     page_title: str,
1517     section_title: str,
1518 ) -> dict[str, Any] | None:
1519     if not re.match(r"(?i)^\s*list of\b", page_title or ""):
1520         return None
1521
1522     text_pairs = [
1523         pair
1524         for pair in highlighted_role_value_pairs
1525         if isinstance(pair, dict)
1526         and (value := _cell_text(pair.get("value")))
1527         and _contains_letter(value)
1528         and not _looks_like_measure_value(value)
1529     ]
1530
1531     if len(text_pairs) != 1:
1532         return None
1533
1534     pair = text_pairs[0]
1535     value = _cell_text(pair.get("value"))
1536     headers = [
1537         str(header).strip()
1538         for header in pair.get("headers", [])
1539         if str(header).strip()
1540     ]
1541     if not headers:
1542         return None
1543
1544     column_label = _lower_initial_label(headers[-1])
1545     if not re.search(
1546         r"\b(event|name|title|team|club|school|person|player|"
1547         r"candidate|office|place|venue|city|country|entry)\b",
1548         column_label,
1549         re.IGNORECASE,
1550     ):
1551         return None
1552
1553     listed_domain = re.sub(

```

```

1554         r"^(?i)^s*list of\s+",
1555         "",
1556         page_title,
1557     ).strip()
1558     member_category = _list_member_category_phrase(
1559         listed_domain=listed_domain,
1560         column_label=column_label,
1561     )
1562     section = section_title.strip()
1563     if section:
1564         if re.fullmatch(r"\d-{4}(\?:[-]\d{2,4})?", section):
1565             prefix = f"In {section}, "
1566         else:
1567             prefix = f"In the {section} section, "
1568     else:
1569         prefix = ""
1570
1571     if member_category:
1572         summary = (
1573             f"{prefix}{value} was {_indefinite_article(member_category)} "
1574             f"{member_category}."
1575         )
1576     else:
1577         summary = (
1578             f"{prefix}{value} was {_indefinite_article(column_label)} "
1579             f"{column_label} in the list of {listed_domain}."
1580         )
1581
1582     return {
1583         "scope": "focused_list_page_relation",
1584         "page_title": page_title or None,
1585         "listed_domain": listed_domain or None,
1586         "member_category": member_category or None,
1587         "section_title": section_title or None,
1588         "column_header": headers[-1],
1589         "value": value,
1590         "relation_summary": summary,
1591     }
1592
1593
1594 def _meaningful_same_row_context(
1595     frame: pd.DataFrame,
1596     highlighted: pd.DataFrame,
1597 ) -> tuple[list[str], list[str]]:
1598     row_context: list[str] = []
1599     row_subjects: list[str] = []
1600
1601     for _, selected_cell in highlighted.iterrows():
1602         row = _row_for_index(frame, selected_cell["row_index"])
1603         selected_column = int(selected_cell["column_index"])
1604         candidates: list[tuple[int, int, str]] = []
1605
1606         for _, cell in row.iterrows():
1607             if bool(cell.get("is_highlighted", False)):
1608                 continue
1609             text = _cell_text(cell.get("cell_value"))
1610             if not text or _is_placeholder_cell(text):
1611                 continue
1612
1613             column_index = _cell_int(cell["column_index"])
1614             row_context.append(text)
1615             side_rank = 0 if column_index < selected_column else 1
1616             distance = abs(column_index - selected_column)
1617             if _contains_letter(text):
1618                 candidates.append((side_rank, distance, text))
1619
1620         if not candidates:
1621             for _, cell in row.iterrows():
1622                 if bool(cell.get("is_highlighted", False)):
1623                     continue
1624                 text = _cell_text(cell.get("cell_value"))
1625                 if not text or _is_placeholder_cell(text):
1626                     continue
1627                 column_index = _cell_int(cell["column_index"])
1628                 side_rank = 0 if column_index < selected_column else 1
1629                 distance = abs(column_index - selected_column)
1630                 candidates.append((side_rank, distance, text))
1631
1632         if candidates:
1633             row_subjects.append(
1634                 sorted(candidates, key=lambda item: (item[0], item[1]))[0][2]
1635             )
1636
1637     return (
1638         list(dict.fromkeys(row_context))[:8],
1639         list(dict.fromkeys(row_subjects))[:3],
1640     )
1641
1642
1643 def _nearby_context_before_highlight(
1644     frame: pd.DataFrame,

```



```

1645     highlighted: pd.DataFrame,
1646 ) -> list[str]:
1647     contexts: list[str] = []
1648     for _, selected_cell in highlighted.iterrows():
1649         row_index = _cell_int(selected_cell["row_index"])
1650         selected_column = _cell_int(selected_cell["column_index"])
1651         preceding = frame[
1652             (frame["row_index"] < row_index)
1653             & (frame["row_index"] >= max(0, row_index - 2))
1654         ].sort_values(["row_index", "column_index"])
1655         for _, cell in preceding.iterrows():
1656             text = _cell_text(cell.get("cell_value"))
1657             if not text or _is_placeholder_cell(text):
1658                 continue
1659             start_column = _cell_int(cell["column_index"])
1660             end_column = _cell_int(cell.get("column_end_index"), start_column)
1661             if (
1662                 start_column <= selected_column <= end_column
1663                 or abs(start_column - selected_column) <= 2
1664                 or _contains_letter(text)
1665             ):
1666                 contexts.append(text)
1667
1668     return list(dict.fromkeys(contexts))[:8]
1669
1670
1671 def _header_context_for_highlighted_cells(
1672     frame: pd.DataFrame,
1673     highlighted: pd.DataFrame,
1674 ) -> tuple[list[str], float]:
1675     if "is_header" not in frame.columns:
1676         return [], 0.0
1677
1678     headers = frame[frame["is_header"].fillna(False).astype(bool)]
1679     if headers.empty:
1680         return [], 0.0
1681
1682     all_headers = _dedupe_meaningful(headers["cell_value"].tolist())
1683     if not all_headers:
1684         return [], 0.0
1685
1686     scored_headers: list[tuple[float, str]] = []
1687     for _, selected_cell in highlighted.iterrows():
1688         value = _cell_text(selected_cell["cell_value"])
1689         selected_column = _cell_int(selected_cell["column_index"])
1690         selected_row = _row_for_index(frame, selected_cell["row_index"])
1691         max_row_column = _cell_int(selected_row["column_index"].max())
1692         cell_fraction = (
1693             (selected_column + 0.5) / (max_row_column + 1)
1694             if max_row_column >= 0
1695             else 0.0
1696         )
1697
1698     for header_row_index, header_row in headers.groupby("row_index"):
1699         ordered = header_row.sort_values("column_index")
1700         row_headers = [
1701             (
1702                 _cell_int(row["column_index"]),
1703                 _cell_int(row.get("column_end_index"), _cell_int(row["column_index"])),
1704                 _cell_text(row["cell_value"]),
1705             )
1706             for _, row in ordered.iterrows()
1707             if _cell_text(row["cell_value"])
1708             and not _is_placeholder_cell(row["cell_value"])
1709         ]
1710         row_headers = [
1711             item for item in row_headers if item[2] in all_headers
1712         ]
1713         if not row_headers:
1714             continue
1715
1716         proportional_index = min(
1717             len(row_headers) - 1,
1718             max(
1719                 0,
1720                 int(cell_fraction * len(row_headers)),
1721             ),
1722         )
1723         for header_position, (
1724             header_column,
1725             header_end_column,
1726             header_text,
1727         ) in enumerate(
1728             row_headers
1729         ):
1730             score = _value_header_semantic_score(value, header_text)
1731             if header_column <= selected_column <= header_end_column:
1732                 score += 8.0
1733             if header_position == proportional_index:
1734                 score += 2.0
1735             position_distance = abs(

```

```

1736         (header_position + 0.5) / len(row_headers)
1737         - cell_fraction
1738     )
1739     score += max(0.0, 2.0 - 8.0 * position_distance)
1740     score -= 0.01 * float(header_row_index)
1741     scored_headers.append((score, header_text))
1742
1743     if not scored_headers:
1744         return all_headers[:8], 0.0
1745
1746     best_score = max(score for score, _ in scored_headers)
1747     best_headers = [
1748         header
1749         for score, header in sorted(
1750             scored_headers,
1751             key=lambda item: item[0],
1752             reverse=True,
1753         )
1754         if score >= best_score - 0.5
1755     ]
1756     return list(dict.fromkeys(best_headers))[:3], best_score
1757
1758
1759 def _focused_description_proposition(
1760     *,
1761     highlighted_values: list[str],
1762     header_context: list[str],
1763     row_context: list[str],
1764     row_subjects: list[str],
1765     page_title: str,
1766     section_title: str,
1767     page_title_subject_candidates: list[dict[str, Any]] | None = None,
1768     nearby_context: list[str] | None = None,
1769 ) -> str:
1770     value_text = ", ".join(highlighted_values)
1771     lower_headers = " ".join(header_context).casefold()
1772     lower_nearby = " ".join(nearby_context or []).casefold()
1773     generic_page_title = bool(
1774         re.search(
1775             r"\b(page|table|list|results?|summary|overview|record)\b",
1776             page_title.casefold(),
1777         )
1778     )
1779     subject_candidates = page_title_subject_candidates or []
1780     if (
1781         value_text
1782         and page_title
1783         and not generic_page_title
1784         and subject_candidates
1785         and "percentage" in lower_headers
1786         and "vote" in lower_nearby
1787     ):
1788         related_headers = " ".join(
1789             " ".join(str(header) for header in candidate.get("related_headers", []))
1790             for candidate in subject_candidates
1791             if isinstance(candidate, dict)
1792         ).casefold()
1793         if "president" in related_headers:
1794             return f"{page_title} received {value_text} of the vote."
1795         return f"{page_title} received {value_text}."
1796
1797     sentence = f"The selected table value is {value_text}"
1798
1799     if header_context:
1800         sentence += f" under the {header_context[0]} header"
1801
1802     if row_context:
1803         context_values = list(
1804             dict.fromkeys([*row_subjects, *row_context])
1805         )[:3]
1806         sentence += (
1807             " in the row containing "
1808             + _natural_join(context_values)
1809         )
1810
1811     table_context = " / ".join(
1812         part
1813         for part in [section_title, page_title]
1814         if part
1815     )
1816     if table_context:
1817         separator = ", within" if row_context else " within"
1818         sentence += f"{separator} {table_context}"
1819
1820     if sentence[-1] not in ".!?" :
1821         sentence += ","
1822     return sentence
1823
1824
1825 def _focused_table_context(
1826     frame: pd.DataFrame,

```

```

1827 ) -> dict[str, Any] | None:
1828     required_columns = {
1829         "cell_value",
1830         "is_highlighted",
1831         "row_index",
1832         "column_index",
1833     }
1834     if not required_columns.issubset(frame.columns):
1835         return None
1836
1837     highlighted = frame[frame["is_highlighted"].fillna(False).astype(bool)]
1838     if highlighted.empty:
1839         return None
1840
1841     highlighted_values = _dedupe_nonempty(highlighted["cell_value"].tolist())
1842     if not highlighted_values:
1843         return None
1844
1845     page_title = (
1846         _cell_text(frame["page_title"].dropna().iloc[0])
1847         if "page_title" in frame.columns and not frame["page_title"].dropna().empty
1848         else ""
1849     )
1850     section_title = (
1851         _cell_text(frame["section_title"].dropna().iloc[0])
1852         if "section_title" in frame.columns and not frame["section_title"].dropna().empty
1853         else ""
1854     )
1855     source_text = (
1856         _cell_text(frame["source_text"].dropna().iloc[0])
1857         if "source_text" in frame.columns and not frame["source_text"].dropna().empty
1858         else ""
1859     )
1860
1861     highlighted_rows = set(highlighted["row_index"].tolist())
1862     same_row = frame[
1863         frame["row_index"].isin(highlighted_rows)
1864         & ~frame["is_highlighted"].fillna(False).astype(bool)
1865     ]
1866     raw_row_context = _dedupe_nonempty(same_row["cell_value"].tolist())[:12]
1867     row_context, row_subjects = _meaningful_same_row_context(
1868         frame,
1869         highlighted,
1870     )
1871     nearby_context = _nearby_context_before_highlight(
1872         frame,
1873         highlighted,
1874     )
1875     header_context, header_confidence = _header_context_for_highlighted_cells(
1876         frame,
1877         highlighted,
1878     )
1879     logical_context = _logical_row_context_for_highlighted_cells(
1880         frame,
1881         highlighted,
1882         page_title=page_title,
1883     )
1884     highlighted_measure_comparisons = (
1885         _same_measure_comparisons_for_highlighted_cells(
1886             frame,
1887             highlighted,
1888         )
1889     )
1890     highlighted_set_contrasts = _highlighted_set_contrasts(
1891         frame,
1892         highlighted,
1893     )
1894     highlighted_record_groups, highlighted_record_group_summary = (
1895         _highlighted_record_groups(
1896             frame,
1897             highlighted,
1898             page_title=page_title,
1899         )
1900     )
1901     focused_record_relation = _focused_record_style_relation(
1902         highlighted_role_value_pairs=logical_context[
1903             "highlighted_role_value_pairs"
1904         ],
1905         page_title=page_title,
1906         section_title=section_title,
1907     )
1908     focused_list_relation = _focused_list_page_relation(
1909         highlighted_role_value_pairs=logical_context[
1910             "highlighted_role_value_pairs"
1911         ],
1912         page_title=page_title,
1913         section_title=section_title,
1914     )
1915     focused_context = {
1916         "highlighted_values": highlighted_values,
1917         "raw_row_context": raw_row_context,

```

```

1918     "row_context": row_context,
1919     **logical_context,
1920 }
1921 focused_numeric_tokens = _focused_support_numbers_from_context(
1922     focused_context
1923 )
1924 description_proposition = _focused_description_proposition(
1925     highlighted_values=highlighted_values,
1926     header_context=header_context,
1927     row_context=row_context,
1928     row_subjects=row_subjects,
1929     page_title=page_title,
1930     section_title=section_title,
1931     page_title_subject_candidates=logical_context[
1932         "page_title_subject_candidates"
1933     ],
1934     nearby_context=nearby_context,
1935 )
1936 if focused_record_relation:
1937     description_proposition = focused_record_relation[
1938         "relation_summary"
1939     ]
1940 elif focused_list_relation:
1941     description_proposition = focused_list_relation[
1942         "relation_summary"
1943     ]
1944
1945 context_parts = [
1946     *([f"page title `{page_title}`"] if page_title else []),
1947     *([f"section `{section_title}`"] if section_title else []),
1948     *([f"column/header context `{', '.join(header_context)}`"] if header_context else []),
1949     *([f"nearby row context `{', '.join(nearby_context)}`"] if nearby_context else []),
1950     *([f"row context `{', '.join(row_context)}`"] if row_context else []),
1951 ]
1952 logical_row_description = _format_role_value_pairs(
1953     logical_context["logical_row_context"]
1954 )
1955 highlighted_role_description = _format_role_value_pairs(
1956     logical_context["highlighted_role_value_pairs"]
1957 )
1958 placeholder_description = _format_role_value_pairs(
1959     logical_context["logical_row_placeholders"],
1960     include_placeholders=True,
1961 )
1962 page_subject_description = [
1963     (
1964         f"{candidate['value']} may supply "
1965         f"{', '.join(candidate['related_headers'])}"
1966     )
1967     for candidate in logical_context["page_title_subject_candidates"]
1968 ]
1969 concise_output_focus = {
1970     "prefer_highlighted_values": True,
1971     "primary_subject_candidates": [
1972         candidate["value"]
1973         for candidate in logical_context[
1974             "page_title_subject_candidates"
1975         ]
1976     ],
1977     "highlighted_role_value_pairs": logical_context[
1978         "highlighted_role_value_pairs"
1979     ],
1980     "highlighted_measure_comparisons": (
1981         highlighted_measure_comparisons
1982     ),
1983     "highlighted_set_contrasts": highlighted_set_contrasts,
1984     "highlighted_record_groups": highlighted_record_groups,
1985     "highlighted_record_group_summary": (
1986         highlighted_record_group_summary
1987     ),
1988     "focused_record_relation": focused_record_relation,
1989     "focused_list_relation": focused_list_relation,
1990     "treat_non_highlighted_row_values_as_context": True,
1991     "omit_non_highlighted_measures_unless_needed": True,
1992 }
1993 if highlighted_role_description:
1994     context_parts.append(
1995         "highlighted role/value context `"
1996         + "; ".join(highlighted_role_description)
1997         + "`"
1998     )
1999 if logical_row_description:
2000     context_parts.append(
2001         "logical row context `"
2002         + "; ".join(logical_row_description)
2003         + "`"
2004     )
2005 if placeholder_description:
2006     context_parts.append(
2007         "logical row placeholders `"
2008         + "; ".join(placeholder_description)

```

```

2009         + ""
2010     )
2011     if page_subject_description:
2012         context_parts.append(
2013             "page-title subject candidates `"
2014             + "; ".join(page_subject_description)
2015             + "`"
2016         )
2017     if concise_output_focus["primary_subject_candidates"]:
2018         context_parts.append(
2019             "short-form focus `prefer highlighted values with primary "
2020             "subject candidate(s) "
2021             + "; ".join(concise_output_focus["primary_subject_candidates"])
2022             + "; treat other row values as optional context`"
2023         )
2024     comparison_description = []
2025     for comparison in highlighted_measure_comparisons:
2026         headers = " > ".join(comparison.get("headers", []))
2027         comparison_description.append(
2028             (
2029                 f"{headers or 'highlighted measure'} "
2030                 f"{comparison['highlighted_value']} ranks "
2031                 f"{comparison['rank_descending']} of "
2032                 f"{comparison['comparable_value_count']} comparable "
2033                 "non-aggregate values"
2034             )
2035             + (
2036                 " and is above half"
2037                 if comparison.get("is_majority_percentage")
2038                 else ""
2039             )
2040         )
2041     if comparison_description:
2042         context_parts.append(
2043             "highlighted measure comparison `"
2044             + "; ".join(comparison_description)
2045             + "`"
2046         )
2047     contrast_description = [
2048         str(contrast.get("contrast_summary") or "").strip()
2049         for contrast in highlighted_set_contrasts
2050         if isinstance(contrast, dict)
2051         and str(contrast.get("contrast_summary") or "").strip()
2052     ]
2053     if contrast_description:
2054         context_parts.append(
2055             "highlighted-set contrast `"
2056             + "; ".join(contrast_description)
2057             + "`"
2058         )
2059     if highlighted_record_group_summary:
2060         context_parts.append(
2061             "highlighted record-group summary `"
2062             + highlighted_record_group_summary
2063             + "`"
2064         )
2065     if focused_record_relation:
2066         context_parts.append(
2067             "focused record-style relation `"
2068             + focused_record_relation["relation_summary"]
2069             + "`"
2070         )
2071     if focused_list_relation:
2072         context_parts.append(
2073             "focused list-page relation `"
2074             + focused_list_relation["relation_summary"]
2075             + "`"
2076         )
2077
2078     return {
2079         "highlighted_values": highlighted_values,
2080         "highlighted_cell_count": int(len(highlighted)),
2081         "highlighted_coordinates": [
2082             {
2083                 "row_index": int(row["row_index"]),
2084                 "column_index": int(row["column_index"]),
2085                 "column_end_index": _cell_int(row.get("column_end_index"), _cell_int(row["column_index"])),
2086             }
2087             for _, row in highlighted.iterrows()
2088         ],
2089         "page_title": page_title or None,
2090         "section_title": section_title or None,
2091         "header_context": header_context,
2092         "header_confidence": header_confidence,
2093         "raw_row_context": raw_row_context,
2094         "row_context": row_context,
2095         "row_subjects": row_subjects,
2096         "nearby_context": nearby_context,
2097         **logical_context,
2098         "focused_numeric_tokens": focused_numeric_tokens,
2099         "highlighted_measure_comparisons": (

```

```

2100         highlighted_measure_comparisons
2101     ),
2102     "highlighted_set_contrasts": highlighted_set_contrasts,
2103     "highlighted_record_groups": highlighted_record_groups,
2104     "highlighted_record_group_summary": (
2105         highlighted_record_group_summary
2106     ),
2107     "focused_record_relation": focused_record_relation,
2108     "focused_list_relation": focused_list_relation,
2109     "concise_output_focus": concise_output_focus,
2110     "context_description": "; ".join(context_parts),
2111     "description_proposition": description_proposition,
2112     "source_text_excerpt": source_text[:2_000] or None,
2113 }
2114
2115
2116 def focused_table_analysis(
2117     bundle: DataBundle,
2118     plan: ExecutionPlan,
2119     builder: EvidenceBuilder,
2120 ) -> None:
2121     if EvidenceCapability.FOCUSED_TABLE_REGION not in set(plan.selected_capabilities):
2122         return
2123
2124     task_ids = [
2125         task.task_id
2126         for task in plan.tasks
2127         if task.capability == EvidenceCapability.FOCUSED_TABLE_REGION
2128     ]
2129
2130     for table_name, frame in bundle.tables.items():
2131         context = _focused_table_context(frame)
2132         if context is None:
2133             continue
2134
2135         values_text = ", ".join(f"`{value}`" for value in context["highlighted_values"])
2136         context_description = context["context_description"]
2137         finding = f"The selected table cell value is {values_text}."
2138         highlighted_roles = _format_role_value_pairs(
2139             context.get("highlighted_role_value_pairs", [])
2140         )
2141         logical_rows = _format_role_value_pairs(
2142             context.get("logical_row_context", [])
2143         )
2144         logical_placeholders = _format_role_value_pairs(
2145             context.get("logical_row_placeholders", []),
2146             include_placeholders=True,
2147         )
2148         page_subject_candidates = [
2149             (
2150                 f"{candidate['value']} may supply "
2151                 f"{candidate['related_headers']}"
2152             )
2153             for candidate in context.get(
2154                 "page_title_subject_candidates",
2155                 [],
2156             )
2157         ]
2158         measure_comparisons = []
2159         for comparison in context.get(
2160             "highlighted_measure_comparisons",
2161             [],
2162         ):
2163             headers = " > ".join(comparison.get("headers", []))
2164             measure_comparisons.append(
2165                 (
2166                     f"{headers or 'highlighted measure'} "
2167                     f"{comparison['highlighted_value']} ranks "
2168                     f"{comparison['rank_descending']} of "
2169                     f"{comparison['comparable_value_count']} comparable "
2170                     "non-aggregate values"
2171                 )
2172                 + (
2173                     " and is above half"
2174                     if comparison.get("is_majority_percentage")
2175                     else ""
2176                 )
2177             )
2178         highlighted_set_contrasts = [
2179             str(contrast.get("contrast_summary") or "").strip()
2180             for contrast in context.get(
2181                 "highlighted_set_contrasts",
2182                 [],
2183             )
2184             if isinstance(contrast, dict)
2185             and str(contrast.get("contrast_summary") or "").strip()
2186         ]
2187         focused_record_relation = context.get("focused_record_relation")
2188         focused_record_summary = (
2189             str(focused_record_relation.get("relation_summary") or "").strip()
2190             if isinstance(focused_record_relation, dict)

```

```

2191         else ""
2192     )
2193     focused_list_relation = context.get("focused_list_relation")
2194     focused_list_summary = (
2195         str(focused_list_relation.get("relation_summary") or "").strip()
2196         if isinstance(focused_list_relation, dict)
2197         else ""
2198     )
2199     highlighted_record_group_summary = str(
2200         context.get("highlighted_record_group_summary") or ""
2201     ).strip()
2202     if highlighted_roles:
2203         finding += (
2204             " Highlighted role/value context: "
2205             + "; ".join(highlighted_roles)
2206             + "."
2207         )
2208     if logical_rows:
2209         finding += (
2210             " Same logical-row role/value context: "
2211             + "; ".join(logical_rows)
2212             + "."
2213         )
2214     if logical_placeholders:
2215         finding += (
2216             " Placeholder roles in the same logical row: "
2217             + "; ".join(logical_placeholders)
2218             + "."
2219         )
2220     if page_subject_candidates:
2221         finding += (
2222             " Page-title subject candidates for placeholder roles: "
2223             + "; ".join(page_subject_candidates)
2224             + "."
2225         )
2226     if measure_comparisons:
2227         finding += (
2228             " Highlighted measure comparison: "
2229             + "; ".join(measure_comparisons)
2230             + "."
2231         )
2232     if highlighted_set_contrasts:
2233         finding += (
2234             " Highlighted-set contrast scoped only to selected cells: "
2235             + "; ".join(highlighted_set_contrasts)
2236         )
2237     if highlighted_record_group_summary:
2238         finding += (
2239             " Highlighted record-group summary: "
2240             + highlighted_record_group_summary
2241         )
2242     if focused_record_summary:
2243         finding += (
2244             " Focused record-style relation: "
2245             + focused_record_summary
2246         )
2247     if focused_list_summary:
2248         finding += (
2249             " Focused list-page relation: "
2250             + focused_list_summary
2251         )
2252     focus = context.get("concise_output_focus", {})
2253     if isinstance(focus, dict) and focus.get(
2254         "primary_subject_candidates"
2255     ):
2256         finding += (
2257             " Short-form output focus: centre the highlighted role/value "
2258             "pair on the most specific supported primary subject "
2259             "candidate; treat other same-row values as optional context "
2260             "unless needed for disambiguation."
2261         )
2262     proposition = context.get("description_proposition")
2263     if isinstance(proposition, str) and proposition.strip():
2264         finding += f" A safe focused description is: {proposition.strip()}"
2265     if context_description:
2266         finding += f" Its table context is {context_description}."
2267
2268     builder.add(
2269         route=AnalysisRoute.DESCRPTIVE,
2270         task_ids=task_ids,
2271         capability=EvidenceCapability.FOCUSED_TABLE_REGION,
2272         evidence_type="focused_table_region",
2273         finding=finding,
2274         metrics=context,
2275         source_tables=[table_name],
2276         source_columns=[
2277             column
2278             for column in [
2279                 "cell_value",
2280                 "is_highlighted",
2281                 "row_index",

```



```

2282         "raw_column_index",
2283         "column_index",
2284         "column_end_index",
2285         "page_title",
2286         "section_title",
2287         "is_header",
2288         "source_text",
2289     ]
2290     if column in frame.columns
2291 ],
2292 method=(
2293     "Direct inspection of the focused table region marked in the "
2294     "benchmark input."
2295 ),
2296 practical_interpretation=(
2297     "This identifies the selected cell and nearby table context. "
2298     "When span-aware logical row context is available, prefer "
2299     "the role/value mapping over raw adjacent-cell context. When "
2300     "multiple highlighted numeric cells share the same measure, "
2301     "a lower/higher contrast is supported only within the "
2302     "highlighted set unless table-wide rank evidence is cited. "
2303     "When a highlighted group or section label is paired with a "
2304     "highlighted record-like value, prefer the supplied "
2305     "record-style relation over a mechanical cell-coordinate "
2306     "description. When a highlighted text cell appears under a "
2307     "meaningful column in a list page, prefer the supplied "
2308     "list-page relation over unrelated row details. "
2309     "When "
2310     "row semantics remain ambiguous, describe the selected value "
2311     "as being in or under the supplied context rather than "
2312     "assigning it to an entity as an action or result."
2313 ),
2314 strength_label="focused_table_region",
2315 claim_permissions=[ClaimPermission.DESCRPTIVE],
2316 factual_confidence=1.0,
2317 methodological_strength=1.0,
2318 user_relevance=1.0,
2319 salience=1.0,
2320 recommended_use=RecommendedUse.HEADLINE,
2321 limitations=[
2322     "The output should describe only the focused table region and its supplied context."
2323 ],
2324 prohibited_interpretations=[
2325     "Do not describe unrelated table cells as the main result.",
2326     "Do not add dataset profiling, missingness, correlation, modelling or data-quality discussion unless
explicitly requested.",
2327 ],
2328 )
2329
2330
2331 def _clean_record_text(value: Any) -> str:
2332     text = str(value or "").strip()
2333     return re.sub(r"\s+", " ", text)
2334
2335
2336 def _parse_pipe_triple(value: Any) -> dict[str, str] | None:
2337     if not isinstance(value, str) or "|" not in value:
2338         return None
2339     parts = [part.strip() for part in value.split("|")]
2340     if len(parts) != 3 or not all(parts):
2341         return None
2342     return {
2343         "record_kind": "triple",
2344         "subject": parts[0],
2345         "relation": parts[1],
2346         "object": parts[2],
2347     }
2348
2349
2350 def _serialized_triples_from_value(value: Any) -> list[dict[str, str]]:
2351     records: list[dict[str, str]] = []
2352     parsed = _parse_pipe_triple(value)
2353     if parsed is not None:
2354         return [parsed]
2355
2356     if isinstance(value, str):
2357         for line in value.splitlines():
2358             parsed_line = _parse_pipe_triple(line.strip())
2359             if parsed_line is not None:
2360                 records.append(parsed_line)
2361     return records
2362
2363     if isinstance(value, dict):
2364         triples = value.get("triples")
2365         if isinstance(triples, list):
2366             for item in triples:
2367                 if isinstance(item, (list, tuple)) and len(item) == 3:
2368                     subject, relation, obj = (
2369                         _clean_record_text(item[0]),
2370                         _clean_record_text(item[1]),
2371                         _clean_record_text(item[2]),

```

```

2372         )
2373         if subject and relation and obj:
2374             records.append(
2375                 {
2376                     "record_kind": "triple",
2377                     "subject": subject,
2378                     "relation": relation,
2379                     "object": obj,
2380                 }
2381             )
2382         continue
2383         records.extend(_serialized_triples_from_value(item))
2384     for child in value.values():
2385         records.extend(_serialized_triples_from_value(child))
2386     return records
2387
2388 if isinstance(value, list):
2389     for item in value:
2390         records.extend(_serialized_triples_from_value(item))
2391 return records
2392
2393
2394 def _structured_record_rows(frame: pd.DataFrame) -> list[dict[str, str]]:
2395     records: list[dict[str, str]] = []
2396     columns = set(frame.columns)
2397     has_triples = {"subject", "relation", "object"}.issubset(columns)
2398
2399     for _, row in frame.iterrows():
2400         record_kind = _clean_record_text(row.get("record_kind"))
2401         if has_triples and _clean_record_text(row.get("subject")):
2402             subject = _clean_record_text(row.get("subject"))
2403             relation = _clean_record_text(row.get("relation"))
2404             obj = _clean_record_text(row.get("object"))
2405             if subject and relation and obj:
2406                 records.append(
2407                     {
2408                         "record_kind": "triple",
2409                         "subject": subject,
2410                         "relation": relation,
2411                         "object": obj,
2412                     }
2413                 )
2414             continue
2415
2416         attribute_name = _clean_record_text(row.get("attribute_name"))
2417         attribute_value = _clean_record_text(row.get("attribute_value"))
2418         if attribute_name and attribute_value:
2419             records.append(
2420                 {
2421                     "record_kind": record_kind or "attribute",
2422                     "attribute_name": attribute_name,
2423                     "attribute_value": attribute_value,
2424                 }
2425             )
2426
2427         for column in ("triples", "source_payload", "source_text"):
2428             if column in columns:
2429                 records.extend(
2430                     _serialized_triples_from_value(row.get(column))
2431                 )
2432
2433     deduplicated: list[dict[str, str]] = []
2434     seen: set[tuple[tuple[str, str], ...]] = set()
2435     for record in records:
2436         key = tuple(sorted(record.items()))
2437         if key in seen:
2438             continue
2439         seen.add(key)
2440         deduplicated.append(record)
2441     records = deduplicated
2442
2443     return records
2444
2445
2446 def _record_numeric_values(records: list[dict[str, str]]) -> list[float]:
2447     values: list[float] = []
2448     for record in records:
2449         for item in record.values():
2450             for match in re.findall(
2451                 r"(?<![\w.])[-+]?[0-9]+(?:\.[0-9]+)?(?:\w|\.(?=\d))",
2452                 str(item),
2453             ):
2454                 try:
2455                     values.append(float(match))
2456                 except ValueError:
2457                     continue
2458     return values
2459
2460
2461 def structured_record_verbalisation_analysis(
2462     bundle: DataBundle,

```

```

2463     plan: ExecutionPlan,
2464     builder: EvidenceBuilder,
2465 ) -> None:
2466     if (
2467         EvidenceCapability.STRUCTURED_RECORD_VERBALISATION
2468         not in set(plan.selected_capabilities)
2469     ):
2470         return
2471
2472     task_ids = [
2473         task.task_id
2474         for task in plan.tasks
2475         if task.capability
2476         == EvidenceCapability.STRUCTURED_RECORD_VERBALISATION
2477     ]
2478
2479     for table_name, frame in bundle.tables.items():
2480         records = _structured_record_rows(frame)
2481         if not records:
2482             continue
2483
2484         triple_count = sum(
2485             record.get("record_kind") == "triple"
2486             for record in records
2487         )
2488         attribute_count = len(records) - triple_count
2489         evidence_type = (
2490             "triple_record"
2491             if triple_count and not attribute_count
2492             else (
2493                 "attribute_record"
2494                 if attribute_count and not triple_count
2495                 else "structured_record"
2496             )
2497         )
2498
2499         if evidence_type == "triple_record":
2500             record_text = "; ".join(
2501                 (
2502                     f"{record['subject']} | {record['relation']} | "
2503                     f"{record['object']}"
2504                 )
2505                 for record in records
2506             )
2507             finding = "The supplied triples are: " + record_text + "."
2508         else:
2509             record_text = "; ".join(
2510                 (
2511                     f"{record['attribute_name']} = "
2512                     f"{record['attribute_value']}"
2513                 )
2514                 for record in records
2515                 if record.get("record_kind") != "triple"
2516             )
2517             finding = "The supplied attributes are: " + record_text + "."
2518
2519         source_text = next(
2520             (
2521                 _clean_record_text(value)
2522                 for value in frame.get("source_text", [])
2523                 if _clean_record_text(value)
2524             ),
2525             "",
2526         )
2527         request = next(
2528             (
2529                 _clean_record_text(value)
2530                 for value in frame.get("request", [])
2531                 if _clean_record_text(value)
2532             ),
2533             "",
2534         )
2535
2536         builder.add(
2537             route=AnalysisRoute.DESCRPTIVE,
2538             task_ids=task_ids,
2539             capability=EvidenceCapability.STRUCTURED_RECORD_VERBALISATION,
2540             evidence_type=evidence_type,
2541             finding=finding,
2542             metrics={
2543                 "records": records,
2544                 "record_count": len(records),
2545                 "attribute_count": attribute_count,
2546                 "triple_count": triple_count,
2547                 "numeric_values": _record_numeric_values(records),
2548                 "source_text": source_text or None,
2549                 "request": request or None,
2550             },
2551             source_tables=[table_name],
2552             source_columns=[
2553                 column

```

```

2554         for column in [
2555             "attribute_name",
2556             "attribute_value",
2557             "subject",
2558             "relation",
2559             "object",
2560             "source_text",
2561         ]
2562         if column in frame.columns
2563     ],
2564     method=(
2565         "Direct extraction of supplied attribute or triple records "
2566         "from the benchmark input representation."
2567     ),
2568     practical_interpretation=(
2569         "This records the complete supplied structured meaning "
2570         "representation for concise natural-language verbalisation."
2571     ),
2572     strength_label=evidence_type,
2573     claim_permissions=[ClaimPermission.DESCRPTIVE],
2574     factual_confidence=1.0,
2575     methodological_strength=1.0,
2576     user_relevance=0.95,
2577     salience=0.95,
2578     recommended_use=RecommendedUse.HEADLINE,
2579     prohibited_interpretations=[
2580         "Do not add attributes, triples, entities or relations that "
2581         "are absent from the supplied structured record.",
2582         "Do not discuss dataset profiling, missingness, correlation, "
2583         "modelling or data quality for this short verbalisation task.",
2584     ],
2585 )
2586
2587
2588 def event_analysis(
2589     bundle: DataBundle,
2590     plan: ExecutionPlan,
2591     builder: EvidenceBuilder,
2592     semantic_map: InputSemanticMap | None = None,
2593 ) -> None:
2594     selected = set(plan.selected_capabilities)
2595     event_capabilities = {
2596         EvidenceCapability.EVENT_OUTCOME,
2597         EvidenceCapability.ENTITY_PERFORMANCE,
2598         EvidenceCapability.RANKING,
2599         EvidenceCapability.GROUP_COMPARISON,
2600     }
2601     if not selected & event_capabilities:
2602         return
2603
2604     event_tasks = [
2605         task
2606         for task in plan.tasks
2607         if task.capability in event_capabilities
2608     ]
2609     fallback_task_ids = [task.task_id for task in event_tasks]
2610
2611     semantic_map_available = bool(semantic_map is not None and semantic_map.bindings)
2612     semantic_query_mode = bool(semantic_map_available and plan.evidence_queries)
2613
2614     if semantic_map_available and not plan.evidence_queries:
2615         builder.execution_notes.append(
2616             "The semantic event map was available, but the frozen plan "
2617             "contained no validated evidence queries. Legacy field-alias "
2618             "extraction was not used."
2619         )
2620     return
2621
2622 for table_name, payload in bundle.structured_inputs.items():
2623     records = (
2624         semantic_query_evidence(
2625             table_name=table_name,
2626             payload=payload,
2627             semantic_map=semantic_map,
2628             queries=plan.evidence_queries,
2629         )
2630         if semantic_query_mode and semantic_map is not None
2631         else event_capability_evidence(payload)
2632     )
2633     if semantic_query_mode:
2634         semantic_supplemental_records = [
2635             record
2636             for record in event_capability_evidence(payload)
2637             if record.evidence_type
2638             in {
2639                 "event_sequence",
2640                 "participant_record_context",
2641                 "score_progression",
2642             }
2643         ]
2644     records = [

```

```

2645         *records,
2646         *semantic_supplemental_records,
2647     ]
2648     if not records:
2649         continue
2650
2651     builder.add(
2652         route=AnalysisRoute.DESCRPTIVE,
2653         task_ids=fallback_task_ids,
2654         capability=EvidenceCapability.DATASET_PROFILE,
2655         evidence_type="event_record_overview",
2656         finding=(
2657             f"{table_name}` contains one structured event record with "
2658             "nested participant and entity information."
2659         ),
2660         metrics={
2661             "event_count": 1,
2662             "input_shape": InputShape.EVENT_RECORD.value,
2663         },
2664         source_tables=[table_name],
2665         source_columns=list(bundle.tables[table_name].columns),
2666         source_paths=[],
2667         entity_scope=[],
2668         semantic_level=SemanticLevel.EVENT,
2669         method="Validated input-structure inspection.",
2670         practical_interpretation=(
2671             "The source is one event, not a flat sample of independent rows."
2672         ),
2673         strength_label="event_record_overview",
2674         claim_permissions=[ClaimPermission.DESCRPTIVE],
2675         factual_confidence=1.0,
2676         methodological_strength=1.0,
2677         user_relevance=0.9,
2678         salience=0.85,
2679         recommended_use=RecommendedUse.SUPPORTING_DETAIL,
2680         eligible_for_writer=not semantic_query_mode,
2681         exclusion_reason=(
2682             "Container-level profile evidence is excluded from the semantic event Writer path."
2683             if semantic_query_mode
2684             else None
2685         ),
2686     )
2687
2688     for record in records:
2689         if record.capability not in selected:
2690             continue
2691         task_ids = [
2692             task.task_id
2693             for task in event_tasks
2694             if task.capability == record.capability
2695         ] or fallback_task_ids
2696         builder.add(
2697             route=(
2698                 AnalysisRoute.ASSOCIATION_COMPARISON
2699                 if record.capability == EvidenceCapability.GROUP_COMPARISON
2700                 else AnalysisRoute.DESCRPTIVE
2701             ),
2702             task_ids=task_ids,
2703             capability=record.capability,
2704             evidence_type=record.evidence_type,
2705             finding=record.finding,
2706             metrics=record.metrics,
2707             source_tables=[table_name],
2708             source_columns=list(
2709                 dict.fromkeys(path.split(".", 1)[0] for path in record.source_paths)
2710             ),
2711             source_paths=record.source_paths,
2712             entity_scope=record.entity_scope,
2713             method=(
2714                 "Validated generic semantic-query execution."
2715                 if semantic_query_mode
2716                 else "Legacy structured-event extraction fallback."
2717             ),
2718             practical_interpretation=record.practical_interpretation,
2719             strength_label=record.strength_label,
2720             claim_permissions=record.claim_permissions,
2721             factual_confidence=record.factual_confidence,
2722             methodological_strength=record.methodological_strength,
2723             user_relevance=record.user_relevance,
2724             salience=record.salience,
2725             recommended_use=record.recommended_use,
2726             limitations=record.limitations,
2727             prohibited_interpretations=record.prohibited_interpretations,
2728             semantic_level=record.semantic_level,
2729             semantic_binding_ids=record.semantic_binding_ids,
2730             analytical_function=record.analytical_function,
2731             query_id=record.query_id,
2732         )
2733
2734
2735 def correlation_strength(value: float) -> str:

```

```

2736     absolute = abs(value)
2737
2738     if absolute >= 0.70:
2739         return "very_strong"
2740     if absolute >= 0.50:
2741         return "strong"
2742     if absolute >= 0.30:
2743         return "moderate"
2744     if absolute >= 0.20:
2745         return "weak_but_reportable"
2746     return "negligible"
2747
2748
2749 def standardised_difference_strength(value: float | None) -> str:
2750     if value is None:
2751         return "not_available"
2752
2753     absolute = abs(value)
2754
2755     if absolute >= 0.80:
2756         return "large"
2757     if absolute >= 0.50:
2758         return "moderate"
2759     if absolute >= 0.20:
2760         return "small"
2761     return "negligible"
2762
2763
2764 def recommendation(
2765     builder: EvidenceBuilder,
2766     action: str,
2767     recommendation_type: str,
2768     priority: str,
2769     justification: str,
2770     affected_analyses: list[str] | None = None,
2771     consequence_if_ignored: str | None = None,
2772     confidence: float = 0.75,
2773 ) -> AnalyticalRecommendation:
2774     count = sum(len(item.recommendations) for item in builder.items) + 1
2775
2776     return AnalyticalRecommendation(
2777         recommendation_id=f"REC_{count:04d}",
2778         action=action,
2779         recommendation_type=recommendation_type,
2780         priority=priority,
2781         justification=justification,
2782         affected_analyses=affected_analyses or [],
2783         consequence_if_ignored=(
2784             consequence_if_ignored
2785             or "The related analysis may be less reliable or harder to interpret."
2786         ),
2787         confidence=confidence,
2788     )
2789
2790
2791 LOW_PRIORITY_STRENGTH_LABELS = {
2792     "negligible",
2793     "negligible_association",
2794     "weak_but_reportable_association",
2795     "small_group_difference",
2796 }
2797
2798
2799 def eligible_as_main_finding(item: EvidenceItem) -> bool:
2800     if not item.eligible_for_writer:
2801         return False
2802
2803     if item.recommended_use not in {
2804         RecommendedUse.HEADLINE,
2805         RecommendedUse.MAIN_FINDING,
2806     }:
2807         return False
2808
2809     if item.strength_label in LOW_PRIORITY_STRENGTH_LABELS:
2810         return False
2811
2812     return (
2813         item.factual_confidence >= 0.90
2814         and item.methodological_strength >= 0.70
2815         and item.user_relevance >= 0.65
2816     )
2817
2818
2819 def evidence_priority_score(item: EvidenceItem) -> float:
2820     use_bonus = {
2821         RecommendedUse.HEADLINE: 0.25,
2822         RecommendedUse.MAIN_FINDING: 0.15,
2823         RecommendedUse.SUPPORTING_DETAIL: 0.0,
2824         RecommendedUse.LIMITATION: 0.10,
2825         RecommendedUse.OMIT_UNLESS_REQUESTED: -0.30,
2826     }[item.recommended_use]

```

```

2827
2828 strength_bonus = {
2829     "very_strong_association": 0.20,
2830     "strong_association": 0.15,
2831     "moderate_association": 0.08,
2832     "large_group_difference": 0.18,
2833     "moderate_group_difference": 0.10,
2834     "small_group_difference": -0.08,
2835     "possible_data_quality_issue": 0.12,
2836     "possible_sentinel_zero": 0.15,
2837     "constant_column": 0.15,
2838     "validated_internal_prediction": 0.15,
2839     "validated_forecast": 0.15,
2840     "model_not_better_than_baseline": 0.10,
2841     "forecast_not_better_than_baseline": 0.10,
2842 }.get(item.strength_label, 0.0)
2843
2844 return (
2845     0.30 * item.salience
2846     + 0.25 * item.user_relevance
2847     + 0.20 * item.methodological_strength
2848     + 0.15 * item.factual_confidence
2849     + use_bonus
2850     + strength_bonus
2851 )
2852
2853
2854 def descriptive_analysis(
2855     bundle: DataBundle,
2856     tasks: List[InvestigationTask],
2857     builder: EvidenceBuilder,
2858 ) -> None:
2859     tasks_by_table: Dict[str, List[InvestigationTask]] = {}
2860
2861     for task in tasks:
2862         tasks_by_table.setdefault(task.table_name, []).append(task)
2863
2864     for table_name, table_tasks in tasks_by_table.items():
2865         if table_name not in bundle.tables:
2866             continue
2867
2868         frame = bundle.tables[table_name]
2869         task_ids = [task.task_id for task in table_tasks]
2870
2871         builder.add(
2872             route=AnalysisRoute.DESCRPTIVE,
2873             task_ids=task_ids,
2874             finding=(
2875                 f"Table `{table_name}` contains {len(frame):,} rows "
2876                 f"and {len(frame.columns):,} columns."
2877             ),
2878             metrics={
2879                 "row_count": len(frame),
2880                 "column_count": len(frame.columns),
2881             },
2882             source_tables=[table_name],
2883             source_columns=list(frame.columns),
2884             method="Direct inspection of loaded table dimensions.",
2885             practical_interpretation=(
2886                 "This establishes the size and dimensionality of the available data."
2887             ),
2888             strength_label="dataset_overview",
2889             claim_permissions=[ClaimPermission.DESCRPTIVE],
2890             factual_confidence=1.0,
2891             methodological_strength=1.0,
2892             user_relevance=0.95,
2893             salience=0.95,
2894             recommended_use=RecommendedUse.HEADLINE,
2895         )
2896
2897         hashable_frame = frame.copy()
2898
2899         for column_name in hashable_frame.columns:
2900             hashable_frame[column_name] = (
2901                 hashable_frame[column_name].map(
2902                     safe_hashable
2903                 )
2904             )
2905
2906         duplicate_row_count = int(
2907             hashable_frame.duplicated().sum()
2908         )
2909
2910         if duplicate_row_count > 0:
2911             duplicate_row_rate = duplicate_row_count / max(
2912                 len(frame),
2913                 1,
2914             )
2915
2916         builder.add(
2917             route=AnalysisRoute.DESCRPTIVE,

```



```

2918         task_ids=task_ids,
2919         finding=(
2920             f"Table `{table_name}` contains "
2921             f"{duplicate_row_count:,} exact duplicate rows "
2922             f"({duplicate_row_rate:.2%} of rows).",
2923         ),
2924         metrics={
2925             "duplicate_row_count": duplicate_row_count,
2926             "duplicate_row_rate": duplicate_row_rate,
2927             "row_count": len(frame),
2928         },
2929         source_tables=[table_name],
2930         source_columns=list(frame.columns),
2931         method="Exact row duplicate inspection.",
2932         practical_interpretation=(
2933             "Exactly repeated rows are present, but the available "
2934             "data do not establish whether they are invalid."
2935         ),
2936         strength_label="duplicate_rows",
2937         limitations=[
2938             "Exact duplicate rows may be genuine repeated observations "
2939             "or unintended duplicates."
2940         ],
2941         prohibited_interpretations=[
2942             "Do not call duplicate rows erroneous without record-level "
2943             "validation.",
2944             "Do not automatically recommend deduplication.",
2945         ],
2946         recommendations=[
2947             AnalyticalRecommendation(
2948                 recommendation_id="REC_DUPLICATE_ROWS",
2949                 action=(
2950                     "Review the exact duplicate rows before deciding "
2951                     "whether to remove them."
2952                 ),
2953                 recommendation_type="data_cleaning",
2954                 priority="medium",
2955                 justification=(
2956                     "Exactly repeated rows can either be valid repeated "
2957                     "observations or unintended duplicates."
2958                 ),
2959                 affected_analyses=[
2960                     "descriptive analysis",
2961                     "correlation analysis",
2962                     "group comparison",
2963                     "predictive modelling",
2964                 ],
2965                 consequence_if_ignored=(
2966                     "Repeated rows may influence summaries or models if "
2967                     "they are unintended duplicates."
2968                 ),
2969                 confidence=1.0,
2970             )
2971         ],
2972         claim_permissions=[
2973             ClaimPermission.DESRIPTIVE,
2974             ClaimPermission.METHODOLOGICAL,
2975         ],
2976         factual_confidence=1.0,
2977         methodological_strength=1.0,
2978         user_relevance=0.75,
2979         salience=0.65,
2980         recommended_use=RecommendedUse.SUPPORTING_DETAIL,
2981     )
2982
2983     missing_counts = frame.isna().sum().sort_values(ascending=False)
2984
2985     for column_name, count in missing_counts.items():
2986         count = int(count)
2987
2988         if count == 0:
2989             continue
2990
2991         rate = count / max(len(frame), 1)
2992
2993         builder.add(
2994             route=AnalysisRoute.DESRIPTIVE,
2995             task_ids=task_ids,
2996             finding=(
2997                 f"`{column_name}` has {count:,} missing values "
2998                 f"({rate:.2%} of rows).",
2999             ),
3000             metrics={
3001                 "missing_count": count,
3002                 "missing_rate": rate,
3003             },
3004             source_tables=[table_name],
3005             source_columns=[column_name],
3006             method="Direct missing-value count.",
3007             practical_interpretation=(
3008                 "The field is largely complete."
3009             )

```

```

3009         if rate < 0.01
3010         else "Missingness may materially affect analysis using this field."
3011     ),
3012     strength_label=(
3013         "low_missingness"
3014         if rate < 0.01
3015         else "material_missingness"
3016     ),
3017     claim_permissions=[ClaimPermission.DESRIPTIVE],
3018     factual_confidence=1.0,
3019     methodological_strength=1.0,
3020     user_relevance=0.75 if rate >= 0.01 else 0.55,
3021     salience=0.75 if rate >= 0.01 else 0.50,
3022     recommended_use=(
3023         RecommendedUse.MAIN_FINDING
3024         if rate >= 0.05
3025         else RecommendedUse.SUPPORTING_DETAIL
3026     ),
3027 )
3028
3029 for column_name in frame.columns:
3030     series = frame[column_name]
3031     safe_series = series.map(safe_hashable)
3032     non_missing = safe_series.dropna()
3033
3034     if non_missing.empty:
3035         continue
3036
3037     unique_count = int(non_missing.nunique())
3038
3039     if unique_count == 1:
3040         value = str(non_missing.iloc[0])
3041
3042         builder.add(
3043             route=AnalysisRoute.DESRIPTIVE,
3044             task_ids=task_ids,
3045             finding=(
3046                 f"`{column_name}` is constant at `{value}` "
3047                 f"across all {len(non_missing):,} non-missing rows."
3048             ),
3049             metrics={
3050                 "constant": True,
3051                 "constant_value": value,
3052                 "non_missing_count": len(non_missing),
3053             },
3054             source_tables=[table_name],
3055             source_columns=[column_name],
3056             method="Unique-value and frequency inspection.",
3057             practical_interpretation=(
3058                 "The column contains no observed variation and should not "
3059                 "be used for correlation, comparison, prediction, or forecasting."
3060             ),
3061             strength_label="constant_column",
3062             claim_permissions=[
3063                 ClaimPermission.DESRIPTIVE,
3064                 ClaimPermission.METHODOLOGICAL,
3065             ],
3066             factual_confidence=1.0,
3067             methodological_strength=1.0,
3068             user_relevance=0.90,
3069             salience=0.90,
3070             recommended_use=RecommendedUse.MAIN_FINDING,
3071             recommendations=[
3072                 recommendation(
3073                     builder,
3074                     action=(
3075                         f"Remove `{column_name}` from correlation, comparison, "
3076                         "and predictive feature sets unless its constant value "
3077                         "has a documented interpretation."
3078                     ),
3079                     recommendation_type="data_cleaning",
3080                     priority="high",
3081                     justification=(
3082                         "The variable contains no observed variation and therefore "
3083                         "cannot distinguish observations or explain differences."
3084                     ),
3085                     affected_analyses=[
3086                         "correlation analysis",
3087                         "group comparison",
3088                         "predictive modelling",
3089                     ],
3090                     consequence_if_ignored=(
3091                         "The field will add no analytical information and may "
3092                         "create unnecessary processing or numerical issues in "
3093                         "methods that expect varying predictors."
3094                     ),
3095                     confidence=1.0,
3096                 )
3097             ],
3098             prohibited_interpretations=[
3099                 "Do not compare groups using this column as an outcome.",

```

```

3100         "Do not describe the column as predictive.",
3101         ],
3102     )
3103     continue
3104
3105 if is_numeric_measure(series):
3106     values = pd.to_numeric(series, errors="coerce").dropna()
3107
3108     if values.empty:
3109         continue
3110
3111     q01 = float(values.quantile(0.01))
3112     q05 = float(values.quantile(0.05))
3113     q25 = float(values.quantile(0.25))
3114     q75 = float(values.quantile(0.75))
3115     q99 = float(values.quantile(0.99))
3116     mean = float(values.mean())
3117     median = float(values.median())
3118     std = float(values.std(ddof=1)) if len(values) > 1 else 0.0
3119     zero_count = int((values == 0).sum())
3120     zero_rate = zero_count / len(values)
3121     zero_risk, zero_risk_reason = classify_zero_risk(
3122         column_name=column_name,
3123         zero_count=zero_count,
3124         zero_rate=zero_rate,
3125         median=median,
3126         q05=q05,
3127     )
3128
3129     metrics = {
3130         "count": int(values.count()),
3131         "mean": mean,
3132         "median": median,
3133         "standard_deviation": std,
3134         "minimum": float(values.min()),
3135         "q01": q01,
3136         "q05": q05,
3137         "q25": q25,
3138         "q75": q75,
3139         "q99": q99,
3140         "maximum": float(values.max()),
3141         "zero_count": zero_count,
3142         "zero_rate": zero_rate,
3143         "zero_risk": zero_risk.value,
3144         "zero_risk_reason": zero_risk_reason,
3145         "negative_count": int((values < 0).sum()),
3146         "skewness": (
3147             float(values.skew())
3148             if len(values) > 2
3149             else 0.0
3150         ),
3151     }
3152
3153     centre_difference = abs(mean - median)
3154     centre_close = std == 0 or centre_difference <= 0.10 * std
3155
3156     if centre_close:
3157         interpretation = (
3158             "The mean and median are close relative to the observed spread, "
3159             "so the centre is not strongly separated by this diagnostic."
3160         )
3161     else:
3162         interpretation = (
3163             "The mean and median differ relative to the observed spread, "
3164             "which may indicate skewness or influential values."
3165         )
3166
3167     suspicious_zero = zero_risk in {
3168         ZeroRisk.UNUSUAL,
3169         ZeroRisk.POSSIBLE_SENTINEL,
3170     }
3171
3172     limitations = [
3173         "Distribution summaries do not establish a trend, prediction, "
3174         "or causal relationship."
3175     ]
3176
3177     recommendations: list[AnalyticalRecommendation] = []
3178
3179     if zero_risk == ZeroRisk.CONTEXT_DEPENDENT:
3180         limitations.append(zero_risk_reason)
3181
3182     if suspicious_zero:
3183         limitations.append(zero_risk_reason)
3184
3185         if zero_risk == ZeroRisk.POSSIBLE_SENTINEL:
3186             priority = "high"
3187             recommendation_use = RecommendedUse.MAIN_FINDING
3188             strength_label = "possible_sentinel_zero"
3189             consequence = (
3190                 "Treating encoded missing values as genuine measurements "

```

```

3191         "could distort means, associations, and fitted model "
3192         "relationships."
3193     )
3194     confidence = 0.85
3195 else:
3196     priority = "medium"
3197     recommendation_use = RecommendedUse.SUPPORTING_DETAIL
3198     strength_label = "possible_data_quality_issue"
3199     consequence = (
3200         "If the zeros are invalid records, summaries and "
3201         "relationships involving this field may be distorted."
3202     )
3203     confidence = 0.70
3204
3205 recommendations.append(
3206     recommendation(
3207         builder,
3208         action=(
3209             f"Validate zero values in `{column_name}` against "
3210             "source records or metadata before relying on analyses "
3211             "involving this field."
3212         ),
3213         recommendation_type="data_cleaning",
3214         priority=priority,
3215         justification=zero_risk_reason,
3216         affected_analyses=[
3217             "descriptive statistics",
3218             "correlation analysis",
3219             "predictive modelling",
3220         ],
3221         consequence_if_ignored=consequence,
3222         confidence=confidence,
3223     )
3224 )
3225 else:
3226     recommendation_use = (
3227         RecommendedUse.OMIT_UNLESS_REQUESTED
3228         if zero_risk == ZeroRisk.CONTEXT_DEPENDENT
3229         else RecommendedUse.SUPPORTING_DETAIL
3230     )
3231     strength_label = "distribution_summary"
3232
3233 builder.add(
3234     route=AnalysisRoute.DESCRPTIVE,
3235     task_ids=task_ids,
3236     finding=(
3237         f"`{column_name}` has mean {mean:.4g}, median {median:.4g}, "
3238         f"minimum {metrics['minimum']:.4g}, and maximum "
3239         f"{metrics['maximum']:.4g} across {len(values):,} "
3240         "non-missing observations."
3241     ),
3242     metrics=metrics,
3243     source_tables=[table_name],
3244     source_columns=[column_name],
3245     method=(
3246         "Direct descriptive statistics with quantiles and "
3247         "distribution diagnostics."
3248     ),
3249     practical_interpretation=interpretation,
3250     strength_label=(
3251         strength_label
3252     ),
3253     claim_permissions=[
3254         ClaimPermission.DESCRPTIVE,
3255         ClaimPermission.METHODOLOGICAL,
3256     ],
3257     factual_confidence=0.99,
3258     methodological_strength=0.98,
3259     user_relevance=0.85 if suspicious_zero else 0.55,
3260     salience=0.90 if suspicious_zero else 0.50,
3261     recommended_use=recommendation_use,
3262     limitations=limitations,
3263     recommendations=recommendations,
3264     prohibited_interpretations=[
3265         "Do not infer temporal change from the distribution alone.",
3266         "Do not treat a suspicious value as definitively erroneous "
3267         "without source validation.",
3268     ],
3269 )
3270
3271 elif unique_count <= 20:
3272     counts = non_missing.astype(str).value_counts().head(10)
3273     top_values = {
3274         str(key): int(value)
3275         for key, value in counts.items()
3276     }
3277
3278 builder.add(
3279     route=AnalysisRoute.DESCRPTIVE,
3280     task_ids=task_ids,
3281     finding=(

```

```

3282         f"The most frequent observed values of `{column_name}` are "
3283         + ", ".join(
3284             f"{key}` ({value:})"
3285             for key, value in list(top_values.items())[:5]
3286         )
3287         + "."
3288     ),
3289     metrics={
3290         "value_counts": top_values,
3291         "unique_count": unique_count,
3292     },
3293     source_tables=[table_name],
3294     source_columns=[column_name],
3295     method="Frequency counts after safe conversion of structured values.",
3296     practical_interpretation=(
3297         "The counts describe the observed category composition and "
3298         "can reveal imbalance between groups."
3299     ),
3300     strength_label="category_composition",
3301     claim_permissions=[ClaimPermission.DESCRPTIVE],
3302     factual_confidence=0.99,
3303     methodological_strength=0.98,
3304     user_relevance=0.65,
3305     salience=0.60,
3306     recommended_use=RecommendedUse.SUPPORTING_DETAIL,
3307     limitations=[
3308         "Counts describe the observed dataset and do not establish "
3309         "population prevalence."
3310     ],
3311 )
3312
3313
3314 def association_analysis(
3315     bundle: DataBundle,
3316     tasks: list[InvestigationTask],
3317     builder: EvidenceBuilder,
3318     settings: Settings,
3319 ) -> None:
3320     for task in tasks:
3321         table_name = task.table_name
3322
3323         if table_name not in bundle.tables:
3324             continue
3325
3326         original = bundle.tables[table_name]
3327
3328         if len(original) <= settings.full_data_correlation_limit:
3329             frame = original.copy()
3330             sampling_method = "full_dataset"
3331         else:
3332             sample_size = min(settings.max_analysis_rows, len(original))
3333             frame = original.sample(
3334                 n=sample_size,
3335                 random_state=settings.random_seed,
3336             ).copy()
3337             sampling_method = "fixed_seed_sample"
3338
3339         numeric_columns = [
3340             column
3341             for column in frame.select_dtypes(include=np.number).columns
3342             if pd.to_numeric(frame[column], errors="coerce").nunique(dropna=True) > 1
3343         ]
3344
3345         correlations: list[tuple[float, str, str, int, float]] = []
3346
3347         for left, right in combinations(numeric_columns, 2):
3348             pair = (
3349                 frame[[left, right]]
3350                 .apply(pd.to_numeric, errors="coerce")
3351                 .dropna()
3352             )
3353
3354             if (
3355                 len(pair) < 20
3356                 or pair[left].nunique() < 2
3357                 or pair[right].nunique() < 2
3358             ):
3359                 continue
3360
3361             correlation = float(pair[left].corr(pair[right]))
3362
3363             if (
3364                 np.isfinite(correlation)
3365                 and abs(correlation) >= settings.min_abs_correlation
3366             ):
3367                 correlations.append(
3368                     (
3369                         abs(correlation),
3370                         left,
3371                         right,
3372                         len(pair),

```

```

3373         correlation,
3374     )
3375 )
3376
3377 for _, left, right, complete_count, value in sorted(
3378     correlations,
3379     reverse=True,
3380 ): settings.max_correlation_findings]:
3381     direction = "positive" if value > 0 else "negative"
3382     strength = correlation_strength(value)
3383
3384     sample_prefix = (
3385         "Using the full available data"
3386         if sampling_method == "full_dataset"
3387         else f"In a fixed-seed sample of {len(frame):,} rows"
3388     )
3389
3390     builder.add(
3391         route=AnalysisRoute.ASSOCIATION_COMPARISON,
3392         task_ids=[task.task_id],
3393         finding=(
3394             f"{sample_prefix}, `{left}` and `{right}` have a {direction} "
3395             f"Pearson correlation of {value:.4f} across "
3396             f"{complete_count:,} complete row pairs."
3397         ),
3398         metrics={
3399             "pearson_r": value,
3400             "complete_pairs": complete_count,
3401             "sampling_method": sampling_method,
3402             "analysed_rows": len(frame),
3403             "strength": strength,
3404         },
3405         source_tables=[table_name],
3406         source_columns=[left, right],
3407         method="Pairwise-complete Pearson correlation.",
3408         practical_interpretation=(
3409             f"Higher values of `{left}` tend to coincide with "
3410             f"{'higher' if value > 0 else 'lower'} values of `{right}`. "
3411             f"The observed linear relationship is classified as {strength}."
3412         ),
3413         strength_label=f"{strength}_association",
3414         claim_permissions=[
3415             ClaimPermission.ASSOCIATIONAL,
3416             ClaimPermission.METHODOLOGICAL,
3417         ],
3418         factual_confidence=0.98,
3419         methodological_strength=0.88,
3420         user_relevance=min(1.0, 0.45 + abs(value)),
3421         salience=min(1.0, 0.40 + abs(value)),
3422         recommended_use=(
3423             RecommendedUse.MAIN_FINDING
3424             if abs(value) >= 0.50
3425             else RecommendedUse.SUPPORTING_DETAIL
3426         ),
3427         limitations=[
3428             "Correlation does not establish causation.",
3429             "Pearson correlation measures linear association and may be "
3430             "affected by outliers or non-linear structure.",
3431         ],
3432         prohibited_interpretations=[
3433             f"Do not say `{left}` causes `{right}`.",
3434             f"Do not say `{right}` causes `{left}`.",
3435             "Do not describe correlation as complete explanation.",
3436         ],
3437     )
3438
3439 categorical_columns = [
3440     column
3441     for column in frame.columns
3442     if not is_numeric_measure(frame[column])
3443     and 2
3444     <= frame[column].map(safe_hashable).nunique(dropna=True)
3445     <= 10
3446 ]
3447
3448 candidates: list[dict[str, Any]] = []
3449
3450 for group_column in categorical_columns[:8]:
3451     groups = frame[group_column].map(safe_hashable)
3452
3453     for outcome_column in numeric_columns[:12]:
3454         working = pd.DataFrame(
3455             {
3456                 "group": groups,
3457                 "outcome": pd.to_numeric(
3458                     frame[outcome_column],
3459                     errors="coerce",
3460                 ),
3461             }
3462         ).dropna()
3463 
```

```

3464         if len(working) < 30:
3465             continue
3466
3467         if working["outcome"].nunique(dropna=True) < 2:
3468             continue
3469
3470         summary = working.groupby("group")["outcome"].agg(
3471             ["mean", "std", "count"]
3472         )
3473         summary = summary[summary["count"] >= 5]
3474
3475         if len(summary) < 2:
3476             continue
3477
3478         if summary["mean"].nunique(dropna=True) < 2:
3479             continue
3480
3481         highest_group = summary["mean"].idxmax()
3482         lowest_group = summary["mean"].idxmin()
3483
3484         if highest_group == lowest_group:
3485             continue
3486
3487         highest_mean = float(summary.loc[highest_group, "mean"])
3488         lowest_mean = float(summary.loc[lowest_group, "mean"])
3489         difference = highest_mean - lowest_mean
3490
3491         high_std = float(summary.loc[highest_group, "std"])
3492         low_std = float(summary.loc[lowest_group, "std"])
3493
3494         pooled_std = math.sqrt(
3495             (
3496                 (0.0 if math.isnan(high_std) else high_std ** 2)
3497                 + (0.0 if math.isnan(low_std) else low_std ** 2)
3498             )
3499             / 2.0
3500         )
3501
3502         standardised_difference = (
3503             difference / pooled_std
3504             if pooled_std > 0
3505             else None
3506         )
3507
3508         strength = standardised_difference_strength(
3509             standardised_difference
3510         )
3511
3512         if strength == "negligible":
3513             continue
3514
3515         group_counts = {
3516             str(key): int(value)
3517             for key, value in summary["count"].items()
3518         }
3519
3520         imbalance_ratio = max(group_counts.values()) / max(
3521             min(group_counts.values()),
3522             1,
3523         )
3524
3525         candidates.append(
3526             {
3527                 "score": abs(standardised_difference or 0.0),
3528                 "group_column": group_column,
3529                 "outcome_column": outcome_column,
3530                 "highest_group": str(highest_group),
3531                 "lowest_group": str(lowest_group),
3532                 "highest_mean": highest_mean,
3533                 "lowest_mean": lowest_mean,
3534                 "difference": difference,
3535                 "standardised_difference": standardised_difference,
3536                 "strength": strength,
3537                 "group_counts": group_counts,
3538                 "imbalance_ratio": imbalance_ratio,
3539             }
3540         )
3541
3542     for candidate in sorted(
3543         candidates,
3544         key=lambda item: item["score"],
3545         reverse=True,
3546     )[: settings.max_group_findings]:
3547         if candidate["imbalance_ratio"] >= 2:
3548             group_imbalance_note = (
3549                 "The groups are unevenly represented. The larger group mean is "
3550                 "estimated from more observations than the smaller group mean, "
3551                 "so the estimates may have different levels of precision and "
3552                 "stability."
3553             )
3554         else:

```



```

3555         group_imbalance_note = "The group sizes are not strongly imbalanced."
3556
3557     builder.add(
3558         route=AnalysisRoute.ASSOCIATION_COMPARISON,
3559         task_ids=[task.task_id],
3560         finding=(
3561             f"For `{candidate['outcome_column']}` grouped by "
3562             f"`{candidate['group_column']}`", "
3563             f"`{candidate['highest_group']}` has the highest observed mean "
3564             f"({candidate['highest_mean']:.4g}) and "
3565             f"`{candidate['lowest_group']}` the lowest "
3566             f"({candidate['lowest_mean']:.4g}), a difference of "
3567             f"({candidate['difference']:.4g})."
3568         ),
3569         metrics=candidate,
3570         source_tables=[table_name],
3571         source_columns=[
3572             candidate["group_column"],
3573             candidate["outcome_column"],
3574         ],
3575         method=(
3576             "Observed group means with at least five observations per "
3577             "retained group, accompanied by group counts and a "
3578             "standardised difference."
3579         ),
3580         practical_interpretation=(
3581             f"The extreme observed group means differ by "
3582             f"({candidate['difference']:.4g}). The standardised difference "
3583             f"is classified as {candidate['strength']}. The comparison is "
3584             "descriptive and unadjusted."
3585         ),
3586         strength_label=(
3587             f"{candidate['strength']}_group_difference"
3588         ),
3589         claim_permissions=[
3590             ClaimPermission.COMPARATIVE,
3591             ClaimPermission.ASSOCIATIONAL,
3592             ClaimPermission.METHODOLOGICAL,
3593         ],
3594         factual_confidence=0.97,
3595         methodological_strength=0.82,
3596         user_relevance=min(1.0, 0.50 + candidate["score"] / 2),
3597         salience=min(1.0, 0.45 + candidate["score"] / 2),
3598         recommended_use=(
3599             RecommendedUse.MAIN_FINDING
3600             if candidate["strength"] in {"large", "moderate"}
3601             else RecommendedUse.SUPPORTING_DETAIL
3602         ),
3603         limitations=[
3604             "This is an unadjusted observed comparison.",
3605             "The comparison does not establish causation.",
3606             group_imbalance_note,
3607         ],
3608         prohibited_interpretations=[
3609             "Do not say group membership caused the observed difference.",
3610             "Do not describe the comparison as adjusted for confounding.",
3611         ],
3612     )
3613
3614 def normalise_name(value: str) -> str:
3615     words = re.findall(r"[a-z0-9]+", value.lower())
3616     ignored = {"c", "km", "h", "degrees", "millibars", "value"}
3617     return " ".join(word for word in words if word not in ignored)
3618
3619
3620 def select_and_audit_features(
3621     frame: pd.DataFrame,
3622     target_column: str,
3623     time_column: str | None,
3624     proxy_threshold: float,
3625 ) -> tuple[list[str], list[str], list[dict[str, str]], list[str]]:
3626     numeric: list[str] = []
3627     categorical: list[str] = []
3628     excluded: list[dict[str, str]] = []
3629     warnings: list[str] = []
3630
3631     numeric_target = pd.to_numeric(frame[target_column], errors="coerce")
3632     target_name = normalise_name(target_column)
3633
3634     for column_name in frame.columns:
3635         if column_name in {target_column, time_column}:
3636             continue
3637
3638         series = frame[column_name]
3639
3640         if series.map(
3641             lambda value: isinstance(value, (list, dict, tuple, set))
3642         ).any():
3643             excluded.append(
3644                 {

```

```

3646         "feature": column_name,
3647         "risk_type": "structured_value",
3648         "reason": "Structured values are not encoded by this modelling route.",
3649     }
3650 )
3651     continue
3652
3653 unique_count = int(
3654     series.map(safe_hashable).nunique(dropna=True)
3655 )
3656
3657 if unique_count <= 1:
3658     excluded.append(
3659         {
3660             "feature": column_name,
3661             "risk_type": "constant",
3662             "reason": "The feature has no observed variation.",
3663         }
3664     )
3665     continue
3666
3667 if unique_count >= max(int(len(frame) * 0.98), 1_000):
3668     excluded.append(
3669         {
3670             "feature": column_name,
3671             "risk_type": "identifier",
3672             "reason": "The field behaves like a high-cardinality identifier.",
3673         }
3674     )
3675     continue
3676
3677 possible_proxy = False
3678 proxy_reason = ""
3679
3680 if is_numeric_measure(series) and numeric_target.notna().any():
3681     numeric_feature = pd.to_numeric(series, errors="coerce")
3682     pair = pd.DataFrame(
3683         {
3684             "feature": numeric_feature,
3685             "target": numeric_target,
3686         }
3687     ).dropna()
3688
3689     if (
3690         len(pair) >= 20
3691         and pair["feature"].nunique() > 1
3692         and pair["target"].nunique() > 1
3693     ):
3694         correlation = abs(
3695             float(pair["feature"].corr(pair["target"]))
3696         )
3697
3698         if correlation >= proxy_threshold:
3699             possible_proxy = True
3700             proxy_reason = (
3701                 f"Absolute feature-target correlation is {correlation:.4f}, "
3702                 f"above the proxy threshold {proxy_threshold:.4f}."
3703             )
3704
3705     feature_name = normalise_name(column_name)
3706
3707     if (
3708         target_name
3709         and feature_name
3710         and (
3711             target_name in feature_name
3712             or feature_name in target_name
3713         )
3714     ):
3715         if column_name != target_column:
3716             possible_proxy = True
3717             proxy_reason = (
3718                 proxy_reason
3719                 or "The feature name strongly overlaps the target name."
3720             )
3721
3722     if possible_proxy:
3723         excluded.append(
3724             {
3725                 "feature": column_name,
3726                 "risk_type": "target_proxy",
3727                 "reason": proxy_reason,
3728             }
3729         )
3730     warnings.append(
3731         f"`{column_name}` was excluded as a possible proxy for "
3732         f"`{target_column}`: {proxy_reason}"
3733     )
3734     continue
3735
3736 if is_numeric_measure(series):

```

```

3737         numeric.append(column_name)
3738     elif unique_count <= 100:
3739         categorical.append(column_name)
3740
3741     return numeric[:30], categorical[:20], excluded, warnings
3742
3743
3744 def make_preprocessor(
3745     numeric_columns: list[str],
3746     categorical_columns: list[str],
3747 ) -> ColumnTransformer:
3748     numeric_pipeline = Pipeline(
3749         [
3750             ("imputer", SimpleImputer(strategy="median")),
3751             ("scale", StandardScaler(with_mean=False)),
3752         ]
3753     )
3754
3755     categorical_pipeline = Pipeline(
3756         [
3757             ("imputer", SimpleImputer(strategy="most_frequent")),
3758             ("one_hot", OneHotEncoder(handle_unknown="ignore")),
3759         ]
3760     )
3761
3762     return ColumnTransformer(
3763         [
3764             ("numeric", numeric_pipeline, numeric_columns),
3765             ("categorical", categorical_pipeline, categorical_columns),
3766         ]
3767     )
3768
3769
3770 def add_predictive_insufficiency(
3771     builder: EvidenceBuilder,
3772     task: InvestigationTask,
3773     finding: str,
3774     metrics: dict[str, Any],
3775     limitations: list[str],
3776     recommendations: list[AnalyticalRecommendation] | None = None,
3777 ) -> None:
3778     builder.add(
3779         route=AnalysisRoute.PREDICTIVE,
3780         task_ids=[task.task_id],
3781         finding=finding,
3782         metrics=metrics,
3783         source_tables=[task.table_name],
3784         source_columns=[
3785             column
3786             for column in [
3787                 task.target_column,
3788                 task.time_column,
3789                 *task.columns,
3790             ]
3791             if column
3792         ],
3793         method="Predictive modelling feasibility and validation assessment.",
3794         validation_strategy=task.validation_strategy,
3795         practical_interpretation=(
3796             "The available evidence does not support a positive predictive claim."
3797         ),
3798         strength_label="predictive_insufficiency",
3799         claim_permissions=[
3800             ClaimPermission.INSUFFICIENCY,
3801             ClaimPermission.METHODOLOGICAL,
3802         ],
3803         factual_confidence=1.0,
3804         methodological_strength=0.95,
3805         user_relevance=0.75,
3806         salience=0.75,
3807         recommended_use=RecommendedUse.LIMITATION,
3808         limitations=limitations,
3809         recommendations=recommendations or [],
3810         prohibited_interpretations=[
3811             "Do not describe the task as successfully validated.",
3812             "Do not claim deployment readiness.",
3813         ],
3814     )
3815
3816
3817 def split_predictive_data(
3818     frame: pd.DataFrame,
3819     features: list[str],
3820     target_column: str,
3821     time_column: str | None,
3822     classification: bool,
3823     seed: int,
3824 ) -> tuple[pd.DataFrame, pd.DataFrame, pd.Series, pd.Series, ValidationStrategy]:
3825     working_columns = [*features, target_column]
3826
3827     if time_column and time_column in frame.columns:

```

```

3828         working_columns.append(time_column)
3829
3830     working = frame[working_columns].copy()
3831     working = working.dropna(subset=[target_column])
3832
3833     if time_column and time_column in working.columns:
3834         parsed_time = pd.to_datetime(
3835             working[time_column],
3836             errors="coerce",
3837             utc=True,
3838         )
3839
3840     parse_rate = float(parsed_time.notna().mean())
3841
3842     if parse_rate >= 0.80:
3843         working = working.loc[parsed_time.notna()].copy()
3844         working["_parsed_time"] = parsed_time.loc[parsed_time.notna()]
3845         working = working.sort_values("_parsed_time")
3846
3847         split_index = int(len(working) * 0.75)
3848         split_index = max(1, min(split_index, len(working) - 1))
3849
3850         train = working.iloc[:split_index]
3851         test = working.iloc[split_index:]
3852
3853         return (
3854             train[features],
3855             test[features],
3856             train[target_column],
3857             test[target_column],
3858             ValidationStrategy.CHRONOLOGICAL_HOLDOUT,
3859         )
3860
3861     x = working[features]
3862     y = working[target_column]
3863
3864     stratify = None
3865
3866     if classification:
3867         counts = y.map(safe_hashable).astype(str).value_counts()
3868         if not counts.empty and counts.min() >= 2:
3869             stratify = y.map(safe_hashable).astype(str)
3870
3871     x_train, x_test, y_train, y_test = train_test_split(
3872         x,
3873         y,
3874         test_size=0.25,
3875         random_state=seed,
3876         stratify=stratify,
3877     )
3878
3879     strategy = (
3880         ValidationStrategy.STRATIFIED_HOLDOUT
3881         if stratify is not None
3882         else ValidationStrategy.RANDOM_HOLDOUT
3883     )
3884
3885     return x_train, x_test, y_train, y_test, strategy
3886
3887 def predictive_analysis(
3888     bundle: DataBundle,
3889     task: InvestigationTask,
3890     builder: EvidenceBuilder,
3891     settings: Settings,
3892 ) -> None:
3893     table_name = task.table_name
3894     target_column = task.target_column
3895
3896     if table_name not in bundle.tables or not target_column:
3897         add_predictive_insufficiency(
3898             builder,
3899             task,
3900             "Predictive modelling was not run because no target was selected.",
3901             {},
3902             ["A prediction target must be selected before model fitting."],
3903         )
3904         return
3905
3906     if target_column not in bundle.tables[table_name].columns:
3907         add_predictive_insufficiency(
3908             builder,
3909             task,
3910             f"The selected target `{target_column}` was not found.",
3911             {},
3912             ["The prediction target is not present in the table."],
3913         )
3914         return
3915
3916     if (
3917         task.target_status == TargetStatus.UNCONFIRMED

```

```

3919         or (
3920             task.target_status == TargetStatus.EXPERIMENTAL_CANDIDATE
3921             and not settings.allow_experimental_targets
3922         )
3923     ):
3924         add_predictive_insufficiency(
3925             builder,
3926             task,
3927             (
3928                 f"`{target_column}` was not used as a predictive target because "
3929                 "the target was not user-selected or metadata-confirmed."
3930             ),
3931             {
3932                 "target_column": target_column,
3933                 "target_status": task.target_status.value,
3934             },
3935             [
3936                 "An unconfirmed target is insufficient for a primary predictive claim.",
3937                 "Experimental candidate targets can be enabled explicitly for ablation work.",
3938             ],
3939             recommendations=[
3940                 recommendation(
3941                     builder,
3942                     action=(
3943                         f"Ask the user to confirm whether `{target_column}` is the "
3944                         "intended prediction target."
3945                     ),
3946                     recommendation_type="methodological_check",
3947                     priority="high",
3948                     justification=(
3949                         "Predictive performance has no stable interpretation without "
3950                         "a defined target and prediction objective."
3951                     ),
3952                 ),
3953             ],
3954         )
3955     return
3956
3957 frame = bundle.tables[table_name].copy()
3958
3959 if len(frame) > settings.max_analysis_rows:
3960     frame = frame.sample(
3961         n=settings.max_analysis_rows,
3962         random_state=settings.random_seed,
3963     ).copy()
3964
3965 frame = frame.dropna(subset=[target_column])
3966
3967 if len(frame) < 80:
3968     add_predictive_insufficiency(
3969         builder,
3970         task,
3971         (
3972             f"Predictive modelling for `{target_column}` was not validated "
3973             f"because only {len(frame):,} rows had a non-missing target."
3974         ),
3975         {"usable_target_rows": len(frame)},
3976         ["At least 80 usable target rows are required."],
3977     )
3978     return
3979
3980 numeric_columns, categorical_columns, excluded, leakage_warnings = (
3981     select_and_audit_features(
3982         frame,
3983         target_column,
3984         task.time_column,
3985         settings.target_proxy_correlation,
3986     )
3987 )
3988
3989 feature_columns = numeric_columns + categorical_columns
3990
3991 if not feature_columns:
3992     add_predictive_insufficiency(
3993         builder,
3994         task,
3995         (
3996             f"No leakage-audited predictor columns remained for "
3997             f"`{target_column}`."
3998         ),
3999         {
4000             "excluded_features": excluded,
4001             "leakage_warnings": leakage_warnings,
4002         },
4003         [
4004             "All candidate predictors were constant, identifier-like, "
4005             "structured, or possible target proxies."
4006         ],
4007     )
4008     return
4009

```

```

4010     target = frame[target_column]
4011
4012     classification = bool(
4013         not is_numeric_measure(target)
4014         or target.map(safe_hashable).nunique(dropna=True) <= 20
4015     )
4016
4017     try:
4018         (
4019             x_train,
4020             x_test,
4021             y_train,
4022             y_test,
4023             validation_strategy,
4024         ) = split_predictive_data(
4025             frame,
4026             feature_columns,
4027             target_column,
4028             task.time_column,
4029             classification,
4030             settings.random_seed,
4031         )
4032
4033         if len(x_test) < 20:
4034             add_predictive_insufficiency(
4035                 builder,
4036                 task,
4037                 "The predictive holdout contained fewer than 20 rows.",
4038                 {
4039                     "train_rows": len(x_train),
4040                     "test_rows": len(x_test),
4041                 },
4042                 ["The holdout is too small for a stable predictive conclusion."],
4043             )
4044         return
4045
4046     if classification:
4047         y_train = y_train.map(safe_hashable).astype(str)
4048         y_test = y_test.map(safe_hashable).astype(str)
4049
4050         baseline = Pipeline(
4051             [
4052                 (
4053                     "preprocess",
4054                     make_preprocessor(
4055                         numeric_columns,
4056                         categorical_columns,
4057                     ),
4058                 ),
4059                 (
4060                     "model",
4061                     DummyClassifier(strategy="most_frequent"),
4062                 ),
4063             ]
4064         )
4065
4066         baseline.fit(x_train, y_train)
4067         baseline_prediction = baseline.predict(x_test)
4068
4069         baseline_f1 = f1_score(
4070             y_test,
4071             baseline_prediction,
4072             average="macro",
4073             zero_division=0,
4074         )
4075
4076         candidates = {
4077             "logistic_regression": LogisticRegression(max_iter=1_000),
4078             "random_forest": RandomForestClassifier(
4079                 n_estimators=150,
4080                 max_depth=10,
4081                 random_state=settings.random_seed,
4082                 n_jobs=-1,
4083             ),
4084         }
4085
4086         results: list[tuple[float, str, float]] = []
4087
4088         for model_name, model in candidates.items():
4089             pipeline = Pipeline(
4090                 [
4091                     (
4092                         "preprocess",
4093                         make_preprocessor(
4094                             numeric_columns,
4095                             categorical_columns,
4096                         ),
4097                     ),
4098                     ("model", model),
4099                 ]
4100             )

```

```

4101         pipeline.fit(x_train, y_train)
4102         prediction = pipeline.predict(x_test)
4103
4104         macro_f1 = f1_score(
4105             y_test,
4106             prediction,
4107             average="macro",
4108             zero_division=0,
4109         )
4110         accuracy = accuracy_score(y_test, prediction)
4111
4112         results.append((macro_f1, model_name, accuracy))
4113
4114     best_f1, best_model, best_accuracy = max(results)
4115     improvement = best_f1 - baseline_f1
4116     validated = improvement > 0.02
4117
4118     metrics = {
4119         "task": "classification",
4120         "target_column": target_column,
4121         "target_status": task.target_status.value,
4122         "prediction_definition": task.prediction_definition,
4123         "validation_strategy": validation_strategy.value,
4124         "best_model": best_model,
4125         "holdout_macro_f1": best_f1,
4126         "holdout_accuracy": best_accuracy,
4127         "baseline_macro_f1": baseline_f1,
4128         "absolute_improvement": improvement,
4129         "train_rows": len(x_train),
4130         "test_rows": len(x_test),
4131         "features_used": feature_columns,
4132         "features_excluded": excluded,
4133         "leakage_warnings": leakage_warnings,
4134     }
4135
4136     finding = (
4137         f"The best leakage-audited classifier for `{target_column}` was "
4138         f"`{best_model}`, with holdout macro-F1 {best_f1:.4f} and "
4139         f"accuracy {best_accuracy:.4f}; the majority-class baseline "
4140         f"macro-F1 was {baseline_f1:.4f}."
4141     )
4142
4143     else:
4144         y_train = pd.to_numeric(y_train, errors="coerce")
4145         y_test = pd.to_numeric(y_test, errors="coerce")
4146
4147         train_valid = y_train.notna()
4148         test_valid = y_test.notna()
4149
4150         x_train = x_train.loc[train_valid]
4151         y_train = y_train.loc[train_valid]
4152         x_test = x_test.loc[test_valid]
4153         y_test = y_test.loc[test_valid]
4154
4155         baseline = Pipeline(
4156             [
4157                 (
4158                     "preprocess",
4159                     make_preprocessor(
4160                         numeric_columns,
4161                         categorical_columns,
4162                     ),
4163                 ),
4164                 ("model", DummyRegressor(strategy="mean")),
4165             ]
4166         )
4167
4168         baseline.fit(x_train, y_train)
4169         baseline_prediction = baseline.predict(x_test)
4170         baseline_mae = mean_absolute_error(y_test, baseline_prediction)
4171
4172         candidates = {
4173             "ridge": Ridge(alpha=1.0),
4174             "random_forest": RandomForestRegressor(
4175                 n_estimators=150,
4176                 max_depth=10,
4177                 random_state=settings.random_seed,
4178                 n_jobs=-1,
4179             ),
4180         }
4181
4182         results: list[
4183             tuple[float, str, float, float, float]
4184         ] = []
4185
4186         for model_name, model in candidates.items():
4187             pipeline = Pipeline(
4188                 [
4189                     (
4190                         "preprocess",
4191                         make_preprocessor(

```



```

4192         numeric_columns,
4193         categorical_columns,
4194     ),
4195     ),
4196     ("model", model),
4197 ]
4198 )
4199 pipeline.fit(x_train, y_train)
4200 prediction = pipeline.predict(x_test)
4201
4202 mae = mean_absolute_error(y_test, prediction)
4203 rmse = mean_squared_error(y_test, prediction) ** 0.5
4204 r_squared = r2_score(y_test, prediction)
4205
4206 results.append(
4207     (-mae, model_name, mae, rmse, r_squared)
4208 )
4209
4210 _, best_model, best_mae, best_rmse, best_r_squared = max(results)
4211
4212 improvement = (
4213     (baseline_mae - best_mae) / baseline_mae
4214     if baseline_mae > 0
4215     else 0.0
4216 )
4217
4218 validated = improvement > 0.05
4219
4220 metrics = {
4221     "task": "regression",
4222     "target_column": target_column,
4223     "target_status": task.target_status.value,
4224     "prediction_definition": task.prediction_definition,
4225     "validation_strategy": validation_strategy.value,
4226     "best_model": best_model,
4227     "holdout_mae": best_mae,
4228     "holdout_rmse": best_rmse,
4229     "holdout_r_squared": best_r_squared,
4230     "baseline_mae": baseline_mae,
4231     "relative_mae_improvement": improvement,
4232     "train_rows": len(x_train),
4233     "test_rows": len(x_test),
4234     "features_used": feature_columns,
4235     "features_excluded": excluded,
4236     "leakage_warnings": leakage_warnings,
4237 }
4238
4239 finding = (
4240     f"The best leakage-audited regressor for `{target_column}` was "
4241     f"`{best_model}`, with holdout MAE {best_mae:.4g}, "
4242     f"RMSE {best_rmse:.4g}, and R2 {best_r_squared:.4f}; "
4243     f"the mean baseline MAE was {baseline_mae:.4g}."
4244 )
4245
4246 if not validated:
4247     finding += (
4248         " The tested model did not improve the relevant baseline "
4249         "by the configured validation threshold."
4250     )
4251
4252 limitations = [
4253     "This is internal validation rather than external validation.",
4254     "Performance may change under distribution shift.",
4255     "Feature availability at the intended prediction time was not "
4256     "independently confirmed.",
4257 ]
4258
4259 if task.target_status == TargetStatus.EXPERIMENTAL_CANDIDATE:
4260     limitations.append(
4261         "The target was selected for an explicit experiment rather than "
4262         "confirmed by the user or metadata."
4263     )
4264
4265 if validation_strategy != ValidationStrategy.CHRONOLOGICAL_HOLDOUT:
4266     limitations.append(
4267         "The evaluation was not chronological; it should not be interpreted "
4268         "as evidence of future performance."
4269     )
4270
4271 if leakage_warnings:
4272     limitations.append(
4273         "Potential proxy features were excluded before modelling."
4274     )
4275
4276 builder.add(
4277     route=AnalysisRoute.PREDICTIVE,
4278     task_ids=[task.task_id],
4279     finding=finding,
4280     metrics=metrics,
4281     source_tables=[table_name],
4282     source_columns=[target_column, *feature_columns],

```

```

4283         method=(
4284             "Leakage-audited baseline comparison using a holdout selected "
4285             "according to the available temporal structure."
4286         ),
4287         validation_strategy=validation_strategy,
4288         practical_interpretation=(
4289             "The model is evidence of internal predictive performance only "
4290             "when it improves the baseline. It is not evidence of causality "
4291             "or deployment readiness."
4292         ),
4293         strength_label=(
4294             "validated_internal_prediction"
4295             if validated
4296             else "model_not_better_than_baseline"
4297         ),
4298         claim_permissions=(
4299             [
4300                 ClaimPermission.PREDICTIVE,
4301                 ClaimPermission.METHODOLOGICAL,
4302             ]
4303             if validated
4304             else [
4305                 ClaimPermission.INSUFFICIENCY,
4306                 ClaimPermission.METHODOLOGICAL,
4307             ]
4308         ),
4309         factual_confidence=0.97,
4310         methodological_strength=(
4311             0.88
4312             if validation_strategy == ValidationStrategy.CHRONOLOGICAL_HOLDOUT
4313             else 0.72
4314         ),
4315         user_relevance=0.85,
4316         salience=0.85,
4317         recommended_use=(
4318             RecommendedUse.MAIN_FINDING
4319             if validated
4320             else RecommendedUse.LIMITATION
4321         ),
4322         limitations=limitations,
4323         recommendations=[
4324             recommendation(
4325                 builder,
4326                 action=(
4327                     "Confirm the prediction time and verify that every retained "
4328                     "feature would be available at that time."
4329                 ),
4330                 recommendation_type="validation",
4331                 priority="high",
4332                 justification=(
4333                     "Internal holdout performance does not prove operational "
4334                     "feature availability or future generalisation."
4335                 ),
4336             )
4337         ],
4338         prohibited_interpretations=[
4339             "Do not claim causality from predictive performance.",
4340             "Do not claim deployment readiness.",
4341             "Do not claim future accuracy unless the validation was explicitly temporal.",
4342             "Do not imply excluded proxy features were used.",
4343         ],
4344     )
4345
4346 except Exception as error:
4347     add_predictive_insufficiency(
4348         builder,
4349         task,
4350         (
4351             f"Predictive modelling for `{target_column}` was inconclusive "
4352             "because execution failed."
4353         ),
4354         {
4355             "error_type": type(error).__name__,
4356             "error_message": str(error),
4357         },
4358         [
4359             f"Execution error: {type(error).__name__}: {error}",
4360         ],
4361     )
4362
4363
4364 def infer_time_structure(
4365     timestamps: pd.Series,
4366 ) -> tuple[str, list[int], int]:
4367     differences = (
4368         timestamps.sort_values()
4369         .diff()
4370         .dropna()
4371         .dt.total_seconds()
4372     )
4373

```

```

4374     if differences.empty:
4375         return "unknown", [], 20
4376
4377     median_seconds = float(differences.median())
4378
4379     if median_seconds <= 5_400:
4380         return "hourly_or_finer", [24, 168], 168
4381
4382     if median_seconds <= 129_600:
4383         return "daily", [7], 28
4384
4385     if median_seconds <= 3_456_000:
4386         return "monthly", [12], 12
4387
4388     return "irregular_or_sparse", [], 20
4389
4390
4391 def autoregressive_predictions(
4392     values: np.ndarray,
4393     train_end: int,
4394     test_start: int,
4395     test_end: int,
4396     lags: list[int],
4397 ) -> np.ndarray | None:
4398     usable_lags = sorted(set(lag for lag in lags if lag > 0))
4399
4400     if not usable_lags:
4401         usable_lags = [1]
4402
4403     maximum_lag = max(usable_lags)
4404
4405     rows: list[list[float]] = []
4406     targets: list[float] = []
4407
4408     for index in range(maximum_lag, train_end):
4409         feature_row = [values[index - lag] for lag in usable_lags]
4410
4411         if not np.isfinite(feature_row).all() or not np.isfinite(values[index]):
4412             continue
4413
4414         rows.append(feature_row)
4415         targets.append(values[index])
4416
4417     if len(rows) < 40:
4418         return None
4419
4420     x_train = np.asarray(rows[-50_000:], dtype=float)
4421     y_train = np.asarray(targets[-50_000:], dtype=float)
4422
4423     model = Ridge(alpha=1.0)
4424     model.fit(x_train, y_train)
4425
4426     test_rows: list[list[float]] = []
4427
4428     for index in range(test_start, test_end):
4429         feature_row = [values[index - lag] for lag in usable_lags]
4430
4431         if not np.isfinite(feature_row).all():
4432             return None
4433
4434         test_rows.append(feature_row)
4435
4436     return model.predict(np.asarray(test_rows, dtype=float))
4437
4438
4439 def forecasting_analysis(
4440     bundle: DataBundle,
4441     task: InvestigationTask,
4442     builder: EvidenceBuilder,
4443     settings: Settings,
4444 ) -> None:
4445     table_name = task.table_name
4446     time_column = task.time_column
4447     target_column = task.target_column
4448
4449     if (
4450         table_name not in bundle.tables
4451         or not time_column
4452         or not target_column
4453         or time_column not in bundle.tables[table_name].columns
4454         or target_column not in bundle.tables[table_name].columns
4455     ):
4456         builder.add(
4457             route=AnalysisRoute.FORECASTING,
4458             task_ids=[task.task_id],
4459             finding=(
4460                 "Forecasting was not run because a valid time column and "
4461                 "numeric target were not available."
4462             ),
4463             metrics={},
4464             source_tables=[table_name],

```

```

4465         source_columns=[
4466             column
4467             for column in [time_column, target_column]
4468             if column
4469         ],
4470         method="Forecast feasibility assessment.",
4471         validation_strategy=ValidationStrategy.NONE,
4472         practical_interpretation=(
4473             "The available plan does not support a validated forecast."
4474         ),
4475         strength_label="forecast_insufficiency",
4476         claim_permissions=[
4477             ClaimPermission.INSUFFICIENCY,
4478             ClaimPermission.METHODOLOGICAL,
4479         ],
4480         factual_confidence=1.0,
4481         methodological_strength=1.0,
4482         user_relevance=0.75,
4483         salience=0.75,
4484         recommended_use=RecommendedUse.LIMITATION,
4485         limitations=[
4486             "No validated forecast claim is available.",
4487         ],
4488     )
4489     return
4490
4491 frame = bundle.tables[table_name][
4492     [time_column, target_column]
4493 ].copy()
4494
4495 frame[time_column] = pd.to_datetime(
4496     frame[time_column],
4497     errors="coerce",
4498     utc=True,
4499 )
4500 frame[target_column] = pd.to_numeric(
4501     frame[target_column],
4502     errors="coerce",
4503 )
4504
4505 frame = frame.dropna().sort_values(time_column)
4506 frame = frame.groupby(
4507     time_column,
4508     as_index=False,
4509 )[target_column].mean()
4510
4511 granularity, seasonal_lags, minimum_test_points = infer_time_structure(
4512     frame[time_column]
4513 )
4514
4515 maximum_lag = max([1, *seasonal_lags])
4516
4517 if len(frame) < maximum_lag + minimum_test_points * 2:
4518     builder.add(
4519         route=AnalysisRoute.FORECASTING,
4520         task_ids=[task.task_id],
4521         finding=(
4522             f"Forecast validation for `{target_column}` was not run because "
4523             f"{len(frame):,} usable time points were insufficient for "
4524             "the inferred evaluation design."
4525         ),
4526         metrics={
4527             "usable_time_points": len(frame),
4528             "time_granularity": granularity,
4529             "minimum_test_points": minimum_test_points,
4530             "seasonal_lags": seasonal_lags,
4531         },
4532         source_tables=[table_name],
4533         source_columns=[time_column, target_column],
4534         method="Temporal coverage and seasonal-lag feasibility assessment.",
4535         validation_strategy=ValidationStrategy.NONE,
4536         practical_interpretation=(
4537             "The time series is too short for the selected rolling evaluation."
4538         ),
4539         strength_label="forecast_insufficiency",
4540         claim_permissions=[
4541             ClaimPermission.INSUFFICIENCY,
4542             ClaimPermission.METHODOLOGICAL,
4543         ],
4544         factual_confidence=1.0,
4545         methodological_strength=1.0,
4546         user_relevance=0.75,
4547         salience=0.75,
4548         recommended_use=RecommendedUse.LIMITATION,
4549         limitations=[
4550             "The series does not contain enough usable points for the "
4551             "required rolling evaluation windows."
4552         ],
4553     )
4554     return
4555

```

```

4556 values = frame[target_column].to_numpy(dtype=float)
4557 length = len(values)
4558
4559 test_window = max(minimum_test_points, int(length * 0.05))
4560 test_window = min(test_window, settings.max_forecast_test_points)
4561 test_window = min(test_window, max(minimum_test_points, length // 5))
4562
4563 available_folds = max(
4564     1,
4565     (length - maximum_lag) // test_window - 1,
4566 )
4567 fold_count = min(settings.forecast_folds, available_folds)
4568
4569 fold_results: list[dict[str, Any]] = []
4570
4571 for fold_index in range(fold_count):
4572     test_end = length - (fold_count - fold_index - 1) * test_window
4573     test_start = test_end - test_window
4574     train_end = test_start
4575
4576     if train_end <= maximum_lag:
4577         continue
4578
4579     actual = values[test_start:test_end]
4580
4581     predictions: dict[str, np.ndarray] = {
4582         "naive_last_value": values[test_start - 1:test_end - 1],
4583     }
4584
4585     for lag in seasonal_lags:
4586         if test_start - lag >= 0:
4587             predictions[
4588                 f"seasonal_naive_lag_{lag}"
4589             ] = values[test_start - lag:test_end - lag]
4590
4591     trend_model = LinearRegression()
4592     training_index = np.arange(train_end).reshape(-1, 1)
4593     testing_index = np.arange(test_start, test_end).reshape(-1, 1)
4594
4595     trend_model.fit(training_index, values[:train_end])
4596     predictions["linear_trend"] = trend_model.predict(testing_index)
4597
4598     autoregressive = autoregressive_predictions(
4599         values,
4600         train_end=train_end,
4601         test_start=test_start,
4602         test_end=test_end,
4603         lags=[1, *seasonal_lags],
4604     )
4605
4606     if autoregressive is not None:
4607         predictions["autoregressive_ridge"] = autoregressive
4608
4609     model_metrics = {
4610         model_name: float(mean_absolute_error(actual, prediction))
4611         for model_name, prediction in predictions.items()
4612     }
4613
4614     fold_results.append(
4615         {
4616             "fold_id": f"FOLD_{fold_index + 1:02d}",
4617             "train_end": frame[time_column].iloc[train_end - 1].isoformat(),
4618             "test_start": frame[time_column].iloc[test_start].isoformat(),
4619             "test_end": frame[time_column].iloc[test_end - 1].isoformat(),
4620             "test_points": len(actual),
4621             "mae": model_metrics,
4622         }
4623     )
4624
4625 if not fold_results:
4626     return
4627
4628 model_names = sorted(
4629     {
4630         model_name
4631         for fold in fold_results
4632         for model_name in fold["mae"]
4633     }
4634 )
4635
4636 mean_mae = {
4637     model_name: float(
4638         np.mean(
4639             [
4640                 fold["mae"][model_name]
4641                 for fold in fold_results
4642                 if model_name in fold["mae"]
4643             ]
4644         )
4645     )
4646     for model_name in model_names

```

```

4647     }
4648
4649     baseline_names = [
4650         name
4651         for name in model_names
4652         if name.startswith("naive_") or name.startswith("seasonal_naive_")
4653     ]
4654     candidate_names = [
4655         name
4656         for name in model_names
4657         if name not in baseline_names
4658     ]
4659
4660     best_baseline = min(
4661         baseline_names,
4662         key=lambda name: mean_mae[name],
4663     )
4664     best_candidate = min(
4665         candidate_names,
4666         key=lambda name: mean_mae[name],
4667     )
4668
4669     baseline_mae = mean_mae[best_baseline]
4670     candidate_mae = mean_mae[best_candidate]
4671
4672     relative_improvement = (
4673         (baseline_mae - candidate_mae) / baseline_mae
4674         if baseline_mae > 0
4675         else 0.0
4676     )
4677
4678     fold_wins = sum(
4679         1
4680         for fold in fold_results
4681         if fold["mae"].get(best_candidate, float("inf"))
4682         < fold["mae"].get(best_baseline, float("inf"))
4683     )
4684
4685     validated = bool(
4686         relative_improvement > 0.05
4687         and fold_wins >= math.ceil(len(fold_results) / 2)
4688     )
4689
4690     if validated:
4691         finding = (
4692             f"`{best_candidate}` achieved mean rolling-origin MAE "
4693             f"`{candidate_mae:.4g}`, compared with `{baseline_mae:.4g}` for "
4694             f"the strongest naive baseline, `{best_baseline}`, across "
4695             f"`{len(fold_results)}` evaluation folds."
4696         )
4697     else:
4698         finding = (
4699             f"The best tested forecasting candidate, `{best_candidate}`, "
4700             f"had mean rolling-origin MAE `{candidate_mae:.4g}`, compared with "
4701             f"`{baseline_mae:.4g}` for the strongest naive baseline, "
4702             f"`{best_baseline}`. It did not provide a validated improvement."
4703         )
4704
4705     builder.add(
4706         route=AnalysisRoute.FORECASTING,
4707         task_ids=[task.task_id],
4708         finding=finding,
4709         metrics={
4710             "target_column": target_column,
4711             "time_column": time_column,
4712             "time_granularity": granularity,
4713             "seasonal_lags": seasonal_lags,
4714             "fold_count": len(fold_results),
4715             "test_window_points": test_window,
4716             "fold_results": fold_results,
4717             "mean_mae": mean_mae,
4718             "best_baseline": best_baseline,
4719             "best_candidate": best_candidate,
4720             "baseline_mae": baseline_mae,
4721             "candidate_mae": candidate_mae,
4722             "relative_improvement": relative_improvement,
4723             "candidate_fold_wins": fold_wins,
4724         },
4725         source_tables=[table_name],
4726         source_columns=[time_column, target_column],
4727         method=(
4728             "Expanding-window rolling-origin, one-step-ahead evaluation "
4729             "against last-value and available seasonal-naive baselines."
4730         ),
4731         validation_strategy=ValidationStrategy.ROLLING_ORIGIN,
4732         practical_interpretation=(
4733             "The candidate is considered useful only when it consistently "
4734             "improves the strongest relevant naive baseline across folds."
4735         ),
4736         strength_label=(
4737             "validated_forecast"

```

```

4738         if validated
4739         else "forecast_not_better_than_baseline"
4740     ),
4741     claim_permissions=(
4742         [
4743             ClaimPermission.FORECAST,
4744             ClaimPermission.METHODOLOGICAL,
4745         ]
4746         if validated
4747         else [
4748             ClaimPermission.INSUFFICIENCY,
4749             ClaimPermission.METHODOLOGICAL,
4750         ]
4751     ),
4752     factual_confidence=0.97,
4753     methodological_strength=0.90,
4754     user_relevance=0.85,
4755     salience=0.85,
4756     recommended_use=(
4757         RecommendedUse.MAIN_FINDING
4758         if validated
4759         else RecommendedUse.LIMITATION
4760     ),
4761     limitations=[
4762         "This is an internal backtest, not a guarantee of live future performance.",
4763         "The evaluation is one-step-ahead and may not represent longer forecast horizons.",
4764         "External drivers and distribution shifts are not represented.",
4765     ],
4766     recommendations=[
4767         recommendation(
4768             builder,
4769             action=(
4770                 "Define the intended forecast horizon explicitly and repeat "
4771                 "evaluation for that horizon."
4772             ),
4773             recommendation_type="validation",
4774             priority="high",
4775             justification=(
4776                 "One-step-ahead performance is not interchangeable with "
4777                 "multi-step forecast performance."
4778             ),
4779         ),
4780     ],
4781     prohibited_interpretations=[
4782         "Do not describe an unsuccessful candidate as a validated forecast.",
4783         "Do not claim certainty about future observations.",
4784         "Do not claim causal explanations for forecast behaviour.",
4785     ],
4786 )
4787
4788
4789 def causal_feasibility_analysis(
4790     bundle: DataBundle,
4791     task: InvestigationTask,
4792     builder: EvidenceBuilder,
4793 ) -> None:
4794     table_name = task.table_name
4795     columns = set(
4796         bundle.tables.get(table_name, pd.DataFrame()).columns
4797     )
4798
4799     exposure_available = bool(
4800         task.exposure_column
4801         and task.exposure_column in columns
4802     )
4803     outcome_available = bool(
4804         task.outcome_column
4805         and task.outcome_column in columns
4806     )
4807     time_available = bool(
4808         task.time_column
4809         and task.time_column in columns
4810     )
4811     confounders = [
4812         column
4813         for column in task.confounder_columns
4814         if column in columns
4815     ]
4816
4817     builder.add(
4818         route=AnalysisRoute.CAUSAL_FEASIBILITY,
4819         task_ids=[task.task_id],
4820         finding=(
4821             "A causal conclusion is not authorised because the workflow has "
4822             "not verified randomisation, a natural experiment, a defensible "
4823             "adjustment set, or another identification strategy."
4824         ),
4825         metrics={
4826             "exposure_available": exposure_available,
4827             "outcome_available": outcome_available,
4828             "time_column_available": time_available,

```



```

4829         "proposed_confounder_count": len(confounders),
4830         "causal_claim_authorised": False,
4831     },
4832     source_tables=[table_name] if table_name in bundle.tables else [],
4833     source_columns=[
4834         column
4835         for column in [
4836             task.exposure_column,
4837             task.outcome_column,
4838             task.time_column,
4839             *confounders,
4840         ]
4841         if column
4842     ],
4843     method="Causal-feasibility checklist; no treatment-effect estimation.",
4844     practical_interpretation=(
4845         "Observed relationships may be described as associations, but the "
4846         "available design does not identify a causal effect."
4847     ),
4848     strength_label="causal_insufficiency",
4849     claim_permissions=[
4850         ClaimPermission.INSUFFICIENCY,
4851         ClaimPermission.METHODOLOGICAL,
4852     ],
4853     factual_confidence=1.0,
4854     methodological_strength=1.0,
4855     user_relevance=0.80,
4856     salience=0.80,
4857     recommended_use=RecommendedUse.LIMITATION,
4858     limitations=[
4859         "The presence of a date column does not establish temporal ordering.",
4860         "Observed association does not identify a causal effect.",
4861     ],
4862     prohibited_interpretations=[
4863         "Do not claim that one observed variable caused another.",
4864         "Do not claim a treatment effect.",
4865     ],
4866 )
4867
4868
4869 def execute_plan(
4870     bundle: DataBundle,
4871     plan: ExecutionPlan,
4872     settings: Settings,
4873     semantic_map: InputSemanticMap | None = None,
4874 ) -> EvidenceLedger:
4875     builder = EvidenceBuilder(bundle.fingerprint)
4876     event_input = bool(
4877         (bundle.input_structure and bundle.input_structure.shape == InputShape.EVENT_RECORD)
4878         or (
4879             semantic_map is not None
4880             and semantic_map.input_shape == InputShape.EVENT_RECORD
4881             and semantic_map.confidence >= 0.7
4882         )
4883     )
4884     event_genre = plan.report_specification.genre in {
4885         ReportGenre.EVENT_REPORT,
4886         ReportGenre.SPORTS_GAME_REPORT,
4887     }
4888     focused_table_task = (
4889         EvidenceCapability.FOCUSED_TABLE_REGION
4890         in set(plan.selected_capabilities)
4891     )
4892     structured_record_task = (
4893         EvidenceCapability.STRUCTURED_RECORD_VERBALISATION
4894         in set(plan.selected_capabilities)
4895     )
4896
4897     if event_input:
4898         event_analysis(bundle, plan, builder, semantic_map)
4899
4900     if focused_table_task:
4901         focused_table_analysis(bundle, plan, builder)
4902
4903     if structured_record_task:
4904         structured_record_verbalisation_analysis(bundle, plan, builder)
4905
4906     for route in plan.route_order:
4907         tasks = tasks_for_route(plan, route)
4908
4909         if route == AnalysisRoute.DESCRPTIVE:
4910             tabular_tasks = [
4911                 task
4912                 for task in tasks
4913                 if task.capability
4914                 not in {
4915                     EvidenceCapability.FOCUSED_TABLE_REGION,
4916                     EvidenceCapability.STRUCTURED_RECORD_VERBALISATION,
4917                     EvidenceCapability.EVENT_OUTCOME,
4918                     EvidenceCapability.ENTITY_PERFORMANCE,
4919                     EvidenceCapability.RANKING,

```

```

4920         EvidenceCapability.GROUP_COMPARISON,
4921     }
4922 ]
4923 if tabular_tasks and not (
4924     (event_input and event_genre)
4925     or focused_table_task
4926     or structured_record_task
4927 ):
4928     descriptive_analysis(bundle, tabular_tasks, builder)
4929
4930 elif route == AnalysisRoute.ASSOCIATION_COMPARISON:
4931     tabular_tasks = [
4932         task
4933         for task in tasks
4934         if task.capability != EvidenceCapability.GROUP_COMPARISON
4935         or not event_input
4936     ]
4937     if tabular_tasks and not (
4938         (event_input and event_genre)
4939         or focused_table_task
4940         or structured_record_task
4941     ):
4942         association_analysis(
4943             bundle,
4944             tabular_tasks,
4945             builder,
4946             settings,
4947         )
4948
4949 elif route == AnalysisRoute.PREDICTIVE:
4950     for task in tasks:
4951         predictive_analysis(
4952             bundle,
4953             task,
4954             builder,
4955             settings,
4956         )
4957
4958 elif route == AnalysisRoute.FORECASTING:
4959     for task in tasks:
4960         forecasting_analysis(
4961             bundle,
4962             task,
4963             builder,
4964             settings,
4965         )
4966
4967 elif route == AnalysisRoute.CAUSAL_FEASIBILITY:
4968     for task in tasks:
4969         causal_feasibility_analysis(
4970             bundle,
4971             task,
4972             builder,
4973         )
4974
4975 if not builder.items:
4976     builder.execution_notes.append(
4977         "The execution plan produced no evidence items."
4978     )
4979
4980 return builder.build()

```

Listing 22: P22: table2text_pydanticai/src/table2text/audit.py

```

1  """Validate, materialise, repair, and audit evidence-grounded writer output."""
2
3  from __future__ import annotations
4
5  import json
6  import math
7  import re
8  from collections.abc import Mapping, Sequence
9  from collections import Counter, defaultdict
10 from typing import Any
11
12 import numpy as np
13 import pandas as pd
14
15 from .capabilities import participation_measure_requested
16 from .schemas import (
17     AnalyticalFunction,
18     AnalysisRoute,
19     AuditAnnotation,
20     AuditDecision,
21     AuditMode,
22     AuditRepairProposal,
23     AuditReport,
24     ClaimPermission,
25     ColumnProfile,

```

```

26     CommunicationTask,
27     DataProfile,
28     ErrorType,
29     EvidenceCapability,
30     EvidenceItem,
31     EvidenceLedger,
32     ExternalTruthSource,
33     FactCandidate,
34     FactCandidateSet,
35     FactLedger,
36     FactReview,
37     GenreQualityAssessment,
38     InsightContribution,
39     InsightLedger,
40     InsightType,
41     InsightVerificationStatus,
42     InputStructureProfile,
43     InterpretationLevel,
44     OutputForm,
45     QualityStatus,
46     ProfileSupportRecord,
47     RecommendedUse,
48     RejectedFact,
49     ReleaseStatus,
50     RepairCandidate,
51     RepairStrategy,
52     ReportComponent,
53     ReportComponentAssessment,
54     ReportGenre,
55     ReportPatch,
56     ReportQualityAssessment,
57     RealisationPolicy,
58     ReviewDecision,
59     SentenceSupport,
60     Severity,
61     SupportMapPatch,
62     SupportType,
63     VerificationMethod,
64     VerificationResult,
65     VerifiedFact,
66     VerifiedInsight,
67     WriterAgentDraft,
68     WriterEvidencePack,
69     WriterOutput,
70 )
71 from .config import Settings
72
73
74 NUMBER_PATTERN = re.compile(
75     r"(?<!\w.)[+]?(?:\d{1,3}(?:,\d{3})+|\d+)(?:\.\d+)?(?:?!w|\.(?=\d))"
76 )
77 DATE_LIKE_TOKEN_PATTERN = re.compile(
78     r"(?<!\w)(\d{1,4})[_/-](\d{1,2})[_/-](\d{1,4})(?!w)"
79 )
80
81 ABBREVIATION_DOT_PLACEHOLDER = "__T2T_ABBR_DOT__"
82 INITIALISM_PATTERN = re.compile(r"\b(?:[A-Z]\.){2,}")
83 SINGLE_INITIAL_PATTERN = re.compile(r"\b([A-Z])\.(?:[s+][A-Z][a-z])")
84 COMMON_SENTENCE_ABBREVIATION_PATTERN = re.compile(
85     r"\b(?:vs|v|etc|e\.g|i\.e)\.",
86     re.IGNORECASE,
87 )
88
89 CAUSAL_PATTERN = re.compile(
90     r"\b(caused|causes|causing|drives?(?!s+in\s+\d)|"
91     r"drove(?:s+in\s+\d)|driven by|led to|effect of|"
92     r"responsible for|result(?:s|ed|ing)? in|"
93     r"contribute(?:e|es|ed|ing) to|because of|due to|explains?)\b",
94     re.IGNORECASE,
95 )
96
97 FACTUAL_TITLE_PATTERN = re.compile(
98     r"\b(defeats?|defeated|beats?|beat|wins?|won|loses?|lost|"
99     r"edges?|edged|outscores?|outscored|leads?|led|draws?|drew|"
100     r"ties?|tied|versus|vs\.\?)\b",
101     re.IGNORECASE,
102 )
103
104 PREDICTIVE_PATTERN = re.compile(
105     r"\b(predicts?|predictive|classifier|regressor|out-of-sample|"
106     r"deployment|generalises?)\b",
107     re.IGNORECASE,
108 )
109
110 FORECAST_PATTERN = re.compile(
111     r"\b(forecasts?|forecasted|future values?|will increase|"
112     r"will decrease|future performance)\b",
113     re.IGNORECASE,
114 )
115
116 APPROXIMATE_PATTERN = re.compile(

```

```

117     r"\b(about|approximately|around|roughly|nearly|close to|"
118     r"more than|over|less than|under)\b",
119     re.IGNORECASE,
120 )
121
122 INTERNAL_CONTROL_PATTERN = re.compile(
123     r"(?i)\b("
124     r"global prohibited interpretations|"
125     r"prohibited interpretations|"
126     r"interpretation notes|"
127     r"recommended use|"
128     r"methodological strength|"
129     r"user relevance|"
130     r"do not say|"
131     r"do not describe"
132     r")\b"
133 )
134
135 FIELD_LABEL_PATTERN = re.compile(
136     r"(?im)^\s*(?:[-*]\s*)?\s*\{0,2}"
137     r"(Finding|Strength|Important Note|Interpretation Notes|"
138     r"Recommended Use|Methodological Strength|User Relevance|"
139     r"Salience|Global Prohibited Interpretations)"
140     r"\s*\{0,2}\s*:"
141 )
142
143 GENERIC_OPENING_PATTERNS = [
144     re.compile(r"\bthis document summarizes\b", re.IGNORECASE),
145     re.compile(r"\bthis report provides\b", re.IGNORECASE),
146     re.compile(r"\bhere's a breakdown\b", re.IGNORECASE),
147     re.compile(r"\bthe goal is to provide\b", re.IGNORECASE),
148 ]
149
150 CAVEAT_PATTERNS = [
151     re.compile(r"does not imply causation", re.IGNORECASE),
152     re.compile(r"does not establish causation", re.IGNORECASE),
153     re.compile(r"\bunadjusted\b", re.IGNORECASE),
154     re.compile(r"\bconfound(?:er|ers|ing)\b", re.IGNORECASE),
155 ]
156
157 IMBALANCE_BIAS_PATTERN = re.compile(
158     r"\b("
159     r"imbalanc\w*[\^.!?]{0,80}bias\w*"
160     r"bias\w*[\^.!?]{0,80}imbalanc\w*"
161     r")\b",
162     re.IGNORECASE,
163 )
164
165 UNSANCTIONED_UNIT_TERMS = {
166     "event",
167     "events",
168     "experiment",
169     "experiments",
170     "subject",
171     "subjects",
172 }
173
174 HOURLY_CADENCE_PATTERN = re.compile(
175     r"\b(hourly observations?|hourly measurements?|every hour|"
176     r"one-hour intervals?)\b",
177     re.IGNORECASE,
178 )
179
180 LOCATION_METADATA_PATTERN = re.compile(
181     r"\b(specific location|weather station|recorded at a location)\b",
182     re.IGNORECASE,
183 )
184
185 CONSTANT_OVERSTATEMENT_PATTERN = re.compile(
186     r"\b(no analytical value|useless|worthless|has no value)\b",
187     re.IGNORECASE,
188 )
189
190 ZERO_OVERCONFIDENCE_PATTERN = re.compile(
191     r"\b(likely represents encoded missingness|definitely represents|"
192     r"is measurement failure|must be erroneous)\b",
193     re.IGNORECASE,
194 )
195
196 MISSINGNESS_HARMLESS_PATTERN = re.compile(
197     r"\b(unlikely to cause major issues|can be safely ignored|"
198     r"will not affect the analysis|has no material effect)\b",
199     re.IGNORECASE,
200 )
201
202 DUPLICATE_REMOVAL_PATTERN = re.compile(
203     r"\b(duplicates(?: rows)? should be removed|"
204     r"duplicates(?: rows)? should likely be removed|"
205     r"likely removed|must be removed|automatically deduplicate)\b",
206     re.IGNORECASE,
207 )

```

```

208
209 PEARSON_IMPRECISE_PATTERN = re.compile(
210     r"\bpearson correlation\b[^\.?]{0,100}\b"
211     r"(?:is|may be)?\s*influenced by non-linear patterns\b",
212     re.IGNORECASE,
213 )
214
215 FUTURE_ANALYSIS_PATTERN = re.compile(
216     r"\b(future work should|next analyses should|could model|"
217     r"multivariate model|explore temporal trends|"
218     r"investigate the relationship)\b",
219     re.IGNORECASE,
220 )
221
222 DATASET_GENERALISATION_PATTERN = re.compile(
223     r"\b(always|in general|universally|proves that|demonstrates that all)\b",
224     re.IGNORECASE,
225 )
226
227 UNSUPPORTED_SPORTS_NARRATIVE_PATTERN = re.compile(
228     r"\b(dominated throughout|single-handedly|comeback|turning point|"
229     r"all-time classic|historic victory|upset|cruised to victory)\b",
230     re.IGNORECASE,
231 )
232
233 SCORE_STATE_TIE_CLAIM_PATTERN = re.compile(
234     r"\b(?:game[- ]tying|tie[ds]?s+(?:the\s+)?game|"
235     r"tying\s+(?:the\s+)?game|level(?:led)?s+(?:the\s+)?score|"
236     r"score\s+was\s+level|equali[sz](?:ed|er))\b",
237     re.IGNORECASE,
238 )
239
240 NEGATED_SCORE_STATE_TIE_PATTERN = re.compile(
241     r"\b(?:could|can|did|does|would|will|was|were|is|are)?\s*"
242     r"(?:not|n't|never|failed\s+to|unable\s+to|without)\s+"
243     r"(?:tie[ds]?s+(?:the\s+)?game|tying\s+(?:the\s+)?game|"
244     r"level(?:led)?s+(?:the\s+)?score|"
245     r"equali[sz](?:ed|er))\b",
246     re.IGNORECASE,
247 )
248
249 INSIGHT_OVERSTATEMENT_PATTERN = re.compile(
250     r"\b(completely redundant|universally redundant|should always be removed|"
251     r"must always be removed|proves that)\b",
252     re.IGNORECASE,
253 )
254
255 HYPOTHESIS_WORDING_PATTERN = re.compile(
256     r"\b(hypothesis|hypothesise|hypothesize|question for further "
257     r"investigation)\b",
258     re.IGNORECASE,
259 )
260
261 UNLABELLED_HYPOTHESIS_PATTERN = re.compile(
262     r"\b(may|might|could|likely)\b[^\.?]{0,100}\b"
263     r"(because|reflects?|indicates?|explains?|due to)\b",
264     re.IGNORECASE,
265 )
266
267 EXPLANATORY_HYPOTHESIS_PATTERN = re.compile(
268     r"(?:"
269     r"\b(?:may|might|could|possibly|potentially|whether)\b"
270     r"[^\.?]{0,180}\b(?:"
271     r"reflect(?:s|ed|ing)?|"
272     r"result(?:s|ed|ing)?\s+from|"
273     r"stems?|stemmed|stemming|"
274     r"arises?|arose|arisen|"
275     r"be\s+due\s+to|"
276     r"be\s+explained\s+by"
277     r")\b|"
278     r"\b(?:possible|plausible|potential)\s+"
279     r"(?:explanation|reason)\b"
280     r")",
281     re.IGNORECASE,
282 )
283
284 ASSOCIATION_STRENGTH_PATTERN = re.compile(
285     r"\b(?:P<label>very strong|strong|moderate|weak|negligible)\b"
286     r"(?:\s+(?:positive|negative))?"
287     r"(?:\s+(?:pearson|spearman|rank-based|linear))?"
288     r"\s+(?:correlations?|associations?|relationships?)\b",
289     re.IGNORECASE,
290 )
291
292 GROUP_STRENGTH_PATTERN = re.compile(
293     r"\b(?:P<label>large|moderate|small|negligible)\b"
294     r"(?:\s+standard(?:ised|ized))?"
295     r"\s+(?:group\s+)?(?:differences?|effect sizes?)\b",
296     re.IGNORECASE,
297 )
298
299 INTERPRETIVE_SYNTHESIS_PATTERN = re.compile(

```

```

299     r"\b(overlapping information|taken together|together suggest|"
300     r"combined with|overall pattern|this indicates|this suggests|therefore)\b",
301     re.IGNORECASE,
302 )
303
304
305 def materially_same_report_text(
306     left: str,
307     right: str,
308 ) -> bool:
309     left_normalised = re.sub(
310         r"^[^a-z0-9]+",
311         "",
312         left.lower(),
313     ).strip()
314     right_normalised = re.sub(
315         r"^[^a-z0-9]+",
316         "",
317         right.lower(),
318     ).strip()
319
320     if left_normalised == right_normalised:
321         return True
322
323     left_tokens = {
324         token
325         for token in left_normalised.split()
326         if len(token) > 2
327     }
328     right_tokens = {
329         token
330         for token in right_normalised.split()
331         if len(token) > 2
332     }
333     if not left_tokens or not right_tokens:
334         return False
335
336     return (
337         len(left_tokens & right_tokens)
338         / len(left_tokens | right_tokens)
339         >= 0.8
340     )
341
342 PROFILE_FACT_KIND_TERMS = {
343     "table_dimensions": re.compile(
344         r"\b(rows?|columns?|observations?|records?)\b",
345         re.IGNORECASE,
346     ),
347     "duplicate_rows": re.compile(
348         r"\b(duplicate|duplicates|duplicate rows?)\b",
349         re.IGNORECASE,
350     ),
351     "column_missingness": re.compile(
352         r"\b(missing|missingness)\b",
353         re.IGNORECASE,
354     ),
355     "constant_column": re.compile(
356         r"\b(constant|all zeros?|no observed variation)\b",
357         re.IGNORECASE,
358     ),
359     "near_constant_column": re.compile(
360         r"\b(near-constant|near constant|dominant value)\b",
361         re.IGNORECASE,
362     ),
363     "numeric_summary": re.compile(
364         r"\b(mean|median|minimum|maximum|standard deviation)\b",
365         re.IGNORECASE,
366     ),
367     "zero_diagnostic": re.compile(
368         r"\b(zero|zeros|sentinel)\b",
369         re.IGNORECASE,
370     ),
371     "datetime_presence": re.compile(
372         r"\b(date|datetime|timestamp|time column)\b",
373         re.IGNORECASE,
374     ),
375     "candidate_key": re.compile(
376         r"\b(candidate key|unique identifier)\b",
377         re.IGNORECASE,
378     ),
379 }
380
381 QUALITY_STATUS_ORDER = {
382     QualityStatus.PASS: 0,
383     QualityStatus.WARNING: 1,
384     QualityStatus.REVISE: 2,
385 }
386
387
388 def count_caveat_mentions(markdown: str) -> int:
389     return sum(len(pattern.findall(markdown)) for pattern in CAVEAT_PATTERNS)

```

```

390
391
392 def minimum_useful_report_words(
393     *,
394     target_words: int,
395     required_component_count: int,
396     settings: Settings,
397 ) -> int:
398     """
399     Return a diagnostic minimum that never exceeds the planned target.
400     """
401
402     bounded_target = max(1, target_words)
403
404     ratio_floor = int(
405         bounded_target
406         * settings.minimum_report_word_ratio
407     )
408
409     component_floor = (
410         required_component_count * 45
411     )
412
413     desired_minimum = max(
414         settings.minimum_report_word_floor,
415         ratio_floor,
416         component_floor,
417     )
418
419     return min(
420         bounded_target,
421         desired_minimum,
422     )
423
424 def json_safe(value: Any) -> Any:
425     if value is None or isinstance(value, (str, int, bool)):
426         return value
427
428     if isinstance(value, float):
429         return None if math.isnan(value) or math.isinf(value) else value
430
431     if isinstance(value, np.integer):
432         return int(value)
433
434     if isinstance(value, np.floating):
435         number = float(value)
436         return None if math.isnan(number) or math.isinf(number) else number
437
438     if isinstance(value, pd.Timestamp):
439         return value.isoformat()
440
441     if isinstance(value, dict):
442         return {
443             str(key): json_safe(item)
444             for key, item in value.items()
445         }
446
447     if isinstance(value, (list, tuple, set)):
448         return [json_safe(item) for item in value]
449
450     if hasattr(value, "model_dump"):
451         return json_safe(value.model_dump(mode="json"))
452
453     return str(value)
454
455
456 def compact_json(
457     value: Any,
458     maximum_characters: int = 160_000,
459 ) -> str:
460     rendered = json.dumps(
461         json_safe(value),
462         indent=2,
463         ensure_ascii=False,
464     )
465
466     if len(rendered) <= maximum_characters:
467         return rendered
468
469     return rendered[:maximum_characters] + "\n... [truncated by controller]"
470
471
472 def flatten_numbers(value: Any) -> list[float]:
473     numbers: list[float] = []
474
475     if isinstance(value, bool) or value is None:
476         return numbers
477
478     if isinstance(value, (int, float, np.integer, np.floating)):
479         number = float(value)
480         if math.isfinite(number):

```



```

481         numbers.append(number)
482     return numbers
483
484     if isinstance(value, str):
485         token = value.strip()
486         if NUMBER_PATTERN.fullmatch(token):
487             percentage = token.endswith("%")
488             number = float(token.rstrip("%").replace(",", ""))
489             if percentage:
490                 number /= 100.0
491             if math.isfinite(number):
492                 numbers.append(number)
493     return numbers
494
495     if isinstance(value, dict):
496         for item in value.values():
497             numbers.extend(flatten_numbers(item))
498     return numbers
499
500     if isinstance(value, (list, tuple, set)):
501         for item in value:
502             numbers.extend(flatten_numbers(item))
503
504     return numbers
505
506
507 def extract_number_tokens(text: str) -> list[tuple[str, float]]:
508     tokens: list[tuple[str, float]] = []
509
510     for raw in NUMBER_PATTERN.findall(text or ""):
511         percentage = raw.endswith("%")
512         cleaned = raw.rstrip("%").replace(",", "")
513
514         try:
515             number = float(cleaned)
516         except ValueError:
517             continue
518
519         if percentage:
520             number /= 100.0
521
522         tokens.append((raw, number))
523
524     for match in DATE_LIKE_TOKEN_PATTERN.finditer(text or ""):
525         parts = [part for part in match.groups()]
526         for part in parts:
527             try:
528                 tokens.append((part, float(part)))
529             except ValueError:
530                 continue
531         if len(parts[-1]) == 2 and len(parts[0]) != 4:
532             try:
533                 year = int(parts[-1])
534             except ValueError:
535                 continue
536         tokens.append((f"20{year:02d}", float(2000 + year)))
537
538     return tokens
539
540
541 def number_supported(
542     raw_token: str,
543     number: float,
544     support_numbers: list[float],
545     sentence: str,
546 ) -> bool:
547     """
548     Check whether a rendered number is supported.
549
550     The tolerance accounts for ordinary display rounding. This is
551     particularly important for percentages, where a deterministic
552     value such as 0.005359 may be rendered as 0.54%.
553     """
554
555     approximate = bool(
556         APPROXIMATE_PATTERN.search(sentence)
557     )
558
559     token_without_percent = (
560         raw_token.rstrip("%").replace(",", "")
561     )
562
563     if "." in token_without_percent:
564         decimal_places = len(
565             token_without_percent.split(".", 1)[1]
566         )
567     else:
568         decimal_places = 0
569
570     displayed_resolution = 10.0 ** (-decimal_places)
571

```

```

572     if raw_token.endswith("%"):
573         displayed_resolution /= 100.0
574
575     rounding_tolerance = (
576         displayed_resolution / 2.0
577     ) + 1e-12
578
579     for candidate in support_numbers:
580         relative_tolerance = max(
581             1e-6,
582             abs(candidate) * 0.001,
583         )
584
585         exact_tolerance = max(
586             rounding_tolerance,
587             relative_tolerance,
588         )
589
590         if abs(number - candidate) <= exact_tolerance:
591             return True
592
593         if approximate:
594             approximate_tolerance = max(
595                 displayed_resolution,
596                 0.01,
597                 abs(candidate) * 0.03,
598             )
599
600             if (
601                 abs(number - candidate)
602                 <= approximate_tolerance
603             ):
604                 return True
605
606     digits = token_without_percent
607
608     if (
609         "." not in digits
610         and len(digits) >= 4
611         and digits.endswith("000")
612     ):
613         if abs(number - candidate) <= 500:
614             return True
615
616     return False
617
618 def numbers_supported(
619     text: str,
620     support_numbers: list[float],
621 ) -> bool:
622     return all(
623         number_supported(
624             raw_token,
625             number,
626             support_numbers,
627             text,
628         )
629         for raw_token, number in extract_number_tokens(text)
630     )
631
632
633 def protect_sentence_abbreviations(text: str) -> str:
634     def replace_initialism(match: re.Match[str]) -> str:
635         return match.group(0).replace(
636             ".",
637             ABBREVIATION_DOT_PLACEHOLDER,
638         )
639
640     protected = INITIALISM_PATTERN.sub(
641         replace_initialism,
642         text,
643     )
644     protected = COMMON_SENTENCE_ABBREVIATION_PATTERN.sub(
645         replace_initialism,
646         protected,
647     )
648     return SINGLE_INITIAL_PATTERN.sub(
649         rf"\1{ABBREVIATION_DOT_PLACEHOLDER}",
650         protected,
651     )
652
653
654 def restore_sentence_abbreviations(text: str) -> str:
655     return text.replace(
656         ABBREVIATION_DOT_PLACEHOLDER,
657         ".",
658     )
659
660
661 def split_markdown_sentences(markdown: str) -> list[str]:
662     sentences: list[str] = []

```

```

663
664 for raw_line in (markdown or "").splitlines():
665     line = raw_line.strip()
666
667     if (
668         not line
669         or line.startswith("#")
670         or line.startswith("|")
671         or line.startswith("`")
672     ):
673         continue
674
675     line = re.sub(r"^[^*]\s+", "", line)
676     line = protect_sentence_abbreviations(line)
677
678     parts = re.split(
679         r"(?<=[.!?])\s+(?=[A-Z0-9`])",
680         line,
681     )
682
683     sentences.extend(
684         restore_sentence_abbreviations(part.strip())
685         for part in parts
686         if part.strip()
687     )
688
689     return sentences
690
691 def looks_factual(sentence: str) -> bool:
692     return bool(
693         extract_number_tokens(sentence)
694         or re.search(
695             r"\b(dataset|table|rows?|columns?|mean|median|correlations?|"
696             r"associations?|relationships?|"
697             r"model|forecast|missing|missingness|temperature|humidity|visibility|"
698             r"higher|lower|increase|decrease|associated|observed|"
699             r"duplicate|constant|zeros?|sentinel|timestamp|hourly|"
700             r"location|station|pearson|future work|next analyses|"
701             r"team|player|game|match|won|scored|points?|rebounds?|"
702             r"turnovers?|comeback|dominated|historic|upset)\b",
703             sentence,
704             re.IGNORECASE,
705         )
706     )
707
708
709 def build_evidence_lookup(
710     ledger: EvidenceLedger,
711 ) -> dict[str, EvidenceItem]:
712     return {
713         item.evidence_id: item
714         for item in ledger.items
715     }
716
717
718 def evidence_lookup(
719     ledger: EvidenceLedger,
720 ) -> dict[str, EvidenceItem]:
721     return build_evidence_lookup(ledger)
722
723
724 def evidence_for_fact(
725     fact: VerifiedFact,
726     lookup: dict[str, EvidenceItem],
727 ) -> list[EvidenceItem]:
728     return [
729         lookup[evidence_id]
730         for evidence_id in fact.evidence_ids
731         if evidence_id in lookup
732     ]
733
734
735 EVENT_CONTENT_UNIT_DEFINITIONS = {
736     "event_result": {
737         "description": "State the supported event result or outcome.",
738         "evidence_types": {"event_outcome"},
739         "minimum": 1,
740     },
741     "event_context": {
742         "description": "Include supplied event-level time or location context.",
743         "evidence_types": {"event_context"},
744         "minimum": 1,
745     },
746     "event_status": {
747         "description": "Include supplied event status when available.",
748         "evidence_types": {"event_status"},
749         "minimum": 1,
750     },
751     "participant_record_context": {
752         "description": (

```

```

754         "Include supplied participant record or standing context when "
755         "it helps frame the event."
756     ),
757     "evidence_types": {"participant_record_context"},
758     "minimum": 1,
759 },
760 "score_progression": {
761     "description": (
762         "Use supplied segment-level score progression when it is available."
763     ),
764     "evidence_types": {"score_progression"},
765     "minimum": 1,
766 },
767 "event_sequence": {
768     "description": (
769         "Use supplied ordered event-sequence or score-changing evidence "
770         "when it is available."
771     ),
772     "evidence_types": {"event_sequence"},
773     "minimum": 1,
774 },
775 "leading_performance": {
776     "description": (
777         "Use leading entity rankings or entity-performance facts."
778     ),
779     "evidence_types": {"entity_ranking", "entity_performance"},
780     "minimum": 2,
781 },
782 "main_contrast": {
783     "description": "Use participant-level contrasts supported by evidence.",
784     "evidence_types": {
785         "participant_comparison",
786         "event_contrast",
787         "group_comparison",
788     },
789     "minimum": 2,
790 },
791 "secondary_performance": {
792     "description": "Use additional supported entity rankings if available.",
793     "evidence_types": {"entity_ranking", "entity_performance"},
794     "minimum": 1,
795 },
796 }
797
798
799 EVENT_NARRATIVE_CONNECTOR_PATTERN = re.compile(
800     r"\b(
801         r"while|whereas|despite|although|however|but|in contrast|compared with|"
802         r"compared to|than|more|fewer|less|higher|lower|advantage|margin|"
803         r"followed by|also|both|combined|concentrated|balanced"
804         r")\b",
805     re.IGNORECASE,
806 )
807
808 EVENT_SCOPE_LIMITATION_PATTERN = re.compile(
809     r"\b(
810         r"describe(?:s|d)? only|supplied event|single event|this event|"
811         r"this game|broader performance|broader outcomes|generaliz|"
812         r"does not establish why|do not establish why|does not imply|"
813         r"do not imply|not evidence of causal|does not establish causality|"
814         r"do not establish causality|does not establish causal|"
815         r"do not establish causal|limited to values present|"
816         r"limited to (?:the )?supplied|scope limitations?"
817         r"rankings? (?:are|is) limited"
818         r")\b",
819     re.IGNORECASE,
820 )
821
822 EVENT_SEQUENCE_ABSENCE_PATTERN = re.compile(
823     r"\b(?:does not|do not|cannot|can't|lacks?|missing|without)\b"
824     r"(:?(?!\\.){0,120})"
825     r"\b(?:chronolog(?:y|ical)|sequence|timeline|progression|"
826     r"event dynamics|game dynamics|score(?:ing)? state|score(?:ing)? "
827     r"progression|play[- ]?by[- ]?play)\b",
828     re.IGNORECASE,
829 )
830
831 EVENT_SEQUENCE_OMISSION_PATTERN = re.compile(
832     r"\b(?:this\s+)?(?:report|summary)\b(?:^|{0,120})"
833     r"\b(?:does\s+not|did\s+not|not|no)\b(?:^|{0,80})"
834     r"\b(?:analy[sz]e[ds]?|include[ds]?|report(?:ed)?|cover(?:ed)?)\b|"
835     r"\b(?:detailed\s+)?(?:event\s+)?(?:chronology|play[- ]?by[- ]?play|"
836     r"sequence)\b(?:^|{0,120})\b(?:not|no)\b(?:^|{0,80})"
837     r"\b(?:analy[sz]ed|included|reported|covered)\b",
838     re.IGNORECASE,
839 )
840
841 EVENT_SEGMENT_RANKING_PATTERN = re.compile(
842     r"\b(?:inning|half(?:-|\s)?inning|period|quarter|round|segment|"
843     r"phase|frame|set|leg|stage|play|turn|timeline|sequence)\b",
844     re.IGNORECASE,
845 )
846
847 EVENT_LOW_PRIORITY_ENTITY_METRIC_PATTERN = re.compile(
848     r"\b(?:against|allowed|at\s+bats?|batters?\s+faced|blown|career|"

```

```

845     r"conceded|earned|errors?|holds?|loss(?:es)?|number\s+of\s+pitches|"
846     r"pitch(?:es)?|putouts?|season|strikes?|turnovers?)\b",
847     re.IGNORECASE,
848 )
849 EVENT_RECAP_CORE_ENTITY_METRIC_PATTERN = re.compile(
850     r"\b(?:assists?|blocks?|doubles?|goals?|hits?|home runs?|points?"
851     r"rebounds?|runs(?: batted in)?|saves?|scor(?:e|ed|ing)|steals?"
852     r"strikeouts?|triples?)\b",
853     re.IGNORECASE,
854 )
855 EVENT_RECAP_LOW_PRIORITY_METRIC_PATTERN = re.compile(
856     r"\b(?:attempt(?:ed|s)?|capacity|field[- ]?goal percentage|"
857     r"free[- ]?throw percentage|game number|minutes?|personal fouls?"
858     r"shoot(?:ing)? percentage|three[- ]?point percentage|"
859     r"turnovers?)\b",
860     re.IGNORECASE,
861 )
862
863
864 def _fact_ids_for_evidence_types(
865     *,
866     facts: list[VerifiedFact],
867     evidence_lookup: dict[str, EvidenceItem],
868     evidence_types: set[str],
869 ) -> list[str]:
870     return [
871         fact.fact_id
872         for fact in facts
873         if any(
874             item.evidence_type in evidence_types
875             for item in evidence_for_fact(
876                 fact,
877                 evidence_lookup,
878             )
879         )
880     ]
881
882
883 def _insight_ids_for_fact_ids(
884     *,
885     insights: list[VerifiedInsight],
886     fact_ids: set[str],
887 ) -> list[str]:
888     return [
889         insight.insight_id
890         for insight in insights
891         if set(insight.source_fact_ids) & fact_ids
892     ]
893
894
895 def sentence_support_narrative_stats(
896     sentence_support: list[Any],
897 ) -> dict[str, int]:
898     stats = {
899         "synthesis_sentences": 0,
900         "insight_sentences": 0,
901         "connective_sentences": 0,
902         "scope_limitation_sentences": 0,
903     }
904
905     for support in sentence_support:
906         text = (
907             getattr(support, "sentence_text", None)
908             or getattr(support, "text", "")
909             or ""
910         )
911         fact_ids = set(getattr(support, "fact_ids", []))
912         evidence_ids = set(getattr(support, "evidence_ids", []))
913         insight_ids = set(getattr(support, "insight_ids", []))
914         support_type = getattr(support, "support_type", None)
915         interpretation_level = getattr(
916             support,
917             "interpretation_level",
918             None,
919         )
920
921         support_item_count = len(fact_ids | evidence_ids) + len(insight_ids)
922         is_insight_sentence = bool(insight_ids) and (
923             interpretation_level == InterpretationLevel.BOUNDED_INSIGHT
924         )
925         is_synthesis_sentence = (
926             support_type == SupportType.MULTI_FACT_SYNTHESIS
927             or is_insight_sentence
928             or support_item_count >= 2
929         )
930
931         if is_synthesis_sentence:
932             stats["synthesis_sentences"] += 1
933             if EVENT_NARRATIVE_CONNECTOR_PATTERN.search(text):
934                 stats["connective_sentences"] += 1
935

```

```

936         if is_insight_sentence:
937             stats["insight_sentences"] += 1
938
939         if EVENT_SCOPE_LIMITATION_PATTERN.search(text):
940             stats["scope_limitation_sentences"] += 1
941
942     return stats
943
944
945 def event_scope_limitation_present(
946     writer_output: WriterOutput,
947 ) -> bool:
948     return bool(
949         EVENT_SCOPE_LIMITATION_PATTERN.search(
950             writer_output.markdown
951         )
952     )
953
954
955 def build_writer_content_requirements(
956     *,
957     report_specification: Any,
958     fact_ledger: FactLedger,
959     evidence: EvidenceLedger,
960     insight_ledger: InsightLedger,
961     settings: Settings,
962 ) -> Dict[str, Any]:
963     minimum_words = minimum_useful_report_words(
964         target_words=report_specification.target_length_words,
965         required_component_count=len(
966             report_specification.required_components
967         ),
968         settings=settings,
969     )
970
971     communication_task = getattr(
972         report_specification,
973         "communication_task",
974         None,
975     )
976
977     output_form = getattr(
978         report_specification,
979         "output_form",
980         None,
981     )
982
983     realisation_policy = getattr(
984         report_specification,
985         "realisation_policy",
986         RealisationPolicy.STRICT_SOURCE_SURFACE,
987     )
988
989     realisation_policy_value = (
990         realisation_policy.value
991         if isinstance(realisation_policy, RealisationPolicy)
992         else str(realisation_policy)
993     )
994
995     if communication_task == CommunicationTask.FOCUSED_TABLE_DESCRIPTION:
996         lookup = build_evidence_lookup(evidence)
997         candidate_fact_ids = _fact_ids_for_evidence_types(
998             facts=fact_ledger.writer_ready_facts,
999             evidence_lookup=lookup,
1000             evidence_types={"focused_table_region", "focused_cell_context"},
1001         )
1002
1003         candidate_insight_ids = [
1004             insight.insight_id
1005             for insight in insight_ledger.verified_insights
1006             if (
1007                 EvidenceCapability.FOCUSED_TABLE_REGION
1008                 in insight.source_capabilities
1009             )
1010         ]
1011
1012         units = []
1013         if candidate_fact_ids or candidate_insight_ids:
1014             units.append(
1015                 {
1016                     "unit_id": "focused_table_region",
1017                     "description": (
1018                         "Express the concise relation conveyed by the "
1019                         "selected table cell or focused region using its "
1020                         "supplied row, header, table and source-text context."
1021                     ),
1022                     "minimum_items": 1,
1023                     "candidate_fact_ids": candidate_fact_ids,
1024                     "candidate_insight_ids": candidate_insight_ids,
1025                 }
1026             )
1027
1028     return {
1029         "minimum_word_count": 1,
1030         "enforce_minimum_words": False,
1031         "output_form": (
1032             output_form.value
1033             if isinstance(output_form, OutputForm)

```

```

1027         else str(output_form or OutputForm.ONE_SENTENCE.value)
1028     ),
1029     "allow_headings": False,
1030     "max_sentences": getattr(report_specification, "max_sentences", 1) or 1,
1031     "max_paragraphs": getattr(report_specification, "max_paragraphs", 1) or 1,
1032     "require_complete_sentence": True,
1033     "realisation_policy": realisation_policy_value,
1034     "style_rewrite_permissions": {
1035         "may_normalise_identifier_separators": False,
1036         "must_preserve_numbers_exactly": True,
1037         "must_preserve_percentages_exactly": True,
1038         "must_not_add_caveats": True,
1039         "must_not_add_headings": True,
1040     },
1041     "short_form_selection_policy": {
1042         "prefer_highlighted_role_value_pairs": True,
1043         "prefer_primary_subject_candidate": True,
1044         "treat_non_highlighted_row_values_as_context": True,
1045         "omit_secondary_numeric_values_unless_needed": True,
1046         "avoid_joint_subject_without_combined_entity_evidence": True,
1047         "use_supplied_highlighted_measure_comparisons": True,
1048         "use_supplied_highlighted_set_contrasts": True,
1049         "use_supplied_highlighted_record_groups": True,
1050         "use_supplied_focused_record_relations": True,
1051         "use_supplied_focused_list_relations": True,
1052         "scope_lower_higher_to_highlighted_set": True,
1053         "preserve_numeric_surface_forms": True,
1054         "preserve_identifier_surface_forms": True,
1055         "avoid_spelling_out_numbers": True,
1056     },
1057     "units": units,
1058 }
1059
1060 if communication_task in {
1061     CommunicationTask.ATTRIBUTE_VERBALISATION,
1062     CommunicationTask.TRIPLE_VERBALISATION,
1063 }:
1064     lookup = build_evidence_lookup(evidence)
1065     candidate_fact_ids = _fact_ids_for_evidence_types(
1066         facts=fact_ledger.writer_ready_facts,
1067         evidence_lookup=lookup,
1068         evidence_types={
1069             "attribute_record",
1070             "triple_record",
1071             "structured_record",
1072         },
1073     )
1074     candidate_insight_ids = [
1075         insight.insight_id
1076         for insight in insight_ledger.verified_insights
1077         if (
1078             EvidenceCapability.STRUCTURED_RECORD_VERBALISATION
1079             in insight.source_capabilities
1080         )
1081     ]
1082     units = []
1083     if candidate_fact_ids or candidate_insight_ids:
1084         units.append(
1085             {
1086                 "unit_id": "structured_record_verbalisation",
1087                 "description": (
1088                     "Express all and only the supplied attributes or "
1089                     "triples as concise natural language. Do not add "
1090                     "dataset-profile, data-quality, correlation or "
1091                     "modelling discussion."
1092                 ),
1093                 "minimum_items": 1,
1094                 "candidate_fact_ids": candidate_fact_ids,
1095                 "candidate_insight_ids": candidate_insight_ids,
1096             }
1097         )
1098     return {
1099         "minimum_word_count": 1,
1100         "enforce_minimum_words": False,
1101         "output_form": (
1102             output_form.value
1103             if isinstance(output_form, OutputForm)
1104             else str(output_form or OutputForm.SHORT_TEXT.value)
1105         ),
1106         "allow_headings": False,
1107         "max_sentences": getattr(report_specification, "max_sentences", 2) or 2,
1108         "max_paragraphs": getattr(report_specification, "max_paragraphs", 1) or 1,
1109         "require_complete_sentence": True,
1110         "realisation_policy": realisation_policy_value,
1111         "style_rewrite_permissions": {
1112             "may_normalise_identifier_separators": (
1113                 realisation_policy_value
1114                 == RealisationPolicy.NATURAL_REFERENCE_STYLE.value
1115             ),
1116             "may_humanise_relation_labels": True,
1117             "must_preserve_numbers_exactly": True,

```



```

1118         "must_preserve_units": True,
1119         "must_not_add_caveats": True,
1120         "must_not_add_headings": True,
1121     },
1122     "short_form_selection_policy": {
1123         "use_all_supplied_records": True,
1124         "prefer_natural_phrasing": True,
1125         "avoid_key_value_dump_when_relation_is_clear": True,
1126         "do_not_add_unsupplied_attributes": True,
1127         "do_not_discuss_dataset_profile": True,
1128         "preserve_numeric_surface_forms": True,
1129         "preserve_identifier_surface_forms": True,
1130         "avoid_spelling_out_numbers": True,
1131         "preserve_units_compactly": True,
1132         "prefer_ordinal_rank_phrasing": True,
1133     },
1134     "units": units,
1135 }
1136
1137 event_report = report_specification.genre in {
1138     ReportGenre.EVENT_REPORT,
1139     ReportGenre.SPORTS_GAME_REPORT,
1140 }
1141 if not event_report:
1142     return {
1143         "minimum_word_count": minimum_words,
1144         "enforce_minimum_words": False,
1145         "units": [],
1146     }
1147
1148 reference_recap_style = (
1149     getattr(report_specification, "focus_scope", None)
1150     == "reference_recap"
1151 )
1152 event_recap_style = (
1153     realisation_policy_value
1154     == RealisationPolicy.EVENT_RECAP_STYLE.value
1155 )
1156 lookup = build_evidence_lookup(evidence)
1157 facts = fact_ledger.writer_ready_facts
1158 insights = [
1159     insight
1160     for insight in insight_ledger.verified_insights
1161     if insight.verification_status
1162     in {
1163         InsightVerificationStatus.VERIFIED,
1164         InsightVerificationStatus.VERIFIED_WITH_CAVEAT,
1165     }
1166 ]
1167
1168 units: list[dict[str, Any]] = []
1169 ordered_slots = [
1170     *report_specification.required_content_slots,
1171     *report_specification.optional_content_slots,
1172 ]
1173 if event_report:
1174     slot_priority = {
1175         "event_result": 0,
1176         "event_context": 1,
1177         "participant_record_context": 2,
1178         "event_status": 3,
1179         "score_progression": 4,
1180         "leading_performance": 5,
1181         "main_contrast": 6,
1182         "event_sequence": 7,
1183         "secondary_performance": 8,
1184         "scope_limitations": 9,
1185     }
1186     ordered_slots = sorted(
1187         dict.fromkeys(ordered_slots),
1188         key=lambda slot: (
1189             slot_priority.get(slot, 100),
1190             ordered_slots.index(slot),
1191         ),
1192     )
1193 for slot in dict.fromkeys(ordered_slots):
1194     definition = EVENT_CONTENT_UNIT_DEFINITIONS.get(slot)
1195     if definition is None:
1196         continue
1197
1198     candidate_fact_ids = [
1199         fact.fact_id
1200         for fact in facts
1201         if (
1202             fact.recommended_use
1203             != RecommendedUse.OMIT_UNLESS_REQUESTED
1204             and event_fact_slot(fact, lookup) == slot
1205         )
1206     ]
1207     if slot == "event_sequence":
1208         actionable_candidate_fact_ids = [

```

```

1209         fact.fact_id
1210     for fact in facts
1211     if (
1212         fact.fact_id in candidate_fact_ids
1213         and event_sequence_fact_is_actionable(fact, lookup)
1214     )
1215     ]
1216     if actionable_candidate_fact_ids:
1217         candidate_fact_ids = actionable_candidate_fact_ids
1218 if not candidate_fact_ids:
1219     continue
1220
1221 candidate_insight_ids = _insight_ids_for_fact_ids(
1222     insights=insights,
1223     fact_ids=set(candidate_fact_ids),
1224 )
1225 minimum_items = min(
1226     int(definition["minimum"]),
1227     len(candidate_fact_ids),
1228 )
1229 units.append(
1230     {
1231         "unit_id": slot,
1232         "description": definition["description"],
1233         "minimum_items": minimum_items,
1234         "candidate_fact_ids": candidate_fact_ids,
1235         "candidate_insight_ids": candidate_insight_ids,
1236         "candidate_insight_fact_ids": {
1237             insight.insight_id: [
1238                 fact_id
1239                 for fact_id in insight.source_fact_ids
1240                 if fact_id in candidate_fact_ids
1241             ]
1242             for insight in insights
1243             if insight.insight_id in candidate_insight_ids
1244         },
1245     }
1246 )
1247
1248 substantive_unit_count = sum(
1249     1
1250     for unit in units
1251     if unit["unit_id"]
1252     in {
1253         "event_result",
1254         "event_context",
1255         "participant_record_context",
1256         "score_progression",
1257         "event_sequence",
1258         "leading_performance",
1259         "main_contrast",
1260     }
1261 )
1262 main_contrast_available = any(
1263     unit["unit_id"] == "main_contrast"
1264     and len(unit.get("candidate_fact_ids", [])) >= 2
1265     for unit in units
1266 )
1267 insight_available = any(
1268     unit.get("candidate_insight_ids")
1269     for unit in units
1270 )
1271 enforce_narrative = bool(
1272     substantive_unit_count >= 3
1273     and len(facts) >= 5
1274 )
1275
1276 return {
1277     "minimum_word_count": minimum_words,
1278     "word_count_validation": "quality",
1279     "narrative_validation": "quality",
1280     "content_unit_validation": "quality",
1281     "enforce_minimum_words": bool(
1282         enforce_narrative
1283     ),
1284     "realisation_policy": realisation_policy_value,
1285     "style_rewrite_permissions": {
1286         "may_normalise_identifier_separators": (
1287             realisation_policy_value
1288             in {
1289                 RealisationPolicy.NATURAL_REFERENCE_STYLE.value,
1290                 RealisationPolicy.EVENT_RECAP_STYLE.value,
1291             }
1292         ),
1293         "may_compress_caveats": reference_recap_style,
1294         "may_use_reference_style_event_transitions": (
1295             realisation_policy_value
1296             == RealisationPolicy.EVENT_RECAP_STYLE.value
1297         ),
1298         "must_preserve_numbers_exactly": True,
1299         "must_not_add_unsupported_chronology": True,

```

```

1300         "must_not_add_unsupported_causality": True,
1301     },
1302     "output_form": (
1303         output_form.value
1304         if isinstance(output_form, OutputForm)
1305         else str(output_form or OutputForm.MULTI_PARAGRAPH_REPORT.value)
1306     ),
1307     "allow_headings": False if reference_recap_style else True,
1308     "reference_recap_style": reference_recap_style,
1309     "event_recap_style": event_recap_style,
1310     "narrative_requirements": {
1311         "enforce": enforce_narrative,
1312         "minimum_synthesis_sentences": 2,
1313         "minimum_insight_sentences": 1 if insight_available else 0,
1314         "minimum_connective_sentences": (
1315             1 if main_contrast_available else 0
1316         ),
1317         "minimum_scope_limitations": (
1318             0 if reference_recap_style else 1
1319         ),
1320     },
1321     "units": units,
1322 }
1323
1324
1325 def content_requirement_errors(
1326     *,
1327     used_fact_ids: set[str],
1328     used_insight_ids: set[str],
1329     word_count: int,
1330     requirements: dict[str, Any] | None,
1331     narrative_stats: dict[str, int] | None = None,
1332     include_word_count: bool = True,
1333     respect_validation_severity: bool = True,
1334 ) -> list[str]:
1335     if not requirements:
1336         return []
1337
1338     errors: list[str] = []
1339     word_count_validation_is_fatal = (
1340         not respect_validation_severity
1341         or requirements.get("word_count_validation", "fatal") == "fatal"
1342     )
1343     narrative_validation_is_fatal = (
1344         not respect_validation_severity
1345         or requirements.get("narrative_validation", "fatal") == "fatal"
1346     )
1347     content_unit_validation_is_fatal = (
1348         not respect_validation_severity
1349         or requirements.get("content_unit_validation", "fatal") == "fatal"
1350     )
1351     minimum_words = requirements.get("minimum_word_count")
1352     if (
1353         include_word_count
1354         and word_count_validation_is_fatal
1355         and requirements.get("enforce_minimum_words")
1356         and minimum_words is not None
1357         and word_count < int(minimum_words)
1358     ):
1359         errors.append(
1360             f"The draft contains {word_count} words; at least "
1361             f"{minimum_words} words are required while supported content "
1362             "remains available."
1363         )
1364
1365     narrative = requirements.get("narrative_requirements") or {}
1366     if (
1367         narrative_validation_is_fatal
1368         and narrative.get("enforce")
1369         and narrative_stats is not None
1370     ):
1371         for field, stat_key, label in [
1372             (
1373                 "minimum_synthesis_sentences",
1374                 "synthesis_sentences",
1375                 "multi-fact or insight-backed narrative sentences",
1376             ),
1377             (
1378                 "minimum_insight_sentences",
1379                 "insight_sentences",
1380                 "verified insight-backed narrative sentences",
1381             ),
1382             (
1383                 "minimum_connective_sentences",
1384                 "connective_sentences",
1385                 "contrastive or connective narrative sentences",
1386             ),
1387             (
1388                 "minimum_scope_limitations",
1389                 "scope_limitation_sentences",
1390                 "event-scoped limitation sentences",

```

```

1391     ),
1392   ]:
1393     minimum = int(narrative.get(field, 0) or 0)
1394     if minimum <= 0:
1395       continue
1396     observed = int(
1397       narrative_stats.get(
1398         stat_key,
1399         0,
1400       )
1401     )
1402     if observed < minimum:
1403       errors.append(
1404         f"The event draft uses {observed} {label}, but "
1405         f"{minimum} are required when supported event material "
1406         "is available."
1407       )
1408
1409   if not content_unit_validation_is_fatal:
1410     return errors
1411
1412   for unit in requirements.get("units", []):
1413     candidate_fact_ids = set(unit.get("candidate_fact_ids", []))
1414     candidate_insight_ids = set(unit.get("candidate_insight_ids", []))
1415     if not candidate_fact_ids and not candidate_insight_ids:
1416       continue
1417     covered_fact_ids = candidate_fact_ids & used_fact_ids
1418     insight_fact_coverage = unit.get("candidate_insight_fact_ids") or {}
1419     for insight_id in candidate_insight_ids & used_insight_ids:
1420       covered_fact_ids.update(
1421         fact_id
1422         for fact_id in insight_fact_coverage.get(insight_id, [])
1423         if fact_id in candidate_fact_ids
1424       )
1425     covered_insight_ids = {
1426       insight_id
1427       for insight_id in candidate_insight_ids & used_insight_ids
1428       if not insight_fact_coverage.get(insight_id)
1429     }
1430     covered_items = covered_fact_ids | covered_insight_ids
1431     minimum_items = int(unit.get("minimum_items", 1))
1432     if len(covered_items) < minimum_items:
1433       errors.append(
1434         f"Content unit `{unit.get('unit_id')}` uses "
1435         f"{len(covered_items)} supported item(s), but "
1436         f"{minimum_items} are required: "
1437         f"{unit.get('description')}"
1438       )
1439
1440   return errors
1441
1442
1443 def writer_output_content_requirement_errors(
1444   *,
1445   writer_output: WriterOutput,
1446   requirements: dict[str, Any] | None,
1447   include_word_count: bool = False,
1448   respect_validation_severity: bool = True,
1449 ) -> list[str]:
1450   requirements = requirements or {}
1451   used_fact_ids = {
1452     *writer_output.title_fact_ids,
1453     *[
1454       fact_id
1455       for support in writer_output.sentence_support
1456       for fact_id in support.fact_ids
1457     ],
1458   }
1459   used_insight_ids = {
1460     insight_id
1461     for support in writer_output.sentence_support
1462     for insight_id in support.insight_ids
1463   }
1464   errors = content_requirement_errors(
1465     used_fact_ids=used_fact_ids,
1466     used_insight_ids=used_insight_ids,
1467     word_count=writer_output.word_count(writer_output),
1468     requirements=requirements,
1469     narrative_stats=sentence_support_narrative_stats(
1470       writer_output.sentence_support
1471     ),
1472     include_word_count=include_word_count,
1473     respect_validation_severity=respect_validation_severity,
1474   )
1475   if requirements.get("allow_headings") is False and re.search(
1476     r"(?m)^(#{1,6})\s+",
1477     writer_output.markdown,
1478   ):
1479     errors.append("The output must not contain Markdown headings.")
1480
1481   max_sentences = requirements.get("max_sentences")

```

```

1482     if max_sentences is not None:
1483         sentence_count = len(
1484             split_markdown_sentences(writer_output.markdown)
1485         )
1486         if sentence_count > int(max_sentences):
1487             errors.append(
1488                 f"The output contains {sentence_count} sentences; at most "
1489                 f"{max_sentences} are allowed for this output form."
1490             )
1491
1492     max_paragraphs = requirements.get("max_paragraphs")
1493     if max_paragraphs is not None:
1494         paragraphs = [
1495             paragraph
1496             for paragraph in re.split(
1497                 r"\n\s*\n",
1498                 writer_output.markdown.strip(),
1499             )
1500             if paragraph.strip()
1501         ]
1502         if len(paragraphs) > int(max_paragraphs):
1503             errors.append(
1504                 f"The output contains {len(paragraphs)} paragraphs; at most "
1505                 f"{max_paragraphs} are allowed for this output form."
1506             )
1507
1508     if requirements.get("require_complete_sentence"):
1509         text = writer_output.markdown.strip()
1510         if text and text[-1] not in ".!?:":
1511             errors.append("The output must be a complete sentence.")
1512
1513     return errors
1514
1515
1516 def fact_support_numbers(
1517     fact: VerifiedFact,
1518     evidence: EvidenceLedger,
1519 ) -> list[float]:
1520     lookup = evidence_lookup(evidence)
1521     numbers = [
1522         number
1523         for _, number in extract_number_tokens(fact.fact_summary)
1524     ]
1525     numbers.extend(flatten_numbers(fact.structured_values))
1526
1527     for evidence_id in fact.evidence_ids:
1528         item = lookup.get(evidence_id)
1529         if item is not None:
1530             numbers.extend(flatten_numbers(item.metrics))
1531
1532     return numbers
1533
1534
1535 def fact_support_text(
1536     fact: VerifiedFact,
1537     evidence: EvidenceLedger,
1538 ) -> str:
1539     lookup = evidence_lookup(evidence)
1540
1541     parts = [
1542         fact.fact_summary,
1543         " ".join(fact.allowed_interpretations),
1544         " ".join(fact.required_caveats),
1545     ]
1546
1547     for evidence_id in fact.evidence_ids:
1548         item = lookup.get(evidence_id)
1549
1550         if item is None:
1551             continue
1552
1553         parts.extend(
1554             [
1555                 item.finding,
1556                 item.practical_interpretation,
1557                 json.dumps(json_safe(item.metrics), sort_keys=True),
1558                 " ".join(item.source_columns),
1559             ]
1560         )
1561
1562     return " ".join(parts)
1563
1564
1565 def _normalise_entity_text(value: str) -> str:
1566     text = value.replace("_", " ")
1567     text = re.sub(
1568         r"([A-Za-z])([A-Z] [a-z])",
1569         " ",
1570         text,
1571     )
1572     return re.sub(

```

```

1573         r"\s+",
1574         " ",
1575         text.replace("`", "").strip().casefold(),
1576     )
1577
1578
1579 def unsupported_backtick_entities(
1580     text: str,
1581     supported_entities: set[str],
1582 ) -> list[str]:
1583     normalised_supported = {
1584         _normalise_entity_text(entity)
1585         for entity in supported_entities
1586         if _normalise_entity_text(entity)
1587     }
1588
1589     return list(
1590         dict.fromkeys(
1591             entity
1592             for entity in re.findall(r"`([^\`]+)`", text)
1593             if _normalise_entity_text(entity)
1594             not in normalised_supported
1595         )
1596     )
1597
1598
1599 def _entity_in_sentence(
1600     entity: str,
1601     sentence: str,
1602 ) -> bool:
1603     entity_text = _normalise_entity_text(entity)
1604     sentence_text = _normalise_entity_text(sentence)
1605
1606     return bool(
1607         entity_text
1608         and re.search(
1609             rf"(?<!\w){re.escape(entity_text)}(?:!\w)",
1610             sentence_text,
1611         )
1612     )
1613
1614
1615 def _profile_support_id(
1616     *parts: str,
1617 ) -> str:
1618     return "PROFILE:." + ":".join(parts)
1619
1620
1621 def _profile_record(
1622     *,
1623     support_id: str,
1624     fact_kind: str,
1625     table_name: str,
1626     column_name: str | None,
1627     statement: str,
1628     structured_values: dict[str, Any],
1629     entities: list[str],
1630     provenance: str,
1631     claim_permissions: list[ClaimPermission] | None = None,
1632 ) -> ProfileSupportRecord:
1633     return ProfileSupportRecord(
1634         support_id=support_id,
1635         fact_kind=fact_kind,
1636         table_name=table_name,
1637         column_name=column_name,
1638         statement=statement,
1639         structured_values=structured_values,
1640         entities=list(dict.fromkeys(entities)),
1641         claim_permissions=(
1642             claim_permissions
1643             or [ClaimPermission.DESRIPTIVE]
1644         ),
1645         provenance=provenance,
1646     )
1647
1648
1649 def _column_numeric_summary_values(
1650     column: ColumnProfile,
1651 ) -> dict[str, Any]:
1652     return {
1653         key: value
1654         for key, value in column.numeric_summary.items()
1655         if isinstance(value, (int, float, np.integer, np.floating))
1656         and math.isfinite(float(value))
1657     }
1658
1659
1660 def build_profile_support_registry(
1661     profile: DataProfile,
1662 ) -> list[ProfileSupportRecord]:
1663     records: list[ProfileSupportRecord] = []

```

```

1664
1665 for table in profile.tables:
1666     records.append(
1667         _profile_record(
1668             support_id=_profile_support_id(
1669                 table.table_name,
1670                 "dimensions",
1671             ),
1672             fact_kind="table_dimensions",
1673             table_name=table.table_name,
1674             column_name=None,
1675             statement=(
1676                 f"Table `{table.table_name}` contains "
1677                 f"{table.row_count:,} rows and "
1678                 f"{table.column_count:,} columns."
1679             ),
1680             structured_values={
1681                 "row_count": table.row_count,
1682                 "column_count": table.column_count,
1683             },
1684             entities=[table.table_name],
1685             provenance="deterministic_data_profile",
1686         )
1687     )
1688
1689 duplicate_rate = (
1690     table.duplicate_row_count
1691     / max(table.row_count, 1)
1692 )
1693 records.append(
1694     _profile_record(
1695         support_id=_profile_support_id(
1696             table.table_name,
1697             "duplicates",
1698         ),
1699         fact_kind="duplicate_rows",
1700         table_name=table.table_name,
1701         column_name=None,
1702         statement=(
1703             f"Table `{table.table_name}` contains "
1704             f"{table.duplicate_row_count:,} exact duplicate rows "
1705             f"({duplicate_rate:.2%} of rows)."
1706         ),
1707         structured_values={
1708             "duplicate_row_count": table.duplicate_row_count,
1709             "duplicate_row_rate": duplicate_rate,
1710             "row_count": table.row_count,
1711         },
1712         entities=[table.table_name],
1713         claim_permissions=[
1714             ClaimPermission.DESRIPTIVE,
1715             ClaimPermission.METHODOLOGICAL,
1716         ],
1717         provenance="deterministic_data_profile",
1718     )
1719 )
1720
1721 for column in table.columns:
1722     entities = [
1723         table.table_name,
1724         column.name,
1725     ]
1726     records.append(
1727         _profile_record(
1728             support_id=_profile_support_id(
1729                 table.table_name,
1730                 column.name,
1731                 "missingness",
1732             ),
1733             fact_kind="column_missingness",
1734             table_name=table.table_name,
1735             column_name=column.name,
1736             statement=(
1737                 f"`{column.name}` has {column.missing_count:,} "
1738                 f"missing values ({column.missing_rate:.2%} "
1739                 f"of rows)."
1740             ),
1741             structured_values={
1742                 "missing_count": column.missing_count,
1743                 "missing_rate": column.missing_rate,
1744                 "row_count": table.row_count,
1745             },
1746             entities=entities,
1747             provenance="deterministic_data_profile",
1748         )
1749     )
1750
1751 if column.constant:
1752     values = {
1753         "constant": True,
1754         "unique_count": column.unique_count,

```



```

1755         "sample_values": column.sample_values,
1756         **_column_numeric_summary_values(column),
1757     }
1758     records.append(
1759         _profile_record(
1760             support_id=profile_support_id(
1761                 table.table_name,
1762                 column.name,
1763                 "constant",
1764             ),
1765             fact_kind="constant_column",
1766             table_name=table.table_name,
1767             column_name=column.name,
1768             statement=(
1769                 f"`{column.name}` contains no observed "
1770                 "variation in the profiled data."
1771             ),
1772             structured_values=values,
1773             entities=entities,
1774             claim_permissions=[
1775                 ClaimPermission.DESCRPTIVE,
1776                 ClaimPermission.METHODOLOGICAL,
1777             ],
1778             provenance="deterministic_data_profile",
1779         )
1780     )
1781
1782     if column.near_constant:
1783         records.append(
1784             _profile_record(
1785                 support_id=profile_support_id(
1786                     table.table_name,
1787                     column.name,
1788                     "near_constant",
1789                 ),
1790                 fact_kind="near_constant_column",
1791                 table_name=table.table_name,
1792                 column_name=column.name,
1793                 statement=(
1794                     f"`{column.name}` is near-constant with a "
1795                     "dominant observed value."
1796                 ),
1797                 structured_values={
1798                     "near_constant": True,
1799                     "dominant_value_rate": (
1800                         column.dominant_value_rate
1801                     ),
1802                     "unique_count": column.unique_count,
1803                 },
1804                 entities=entities,
1805                 claim_permissions=[
1806                     ClaimPermission.DESCRPTIVE,
1807                     ClaimPermission.METHODOLOGICAL,
1808                 ],
1809                 provenance="deterministic_data_profile",
1810             )
1811         )
1812
1813     if column.numeric_summary:
1814         records.append(
1815             _profile_record(
1816                 support_id=profile_support_id(
1817                     table.table_name,
1818                     column.name,
1819                     "numeric_summary",
1820                 ),
1821                 fact_kind="numeric_summary",
1822                 table_name=table.table_name,
1823                 column_name=column.name,
1824                 statement=(
1825                     f"`{column.name}` has deterministic numeric "
1826                     "summary statistics in the profile."
1827                 ),
1828                 structured_values=_column_numeric_summary_values(
1829                     column
1830                 ),
1831                 entities=entities,
1832                 provenance="deterministic_data_profile",
1833             )
1834         )
1835
1836     if (
1837         column.suspicious_zero_values
1838         or column.possible_sentinel_values
1839         or column.zero_risk.value != "none"
1840     ):
1841         zero_values = {
1842             *_column_numeric_summary_values(column),
1843             "zero_risk": column.zero_risk.value,
1844             "zero_risk_reason": column.zero_risk_reason,
1845             "suspicious_zero_values": (

```

```

1846         column.suspicious_zero_values
1847     ),
1848     "possible_sentinel_values": (
1849         column.possible_sentinel_values
1850     ),
1851 }
1852 records.append(
1853     _profile_record(
1854         support_id=profile_support_id(
1855             table.table_name,
1856             column.name,
1857             "zero_diagnostic",
1858         ),
1859         fact_kind="zero_diagnostic",
1860         table_name=table.table_name,
1861         column_name=column.name,
1862         statement=(
1863             f"`{column.name}` has deterministic zero-value "
1864             "diagnostics in the profile."
1865         ),
1866         structured_values=zero_values,
1867         entities=entities,
1868         claim_permissions=[
1869             ClaimPermission.DESCRPTIVE,
1870             ClaimPermission.METHODOLOGICAL,
1871         ],
1872         provenance="deterministic_data_profile",
1873     )
1874 )
1875
1876 if column.datetime_parse_rate > 0:
1877     records.append(
1878         _profile_record(
1879             support_id=profile_support_id(
1880                 table.table_name,
1881                 column.name,
1882                 "datetime_presence",
1883             ),
1884             fact_kind="datetime_presence",
1885             table_name=table.table_name,
1886             column_name=column.name,
1887             statement=(
1888                 f"`{column.name}` is date/time-like with "
1889                 f"parse rate {column.datetime_parse_rate:.2%}."
1890             ),
1891             structured_values={
1892                 "datetime_parse_rate": (
1893                     column.datetime_parse_rate
1894                 ),
1895                 "row_count": table.row_count,
1896             },
1897             entities=entities,
1898             provenance="deterministic_data_profile",
1899         )
1900     )
1901
1902 if column.candidate_key:
1903     records.append(
1904         _profile_record(
1905             support_id=profile_support_id(
1906                 table.table_name,
1907                 column.name,
1908                 "candidate_key",
1909             ),
1910             fact_kind="candidate_key",
1911             table_name=table.table_name,
1912             column_name=column.name,
1913             statement=(
1914                 f"`{column.name}` is a deterministic candidate "
1915                 "key in the profile."
1916             ),
1917             structured_values={
1918                 "candidate_key": True,
1919                 "unique_count": column.unique_count,
1920                 "row_count": table.row_count,
1921             },
1922             entities=entities,
1923             provenance="deterministic_data_profile",
1924         )
1925     )
1926
1927 return records
1928
1929
1930 def profile_record_numbers(
1931     record: ProfileSupportRecord,
1932 ) -> list[float]:
1933     return flatten_numbers(record.structured_values)
1934
1935
1936 def _profile_entity_aligned(

```

```

1937     sentence: str,
1938     record: ProfileSupportRecord,
1939     records: list[ProfileSupportRecord],
1940 ) -> bool:
1941     if record.column_name is not None:
1942         return _entity_in_sentence(
1943             record.column_name,
1944             sentence,
1945         )
1946
1947     if _entity_in_sentence(record.table_name, sentence):
1948         return True
1949
1950     table_count = len(
1951         {
1952             candidate.table_name
1953             for candidate in records
1954         }
1955     )
1956
1957     return (
1958         table_count == 1
1959         and re.search(
1960             r"\b(dataset|table|rows?|records?|observations?)\b",
1961             sentence,
1962             re.IGNORECASE,
1963         )
1964         is not None
1965     )
1966
1967 def _profile_fact_kind_aligned(
1968     sentence: str,
1969     record: ProfileSupportRecord,
1970 ) -> bool:
1971     pattern = PROFILE_FACT_KIND_TERMS.get(
1972         record.fact_kind
1973     )
1974
1975     return bool(pattern and pattern.search(sentence))
1976
1977
1978 def _profile_semantic_escalation(
1979     sentence: str,
1980 ) -> bool:
1981     return bool(
1982         HOURLY_CADENCE_PATTERN.search(sentence)
1983         or LOCATION_METADATA_PATTERN.search(sentence)
1984         or CONSTANT_OVERSTATEMENT_PATTERN.search(sentence)
1985         or ZERO_OVERCONFIDENCE_PATTERN.search(sentence)
1986         or MISSINGNESS_HARMLESS_PATTERN.search(sentence)
1987         or DUPLICATE_REMOVAL_PATTERN.search(sentence)
1988         or PEARSON_IMPRECISE_PATTERN.search(sentence)
1989     )
1990
1991
1992 def matching_profile_support_records(
1993     *,
1994     sentence: str,
1995     records: list[ProfileSupportRecord],
1996 ) -> list[ProfileSupportRecord]:
1997     if _profile_semantic_escalation(sentence):
1998         return []
1999
2000     matches: list[ProfileSupportRecord] = []
2001
2002     for record in records:
2003         if not _profile_entity_aligned(
2004             sentence,
2005             record,
2006             records,
2007         ):
2008             continue
2009
2010         if not _profile_fact_kind_aligned(
2011             sentence,
2012             record,
2013         ):
2014             continue
2015
2016         if not numbers_supported(
2017             sentence,
2018             profile_record_numbers(record),
2019         ):
2020             continue
2021
2022         matches.append(record)
2023
2024     return matches
2025
2026
2027

```

```

2028 def _profile_records_support_sentence(
2029     *,
2030     sentence: str,
2031     support_ids: list[str],
2032     records_by_id: dict[str, ProfileSupportRecord],
2033 ) -> bool:
2034     selected = [
2035         records_by_id[support_id]
2036         for support_id in support_ids
2037         if support_id in records_by_id
2038     ]
2039     if not selected:
2040         return False
2041
2042     return bool(
2043         matching_profile_support_records(
2044             sentence=sentence,
2045             records=selected,
2046         )
2047     )
2048
2049
2050 def _focused_role_value_text(pair: dict[str, Any]) -> str | None:
2051     headers = [
2052         str(header)
2053         for header in pair.get("headers", [])
2054         if str(header).strip()
2055     ]
2056     value = str(pair.get("value", "")).strip()
2057     if not value:
2058         return None
2059
2060     header_text = " / ".join(headers) if headers else "value"
2061     return f"{header_text} = {value}"
2062
2063
2064 def _focused_table_fact_summary(item: EvidenceItem) -> str:
2065     metrics = item.metrics
2066     record_relation = metrics.get("focused_record_relation")
2067     record_summary = (
2068         str(record_relation.get("relation_summary") or "").strip()
2069         if isinstance(record_relation, dict)
2070         else ""
2071     )
2072     list_relation = metrics.get("focused_list_relation")
2073     list_summary = (
2074         str(list_relation.get("relation_summary") or "").strip()
2075         if isinstance(list_relation, dict)
2076         else ""
2077     )
2078     record_group_summary = str(
2079         metrics.get("highlighted_record_group_summary") or ""
2080     ).strip()
2081     pair_texts = [
2082         text
2083         for pair in metrics.get("highlighted_role_value_pairs", [])
2084         if isinstance(pair, dict)
2085         and (text := _focused_role_value_text(pair))
2086     ]
2087
2088     if record_summary:
2089         summary = record_summary
2090     elif list_summary:
2091         summary = list_summary
2092     elif record_group_summary:
2093         summary = record_group_summary
2094     elif pair_texts:
2095         summary = "Highlighted table values: " + "; ".join(pair_texts) + "."
2096     else:
2097         highlighted_values = [
2098             str(value)
2099             for value in metrics.get("highlighted_values", [])
2100             if str(value).strip()
2101         ]
2102         if highlighted_values:
2103             summary = (
2104                 "Highlighted table value"
2105                 + ("s" if len(highlighted_values) != 1 else "")
2106                 + ": "
2107                 + ", ".join(highlighted_values)
2108                 + "."
2109             )
2110         else:
2111             proposition = str(
2112                 metrics.get("description_proposition") or ""
2113             ).strip()
2114             summary = proposition or item.finding
2115
2116     primary_subjects = [
2117         str(subject)
2118         for subject in (

```

```

2119         (metrics.get("concise_output_focus") or {})
2120         .get("primary_subject_candidates", [])
2121     )
2122     if str(subject).strip()
2123 ]
2124 if primary_subjects:
2125     summary += (
2126         " Structurally supported primary subject candidate"
2127         + ("s" if len(primary_subjects) != 1 else "")
2128         + ": "
2129         + ", ".join(primary_subjects)
2130         + "."
2131     )
2132
2133 highlighted_set_contrasts = [
2134     str(contrast.get("contrast_summary") or "").strip()
2135     for contrast in metrics.get("highlighted_set_contrasts", [])
2136     if isinstance(contrast, dict)
2137     and str(contrast.get("contrast_summary") or "").strip()
2138 ]
2139 if highlighted_set_contrasts:
2140     summary += (
2141         " Highlighted-set contrast scoped only to selected cells: "
2142         + "; ".join(highlighted_set_contrasts)
2143     )
2144
2145 record_group_summary = str(
2146     metrics.get("highlighted_record_group_summary") or ""
2147 ).strip()
2148 if record_group_summary and record_group_summary not in summary:
2149     summary += " Highlighted record-group summary: "
2150     summary += record_group_summary
2151
2152 context_parts = [
2153     str(metrics.get("section_title") or "").strip(),
2154     str(metrics.get("page_title") or "").strip(),
2155 ]
2156 context = " / ".join(
2157     part
2158     for part in context_parts
2159     if part and not extract_number_tokens(part)
2160 )
2161 if context:
2162     summary += f" Context: {context}."
2163
2164 comparisons = metrics.get("highlighted_measure_comparisons") or []
2165 comparison_notes = []
2166 for comparison in comparisons:
2167     if not isinstance(comparison, dict):
2168         continue
2169     value = str(comparison.get("highlighted_value") or "").strip()
2170     if not value:
2171         continue
2172     note_parts = [value]
2173     if comparison.get("is_highest_comparable_value"):
2174         note_parts.append("is the highest comparable highlighted measure")
2175     if comparison.get("is_majority_percentage"):
2176         note_parts.append("is above half")
2177     if len(note_parts) > 1:
2178         comparison_notes.append(" ".join(note_parts))
2179 if comparison_notes:
2180     summary += " Highlighted measure context: "
2181     summary += "; ".join(comparison_notes) + "."
2182
2183 return summary
2184
2185
2186 def _focused_table_local_contrast_summary(
2187     metrics: dict[str, Any],
2188 ) -> str | None:
2189     contrasts = metrics.get("highlighted_set_contrasts") or []
2190     for contrast in contrasts:
2191         if not isinstance(contrast, dict):
2192             continue
2193         summary = str(contrast.get("contrast_summary") or "").strip()
2194         if summary:
2195             return summary
2196     return None
2197
2198
2199 def _structured_record_fact_summary(item: EvidenceItem) -> str:
2200     records = [
2201         record
2202         for record in item.metrics.get("records", [])
2203         if isinstance(record, dict)
2204     ]
2205     triples = [
2206         record
2207         for record in records
2208         if str(record.get("record_kind") or "") == "triple"
2209     ]

```

```

2210     attributes = [
2211         record
2212         for record in records
2213         if str(record.get("record_kind") or "") != "triple"
2214     ]
2215
2216     if triples and not attributes:
2217         parts = [
2218             (
2219                 f"{record.get('subject')} | {record.get('relation')} | "
2220                 f"{record.get('object')}"
2221             )
2222             for record in triples
2223             if record.get("subject")
2224             and record.get("relation")
2225             and record.get("object")
2226         ]
2227         if parts:
2228             return "Supplied triples: " + "; ".join(parts) + "."
2229
2230     parts = [
2231         f"{record.get('attribute_name')} = {record.get('attribute_value')}"
2232         for record in attributes
2233         if record.get("attribute_name") and record.get("attribute_value")
2234     ]
2235     if parts:
2236         return "Supplied attributes: " + "; ".join(parts) + "."
2237
2238     return item.finding
2239
2240
2241 def _semantic_event_fact_summary(item: EvidenceItem) -> str:
2242     if item.evidence_type in {"event_context", "event_status"}:
2243         values = [
2244             value
2245             for value in item.metrics.get("values", [])
2246             if isinstance(value, dict)
2247             and value.get("label") is not None
2248             and value.get("value") is not None
2249         ]
2250         if values:
2251             parts = [
2252                 f"{value['label']} is {value['value']}"
2253                 for value in values[:4]
2254             ]
2255             return "Event context includes " + ", ".join(parts) + "."
2256
2257     if item.evidence_type in {"entity_ranking", "entity_performance"}:
2258         ranking = [
2259             record
2260             for record in item.metrics.get("ranking", [])
2261             if isinstance(record, dict)
2262             and record.get("entity") is not None
2263             and record.get("value") is not None
2264         ]
2265         if ranking:
2266             measure = str(
2267                 item.metrics.get("semantic_label")
2268                 or ranking[0].get("measure")
2269                 or "recorded value"
2270             )
2271             measure = re.sub(
2272                 r"^entity ranking for\s+",
2273                 "",
2274                 measure,
2275                 flags=re.IGNORECASE,
2276             )
2277
2278             def entity_name(record: dict[str, Any]) -> str:
2279                 entity = str(record["entity"])
2280                 group = record.get("group")
2281                 if group and str(group) not in entity:
2282                     return f"{entity} ({group})"
2283                 return entity
2284
2285             leaders = [
2286                 record
2287                 for record in ranking
2288                 if record.get("rank") == 1
2289             ]
2290             if len(leaders) > 1:
2291                 names = ", ".join(entity_name(record) for record in leaders[:4])
2292                 return (
2293                     f"In the ranking for {measure}, {names} tied for the lead "
2294                     f"with {float(leaders[0]['value']):g} each."
2295                 )
2296
2297             leader = ranking[0]
2298             summary = (
2299                 f"In the ranking for {measure}, {entity_name(leader)} led "
2300                 f"with {float(leader['value']):g}"

```

```

2301         )
2302         followers = [
2303             record
2304             for record in ranking[1:4]
2305             if record.get("value") is not None
2306         ]
2307         if followers:
2308             summary += ", followed by " + ", ".join(
2309                 f"{entity_name(record)} ({float(record['value']):g})"
2310                 for record in followers
2311             )
2312         return summary + "."
2313
2314     if item.evidence_type not in {
2315         "event_outcome",
2316         "participant_comparison",
2317         "event_contrast",
2318     }:
2319         return item.finding
2320
2321     records = [
2322         record
2323         for record in item.metrics.get("records", [])
2324         if isinstance(record, dict)
2325         and record.get("entity") is not None
2326         and record.get("value") is not None
2327     ]
2328     if len(records) < 2:
2329         return item.finding
2330
2331     ordered = sorted(
2332         records,
2333         key=lambda record: float(record["value"]),
2334         reverse=True,
2335     )
2336     high = ordered[0]
2337     low = ordered[-1]
2338     high_entity = str(high["entity"])
2339     low_entity = str(low["entity"])
2340     high_value = float(high["value"])
2341     low_value = float(low["value"])
2342     high_measure = str(high.get("measure") or "outcome value")
2343     low_measure = str(low.get("measure") or high_measure)
2344     difference = abs(high_value - low_value)
2345
2346     if high_value == low_value:
2347         return (
2348             f"{high_entity} and {low_entity} both recorded "
2349             f"{high_value:g} for the supplied outcome measure."
2350         )
2351
2352     return (
2353         f"{high_entity} recorded {high_value:g} for {high_measure}, "
2354         f"while {low_entity} recorded {low_value:g} for {low_measure}; "
2355         f"the difference is {difference:g}."
2356     )
2357
2358
2359 def event_sequence_evidence_is_actionable(item: EvidenceItem) -> bool:
2360     if item.evidence_type != "event_sequence":
2361         return False
2362
2363     if normalise_strength_label(item.strength_label) == (
2364         "event_sequence_highlight"
2365     ):
2366         return True
2367
2368     metrics = item.metrics
2369     if metrics.get("sequence_type") == "score_changing_sequence":
2370         return True
2371
2372     return any(
2373         isinstance(highlight, dict)
2374         and str(highlight.get("event_text") or "").strip()
2375         and str(highlight.get("score_phrase") or "").strip()
2376         for highlight in metrics.get("highlights", [])
2377     )
2378
2379
2380 def event_sequence_fact_is_actionable(
2381     fact: VerifiedFact,
2382     evidence_lookup: dict[str, EvidenceItem],
2383 ) -> bool:
2384     return any(
2385         event_sequence_evidence_is_actionable(item)
2386         for item in evidence_for_fact(fact, evidence_lookup)
2387     )
2388
2389
2390 def _event_sequence_highlight_summary(
2391     highlight: dict[str, Any],

```



```

2392 ) -> str | None:
2393     event_text = str(highlight.get("event_text") or "").strip()
2394     score_phrase = str(highlight.get("score_phrase") or "").strip()
2395     if not event_text or not score_phrase:
2396         return None
2397
2398     return f"{event_text}, after which {score_phrase}."
2399
2400
2401 def _event_sequence_highlight_salience(
2402     *,
2403     item: EvidenceItem,
2404     highlight: dict[str, Any],
2405 ) -> float:
2406     roles = {
2407         str(role)
2408         for role in highlight.get("sequence_roles", [])
2409     }
2410     bonus = 0.0
2411     if roles & {"lead_change", "tie", "largest_score_change"}:
2412         bonus += 0.04
2413     if roles & {"late_score_change", "final_score_change", "late_narrowing"}:
2414         bonus += 0.03
2415     if roles & {"opening_score"}:
2416         bonus += 0.02
2417     return min(1.0, item.salience + bonus)
2418
2419
2420 def deterministic_fact_candidates_from_evidence(
2421     *,
2422     item: EvidenceItem,
2423     start_ordinal: int,
2424 ) -> list[FactCandidate]:
2425     highlights = [
2426         highlight
2427         for highlight in item.metrics.get("highlights", [])
2428         if isinstance(highlight, dict)
2429     ]
2430     if event_sequence_evidence_is_actionable(item) and highlights:
2431         candidates: list[FactCandidate] = []
2432         for offset, highlight in enumerate(highlights):
2433             summary = _event_sequence_highlight_summary(highlight)
2434             if summary is None:
2435                 continue
2436             candidates.append(
2437                 FactCandidate(
2438                     candidate_id=f"CAN_{start_ordinal + offset:04d}",
2439                     fact_summary=summary,
2440                     evidence_ids=[item.evidence_id],
2441                     claim_permissions=item.claim_permissions,
2442                     allowed_interpretations=(
2443                         [item.practical_interpretation]
2444                         if item.practical_interpretation
2445                         else []
2446                     ),
2447                     prohibited_interpretations=item.prohibited_interpretations,
2448                     required_caveats=item.limitations,
2449                     factual_confidence=item.factual_confidence,
2450                     methodological_strength=item.methodological_strength,
2451                     user_relevance=item.user_relevance,
2452                     salience=_event_sequence_highlight_salience(
2453                         item=item,
2454                         highlight=highlight,
2455                     ),
2456                     recommended_use=RecommendedUse.MAIN_FINDING,
2457                     eligible_for_writer=True,
2458                 )
2459             )
2460         if candidates:
2461             return candidates
2462
2463     return [
2464         deterministic_fact_candidate_from_evidence(
2465             item=item,
2466             ordinal=start_ordinal,
2467         )
2468     ]
2469
2470
2471 def candidate_support(
2472     candidate: FactCandidate,
2473     evidence: EvidenceLedger,
2474 ) -> tuple[str, list[float], set[ClaimPermission]]:
2475     lookup = evidence_lookup(evidence)
2476     text_parts: list[str] = []
2477     numbers: list[float] = []
2478     permissions: set[ClaimPermission] = set()
2479
2480     for evidence_id in candidate.evidence_ids:
2481         item = lookup.get(evidence_id)
2482

```

```

2483     if item is None:
2484         continue
2485
2486     text_parts.extend(
2487         [
2488             item.finding,
2489             item.practical_interpretation,
2490             json.dumps(json_safe(item.metrics), sort_keys=True),
2491         ]
2492     )
2493     numbers.extend(flatten_numbers(item.metrics))
2494     permissions.update(item.claim_permissions)
2495
2496     return " ".join(text_parts), numbers, permissions
2497
2498
2499 def validate_fact_candidates(
2500     candidate_set: FactCandidateSet,
2501     evidence: EvidenceLedger,
2502 ) -> list[str]:
2503     errors: list[str] = []
2504     lookup = evidence_lookup(evidence)
2505     seen: set[str] = set()
2506
2507     for candidate in candidate_set.candidates:
2508         if candidate.candidate_id in seen:
2509             errors.append(
2510                 f"Duplicate candidate ID: {candidate.candidate_id}"
2511             )
2512
2513         seen.add(candidate.candidate_id)
2514
2515         if not candidate.evidence_ids:
2516             errors.append(
2517                 f"{candidate.candidate_id} has no evidence IDs."
2518             )
2519             continue
2520
2521         unknown = [
2522             evidence_id
2523             for evidence_id in candidate.evidence_ids
2524             if evidence_id not in lookup
2525         ]
2526
2527         if unknown:
2528             errors.append(
2529                 f"{candidate.candidate_id} cites unknown evidence IDs: {unknown}"
2530             )
2531             continue
2532
2533         _, support_numbers, permissions = candidate_support(
2534             candidate,
2535             evidence,
2536         )
2537
2538         if not numbers_supported(
2539             candidate.fact_summary,
2540             support_numbers,
2541         ):
2542             errors.append(
2543                 f"{candidate.candidate_id} contains unsupported numbers."
2544             )
2545
2546         if not set(candidate.claim_permissions).issubset(permissions):
2547             errors.append(
2548                 f"{candidate.candidate_id} requests unsupported permissions."
2549             )
2550
2551         if (
2552             CAUSAL_PATTERN.search(candidate.fact_summary)
2553             and ClaimPermission.CAUSAL not in permissions
2554         ):
2555             errors.append(f"{candidate.candidate_id} introduces unsupported causal wording.")
2556
2557         if candidate.eligible_for_writer and not all(
2558             lookup[evidence_id].eligible_for_writer
2559             for evidence_id in candidate.evidence_ids
2560         ):
2561             errors.append(
2562                 f"{candidate.candidate_id} is writer-eligible but cites excluded evidence."
2563             )
2564
2565     return errors
2566
2567
2568 def collect_entity_strings(value: Any) -> set[str]:
2569     entities: set[str] = set()
2570
2571     if isinstance(value, str):
2572         entities.add(value)
2573     elif isinstance(value, dict):

```

```

2574         for key, item in value.items():
2575             entities.add(str(key))
2576             entities.update(collect_entity_strings(item))
2577     elif isinstance(value, (list, tuple, set)):
2578         for item in value:
2579             entities.update(collect_entity_strings(item))
2580
2581     return entities
2582
2583
2584 def deterministic_fact_candidate_from_evidence(
2585     *,
2586     item: EvidenceItem,
2587     ordinal: int,
2588 ) -> FactCandidate:
2589     return FactCandidate(
2590         candidate_id=f"CAN_{ordinal:04d}",
2591         fact_summary=deterministic_fact_summary_from_evidence(item),
2592         evidence_ids=[item.evidence_id],
2593         claim_permissions=item.claim_permissions,
2594         allowed_interpretations=(
2595             [item.practical_interpretation]
2596             if item.practical_interpretation
2597             else []
2598         ),
2599         prohibited_interpretations=item.prohibited_interpretations,
2600         required_caveats=item.limitations,
2601         factual_confidence=item.factual_confidence,
2602         methodological_strength=item.methodological_strength,
2603         user_relevance=item.user_relevance,
2604         salience=item.salience,
2605         recommended_use=item.recommended_use,
2606         eligible_for_writer=True,
2607     )
2608
2609
2610 def deterministic_fact_candidate_scaffold(
2611     evidence: EvidenceLedger,
2612     maximum_facts: int | None,
2613     *,
2614     synthesis_note: str = (
2615         "Deterministic evidence-to-fact scaffold was created before "
2616         "LLM enrichment."
2617     ),
2618 ) -> FactCandidateSet:
2619     eligible_items = [
2620         item
2621         for item in evidence.items
2622         if item.eligible_for_writer
2623     ]
2624
2625     ranked = sorted(
2626         eligible_items,
2627         key=lambda item: (
2628             item.salience
2629             + item.user_relevance
2630             + item.methodological_strength
2631         ),
2632         reverse=True,
2633     )
2634
2635     selected_items = (
2636         ranked
2637         if maximum_facts is None
2638         else ranked[:maximum_facts]
2639     )
2640     candidates: list[FactCandidate] = []
2641     next_ordinal = 1
2642     for item in selected_items:
2643         item_candidates = deterministic_fact_candidates_from_evidence(
2644             item=item,
2645             start_ordinal=next_ordinal,
2646         )
2647         candidates.extend(item_candidates)
2648         next_ordinal += len(item_candidates)
2649
2650     return FactCandidateSet(
2651         candidates=candidates,
2652         synthesis_notes=[synthesis_note],
2653     )
2654
2655
2656 def fallback_fact_candidates(
2657     evidence: EvidenceLedger,
2658     maximum_facts: int | None,
2659 ) -> FactCandidateSet:
2660     return deterministic_fact_candidate_scaffold(
2661         evidence,
2662         maximum_facts,
2663         synthesis_note="Deterministic evidence-to-fact fallback was used.",
2664     )

```

```

2665
2666
2667 def empty_fact_candidate_enrichment() -> FactCandidateSet:
2668     return FactCandidateSet(
2669         candidates=[],
2670         synthesis_notes=[
2671             "LLM fact-candidate enrichment was unavailable; "
2672             "the deterministic scaffold was retained."
2673         ],
2674     )
2675
2676
2677 def merge_fact_candidate_scaffold(
2678     *,
2679     scaffold: FactCandidateSet,
2680     enrichment: FactCandidateSet,
2681     evidence: EvidenceLedger,
2682 ) -> FactCandidateSet:
2683     """Preserve deterministic coverage while accepting valid LLM enrichment."""
2684
2685     valid_enrichment: list[FactCandidate] = []
2686     dropped_notes: list[str] = []
2687     seen_enrichment_ids: set[str] = set()
2688
2689     for candidate in enrichment.candidates:
2690         if candidate.candidate_id in seen_enrichment_ids:
2691             dropped_notes.append(
2692                 f"Dropped duplicate enriched candidate {candidate.candidate_id}."
2693             )
2694             continue
2695         seen_enrichment_ids.add(candidate.candidate_id)
2696
2697         errors = validate_fact_candidates(
2698             FactCandidateSet(candidates=[candidate]),
2699             evidence,
2700         )
2701         if errors:
2702             dropped_notes.append(
2703                 f"Dropped invalid enriched candidate {candidate.candidate_id}: "
2704                 + "; ".join(errors)
2705             )
2706             continue
2707         valid_enrichment.append(candidate)
2708
2709     lookup = evidence_lookup(evidence)
2710     merged: list[FactCandidate] = list(scaffold.candidates)
2711     scaffold_indices_by_evidence_id: dict[str, list[int]] = defaultdict(list)
2712     for index, candidate in enumerate(merged):
2713         if len(candidate.evidence_ids) == 1:
2714             scaffold_indices_by_evidence_id[candidate.evidence_ids[0]].append(
2715                 index
2716             )
2717
2718     replaced_count = 0
2719     added_count = 0
2720     seen_signatures: set[tuple[tuple[str, ...], str]] = {
2721         (
2722             tuple(candidate.evidence_ids),
2723             candidate.fact_summary.strip().lower(),
2724         )
2725         for candidate in merged
2726     }
2727
2728     for candidate in valid_enrichment:
2729         signature = (
2730             tuple(candidate.evidence_ids),
2731             candidate.fact_summary.strip().lower(),
2732         )
2733         if signature in seen_signatures:
2734             continue
2735
2736         if (
2737             len(candidate.evidence_ids) == 1
2738             and len(
2739                 scaffold_indices_by_evidence_id.get(
2740                     candidate.evidence_ids[0],
2741                     [],
2742                 )
2743             )
2744             == 1
2745             and not event_sequence_evidence_is_actionable(
2746                 lookup[candidate.evidence_ids[0]]
2747             )
2748         ):
2749             index = scaffold_indices_by_evidence_id[
2750                 candidate.evidence_ids[0]
2751             ][0]
2752             seen_signatures.discard(
2753                 (
2754                     tuple(merged[index].evidence_ids),
2755                     merged[index].fact_summary.strip().lower(),

```

```

2756         )
2757     )
2758     merged[index] = candidate
2759     seen_signatures.add(signature)
2760     replaced_count += 1
2761     continue
2762
2763     merged.append(candidate)
2764     seen_signatures.add(signature)
2765     added_count += 1
2766
2767     renumbered = [
2768         candidate.model_copy(
2769             update={"candidate_id": f"CAN_{index:04d}"})
2770     ]
2771     for index, candidate in enumerate(merged, start=1)
2772 ]
2773
2774 return FactCandidateSet(
2775     candidates=renumbered,
2776     synthesis_notes=[
2777         *scaffold.synthesis_notes,
2778         *enrichment.synthesis_notes,
2779         (
2780             "Merged fact candidates by retaining deterministic "
2781             f"coverage, replacing {replaced_count} scaffold "
2782             f"candidate(s), and appending {added_count} enriched "
2783             "synthesis candidate(s).",
2784         ),
2785         *dropped_notes,
2786     ],
2787 )
2788
2789 def fallback_verification(
2790     candidate_set: FactCandidateSet,
2791 ) -> VerificationResult:
2792     return VerificationResult(
2793         reviews=[
2794             FactReview(
2795                 candidate_id=candidate.candidate_id,
2796                 decision=(
2797                     ReviewDecision.CAUTION
2798                     if candidate.required_caveats
2799                     else ReviewDecision.APPROVE
2800                 ),
2801                 rationale=(
2802                     "The candidate is directly grounded in validated evidence."
2803                 ),
2804                 required_caveats=candidate.required_caveats,
2805                 prohibited_interpretations=(
2806                     candidate.prohibited_interpretations
2807                 ),
2808             ),
2809             for candidate in candidate_set.candidates
2810         ],
2811         overall_notes=[
2812             "Deterministic verification fallback was used."
2813         ],
2814     )
2815
2816
2817 MISSING_EVIDENCE_REJECTION_PATTERN = re.compile(
2818     r"\b(?:cited\s+)?evidence\b.*\b(?:not present|not found|unknown|missing)\b|"
2819     r"\b(?:not present|not found|unknown|missing)\b.*\bevidence\b",
2820     re.IGNORECASE,
2821 )
2822
2823
2824 def repair_spurious_missing_evidence_rejections(
2825     *,
2826     candidate_set: FactCandidateSet,
2827     verification: VerificationResult,
2828     evidence: EvidenceLedger,
2829 ) -> VerificationResult:
2830     """Keep local evidence-ID validation authoritative over verifier slips."""
2831
2832     candidates = {
2833         candidate.candidate_id: candidate
2834         for candidate in candidate_set.candidates
2835     }
2836     repaired_reviews: list[FactReview] = []
2837     repair_notes: list[str] = []
2838
2839     for review in verification.reviews:
2840         candidate = candidates.get(review.candidate_id)
2841         if (
2842             candidate is not None
2843             and review.decision == ReviewDecision.REJECT
2844             and MISSING_EVIDENCE_REJECTION_PATTERN.search(
2845                 review.rationale

```

```

2847         )
2848         and not validate_fact_candidates(
2849             FactCandidateSet(candidates=[candidate]),
2850             evidence,
2851         )
2852     ):
2853         repaired_reviews.append(
2854             FactReview(
2855                 candidate_id=review.candidate_id,
2856                 decision=(
2857                     ReviewDecision.CAUTION
2858                     if candidate.required_caveats
2859                     else ReviewDecision.APPROVE
2860                 ),
2861                 rationale=(
2862                     "Local deterministic validation confirmed the cited "
2863                     "evidence IDs exist and support this candidate; "
2864                     "overrode a spurious missing-evidence rejection."
2865                 ),
2866                 required_caveats=[
2867                     *candidate.required_caveats,
2868                     *review.required_caveats,
2869                 ],
2870                 prohibited_interpretations=[
2871                     *candidate.prohibited_interpretations,
2872                     *review.prohibited_interpretations,
2873                 ],
2874             )
2875         )
2876         repair_notes.append(
2877             f"Repaired spurious missing-evidence rejection for "
2878             f"{review.candidate_id}."
2879         )
2880         continue
2881     repaired_reviews.append(review)
2882
2883 if not repair_notes:
2884     return verification
2885
2886 return verification.model_copy(
2887     update={
2888         "reviews": repaired_reviews,
2889         "overall_notes": [
2890             *verification.overall_notes,
2891             *repair_notes,
2892         ],
2893     },
2894 )
2895
2896
2897 def finalise_fact_ledger(
2898     candidate_set: FactCandidateSet,
2899     verification: VerificationResult,
2900     evidence: EvidenceLedger,
2901 ) -> FactLedger:
2902     errors = validate_fact_candidates(candidate_set, evidence)
2903     reviews = {
2904         review.candidate_id: review
2905         for review in verification.reviews
2906     }
2907     lookup = evidence_lookup(evidence)
2908
2909     facts: list[VerifiedFact] = []
2910     rejected: list[RejectedFact] = []
2911
2912     for candidate in candidate_set.candidates:
2913         candidate_errors = [
2914             error
2915             for error in errors
2916             if candidate.candidate_id in error
2917         ]
2918
2919         review = reviews.get(candidate.candidate_id)
2920
2921         if candidate_errors:
2922             rejected.append(
2923                 RejectedFact(
2924                     source_candidate_id=candidate.candidate_id,
2925                     fact_summary=candidate.fact_summary,
2926                     reason=" ".join(candidate_errors),
2927                 )
2928             )
2929             continue
2930
2931         if review is None or review.decision == ReviewDecision.REJECT:
2932             rejected.append(
2933                 RejectedFact(
2934                     source_candidate_id=candidate.candidate_id,
2935                     fact_summary=candidate.fact_summary,
2936                     reason=(

```

```

2938         review.rationale
2939         if review
2940             else "The verifier did not review this candidate."
2941     ),
2942 )
2943 )
2944 continue
2945
2946 structured_values: dict[str, Any] = {}
2947 entities: set[str] = set()
2948 source_capabilities = []
2949
2950 for evidence_id in candidate.evidence_ids:
2951     item = lookup[evidence_id]
2952     structured_values[evidence_id] = item.metrics
2953     entities.update(item.source_tables)
2954     entities.update(item.source_columns)
2955     entities.update(item.entity_scope)
2956     source_capabilities.append(item.capability)
2957
2958     entities.update(collect_entity_strings(item.metrics))
2959
2960 facts.append(
2961     VerifiedFact(
2962         fact_id=f"FACT_{len(facts) + 1:04d}",
2963         source_candidate_id=candidate.candidate_id,
2964         fact_summary=candidate.fact_summary,
2965         evidence_ids=candidate.evidence_ids,
2966         source_capabilities=list(
2967             dict.fromkeys(source_capabilities)
2968         ),
2969         structured_values=structured_values,
2970         entities=sorted(entities),
2971         claim_permissions=candidate.claim_permissions,
2972         allowed_interpretations=candidate.allowed_interpretations,
2973         prohibited_interpretations=list(
2974             dict.fromkeys(
2975                 candidate.prohibited_interpretations
2976                 + review.prohibited_interpretations
2977             )
2978         ),
2979         required_caveats=list(
2980             dict.fromkeys(
2981                 candidate.required_caveats
2982                 + review.required_caveats
2983             )
2984         ),
2985         factual_confidence=candidate.factual_confidence,
2986         methodological_strength=candidate.methodological_strength,
2987         user_relevance=candidate.user_relevance,
2988         salience=candidate.salience,
2989         recommended_use=candidate.recommended_use,
2990     )
2991 )
2992
2993 return FactLedger(
2994     writer_ready_facts=facts,
2995     rejected_facts=rejected,
2996     verifier_notes=verification.overall_notes,
2997 )
2998
2999
3000
3001 def normalise_strength_label(
3002     label: str,
3003 ) -> str:
3004     mapping = {
3005         "very_strong": "very_strong_association",
3006         "strong": "strong_association",
3007         "moderate": "moderate_association",
3008         "weak": "weak_but_reportable_association",
3009         "weak_but_reportable": (
3010             "weak_but_reportable_association"
3011         ),
3012         "large": "large_group_difference",
3013         "small": "small_group_difference",
3014         "negligible": "negligible_group_difference",
3015     }
3016
3017     return mapping.get(label, label)
3018
3019
3020 ASSOCIATION_STRENGTH_TERMS = {
3021     "very strong": "very_strong_association",
3022     "strong": "strong_association",
3023     "moderate": "moderate_association",
3024     "weak": "weak_but_reportable_association",
3025     "negligible": "negligible_association",
3026 }
3027
3028 GROUP_STRENGTH_TERMS = {

```



```

3029     "large": "large_group_difference",
3030     "moderate": "moderate_group_difference",
3031     "small": "small_group_difference",
3032     "negligible": "negligible_group_difference",
3033 }
3034
3035
3036 def qualitative_strength_conflicts(
3037     sentence: str,
3038     evidence_items: list[EvidenceItem],
3039 ) -> list[str]:
3040     normalised_labels = {
3041         normalise_strength_label(item.strength_label)
3042         for item in evidence_items
3043     }
3044     association_labels = normalised_labels & set(
3045         ASSOCIATION_STRENGTH_TERMS.values()
3046     )
3047     group_labels = normalised_labels & set(
3048         GROUP_STRENGTH_TERMS.values()
3049     )
3050     conflicts: list[str] = []
3051
3052     for match in ASSOCIATION_STRENGTH_PATTERN.finditer(sentence):
3053         visible_label = match.group("label").lower()
3054         expected_label = ASSOCIATION_STRENGTH_TERMS[visible_label]
3055         if association_labels and expected_label not in association_labels:
3056             conflicts.append(
3057                 f"{visible_label} association conflicts with "
3058                 f"{sorted(association_labels)}"
3059             )
3060
3061     for match in GROUP_STRENGTH_PATTERN.finditer(sentence):
3062         visible_label = match.group("label").lower()
3063         expected_label = GROUP_STRENGTH_TERMS[visible_label]
3064         if group_labels and expected_label not in group_labels:
3065             conflicts.append(
3066                 f"{visible_label} group difference conflicts with "
3067                 f"{sorted(group_labels)}"
3068             )
3069
3070     return list(dict.fromkeys(conflicts))
3071
3072 LOW_PRIORITY_STRENGTH_LABELS = {
3073     "negligible",
3074     "negligible_association",
3075     "negligible_group_difference",
3076     "weak_but_reportable_association",
3077     "small_group_difference",
3078 }
3079
3080 RECOVERABLE_CORRELATION_LABELS = {
3081     "very_strong_association",
3082     "strong_association",
3083     "moderate_association",
3084 }
3085
3086 RECOVERABLE_GROUP_LABELS = {
3087     "large_group_difference",
3088     "moderate_group_difference",
3089 }
3090
3091 RECOVERABLE_MODELLING_LABELS = {
3092     "validated_internal_prediction",
3093     "model_not_better_than_baseline",
3094     "validated_forecast",
3095     "forecast_not_better_than_baseline",
3096     "predictive_insufficiency",
3097     "forecast_insufficiency",
3098 }
3099
3100 RECOVERABLE_DATA_QUALITY_LABELS = {
3101     "constant_column",
3102     "duplicate_rows",
3103     "possible_sentinel_zero",
3104     "possible_data_quality_issue",
3105     "material_missingness",
3106     "low_missingness",
3107 }
3108
3109
3110 def evidence_subtype(
3111     item: EvidenceItem,
3112 ) -> str:
3113     metrics = item.metrics
3114     label = normalise_strength_label(
3115         item.strength_label
3116     )
3117
3118     if item.capability == EvidenceCapability.FOCUSED_TABLE_REGION:
3119         return "focused_table_region"

```

```

3120
3121     if (
3122         item.capability
3123         == EvidenceCapability.STRUCTURED_RECORD_VERBALISATION
3124     ):
3125         return "structured_record_verbalisation"
3126
3127     if item.evidence_type == "event_context":
3128         return "event_context"
3129
3130     if item.evidence_type == "event_status":
3131         return "event_status"
3132
3133     if item.evidence_type == "participant_record_context":
3134         return "participant_record_context"
3135
3136     if item.evidence_type == "score_progression":
3137         return "score_progression"
3138
3139     if item.evidence_type == "event_sequence":
3140         return "event_sequence"
3141
3142     if item.capability == EvidenceCapability.EVENT_OUTCOME:
3143         return "event_outcome"
3144
3145     if item.capability in {
3146         EvidenceCapability.ENTITY_PERFORMANCE,
3147         EvidenceCapability.RANKING,
3148     }:
3149         return "entity_performance"
3150
3151     if (
3152         item.capability == EvidenceCapability.GROUP_COMPARISON
3153         and item.evidence_type == "participant_comparison"
3154     ):
3155         return "group_comparison"
3156
3157     if item.route == AnalysisRoute.DESCRPTIVE:
3158         if (
3159             "row_count" in metrics
3160             and "column_count" in metrics
3161         ):
3162             return "dataset_overview"
3163
3164         if (
3165             metrics.get("constant") is True
3166             or "missing_count" in metrics
3167             or "missing_rate" in metrics
3168             or "duplicate_row_count" in metrics
3169             or label
3170             in {
3171                 "constant_column",
3172                 "duplicate_rows",
3173                 "possible_sentinel_zero",
3174                 "possible_data_quality_issue",
3175                 "material_missingness",
3176                 "low_missingness",
3177             }
3178         ):
3179             return "data_quality"
3180
3181         return "descriptive_detail"
3182
3183     if (
3184         item.route
3185         == AnalysisRoute.ASSOCIATION_COMPARISON
3186     ):
3187         if (
3188             "pearson_r" in metrics
3189             or "spearman_r" in metrics
3190             or "correlation" in metrics
3191         ):
3192             return "correlation"
3193
3194         if (
3195             "highest_group" in metrics
3196             or "lowest_group" in metrics
3197             or "group_counts" in metrics
3198             or "standardised_difference"
3199             in metrics
3200             or "standardized_difference"
3201             in metrics
3202         ):
3203             return "group_comparison"
3204
3205         return "association_other"
3206
3207     if item.route == AnalysisRoute.PREDICTIVE:
3208         return "predictive_validation"
3209
3210     if item.route == AnalysisRoute.FORECASTING:

```

```

3211         return "forecast_validation"
3212
3213     if (
3214         item.route
3215         == AnalysisRoute.CAUSAL_FEASIBILITY
3216     ):
3217         return "causal_feasibility"
3218
3219     return "other"
3220
3221 def fact_priority_score(
3222     fact: VerifiedFact,
3223     evidence_lookup: dict[str, EvidenceItem],
3224 ) -> float:
3225     evidence_items = evidence_for_fact(
3226         fact,
3227         evidence_lookup,
3228     )
3229
3230     use_bonus = {
3231         RecommendedUse.HEADLINE: 0.25,
3232         RecommendedUse.MAIN_FINDING: 0.15,
3233         RecommendedUse.SUPPORTING_DETAIL: 0.0,
3234         RecommendedUse.LIMITATION: 0.10,
3235         RecommendedUse.OMIT_UNLESS_REQUESTED: -0.30,
3236     }.get(
3237         fact.recommended_use,
3238         0.0,
3239     )
3240
3241     bonus_by_label = {
3242         "dataset_overview": 0.18,
3243         "constant_column": 0.15,
3244         "possible_sentinel_zero": 0.15,
3245         "possible_data_quality_issue": 0.12,
3246         "material_missingness": 0.12,
3247         "low_missingness": 0.03,
3248         "very_strong_association": 0.22,
3249         "strong_association": 0.17,
3250         "moderate_association": 0.10,
3251         "weak_but_reportable_association": -0.08,
3252         "large_group_difference": 0.18,
3253         "moderate_group_difference": 0.10,
3254         "small_group_difference": -0.15,
3255         "negligible_group_difference": -0.35,
3256         "validated_internal_prediction": 0.15,
3257         "model_not_better_than_baseline": 0.12,
3258         "validated_forecast": 0.15,
3259         "forecast_not_better_than_baseline": 0.12,
3260     }
3261
3262     strength_bonuses = [
3263         bonus_by_label.get(
3264             normalise_strength_label(
3265                 item.strength_label
3266             ),
3267             0.0,
3268         )
3269         for item in evidence_items
3270     ]
3271
3272     strength_bonus = (
3273         max(strength_bonuses)
3274         if strength_bonuses
3275         else 0.0
3276     )
3277
3278     return (
3279         0.30 * fact.salience
3280         + 0.25 * fact.user_relevance
3281         + 0.20 * fact.methodological_strength
3282         + 0.15 * fact.factual_confidence
3283         + use_bonus
3284         + strength_bonus
3285     )
3286
3287 def evidence_priority_score_for_fact(
3288     fact: VerifiedFact,
3289     evidence_lookup: dict[str, EvidenceItem],
3290 ) -> float:
3291     return fact_priority_score(fact, evidence_lookup)
3292
3293
3294 def classify_fact_component(
3295     fact: VerifiedFact,
3296     evidence_lookup: dict[str, EvidenceItem],
3297 ) -> ReportComponent:
3298     items = evidence_for_fact(fact, evidence_lookup)
3299     subtypes = {evidence_subtype(item) for item in items}
3300     permissions = set(fact.claim_permissions)
3301

```

```

3302     if "dataset_overview" in subtypes:
3303         return ReportComponent.DATASET_OVERVIEW
3304
3305     if "data_quality" in subtypes:
3306         return ReportComponent.DATA_QUALITY
3307
3308     if subtypes & {
3309         "event_outcome",
3310         "entity_performance",
3311         "event_sequence",
3312         "participant_record_context",
3313         "score_progression",
3314     }:
3315         return ReportComponent.DATASET_OVERVIEW
3316
3317     if (
3318         "predictive_validation" in subtypes
3319         or "forecast_validation" in subtypes
3320     ):
3321         return ReportComponent.MODELLING_VALIDATION
3322
3323     if (
3324         "correlation" in subtypes
3325         or "group_comparison" in subtypes
3326         or "association_other" in subtypes
3327     ):
3328         return ReportComponent.STRONGEST_RELATIONSHIPS
3329
3330     if (
3331         ClaimPermission.INSUFFICIENCY in permissions
3332         or fact.recommended_use == RecommendedUse.LIMITATION
3333         or fact.required_caveats
3334     ):
3335         return ReportComponent.LIMITATIONS_NEXT_STEPS
3336
3337     return ReportComponent.DATASET_OVERVIEW
3338
3339 def eligible_fact_as_priority(
3340     fact: VerifiedFact,
3341     evidence_lookup: dict[str, EvidenceItem],
3342 ) -> bool:
3343     if fact.recommended_use not in {
3344         RecommendedUse.HEADLINE,
3345         RecommendedUse.MAIN_FINDING,
3346         RecommendedUse.LIMITATION,
3347     }:
3348         return False
3349
3350     strength_labels = {
3351         normalise_strength_label(
3352             item.strength_label
3353         )
3354         for item
3355         in evidence_for_fact(
3356             fact,
3357             evidence_lookup,
3358         )
3359     }
3360
3361     if (
3362         strength_labels
3363         & LOW_PRIORITY_STRENGTH_LABELS
3364     ):
3365         return False
3366
3367     return (
3368         fact.factual_confidence >= 0.90
3369         and fact.methodological_strength
3370         >= 0.70
3371         and fact.user_relevance >= 0.60
3372     )
3373
3374 def evidence_priority_score(
3375     item: EvidenceItem,
3376 ) -> float:
3377     temporary_fact = VerifiedFact(
3378         fact_id="FACT_SCORE_ONLY",
3379         source_candidate_id=(
3380             f"SCORE_{item.evidence_id}"
3381         ),
3382         verification_method=(
3383             VerificationMethod
3384                 .DETERMINISTIC_EVIDENCE_RECOVERY
3385         ),
3386         fact_summary=item.finding,
3387         evidence_ids=[item.evidence_id],
3388         structured_values={
3389             item.evidence_id: item.metrics
3390         },
3391         entities=[

```

```

3393         *item.source_tables,
3394         *item.source_columns,
3395     ],
3396     claim_permissions=(
3397         item.claim_permissions
3398     ),
3399     allowed_interpretations=[
3400         item.practical_interpretation
3401     ],
3402     prohibited_interpretations=(
3403         item.prohibited_interpretations
3404     ),
3405     required_caveats=item.limitations,
3406     factual_confidence=(
3407         item.factual_confidence
3408     ),
3409     methodological_strength=(
3410         item.methodological_strength
3411     ),
3412     user_relevance=item.user_relevance,
3413     salience=item.salience,
3414     recommended_use=item.recommended_use,
3415 )
3416
3417 return fact_priority_score(
3418     temporary_fact,
3419     {
3420         item.evidence_id: item,
3421     },
3422 )
3423
3424 def eligible_for_deterministic_fact_recovery(
3425     item: EvidenceItem,
3426     *,
3427     allow_query_evidence: bool = False,
3428 ) -> bool:
3429     if item.query_id is not None and not allow_query_evidence:
3430         return False
3431
3432     if not item.eligible_for_writer:
3433         return False
3434
3435     if item.factual_confidence < 0.90:
3436         return False
3437
3438     if item.methodological_strength < 0.65:
3439         return False
3440
3441     subtype = evidence_subtype(item)
3442     label = normalise_strength_label(
3443         item.strength_label
3444     )
3445
3446     if subtype == "dataset_overview":
3447         return True
3448
3449     if subtype == "focused_table_region":
3450         return True
3451
3452     if subtype in {
3453         "event_context",
3454         "event_status",
3455         "participant_record_context",
3456         "score_progression",
3457         "event_outcome",
3458         "entity_performance",
3459         "event_sequence",
3460     }:
3461         return True
3462
3463     if subtype == "data_quality":
3464         return (
3465             label
3466             in RECOVERABLE_DATA_QUALITY_LABELS
3467         )
3468
3469     if subtype == "correlation":
3470         return (
3471             label
3472             in RECOVERABLE_CORRELATION_LABELS
3473         )
3474
3475     if subtype == "group_comparison":
3476         if item.capability == EvidenceCapability.GROUP_COMPARISON:
3477             return True
3478         return (
3479             label in RECOVERABLE_GROUP_LABELS
3480         )
3481
3482     if subtype in {

```

```

3484     "predictive_validation",
3485     "forecast_validation",
3486   }:
3487     return (
3488       label
3489       in RECOVERABLE_MODELLING_LABELS
3490       or ClaimPermission.INSUFFICIENCY
3491       in item.claim_permissions
3492     )
3493
3494   if subtype == "causal_feasibility":
3495     return (
3496       ClaimPermission.INSUFFICIENCY
3497       in item.claim_permissions
3498     )
3499
3500   return False
3501
3502
3503 def deterministic_fact_summary_from_evidence(
3504     item: EvidenceItem,
3505 ) -> str:
3506     if item.capability == EvidenceCapability.FOCUSED_TABLE_REGION:
3507         return _focused_table_fact_summary(item)
3508
3509     if (
3510         item.capability
3511         == EvidenceCapability.STRUCTURED_RECORD_VERBALISATION
3512     ):
3513         return _structured_record_fact_summary(item)
3514
3515     if (
3516         item.capability
3517         in {
3518             EvidenceCapability.EVENT_OUTCOME,
3519             EvidenceCapability.ENTITY_PERFORMANCE,
3520             EvidenceCapability.RANKING,
3521             EvidenceCapability.GROUP_COMPARISON,
3522         }
3523         or evidence_subtype(item)
3524         in {
3525             "event_context",
3526             "event_status",
3527             "participant_record_context",
3528             "score_progression",
3529             "event_outcome",
3530             "entity_performance",
3531             "event_sequence",
3532             "group_comparison",
3533         }
3534     ):
3535         return _semantic_event_fact_summary(item)
3536
3537     return item.finding
3538
3539
3540 def deterministic_fact_from_evidence(
3541     *,
3542     item: EvidenceItem,
3543     ordinal: int,
3544 ) -> VerifiedFact:
3545     entities = set(
3546         [
3547             *item.source_tables,
3548             *item.source_columns,
3549         ]
3550     )
3551
3552     entities.update(
3553         collect_entity_strings(item.metrics)
3554     )
3555     entities.update(item.entity_scope)
3556
3557     return VerifiedFact(
3558         fact_id=f"FACT_REC_{ordinal:04d}",
3559         source_candidate_id=(
3560             f"RECOVERY_{item.evidence_id}"
3561         ),
3562         verification_method=(
3563             VerificationMethod
3564             .DETERMINISTIC_EVIDENCE_RECOVERY
3565         ),
3566         fact_summary=deterministic_fact_summary_from_evidence(item),
3567         evidence_ids=[item.evidence_id],
3568         source_capabilities=[item.capability],
3569         structured_values={
3570             item.evidence_id: item.metrics
3571         },
3572         entities=sorted(entities),
3573         claim_permissions=(
3574             item.claim_permissions

```

```

3575         ),
3576         allowed_interpretations=(
3577             [item.practical_interpretation]
3578             if item.practical_interpretation
3579             else []
3580         ),
3581         prohibited_interpretations=(
3582             item.prohibited_interpretations
3583         ),
3584         required_caveats=item.limitations,
3585         factual_confidence=(
3586             item.factual_confidence
3587         ),
3588         methodological_strength=(
3589             item.methodological_strength
3590         ),
3591         user_relevance=item.user_relevance,
3592         salience=item.salience,
3593         recommended_use=item.recommended_use,
3594     )
3595
3596
3597 def fact_subtype_counts(
3598     *,
3599     facts: list[VerifiedFact],
3600     evidence_lookup: dict[str, EvidenceItem],
3601 ) -> Counter[str]:
3602     counts: Counter[str] = Counter()
3603
3604     for fact in facts:
3605         represented_subtypes = {
3606             evidence_subtype(
3607                 evidence_lookup[evidence_id]
3608             )
3609             for evidence_id in fact.evidence_ids
3610             if evidence_id in evidence_lookup
3611         }
3612
3613         for subtype in represented_subtypes:
3614             counts[subtype] += 1
3615
3616     return counts
3617
3618
3619 def augment_fact_ledger_for_report_coverage(
3620     *,
3621     fact_ledger: FactLedger,
3622     evidence: EvidenceLedger,
3623     required_components: list[ReportComponent],
3624     required_content_slots: list[str] | None = None,
3625     settings: Settings,
3626 ) -> FactLedger:
3627     """
3628     Augment a thin fact ledger using exact, trusted deterministic
3629     evidence statements.
3630
3631     This does not create new calculations or LLM interpretations.
3632     """
3633
3634     lookup = build_evidence_lookup(evidence)
3635
3636     represented_evidence_ids = {
3637         evidence_id
3638         for fact
3639         in fact_ledger.writer_ready_facts
3640         for evidence_id in fact.evidence_ids
3641     }
3642
3643     existing_counts = fact_subtype_counts(
3644         facts=fact_ledger.writer_ready_facts,
3645         evidence_lookup=lookup,
3646     )
3647
3648     candidates = sorted(
3649         [
3650             item
3651             for item in evidence.items
3652             if (
3653                 item.evidence_id
3654                 not in represented_evidence_ids
3655                 and eligible_for_deterministic_fact_recovery(item)
3656             )
3657         ],
3658         key=evidence_priority_score,
3659         reverse=True,
3660     )
3661
3662     subtype_limits = {
3663         "dataset_overview": (
3664             settings
3665             .maximum_recovered_overview_facts

```



```

3666     ),
3667     "data_quality": (
3668         settings
3669         .maximum_recovered_data_quality_facts
3670     ),
3671     "correlation": (
3672         settings
3673         .maximum_recovered_correlation_facts
3674     ),
3675     "group_comparison": (
3676         settings
3677         .maximum_recovered_group_comparison_facts
3678     ),
3679     "predictive_validation": (
3680         settings
3681         .maximum_recovered_modelling_facts
3682     ),
3683     "forecast_validation": (
3684         settings
3685         .maximum_recovered_modelling_facts
3686     ),
3687     "causal_feasibility": 1,
3688     "event_context": 2,
3689     "event_status": 2,
3690     "participant_record_context": 2,
3691     "score_progression": 2,
3692     "event_outcome": 2,
3693     "entity_performance": 4,
3694     "event_sequence": 2,
3695 }
3696
3697 recovered: list[VerifiedFact] = []
3698 recovered_counts: Counter[str] = Counter()
3699 recovery_notes: list[str] = []
3700 recovered_evidence_ids: set[str] = set()
3701
3702 existing_fact_ids = {
3703     fact.fact_id
3704     for fact
3705     in fact_ledger.writer_ready_facts
3706 }
3707
3708 next_ordinal = 1
3709
3710 def next_fact_id_ordinal() -> int:
3711     nonlocal next_ordinal
3712
3713     while (
3714         f"FACT_REC_{next_ordinal:04d}"
3715         in existing_fact_ids
3716     ):
3717         next_ordinal += 1
3718
3719     value = next_ordinal
3720     next_ordinal += 1
3721
3722     return value
3723
3724 def recover(item: EvidenceItem) -> bool:
3725     subtype = evidence_subtype(item)
3726
3727     if (
3728         item.evidence_id
3729         in recovered_evidence_ids
3730     ):
3731         return False
3732
3733     maximum = subtype_limits.get(
3734         subtype,
3735         0,
3736     )
3737
3738     if (
3739         recovered_counts[subtype]
3740         >= maximum
3741     ):
3742         return False
3743
3744     fact = deterministic_fact_from_evidence(
3745         item=item,
3746         ordinal=next_fact_id_ordinal(),
3747     )
3748
3749     recovered.append(fact)
3750     recovered_counts[subtype] += 1
3751     recovered_evidence_ids.add(
3752         item.evidence_id
3753     )
3754
3755     recovery_notes.append(
3756         "Recovered writer-ready fact "

```

```

3757         f"{fact.fact_id}` from deterministic "
3758         f"evidence `{item.evidence_id}` for "
3759         f"subtype `{subtype}`."
3760     )
3761
3762     return True
3763
3764 def recover_best(
3765     subtype: str,
3766     *,
3767     allow_query_evidence: bool = False,
3768     allow_represented_evidence: bool = False,
3769 ) -> bool:
3770     source_items = candidates
3771     if allow_query_evidence or allow_represented_evidence:
3772         source_items = sorted(
3773             [
3774                 item
3775                 for item in evidence.items
3776                 if (
3777                     (
3778                         allow_represented_evidence
3779                         or item.evidence_id
3780                         not in represented_evidence_ids
3781                     )
3782                     and eligible_for_deterministic_fact_recovery(
3783                         item,
3784                         allow_query_evidence=allow_query_evidence,
3785                     )
3786                 )
3787             ],
3788             key=evidence_priority_score,
3789             reverse=True,
3790         )
3791
3792     for item in source_items:
3793         if (
3794             evidence_subtype(item)
3795             == subtype
3796             and item.evidence_id
3797             not in recovered_evidence_ids
3798         ):
3799             if recover(item):
3800                 return True
3801
3802     return False
3803
3804 slot_subtypes = {
3805     "event_context": "event_context",
3806     "event_status": "event_status",
3807     "participant_record_context": "participant_record_context",
3808     "score_progression": "score_progression",
3809     "event_sequence": "event_sequence",
3810     "event_result": "event_outcome",
3811     "leading_performance": "entity_performance",
3812     "main_contrast": "group_comparison",
3813     "secondary_performance": "entity_performance",
3814 }
3815
3816 def existing_required_slot_fact_count(
3817     subtype: str,
3818 ) -> int:
3819     return sum(
3820         1
3821         for fact in fact_ledger.writer_ready_facts
3822         if (
3823             fact.recommended_use
3824             != RecommendedUse.OMIT_UNLESS_REQUESTED
3825             and any(
3826                 evidence_id in lookup
3827                 and evidence_subtype(lookup[evidence_id])
3828                 == subtype
3829                 for evidence_id in fact.evidence_ids
3830             )
3831         )
3832     )
3833
3834 for slot in required_content_slots or []:
3835     subtype = slot_subtypes.get(slot)
3836     if subtype is None:
3837         continue
3838     if (
3839         existing_required_slot_fact_count(subtype)
3840         + recovered_counts[subtype]
3841         == 0
3842     ):
3843         recover_best(
3844             subtype,
3845             allow_query_evidence=True,
3846             allow_represented_evidence=True,
3847         )

```

```

3848
3849 if (
3850     ReportComponent.DATASET_OVERVIEW
3851     in required_components
3852 ):
3853     while (
3854         existing_counts["dataset_overview"]
3855         + recovered_counts[
3856             "dataset_overview"
3857         ]
3858         < settings.minimum_overview_fact_count
3859     ):
3860         if not recover_best(
3861             "dataset_overview"
3862         ):
3863             break
3864
3865 if (
3866     ReportComponent.DATA_QUALITY
3867     in required_components
3868 ):
3869     while (
3870         existing_counts["data_quality"]
3871         + recovered_counts["data_quality"]
3872         < settings.minimum_data_quality_fact_count
3873     ):
3874         if not recover_best("data_quality"):
3875             break
3876
3877 if (
3878     ReportComponent.STRONGEST_RELATIONSHIPS
3879     in required_components
3880 ):
3881     # Prefer subtype diversity when both forms are available.
3882     if (
3883         existing_counts["correlation"] == 0
3884         and any(
3885             evidence_subtype(item)
3886             == "correlation"
3887             for item in candidates
3888         )
3889     ):
3890         recover_best("correlation")
3891
3892     if (
3893         existing_counts["group_comparison"]
3894         == 0
3895         and any(
3896             evidence_subtype(item)
3897             == "group_comparison"
3898             for item in candidates
3899         )
3900     ):
3901         recover_best("group_comparison")
3902
3903     def relationship_count() -> int:
3904         return sum(
3905             existing_counts[subtype]
3906             + recovered_counts[subtype]
3907             for subtype in {
3908                 "correlation",
3909                 "group_comparison",
3910                 "association_other",
3911             }
3912         )
3913
3914     while (
3915         relationship_count()
3916         < settings.minimum_relationship_fact_count
3917     ):
3918         recovered_one = (
3919             recover_best("correlation")
3920             or recover_best(
3921                 "group_comparison"
3922             )
3923         )
3924
3925         if not recovered_one:
3926             break
3927
3928     # Fill a thin ledger only with remaining eligible evidence.
3929     for item in candidates:
3930         if (
3931             len(
3932                 fact_ledger.writer_ready_facts
3933             )
3934             + len(recovered)
3935             >= settings.minimum_writer_ready_fact_count
3936         ):
3937             break
3938

```

```

3939         recover(item)
3940
3941     if not recovered:
3942         return fact_ledger
3943
3944     return fact_ledger.model_copy(
3945         update={
3946             "writer_ready_facts": [
3947                 *fact_ledger.writer_ready_facts,
3948                 *recovered,
3949             ],
3950             "deterministically_recovered_fact_ids": [
3951                 *fact_ledger
3952                 .deterministically_recovered_fact_ids,
3953                 *[
3954                     fact.fact_id
3955                     for fact in recovered
3956                 ],
3957             ],
3958             "coverage_recovery_notes": [
3959                 *fact_ledger.coverage_recovery_notes,
3960                 *recovery_notes,
3961             ],
3962         }
3963     )
3964
3965 def select_balanced_priority_facts(
3966     *,
3967     facts: list[VerifiedFact],
3968     evidence: EvidenceLedger,
3969     required_components: list[ReportComponent],
3970     settings: Settings,
3971 ) -> list[VerifiedFact]:
3972     """
3973     Select facts by coverage and analytical strength.
3974
3975     There is deliberately no unrestricted final fill stage. Weak or
3976     small facts remain available in supporting_facts but are not
3977     promoted simply because unused capacity remains.
3978     """
3979
3980     lookup = build_evidence_lookup(evidence)
3981
3982     ranked = sorted(
3983         facts,
3984         key=lambda fact: fact_priority_score(
3985             fact,
3986             lookup,
3987         ),
3988         reverse=True,
3989     )
3990
3991     selected: list[VerifiedFact] = []
3992     selected_ids: set[str] = set()
3993     subtype_counts: Counter[str] = Counter()
3994
3995     def primary_subtype(
3996         fact: VerifiedFact,
3997     ) -> str:
3998         subtypes = [
3999             evidence_subtype(item)
4000             for item
4001             in evidence_for_fact(
4002                 fact,
4003                 lookup,
4004             )
4005         ]
4006
4007         return (
4008             subtypes[0]
4009             if subtypes
4010             else "other"
4011         )
4012
4013     def priority_eligible(
4014         fact: VerifiedFact,
4015     ) -> bool:
4016         return (
4017             eligible_fact_as_priority(
4018                 fact,
4019                 lookup,
4020             )
4021             and fact_priority_score(
4022                 fact,
4023                 lookup,
4024             )
4025             >= settings.minimum_main_finding_score
4026         )
4027
4028     def add_fact(
4029         fact: VerifiedFact,

```

```

4030     *,
4031     require_priority_eligibility: bool,
4032 ) -> bool:
4033     if fact.fact_id in selected_ids:
4034         return False
4035
4036     subtype = primary_subtype(fact)
4037
4038     if (
4039         require_priority_eligibility
4040         and not priority_eligible(fact)
4041     ):
4042         return False
4043
4044     selected.append(fact)
4045     selected_ids.add(fact.fact_id)
4046     subtype_counts[subtype] += 1
4047
4048     return True
4049
4050 def add_best_for_subtype(
4051     subtype: str,
4052     *,
4053     require_priority_eligibility: bool,
4054 ) -> bool:
4055     for fact in ranked:
4056         if primary_subtype(fact) != subtype:
4057             continue
4058
4059         if add_fact(
4060             fact,
4061             require_priority_eligibility=(
4062                 require_priority_eligibility
4063             ),
4064         ):
4065             return True
4066
4067     return False
4068
4069 add_best_for_subtype(
4070     "event_outcome",
4071     require_priority_eligibility=False,
4072 )
4073 add_best_for_subtype(
4074     "entity_performance",
4075     require_priority_eligibility=False,
4076 )
4077
4078 if (
4079     ReportComponent.DATASET_OVERVIEW
4080     in required_components
4081 ):
4082     add_best_for_subtype(
4083         "dataset_overview",
4084         require_priority_eligibility=False,
4085     )
4086
4087 if (
4088     ReportComponent.DATA_QUALITY
4089     in required_components
4090 ):
4091     add_best_for_subtype(
4092         "data_quality",
4093         require_priority_eligibility=False,
4094     )
4095
4096 if (
4097     ReportComponent.STRONGEST_RELATIONSHIPS
4098     in required_components
4099 ):
4100     # Prefer one strong correlation and one strong group comparison.
4101     add_best_for_subtype(
4102         "correlation",
4103         require_priority_eligibility=True,
4104     )
4105
4106     event_facts_present = any(
4107         EvidenceCapability.EVENT_OUTCOME
4108         in fact.source_capabilities
4109         for fact in facts
4110     )
4111
4112     add_best_for_subtype(
4113         "group_comparison",
4114         require_priority_eligibility=(
4115             not event_facts_present
4116         ),
4117     )
4118
4119 def selected_relationship_count() -> int:
4120     return sum(

```

```

4121         subtype_counts[subtype]
4122         for subtype in {
4123             "correlation",
4124             "group_comparison",
4125             "association_other",
4126         }
4127     )
4128
4129     for fact in ranked:
4130         if (
4131             selected_relationship_count()
4132             >= settings.minimum_relationship_fact_count
4133         ):
4134             break
4135
4136         if primary_subtype(fact) not in {
4137             "correlation",
4138             "group_comparison",
4139             "association_other",
4140         }:
4141             continue
4142
4143         add_fact(
4144             fact,
4145             require_priority_eligibility=True,
4146         )
4147
4148     if (
4149         ReportComponent.MODELLING_VALIDATION
4150         in required_components
4151     ):
4152         add_best_for_subtype(
4153             "predictive_validation",
4154             require_priority_eligibility=False,
4155         )
4156
4157         if not any(
4158             primary_subtype(fact)
4159             in {
4160                 "predictive_validation",
4161                 "forecast_validation",
4162             }
4163             for fact in selected
4164         ):
4165             add_best_for_subtype(
4166                 "forecast_validation",
4167                 require_priority_eligibility=False,
4168             )
4169
4170     if (
4171         ReportComponent.LIMITATIONS_NEXT_STEPS
4172         in required_components
4173     ):
4174         for fact in ranked:
4175             if (
4176                 fact.recommended_use
4177                 == RecommendedUse.LIMITATION
4178                 or ClaimPermission.INSUFFICIENCY
4179                 in fact.claim_permissions
4180             ):
4181                 if add_fact(
4182                     fact,
4183                     require_priority_eligibility=False,
4184                 ):
4185                     break
4186
4187     # Add only high-quality eligible facts. Do not fill with weak facts.
4188     for fact in ranked:
4189         if (
4190             settings.writer_priority_fact_limit is not None
4191             and len(selected)
4192             >= settings.writer_priority_fact_limit
4193         ):
4194             break
4195         add_fact(
4196             fact,
4197             require_priority_eligibility=True,
4198         )
4199
4200     return selected
4201
4202 def select_priority_facts(
4203     facts: list[VerifiedFact],
4204     evidence: EvidenceLedger,
4205     required_components: list[ReportComponent],
4206     settings: Settings,
4207 ) -> list[VerifiedFact]:
4208     return select_balanced_priority_facts(
4209         facts=facts,
4210         evidence=evidence,
4211         required_components=required_components,

```

```

4212         settings=settings,
4213     )
4214
4215
4216 def reader_facing_caveat(text: str) -> str | None:
4217     lowered = text.strip().lower()
4218
4219     if not text.strip():
4220         return None
4221
4222     if lowered.startswith(("do not", "never", "prohibited", "internal")):
4223         if "caus" in lowered:
4224             return (
4225                 "These are unadjusted descriptive comparisons and should not "
4226                 "be interpreted as causal effects."
4227             )
4228         if "confounding" in lowered or "adjusted" in lowered:
4229             return "The comparisons are unadjusted and do not control for confounding."
4230         if "predict" in lowered:
4231             return (
4232                 "Predictive wording should be limited to validated modelling "
4233                 "results and not inferred from descriptive evidence."
4234             )
4235         return None
4236
4237     return text.strip()
4238
4239
4240 def build_reader_facing_limitations(
4241     facts: list[VerifiedFact],
4242 ) -> list[str]:
4243     limitations: list[str] = []
4244     has_group_comparison = any(
4245         ClaimPermission.COMPARATIVE in fact.claim_permissions
4246         for fact in facts
4247     )
4248     has_association = any(
4249         ClaimPermission.ASSOCIATIONAL in fact.claim_permissions
4250         for fact in facts
4251     )
4252     has_predictive = any(
4253         ClaimPermission.PREDICTIVE in fact.claim_permissions
4254         for fact in facts
4255     )
4256     has_forecast = any(
4257         ClaimPermission.FORECAST in fact.claim_permissions
4258         for fact in facts
4259     )
4260
4261     if has_group_comparison or has_association:
4262         limitations.append(
4263             "Observed associations and group comparisons are descriptive. "
4264             "They are not evidence of causal effects."
4265         )
4266
4267     if has_group_comparison:
4268         limitations.append(
4269             "Group comparisons are unadjusted unless the evidence explicitly "
4270             "states otherwise. Unequal group sizes may lead to different "
4271             "levels of precision and stability."
4272         )
4273
4274     if has_predictive:
4275         limitations.append(
4276             "Predictive results describe internal validation under the tested "
4277             "setup and do not establish deployment readiness."
4278         )
4279
4280     if has_forecast:
4281         limitations.append(
4282             "Forecast results are internal backtests and do not guarantee "
4283             "future performance."
4284         )
4285
4286     return list(dict.fromkeys(limitations))
4287
4288
4289 def select_priority_verified_insights(
4290     *,
4291     insight_ledger: InsightLedger,
4292     maximum: int | None,
4293 ) -> tuple[list[VerifiedInsight], list[VerifiedInsight]]:
4294     eligible = [
4295         insight
4296         for insight in insight_ledger.verified_insights
4297         if insight.verification_status
4298         in {
4299             InsightVerificationStatus.VERIFIED,
4300             InsightVerificationStatus.VERIFIED_WITH_CAVEAT,
4301         }
4302         and insight.interpretation_level

```



```

4303     == InterpretationLevel.BOUNDED_INSIGHT
4304 ]
4305 ranked = sorted(
4306     enumerate(eligible),
4307     key=lambda item: (
4308         -item[1].salience,
4309         -item[1].confidence,
4310         item[0],
4311     ),
4312 )
4313
4314 priority: list[VerifiedInsight] = []
4315 selected_ids: set[str] = set()
4316 selected_types: set[InsightType] = set()
4317
4318 priority_limit = maximum or len(ranked)
4319
4320 for _, insight in ranked:
4321     if len(priority) >= priority_limit:
4322         break
4323     if insight.insight_type in selected_types:
4324         continue
4325     priority.append(insight)
4326     selected_ids.add(insight.insight_id)
4327     selected_types.add(insight.insight_type)
4328
4329 for _, insight in ranked:
4330     if len(priority) >= priority_limit:
4331         break
4332     if insight.insight_id in selected_ids:
4333         continue
4334     priority.append(insight)
4335     selected_ids.add(insight.insight_id)
4336
4337 supporting = [
4338     insight
4339     for _, insight in ranked
4340     if insight.insight_id not in selected_ids
4341 ]
4342
4343 return priority, supporting
4344
4345
4346 def fact_is_relevant_to_genre(
4347     fact: VerifiedFact,
4348     evidence_lookup: dict[str, EvidenceItem],
4349     genre: ReportGenre,
4350 ) -> bool:
4351     if genre not in {
4352         ReportGenre.EVENT_REPORT,
4353         ReportGenre.SPORTS_GAME_REPORT,
4354     }:
4355         return True
4356
4357     items = [
4358         evidence_lookup[evidence_id]
4359         for evidence_id in fact.evidence_ids
4360         if evidence_id in evidence_lookup
4361     ]
4362     if not items:
4363         return False
4364
4365     event_evidence_types = {
4366         "event_outcome",
4367         "event_context",
4368         "event_status",
4369         "participant_record_context",
4370         "score_progression",
4371         "entity_ranking",
4372         "entity_performance",
4373         "event_sequence",
4374         "participant_comparison",
4375         "event_contrast",
4376     }
4377     return any(
4378         item.eligible_for_writer
4379         and item.evidence_type in event_evidence_types
4380         for item in items
4381     )
4382
4383
4384 def evidence_analytical_function(
4385     item: EvidenceItem,
4386 ) -> AnalyticalFunction | None:
4387     if item.analytical_function is not None:
4388         return item.analytical_function
4389
4390     value = item.metrics.get("analytical_function")
4391     if not isinstance(value, str):
4392         return None
4393

```

```

4394     try:
4395         return AnalyticalFunction(value)
4396     except ValueError:
4397         return None
4398
4399
4400 def event_evidence_is_segment_ranking(
4401     item: EvidenceItem,
4402 ) -> bool:
4403     if item.evidence_type not in {
4404         "entity_ranking",
4405         "entity_performance",
4406     }:
4407         return False
4408
4409     text_parts = [
4410         item.finding,
4411         item.strength_label,
4412         item.metrics.get("semantic_label"),
4413         item.metrics.get("question"),
4414         item.metrics.get("measure"),
4415         *item.source_paths,
4416     ]
4417     text = " ".join(str(part) for part in text_parts if part)
4418     return bool(EVENT_SEGMENT_RANKING_PATTERN.search(text))
4419
4420
4421 def event_fact_has_low_priority_entity_metric(
4422     fact: VerifiedFact,
4423     evidence_lookup: dict[str, EvidenceItem],
4424 ) -> bool:
4425     for item in evidence_for_fact(fact, evidence_lookup):
4426         if item.evidence_type not in {
4427             "entity_ranking",
4428             "entity_performance",
4429         }:
4430             continue
4431         text_parts = [
4432             item.finding,
4433             item.strength_label,
4434             item.metrics.get("semantic_label"),
4435             item.metrics.get("question"),
4436             item.metrics.get("measure"),
4437             fact.fact_summary,
4438             *item.source_paths,
4439         ]
4440         text = " ".join(str(part) for part in text_parts if part)
4441         if (
4442             item.evidence_type == "entity_performance"
4443             and EVENT_RECAP_CORE_ENTITY_METRIC_PATTERN.search(text)
4444         ):
4445             continue
4446         if EVENT_LOW_PRIORITY_ENTITY_METRIC_PATTERN.search(text):
4447             return True
4448     return False
4449
4450
4451 def event_fact_is_low_priority_for_recap(
4452     fact: VerifiedFact,
4453     evidence_lookup: dict[str, EvidenceItem],
4454 ) -> bool:
4455     slot = event_fact_slot(fact, evidence_lookup)
4456     items = evidence_for_fact(fact, evidence_lookup)
4457     text = " ".join(
4458         str(part)
4459         for item in items
4460         for part in [
4461             item.finding,
4462             item.strength_label,
4463             item.metrics.get("metric"),
4464             item.metrics.get("semantic_label"),
4465             item.metrics.get("question"),
4466             item.metrics.get("measure"),
4467             fact.fact_summary,
4468         ]
4469         if part
4470     )
4471     if slot == "leading_performance":
4472         if event_fact_has_low_priority_entity_metric(fact, evidence_lookup):
4473             return True
4474         if (
4475             any(item.evidence_type == "entity_performance" for item in items)
4476             and EVENT_RECAP_CORE_ENTITY_METRIC_PATTERN.search(text)
4477         ):
4478             return False
4479         if (
4480             any(item.evidence_type == "entity_ranking" for item in items)
4481             and not EVENT_RECAP_CORE_ENTITY_METRIC_PATTERN.search(text)
4482         ):
4483             return True
4484     if slot == "main_contrast":

```

```

4485         if EVENT_RECAP_LOW_PRIORITY_METRIC_PATTERN.search(text):
4486             return True
4487         if re.search(r"\bpoints?\b", text, re.IGNORECASE):
4488             return True
4489         if slot == "participant_record_context":
4490             return False
4491         return bool(EVENT_RECAP_LOW_PRIORITY_METRIC_PATTERN.search(text))
4492
4493
4494 def event_fact_slot(
4495     fact: VerifiedFact,
4496     evidence_lookup: dict[str, EvidenceItem],
4497 ) -> str | None:
4498     items = evidence_for_fact(fact, evidence_lookup)
4499     evidence_types = {
4500         item.evidence_type
4501         for item in items
4502     }
4503
4504     if "event_outcome" in evidence_types:
4505         return "event_result"
4506
4507     if "event_status" in evidence_types:
4508         return "event_status"
4509
4510     if "participant_record_context" in evidence_types:
4511         return "participant_record_context"
4512
4513     if "score_progression" in evidence_types:
4514         return "score_progression"
4515
4516     if "event_sequence" in evidence_types:
4517         return "event_sequence"
4518
4519     if "event_context" in evidence_types:
4520         return "event_context"
4521
4522     if evidence_types & {"entity_ranking", "entity_performance"}:
4523         if any(
4524             evidence_analytical_function(item)
4525             == AnalyticalFunction.PARTICIPATION
4526             for item in items
4527         ):
4528             return "participation"
4529
4530         if any(event_evidence_is_segment_ranking(item) for item in items):
4531             return "event_sequence"
4532
4533         return "leading_performance"
4534
4535     if evidence_types & {
4536         "participant_comparison",
4537         "event_contrast",
4538     }:
4539         return "main_contrast"
4540
4541     return None
4542
4543
4544 def select_event_priority_facts(
4545     *,
4546     facts: list[VerifiedFact],
4547     evidence: EvidenceLedger,
4548     settings: Settings,
4549     request: str,
4550     report_specification: Any | None = None,
4551 ) -> tuple[list[VerifiedFact], list[VerifiedFact]]:
4552     evidence_lookup = build_evidence_lookup(evidence)
4553     realisation_policy = getattr(
4554         report_specification,
4555         "realisation_policy",
4556         None,
4557     )
4558     realisation_policy_value = (
4559         realisation_policy.value
4560         if isinstance(realisation_policy, RealisationPolicy)
4561         else str(realisation_policy)
4562     )
4563     reference_recap_style = bool(
4564         report_specification is not None
4565         and (
4566             getattr(report_specification, "focus_scope", None)
4567             == "reference_recap"
4568             or realisation_policy_value
4569             == RealisationPolicy.EVENT_RECAP_STYLE.value
4570         )
4571     )
4572
4573 def event_priority_score(
4574     fact: VerifiedFact,
4575 ) -> float:

```

```

4576     slot = event_fact_slot(fact, evidence_lookup)
4577     analytical_functions = {
4578         evidence_analytical_function(item)
4579         for item in evidence_for_fact(
4580             fact,
4581             evidence_lookup,
4582         )
4583     }
4584     component_bonus = (
4585         0.20
4586         if (
4587             slot == "main_contrast"
4588             and AnalyticalFunction.OUTCOME_COMPONENT
4589             in analytical_functions
4590         )
4591         else 0.0
4592     )
4593     sequence_bonus = 0.0
4594     if slot == "event_sequence":
4595         sequence_bonus = (
4596             0.35
4597             if event_sequence_fact_is_actionable(
4598                 fact,
4599                 evidence_lookup,
4600             )
4601             else -0.25
4602         )
4603
4604     return (
4605         evidence_priority_score_for_fact(
4606             fact,
4607             evidence_lookup,
4608         )
4609         + component_bonus
4610         + sequence_bonus
4611     )
4612
4613     ranked = sorted(
4614         facts,
4615         key=event_priority_score,
4616         reverse=True,
4617     )
4618     explicit_detail_requested = bool(
4619         re.search(
4620             r"\b(all|complete|detailed|every|exhaustive|full)\b",
4621             request,
4622             re.IGNORECASE,
4623         )
4624     )
4625     priority_limit = settings.writer_priority_fact_limit
4626     supporting_limit = (
4627         settings.writer_supporting_fact_limit
4628         if settings.writer_supporting_fact_limit is not None
4629         else None
4630     )
4631     uncapped_event_selection = (
4632         priority_limit is None
4633         and settings.writer_max_words is None
4634     )
4635     actionable_sequence_available = any(
4636         event_fact_slot(fact, evidence_lookup) == "event_sequence"
4637         and event_sequence_fact_is_actionable(fact, evidence_lookup)
4638         for fact in facts
4639     )
4640
4641     selected: list[VerifiedFact] = []
4642     selected_ids: set[str] = set()
4643     slot_counts: dict[str, int] = {}
4644     if uncapped_event_selection:
4645         slot_limits: dict[str, int | None] = {
4646             "event_result": None,
4647             "event_context": None,
4648             "event_status": None,
4649             "participant_record_context": None,
4650             "score_progression": None,
4651             "event_sequence": None,
4652             "main_contrast": None,
4653             "leading_performance": None,
4654             "participation": None if explicit_detail_requested else 0,
4655         }
4656     else:
4657         slot_limits = {
4658             "event_result": 2,
4659             "event_context": 1,
4660             "event_status": 1,
4661             "participant_record_context": 1,
4662             "score_progression": 1,
4663             "event_sequence": None,
4664             "main_contrast": 2,
4665             "leading_performance": None,
4666             "participation": None if explicit_detail_requested else 0,

```

```

4667     }
4668     if reference_recap_style and not explicit_detail_requested:
4669         slot_limits.update(
4670             {
4671                 "event_result": 2,
4672                 "event_context": 1,
4673                 "event_status": 1,
4674                 "participant_record_context": 3,
4675                 "score_progression": 1,
4676                 "main_contrast": 3,
4677                 "leading_performance": 10,
4678                 "participation": 0,
4679             }
4680         )
4681
4682     def can_use_fact(
4683         fact: VerifiedFact,
4684         *,
4685         as_supporting: bool = False,
4686     ) -> bool:
4687         if fact.fact_id in selected_ids and not as_supporting:
4688             return False
4689         if (
4690             fact.recommended_use == RecommendedUse.OMIT_UNLESS_REQUESTED
4691             and not explicit_detail_requested
4692         ):
4693             return False
4694         if (
4695             reference_recap_style
4696             and not explicit_detail_requested
4697             and event_fact_is_low_priority_for_recap(
4698                 fact,
4699                 evidence_lookup,
4700             )
4701         ):
4702             return False
4703         slot = event_fact_slot(fact, evidence_lookup)
4704         if slot == "participation" and not explicit_detail_requested:
4705             return False
4706         if (
4707             slot == "event_sequence"
4708             and actionable_sequence_available
4709             and not event_sequence_fact_is_actionable(
4710                 fact,
4711                 evidence_lookup,
4712             )
4713         ):
4714             return False
4715         if (
4716             slot == "leading_performance"
4717             and event_fact_has_low_priority_entity_metric(
4718                 fact,
4719                 evidence_lookup,
4720             )
4721             and not explicit_detail_requested
4722         ):
4723             return False
4724         return True
4725
4726     for slot in (
4727         "event_result",
4728         "event_context",
4729         "participant_record_context",
4730         "event_status",
4731         "score_progression",
4732         "leading_performance",
4733         "main_contrast",
4734         "event_sequence",
4735         "participation",
4736     ):
4737         if priority_limit is not None and len(selected) >= priority_limit:
4738             break
4739         limit = slot_limits.get(slot)
4740         if limit == 0:
4741             continue
4742         for fact in ranked:
4743             if priority_limit is not None and len(selected) >= priority_limit:
4744                 break
4745             if event_fact_slot(fact, evidence_lookup) != slot:
4746                 continue
4747             if not can_use_fact(fact):
4748                 continue
4749             if (
4750                 limit is not None
4751                 and slot_counts.get(slot, 0) >= limit
4752             ):
4753                 break
4754
4755             selected.append(fact)
4756             selected_ids.add(fact.fact_id)
4757             slot_counts[slot] = slot_counts.get(slot, 0) + 1

```

```

4758 supporting: list[VerifiedFact] = []
4759 for fact in ranked:
4760     if fact.fact_id in selected_ids:
4761         continue
4762     if not can_use_fact(fact, as_supporting=True):
4763         continue
4764     supporting.append(fact)
4765     if (
4766         supporting_limit is not None
4767         and len(supporting) >= supporting_limit
4768     ):
4769         break
4770     break
4771
4772 return selected, supporting
4773
4774 EVENT_PROFILE_INSIGHT_PATTERN = re.compile(
4775     r"\b(missing(?:ness| data| values?))|duplicates?|rows?|columns?|"
4776     r"constant columns?|correlations?|regressions?|"
4777     r"statistical(?:modelling|modeling|power)|"
4778     r"predictive(?:modelling|modeling)|feature sets?)\b",
4779     re.IGNORECASE,
4780 )
4781
4782
4783 def event_insight_is_relevant(
4784     insight: VerifiedInsight,
4785     request: str,
4786 ) -> bool:
4787     text = f"{insight.statement} {insight.why_it_matters}"
4788     if not EVENT_PROFILE_INSIGHT_PATTERN.search(text):
4789         return True
4790
4791     return bool(EVENT_PROFILE_INSIGHT_PATTERN.search(request))
4792
4793
4794 def scope_fact_ledger_for_genre(
4795     fact_ledger: FactLedger,
4796     evidence: EvidenceLedger,
4797     genre: ReportGenre,
4798 ) -> FactLedger:
4799     evidence_lookup = {item.evidence_id: item for item in evidence.items}
4800     scoped_facts = [
4801         fact
4802         for fact in fact_ledger.writer_ready_facts
4803         if fact_is_relevant_to_genre(
4804             fact,
4805             evidence_lookup,
4806             genre,
4807         )
4808     ]
4809     return fact_ledger.model_copy(update={"writer_ready_facts": scoped_facts})
4810
4811
4812 def build_writer_evidence_pack(
4813     request: str,
4814     understanding: Any,
4815     plan: Any,
4816     evidence: EvidenceLedger,
4817     fact_ledger: FactLedger,
4818     settings: Settings,
4819     insight_ledger: InsightLedger | None = None,
4820     input_structure: InputStructureProfile | None = None,
4821     available_capabilities: list[EvidenceCapability] | None = None,
4822 ) -> WriterEvidencePack:
4823     insight_ledger = insight_ledger or InsightLedger(
4824         synthesis_enabled=False,
4825         fallback_reason="No Insight Ledger was supplied.",
4826     )
4827     evidence_lookup = {item.evidence_id: item for item in evidence.items}
4828     scoped_fact_ledger = scope_fact_ledger_for_genre(
4829         fact_ledger,
4830         evidence,
4831         plan.report_specification.genre,
4832     )
4833     facts = scoped_fact_ledger.writer_ready_facts
4834
4835     event_genre = plan.report_specification.genre in {
4836         ReportGenre.EVENT_REPORT,
4837         ReportGenre.SPORTS_GAME_REPORT,
4838     }
4839     focused_table_task = (
4840         getattr(plan.report_specification, "communication_task", None)
4841         == CommunicationTask.FOCUSED_TABLE_DESCRIPTION
4842     )
4843
4844     if focused_table_task:
4845         focused_facts = [
4846             fact
4847             for fact in facts

```

```

4849         if EvidenceCapability.FOCUSED_TABLE_REGION
4850         in fact.source_capabilities
4851         or any(
4852             evidence_lookup[evidence_id].capability
4853             == EvidenceCapability.FOCUSED_TABLE_REGION
4854             for evidence_id in fact.evidence_ids
4855             if evidence_id in evidence_lookup
4856         )
4857     ]
4858     priority = sorted(
4859         focused_facts,
4860         key=lambda fact: evidence_priority_score_for_fact(
4861             fact,
4862             evidence_lookup,
4863         ),
4864         reverse=True,
4865     )
4866     supporting = []
4867 elif event_genre:
4868     priority, supporting = select_event_priority_facts(
4869         facts=facts,
4870         evidence=evidence,
4871         settings=settings,
4872         request=request,
4873         report_specification=plan.report_specification,
4874     )
4875 else:
4876     priority = select_balanced_priority_facts(
4877         facts=facts,
4878         evidence=evidence,
4879         required_components=(
4880             plan.report_specification.required_components
4881         ),
4882         settings=settings,
4883     )
4884     priority_ids = {
4885         fact.fact_id
4886         for fact in priority
4887     }
4888     ranked_facts = sorted(
4889         facts,
4890         key=lambda fact: evidence_priority_score_for_fact(
4891             fact,
4892             evidence_lookup,
4893         ),
4894         reverse=True,
4895     )
4896     supporting = [
4897         fact
4898         for fact in ranked_facts
4899         if fact.fact_id not in priority_ids
4900         and fact.recommended_use
4901         != RecommendedUse.OMIT_UNLESS_REQUESTED
4902     ]
4903     if settings.writer_supporting_fact_limit is not None:
4904         supporting = supporting[
4905             : settings.writer_supporting_fact_limit
4906         ]
4907
4908     limitations = sorted(
4909         [
4910             fact
4911             for fact in facts
4912             if (
4913                 fact.recommended_use == RecommendedUse.LIMITATION
4914                 or ClaimPermission.INSUFFICIENCY in fact.claim_permissions
4915                 or (fact.required_caveats and not event_genre)
4916             )
4917         ],
4918         key=lambda fact: evidence_priority_score_for_fact(fact, evidence_lookup),
4919         reverse=True,
4920     )
4921
4922     recommendations = sorted(
4923         [
4924             recommendation
4925             for item in evidence.items
4926             for recommendation in item.recommendations
4927             if recommendation.priority in {"high", "medium"}
4928         ],
4929         key=lambda item: (
4930             {"high": 2, "medium": 1, "low": 0}.get(item.priority, 0),
4931             getattr(item, "confidence", 0.5),
4932         ),
4933         reverse=True,
4934     )
4935
4936     prohibited = list(
4937         dict.fromkeys(
4938             interpretation
4939             for fact in facts

```

```

4940         for interpretation in fact.prohibited_interpretations
4941     )
4942 )
4943 scoped_fact_ids = {fact.fact_id for fact in facts}
4944 scoped_insight_ledger = insight_ledger.model_copy(
4945     update={
4946         "verified_insights": [
4947             insight
4948             for insight in insight_ledger.verified_insights
4949             if insight.source_fact_ids
4950             and set(insight.source_fact_ids).issubset(scoped_fact_ids)
4951             and (
4952                 not event_genre
4953                 or event_insight_is_relevant(insight, request)
4954             )
4955         ],
4956         "hypothesis_only_insights": [
4957             insight
4958             for insight in insight_ledger.hypothesis_only_insights
4959             if insight.source_fact_ids
4960             and set(insight.source_fact_ids).issubset(scoped_fact_ids)
4961             and (
4962                 not event_genre
4963                 or event_insight_is_relevant(insight, request)
4964             )
4965         ],
4966     }
4967 )
4968 priority_insights, supporting_insights = select_priority_verified_insights(
4969     insight_ledger=scoped_insight_ledger,
4970     maximum=settings.max_verified_main_insights,
4971 )
4972
4973 return WriterEvidencePack(
4974     user_request=request,
4975     report_specification=plan.report_specification,
4976     dataset_understanding=understanding,
4977     input_structure=input_structure,
4978     available_capabilities=available_capabilities or [],
4979     priority_facts=priority,
4980     supporting_facts=supporting,
4981     limitation_facts=limitations,
4982     evidence_ledger=evidence,
4983     insight_ledger=scoped_insight_ledger,
4984     priority_verified_insights=priority_insights,
4985     supporting_verified_insights=supporting_insights,
4986     analytical_recommendations=recommendations,
4987     reader_facing_limitations=build_reader_facing_limitations(
4988         priority + limitations
4989     ),
4990     internal_prohibited_interpretations=prohibited,
4991 )
4992
4993
4994 def _sentence_section_headings(
4995     markdown: str,
4996 ) -> dict[str, str]:
4997     headings: dict[str, str] = {}
4998     current_heading = ""
4999
5000     for raw_line in markdown.splitlines():
5001         line = raw_line.strip()
5002
5003         if line.startswith("## "):
5004             current_heading = line[3:].strip()
5005             continue
5006
5007         for sentence in split_markdown_sentences(line):
5008             headings[sentence] = current_heading
5009
5010     return headings
5011
5012
5013 def validate_writer_output(
5014     output: WriterOutput,
5015     fact_ledger: FactLedger,
5016     insight_ledger: InsightLedger | None = None,
5017     allow_hypotheses_in_report: bool = False,
5018 ) -> list[str]:
5019     errors: list[str] = []
5020     insight_ledger = insight_ledger or InsightLedger(
5021         synthesis_enabled=False
5022     )
5023     valid_fact_ids = {
5024         fact.fact_id
5025         for fact in fact_ledger.writer_ready_facts
5026     }
5027     fact_lookup = {
5028         fact.fact_id: fact
5029         for fact in fact_ledger.writer_ready_facts
5030     }

```



```

5031     verified_insights = {
5032         insight.insight_id: insight
5033         for insight in insight_ledger.verified_insights
5034     }
5035     hypothesis_insights = {
5036         insight.insight_id: insight
5037         for insight in insight_ledger.hypothesis_only_insights
5038     }
5039     all_insight_ids = {
5040         *verified_insights,
5041         *hypothesis_insights,
5042     }
5043     section_headings = _sentence_section_headings(output.markdown)
5044
5045     unknown_title_fact_ids = set(output.title_fact_ids) - valid_fact_ids
5046     if unknown_title_fact_ids:
5047         errors.append(f"The title cites unknown fact IDs: {sorted(unknown_title_fact_ids)}")
5048     title_facts = [
5049         fact_lookup[fact_id] for fact_id in output.title_fact_ids if fact_id in fact_lookup
5050     ]
5051     all_title_entities = {
5052         entity
5053         for fact in fact_ledger.writer_ready_facts
5054         for entity in fact.entities
5055         if len(entity.strip()) >= 3
5056         and entity.casefold() in output.title.casefold()
5057     }
5058     title_requires_support = bool(
5059         extract_number_tokens(output.title)
5060         or (
5061             all_title_entities
5062             and FACTUAL_TITLE_PATTERN.search(output.title)
5063         )
5064     )
5065     if title_requires_support and not output.title_fact_ids:
5066         errors.append("A factual title must cite supporting fact IDs.")
5067     supported_title_entities = {
5068         entity
5069         for fact in title_facts
5070         for entity in fact.entities
5071     }
5072     unsupported_title_entities = (
5073         all_title_entities - supported_title_entities
5074     )
5075     if title_requires_support and unsupported_title_entities:
5076         errors.append(
5077             "The title contains entities unsupported by its facts: "
5078             f"{sorted(unsupported_title_entities)}"
5079         )
5080     title_support_numbers = [
5081         number
5082         for fact in title_facts
5083         for number in [
5084             *[
5085                 value
5086                 for _, value in extract_number_tokens(
5087                     fact.fact_summary
5088                 )
5089             ],
5090             *flatten_numbers(fact.structured_values),
5091         ]
5092     ]
5093     if title_facts and not numbers_supported(
5094         output.title,
5095         title_support_numbers,
5096     ):
5097         errors.append("The title contains a number unsupported by its facts.")
5098     if CAUSAL_PATTERN.search(output.title) and not any(
5099         ClaimPermission.CAUSAL in fact.claim_permissions for fact in title_facts
5100     ):
5101         errors.append("The title introduces unsupported causal wording.")
5102
5103     if re.search(r"\[(:CLM|FACT)_d+", output.markdown):
5104         errors.append(
5105             "Internal fact or claim IDs must not appear in the visible report."
5106         )
5107
5108     if INTERNAL_CONTROL_PATTERN.search(output.markdown):
5109         errors.append(
5110             "The visible report exposes an internal writer or auditor control."
5111         )
5112
5113     if FIELD_LABEL_PATTERN.search(output.markdown):
5114         errors.append(
5115             "The visible report renders internal evidence fields such as "
5116             "`Finding`, `Strength`, or `Important Note` rather than natural "
5117             "data-science prose."
5118         )
5119
5120     seen_sentence_ids: set[str] = set()
5121     mapped_sentences = {

```

```

5122     support.sentence_text
5123     for support in output.sentence_support
5124 }
5125 factual_sentences = [
5126     sentence
5127     for sentence in split_markdown_sentences(output.markdown)
5128     if looks_factual(sentence)
5129 ]
5130 missing_mappings = [
5131     sentence
5132     for sentence in factual_sentences
5133     if sentence not in mapped_sentences
5134 ]
5135 if missing_mappings:
5136     errors.append(
5137         "Every factual sentence must appear exactly in the hidden sentence support map."
5138     )
5139
5140 for support in output.sentence_support:
5141     if support.sentence_id in seen_sentence_ids:
5142         errors.append(
5143             f"Duplicate sentence ID: {support.sentence_id}"
5144         )
5145     seen_sentence_ids.add(support.sentence_id)
5146
5147     if support.sentence_text not in output.markdown:
5148         errors.append(
5149             f"{support.sentence_id} text does not appear in the report."
5150         )
5151
5152     unknown = set(support.fact_ids) - valid_fact_ids
5153     if unknown:
5154         errors.append(
5155             f"{support.sentence_id} cites unknown fact IDs: {sorted(unknown)}"
5156         )
5157
5158     unknown_insights = (
5159         set(support.insight_ids)
5160         - all_insight_ids
5161     )
5162     if unknown_insights:
5163         errors.append(
5164             f"{support.sentence_id} cites unknown insight IDs: "
5165             f"{sorted(unknown_insights)}"
5166         )
5167
5168     if (
5169         EXPLANATORY_HYPOTHESIS_PATTERN.search(
5170             support.sentence_text
5171         )
5172         and support.interpretation_level
5173         != InterpretationLevel.HYPOTHESIS
5174     ):
5175         errors.append(
5176             f"{support.sentence_id} presents a possible explanation "
5177             "without classifying it as a hypothesis."
5178         )
5179
5180     if (
5181         support.interpretation_level
5182         == InterpretationLevel.BOUNDED_INSIGHT
5183     ):
5184         if not support.insight_ids:
5185             errors.append(
5186                 f"{support.sentence_id} is a bounded insight but has no "
5187                 "insight ID."
5188             )
5189         elif not set(support.insight_ids).issubset(
5190             set(verified_insights)
5191         ):
5192             errors.append(
5193                 f"{support.sentence_id} cites an insight that is not a "
5194                 "verified main insight."
5195             )
5196     else:
5197         required_fact_ids = {
5198             fact_id
5199             for insight_id in support.insight_ids
5200             for fact_id in verified_insights[
5201                 insight_id
5202             ].source_fact_ids
5203         }
5204         required_evidence_ids = {
5205             evidence_id
5206             for insight_id in support.insight_ids
5207             for evidence_id in verified_insights[
5208                 insight_id
5209             ].source_evidence_ids
5210         }
5211         if not required_fact_ids.issubset(
5212             set(support.fact_ids)

```

```

5213         ):
5214             errors.append(
5215                 f"{support.sentence_id} omits source facts for its "
5216                 "verified insight."
5217             )
5218         if not required_evidence_ids.issubset(
5219             set(support.evidence_ids)
5220         ):
5221             errors.append(
5222                 f"{support.sentence_id} omits source evidence for its "
5223                 "verified insight."
5224             )
5225
5226     if (
5227         support.interpretation_level
5228         == InterpretationLevel.HYPOTHESIS
5229     ):
5230         if not allow_hypotheses_in_report:
5231             errors.append(
5232                 f"{support.sentence_id} is a hypothesis while hypotheses "
5233                 "are disabled."
5234             )
5235         if not support.insight_ids or not set(
5236             support.insight_ids
5237         ).issubset(set(hypothesis_insights)):
5238             errors.append(
5239                 f"{support.sentence_id} lacks valid hypothesis-only "
5240                 "provenance."
5241             )
5242         if section_headings.get(support.sentence_text, "").lower() != (
5243             "questions for further investigation"
5244         ):
5245             errors.append(
5246                 f"{support.sentence_id} places a hypothesis outside the "
5247                 "allowed section."
5248             )
5249         if (
5250             not HYPOTHESIS_WORDING_PATTERN.search(
5251                 support.sentence_text
5252             )
5253             and not support.sentence_text.strip().endswith("?")
5254         ):
5255             errors.append(
5256                 f"{support.sentence_id} does not explicitly label the "
5257                 "hypothesis or frame it as a question."
5258             )
5259
5260     if (
5261         support.interpretation_level
5262         == InterpretationLevel.FINDING
5263         and support.insight_ids
5264     ):
5265         errors.append(
5266             f"{support.sentence_id} is a finding but cites insight IDs."
5267         )
5268
5269     if (
5270         support.support_type != SupportType.NON_FACTUAL
5271         and not support.fact_ids
5272         and not support.profile_support_ids
5273         and not support.insight_ids
5274     ):
5275         errors.append(
5276             f"{support.sentence_id} is factual but has no supporting "
5277             "provenance."
5278         )
5279
5280     supporting_facts = [
5281         fact_lookup[fact_id]
5282         for fact_id in support.fact_ids
5283         if fact_id in fact_lookup
5284     ]
5285     support_numbers = [
5286         number
5287         for fact in supporting_facts
5288         for number in [
5289             *[
5290                 value
5291                 for _, value in extract_number_tokens(
5292                     fact.fact_summary
5293                 )
5294             ],
5295             *flatten_numbers(fact.structured_values),
5296         ]
5297     ]
5298     if (
5299         supporting_facts
5300         and not numbers_supported(
5301             support.sentence_text,
5302             support_numbers,
5303         )

```

```

5304         ):
5305             errors.append(
5306                 f"{support.sentence_id} contains a number unsupported by its "
5307                 "mapped facts."
5308             )
5309
5310         known_entities = {
5311             entity
5312             for fact in supporting_facts
5313             for entity in fact.entities
5314         }
5315         unsupported_entities = unsupported_backtick_entities(
5316             support.sentence_text,
5317             known_entities,
5318         )
5319         if supporting_facts and unsupported_entities:
5320             errors.append(
5321                 f"{support.sentence_id} contains unsupported entities "
5322                 f"{unsupported_entities}; mapped fact IDs: "
5323                 f"{support.fact_ids}."
5324             )
5325
5326         declared = set(output.selected_fact_ids)
5327         used = set(output.title_fact_ids)
5328         used.update(
5329             fact_id
5330             for support in output.sentence_support
5331             for fact_id in support.fact_ids
5332         )
5333
5334         if declared != used:
5335             errors.append(
5336                 "selected_fact_ids must match the facts used in sentence_support."
5337             )
5338
5339         return errors
5340
5341     def apply_support_map_patches(
5342         writer_output: WriterOutput,
5343         patches: list[SupportMapPatch],
5344         valid_profile_support_ids: set[str],
5345     ) -> WriterOutput:
5346         original_markdown = writer_output.markdown
5347         support_map = [
5348             support.model_copy()
5349             for support in writer_output.sentence_support
5350         ]
5351         support_by_id = {
5352             support.sentence_id: support
5353             for support in support_map
5354         }
5355
5356         for patch in patches:
5357             unknown = [
5358                 support_id
5359                 for support_id in patch.added_profile_support_ids
5360                 if support_id not in valid_profile_support_ids
5361             ]
5362
5363             if unknown:
5364                 raise ValueError(
5365                     "Support-map patch references unknown profile support "
5366                     f"IDs: {unknown}"
5367                 )
5368
5369             support = support_by_id.get(patch.sentence_id)
5370
5371             if support is None:
5372                 raise ValueError(
5373                     "Support-map patch references unknown sentence ID: "
5374                     f"{patch.sentence_id}"
5375                 )
5376
5377             if support.sentence_text != patch.sentence_text:
5378                 raise ValueError(
5379                     "Support-map patch sentence text does not exactly match "
5380                     "the hidden support entry."
5381                 )
5382
5383             merged_profile_ids = list(
5384                 dict.fromkeys(
5385                     [
5386                         *support.profile_support_ids,
5387                         *patch.added_profile_support_ids,
5388                     ]
5389                 )
5390             )
5391
5392             replacement = support.model_copy(
5393                 update={

```

```

5395         "profile_support_ids": merged_profile_ids
5396     }
5397 )
5398 support_index = support_map.index(support)
5399 support_map[support_index] = replacement
5400 support_by_id[patch.sentence_id] = replacement
5401
5402 patched = writer_output.model_copy(
5403     update={
5404         "sentence_support": support_map
5405     }
5406 )
5407
5408 assert patched.markdown == original_markdown
5409 return patched
5410
5411
5412 def materialise_writer_output(
5413     draft: WriterAgentDraft,
5414     fact_ledger: FactLedger,
5415     *,
5416     insight_ledger: InsightLedger | None = None,
5417     allow_hypotheses_in_report: bool = False,
5418     content_requirements: dict[str, Any] | None = None,
5419     writer_mode: str = "llm_writer",
5420     eligible_for_primary_evaluation: bool = True,
5421     quality_revision_round: int = 0,
5422     quality_revision_summary: str | None = None,
5423 ) -> WriterOutput:
5424     content_requirements = content_requirements or {}
5425     insight_ledger = insight_ledger or InsightLedger(
5426         synthesis_enabled=False
5427     )
5428     fact_lookup = {
5429         fact.fact_id: fact
5430         for fact in fact_ledger.writer_ready_facts
5431     }
5432     verified_insight_lookup = {
5433         insight.insight_id: insight
5434         for insight in insight_ledger.verified_insights
5435     }
5436     hypothesis_insight_lookup = {
5437         insight.insight_id: insight
5438         for insight in insight_ledger.hypothesis_only_insights
5439     }
5440     all_insight_lookup = {
5441         **verified_insight_lookup,
5442         **hypothesis_insight_lookup,
5443     }
5444     output_form = content_requirements.get("output_form")
5445     short_form_without_headings = bool(
5446         content_requirements.get("allow_headings") is False
5447         or output_form
5448         in {
5449             OutputForm.ONE_SENTENCE.value,
5450             OutputForm.DIRECT_ANSWER.value,
5451             OutputForm.SHORT_TEXT.value,
5452         }
5453     )
5454     unknown_title_fact_ids = [
5455         fact_id for fact_id in draft.title_fact_ids if fact_id not in fact_lookup
5456     ]
5457     if unknown_title_fact_ids and not short_form_without_headings:
5458         raise ValueError(f"Writer draft title contains unknown fact IDs: {unknown_title_fact_ids}")
5459
5460     lines: list[str] = (
5461         []
5462         if short_form_without_headings
5463         else [
5464             f"# {draft.title.strip()}",
5465             "",
5466         ]
5467     )
5468     max_rendered_sentences = (
5469         int(content_requirements["max_sentences"])
5470         if short_form_without_headings
5471         and content_requirements.get("max_sentences") is not None
5472         else None
5473     )
5474
5475     sentence_support: list[SentenceSupport] = []
5476     selected_fact_ids: list[str] = (
5477         []
5478         if short_form_without_headings
5479         else list(dict.fromkeys(draft.title_fact_ids))
5480     )
5481     sentence_number = 1
5482
5483     for section in draft.sections:
5484         heading = section.heading.strip()
5485         heading = re.sub(

```

```

5486         r"^\#\s*",
5487         """
5488         heading,
5489     )
5490
5491     if not heading:
5492         continue
5493
5494     if (
5495         short_form_without_headings
5496         and max_rendered_sentences is not None
5497         and sentence_number > max_rendered_sentences
5498     ):
5499         break
5500
5501     if not short_form_without_headings:
5502         lines.extend(
5503             [
5504                 f"## {heading}",
5505                 """
5506             ]
5507         )
5508
5509     for sentence_draft in section.sentences:
5510         if (
5511             short_form_without_headings
5512             and max_rendered_sentences is not None
5513             and sentence_number > max_rendered_sentences
5514         ):
5515             break
5516
5517         sentence_text = re.sub(
5518             r"\s+",
5519             " ",
5520             sentence_draft.text,
5521         ).strip()
5522
5523         if not sentence_text:
5524             continue
5525
5526         if sentence_text[-1] not in ".!?":
5527             sentence_text += "."
5528
5529         unknown_fact_ids = [
5530             fact_id
5531             for fact_id in sentence_draft.fact_ids
5532             if fact_id not in fact_lookup
5533         ]
5534
5535         if unknown_fact_ids:
5536             raise ValueError(
5537                 "Writer draft contains unknown "
5538                 f"fact IDs: {unknown_fact_ids}"
5539             )
5540
5541         unknown_insight_ids = [
5542             insight_id
5543             for insight_id in sentence_draft.insight_ids
5544             if insight_id not in all_insight_lookup
5545         ]
5546         if unknown_insight_ids:
5547             raise ValueError(
5548                 "Writer draft contains unknown insight IDs: "
5549                 f"{unknown_insight_ids}"
5550             )
5551
5552         if (
5553             sentence_draft.interpretation_level
5554             == InterpretationLevel.BOUNDED_INSIGHT
5555         ):
5556             if not sentence_draft.insight_ids:
5557                 raise ValueError(
5558                     "A bounded-insight Writer sentence has no verified "
5559                     "insight ID."
5560                 )
5561             if not set(sentence_draft.insight_ids).issubset(
5562                 set(verified_insight_lookup)
5563             ):
5564                 raise ValueError(
5565                     "A bounded-insight Writer sentence cites an insight "
5566                     "that is not verified for the main report."
5567                 )
5568
5569         if (
5570             sentence_draft.interpretation_level
5571             == InterpretationLevel.HYPOTHESIS
5572         ):
5573             if not allow_hypotheses_in_report:
5574                 raise ValueError(
5575                     "Writer hypotheses are disabled by configuration."
5576                 )

```

```

5577         if not sentence_draft.insight_ids or not set(
5578             sentence_draft.insight_ids
5579         ).issubset(set(hypothesis_insight_lookup)):
5580             raise ValueError(
5581                 "A hypothesis Writer sentence must cite a valid "
5582                 "hypothesis-only insight."
5583             )
5584
5585     if (
5586         sentence_draft.interpretation_level
5587         == InterpretationLevel.FINDING
5588         and sentence_draft.insight_ids
5589     ):
5590         raise ValueError(
5591             "A direct Writer finding must not cite insight IDs."
5592         )
5593
5594     expanded_fact_ids = list(
5595         dict.fromkeys(
5596             [
5597                 *sentence_draft.fact_ids,
5598                 *[
5599                     fact_id
5600                     for insight_id in sentence_draft.insight_ids
5601                     for fact_id in all_insight_lookup[
5602                         insight_id
5603                     ].source_fact_ids
5604                     if fact_id in fact_lookup
5605                 ],
5606             ],
5607         )
5608     )
5609
5610     if (
5611         sentence_draft.support_type
5612         != SupportType.NON_FACTUAL
5613         and not expanded_fact_ids
5614     ):
5615         raise ValueError(
5616             "A factual Writer sentence has no "
5617             "supporting fact IDs."
5618         )
5619
5620     evidence_ids = list(
5621         dict.fromkeys(
5622             [
5623                 *[
5624                     evidence_id
5625                     for fact_id in expanded_fact_ids
5626                     for evidence_id in fact_lookup[
5627                         fact_id
5628                     ].evidence_ids
5629                 ],
5630                 *[
5631                     evidence_id
5632                     for insight_id in sentence_draft.insight_ids
5633                     for evidence_id in all_insight_lookup[
5634                         insight_id
5635                     ].source_evidence_ids
5636                 ],
5637             ],
5638         )
5639     )
5640
5641     rendered_sentences = (
5642         split_markdown_sentences(sentence_text)
5643         or [sentence_text]
5644     )
5645
5646     for rendered_sentence in rendered_sentences:
5647         if (
5648             short_form_without_headings
5649             and max_rendered_sentences is not None
5650             and sentence_number > max_rendered_sentences
5651         ):
5652             break
5653
5654         lines.append(rendered_sentence)
5655
5656         sentence_support.append(
5657             SentenceSupport(
5658                 sentence_id=(
5659                     f"SENT_{sentence_number:04d}"
5660                 ),
5661                 sentence_text=rendered_sentence,
5662                 fact_ids=expanded_fact_ids,
5663                 evidence_ids=evidence_ids,
5664                 insight_ids=list(
5665                     dict.fromkeys(
5666                         sentence_draft.insight_ids

```

```

5668         ),
5669         interpretation_level=(
5670             sentence_draft.interpretation_level
5671         ),
5672         support_type=(
5673             sentence_draft.support_type
5674         ),
5675     )
5676 )
5677
5678     selected_fact_ids.extend(
5679         expanded_fact_ids
5680     )
5681
5682     sentence_number += 1
5683
5684     if not short_form_without_headings:
5685         lines.append("")
5686
5687     selected_fact_ids = list(
5688         dict.fromkeys(selected_fact_ids)
5689     )
5690
5691     all_fact_ids = [
5692         fact.fact_id
5693         for fact in fact_ledger.writer_ready_facts
5694     ]
5695
5696     output = WriterOutput(
5697         title=(
5698             "Focused table description"
5699             if short_form_without_headings
5700             else draft.title.strip()
5701         ),
5702         title_fact_ids=(
5703             []
5704             if short_form_without_headings
5705             else list(dict.fromkeys(draft.title_fact_ids))
5706         ),
5707         markdown=(
5708             "\n".join(lines).strip()
5709             + "\n"
5710         ),
5711         sentence_support=sentence_support,
5712         selected_fact_ids=selected_fact_ids,
5713         omitted_fact_ids=[
5714             fact_id
5715             for fact_id in all_fact_ids
5716             if fact_id not in selected_fact_ids
5717         ],
5718         writer_notes=draft.writer_notes,
5719         writer_mode=writer_mode,
5720         eligible_for_primary_evaluation=(
5721             eligible_for_primary_evaluation
5722         ),
5723         quality_revision_round=(
5724             quality_revision_round
5725         ),
5726         quality_revision_summary=(
5727             quality_revision_summary
5728         ),
5729     )
5730
5731     errors = validate_writer_output(
5732         output,
5733         fact_ledger,
5734         insight_ledger,
5735         allow_hypotheses_in_report,
5736     )
5737     errors.extend(
5738         writer_output_content_requirement_errors(
5739             writer_output=output,
5740             requirements=content_requirements,
5741             include_word_count=True,
5742         )
5743     )
5744
5745     if errors:
5746         raise ValueError(
5747             "Controller-generated WriterOutput "
5748             "failed validation:\n- "
5749             + "\n- ".join(errors)
5750         )
5751
5752     return output
5753
5754
5755 def writer_output_word_count(
5756     output: WriterOutput,
5757 ) -> int:

```



```

5759     return len(
5760         re.findall(
5761             r"\b[\w'-]+\b",
5762             output.markdown,
5763         )
5764     )
5765
5766
5767 def accept_writer_quality_revision(
5768     *,
5769     before: WriterOutput,
5770     after: WriterOutput,
5771     before_audit: AuditReport,
5772     after_audit: AuditReport,
5773     validation_errors: list[str],
5774     report_specification: Any,
5775     settings: Settings,
5776 ) -> tuple[bool, list[str]]:
5777     """
5778     Accept a whole-report quality revision only when it is valid and
5779     measurably improves the incomplete draft.
5780     """
5781
5782     reasons: list[str] = list(
5783         validation_errors
5784     )
5785
5786     before_missing = sum(
5787         not assessment.covered
5788         for assessment
5789         in before_audit.component_assessments
5790     )
5791
5792     after_missing = sum(
5793         not assessment.covered
5794         for assessment
5795         in after_audit.component_assessments
5796     )
5797
5798     before_words = writer_output_word_count(
5799         before
5800     )
5801     after_words = writer_output_word_count(
5802         after
5803     )
5804
5805     minimum_words = minimum_useful_report_words(
5806         target_words=(
5807             report_specification
5808             .target_length_words
5809         ),
5810         required_component_count=len(
5811             report_specification
5812             .required_components
5813         ),
5814         settings=settings,
5815     )
5816     maximum_words = report_specification.maximum_length_words
5817
5818     if maximum_words is not None and after_words > maximum_words:
5819         reasons.append(
5820             "The revision exceeds the configured report word ceiling."
5821         )
5822
5823     if (
5824         before_missing > 0
5825         and after_missing >= before_missing
5826     ):
5827         reasons.append(
5828             "The revision did not reduce the number "
5829             "of missing required components."
5830         )
5831
5832     if (
5833         before_words < minimum_words
5834         and after_words <= before_words
5835     ):
5836         reasons.append(
5837             "The revision did not improve the "
5838             "under-length report."
5839         )
5840
5841     if (
5842         before_missing > 0
5843         and len(after.sentence_support)
5844         < len(before.sentence_support)
5845     ):
5846         reasons.append(
5847             "The revision reduced supported sentence "
5848             "coverage while the report was incomplete."
5849         )

```

```

5850
5851     serious_after = {
5852         (
5853             annotation.sentence,
5854             annotation.subtype,
5855         )
5856         for annotation
5857         in after_audit.annotations
5858         if annotation.severity
5859         in {
5860             Severity.HIGH,
5861             Severity.CRITICAL,
5862         }
5863     }
5864
5865     serious_before = {
5866         (
5867             annotation.sentence,
5868             annotation.subtype,
5869         )
5870         for annotation
5871         in before_audit.annotations
5872         if annotation.severity
5873         in {
5874             Severity.HIGH,
5875             Severity.CRITICAL,
5876         }
5877     }
5878
5879     introduced_serious = (
5880         serious_after - serious_before
5881     )
5882
5883     if introduced_serious:
5884         reasons.append(
5885             "The revision introduced a new serious "
5886             "factual or contextual annotation."
5887         )
5888
5889     return (
5890         not reasons,
5891         list(dict.fromkeys(reasons)),
5892     )
5893
5894
5895 def _focused_table_fallback_writer(
5896     pack: WriterEvidencePack,
5897     evidence_by_id: dict[str, EvidenceItem],
5898 ) -> WriterOutput | None:
5899     available_facts = list(
5900         {
5901             fact.fact_id: fact
5902             for fact in (
5903                 pack.priority_facts
5904                 + pack.supporting_facts
5905                 + pack.limitation_facts
5906             )
5907         }.values()
5908     )
5909     fact_lookup = {
5910         fact.fact_id: fact
5911         for fact in available_facts
5912     }
5913     focused_insight = next(
5914         (
5915             insight
5916             for insight in [
5917                 *pack.priority_verified_insights,
5918                 *pack.supporting_verified_insights,
5919             ]
5920             if any(
5921                 evidence_by_id[evidence_id].capability
5922                 == EvidenceCapability.FOCUSED_TABLE_REGION
5923                 for evidence_id in insight.source_evidence_ids
5924                 if evidence_id in evidence_by_id
5925             )
5926         ),
5927         None,
5928     )
5929     if focused_insight is not None:
5930         sentence = re.sub(
5931             r"\s+",
5932             " ",
5933             focused_insight.statement,
5934         ).strip()
5935         if sentence and sentence[-1] not in ".!?:":
5936             sentence += "."
5937         fact_ids = [
5938             fact_id
5939             for fact_id in focused_insight.source_fact_ids
5940             if fact_id in fact_lookup

```

```

5941     ]
5942     evidence_ids = list(focused_insight.source_evidence_ids)
5943     support = SentenceSupport(
5944         sentence_id="SENT_0001",
5945         sentence_text=sentence,
5946         fact_ids=fact_ids,
5947         evidence_ids=evidence_ids,
5948         insight_ids=[focused_insight.insight_id],
5949         interpretation_level=InterpretationLevel.BOUNDED_INSIGHT,
5950         support_type=SupportType.MULTI_FACT_SYNTHESIS,
5951     )
5952     return WriterOutput(
5953         title="Focused table description",
5954         markdown=sentence + "\n",
5955         sentence_support=[support],
5956         selected_fact_ids=fact_ids,
5957         omitted_fact_ids=[
5958             fact.fact_id
5959             for fact in available_facts
5960             if fact.fact_id not in set(fact_ids)
5961         ],
5962         writer_notes=[
5963             "Deterministic short-form writer used a verified "
5964             "focused-table insight.",
5965             "This requested short-form verbalisation is eligible for "
5966             "primary evaluation because it is directly support-mapped.",
5967         ],
5968         writer_mode="deterministic_short_form_writer",
5969         eligible_for_primary_evaluation=True,
5970     )
5971
5972     focused_fact = next(
5973         (
5974             fact
5975             for fact in available_facts
5976             if any(
5977                 evidence_by_id[evidence_id].capability
5978                 == EvidenceCapability.FOCUSED_TABLE_REGION
5979                 for evidence_id in fact.evidence_ids
5980                 if evidence_id in evidence_by_id
5981             )
5982         ),
5983         None,
5984     )
5985     if focused_fact is None:
5986         return None
5987
5988     evidence_item = next(
5989         (
5990             evidence_by_id[evidence_id]
5991             for evidence_id in focused_fact.evidence_ids
5992             if evidence_id in evidence_by_id
5993             and evidence_by_id[evidence_id].capability
5994             == EvidenceCapability.FOCUSED_TABLE_REGION
5995         ),
5996         None,
5997     )
5998     metrics = evidence_item.metrics if evidence_item is not None else {}
5999
6000     values = [
6001         str(value).strip()
6002         for value in metrics.get("highlighted_values", [])
6003         if str(value).strip()
6004     ]
6005     row_context = [
6006         str(value).strip()
6007         for value in metrics.get("row_context", [])
6008         if str(value).strip()
6009     ]
6010     header_context = [
6011         str(value).strip()
6012         for value in metrics.get("header_context", [])
6013         if str(value).strip()
6014     ]
6015     page_title = str(metrics.get("page_title") or "").strip()
6016     section_title = str(metrics.get("section_title") or "").strip()
6017     proposition = str(metrics.get("description_proposition") or "").strip()
6018     local_contrast = _focused_table_local_contrast_summary(metrics)
6019     record_relation = metrics.get("focused_record_relation")
6020     record_summary = (
6021         str(record_relation.get("relation_summary") or "").strip()
6022         if isinstance(record_relation, dict)
6023         else ""
6024     )
6025     list_relation = metrics.get("focused_list_relation")
6026     list_summary = (
6027         str(list_relation.get("relation_summary") or "").strip()
6028         if isinstance(list_relation, dict)
6029         else ""
6030     )
6031     record_group_summary = str(

```

```

6032         metrics.get("highlighted_record_group_summary") or ""
6033     ).strip()
6034
6035     value_text = ", ".join(values)
6036     sentence = focused_fact.fact_summary
6037     if local_contrast:
6038         sentence = local_contrast
6039     elif record_summary:
6040         sentence = record_summary
6041     elif list_summary:
6042         sentence = list_summary
6043     elif record_group_summary:
6044         sentence = record_group_summary
6045     elif proposition:
6046         sentence = proposition
6047     elif value_text:
6048         if row_context and header_context:
6049             sentence = (
6050                 f"The selected table value is {value_text} under the "
6051                 f"{header_context[0]} header in the row containing "
6052                 f"{row_context[0]}"
6053             )
6054         elif row_context:
6055             sentence = (
6056                 f"The selected table value is {value_text} in the row "
6057                 f"containing {row_context[0]}"
6058             )
6059         else:
6060             sentence = f"The selected table cell value is {value_text}"
6061
6062     context_parts = [
6063         part
6064         for part in [section_title, page_title]
6065         if part
6066     ]
6067     if context_parts:
6068         sentence += " in " + " / ".join(context_parts)
6069
6070     sentence = re.sub(r"\s+", " ", sentence).strip()
6071     if sentence and sentence[-1] not in ".!?:":
6072         sentence += "."
6073
6074     support = SentenceSupport(
6075         sentence_id="SENT_0001",
6076         sentence_text=sentence,
6077         fact_ids=[focused_fact.fact_id],
6078         evidence_ids=list(focused_fact.evidence_ids),
6079         support_type=SupportType.DIRECT,
6080     )
6081
6082     return WriterOutput(
6083         title="Focused table description",
6084         markdown=sentence + "\n",
6085         sentence_support=[support],
6086         selected_fact_ids=[focused_fact.fact_id],
6087         omitted_fact_ids=[
6088             fact.fact_id
6089             for fact in available_facts
6090             if fact.fact_id != focused_fact.fact_id
6091         ],
6092         writer_notes=[
6093             "Deterministic short-form writer used verified focused-table "
6094             "evidence.",
6095             "This requested short-form verbalisation is eligible for "
6096             "primary evaluation because it is directly support-mapped.",
6097         ],
6098         writer_mode="deterministic_short_form_writer",
6099         eligible_for_primary_evaluation=True,
6100     )
6101
6102 def _humanise_record_key(value: str) -> str:
6103     text = re.sub(r"(<=[a-z])(<=[A-Z])", " ", value)
6104     text = text.replace("-", " ").replace("_", " ")
6105     return re.sub(r"\s+", " ", text).strip()
6106
6107
6108 def _compact_record_value(value: Any) -> str:
6109     text = str(value or "").strip()
6110     text = text.replace(" ", "")
6111     text = re.sub(r"\\([^\]]+)\)", r"\1", text)
6112     return re.sub(r"\s+", " ", text).strip()
6113
6114
6115 def _record_subject_text(
6116     value: Any,
6117     *,
6118     realisation_policy: RealisationPolicy,
6119 ) -> str:
6120     text = str(value or "").strip()
6121     if realisation_policy == RealisationPolicy.NATURAL_REFERENCE_STYLE:

```

```

6123     text = text.replace("-", " ")
6124     return re.sub(r"\s+", " ", text).strip()
6125
6126
6127 def _ordinal_text(value: Any) -> str:
6128     text = str(value or "").strip()
6129     try:
6130         number = int(float(text))
6131     except ValueError:
6132         return text
6133     suffix = "th"
6134     if number % 100 not in {11, 12, 13}:
6135         suffix = {1: "st", 2: "nd", 3: "rd"}.get(number % 10, "th")
6136     return f"{number}{suffix}"
6137
6138
6139 def _attribute_sentence(
6140     records: list[dict[str, Any]],
6141 ) -> str:
6142     attributes = [
6143         (
6144             _humanise_record_key(str(record.get("attribute_name") or "")),
6145             str(record.get("attribute_value") or "").strip(),
6146         )
6147         for record in records
6148         if str(record.get("record_kind") or "") != "triple"
6149         and str(record.get("attribute_name") or "").strip()
6150         and str(record.get("attribute_value") or "").strip()
6151     ]
6152     if not attributes:
6153         return ""
6154
6155     by_key = {
6156         re.sub(r"^[^a-z0-9]+", "", key.casefold()): (key, value)
6157         for key, value in attributes
6158     }
6159     subject = next(
6160         (
6161             value
6162             for key in ["name", "title", "label"]
6163             if key in by_key
6164             for _, value in [by_key[key]]
6165         ),
6166         None,
6167     )
6168     if subject is None:
6169         subject = attributes[0][1]
6170
6171     used_keys: set[str] = set()
6172     clauses: list[str] = []
6173     type_value = next(
6174         (
6175             value
6176             for key in ["eatype", "type", "category"]
6177             if key in by_key
6178             for _, value in [by_key[key]]
6179         ),
6180         None,
6181     )
6182     if type_value:
6183         article = "an" if type_value[1].casefold() in {"a", "e", "i", "o", "u"} else "a"
6184         clauses.append(f"is {article} {type_value}")
6185         used_keys.update({"eatype", "type", "category"})
6186
6187     relation_clauses: list[str] = []
6188     for normalised_key, (key, value) in by_key.items():
6189         if normalised_key in used_keys or normalised_key in {"name", "title", "label"}:
6190             continue
6191         if normalised_key == "near":
6192             relation_clauses.append(f"near {value}")
6193         elif normalised_key == "area":
6194             relation_clauses.append(f"in the {value} area")
6195         elif normalised_key == "food":
6196             relation_clauses.append(f"serves {value} food")
6197         elif normalised_key in {"customerrating", "rating"}:
6198             relation_clauses.append(f"has a {key} of {value}")
6199         elif normalised_key == "pricerange":
6200             relation_clauses.append(f"has a {key} of {value}")
6201         elif normalised_key == "rank":
6202             relation_clauses.append(f"ranks {_ordinal_text(value)}")
6203         elif normalised_key == "total":
6204             relation_clauses.append(f"has a total of {value}")
6205         elif normalised_key == "familyfriendly":
6206             if value.casefold() in {"yes", "true", "1"}:
6207                 relation_clauses.append("is family friendly")
6208             elif value.casefold() in {"no", "false", "0"}:
6209                 relation_clauses.append("is not family friendly")
6210             else:
6211                 relation_clauses.append(f"has {key} {value}")
6212         else:
6213             relation_clauses.append(f"has {key} {value}")

```

```

6214
6215 if clauses:
6216     sentence = f"{subject} " + " and ".join(clauses)
6217     if relation_clauses:
6218         prepositional = [
6219             clause
6220             for clause in relation_clauses
6221             if clause.startswith(("near ", "in ", "at ", "on "))
6222         ]
6223         predicates = [
6224             clause
6225             for clause in relation_clauses
6226             if clause not in prepositional
6227         ]
6228         if prepositional:
6229             sentence += " " + " and ".join(prepositional)
6230         if predicates:
6231             sentence += " and " + " and ".join(predicates)
6232 else:
6233     details = [
6234         f"{key} {value}"
6235         for key, value in attributes
6236         if value != subject
6237     ]
6238     sentence = (
6239         f"{subject} has " + ", ".join(details)
6240         if details
6241         else str(subject)
6242     )
6243
6244     return sentence.strip()
6245
6246 def _triple_relation_clause(relation: str, obj: str) -> str:
6247     normalised = re.sub(r"^[a-z0-9]+", "", relation.casefold())
6248     value = _compact_record_value(obj)
6249     if normalised in {"engine", "enginetype"}:
6250         if (
6251             value
6252             and not value.isupper()
6253             and not re.search(r"\b[A-Z]{2,}\b", value)
6254         ):
6255             value = value[:1].lower() + value[1:]
6256         if value.casefold().endswith("engine"):
6257             return f"has a {value}"
6258         article = "an" if value[:1].casefold() in {"a", "e", "i", "o", "u"} else "a"
6259         return f"has {article} {value} engine"
6260     if normalised in {"cylindercount", "cylinders", "numberofcylinders"}:
6261         return f"has {value} cylinders"
6262     if normalised in {"length", "height", "width", "diameter"}:
6263         return f"has a {relation} of {value}"
6264     if normalised == "rank":
6265         return f"ranks {_ordinal_text(value)}"
6266     if normalised == "total":
6267         return f"has a total of {value}"
6268     return f"has {_humanise_record_key(relation)} {value}"
6269
6270
6271 def _triple_sentence(
6272     records: list[dict[str, Any]],
6273     *,
6274     realisation_policy: RealisationPolicy = (
6275         RealisationPolicy.STRICT_SOURCE_SURFACE
6276     ),
6277 ) -> str:
6278     triples = [
6279         (
6280             _record_subject_text(
6281                 record.get("subject"),
6282                 realisation_policy=realisation_policy,
6283             ),
6284             _humanise_record_key(str(record.get("relation") or "")),
6285             str(record.get("object") or "").strip(),
6286         )
6287     ]
6288     for record in records
6289     if str(record.get("record_kind") or "") == "triple"
6290     and str(record.get("subject") or "").strip()
6291     and str(record.get("relation") or "").strip()
6292     and str(record.get("object") or "").strip()
6293 ]
6294 if not triples:
6295     return ""
6296
6297 grouped: dict[str, list[tuple[str, str]]] = {}
6298 for subject, relation, obj in triples:
6299     grouped.setdefault(subject, []).append((relation, obj))
6300
6301 sentences: list[str] = []
6302 for subject, relations in grouped.items():
6303     relation_text = ", ".join(
6304         _triple_relation_clause(relation, obj)

```

```

6305         for relation, obj in relations
6306     )
6307     if len(relations) > 1 and "," in relation_text:
6308         head, _, tail = relation_text.rpartition(", ")
6309         relation_text = f"{head}, and {tail}"
6310     sentences.append(f"{subject} {relation_text}")
6311     return "; ".join(sentences)
6312
6313
6314 def _structured_record_fallback_writer(
6315     pack: WriterEvidencePack,
6316     evidence_by_id: dict[str, EvidenceItem],
6317 ) -> WriterOutput | None:
6318     realisation_policy = getattr(
6319         pack.report_specification,
6320         "realisation_policy",
6321         RealisationPolicy.STRICT_SOURCE_SURFACE,
6322     )
6323     if not isinstance(realisation_policy, RealisationPolicy):
6324         try:
6325             realisation_policy = RealisationPolicy(str(realisation_policy))
6326         except ValueError:
6327             realisation_policy = RealisationPolicy.STRICT_SOURCE_SURFACE
6328     available_facts = list(
6329         {
6330             fact.fact_id: fact
6331             for fact in (
6332                 pack.priority_facts
6333                 + pack.supporting_facts
6334                 + pack.limitation_facts
6335             )
6336         }.values()
6337     )
6338     fact_lookup = {
6339         fact.fact_id: fact
6340         for fact in available_facts
6341     }
6342     structured_insight = next(
6343         (
6344             insight
6345             for insight in [
6346                 *pack.priority_verified_insights,
6347                 *pack.supporting_verified_insights,
6348             ]
6349             if any(
6350                 evidence_by_id[evidence_id].capability
6351                 == EvidenceCapability.STRUCTURED_RECORD_VERBALISATION
6352                 for evidence_id in insight.source_evidence_ids
6353                 if evidence_id in evidence_by_id
6354             )
6355         ),
6356         None,
6357     )
6358     if structured_insight is not None:
6359         sentence = re.sub(r"\s+", " ", structured_insight.statement).strip()
6360         if sentence and sentence[-1] not in ".!?:":
6361             sentence += "."
6362         fact_ids = [
6363             fact_id
6364             for fact_id in structured_insight.source_fact_ids
6365             if fact_id in fact_lookup
6366         ]
6367         evidence_ids = list(structured_insight.source_evidence_ids)
6368         return WriterOutput(
6369             title="Structured record verbalisation",
6370             markdown=sentence + "\n",
6371             sentence_support=[
6372                 SentenceSupport(
6373                     sentence_id="SENT_0001",
6374                     sentence_text=sentence,
6375                     fact_ids=fact_ids,
6376                     evidence_ids=evidence_ids,
6377                     insight_ids=[structured_insight.insight_id],
6378                     interpretation_level=InterpretationLevel.BOUNDED_INSIGHT,
6379                     support_type=SupportType.MULTI_FACT_SYNTHESIS,
6380                 )
6381             ],
6382             selected_fact_ids=fact_ids,
6383             omitted_fact_ids=[
6384                 fact.fact_id
6385                 for fact in available_facts
6386                 if fact.fact_id not in set(fact_ids)
6387             ],
6388             writer_notes=[
6389                 "Deterministic short-form writer used a verified "
6390                 "structured-record insight.",
6391                 "This requested short-form verbalisation is eligible for "
6392                 "primary evaluation because it is directly support-mapped.",
6393             ],
6394             writer_mode="deterministic_short_form_writer",
6395             eligible_for_primary_evaluation=True,

```

```

6396     )
6397
6398     structured_fact = next(
6399         (
6400             fact
6401             for fact in available_facts
6402             if any(
6403                 evidence_by_id[evidence_id].capability
6404                 == EvidenceCapability.STRUCTURED_RECORD_VERBALISATION
6405                 for evidence_id in fact.evidence_ids
6406                 if evidence_id in evidence_by_id
6407             )
6408         ),
6409         None,
6410     )
6411     if structured_fact is None:
6412         return None
6413
6414     evidence_item = next(
6415         (
6416             evidence_by_id[evidence_id]
6417             for evidence_id in structured_fact.evidence_ids
6418             if evidence_id in evidence_by_id
6419             and evidence_by_id[evidence_id].capability
6420             == EvidenceCapability.STRUCTURED_RECORD_VERBALISATION
6421         ),
6422         None,
6423     )
6424     metrics = evidence_item.metrics if evidence_item is not None else {}
6425     records = [
6426         record
6427         for record in metrics.get("records", [])
6428         if isinstance(record, dict)
6429     ]
6430     sentence = _triple_sentence(
6431         records,
6432         realisation_policy=realisation_policy,
6433     ) or _attribute_sentence(records)
6434     sentence = sentence or structured_fact.fact_summary
6435     sentence = re.sub(r"\s+", " ", sentence).strip()
6436     if sentence and sentence[-1] not in ".!?:":
6437         sentence += "."
6438
6439     return WriterOutput(
6440         title="Structured record verbalisation",
6441         markdown=sentence + "\n",
6442         sentence_support=[
6443             SentenceSupport(
6444                 sentence_id="SENT_0001",
6445                 sentence_text=sentence,
6446                 fact_ids=[structured_fact.fact_id],
6447                 evidence_ids=list(structured_fact.evidence_ids),
6448                 support_type=SupportType.DIRECT,
6449             )
6450         ],
6451         selected_fact_ids=[structured_fact.fact_id],
6452         omitted_fact_ids=[
6453             fact.fact_id
6454             for fact in available_facts
6455             if fact.fact_id != structured_fact.fact_id
6456         ],
6457         writer_notes=[
6458             "Deterministic short-form writer used verified structured-record "
6459             "evidence.",
6460             "This requested short-form verbalisation is eligible for primary "
6461             "evaluation because it is directly support-mapped.",
6462         ],
6463         writer_mode="deterministic_short_form_writer",
6464         eligible_for_primary_evaluation=True,
6465     )
6466
6467
6468 def fallback_writer(
6469     pack: WriterEvidencePack,
6470 ) -> WriterOutput:
6471     """
6472     Safe deterministic fallback.
6473
6474     It is deliberately richer than a two-sentence renderer, but remains
6475     ineligible for primary evaluation because it is not the natural LLM
6476     Writer.
6477     """
6478
6479     event_report = (
6480         pack.report_specification.genre
6481         in {
6482             ReportGenre.EVENT_REPORT,
6483             ReportGenre.SPORTS_GAME_REPORT,
6484         }
6485     )
6486     reference_recap_style = (

```



```

6487     event_report
6488     and pack.report_specification.focus_scope == "reference_recap"
6489 )
6490 evidence_by_id = build_evidence_lookup(
6491     pack.evidence_ledger
6492 )
6493 if (
6494     pack.report_specification.communication_task
6495     == CommunicationTask.FOCUSED_TABLE_DESCRIPTION
6496 ):
6497     focused_output = _focused_table_fallback_writer(
6498         pack,
6499         evidence_by_id,
6500     )
6501     if focused_output is not None:
6502         return focused_output
6503 if pack.report_specification.communication_task in {
6504     CommunicationTask.ATTRIBUTE_VERBALISATION,
6505     CommunicationTask.TRIPLE_VERBALISATION,
6506 }:
6507     structured_output = _structured_record_fallback_writer(
6508         pack,
6509         evidence_by_id,
6510     )
6511     if structured_output is not None:
6512         return structured_output
6513
6514 priority_facts = pack.priority_facts
6515 if pack.report_specification.maximum_main_findings is not None:
6516     priority_facts = priority_facts[
6517         : pack.report_specification.maximum_main_findings
6518     ]
6519
6520 selected = list(
6521     {
6522         fact.fact_id: fact
6523         for fact in (
6524             priority_facts
6525             + pack.limitation_facts
6526         )
6527     }.values()
6528 )
6529
6530 sections: dict[
6531     ReportComponent,
6532     list[VerifiedFact],
6533 ] = {
6534     ReportComponent.DATASET_OVERVIEW: [],
6535     ReportComponent.DATA_QUALITY: [],
6536     ReportComponent.STRONGEST_RELATIONSHIPS: [],
6537     ReportComponent.MODELLING_VALIDATION: [],
6538     ReportComponent.LIMITATIONS_NEXT_STEPS: [],
6539 }
6540
6541 for fact in selected:
6542     component = classify_fact_component(
6543         fact,
6544         evidence_by_id,
6545     )
6546
6547     sections.setdefault(
6548         component,
6549         [],
6550     ).append(fact)
6551
6552 headings = (
6553     {
6554         ReportComponent.DATASET_OVERVIEW: (
6555             "Result and leading performances"
6556         ),
6557         ReportComponent.DATA_QUALITY: (
6558             "Record quality"
6559         ),
6560         ReportComponent.STRONGEST_RELATIONSHIPS: (
6561             "Main participant contrasts"
6562         ),
6563         ReportComponent.MODELLING_VALIDATION: (
6564             "Supporting analysis"
6565         ),
6566         ReportComponent.LIMITATIONS_NEXT_STEPS: (
6567             "Limitations"
6568         ),
6569     }
6570     if event_report
6571     else {
6572         ReportComponent.DATASET_OVERVIEW: (
6573             "Dataset overview"
6574         ),
6575         ReportComponent.DATA_QUALITY: (
6576             "Data quality"
6577         ),

```

```

6578         ReportComponent.STRONGEST_RELATIONSHIPS: (
6579             "Strongest observed relationships"
6580         ),
6581         ReportComponent.MODELLING_VALIDATION: (
6582             "Modelling and validation"
6583         ),
6584         ReportComponent.LIMITATIONS_NEXT_STEPS: (
6585             "Limitations and next steps"
6586         ),
6587     }
6588 )
6589
6590 report_title = (
6591     "Evidence-grounded event report"
6592     if event_report
6593     else "Evidence-grounded data-science report"
6594 )
6595
6596 lines = (
6597     []
6598     if reference_recap_style
6599     else [
6600         f"# {report_title}",
6601         "",
6602     ]
6603 )
6604
6605 support_map: list[SentenceSupport] = []
6606 sentence_counter = 1
6607 rendered_recommendations: set[str] = set()
6608
6609 def add_supported_sentence(
6610     sentence: str,
6611     facts: list[VerifiedFact],
6612     support_type: SupportType,
6613 ) -> None:
6614     nonlocal sentence_counter
6615
6616     cleaned = sentence.strip()
6617
6618     if not cleaned:
6619         return
6620
6621     if cleaned[-1] not in ".!?:":
6622         cleaned += "."
6623
6624     lines.append(cleaned)
6625
6626     fact_ids = list(
6627         dict.fromkeys(
6628             fact.fact_id
6629             for fact in facts
6630         )
6631     )
6632
6633     evidence_ids = list(
6634         dict.fromkeys(
6635             evidence_id
6636             for fact in facts
6637             for evidence_id in fact.evidence_ids
6638         )
6639     )
6640
6641     rendered_sentences = (
6642         split_markdown_sentences(cleaned)
6643         or [cleaned]
6644     )
6645     for rendered_sentence in rendered_sentences:
6646         support_map.append(
6647             SentenceSupport(
6648                 sentence_id=(
6649                     f"SENT_{sentence_counter:04d}"
6650                 ),
6651                 sentence_text=rendered_sentence,
6652                 fact_ids=fact_ids,
6653                 evidence_ids=evidence_ids,
6654                 support_type=support_type,
6655             )
6656         )
6657
6658         sentence_counter += 1
6659
6660 for component in [
6661     ReportComponent.DATASET_OVERVIEW,
6662     ReportComponent.DATA_QUALITY,
6663     ReportComponent.STRONGEST_RELATIONSHIPS,
6664     ReportComponent.MODELLING_VALIDATION,
6665 ]:
6666     component_facts = sections.get(
6667         component,
6668         [],

```

```

6669     )
6670
6671     if not component_facts:
6672         continue
6673
6674     if not reference_recap_style:
6675         lines.extend(
6676             [
6677                 f"## {headings[component]}",
6678                 "",
6679             ]
6680         )
6681
6682     for fact in component_facts:
6683         sentence = (
6684             reader_facing_caveat(
6685                 fact.fact_summary
6686             )
6687             or fact.fact_summary
6688         )
6689
6690         add_supported_sentence(
6691             sentence,
6692             [fact],
6693             SupportType.DIRECT,
6694         )
6695
6696     if (
6697         component
6698         == ReportComponent.DATA_QUALITY
6699     ):
6700         for evidence_id in fact.evidence_ids:
6701             item = evidence_by_id.get(
6702                 evidence_id
6703             )
6704
6705             if item is None:
6706                 continue
6707
6708             for recommendation in (
6709                 item.recommendations
6710             ):
6711                 if (
6712                     recommendation.priority
6713                     not in {"high", "medium"}
6714                 ):
6715                     continue
6716
6717                 action = (
6718                     recommendation.action.strip()
6719                 )
6720
6721                 if (
6722                     action
6723                     in rendered_recommendations
6724                 ):
6725                     continue
6726
6727                 rendered_recommendations.add(
6728                     action
6729                 )
6730
6731                 add_supported_sentence(
6732                     action,
6733                     [fact],
6734                     SupportType.PARAPHRASE,
6735                 )
6736
6737     if not reference_recap_style:
6738         lines.append("")
6739
6740 limitation_facts = list(
6741     {
6742         fact.fact_id: fact
6743         for fact in (
6744             sections.get(
6745                 ReportComponent
6746                 .LIMITATIONS_NEXT_STEPS,
6747                 [],
6748             )
6749             + [
6750                 fact
6751                 for fact in selected
6752                 if (
6753                     fact.required_caveats
6754                     or ClaimPermission.COMPARATIVE
6755                     in fact.claim_permissions
6756                     or ClaimPermission.ASSOCIATIONAL
6757                     in fact.claim_permissions
6758                 )
6759             ]

```

```

6760         )
6761         }.values()
6762     )
6763     event_limitations = list(
6764         dict.fromkeys(
6765             caveat
6766             for fact in selected
6767             for caveat in fact.required_caveats
6768         )
6769     )
6770
6771     if (
6772         not reference_recap_style
6773         and (
6774             (
6775                 event_limitations
6776                 if event_report
6777                 else pack.reader_facing_limitations
6778             )
6779             or limitation_facts
6780             or rendered_recommendations
6781         )
6782     ):
6783         lines.extend(
6784             [
6785                 "## "
6786                 + headings[
6787                     ReportComponent.LIMITATIONS_NEXT_STEPS
6788                 ],
6789                 "",
6790             ]
6791         )
6792
6793         for fact in sections.get(
6794             ReportComponent
6795             .LIMITATIONS_NEXT_STEPS,
6796             [],
6797         ):
6798             add_supported_sentence(
6799                 fact.fact_summary,
6800                 [fact],
6801                 SupportType.DIRECT,
6802             )
6803
6804         if event_report:
6805             for limitation in event_limitations:
6806                 supporting = [
6807                     fact
6808                     for fact in selected
6809                     if limitation in fact.required_caveats
6810                 ]
6811                 if supporting:
6812                     add_supported_sentence(
6813                         limitation,
6814                         supporting,
6815                         SupportType.PARAPHRASE,
6816                     )
6817             else:
6818                 relationship_facts = [
6819                     fact
6820                     for fact in selected
6821                     if (
6822                         ClaimPermission.COMPARATIVE
6823                         in fact.claim_permissions
6824                         or ClaimPermission.ASSOCIATIONAL
6825                         in fact.claim_permissions
6826                     )
6827                 ]
6828
6829                 for limitation in (
6830                     pack.reader_facing_limitations
6831                 ):
6832                     supporting = (
6833                         relationship_facts
6834                         or limitation_facts
6835                     )
6836
6837                     if supporting:
6838                         add_supported_sentence(
6839                             limitation,
6840                             supporting,
6841                             SupportType.MULTI_FACT_SYNTHESIS,
6842                         )
6843
6844                 lines.append("")
6845
6846     used_fact_ids = list(
6847         dict.fromkeys(
6848             fact_id
6849             for support in support_map
6850             for fact_id in support.fact_ids

```

```

6851     )
6852 )
6853
6854 available_facts = list(
6855     {
6856         fact.fact_id: fact
6857         for fact in (
6858             pack.priority_facts
6859             + pack.supporting_facts
6860             + pack.limitation_facts
6861         )
6862     }.values()
6863 )
6864
6865 return WriterOutput(
6866     title=report_title,
6867     markdown=(
6868         "\n".join(lines).strip()
6869         + "\n"
6870     ),
6871     sentence_support=support_map,
6872     selected_fact_ids=used_fact_ids,
6873     omitted_fact_ids=[
6874         fact.fact_id
6875         for fact in available_facts
6876         if fact.fact_id
6877         not in set(used_fact_ids)
6878     ],
6879     writer_notes=[
6880         "Deterministic writer fallback was used.",
6881         "This output is preserved for debugging "
6882         "and is not eligible for primary evaluation.",
6883     ],
6884     writer_mode=(
6885         "deterministic_fallback"
6886     ),
6887     eligible_for_primary_evaluation=False,
6888 )
6889
6890 def default_quality_assessment() -> ReportQualityAssessment:
6891     return ReportQualityAssessment(
6892         status=QualityStatus.PASS,
6893         request_responsiveness=1.0,
6894         finding_selection=1.0,
6895         coherence=1.0,
6896         concision=1.0,
6897         caveat_integration=1.0,
6898         data_science_interpretation=1.0,
6899         findings=[],
6900         recommendations=[],
6901     )
6902
6903
6904 def assess_report_component_coverage(
6905     *,
6906     writer_output: WriterOutput,
6907     fact_ledger: FactLedger,
6908     evidence: EvidenceLedger,
6909     required_components: list[ReportComponent],
6910 ) -> list[ReportComponentAssessment]:
6911     """
6912     Assess coverage using the sentence support map.
6913
6914     Limitations and next steps may be supported by the same verified
6915     relationship or quality fact used elsewhere, so they are not
6916     restricted to facts whose primary component is LIMITATIONS.
6917     """
6918
6919     fact_lookup = {
6920         fact.fact_id: fact
6921         for fact
6922         in fact_ledger.writer_ready_facts
6923     }
6924
6925     evidence_by_id = build_evidence_lookup(
6926         evidence
6927     )
6928
6929     support_by_component: defaultdict[
6930         ReportComponent,
6931         list[str],
6932     ] = defaultdict(list)
6933     component_covered_without_facts: set[ReportComponent] = set()
6934
6935     for component in required_components:
6936         support_by_component[component]
6937
6938     limitation_language = re.compile(
6939         r"\b("
6940         r"limitation|"
6941         r"unadjusted|"

```

```

6942         r"causal|causation|"
6943         r"confound|"
6944         r"precision|stability|"
6945         r"validate|verify|inspect|investigate|"
6946         r"remove|exclude|check|"
6947         r"next step|further analysis|"
6948         r"deployment|backtest"
6949         r")\b",
6950         re.IGNORECASE,
6951     )
6952
6953     for support in writer_output.sentence_support:
6954         supported_facts = [
6955             fact_lookup[fact_id]
6956             for fact_id in support.fact_ids
6957             if fact_id in fact_lookup
6958         ]
6959
6960         for fact in supported_facts:
6961             component = classify_fact_component(
6962                 fact,
6963                 evidence_by_id,
6964             )
6965
6966             if (
6967                 component
6968                 in required_components
6969             ):
6970                 support_by_component[
6971                     component
6972                 ].append(fact.fact_id)
6973
6974             if (
6975                 ReportComponent
6976                 .LIMITATIONS_NEXT_STEPS
6977                 in required_components
6978                 and limitation_language.search(
6979                     support.sentence_text
6980                 )
6981             ):
6982                 if supported_facts:
6983                     support_by_component[
6984                         ReportComponent
6985                         .LIMITATIONS_NEXT_STEPS
6986                     ].extend(
6987                         fact.fact_id
6988                         for fact in supported_facts
6989                     )
6990                 else:
6991                     component_covered_without_facts.add(
6992                         ReportComponent.LIMITATIONS_NEXT_STEPS
6993                     )
6994
6995     assessments: list[
6996         ReportComponentAssessment
6997     ] = []
6998
6999     for component in required_components:
7000         fact_ids = list(
7001             dict.fromkeys(
7002                 support_by_component[component]
7003             )
7004         )
7005         covered = (
7006             bool(fact_ids)
7007             or component in component_covered_without_facts
7008         )
7009
7010         assessments.append(
7011             ReportComponentAssessment(
7012                 component=component,
7013                 covered=covered,
7014                 supporting_fact_ids=fact_ids,
7015                 explanation=(
7016                     "At least one report sentence is "
7017                     "mapped to verified support for "
7018                     "this component."
7019                     if fact_ids
7020                     else (
7021                         "A scoped non-factual caveat clearly covers "
7022                         "this component."
7023                         if covered
7024                         else (
7025                             "No supported report sentence "
7026                             "clearly covers this required "
7027                             "component."
7028                         )
7029                     )
7030                 ),
7031             )
7032         )

```

```

7033     return assessments
7034
7035
7036 def assess_report_components(
7037     writer_output: WriterOutput,
7038     fact_ledger: FactLedger,
7039     evidence: EvidenceLedger,
7040     required_components: List[ReportComponent],
7041 ) -> List[ReportComponentAssessment]:
7042     return assess_report_component_coverage(
7043         writer_output=writer_output,
7044         fact_ledger=fact_ledger,
7045         evidence=evidence,
7046         required_components=required_components,
7047     )
7048
7049
7050 def assess_genre_quality(
7051     writer_output: WriterOutput,
7052     report_specification: Any,
7053     evidence: EvidenceLedger,
7054 ) -> GenreQualityAssessment:
7055     event_report = report_specification.genre in {
7056         ReportGenre.EVENT_REPORT,
7057         ReportGenre.SPORTS_GAME_REPORT,
7058     }
7059     realisation_policy = getattr(
7060         report_specification,
7061         "realisation_policy",
7062         None,
7063     )
7064     realisation_policy_value = (
7065         realisation_policy.value
7066         if isinstance(realisation_policy, RealisationPolicy)
7067         else str(realisation_policy)
7068     )
7069     reference_recap_style = bool(
7070         event_report
7071         and (
7072             getattr(report_specification, "focus_scope", None)
7073             == "reference_recap"
7074             or realisation_policy_value
7075             == RealisationPolicy.EVENT_RECAP_STYLE.value
7076         )
7077     )
7078     evidence_lookup = {
7079         item.evidence_id: item
7080         for item in evidence.items
7081         if item.eligible_for_writer
7082     }
7083     actionable_sequence_evidence_ids = {
7084         evidence_id
7085         for evidence_id, item in evidence_lookup.items()
7086         if event_sequence_evidence_is_actionable(item)
7087     }
7088     used_evidence_ids = {
7089         evidence_id
7090         for support in writer_output.sentence_support
7091         for evidence_id in support.evidence_ids
7092     }
7093
7094 def matching_evidence(slot: str) -> set[str]:
7095     matches: set[str] = set()
7096     for evidence_id, item in evidence_lookup.items():
7097         if (
7098             slot in {"focused_table_region", "focused_cell_context"}
7099             and item.capability == EvidenceCapability.FOCUSED_TABLE_REGION
7100         ):
7101             matches.add(evidence_id)
7102         elif (
7103             slot == "structured_record_verbalisation"
7104             and item.capability
7105             == EvidenceCapability.STRUCTURED_RECORD_VERBALISATION
7106         ):
7107             matches.add(evidence_id)
7108         elif slot == "event_result" and item.evidence_type == "event_outcome":
7109             matches.add(evidence_id)
7110         elif slot == "leading_performance" and item.capability in {
7111             EvidenceCapability.ENTITY_PERFORMANCE,
7112             EvidenceCapability.RANKING,
7113         } and evidence_analytical_function(item) != (
7114             AnalyticalFunction.PARTICIPATION
7115         ) and not event_evidence_is_segment_ranking(item):
7116             matches.add(evidence_id)
7117         elif slot == "main_contrast" and item.evidence_type in {
7118             "participant_comparison",
7119             "event_contrast",
7120             "group_comparison",
7121         }:
7122             matches.add(evidence_id)
7123         elif slot == "event_context" and item.evidence_type == ("event_context"):

```

```

7124         matches.add(evidence_id)
7125     elif slot == "event_status" and item.evidence_type == "event_status":
7126         matches.add(evidence_id)
7127     elif (
7128         slot == "participant_record_context"
7129         and item.evidence_type == "participant_record_context"
7130     ):
7131         matches.add(evidence_id)
7132     elif (
7133         slot == "score_progression"
7134         and item.evidence_type == "score_progression"
7135     ):
7136         matches.add(evidence_id)
7137     elif (
7138         slot == "event_sequence"
7139         and item.evidence_type == "event_sequence"
7140     ):
7141         if (
7142             actionable_sequence_evidence_ids
7143             and evidence_id not in actionable_sequence_evidence_ids
7144         ):
7145             continue
7146         matches.add(evidence_id)
7147     elif slot == "secondary_performance" and item.capability == (
7148         EvidenceCapability.RANKING
7149     ) and not event_evidence_is_segment_ranking(item):
7150         matches.add(evidence_id)
7151     elif slot == "dataset_scope" and item.evidence_type in {
7152         "dataset_overview",
7153         "event_record_overview",
7154     }:
7155         matches.add(evidence_id)
7156     elif slot == "material_data_quality_issue" and evidence_subtype(item) == (
7157         "data_quality"
7158     ):
7159         matches.add(evidence_id)
7160     elif slot == "strongest_analytical_finding" and item.capability in {
7161         EvidenceCapability.ASSOCIATION,
7162         EvidenceCapability.GROUP_COMPARISON,
7163         EvidenceCapability.EVENT_OUTCOME,
7164     }:
7165         matches.add(evidence_id)
7166     return matches
7167
7168 required_slots = list(report_specification.required_content_slots)
7169 supported_slots: list[str] = []
7170 covered_slots: list[str] = []
7171
7172 for slot in [
7173     *required_slots,
7174     *report_specification.optional_content_slots,
7175 ]:
7176     matching = matching_evidence(slot)
7177     if matching:
7178         supported_slots.append(slot)
7179     if matching & used_evidence_ids:
7180         covered_slots.append(slot)
7181
7182 if "limitation" in required_slots:
7183     supported_slots.append("limitation")
7184     if re.search(
7185         r"\b(limitations?|caveats?|does not|cannot|uncertain|missing)\b",
7186         writer_output.markdown,
7187         re.IGNORECASE,
7188     ):
7189         covered_slots.append("limitation")
7190
7191 supported_slot_set = set(supported_slots)
7192 event_material_available = event_report and {
7193     "event_result",
7194     "leading_performance",
7195     "main_contrast",
7196 }.issubset(supported_slot_set)
7197
7198 visible_scope_limitation_required = (
7199     event_material_available
7200     and not reference_recap_style
7201 )
7202
7203 if visible_scope_limitation_required:
7204     supported_slots.append("scope_limitations")
7205     if event_scope_limitation_present(writer_output):
7206         covered_slots.append("scope_limitations")
7207
7208 missing = [
7209     slot
7210     for slot in required_slots
7211     if slot in supported_slots and slot not in covered_slots
7212 ]
7213 findings = [
7214     f"Required supported content slot `{slot}` is missing from the report."

```



```

7215     for slot in missing
7216 ]
7217 recommendations = [
7218     "Use the available verified evidence to cover each missing content slot."
7219 ] if missing else []
7220
7221 if event_report and re.search(
7222     r"\b(rows?|columns?|constant columns?|missingness|schema|"
7223     r"correlations?|regressions?|statistical modelling|statistical modeling|"
7224     r"predictive modelling|predictive modeling|statistical power|"
7225     r"feature sets?|observed associations?|"
7226     r"group comparisons? (?:(are|is) unadjusted)\b",
7227     writer_output.markdown,
7228     re.IGNORECASE,
7229 ):
7230     findings.append(
7231         "The event report includes flat-table profiling or modelling "
7232         "discussion that displaces the supported event narrative."
7233     )
7234     recommendations.append(
7235         "Remove wrapper-level profiling and modelling discussion; use the "
7236         "supported event context, performances and participant contrasts."
7237     )
7238
7239 if (
7240     event_report
7241     and (
7242         actionable_sequence_evidence_ids
7243         or any(
7244             item.evidence_type == "event_sequence"
7245             for item in evidence_lookup.values()
7246         )
7247     )
7248     and (
7249         EVENT_SEQUENCE_ABSENCE_PATTERN.search(writer_output.markdown)
7250         or EVENT_SEQUENCE_OMISSION_PATTERN.search(writer_output.markdown)
7251     )
7252 ):
7253     findings.append(
7254         "The event report omits or disclaims event-sequence narration "
7255         "even though supported event-sequence evidence is available."
7256     )
7257     recommendations.append(
7258         "Use the supported event-sequence evidence without inferring "
7259         "unsupported causes, momentum or turning points."
7260     )
7261
7262 if event_material_available:
7263     narrative_stats = sentence_support_narrative_stats(
7264         writer_output.sentence_support
7265     )
7266     if narrative_stats["synthesis_sentences"] < 2:
7267         findings.append(
7268             "The event report lists supported facts without enough "
7269             "multi-fact or insight-backed narrative synthesis."
7270         )
7271         recommendations.append(
7272             "Relate the supported result, performances and participant "
7273             "contrasts in connected event prose."
7274         )
7275     if narrative_stats["connective_sentences"] < 1:
7276         findings.append(
7277             "The event report does not clearly connect supported "
7278             "participant contrasts into a narrative comparison."
7279         )
7280         recommendations.append(
7281             "Use bounded connective wording such as while, compared with "
7282             "or despite when the verified facts support a contrast."
7283         )
7284     if (
7285         visible_scope_limitation_required
7286         and narrative_stats["scope_limitation_sentences"] < 1
7287     ):
7288         findings.append(
7289             "The event report omits an event-scoped limitation."
7290         )
7291         recommendations.append(
7292             "Add a short caveat that the comparisons describe only the "
7293             "supplied event and do not explain why the result occurred."
7294         )
7295
7296 if event_report and not participation_measure_requested(
7297     " ".join(
7298         [
7299             report_specification.report_purpose,
7300             report_specification.communication_goal,
7301         ]
7302     )
7303 ):
7304     available_substantive = {
7305         evidence_id

```

```

7306         for evidence_id, item in evidence_lookup.items()
7307         if item.evidence_type
7308         in {
7309             "entity_ranking",
7310             "entity_performance",
7311         }
7312         and evidence_analytical_function(item)
7313         != AnalyticalFunction.PARTICIPATION
7314         and not event_evidence_is_segment_ranking(item)
7315     }
7316     used_participation = {
7317         evidence_id
7318         for evidence_id, item in evidence_lookup.items()
7319         if evidence_id in used_evidence_ids
7320         and item.evidence_type
7321         in {
7322             "entity_ranking",
7323             "entity_performance",
7324         }
7325         and evidence_analytical_function(item)
7326         == AnalyticalFunction.PARTICIPATION
7327     }
7328     omitted_substantive = (
7329         available_substantive - used_evidence_ids
7330     )
7331
7332     if used_participation and omitted_substantive:
7333         findings.append(
7334             "The event report uses participation evidence while available "
7335             "substantive entity-performance evidence is omitted."
7336         )
7337         recommendations.append(
7338             "Prioritise distinct substantive performances over duration or "
7339             "exposure unless participation was explicitly requested."
7340         )
7341
7342     return GenreQualityAssessment(
7343         status=(QualityStatus.REVISE if findings else QualityStatus.PASS),
7344         genre=report_specification.genre,
7345         required_slots=required_slots,
7346         supported_slots=list(dict.fromkeys(supported_slots)),
7347         covered_slots=list(dict.fromkeys(covered_slots)),
7348         missing_supported_slots=missing,
7349         findings=findings,
7350         recommendations=recommendations,
7351     )
7352
7353
7354 REPAIR_DRIVING_ERROR_TYPES = {
7355     ErrorType.INCORRECT_NAMED_ENTITY,
7356     ErrorType.INCORRECT_NUMBER,
7357     ErrorType.INCORRECT_WORD,
7358     ErrorType.CONTEXT_ERROR,
7359     ErrorType.SUPPORT_MAPPING_ERROR,
7360 }
7361
7362
7363 def annotation_requires_repair(
7364     annotation: AuditAnnotation,
7365 ) -> bool:
7366     return (
7367         annotation.error_type in REPAIR_DRIVING_ERROR_TYPES
7368         and annotation.severity
7369         in {
7370             Severity.MEDIUM,
7371             Severity.HIGH,
7372             Severity.CRITICAL,
7373         }
7374         and annotation.confidence >= 0.80
7375         and bool(annotation.correction_goal.strip())
7376     )
7377
7378
7379 def decide_release_status(
7380     *,
7381     annotations: list[AuditAnnotation],
7382     quality: ReportQualityAssessment,
7383     methodological_warnings: list[str],
7384     repair_budget_exhausted: bool,
7385     audit_mode: AuditMode,
7386 ) -> ReleaseStatus:
7387     if audit_mode == AuditMode.ANNOTATION_ONLY:
7388         if (
7389             annotations
7390             or methodological_warnings
7391             or quality.status != QualityStatus.PASS
7392         ):
7393             return ReleaseStatus.APPROVED_WITH_WARNINGS
7394
7395     return ReleaseStatus.APPROVED
7396

```

```

7397     unresolved_critical = any(
7398         annotation.severity == Severity.CRITICAL
7399         and annotation.confidence >= 0.80
7400         for annotation in annotations
7401     )
7402     unresolved_high = any(
7403         annotation.severity == Severity.HIGH
7404         and annotation.confidence >= 0.80
7405         for annotation in annotations
7406     )
7407     unresolved_repairable = any(
7408         annotation_requires_repair(annotation)
7409         for annotation in annotations
7410     )
7411
7412     if unresolved_critical:
7413         return ReleaseStatus.HUMAN_REVIEW_REQUIRED
7414
7415     if (
7416         repair_budget_exhausted
7417         and (unresolved_high or unresolved_repairable)
7418     ):
7419         return ReleaseStatus.HUMAN_REVIEW_REQUIRED
7420
7421     if (
7422         annotations
7423         or methodological_warnings
7424         or quality.status != QualityStatus.PASS
7425     ):
7426         return ReleaseStatus.APPROVED_WITH_WARNINGS
7427
7428     return ReleaseStatus.APPROVED
7429
7430
7431 def add_annotation(
7432     annotations: list[AuditAnnotation],
7433     *,
7434     sentence: str,
7435     text_span: str,
7436     error_type: ErrorType,
7437     subtype: str,
7438     severity: Severity,
7439     explanation: str,
7440     correction_goal: str,
7441     fact_ids: list[str] | None = None,
7442     evidence_ids: list[str] | None = None,
7443     profile_support_ids: list[str] | None = None,
7444     insight_ids: list[str] | None = None,
7445     confidence: float = 1.0,
7446 ) -> None:
7447     key = (
7448         sentence,
7449         text_span,
7450         error_type.value,
7451         subtype,
7452     )
7453
7454     if any(
7455         (
7456             annotation.sentence,
7457             annotation.text_span,
7458             annotation.error_type.value,
7459             annotation.subtype,
7460         )
7461         == key
7462         for annotation in annotations
7463     ):
7464         return
7465
7466     annotations.append(
7467         AuditAnnotation(
7468             annotation_id=f"ANN_{len(annotations) + 1:04d}",
7469             sentence=sentence,
7470             text_span=text_span,
7471             error_type=error_type,
7472             subtype=subtype,
7473             severity=severity,
7474             explanation=explanation,
7475             correction_goal=correction_goal,
7476             fact_ids=fact_ids or [],
7477             evidence_ids=evidence_ids or [],
7478             profile_support_ids=profile_support_ids or [],
7479             insight_ids=insight_ids or [],
7480             confidence=confidence,
7481         )
7482     )
7483
7484
7485 def negative_causal(sentence: str) -> bool:
7486     return bool(
7487         re.search(

```

```

7488         r"\b(no causal|not causal|does not establish causation|"
7489         r"causality is not established|causal conclusion is not|"
7490         r"do(?:es)? not explain why|cannot explain why|"
7491         r"do(?:es)? not establish why|"
7492         r"without explaining why)\b",
7493         sentence,
7494         re.IGNORECASE,
7495     )
7496 )
7497
7498
7499 def _normalised_text_contains(
7500     haystack: str,
7501     needle: Any,
7502 ) -> bool:
7503     needle_text = str(needle or "").strip()
7504     if not needle_text:
7505         return False
7506
7507     return needle_text.casefold() in haystack.casefold()
7508
7509
7510 def _highlight_describes_tied_score(
7511     highlight: Mapping[str, Any],
7512 ) -> bool:
7513     left_value = highlight.get("left_value")
7514     right_value = highlight.get("right_value")
7515     if isinstance(left_value, (int, float)) and isinstance(
7516         right_value,
7517         (int, float),
7518     ):
7519         return float(left_value) == float(right_value)
7520
7521     score_phrase = str(highlight.get("score_phrase") or "")
7522     return bool(
7523         re.search(
7524             r"\b(?:level|tie[sd]?)\b",
7525             score_phrase,
7526             re.IGNORECASE,
7527         )
7528     )
7529
7530
7531 def _highlight_referenced_by_sentence(
7532     sentence: str,
7533     highlight: Mapping[str, Any],
7534 ) -> bool:
7535     event_text = str(highlight.get("event_text") or "")
7536     actor = event_text.split(" recorded ", 1)[0]
7537     actor = re.sub(
7538         r"^\.*?:\s*",
7539         "",
7540         actor,
7541     ).strip()
7542     score_phrase = str(highlight.get("score_phrase") or "")
7543
7544     return any(
7545         _normalised_text_contains(sentence, value)
7546         for value in [
7547             actor,
7548             score_phrase,
7549             f"{highlight.get('left_value'):g}-{highlight.get('right_value'):g}"
7550             if isinstance(highlight.get("left_value"), (int, float))
7551             and isinstance(highlight.get("right_value"), (int, float))
7552             else None,
7553         ]
7554     )
7555
7556
7557 def score_state_tie_claim_supported(
7558     sentence: str,
7559     evidence_items: Sequence[EvidenceItem],
7560 ) -> bool:
7561     if not SCORE_STATE_TIE_CLAIM_PATTERN.search(sentence):
7562         return True
7563     if NEGATED_SCORE_STATE_TIE_PATTERN.search(sentence):
7564         return True
7565
7566     sequence_items = [
7567         item
7568         for item in evidence_items
7569         if item.evidence_type == "event_sequence"
7570     ]
7571     if not sequence_items:
7572         return False
7573
7574     for item in sequence_items:
7575         highlights = item.metrics.get("highlights", [])
7576         if not isinstance(highlights, list):
7577             continue

```

```

7579         for highlight in highlights:
7580             if not isinstance(highlight, Mapping):
7581                 continue
7582             if not _highlight_describes_tied_score(highlight):
7583                 continue
7584             if _highlight_referenced_by_sentence(sentence, highlight):
7585                 return True
7586
7587         return False
7588
7589
7590 def negative_predictive(sentence: str) -> bool:
7591     return bool(
7592         re.search(
7593             r"\b(not validated|did not improve|no predictive|"
7594             r"cannot support prediction|not deployment ready)\b",
7595             sentence,
7596             re.IGNORECASE,
7597         )
7598     )
7599
7600
7601 def negative_forecast(sentence: str) -> bool:
7602     return bool(
7603         re.search(
7604             r"\b(not validated|did not improve|no forecast|"
7605             r"did not outperform|not a live future forecast)\b",
7606             sentence,
7607             re.IGNORECASE,
7608         )
7609     )
7610
7611
7612 def _support_text_for_facts(
7613     facts: list[VerifiedFact],
7614     evidence: EvidenceLedger,
7615 ) -> str:
7616     return " ".join(
7617         fact_support_text(fact, evidence)
7618         for fact in facts
7619     ).lower()
7620
7621
7622 def _mapped_facts_semantically_support_profile_match(
7623     *,
7624     support_text: str,
7625     matches: list[ProfileSupportRecord],
7626 ) -> bool:
7627     for record in matches:
7628         pattern = PROFILE_FACT_KIND_TERMS.get(
7629             record.fact_kind
7630         )
7631         if (
7632             pattern is not None
7633             and pattern.search(support_text)
7634         ):
7635             return True
7636
7637     return False
7638
7639
7640 def _recommendation_supported_by_evidence(
7641     sentence: str,
7642     facts: list[VerifiedFact],
7643     evidence_lookup_by_id: dict[str, EvidenceItem],
7644 ) -> bool:
7645     sentence_text = sentence.lower()
7646
7647     for fact in facts:
7648         for item in evidence_for_fact(
7649             fact,
7650             evidence_lookup_by_id,
7651         ):
7652             for recommendation in item.recommendations:
7653                 action = recommendation.action.lower()
7654                 if action and (
7655                     action in sentence_text
7656                     or sentence_text in action
7657                 ):
7658                     return True
7659
7660     return False
7661
7662
7663 def add_wording_guardrail_annotations(
7664     *,
7665     annotations: list[AuditAnnotation],
7666     sentence: str,
7667     support: SentenceSupport,
7668     supporting_facts: list[VerifiedFact],
7669     evidence: EvidenceLedger,

```

```

7670     report_specification: Any,
7671 ) -> None:
7672     support_text = _support_text_for_facts(
7673         supporting_facts,
7674         evidence,
7675     )
7676     evidence_lookup_by_id = build_evidence_lookup(evidence)
7677     request_text = (
7678         getattr(
7679             report_specification,
7680             "report_purpose",
7681             "",
7682         )
7683         or ""
7684     ).lower()
7685
7686     hourly_match = HOURLY_CADENCE_PATTERN.search(sentence)
7687     if hourly_match and not HOURLY_CADENCE_PATTERN.search(support_text):
7688         add_annotation(
7689             annotations,
7690             sentence=sentence,
7691             text_span=hourly_match.group(0),
7692             error_type=ErrorType.CONTEXT_ERROR,
7693             subtype="unsupported_temporal_cadence",
7694             severity=Severity.HIGH,
7695             explanation=(
7696                 "A date/time-like field or parse rate does not establish "
7697                 "regular hourly spacing."
7698             ),
7699             correction_goal=(
7700                 "Use neutral wording such as timestamped weather observations."
7701             ),
7702             fact_ids=support.fact_ids,
7703             evidence_ids=support.evidence_ids,
7704             profile_support_ids=support.profile_support_ids,
7705         )
7706
7707     location_match = LOCATION_METADATA_PATTERN.search(sentence)
7708     if location_match and not LOCATION_METADATA_PATTERN.search(support_text):
7709         add_annotation(
7710             annotations,
7711             sentence=sentence,
7712             text_span=location_match.group(0),
7713             error_type=ErrorType.CONTEXT_ERROR,
7714             subtype="unsupported_location_metadata",
7715             severity=Severity.HIGH,
7716             explanation=(
7717                 "The supplied facts do not establish collection at a "
7718                 "specific location or weather station."
7719             ),
7720             correction_goal="Remove unsupported location metadata.",
7721             fact_ids=support.fact_ids,
7722             evidence_ids=support.evidence_ids,
7723             profile_support_ids=support.profile_support_ids,
7724         )
7725
7726     constant_match = CONSTANT_OVERSTATEMENT_PATTERN.search(sentence)
7727     if constant_match:
7728         add_annotation(
7729             annotations,
7730             sentence=sentence,
7731             text_span=constant_match.group(0),
7732             error_type=ErrorType.CONTEXT_ERROR,
7733             subtype="overbroad_constant_interpretation",
7734             severity=Severity.HIGH,
7735             explanation=(
7736                 "A constant field should not be described as universally "
7737                 "worthless or valueless."
7738             ),
7739             correction_goal=(
7740                 "Say it contains no observed variation for analyses that "
7741                 "depend on variation."
7742             ),
7743             fact_ids=support.fact_ids,
7744             evidence_ids=support.evidence_ids,
7745             profile_support_ids=support.profile_support_ids,
7746         )
7747
7748     zero_match = ZERO_OVERCONFIDENCE_PATTERN.search(sentence)
7749     if zero_match:
7750         add_annotation(
7751             annotations,
7752             sentence=sentence,
7753             text_span=zero_match.group(0),
7754             error_type=ErrorType.CONTEXT_ERROR,
7755             subtype="overconfident_zero_interpretation",
7756             severity=Severity.HIGH,
7757             explanation=(
7758                 "Suspicious zero diagnostics do not establish what the zero "
7759                 "values actually mean."
7760             ),

```

```

7761         correction_goal=(
7762             "Use cautious wording such as may represent encoded "
7763             "missingness or measurement failure and should be validated."
7764         ),
7765         fact_ids=support.fact_ids,
7766         evidence_ids=support.evidence_ids,
7767         profile_support_ids=support.profile_support_ids,
7768     )
7769
7770 missingness_match = MISSINGNESS_HARMLESS_PATTERN.search(sentence)
7771 if missingness_match:
7772     add_annotation(
7773         annotations,
7774         sentence=sentence,
7775         text_span=missingness_match.group(0),
7776         error_type=ErrorType.CONTEXT_ERROR,
7777         subtype="unsupported_missingness_impact",
7778         severity=Severity.HIGH,
7779         explanation=(
7780             "A missingness rate alone does not establish that the "
7781             "missingness is harmless."
7782         ),
7783         correction_goal=(
7784             "Report the rate and recommend examining the missingness "
7785             "pattern."
7786         ),
7787         fact_ids=support.fact_ids,
7788         evidence_ids=support.evidence_ids,
7789         profile_support_ids=support.profile_support_ids,
7790     )
7791
7792 duplicate_match = DUPLICATE_REMOVAL_PATTERN.search(sentence)
7793 if duplicate_match:
7794     add_annotation(
7795         annotations,
7796         sentence=sentence,
7797         text_span=duplicate_match.group(0),
7798         error_type=ErrorType.CONTEXT_ERROR,
7799         subtype="unsupported_duplicate_removal",
7800         severity=Severity.HIGH,
7801         explanation=(
7802             "Exact duplicate rows may be valid repeated observations or "
7803             "unintended duplicates."
7804         ),
7805         correction_goal=(
7806             "Say duplicate rows should be reviewed before any decision "
7807             "to remove them."
7808         ),
7809         fact_ids=support.fact_ids,
7810         evidence_ids=support.evidence_ids,
7811         profile_support_ids=support.profile_support_ids,
7812     )
7813
7814 pearson_match = PEARSON_IMPRECISE_PATTERN.search(sentence)
7815 if pearson_match:
7816     add_annotation(
7817         annotations,
7818         sentence=sentence,
7819         text_span=pearson_match.group(0),
7820         error_type=ErrorType.CONTEXT_ERROR,
7821         subtype="imprecise_pearson_limitation",
7822         severity=Severity.MEDIUM,
7823         explanation=(
7824             "Pearson correlation may not capture non-linear relationships; "
7825             "it is not literally influenced by a pattern it does not measure."
7826         ),
7827         correction_goal=(
7828             "Say Pearson correlation may not capture non-linear "
7829             "relationships and can be sensitive to influential observations."
7830         ),
7831         fact_ids=support.fact_ids,
7832         evidence_ids=support.evidence_ids,
7833         profile_support_ids=support.profile_support_ids,
7834         confidence=0.9,
7835     )
7836
7837 future_match = FUTURE_ANALYSIS_PATTERN.search(sentence)
7838 if future_match:
7839     requested = bool(
7840         re.search(
7841             r"\b(model|modelling|modeling|temporal|trend|future|"
7842             r"relationship|multivariate)\b",
7843             request_text,
7844         )
7845     )
7846     supported_recommendation = (
7847         _recommendation_supported_by_evidence(
7848             sentence,
7849             supporting_facts,
7850             evidence_lookup_by_id,
7851         )

```

```

7852     )
7853
7854     if not requested and not supported_recommendation:
7855         add_annotation(
7856             annotations,
7857             sentence=sentence,
7858             text_span=future_match.group(0),
7859             error_type=ErrorType.CONTEXT_ERROR,
7860             subtype="unsupported_analytical_recommendation",
7861             severity=Severity.MEDIUM,
7862             explanation=(
7863                 "Reader-facing next steps must come from supplied "
7864                 "recommendations, verified methodological facts, or the "
7865                 "explicit user request."
7866             ),
7867             correction_goal=(
7868                 "Remove the unsupported recommendation or ground it in a "
7869                 "supplied analytical recommendation."
7870             ),
7871             fact_ids=support.fact_ids,
7872             evidence_ids=support.evidence_ids,
7873             profile_support_ids=support.profile_support_ids,
7874             confidence=0.9,
7875         )
7876
7877 def deterministic_audit(
7878     writer_output: WriterOutput,
7879     fact_ledger: FactLedger,
7880     evidence: EvidenceLedger,
7881     mode: AuditMode,
7882     external_sources: list[ExternalTruthSource],
7883     revision_round: int,
7884     report_specification: Any,
7885     settings: Settings | None = None,
7886     profile_support_records: list[
7887         ProfileSupportRecord
7888     ] | None = None,
7889     insight_ledger: InsightLedger | None = None,
7890 ) -> AuditReport:
7891     settings = settings or Settings()
7892     profile_support_records = profile_support_records or []
7893     insight_ledger = insight_ledger or InsightLedger(
7894         synthesis_enabled=False
7895     )
7896     profile_records_by_id = {
7897         record.support_id: record
7898         for record in profile_support_records
7899     }
7900     facts = {
7901         fact.fact_id: fact
7902         for fact in fact_ledger.writer_ready_facts
7903     }
7904     verified_insights = {
7905         insight.insight_id: insight
7906         for insight in insight_ledger.verified_insights
7907     }
7908     hypothesis_insights = {
7909         insight.insight_id: insight
7910         for insight in insight_ledger.hypothesis_only_insights
7911     }
7912     all_insight_ids = {
7913         *verified_insights,
7914         *hypothesis_insights,
7915     }
7916
7917     support_by_sentence = {
7918         support.sentence_text: support
7919         for support in writer_output.sentence_support
7920     }
7921
7922     annotations: list[AuditAnnotation] = []
7923     support_map_patches: list[SupportMapPatch] = []
7924     sentences = split_markdown_sentences(writer_output.markdown)
7925     section_headings = _sentence_section_headings(
7926         writer_output.markdown
7927     )
7928     evidence_lookup_by_id = build_evidence_lookup(evidence)
7929
7930     factual_count = 0
7931     supported_count = 0
7932
7933     for sentence in sentences:
7934         support = support_by_sentence.get(sentence)
7935
7936         if (
7937             not looks_factual(sentence)
7938             and not (
7939                 support is not None
7940                 and support.interpretation_level
7941                 != InterpretationLevel.FINDING
7942

```



```

7943     )
7944 ):
7945     continue
7946
7947     factual_count += 1
7948
7949     if support is None:
7950         add_annotation(
7951             annotations,
7952             sentence=sentence,
7953             text_span=sentence,
7954             error_type=ErrorType.NOT_CHECKABLE,
7955             subtype="missing_support_map_entry",
7956             severity=Severity.HIGH,
7957             explanation=(
7958                 "The sentence appears factual but is absent from the hidden "
7959                 "sentence support map."
7960             ),
7961             correction_goal=(
7962                 "Attach relevant verified facts or remove unsupported factual content."
7963             ),
7964         )
7965         continue
7966
7967     unknown_fact_ids = [
7968         fact_id
7969         for fact_id in support.fact_ids
7970         if fact_id not in facts
7971     ]
7972
7973     if unknown_fact_ids:
7974         add_annotation(
7975             annotations,
7976             sentence=sentence,
7977             text_span=sentence,
7978             error_type=ErrorType.NOT_CHECKABLE,
7979             subtype="unknown_fact_id",
7980             severity=Severity.HIGH,
7981             explanation=(
7982                 "The support map references facts not present in the verified ledger."
7983             ),
7984             correction_goal="Use valid verified facts.",
7985             fact_ids=unknown_fact_ids,
7986         )
7987         continue
7988
7989     supporting_facts = [
7990         facts[fact_id]
7991         for fact_id in support.fact_ids
7992     ]
7993
7994     supporting_evidence_ids = {
7995         *support.evidence_ids,
7996         *[
7997             evidence_id
7998             for fact in supporting_facts
7999             for evidence_id in fact.evidence_ids
8000         ],
8001     }
8002     supporting_evidence_items = [
8003         evidence_lookup_by_id[evidence_id]
8004         for evidence_id in supporting_evidence_ids
8005         if evidence_id in evidence_lookup_by_id
8006     ]
8007     if not score_state_tie_claim_supported(
8008         sentence,
8009         supporting_evidence_items,
8010     ):
8011         add_annotation(
8012             annotations,
8013             sentence=sentence,
8014             text_span=sentence,
8015             error_type=ErrorType.CONTEXT_ERROR,
8016             subtype="unsupported_event_score_state",
8017             severity=Severity.HIGH,
8018             explanation=(
8019                 "The sentence uses tying or score-level wording without "
8020                 "mapped event-sequence evidence showing that the referenced "
8021                 "event left the score tied."
8022             ),
8023             correction_goal=(
8024                 "Remove the tying wording or map the sentence to exact "
8025                 "event-sequence evidence whose score state supports it."
8026             ),
8027             fact_ids=support.fact_ids,
8028             evidence_ids=sorted(supporting_evidence_ids),
8029             insight_ids=support.insight_ids,
8030             confidence=0.95,
8031         )
8032
8033     for conflict in qualitative_strength_conflicts(

```

```

8034         sentence,
8035         supporting_evidence_items,
8036     ):
8037         add_annotation(
8038             annotations,
8039             sentence=sentence,
8040             text_span=sentence,
8041             error_type=ErrorType.CONTEXT_ERROR,
8042             subtype="inconsistent_strength_label",
8043             severity=Severity.HIGH,
8044             explanation=(
8045                 "The qualitative relationship strength conflicts with "
8046                 f"the mapped deterministic evidence: {conflict}."
8047             ),
8048             correction_goal=(
8049                 "Use the qualitative strength classification recorded in "
8050                 "the mapped evidence."
8051             ),
8052             fact_ids=support.fact_ids,
8053             evidence_ids=sorted(supporting_evidence_ids),
8054             insight_ids=support.insight_ids,
8055         )
8056
8057     explanatory_hypothesis = (
8058         EXPLANATORY_HYPOTHESIS_PATTERN.search(sentence)
8059         or UNLABELLED_HYPOTHESIS_PATTERN.search(sentence)
8060     )
8061     if (
8062         explanatory_hypothesis
8063         and support.interpretation_level
8064         != InterpretationLevel.HYPOTHESIS
8065     ):
8066         add_annotation(
8067             annotations,
8068             sentence=sentence,
8069             text_span=explanatory_hypothesis.group(0),
8070             error_type=ErrorType.CONTEXT_ERROR,
8071             subtype="unlabelled_hypothesis",
8072             severity=Severity.HIGH,
8073             explanation=(
8074                 "The sentence proposes a possible explanation without "
8075                 "verified hypothesis provenance."
8076             ),
8077             correction_goal=(
8078                 "Remove the explanation or present it as an explicitly "
8079                 "labelled hypothesis in the permitted section when enabled."
8080             ),
8081             fact_ids=support.fact_ids,
8082             evidence_ids=support.evidence_ids,
8083             insight_ids=support.insight_ids,
8084         )
8085
8086     unknown_insight_ids = [
8087         insight_id
8088         for insight_id in support.insight_ids
8089         if insight_id not in all_insight_ids
8090     ]
8091     if unknown_insight_ids:
8092         add_annotation(
8093             annotations,
8094             sentence=sentence,
8095             text_span=sentence,
8096             error_type=ErrorType.NOT_CHECKABLE,
8097             subtype="unsupported_insight",
8098             severity=Severity.HIGH,
8099             explanation=(
8100                 "The support map references insight IDs that are absent "
8101                 "from the verified Insight Ledger."
8102             ),
8103             correction_goal=(
8104                 "Remove the unsupported interpretation or map it to an "
8105                 "existing verified insight."
8106             ),
8107             fact_ids=support.fact_ids,
8108             evidence_ids=support.evidence_ids,
8109             insight_ids=unknown_insight_ids,
8110         )
8111
8112     mapped_verified_insights = [
8113         verified_insights[insight_id]
8114         for insight_id in support.insight_ids
8115         if insight_id in verified_insights
8116     ]
8117     mapped_hypotheses = [
8118         hypothesis_insights[insight_id]
8119         for insight_id in support.insight_ids
8120         if insight_id in hypothesis_insights
8121     ]
8122
8123     if (
8124         support.interpretation_level

```

```

8125 == InterpretationLevel.BOUNDED_INSIGHT
8126 ):
8127     if not support.insight_ids:
8128         add_annotation(
8129             annotations,
8130             sentence=sentence,
8131             text_span=sentence,
8132             error_type=ErrorType.NOT_CHECKABLE,
8133             subtype="unsupported_insight",
8134             severity=Severity.HIGH,
8135             explanation=(
8136                 "The sentence presents a bounded interpretation but "
8137                 "has no verified insight provenance."
8138             ),
8139             correction_goal=(
8140                 "Map the sentence to an exact verified insight or "
8141                 "rewrite it as directly supported findings."
8142             ),
8143             fact_ids=support.fact_ids,
8144             evidence_ids=support.evidence_ids,
8145         )
8146     elif len(mapped_verified_insights) != len(
8147         support.insight_ids
8148     ):
8149         add_annotation(
8150             annotations,
8151             sentence=sentence,
8152             text_span=sentence,
8153             error_type=ErrorType.CONTEXT_ERROR,
8154             subtype="unsupported_insight",
8155             severity=Severity.HIGH,
8156             explanation=(
8157                 "A bounded-insight sentence may cite only insights "
8158                 "verified for the main report."
8159             ),
8160             correction_goal=(
8161                 "Use a verified main insight or remove the "
8162                 "interpretive claim."
8163             ),
8164             fact_ids=support.fact_ids,
8165             evidence_ids=support.evidence_ids,
8166             insight_ids=support.insight_ids,
8167         )
8168
8169     required_fact_ids = {
8170         fact_id
8171         for insight in mapped_verified_insights
8172         for fact_id in insight.source_fact_ids
8173     }
8174     required_evidence_ids = {
8175         evidence_id
8176         for insight in mapped_verified_insights
8177         for evidence_id in insight.source_evidence_ids
8178     }
8179     if (
8180         mapped_verified_insights
8181         and (
8182             not required_fact_ids.issubset(
8183                 set(support.fact_ids)
8184             )
8185             or not required_evidence_ids.issubset(
8186                 set(support.evidence_ids)
8187             )
8188         )
8189     ):
8190         add_annotation(
8191             annotations,
8192             sentence=sentence,
8193             text_span=sentence,
8194             error_type=ErrorType.SUPPORT_MAPPING_ERROR,
8195             subtype="insight_missing_source_support",
8196             severity=Severity.HIGH,
8197             explanation=(
8198                 "The bounded insight is mapped without all of its "
8199                 "verified source facts and evidence."
8200             ),
8201             correction_goal=(
8202                 "Restore the verified insight's source fact and "
8203                 "evidence provenance in the hidden support map."
8204             ),
8205             fact_ids=support.fact_ids,
8206             evidence_ids=support.evidence_ids,
8207             insight_ids=support.insight_ids,
8208         )
8209
8210     overstatement = INSIGHT_OVERSTATEMENT_PATTERN.search(sentence)
8211     if overstatement:
8212         add_annotation(
8213             annotations,
8214             sentence=sentence,
8215             text_span=overstatement.group(0),

```

```

8216         error_type=ErrorType.CONTEXT_ERROR,
8217         subtype="insight_exceeds_verified_wording",
8218         severity=Severity.HIGH,
8219         explanation=(
8220             "The report wording is stronger than the bounded "
8221             "interpretation authorised by the Insight Ledger."
8222         ),
8223         correction_goal=(
8224             "Use the verified dataset-scoped insight wording."
8225         ),
8226         fact_ids=support.fact_ids,
8227         evidence_ids=support.evidence_ids,
8228         insight_ids=support.insight_ids,
8229     )
8230
8231     if (
8232         support.interpretation_level
8233         == InterpretationLevel.HYPOTHESIS
8234     ):
8235         if not settings.allow_hypotheses_in_report:
8236             add_annotation(
8237                 annotations,
8238                 sentence=sentence,
8239                 text_span=sentence,
8240                 error_type=ErrorType.CONTEXT_ERROR,
8241                 subtype="hypothesis_presented_as_conclusion",
8242                 severity=Severity.HIGH,
8243                 explanation=(
8244                     "Hypotheses are disabled for this report."
8245                 ),
8246                 correction_goal="Remove the hypothesis from the report.",
8247                 fact_ids=support.fact_ids,
8248                 evidence_ids=support.evidence_ids,
8249                 insight_ids=support.insight_ids,
8250             )
8251         if not mapped_hypotheses or len(mapped_hypotheses) != len(
8252             support.insight_ids
8253         ):
8254             add_annotation(
8255                 annotations,
8256                 sentence=sentence,
8257                 text_span=sentence,
8258                 error_type=ErrorType.NOT_CHECKABLE,
8259                 subtype="unlabelled_hypothesis",
8260                 severity=Severity.HIGH,
8261                 explanation=(
8262                     "The sentence lacks valid hypothesis-only provenance."
8263                 ),
8264                 correction_goal=(
8265                     "Map it to a hypothesis-only ledger entry or remove it."
8266                 ),
8267                 fact_ids=support.fact_ids,
8268                 evidence_ids=support.evidence_ids,
8269                 insight_ids=support.insight_ids,
8270             )
8271         if (
8272             not HYPOTHESIS_WORDING_PATTERN.search(sentence)
8273             and not sentence.strip().endswith("?")
8274         ):
8275             add_annotation(
8276                 annotations,
8277                 sentence=sentence,
8278                 text_span=sentence,
8279                 error_type=ErrorType.CONTEXT_ERROR,
8280                 subtype="unlabelled_hypothesis",
8281                 severity=Severity.HIGH,
8282                 explanation=(
8283                     "A hypothesis must be explicitly labelled rather than "
8284                     "presented as established knowledge."
8285                 ),
8286                 correction_goal="Label the statement explicitly as a hypothesis.",
8287                 fact_ids=support.fact_ids,
8288                 evidence_ids=support.evidence_ids,
8289                 insight_ids=support.insight_ids,
8290             )
8291         if section_headings.get(sentence, "").lower() != (
8292             "questions for further investigation"
8293         ):
8294             add_annotation(
8295                 annotations,
8296                 sentence=sentence,
8297                 text_span=sentence,
8298                 error_type=ErrorType.CONTEXT_ERROR,
8299                 subtype="hypothesis_presented_as_conclusion",
8300                 severity=Severity.HIGH,
8301                 explanation=(
8302                     "Hypotheses may appear only in the separate Questions "
8303                     "for Further Investigation section."
8304                 ),
8305                 correction_goal=(
8306                     "Move the labelled hypothesis to the permitted section."

```

```

8307         ),
8308         fact_ids=support.fact_ids,
8309         evidence_ids=support.evidence_ids,
8310         insight_ids=support.insight_ids,
8311     )
8312
8313     if (
8314         support.interpretation_level
8315         == InterpretationLevel.FINDING
8316     ):
8317         if support.insight_ids:
8318             add_annotation(
8319                 annotations,
8320                 sentence=sentence,
8321                 text_span=sentence,
8322                 error_type=ErrorType.SUPPORT_MAPPING_ERROR,
8323                 subtype="single_fact_relabelled_as_insight",
8324                 severity=Severity.MEDIUM,
8325                 explanation=(
8326                     "A direct finding is incorrectly mapped as an insight."
8327                 ),
8328                 correction_goal=(
8329                     "Remove the insight mapping or mark a genuinely "
8330                     "verified bounded interpretation."
8331                 ),
8332                 fact_ids=support.fact_ids,
8333                 evidence_ids=support.evidence_ids,
8334                 insight_ids=support.insight_ids,
8335             )
8336         unknown_profile_ids = [
8337             support_id
8338             for support_id in support.profile_support_ids
8339             if support_id not in profile_records_by_id
8340         ]
8341
8342         if unknown_profile_ids:
8343             add_annotation(
8344                 annotations,
8345                 sentence=sentence,
8346                 text_span=sentence,
8347                 error_type=ErrorType.NOT_CHECKABLE,
8348                 subtype="unknown_profile_support_id",
8349                 severity=Severity.HIGH,
8350                 explanation=(
8351                     "The support map references deterministic profile support "
8352                     "not present in the profile registry."
8353                 ),
8354                 correction_goal="Use valid deterministic profile support IDs.",
8355                 fact_ids=support.fact_ids,
8356                 evidence_ids=support.evidence_ids,
8357                 profile_support_ids=unknown_profile_ids,
8358             )
8359             continue
8360
8361         attached_profile_supports = _profile_records_support_sentence(
8362             sentence=sentence,
8363             support_ids=support.profile_support_ids,
8364             records_by_id=profile_records_by_id,
8365         )
8366
8367         fact_numbers = [
8368             number
8369             for fact in supporting_facts
8370             for number in fact_support_numbers(fact, evidence)
8371         ]
8372
8373         attached_profile_numbers = [
8374             number
8375             for support_id in support.profile_support_ids
8376             if support_id in profile_records_by_id
8377             for number in profile_record_numbers(
8378                 profile_records_by_id[support_id]
8379             )
8380         ]
8381
8382         combined_numbers = [
8383             *fact_numbers,
8384             *(
8385                 attached_profile_numbers
8386                 if attached_profile_supports
8387                 else []
8388             ),
8389         ]
8390
8391         profile_matches = matching_profile_support_records(
8392             sentence=sentence,
8393             records=profile_support_records,
8394         )
8395
8396         if not numbers_supported(sentence, combined_numbers):
8397             if profile_matches:

```

```

8398         matched_ids = [
8399             record.support_id
8400             for record in profile_matches
8401         ]
8402     add_annotation(
8403         annotations,
8404         sentence=sentence,
8405         text_span=sentence,
8406         error_type=ErrorType.SUPPORT_MAPPING_ERROR,
8407         subtype=(
8408             "deterministic_profile_support_missing_from_map"
8409         ),
8410         severity=Severity.MEDIUM,
8411         explanation=(
8412             "The visible statement is supported by deterministic "
8413             "profile data, but that support is missing from the "
8414             "hidden sentence support map."
8415         ),
8416         correction_goal=(
8417             "Attach the relevant deterministic profile support "
8418             "IDs to the hidden sentence support map."
8419         ),
8420         fact_ids=support.fact_ids,
8421         evidence_ids=support.evidence_ids,
8422         profile_support_ids=matched_ids,
8423         confidence=1.0,
8424     )
8425     support_map_patches.append(
8426         SupportMapPatch(
8427             sentence_id=support.sentence_id,
8428             sentence_text=support.sentence_text,
8429             added_profile_support_ids=matched_ids,
8430             reason=(
8431                 "Deterministic profile data exactly supports "
8432                 "the visible structural statement."
8433             ),
8434         )
8435     )
8436 else:
8437     add_annotation(
8438         annotations,
8439         sentence=sentence,
8440         text_span=sentence,
8441         error_type=ErrorType.INCORRECT_NUMBER,
8442         subtype="unsupported_number",
8443         severity=Severity.HIGH,
8444         explanation=(
8445             "One or more numbers are not supported by the mapped "
8446             "verified facts or deterministic profile support."
8447         ),
8448         correction_goal=(
8449             "Use a supported exact or appropriately qualified "
8450             "rounded value."
8451         ),
8452         fact_ids=support.fact_ids,
8453         evidence_ids=support.evidence_ids,
8454         profile_support_ids=support.profile_support_ids,
8455     )
8456 elif (
8457     profile_matches
8458     and not attached_profile_supports
8459     and not _mapped_facts_semantically_supported_profile_match(
8460         support_text=_support_text_for_facts(
8461             supporting_facts,
8462             evidence,
8463         ),
8464         matches=profile_matches,
8465     )
8466 ):
8467     matched_ids = [
8468         record.support_id
8469         for record in profile_matches
8470     ]
8471     add_annotation(
8472         annotations,
8473         sentence=sentence,
8474         text_span=sentence,
8475         error_type=ErrorType.SUPPORT_MAPPING_ERROR,
8476         subtype="deterministic_profile_support_missing_from_map",
8477         severity=Severity.MEDIUM,
8478         explanation=(
8479             "The visible structural statement is supported by "
8480             "deterministic profile data, but the mapped verified facts "
8481             "do not provide that provenance."
8482         ),
8483         correction_goal=(
8484             "Attach the relevant deterministic profile support IDs "
8485             "to the hidden sentence support map."
8486         ),
8487         fact_ids=support.fact_ids,
8488         evidence_ids=support.evidence_ids,

```

```

8489         profile_support_ids=matched_ids,
8490         confidence=1.0,
8491     )
8492     support_map_patches.append(
8493         SupportMapPatch(
8494             sentence_id=support.sentence_id,
8495             sentence_text=support.sentence_text,
8496             added_profile_support_ids=matched_ids,
8497             reason=(
8498                 "Deterministic profile data exactly supports the "
8499                 "visible structural statement."
8500             ),
8501         )
8502     )
8503
8504     permissions = {
8505         permission
8506         for fact in supporting_facts
8507         for permission in fact.claim_permissions
8508     }
8509     mapped_interpretations = [
8510         *mapped_verified_insights,
8511         *mapped_hypotheses,
8512     ]
8513     mapped_insight_text = " ".join(
8514         insight.statement
8515         for insight in mapped_interpretations
8516     )
8517
8518     if IMBALANCE_BIAS_PATTERN.search(sentence):
8519         support_text = " ".join(
8520             [
8521                 sentence,
8522                 *[fact.fact_summary for fact in supporting_facts],
8523                 *[
8524                     item.practical_interpretation
8525                     for fact in supporting_facts
8526                     for item in evidence_for_fact(
8527                         fact,
8528                         evidence_lookup_by_id,
8529                     )
8530                 ],
8531                 *[
8532                     limitation
8533                     for fact in supporting_facts
8534                     for item in evidence_for_fact(
8535                         fact,
8536                         evidence_lookup_by_id,
8537                     )
8538                     for limitation in getattr(item, "limitations", [])
8539                 ],
8540             ]
8541         ).lower()
8542
8543     if "sampling bias" not in support_text and "selection bias" not in support_text:
8544         add_annotation(
8545             annotations,
8546             sentence=sentence,
8547             text_span=sentence,
8548             error_type=ErrorType.CONTEXT_ERROR,
8549             subtype="unsupported_methodological_interpretation",
8550             severity=Severity.MEDIUM,
8551             explanation=(
8552                 "The evidence can support noting that unequal group sizes "
8553                 "may affect precision, stability, or representation, but it "
8554                 "does not establish that the reported means are biased."
8555             ),
8556             correction_goal=(
8557                 "Replace bias wording with a precision, stability, or "
8558                 "representation caveat grounded in the evidence."
8559             ),
8560             fact_ids=support.fact_ids,
8561             evidence_ids=support.evidence_ids,
8562             confidence=0.85,
8563         )
8564
8565     if (
8566         CAUSAL_PATTERN.search(sentence)
8567         and not negative_causal(sentence)
8568         and (
8569             ClaimPermission.CAUSAL not in permissions
8570             or (
8571                 support.interpretation_level
8572                 != InterpretationLevel.FINDING
8573                 and not CAUSAL_PATTERN.search(
8574                     mapped_insight_text
8575                 )
8576             )
8577         )
8578     ):
8579         add_annotation(

```

```

8580         annotations,
8581         sentence=sentence,
8582         text_span=sentence,
8583         error_type=ErrorType.CONTEXT_ERROR,
8584         subtype=(
8585             "unsupported_causal_interpretation"
8586             if support.interpretation_level
8587             != InterpretationLevel.FINDING
8588             else "causal_overclaim"
8589         ),
8590         severity=Severity.CRITICAL,
8591         explanation=(
8592             "The sentence uses causal language without verified causal permission."
8593         ),
8594         correction_goal=(
8595             "Rewrite as association or explicitly state that causality "
8596             "was not established."
8597         ),
8598         fact_ids=support.fact_ids,
8599         evidence_ids=support.evidence_ids,
8600         insight_ids=support.insight_ids,
8601     )
8602
8603     if (
8604         PREDICTIVE_PATTERN.search(sentence)
8605         and not negative_predictive(sentence)
8606         and (
8607             ClaimPermission.PREDICTIVE not in permissions
8608             or (
8609                 support.interpretation_level
8610                 != InterpretationLevel.FINDING
8611                 and not PREDICTIVE_PATTERN.search(
8612                     mapped_insight_text
8613                 )
8614             )
8615         )
8616     ):
8617         add_annotation(
8618             annotations,
8619             sentence=sentence,
8620             text_span=sentence,
8621             error_type=ErrorType.CONTEXT_ERROR,
8622             subtype="predictive_overclaim",
8623             severity=Severity.HIGH,
8624             explanation=(
8625                 "Predictive wording is used without validated predictive permission."
8626             ),
8627             correction_goal=(
8628                 "Describe the feasibility limitation or validated internal result accurately."
8629             ),
8630             fact_ids=support.fact_ids,
8631             evidence_ids=support.evidence_ids,
8632             insight_ids=support.insight_ids,
8633         )
8634
8635     if (
8636         FORECAST_PATTERN.search(sentence)
8637         and not negative_forecast(sentence)
8638         and (
8639             ClaimPermission.FORECAST not in permissions
8640             or (
8641                 support.interpretation_level
8642                 != InterpretationLevel.FINDING
8643                 and not FORECAST_PATTERN.search(
8644                     mapped_insight_text
8645                 )
8646             )
8647         )
8648     ):
8649         add_annotation(
8650             annotations,
8651             sentence=sentence,
8652             text_span=sentence,
8653             error_type=ErrorType.CONTEXT_ERROR,
8654             subtype="forecast_overclaim",
8655             severity=Severity.HIGH,
8656             explanation=(
8657                 "Forecast wording is used without validated forecast permission."
8658             ),
8659             correction_goal=(
8660                 "Describe the backtest or insufficiency finding without "
8661                 "claiming unsupported future performance."
8662             ),
8663             fact_ids=support.fact_ids,
8664             evidence_ids=support.evidence_ids,
8665             insight_ids=support.insight_ids,
8666         )
8667
8668     generalisation_match = DATASET_GENERALISATION_PATTERN.search(
8669         sentence
8670     )

```



```

8671     if (
8672         generalisation_match
8673         and not DATASET_GENERALISATION_PATTERN.search(
8674             mapped_insight_text
8675         )
8676     ):
8677         add_annotation(
8678             annotations,
8679             sentence=sentence,
8680             text_span=generalisation_match.group(0),
8681             error_type=ErrorType.CONTEXT_ERROR,
8682             subtype="insight_exceeds_verified_wording",
8683             severity=Severity.HIGH,
8684             explanation=(
8685                 "The sentence generalises a dataset-scoped finding or "
8686                 "insight beyond the analysed data."
8687             ),
8688             correction_goal=(
8689                 "Scope the statement to this dataset and the verified "
8690                 "interpretation."
8691             ),
8692             fact_ids=support.fact_ids,
8693             evidence_ids=support.evidence_ids,
8694             insight_ids=support.insight_ids,
8695         )
8696
8697     sports_narrative_match = (
8698         UNSUPPORTED_SPORTS_NARRATIVE_PATTERN.search(
8699             sentence
8700         )
8701     )
8702     support_text_for_narrative = (
8703         _support_text_for_facts(
8704             supporting_facts,
8705             evidence,
8706         )
8707         + " "
8708         + mapped_insight_text.lower()
8709     )
8710     if (
8711         sports_narrative_match
8712         and not UNSUPPORTED_SPORTS_NARRATIVE_PATTERN.search(
8713             support_text_for_narrative
8714         )
8715     ):
8716         add_annotation(
8717             annotations,
8718             sentence=sentence,
8719             text_span=sports_narrative_match.group(0),
8720             error_type=ErrorType.CONTEXT_ERROR,
8721             subtype="unsupported_sports_narrative",
8722             severity=Severity.HIGH,
8723             explanation=(
8724                 "The sports narrative introduces chronology, evaluation, "
8725                 "or historical significance absent from verified support."
8726             ),
8727             correction_goal=(
8728                 "Use only the supported result, performances and team "
8729                 "contrasts without invented narrative context."
8730             ),
8731             fact_ids=support.fact_ids,
8732             evidence_ids=support.evidence_ids,
8733             insight_ids=support.insight_ids,
8734         )
8735
8736     add_wording_guardrail_annotations(
8737         annotations=annotations,
8738         sentence=sentence,
8739         support=support,
8740         supporting_facts=supporting_facts,
8741         evidence=evidence,
8742         report_specification=report_specification,
8743     )
8744
8745     known_entities = {
8746         entity
8747         for fact in supporting_facts
8748         for entity in fact.entities
8749     }
8750     known_entities.update(
8751         entity
8752         for support_id in support.profile_support_ids
8753         if support_id in profile_records_by_id
8754         for entity in profile_records_by_id[
8755             support_id
8756         ].entities
8757     )
8758
8759     for backtick_entity in unsupported_backtick_entities(
8760         sentence,
8761         known_entities,

```

```

8762 ):
8763     add_annotation(
8764         annotations,
8765         sentence=sentence,
8766         text_span=backtick_entity,
8767         error_type=ErrorType.INCORRECT_NAMED_ENTITY,
8768         subtype="unsupported_backtick_entity",
8769         severity=Severity.HIGH,
8770         explanation=(
8771             "The named entity is not present in the mapped facts."
8772         ),
8773         correction_goal=(
8774             "Use an entity present in the verified facts or remove it."
8775         ),
8776         fact_ids=support.fact_ids,
8777         evidence_ids=support.evidence_ids,
8778     )
8779
8780     if not any(
8781         annotation.sentence == sentence
8782         for annotation in annotations
8783     ):
8784         supported_count += 1
8785
8786 word_count = len(
8787     re.findall(r"\b\[w'-]+\b", writer_output.markdown)
8788 )
8789
8790 quality_findings: list[str] = []
8791 quality_recommendations: list[str] = []
8792 methodological_warnings: list[str] = []
8793
8794 if INTERNAL_CONTROL_PATTERN.search(writer_output.markdown) or FIELD_LABEL_PATTERN.search(
8795     writer_output.markdown
8796 ):
8797     quality_findings.append(
8798         "The report exposes internal writer or auditor guardrails."
8799     )
8800     quality_recommendations.append(
8801         "Convert internal constraints into concise reader-facing caveats."
8802     )
8803
8804 if any(pattern.search(writer_output.markdown) for pattern in GENERIC_OPENING_PATTERNS):
8805     quality_findings.append(
8806         "The report opens with generic report boilerplate instead of dataset-specific substance."
8807     )
8808     quality_recommendations.append(
8809         "Start with a concrete dataset overview or leading supported finding."
8810     )
8811
8812 maximum_words = report_specification.maximum_length_words
8813 if maximum_words is not None and word_count > maximum_words:
8814     quality_findings.append(
8815         f"The report contains {word_count} words and exceeds the "
8816         f"{maximum_words}-word ceiling."
8817     )
8818     quality_recommendations.append(
8819         "Remove low-priority detail and consolidate methodological caveats."
8820     )
8821
8822 minimum_words = minimum_useful_report_words(
8823     target_words=report_specification.target_length_words,
8824     required_component_count=len(report_specification.required_components),
8825     settings=settings,
8826 )
8827 if word_count < minimum_words:
8828     quality_findings.append(
8829         f"The report contains {word_count} words, below the minimum useful "
8830         f"coverage threshold of {minimum_words}."
8831     )
8832     if report_specification.genre in {
8833         ReportGenre.EVENT_REPORT,
8834         ReportGenre.SPORTS_GAME_REPORT,
8835     }:
8836         coverage_target = (
8837             "the supported event result, context, leading performances, "
8838             "participant contrasts, and scope caveats"
8839         )
8840     else:
8841         coverage_target = (
8842             "the required dataset overview, quality, relationship, and "
8843             "limitation components"
8844         )
8845     quality_recommendations.append(
8846         f"Expand the report using verified facts covering {coverage_target}."
8847     )
8848
8849 content_requirements = build_writer_content_requirements(
8850     report_specification=report_specification,
8851     fact_ledger=fact_ledger,
8852     evidence=evidence,

```

```

8853         insight_ledger=insight_ledger,
8854         settings=settings,
8855     )
8856     for content_error in writer_output_content_requirement_errors(
8857         writer_output=writer_output,
8858         requirements=content_requirements,
8859         include_word_count=False,
8860         respect_validation_severity=False,
8861     ):
8862         quality_findings.append(content_error)
8863         quality_recommendations.append(
8864             "Use the controller-enforced content requirements to include "
8865             "the supported event material before adding lower-priority prose."
8866         )
8867
8868     configured_fact_limits = [
8869         limit
8870         for limit in {
8871             report_specification.maximum_main_findings,
8872             report_specification.maximum_supporting_facts,
8873         }
8874         if limit is not None
8875     ]
8876     if (
8877         configured_fact_limits
8878         and len(writer_output.selected_fact_ids)
8879         > max(configured_fact_limits)
8880     ):
8881         quality_findings.append(
8882             "The report uses more facts than the configured report budget."
8883         )
8884         quality_recommendations.append(
8885             "Prioritise headline findings and omit weaker supporting detail."
8886         )
8887
8888     used_verified_insight_ids = {
8889         insight_id
8890         for support in writer_output.sentence_support
8891         if support.interpretation_level
8892         == InterpretationLevel.BOUNDED_INSIGHT
8893         for insight_id in support.insight_ids
8894         if insight_id in verified_insights
8895     }
8896     if verified_insights and not used_verified_insight_ids:
8897         quality_findings.append(
8898             "The report lists findings without relating them through an "
8899             "available verified insight."
8900         )
8901         quality_recommendations.append(
8902             "Use at least one salient verified bounded insight in a main "
8903             "analytical paragraph."
8904         )
8905
8906     insights_without_implication = []
8907     for insight_id in used_verified_insight_ids:
8908         insight = verified_insights[insight_id]
8909         if (
8910             report_specification.genre
8911             in {
8912                 ReportGenre.EVENT_REPORT,
8913                 ReportGenre.SPORTS_GAME_REPORT,
8914             }
8915             or
8916             insight.contribution
8917             != InsightContribution.ANALYTICAL_IMPLICATION
8918         ):
8919             continue
8920     mapped_sentences = [
8921         support.sentence_text
8922         for support in writer_output.sentence_support
8923         if insight_id in support.insight_ids
8924     ]
8925     direct_source_texts = [
8926         insight.statement,
8927         *[
8928             facts[fact_id].fact_summary
8929             for fact_id in insight.source_fact_ids
8930             if fact_id in facts
8931         ],
8932     ]
8933     if mapped_sentences and all(
8934         any(
8935             materially_same_report_text(
8936                 sentence,
8937                 source_text,
8938             )
8939             for source_text in direct_source_texts
8940         )
8941         for sentence in mapped_sentences
8942     ):
8943         insights_without_implication.append(insight_id)

```

```

8944
8945 if insights_without_implication:
8946     quality_findings.append(
8947         "The report states a verified insight but does not explain its "
8948         "supported analytical implication."
8949     )
8950     quality_recommendations.append(
8951         "Use the verified `why_it_matters` content to explain why the "
8952         "combined findings matter instead of restating them."
8953     )
8954
8955 if verified_insights:
8956     most_salient_insight = max(
8957         verified_insights.values(),
8958         key=lambda insight: (
8959             insight.salience,
8960             insight.confidence,
8961         ),
8962     )
8963     if (
8964         most_salient_insight.insight_id
8965         not in used_verified_insight_ids
8966     ):
8967         quality_findings.append(
8968             "The report omits the most salient verified insight."
8969         )
8970         quality_recommendations.append(
8971             "Prioritise the most salient verified insight or explain the "
8972             "selection through stronger supported content."
8973         )
8974
8975 if (
8976     report_specification.genre
8977     in {
8978         ReportGenre.EVENT_REPORT,
8979         ReportGenre.SPORTS_GAME_REPORT,
8980     }
8981 ):
8982     sports_text = writer_output.markdown.lower()
8983     game_content = re.search(
8984         r"\b(won|winner|score|points?|rebounds?|turnovers?|team|player)\b",
8985         sports_text,
8986     )
8987     profile_dominant = bool(
8988         re.search(
8989             r"\b(columns?|schema|missingness|dtype|data type)\b",
8990             sports_text,
8991         )
8992         and not game_content
8993     )
8994     if not game_content or profile_dominant:
8995         quality_findings.append(
8996             "The event report reads like a dataset profile rather than "
8997             "communicating the supported result and performances."
8998         )
8999         quality_recommendations.append(
9000             "Prioritise the supported result, salient performances and "
9001             "team-level contrasts required by the selected genre."
9002         )
9003
9004 if any(
9005     annotation.subtype
9006     in {
9007         "unlabelled_hypothesis",
9008         "hypothesis_presented_as_conclusion",
9009     }
9010     for annotation in annotations
9011 ):
9012     quality_findings.append(
9013         "The report uses a hypothesis as a conclusion."
9014     )
9015     quality_recommendations.append(
9016         "Remove the hypothesis or label it in the permitted further-"
9017         "investigation section."
9018     )
9019
9020 repeated_caveat_count = count_caveat_mentions(writer_output.markdown)
9021
9022 if repeated_caveat_count > settings.maximum_repeated_caveat_mentions:
9023     quality_findings.append(
9024         "The same causal caveat is repeated several times."
9025     )
9026     quality_recommendations.append(
9027         "Consolidate recurring caveats at section level."
9028     )
9029
9030 component_assessments = assess_report_component_coverage(
9031     writer_output=writer_output,
9032     fact_ledger=fact_ledger,
9033     evidence=evidence,
9034     required_components=report_specification.required_components,

```

```

9035 )
9036 missing_required_components = False
9037 for assessment in component_assessments:
9038     if not assessment.covered:
9039         missing_required_components = True
9040         quality_findings.append(
9041             f"Required report component `{assessment.component.value}` is not clearly covered."
9042         )
9043         quality_recommendations.append(
9044             "Revise the report to cover the required component using verified facts."
9045         )
9046
9047 fact_text = " ".join(
9048     fact.fact_summary.lower()
9049     + " "
9050     + " ".join(entity.lower() for entity in fact.entities)
9051     for fact in fact_ledger.writer_ready_facts
9052 )
9053 allowed_unit_terms = set()
9054 if report_specification.genre in {
9055     ReportGenre.EVENT_REPORT,
9056     ReportGenre.SPORTS_GAME_REPORT,
9057 }:
9058     allowed_unit_terms.update({"event", "events"})
9059
9060 unsupported_unit_terms = [
9061     term
9062     for term in UNSANCTIONED_UNIT_TERMS
9063     if term not in allowed_unit_terms
9064     if re.search(rf"\b{re.escape(term)}\b", writer_output.markdown, re.IGNORECASE)
9065     and term not in fact_text
9066 ]
9067 if unsupported_unit_terms:
9068     quality_findings.append(
9069         "The report may substitute an unsupported unit of observation: "
9070         + ", ".join(sorted(set(unsupported_unit_terms)))
9071         + "."
9072     )
9073     quality_recommendations.append(
9074         "Use neutral unit wording unless verified facts or deterministic "
9075         "profile support establish a more specific unit."
9076     )
9077
9078 for annotation in annotations:
9079     if annotation.subtype == "unsupported_methodological_interpretation":
9080         methodological_warnings.append(annotation.explanation)
9081
9082 quality = ReportQualityAssessment(
9083     status=(
9084         QualityStatus.REVISE
9085         if missing_required_components
9086         else (
9087             QualityStatus.WARNING
9088             if quality_findings
9089             else QualityStatus.PASS
9090         )
9091     ),
9092     request_responsiveness=0.8 if quality_findings else 1.0,
9093     finding_selection=0.7 if quality_findings else 1.0,
9094     coherence=0.9,
9095     concision=0.7 if quality_findings else 1.0,
9096     caveat_integration=(
9097         0.6
9098         if repeated_caveat_count > settings.maximum_repeated_caveat_mentions
9099         else 1.0
9100     ),
9101     data_science_interpretation=0.9,
9102     findings=quality_findings,
9103     recommendations=quality_recommendations,
9104 )
9105
9106 serious = any(
9107     annotation_requires_repair(annotation)
9108     for annotation in annotations
9109 )
9110
9111 if mode == AuditMode.ANNOTATION_ONLY:
9112     decision = AuditDecision.PASS
9113 elif serious:
9114     decision = AuditDecision.REVISE
9115 else:
9116     decision = AuditDecision.PASS
9117
9118 release_status = decide_release_status(
9119     annotations=annotations,
9120     quality=quality,
9121     methodological_warnings=methodological_warnings,
9122     repair_budget_exhausted=False,
9123     audit_mode=mode,
9124 )
9125

```

```

9126     return AuditReport(
9127         mode=mode,
9128         decision=decision,
9129         release_status=release_status,
9130         annotations=annotations,
9131         applied_patches=[],
9132         support_map_patches=support_map_patches,
9133         factual_sentence_count=factual_count,
9134         supported_sentence_count=supported_count,
9135         support_rate=(
9136             supported_count / factual_count
9137             if factual_count
9138             else 1.0
9139         ),
9140         residual_risk=(
9141             "High-confidence factual issues require repair."
9142             if serious
9143             else (
9144                 "No high-confidence factual error was detected; "
9145                 "residual writer and auditor error remain possible."
9146             )
9147         ),
9148         revision_instructions=list(
9149             dict.fromkeys(
9150                 annotation.correction_goal
9151                 for annotation in annotations
9152             )
9153         ),
9154         quality_assessment=quality,
9155         revision_round=revision_round,
9156         component_assessments=component_assessments,
9157         methodological_warnings=methodological_warnings,
9158     )
9159
9160 def _ordered_dedupe(
9161     values: list[str],
9162 ) -> list[str]:
9163     return list(
9164         dict.fromkeys(
9165             value
9166             for value in values
9167             if value
9168         )
9169     )
9170
9171
9172 def merge_quality_assessments(
9173     deterministic: ReportQualityAssessment,
9174     semantic: ReportQualityAssessment,
9175 ) -> ReportQualityAssessment:
9176     status = max(
9177         [deterministic.status, semantic.status],
9178         key=lambda item: QUALITY_STATUS_ORDER[item],
9179     )
9180
9181     return ReportQualityAssessment(
9182         status=status,
9183         request_responsiveness=min(
9184             deterministic.request_responsiveness,
9185             semantic.request_responsiveness,
9186         ),
9187         finding_selection=min(
9188             deterministic.finding_selection,
9189             semantic.finding_selection,
9190         ),
9191         coherence=min(
9192             deterministic.coherence,
9193             semantic.coherence,
9194         ),
9195         concision=min(
9196             deterministic.concision,
9197             semantic.concision,
9198         ),
9199         caveat_integration=min(
9200             deterministic.caveat_integration,
9201             semantic.caveat_integration,
9202         ),
9203         data_science_interpretation=min(
9204             deterministic.data_science_interpretation,
9205             semantic.data_science_interpretation,
9206         ),
9207         findings=_ordered_dedupe(
9208             deterministic.findings
9209             + semantic.findings
9210         ),
9211         recommendations=_ordered_dedupe(
9212             deterministic.recommendations
9213             + semantic.recommendations
9214         ),
9215     )
9216

```

```

9217
9218
9219 def merge_audit_proposal(
9220     deterministic: AuditReport,
9221     proposal: AuditRepairProposal,
9222 ) -> AuditReport:
9223     annotations = list(deterministic.annotations)
9224
9225     for proposed in proposal.annotations:
9226         duplicate = any(
9227             existing.sentence == proposed.sentence
9228             and existing.text_span == proposed.text_span
9229             and existing.subtype == proposed.subtype
9230             for existing in annotations
9231         )
9232
9233         if duplicate:
9234             continue
9235
9236         annotations.append(
9237             proposed.model_copy(
9238                 update={
9239                     "annotation_id": f"ANN_{len(annotations) + 1:04d}"
9240                 }
9241             )
9242         )
9243
9244     merged_quality = merge_quality_assessments(
9245         deterministic.quality_assessment,
9246         proposal.quality_assessment,
9247     )
9248
9249     serious = any(
9250         annotation_requires_repair(annotation)
9251         for annotation in annotations
9252     )
9253
9254     if deterministic.mode == AuditMode.ANNOTATION_ONLY:
9255         decision = AuditDecision.PASS
9256     elif serious:
9257         decision = AuditDecision.REVISE
9258     else:
9259         decision = AuditDecision.PASS
9260
9261     release_status = decide_release_status(
9262         annotations=annotations,
9263         quality=merged_quality,
9264         methodological_warnings=deterministic.methodological_warnings,
9265         repair_budget_exhausted=False,
9266         audit_mode=deterministic.mode,
9267     )
9268
9269     return deterministic.model_copy(
9270         update={
9271             "annotations": annotations,
9272             "decision": decision,
9273             "release_status": release_status,
9274             "residual_risk": " ".join(
9275                 _ordered_dedupe(
9276                     [
9277                         deterministic.residual_risk,
9278                         proposal.residual_risk,
9279                     ]
9280                 )
9281             ),
9282             "revision_instructions": _ordered_dedupe(
9283                 deterministic.revision_instructions
9284                 + proposal.revision_instructions
9285             ),
9286             "quality_assessment": merged_quality,
9287             "methodological_warnings": deterministic.methodological_warnings,
9288             "component_assessments": deterministic.component_assessments,
9289             "support_map_patches": deterministic.support_map_patches,
9290         }
9291     )
9292
9293
9294
9295 def fallback_audit_proposal(
9296     deterministic: AuditReport,
9297 ) -> AuditRepairProposal:
9298     return AuditRepairProposal(
9299         annotations=[],
9300         repairs=[],
9301         recommended_decision=deterministic.decision,
9302         residual_risk=(
9303             "Semantic auditing was unavailable. "
9304             + deterministic.residual_risk
9305         ),
9306         quality_assessment=deterministic.quality_assessment,
9307         revision_instructions=deterministic.revision_instructions,

```

```

9308     )
9309
9310
9311 def repair_score(candidate: RepairCandidate) -> float:
9312     return (
9313         0.45 * candidate.factual_support_score
9314         + 0.25 * candidate.meaning_preservation_score
9315         + 0.20 * candidate.readability_score
9316         - 0.10 * candidate.residual_hallucination_risk
9317     )
9318
9319
9320 def validate_repair_candidate(
9321     candidate: RepairCandidate,
9322     fact_ledger: FactLedger,
9323     evidence: EvidenceLedger,
9324     insight_ledger: InsightLedger | None = None,
9325     allow_hypotheses_in_report: bool = False,
9326     original_text: str | None = None,
9327 ) -> list[str]:
9328     errors: list[str] = []
9329     insight_ledger = insight_ledger or InsightLedger(
9330         synthesis_enabled=False
9331     )
9332     facts = {
9333         fact.fact_id: fact
9334         for fact in fact_ledger.writer_ready_facts
9335     }
9336     verified_insights = {
9337         insight.insight_id: insight
9338         for insight in insight_ledger.verified_insights
9339     }
9340     hypothesis_insights = {
9341         insight.insight_id: insight
9342         for insight in insight_ledger.hypothesis_only_insights
9343     }
9344     all_insights = {
9345         **verified_insights,
9346         **hypothesis_insights,
9347     }
9348
9349     unknown_insights = [
9350         insight_id
9351         for insight_id in candidate.supporting_insight_ids
9352         if insight_id not in all_insights
9353     ]
9354     if unknown_insights:
9355         errors.append(
9356             f"Unknown repair insight IDs: {unknown_insights}"
9357         )
9358     return errors
9359
9360     if (
9361         set(candidate.supporting_insight_ids)
9362         & set(hypothesis_insights)
9363         and not allow_hypotheses_in_report
9364     ):
9365         errors.append(
9366             "The replacement introduces a hypothesis while hypotheses are "
9367             "disabled."
9368         )
9369
9370     expanded_fact_ids = list(
9371         dict.fromkeys(
9372             [
9373                 *candidate.supporting_fact_ids,
9374                 *[
9375                     fact_id
9376                     for insight_id in candidate.supporting_insight_ids
9377                     if insight_id in all_insights
9378                     for fact_id in all_insights[
9379                         insight_id
9380                     ].source_fact_ids
9381                 ],
9382             ],
9383         )
9384     )
9385
9386     unknown = [
9387         fact_id
9388         for fact_id in expanded_fact_ids
9389         if fact_id not in facts
9390     ]
9391
9392     if unknown:
9393         errors.append(f"Unknown repair fact IDs: {unknown}")
9394     return errors
9395
9396     if candidate.strategy == RepairStrategy.DELETE:
9397         if candidate.replacement_text.strip():
9398             errors.append(

```



```

9399         "Delete repairs must use an empty replacement_text."
9400     )
9401     return errors
9402
9403 if not candidate.replacement_text.strip():
9404     errors.append("Non-delete repairs require replacement text.")
9405     return errors
9406
9407 if (
9408     original_text is not None
9409     and " ".join(candidate.replacement_text.split())
9410     == " ".join(original_text.split())
9411 ):
9412     errors.append(
9413         "The replacement is identical to the original sentence."
9414     )
9415     return errors
9416
9417 supporting_facts = [
9418     facts[fact_id]
9419     for fact_id in expanded_fact_ids
9420 ]
9421
9422 replacement_hypothesis = EXPLANATORY_HYPOTHESIS_PATTERN.search(
9423     candidate.replacement_text
9424 )
9425 supporting_hypothesis_ids = set(
9426     candidate.supporting_insight_ids
9427 ) & set(hypothesis_insights)
9428 if replacement_hypothesis:
9429     if (
9430         not allow_hypotheses_in_report
9431         or not supporting_hypothesis_ids
9432     ):
9433         errors.append(
9434             "The replacement introduces an explanatory hypothesis "
9435             "without permitted hypothesis-only provenance."
9436         )
9437     elif (
9438         not HYPOTHESIS_WORDING_PATTERN.search(
9439             candidate.replacement_text
9440         )
9441         and not candidate.replacement_text.strip().endswith("?")
9442     ):
9443         errors.append(
9444             "The replacement does not explicitly label its hypothesis."
9445         )
9446
9447 supporting_evidence_ids = {
9448     evidence_id
9449     for fact in supporting_facts
9450     for evidence_id in fact.evidence_ids
9451 }
9452 evidence_lookup = build_evidence_lookup(evidence)
9453 strength_conflicts = qualitative_strength_conflicts(
9454     candidate.replacement_text,
9455     [
9456         evidence_lookup[evidence_id]
9457         for evidence_id in supporting_evidence_ids
9458         if evidence_id in evidence_lookup
9459     ],
9460 )
9461 if strength_conflicts:
9462     errors.append(
9463         "The replacement uses a qualitative strength label that conflicts "
9464         "with its mapped evidence: "
9465         + "; ".join(strength_conflicts)
9466     )
9467
9468 if (
9469     INTERPRETIVE_SYNTHESIS_PATTERN.search(
9470         candidate.replacement_text
9471     )
9472     and not candidate.supporting_insight_ids
9473 ):
9474     replacement_normalised = re.sub(
9475         r"\W+",
9476         " ",
9477         candidate.replacement_text.lower(),
9478     ).strip()
9479     exact_fact_wording = {
9480         re.sub(r"\W+", " ", text.lower()).strip()
9481         for fact in supporting_facts
9482         for text in [
9483             fact.fact_summary,
9484             *fact.allowed_interpretations,
9485         ]
9486     }
9487     if replacement_normalised not in exact_fact_wording:
9488         errors.append(
9489             "The replacement introduces an interpretive synthesis "

```

```

9490         "without a verified insight ID."
9491     )
9492
9493     support_numbers = [
9494         number
9495         for fact in supporting_facts
9496         for number in fact_support_numbers(fact, evidence)
9497     ]
9498
9499     if not numbers_supported(
9500         candidate.replacement_text,
9501         support_numbers,
9502     ):
9503         errors.append(
9504             "The replacement introduces unsupported numbers."
9505         )
9506
9507     permissions = {
9508         permission
9509         for fact in supporting_facts
9510         for permission in fact.claim_permissions
9511     }
9512
9513     if (
9514         CAUSAL_PATTERN.search(candidate.replacement_text)
9515         and not negative_causal(candidate.replacement_text)
9516         and (
9517             ClaimPermission.CAUSAL not in permissions
9518             or (
9519                 candidate.supporting_insight_ids
9520                 and not any(
9521                     CAUSAL_PATTERN.search(
9522                         all_insights[insight_id].statement
9523                     )
9524                     for insight_id in candidate.supporting_insight_ids
9525                     if insight_id in all_insights
9526                 )
9527             )
9528         )
9529     ):
9530         errors.append(
9531             "The replacement introduces unsupported causal language."
9532         )
9533
9534     if (
9535         PREDICTIVE_PATTERN.search(candidate.replacement_text)
9536         and not negative_predictive(candidate.replacement_text)
9537         and (
9538             ClaimPermission.PREDICTIVE not in permissions
9539             or (
9540                 candidate.supporting_insight_ids
9541                 and not any(
9542                     PREDICTIVE_PATTERN.search(
9543                         all_insights[insight_id].statement
9544                     )
9545                     for insight_id in candidate.supporting_insight_ids
9546                     if insight_id in all_insights
9547                 )
9548             )
9549         )
9550     ):
9551         errors.append(
9552             "The replacement introduces unsupported predictive language."
9553         )
9554
9555     if (
9556         FORECAST_PATTERN.search(candidate.replacement_text)
9557         and not negative_forecast(candidate.replacement_text)
9558         and (
9559             ClaimPermission.FORECAST not in permissions
9560             or (
9561                 candidate.supporting_insight_ids
9562                 and not any(
9563                     FORECAST_PATTERN.search(
9564                         all_insights[insight_id].statement
9565                     )
9566                     for insight_id in candidate.supporting_insight_ids
9567                     if insight_id in all_insights
9568                 )
9569             )
9570         )
9571     ):
9572         errors.append(
9573             "The replacement introduces unsupported forecast language."
9574         )
9575
9576     if (
9577         DATASET_GENERALISATION_PATTERN.search(
9578             candidate.replacement_text
9579         )
9580         and not any(

```

```

9581         DATASET_GENERALISATION_PATTERN.search(
9582             all_insights[insight_id].statement
9583         )
9584         for insight_id in candidate.supporting_insight_ids
9585         if insight_id in all_insights
9586     )
9587 ):
9588     errors.append(
9589         "The replacement generalises beyond its verified support."
9590     )
9591
9592 if (
9593     INSIGHT_OVERSTATEMENT_PATTERN.search(
9594         candidate.replacement_text
9595     )
9596     and not any(
9597         INSIGHT_OVERSTATEMENT_PATTERN.search(
9598             all_insights[insight_id].statement
9599         )
9600         for insight_id in candidate.supporting_insight_ids
9601         if insight_id in all_insights
9602     )
9603 ):
9604     errors.append(
9605         "The replacement is stronger than its verified insight wording."
9606     )
9607
9608 known_entities = {
9609     entity
9610     for fact in supporting_facts
9611     for entity in fact.entities
9612 }
9613
9614 unsupported_entities = unsupported_backtick_entities(
9615     candidate.replacement_text,
9616     known_entities,
9617 )
9618 if unsupported_entities:
9619     errors.append(
9620         "Unsupported named entities in replacement: "
9621         f"{unsupported_entities}"
9622     )
9623
9624 return errors
9625
9626 def apply_repair_proposal(
9627     writer_output: WriterOutput,
9628     proposal: AuditRepairProposal,
9629     fact_ledger: FactLedger,
9630     evidence: EvidenceLedger,
9631     insight_ledger: InsightLedger | None = None,
9632     allow_hypotheses_in_report: bool = False,
9633 ) -> tuple[WriterOutput, list[ReportPatch]]:
9634     insight_ledger = insight_ledger or InsightLedger(
9635         synthesis_enabled=False
9636     )
9637     markdown = writer_output.markdown
9638     support_map = list(writer_output.sentence_support)
9639     patches: list[ReportPatch] = []
9640
9641     support_by_sentence = {
9642         support.sentence_text: support
9643         for support in support_map
9644     }
9645     flagged_sentences = {
9646         repair.original_sentence
9647         for repair in proposal.repairs
9648     }
9649     original_unflagged_sentences = {
9650         support.sentence_text
9651         for support in writer_output.sentence_support
9652         if support.sentence_text not in flagged_sentences
9653     }
9654
9655     for repair in proposal.repairs:
9656         original = repair.original_sentence
9657
9658         if original not in markdown:
9659             continue
9660
9661         ranked_candidates = sorted(
9662             repair.candidates,
9663             key=repair_score,
9664             reverse=True,
9665         )
9666
9667         selected: RepairCandidate | None = None
9668
9669         if repair.preferred_repair_id:
9670             preferred = next(

```

```

9672         (
9673             candidate
9674             for candidate in ranked_candidates
9675             if candidate.repair_id == repair.preferred_repair_id
9676         ),
9677         None,
9678     )
9679
9680     if preferred is not None:
9681         ranked_candidates = [
9682             preferred,
9683             *[
9684                 candidate
9685                 for candidate in ranked_candidates
9686                 if candidate.repair_id != preferred.repair_id
9687             ],
9688         ]
9689
9690     for candidate in ranked_candidates:
9691         if not validate_repair_candidate(
9692             candidate,
9693             fact_ledger,
9694             evidence,
9695             insight_ledger,
9696             allow_hypotheses_in_report,
9697             original_text=original,
9698         ):
9699             selected = candidate
9700             break
9701
9702     if selected is None:
9703         continue
9704
9705     replacement = selected.replacement_text
9706     operation = (
9707         "delete"
9708         if selected.strategy == RepairStrategy.DELETE
9709         else "replace"
9710     )
9711
9712     markdown = markdown.replace(
9713         original,
9714         replacement,
9715         1,
9716     )
9717
9718     existing_support = support_by_sentence.get(original)
9719
9720     if existing_support is not None:
9721         support_map.remove(existing_support)
9722         support_by_sentence.pop(original, None)
9723
9724     if replacement.strip():
9725         supporting_facts = {
9726             fact.fact_id: fact
9727             for fact in fact_ledger.writer_ready_facts
9728         }
9729         main_insights = {
9730             insight.insight_id: insight
9731             for insight in insight_ledger.verified_insights
9732         }
9733         hypothesis_insights = {
9734             insight.insight_id: insight
9735             for insight in insight_ledger.hypothesis_only_insights
9736         }
9737         all_insights = {
9738             **main_insights,
9739             **hypothesis_insights,
9740         }
9741         replacement_fact_ids = list(
9742             dict.fromkeys(
9743                 [
9744                     *selected.supporting_fact_ids,
9745                     *[
9746                         fact_id
9747                         for insight_id in selected.supporting_insight_ids
9748                         if insight_id in all_insights
9749                         for fact_id in all_insights[
9750                             insight_id
9751                         ].source_fact_ids
9752                     ],
9753                 ],
9754             )
9755         )
9756
9757         replacement_evidence = list(
9758             dict.fromkeys(
9759                 [
9760                     *[
9761                         evidence_id
9762                         for fact_id in replacement_fact_ids

```

```

9763         if fact_id in supporting_facts
9764         for evidence_id in supporting_facts[
9765             fact_id
9766         ].evidence_ids
9767     ],
9768     *[
9769         evidence_id
9770         for insight_id in selected.supporting_insight_ids
9771         if insight_id in all_insights
9772         for evidence_id in all_insights[
9773             insight_id
9774         ].source_evidence_ids
9775     ],
9776 ]
9777 )
9778 )
9779 replacement_level = InterpretationLevel.FINDING
9780 if set(selected.supporting_insight_ids) & set(
9781     hypothesis_insights
9782 ):
9783     replacement_level = InterpretationLevel.HYPOTHESIS
9784 elif set(selected.supporting_insight_ids) & set(
9785     main_insights
9786 ):
9787     replacement_level = InterpretationLevel.BOUNDED_INSIGHT
9788
9789 replacement_sentences = (
9790     split_markdown_sentences(replacement)
9791     or [replacement]
9792 )
9793 used_sentence_ids = {
9794     support.sentence_id
9795     for support in support_map
9796 }
9797 numeric_sentence_ids = [
9798     int(match.group(1))
9799     for support in support_map
9800     if (
9801         match := re.fullmatch(
9802             r"SENT_(\d+)",
9803             support.sentence_id,
9804         )
9805     )
9806 ]
9807 next_sentence_number = max(
9808     numeric_sentence_ids,
9809     default=0,
9810 ) + 1
9811
9812 for index, replacement_sentence in enumerate(
9813     replacement_sentences
9814 ):
9815     sentence_id = repair.sentence_id
9816     if index > 0 or sentence_id in used_sentence_ids:
9817         while (
9818             f"SENT_{next_sentence_number:04d}"
9819             in used_sentence_ids
9820         ):
9821             next_sentence_number += 1
9822         sentence_id = (
9823             f"SENT_{next_sentence_number:04d}"
9824         )
9825         next_sentence_number += 1
9826
9827     new_support = SentenceSupport(
9828         sentence_id=sentence_id,
9829         sentence_text=replacement_sentence,
9830         fact_ids=replacement_fact_ids,
9831         evidence_ids=replacement_evidence,
9832         insight_ids=selected.supporting_insight_ids,
9833         interpretation_level=replacement_level,
9834         support_type=(
9835             SupportType.MULTI_FACT_SYNTHESIS
9836             if replacement_level
9837             != InterpretationLevel.FINDING
9838             else SupportType.PARAPHRASE
9839         ),
9840     )
9841
9842     support_map.append(new_support)
9843     support_by_sentence[
9844         replacement_sentence
9845     ] = new_support
9846     used_sentence_ids.add(sentence_id)
9847
9848 patches.append(
9849     ReportPatch(
9850         sentence_id=repair.sentence_id,
9851         original_text=original,
9852         replacement_text=replacement,
9853         operation=operation,

```

```

9854         selected_repair_id=selected.repair_id,
9855     )
9856 )
9857
9858 selected_fact_ids = list(
9859     dict.fromkeys(
9860         fact_id
9861         for support in support_map
9862         for fact_id in support.fact_ids
9863     )
9864 )
9865
9866 all_fact_ids = [
9867     fact.fact_id
9868     for fact in fact_ledger.writer_ready_facts
9869 ]
9870
9871 for sentence in original_unflagged_sentences:
9872     if sentence and sentence not in markdown:
9873         raise ValueError(
9874             "A targeted repair modified or removed unflagged report content."
9875         )
9876
9877 return (
9878     writer_output.model_copy(
9879         update={
9880             "markdown": markdown,
9881             "sentence_support": support_map,
9882             "selected_fact_ids": selected_fact_ids,
9883             "omitted_fact_ids": [
9884                 fact_id
9885                 for fact_id in all_fact_ids
9886                 if fact_id not in selected_fact_ids
9887             ],
9888             "writer_mode": "auditor_repaired",
9889         }
9890     ),
9891     patches,
9892 )

```

Listing 23: P23: table2text_pydanticai/src/table2text/capabilities.py

```

1  """Resolve generic evidence capabilities and execute structure-aware tasks."""
2
3  from __future__ import annotations
4
5  import re
6  from collections.abc import Mapping
7  from dataclasses import dataclass, field
8  from typing import Any
9
10 from .schemas import (
11     AnalyticalFunction,
12     CapabilityDefinition,
13     ClaimPermission,
14     EvidenceCapability,
15     EvidenceOperation,
16     EvidenceQuery,
17     InvestigationTask,
18     InputSemanticMap,
19     InputShape,
20     RecommendedUse,
21     SemanticBinding,
22     SemanticLevel,
23     SemanticRole,
24     StructuralField,
25 )
26 from .structure import find_participant_container, normalise_key
27
28
29 CAPABILITY_REGISTRY: dict[EvidenceCapability, CapabilityDefinition] = {
30     EvidenceCapability.DATASET_PROFILE: CapabilityDefinition(
31         capability=EvidenceCapability.DATASET_PROFILE,
32         supported_input_shapes=list(InputShape),
33         output_evidence_types=["dataset_profile", "event_record_overview"],
34     ),
35     EvidenceCapability.FOCUSED_TABLE_REGION: CapabilityDefinition(
36         capability=EvidenceCapability.FOCUSED_TABLE_REGION,
37         supported_input_shapes=[
38             InputShape.FLAT_TABLE,
39             InputShape.NESTED_RECORD,
40             InputShape.ENTITY_COLLECTION,
41             InputShape.INPUT_REFERENCE_PAIRS,
42             InputShape.AMBIGUOUS,
43         ],
44         output_evidence_types=[
45             "focused_table_region",
46             "focused_cell_context",
47         ],

```

```

48     ),
49     EvidenceCapability.STRUCTURED_RECORD_VERBALISATION: CapabilityDefinition(
50         capability=EvidenceCapability.STRUCTURED_RECORD_VERBALISATION,
51         supported_input_shapes=[
52             InputShape.FLAT_TABLE,
53             InputShape.NESTED_RECORD,
54             InputShape.ENTITY_COLLECTION,
55             InputShape.INPUT_REFERENCE_PAIRS,
56             InputShape.AMBIGUOUS,
57         ],
58         output_evidence_types=[
59             "attribute_record",
60             "triple_record",
61             "structured_record",
62         ],
63     ),
64     EvidenceCapability.MISSINGNESS: CapabilityDefinition(
65         capability=EvidenceCapability.MISSINGNESS,
66         supported_input_shapes=[
67             InputShape.FLAT_TABLE,
68             InputShape.ENTITY_COLLECTION,
69             InputShape.TIME_SERIES,
70         ],
71         output_evidence_types=["missingness"],
72     ),
73     EvidenceCapability.DUPLICATES: CapabilityDefinition(
74         capability=EvidenceCapability.DUPLICATES,
75         supported_input_shapes=[
76             InputShape.FLAT_TABLE,
77             InputShape.ENTITY_COLLECTION,
78             InputShape.TIME_SERIES,
79         ],
80         output_evidence_types=["duplicate_rows"],
81     ),
82     EvidenceCapability.DISTRIBUTION_SUMMARY: CapabilityDefinition(
83         capability=EvidenceCapability.DISTRIBUTION_SUMMARY,
84         supported_input_shapes=[
85             InputShape.FLAT_TABLE,
86             InputShape.ENTITY_COLLECTION,
87             InputShape.TIME_SERIES,
88         ],
89         requires_numeric_fields=True,
90         output_evidence_types=["distribution_summary"],
91     ),
92     EvidenceCapability.ASSOCIATION: CapabilityDefinition(
93         capability=EvidenceCapability.ASSOCIATION,
94         supported_input_shapes=[
95             InputShape.FLAT_TABLE,
96             InputShape.ENTITY_COLLECTION,
97             InputShape.TIME_SERIES,
98         ],
99         requires_numeric_fields=True,
100         minimum_observations=20,
101         output_evidence_types=["correlation"],
102     ),
103     EvidenceCapability.GROUP_COMPARISON: CapabilityDefinition(
104         capability=EvidenceCapability.GROUP_COMPARISON,
105         supported_input_shapes=[
106             InputShape.FLAT_TABLE,
107             InputShape.ENTITY_COLLECTION,
108             InputShape.TIME_SERIES,
109             InputShape.EVENT_RECORD,
110         ],
111         requires_entity_fields=True,
112         output_evidence_types=["group_comparison", "participant_comparison"],
113     ),
114     EvidenceCapability.RANKING: CapabilityDefinition(
115         capability=EvidenceCapability.RANKING,
116         supported_input_shapes=[
117             InputShape.ENTITY_COLLECTION,
118             InputShape.EVENT_RECORD,
119         ],
120         requires_entity_fields=True,
121         output_evidence_types=["entity_ranking"],
122     ),
123     EvidenceCapability.EVENT_OUTCOME: CapabilityDefinition(
124         capability=EvidenceCapability.EVENT_OUTCOME,
125         supported_input_shapes=[InputShape.EVENT_RECORD],
126         requires_event_participants=True,
127         requires_outcome_field=True,
128         output_evidence_types=[
129             "event_outcome",
130             "event_status",
131             "event_sequence",
132             "participant_record_context",
133             "score_progression",
134         ],
135     ),
136     EvidenceCapability.ENTITY_PERFORMANCE: CapabilityDefinition(
137         capability=EvidenceCapability.ENTITY_PERFORMANCE,
138         supported_input_shapes=[InputShape.EVENT_RECORD],

```

```

139         requires_entity_fields=True,
140         requires_event_participants=True,
141         output_evidence_types=["entity_performance"],
142     ),
143 }
144
145 QUERY_EVIDENCE_TYPES: dict[EvidenceCapability, set[str]] = {
146     EvidenceCapability.DATASET_PROFILE: {
147         "event_context",
148         "event_status",
149     },
150     EvidenceCapability.FOCUSED_TABLE_REGION: {
151         "focused_table_region",
152         "focused_cell_context",
153     },
154     EvidenceCapability.STRUCTURED_RECORD_VERBALISATION: {
155         "attribute_record",
156         "triple_record",
157         "structured_record",
158     },
159     EvidenceCapability.EVENT_OUTCOME: {
160         "event_outcome",
161         "event_context",
162         "event_status",
163         "event_sequence",
164     },
165     EvidenceCapability.ENTITY_PERFORMANCE: {"entity_performance"},
166     EvidenceCapability.RANKING: {"entity_ranking"},
167     EvidenceCapability.GROUP_COMPARISON: {
168         "participant_comparison",
169         "event_contrast",
170     },
171 }
172
173 QUERY_OPERATIONS: dict[str, EvidenceOperation] = {
174     "focused_table_region": EvidenceOperation.RETRIEVE,
175     "focused_cell_context": EvidenceOperation.RETRIEVE,
176     "attribute_record": EvidenceOperation.RETRIEVE,
177     "triple_record": EvidenceOperation.RETRIEVE,
178     "structured_record": EvidenceOperation.RETRIEVE,
179     "event_outcome": EvidenceOperation.COMPARE,
180     "event_context": EvidenceOperation.RETRIEVE,
181     "event_status": EvidenceOperation.RETRIEVE,
182     "event_sequence": EvidenceOperation.RETRIEVE,
183     "entity_performance": EvidenceOperation.RETRIEVE,
184     "entity_ranking": EvidenceOperation.RANK,
185     "participant_comparison": EvidenceOperation.COMPARE,
186     "event_contrast": EvidenceOperation.COMPARE,
187 }
188
189
190 def _parse_pipe_triple(value: Any) -> tuple[str, str, str] | None:
191     if not isinstance(value, str) or "|" not in value:
192         return None
193     parts = [part.strip() for part in value.split("|")]
194     if len(parts) != 3 or not all(parts):
195         return None
196     return parts[0], parts[1], parts[2]
197
198
199 def _value_contains_serialized_triple(value: Any) -> bool:
200     if _parse_pipe_triple(value) is not None:
201         return True
202     if isinstance(value, Mapping):
203         triples = value.get("triples")
204         if isinstance(triples, list) and any(
205             _value_contains_serialized_triple(item)
206             or (
207                 isinstance(item, (list, tuple))
208                 and len(item) == 3
209                 and all(str(part).strip() for part in item)
210             )
211             for item in triples
212         ):
213             return True
214     return any(_value_contains_serialized_triple(item) for item in value.values())
215 if isinstance(value, list):
216     return any(_value_contains_serialized_triple(item) for item in value)
217 return False
218
219
220 def _bundle_contains_serialized_triples(bundle: Any) -> bool:
221     structured_inputs = getattr(bundle, "structured_inputs", {}) or {}
222     if any(_value_contains_serialized_triple(value) for value in structured_inputs.values()):
223         return True
224
225     for frame in getattr(bundle, "tables", {}).values():
226         columns = set(getattr(frame, "columns", []))
227         candidate_columns = columns & {"source_payload", "source_text", "triples"}
228         for column in candidate_columns:
229             try:

```



```

230         values = frame[column].tolist()
231     except Exception:
232         continue
233     if any(_value_contains_serialized_triple(value) for value in values):
234         return True
235     return False
236
237
238 PARTICIPATION_REQUEST_PATTERN = re.compile(
239     r"\b(duration|time played|playing time|minutes played|seconds played|"
240     r"participation|exposure|attendance|appearances?)\b",
241     re.IGNORECASE,
242 )
243
244
245 def participation_measure_requested(request: str) -> bool:
246     return bool(PARTICIPATION_REQUEST_PATTERN.search(request))
247
248
249 EVENT_RANKING_RESULT_LIMIT = 200
250
251
252 def _binding_text(binding_id: str, semantic_map: InputSemanticMap) -> str:
253     binding = next(
254         item
255         for item in semantic_map.bindings
256         if item.binding_id == binding_id
257     )
258     return " ".join(
259         item
260         for item in [
261             binding.label,
262             binding.path_pattern,
263             binding.unit or "",
264         ]
265         if item
266     ).lower()
267
268
269 IDENTIFIER_ROLES = {
270     SemanticRole.PARTICIPANT_IDENTIFIER,
271     SemanticRole.ENTITY_IDENTIFIER,
272     SemanticRole.IDENTIFIER,
273 }
274
275
276 def _path_parent(path_pattern: str) -> str:
277     return path_pattern.rsplit(".", 1)[0] if "." in path_pattern else ""
278
279
280 def _repeated_parent(path_pattern: str) -> str | None:
281     parent = _path_parent(path_pattern)
282     if "*" not in parent and "[" not in parent:
283         return None
284     return parent
285
286
287 def _binding_label_text(binding: SemanticBinding) -> str:
288     return " ".join(
289         part
290         for part in [
291             binding.label,
292             binding.path_pattern,
293             binding.description,
294             binding.unit or "",
295         ]
296         if part
297     ).lower()
298
299
300 def _identity_score(binding: SemanticBinding) -> float:
301     text = _binding_label_text(binding)
302     key = normalise_key(binding.path_pattern.rsplit(".", 1)[-1])
303     score = 0.0
304     if key in NAME_FIELD_NAMES:
305         score += 8.0
306     if any(
307         term in text
308         for term in [
309             "name",
310             "label",
311             "actor",
312             "entity",
313             "member",
314             "person",
315             "performer",
316             "player",
317             "subject",
318         ]
319     ):
320         score += 4.0

```

```

321     if any(
322         term in text
323         for term in [
324             "team",
325             "side",
326             "group",
327             "participant",
328             "affiliation",
329         ]
330     ):
331         score -= 3.0
332     if any(term in text for term in ["id", "code"]):
333         score -= 4.0
334     return score
335
336 def _affiliation_score(binding: SemanticBinding) -> float:
337     text = _binding_label_text(binding)
338     key = normalise_key(binding.path_pattern.rsplit(".", 1)[-1])
339     score = 0.0
340     if key in AFFILIATION_FIELD_NAMES:
341         score += 6.0
342     if any(
343         term in text
344         for term in [
345             "team",
346             "side",
347             "group",
348             "participant",
349             "affiliation",
350         ]
351     ):
352         score += 4.0
353     if any(term in text for term in ["name", "label"]):
354         score += 1.0
355     return score
356
357
358 def _is_affiliation_binding(binding: SemanticBinding) -> bool:
359     if binding.role in IDENTIFIER_ROLES:
360         return True
361
362     if binding.role not in {
363         SemanticRole.CONTEXT,
364         SemanticRole.METADATA,
365         SemanticRole.LOCATION,
366     }:
367         return False
368
369     return _affiliation_score(binding) > 0
370
371
372 def _measure_bindings_for_repeated_parents(
373     bindings: list[SemanticBinding],
374 ) -> dict[str, list[SemanticBinding]]:
375     grouped: dict[str, list[SemanticBinding]] = {}
376     for binding in bindings:
377         parent = _repeated_parent(binding.path_pattern)
378         if parent is None:
379             continue
380         if binding.role not in {
381             SemanticRole.PERFORMANCE_MEASURE,
382             SemanticRole.MEASURE,
383             SemanticRole.OUTCOME_MEASURE,
384         }:
385             continue
386         grouped.setdefault(parent, []).append(binding)
387     return grouped
388
389
390 def _identifier_bindings_for_repeated_parents(
391     bindings: list[SemanticBinding],
392 ) -> dict[str, list[SemanticBinding]]:
393     grouped: dict[str, list[SemanticBinding]] = {}
394     for binding in bindings:
395         parent = _repeated_parent(binding.path_pattern)
396         if parent is None or binding.role not in IDENTIFIER_ROLES:
397             continue
398         grouped.setdefault(parent, []).append(binding)
399     return grouped
400
401
402 def _repeated_entity_binding_ids(
403     bindings: list[SemanticBinding],
404 ) -> set[str]:
405     measure_groups = _measure_bindings_for_repeated_parents(bindings)
406     identifier_groups = _identifier_bindings_for_repeated_parents(bindings)
407     repeated_parents = set(measure_groups) | set(identifier_groups)
408     selected: set[str] = set()
409
410     for parent, measures in measure_groups.items():

```

```

412     if any(
413         candidate != parent and candidate.startswith(f"{parent}.")
414         for candidate in repeated_parents
415     ):
416         continue
417     identifiers = identifier_groups.get(parent, [])
418     if not identifiers or not measures:
419         continue
420
421     scored = sorted(
422         identifiers,
423         key=_identity_score,
424         reverse=True,
425     )
426     best = scored[0]
427     if _identity_score(best) <= 0 and len(scored) > 1:
428         continue
429
430     selected.add(best.binding_id)
431     selected.update(binding.binding_id for binding in measures)
432
433     return selected
434
435 def _local_entity_identifier(
436     measure_binding: SemanticBinding,
437     bindings: list[SemanticBinding],
438 ) -> SemanticBinding | None:
439     parent = _repeated_parent(measure_binding.path_pattern)
440     candidates = [
441         binding
442         for binding in bindings
443         if parent is not None
444         and _repeated_parent(binding.path_pattern) == parent
445         and binding.role == SemanticRole.ENTITY_IDENTIFIER
446         and binding.level == SemanticLevel.ENTITY
447     ]
448     if not candidates and parent is None:
449         candidates = [
450             binding
451             for binding in bindings
452             if binding.role == SemanticRole.ENTITY_IDENTIFIER
453             and binding.level == SemanticLevel.ENTITY
454         ]
455     if not candidates:
456         return None
457     return max(candidates, key=_identity_score)
458
459 def _global_entity_identifier(
460     bindings: list[SemanticBinding],
461 ) -> SemanticBinding | None:
462     candidates = [
463         binding
464         for binding in bindings
465         if binding.role == SemanticRole.ENTITY_IDENTIFIER
466         and binding.level == SemanticLevel.ENTITY
467     ]
468     if not candidates:
469         return None
470     return max(candidates, key=_identity_score)
471
472 def _local_group_identifier(
473     measure_binding: SemanticBinding,
474     entity_binding: SemanticBinding | None,
475     bindings: list[SemanticBinding],
476     fallback_participant: SemanticBinding | None,
477 ) -> SemanticBinding | None:
478     parent = _repeated_parent(measure_binding.path_pattern)
479     candidates = [
480         binding
481         for binding in bindings
482         if parent is not None
483         and _repeated_parent(binding.path_pattern) == parent
484         and binding.binding_id != (
485             entity_binding.binding_id if entity_binding else None
486         )
487         and _is_affiliation_binding(binding)
488     ]
489     if candidates:
490         return max(candidates, key=_affiliation_score)
491     return fallback_participant
492
493 def _measure_priority(
494     binding_id: str,
495     semantic_map: InputSemanticMap,
496 ) -> float:
497     binding = next(
498         item

```

```

503     for item in semantic_map.bindings
504     if item.binding_id == binding_id
505     )
506     terminal_key = binding.path_pattern.rsplit(".", 1)[-1]
507     text = " ".join(
508         part
509         for part in [
510             binding.label,
511             terminal_key,
512             binding.unit or "",
513         ]
514         if part
515     ).lower()
516     text = re.sub(r"[_*\.-/+]", " ", text)
517     score = (
518         -90.0
519         if re.search(
520             r"\b(avg|average|era|loss(?:es)?|obp|ops|pct|percentage|"
521             r"rate|record|standing|wins?)\b",
522             text,
523         )
524         else 0.0
525     )
526     structural_text = re.sub(r"[_*\.-/+]", " ", _binding_label_text(binding))
527     if re.search(
528         r"\b(?:inning|half(?:-|\s)?inning|period|quarter|round|segment|"
529         r"phase|frame|stage|play|turn|timeline|sequence)\b",
530         structural_text,
531     ):
532         score -= 150.0
533     if re.search(
534         r"\b(?:allowed|against|caught\s+stealing|conceded|earned|error|"
535         r"errors|loss|losses)\b",
536         text,
537     ):
538         score -= 130.0
539     if (
540         re.search(r"\bstrikeout\b", text)
541         and not re.search(
542             r"\b(?:defensive|pitcher|pitchers|pitching)\b",
543             text,
544         )
545     ):
546         score -= 130.0
547     priority_terms = [
548         (120.0, ("point", "score", "total", "run")),
549         (100.0, ("goal", "made", "converted", "hit")),
550         (95.0, ("rbi", "assist", "support")),
551         (75.0, ("rebound", "recovery")),
552         (72.0, ("strikeout",)),
553         (71.0, ("double", "triple", "stolen")),
554         (70.0, ("attempt", "opportunity")),
555         (55.0, ("turnover", "error")),
556         (50.0, ("steal", "block", "defen")),
557         (20.0, ("foul", "penalt")),
558         (-100.0, ("second", "minute", "duration", "time played", "sec")),
559     ]
560     for value, terms in priority_terms:
561         if any(
562             re.search(rf"\b{re.escape(term)}\b", text)
563             for term in terms
564         ):
565             score += value
566             break
567
568     binding = next(
569         item
570         for item in semantic_map.bindings
571         if item.binding_id == binding_id
572     )
573     if binding.analytical_function == AnalyticalFunction.OUTCOME:
574         score += 120.0
575     elif binding.analytical_function == AnalyticalFunction.OUTCOME_COMPONENT:
576         score += 60.0
577     elif binding.analytical_function == AnalyticalFunction.PERFORMANCE:
578         score += 45.0
579     elif binding.analytical_function == AnalyticalFunction.PARTICIPATION:
580         score -= 120.0
581
582     return score + binding.confidence
583
584
585 def _preferred_identifier(
586     bindings: list,
587     *,
588     preferred_terms: tuple[str, ...],
589     excluded_terms: tuple[str, ...] = (),
590 ):
591     def score(binding) -> tuple[int, float]:
592         text = " ".join(
593             item

```

```

594         for item in [
595             binding.label,
596             binding.path_pattern,
597         ]
598         if item
599     ).lower()
600     preferred = sum(term in text for term in preferred_terms)
601     excluded = sum(term in text for term in excluded_terms)
602     return (preferred - excluded, binding.confidence)
603
604     eligible = [
605         binding
606         for binding in bindings
607         if not any(
608             term
609             in " ".join([binding.label, binding.path_pattern]).lower()
610             for term in excluded_terms
611         )
612     ]
613     candidates = eligible or bindings
614     return max(candidates, key=score) if candidates else None
615
616 def _task_id_for_capability(
617     tasks: list[InvestigationTask],
618     capability: EvidenceCapability,
619 ) -> str | None:
620     for task in tasks:
621         if task.capability == capability:
622             return task.task_id
623
624     return None
625
626
627 PARTICIPANT_SIDE_ALIASES = {
628     "home": "home",
629     "host": "home",
630     "local": "home",
631     "away": "away",
632     "guest": "away",
633     "road": "away",
634     "vis": "away",
635     "visitor": "away",
636     "visiting": "away",
637     "left": "left",
638     "right": "right",
639     "north": "north",
640     "south": "south",
641     "east": "east",
642     "west": "west",
643     "first": "first",
644     "second": "second",
645 }
646
647
648 def _participant_side_key(binding: SemanticBinding) -> str | None:
649     texts = [
650         binding.path_pattern.rsplit(".", 1)[0],
651         binding.path_pattern,
652         binding.label,
653     ]
654     for text in texts:
655         tokens = [
656             token
657             for token in normalise_key(text).split("_")
658             if token
659         ]
660         for token in tokens[:3]:
661             side = PARTICIPANT_SIDE_ALIASES.get(token)
662             if side is not None:
663                 return side
664     return None
665
666
667 def _participant_identifier_for_measure(
668     measure_binding: SemanticBinding,
669     participant_identifiers: list[SemanticBinding],
670 ) -> SemanticBinding | None:
671     side_key = _participant_side_key(measure_binding)
672     if side_key is None:
673         return None
674     candidates = [
675         binding
676         for binding in participant_identifiers
677         if _participant_side_key(binding) == side_key
678     ]
679     if not candidates:
680         return None
681     return max(candidates, key=lambda binding: binding.confidence)
682
683
684

```

```

685 def _is_single_participant_side_binding(
686     binding: SemanticBinding,
687 ) -> bool:
688     return (
689         _participant_side_key(binding) is not None
690         and _repeated_parent(binding.path_pattern) is None
691     )
692
693
694 EVENT_RESULT_MEASURE_TERMS = {
695     "points",
696     "result",
697     "runs",
698     "score",
699     "scored",
700     "seats",
701     "tally",
702     "total",
703     "votes",
704 }
705
706
707 def _looks_like_event_result_measure(
708     binding: SemanticBinding,
709 ) -> bool:
710     tokens = set(
711         token
712         for token in normalise_key(
713             " ".join(
714                 [
715                     binding.path_pattern.rsplit(".", 1)[-1],
716                     binding.label,
717                 ]
718             )
719             .split("_")
720             if token
721         )
722     )
723     return bool(tokens & EVENT_RESULT_MEASURE_TERMS)
724
725 PARALLEL_PARTICIPANT_COMPARISON_TYPES = {
726     "event_outcome",
727     "participant_comparison",
728     "event_contrast",
729 }
730
731
732 def _participant_measure_family_key(
733     binding: SemanticBinding,
734 ) -> str:
735     leaf = binding.path_pattern.rsplit(".", 1)[-1]
736     side_tokens = set(PARTICIPANT_SIDE_ALIASES)
737     tokens = [
738         token
739         for token in normalise_key(f"{leaf} {binding.label}").split("_")
740         if token and token not in side_tokens
741     ]
742     return "_".join(tokens)
743
744
745 def _allows_parallel_participant_value_bindings(
746     *,
747     evidence_type: str,
748     value_bindings: list[SemanticBinding],
749 ) -> bool:
750     if evidence_type not in PARALLEL_PARTICIPANT_COMPARISON_TYPES:
751         return False
752     if len(value_bindings) < 2:
753         return False
754     side_keys = [
755         _participant_side_key(binding)
756         for binding in value_bindings
757     ]
758     family_keys = {
759         _participant_measure_family_key(binding)
760         for binding in value_bindings
761     }
762     return (
763         all(side_keys)
764         and len(set(side_keys)) == len(side_keys)
765         and len(family_keys) == 1
766     )
767
768
769 def build_event_evidence_queries(
770     *,
771     semantic_map: InputSemanticMap | None,
772     tasks: list[InvestigationTask],
773     available_capabilities: set[EvidenceCapability],
774     request: str,
775 ) -> list[EvidenceQuery]:

```

```

776 semantic_map = normalise_semantic_map(semantic_map)
777 if semantic_map is None or semantic_map.input_shape != InputShape.EVENT_RECORD:
778     return []
779
780 bindings = semantic_map.bindings
781 if not bindings:
782     return []
783
784 table_name = bindings[0].table_name
785 participant_identifiers = [
786     binding
787     for binding in bindings
788     if binding.role == SemanticRole.PARTICIPANT_IDENTIFIER
789     and binding.level == SemanticLevel.PARTICIPANT
790 ]
791 entity_identifiers = [
792     binding
793     for binding in bindings
794     if binding.role == SemanticRole.ENTITY_IDENTIFIER
795     and binding.level == SemanticLevel.ENTITY
796 ]
797 participant_id = _preferred_identifier(
798     participant_identifiers,
799     preferred_terms=("name", "label", "team", "participant"),
800     excluded_terms=("place", "city", "location", "code", "record"),
801 )
802 participant_group = _preferred_identifier(
803     [
804         binding
805         for binding in participant_identifiers
806         if participant_id is None
807         or binding.binding_id != participant_id.binding_id
808     ],
809     preferred_terms=("place", "city", "location"),
810 )
811 entity_id = _preferred_identifier(
812     entity_identifiers,
813     preferred_terms=("name", "label", "entity", "player", "person"),
814     excluded_terms=("id", "code"),
815 )
816 context_bindings = [
817     binding
818     for binding in bindings
819     if binding.level == SemanticLevel.EVENT
820     and binding.role
821     in {
822         SemanticRole.CONTEXT,
823         SemanticRole.TIME,
824         SemanticRole.LOCATION,
825         SemanticRole.METADATA,
826     }
827 ]
828 compact_context_bindings = [
829     binding
830     for binding in context_bindings
831     if _repeated_parent(binding.path_pattern) is None
832 ]
833 context_ids = [
834     binding.binding_id
835     for binding in (compact_context_bindings or context_bindings)
836 ][0:6]
837 status_ids = [
838     binding.binding_id
839     for binding in bindings
840     if binding.level == SemanticLevel.EVENT
841     and binding.role == SemanticRole.STATUS
842 ][0:1]
843 outcome_ids = [
844     binding.binding_id
845     for binding in bindings
846     if binding.level == SemanticLevel.PARTICIPANT
847     and binding.role == SemanticRole.OUTCOME_MEASURE
848     and binding.analytical_function == AnalyticalFunction.OUTCOME
849 ]
850 entity_measure_ids = [
851     binding.binding_id
852     for binding in bindings
853     if binding.level == SemanticLevel.ENTITY
854     and binding.role
855     in {
856         SemanticRole.PERFORMANCE_MEASURE,
857         SemanticRole.MEASURE,
858     }
859     and (
860         participation_measure_requested(request)
861         or binding.analytical_function
862         != AnalyticalFunction.PARTICIPATION
863     )
864 ]
865 participant_component_ids = [
866     binding.binding_id

```

```

867     for binding in bindings
868     if binding.level == SemanticLevel.PARTICIPANT
869     and binding.role
870     in {
871         SemanticRole.PERFORMANCE_MEASURE,
872         SemanticRole.MEASURE,
873     }
874     and binding.analytical_function
875     == AnalyticalFunction.OUTCOME_COMPONENT
876 ]
877
878 queries: list[EvidenceQuery] = []
879
880 event_task_id = _task_id_for_capability(
881     tasks,
882     EvidenceCapability.EVENT_OUTCOME,
883 )
884 ranking_task_id = _task_id_for_capability(
885     tasks,
886     EvidenceCapability.RANKING,
887 )
888 entity_performance_task_id = _task_id_for_capability(
889     tasks,
890     EvidenceCapability.ENTITY_PERFORMANCE,
891 )
892 comparison_task_id = _task_id_for_capability(
893     tasks,
894     EvidenceCapability.GROUP_COMPARISON,
895 )
896
897 common = {
898     "table_name": table_name,
899     "user_relevance": 0.95,
900     "salience": 0.95,
901 }
902
903 if (
904     event_task_id
905     and EvidenceCapability.EVENT_OUTCOME in available_capabilities
906     and context_ids
907 ):
908     queries.append(
909         EvidenceQuery(
910             query_id="QUERY_EVENT_CONTEXT_AUTO",
911             task_id=event_task_id,
912             operation=EvidenceOperation.RETRIEVE,
913             capability=EvidenceCapability.EVENT_OUTCOME,
914             evidence_type="event_context",
915             semantic_label="event context",
916             question="What supplied context locates this event?",
917             semantic_level=SemanticLevel.EVENT,
918             value_binding_ids=context_ids,
919             recommended_use=RecommendedUse.HEADLINE,
920             **common,
921         )
922     )
923
924 if (
925     event_task_id
926     and EvidenceCapability.EVENT_OUTCOME in available_capabilities
927     and status_ids
928 ):
929     queries.append(
930         EvidenceQuery(
931             query_id="QUERY_EVENT_STATUS_AUTO",
932             task_id=event_task_id,
933             operation=EvidenceOperation.RETRIEVE,
934             capability=EvidenceCapability.EVENT_OUTCOME,
935             evidence_type="event_status",
936             semantic_label="event status",
937             question="What status is recorded for this event?",
938             semantic_level=SemanticLevel.EVENT,
939             value_binding_ids=status_ids,
940             recommended_use=RecommendedUse.SUPPORTING_DETAIL,
941             **common,
942         )
943     )
944
945 if (
946     event_task_id
947     and participant_id
948     and outcome_ids
949     and EvidenceCapability.EVENT_OUTCOME in available_capabilities
950 ):
951     paired_outcome_ids = [
952         binding_id
953         for binding_id in sorted(
954             outcome_ids,
955             key=lambda item: _measure_priority(item, semantic_map),
956             reverse=True,
957         )

```



```

958         if _participant_identifier_for_measure(
959             next(
960                 binding
961                 for binding in bindings
962                 if binding.binding_id == binding_id
963             ),
964             participant_identifiers,
965         )
966         is not None
967     ]
968     selected_outcome_ids = (
969         paired_outcome_ids
970         if len(paired_outcome_ids) >= 2
971         else [
972             max(
973                 outcome_ids,
974                 key=lambda binding_id: _measure_priority(
975                     binding_id,
976                     semantic_map,
977                 ),
978             )
979         ]
980     )
981     queries.append(
982         EvidenceQuery(
983             query_id="QUERY_EVENT_OUTCOME_AUTO",
984             task_id=event_task_id,
985             operation=EvidenceOperation.COMPARE,
986             capability=EvidenceCapability.EVENT_OUTCOME,
987             evidence_type="event_outcome",
988             semantic_label="event outcome",
989             question="How do the participant outcome measures compare?",
990             semantic_level=SemanticLevel.PARTICIPANT,
991             value_binding_ids=selected_outcome_ids,
992             entity_binding_id=participant_id.binding_id,
993             group_binding_id=(
994                 participant_group.binding_id
995                 if participant_group is not None
996                 and len(selected_outcome_ids) == 1
997                 else None
998             ),
999             recommended_use=RecommendedUse.HEADLINE,
1000             **common,
1001         )
1002     )
1003
1004 if (
1005     ranking_task_id
1006     and EvidenceCapability.RANKING in available_capabilities
1007 ):
1008     for index, binding_id in enumerate(
1009         sorted(
1010             entity_measure_ids,
1011             key=lambda item: _measure_priority(item, semantic_map),
1012             reverse=True,
1013         ),
1014         start=1,
1015     ):
1016         measure_binding = next(
1017             binding
1018             for binding in bindings
1019             if binding.binding_id == binding_id
1020         )
1021         local_entity_id = _local_entity_identifier(
1022             measure_binding,
1023             bindings,
1024         )
1025         if (
1026             local_entity_id is None
1027             and _repeated_parent(measure_binding.path_pattern) is None
1028         ):
1029             local_entity_id = entity_id
1030         if local_entity_id is None:
1031             continue
1032         local_group_id = _local_group_identifier(
1033             measure_binding,
1034             local_entity_id,
1035             bindings,
1036             participant_id,
1037         )
1038         priority_score = _measure_priority(binding_id, semantic_map)
1039         recommended_use = (
1040             RecommendedUse.MAIN_FINDING
1041             if priority_score >= 70
1042             else (
1043                 RecommendedUse.SUPPORTING_DETAIL
1044                 if priority_score >= 20
1045                 else RecommendedUse.OMIT_UNLESS_REQUESTED
1046             )
1047         )
1048         salience = (

```

```

1049         0.9
1050         if priority_score >= 70
1051         else (0.65 if priority_score >= 20 else 0.30)
1052     )
1053     queries.append(
1054         EvidenceQuery(
1055             query_id=f"QUERY_ENTITY_RANKING_AUTO_{index:02d}",
1056             task_id=ranking_task_id,
1057             operation=EvidenceOperation.RANK,
1058             capability=EvidenceCapability.RANKING,
1059             evidence_type="entity_ranking",
1060             semantic_label=(
1061                 "entity ranking for "
1062                 + next(
1063                     binding.label
1064                     for binding in bindings
1065                     if binding.binding_id == binding_id
1066                 )
1067             ),
1068             question="Which entities have the highest recorded values?",
1069             semantic_level=SemanticLevel.ENTITY,
1070             value_binding_ids=[binding_id],
1071             entity_binding_id=local_entity_id.binding_id,
1072             group_binding_id=(
1073                 local_group_id.binding_id
1074                 if local_group_id is not None
1075                 else None
1076             ),
1077             limit=EVENT_RANKING_RESULT_LIMIT,
1078             recommended_use=recommended_use,
1079             user_relevance=salience,
1080             salience=salience,
1081             table_name=table_name,
1082         )
1083     )
1084
1085     if (
1086         entity_performance_task_id
1087         and EvidenceCapability.ENTITY_PERFORMANCE in available_capabilities
1088         and not ranking_task_id
1089     ):
1090         ranked_measure_ids = sorted(
1091             entity_measure_ids,
1092             key=lambda item: _measure_priority(item, semantic_map),
1093             reverse=True,
1094         )
1095         for index, binding_id in enumerate(ranked_measure_ids, start=1):
1096             measure_binding = next(
1097                 binding
1098                 for binding in bindings
1099                 if binding.binding_id == binding_id
1100             )
1101             local_entity_id = _local_entity_identifier(
1102                 measure_binding,
1103                 bindings,
1104             )
1105             if (
1106                 local_entity_id is None
1107                 and _repeated_parent(measure_binding.path_pattern) is None
1108             ):
1109                 local_entity_id = entity_id
1110             if local_entity_id is None:
1111                 continue
1112             local_group_id = _local_group_identifier(
1113                 measure_binding,
1114                 local_entity_id,
1115                 bindings,
1116                 participant_id,
1117             )
1118             priority_score = _measure_priority(binding_id, semantic_map)
1119             recommended_use = (
1120                 RecommendedUse.MAIN_FINDING
1121                 if priority_score >= 70
1122                 else (
1123                     RecommendedUse.SUPPORTING_DETAIL
1124                     if priority_score >= 20
1125                     else RecommendedUse.OMIT_UNLESS_REQUESTED
1126                 )
1127             )
1128             salience = (
1129                 0.88
1130                 if priority_score >= 70
1131                 else (0.62 if priority_score >= 20 else 0.25)
1132             )
1133             queries.append(
1134                 EvidenceQuery(
1135                     query_id=f"QUERY_ENTITY_PERFORMANCE_AUTO_{index:02d}",
1136                     task_id=entity_performance_task_id,
1137                     operation=EvidenceOperation.RETRIEVE,
1138                     capability=EvidenceCapability.ENTITY_PERFORMANCE,
1139                     evidence_type="entity_performance",

```

```

1140         semantic_label=(
1141             "entity performance for "
1142             + measure_binding.label
1143         ),
1144         question="What entity-level performance values are recorded?",
1145         semantic_level=SemanticLevel.ENTITY,
1146         value_binding_ids=[binding_id],
1147         entity_binding_id=local_entity_id.binding_id,
1148         group_binding_id=(
1149             local_group_id.binding_id
1150             if local_group_id is not None
1151             else None
1152         ),
1153         limit=EVENT_RANKING_RESULT_LIMIT,
1154         recommended_use=recommended_use,
1155         user_relevance=salience,
1156         salience=salience,
1157         table_name=table_name,
1158     )
1159 )
1160
1161 if (
1162     comparison_task_id
1163     and participant_id
1164     and EvidenceCapabilities.GROUP_COMPARISON in available_capabilities
1165 ):
1166     participant_component_lookup = {
1167         binding.binding_id: binding
1168         for binding in bindings
1169         if binding.binding_id in participant_component_ids
1170     }
1171     paired_component_groups: dict[str, list[SemanticBinding]] = {}
1172     repeated_component_ids: list[str] = []
1173     for binding_id in sorted(
1174         participant_component_ids,
1175         key=lambda item: _measure_priority(item, semantic_map),
1176         reverse=True,
1177     ):
1178         binding = participant_component_lookup[binding_id]
1179         if _participant_identifier_for_measure(
1180             binding,
1181             participant_identifiers,
1182         ) is None:
1183             repeated_component_ids.append(binding_id)
1184             continue
1185         paired_component_groups.setdefault(
1186             _participant_measure_family_key(binding),
1187             [],
1188         ).append(binding)
1189
1190     paired_queries: list[list[str]] = []
1191     used_component_ids: set[str] = set()
1192     for component_group in paired_component_groups.values():
1193         side_unique: dict[str, SemanticBinding] = {}
1194         for binding in component_group:
1195             side_key = _participant_side_key(binding)
1196             if side_key is None or side_key in side_unique:
1197                 continue
1198             side_unique[side_key] = binding
1199         if len(side_unique) < 2:
1200             repeated_component_ids.extend(
1201                 binding.binding_id
1202                 for binding in component_group
1203                 if binding.binding_id not in used_component_ids
1204             )
1205             continue
1206         selected_ids = [
1207             binding.binding_id
1208             for binding in sorted(
1209                 side_unique.values(),
1210                 key=lambda item: _measure_priority(
1211                     item.binding_id,
1212                     semantic_map,
1213                 ),
1214                 reverse=True,
1215             )
1216         ]
1217         used_component_ids.update(selected_ids)
1218         paired_queries.append(selected_ids)
1219
1220     comparison_value_sets = [
1221         *paired_queries,
1222         *[
1223             [binding_id]
1224             for binding_id in repeated_component_ids
1225             if binding_id not in used_component_ids
1226         ],
1227     ]
1228
1229     for index, value_binding_ids in enumerate(
1230         comparison_value_sets,

```

```

1231         start=1,
1232     ):
1233         priority_score = max(
1234             _measure_priority(binding_id, semantic_map)
1235             for binding_id in value_binding_ids
1236         )
1237         recommended_use = (
1238             RecommendedUse.SUPPORTING_DETAIL
1239             if priority_score >= 20
1240             else RecommendedUse.OMIT_UNLESS_REQUESTED
1241         )
1242         salience = 0.85 if priority_score >= 20 else 0.30
1243         primary_binding_id = value_binding_ids[0]
1244         queries.append(
1245             EvidenceQuery(
1246                 query_id=f"QUERY_PARTICIPANT_CONTRAST_AUTO_{index:02d}",
1247                 task_id=comparison_task_id,
1248                 operation=EvidenceOperation.COMPARE,
1249                 capability=EvidenceCapability.GROUP_COMPARISON,
1250                 evidence_type="participant_comparison",
1251                 semantic_label=(
1252                     "participant contrast for "
1253                     + next(
1254                         binding.label
1255                         for binding in bindings
1256                         if binding.binding_id == primary_binding_id
1257                     )
1258                 ),
1259                 question="How do participant-level measures compare?",
1260                 semantic_level=SemanticLevel.PARTICIPANT,
1261                 value_binding_ids=value_binding_ids,
1262                 entity_binding_id=participant_id.binding_id,
1263                 group_binding_id=(
1264                     participant_group.binding_id
1265                     if participant_group is not None
1266                     and len(value_binding_ids) == 1
1267                     else None
1268                 ),
1269                 recommended_use=recommended_use,
1270                 user_relevance=salience,
1271                 salience=salience,
1272                 table_name=table_name,
1273             )
1274         )
1275     return queries
1276
1277
1278
1279 def _normalise_event_query_priority(
1280     query: EvidenceQuery,
1281     semantic_map: InputSemanticMap,
1282 ) -> EvidenceQuery:
1283     if query.evidence_type not in {
1284         "entity_ranking",
1285         "entity_performance",
1286         "participant_comparison",
1287         "event_contrast",
1288     }:
1289         return query
1290
1291     priority_scores = [
1292         _measure_priority(binding_id, semantic_map)
1293         for binding_id in query.value_binding_ids
1294     ]
1295     if not priority_scores:
1296         return query
1297
1298     priority_score = max(priority_scores)
1299     if query.evidence_type in {
1300         "entity_ranking",
1301         "entity_performance",
1302     }:
1303         recommended_use = (
1304             RecommendedUse.MAIN_FINDING
1305             if priority_score >= 70
1306             else (
1307                 RecommendedUse.SUPPORTING_DETAIL
1308                 if priority_score >= 20
1309                 else RecommendedUse.OMIT_UNLESS_REQUESTED
1310             )
1311         )
1312         salience = (
1313             0.9
1314             if priority_score >= 70
1315             else (0.65 if priority_score >= 20 else 0.30)
1316         )
1317     else:
1318         recommended_use = (
1319             RecommendedUse.SUPPORTING_DETAIL
1320             if priority_score >= 20
1321             else RecommendedUse.OMIT_UNLESS_REQUESTED

```

```

1322     )
1323     salience = 0.85 if priority_score >= 20 else 0.30
1324
1325     return query.model_copy(
1326         update={
1327             "recommended_use": recommended_use,
1328             "user_relevance": min(query.user_relevance, salience),
1329             "salience": min(query.salience, salience),
1330         }
1331     )
1332
1333
1334 def normalise_event_evidence_queries(
1335     *,
1336     queries: list[EvidenceQuery],
1337     semantic_map: InputSemanticMap | None,
1338     tasks: list[InvestigationTask],
1339     available_capabilities: set[EvidenceCapability],
1340     request: str,
1341     structural_catalog: list[StructuralField] | None = None,
1342 ) -> list[EvidenceQuery]:
1343     semantic_map = normalise_semantic_map(semantic_map)
1344     if semantic_map is None or semantic_map.input_shape != InputShape.EVENT_RECORD:
1345         return queries
1346
1347     generated = build_event_evidence_queries(
1348         semantic_map=semantic_map,
1349         tasks=tasks,
1350         available_capabilities=available_capabilities,
1351         request=request,
1352     )
1353     combined = [
1354         _normalise_event_query_priority(query, semantic_map)
1355         for query in [*queries, *generated]
1356     ]
1357     unique: list[EvidenceQuery] = []
1358     signatures: set[
1359         tuple[
1360             str,
1361             tuple[str, ...],
1362             str | None,
1363             str | None,
1364         ]
1365     ] = set()
1366     task_capabilities = {
1367         task.task_id: task.capability
1368         for task in tasks
1369     }
1370     for query in combined:
1371         if not _event_query_is_semantically_executable(
1372             query=query,
1373             semantic_map=semantic_map,
1374             structural_catalog=structural_catalog or [],
1375             available_capabilities=available_capabilities,
1376             task_capabilities=task_capabilities,
1377         ):
1378             continue
1379         signature = (
1380             query.operation.value,
1381             tuple(query.value_binding_ids),
1382             query.entity_binding_id,
1383             query.group_binding_id,
1384         )
1385         if signature in signatures:
1386             continue
1387         signatures.add(signature)
1388         unique.append(query)
1389
1390     binding_ids = {
1391         binding.binding_id
1392         for binding in semantic_map.bindings
1393     }
1394
1395     def query_score(query: EvidenceQuery) -> float:
1396         score = query.salience + query.user_relevance
1397         scored_binding_ids = [
1398             binding_id
1399             for binding_id in query.value_binding_ids
1400             if binding_id in binding_ids
1401         ]
1402         if scored_binding_ids:
1403             score += max(
1404                 _measure_priority(binding_id, semantic_map)
1405                 for binding_id in scored_binding_ids
1406             )
1407         if query.evidence_type in PARALLEL_PARTICIPANT_COMPARISON_TYPES:
1408             score += 2.0 * len(query.value_binding_ids)
1409         if query.query_id.endswith("_AUTO"):
1410             score += 1.0
1411         return score
1412

```

```

1413     essential_types = (
1414         "event_context",
1415         "event_status",
1416         "event_outcome",
1417     )
1418     essential: list[EvidenceQuery] = []
1419     for evidence_type in essential_types:
1420         candidates = [
1421             query
1422             for query in unique
1423             if query.evidence_type == evidence_type
1424         ]
1425         if candidates:
1426             essential.append(
1427                 max(candidates, key=query_score)
1428             )
1429
1430     rankings = sorted(
1431         [
1432             query
1433             for query in unique
1434             if query.evidence_type == "entity_ranking"
1435         ],
1436         key=query_score,
1437         reverse=True,
1438     )
1439     comparisons = sorted(
1440         [
1441             query
1442             for query in unique
1443             if query.evidence_type
1444             in {"participant_comparison", "event_contrast"}
1445         ],
1446         key=query_score,
1447         reverse=True,
1448     )
1449     others = [
1450         query
1451         for query in unique
1452         if query.evidence_type
1453         not in {
1454             *essential_types,
1455             "entity_ranking",
1456             "participant_comparison",
1457             "event_contrast",
1458         }
1459     ]
1460
1461     ordered = [*essential, *rankings, *comparisons, *others]
1462     return ordered
1463
1464
1465 def _event_query_is_semantically_executable(
1466     *,
1467     query: EvidenceQuery,
1468     semantic_map: InputSemanticMap,
1469     structural_catalog: list[StructuralField],
1470     available_capabilities: set[EvidenceCapability],
1471     task_capabilities: dict[str, EvidenceCapability | None],
1472 ) -> bool:
1473     """Return whether an event query can be safely executed.
1474
1475     This is intentionally a permissive sanitiser for LLM-authored and
1476     controller-completed queries. The formal validator still reports errors;
1477     this gate prevents known-bad event queries from poisoning an otherwise
1478     recoverable plan.
1479     """
1480
1481     if query.capability not in available_capabilities:
1482         return False
1483     if (
1484         query.task_id in task_capabilities
1485         and task_capabilities[query.task_id] is not None
1486         and task_capabilities[query.task_id] != query.capability
1487     ):
1488         return False
1489
1490     allowed_evidence_types = QUERY_EVIDENCE_TYPES.get(query.capability)
1491     if (
1492         allowed_evidence_types is not None
1493         and query.evidence_type not in allowed_evidence_types
1494     ):
1495         return False
1496
1497     expected_operation = QUERY_OPERATIONS.get(query.evidence_type)
1498     if expected_operation is not None and query.operation != expected_operation:
1499         return False
1500
1501     binding_lookup = {
1502         binding.binding_id: binding
1503         for binding in semantic_map.bindings

```

```

1504     }
1505     referenced_ids = [
1506         *query.value_binding_ids,
1507         *query.context_binding_ids,
1508         *([query.entity_binding_id] if query.entity_binding_id else []),
1509         *([query.group_binding_id] if query.group_binding_id else []),
1510     ]
1511     if any(binding_id not in binding_lookup for binding_id in referenced_ids):
1512         return False
1513     if any(
1514         binding_lookup[binding_id].table_name != query.table_name
1515         for binding_id in referenced_ids
1516     ):
1517         return False
1518
1519     value_bindings = [
1520         binding_lookup[binding_id]
1521         for binding_id in query.value_binding_ids
1522     ]
1523     allowed_value_roles = {
1524         "event_outcome": {SemanticRole.OUTCOME_MEASURE},
1525         "event_context": {
1526             SemanticRole.CONTEXT,
1527             SemanticRole.IDENTIFIER,
1528             SemanticRole.LOCATION,
1529             SemanticRole.METADATA,
1530             SemanticRole.TIME,
1531         },
1532         "event_status": {SemanticRole.STATUS},
1533         "entity_performance": {
1534             SemanticRole.MEASURE,
1535             SemanticRole.PERFORMANCE_MEASURE,
1536         },
1537         "entity_ranking": {
1538             SemanticRole.MEASURE,
1539             SemanticRole.PERFORMANCE_MEASURE,
1540         },
1541         "participant_comparison": {
1542             SemanticRole.MEASURE,
1543             SemanticRole.OUTCOME_MEASURE,
1544             SemanticRole.PERFORMANCE_MEASURE,
1545         },
1546         "event_contrast": {
1547             SemanticRole.MEASURE,
1548             SemanticRole.OUTCOME_MEASURE,
1549             SemanticRole.PERFORMANCE_MEASURE,
1550         },
1551     }.get(query.evidence_type)
1552     if allowed_value_roles is not None and any(
1553         binding.role not in allowed_value_roles
1554         for binding in value_bindings
1555     ):
1556         return False
1557
1558     entity_binding = (
1559         binding_lookup.get(query.entity_binding_id)
1560         if query.entity_binding_id
1561         else None
1562     )
1563     expected_entity_level = {
1564         "event_outcome": SemanticLevel.PARTICIPANT,
1565         "participant_comparison": SemanticLevel.PARTICIPANT,
1566         "event_contrast": SemanticLevel.PARTICIPANT,
1567         "entity_performance": SemanticLevel.ENTITY,
1568         "entity_ranking": SemanticLevel.ENTITY,
1569     }.get(query.evidence_type)
1570     if expected_entity_level is not None:
1571         if entity_binding is None:
1572             return False
1573         if entity_binding.level != expected_entity_level:
1574             return False
1575
1576     if not query.value_binding_ids:
1577         return False
1578     if query.operation in {
1579         EvidenceOperation.COMPARE,
1580         EvidenceOperation.RANK,
1581     }:
1582         if (
1583             query.evidence_type not in PARALLEL_PARTICIPANT_COMPARISON_TYPES
1584             and len(query.value_binding_ids) != 1
1585         ):
1586             return False
1587     if query.evidence_type in PARALLEL_PARTICIPANT_COMPARISON_TYPES:
1588         if not query.value_binding_ids:
1589             return False
1590         if len(query.value_binding_ids) > 1 and not (
1591             query.operation == EvidenceOperation.COMPARE
1592             and _allows_parallel_participant_value_bindings(
1593                 evidence_type=query.evidence_type,
1594                 value_bindings=value_bindings,

```

```

1595         )
1596     ):
1597         return False
1598     if query.entity_binding_id is None:
1599         return False
1600
1601     catalog_lookup = {
1602         (field.table_name, field.path_pattern): field
1603         for field in structural_catalog
1604     }
1605     for binding in value_bindings:
1606         field = catalog_lookup.get(
1607             (binding.table_name, binding.path_pattern)
1608         )
1609         if field is not None and not _field_supports_numeric_measure(field):
1610             return False
1611
1612     return True
1613
1614
1615 ENTITY_CONTAINER_NAMES = {
1616     "box_score",
1617     "entities",
1618     "members",
1619     "participants",
1620     "performers",
1621     "players",
1622 }
1623 NAME_FIELD_NAMES = {
1624     "display_name",
1625     "entity_name",
1626     "full_name",
1627     "name",
1628     "participant_name",
1629     "team_name",
1630 }
1631 PLACE_FIELD_NAMES = {"place", "team_place"}
1632 AFFILIATION_FIELD_NAMES = {
1633     "participant",
1634     "participant_name",
1635     "side",
1636     "team",
1637     "team_name",
1638 }
1639 LINE_RECORD_SUFFIXES = {
1640     "line",
1641     "record",
1642     "summary",
1643 }
1644 SCORE_FIELD_NAMES = {
1645     "final_score",
1646     "points",
1647     "pts",
1648     "score",
1649     "team_points",
1650     "team_runs",
1651     "total",
1652 }
1653 EVENT_AGGREGATE_LEVEL_TERMS = {
1654     "event",
1655     "final",
1656     "full",
1657     "game",
1658     "match",
1659     "overall",
1660     "total",
1661 }
1662 EVENT_SEGMENT_LEVEL_TERMS = {
1663     "half",
1664     "h1",
1665     "h2",
1666     "inning",
1667     "innings",
1668     "ot",
1669     "overtime",
1670     "period",
1671     "periods",
1672     "q1",
1673     "q2",
1674     "q3",
1675     "q4",
1676     "quarter",
1677     "quarters",
1678     "segment",
1679     "segments",
1680 }
1681 PARTICIPANT_RECORD_CONTEXT_KEYS = {
1682     "draws",
1683     "game_number",
1684     "losses",
1685     "rank",

```



```

1686     "ranking",
1687     "record",
1688     "seed",
1689     "standing",
1690     "ties",
1691     "wins",
1692 }
1693 METRIC_ALIASES = {
1694     "points": {"points", "pts", "score"},
1695     "runs": {"r", "runs", "team_runs"},
1696     "hits": {"h", "hits", "team_hits"},
1697     "errors": {"e", "errors", "team_errors"},
1698     "runs batted in": {"rbi", "runs_batted_in"},
1699     "home runs": {"hr", "home_runs"},
1700     "triples": {"t", "triples"},
1701     "doubles": {"d", "doubles"},
1702     "strikeouts": {"so", "strikeouts"},
1703     "pitching strikeouts": {"p_so"},
1704     "walks": {"bb", "walks"},
1705     "pitching walks": {"p_bb"},
1706     "earned runs allowed": {"p_er"},
1707     "runs allowed": {"p_r"},
1708     "rebounds": {"reb", "rebounds", "total_rebounds", "treb"},
1709     "assists": {"ast", "assists"},
1710     "turnovers": {"tov", "turnovers"},
1711     "steals": {"stl", "steals"},
1712     "blocks": {"blk", "blocks"},
1713     "field goals made": {"fgm", "field_goals_made"},
1714     "field goals attempted": {"fga", "field_goals_attempted"},
1715     "three-pointers made": {"fg3m", "three_pointers_made"},
1716     "three-pointers attempted": {"fg3a", "three_pointers_attempted"},
1717     "free throws made": {"ftm", "free_throws_made"},
1718     "free throws attempted": {"fta", "free_throws_attempted"},
1719 }
1720
1721 PRIMARY_ENTITY_METRIC_PRIORITY = [
1722     "points",
1723     "runs",
1724     "hits",
1725     "runs batted in",
1726     "home runs",
1727     "pitching strikeouts",
1728     "strikeouts",
1729     "assists",
1730     "rebounds",
1731     "triples",
1732     "doubles",
1733     "walks",
1734 ]
1735 EVENT_CONTEXT_ROLE_KEYS = {
1736     "context": {
1737         "event",
1738         "game",
1739         "context",
1740         "title",
1741         "name",
1742     },
1743     "time": {
1744         "date",
1745         "day",
1746         "dayname",
1747         "month",
1748         "monthname",
1749         "season",
1750         "time",
1751         "when",
1752         "year",
1753     },
1754     "location": {
1755         "arena",
1756         "city",
1757         "country",
1758         "location",
1759         "place",
1760         "site",
1761         "stadium",
1762         "state",
1763         "venue",
1764         "where",
1765     },
1766 }
1767 EVENT_STATUS_KEYS = {
1768     "cancelled",
1769     "completed",
1770     "extra_period",
1771     "extra_time",
1772     "overtime",
1773     "postponed",
1774     "status",
1775     "tied",
1776 }

```

```

1777 EVENT_CONTEXT_EXCLUDED_KEYS = {
1778     "attendance",
1779     "capacity",
1780     "game_id",
1781     "id",
1782     "next_game_id",
1783     "previous_game_id",
1784 }
1785 SEQUENCE_CONTAINER_TERMS = {
1786     "action",
1787     "actions",
1788     "event",
1789     "events",
1790     "phase",
1791     "phases",
1792     "play",
1793     "plays",
1794     "round",
1795     "rounds",
1796     "segment",
1797     "segments",
1798     "sequence",
1799     "step",
1800     "steps",
1801     "timeline",
1802     "turn",
1803     "turns",
1804 }
1805 SEQUENCE_ORDER_KEYS = {
1806     "frame",
1807     "half",
1808     "inning",
1809     "index",
1810     "order",
1811     "period",
1812     "phase",
1813     "quarter",
1814     "round",
1815     "segment",
1816     "sequence",
1817     "step",
1818     "time",
1819     "timestamp",
1820     "turn",
1821 }
1822 SEQUENCE_ACTION_KEYS = {
1823     "action",
1824     "description",
1825     "event",
1826     "event_type",
1827     "outcome",
1828     "result",
1829     "summary",
1830     "type",
1831 }
1832 SEQUENCE_PARTICIPANT_KEYS = {
1833     "actor",
1834     "batter",
1835     "competitor",
1836     "entity",
1837     "participant",
1838     "participant_name",
1839     "performer",
1840     "pitcher",
1841     "player",
1842     "scorer",
1843     "scorers",
1844     "side",
1845     "subject",
1846     "team",
1847     "team_name",
1848 }
1849 SEQUENCE_SCORE_KEYS = {
1850     "goals",
1851     "score",
1852     "score_state",
1853     "scores",
1854     "points",
1855     "runs",
1856     "total",
1857 }
1858
1859
1860 @dataclass(frozen=True)
1861 class NumericLeaf:
1862     path: str
1863     key: str
1864     value: float
1865
1866
1867 def _numeric_value(value: Any) -> float | None:

```

```

1868     if isinstance(value, bool) or value is None:
1869         return None
1870     if isinstance(value, (int, float)):
1871         return float(value)
1872     if isinstance(value, str):
1873         cleaned = value.strip().replace(",", "")
1874         if re.fullmatch(r"[+-]?(\d+(\.\d*)?|\.\d+)", cleaned):
1875             return float(cleaned)
1876     return None
1877
1878
1879 @dataclass
1880 class EventEntity:
1881     name: str
1882     participant_name: str
1883     metrics: dict[str, float]
1884     metric_paths: dict[str, str]
1885     identity_paths: list[str] = field(default_factory=list)
1886
1887
1888 @dataclass
1889 class EventParticipant:
1890     key: str
1891     name: str
1892     source_path: str
1893     identity_paths: list[str]
1894     score: float | None
1895     score_path: str | None
1896     metrics: dict[str, float] = field(default_factory=dict)
1897     metric_paths: dict[str, str] = field(default_factory=dict)
1898     entities: list[EventEntity] = field(default_factory=list)
1899     record_context: dict[str, Any] = field(default_factory=dict)
1900     record_context_paths: dict[str, str] = field(default_factory=dict)
1901     adjacent_event_context: dict[str, dict[str, Any]] = field(default_factory=dict)
1902     adjacent_event_context_paths: dict[str, dict[str, str]] = field(default_factory=dict)
1903     segment_scores: dict[str, tuple[float, str]] = field(default_factory=dict)
1904
1905
1906 @dataclass(frozen=True)
1907 class EventContextValue:
1908     label: str
1909     value: Any
1910     source_path: str
1911     role: str
1912
1913
1914 @dataclass(frozen=True)
1915 class EventSequenceRecord:
1916     source_path: str
1917     order_values: dict[str, Any] = field(default_factory=dict)
1918     action_values: dict[str, Any] = field(default_factory=dict)
1919     participant_values: dict[str, Any] = field(default_factory=dict)
1920     score_values: dict[str, Any] = field(default_factory=dict)
1921
1922
1923 @dataclass(frozen=True)
1924 class CapabilityEvidence:
1925     capability: EvidenceCapability
1926     evidence_type: str
1927     finding: str
1928     metrics: dict[str, Any]
1929     source_paths: list[str]
1930     entity_scope: list[str]
1931     practical_interpretation: str
1932     strength_label: str
1933     claim_permissions: list[ClaimPermission]
1934     factual_confidence: float
1935     methodological_strength: float
1936     user_relevance: float
1937     salience: float
1938     recommended_use: RecommendedUse
1939     semantic_level: SemanticLevel = SemanticLevel.DATASET
1940     semantic_binding_ids: list[str] = field(default_factory=list)
1941     analytical_function: AnalyticalFunction | None = None
1942     query_id: str | None = None
1943     limitations: list[str] = field(default_factory=list)
1944     prohibited_interpretations: list[str] = field(default_factory=list)
1945
1946
1947 def _numeric_leaves(
1948     value: Any,
1949     prefix: str,
1950     *,
1951     max_depth: int = 7,
1952 ) -> list[NumericLeaf]:
1953     leaves: list[NumericLeaf] = []
1954
1955     def visit(current: Any, path: str, depth: int) -> None:
1956         if depth > max_depth:
1957             return
1958         if isinstance(current, Mapping):

```

```

1959         for key, child in current.items():
1960             child_path = f"{path}.{key}" if path else str(key)
1961             visit(child, child_path, depth + 1)
1962         elif (numeric := _numeric_value(current)) is not None:
1963             key = normalise_key(path.rsplit(".", 1)[-1])
1964             leaves.append(NumericLeaf(path=path, key=key, value=numeric))
1965
1966     visit(value, prefix, 0)
1967     return leaves
1968
1969
1970 def _first_named_item(
1971     value: Mapping[str, Any],
1972     names: set[str],
1973     prefix: str = "",
1974 ) -> tuple[str | None, str | None]:
1975     for key, child in value.items():
1976         if normalise_key(str(key)) in names and isinstance(child, str) and child.strip():
1977             path = f"{prefix}.{key}" if prefix else str(key)
1978             return child.strip(), path
1979     return None, None
1980
1981
1982 def _first_named_value(value: Mapping[str, Any], names: set[str]) -> str | None:
1983     return _first_named_item(value, names)[0]
1984
1985
1986 def _participant_identity(
1987     key: str,
1988     value: Mapping[str, Any],
1989     prefix: str,
1990 ) -> tuple[str, list[str]]:
1991     name, name_path = _first_named_item(value, NAME_FIELD_NAMES, prefix)
1992     place, place_path = _first_named_item(value, PLACE_FIELD_NAMES, prefix)
1993     name = name or str(key)
1994     identity_paths = [path for path in [place_path, name_path] if path]
1995     if place and normalise_key(place) not in normalise_key(names):
1996         name = f"{place} {name}"
1997     return name, identity_paths
1998
1999
2000 def _canonical_metric(key: str) -> str | None:
2001     normalised = normalise_key(key)
2002     for label, aliases in METRIC_ALIASES.items():
2003         if normalised in aliases:
2004             return label
2005     return None
2006
2007
2008 def _metric_priority(metric: str) -> tuple[int, str]:
2009     try:
2010         return (
2011             -PRIMARY_ENTITY_METRIC_PRIORITY.index(metric),
2012             metric,
2013         )
2014     except ValueError:
2015         return (-len(PRIMARY_ENTITY_METRIC_PRIORITY), metric)
2016
2017
2018 def _normalised_path_tokens(path: str) -> list[str]:
2019     return [
2020         token
2021         for token in normalise_key(path).split("_")
2022         if token
2023     ]
2024
2025
2026 def _event_measure_path_is_aggregate(path: str) -> bool:
2027     tokens = set(_normalised_path_tokens(path))
2028     return bool(tokens & EVENT_AGGREGATE_LEVEL_TERMS)
2029
2030
2031 def _event_measure_path_is_segment(path: str) -> bool:
2032     tokens = set(_normalised_path_tokens(path))
2033     if tokens & EVENT_SEGMENT_LEVEL_TERMS:
2034         return True
2035     return any(
2036         re.fullmatch(r"(?:q|h|p|period|quarter|inning)\d+", token)
2037         for token in tokens
2038     )
2039
2040
2041 def _event_record_identity(record: Mapping[str, Any]) -> tuple[str, str]:
2042     return (
2043         str(record.get("entity") or ""),
2044         str(record.get("group") or ""),
2045     )
2046
2047
2048 def _event_records_cover_multiple_entities(
2049     records: list[dict[str, Any]],

```

```

2050 ) -> bool:
2051     return len({_event_record_identity(record) for record in records}) >= 2
2052
2053
2054 def _deduplicate_event_records(
2055     records: list[dict[str, Any]],
2056 ) -> list[dict[str, Any]]:
2057     retained: list[dict[str, Any]] = []
2058     seen: set[tuple[str, str, str]] = set()
2059     for record in records:
2060         key = (
2061             *_event_record_identity(record),
2062             str(record.get("measure") or ""),
2063         )
2064         if key in seen:
2065             continue
2066         seen.add(key)
2067         retained.append(record)
2068     return retained
2069
2070
2071 def _comparable_event_records(
2072     records: list[dict[str, Any]],
2073 ) -> tuple[list[dict[str, Any]], str | None]:
2074     """Prefer event/game totals over segment values for direct comparisons."""
2075
2076     if len(records) < 3:
2077         return records, None
2078
2079     aggregate_records = [
2080         record
2081         for record in records
2082         if _event_measure_path_is_aggregate(str(record.get("source_path") or ""))
2083     ]
2084     if _event_records_cover_multiple_entities(aggregate_records):
2085         return (
2086             _deduplicate_event_records(aggregate_records),
2087             "aggregate_event_level",
2088         )
2089
2090     non_segment_records = [
2091         record
2092         for record in records
2093         if not _event_measure_path_is_segment(
2094             str(record.get("source_path") or "")
2095         )
2096     ]
2097     if _event_records_cover_multiple_entities(non_segment_records):
2098         return (
2099             _deduplicate_event_records(non_segment_records),
2100             "non_segment_event_level",
2101         )
2102
2103     return records, None
2104
2105
2106 EVENT_MEASURE_ROLES = {
2107     SemanticRole.OUTCOME_MEASURE,
2108     SemanticRole.PERFORMANCE_MEASURE,
2109     SemanticRole.MEASURE,
2110 }
2111 MEASURE_ANALYTICAL_FUNCTIONS = {
2112     AnalyticalFunction.OUTCOME,
2113     AnalyticalFunction.OUTCOME_COMPONENT,
2114     AnalyticalFunction.PERFORMANCE,
2115     AnalyticalFunction.PARTICIPATION,
2116 }
2117
2118
2119 def _default_event_analytical_function(
2120     binding: Any,
2121 ) -> AnalyticalFunction:
2122     if binding.role == SemanticRole.OUTCOME_MEASURE:
2123         return AnalyticalFunction.OUTCOME
2124     if binding.level == SemanticLevel.PARTICIPANT:
2125         return AnalyticalFunction.OUTCOME_COMPONENT
2126     if binding.level == SemanticLevel.ENTITY:
2127         return AnalyticalFunction.PERFORMANCE
2128     return AnalyticalFunction.CONTEXT
2129
2130
2131 def normalise_semantic_map(
2132     semantic_map: InputSemanticMap | None,
2133 ) -> InputSemanticMap | None:
2134     if semantic_map is None or semantic_map.input_shape != InputShape.EVENT_RECORD:
2135         return semantic_map
2136
2137     changed = False
2138     repeated_entity_ids = _repeated_entity_binding_ids(semantic_map.bindings)
2139     bindings = []
2140     for binding in semantic_map.bindings:

```

```

2141 analytical_function = binding.analytical_function
2142 role = binding.role
2143 level = binding.level
2144
2145 if _is_single_participant_side_binding(binding):
2146     if role in IDENTIFIER_ROLES:
2147         role = SemanticRole.PARTICIPANT_IDENTIFIER
2148         level = SemanticLevel.PARTICIPANT
2149     elif role == SemanticRole.OUTCOME_MEASURE:
2150         level = SemanticLevel.PARTICIPANT
2151         if _looks_like_event_result_measure(binding):
2152             analytical_function = AnalyticalFunction.OUTCOME
2153         else:
2154             role = SemanticRole.MEASURE
2155             analytical_function = AnalyticalFunction.OUTCOME_COMPONENT
2156     elif role in {
2157         SemanticRole.PERFORMANCE_MEASURE,
2158         SemanticRole.MEASURE,
2159     }:
2160         level = SemanticLevel.PARTICIPANT
2161         if analytical_function in {
2162             None,
2163             AnalyticalFunction.PERFORMANCE,
2164             AnalyticalFunction.OUTCOME_COMPONENT,
2165         }:
2166             analytical_function = AnalyticalFunction.OUTCOME_COMPONENT
2167     elif role == SemanticRole.STATUS:
2168         level = SemanticLevel.PARTICIPANT
2169
2170 elif binding.binding_id in repeated_entity_ids:
2171     if role in IDENTIFIER_ROLES:
2172         role = SemanticRole.ENTITY_IDENTIFIER
2173         level = SemanticLevel.ENTITY
2174     elif role in EVENT_MEASURE_ROLES:
2175         level = SemanticLevel.ENTITY
2176
2177 if (
2178     role == SemanticRole.OUTCOME_MEASURE
2179     and analytical_function != AnalyticalFunction.OUTCOME
2180 ):
2181     analytical_function = AnalyticalFunction.OUTCOME
2182 elif (
2183     role in EVENT_MEASURE_ROLES
2184     and analytical_function is None
2185 ):
2186     analytical_function = _default_event_analytical_function(
2187         binding.model_copy(update={"role": role, "level": level})
2188     )
2189 elif (
2190     role not in EVENT_MEASURE_ROLES
2191     and analytical_function in MEASURE_ANALYTICAL_FUNCTIONS
2192 ):
2193     analytical_function = None
2194
2195 if (
2196     analytical_function != binding.analytical_function
2197     or role != binding.role
2198     or level != binding.level
2199 ):
2200     changed = True
2201     bindings.append(
2202         binding.model_copy(
2203             update={
2204                 "analytical_function": analytical_function,
2205                 "role": role,
2206                 "level": level,
2207             }
2208         )
2209     )
2210 else:
2211     bindings.append(binding)
2212
2213 if not changed:
2214     return semantic_map
2215
2216 return semantic_map.model_copy(update={"bindings": bindings})
2217
2218
2219 def _field_supports_numeric_measure(field: StructuralField) -> bool:
2220     if set(field.value_types) & {"integer", "number"}:
2221         return True
2222
2223     meaningful_samples = [
2224         str(value).strip()
2225         for value in field.sample_values
2226         if str(value).strip().lower()
2227         not in {"", "n/a", "na", "nan", "none", "null", "-"}
2228     ]
2229     if not meaningful_samples:
2230         return False
2231

```

```

2232     numeric_samples = [
2233         value
2234         for value in meaningful_samples
2235         if _numeric_value(value) is not None
2236     ]
2237     return bool(numeric_samples) and (
2238         len(numeric_samples) == len(meaningful_samples)
2239         or len(numeric_samples) >= 2
2240     )
2241
2242
2243 def _participant_line_prefix(key: str) -> str:
2244     normalised = normalise_key(key)
2245     parts = normalised.split("_")
2246     if len(parts) > 1 and parts[-1] in LINE_RECORD_SUFFIXES:
2247         return "_".join(parts[:-1])
2248     return normalised
2249
2250
2251 def _participant_line_items(
2252     payload: Any,
2253 ) -> list[tuple[str, Mapping[str, Any], str]]:
2254     if not isinstance(payload, Mapping):
2255         return []
2256
2257     records: list[tuple[str, Mapping[str, Any], str]] = []
2258     for key, child in payload.items():
2259         if not isinstance(child, Mapping):
2260             continue
2261         prefix = _participant_line_prefix(str(key))
2262         if prefix == normalise_key(str(key)):
2263             continue
2264         child_keys = {normalise_key(str(child_key)) for child_key in child}
2265         if "result" not in child_keys:
2266             continue
2267         if not (
2268             child_keys & SCORE_FIELD_NAMES
2269             or child_keys & NAME_FIELD_NAMES
2270         ):
2271             continue
2272         records.append((prefix, child, str(key)))
2273     return records
2274
2275
2276 def _external_entity_collections(
2277     payload: Any,
2278 ) -> list[tuple[str, Mapping[str, Any] | list[Any]]]:
2279     if not isinstance(payload, Mapping):
2280         return []
2281     collections: list[tuple[str, Mapping[str, Any] | list[Any]]] = []
2282     for key, child in payload.items():
2283         if normalise_key(str(key)) not in ENTITY_CONTAINER_NAMES:
2284             continue
2285         if isinstance(child, list) and any(isinstance(item, Mapping) for item in child):
2286             collections.append((str(key), child))
2287         elif (
2288             isinstance(child, Mapping)
2289             and child
2290             and all(isinstance(item, Mapping) for item in child.values())
2291         ):
2292             collections.append((str(key), child))
2293     return collections
2294
2295
2296 def _entity_affiliation(
2297     entity: Mapping[str, Any],
2298 ) -> tuple[str | None, str | None]:
2299     return _first_named_item(entity, AFFILIATION_FIELD_NAMES)
2300
2301
2302 def _normalised_text(value: str) -> str:
2303     return normalise_key(value).replace("_", "")
2304
2305
2306 def _matching_participant(
2307     participants: list[EventParticipant],
2308     affiliation: str,
2309 ) -> EventParticipant | None:
2310     target = _normalised_text(affiliation)
2311     for participant in participants:
2312         candidates = {
2313             _normalised_text(participant.key),
2314             _normalised_text(participant.name),
2315         }
2316         if target in candidates or any(
2317             target in candidate or candidate in target
2318             for candidate in candidates
2319         ):
2320             return participant
2321     return None
2322

```

```

2323
2324 def _score_leaf(value: Mapping[str, Any], prefix: str) -> NumericLeaf | None:
2325     candidates: list[tuple[int, NumericLeaf]] = []
2326     for leaf in _numeric_leaves(value, prefix):
2327         relative_path = leaf.path.removeprefix(prefix).lstrip(".")
2328         path_parts = {
2329             normalise_key(part)
2330             for part in relative_path.split(".")
2331         }
2332         if path_parts & ENTITY_CONTAINER_NAMES:
2333             continue
2334         if leaf.key not in SCORE_FIELD_NAMES:
2335             continue
2336         score = 20
2337         if "game" in path_parts or "event" in path_parts:
2338             score += 30
2339         if "team" in path_parts or "participant" in path_parts:
2340             score += 20
2341         if leaf.key in {"score", "final_score"}:
2342             score += 15
2343         candidates.append((score, leaf))
2344     return max(candidates, default=(0, None), key=lambda item: item[0])[1]
2345
2346 def _team_metric_mapping(
2347     value: Mapping[str, Any],
2348     prefix: str,
2349 ) -> tuple[dict[str, float], dict[str, str]]:
2350     best_score = -1
2351     best_metrics: dict[str, float] = {}
2352     best_paths: dict[str, str] = {}
2353
2354     def visit(current: Any, path: str, depth: int) -> None:
2355         nonlocal best_score, best_metrics, best_paths
2356         if depth > 6 or not isinstance(current, Mapping):
2357             return
2358
2359         metrics: dict[str, float] = {}
2360         paths: dict[str, str] = {}
2361         for key, child in current.items():
2362             numeric = _numeric_value(child)
2363             if numeric is not None:
2364                 canonical = _canonical_metric(str(key))
2365                 if canonical:
2366                     metrics[canonical] = numeric
2367                     paths[canonical] = f"{path}.{key}" if path else str(key)
2368
2369         relative_path = path.removeprefix(prefix).lstrip(".")
2370         path_names = {
2371             normalise_key(part)
2372             for part in relative_path.split(".")
2373         }
2374         score = len(metrics)
2375         if "game" in path_names or "event" in path_names:
2376             score += 10
2377         if "team" in path_names or "participant" in path_names:
2378             score += 5
2379         if "period" in path_names or path_names & ENTITY_CONTAINER_NAMES:
2380             score -= 20
2381         if score > best_score:
2382             best_score = score
2383             best_metrics = metrics
2384             best_paths = paths
2385
2386         for key, child in current.items():
2387             if isinstance(child, Mapping):
2388                 child_path = f"{path}.{key}" if path else str(key)
2389                 visit(child, child_path, depth + 1)
2390
2391     visit(value, prefix, 0)
2392     return best_metrics, best_paths
2393
2394
2395 def _participant_record_context(
2396     value: Mapping[str, Any],
2397     prefix: str,
2398 ) -> tuple[dict[str, Any], dict[str, str]]:
2399     context: dict[str, Any] = {}
2400     paths: dict[str, str] = {}
2401     for key, child in value.items():
2402         if isinstance(child, (Mapping, list)) or child is None:
2403             continue
2404         normalised = normalise_key(str(key))
2405         if (
2406             normalised not in PARTICIPANT_RECORD_CONTEXT_KEYS
2407             and not normalised.endswith("_standing")
2408         ):
2409             continue
2410         text = str(child).strip()
2411         if not text:
2412             continue
2413

```



```

2414         label = str(key).replace("_", " ")
2415         context[label] = child
2416         paths[label] = f"{prefix}.{key}" if prefix else str(key)
2417     return context, paths
2418
2419
2420 ADJACENT_EVENT_CONTEXT_KEYS = {
2421     "next_event",
2422     "next_game",
2423     "next_match",
2424     "previous_event",
2425     "previous_game",
2426     "previous_match",
2427 }
2428 ADJACENT_EVENT_CONTEXT_VALUE_KEYS = {
2429     "city",
2430     "date",
2431     "day",
2432     "dayname",
2433     "is_home",
2434     "month",
2435     "opponent",
2436     "opponent_name",
2437     "opponent_place",
2438     "stadium",
2439     "venue",
2440     "year",
2441 }
2442
2443
2444 def _participant_adjacent_event_context(
2445     value: Mapping[str, Any],
2446     prefix: str,
2447 ) -> tuple[dict[str, dict[str, Any]], dict[str, dict[str, str]]]:
2448     context: dict[str, dict[str, Any]] = {}
2449     paths: dict[str, dict[str, str]] = {}
2450     for key, child in value.items():
2451         normalised = normalise_key(str(key))
2452         if normalised not in ADJACENT_EVENT_CONTEXT_KEYS:
2453             continue
2454         if not isinstance(child, Mapping):
2455             continue
2456         values: dict[str, Any] = {}
2457         value_paths: dict[str, str] = {}
2458         for child_key, child_value in child.items():
2459             if isinstance(child_value, (Mapping, list)) or child_value is None:
2460                 continue
2461             child_normalised = normalise_key(str(child_key))
2462             if child_normalised not in ADJACENT_EVENT_CONTEXT_VALUE_KEYS:
2463                 continue
2464             text = str(child_value).strip()
2465             if not text:
2466                 continue
2467             label = str(child_key).replace("_", " ")
2468             values[label] = child_value
2469             value_paths[label] = (
2470                 f"{prefix}.{key}.{child_key}" if prefix else f"{key}.{child_key}"
2471             )
2472             if values:
2473                 label = str(key).replace("_", " ")
2474                 context[label] = values
2475                 paths[label] = value_paths
2476     return context, paths
2477
2478
2479 def _segment_sort_key(segment: str) -> tuple[int, int, str] | None:
2480     normalised = normalise_key(segment)
2481     if normalised in EVENT_AGGREGATE_LEVEL_TERMS:
2482         return None
2483     if normalised in {"ot", "overtime", "extra_time", "extra_period"}:
2484         return (90, 1, normalised)
2485     match = re.fullmatch(r"q(?:quarter)?_(\d+)", normalised)
2486     if match:
2487         return (10, int(match.group(1)), normalised)
2488     match = re.fullmatch(r"h(?:alf)?_(\d+)", normalised)
2489     if match:
2490         return (20, int(match.group(1)), normalised)
2491     match = re.fullmatch(r"(?:p|period)?_(\d+)", normalised)
2492     if match:
2493         return (30, int(match.group(1)), normalised)
2494     match = re.fullmatch(r"(?:inning|frame)?_(\d+)", normalised)
2495     if match:
2496         return (40, int(match.group(1)), normalised)
2497     if normalised.isdigit():
2498         return (50, int(normalised), normalised)
2499     if normalised in EVENT_SEGMENT_LEVEL_TERMS:
2500         return (80, 0, normalised)
2501     return None
2502
2503
2504 def _segment_family(segment: str) -> str:

```

```

2505     sort_key = _segment_sort_key(segment)
2506     if sort_key is None:
2507         return "unknown"
2508     return {
2509         10: "quarter",
2510         20: "half",
2511         30: "period",
2512         40: "inning",
2513         50: "numeric",
2514         80: "named_segment",
2515         90: "extra_period",
2516     }.get(sort_key[0], "unknown")
2517
2518
2519 def _participant_segment_scores(
2520     value: Mapping[str, Any],
2521     prefix: str,
2522 ) -> dict[str, tuple[float, str]]:
2523     scores: dict[str, tuple[float, str]] = {}
2524
2525     def visit(current: Any, path: str, depth: int) -> None:
2526         if depth > 5 or not isinstance(current, Mapping):
2527             return
2528         for key, child in current.items():
2529             child_path = f"{path}.{key}" if path else str(key)
2530             segment_key = str(key)
2531             if isinstance(child, Mapping) and _segment_sort_key(segment_key) is not None:
2532                 score_leaf = _score_leaf(child, child_path)
2533                 if score_leaf is not None:
2534                     scores[segment_key] = (score_leaf.value, score_leaf.path)
2535                 continue
2536             if isinstance(child, Mapping):
2537                 visit(child, child_path, depth + 1)
2538
2539     visit(value, prefix, 0)
2540     return scores
2541
2542
2543 def _entity_container(
2544     value: Mapping[str, Any],
2545     prefix: str,
2546 ) -> tuple[str, Mapping[str, Any] | list[Any] | None:
2547     stack: list[tuple[str, Any, int]] = [(prefix, value, 0)]
2548     while stack:
2549         path, current, depth = stack.pop()
2550         if depth > 6 or not isinstance(current, Mapping):
2551             continue
2552         for key, child in current.items():
2553             child_path = f"{path}.{key}" if path else str(key)
2554             normalised_key = normalise_key(str(key))
2555             if (
2556                 normalised_key in ENTITY_CONTAINER_NAMES
2557                 and isinstance(child, Mapping)
2558                 and child
2559                 and all(isinstance(item, Mapping) for item in child.values())
2560             ):
2561                 return child_path, child
2562             if (
2563                 normalised_key in ENTITY_CONTAINER_NAMES
2564                 and isinstance(child, list)
2565                 and child
2566                 and all(isinstance(item, Mapping) for item in child)
2567             ):
2568                 return child_path, child
2569             if isinstance(child, Mapping):
2570                 stack.append((child_path, child, depth + 1))
2571     return None
2572
2573
2574 def _entity_items(
2575     entities: Mapping[str, Any] | list[Any],
2576 ) -> list[tuple[str, Mapping[str, Any]]]:
2577     if isinstance(entities, Mapping):
2578         return [
2579             (str(key), value)
2580             for key, value in entities.items()
2581             if isinstance(value, Mapping)
2582         ]
2583     return [
2584         (str(index), value)
2585         for index, value in enumerate(entities)
2586         if isinstance(value, Mapping)
2587     ]
2588
2589
2590 def extract_event_participants(payload: Any) -> list[EventParticipant]:
2591     located = find_participant_container(payload)
2592     participants: list[EventParticipant] = []
2593
2594     if located is not None:
2595         container_path, container = located

```

```

2596     participant_items = [
2597         (str(key), raw_participant, f"{container_path}.{key}")
2598         for key, raw_participant in container.items()
2599         if isinstance(raw_participant, Mapping)
2600     ]
2601 else:
2602     participant_items = [
2603         (prefix, raw_participant, source_path)
2604         for prefix, raw_participant, source_path
2605         in _participant_line_items(payload)
2606     ]
2607
2608 if not participant_items:
2609     return []
2610
2611 for key, raw_participant, source_path in participant_items:
2612     participant_name, identity_paths = _participant_identity(
2613         str(key), raw_participant, source_path
2614     )
2615     score_leaf = _score_leaf(raw_participant, source_path)
2616     metrics, metric_paths = _team_metric_mapping(raw_participant, source_path)
2617     record_context, record_context_paths = _participant_record_context(
2618         raw_participant,
2619         source_path,
2620     )
2621     adjacent_context, adjacent_context_paths = (
2622         _participant_adjacent_event_context(
2623             raw_participant,
2624             source_path,
2625         )
2626     )
2627     segment_scores = _participant_segment_scores(
2628         raw_participant,
2629         source_path,
2630     )
2631     participant = EventParticipant(
2632         key=str(key),
2633         name=participant_name,
2634         source_path=source_path,
2635         identity_paths=identity_paths,
2636         score=(score_leaf.value if score_leaf else None),
2637         score_path=(score_leaf.path if score_leaf else None),
2638         metrics=metrics,
2639         metric_paths=metric_paths,
2640         record_context=record_context,
2641         record_context_paths=record_context_paths,
2642         adjacent_event_context=adjacent_context,
2643         adjacent_event_context_paths=adjacent_context_paths,
2644         segment_scores=segment_scores,
2645     )
2646
2647     located_entities = _entity_container(raw_participant, source_path)
2648     if located_entities is not None:
2649         entity_path, entities = located_entities
2650         for entity_key, raw_entity in _entity_items(entities):
2651             entity_metrics: dict[str, float] = {}
2652             entity_metric_paths: dict[str, str] = {}
2653             for raw_metric, raw_value in raw_entity.items():
2654                 numeric = _numeric_value(raw_value)
2655                 if numeric is None:
2656                     continue
2657                 canonical = _canonical_metric(str(raw_metric))
2658                 if canonical:
2659                     entity_metrics[canonical] = numeric
2660                     entity_metric_paths[canonical] = (
2661                         f"{entity_path}.{entity_key}.{raw_metric}"
2662                     )
2663             entity_name, entity_name_path = _first_named_item(
2664                 raw_entity,
2665                 NAME_FIELD_NAMES,
2666                 f"{entity_path}.{entity_key}",
2667             )
2668             entity_name = entity_name or str(entity_key)
2669             participant.entities.append(
2670                 EventEntity(
2671                     name=entity_name,
2672                     participant_name=participant.name,
2673                     metrics=entity_metrics,
2674                     metric_paths=entity_metric_paths,
2675                     identity_paths=list(
2676                         dict.fromkeys(
2677                             [
2678                                 *participant.identity_paths,
2679                                 *(
2680                                     [entity_name_path]
2681                                     if entity_name_path
2682                                     else []
2683                                 ),
2684                             ],
2685                         ),
2686                 ),

```

```

2687         )
2688     )
2689     participants.append(participant)
2690
2691     if located is None and isinstance(payload, Mapping):
2692         for entity_collection_path, entities in _external_entity_collections(payload):
2693             for entity_key, raw_entity in _entity_items(entities):
2694                 affiliation, affiliation_path = _entity_affiliation(raw_entity)
2695                 if not affiliation:
2696                     continue
2697                 participant = _matching_participant(
2698                     participants,
2699                     affiliation,
2700                 )
2701                 if participant is None:
2702                     continue
2703
2704                 entity_path = f"{entity_collection_path}.{entity_key}"
2705                 entity_metrics: dict[str, float] = {}
2706                 entity_metric_paths: dict[str, str] = {}
2707                 for raw_metric, raw_value in raw_entity.items():
2708                     numeric = _numeric_value(raw_value)
2709                     if numeric is None:
2710                         continue
2711                     canonical = _canonical_metric(str(raw_metric))
2712                     if canonical:
2713                         entity_metrics[canonical] = numeric
2714                         entity_metric_paths[canonical] = (
2715                             f"{entity_path}.{raw_metric}"
2716                         )
2717
2718                 if not entity_metrics:
2719                     continue
2720
2721                 entity_name, entity_name_path = _first_named_item(
2722                     raw_entity,
2723                     NAME_FIELD_NAMES,
2724                     entity_path,
2725                 )
2726                 entity_name = entity_name or str(entity_key)
2727                 participant.entities.append(
2728                     EventEntity(
2729                         name=entity_name,
2730                         participant_name=participant.name,
2731                         metrics=entity_metrics,
2732                         metric_paths=entity_metric_paths,
2733                         identity_paths=list(
2734                             dict.fromkeys(
2735                                 [
2736                                     *participant.identity_paths,
2737                                     *(
2738                                         [affiliation_path]
2739                                         if affiliation_path
2740                                         else []
2741                                     ),
2742                                     *(
2743                                         [entity_name_path]
2744                                         if entity_name_path
2745                                         else []
2746                                     ),
2747                                 ],
2748                             ),
2749                         ),
2750                     )
2751                 )
2752
2753     return participants
2754
2755
2756 def _event_level_scalar_values(
2757     payload: Any,
2758     *,
2759     excluded_prefixes: list[str],
2760     max_depth: int = 4,
2761 ) -> list[EventContextValue]:
2762     values: list[EventContextValue] = []
2763
2764     def visit(current: Any, path: str, depth: int) -> None:
2765         if depth > max_depth:
2766             return
2767         if path and any(
2768             path == prefix or path.startswith(f"{prefix}.")
2769             for prefix in excluded_prefixes
2770         ):
2771             return
2772         if isinstance(current, Mapping):
2773             for key, child in current.items():
2774                 child_path = f"{path}.{key}" if path else str(key)
2775                 visit(child, child_path, depth + 1)
2776             return
2777         if isinstance(current, (Mapping, list)) or current is None:

```

```

2778         return
2779
2780     key = normalise_key(path.rsplit(".", 1)[-1])
2781     if key in EVENT_CONTEXT_EXCLUDED_KEYS:
2782         return
2783
2784     role = next(
2785         (
2786             role_name
2787             for role_name, keys in EVENT_CONTEXT_ROLE_KEYS.items()
2788             if key in keys
2789         ),
2790         None,
2791     )
2792     if role is None and key in EVENT_STATUS_KEYS:
2793         role = "status"
2794     if role is None:
2795         return
2796
2797     values.append(
2798         EventContextValue(
2799             label=path.rsplit(".", 1)[-1].replace("_", " "),
2800             value=current,
2801             source_path=path,
2802             role=role,
2803         )
2804     )
2805
2806     visit(payload, "", 0)
2807     return values
2808
2809
2810 def _stringify_context_value(value: Any) -> str:
2811     if isinstance(value, bool):
2812         return "true" if value else "false"
2813     return str(value).strip()
2814
2815
2816 def _context_metric_map(
2817     values: list[EventContextValue],
2818 ) -> dict[str, EventContextValue]:
2819     mapped: dict[str, EventContextValue] = {}
2820     for value in values:
2821         key = normalise_key(value.label)
2822         mapped.setdefault(key, value)
2823     return mapped
2824
2825
2826 def _event_context_finding(
2827     values: list[EventContextValue],
2828 ) -> str:
2829     by_key = _context_metric_map(values)
2830     date_parts = [
2831         by_key[key]
2832         for key in [
2833             "day",
2834             "monthname" if "monthname" in by_key else "month",
2835             "year",
2836         ]
2837         if key in by_key
2838     ]
2839     location_parts = [
2840         by_key[key]
2841         for key in ["venue", "stadium", "arena", "site", "city", "state", "country"]
2842         if key in by_key
2843     ]
2844
2845     clauses: list[str] = []
2846     if date_parts:
2847         clauses.append(
2848             "on "
2849             + " ".join(_stringify_context_value(item.value) for item in date_parts)
2850         )
2851     if location_parts:
2852         first_location = _stringify_context_value(location_parts[0].value)
2853         remaining_location = ", ".join(
2854             _stringify_context_value(item.value)
2855             for item in location_parts[1:]
2856         )
2857         clauses.append(
2858             f"at {first_location}"
2859             if not remaining_location
2860             else f"at {first_location} in {remaining_location}"
2861         )
2862
2863     if clauses:
2864         return (
2865             "The supplied event context records the event "
2866             + " ".join(clauses)
2867             + "."
2868         )

```

```

2869
2870     fallback_values = "; ".join(
2871         f"{item.label}: {_stringify_context_value(item.value)}"
2872         for item in values[:8]
2873     )
2874     return f"The supplied event context records {fallback_values}."
2875
2876
2877 def _event_context_evidence(payload: Any) -> List[CapabilityEvidence]:
2878     if not isinstance(payload, Mapping):
2879         return []
2880
2881     located = find_participant_container(payload)
2882     excluded_prefixes = [located[0]] if located is not None else []
2883     values = _event_level_scalar_values(
2884         payload,
2885         excluded_prefixes=excluded_prefixes,
2886     )
2887     context_values = [
2888         item
2889         for item in values
2890         if item.role in {"context", "time", "location"}
2891     ]
2892     status_values = [
2893         item
2894         for item in values
2895         if item.role == "status"
2896     ]
2897
2898     evidence: List[CapabilityEvidence] = []
2899     if context_values:
2900         evidence.append(
2901             CapabilityEvidence(
2902                 capability=EvidenceCapability.DATASET_PROFILE,
2903                 evidence_type="event_context",
2904                 finding=_event_context_finding(context_values),
2905                 metrics={
2906                     "values": [
2907                         {
2908                             "label": item.label,
2909                             "value": item.value,
2910                             "role": item.role,
2911                             "source_path": item.source_path,
2912                         }
2913                         for item in context_values
2914                     ],
2915                 },
2916                 source_paths=[item.source_path for item in context_values],
2917                 entity_scope=[
2918                     _stringify_context_value(item.value)
2919                     for item in context_values
2920                     if item.role in {"location", "context"}
2921                 ],
2922                 practical_interpretation=(
2923                     "This records supplied event-level context without "
2924                     "using participant or entity statistics."
2925                 ),
2926                 strength_label="event_context",
2927                 claim_permissions=[ClaimPermission.DESRIPTIVE],
2928                 factual_confidence=1.0,
2929                 methodological_strength=1.0,
2930                 user_relevance=0.75,
2931                 salience=0.70,
2932                 recommended_use=RecommendedUse.SUPPORTING_DETAIL,
2933                 semantic_level=SemanticLevel.EVENT,
2934             )
2935         )
2936
2937     for item in status_values:
2938         value_text = _stringify_context_value(item.value)
2939         normalised_label = normalise_key(item.label)
2940         if isinstance(item.value, bool) and normalised_label in {
2941             "extra_period",
2942             "extra_time",
2943             "overtime",
2944         }:
2945             status_finding = (
2946                 f"The event required {item.label.replace('_', ' ')}."
2947                 if item.value
2948                 else f"The event did not require {item.label.replace('_', ' ')}."
2949             )
2950         else:
2951             status_finding = (
2952                 f"The supplied event status records {item.label}: {value_text}."
2953             )
2954         evidence.append(
2955             CapabilityEvidence(
2956                 capability=EvidenceCapability.EVENT_OUTCOME,
2957                 evidence_type="event_status",
2958                 finding=status_finding,
2959                 metrics={

```

```

2960         "label": item.label,
2961         "value": item.value,
2962         "source_path": item.source_path,
2963     },
2964     source_paths=[item.source_path],
2965     entity_scope=[],
2966     practical_interpretation=(
2967         "This records a supplied event-status field."
2968     ),
2969     strength_label="event_status",
2970     claim_permissions=[ClaimPermission.DESRIPTIVE],
2971     factual_confidence=1.0,
2972     methodological_strength=1.0,
2973     user_relevance=0.65,
2974     salience=0.55,
2975     recommended_use=RecommendedUse.SUPPORTING_DETAIL,
2976     semantic_level=SemanticLevel.EVENT,
2977 )
2978 )
2979
2980 return evidence
2981
2982
2983 def _sequence_container_name(key: str) -> bool:
2984     normalised = normalise_key(key)
2985     return any(term in normalised.split("_") for term in SEQUENCE_CONTAINER_TERMS)
2986
2987
2988 def _sequence_score_key(key: str) -> bool:
2989     normalised = normalise_key(key)
2990     return (
2991         normalised in SEQUENCE_SCORE_KEYS
2992         or normalised.endswith("_score")
2993         or normalised.endswith("_points")
2994         or normalised.endswith("_total")
2995         or normalised.endswith("_runs")
2996     )
2997
2998
2999 def _sequence_signal_count(item: Mapping[str, Any]) -> int:
3000     signals = 0
3001     for key, value in item.items():
3002         if isinstance(value, (Mapping, list)):
3003             continue
3004         normalised = normalise_key(str(key))
3005         if normalised in SEQUENCE_ORDER_KEYS:
3006             signals += 1
3007         if normalised in SEQUENCE_ACTION_KEYS:
3008             signals += 1
3009         if normalised in SEQUENCE_PARTICIPANT_KEYS:
3010             signals += 1
3011         if _sequence_score_key(normalised):
3012             signals += 1
3013     return signals
3014
3015
3016 def _sequence_field_values(
3017     item: Mapping[str, Any],
3018     *,
3019     path: str,
3020 ) -> tuple[dict[str, Any], dict[str, Any], dict[str, Any], dict[str, Any]]:
3021     order_values: dict[str, Any] = {}
3022     action_values: dict[str, Any] = {}
3023     participant_values: dict[str, Any] = {}
3024     score_values: dict[str, Any] = {}
3025
3026     for key, value in item.items():
3027         if isinstance(value, Mapping) or value is None:
3028             continue
3029         if isinstance(value, list) and any(
3030             isinstance(child, (Mapping, list))
3031             for child in value
3032         ):
3033             continue
3034         normalised = normalise_key(str(key))
3035         source_path = f"{path}.{key}" if path else str(key)
3036         labelled_value = {
3037             "label": str(key),
3038             "value": value,
3039             "source_path": source_path,
3040         }
3041         if normalised in SEQUENCE_ORDER_KEYS:
3042             order_values[str(key)] = labelled_value
3043         if normalised in SEQUENCE_ACTION_KEYS:
3044             action_values[str(key)] = labelled_value
3045         if normalised in SEQUENCE_PARTICIPANT_KEYS:
3046             participant_values[str(key)] = labelled_value
3047         if _sequence_score_key(normalised):
3048             score_values[str(key)] = labelled_value
3049
3050     return order_values, action_values, participant_values, score_values

```

```

3051
3052
3053 def _sequence_record(
3054     item: Mapping[str, Any],
3055     *,
3056     path: str,
3057     inherited_order: dict[str, Any] | None = None,
3058     inherited_participants: dict[str, Any] | None = None,
3059 ) -> EventSequenceRecord | None:
3060     order_values, action_values, participant_values, score_values = (
3061         _sequence_field_values(item, path=path)
3062     )
3063     order_values = {*(inherited_order or {}), **order_values}
3064     participant_values = {
3065         *(inherited_participants or {}),
3066         **participant_values,
3067     }
3068
3069     if not (action_values or score_values):
3070         return None
3071
3072     return EventSequenceRecord(
3073         source_path=path,
3074         order_values=order_values,
3075         action_values=action_values,
3076         participant_values=participant_values,
3077         score_values=score_values,
3078     )
3079
3080
3081 def _event_sequence_records(payload: Any) -> list[EventSequenceRecord]:
3082     records: list[EventSequenceRecord] = []
3083
3084     def visit(current: Any, path: str, depth: int) -> None:
3085         if depth > 5:
3086             return
3087         if isinstance(current, Mapping):
3088             for key, child in current.items():
3089                 child_path = f"{path}.{key}" if path else str(key)
3090                 if isinstance(child, list):
3091                     visit_list(str(key), child, child_path, depth + 1)
3092                 elif isinstance(child, Mapping):
3093                     visit(child, child_path, depth + 1)
3094             return
3095         if isinstance(current, list):
3096             visit_list(path.rsplit(".", 1)[-1], current, path, depth + 1)
3097
3098     def visit_list(key: str, items: list[Any], path: str, depth: int) -> None:
3099         mapping_items = [
3100             (index, item)
3101             for index, item in enumerate(items)
3102             if isinstance(item, Mapping)
3103         ]
3104         if len(mapping_items) < 2:
3105             for index, item in mapping_items:
3106                 visit(item, f"{path}[{index}]", depth + 1)
3107             return
3108
3109         sequence_like = _sequence_container_name(key) or any(
3110             _sequence_signal_count(item) >= 2
3111             for _, item in mapping_items[:5]
3112         )
3113         if not sequence_like:
3114             for index, item in mapping_items:
3115                 visit(item, f"{path}[{index}]", depth + 1)
3116             return
3117
3118         for index, item in mapping_items:
3119             item_path = f"{path}[{index}]"
3120             parent_order, _, parent_participants, _ = _sequence_field_values(
3121                 item,
3122                 path=item_path,
3123             )
3124             record = _sequence_record(item, path=item_path)
3125             if record is not None:
3126                 records.append(record)
3127
3128         for child_key, child in item.items():
3129             if not isinstance(child, list):
3130                 continue
3131             child_mappings = [
3132                 (child_index, child_item)
3133                 for child_index, child_item in enumerate(child)
3134                 if isinstance(child_item, Mapping)
3135             ]
3136             if not child_mappings:
3137                 continue
3138             segment_value = {
3139                 "label": "segment",
3140                 "value": child_key,
3141                 "source_path": f"{item_path}.{child_key}",

```



```

3142         }
3143         inherited_order = {
3144             **parent_order,
3145             f"{child_key} segment": segment_value,
3146         }
3147         for child_index, child_item in child_mappings:
3148             child_path = f"{item_path}.{child_key}[{child_index}]"
3149             nested_record = _sequence_record(
3150                 child_item,
3151                 path=child_path,
3152                 inherited_order=inherited_order,
3153                 inherited_participants=parent_participants,
3154             )
3155             if nested_record is not None:
3156                 records.append(nested_record)
3157
3158     visit(payload, "", 0)
3159     return records
3160
3161
3162 def _clean_sequence_scalar(value: Any) -> str | None:
3163     if value is None:
3164         return None
3165     if isinstance(value, list):
3166         cleaned_items = [
3167             item
3168             for item in (
3169                 _clean_sequence_scalar(child)
3170                 for child in value
3171             )
3172             if item
3173         ]
3174         return ", ".join(cleaned_items) if cleaned_items else None
3175     text = str(value).strip()
3176     if not text or text.upper() in {"N/A", "NA", "NONE", "NULL"}:
3177         return None
3178     return text
3179
3180
3181 def _sequence_value_number(labelled_value: Any) -> float | None:
3182     if not isinstance(labelled_value, Mapping):
3183         return None
3184     return _numeric_value(labelled_value.get("value"))
3185
3186
3187 def _side_key_from_text(text: str) -> str | None:
3188     tokens = [
3189         token
3190         for token in normalise_key(text).split("_")
3191         if token
3192     ]
3193     for token in tokens[:4]:
3194         side = PARTICIPANT_SIDE_ALIASES.get(token)
3195         if side is not None:
3196             return side
3197     return None
3198
3199
3200 def _participant_name_for_side(
3201     payload: Any,
3202     side: str,
3203 ) -> str:
3204     side_aliases = {
3205         alias
3206         for alias, canonical in PARTICIPANT_SIDE_ALIASES.items()
3207         if canonical == side
3208     } | {side}
3209     name_terms = {"name", "team", "participant", "side", "label"}
3210     candidates: list[tuple[int, str]] = []
3211
3212     def visit(current: Any, path: str, depth: int) -> None:
3213         if depth > 3 or not isinstance(current, Mapping):
3214             return
3215         for key, value in current.items():
3216             key_tokens = set(normalise_key(str(key)).split("_"))
3217             path_tokens = set(normalise_key(path).split("_"))
3218             side_match = bool((key_tokens | path_tokens) & side_aliases)
3219             if isinstance(value, str):
3220                 value_text = _clean_sequence_scalar(value)
3221                 if not value_text:
3222                     continue
3223                 if side_match and key_tokens & name_terms:
3224                     candidates.append((3, value_text))
3225                 elif side_match:
3226                     candidates.append((1, value_text))
3227             elif isinstance(value, Mapping):
3228                 child_path = f"{path}.{key}" if path else str(key)
3229                 visit(value, child_path, depth + 1)
3230
3231     visit(payload, "", 0)
3232     if candidates:

```

```

3233         return sorted(candidates, reverse=True)[0][1]
3234     return side.title()
3235
3236
3237 def _sequence_order_text(record: EventSequenceRecord) -> str | None:
3238     parts = []
3239     for labelled_value in record.order_values.values():
3240         if not isinstance(labelled_value, Mapping):
3241             continue
3242         label = str(labelled_value.get("label") or "").replace("_", " ")
3243         value = _clean_sequence_scalar(labelled_value.get("value"))
3244         if label and value:
3245             parts.append(f"{label} {value}")
3246     return ", ".join(dict.fromkeys(parts)) or None
3247
3248
3249 def _sequence_action_text(record: EventSequenceRecord) -> str | None:
3250     preferred = []
3251     for labelled_value in record.action_values.values():
3252         if not isinstance(labelled_value, Mapping):
3253             continue
3254         value = _clean_sequence_scalar(labelled_value.get("value"))
3255         if value:
3256             preferred.append(value)
3257     return "; ".join(dict.fromkeys(preferred)) or None
3258
3259
3260 def _sequence_actor_text(record: EventSequenceRecord) -> str | None:
3261     primary_actors = []
3262     secondary_actors = []
3263     primary_terms = {
3264         "actor",
3265         "batter",
3266         "competitor",
3267         "entity",
3268         "participant",
3269         "performer",
3270         "player",
3271         "subject",
3272         "team",
3273         "team_name",
3274     }
3275     secondary_terms = {
3276         "pitcher",
3277         "scorer",
3278         "scorers",
3279     }
3280     for labelled_value in record.participant_values.values():
3281         if not isinstance(labelled_value, Mapping):
3282             continue
3283         label = normalise_key(str(labelled_value.get("label") or ""))
3284         value = _clean_sequence_scalar(labelled_value.get("value"))
3285         if not value:
3286             continue
3287         if label in primary_terms:
3288             primary_actors.append(value)
3289         elif label in secondary_terms:
3290             secondary_actors.append(value)
3291     actors = primary_actors or secondary_actors
3292     return ", ".join(dict.fromkeys(actors)) or None
3293
3294
3295 def _sequence_score_state(
3296     record: EventSequenceRecord,
3297     payload: Any,
3298 ) -> dict[str, Any] | None:
3299     side_values: dict[str, float] = {}
3300     source_paths: list[str] = []
3301     for labelled_value in record.score_values.values():
3302         if not isinstance(labelled_value, Mapping):
3303             continue
3304         side = _side_key_from_text(
3305             " ".join(
3306                 str(labelled_value.get(part) or "")
3307                 for part in ["label", "source_path"]
3308             )
3309         )
3310         if side is None:
3311             continue
3312         number = _sequence_value_number(labelled_value)
3313         if number is None:
3314             continue
3315         side_values[side] = number
3316         source_path = labelled_value.get("source_path")
3317         if source_path:
3318             source_paths.append(str(source_path))
3319
3320     if len(side_values) < 2:
3321         return None
3322     sides = list(side_values)[:2]
3323     left, right = sides[0], sides[1]

```

```

3324     return {
3325         "left_side": left,
3326         "right_side": right,
3327         "left_name": _participant_name_for_side(payload, left),
3328         "right_name": _participant_name_for_side(payload, right),
3329         "left_value": side_values[left],
3330         "right_value": side_values[right],
3331         "source_paths": source_paths,
3332     }
3333
3334
3335 def _sequence_score_delta(record: EventSequenceRecord) -> float | None:
3336     for labelled_value in record.score_values.values():
3337         if not isinstance(labelled_value, Mapping):
3338             continue
3339         text = " ".join(
3340             str(labelled_value.get(part) or "")
3341             for part in ["label", "source_path"]
3342         )
3343         if _side_key_from_text(text) is not None:
3344             continue
3345         number = _sequence_value_number(labelled_value)
3346         if number is not None and number > 0:
3347             return number
3348     return None
3349
3350
3351 def _event_sequence_highlights(
3352     records: list[EventSequenceRecord],
3353     payload: Any,
3354 ) -> list[dict[str, Any]]:
3355     highlights: list[dict[str, Any]] = []
3356     for record in records:
3357         score_state = _sequence_score_state(record, payload)
3358         score_delta = _sequence_score_delta(record)
3359         if score_state is None or score_delta is None:
3360             continue
3361
3362         left_value = float(score_state["left_value"])
3363         right_value = float(score_state["right_value"])
3364         if left_value == right_value:
3365             score_phrase = (
3366                 f"the score was level at {left_value:g}-{right_value:g}"
3367             )
3368         else:
3369             leader = (
3370                 score_state["left_name"]
3371                 if left_value > right_value
3372                 else score_state["right_name"]
3373             )
3374             score_phrase = (
3375                 f"{leader} led {max(left_value, right_value):g}-"
3376                 f"{min(left_value, right_value):g}"
3377             )
3378
3379         action = _sequence_action_text(record)
3380         actor = _sequence_actor_text(record)
3381         order_text = _sequence_order_text(record)
3382         if action and actor:
3383             event_text = f"{actor} recorded {action}"
3384         elif action:
3385             event_text = f"the recorded action was {action}"
3386         elif actor:
3387             event_text = f"{actor} was recorded"
3388         else:
3389             event_text = "a score-changing step was recorded"
3390         if order_text:
3391             event_text = f"{order_text}: {event_text}"
3392
3393         source_paths = [
3394             record.source_path,
3395             *score_state.get("source_paths", []),
3396             *[
3397                 str(value.get("source_path"))
3398                 for value in [
3399                     *record.action_values.values(),
3400                     *record.participant_values.values(),
3401                 ]
3402                 if isinstance(value, Mapping)
3403                 and value.get("source_path")
3404             ],
3405         ]
3406         highlights.append(
3407             {
3408                 "order": len(highlights) + 1,
3409                 "event_text": event_text,
3410                 "score_phrase": score_phrase,
3411                 "score_delta": score_delta,
3412                 "left_name": score_state["left_name"],
3413                 "right_name": score_state["right_name"],
3414                 "left_value": left_value,

```

```

3415         "right_value": right_value,
3416         "source_path": record.source_path,
3417         "source_paths": list(dict.fromkeys(source_paths)),
3418     }
3419 )
3420
3421 if not highlights:
3422     return highlights
3423
3424 def leader_after(highlight: dict[str, Any]) -> str | None:
3425     left_value = float(highlight["left_value"])
3426     right_value = float(highlight["right_value"])
3427     if left_value == right_value:
3428         return None
3429     return (
3430         str(highlight["left_name"])
3431         if left_value > right_value
3432         else str(highlight["right_name"])
3433     )
3434
3435 largest_delta = max(float(item["score_delta"]) for item in highlights)
3436 previous_leader: str | None = None
3437 previous_margin: float | None = None
3438 total_highlights = len(highlights)
3439 for index, highlight in enumerate(highlights):
3440     roles: list[str] = []
3441     current_leader = leader_after(highlight)
3442     current_margin = abs(
3443         float(highlight["left_value"]) - float(highlight["right_value"])
3444     )
3445
3446     if current_leader is None:
3447         roles.append("tie")
3448     elif previous_leader is None and index == 0:
3449         roles.append("opening_score")
3450     elif previous_leader is not None and previous_leader != current_leader:
3451         roles.append("lead_change")
3452     elif previous_leader is None and index > 0:
3453         roles.append("tie_broken")
3454
3455     if float(highlight["score_delta"]) == largest_delta:
3456         roles.append("largest_score_change")
3457
3458     sequence_position = (index + 1) / total_highlights
3459     if sequence_position >= 0.75:
3460         roles.append("late_score_change")
3461     if index == total_highlights - 1:
3462         roles.append("final_score_change")
3463         if (
3464             previous_margin is not None
3465             and current_margin < previous_margin
3466         ):
3467             roles.append("late_narrowing")
3468
3469     highlight["leader_after"] = current_leader
3470     highlight["margin_after"] = current_margin
3471     highlight["sequence_position"] = sequence_position
3472     highlight["sequence_roles"] = roles
3473
3474     previous_leader = current_leader
3475     previous_margin = current_margin
3476
3477 return highlights
3478
3479
3480 def _event_sequence_evidence(payload: Any) -> list[CapabilityEvidence]:
3481     records = _event_sequence_records(payload)
3482     if len(records) < 2:
3483         return []
3484
3485     action_count = sum(bool(record.action_values) for record in records)
3486     score_count = sum(bool(record.score_values) for record in records)
3487     participant_count = sum(bool(record.participant_values) for record in records)
3488     if not (action_count or score_count):
3489         return []
3490
3491     source_roots = list(
3492         dict.fromkeys(
3493             record.source_path.split("[", 1)[0]
3494             for record in records
3495             if record.source_path
3496         )
3497     )
3498     available_parts = []
3499     if action_count:
3500         available_parts.append("action labels")
3501     if participant_count:
3502         available_parts.append("participant roles")
3503     if score_count:
3504         available_parts.append("score-state fields")
3505

```

```

3506     evidence = [
3507         CapabilityEvidence(
3508             capability=EvidenceCapability.EVENT_OUTCOME,
3509             evidence_type="event_sequence",
3510             finding=(
3511                 "The supplied event structure includes an ordered event "
3512                 f"sequence with {len(records)} recorded steps"
3513                 + (
3514                     f" across {' '.join(source_roots[:3])}"
3515                     if source_roots
3516                     else ""
3517                 )
3518                 + (
3519                     f", including {' '.join(available_parts)}."
3520                     if available_parts
3521                     else "."
3522                 )
3523             ),
3524             metrics={
3525                 "step_count": len(records),
3526                 "action_step_count": action_count,
3527                 "participant_step_count": participant_count,
3528                 "score_state_step_count": score_count,
3529                 "source_roots": source_roots,
3530             },
3531             source_paths=[
3532                 path
3533                 for record in records[:12]
3534                 for path in [record.source_path]
3535                 if path
3536             ],
3537             entity_scope=[],
3538             practical_interpretation=(
3539                 "This permits bounded descriptions of recorded event order "
3540                 "and score-state availability without inferring causes."
3541             ),
3542             strength_label="event_sequence",
3543             claim_permissions=[ClaimPermission.DESRIPTIVE],
3544             factual_confidence=1.0,
3545             methodological_strength=1.0,
3546             user_relevance=0.75,
3547             salience=0.65,
3548             recommended_use=RecommendedUse.SUPPORTING_DETAIL,
3549             semantic_level=SemanticLevel.EVENT,
3550             limitations=[
3551                 "Sequence evidence supports the supplied recorded order only; "
3552                 "it does not by itself establish momentum, tactics or causes."
3553             ],
3554             prohibited_interpretations=[
3555                 "Do not infer a comeback, turning point, dominance or causal "
3556                 "game mechanism unless directly supported."
3557             ],
3558         )
3559     ]
3560     highlights = _event_sequence_highlights(records, payload)
3561     if highlights:
3562         retained = highlights[:6]
3563         highlight_text = "; ".join(
3564             f"{item['event_text']}, after which {item['score_phrase']}"
3565             for item in retained
3566         )
3567         if len(highlights) > len(retained):
3568             highlight_text += (
3569                 f"; {len(highlights) - len(retained)} later score-changing "
3570                 "steps are omitted from this compact evidence item"
3571             )
3572     evidence.append(
3573         CapabilityEvidence(
3574             capability=EvidenceCapability.EVENT_OUTCOME,
3575             evidence_type="event_sequence",
3576             finding=(
3577                 "The recorded score-changing sequence includes: "
3578                 + highlight_text
3579                 + "."
3580             ),
3581             metrics={
3582                 "sequence_type": "score_changing_sequence",
3583                 "highlight_count": len(highlights),
3584                 "summary_highlight_count": len(retained),
3585                 "highlights": highlights,
3586                 "summary_highlights": retained,
3587                 "omitted_highlight_count": max(
3588                     0,
3589                     len(highlights) - len(retained),
3590                 ),
3591             },
3592             source_paths=list(
3593                 dict.fromkeys(
3594                     path
3595                     for item in highlights
3596                     for path in item.get("source_paths", [])

```

```

3597         if path
3598     )
3599 ),
3600 entity_scope=list(
3601     dict.fromkeys(
3602         name
3603         for item in highlights
3604         for name in [
3605             item.get("left_name"),
3606             item.get("right_name"),
3607         ]
3608         if name
3609     )
3610 ),
3611 practical_interpretation=(
3612     "This supports a bounded recap of recorded score-changing "
3613     "steps and score states, without assigning causes."
3614 ),
3615 strength_label="event_sequence_highlight",
3616 claim_permissions=[
3617     ClaimPermission.DESCRPTIVE,
3618     ClaimPermission.COMPARATIVE,
3619 ],
3620 factual_confidence=1.0,
3621 methodological_strength=1.0,
3622 user_relevance=0.95,
3623 salience=0.92,
3624 recommended_use=RecommendedUse.MAIN_FINDING,
3625 semantic_level=SemanticLevel.EVENT,
3626 limitations=[
3627     "Sequence highlights describe recorded score-changing "
3628     "steps only and do not establish causality or momentum."
3629 ],
3630 prohibited_interpretations=[
3631     "Do not call a score-changing step a turning point, comeback "
3632     "or cause unless explicitly supported by the source."
3633 ],
3634 )
3635 )
3636
3637 return evidence
3638
3639
3640 def _participant_record_context_finding(
3641     participants: list[EventParticipant],
3642 ) -> str:
3643     clauses: list[str] = []
3644     for participant in participants:
3645         if not participant.record_context:
3646             continue
3647         wins_key = next(
3648             (
3649                 key
3650                 for key in participant.record_context
3651                 if normalise_key(key) == "wins"
3652             ),
3653             None,
3654         )
3655         losses_key = next(
3656             (
3657                 key
3658                 for key in participant.record_context
3659                 if normalise_key(key) == "losses"
3660             ),
3661             None,
3662         )
3663         if wins_key and losses_key:
3664             clauses.append(
3665                 f"{participant.name} entered with "
3666                 f"{participant.record_context[wins_key]} wins and "
3667                 f"{participant.record_context[losses_key]} losses"
3668             )
3669             continue
3670         values = [
3671             f"{label} {value}"
3672             for label, value in list(participant.record_context.items())[:3]
3673         ]
3674         if values:
3675             clauses.append(f"{participant.name} had " + ", ".join(values))
3676
3677     return (
3678         "Participant context records "
3679         + "; ".join(clauses)
3680         + "."
3681     )
3682
3683
3684 def _participant_adjacent_event_context_finding(
3685     participants: list[EventParticipant],
3686 ) -> str:
3687     clauses: list[str] = []

```

```

3688     for participant in participants:
3689         for relation, values in participant.adjacent_event_context.items():
3690             normalised_relation = relation.replace("_", " ")
3691             opponent = (
3692                 values.get("opponent name")
3693                 or values.get("opponent")
3694                 or values.get("opponent place")
3695             )
3696             date_parts = [
3697                 values[key]
3698                 for key in ["dayname", "day", "month", "year"]
3699                 if key in values
3700             ]
3701             venue = values.get("stadium") or values.get("venue")
3702             city = values.get("city")
3703             location = ", ".join(
3704                 str(item)
3705                 for item in [venue, city]
3706                 if item not in {None, ""}
3707             )
3708             home_value = str(values.get("is home", "")).strip().lower()
3709             home_text = (
3710                 "at home"
3711                 if home_value == "true"
3712                 else "away"
3713                 if home_value == "false"
3714                 else ""
3715             )
3716             parts = [
3717                 f"{participant.name} {normalised_relation}",
3718                 home_text,
3719                 f"against {opponent}" if opponent else "",
3720                 "on " + " ".join(str(item) for item in date_parts)
3721                 if date_parts
3722                 else "",
3723                 f"at {location}" if location else "",
3724             ]
3725             clause = " ".join(part for part in parts if part).strip()
3726             if clause:
3727                 clauses.append(clause)
3728
3729     return (
3730         "Adjacent event context records "
3731         + "; ".join(clauses)
3732         + "."
3733     )
3734
3735
3736 def _participant_record_context_evidence(
3737     participants: list[EventParticipant],
3738 ) -> list[CapabilityEvidence]:
3739     participants_with_context = [
3740         participant
3741         for participant in participants
3742         if participant.record_context
3743     ]
3744     if len(participants_with_context) < 2:
3745         return []
3746
3747     values = [
3748         {
3749             "participant": participant.name,
3750             "values": participant.record_context,
3751             "source_paths": list(participant.record_context_paths.values()),
3752         }
3753         for participant in participants_with_context
3754     ]
3755     return [
3756         CapabilityEvidence(
3757             capability=EvidenceCapability.EVENT_OUTCOME,
3758             evidence_type="participant_record_context",
3759             finding=_participant_record_context_finding(
3760                 participants_with_context
3761             ),
3762             metrics={"values": values},
3763             source_paths=list(
3764                 dict.fromkeys(
3765                     path
3766                     for participant in participants_with_context
3767                     for path in participant.record_context_paths.values()
3768                 )
3769             ),
3770             entity_scope=[participant.name for participant in participants_with_context],
3771             practical_interpretation=(
3772                 "This supplies participant-level record or standing context "
3773                 "without using it as causal evidence."
3774             ),
3775             strength_label="participant_record_context",
3776             claim_permissions=[ClaimPermission.DESRIPTIVE],
3777             factual_confidence=1.0,
3778             methodological_strength=1.0,

```



```

3779         user_relevance=0.80,
3780         salience=0.82,
3781         recommended_use=RecommendedUse.SUPPORTING_DETAIL,
3782         semantic_level=SemanticLevel.PARTICIPANT,
3783         prohibited_interpretations=[
3784             "Do not infer motivation, upset status or historical "
3785             "significance from participant records alone."
3786         ],
3787     )
3788 ]
3789
3790
3791 def _participant_adjacent_event_context_evidence(
3792     participants: list[EventParticipant],
3793 ) -> list[CapabilityEvidence]:
3794     participants_with_context = [
3795         participant
3796         for participant in participants
3797         if participant.adjacent_event_context
3798     ]
3799     if len(participants_with_context) < 1:
3800         return []
3801
3802     values = [
3803         {
3804             "participant": participant.name,
3805             "values": participant.adjacent_event_context,
3806             "source_paths": [
3807                 path
3808                 for paths in participant.adjacent_event_context_paths.values()
3809                 for path in paths.values()
3810             ],
3811         }
3812         for participant in participants_with_context
3813     ]
3814     return [
3815         CapabilityEvidence(
3816             capability=EvidenceCapability.EVENT_OUTCOME,
3817             evidence_type="participant_record_context",
3818             finding=_participant_adjacent_event_context_finding(
3819                 participants_with_context
3820             ),
3821             metrics={"values": values, "context_kind": "adjacent_event"},
3822             source_paths=list(
3823                 dict.fromkeys(
3824                     path
3825                     for participant in participants_with_context
3826                     for paths in participant.adjacent_event_context_paths.values()
3827                     for path in paths.values()
3828                 )
3829             ),
3830             entity_scope=[
3831                 participant.name
3832                 for participant in participants_with_context
3833             ],
3834             practical_interpretation=(
3835                 "This supplies adjacent scheduled-event context without using "
3836                 "it as evidence about the current event result."
3837             ),
3838             strength_label="adjacent_event_context",
3839             claim_permissions=[ClaimPermission.DESCRPTIVE],
3840             factual_confidence=1.0,
3841             methodological_strength=1.0,
3842             user_relevance=0.78,
3843             salience=0.78,
3844             recommended_use=RecommendedUse.SUPPORTING_DETAIL,
3845             semantic_level=SemanticLevel.PARTICIPANT,
3846             prohibited_interpretations=[
3847                 "Do not infer future outcomes or participant motivation from "
3848                 "adjacent scheduled-event context."
3849             ],
3850         )
3851     ]
3852
3853
3854 ENTITY_DETAIL_METRIC_NAMES = {
3855     "assists",
3856     "blocks",
3857     "doubles",
3858     "earned runs allowed",
3859     "field goals attempted",
3860     "field goals made",
3861     "free throws attempted",
3862     "free throws made",
3863     "hits",
3864     "home runs",
3865     "pitching strikeouts",
3866     "points",
3867     "rebounds",
3868     "runs",
3869     "runs allowed",

```



```

3870     "runs batted in",
3871     "steals",
3872     "strikeouts",
3873     "three-pointers attempted",
3874     "three-pointers made",
3875     "triples",
3876     "walks",
3877 }
3878 SINGULAR_ENTITY_METRIC_NAMES = {
3879     "assists": "assist",
3880     "blocks": "block",
3881     "doubles": "double",
3882     "earned runs allowed": "earned run allowed",
3883     "field goals attempted": "field goal attempted",
3884     "field goals made": "field goal made",
3885     "free throws attempted": "free throw attempted",
3886     "free throws made": "free throw made",
3887     "hits": "hit",
3888     "home runs": "home run",
3889     "pitching strikeouts": "pitching strikeout",
3890     "points": "point",
3891     "rebounds": "rebound",
3892     "runs": "run",
3893     "runs allowed": "run allowed",
3894     "runs batted in": "run batted in",
3895     "steals": "steal",
3896     "strikeouts": "strikeout",
3897     "three-pointers attempted": "three-pointer attempted",
3898     "three-pointers made": "three-pointer made",
3899     "triples": "triple",
3900     "walks": "walk",
3901 }
3902
3903
3904 def _format_entity_metric_phrase(
3905     metric: str,
3906     value: float,
3907 ) -> str:
3908     label = (
3909         SINGULAR_ENTITY_METRIC_NAMES.get(metric, metric)
3910         if abs(value) == 1
3911         else metric
3912     )
3913     return f"{value:g} {label}"
3914
3915
3916 def _entity_detail_metrics(
3917     entity: EventEntity,
3918     primary_metric: str,
3919 ) -> list[str]:
3920     preferred = [
3921         primary_metric,
3922         "rebounds",
3923         "assists",
3924         "steals",
3925         "blocks",
3926         "hits",
3927         "runs batted in",
3928         "home runs",
3929         "runs",
3930         "pitching strikeouts",
3931         "strikeouts",
3932         "doubles",
3933         "triples",
3934         "walks",
3935         "field goals made",
3936         "field goals attempted",
3937         "three-pointers made",
3938         "three-pointers attempted",
3939         "free throws made",
3940         "free throws attempted",
3941     ]
3942     selected: list[str] = []
3943     for metric in dict.fromkeys(preferred):
3944         if metric not in entity.metrics:
3945             continue
3946         if metric not in ENTITY_DETAIL_METRIC_NAMES:
3947             continue
3948         value = entity.metrics[metric]
3949         if value <= 0 and metric != primary_metric:
3950             continue
3951         selected.append(metric)
3952         if len(selected) >= 6:
3953             break
3954     return selected
3955
3956
3957 def _entity_performance_evidence(
3958     *,
3959     entities: list[EventEntity],
3960     participants: list[EventParticipant],

```

```

3961     entity_metrics: list[str],
3962 ) -> list[CapabilityEvidence]:
3963     primary_metric = entity_metrics[0] if entity_metrics else None
3964     if not entities or not primary_metric:
3965         return []
3966
3967     selected: dict[str, EventEntity] = {}
3968     primary_ranked = sorted(
3969         [
3970             entity
3971             for entity in entities
3972             if primary_metric in entity.metrics
3973         ],
3974         key=lambda entity: entity.metrics.get(primary_metric, float("-inf")),
3975         reverse=True,
3976     )
3977     for entity in primary_ranked[:6]:
3978         selected.setdefault(entity.name, entity)
3979
3980     for participant in participants:
3981         participant_ranked = [
3982             entity
3983             for entity in primary_ranked
3984             if entity.participant_name == participant.name
3985         ]
3986         for entity in participant_ranked[:3]:
3987             selected.setdefault(entity.name, entity)
3988
3989     for metric in entity_metrics[:6]:
3990         ranked = sorted(
3991             [
3992                 entity
3993                 for entity in entities
3994                 if metric in entity.metrics and entity.metrics[metric] > 0
3995             ],
3996             key=lambda entity: entity.metrics[metric],
3997             reverse=True,
3998         )
3999         if ranked:
4000             selected.setdefault(ranked[0].name, ranked[0])
4001
4002     evidence: list[CapabilityEvidence] = []
4003     for index, entity in enumerate(selected.values(), start=1):
4004         visible_metrics = _entity_detail_metrics(entity, primary_metric)
4005         if not visible_metrics:
4006             continue
4007         metric_text = ", ".join(
4008             _format_entity_metric_phrase(
4009                 metric,
4010                 entity.metrics[metric],
4011             )
4012             for metric in visible_metrics
4013         )
4014         evidence.append(
4015             CapabilityEvidence(
4016                 capability=EvidenceCapability.ENTITY_PERFORMANCE,
4017                 evidence_type="entity_performance",
4018                 finding=(
4019                     f"{entity.name} recorded {metric_text} for "
4020                     f"{entity.participant_name}."
4021                 ),
4022                 metrics={
4023                     "entity": entity.name,
4024                     "participant": entity.participant_name,
4025                     "primary_metric": primary_metric,
4026                     "selection_rank": index,
4027                     **{
4028                         metric: entity.metrics[metric]
4029                         for metric in visible_metrics
4030                     },
4031                 },
4032                 source_paths=[
4033                     path
4034                     for path in [
4035                         *entity.identity_paths,
4036                         *[
4037                             entity.metric_paths[metric]
4038                             for metric in visible_metrics
4039                         ],
4040                     ]
4041                     if path
4042                 ],
4043                 entity_scope=[entity.name, entity.participant_name],
4044                 practical_interpretation=(
4045                     "This records a salient entity performance within the "
4046                     "supplied event without adding a domain-specific milestone."
4047                 ),
4048                 strength_label="entity_performance",
4049                 claim_permissions=[ClaimPermission.DESCRPTIVE],
4050                 factual_confidence=1.0,
4051                 methodological_strength=1.0,

```

```

4052         user_relevance=0.95,
4053         salience=0.95 if index <= 3 else 0.88,
4054         recommended_use=(
4055             RecommendedUse.MAIN_FINDING
4056             if index <= 3
4057             else RecommendedUse.SUPPORTING_DETAIL
4058         ),
4059         prohibited_interpretations=[
4060             "Do not infer that this performance caused the event result."
4061         ],
4062     )
4063 )
4064 return evidence
4065
4066 def _preferred_score_segment_family(
4067     participants: list[EventParticipant],
4068 ) -> str | None:
4069     family_counts: dict[str, set[str]] = {}
4070     for participant in participants:
4071         for segment in participant.segment_scores:
4072             family = _segment_family(segment)
4073             if family == "extra_period":
4074                 continue
4075             family_counts.setdefault(family, set()).add(segment)
4076     eligible = {
4077         family: segments
4078         for family, segments in family_counts.items()
4079         if family != "unknown" and len(segments) >= 2
4080     }
4081     if not eligible:
4082         return None
4083     family_priority = {
4084         "quarter": 0,
4085         "period": 1,
4086         "inning": 2,
4087         "numeric": 3,
4088         "half": 4,
4089         "named_segment": 5,
4090     }
4091     return min(
4092         eligible,
4093         key=lambda family: family_priority.get(family, 99),
4094     )
4095
4096 def _score_progression_evidence(
4097     participants: list[EventParticipant],
4098 ) -> list[CapabilityEvidence]:
4099     if len(participants) < 2:
4100         return []
4101     family = _preferred_score_segment_family(participants)
4102     if family is None:
4103         return []
4104
4105     segment_names = sorted(
4106         {
4107             segment
4108             for participant in participants
4109             for segment in participant.segment_scores
4110             if _segment_family(segment) == family
4111         },
4112         key=lambda segment: _segment_sort_key(segment) or (99, 99, segment),
4113     )
4114     cumulative: dict[str, float] = {
4115         participant.name: 0.0
4116         for participant in participants
4117     }
4118     progression: list[dict[str, Any]] = []
4119     source_paths: list[str] = []
4120     for segment in segment_names:
4121         segment_values: dict[str, float] = {}
4122         segment_paths: list[str] = []
4123         for participant in participants:
4124             if segment not in participant.segment_scores:
4125                 continue
4126             value, path = participant.segment_scores[segment]
4127             segment_values[participant.name] = value
4128             segment_paths.append(path)
4129         if len(segment_values) < 2:
4130             continue
4131         if all(value == 0 for value in segment_values.values()):
4132             continue
4133         for participant_name, value in segment_values.items():
4134             cumulative[participant_name] = cumulative.get(participant_name, 0.0) + value
4135     ordered_cumulative = sorted(
4136         cumulative.items(),
4137         key=lambda item: item[1],
4138         reverse=True,
4139     )
4140     leader = ordered_cumulative[0][0]

```

```

4143     tied = (
4144         len(ordered_cumulative) > 1
4145         and ordered_cumulative[0][1] == ordered_cumulative[1][1]
4146     )
4147     progression.append(
4148         {
4149             "segment": segment,
4150             "segment_values": segment_values,
4151             "cumulative_values": dict(cumulative),
4152             "leader": None if tied else leader,
4153             "tied": tied,
4154             "source_paths": segment_paths,
4155         }
4156     )
4157     source_paths.extend(segment_paths)
4158
4159     if len(progression) < 2:
4160         return []
4161
4162     highlight_parts: list[str] = []
4163     for item in progression[:4]:
4164         ordered_scores = sorted(
4165             item["cumulative_values"].items(),
4166             key=lambda score_item: score_item[1],
4167             reverse=True,
4168         )
4169         score_text = "-".join(f"{value:g}" for _, value in ordered_scores[:2])
4170         if item["tied"]:
4171             highlight_parts.append(
4172                 f"after {item['segment']}, the cumulative score was level at {score_text}"
4173             )
4174         else:
4175             highlight_parts.append(
4176                 f"after {item['segment']}, {item['leader']} led {score_text}"
4177             )
4178
4179     return [
4180         CapabilityEvidence(
4181             capability=EvidenceCapability.EVENT_OUTCOME,
4182             evidence_type="score_progression",
4183             finding=(
4184                 "The supplied score progression records "
4185                 + "; ".join(highlight_parts)
4186                 + "."
4187             ),
4188             metrics={
4189                 "segment_family": family,
4190                 "segments": progression,
4191             },
4192             source_paths=list(dict.fromkeys(source_paths)),
4193             entity_scope=[participant.name for participant in participants],
4194             practical_interpretation=(
4195                 "This supports bounded narration of recorded score progression "
4196                 "without inferring momentum or causes."
4197             ),
4198             strength_label="score_progression",
4199             claim_permissions=[
4200                 ClaimPermission.DESCRPTIVE,
4201                 ClaimPermission.COMPARATIVE,
4202             ],
4203             factual_confidence=1.0,
4204             methodological_strength=1.0,
4205             user_relevance=0.88,
4206             salience=0.88,
4207             recommended_use=RecommendedUse.MAIN_FINDING,
4208             semantic_level=SemanticLevel.EVENT,
4209             limitations=[
4210                 "Score progression describes supplied segment scores only and "
4211                 "does not establish why the result occurred."
4212             ],
4213             prohibited_interpretations=[
4214                 "Do not infer momentum, a turning point or a comeback unless "
4215                 "a recorded event sequence directly supports it."
4216             ],
4217         )
4218     ]
4219
4220
4221 def available_capabilities(
4222     bundle: Any,
4223     semantic_map: InputSemanticMap | None = None,
4224 ) -> list[EvidenceCapability]:
4225     shape = getattr(getattr(bundle, "input_structure", None), "shape", None)
4226     capabilities = [EvidenceCapability.DATASET_PROFILE]
4227
4228     tables = getattr(bundle, "tables", {})
4229     if any(
4230         {
4231             "attribute_name", "attribute_value"
4232         }.issubset(
4233             set(getattr(frame, "columns", []))
4234         )

```

```

4234         or {"subject", "relation", "object"}.issubset(
4235             set(getattr(frame, "columns", []))
4236         )
4237     )
4238     for frame in tables.values()
4239 ) or _bundle_contains_serialized_triples(bundle):
4240     capabilities.append(EvidenceCapability.STRUCTURED_RECORD_VERBALISATION)
4241
4242 if any(
4243     "is_highlighted" in getattr(frame, "columns", [])
4244     and bool(frame["is_highlighted"].fillna(False).any())
4245     for frame in tables.values()
4246 ):
4247     capabilities.append(EvidenceCapability.FOCUSED_TABLE_REGION)
4248
4249 if shape in {
4250     InputShape.FLAT_TABLE,
4251     InputShape.ENTITY_COLLECTION,
4252     InputShape.TIME_SERIES,
4253 }:
4254     capabilities.extend(
4255         [
4256             EvidenceCapability.MISSINGNESS,
4257             EvidenceCapability.DUPPLICATES,
4258             EvidenceCapability.DISTRIBUTION_SUMMARY,
4259             EvidenceCapability.ASSOCIATION,
4260             EvidenceCapability.GROUP_COMPARISON,
4261         ]
4262     )
4263
4264 if semantic_map is not None and semantic_map.bindings:
4265     semantic_shape = semantic_map.input_shape
4266     roles_by_table: dict[str, set[SemanticRole]] = {}
4267     for binding in semantic_map.bindings:
4268         roles_by_table.setdefault(binding.table_name, set()).add(
4269             binding.role
4270         )
4271     role_sets = list(roles_by_table.values())
4272     if (
4273         semantic_shape == InputShape.EVENT_RECORD
4274         and any(
4275             {
4276                 SemanticRole.PARTICIPANT_IDENTIFIER,
4277                 SemanticRole.OUTCOME_MEASURE,
4278             }.issubset(roles)
4279             for roles in role_sets
4280         )
4281     ):
4282         capabilities.append(EvidenceCapability.EVENT_OUTCOME)
4283     if (
4284         semantic_shape in {
4285             InputShape.ENTITY_COLLECTION,
4286             InputShape.EVENT_RECORD,
4287         }
4288         and any(
4289             SemanticRole.ENTITY_IDENTIFIER in roles
4290             and bool(
4291                 roles
4292                 & {
4293                     SemanticRole.PERFORMANCE_MEASURE,
4294                     SemanticRole.MEASURE,
4295                 }
4296             )
4297             for roles in role_sets
4298         )
4299     ):
4300         capabilities.append(EvidenceCapability.RANKING)
4301     if (
4302         semantic_shape == InputShape.EVENT_RECORD
4303         and any(
4304             {
4305                 SemanticRole.PARTICIPANT_IDENTIFIER,
4306                 SemanticRole.ENTITY_IDENTIFIER,
4307             }.issubset(roles)
4308             and bool(
4309                 roles
4310                 & {
4311                     SemanticRole.PERFORMANCE_MEASURE,
4312                     SemanticRole.MEASURE,
4313                 }
4314             )
4315             for roles in role_sets
4316         )
4317     ):
4318         capabilities.append(EvidenceCapability.ENTITY_PERFORMANCE)
4319     if (
4320         semantic_shape == InputShape.EVENT_RECORD
4321         and any(
4322             SemanticRole.PARTICIPANT_IDENTIFIER in roles
4323             and bool(
4324                 roles

```

```

4325         & {
4326             SemanticRole.PERFORMANCE_MEASURE,
4327             SemanticRole.OUTCOME_MEASURE,
4328             SemanticRole.MEASURE,
4329         }
4330     )
4331     for roles in role_sets
4332 )
4333 ):
4334     capabilities.append(EvidenceCapability.GROUP_COMPARISON)
4335 else:
4336     participants = [
4337         participant
4338         for payload in getattr(bundle, "structured_inputs", {}).values()
4339         for participant in extract_event_participants(payload)
4340     ]
4341     if len(participants) >= 2:
4342         if all(participant.score is not None for participant in participants):
4343             capabilities.append(EvidenceCapability.EVENT_OUTCOME)
4344         if any(participant.entities for participant in participants):
4345             capabilities.extend(
4346                 [
4347                     EvidenceCapability.ENTITY_PERFORMANCE,
4348                     EvidenceCapability.RANKING,
4349                 ]
4350             )
4351         if len([participant for participant in participants if participant.metrics]) >= 2:
4352             capabilities.append(EvidenceCapability.GROUP_COMPARISON)
4353
4354     return list(dict.fromkeys(capabilities))
4355
4356 def event_capability_evidence(payload: Any) -> list[CapabilityEvidence]:
4357     participants = extract_event_participants(payload)
4358     evidence: list[CapabilityEvidence] = [
4359         *_event_context_evidence(payload),
4360         *_event_sequence_evidence(payload),
4361     ]
4362     if len(participants) < 2:
4363         return evidence
4364
4365     evidence.extend(
4366         [
4367             *_participant_record_context_evidence(participants),
4368             *_participant_adjacent_event_context_evidence(participants),
4369             *_score_progression_evidence(participants),
4370         ]
4371     )
4372
4373     scored = [participant for participant in participants if participant.score is not None]
4374
4375     if len(scored) >= 2:
4376         ordered = sorted(scored, key=lambda item: float(item.score or 0), reverse=True)
4377         winner, runner_up = ordered[0], ordered[1]
4378         margin = float(winner.score or 0) - float(runner_up.score or 0)
4379         tied = margin == 0
4380         if tied:
4381             finding = (
4382                 f"{winner.name} and {runner_up.name} finished level at "
4383                 f"{winner.score:g}-{runner_up.score:g}."
4384             )
4385         else:
4386             finding = (
4387                 f"{winner.name} defeated {runner_up.name} "
4388                 f"{winner.score:g}-{runner_up.score:g}, a margin of {margin:g}."
4389             )
4390
4391     evidence.append(
4392         CapabilityEvidence(
4393             capability=EvidenceCapability.EVENT_OUTCOME,
4394             evidence_type="event_outcome",
4395             finding=finding,
4396             metrics={
4397                 "winner": None if tied else winner.name,
4398                 "loser": None if tied else runner_up.name,
4399                 "winner_score": winner.score,
4400                 "loser_score": runner_up.score,
4401                 "margin": margin,
4402                 "tied": tied,
4403             },
4404             source_paths=[
4405                 path
4406                 for path in [
4407                     *winner.identity_paths,
4408                     winner.score_path,
4409                     *runner_up.identity_paths,
4410                     runner_up.score_path,
4411                 ]
4412                 if path
4413             ],
4414             entity_scope=[winner.name, runner_up.name],
4415             practical_interpretation=(

```

```

4416         "This establishes the supported event result and score margin."
4417     ),
4418     strength_label="event_outcome",
4419     claim_permissions=[
4420         ClaimPermission.DESCRPTIVE,
4421         ClaimPermission.COMPARATIVE,
4422     ],
4423     factual_confidence=1.0,
4424     methodological_strength=1.0,
4425     user_relevance=1.0,
4426     salience=1.0,
4427     recommended_use=RecommendedUse.HEADLINE,
4428     prohibited_interpretations=[
4429         "Do not infer chronology, a comeback, dominance, or historical "
4430         "significance from the final score alone."
4431     ],
4432 )
4433 )
4434
4435 all_entities = [entity for participant in participants for entity in participant.entities]
4436 entity_metrics = sorted(
4437     {
4438         metric
4439         for entity in all_entities
4440         for metric in entity.metrics
4441     },
4442     key=_metric_priority,
4443     reverse=True,
4444 )
4445 for metric in entity_metrics[:8]:
4446     ranked = sorted(
4447         [entity for entity in all_entities if metric in entity.metrics],
4448         key=lambda entity: entity.metrics[metric],
4449         reverse=True,
4450     )
4451     if not ranked:
4452         continue
4453     if ranked[0].metrics[metric] > 0:
4454         ranked = [
4455             entity
4456             for entity in ranked
4457             if entity.metrics[metric] > 0
4458         ]
4459     if not ranked:
4460         continue
4461     leaders = ranked[:3]
4462     ranking_text = "; ".join(
4463         f"{entity.name} ({entity.participant_name}) recorded "
4464         f"{entity.metrics[metric]:g}"
4465         for entity in leaders
4466     )
4467     evidence.append(
4468         CapabilityEvidence(
4469             capability=EvidenceCapability.RANKING,
4470             evidence_type="entity_ranking",
4471             finding=f"The leading recorded {metric} performances were: {ranking_text}.",
4472             metrics={
4473                 "metric": metric,
4474                 "ranking": [
4475                     {
4476                         "rank": index,
4477                         "entity": entity.name,
4478                         "participant": entity.participant_name,
4479                         "value": entity.metrics[metric],
4480                     }
4481                     for index, entity in enumerate(leaders, start=1)
4482                 ],
4483             },
4484             source_paths=[
4485                 path
4486                 for entity in leaders
4487                 for path in [
4488                     *entity.identity_paths,
4489                     entity.metric_paths.get(metric),
4490                 ]
4491                 if path
4492             ],
4493             entity_scope=[entity.name for entity in leaders],
4494             practical_interpretation=(
4495                 f"This ranks entities by the observed {metric} metric within "
4496                 "the supplied event."
4497             ),
4498             strength_label="entity_ranking",
4499             claim_permissions=[
4500                 ClaimPermission.DESCRPTIVE,
4501                 ClaimPermission.COMPARATIVE,
4502             ],
4503             factual_confidence=1.0,
4504             methodological_strength=1.0,
4505             user_relevance=0.95,
4506             salience=0.95 if metric == "points" else 0.85,

```

```

4507         recommended_use=(
4508             RecommendedUse.MAIN_FINDING
4509             if metric == entity_metrics[0]
4510             else RecommendedUse.SUPPORTING_DETAIL
4511         ),
4512         limitations=[
4513             "The ranking is limited to entities and metrics recorded in "
4514             "the supplied event structure."
4515         ],
4516         prohibited_interpretations=[
4517             "Do not call a ranked entity historically best or dominant."
4518         ],
4519     )
4520 )
4521
4522 evidence.extend(
4523     _entity_performance_evidence(
4524         entities=all_entities,
4525         participants=participants,
4526         entity_metrics=entity_metrics,
4527     )
4528 )
4529
4530 metric_participants = [participant for participant in participants if participant.metrics]
4531 if len(metric_participants) >= 2:
4532     left, right = metric_participants[:2]
4533     common_metrics = set(left.metrics) & set(right.metrics)
4534     comparisons = sorted(
4535         common_metrics,
4536         key=lambda metric: abs(left.metrics[metric] - right.metrics[metric]),
4537         reverse=True,
4538     )
4539     for metric in comparisons[:6]:
4540         left_value = left.metrics[metric]
4541         right_value = right.metrics[metric]
4542         if left_value == right_value:
4543             continue
4544         leader, trailer = (
4545             (left, right) if left_value > right_value else (right, left)
4546         )
4547         leader_value = leader.metrics[metric]
4548         trailer_value = trailer.metrics[metric]
4549         evidence.append(
4550             CapabilityEvidence(
4551                 capability=EvidenceCapability.GROUP_COMPARISON,
4552                 evidence_type="participant_comparison",
4553                 finding=(
4554                     f"{leader.name} recorded more {metric} than {trailer.name} "
4555                     f"({leader_value:g} versus {trailer_value:g}), a difference "
4556                     f"of {abs(leader_value - trailer_value):g}."
4557                 ),
4558                 metrics={
4559                     "metric": metric,
4560                     "higher_participant": leader.name,
4561                     "lower_participant": trailer.name,
4562                     "higher_value": leader_value,
4563                     "lower_value": trailer_value,
4564                     "difference": abs(leader_value - trailer_value),
4565                 },
4566                 source_paths=[
4567                     *leader.identity_paths,
4568                     leader.metric_paths[metric],
4569                     *trailer.identity_paths,
4570                     trailer.metric_paths[metric],
4571                 ],
4572                 entity_scope=[leader.name, trailer.name],
4573                 practical_interpretation=(
4574                     "This is a direct participant-level contrast within the event."
4575                 ),
4576                 strength_label="participant_comparison",
4577                 claim_permissions=[
4578                     ClaimPermission.DESCRPTIVE,
4579                     ClaimPermission.COMPARATIVE,
4580                 ],
4581                 factual_confidence=1.0,
4582                 methodological_strength=1.0,
4583                 user_relevance=0.8,
4584                 salience=0.75,
4585                 recommended_use=RecommendedUse.SUPPORTING_DETAIL,
4586                 prohibited_interpretations=[
4587                     "Do not call the contrast decisive without explicit evidence."
4588                 ],
4589             )
4590         )
4591
4592 return evidence
4593
4594 @dataclass(frozen=True)
4595 class PathMatch:
4596     source_path: str

```



```

4598     captures: tuple[str, ...]
4599     value: Any
4600
4601
4602 def match_path_pattern(payload: Any, pattern: str) -> list[PathMatch]:
4603     """Resolve a dot path containing mapping/list wildcards."""
4604
4605     parts = [part for part in pattern.split(".") if part]
4606     matches: list[PathMatch] = []
4607
4608     def visit(
4609         current: Any,
4610         index: int,
4611         path_parts: list[str],
4612         captures: list[str],
4613     ) -> None:
4614         if index == len(parts):
4615             matches.append(
4616                 PathMatch(
4617                     source_path=".".join(path_parts),
4618                     captures=tuple(captures),
4619                     value=current,
4620                 )
4621             )
4622             return
4623
4624         part = parts[index]
4625         if part == "*":
4626             if isinstance(current, Mapping):
4627                 for key, child in current.items():
4628                     visit(
4629                         child,
4630                         index + 1,
4631                         [*path_parts, str(key)],
4632                         [*captures, str(key)],
4633                     )
4634             elif isinstance(current, list):
4635                 for item_index, child in enumerate(current):
4636                     visit(
4637                         child,
4638                         index + 1,
4639                         [*path_parts, str(item_index)],
4640                         [*captures, str(item_index)],
4641                     )
4642             return
4643
4644         if isinstance(current, Mapping) and part in current:
4645             visit(
4646                 current[part],
4647                 index + 1,
4648                 [*path_parts, part],
4649                 captures,
4650             )
4651             return
4652
4653         if isinstance(current, list) and part.isdigit():
4654             item_index = int(part)
4655             if 0 <= item_index < len(current):
4656                 visit(
4657                     current[item_index],
4658                     index + 1,
4659                     [*path_parts, part],
4660                     captures,
4661                 )
4662
4663     visit(payload, 0, [], [])
4664     return matches
4665
4666
4667 def validate_semantic_map(
4668     semantic_map: InputSemanticMap | None,
4669     structural_catalog: list[StructuralField],
4670     *,
4671     require_entity_measure_coverage: bool = False,
4672 ) -> list[str]:
4673     if semantic_map is None:
4674         return []
4675
4676     errors: list[str] = []
4677     seen: set[str] = set()
4678     seen_paths: set[tuple[str, str]] = set()
4679     catalog_paths = {(field.table_name, field.path_pattern) for field in structural_catalog}
4680
4681     for binding in semantic_map.bindings:
4682         if binding.binding_id in seen:
4683             errors.append(f"Duplicate semantic binding ID: {binding.binding_id}.")
4684             seen.add(binding.binding_id)
4685         if (binding.table_name, binding.path_pattern) not in catalog_paths:
4686             errors.append(
4687                 f"Semantic binding {binding.binding_id} uses an unknown "
4688                 f"catalog path: {binding.table_name}:{binding.path_pattern}."

```

```

4689         )
4690
4691         binding_path = (binding.table_name, binding.path_pattern)
4692         if binding_path in seen_paths:
4693             errors.append(
4694                 f"Semantic binding {binding.binding_id} repeats catalog path "
4695                 f"{binding.table_name}:{binding.path_pattern}."
4696             )
4697         seen_paths.add(binding_path)
4698
4699     if semantic_map.input_shape != InputShape.EVENT_RECORD:
4700         return errors
4701
4702     measure_roles = {
4703         SemanticRole.OUTCOME_MEASURE,
4704         SemanticRole.PERFORMANCE_MEASURE,
4705         SemanticRole.MEASURE,
4706     }
4707     missing_function_ids = [
4708         binding.binding_id
4709         for binding in semantic_map.bindings
4710         if binding.role in measure_roles
4711         and binding.analytical_function is None
4712     ]
4713     if missing_function_ids:
4714         errors.append(
4715             "Event measure bindings must declare analytical_function: "
4716             + ", ".join(missing_function_ids)
4717             + "."
4718         )
4719
4720     invalid_outcome_ids = [
4721         binding.binding_id
4722         for binding in semantic_map.bindings
4723         if binding.role == SemanticRole.OUTCOME_MEASURE
4724         and binding.analytical_function != AnalyticalFunction.OUTCOME
4725     ]
4726     if invalid_outcome_ids:
4727         errors.append(
4728             "Event outcome bindings must use analytical function 'outcome': "
4729             + ", ".join(invalid_outcome_ids)
4730             + "."
4731         )
4732
4733     invalid_function_ids = [
4734         binding.binding_id
4735         for binding in semantic_map.bindings
4736         if binding.analytical_function
4737         in {
4738             AnalyticalFunction.OUTCOME,
4739             AnalyticalFunction.OUTCOME_COMPONENT,
4740             AnalyticalFunction.PERFORMANCE,
4741             AnalyticalFunction.PARTICIPATION,
4742         }
4743         and binding.role not in measure_roles
4744     ]
4745     if invalid_function_ids:
4746         errors.append(
4747             "Measure analytical functions cannot be assigned to non-measure "
4748             "bindings: "
4749             + ", ".join(invalid_function_ids)
4750             + "."
4751         )
4752
4753     if require_entity_measure_coverage:
4754         entity_measure_groups: dict[tuple[str, str], set[str]] = {}
4755         for binding in semantic_map.bindings:
4756             if binding.role != SemanticRole.ENTITY_IDENTIFIER:
4757                 continue
4758             parent_path = binding.path_pattern.rsplit(".", 1)[0]
4759             key = (binding.table_name, parent_path)
4760             entity_measure_groups.setdefault(key, set())
4761
4762         for key in entity_measure_groups:
4763             table_name, parent_path = key
4764             entity_measure_groups[key] = {
4765                 field.path_pattern
4766                 for field in structural_catalog
4767                 if field.table_name == table_name
4768                 and field.path_pattern.rsplit(".", 1)[0] == parent_path
4769                 and _field_supports_numeric_measure(field)
4770             }
4771
4772     substantive_functions = {
4773         AnalyticalFunction.PERFORMANCE,
4774         AnalyticalFunction.OUTCOME_COMPONENT,
4775     }
4776     for (table_name, parent_path), available_paths in entity_measure_groups.items():
4777         required = min(3, len(available_paths))
4778         if required == 0:
4779             continue

```

```

4780         selected = {
4781             binding.path_pattern
4782             for binding in semantic_map.bindings
4783             if binding.table_name == table_name
4784             and binding.path_pattern.rsplit(".", 1)[0] == parent_path
4785             and binding.level == SemanticLevel.ENTITY
4786             and binding.analytical_function in substantive_functions
4787         }
4788         if len(selected) < required:
4789             errors.append(
4790                 "Event semantic map must reserve substantive entity-performance "
4791                 f"bindings under {table_name}:{parent_path}; found {len(selected)} "
4792                 f"but the catalog supports at least {required}."
4793             )
4794     return errors
4795
4796
4797 def validate_event_query_priorities(
4798     queries: list[EvidenceQuery],
4799     semantic_map: InputSemanticMap | None,
4800     request: str,
4801 ) -> list[str]:
4802     semantic_map = normalise_semantic_map(semantic_map)
4803     if semantic_map is None or semantic_map.input_shape != InputShape.EVENT_RECORD:
4804         return []
4805     substantive_ids = {
4806         binding.binding_id
4807         for binding in semantic_map.bindings
4808         if binding.level == SemanticLevel.ENTITY
4809         and binding.analytical_function
4810         in {
4811             AnalyticalFunction.PERFORMANCE,
4812             AnalyticalFunction.OUTCOME_COMPONENT,
4813         }
4814     }
4815     entity_queries = [
4816         query
4817         for query in queries
4818         if query.evidence_type
4819         in {"entity_ranking", "entity_performance"}
4820     ]
4821     ranking_queries = [
4822         query
4823         for query in entity_queries
4824         if query.evidence_type == "entity_ranking"
4825     ]
4826     errors: list[str] = []
4827
4828     queried_substantive_ids = {
4829         binding_id
4830         for query in ranking_queries
4831         for binding_id in query.value_binding_ids
4832         if binding_id in substantive_ids
4833     }
4834     required = min(3, len(substantive_ids))
4835     if len(queried_substantive_ids) < required:
4836         errors.append(
4837             "Event plan must rank distinct substantive entity-performance "
4838             f"measures when available; found {len(queried_substantive_ids)} "
4839             f"but {required} are required."
4840         )
4841
4842     component_ids = {
4843         binding.binding_id
4844         for binding in semantic_map.bindings
4845         if binding.level == SemanticLevel.PARTICIPANT
4846         and binding.analytical_function
4847         == AnalyticalFunction.OUTCOME_COMPONENT
4848     }
4849     comparison_queries = [
4850         query
4851         for query in queries
4852         if query.evidence_type
4853         in {"participant_comparison", "event_contrast"}
4854     ]
4855     queried_component_ids = {
4856         binding_id
4857         for query in comparison_queries
4858         for binding_id in query.value_binding_ids
4859         if binding_id in component_ids
4860     }
4861     required_components = min(2, len(component_ids))
4862     if len(queried_component_ids) < required_components:
4863         errors.append(
4864             "Event plan must compare distinct participant-level outcome "
4865             f"components when available; found {len(queried_component_ids)} "
4866             f"but {required_components} are required."
4867         )
4868
4869     return errors
4870

```

```

4871
4872
4873 def validate_evidence_queries(
4874     queries: list[EvidenceQuery],
4875     semantic_map: InputSemanticMap | None,
4876     structural_catalog: list[StructuralField],
4877     *,
4878     task_ids: set[str],
4879     available: set[EvidenceCapability],
4880     task_capabilities: dict[str, EvidenceCapability | None] | None = None,
4881 ) -> list[str]:
4882     semantic_map = normalise_semantic_map(semantic_map)
4883     errors = validate_semantic_map(semantic_map, structural_catalog)
4884     if semantic_map is None:
4885         if queries:
4886             errors.append("Evidence queries require an input semantic map.")
4887         return errors
4888
4889     binding_lookup = {binding.binding_id: binding for binding in semantic_map.bindings}
4890     catalog_lookup = {(field.table_name, field.path_pattern): field for field in structural_catalog}
4891     seen_query_ids: set[str] = set()
4892
4893     for query in queries:
4894         if query.query_id in seen_query_ids:
4895             errors.append(f"Duplicate evidence query ID: {query.query_id}.")
4896             seen_query_ids.add(query.query_id)
4897
4898         if re.search(r"(?<!\w)\d+(?:[.,]\d+)?", query.question):
4899             errors.append(
4900                 f"Evidence query {query.query_id} contains a result value in "
4901                 f"its pre-result analytical question."
4902             )
4903
4904         if query.task_id not in task_ids:
4905             errors.append(f"Evidence query {query.query_id} uses unknown task {query.task_id}.")
4906         elif (
4907             task_capabilities is not None
4908             and task_capabilities.get(query.task_id) is not None
4909             and task_capabilities[query.task_id] != query.capability
4910         ):
4911             errors.append(
4912                 f"Evidence query {query.query_id} does not match the capability "
4913                 f"of task {query.task_id}."
4914             )
4915         if query.capability not in available:
4916             errors.append(
4917                 f"Evidence query {query.query_id} uses unavailable capability "
4918                 f"{query.capability.value}."
4919             )
4920         allowed_evidence_types = QUERY_EVIDENCE_TYPES.get(query.capability)
4921         if allowed_evidence_types is not None and query.evidence_type not in allowed_evidence_types:
4922             errors.append(
4923                 f"Evidence query {query.query_id} uses evidence_type "
4924                 f"{query.evidence_type!r}; allowed values for "
4925                 f"{query.capability.value} are "
4926                 f"{sorted(allowed_evidence_types)}."
4927             )
4928         expected_operation = QUERY_OPERATIONS.get(query.evidence_type)
4929         if expected_operation is not None and query.operation != expected_operation:
4930             errors.append(
4931                 f"Evidence query {query.query_id} must use operation "
4932                 f"{expected_operation.value!r} for evidence_type "
4933                 f"{query.evidence_type!r}."
4934             )
4935
4936         referenced_ids = [
4937             *query.value_binding_ids,
4938             *query.context_binding_ids,
4939             *([query.entity_binding_id] if query.entity_binding_id else []),
4940             *([query.group_binding_id] if query.group_binding_id else []),
4941         ]
4942         unknown_ids = [
4943             binding_id for binding_id in referenced_ids if binding_id not in binding_lookup
4944         ]
4945         if unknown_ids:
4946             errors.append(
4947                 f"Evidence query {query.query_id} uses unknown semantic bindings: {unknown_ids}."
4948             )
4949             continue
4950
4951         wrong_table = [
4952             binding_id
4953             for binding_id in referenced_ids
4954             if binding_lookup[binding_id].table_name != query.table_name
4955         ]
4956         if wrong_table:
4957             errors.append(
4958                 f"Evidence query {query.query_id} mixes bindings from another table: {wrong_table}."
4959             )
4960
4961         value_bindings = [

```

```

4962         binding_lookup[binding_id]
4963     for binding_id in query.value_binding_ids
4964         if binding_id in binding_lookup
4965     ]
4966     entity_binding = (
4967         binding_lookup.get(query.entity_binding_id)
4968         if query.entity_binding_id
4969         else None
4970     )
4971     allowed_value_roles = {
4972         "event_outcome": {SemanticRole.OUTCOME_MEASURE},
4973         "event_context": {
4974             SemanticRole.CONTEXT,
4975             SemanticRole.IDENTIFIER,
4976             SemanticRole.LOCATION,
4977             SemanticRole.METADATA,
4978             SemanticRole.TIME,
4979         },
4980         "event_status": {SemanticRole.STATUS},
4981         "entity_performance": {
4982             SemanticRole.MEASURE,
4983             SemanticRole.PERFORMANCE_MEASURE,
4984         },
4985         "entity_ranking": {
4986             SemanticRole.MEASURE,
4987             SemanticRole.PERFORMANCE_MEASURE,
4988         },
4989         "participant_comparison": {
4990             SemanticRole.MEASURE,
4991             SemanticRole.OUTCOME_MEASURE,
4992             SemanticRole.PERFORMANCE_MEASURE,
4993         },
4994         "event_contrast": {
4995             SemanticRole.MEASURE,
4996             SemanticRole.OUTCOME_MEASURE,
4997             SemanticRole.PERFORMANCE_MEASURE,
4998         },
4999     }.get(query.evidence_type)
5000     if allowed_value_roles is not None:
5001         invalid_value_bindings = [
5002             binding.binding_id
5003             for binding in value_bindings
5004             if binding.role not in allowed_value_roles
5005         ]
5006         if invalid_value_bindings:
5007             errors.append(
5008                 f"Evidence query {query.query_id} uses semantically "
5009                 f"incompatible value bindings: {invalid_value_bindings}."
5010             )
5011     expected_entity_level = {
5012         "event_outcome": SemanticLevel.PARTICIPANT,
5013         "participant_comparison": SemanticLevel.PARTICIPANT,
5014         "event_contrast": SemanticLevel.PARTICIPANT,
5015         "entity_performance": SemanticLevel.ENTITY,
5016         "entity_ranking": SemanticLevel.ENTITY,
5017     }.get(query.evidence_type)
5018     if expected_entity_level is not None:
5019         if entity_binding is None:
5020             errors.append(
5021                 f"Evidence query {query.query_id} requires an identifier "
5022                 f"binding at semantic level {expected_entity_level.value!r}."
5023             )
5024         elif entity_binding.level != expected_entity_level:
5025             errors.append(
5026                 f"Evidence query {query.query_id} requires an identifier at "
5027                 f"semantic level {expected_entity_level.value!r}."
5028             )
5029
5030     if not query.value_binding_ids:
5031         errors.append(f"Evidence query {query.query_id} has no value bindings.")
5032     if query.operation in {
5033         EvidenceOperation.COMPARE,
5034         EvidenceOperation.RANK,
5035     }:
5036         if (
5037             query.evidence_type not in PARALLEL_PARTICIPANT_COMPARISON_TYPES
5038             and len(query.value_binding_ids) != 1
5039         ):
5040             errors.append(
5041                 f"Evidence query {query.query_id} must use exactly one measure binding."
5042             )
5043         if query.evidence_type in PARALLEL_PARTICIPANT_COMPARISON_TYPES:
5044             if not query.value_binding_ids:
5045                 errors.append(
5046                     f"Evidence query {query.query_id} must use at least one measure binding."
5047                 )
5048             elif len(query.value_binding_ids) > 1 and not (
5049                 query.operation == EvidenceOperation.COMPARE
5050                 and _allows_parallel_participant_value_bindings(
5051                     evidence_type=query.evidence_type,
5052                     value_bindings=value_bindings,

```

```

5053         )
5054     ):
5055         errors.append(
5056             f"Evidence query {query.query_id} can only use "
5057             "multiple measure bindings when they are aligned "
5058             "side-specific participant values from the same "
5059             "measure family."
5060         )
5061     if query.entity_binding_id is None:
5062         errors.append(
5063             f"Evidence query {query.query_id} requires an entity identifier binding."
5064         )
5065     for binding_id in query.value_binding_ids:
5066         binding = binding_lookup.get(binding_id)
5067         if binding is None:
5068             continue
5069         field = catalog_lookup.get((binding.table_name, binding.path_pattern))
5070         if field is not None and not _field_supports_numeric_measure(field):
5071             errors.append(
5072                 f"Evidence query {query.query_id} uses non-numeric "
5073                 f"measure binding {binding_id}."
5074             )
5075
5076     return errors
5077
5078 def _aligned_label_match(
5079     match: PathMatch,
5080     label_matches: list[PathMatch],
5081 ) -> PathMatch | None:
5082     compatible = [
5083         candidate
5084         for candidate in label_matches
5085         if candidate.captures == match.captures[: len(candidate.captures)]
5086     ]
5087     if not compatible:
5088         return None
5089     return max(compatible, key=lambda item: len(item.captures))
5090
5091 def _aligned_label(
5092     match: PathMatch,
5093     label_matches: list[PathMatch],
5094 ) -> tuple[str | None, str | None]:
5095     selected = _aligned_label_match(match, label_matches)
5096     if selected is None:
5097         return None, None
5098     return str(selected.value), selected.source_path
5099
5100 def _query_permissions(
5101     operation: EvidenceOperation,
5102 ) -> list[ClaimPermission]:
5103     permissions = [ClaimPermission.DESCRPTIVE]
5104     if operation in {EvidenceOperation.COMPARE, EvidenceOperation.RANK}:
5105         permissions.append(ClaimPermission.COMPARATIVE)
5106     return permissions
5107
5108 def semantic_query_evidence(
5109     *,
5110     table_name: str,
5111     payload: Any,
5112     semantic_map: InputSemanticMap,
5113     queries: list[EvidenceQuery],
5114 ) -> list[CapabilityEvidence]:
5115     """Execute validated generic semantic queries without authoring claims."""
5116
5117     semantic_map = normalise_semantic_map(semantic_map) or semantic_map
5118     binding_lookup = {
5119         binding.binding_id: binding
5120         for binding in semantic_map.bindings
5121         if binding.table_name == table_name
5122     }
5123     evidence: list[CapabilityEvidence] = []
5124
5125     def matches_for(binding_id: str) -> list[PathMatch]:
5126         binding = binding_lookup[binding_id]
5127         return match_path_pattern(payload, binding.path_pattern)
5128
5129     for query in queries:
5130         if query.table_name != table_name:
5131             continue
5132         if any(
5133             binding_id not in binding_lookup
5134             for binding_id in [
5135                 *query.value_binding_ids,
5136                 *query.context_binding_ids,
5137                 *(query.entity_binding_id if query.entity_binding_id else []),
5138                 *(query.group_binding_id if query.group_binding_id else []),
5139             ]
5140         ):

```

```

5144 ):
5145     continue
5146
5147 binding_ids = list(
5148     dict.fromkeys(
5149         [
5150             *query.value_binding_ids,
5151             *query.context_binding_ids,
5152             *[query.entity_binding_id if query.entity_binding_id else []],
5153             *[query.group_binding_id if query.group_binding_id else []],
5154         ]
5155     )
5156 )
5157 source_paths: list[str] = []
5158 entity_scope: list[str] = []
5159 metrics: dict[str, Any] = {
5160     "operation": query.operation.value,
5161     "semantic_label": query.semantic_label,
5162     "question": query.question,
5163 }
5164 value_functions = {
5165     binding_id: binding_lookup[binding_id].analytical_function
5166     for binding_id in query.value_binding_ids
5167     if binding_lookup[binding_id].analytical_function is not None
5168 }
5169 query_function = (
5170     next(iter(value_functions.values()))
5171     if len(set(value_functions.values())) == 1
5172     else None
5173 )
5174 if value_functions:
5175     metrics["analytical_functions"] = {
5176         binding_id: analytical_function.value
5177         for binding_id, analytical_function
5178         in value_functions.items()
5179     }
5180 if query_function is not None:
5181     metrics["analytical_function"] = query_function.value
5182
5183 if query.operation == EvidenceOperation.RETRIEVE:
5184     values: list[dict[str, Any]] = []
5185     entity_matches = matches_for(query.entity_binding_id) if query.entity_binding_id else []
5186     group_matches = matches_for(query.group_binding_id) if query.group_binding_id else []
5187     context_matches = {
5188         binding_id: matches_for(binding_id) for binding_id in query.context_binding_ids
5189     }
5190     for binding_id in query.value_binding_ids:
5191         binding = binding_lookup[binding_id]
5192         for match in matches_for(binding_id):
5193             entity, entity_path = _aligned_label(
5194                 match,
5195                 entity_matches,
5196             )
5197             group, group_path = _aligned_label(
5198                 match,
5199                 group_matches,
5200             )
5201             context: dict[str, Any] = {}
5202             context_paths: list[str] = []
5203             for context_id, candidates in context_matches.items():
5204                 context_value, context_path = _aligned_label(
5205                     match,
5206                     candidates,
5207                 )
5208                 if context_value is not None:
5209                     context[binding_lookup[context_id].label] = context_value
5210                 if context_path is not None:
5211                     context_paths.append(context_path)
5212             values.append(
5213                 {
5214                     "binding_id": binding_id,
5215                     "label": binding.label,
5216                     "role": binding.role.value,
5217                     "analytical_function": (
5218                         binding.analytical_function.value
5219                         if binding.analytical_function is not None
5220                         else None
5221                     ),
5222                     "value": match.value,
5223                     "entity": entity,
5224                     "group": group,
5225                     "context": context,
5226                     "source_path": match.source_path,
5227                 }
5228             )
5229     entity_scope.extend(
5230         value for value in [entity, group, *context.values()] if value
5231     )
5232     source_paths.extend(
5233         path
5234         for path in [

```



```

5235         match.source_path,
5236         entity_path,
5237         group_path,
5238         *context_paths,
5239     ]
5240     if path
5241 )
5242 if not values:
5243     continue
5244 metrics["values"] = values
5245
5246 else:
5247     value_binding_ids = query.value_binding_ids
5248     value_bindings_for_query = [
5249         binding_lookup[binding_id]
5250         for binding_id in value_binding_ids
5251         if binding_id in binding_lookup
5252     ]
5253     participant_identifiers = [
5254         binding
5255         for binding in binding_lookup.values()
5256         if binding.role == SemanticRole.PARTICIPANT_IDENTIFIER
5257         and binding.level == SemanticLevel.PARTICIPANT
5258     ]
5259     side_specific_values = _allows_parallel_participant_value_bindings(
5260         evidence_type=query.evidence_type,
5261         value_bindings=value_bindings_for_query,
5262     )
5263     entity_matches_by_binding: dict[str, list[PathMatch]] = {}
5264     if side_specific_values:
5265         participant_binding_ids = [
5266             binding.binding_id
5267             for binding in participant_identifiers
5268         ]
5269         entity_matches_by_binding = {
5270             binding_id: matches_for(binding_id)
5271             for binding_id in participant_binding_ids
5272         }
5273     else:
5274         entity_matches_by_binding = {
5275             query.entity_binding_id or "": matches_for(
5276                 query.entity_binding_id or ""
5277             )
5278         }
5279     group_matches = (
5280         matches_for(query.group_binding_id)
5281         if query.group_binding_id
5282         else []
5283     )
5284     context_matches = {
5285         binding_id: matches_for(binding_id)
5286         for binding_id in query.context_binding_ids
5287     }
5288     records: list[dict[str, Any]] = []
5289     extra_binding_ids: list[str] = []
5290
5291     for value_binding_id in value_binding_ids:
5292         current_value_binding = binding_lookup[value_binding_id]
5293         value_matches = [
5294             match
5295             for match in matches_for(value_binding_id)
5296             if _numeric_value(match.value) is not None
5297         ]
5298         current_entity_binding = (
5299             _participant_identifier_for_measure(
5300                 current_value_binding,
5301                 participant_identifiers,
5302             )
5303             if side_specific_values
5304             else binding_lookup.get(query.entity_binding_id or "")
5305         )
5306         current_entity_matches = (
5307             entity_matches_by_binding.get(
5308                 current_entity_binding.binding_id,
5309                 [],
5310             )
5311             if current_entity_binding is not None
5312             else []
5313         )
5314         if current_entity_binding is not None:
5315             extra_binding_ids.append(current_entity_binding.binding_id)
5316
5317         for value_match in value_matches:
5318             entity, entity_path = _aligned_label(
5319                 value_match,
5320                 current_entity_matches,
5321             )
5322             if entity is None:
5323                 continue
5324             group_match = _aligned_label_match(
5325                 value_match,

```



```

5326         group_matches,
5327     )
5328     if (
5329         group_match is not None
5330         and value_match.captures
5331         and not group_match.captures
5332     ):
5333         group_match = None
5334     group = (
5335         str(group_match.value)
5336         if group_match is not None
5337         else None
5338     )
5339     group_path = (
5340         group_match.source_path
5341         if group_match is not None
5342         else None
5343     )
5344     context: dict[str, Any] = {}
5345     context_paths: list[str] = []
5346     for binding_id, candidates in context_matches.items():
5347         context_value, context_path = _aligned_label(
5348             value_match,
5349             candidates,
5350         )
5351         if context_value is not None:
5352             context[binding_lookup[binding_id].label] = context_value
5353         if context_path is not None:
5354             context_paths.append(context_path)
5355
5356     record = {
5357         "entity": entity,
5358         "group": group,
5359         "value": _numeric_value(value_match.value),
5360         "measure": current_value_binding.label,
5361         "context": context,
5362         "source_path": value_match.source_path,
5363     }
5364     records.append(record)
5365     entity_scope.extend(
5366         value
5367         for value in [entity, group, *context.values()]
5368         if value
5369     )
5370     source_paths.extend(
5371         path
5372         for path in [
5373             value_match.source_path,
5374             entity_path,
5375             group_path,
5376             *context_paths,
5377         ]
5378         if path
5379     )
5380
5381     if not records:
5382         continue
5383     ordered = sorted(
5384         records,
5385         key=lambda item: item["value"],
5386         reverse=query.descending,
5387     )
5388     if query.operation == EvidenceOperation.RANK:
5389         if (
5390             query.descending
5391             and ordered
5392             and ordered[0]["value"] <= 0
5393             and query_function
5394             in {
5395                 AnalyticalFunction.OUTCOME_COMPONENT,
5396                 AnalyticalFunction.PERFORMANCE,
5397             }
5398         ):
5399             continue
5400     ranking_source = ordered
5401     if (
5402         query.descending
5403         and ordered
5404         and ordered[0]["value"] > 0
5405     ):
5406         ranking_source = [
5407             record
5408             for record in ordered
5409             if record["value"] > 0
5410         ]
5411     selected_records = ranking_source[: query.limit]
5412     value_counts = {
5413         record["value"]: sum(
5414             candidate["value"] == record["value"]
5415             for candidate in ranking_source
5416         )

```

```

5417         for record in selected_records
5418     }
5419     ranking: list[dict[str, Any]] = []
5420     previous_value: float | None = None
5421     current_rank = 0
5422     for index, record in enumerate(
5423         selected_records,
5424         start=1,
5425     ):
5426         if record["value"] != previous_value:
5427             current_rank = index
5428             previous_value = record["value"]
5429             ranking.append(
5430                 {
5431                     **record,
5432                     "rank": current_rank,
5433                     "tied": value_counts[record["value"]] > 1,
5434                 }
5435             )
5436         metrics["ranking"] = ranking
5437         metrics["ties_present"] = any(
5438             record["tied"] for record in ranking
5439         )
5440     else:
5441         comparable_records, level_filter = _comparable_event_records(
5442             ordered
5443         )
5444         metrics["records"] = comparable_records
5445         if level_filter is not None:
5446             metrics["record_level_filter"] = level_filter
5447             metrics["unfiltered_record_count"] = len(ordered)
5448             source_paths = [
5449                 str(record["source_path"])
5450                 for record in comparable_records
5451                 if record.get("source_path")
5452             ]
5453             entity_scope = [
5454                 value
5455                 for record in comparable_records
5456                 for value in [
5457                     record.get("entity"),
5458                     record.get("group"),
5459                     *record.get("context", {}).values(),
5460                 ]
5461                 if value
5462             ]
5463             if len(comparable_records) >= 2:
5464                 metrics["difference"] = abs(
5465                     comparable_records[0]["value"]
5466                     - comparable_records[-1]["value"]
5467                 )
5468                 metrics["tied"] = (
5469                     comparable_records[0]["value"]
5470                     == comparable_records[-1]["value"]
5471                 )
5472             binding_ids = list(dict.fromkeys([*binding_ids, *extra_binding_ids]))
5473
5474     confidences = [binding_lookup[binding_id].confidence for binding_id in binding_ids]
5475     evidence.append(
5476         CapabilityEvidence(
5477             capability=query.capability,
5478             evidence_type=query.evidence_type,
5479             finding=f"Validated semantic query result for `{query.semantic_label}`.",
5480             metrics=metrics,
5481             source_paths=list(dict.fromkeys(source_paths)),
5482             entity_scope=list(dict.fromkeys(entity_scope)),
5483             practical_interpretation=query.question,
5484             strength_label=f"semantic_{query.operation.value}",
5485             claim_permissions=_query_permissions(query.operation),
5486             factual_confidence=min(confidences, default=0.75),
5487             methodological_strength=1.0,
5488             user_relevance=query.user_relevance,
5489             salience=query.salience,
5490             recommended_use=query.recommended_use,
5491             semantic_level=query.semantic_level,
5492             semantic_binding_ids=binding_ids,
5493             analytical_function=query_function,
5494             query_id=query.query_id,
5495             limitations=["The result is limited to values present in the supplied record."],
5496             prohibited_interpretations=[
5497                 "Do not infer causality, chronology, or broader historical "
5498                 "significance from this result."
5499             ],
5500         ),
5501     )
5502
5503     return evidence

```

Listing 24: P24: table2text_pydanticai/src/table2text/cli.py

```

1  """Command-line entry points for running the Table2Text workflow."""
2
3  from __future__ import annotations
4
5  import argparse
6  import json
7  from dataclasses import replace
8  from pathlib import Path
9
10 from .config import Settings
11 from .schemas import (
12     AuditMode,
13     EvaluationFieldPolicy,
14     ExternalTruthSource,
15     ReportGenre,
16 )
17 from .workflow import Table2TextWorkflow
18
19
20 def load_external_truth(
21     path: str | None,
22 ) -> list[ExternalTruthSource]:
23     if not path:
24         return []
25
26     payload = json.loads(
27         Path(path).read_text(encoding="utf-8")
28     )
29
30     if isinstance(payload, dict):
31         payload = payload.get("sources", [payload])
32
33     if not isinstance(payload, list):
34         raise ValueError(
35             "External truth JSON must contain a source list."
36         )
37
38     return [
39         ExternalTruthSource.model_validate(source)
40         for source in payload
41     ]
42
43
44 def build_parser() -> argparse.ArgumentParser:
45     parser = argparse.ArgumentParser(
46         prog="table2text",
47         description=(
48             "Run the six-agent PydanticAI Table2Text pipeline."
49         ),
50     )
51
52     subparsers = parser.add_subparsers(
53         dest="command",
54         required=True,
55     )
56
57     run_parser = subparsers.add_parser(
58         "run",
59         help="Run the full Table2Text workflow.",
60     )
61
62     run_parser.add_argument(
63         "inputs",
64         nargs="+",
65         help="Input tables or directories.",
66     )
67
68     run_parser.add_argument(
69         "--request",
70         required=True,
71         help="The data-science reporting objective.",
72     )
73
74     run_parser.add_argument(
75         "--audit-mode",
76         default=AuditMode.INTERNAL.value,
77         choices=[mode.value for mode in AuditMode],
78     )
79
80     run_parser.add_argument(
81         "--external-truth",
82         help="JSON file containing trusted external facts.",
83     )
84
85     run_parser.add_argument(
86         "--no-llm",
87         action="store_true",
88         help="Use deterministic fallbacks without LLM calls.",
89     )

```

```

90
91     run_parser.add_argument(
92         "--output-dir",
93         help="Override the run artifact directory.",
94     )
95
96     run_parser.add_argument(
97         "--allow-experimental-targets",
98         action="store_true",
99         help=(
100             "Allow unconfirmed candidate targets for explicit "
101             "modelling experiments."
102         ),
103     )
104
105     run_parser.add_argument(
106         "--report-genre",
107         choices=[genre.value for genre in ReportGenre],
108         help="Set an experiment-level report-genre contract.",
109     )
110     run_parser.add_argument(
111         "--operational-input-path",
112         action="append",
113         default=[],
114         help="Declare an operational JSON path; repeat for multiple paths.",
115     )
116     run_parser.add_argument(
117         "--held-out-reference-path",
118         action="append",
119         default=[],
120         help="Declare a held-out evaluation-reference JSON path.",
121     )
122     run_parser.add_argument(
123         "--metadata-path",
124         action="append",
125         default=[],
126         help="Declare a metadata-only JSON path.",
127     )
128
129     return parser
130
131
132 def main() -> None:
133     arguments = build_parser().parse_args()
134     settings = Settings.from_env()
135
136     if arguments.no_llm:
137         settings = replace(
138             settings,
139             use_llm=False,
140         )
141
142     if arguments.output_dir:
143         settings = replace(
144             settings,
145             output_dir=Path(arguments.output_dir),
146         )
147
148     if arguments.allow_experimental_targets:
149         settings = replace(
150             settings,
151             allow_experimental_targets=True,
152         )
153
154     workflow = Table2TextWorkflow(settings)
155
156     result = workflow.run_sync(
157         inputs=arguments.inputs,
158         request=arguments.request,
159         audit_mode=AuditMode(arguments.audit_mode),
160         external_truth_sources=load_external_truth(
161             arguments.external_truth
162         ),
163         evaluation_field_policy=EvaluationFieldPolicy(
164             operational_input_paths=arguments.operational_input_path,
165             held_out_reference_paths=arguments.held_out_reference_path,
166             metadata_paths=arguments.metadata_path,
167         ),
168         report_genre=(
169             ReportGenre(arguments.report_genre)
170             if arguments.report_genre
171             else None
172         ),
173     )
174
175     print(f"Run ID: {result.run_id}")
176     print(f"Release status: {result.release_status.value}")
177     print(f"Approved for release: {result.approved_for_release}")
178     print(f"Repair rounds: {result.repair_rounds_used}")
179     print(f"Writer mode: {result.raw_writer_output.writer_mode}")
180     print(f"Artifacts: {settings.output_dir / result.run_id}")

```

Listing 25: P25: table2text_pydanticai/src/table2text/config.py

```

1  """Environment-backed configuration for workflow models, budgets, and gates."""
2
3  from __future__ import annotations
4
5  import os
6  from dataclasses import dataclass
7  from pathlib import Path
8
9
10 _DOTENV_LOADED_KEYS: set[str] = set()
11
12
13 def _candidate_env_files() -> list[Path]:
14     cwd = Path.cwd()
15     package_root = Path(__file__).resolve().parents[2]
16     candidates = [
17         *(directory / ".env" for directory in reversed([cwd, *cwd.parents])),
18         package_root / ".env",
19     ]
20     unique: list[Path] = []
21     seen: set[Path] = set()
22     for candidate in candidates:
23         resolved = candidate.resolve()
24         if resolved not in seen:
25             unique.append(candidate)
26             seen.add(resolved)
27     return unique
28
29
30 def _parse_env_line(line: str) -> tuple[str, str] | None:
31     stripped = line.strip()
32     if not stripped or stripped.startswith("#"):
33         return None
34     if stripped.startswith("export "):
35         stripped = stripped[len("export "):].strip()
36     if "=" not in stripped:
37         return None
38     key, value = stripped.split("=", 1)
39     key = key.strip()
40     value = value.strip()
41     if not key:
42         return None
43     if len(value) >= 2 and value[0] == value[-1] and value[0] in {"'", '"'}:
44         value = value[1:-1]
45     return key, value
46
47
48 def load_env_files(paths: list[Path] | None = None) -> None:
49     for path in paths or _candidate_env_files():
50         if not path.exists() or not path.is_file():
51             continue
52         for line in path.read_text(encoding="utf-8").splitlines():
53             parsed = _parse_env_line(line)
54             if parsed is None:
55                 continue
56             key, value = parsed
57             if key in os.environ and key not in _DOTENV_LOADED_KEYS:
58                 continue
59             os.environ[key] = value
60             _DOTENV_LOADED_KEYS.add(key)
61
62
63 def env_bool(name: str, default: bool) -> bool:
64     value = os.getenv(name)
65     if value is None:
66         return default
67     return value.strip().lower() in {"1", "true", "yes", "on"}
68
69
70 def env_int(name: str, default: int) -> int:
71     value = os.getenv(name)
72     return default if value is None else int(value)
73
74
75 def env_optional_int(
76     name: str,
77     default: int | None = None,
78 ) -> int | None:
79     value = os.getenv(name)
80     if value is None:
81         return default
82     if value.strip().lower() in {"", "none", "null", "unlimited", "uncapped"}:
83         return None
84     return int(value)
85
86
87 def env_float(name: str, default: float) -> float:
88     value = os.getenv(name)
89     return default if value is None else float(value)

```

```

90
91
92 @dataclass(frozen=True)
93 class Settings:
94     use_llm: bool = True
95     output_dir: Path = Path("runs")
96
97     max_revision_rounds: int = 2
98     max_agent_requests: int = 4
99     max_total_tokens: int = 24_000
100
101     random_seed: int = 42
102     max_analysis_rows: int = 50_000
103     structured_output_mode: str = "native"
104
105     full_data_correlation_limit: int = 250_000
106     min_abs_correlation: float = 0.20
107     max_correlation_findings: int = 6
108     max_group_findings: int = 8
109
110     target_proxy_correlation: float = 0.98
111     allow_experimental_targets: bool = False
112
113     forecast_folds: int = 3
114     max_forecast_test_points: int = 2_000
115
116     writer_target_words: int = 650
117     writer_max_words: int | None = None
118     writer_max_main_findings: int | None = None
119     repair_candidates_per_sentence: int = 3
120     writer_quality_revision_rounds: int = 1
121     writer_priority_fact_limit: int | None = None
122     writer_supporting_fact_limit: int | None = None
123     force_llm_short_form_writer: bool = False
124
125     enable_insight_synthesis: bool = True
126     max_insight_candidates: int | None = None
127     max_verified_main_insights: int | None = None
128     min_insight_confidence: float = 0.75
129     min_insight_salience: float = 0.65
130     min_facts_per_bounded_insight: int = 2
131     allow_hypotheses_in_report: bool = False
132
133     # Deterministic coverage recovery from the trusted evidence ledger.
134     minimum_writer_ready_fact_count: int = 6
135     minimum_overview_fact_count: int = 1
136     minimum_data_quality_fact_count: int = 1
137     minimum_relationship_fact_count: int = 2
138
139     maximum_recovered_overview_facts: int = 2
140     maximum_recovered_data_quality_facts: int = 3
141     maximum_recovered_correlation_facts: int = 3
142     maximum_recovered_group_comparison_facts: int = 2
143     maximum_recovered_modelling_facts: int = 1
144
145     minimum_main_finding_score: float = 0.65
146     minimum_main_effect_strength: str = "moderate"
147
148     minimum_report_word_ratio: float = 0.45
149     minimum_report_word_floor: int = 160
150     maximum_repeated_caveat_mentions: int = 4
151
152     zero_unusual_rate_threshold: float = 0.05
153
154     ollama_base_url: str = "http://localhost:11434/v1"
155
156     data_understanding_model: str = "ollama:gemma3:4b"
157     orchestrator_model: str = "ollama:gemma3:12b"
158     evidence_model: str = "ollama:gemma3:12b"
159     verifier_model: str = "ollama:gemma3:12b"
160     writer_model: str = "ollama:gemma3:12b"
161     auditor_model: str = "ollama:gemma3:12b"
162
163     def __post_init__(self) -> None:
164         if (
165             self.max_insight_candidates is not None
166             and self.max_insight_candidates <= 0
167         ):
168             raise ValueError(
169                 "max_insight_candidates must be positive."
170             )
171
172         if (
173             self.max_verified_main_insights is not None
174             and self.max_verified_main_insights <= 0
175         ):
176             raise ValueError(
177                 "max_verified_main_insights must be positive."
178             )
179
180         if (

```

```

181         self.max_verified_main_insights is not None
182         and self.max_insight_candidates is not None
183         and self.max_verified_main_insights
184         > self.max_insight_candidates
185     ):
186         raise ValueError(
187             "max_verified_main_insights cannot exceed "
188             "max_insight_candidates."
189         )
190
191     if (
192         self.writer_max_words is not None
193         and self.writer_max_words < 150
194     ):
195         raise ValueError(
196             "writer_max_words must be at least 150."
197         )
198
199     for field_name, value in {
200         "writer_max_main_findings": self.writer_max_main_findings,
201         "writer_priority_fact_limit": self.writer_priority_fact_limit,
202         "writer_supporting_fact_limit": self.writer_supporting_fact_limit,
203     }.items():
204         if value is not None and value <= 0:
205             raise ValueError(
206                 f"{field_name} must be positive when configured."
207             )
208
209     for field_name, value in {
210         "min_insight_confidence": self.min_insight_confidence,
211         "min_insight_salience": self.min_insight_salience,
212     }.items():
213         if not 0.0 <= value <= 1.0:
214             raise ValueError(
215                 f"{field_name} must be between 0 and 1."
216             )
217
218     if self.min_facts_per_bounded_insight < 1:
219         raise ValueError(
220             "min_facts_per_bounded_insight must be at least 1."
221         )
222
223     @classmethod
224     def from_env(cls) -> "Settings":
225         load_env_files()
226
227         output_mode = os.getenv(
228             "T2T_STRUCTURED_OUTPUT_MODE",
229             "native",
230         ).strip().lower()
231
232         if output_mode not in {"native", "prompted"}:
233             raise ValueError(
234                 "T2T_STRUCTURED_OUTPUT_MODE must be 'native' or 'prompted'."
235             )
236
237         return cls(
238             use_llm=env_bool("T2T_USE_LLM", True),
239             output_dir=Path(os.getenv("T2T_OUTPUT_DIR", "runs")),
240             max_revision_rounds=env_int("T2T_MAX_REVISION_ROUNDS", 2),
241             max_agent_requests=env_int("T2T_MAX_AGENT_REQUESTS", 4),
242             max_total_tokens=env_int("T2T_MAX_TOTAL_TOKENS", 24_000),
243             random_seed=env_int("T2T_RANDOM_SEED", 42),
244             max_analysis_rows=env_int("T2T_MAX_ANALYSIS_ROWS", 50_000),
245             structured_output_mode=output_mode,
246             full_data_correlation_limit=env_int(
247                 "T2T_FULL_DATA_CORRELATION_LIMIT",
248                 250_000,
249             ),
250             min_abs_correlation=env_float(
251                 "T2T_MIN_ABS_CORRELATION",
252                 0.20,
253             ),
254             max_correlation_findings=env_int(
255                 "T2T_MAX_CORRELATION_FINDINGS",
256                 6,
257             ),
258             max_group_findings=env_int(
259                 "T2T_MAX_GROUP_FINDINGS",
260                 8,
261             ),
262             target_proxy_correlation=env_float(
263                 "T2T_TARGET_PROXY_CORRELATION",
264                 0.98,
265             ),
266             allow_experimental_targets=env_bool(
267                 "T2T_ALLOW_EXPERIMENTAL_TARGETS",
268                 False,
269             ),
270             forecast_folds=env_int("T2T_FORECAST_FOLDS", 3),
271             max_forecast_test_points=env_int(

```

```

272         "T2T_MAX_FORECAST_TEST_POINTS",
273         2_000,
274     ),
275     writer_target_words=env_int(
276         "T2T_WRITER_TARGET_WORDS",
277         650,
278     ),
279     writer_max_words=env_optional_int(
280         "T2T_WRITER_MAX_WORDS",
281     ),
282     writer_max_main_findings=env_optional_int(
283         "T2T_WRITER_MAX_MAIN_FINDINGS",
284     ),
285     repair_candidates_per_sentence=env_int(
286         "T2T_REPAIR_CANDIDATES_PER_SENTENCE",
287         3,
288     ),
289     writer_quality_revision_rounds=min(
290         env_int("T2T_WRITER_QUALITY_REVISION_ROUNDS", 1),
291         1,
292     ),
293     writer_priority_fact_limit=env_optional_int(
294         "T2T_WRITER_PRIORITY_FACT_LIMIT",
295     ),
296     writer_supporting_fact_limit=env_optional_int(
297         "T2T_WRITER_SUPPORTING_FACT_LIMIT",
298     ),
299     force_llm_short_form_writer=env_bool(
300         "T2T_FORCE_LLM_SHORT_FORM_WRITER",
301         False,
302     ),
303     enable_insight_synthesis=env_bool(
304         "T2T_ENABLE_INSIGHT_SYNTHESIS",
305         True,
306     ),
307     max_insight_candidates=env_optional_int(
308         "T2T_MAX_INSIGHT_CANDIDATES",
309     ),
310     max_verified_main_insights=env_optional_int(
311         "T2T_MAX_VERIFIED_MAIN_INSIGHTS",
312     ),
313     min_insight_confidence=env_float(
314         "T2T_MIN_INSIGHT_CONFIDENCE",
315         0.75,
316     ),
317     min_insight_salience=env_float(
318         "T2T_MIN_INSIGHT_SALIENCE",
319         0.65,
320     ),
321     min_facts_per_bounded_insight=env_int(
322         "T2T_MIN_FACTS_PER_BOUNDED_INSIGHT",
323         2,
324     ),
325     allow_hypotheses_in_report=env_bool(
326         "T2T_ALLOW_HYPOTHESES_IN_REPORT",
327         False,
328     ),
329     minimum_writer_ready_fact_count=env_int(
330         "T2T_MINIMUM_WRITER_READY_FACT_COUNT",
331         6,
332     ),
333     minimum_overview_fact_count=env_int(
334         "T2T_MINIMUM_OVERVIEW_FACT_COUNT",
335         1,
336     ),
337     minimum_data_quality_fact_count=env_int(
338         "T2T_MINIMUM_DATA_QUALITY_FACT_COUNT",
339         1,
340     ),
341     minimum_relationship_fact_count=env_int(
342         "T2T_MINIMUM_RELATIONSHIP_FACT_COUNT",
343         2,
344     ),
345     maximum_recovered_overview_facts=env_int(
346         "T2T_MAXIMUM_RECOVERED_OVERVIEW_FACTS",
347         2,
348     ),
349     maximum_recovered_data_quality_facts=env_int(
350         "T2T_MAXIMUM_RECOVERED_DATA_QUALITY_FACTS",
351         3,
352     ),
353     maximum_recovered_correlation_facts=env_int(
354         "T2T_MAXIMUM_RECOVERED_CORRELATION_FACTS",
355         3,
356     ),
357     maximum_recovered_group_comparison_facts=env_int(
358         "T2T_MAXIMUM_RECOVERED_GROUP_COMPARISON_FACTS",
359         2,
360     ),
361     maximum_recovered_modelling_facts=env_int(
362         "T2T_MAXIMUM_RECOVERED_MODELING_FACTS",

```



```

363         1,
364     ),
365     minimum_main_finding_score=env_float(
366         "T2T_MINIMUM_MAIN_FINDING_SCORE",
367         0.65,
368     ),
369     minimum_main_effect_strength=os.getenv(
370         "T2T_MINIMUM_MAIN_EFFECT_STRENGTH",
371         "moderate",
372     ),
373     minimum_report_word_ratio=env_float(
374         "T2T_MINIMUM_REPORT_WORD_RATIO",
375         0.45,
376     ),
377     minimum_report_word_floor=env_int(
378         "T2T_MINIMUM_REPORT_WORD_FLOOR",
379         160,
380     ),
381     maximum_repeated_caveat_mentions=env_int(
382         "T2T_MAXIMUM_REPEATED_CAVEAT_MENTIONS",
383         4,
384     ),
385     zero_unusual_rate_threshold=env_float(
386         "T2T_ZERO_UNUSUAL_RATE_THRESHOLD",
387         0.05,
388     ),
389     ollama_base_url=os.getenv(
390         "OLLAMA_BASE_URL",
391         "http://localhost:11434/v1",
392     ),
393     data_understanding_model=os.getenv(
394         "T2T_MODEL_DATA_UNDERSTANDING",
395         "ollama:gemma3:4b",
396     ),
397     orchestrator_model=os.getenv(
398         "T2T_MODEL_ORCHESTRATOR",
399         "ollama:gemma3:12b",
400     ),
401     evidence_model=os.getenv(
402         "T2T_MODEL_EVIDENCE",
403         "ollama:gemma3:12b",
404     ),
405     verifier_model=os.getenv(
406         "T2T_MODEL_VERIFIER",
407         "ollama:gemma3:12b",
408     ),
409     writer_model=os.getenv(
410         "T2T_MODEL_WRITER",
411         "ollama:gemma3:12b",
412     ),
413     auditor_model=os.getenv(
414         "T2T_MODEL_AUDITOR",
415         "ollama:gemma3:12b",
416     ),
417 )
418
419 def model_for(self, role: str) -> str:
420     mapping = {
421         "data_understanding": self.data_understanding_model,
422         "orchestrator": self.orchestrator_model,
423         "evidence": self.evidence_model,
424         "verifier": self.verifier_model,
425         "writer": self.writer_model,
426         "auditor": self.auditor_model,
427     }
428
429     try:
430         return mapping[role]
431     except KeyError as error:
432         raise ValueError(f"Unknown model role: {role}") from error

```

Listing 26: P26: table2text_pydanticai/src/table2text/data.py

```

1  """Load supported data formats and build deterministic table profiles."""
2
3  from __future__ import annotations
4
5  import hashlib
6  import json
7  import math
8  import re
9  from dataclasses import dataclass, field
10 from pathlib import Path
11 from typing import Any, Iterable
12
13 import pandas as pd
14
15 from .schemas import (
16     ColumnProfile,

```

```

17     DataProfile,
18     EvaluationFieldPolicy,
19     InputRepresentationStatus,
20     InputShape,
21     InputStructureProfile,
22     TableProfile,
23     ZeroRisk,
24 )
25 from .structure import combine_structure_profiles, inspect_and_filter_payload
26
27
28 SUPPORTED_EXTENSIONS = {
29     ".csv",
30     ".json",
31     ".jsonl",
32     ".ndjson",
33     ".parquet",
34     ".xlsx",
35     ".xls",
36 }
37
38 VALID_ZERO_TERMS = {
39     "bearing",
40     "direction",
41     "angle",
42     "degree",
43     "degrees",
44     "count",
45     "number",
46     "index",
47     "flag",
48     "binary",
49 }
50
51 CONTEXT_DEPENDENT_ZERO_TERMS = {
52     "visibility",
53     "speed",
54     "precipitation",
55     "rain",
56     "snow",
57     "distance",
58 }
59
60 POSSIBLE_SENTINEL_ZERO_TERMS = {
61     "pressure",
62     "blood pressure",
63 }
64
65
66 @dataclass
67 class DataBundle:
68     tables: dict[str, pd.DataFrame]
69     source_paths: list[Path]
70     fingerprint: str
71     structured_inputs: dict[str, Any] = field(default_factory=dict)
72     input_structure: InputStructureProfile | None = None
73     evaluation_field_policy: EvaluationFieldPolicy = field(
74         default_factory=EvaluationFieldPolicy
75     )
76
77
78 def safe_hashable(value: Any) -> Any:
79     if value is None:
80         return None
81
82     if isinstance(value, float) and math.isnan(value):
83         return None
84
85     if isinstance(value, (list, dict, tuple, set)):
86         return json.dumps(value, sort_keys=True, default=str)
87
88     try:
89         if bool(pd.isna(value)):
90             return None
91     except Exception:
92         pass
93
94     return value
95
96
97 def profile_sample_value(value: Any) -> str:
98     if isinstance(value, dict):
99         return json.dumps(
100             {
101                 "type": "object",
102                 "keys": [str(key) for key in list(value)[:20]],
103             },
104             sort_keys=True,
105         )
106     if isinstance(value, list):
107         return json.dumps(

```

```

108         {
109             "type": "array",
110             "length": len(value),
111             "item_type": (
112                 type(value[0]).__name__ if value else "unknown"
113             ),
114         },
115         sort_keys=True,
116     )
117
118     text = str(value)
119     return text if len(text) <= 240 else text[:237] + "..."
120
121
122 def fingerprint_files(paths: list[Path]) -> str:
123     digest = hashlib.sha256()
124
125     for path in sorted(paths, key=lambda item: str(item.resolve())):
126         digest.update(str(path.resolve()).encode("utf-8"))
127
128         with path.open("rb") as handle:
129             while chunk := handle.read(1024 * 1024):
130                 digest.update(chunk)
131
132     return digest.hexdigest()
133
134
135 def expand_inputs(inputs: Iterable[str | Path]) -> list[Path]:
136     paths: list[Path] = []
137
138     for raw_path in inputs:
139         path = Path(raw_path).expanduser()
140
141         if path.is_dir():
142             paths.extend(
143                 item
144                 for item in sorted(path.iterdir())
145                 if item.is_file() and item.suffix.lower() in SUPPORTED_EXTENSIONS
146             )
147         elif path.is_file():
148             paths.append(path)
149         else:
150             raise FileNotFoundError(f"Input path does not exist: {path}")
151
152     if not paths:
153         raise ValueError("No supported input files were found.")
154
155     return paths
156
157
158 def unique_table_name(name: str, tables: dict[str, pd.DataFrame]) -> str:
159     candidate = name
160     counter = 2
161
162     while candidate in tables:
163         candidate = f"{name}_{counter}"
164         counter += 1
165
166     return candidate
167
168
169 def _columnar_mapping(payload: dict[str, Any]) -> bool:
170     values = list(payload.values())
171     if not values or not all(isinstance(value, list) for value in values):
172         return False
173     lengths = {len(value) for value in values}
174     return len(lengths) == 1 and not all(
175         not value or isinstance(value[0], dict)
176         for value in values
177     )
178
179
180 def _highlight_pairs(value: Any) -> set[tuple[int, int]]:
181     pairs: set[tuple[int, int]] = set()
182     if not isinstance(value, list):
183         return pairs
184
185     for item in value:
186         if (
187             isinstance(item, list | tuple)
188             and len(item) >= 2
189             and isinstance(item[0], int)
190             and isinstance(item[1], int)
191         ):
192             pairs.add((item[0], item[1]))
193
194     return pairs
195
196
197 def _meaning_representation_pairs(value: Any) -> list[list[str]]:
198     text = str(value or "")

```

```

199 pattern = re.compile(r"([A-Za-z0-9 _-])\s*([^\s]*)")
200 pairs = [
201     [key.strip(), item.strip()]
202     for key, item in pattern.findall(text)
203     if key.strip() and item.strip()
204 ]
205 return pairs or ([["meaning_representation", text]] if text.strip() else [])
206
207
208 def _normalise_record_rows(value: Any) -> list[list[str]]:
209     if not isinstance(value, list):
210         return []
211
212     rows: list[list[str]] = []
213     for item in value:
214         if isinstance(item, dict):
215             subject = item.get("subject") or item.get("head") or item.get("s")
216             relation = (
217                 item.get("relation")
218                 or item.get("predicate")
219                 or item.get("property")
220                 or item.get("r")
221             )
222             obj = item.get("object") or item.get("tail") or item.get("value") or item.get("o")
223             if subject is not None and relation is not None and obj is not None:
224                 rows.append([str(subject), str(relation), str(obj)])
225             else:
226                 for key, child in item.items():
227                     if not isinstance(child, (dict, list)):
228                         rows.append([str(key), str(child)])
229             elif isinstance(item, list | tuple):
230                 values = [str(part) for part in item if str(part).strip()]
231                 if values:
232                     rows.append(values)
233     return rows
234
235
236 def _structured_record_table(payload: dict[str, Any]) -> pd.DataFrame | None:
237     if payload.get("__table2text_benchmark_example__"):
238         task_family = str(payload.get("task_family") or "")
239         if task_family not in {
240             "attribute_verbalisation",
241             "triple_verbalisation",
242         }:
243             return None
244         source_payload = payload.get("source_payload")
245         parent_table = payload.get("parent_table")
246         request = payload.get("request")
247         source_text = payload.get("source_text")
248         output_mode = payload.get("output_mode")
249     else:
250         task_family = ""
251         source_payload = payload
252         parent_table = None
253         request = None
254         source_text = None
255         output_mode = None
256
257     rows = _normalise_record_rows(parent_table)
258     if not rows and isinstance(source_payload, dict):
259         if "meaning_representation" in source_payload:
260             rows = _meaning_representation_pairs(
261                 source_payload.get("meaning_representation")
262             )
263             task_family = task_family or "attribute_verbalisation"
264             source_text = source_text or str(
265                 source_payload.get("meaning_representation") or ""
266             )
267         elif "triples" in source_payload:
268             rows = _normalise_record_rows(source_payload.get("triples"))
269             task_family = task_family or "triple_verbalisation"
270
271     if not rows:
272         return None
273
274     records: list[dict[str, Any]] = []
275     record_kind = (
276         "triple"
277         if task_family == "triple_verbalisation"
278         or any(len(row) >= 3 for row in rows)
279         else "attribute"
280     )
281     for row_index, row in enumerate(rows):
282         if len(row) >= 3:
283             subject, relation, obj = row[0], row[1], row[2]
284             records.append(
285                 {
286                     "row_index": row_index,
287                     "record_kind": "triple",
288                     "subject": subject,
289                     "relation": relation,

```

```

290         "object": obj,
291         "attribute_name": relation,
292         "attribute_value": obj,
293         "task_family": task_family or "triple_verbalisation",
294         "output_mode": output_mode,
295         "request": request,
296         "source_text": source_text,
297     }
298 )
299 elif len(row) >= 2:
300     records.append(
301         {
302             "row_index": row_index,
303             "record_kind": record_kind,
304             "subject": None,
305             "relation": row[0],
306             "object": row[1],
307             "attribute_name": row[0],
308             "attribute_value": row[1],
309             "task_family": task_family or "attribute_verbalisation",
310             "output_mode": output_mode,
311             "request": request,
312             "source_text": source_text,
313         }
314     )
315
316 return pd.DataFrame(records) if records else None
317
318
319 def _benchmark_cell_table(payload: dict[str, Any]) -> pd.DataFrame | None:
320     benchmark_wrapper = bool(payload.get("__table2text_benchmark_example__"))
321     source_payload = payload.get("source_payload") if benchmark_wrapper else payload
322     if not isinstance(source_payload, dict):
323         return None
324
325     highlighted_source = (
326         source_payload.get("highlighted_cells")
327         or source_payload.get("highlighted_cell_ids")
328     )
329     if not highlighted_source:
330         return None
331
332     table = source_payload.get("table")
333     if not isinstance(table, list):
334         table = payload.get("parent_table") if benchmark_wrapper else None
335
336     if not isinstance(table, list):
337         return None
338
339     highlighted = _highlight_pairs(highlighted_source)
340     page_title = (
341         source_payload.get("table_page_title")
342         or source_payload.get("page_title")
343     )
344     section_title = (
345         source_payload.get("table_section_title")
346         or source_payload.get("table_title")
347     )
348
349     records: list[dict[str, Any]] = []
350     occupied_columns_by_row: dict[int, set[int]] = {}
351     for row_index, table_row in enumerate(table):
352         if not isinstance(table_row, list):
353             continue
354         expanded_column_index = 0
355         for raw_column_index, cell in enumerate(table_row):
356             occupied_columns = occupied_columns_by_row.get(row_index, set())
357             while expanded_column_index in occupied_columns:
358                 expanded_column_index += 1
359
360             if isinstance(cell, dict):
361                 value = cell.get("value", "")
362                 is_header = bool(cell.get("is_header", False))
363                 row_span = cell.get("row_span")
364                 column_span = cell.get("column_span")
365             else:
366                 value = cell
367                 is_header = False
368                 row_span = None
369                 column_span = None
370
371             span_width = (
372                 column_span
373                 if isinstance(column_span, int) and column_span > 0
374                 else 1
375             )
376             span_height = (
377                 row_span
378                 if isinstance(row_span, int) and row_span > 0
379                 else 1
380             )

```

```

381     column_index = expanded_column_index
382     column_end_index = expanded_column_index + span_width - 1
383     # Benchmark highlighted-cell coordinates are raw table-cell
384     # coordinates. Keep span-expanded columns for structural context,
385     # but do not let a preceding colspan absorb a later highlighted
386     # raw cell in the same row.
387     cell_highlighted = (row_index, raw_column_index) in highlighted
388
389     records.append(
390         {
391             "page_title": page_title,
392             "section_title": section_title,
393             "row_index": row_index,
394             "raw_column_index": raw_column_index,
395             "column_index": column_index,
396             "column_end_index": column_end_index,
397             "cell_value": value,
398             "is_header": is_header,
399             "is_highlighted": cell_highlighted,
400             "row_span": row_span,
401             "column_span": column_span,
402             "task_family": payload.get("task_family") if benchmark_wrapper else None,
403             "output_mode": payload.get("output_mode") if benchmark_wrapper else None,
404             "request": payload.get("request") if benchmark_wrapper else None,
405             "source_text": payload.get("source_text") if benchmark_wrapper else None,
406         }
407     )
408     expanded_column_index += span_width
409     if span_height > 1:
410         spanned_columns = set(
411             range(column_index, column_end_index + 1)
412         )
413         for spanned_row_index in range(
414             row_index + 1,
415             row_index + span_height,
416         ):
417             occupied_columns_by_row.setdefault(
418                 spanned_row_index,
419                 set(),
420             ).update(spanned_columns)
421
422     if not records:
423         return None
424
425     return pd.DataFrame(records)
426
427
428 def load_json_tables(
429     path: Path,
430     payload: Any | None = None,
431 ) -> dict[str, pd.DataFrame]:
432     if payload is None:
433         with path.open("r", encoding="utf-8") as handle:
434             payload = json.load(handle)
435
436     if isinstance(payload, dict):
437         structured_record = _structured_record_table(payload)
438         if structured_record is not None:
439             return {path.stem: structured_record}
440
441         benchmark_table = _benchmark_cell_table(payload)
442         if benchmark_table is not None:
443             return {path.stem: benchmark_table}
444
445     if isinstance(payload, list):
446         return {path.stem: pd.json_normalize(payload)}
447
448     if isinstance(payload, dict):
449         nested_tables = {
450             str(key): pd.json_normalize(value)
451             for key, value in payload.items()
452             if isinstance(value, list)
453             and (not value or isinstance(value[0], dict))
454         }
455
456         if nested_tables and len(nested_tables) == len(payload):
457             return nested_tables
458
459         if _columnar_mapping(payload):
460             return {path.stem: pd.DataFrame(payload)}
461
462         # A mixed scalar/nested mapping is one structured record. Passing it
463         # directly to DataFrame aligns nested keys into artificial rows.
464         return {path.stem: pd.DataFrame([payload])}
465
466     return {path.stem: pd.DataFrame({"value": [payload]})}
467
468
469 def load_data(
470     inputs: Iterable[str | Path],
471     evaluation_field_policy: EvaluationFieldPolicy | None = None,

```

```

472 ) -> DataBundle:
473     paths = expand_inputs(inputs)
474     tables: dict[str, pd.DataFrame] = {}
475     structured_inputs: dict[str, Any] = {}
476     structure_profiles: list[InputStructureProfile] = []
477     effective_policies: list[EvaluationFieldPolicy] = []
478
479     for path in paths:
480         extension = path.suffix.lower()
481
482         operational_payload: Any | None = None
483
484         if extension == ".csv":
485             loaded = {path.stem: pd.read_csv(path, low_memory=False)}
486         elif extension == ".json":
487             with path.open("r", encoding="utf-8") as handle:
488                 payload = json.load(handle)
489                 (
490                     operational_payload,
491                     structure_profile,
492                     effective_policy,
493                 ) = inspect_and_filter_payload(
494                     payload=payload,
495                     source_path=path,
496                     field_policy=evaluation_field_policy,
497                 )
498                 structure_profiles.append(structure_profile)
499                 effective_policies.append(effective_policy)
500                 loaded = load_json_tables(path, operational_payload)
501         elif extension in {".jsonl", ".ndjson"}:
502             loaded = {path.stem: pd.read_json(path, lines=True)}
503         elif extension == ".parquet":
504             loaded = {path.stem: pd.read_parquet(path)}
505         elif extension in {".xlsx", ".xls"}:
506             sheets = pd.read_excel(path, sheet_name=None)
507             loaded = {
508                 f"{path.stem}_{sheet_name}": frame
509                 for sheet_name, frame in sheets.items()
510             }
511         else:
512             raise ValueError(f"Unsupported format: {path}")
513
514         if extension != ".json":
515             structure_profiles.append(
516                 InputStructureProfile(
517                     shape=(
518                         InputShape.TIME_SERIES
519                         if any(
520                             "date" in str(column).casefold()
521                             or "time" in str(column).casefold()
522                             for frame in loaded.values()
523                             for column in frame.columns
524                         )
525                         else InputShape.FLAT_TABLE
526                     ),
527                     representation_status=InputRepresentationStatus.VALID,
528                     source_paths=[str(path)],
529                     row_semantics="one observation per row",
530                     confidence=0.95,
531                 )
532             )
533
534         for proposed_name, frame in loaded.items():
535             table_name = unique_table_name(proposed_name, tables)
536             frame = frame.copy()
537             frame.columns = [str(column) for column in frame.columns]
538             tables[table_name] = frame
539             if operational_payload is not None:
540                 if len(loaded) == 1:
541                     structured_inputs[table_name] = operational_payload
542                 elif isinstance(operational_payload, dict):
543                     structured_inputs[table_name] = operational_payload.get(
544                         proposed_name
545                     )
546
547     if effective_policies:
548         effective_policy = EvaluationFieldPolicy(
549             operational_input_paths=list(
550                 dict.fromkeys(
551                     path
552                     for policy in effective_policies
553                     for path in policy.operational_input_paths
554                 )
555             ),
556             held_out_reference_paths=list(
557                 dict.fromkeys(
558                     path
559                     for policy in effective_policies
560                     for path in policy.held_out_reference_paths
561                 )
562             ),

```

```

563         metadata_paths=list(
564             dict.fromkeys(
565                 path
566                 for policy in effective_policies
567                 for path in policy.metadata_paths
568             )
569         ),
570     )
571 else:
572     effective_policy = evaluation_field_policy or EvaluationFieldPolicy()
573
574     return DataBundle(
575         tables=tables,
576         source_paths=paths,
577         fingerprint=fingerprint_files(paths),
578         structured_inputs=structured_inputs,
579         input_structure=combine_structure_profiles(structure_profiles),
580         evaluation_field_policy=effective_policy,
581     )
582
583
584 def normalise_column_name(column_name: str) -> str:
585     return re.sub(r"^[a-z0-9]+", " ", column_name.lower()).strip()
586
587
588 def looks_datetime_like(series: pd.Series) -> bool:
589     sample = series.dropna().astype(str).head(200)
590
591     if sample.empty:
592         return False
593
594     datetime_pattern_rate = sample.str.contains(
595         r"\d{4}[-/]\d{1,2}[-/]\d{1,2}"
596         r"|"
597         r"\d{1,2}[-/]\d{1,2}[-/]\d{2,4}"
598         r"|"
599         r"\d{1,2}:\d{2}",
600         regex=True,
601     ).mean()
602
603     return bool(datetime_pattern_rate >= 0.5)
604
605
606 def classify_zero_risk(
607     *,
608     column_name: str,
609     zero_count: int,
610     zero_rate: float,
611     median: float,
612     q05: float,
613 ) -> tuple[ZeroRisk, str]:
614     if zero_count == 0:
615         return ZeroRisk.NONE, "No zero observations were recorded."
616
617     name = normalise_column_name(column_name)
618
619     if any(term in name for term in VALID_ZERO_TERMS):
620         return (
621             ZeroRisk.LIKELY_VALID,
622             "Zero is valid on the apparent measurement or coding scale.",
623         )
624
625     if any(term in name for term in CONTEXT_DEPENDENT_ZERO_TERMS):
626         return (
627             ZeroRisk.CONTEXT_DEPENDENT,
628             "Zero may be a genuine extreme observation and should not be "
629             "treated as erroneous without contextual evidence.",
630         )
631
632     if (
633         any(term in name for term in POSSIBLE_SENTINEL_ZERO_TERMS)
634         and median > 0
635         and q05 > 0
636     ):
637         return (
638             ZeroRisk.POSSIBLE_SENTINEL,
639             "Zero is separated from the main positive distribution and may "
640             "represent encoded missingness or measurement failure.",
641         )
642
643     if median >= 10 and q05 > 0 and zero_rate <= 0.05:
644         return (
645             ZeroRisk.UNUSUAL,
646             "Zero is unusual relative to the observed distribution, but its "
647             "validity cannot be established without metadata.",
648         )
649
650     return (
651         ZeroRisk.NONE,
652         "The observed distribution does not provide sufficient evidence that "
653         "zero is problematic.",
654     )

```



```

654     )
655
656
657 def datetime_parse_rate(series: pd.Series) -> float:
658     non_missing = series.dropna()
659
660     if non_missing.empty:
661         return 0.0
662
663     sample = non_missing.head(1_000)
664
665     if pd.api.types.is_datetime64_any_dtype(sample):
666         return 1.0
667
668     if pd.api.types.is_numeric_dtype(sample):
669         return 0.0
670
671     if not looks_datetime_like(sample):
672         return 0.0
673
674     parsed = pd.to_datetime(sample, errors="coerce", utc=True)
675     return float(parsed.notna().mean())
676
677
678 def infer_semantic_type(
679     series: pd.Series,
680     unique_count: int,
681     parse_rate: float,
682 ) -> str:
683     if series.map(
684         lambda value: isinstance(value, (list, dict, tuple, set))
685     ).any():
686         return "structured"
687
688     if pd.api.types.is_bool_dtype(series):
689         return "boolean"
690
691     if pd.api.types.is_numeric_dtype(series):
692         return "numeric"
693
694     if pd.api.types.is_datetime64_any_dtype(series) or parse_rate >= 0.8:
695         return "datetime"
696
697     row_count = max(len(series), 1)
698
699     if unique_count <= max(20, int(row_count * 0.05)):
700         return "categorical"
701
702     return "text"
703
704
705 def numeric_diagnostics(
706     column_name: str,
707     series: pd.Series,
708 ) -> tuple[dict[str, float | int], list[str], ZeroRisk, str, bool]:
709     numeric = pd.to_numeric(series, errors="coerce").dropna()
710
711     if numeric.empty:
712         return {}, [], ZeroRisk.NONE, "No numeric observations were available.", False
713
714     q01 = float(numeric.quantile(0.01))
715     q05 = float(numeric.quantile(0.05))
716     q25 = float(numeric.quantile(0.25))
717     q75 = float(numeric.quantile(0.75))
718     q99 = float(numeric.quantile(0.99))
719
720     iqr = q75 - q25
721
722     if iqr > 0:
723         lower_bound = q25 - 1.5 * iqr
724         upper_bound = q75 + 1.5 * iqr
725         outlier_count = int(
726             ((numeric < lower_bound) | (numeric > upper_bound)).sum()
727         )
728     else:
729         outlier_count = 0
730
731     zero_count = int((numeric == 0).sum())
732     zero_rate = zero_count / len(numeric)
733
734     median = float(numeric.median())
735
736     zero_risk, zero_risk_reason = classify_zero_risk(
737         column_name=column_name,
738         zero_count=zero_count,
739         zero_rate=float(zero_rate),
740         median=median,
741         q05=q05,
742     )
743     suspicious_zero = zero_risk in {
744         ZeroRisk.UNUSUAL,

```

```

745     ZeroRisk.POSSIBLE_SENTINEL,
746 }
747
748 warnings: List[str] = []
749
750 if suspicious_zero:
751     warnings.append(zero_risk_reason)
752
753 summary: dict[str, float | int] = {
754     "count": int(numeric.count()),
755     "mean": float(numeric.mean()),
756     "median": median,
757     "standard_deviation": (
758         float(numeric.std(ddof=1))
759         if len(numeric) > 1
760         else 0.0
761     ),
762     "minimum": float(numeric.min()),
763     "q01": q01,
764     "q05": q05,
765     "q25": q25,
766     "q75": q75,
767     "q99": q99,
768     "maximum": float(numeric.max()),
769     "zero_count": zero_count,
770     "zero_rate": float(zero_rate),
771     "negative_count": int((numeric < 0).sum()),
772     "skewness": (
773         float(numeric.skew())
774         if len(numeric) > 2
775         else 0.0
776     ),
777     "iqr_outlier_count": outlier_count,
778 }
779
780 return summary, warnings, zero_risk, zero_risk_reason, suspicious_zero
781
782
783 def profile_data(bundle: DataBundle) -> DataProfile:
784     path_lookup = {
785         path.stem: str(path)
786         for path in bundle.source_paths
787     }
788
789     table_profiles: List[TableProfile] = []
790
791     for table_name, frame in bundle.tables.items():
792         hashable_frame = frame.copy()
793
794         for column in hashable_frame.columns:
795             hashable_frame[column] = hashable_frame[column].map(safe_hashable)
796
797         duplicate_count = int(hashable_frame.duplicated().sum())
798
799         candidate_keys: List[str] = []
800         columns: List[ColumnProfile] = []
801         table_warnings: List[str] = []
802
803         for column_name in frame.columns:
804             series = frame[column_name]
805             safe_series = series.map(safe_hashable)
806
807             missing_count = int(safe_series.isna().sum())
808             unique_count = int(safe_series.nunique(dropna=True))
809             parse_rate = datetime_parse_rate(series)
810
811             candidate_key = bool(
812                 len(frame) > 0
813                 and missing_count == 0
814                 and unique_count == len(frame)
815             )
816
817             if candidate_key:
818                 candidate_keys.append(column_name)
819
820             semantic_type = infer_semantic_type(
821                 series,
822                 unique_count,
823                 parse_rate,
824             )
825
826             non_missing = safe_series.dropna()
827
828             if non_missing.empty:
829                 dominant_rate = 0.0
830             else:
831                 dominant_rate = float(
832                     non_missing.value_counts(normalize=True).iloc[0]
833                 )
834
835             constant = unique_count <= 1 and len(frame) > 0

```

```

836     near_constant = not constant and dominant_rate >= 0.995
837
838     summary: dict[str, float | int] = {}
839     quality_warnings: list[str] = []
840     suspicious_zero = False
841     zero_risk = ZeroRisk.NONE
842     zero_risk_reason = None
843
844     if semantic_type == "numeric":
845         (
846             summary,
847             numeric_warnings,
848             zero_risk,
849             zero_risk_reason,
850             suspicious_zero,
851         ) = numeric_diagnostics(
852             column_name,
853             series,
854         )
855         quality_warnings.extend(numeric_warnings)
856
857     if constant:
858         quality_warnings.append(
859             "The column is constant and has no observed analytical variation."
860         )
861
862     if near_constant:
863         quality_warnings.append(
864             "The column is near-constant and may contribute little analytical information."
865         )
866
867     samples = list(
868         dict.fromkeys(
869             profile_sample_value(value)
870             for value in series.dropna().head(20)
871         )
872     )[:5]
873
874     columns.append(
875         ColumnProfile(
876             name=column_name,
877             dtype=str(series.dtype),
878             semantic_type=semantic_type,
879             missing_count=missing_count,
880             missing_rate=round(
881                 missing_count / max(len(frame), 1),
882                 6,
883             ),
884             unique_count=unique_count,
885             sample_values=samples,
886             numeric_summary=summary,
887             datetime_parse_rate=round(parse_rate, 6),
888             candidate_key=candidate_key,
889             structured_values=semantic_type == "structured",
890             constant=constant,
891             near_constant=near_constant,
892             dominant_value_rate=round(dominant_rate, 6),
893             suspicious_zero_values=suspicious_zero,
894             possible_sentinel_values=suspicious_zero,
895             zero_risk=zero_risk,
896             zero_risk_reason=zero_risk_reason,
897             quality_warnings=quality_warnings,
898         )
899     )
900
901     if len(frame) == 0:
902         table_warnings.append("The table contains no rows.")
903
904     if duplicate_count:
905         table_warnings.append(
906             f"{duplicate_count} duplicate rows were detected."
907         )
908
909     if any(column.missing_rate >= 0.5 for column in columns):
910         table_warnings.append(
911             "At least one column has 50% or more missing values."
912         )
913
914     constant_columns = [
915         column.name for column in columns if column.constant
916     ]
917
918     if constant_columns:
919         table_warnings.append(
920             "Constant columns detected: "
921             + ", ".join(f"{column}" for column in constant_columns)
922             + "."
923         )
924
925     suspicious_zero_columns = [
926         column.name

```

```

927         for column in columns
928         if column.suspicious_zero_values
929     ]
930
931     if suspicious_zero_columns:
932         table_warnings.append(
933             "Potentially suspicious zero values detected in: "
934             + ", ".join(
935                 f"`{column}`"
936                 for column in suspicious_zero_columns
937             )
938             + "."
939         )
940
941     source_path = next(
942         (
943             path
944             for stem, path in path_lookup.items()
945             if table_name.startswith(stem)
946         ),
947         str(bundle.source_paths[0]),
948     )
949
950     table_profiles.append(
951         TableProfile(
952             table_name=table_name,
953             source_path=source_path,
954             row_count=int(len(frame)),
955             column_count=int(len(frame.columns)),
956             duplicate_row_count=duplicate_count,
957             candidate_keys=candidate_keys,
958             columns=columns,
959             warnings=table_warnings,
960         )
961     )
962
963     return DataProfile(
964         fingerprint=bundle.fingerprint,
965         source_paths=[str(path) for path in bundle.source_paths],
966         tables=table_profiles,
967     )

```

Listing 27: P27: table2text_pydanticai/src/table2text/evaluation/__init__.py

```

1  """Public API for dataset preparation, generation, metrics, and human studies."""
2
3  from __future__ import annotations
4
5  from .models import (
6      BenchmarkExample,
7      DatasetConfig,
8      DeepEvalObservation,
9      ExperimentConfig,
10     GenerationRecord,
11     LLMJudgeAnnotationRecord,
12     LLMJudgeErrorAnnotation,
13     MetricObservation,
14     VariantConfig,
15 )
16 from .alignscore_client import AlignScoreClient
17 from .external_factuality import ExternalFactualityResult, HHMEvaluator
18 from .notebook import (
19     aggregate_for_notebook,
20     annotate_with_openai_judge_for_notebook,
21     default_paths,
22     diagnostics_for_notebook,
23     generate_reports_for_notebook,
24     init_notebook_evaluation,
25     load_project_env,
26     prepare_examples_for_notebook,
27     read_jsonl_as_frame,
28     score_deepeval_for_notebook,
29     score_openai_judge_for_notebook,
30     score_reference_metrics_for_notebook,
31 )
32
33 __all__ = [
34     "aggregate_for_notebook",
35     "annotate_with_openai_judge_for_notebook",
36     "AlignScoreClient",
37     "BenchmarkExample",
38     "DatasetConfig",
39     "default_paths",
40     "DeepEvalObservation",
41     "diagnostics_for_notebook",
42     "ExternalFactualityResult",
43     "ExperimentConfig",
44     "generate_reports_for_notebook",
45     "GenerationRecord",

```

```

46     "HHEvaluator",
47     "init_notebook_evaluation",
48     "load_project_env",
49     "LLMJudgeAnnotationRecord",
50     "LLMJudgeErrorAnnotation",
51     "MetricObservation",
52     "prepare_examples_for_notebook",
53     "read_jsonl_as_frame",
54     "score_deepeval_for_notebook",
55     "score_openai_judge_for_notebook",
56     "score_reference_metrics_for_notebook",
57     "VariantConfig",
58 ]
59
60 __version__ = "2.0.0"

```

Listing 28: P28: table2text_pydanticai/src/table2text/evaluation/alignscore_client.py

```

1  """Invoke an isolated AlignScore worker and normalise its factuality scores."""
2
3  from __future__ import annotations
4
5  import json
6  import os
7  import subprocess
8  import tempfile
9  import uuid
10 from pathlib import Path
11
12 from .external_factuality import ExternalFactualityResult
13
14 class AlignScoreClient:
15     def __init__(
16         self,
17         *,
18         python_executable: Path,
19         worker_path: Path,
20         model_size: str = "base",
21         device: str = "cpu",
22         batch_size: int = 8,
23         threshold: float = 0.5,
24         local_files_only: bool = True,
25     ) -> None:
26         self.threshold = threshold
27         self.model_size = model_size
28
29         command = [
30             str(python_executable),
31             str(worker_path),
32             "--model-size",
33             model_size,
34             "--device",
35             device,
36             "--batch-size",
37             str(batch_size),
38         ]
39
40         environment = os.environ.copy()
41         if local_files_only:
42             command.append("--local-files-only")
43             environment["HF_HUB_OFFLINE"] = "1"
44             environment["TRANSFORMERS_OFFLINE"] = "1"
45         environment.setdefault(
46             "HF_MODULES_CACHE",
47             str(Path(tempfile.gettempdir()) / "table2text_hf_modules"),
48         )
49
50         self._process = subprocess.Popen(
51             command,
52             stdin=subprocess.PIPE,
53             stdout=subprocess.PIPE,
54             stderr=subprocess.PIPE,
55             text=True,
56             bufsize=1,
57             env=environment,
58         )
59
60     @property
61     def metric_name(self) -> str:
62         return f"alignscore_{self.model_size}"
63
64     def evaluate(
65         self,
66         *,
67         context: str,
68         generated_text: str,
69     ) -> ExternalFactualityResult:
70         if not context.strip():
71             return ExternalFactualityResult(

```

```

72         metric_name=self.metric_name,
73         status="skipped",
74         threshold=self.threshold,
75         details={"reason": "The factuality context is empty."},
76     )
77     if not generated_text.strip():
78         return ExternalFactualityResult(
79             metric_name=self.metric_name,
80             status="skipped",
81             threshold=self.threshold,
82             details={"reason": "The generated output is empty."},
83         )
84     if self._process.stdin is None or self._process.stdout is None:
85         return ExternalFactualityResult(
86             metric_name=self.metric_name,
87             status="error",
88             threshold=self.threshold,
89             error="AlignScore worker pipes are unavailable.",
90         )
91
92     request_id = uuid.uuid4().hex
93     request = {
94         "request_id": request_id,
95         "context": context,
96         "claim": generated_text,
97     }
98
99     self._process.stdin.write(json.dumps(request) + "\n")
100    self._process.stdin.flush()
101
102    response_line = self._process.stdout.readline()
103    if not response_line:
104        stderr = self._process.stderr.read() if self._process.stderr else ""
105        return ExternalFactualityResult(
106            metric_name=self.metric_name,
107            status="error",
108            threshold=self.threshold,
109            error=f"AlignScore worker terminated. stderr={stderr}",
110        )
111
112    response = json.loads(response_line)
113    if response.get("request_id") != request_id:
114        return ExternalFactualityResult(
115            metric_name=self.metric_name,
116            status="error",
117            threshold=self.threshold,
118            error="AlignScore response ID mismatch.",
119        )
120    if response.get("status") != "scored":
121        return ExternalFactualityResult(
122            metric_name=self.metric_name,
123            status="error",
124            threshold=self.threshold,
125            error=response.get("error"),
126        )
127
128    return ExternalFactualityResult(
129        metric_name=self.metric_name,
130        status="scored",
131        overall_score=float(response["score"]),
132        threshold=self.threshold,
133        details={
134            "evaluation_mode": "nli_sp",
135            "model_size": self.model_size,
136        },
137    )
138
139    def close(self) -> None:
140        if self._process.stdin:
141            self._process.stdin.close()
142        self._process.terminate()
143        try:
144            self._process.wait(timeout=10)
145        except subprocess.TimeoutExpired:
146            self._process.kill()
147            self._process.wait(timeout=10)

```

Listing 29: P29: table2text_pydanticai/src/table2text/evaluation/cli.py

```

1  """Command-line interface for the reproducible evaluation subsystem."""
2
3  from __future__ import annotations
4
5  import argparse
6  import json
7  from pathlib import Path
8
9  from .datasets import (
10     default_dataset_configs,

```

```

11     load_dataset_configs,
12     prepare_datasets,
13     read_examples,
14     save_default_dataset_configs,
15 )
16 from .deepeval_metrics import evaluate_deepeval
17 from .diagnostics import write_diagnostics
18 from .generation import generate_all, load_variants, read_generations
19 from .human_evaluation import (
20     decode_human_scores,
21     export_human_packet,
22     export_reviewer_packet,
23     inter_rater_statistics,
24     load_human_judgements,
25     make_blinded_pairs,
26 )
27 from .models import (
28     DeepEvalConfig,
29     ExperimentConfig,
30     HumanEvaluationPair,
31     ReferenceMetricConfig,
32     VariantConfig,
33 )
34 from .reference_metrics import evaluate_reference_metrics
35 from .statistics import write_analysis
36
37
38 def load_json(path: Path) -> dict:
39     return json.loads(path.read_text(encoding="utf-8"))
40
41
42 def write_default_variants(path: Path) -> None:
43     variants = [
44         VariantConfig(
45             variant_id="single_agent_baseline",
46             enabled=False,
47             backend="callable",
48             description=(
49                 "Configure callable_path with a direct single-agent generation function."
50             ),
51             callable_path="table2text.evaluation_backends.single_agent_baseline",
52             repetitions=1,
53             seeds=[42],
54         ),
55         VariantConfig(
56             variant_id="full_system",
57             enabled=True,
58             backend="table2text",
59             description="Current full multi-agent system.",
60             settings_overrides={},
61             repetitions=1,
62             seeds=[42],
63         ),
64         VariantConfig(
65             variant_id="insight_synthesis_off",
66             enabled=True,
67             backend="table2text",
68             description="Full system with insight synthesis disabled.",
69             settings_overrides={"enable_insight_synthesis": False},
70             repetitions=1,
71             seeds=[42],
72         ),
73         VariantConfig(
74             variant_id="verifier_off",
75             enabled=False,
76             backend="precomputed",
77             description=(
78                 "Enable after the operational pipeline exposes a verifier feature "
79                 "flag, or provide precomputed outputs."
80             ),
81             precomputed_path=Path("evaluation/generations/verifier_off.jsonl"),
82             repetitions=1,
83             seeds=[42],
84         ),
85         VariantConfig(
86             variant_id="auditor_off",
87             enabled=False,
88             backend="precomputed",
89             description=(
90                 "Enable after generating outputs with the final Auditor and repair "
91                 "stages disabled."
92             ),
93             precomputed_path=Path("evaluation/generations/auditor_off.jsonl"),
94             repetitions=1,
95             seeds=[42],
96         ),
97         VariantConfig(
98             variant_id="auditor_detection_only",
99             enabled=False,
100            backend="precomputed",
101            description="Auditor detects issues but does not repair them.",

```

```

102         precomputed_path=Path("evaluation/generations/auditor_detection_only.jsonl"),
103         repetitions=1,
104         seeds=[42],
105     ),
106 ]
107 path.parent.mkdir(parents=True, exist_ok=True)
108 path.write_text(
109     json.dumps(
110         {"variants": [item.model_dump(mode="json") for item in variants]},
111         indent=2,
112         ensure_ascii=False,
113     )
114     + "\n",
115     encoding="utf-8",
116 )
117
118
119 def write_default_metrics(path: Path) -> None:
120     payload = {
121         "experiment_id": "table2text_reference_evaluation_v1",
122         "prepared_examples_path": "evaluation/prepared/all_examples.jsonl",
123         "generations_path": "evaluation/generations/generations.jsonl",
124         "result_directory": "evaluation/results",
125         "baseline_variant": "single_agent_baseline",
126         "bootstrap_resamples": 5000,
127         "confidence_level": 0.95,
128         "random_seed": 42,
129         "reference_metrics": ReferenceMetricConfig().model_dump(mode="json"),
130         "deepeval": DeepEvalConfig().model_dump(mode="json"),
131     }
132     path.parent.mkdir(parents=True, exist_ok=True)
133     path.write_text(json.dumps(payload, indent=2, ensure_ascii=False) + "\n", encoding="utf-8")
134
135
136 def init_config(arguments: argparse.Namespace) -> None:
137     root = Path(arguments.directory)
138     save_default_dataset_configs(root / "datasets.json")
139     write_default_variants(root / "variants.json")
140     write_default_metrics(root / "metrics.json")
141     print(f"Created evaluation configuration in {root}")
142
143
144 def list_datasets(_: argparse.Namespace) -> None:
145     for config in default_dataset_configs():
146         location = config.hub_id if config.hub_id else str(config.local_path)
147         print(
148             f"{config.dataset_id:32s} "
149             f"{config.source.value:12s} "
150             f"{config.task_family.value:36s} "
151             f"{location}"
152         )
153
154
155 def prepare(arguments: argparse.Namespace) -> None:
156     statuses = prepare_datasets(
157         load_dataset_configs(Path(arguments.config)),
158         Path(arguments.output_directory),
159         skip_unavailable=arguments.skip_unavailable,
160     )
161     for status in statuses:
162         print(
163             f"{status.dataset_id}: {status.status.value}; examples={status.example_count}"
164             + (f"; error={status.error}" if status.error else "")
165         )
166
167
168 def generate(arguments: argparse.Namespace) -> None:
169     records = generate_all(
170         read_examples(Path(arguments.examples)),
171         load_variants(Path(arguments.variants)),
172         Path(arguments.output),
173         Path(arguments.run_root),
174         resume=not arguments.no_resume,
175     )
176     successful = sum(item.error is None for item in records)
177     print(
178         f"Generation records: {len(records)}; successful: {successful}; "
179         f"errors: {len(records) - successful}"
180     )
181
182
183 def reference_metrics(arguments: argparse.Namespace) -> None:
184     records = read_generations(Path(arguments.generations))
185     experiment = ExperimentConfig.model_validate(load_json(Path(arguments.config)))
186     observations = evaluate_reference_metrics(
187         records,
188         experiment.reference_metrics,
189         Path(arguments.output),
190         include_ineligible=arguments.include_ineligible,
191     )
192     scored = sum(item.status.value == "scored" for item in observations)

```



```

193     print(f"Reference metric observations: {len(observations)}; scored: {scored}")
194
195
196 def deepeval_metrics(arguments: argparse.Namespace) -> None:
197     records = read_generations(Path(arguments.generations))
198     experiment = ExperimentConfig.model_validate(load_json(Path(arguments.config)))
199     observations = evaluate_deepeval(
200         records,
201         experiment.deepeval,
202         Path(arguments.output),
203         resume=not arguments.no_resume,
204     )
205     scored = sum(item.status.value == "scored" for item in observations)
206     print(f"DeepEval observations: {len(observations)}; scored: {scored}")
207
208
209 def diagnostics(arguments: argparse.Namespace) -> None:
210     frame = write_diagnostics(Path(arguments.generations), Path(arguments.output))
211     print(f"Wrote {len(frame)} diagnostic rows.")
212
213
214 def export_human(arguments: argparse.Namespace) -> None:
215     pairs = make_blinded_pairs(
216         read_generations(Path(arguments.generations)),
217         variants=arguments.variant if arguments.variant else None,
218         examples_per_dataset=arguments.examples_per_dataset,
219         seed=arguments.seed,
220     )
221     export_human_packet(
222         pairs,
223         Path(arguments.hidden_packet),
224         Path(arguments.response_template),
225     )
226     export_reviewer_packet(pairs, Path(arguments.reviewer_packet))
227     print(f"Created {len(pairs)} blinded comparisons.")
228
229
230 def analyse_human(arguments: argparse.Namespace) -> None:
231     pairs = [
232         HumanEvaluationPair.model_validate_json(line)
233         for line in Path(arguments.hidden_packet).read_text(encoding="utf-8").splitlines()
234         if line.strip()
235     ]
236     judgements = load_human_judgements(Path(arguments.responses))
237     decoded = decode_human_scores(pairs, judgements)
238     output_directory = Path(arguments.output_directory)
239     output_directory.mkdir(parents=True, exist_ok=True)
240     decoded.to_csv(output_directory / "human_scores_decoded.csv", index=False)
241     inter_rater_statistics(judgements).to_csv(
242         output_directory / "inter_rater_agreement.csv",
243         index=False,
244     )
245     summary = (
246         decoded.groupby(["dataset_id", "variant_id"], as_index=False)
247         .agg(
248             factual_correctness=("factual_correctness", "mean"),
249             coverage=("coverage", "mean"),
250             coherence=("coherence", "mean"),
251             usefulness=("usefulness", "mean"),
252             preference_rate=("preferred", "mean"),
253             tie_rate=("tie", "mean"),
254             rating_count=("pair_id", "count"),
255         )
256     )
257     summary.to_csv(output_directory / "human_summary.csv", index=False)
258     print(f"Analysed {len(judgements)} judgements.")
259
260
261 def aggregate(arguments: argparse.Namespace) -> None:
262     experiment = ExperimentConfig.model_validate(load_json(Path(arguments.config)))
263     write_analysis(
264         reference_observations_path=Path(arguments.reference_metrics),
265         deepeval_observations_path=(
266             Path(arguments.deepeval_metrics) if arguments.deepeval_metrics else None
267         ),
268         diagnostics_path=Path(arguments.diagnostics) if arguments.diagnostics else None,
269         output_directory=Path(arguments.output_directory),
270         baseline_variant=experiment.baseline_variant,
271         bootstrap_resamples=experiment.bootstrap_resamples,
272         confidence_level=experiment.confidence_level,
273         seed=experiment.random_seed,
274     )
275     print(f"Analysis written to {arguments.output_directory}")
276
277
278 def build_parser() -> argparse.ArgumentParser:
279     parser = argparse.ArgumentParser(description="Reference-based Table2Text evaluation system.")
280     subparsers = parser.add_subparsers(required=True)
281
282     command = subparsers.add_parser("init-config")
283     command.add_argument("--directory", default="evaluation/config")

```

```

284     command.set_defaults(function=init_config)
285
286     command = subparsers.add_parser("list-datasets")
287     command.set_defaults(function=list_datasets)
288
289     command = subparsers.add_parser("prepare")
290     command.add_argument("--config", default="evaluation/config/datasets.json")
291     command.add_argument("--output-directory", default="evaluation/prepared")
292     command.add_argument("--skip-unavailable", action="store_true")
293     command.set_defaults(function=prepare)
294
295     command = subparsers.add_parser("generate")
296     command.add_argument("--examples", default="evaluation/prepared/all_examples.jsonl")
297     command.add_argument("--variants", default="evaluation/config/variants.json")
298     command.add_argument("--output", default="evaluation/generations/generations.jsonl")
299     command.add_argument("--run-root", default="evaluation/generations/runs")
300     command.add_argument("--no-resume", action="store_true")
301     command.set_defaults(function=generate)
302
303     command = subparsers.add_parser("reference-metrics")
304     command.add_argument("--generations", default="evaluation/generations/generations.jsonl")
305     command.add_argument("--config", default="evaluation/config/metrics.json")
306     command.add_argument("--output", default="evaluation/results/reference_metrics.jsonl")
307     command.add_argument(
308         "--include-ineligible",
309         action="store_true",
310         help=(
311             "Score generations marked ineligible for primary evaluation. "
312             "Useful for notebook smoke tests; protected evaluation should omit this."
313         ),
314     )
315     command.set_defaults(function=reference_metrics)
316
317     command = subparsers.add_parser("deepeval")
318     command.add_argument("--generations", default="evaluation/generations/generations.jsonl")
319     command.add_argument("--config", default="evaluation/config/metrics.json")
320     command.add_argument("--output", default="evaluation/results/deepeval_metrics.jsonl")
321     command.add_argument("--no-resume", action="store_true")
322     command.set_defaults(function=deepeval_metrics)
323
324     command = subparsers.add_parser("diagnostics")
325     command.add_argument("--generations", default="evaluation/generations/generations.jsonl")
326     command.add_argument("--output", default="evaluation/results/diagnostics.csv")
327     command.set_defaults(function=diagnostics)
328
329     command = subparsers.add_parser("export-human")
330     command.add_argument("--generations", default="evaluation/generations/generations.jsonl")
331     command.add_argument("--variant", action="append", default=[])
332     command.add_argument("--examples-per-dataset", type=int, default=10)
333     command.add_argument("--seed", type=int, default=42)
334     command.add_argument("--hidden-packet", default="evaluation/human/hidden_pair_mapping.jsonl")
335     command.add_argument("--reviewer-packet", default="evaluation/human/reviewer_packet.jsonl")
336     command.add_argument("--response-template", default="evaluation/human/response_template.csv")
337     command.set_defaults(function=export_human)
338
339     command = subparsers.add_parser("analyse-human")
340     command.add_argument("--hidden-packet", default="evaluation/human/hidden_pair_mapping.jsonl")
341     command.add_argument("--responses", required=True)
342     command.add_argument("--output-directory", default="evaluation/results/human")
343     command.set_defaults(function=analyse_human)
344
345     command = subparsers.add_parser("aggregate")
346     command.add_argument("--config", default="evaluation/config/metrics.json")
347     command.add_argument("--reference-metrics", default="evaluation/results/reference_metrics.jsonl")
348     command.add_argument("--deepeval-metrics", default="evaluation/results/deepeval_metrics.jsonl")
349     command.add_argument("--diagnostics", default="evaluation/results/diagnostics.csv")
350     command.add_argument("--output-directory", default="evaluation/results/analysis")
351     command.set_defaults(function=aggregate)
352     return parser
353
354
355 def main(argv: List[str] | None = None) -> None:
356     arguments = build_parser().parse_args(argv)
357     arguments.function(arguments)
358
359
360 if __name__ == "__main__":
361     main()

```

Listing 30: P30: table2text_pydanticai/src/table2text/evaluation/datasets.py

```

1  """Prepare heterogeneous benchmark datasets as canonical evaluation examples."""
2
3  from __future__ import annotations
4
5  import hashlib
6  import json
7  import random
8  import re

```

```

9  from collections import defaultdict
10 from pathlib import Path
11 from typing import Any, Iterable
12
13 import pandas as pd
14
15 from .models import (
16     BenchmarkExample,
17     DatasetConfig,
18     DatasetPreparationStatus,
19     DatasetSource,
20     MetricStatus,
21     OutputMode,
22     TaskFamily,
23 )
24
25
26 REFERENCE_FIELD_NAMES = {
27     "answer",
28     "answers",
29     "article",
30     "description",
31     "descriptions",
32     "gold",
33     "news_article",
34     "ref",
35     "reference",
36     "references",
37     "summary",
38     "summaries",
39     "summary_de",
40     "summary_en",
41     "target",
42     "targets",
43     "text",
44 }
45
46
47 def sha256_text(value: str) -> str:
48     return hashlib.sha256(value.encode("utf-8")).hexdigest()
49
50
51 def json_safe(value: Any) -> Any:
52     if value is None:
53         return None
54     if isinstance(value, dict):
55         return {key: json_safe(item) for key, item in value.items()}
56     if isinstance(value, list | tuple | set):
57         return [json_safe(item) for item in value]
58     if hasattr(value, "item"):
59         try:
60             return value.item()
61         except Exception:
62             pass
63     try:
64         if pd.isna(value):
65             return None
66     except (TypeError, ValueError):
67         pass
68     return value
69
70
71 def compact_json(value: Any) -> str:
72     return json.dumps(
73         json_safe(value),
74         ensure_ascii=False,
75         sort_keys=True,
76         separators=(",", ":"),
77     )
78
79
80 def pretty_json(value: Any) -> str:
81     return json.dumps(json_safe(value), ensure_ascii=False, sort_keys=True, indent=2)
82
83
84 def as_string_list(value: Any) -> list[str]:
85     if value is None:
86         return []
87     if isinstance(value, str):
88         return [value] if value.strip() else []
89     if isinstance(value, dict):
90         results: list[str] = []
91         for candidate in (
92             "text",
93             "target",
94             "reference",
95             "final_sentence",
96             "answer",
97             "summary",
98             "news_article",
99         ):

```

```

100         if candidate in value:
101             results.extend(as_string_list(value[candidate]))
102         return results
103     if isinstance(value, list | tuple):
104         results: list[str] = []
105         for item in value:
106             results.extend(as_string_list(item))
107         return results
108     return [str(value)]
109
110
111 def deduplicate_strings(values: Iterable[str]) -> list[str]:
112     result: list[str] = []
113     seen: set[str] = set()
114     for value in values:
115         cleaned = re.sub(r"\s+", " ", value).strip()
116         if not cleaned or cleaned in seen:
117             continue
118         seen.add(cleaned)
119         result.append(cleaned)
120     return result
121
122
123 def value_at_path(value: Any, path: str) -> Any:
124     current = value
125     for part in path.split("."):
126         if not isinstance(current, dict):
127             return None
128         current = current.get(part)
129     return current
130
131
132 def references_from_row(row: dict[str, Any], config: DatasetConfig) -> list[str]:
133     values: list[str] = []
134     for field in config.reference_fields:
135         candidate = value_at_path(row, field)
136         if candidate is not None:
137             values.extend(as_string_list(candidate))
138
139     if config.normalizer == "sportsett":
140         values.extend(as_string_list(row.get("summaries")))
141     if config.normalizer == "totto":
142         for annotation in row.get("sentence_annotations", []) or []:
143             if isinstance(annotation, dict):
144                 values.extend(as_string_list(annotation.get("final_sentence")))
145     if config.normalizer == "rotowire_en_de":
146         if config.language.casefold().startswith("de"):
147             values.extend(as_string_list(row.get("summary_de")))
148         else:
149             values.extend(as_string_list(row.get("summary_en")))
150     if config.normalizer == "turku_hockey":
151         values.extend(as_string_list(row.get("news_article")))
152
153     return deduplicate_strings(values)
154
155
156 def example_id_from_row(row: dict[str, Any], config: DatasetConfig, index: int) -> str:
157     for field in [*config.group_fields, *config.id_fields]:
158         candidate = value_at_path(row, field)
159         if candidate not in (None, ""):
160             return str(candidate)
161     source_key = compact_json(source_payload_from_row(row, config))
162     return f"{config.dataset_id}-{sha256_text(source_key)[:16]}-{index}"
163
164
165 def parse_meaning_representation(value: str) -> list[list[str]]:
166     pattern = re.compile(r"([A-Za-z0-9_-]+\.[^]]*)")
167     pairs = [[key.strip(), item.strip()] for key, item in pattern.findall(value)]
168     return pairs or [{"meaning_representation", value}]
169
170
171 def triples_to_parent_table(value: Any) -> list[list[str]] | None:
172     if not isinstance(value, list):
173         return None
174     rows: list[list[str]] = []
175     for item in value:
176         if isinstance(item, list | tuple):
177             if len(item) >= 3:
178                 rows.append([str(item[0]), str(item[1]), str(item[2])])
179             elif item:
180                 rows.append([str(part) for part in item])
181         elif isinstance(item, dict):
182             subject = item.get("subject") or item.get("head") or item.get("s")
183             relation = (
184                 item.get("relation")
185                 or item.get("predicate")
186                 or item.get("property")
187                 or item.get("r")
188             )
189             obj = item.get("object") or item.get("tail") or item.get("value") or item.get("o")
190             if subject is not None and relation is not None and obj is not None:

```

```

191         rows.append([str(subject), str(relation), str(obj)])
192     return rows or None
193
194
195 def totto_parent_table(row: dict[str, Any]) -> list[list[str]] | None:
196     table = row.get("table")
197     if not isinstance(table, list):
198         return None
199     result: list[list[str]] = []
200     for table_row in table:
201         if not isinstance(table_row, list):
202             continue
203         values = [
204             str(cell.get("value", "")) if isinstance(cell, dict) else str(cell)
205             for cell in table_row
206         ]
207         result.append(values)
208     return result or None
209
210
211 def flat_parent_table(payload: Any) -> list[list[str]] | None:
212     if not isinstance(payload, dict):
213         return None
214     rows = [
215         [str(key), str(value)]
216         for key, value in payload.items()
217         if isinstance(value, str | int | float | bool)
218     ]
219     return rows or None
220
221
222 def source_payload_from_row(row: dict[str, Any], config: DatasetConfig) -> Any:
223     normalizer = config.normalizer
224     if normalizer == "sportsett":
225         return {"game": json_safe(row.get("game")), "teams": json_safe(row.get("teams"))}
226     if normalizer == "mlb":
227         excluded = {*config.reference_fields, "summary", "summary_eval", "target", "references"}
228         return {key: json_safe(value) for key, value in row.items() if key not in excluded}
229     if normalizer == "totto":
230         return {
231             "table_page_title": row.get("table_page_title"),
232             "table_section_title": row.get("table_section_title"),
233             "table_section_text": row.get("table_section_text"),
234             "table": json_safe(row.get("table")),
235             "highlighted_cells": json_safe(row.get("highlighted_cells")),
236         }
237     if normalizer in {"e2e", "viggo"}:
238         field = "meaning_representation" if "meaning_representation" in row else "mr"
239         return {"meaning_representation": row.get(field)}
240     if normalizer in {"webnlg", "dart"}:
241         triples = row.get("input") if normalizer == "webnlg" else row.get("tripleset")
242         return {"triples": json_safe(triples), "category": row.get("category")}
243     if normalizer == "logicnlg":
244         return {
245             "title": row.get("title"),
246             "table": json_safe(row.get("table")),
247             "linked_columns": json_safe(row.get("linked_columns")),
248             "template": row.get("template"),
249         }
250     if normalizer == "fetaqa":
251         table = row.get("table_array") or row.get("table")
252         return {
253             "page_title": row.get("table_page_title") or row.get("page_title"),
254             "table_title": row.get("table_section_title")
255             or row.get("table_title")
256             or row.get("title"),
257             "table": json_safe(table),
258             "highlighted_cell_ids": json_safe(row.get("highlighted_cell_ids")),
259             "question": row.get("question"),
260         }
261     if normalizer == "rotowire_en_de":
262         return {
263             "home_line": json_safe(row.get("home_line")),
264             "vis_line": json_safe(row.get("vis_line")),
265             "box_score": json_safe(row.get("box_score")),
266             "home_name": row.get("home_name"),
267             "vis_name": row.get("vis_name"),
268             "day": row.get("day"),
269         }
270     if normalizer == "turku_hockey":
271         excluded = {*config.reference_fields, "news_article", "target", "references"}
272         return {key: json_safe(value) for key, value in row.items() if key not in excluded}
273
274     if config.source_fields:
275         selected: dict[str, Any] = {}
276         for field in config.source_fields:
277             candidate = value_at_path(row, field)
278             if candidate is not None:
279                 selected[field] = json_safe(candidate)
280     if selected:
281         return selected

```

```

282 excluded_fields = {*config.reference_fields, *config.id_fields, *REFERENCE_FIELD_NAMES}
283 return {key: json_safe(value) for key, value in row.items() if key not in excluded_fields}
284
285
286
287 def source_text_from_payload(payload: Any, row: dict[str, Any], config: DatasetConfig) -> str:
288     if config.normalizer == "totto":
289         lines: list[str] = []
290         if payload.get("table_page_title"):
291             lines.append(f"Page: {payload['table_page_title']}")
292         if payload.get("table_section_title"):
293             lines.append(f"Section: {payload['table_section_title']}")
294         highlights = {
295             tuple(item)
296             for item in (payload.get("highlighted_cells") or [])
297             if isinstance(item, list) and len(item) == 2
298         }
299         lines.append("Table:")
300         for row_index, table_row in enumerate(payload.get("table") or []):
301             cells: list[str] = []
302             for column_index, cell in enumerate(table_row):
303                 value = str(cell.get("value", "")) if isinstance(cell, dict) else str(cell)
304                 marker = "*" if (row_index, column_index) in highlights else ""
305                 cells.append(f"{marker}{value}{marker}")
306             lines.append(" | ".join(cells))
307         lines.append("Cells surrounded by * are highlighted.")
308         return "\n".join(lines)
309
310     if config.normalizer == "fetaqa":
311         lines: list[str] = []
312         if payload.get("page_title"):
313             lines.append(f"Page: {payload['page_title']}")
314         if payload.get("table_title"):
315             lines.append(f"Table: {payload['table_title']}")
316         lines.append(pretty_json(payload.get("table")))
317         if payload.get("question"):
318             lines.append(f"Question: {payload['question']}")
319         return "\n".join(lines)
320
321     if config.normalizer in {"e2e", "viggo":
322         return str(payload.get("meaning_representation", ""))
323     if config.normalizer in {"webnlg", "dart":
324         return "\n".join(
325             " | ".join(map(str, item)) if isinstance(item, list) else str(item)
326             for item in (payload.get("triples") or [])
327         )
328     if config.normalizer == "logicnlg":
329         lines = []
330         if payload.get("title"):
331             lines.append(f"Title: {payload['title']}")
332         lines.append(str(payload.get("table", "")))
333         return "\n".join(lines)
334     return pretty_json(payload)
335
336
337 def parent_table_from_row(
338     row: dict[str, Any],
339     payload: Any,
340     config: DatasetConfig,
341 ) -> list[list[str]] | None:
342     if config.normalizer == "totto":
343         return totto_parent_table(row)
344     if config.normalizer == "webnlg":
345         return triples_to_parent_table(row.get("input"))
346     if config.normalizer == "dart":
347         return triples_to_parent_table(row.get("tripleset"))
348     if config.normalizer in {"e2e", "viggo":
349         return parse_meaning_representation(str(payload.get("meaning_representation", "")))
350     if config.normalizer in {"fetaqa", "logicnlg":
351         table = payload.get("table")
352         if isinstance(table, list):
353             return [
354                 [str(cell) for cell in table_row]
355                 for table_row in table
356                 if isinstance(table_row, list)
357             ] or None
358         return flat_parent_table(payload)
359
360
361 def request_for_config(config: DatasetConfig, payload: Any) -> str:
362     language_name = {
363         "de": "German",
364         "fi": "Finnish",
365         "en": "English",
366     }.get(config.language.split("-")[0].casefold(), config.language)
367
368     match config.task_family:
369         case TaskFamily.EVENT_REPORT:
370             return (
371                 "Write a coherent game report from the supplied structured game data. "
372                 "Lead with the result, select the most important performances and "

```



```

373         "contrasts, and do not invent information."
374     )
375     case TaskFamily.LONG_FORM_TABLE_REPORT:
376         return (
377             "Write a coherent factual report from the supplied structured data. "
378             "Select the most important information, organise it clearly, and "
379             "avoid unsupported claims."
380         )
381     case TaskFamily.HIGHLIGHTED_TABLE_DESCRIPTION:
382         return (
383             "Write exactly one concise sentence describing the highlighted table "
384             "cells. Do not discuss unrelated cells and do not add headings."
385         )
386     case TaskFamily.LOGICAL_TABLE_STATEMENT:
387         return (
388             "Write one concise statement that is logically entailed by the supplied "
389             "table. Do not introduce outside knowledge."
390         )
391     case TaskFamily.TABLE_QUESTION_ANSWERING:
392         question = payload.get("question") if isinstance(payload, dict) else None
393         return (
394             "Answer the supplied question directly using only the table. Give a "
395             "concise free-form answer."
396             + (f"\nQuestion: {question}" if question else "")
397         )
398     case TaskFamily.ATTRIBUTE_VERBALISATION:
399         return (
400             "Express all and only the supplied attributes in one or two fluent "
401             "sentences. Do not add headings or unsupported details."
402         )
403     case TaskFamily.TRIPLE_VERBALISATION:
404         return (
405             "Express all and only the supplied triples as short, coherent natural "
406             "language. Do not add unsupported facts."
407         )
408     case TaskFamily.BIOGRAPHY:
409         return (
410             "Write one concise biographical opening sentence using only the "
411             "supplied infobox information."
412         )
413     case TaskFamily.WEATHER_RESPONSE:
414         return "Write a concise weather response using only the supplied weather attributes."
415     case TaskFamily.CROSS_LINGUAL_EVENT_REPORT:
416         return f"Write a coherent game report in {language_name}. Use only the supplied structured game data."
417     case TaskFamily.ANALYTICAL_EXPLANATION:
418         return (
419             "Write a concise analytical explanation of the supplied "
420             "model-performance table. Compare the reported metrics accurately "
421             "and do not invent causal explanations."
422         )
423
424     raise ValueError(f"Unsupported task family: {config.task_family}")
425
426
427 def normalise_row(row: dict[str, Any], config: DatasetConfig, index: int) -> BenchmarkExample:
428     payload = source_payload_from_row(row, config)
429     references = references_from_row(row, config)
430     example_id = example_id_from_row(row, config, index)
431     source_text = source_text_from_payload(payload, row, config)
432     parent_table = parent_table_from_row(row, payload, config)
433     metadata = {
434         field: json_safe(value_at_path(row, field))
435         for field in config.metadata_fields
436         if value_at_path(row, field) is not None
437     }
438     metadata.update(
439         {
440             "normalizer": config.normalizer,
441             "requested_split": config.split,
442             "hub_id": config.hub_id,
443             "config_name": config.config_name,
444         }
445     )
446     return BenchmarkExample(
447         dataset_id=config.dataset_id,
448         example_id=example_id,
449         task_family=config.task_family,
450         output_mode=config.output_mode,
451         language=config.language,
452         source_payload=payload,
453         source_text=source_text,
454         references=references,
455         request=request_for_config(config, payload),
456         parent_table=parent_table,
457         metadata=metadata,
458         source_sha256=sha256_text(compact_json(payload)),
459         reference_sha256=sha256_text(compact_json(references)),
460     )
461
462
463 def load_json_rows(path: Path, split: str, root_field: str | None) -> list[dict[str, Any]]:

```

```

464     payload = json.loads(path.read_text(encoding="utf-8"))
465     if root_field:
466         payload = value_at_path(payload, root_field)
467     elif isinstance(payload, dict):
468         if split in payload:
469             payload = payload[split]
470         elif "data" in payload:
471             payload = payload["data"]
472         elif "examples" in payload:
473             payload = payload["examples"]
474     if isinstance(payload, dict):
475         return [
476             {"_mapping_key": key, **value}
477             if isinstance(value, dict)
478             else {"_mapping_key": key, "value": value}
479             for key, value in payload.items()
480         ]
481     if not isinstance(payload, list):
482         raise ValueError(f"Expected a list or mapping in {path}.")
483     return [dict(item) for item in payload if isinstance(item, dict)]
484
485
486 def load_jsonl_rows(path: Path) -> list[dict[str, Any]]:
487     rows: list[dict[str, Any]] = []
488     with path.open(encoding="utf-8") as handle:
489         for line_number, line in enumerate(handle, start=1):
490             if not line.strip():
491                 continue
492             value = json.loads(line)
493             if not isinstance(value, dict):
494                 raise ValueError(f"{path}:{line_number} is not an object.")
495             rows.append(value)
496     return rows
497
498
499 def resolve_local_file(path: Path, split: str) -> Path:
500     if path.is_file():
501         return path
502     for candidate in (
503         path / f"{split}.jsonl",
504         path / f"{split}.json",
505         path / f"{split}.parquet",
506         path / f"{split}.csv",
507         path / f"{split}.tsv",
508     ):
509         if candidate.exists():
510             return candidate
511     raise FileNotFoundError(f"No file for split `{split}` was found in {path}.")
512
513
514 def load_local_rows(config: DatasetConfig) -> list[dict[str, Any]]:
515     assert config.local_path is not None
516     path = resolve_local_file(config.local_path, config.split)
517     suffix = path.suffix.casefold()
518     if suffix == ".jsonl":
519         return load_jsonl_rows(path)
520     if suffix == ".json":
521         return load_json_rows(path, config.split, config.options.get("root_field"))
522     if suffix == ".parquet":
523         return pd.read_parquet(path).to_dict(orient="records")
524     if suffix in {".csv", ".tsv"}:
525         separator = "\t" if suffix == ".tsv" else config.options.get("separator", ",")
526         return pd.read_csv(path, sep=separator, low_memory=False).to_dict(orient="records")
527     raise ValueError(f"Unsupported local dataset format: {path}")
528
529
530 def load_huggingface_rows(config: DatasetConfig) -> list[dict[str, Any]]:
531     try:
532         from datasets import load_dataset
533     except ImportError as exc:
534         raise RuntimeError("Install the evaluation dependencies first.") from exc
535
536     kwargs: dict[str, Any] = {"path": config.hub_id, "split": config.split}
537     if config.config_name:
538         kwargs["name"] = config.config_name
539     if config.revision:
540         kwargs["revision"] = config.revision
541     if config.trust_remote_code:
542         kwargs["trust_remote_code"] = True
543     try:
544         dataset = load_dataset(**kwargs)
545     except RuntimeError as exc:
546         if "Dataset scripts are no longer supported" in str(exc):
547             raise RuntimeError(
548                 "This benchmark dataset still uses a Hugging Face loading script. "
549                 "Install the evaluation extra with datasets<4.0, then restart the notebook kernel."
550             ) from exc
551         raise
552     except TypeError:
553         kwargs.pop("trust_remote_code", None)
554         dataset = load_dataset(**kwargs)

```



```

555     return [dict(row) for row in dataset]
556
557
558 def merge_examples(examples: list[BenchmarkExample]) -> list[BenchmarkExample]:
559     grouped: dict[tuple[str, str], list[BenchmarkExample]] = defaultdict(list)
560     for example in examples:
561         grouped[(example.dataset_id, example.example_id)].append(example)
562
563     merged: list[BenchmarkExample] = []
564     for group in grouped.values():
565         first = group[0]
566         references = deduplicate_strings(
567             reference for example in group for reference in example.references
568         )
569         merged.append(
570             first.model_copy(
571                 update={
572                     "references": references,
573                     "reference_sha256": sha256_text(compact_json(references)),
574                 }
575             )
576         )
577     return merged
578
579
580 def deterministic_sample(
581     examples: list[BenchmarkExample],
582     sample_size: int | None,
583     seed: int,
584 ) -> list[BenchmarkExample]:
585     if sample_size is None or sample_size >= len(examples):
586         return examples
587     random_generator = random.Random(seed)
588     selected_indexes = sorted(random_generator.sample(range(len(examples)), sample_size))
589     return [examples[index] for index in selected_indexes]
590
591
592 def load_and_normalise_dataset(config: DatasetConfig) -> list[BenchmarkExample]:
593     rows = load_huggingface_rows(config) if config.source == DatasetSource.HUGGINGFACE else load_local_rows(config)
594     examples: list[BenchmarkExample] = []
595     errors: list[str] = []
596     for index, row in enumerate(rows):
597         try:
598             examples.append(normalise_row(json_safe(row), config, index))
599         except Exception as exc:
600             errors.append(f"row {index}: {type(exc).__name__}: {exc}")
601     examples = deterministic_sample(merge_examples(examples), config.sample_size, config.seed)
602     if not examples:
603         details = "\n".join(errors[:10])
604         raise RuntimeError(
605             f"{config.dataset_id} produced no usable examples."
606             + (f"\nFirst errors:\n{details}" if details else "")
607         )
608     return examples
609
610
611 def write_jsonl(path: Path, values: Iterable[Any]) -> None:
612     path.parent.mkdir(parents=True, exist_ok=True)
613     with path.open("w", encoding="utf-8") as handle:
614         for value in values:
615             payload = value.model_dump(mode="json") if hasattr(value, "model_dump") else json_safe(value)
616             handle.write(json.dumps(payload, ensure_ascii=False) + "\n")
617
618
619 def read_examples(path: Path) -> list[BenchmarkExample]:
620     return [
621         BenchmarkExample.model_validate(json.loads(line))
622         for line in path.read_text(encoding="utf-8").splitlines()
623         if line.strip()
624     ]
625
626
627 def prepare_datasets(
628     configs: list[DatasetConfig],
629     output_directory: Path,
630     *,
631     skip_unavailable: bool = False,
632 ) -> list[DatasetPreparationStatus]:
633     output_directory.mkdir(parents=True, exist_ok=True)
634     statuses: list[DatasetPreparationStatus] = []
635     all_examples: list[BenchmarkExample] = []
636
637     for config in configs:
638         if not config.enabled:
639             continue
640         try:
641             examples = load_and_normalise_dataset(config)
642             dataset_path = output_directory / f"{config.dataset_id}.jsonl"
643             write_jsonl(dataset_path, examples)
644             all_examples.extend(examples)
645             statuses.append(

```

```

646         DatasetPreparationStatus(
647             dataset_id=config.dataset_id,
648             status=MetricStatus.SCORED,
649             requested_split=config.split,
650             example_count=len(examples),
651             output_path=dataset_path,
652         )
653     )
654 except Exception as exc:
655     status = DatasetPreparationStatus(
656         dataset_id=config.dataset_id,
657         status=MetricStatus.UNAVAILABLE,
658         requested_split=config.split,
659         error=f"{type(exc).__name__}: {exc}",
660     )
661     statuses.append(status)
662     if not skip_unavailable:
663         write_jsonl(output_directory / "preparation_status.jsonl", statuses)
664     raise
665
666 write_jsonl(output_directory / "all_examples.jsonl", all_examples)
667 write_jsonl(output_directory / "preparation_status.jsonl", statuses)
668 return statuses
669
670
671 def default_dataset_configs() -> list[DatasetConfig]:
672     configs = [
673         DatasetConfig(
674             dataset_id="sportsett_basketball",
675             source=DatasetSource.HUGGINGFACE,
676             hub_id="GEM/sportsett_basketball",
677             split="test",
678             normalizer="sportsett",
679             task_family=TaskFamily.EVENT_REPORT,
680             output_mode=OutputMode.MULTI_PARAGRAPH_REPORT,
681             sample_size=30,
682             reference_fields=["references", "target", "summaries"],
683             id_fields=["sportsett_id", "gem_parent_id", "gem_id"],
684             metadata_fields=["sportsett_id"],
685         ),
686         DatasetConfig(
687             dataset_id="totto",
688             source=DatasetSource.HUGGINGFACE,
689             hub_id="GEM/totto",
690             split="validation",
691             normalizer="totto",
692             task_family=TaskFamily.HIGHLIGHTED_TABLE_DESCRIPTION,
693             output_mode=OutputMode.ONE_SENTENCE,
694             sample_size=30,
695             reference_fields=["references", "target"],
696             metadata_fields=["overlap_subset", "table_page_title", "table_section_title"],
697         ),
698         DatasetConfig(
699             dataset_id="e2e_nlg",
700             source=DatasetSource.HUGGINGFACE,
701             hub_id="GEM/e2e_nlg",
702             split="test",
703             normalizer="e2e",
704             task_family=TaskFamily.ATTRIBUTE_VERBALISATION,
705             output_mode=OutputMode.SHORT_TEXT,
706             sample_size=30,
707             reference_fields=["references", "target"],
708         ),
709         DatasetConfig(
710             dataset_id="web_nlg",
711             source=DatasetSource.HUGGINGFACE,
712             hub_id="GEM/web_nlg",
713             config_name="en",
714             split="test",
715             normalizer="webnlg",
716             task_family=TaskFamily.TRIPLE_VERBALISATION,
717             output_mode=OutputMode.SHORT_TEXT,
718             sample_size=30,
719             reference_fields=["references", "target"],
720             metadata_fields=["category"],
721         ),
722         DatasetConfig(
723             dataset_id="dart",
724             source=DatasetSource.HUGGINGFACE,
725             hub_id="GEM/dart",
726             split="test",
727             normalizer="dart",
728             task_family=TaskFamily.TRIPLE_VERBALISATION,
729             output_mode=OutputMode.SHORT_TEXT,
730             sample_size=30,
731             reference_fields=["references", "target"],
732             metadata_fields=["target_sources", "subtree_was_extended"],
733         ),
734         DatasetConfig(
735             dataset_id="logicnlg",
736             source=DatasetSource.HUGGINGFACE,

```

```

737         hub_id="kasnerz/logicnlg",
738         split="test",
739         normalizer="logicnlg",
740         task_family=TaskFamily.LOGICAL_TABLE_STATEMENT,
741         output_mode=OutputMode.ONE_SENTENCE,
742         sample_size=30,
743         reference_fields=["ref", "references", "target"],
744         id_fields=["table_id", "id"],
745         metadata_fields=["title", "template"],
746     ),
747     DatasetConfig(
748         dataset_id="fetaqa",
749         source=DatasetSource.HUGGINGFACE,
750         hub_id="table-benchmark/fetaqa",
751         split="test",
752         normalizer="fetaqa",
753         task_family=TaskFamily.TABLE_QUESTION_ANSWERING,
754         output_mode=OutputMode.DIRECT_ANSWER,
755         sample_size=30,
756         reference_fields=["answer", "references", "target"],
757         id_fields=["feta_id", "id"],
758         metadata_fields=["table_page_title", "table_section_title"],
759     ),
760     DatasetConfig(
761         dataset_id="viggo",
762         source=DatasetSource.HUGGINGFACE,
763         hub_id="GEM/viggo",
764         split="test",
765         normalizer="viggo",
766         task_family=TaskFamily.ATTRIBUTE_VERBALISATION,
767         output_mode=OutputMode.SHORT_TEXT,
768         sample_size=30,
769         reference_fields=["references", "target"],
770     ),
771     DatasetConfig(
772         dataset_id="mlb_data_to_text",
773         source=DatasetSource.HUGGINGFACE,
774         hub_id="GEM/mlb_data_to_text",
775         split="test",
776         trust_remote_code=True,
777         normalizer="mlb",
778         task_family=TaskFamily.EVENT_REPORT,
779         output_mode=OutputMode.MULTI_PARAGRAPH_REPORT,
780         sample_size=20,
781         reference_fields=["references", "target", "summary_eval", "summary"],
782         id_fields=["gem_parent_id", "gem_id", "game_id", "id"],
783     ),
784     DatasetConfig(
785         dataset_id="conversational_weather",
786         source=DatasetSource.HUGGINGFACE,
787         hub_id="GEM/conversational_weather",
788         split="test",
789         trust_remote_code=True,
790         normalizer="generic",
791         task_family=TaskFamily.WEATHER_RESPONSE,
792         output_mode=OutputMode.SHORT_TEXT,
793         sample_size=30,
794         source_fields=["dialogue_act", "weather_data", "input", "linearized_input"],
795         reference_fields=["references", "target", "response"],
796     ),
797     DatasetConfig(
798         dataset_id="turku_hockey",
799         source=DatasetSource.HUGGINGFACE,
800         hub_id="GEM/turku_hockey_data2text",
801         split="test",
802         trust_remote_code=True,
803         normalizer="turku_hockey",
804         task_family=TaskFamily.EVENT_REPORT,
805         output_mode=OutputMode.PARAGRAPH,
806         language="fi",
807         sample_size=20,
808         reference_fields=["references", "target", "news_article"],
809     ),
810     DatasetConfig(
811         dataset_id="rotowire_english_german",
812         source=DatasetSource.HUGGINGFACE,
813         hub_id="GEM/RotoWire_English-German",
814         split="test",
815         trust_remote_code=True,
816         normalizer="rotowire_en_de",
817         task_family=TaskFamily.CROSS_LINGUAL_EVENT_REPORT,
818         output_mode=OutputMode.MULTI_PARAGRAPH_REPORT,
819         language="de",
820         sample_size=20,
821         reference_fields=["references", "target", "summary_de"],
822     ),
823 ]
824
825 local_configs = [
826     ("rotowire_fg", TaskFamily.EVENT_REPORT, OutputMode.MULTI_PARAGRAPH_REPORT),
827     ("rotowire_original", TaskFamily.EVENT_REPORT, OutputMode.MULTI_PARAGRAPH_REPORT),

```

```

828     ("wikitablet", TaskFamily.LONG_FORM_TABLE_REPORT, OutputMode.PARAGRAPH),
829     ("tagg", TaskFamily.LONG_FORM_TABLE_REPORT, OutputMode.PARAGRAPH),
830     ("ml_performance_explanations", TaskFamily.ANALYTICAL_EXPLANATION, OutputMode.PARAGRAPH),
831 ]
832 for dataset_id, task_family, output_mode in local_configs:
833     configs.append(
834         DatasetConfig(
835             dataset_id=dataset_id,
836             enabled=False,
837             source=DatasetSource.LOCAL,
838             local_path=Path("evaluation/local_datasets") / dataset_id,
839             split="test",
840             normalizer="generic",
841             task_family=task_family,
842             output_mode=output_mode,
843             sample_size=30,
844             source_fields=["table", "metadata", "input", "box_score", "line_score"],
845             reference_fields=["summary", "reference", "references", "target", "explanation"],
846             id_fields=["game_id", "id", "table_id"],
847         )
848     )
849 return configs
850
851 def load_dataset_configs(path: Path) -> list[DatasetConfig]:
852     payload = json.loads(path.read_text(encoding="utf-8"))
853     if isinstance(payload, dict):
854         payload = payload.get("datasets", payload)
855     if not isinstance(payload, list):
856         raise ValueError("The dataset config must contain a list.")
857     return [DatasetConfig.model_validate(item) for item in payload]
858
859 def save_default_dataset_configs(path: Path) -> None:
860     path.parent.mkdir(parents=True, exist_ok=True)
861     path.write_text(
862         json.dumps(
863             {"datasets": [item.model_dump(mode="json") for item in default_dataset_configs()]},
864             indent=2,
865             ensure_ascii=False,
866         )
867         + "\n",
868         encoding="utf-8",
869     )
870
871

```

Listing 31: P31: table2text_pydanticai/src/table2text/evaluation/deepeval_metrics.py

```

1  """Run DeepEval metrics through the project's configured independent judge."""
2
3  from __future__ import annotations
4
5  import os
6  import time
7  from pathlib import Path
8  from typing import Any, Callable
9
10 from .datasets import write_jsonl
11 from .models import DeepEvalConfig, DeepEvalObservation, GenerationRecord, MetricStatus
12 from .reference_metrics import plain_text
13 from table2text.config import load_env_files
14
15 def make_observation(
16     record: GenerationRecord,
17     *,
18     metric_name: str,
19     judge_model: str,
20     judge_repetition: int,
21     status: MetricStatus,
22     score: float | None = None,
23     reason: str | None = None,
24     threshold: float | None = None,
25     success: bool | None = None,
26     duration: float | None = None,
27     error: str | None = None,
28 ) -> DeepEvalObservation:
29     return DeepEvalObservation(
30         generation_id=record.generation_id,
31         dataset_id=record.dataset_id,
32         example_id=record.example_id,
33         variant_id=record.variant_id,
34         repetition=record.repetition,
35         metric_name=metric_name,
36         judge_model=judge_model,
37         judge_repetition=judge_repetition,
38         status=status,
39         score=score,
40         reason=reason,
41         threshold=threshold,
42

```

```

43         success=success,
44         duration_seconds=duration,
45         error=error,
46     )
47
48
49 def source_for_judge(record: GenerationRecord, config: DeepEvalConfig) -> str:
50     source = record.source_text
51     if len(source) <= config.max_source_characters:
52         return source
53     return source[: config.max_source_characters] + "\n\n[Source truncated by evaluation configuration.]"
54
55
56 def _enum_value(value: Any) -> str:
57     return str(getattr(value, "value", value) or "")
58
59
60 def task_guidance_for_judge(record: GenerationRecord) -> str:
61     task_family = _enum_value(record.task_family)
62     output_mode = _enum_value(record.output_mode)
63     guidance = [
64         "Judge the actual output against the requested data-to-text task, "
65         "not against a generic summarisation task.",
66     ]
67
68     if task_family == "highlighted_table_description":
69         guidance.extend(
70             [
71                 "This is a focused-table task: identify the concise "
72                 "table-local proposition expressed by the highlighted cell "
73                 "or focused table region.",
74                 "Use page title, section title, row labels, column headers, "
75                 "highlighted values and source-text context to decide the "
76                 "correct subject and relation.",
77                 "Penalise outputs that merely describe cell coordinates or "
78                 "assign the highlighted value to a row co-entity when the "
79                 "local table context identifies a different primary subject.",
80             ]
81         )
82     elif task_family in {
83         "attribute_verbalisation",
84         "triple_verbalisation",
85     }:
86         guidance.extend(
87             [
88                 "This is a short structured-record verbalisation task: the "
89                 "answer should express all and only the supplied attributes "
90                 "or triples.",
91                 "Prefer concise predicate-preserving wording. Do not reward "
92                 "unrelated background, extra attributes, changed numbers, "
93                 "changed units or changed entity identity.",
94             ]
95         )
96
97     if output_mode in {
98         "one_sentence",
99         "short_text",
100         "direct_answer",
101     }:
102         guidance.append(
103             "The expected output is short; brevity is appropriate when the "
104             "essential supported proposition is complete."
105         )
106
107     return "\n".join(f"-- {item}" for item in guidance)
108
109
110 def input_for_judge(record: GenerationRecord, config: DeepEvalConfig) -> str:
111     source = source_for_judge(record, config)
112     return (
113         f"Dataset ID: {record.dataset_id}\n"
114         f"Example ID: {record.example_id}\n"
115         f"Task family: {_enum_value(record.task_family)}\n"
116         f"Output mode: {_enum_value(record.output_mode)}\n"
117         f"Request: {record.request}\n\n"
118         "Task-specific evaluation guidance:\n"
119         f"{task_guidance_for_judge(record)}\n\n"
120         "Structured source:\n"
121         f"{source}"
122     )
123
124
125 def reference_for_judge(record: GenerationRecord) -> str | None:
126     if not record.references:
127         return None
128     return "\n\n--- ALTERNATIVE REFERENCE ---\n\n".join(record.references)
129
130
131 def source_chunks_for_judge(record: GenerationRecord, config: DeepEvalConfig) -> list[str]:
132     source = source_for_judge(record, config)
133     chunks = [chunk.strip() for chunk in source.split("\n\n") if chunk.strip()]

```

```

134     return chunks or ([source] if source.strip() else [])
135
136
137 def _first_env(*names: str) -> str | None:
138     for name in names:
139         value = os.getenv(name)
140         if value is not None and value.strip():
141             return value.strip()
142     return None
143
144
145 def _normalise_deepseek_model_name(model_name: str) -> str:
146     if model_name.startswith("deepseek:"):
147         return model_name.split(":", 1)[1]
148     return model_name
149
150
151 def _normalise_openai_model_name(model_name: str) -> str:
152     if model_name.startswith("openai:"):
153         return model_name.split(":", 1)[1]
154     return model_name
155
156
157 def apply_deepeval_env_overrides(config: DeepEvalConfig) -> DeepEvalConfig:
158     updates: dict[str, Any] = {}
159     provider = _first_env(
160         "T2T_DEEPEVAL_JUDGE_PROVIDER",
161         "DEEPEVAL_JUDGE_PROVIDER",
162     )
163     if provider is not None:
164         updates["judge_provider"] = provider.lower()
165
166     model = _first_env(
167         "T2T_DEEPEVAL_JUDGE_MODEL",
168         "DEEPEVAL_JUDGE_MODEL",
169     )
170     if model is not None:
171         updates["judge_model"] = model
172
173     repetitions = _first_env(
174         "T2T_DEEPEVAL_JUDGE_REPETITIONS",
175         "DEEPEVAL_JUDGE_REPETITIONS",
176     )
177     if repetitions is not None:
178         updates["judge_repetitions"] = int(repetitions)
179
180     if not updates:
181         return config
182     return DeepEvalConfig.model_validate(config.model_dump() | updates)
183
184
185 def build_judge_model(config: DeepEvalConfig) -> Any:
186     provider = config.judge_provider
187     model_name = config.judge_model
188     if model_name.startswith("deepseek:"):
189         provider = "deepseek"
190         model_name = _normalise_deepseek_model_name(model_name)
191     elif model_name.startswith("openai:"):
192         provider = "openai"
193         model_name = _normalise_openai_model_name(model_name)
194
195     if provider == "deepseek":
196         api_key = _first_env("DEEPSEEK_API_KEY")
197         if api_key is None:
198             raise RuntimeError(
199                 "DeepEval is configured to use DeepSeek, but DEEPSEEK_API_KEY "
200                 "is not set in the environment or project .env file."
201             )
202         from deepeval.models import DeepSeekModel
203
204         return DeepSeekModel(
205             model=_normalise_deepseek_model_name(model_name),
206             api_key=api_key,
207         )
208
209     if provider == "openai":
210         api_key = _first_env("OPENAI_API_KEY", "T2T_OPENAI_API_KEY")
211         if api_key is None:
212             raise RuntimeError(
213                 "DeepEval is configured to use OpenAI, but OPENAI_API_KEY "
214                 "is not set in the environment or project .env file."
215             )
216         from deepeval.models import GPTModel
217
218         return GPTModel(
219             model=_normalise_openai_model_name(model_name),
220             api_key=api_key,
221             base_url=_first_env("OPENAI_BASE_URL", "T2T_OPENAI_BASE_URL"),
222         )
223
224     return model_name

```

```

225
226
227 def run_metric(
228     record: GenerationRecord,
229     metric_name: str,
230     metric_factory: Callable[[], Any],
231     test_case: Any,
232     config: DeepEvalConfig,
233     judge_repetition: int,
234 ) -> DeepEvalObservation:
235     started = time.perf_counter()
236     try:
237         metric = metric_factory()
238         metric.measure(test_case)
239         score = float(metric.score) if metric.score is not None else None
240         return make_observation(
241             record,
242             metric_name=metric_name,
243             judge_model=config.judge_model,
244             judge_repetition=judge_repetition,
245             status=MetricStatus.SCORED,
246             score=score,
247             reason=getattr(metric, "reason", None),
248             threshold=getattr(metric, "threshold", config.threshold),
249             success=getattr(metric, "success", None),
250             duration=time.perf_counter() - started,
251         )
252     except Exception as exc:
253         return make_observation(
254             record,
255             metric_name=metric_name,
256             judge_model=config.judge_model,
257             judge_repetition=judge_repetition,
258             status=MetricStatus.ERROR,
259             duration=time.perf_counter() - started,
260             error=f"{type(exc).__name__}: {exc}",
261         )
262
263
264 def evaluate_record(record: GenerationRecord, config: DeepEvalConfig) -> list[DeepEvalObservation]:
265     try:
266         from deepeval.metrics import FaithfulnessMetric, GEval, SummarizationMetric
267         from deepeval.test_case import LLMTestCase, SingleTurnParams
268     except ImportError:
269         return [
270             make_observation(
271                 record,
272                 metric_name="deepeval",
273                 judge_model=config.judge_model,
274                 judge_repetition=0,
275                 status=MetricStatus.UNAVAILABLE,
276                 error="deepeval is not installed.",
277             )
278         ]
279
280     try:
281         judge_model = build_judge_model(config)
282     except Exception as exc:
283         return [
284             make_observation(
285                 record,
286                 metric_name="deepeval_judge_model",
287                 judge_model=config.judge_model,
288                 judge_repetition=0,
289                 status=MetricStatus.ERROR,
290                 error=f"{type(exc).__name__}: {exc}",
291             )
292         ]
293
294     source = input_for_judge(record, config)
295     source_chunks = source_chunks_for_judge(record, config)
296     generated = plain_text(record.generated_text)
297     expected = reference_for_judge(record)
298     test_case = LLMTestCase(
299         input=source,
300         actual_output=generated,
301         expected_output=expected,
302         retrieval_context=source_chunks,
303     )
304     factories: list[tuple[str, Callable[[], Any]]] = []
305
306     if config.run_summarization:
307         factories.append(
308             (
309                 "deepeval_summarization",
310                 lambda: SummarizationMetric(
311                     threshold=config.threshold,
312                     model=judge_model,
313                     include_reason=True,
314                     async_mode=False,
315                 ),
316             ),

```



```

316     )
317 )
318 if config.run_faithfulness:
319     factories.append(
320         (
321             "deepeval_faithfulness",
322             lambda: FaithfulnessMetric(
323                 threshold=config.threshold,
324                 model=judge_model,
325                 include_reason=True,
326                 async_mode=False,
327             ),
328         )
329 )
330 if config.run_factual_correctness:
331     factories.append(
332         (
333             "geval_factual_correctness",
334             lambda: GEval(
335                 name="Factual correctness",
336                 criteria=(
337                     "Determine whether every factual statement in the actual output "
338                     "is supported by the structured source in the input. Penalise "
339                     "incorrect numbers, entities, rankings, comparisons, chronology "
340                     "and unsupported causal claims. Do not require wording to match "
341                     "a reference."
342                 ),
343                 evaluation_params=[SingleTurnParams.INPUT, SingleTurnParams.ACTUAL_OUTPUT],
344                 threshold=config.threshold,
345                 model=judge_model,
346                 async_mode=False,
347             ),
348         )
349 )
350 if config.run_reference_adequacy and expected is not None:
351     factories.append(
352         (
353             "geval_reference_adequacy",
354             lambda: GEval(
355                 name="Reference adequacy",
356                 criteria=(
357                     "Determine whether the actual output communicates the same "
358                     "important content as one or more expected outputs. Accept valid "
359                     "paraphrases and different organisation. Penalise material "
360                     "omissions and unrelated content."
361                 ),
362                 evaluation_params=[
363                     SingleTurnParams.ACTUAL_OUTPUT,
364                     SingleTurnParams.EXPECTED_OUTPUT,
365                 ],
366                 threshold=config.threshold,
367                 model=judge_model,
368                 async_mode=False,
369             ),
370         )
371 )
372 if config.run_task_relevance:
373     factories.append(
374         (
375             "geval_task_relevance",
376             lambda: GEval(
377                 name="Task relevance",
378                 criteria=(
379                     "Determine whether the actual output follows the requested "
380                     "data-to-text task, stays within the expected scope and avoids "
381                     "irrelevant analysis."
382                 ),
383                 evaluation_params=[SingleTurnParams.INPUT, SingleTurnParams.ACTUAL_OUTPUT],
384                 threshold=config.threshold,
385                 model=judge_model,
386                 async_mode=False,
387             ),
388         )
389 )
390 if config.run_coherence:
391     factories.append(
392         (
393             "geval_coherence",
394             lambda: GEval(
395                 name="Coherence and readability",
396                 criteria=(
397                     "Evaluate whether the actual output is clear, well organised, "
398                     "fluent and non-repetitive for its expected length. Do not judge "
399                     "factual correctness in this criterion."
400                 ),
401                 evaluation_params=[SingleTurnParams.ACTUAL_OUTPUT],
402                 threshold=config.threshold,
403                 model=judge_model,
404                 async_mode=False,
405             ),
406         )

```



```

407     )
408     if config.run_usefulness:
409         factories.append(
410             (
411                 "geval_usefulness",
412                 lambda: GEval(
413                     name="Analytical usefulness",
414                     criteria=(
415                         "Evaluate whether the actual output selects and communicates "
416                         "useful information for the requested task without "
417                         "over-interpreting the source. For simple verbalisation tasks, "
418                         "usefulness means concise complete expression of the supplied facts."
419                     ),
420                     evaluation_params=[SingleTurnParams.INPUT, SingleTurnParams.ACTUAL_OUTPUT],
421                     threshold=config.threshold,
422                     model=judge_model,
423                     async_mode=False,
424                 ),
425             )
426         )
427
428     observations: list[DeepEvalObservation] = []
429     for judge_repetition in range(config.judge_repetitions):
430         for metric_name, factory in factories:
431             observations.append(
432                 run_metric(record, metric_name, factory, test_case, config, judge_repetition)
433             )
434     return observations
435
436
437 def evaluate_deepeval(
438     records: list[GenerationRecord],
439     config: DeepEvalConfig,
440     output_path: Path,
441     *,
442     resume: bool = True,
443 ) -> list[DeepEvalObservation]:
444     load_env_files()
445     config = apply_deepeval_env_overrides(config)
446     if not config.enabled:
447         return []
448     existing: dict[tuple[str, str, int], DeepEvalObservation] = {}
449     if resume and output_path.exists():
450         for line in output_path.read_text(encoding="utf-8").splitlines():
451             if not line.strip():
452                 continue
453             item = DeepEvalObservation.model_validate_json(line)
454             existing[(item.generation_id, item.metric_name, item.judge_repetition)] = item
455
456     observations = dict(existing)
457     for record in records:
458         if record.error is not None or not record.generated_text.strip():
459             continue
460         if record.primary_evaluation_eligible is False:
461             continue
462         for item in evaluate_record(record, config):
463             observations[(item.generation_id, item.metric_name, item.judge_repetition)] = item
464         write_jsonl(output_path, observations.values())
465
466     ordered = sorted(
467         observations.values(),
468         key=lambda item: (
469             item.dataset_id,
470             item.example_id,
471             item.variant_id,
472             item.repetition,
473             item.metric_name,
474             item.judge_repetition,
475         ),
476     )
477     write_jsonl(output_path, ordered)
478     return ordered

```

Listing 32: P32: table2text_pydanticai/src/table2text/evaluation/diagnostics.py

```

1  """Derive generation diagnostics from workflow artifacts and metric records."""
2
3  from __future__ import annotations
4
5  import re
6  from pathlib import Path
7  from typing import Any, Iterable
8
9  import pandas as pd
10
11 from .generation import read_generations
12 from .reference_metrics import plain_text
13
14

```

```

15 NUMBER_PATTERN = re.compile(
16     r"(?![\w.])[-+]?(?:\d{1,3}(?:,\d{3})+|\d+)(?:\.\d+)?%"
17 )
18
19
20 def extract_numbers(value: str) -> list[str]:
21     numbers: list[str] = []
22     for token in NUMBER_PATTERN.findall(value):
23         cleaned = token.replace(",", "").rstrip("%")
24         if cleaned.startswith(("+", "-")):
25             cleaned = cleaned[1:]
26         if cleaned:
27             numbers.append(cleaned)
28     return numbers
29
30
31 def sentence_count(value: str) -> int:
32     text = plain_text(value)
33     return len([item for item in re.split(r"(?<=[.!?])\s+", text) if item.strip()])
34
35
36 def safe_ratio(numerator: float, denominator: float) -> float | None:
37     if denominator == 0:
38         return None
39     return numerator / denominator
40
41
42 def number_diagnostics(
43     generated: str,
44     source: str,
45     references: list[str],
46 ) -> dict[str, float | int | None]:
47     generated_numbers = extract_numbers(generated)
48     source_numbers = set(extract_numbers(source))
49     reference_numbers = {
50         number for reference in references for number in extract_numbers(reference)
51     }
52     source_supported = sum(number in source_numbers for number in generated_numbers)
53     reference_supported = sum(number in reference_numbers for number in generated_numbers)
54     return {
55         "generated_number_count": len(generated_numbers),
56         "source_number_count": len(source_numbers),
57         "reference_number_count": len(reference_numbers),
58         "generated_number_source_precision": safe_ratio(
59             source_supported,
60             len(generated_numbers),
61         ),
62         "generated_number_reference_precision": safe_ratio(
63             reference_supported,
64             len(generated_numbers),
65         ),
66     }
67
68
69 def generation_diagnostics(records: Iterable[Any]) -> pd.DataFrame:
70     rows: list[dict[str, Any]] = []
71     for record in records:
72         generated = plain_text(record.generated_text)
73         support_count = record.support_sentence_count or 0
74         mapped_count = record.mapped_support_sentence_count or 0
75         row = {
76             "generation_id": record.generation_id,
77             "dataset_id": record.dataset_id,
78             "example_id": record.example_id,
79             "variant_id": record.variant_id,
80             "repetition": record.repetition,
81             "seed": record.seed,
82             "language": record.language,
83             "writer_mode": record.writer_mode,
84             "release_status": record.release_status,
85             "approved_for_release": record.approved_for_release,
86             "primary_evaluation_eligible": record.primary_evaluation_eligible,
87             "repair_rounds_used": record.repair_rounds_used,
88             "audit_support_rate": record.audit_support_rate,
89             "elapsed_seconds": record.elapsed_seconds,
90             "word_count": len(generated.split()),
91             "character_count": len(generated),
92             "sentence_count": sentence_count(generated),
93             "reference_count": len(record.references),
94             "support_sentence_count": support_count,
95             "mapped_support_sentence_count": mapped_count,
96             "provenance_coverage": safe_ratio(mapped_count, support_count),
97             "fact_id_reference_count": record.fact_id_reference_count,
98             "evidence_id_reference_count": record.evidence_id_reference_count,
99             "invalid_fact_id_count": record.invalid_fact_id_count,
100             "invalid_evidence_id_count": record.invalid_evidence_id_count,
101             "input_tokens": record.input_tokens,
102             "output_tokens": record.output_tokens,
103             "total_tokens": record.total_tokens,
104             "estimated_cost_gbp": record.estimated_cost_gbp,
105             "generation_error": record.error,

```

```

106     }
107     row.update(number_diagnostics(generated, record.source_text, record.references))
108     rows.append(row)
109     return pd.DataFrame(rows)
110
111
112 def write_diagnostics(generations_path: Path, output_path: Path) -> pd.DataFrame:
113     records = read_generations(generations_path)
114     frame = generation_diagnostics(records)
115     output_path.parent.mkdir(parents=True, exist_ok=True)
116     frame.to_csv(output_path, index=False)
117     return frame

```

Listing 33: P33: table2text_pydanticai/src/table2text/evaluation/external_factuality.py

```

1  """Compute optional source-grounded factuality metrics such as HHEM."""
2
3  from __future__ import annotations
4
5  import os
6  import re
7  import tempfile
8  from contextlib import contextmanager
9  from pathlib import Path
10 from statistics import mean
11 from typing import Any
12
13 from pydantic import BaseModel, ConfigDict, Field
14
15
16 class ExternalFactualityResult(BaseModel):
17     model_config = ConfigDict(extra="forbid")
18
19     metric_name: str
20     status: str
21
22     overall_score: float | None = None
23     sentence_scores: list[float] = Field(default_factory=list)
24     minimum_sentence_score: float | None = None
25     unsupported_sentence_rate: float | None = None
26
27     threshold: float | None = None
28     details: dict[str, Any] = Field(default_factory=dict)
29     error: str | None = None
30
31
32 _SENTENCE_BOUNDARY = re.compile(r"(?<=[.!?])\s+(?=[A-Z0-9])")
33 _TOKEN = re.compile(r"[A-Za-z0-9]+")
34
35
36 @contextmanager
37 def huggingface_offline(enabled: bool):
38     keys = ("HF_HUB_OFFLINE", "TRANSFORMERS_OFFLINE", "HF_MODULES_CACHE")
39     previous = {key: os.environ.get(key) for key in keys}
40     if enabled:
41         os.environ["HF_HUB_OFFLINE"] = "1"
42         os.environ["TRANSFORMERS_OFFLINE"] = "1"
43     if previous["HF_MODULES_CACHE"] is None:
44         modules_cache = Path(tempfile.gettempdir()) / "table2text_hf_modules"
45         modules_cache.mkdir(parents=True, exist_ok=True)
46         os.environ["HF_MODULES_CACHE"] = str(modules_cache)
47     try:
48         yield
49     finally:
50         for key, value in previous.items():
51             if value is None:
52                 os.environ.pop(key, None)
53             else:
54                 os.environ[key] = value
55
56
57 def split_sentences(text: str) -> list[str]:
58     cleaned = re.sub(r"\s+", " ", text).strip()
59     if not cleaned:
60         return []
61     return [
62         sentence.strip()
63         for sentence in _SENTENCE_BOUNDARY.split(cleaned)
64         if sentence.strip()
65     ]
66
67
68 def summarise_sentence_scores(
69     *,
70     metric_name: str,
71     scores: list[float],
72     threshold: float,
73     details: dict[str, Any] | None = None,
74 ) -> ExternalFactualityResult:

```

```

75     if not scores:
76         return ExternalFactualityResult(
77             metric_name=metric_name,
78             status="skipped",
79             threshold=threshold,
80             details={"reason": "No factual sentences were supplied."},
81         )
82
83     unsupported_count = sum(score < threshold for score in scores)
84
85     return ExternalFactualityResult(
86         metric_name=metric_name,
87         status="scored",
88         overall_score=mean(scores),
89         sentence_scores=scores,
90         minimum_sentence_score=min(scores),
91         unsupported_sentence_rate=unsupported_count / len(scores),
92         threshold=threshold,
93         details=details or {},
94     )
95
96
97 def _token_set(text: str) -> set[str]:
98     return {token.casefold() for token in _TOKEN.findall(text)}
99
100
101 def compact_support_context(
102     *,
103     context: str,
104     claim: str,
105     max_characters: int,
106 ) -> str:
107     context = re.sub(r"\s+", " ", context).strip()
108     if len(context) <= max_characters:
109         return context
110
111     claim_tokens = _token_set(claim)
112     units = split_sentences(context) or [context]
113     ranked: list[tuple[int, int, str]] = []
114     for index, unit in enumerate(units):
115         overlap = len(claim_tokens & _token_set(unit))
116         ranked.append((overlap, index, unit))
117
118     selected: list[tuple[int, str]] = []
119     used = 0
120     for overlap, index, unit in sorted(
121         ranked,
122         key=lambda item: (item[0], -item[1]),
123         reverse=True,
124     ):
125         if overlap == 0 and selected:
126             continue
127         next_length = len(unit) + (1 if selected else 0)
128         if used + next_length > max_characters and selected:
129             continue
130         selected.append((index, unit[:max_characters]))
131         used += min(next_length, max_characters)
132         if used >= max_characters:
133             break
134
135     if not selected:
136         return context[:max_characters].strip()
137
138     ordered = [unit for _, unit in sorted(selected, key=lambda item: item[0])]
139     compacted = " ".join(ordered).strip()
140     return compacted[:max_characters].strip()
141
142
143 class HHEvaluator:
144     def __init__(
145         self,
146         *,
147         model_name: str = "vectara/hallucination_evaluation_model",
148         foundation_model_name: str = "google/flan-t5-base",
149         threshold: float = 0.5,
150         batch_size: int = 16,
151         device: str | None = None,
152         local_files_only: bool = True,
153         max_context_characters: int = 1_000,
154     ) -> None:
155         self.model_name = model_name
156         self.foundation_model_name = foundation_model_name
157         self.threshold = threshold
158         self.batch_size = batch_size
159         self.device = device
160         self.local_files_only = local_files_only
161         self.max_context_characters = max_context_characters
162         self._model: Any | None = None
163
164     def _load_model(self) -> Any:
165         if self._model is not None:

```

```

166         return self._model
167
168     from huggingface_hub import snapshot_download
169
170     model_path = self.model_name
171     foundation_path = self.foundation_model_name
172     if self.local_files_only:
173         model_path = snapshot_download(
174             repo_id=self.model_name,
175             local_files_only=True,
176         )
177         foundation_path = snapshot_download(
178             repo_id=self.foundation_model_name,
179             local_files_only=True,
180         )
181
182     with huggingface_offline(self.local_files_only):
183         from transformers import AutoConfig, AutoModelForSequenceClassification
184
185         model_config = AutoConfig.from_pretrained(
186             model_path,
187             trust_remote_code=True,
188             local_files_only=self.local_files_only,
189         )
190         model_config.foundation = foundation_path
191
192         kwargs: dict[str, Any] = {
193             "pretrained_model_name_or_path": model_path,
194             "config": model_config,
195             "trust_remote_code": True,
196             "local_files_only": self.local_files_only,
197         }
198         if self.device and self.device != "cpu":
199             kwargs["device_map"] = self.device
200         self._model = AutoModelForSequenceClassification.from_pretrained(**kwargs)
201     self._model.eval()
202     return self._model
203
204 def evaluate(
205     self,
206     *,
207     context: str,
208     generated_text: str,
209 ) -> ExternalFactualityResult:
210     sentences = split_sentences(generated_text)
211     metric_name = "hhem_2_1_open"
212
213     if not context.strip():
214         return ExternalFactualityResult(
215             metric_name=metric_name,
216             status="skipped",
217             threshold=self.threshold,
218             details={"reason": "The factuality context is empty."},
219         )
220     if not sentences:
221         return ExternalFactualityResult(
222             metric_name=metric_name,
223             status="skipped",
224             threshold=self.threshold,
225             details={"reason": "The generated output is empty."},
226         )
227
228     try:
229         import torch
230     except ImportError as exc:
231         return ExternalFactualityResult(
232             metric_name=metric_name,
233             status="unavailable",
234             threshold=self.threshold,
235             error=f"torch is not installed: {exc}",
236         )
237
238     try:
239         model = self._load_model()
240         if not hasattr(model, "predict"):
241             return ExternalFactualityResult(
242                 metric_name=metric_name,
243                 status="error",
244                 threshold=self.threshold,
245                 error="The loaded HHEM model does not expose a predict method.",
246             )
247
248         scores: list[float] = []
249         with torch.inference_mode():
250             for start in range(0, len(sentences), self.batch_size):
251                 batch = sentences[start : start + self.batch_size]
252                 pairs = [
253                     (
254                         compact_support_context(
255                             context=context,
256                             claim=sentence,

```

```

257         max_characters=self.max_context_characters,
258     ),
259     sentence,
260 )
261     for sentence in batch
262 ]
263 with huggingface_offline(self.local_files_only):
264     batch_scores = model.predict(pairs)
265     if hasattr(batch_scores, "detach"):
266         batch_scores = batch_scores.detach().cpu().tolist()
267     elif hasattr(batch_scores, "tolist"):
268         batch_scores = batch_scores.tolist()
269     for score in batch_scores:
270         if isinstance(score, (list, tuple)):
271             score = score[0]
272         scores.append(float(score))
273
274     return summarise_sentence_scores(
275         metric_name=metric_name,
276         scores=scores,
277         threshold=self.threshold,
278         details={
279             "model_name": self.model_name,
280             "foundation_model_name": self.foundation_model_name,
281             "sentence_count": len(sentences),
282             "sentences": sentences,
283         },
284     )
285 except ImportError as exc:
286     return ExternalFactualityResult(
287         metric_name=metric_name,
288         status="unavailable",
289         threshold=self.threshold,
290         error=f"transformers is not installed: {exc}",
291     )
292 except Exception as exc:
293     return ExternalFactualityResult(
294         metric_name=metric_name,
295         status="error",
296         threshold=self.threshold,
297         error=f"{type(exc).__name__}: {exc}",
298     )

```

Listing 34: P34: table2text_pydanticai/src/table2text/evaluation/generation.py

```

1  """Run evaluation variants and persist resumable generation records."""
2
3  from __future__ import annotations
4
5  import importlib
6  import json
7  import os
8  import subprocess
9  import tempfile
10 import time
11 from dataclasses import fields, replace
12 from pathlib import Path
13 from typing import Any, Callable
14
15 from .datasets import write_jsonl
16 from .models import (
17     BenchmarkExample,
18     GenerationBackend,
19     GenerationRecord,
20     OutputMode,
21     TaskFamily,
22     VariantConfig,
23 )
24
25
26 def load_variants(path: Path) -> list[VariantConfig]:
27     payload = json.loads(path.read_text(encoding="utf-8"))
28     if isinstance(payload, dict):
29         payload = payload.get("variants", payload)
30     if not isinstance(payload, list):
31         raise ValueError("The variant configuration must be a list.")
32     return [VariantConfig.model_validate(item) for item in payload]
33
34
35 def plain_generation_id(
36     example: BenchmarkExample,
37     variant: VariantConfig,
38     repetition: int,
39     seed: int,
40 ) -> str:
41     return (
42         f"{example.dataset_id}_{example.example_id}_"
43         f"{variant.variant_id}_r{repetition}_s{seed}"

```

```

44     )
45
46
47 def materialise_input(
48     example: BenchmarkExample,
49     directory: Path,
50     *,
51     source_only: bool = False,
52 ) -> Path:
53     directory.mkdir(parents=True, exist_ok=True)
54     path = directory / f"{example.dataset_id}_{example.example_id}.json"
55     if source_only:
56         payload = example.source_payload
57     elif example.task_family in {
58         TaskFamily.HIGHLIGHTED_TABLE_DESCRIPTION,
59         TaskFamily.ATTRIBUTE_VERBALISATION,
60         TaskFamily.TRIPLE_VERBALISATION,
61     }:
62         payload = {
63             "__table2text_benchmark_example__": True,
64             "dataset_id": example.dataset_id,
65             "example_id": example.example_id,
66             "task_family": example.task_family.value,
67             "output_mode": example.output_mode.value,
68             "request": example.request,
69             "source_text": example.source_text,
70             "source_payload": example.source_payload,
71             "parent_table": example.parent_table,
72             "metadata": example.metadata,
73         }
74     else:
75         payload = example.source_payload
76     path.write_text(
77         json.dumps(payload, indent=2, ensure_ascii=False) + "\n",
78         encoding="utf-8",
79     )
80     return path
81
82
83 def report_genre_for_task(task_family: TaskFamily) -> Any:
84     from table2text.schemas import ReportGenre
85
86     if task_family in {TaskFamily.EVENT_REPORT, TaskFamily.CROSS_LINGUAL_EVENT_REPORT}:
87         return ReportGenre.EVENT_REPORT
88     if task_family in {TaskFamily.LONG_FORM_TABLE_REPORT, TaskFamily.ANALYTICAL_EXPLANATION}:
89         return ReportGenre.DATA_SCIENCE_REPORT
90     return ReportGenre.DATASET_OVERVIEW
91
92
93 def communication_task_for_task(task_family: TaskFamily) -> Any:
94     from table2text.schemas import CommunicationTask
95
96     mapping = {
97         TaskFamily.EVENT_REPORT: CommunicationTask.EVENT_REPORT,
98         TaskFamily.CROSS_LINGUAL_EVENT_REPORT: CommunicationTask.EVENT_REPORT,
99         TaskFamily.LONG_FORM_TABLE_REPORT: CommunicationTask.DATA_SCIENCE_REPORT,
100        TaskFamily.ANALYTICAL_EXPLANATION: CommunicationTask.DATA_SCIENCE_REPORT,
101        TaskFamily.HIGHLIGHTED_TABLE_DESCRIPTION: (
102            CommunicationTask.FOCUSED_TABLE_DESCRIPTION
103        ),
104        TaskFamily.LOGICAL_TABLE_STATEMENT: CommunicationTask.TABLE_ENTAILMENT,
105        TaskFamily.TABLE_QUESTION_ANSWERING: (
106            CommunicationTask.TABLE_QUESTION_ANSWERING
107        ),
108        TaskFamily.ATTRIBUTE_VERBALISATION: (
109            CommunicationTask.ATTRIBUTE_VERBALISATION
110        ),
111        TaskFamily.TRIPLE_VERBALISATION: (
112            CommunicationTask.TRIPLE_VERBALISATION
113        ),
114    }
115     return mapping.get(task_family, CommunicationTask.CUSTOM)
116
117
118 def output_form_for_mode(output_mode: Any) -> Any:
119     from table2text.schemas import OutputForm
120
121     mapping = {
122         OutputMode.ONE_SENTENCE: OutputForm.ONE_SENTENCE,
123         OutputMode.DIRECT_ANSWER: OutputForm.DIRECT_ANSWER,
124         OutputMode.SHORT_TEXT: OutputForm.SHORT_TEXT,
125         OutputMode.PARAGRAPH: OutputForm.PARAGRAPH,
126         OutputMode.MULTI_PARAGRAPH_REPORT: (
127             OutputForm.MULTI_PARAGRAPH_REPORT
128         ),
129     }
130     return mapping.get(output_mode, OutputForm.MULTI_PARAGRAPH_REPORT)
131
132
133 def focus_scope_for_task(task_family: TaskFamily) -> str | None:
134     if task_family in {

```

```

135     TaskFamily.EVENT_REPORT,
136     TaskFamily.CROSS_LINGUAL_EVENT_REPORT,
137     }:
138     return "reference_recap"
139 if task_family == TaskFamily.HIGHLIGHTED_TABLE_DESCRIPTION:
140     return "highlighted_cells"
141 return None
142
143
144 def workflow_contract_for_variant(
145     example: BenchmarkExample,
146     variant: VariantConfig,
147 ) -> dict[str, Any]:
148     if variant.task_contract_mode == "inferred":
149         return {
150             "report_genre": None,
151             "communication_task": None,
152             "output_form": None,
153             "focus_scope": None,
154         }
155     return {
156         "report_genre": report_genre_for_task(example.task_family),
157         "communication_task": communication_task_for_task(example.task_family),
158         "output_form": output_form_for_mode(example.output_mode),
159         "focus_scope": focus_scope_for_task(example.task_family),
160     }
161
162
163 def valid_setting_names() -> set[str]:
164     from table2text.config import Settings
165
166     return {field.name for field in fields(Settings)}
167
168
169 def _base_generation_record(
170     *,
171     example: BenchmarkExample,
172     variant: VariantConfig,
173     repetition: int,
174     seed: int,
175     generated_text: str,
176     backend: GenerationBackend,
177     elapsed_seconds: float | None = None,
178     metadata: dict[str, Any] | None = None,
179     error: str | None = None,
180     request: str | None = None,
181 ) -> GenerationRecord:
182     return GenerationRecord(
183         generation_id=plain_generation_id(example, variant, repetition, seed),
184         dataset_id=example.dataset_id,
185         example_id=example.example_id,
186         variant_id=variant.variant_id,
187         repetition=repetition,
188         seed=seed,
189         task_family=example.task_family,
190         output_mode=example.output_mode,
191         language=example.language,
192         source_text=example.source_text,
193         references=example.references,
194         parent_table=example.parent_table,
195         request=request or example.request,
196         generated_text=generated_text,
197         backend=backend,
198         elapsed_seconds=elapsed_seconds,
199         metadata=metadata or {},
200         error=error,
201     )
202
203
204 def table2text_generate(
205     example: BenchmarkExample,
206     variant: VariantConfig,
207     repetition: int,
208     seed: int,
209     run_root: Path,
210 ) -> GenerationRecord:
211     from table2text import Settings, Table2TextWorkflow
212     from table2text.schemas import AuditMode, EvaluationFieldPolicy
213
214     started = time.perf_counter()
215     run_request = variant.request_override or example.request
216     try:
217         settings = Settings.from_env()
218         overrides = {
219             **variant.settings_overrides,
220             "random_seed": seed,
221             "output_dir": run_root / variant.variant_id / example.dataset_id,
222         }
223         unknown = set(overrides) - valid_setting_names()
224         if unknown:
225             raise ValueError(

```



```

226         f"Variant `{variant.variant_id}` contains unknown Settings fields: "
227         f"{sorted(unknown)})."
228     )
229     settings = replace(settings, **overrides)
230     input_path = materialise_input(
231         example,
232         run_root / "_inputs",
233         source_only=variant.task_contract_mode == "inferred",
234     )
235     workflow = Table2TextWorkflow(settings)
236     result = workflow.run_sync(
237         inputs=[input_path],
238         request=run_request,
239         audit_mode=AuditMode.INTERNAL,
240         evaluation_field_policy=EvaluationFieldPolicy(
241             operational_input_paths=["$"],
242             held_out_reference_paths=[],
243             metadata_paths=[],
244         ),
245         **workflow_contract_for_variant(example, variant),
246     )
247     output = result.final_writer_output
248     pipeline_result_path = settings.output_dir / result.run_id / "pipeline_result.json"
249     pipeline_result_path.parent.mkdir(parents=True, exist_ok=True)
250     pipeline_result_path.write_text(result.model_dump_json(indent=2), encoding="utf-8")
251
252     fact_ids = {fact.fact_id for fact in result.fact_ledger.writer_ready_facts}
253     evidence_ids = {item.evidence_id for item in result.evidence_ledger.items}
254     referenced_fact_ids = [
255         fact_id for support in output.sentence_support for fact_id in support.fact_ids
256     ]
257     referenced_evidence_ids = [
258         evidence_id
259         for support in output.sentence_support
260         for evidence_id in support.evidence_ids
261     ]
262     mapped_support_count = sum(
263         bool(
264             support.fact_ids
265             or support.evidence_ids
266             or support.profile_support_ids
267             or support.insight_ids
268         )
269         for support in output.sentence_support
270     )
271     base = _base_generation_record(
272         example=example,
273         variant=variant,
274         repetition=repetition,
275         seed=seed,
276         generated_text=output.markdown,
277         backend=GenerationBackend.TABLE2TEXT,
278         elapsed_seconds=time.perf_counter() - started,
279         request=run_request,
280     )
281     return base.model_copy(
282         update={
283             "run_id": result.run_id,
284             "pipeline_result_path": pipeline_result_path,
285             "writer_mode": output.writer_mode,
286             "release_status": result.release_status.value,
287             "approved_for_release": result.approved_for_release,
288             "primary_evaluation_eligible": (
289                 result.primary_evaluation_eligible
290                 and output.eligible_for_primary_evaluation
291             ),
292             "primary_evaluation_reason": result.primary_evaluation_reason,
293             "repair_rounds_used": result.repair_rounds_used,
294             "audit_support_rate": result.final_audit.support_rate,
295             "support_sentence_count": len(output.sentence_support),
296             "mapped_support_sentence_count": mapped_support_count,
297             "fact_id_reference_count": len(referenced_fact_ids),
298             "evidence_id_reference_count": len(referenced_evidence_ids),
299             "invalid_fact_id_count": sum(
300                 fact_id not in fact_ids for fact_id in referenced_fact_ids
301             ),
302             "invalid_evidence_id_count": sum(
303                 evidence_id not in evidence_ids
304                 for evidence_id in referenced_evidence_ids
305             ),
306             "metadata": {
307                 "title": output.title,
308                 "selected_fact_count": len(output.selected_fact_ids),
309                 "omitted_fact_count": len(output.omitted_fact_ids),
310                 "quality_revision_round": output.quality_revision_round,
311                 "inferred_task_contract": (
312                     result.inferred_task_contract.model_dump(mode="json")
313                     if result.inferred_task_contract is not None
314                     else None
315                 ),
316                 "resolved_task_contract": (

```

```

317         result.resolved_task_contract.model_dump(mode="json")
318         if result.resolved_task_contract is not None
319             else None
320     ),
321     "task_contract_agreement": result.task_contract_agreement,
322 },
323 },
324 )
325 except Exception as exc:
326     return _base_generation_record(
327         example=example,
328         variant=variant,
329         repetition=repetition,
330         seed=seed,
331         generated_text="",
332         backend=GenerationBackend.TABLE2TEXT,
333         elapsed_seconds=time.perf_counter() - started,
334         error=f"{type(exc).__name__}: {exc}",
335         request=run_request,
336     )
337
338
339 async def table2text_generate_async(
340     example: BenchmarkExample,
341     variant: VariantConfig,
342     repetition: int,
343     seed: int,
344     run_root: Path,
345 ) -> GenerationRecord:
346     from table2text import Settings, Table2TextWorkflow
347     from table2text.schemas import AuditMode, EvaluationFieldPolicy
348
349     started = time.perf_counter()
350     run_request = variant.request_override or example.request
351     try:
352         settings = Settings.from_env()
353         overrides = {
354             **variant.settings_overrides,
355             "random_seed": seed,
356             "output_dir": run_root / variant.variant_id / example.dataset_id,
357         }
358         unknown = set(overrides) - valid_setting_names()
359         if unknown:
360             raise ValueError(
361                 f"Variant `{variant.variant_id}` contains unknown Settings fields: "
362                 f"{sorted(unknown)}."
363             )
364         settings = replace(settings, **overrides)
365         input_path = materialise_input(
366             example,
367             run_root / "_inputs",
368             source_only=variant.task_contract_mode == "inferred",
369         )
370         workflow = Table2TextWorkflow(settings)
371         result = await workflow.run(
372             inputs=[input_path],
373             request=run_request,
374             audit_mode=AuditMode.INTERNAL,
375             evaluation_field_policy=EvaluationFieldPolicy(
376                 operational_input_paths=["$"],
377                 held_out_reference_paths=[],
378                 metadata_paths=[],
379             ),
380             **workflow_contract_for_variant(example, variant),
381         )
382         output = result.final_writer_output
383         pipeline_result_path = settings.output_dir / result.run_id / "pipeline_result.json"
384         pipeline_result_path.parent.mkdir(parents=True, exist_ok=True)
385         pipeline_result_path.write_text(result.model_dump_json(indent=2), encoding="utf-8")
386
387         fact_ids = {fact.fact_id for fact in result.fact_ledger.writer_ready_facts}
388         evidence_ids = {item.evidence_id for item in result.evidence_ledger.items}
389         referenced_fact_ids = [
390             fact_id for support in output.sentence_support for fact_id in support.fact_ids
391         ]
392         referenced_evidence_ids = [
393             evidence_id
394             for support in output.sentence_support
395             for evidence_id in support.evidence_ids
396         ]
397         mapped_support_count = sum(
398             bool(
399                 support.fact_ids
400                 or support.evidence_ids
401                 or support.profile_support_ids
402                 or support.insight_ids
403             )
404             for support in output.sentence_support
405         )
406         base = _base_generation_record(
407             example=example,

```

```

408         variant=variant,
409         repetition=repetition,
410         seed=seed,
411         generated_text=output.markdown,
412         backend=GenerationBackend.TABLE2TEXT,
413         elapsed_seconds=time.perf_counter() - started,
414         request=run_request,
415     )
416     return base.model_copy(
417         update={
418             "run_id": result.run_id,
419             "pipeline_result_path": pipeline_result_path,
420             "writer_mode": output.writer_mode,
421             "release_status": result.release_status.value,
422             "approved_for_release": result.approved_for_release,
423             "primary_evaluation_eligible": (
424                 result.primary_evaluation_eligible
425                 and output.eligible_for_primary_evaluation
426             ),
427             "primary_evaluation_reason": result.primary_evaluation_reason,
428             "repair_rounds_used": result.repair_rounds_used,
429             "audit_support_rate": result.final_audit.support_rate,
430             "support_sentence_count": len(output.sentence_support),
431             "mapped_support_sentence_count": mapped_support_count,
432             "fact_id_reference_count": len(referenced_fact_ids),
433             "evidence_id_reference_count": len(referenced_evidence_ids),
434             "invalid_fact_id_count": sum(
435                 fact_id not in fact_ids for fact_id in referenced_fact_ids
436             ),
437             "invalid_evidence_id_count": sum(
438                 evidence_id not in evidence_ids
439                 for evidence_id in referenced_evidence_ids
440             ),
441             "metadata": {
442                 "title": output.title,
443                 "selected_fact_count": len(output.selected_fact_ids),
444                 "omitted_fact_count": len(output.omitted_fact_ids),
445                 "quality_revision_round": output.quality_revision_round,
446                 "inferred_task_contract": (
447                     result.inferred_task_contract.model_dump(mode="json")
448                     if result.inferred_task_contract is not None
449                     else None
450                 ),
451                 "resolved_task_contract": (
452                     result.resolved_task_contract.model_dump(mode="json")
453                     if result.resolved_task_contract is not None
454                     else None
455                 ),
456                 "task_contract_agreement": result.task_contract_agreement,
457             },
458         },
459     )
460     except Exception as exc:
461         return _base_generation_record(
462             example=example,
463             variant=variant,
464             repetition=repetition,
465             seed=seed,
466             generated_text="",
467             backend=GenerationBackend.TABLE2TEXT,
468             elapsed_seconds=time.perf_counter() - started,
469             error=f"{type(exc).__name__}: {exc}",
470             request=run_request,
471         )
472
473 def resolve_callable(dotted_path: str) -> Callable[..., Any]:
474     module_name, separator, attribute_name = dotted_path.rpartition(".")
475     if not separator:
476         raise ValueError("callable_path must be formatted as `package.module.function`.")
477     module = importlib.import_module(module_name)
478     value = getattr(module, attribute_name)
479     if not callable(value):
480         raise TypeError(f"{dotted_path} is not callable.")
481     return value
482
483
484 def callable_generate(
485     example: BenchmarkExample,
486     variant: VariantConfig,
487     repetition: int,
488     seed: int,
489 ) -> GenerationRecord:
490     assert variant.callable_path is not None
491     started = time.perf_counter()
492     try:
493         function = resolve_callable(variant.callable_path)
494         result = function(example=example, variant=variant, repetition=repetition, seed=seed)
495         if isinstance(result, str):
496             text = result
497             metadata: dict[str, Any] = {}
498

```

```

499     elif isinstance(result, dict):
500         text = str(result.get("generated_text", ""))
501         metadata = {key: value for key, value in result.items() if key != "generated_text"}
502     else:
503         raise TypeError("A generation callable must return a string or dictionary.")
504     return _base_generation_record(
505         example=example,
506         variant=variant,
507         repetition=repetition,
508         seed=seed,
509         generated_text=text,
510         backend=GenerationBackend.CALLABLE,
511         elapsed_seconds=time.perf_counter() - started,
512         metadata=metadata,
513     )
514 except Exception as exc:
515     return _base_generation_record(
516         example=example,
517         variant=variant,
518         repetition=repetition,
519         seed=seed,
520         generated_text="",
521         backend=GenerationBackend.CALLABLE,
522         elapsed_seconds=time.perf_counter() - started,
523         error=f"{type(exc).__name__}: {exc}",
524     )
525
526
527 def command_generate(
528     example: BenchmarkExample,
529     variant: VariantConfig,
530     repetition: int,
531     seed: int,
532 ) -> GenerationRecord:
533     started = time.perf_counter()
534     try:
535         with tempfile.TemporaryDirectory() as directory:
536             root = Path(directory)
537             request_path = root / "request.json"
538             response_path = root / "response.json"
539             request_path.write_text(
540                 json.dumps(
541                     {
542                         "example": example.model_dump(mode="json"),
543                         "variant": variant.model_dump(mode="json"),
544                         "repetition": repetition,
545                         "seed": seed,
546                     },
547                     ensure_ascii=False,
548                     indent=2,
549                 ),
550                 encoding="utf-8",
551             )
552             replacements = {
553                 "{request_json}": str(request_path),
554                 "{response_json}": str(response_path),
555                 "{seed}": str(seed),
556             }
557             command = []
558             for part in variant.command:
559                 rendered = part
560                 for old, new in replacements.items():
561                     rendered = rendered.replace(old, new)
562                 command.append(rendered)
563             completed = subprocess.run(
564                 command,
565                 check=False,
566                 text=True,
567                 capture_output=True,
568                 env={**os.environ, "TABLE2TEXT_EVALUATION_SEED": str(seed)},
569             )
570             if completed.returncode != 0:
571                 raise RuntimeError(
572                     f"Command exited with {completed.returncode}: {completed.stderr.strip()}"
573                 )
574             if response_path.exists():
575                 response = json.loads(response_path.read_text(encoding="utf-8"))
576                 text = str(response.get("generated_text", ""))
577                 metadata = {key: value for key, value in response.items() if key != "generated_text"}
578             else:
579                 text = completed.stdout.strip()
580                 metadata = {"stderr": completed.stderr}
581     return _base_generation_record(
582         example=example,
583         variant=variant,
584         repetition=repetition,
585         seed=seed,
586         generated_text=text,
587         backend=GenerationBackend.COMMAND,
588         elapsed_seconds=time.perf_counter() - started,
589         metadata=metadata,

```

```

590 )
591 except Exception as exc:
592     return _base_generation_record(
593         example=example,
594         variant=variant,
595         repetition=repetition,
596         seed=seed,
597         generated_text="",
598         backend=GenerationBackend.COMMAND,
599         elapsed_seconds=time.perf_counter() - started,
600         error=f"{type(exc).__name__}: {exc}",
601     )
602
603
604 def load_precomputed(path: Path) -> dict[tuple[str, str, int], GenerationRecord]:
605     result: dict[tuple[str, str, int], GenerationRecord] = {}
606     for line in path.read_text(encoding="utf-8").splitlines():
607         if not line.strip():
608             continue
609         record = GenerationRecord.model_validate(json.loads(line))
610         result[(record.dataset_id, record.example_id, record.repetition)] = record
611     return result
612
613
614 def seeds_for_variant(variant: VariantConfig) -> list[int]:
615     if variant.seeds:
616         if len(variant.seeds) < variant.repetitions:
617             raise ValueError(f"{variant.variant_id}: not enough seeds for the requested repetitions.")
618         return variant.seeds[: variant.repetitions]
619     return [42 + repetition for repetition in range(variant.repetitions)]
620
621
622 def generate_all(
623     examples: list[BenchmarkExample],
624     variants: list[VariantConfig],
625     output_path: Path,
626     run_root: Path,
627     *,
628     resume: bool = True,
629 ) -> list[GenerationRecord]:
630     existing: dict[str, GenerationRecord] = {}
631     if resume and output_path.exists():
632         for line in output_path.read_text(encoding="utf-8").splitlines():
633             if not line.strip():
634                 continue
635             record = GenerationRecord.model_validate(json.loads(line))
636             existing[record.generation_id] = record
637     records = dict(existing)
638     for variant in variants:
639         if not variant.enabled:
640             continue
641         precomputed = (
642             load_precomputed(variant.precomputed_path)
643             if variant.backend == GenerationBackend.PRECOMPUTED
644             and variant.precomputed_path is not None
645             and variant.precomputed_path.exists()
646             else {}
647         )
648         for example in examples:
649             for repetition, seed in enumerate(seeds_for_variant(variant)):
650                 generation_id = plain_generation_id(example, variant, repetition, seed)
651                 if generation_id in records:
652                     continue
653                 if variant.backend == GenerationBackend.TABLE2TEXT:
654                     record = table2text_generate(example, variant, repetition, seed, run_root)
655                 elif variant.backend == GenerationBackend.CALLABLE:
656                     record = callable_generate(example, variant, repetition, seed)
657                 elif variant.backend == GenerationBackend.COMMAND:
658                     record = command_generate(example, variant, repetition, seed)
659                 elif variant.backend == GenerationBackend.PRECOMPUTED:
660                     key = (example.dataset_id, example.example_id, repetition)
661                     if key in precomputed:
662                         record = precomputed[key].model_copy(
663                             update={
664                                 "generation_id": generation_id,
665                                 "variant_id": variant.variant_id,
666                                 "seed": seed,
667                             }
668                         )
669                 else:
670                     record = _base_generation_record(
671                         example=example,
672                         variant=variant,
673                         repetition=repetition,
674                         seed=seed,
675                         generated_text="",
676                         backend=GenerationBackend.PRECOMPUTED,
677                         error="No matching precomputed generation was found.",
678                     )
679             else:
680                 raise ValueError(f"Unsupported backend: {variant.backend}")

```

```

681         records[generation_id] = record
682         write_jsonl(output_path, records.values())
683
684     ordered = sorted(
685         records.values(),
686         key=lambda item: (item.dataset_id, item.example_id, item.variant_id, item.repetition),
687     )
688     write_jsonl(output_path, ordered)
689     return ordered
690
691
692 async def generate_all_async(
693     examples: list[BenchmarkExample],
694     variants: list[VariantConfig],
695     output_path: Path,
696     run_root: Path,
697     *,
698     resume: bool = True,
699 ) -> list[GenerationRecord]:
700     existing: dict[str, GenerationRecord] = {}
701     if resume and output_path.exists():
702         for line in output_path.read_text(encoding="utf-8").splitlines():
703             if not line.strip():
704                 continue
705             record = GenerationRecord.model_validate(json.loads(line))
706             existing[record.generation_id] = record
707     records = dict(existing)
708     for variant in variants:
709         if not variant.enabled:
710             continue
711         precomputed = (
712             load_precomputed(variant.precomputed_path)
713             if variant.backend == GenerationBackend.PRECOMPUTED
714             and variant.precomputed_path is not None
715             and variant.precomputed_path.exists()
716             else {}
717         )
718         for example in examples:
719             for repetition, seed in enumerate(seeds_for_variant(variant)):
720                 generation_id = plain_generation_id(example, variant, repetition, seed)
721                 if generation_id in records:
722                     continue
723                 if variant.backend == GenerationBackend.TABLE2TEXT:
724                     record = await table2text_generate_async(
725                         example,
726                         variant,
727                         repetition,
728                         seed,
729                         run_root,
730                     )
731                 elif variant.backend == GenerationBackend.CALLABLE:
732                     record = callable_generate(example, variant, repetition, seed)
733                 elif variant.backend == GenerationBackend.COMMAND:
734                     record = command_generate(example, variant, repetition, seed)
735                 elif variant.backend == GenerationBackend.PRECOMPUTED:
736                     key = (example.dataset_id, example.example_id, repetition)
737                     if key in precomputed:
738                         record = precomputed[key].model_copy(
739                             update={
740                                 "generation_id": generation_id,
741                                 "variant_id": variant.variant_id,
742                                 "seed": seed,
743                             }
744                         )
745                 else:
746                     record = _base_generation_record(
747                         example=example,
748                         variant=variant,
749                         repetition=repetition,
750                         seed=seed,
751                         generated_text="",
752                         backend=GenerationBackend.PRECOMPUTED,
753                         error="No matching precomputed generation was found.",
754                     )
755                 else:
756                     raise ValueError(f"Unsupported backend: {variant.backend}")
757                 records[generation_id] = record
758                 write_jsonl(output_path, records.values())
759
760     ordered = sorted(
761         records.values(),
762         key=lambda item: (item.dataset_id, item.example_id, item.variant_id, item.repetition),
763     )
764     write_jsonl(output_path, ordered)
765     return ordered
766
767
768 def read_generations(path: Path) -> list[GenerationRecord]:
769     return [
770         GenerationRecord.model_validate(json.loads(line))
771         for line in path.read_text(encoding="utf-8").splitlines()

```

```

772     if line.strip()
773 ]

```

Listing 35: P35: table2text_pydanticai/src/table2text/evaluation/human_evaluation.py

```

1  """Build blinded human-study packets and analyse completed annotations."""
2
3  from __future__ import annotations
4
5  import csv
6  import hashlib
7  import json
8  import random
9  from collections import defaultdict
10 from itertools import combinations
11 from pathlib import Path
12 from typing import Any
13
14 import pandas as pd
15 from sklearn.metrics import cohen_kappa_score
16
17 from .datasets import write_jsonl
18 from .models import GenerationRecord, HumanEvaluationPair, HumanJudgement
19
20
21 def pair_identifier(
22     dataset_id: str,
23     example_id: str,
24     first_variant: str,
25     second_variant: str,
26 ) -> str:
27     raw = f"{dataset_id}|{example_id}|{first_variant}|{second_variant}"
28     return hashlib.sha256(raw.encode("utf-8")).hexdigest()[:20]
29
30
31 def make_blinded_pairs(
32     records: list[GenerationRecord],
33     *,
34     variants: list[str] | None = None,
35     examples_per_dataset: int = 10,
36     seed: int = 42,
37 ) -> list[HumanEvaluationPair]:
38     eligible = [
39         record
40         for record in records
41         if (
42             record.error is None
43             and record.generated_text.strip()
44             and (variants is None or record.variant_id in variants)
45         )
46     ]
47     grouped: dict[tuple[str, str], dict[str, GenerationRecord]] = defaultdict(dict)
48     for record in eligible:
49         if record.repetition == 0:
50             grouped[(record.dataset_id, record.example_id)][record.variant_id] = record
51
52     by_dataset: dict[str, list[tuple[tuple[str, str], dict[str, GenerationRecord]]]] = (
53         defaultdict(list)
54     )
55     for key, value in grouped.items():
56         if len(value) >= 2:
57             by_dataset[key[0]].append((key, value))
58
59     random_generator = random.Random(seed)
60     pairs: list[HumanEvaluationPair] = []
61     for dataset_id, candidates in by_dataset.items():
62         candidates = list(candidates)
63         random_generator.shuffle(candidates)
64         for (_, example_id), variant_records in candidates[:examples_per_dataset]:
65             for first_variant, second_variant in combinations(sorted(variant_records), 2):
66                 first = variant_records[first_variant]
67                 second = variant_records[second_variant]
68                 pair_id = pair_identifier(dataset_id, example_id, first_variant, second_variant)
69                 pair_seed = int(
70                     hashlib.sha256(f"{pair_id}|{seed}".encode("utf-8")).hexdigest()[:8],
71                     16,
72                 )
73                 swap = random.Random(pair_seed).choice([False, True])
74                 if swap:
75                     output_a, output_b = second.generated_text, first.generated_text
76                     hidden_a, hidden_b = second_variant, first_variant
77                 else:
78                     output_a, output_b = first.generated_text, second.generated_text
79                     hidden_a, hidden_b = first_variant, second_variant
80                 pairs.append(
81                     HumanEvaluationPair(
82                         pair_id=pair_id,
83                         dataset_id=dataset_id,

```



```

84         example_id=example_id,
85         source_text=first.source_text,
86         references=first.references,
87         output_a=output_a,
88         output_b=output_b,
89         hidden_variant_a=hidden_a,
90         hidden_variant_b=hidden_b,
91         order_seed=pair_seed,
92     )
93 )
94 random_generator.shuffle(pairs)
95 return pairs
96
97
98 def export_human_packet(
99     pairs: List[HumanEvaluationPair],
100     packet_path: Path,
101     response_template_path: Path,
102 ) -> None:
103     packet_path.parent.mkdir(parents=True, exist_ok=True)
104     write_jsonl(packet_path, pairs)
105     columns = [
106         "pair_id",
107         "evaluator_id",
108         "factual_correctness_a",
109         "factual_correctness_b",
110         "coverage_a",
111         "coverage_b",
112         "coherence_a",
113         "coherence_b",
114         "usefulness_a",
115         "usefulness_b",
116         "preference",
117         "comments",
118     ]
119     response_template_path.parent.mkdir(parents=True, exist_ok=True)
120     with response_template_path.open("w", encoding="utf-8", newline="") as handle:
121         writer = csv.DictWriter(handle, fieldnames=columns)
122         writer.writeheader()
123         for pair in pairs:
124             writer.writerow({"pair_id": pair.pair_id, **{column: "" for column in columns[1:]}})
125
126
127 def export_reviewer_packet(pairs: List[HumanEvaluationPair], path: Path) -> None:
128     path.parent.mkdir(parents=True, exist_ok=True)
129     with path.open("w", encoding="utf-8") as handle:
130         for pair in pairs:
131             payload = {
132                 "pair_id": pair.pair_id,
133                 "dataset_id": pair.dataset_id,
134                 "example_id": pair.example_id,
135                 "source_text": pair.source_text,
136                 "references": pair.references,
137                 "output_a": pair.output_a,
138                 "output_b": pair.output_b,
139                 "rubric": {
140                     "factual_correctness": "1 = materially incorrect; 5 = fully supported.",
141                     "coverage": "1 = omits most important content; 5 = strong content coverage.",
142                     "coherence": "1 = difficult to read; 5 = clear and well organised.",
143                     "usefulness": (
144                         "1 = not useful for the task; 5 = highly useful and appropriately scoped."
145                     ),
146                     "preference": "A, B, or tie",
147                 },
148             }
149             handle.write(json.dumps(payload, ensure_ascii=False) + "\n")
150
151
152 def load_human_judgements(path: Path) -> List[HumanJudgement]:
153     frame = pd.read_csv(path)
154     return [
155         HumanJudgement.model_validate(
156             {key: "" if pd.isna(value) else value for key, value in row.items()}
157         )
158         for row in frame.to_dict(orient="records")
159     ]
160
161
162 def decode_human_scores(
163     pairs: List[HumanEvaluationPair],
164     judgements: List[HumanJudgement],
165 ) -> pd.DataFrame:
166     pair_lookup = {pair.pair_id: pair for pair in pairs}
167     rows: List[dict[str, Any]] = []
168     for judgement in judgements:
169         pair = pair_lookup.get(judgement.pair_id)
170         if pair is None:
171             raise ValueError(f"Unknown pair ID: {judgement.pair_id}")
172         for label, variant, suffix in (
173             ("A", pair.hidden_variant_a, "a"),
174             ("B", pair.hidden_variant_b, "b"),

```



```

175     ):
176         rows.append(
177             {
178                 "pair_id": pair.pair_id,
179                 "dataset_id": pair.dataset_id,
180                 "example_id": pair.example_id,
181                 "evaluator_id": judgement.evaluator_id,
182                 "position": label,
183                 "variant_id": variant,
184                 "factual_correctness": getattr(
185                     judgement,
186                     f"factual_correctness_{suffix}",
187                 ),
188                 "coverage": getattr(judgement, f"coverage_{suffix}"),
189                 "coherence": getattr(judgement, f"coherence_{suffix}"),
190                 "usefulness": getattr(judgement, f"usefulness_{suffix}"),
191                 "preferred": judgement.preference == label,
192                 "tie": judgement.preference == "tie",
193             }
194         )
195     return pd.DataFrame(rows)
196
197
198 def inter_rater_statistics(judgements: list[HumanJudgement]) -> pd.DataFrame:
199     by_pair: dict[str, list[HumanJudgement]] = defaultdict(list)
200     for judgement in judgements:
201         by_pair[judgement.pair_id].append(judgement)
202
203     metrics = [
204         "factual_correctness_a",
205         "factual_correctness_b",
206         "coverage_a",
207         "coverage_b",
208         "coherence_a",
209         "coherence_b",
210         "usefulness_a",
211         "usefulness_b",
212     ]
213     rows: list[dict[str, Any]] = []
214     for metric in metrics:
215         first_values = []
216         second_values = []
217         for pair_judgements in by_pair.values():
218             if len(pair_judgements) < 2:
219                 continue
220             ordered = sorted(pair_judgements, key=lambda item: item.evaluator_id)
221             first_values.append(getattr(ordered[0], metric))
222             second_values.append(getattr(ordered[1], metric))
223         rows.append(
224             {
225                 "measure": metric,
226                 "paired_rating_count": len(first_values),
227                 "quadratic_weighted_kappa": (
228                     cohen_kappa_score(first_values, second_values, weights="quadratic")
229                     if len(first_values) >= 2
230                     else None
231                 ),
232             }
233         )
234
235     first_preferences = []
236     second_preferences = []
237     for pair_judgements in by_pair.values():
238         if len(pair_judgements) < 2:
239             continue
240         ordered = sorted(pair_judgements, key=lambda item: item.evaluator_id)
241         first_preferences.append(ordered[0].preference)
242         second_preferences.append(ordered[1].preference)
243     rows.append(
244         {
245             "measure": "pairwise_preference",
246             "paired_rating_count": len(first_preferences),
247             "quadratic_weighted_kappa": (
248                 cohen_kappa_score(first_preferences, second_preferences)
249                 if len(first_preferences) >= 2
250                 else None
251             ),
252         }
253     )
254     return pd.DataFrame(rows)

```

Listing 36: P36: table2text_pydanticai/src/table2text/evaluation/llm_judge_annotations.py

```

1  """Produce structured, source-only factual error annotations with an LLM judge."""
2
3  from __future__ import annotations
4
5  import os

```

```

6 import time
7 from pathlib import Path
8 from typing import Literal
9
10 from pydantic import Field
11
12 from table2text.config import load_env_files
13
14 from .datasets import write_jsonl
15 from .models import (
16     GenerationRecord,
17     LLMJudgeAnnotationPayload,
18     LLMJudgeAnnotationRecord,
19     MetricStatus,
20     StrictModel,
21 )
22 from .reference_metrics import plain_text
23
24
25 class OpenAIJudgeAnnotationConfig(StrictModel):
26     judge_model: str = "gpt-5.6-sol"
27     judge_repetitions: int = Field(default=1, ge=1)
28     reasoning_effort: Literal[
29         "minimal",
30         "low",
31         "medium",
32         "high",
33         "xhigh",
34         "max",
35     ] = "high"
36     max_source_characters: int = Field(default=50_000, ge=1_000)
37     max_output_tokens: int = Field(default=2_500, ge=200)
38     include_references: bool = False
39     include_system_identity: bool = False
40     include_metric_scores: bool = False
41
42
43 ERROR_TAXONOMY = """\
44 Use exactly one category per error:
45 - NAME: wrong, missing, swapped or misattributed named entity.
46 - NUMBER: wrong, missing, swapped, rounded incorrectly or unsupported number, date, score, rank, unit or quantity.
47 - WORD: incorrect word choice that changes the factual meaning, including wrong predicate, relation, role or action.
48 - CONTEXT: unsupported or incorrect broader interpretation, chronology, comparison, causal implication, trend, scope or
49     situation.
50 - NOT CHECKABLE: claim cannot be verified from the supplied source data alone.
51 - OTHER: factual error that does not fit the other categories.
52 - OMISSION: important source-supported content required by the task is absent from the output.
53 - TASK/FORMAT: output does not follow the requested task, genre, scope, language or format.
54 """
55
56 SYSTEM_INSTRUCTIONS = f"""
57 You are an independent factual-accuracy annotator for structured data-to-text outputs.
58
59 You must inspect one generated output at a time. Use only the supplied source data and task request.
60 Do not use outside knowledge, web search, hidden references, metric scores, or any other system output.
61
62 Return structured JSON matching the requested schema. If the output has no errors, return an empty errors list.
63
64 Do not reward or punish style unless it creates a TASK/FORMAT error. Do not require wording to match a human
65 reference. Do not report harmless paraphrases as errors. Do not infer facts that are not in the source.
66
67 For OMISSION, only report missing information when it is clearly required by the task or when omitting it changes
68 the adequacy of the output. Do not report every unused source field as an omission.
69
70 {ERROR_TAXONOMY}
71 """
72
73
74 def _first_env(*names: str) -> str | None:
75     for name in names:
76         value = os.getenv(name)
77         if value is not None and value.strip():
78             return value.strip()
79     return None
80
81
82 def _normalise_openai_model_name(model_name: str) -> str:
83     if model_name.startswith("openai:"):
84         return model_name.split(":", 1)[1]
85     return model_name
86
87
88 def source_for_annotation_judge(
89     record: GenerationRecord,
90     config: OpenAIJudgeAnnotationConfig,
91 ) -> str:
92     source = record.source_text
93     if len(source) <= config.max_source_characters:
94         return source
95     return source[: config.max_source_characters] + "\n\n[Source truncated by LLM judge configuration.]"

```

```

96
97
98 def build_annotation_judge_input(
99     record: GenerationRecord,
100     config: OpenAIJudgeAnnotationConfig,
101 ) -> str:
102     sections = [
103         "TASK",
104         f"Dataset ID: {record.dataset_id}",
105         f"Example ID: {record.example_id}",
106         f"Task family: {getattr(record.task_family, 'value', record.task_family)}",
107         f"Output mode: {getattr(record.output_mode, 'value', record.output_mode)}",
108         f"Language: {record.language}",
109         f"Request: {record.request}",
110         "",
111         "SOURCE DATA",
112         source_for_annotation_judge(record, config),
113         "",
114         "GENERATED OUTPUT",
115         plain_text(record.generated_text),
116         "",
117         "ERROR TAXONOMY",
118         ERROR_TAXONOMY,
119     ]
120     if config.include_references:
121         sections.extend(
122             [
123                 "",
124                 "HUMAN REFERENCES",
125                 f"\n\n--- ALTERNATIVE REFERENCE ---\n\n".join(record.references),
126             ]
127         )
128     if config.include_system_identity:
129         sections.extend(
130             [
131                 "",
132                 "SYSTEM IDENTITY",
133                 f"Variant ID: {record.variant_id}",
134                 f"Writer mode: {record.writer_mode}",
135                 f"Release status: {record.release_status}",
136             ]
137         )
138     if config.include_metric_scores:
139         sections.extend(
140             [
141                 "",
142                 "SYSTEM METADATA",
143                 f"Audit support rate: {record.audit_support_rate}",
144                 f"Repair rounds used: {record.repair_rounds_used}",
145             ]
146         )
147     return "\n".join(sections)
148
149
150 def _usage_value(usage: object, name: str) -> int | None:
151     value = getattr(usage, name, None)
152     return int(value) if value is not None else None
153
154
155 def _parsed_payload(response: object) -> LLMJudgeAnnotationPayload:
156     parsed = getattr(response, "output_parsed", None)
157     if parsed is not None:
158         return LLMJudgeAnnotationPayload.model_validate(parsed)
159
160     for output in getattr(response, "output", []) or []:
161         if getattr(output, "type", None) != "message":
162             continue
163         for item in getattr(output, "content", []) or []:
164             parsed = getattr(item, "parsed", None)
165             if parsed is not None:
166                 return LLMJudgeAnnotationPayload.model_validate(parsed)
167     raise RuntimeError("OpenAI response did not contain parsed structured output.")
168
169
170 def _make_record(
171     record: GenerationRecord,
172     config: OpenAIJudgeAnnotationConfig,
173     *,
174     judge_repetition: int,
175     status: MetricStatus,
176     payload: LLMJudgeAnnotationPayload | None = None,
177     duration: float | None = None,
178     input_tokens: int | None = None,
179     output_tokens: int | None = None,
180     total_tokens: int | None = None,
181     error: str | None = None,
182 ) -> LLMJudgeAnnotationRecord:
183     errors = payload.errors if payload is not None else []
184     return LLMJudgeAnnotationRecord(
185         generation_id=record.generation_id,
186         dataset_id=record.dataset_id,

```

```

187     example_id=record.example_id,
188     variant_id=record.variant_id,
189     repetition=record.repetition,
190     judge_model=config.judge_model,
191     judge_repetition=judge_repetition,
192     status=status,
193     errors=errors,
194     error_count=len(errors),
195     duration_seconds=duration,
196     input_tokens=input_tokens,
197     output_tokens=output_tokens,
198     total_tokens=total_tokens,
199     error=error,
200 )
201
202
203 def annotate_record_with_openai_judge(
204     record: GenerationRecord,
205     config: OpenAIJudgeAnnotationConfig,
206     *,
207     judge_repetition: int,
208 ) -> LLMJudgeAnnotationRecord:
209     started = time.perf_counter()
210     try:
211         try:
212             from openai import OpenAI
213         except ImportError as exc:
214             raise RuntimeError(
215                 "The openai package is required to run OpenAI LLM judge annotations."
216             ) from exc
217
218         api_key = _first_env("OPENAI_API_KEY", "T2T_OPENAI_API_KEY")
219         if api_key is None:
220             raise RuntimeError(
221                 "OPENAI_API_KEY is not set in the environment or project .env file."
222             )
223
224         client = OpenAI(
225             api_key=api_key,
226             base_url=_first_env("OPENAI_BASE_URL", "T2T_OPENAI_BASE_URL"),
227         )
228         response = client.responses.parse(
229             model=_normalise_openai_model_name(config.judge_model),
230             instructions=SYSTEM_INSTRUCTIONS,
231             input=build_annotation_judge_input(record, config),
232             text_format=LLMJudgeAnnotationPayload,
233             reasoning={"effort": config.reasoning_effort},
234             max_output_tokens=config.max_output_tokens,
235             store=False,
236         )
237         payload = _parsed_payload(response)
238         usage = getattr(response, "usage", None)
239         return _make_record(
240             record,
241             config,
242             judge_repetition=judge_repetition,
243             status=MetricStatus.SCORED,
244             payload=payload,
245             duration=time.perf_counter() - started,
246             input_tokens=_usage_value(usage, "input_tokens") if usage else None,
247             output_tokens=_usage_value(usage, "output_tokens") if usage else None,
248             total_tokens=_usage_value(usage, "total_tokens") if usage else None,
249         )
250     except Exception as exc:
251         return _make_record(
252             record,
253             config,
254             judge_repetition=judge_repetition,
255             status=MetricStatus.ERROR,
256             duration=time.perf_counter() - started,
257             error=f"{type(exc).__name__}: {exc}",
258         )
259
260
261 def annotate_generations_with_openai_judge(
262     records: list[GenerationRecord],
263     config: OpenAIJudgeAnnotationConfig,
264     output_path: Path,
265     *,
266     resume: bool = True,
267 ) -> list[LLMJudgeAnnotationRecord]:
268     load_env_files()
269     existing: dict[tuple[str, str, int], LLMJudgeAnnotationRecord] = {}
270     if resume and output_path.exists():
271         for line in output_path.read_text(encoding="utf-8").splitlines():
272             if line.strip():
273                 item = LLMJudgeAnnotationRecord.model_validate_json(line)
274                 existing[(item.generation_id, item.judge_model, item.judge_repetition)] = item
275
276     annotations = dict(existing)
277     for record in records:

```

```

278     if record.error is not None or not record.generated_text.strip():
279         continue
280     if record.primary_evaluation_eligible is False:
281         continue
282     for judge_repetition in range(config.judge_repetitions):
283         key = (record.generation_id, config.judge_model, judge_repetition)
284         if resume and key in annotations:
285             continue
286         annotations[key] = annotate_record_with_openai_judge(
287             record,
288             config,
289             judge_repetition=judge_repetition,
290         )
291     write_jsonl(output_path, annotations.values())
292
293     ordered = sorted(
294         annotations.values(),
295         key=lambda item: (
296             item.dataset_id,
297             item.example_id,
298             item.variant_id,
299             item.repetition,
300             item.judge_model,
301             item.judge_repetition,
302         ),
303     )
304     write_jsonl(output_path, ordered)
305     return ordered

```

Listing 37: P37: table2text_pydanticai/src/table2text/evaluation/models.py

```

1  """Typed configuration and observation models for evaluation experiments."""
2
3  from __future__ import annotations
4
5  from enum import Enum
6  from pathlib import Path
7  from typing import Any, Literal
8
9  from pydantic import BaseModel, ConfigDict, Field, model_validator
10
11
12  class StrictModel(BaseModel):
13      model_config = ConfigDict(extra="forbid", validate_assignment=True)
14
15
16  class TaskFamily(str, Enum):
17      EVENT_REPORT = "event_report"
18      LONG_FORM_TABLE_REPORT = "long_form_table_report"
19      HIGHLIGHTED_TABLE_DESCRIPTION = "highlighted_table_description"
20      LOGICAL_TABLE_STATEMENT = "logical_table_statement"
21      TABLE_QUESTION_ANSWERING = "table_question_answering"
22      ATTRIBUTE_VERBALISATION = "attribute_verbalisation"
23      TRIPLE_VERBALISATION = "triple_verbalisation"
24      BIOGRAPHY = "biography"
25      WEATHER_RESPONSE = "weather_response"
26      CROSS_LINGUAL_EVENT_REPORT = "cross_lingual_event_report"
27      ANALYTICAL_EXPLANATION = "analytical_explanation"
28
29
30  class OutputMode(str, Enum):
31      ONE_SENTENCE = "one_sentence"
32      SHORT_TEXT = "short_text"
33      PARAGRAPH = "paragraph"
34      MULTI_PARAGRAPH_REPORT = "multi_paragraph_report"
35      DIRECT_ANSWER = "direct_answer"
36
37
38  class DatasetSource(str, Enum):
39      HUGGINGFACE = "huggingface"
40      LOCAL = "local"
41
42
43  class GenerationBackend(str, Enum):
44      TABLE2TEXT = "table2text"
45      PRECOMPUTED = "precomputed"
46      CALLABLE = "callable"
47      COMMAND = "command"
48
49
50  class MetricStatus(str, Enum):
51      SCORED = "scored"
52      SKIPPED = "skipped"
53      UNAVAILABLE = "unavailable"
54      ERROR = "error"
55
56
57  class DatasetConfig(StrictModel):
58      dataset_id: str

```

```

59     enabled: bool = True
60     source: DatasetSource = DatasetSource.HUGGINGFACE
61     hub_id: str | None = None
62     config_name: str | None = None
63     revision: str | None = None
64     split: str = "test"
65     trust_remote_code: bool = False
66     local_path: Path | None = None
67     normalizer: str = "generic"
68     task_family: TaskFamily
69     output_mode: OutputMode
70     language: str = "en"
71     sample_size: int | None = 30
72     seed: int = 42
73     source_fields: list[str] = Field(default_factory=list)
74     reference_fields: list[str] = Field(
75         default_factory=lambda: ["references", "target", "reference", "ref"]
76     )
77     id_fields: list[str] = Field(
78         default_factory=lambda: ["gem_parent_id", "gem_id", "example_id", "id"]
79     )
80     group_fields: list[str] = Field(default_factory=list)
81     metadata_fields: list[str] = Field(default_factory=list)
82     options: dict[str, Any] = Field(default_factory=dict)
83
84     @model_validator(mode="after")
85     def validate_source(self) -> "DatasetConfig":
86         if self.source == DatasetSource.HUGGINGFACE and not self.hub_id:
87             raise ValueError(
88                 f"{self.dataset_id}: hub_id is required for a Hugging Face source."
89             )
90         if self.source == DatasetSource.LOCAL and self.local_path is None:
91             raise ValueError(f"{self.dataset_id}: local_path is required for a local source.")
92         if self.sample_size is not None and self.sample_size <= 0:
93             raise ValueError("sample_size must be positive or null.")
94         return self
95
96 class BenchmarkExample(StrictModel):
97     dataset_id: str
98     example_id: str
99     task_family: TaskFamily
100     output_mode: OutputMode
101     language: str
102     source_payload: Any
103     source_text: str
104     references: list[str]
105     request: str
106     parent_table: list[list[str]] | None = None
107     metadata: dict[str, Any] = Field(default_factory=dict)
108     source_sha256: str
109     reference_sha256: str
110
111     @model_validator(mode="after")
112     def validate_references(self) -> "BenchmarkExample":
113         cleaned = [
114             reference.strip()
115             for reference in self.references
116             if isinstance(reference, str) and reference.strip()
117         ]
118         object.__setattr__(self, "references", list(dict.fromkeys(cleaned)))
119         if not self.references:
120             raise ValueError(f"{self.dataset_id}/{self.example_id} has no usable reference output.")
121         return self
122
123
124 class DatasetPreparationStatus(StrictModel):
125     dataset_id: str
126     status: MetricStatus
127     requested_split: str
128     example_count: int = 0
129     output_path: Path | None = None
130     warnings: list[str] = Field(default_factory=list)
131     error: str | None = None
132
133
134 class VariantConfig(StrictModel):
135     variant_id: str
136     enabled: bool = True
137     backend: GenerationBackend = GenerationBackend.TABLE2TEXT
138     description: str = ""
139     settings_overrides: dict[str, Any] = Field(default_factory=dict)
140     task_contract_mode: Literal["explicit", "inferred"] = "explicit"
141     request_override: str | None = None
142     callable_path: str | None = None
143     command: list[str] = Field(default_factory=list)
144     precomputed_path: Path | None = None
145     repetitions: int = Field(default=1, ge=1)
146     seeds: list[int] = Field(default_factory=list)
147
148     @model_validator(mode="after")
149

```

```

150 def validate_backend(self) -> "VariantConfig":
151     if self.backend == GenerationBackend.CALLABLE and not self.callable_path:
152         raise ValueError(f"{self.variant_id}: callable_path is required.")
153     if self.backend == GenerationBackend.COMMAND and not self.command:
154         raise ValueError(f"{self.variant_id}: command is required.")
155     if self.backend == GenerationBackend.PRECOMPUTED and self.precomputed_path is None:
156         raise ValueError(f"{self.variant_id}: precomputed_path is required.")
157     return self
158
159 class GenerationRecord(StrictModel):
160     generation_id: str
161     dataset_id: str
162     example_id: str
163     variant_id: str
164     repetition: int
165     seed: int
166     task_family: TaskFamily
167     output_mode: OutputMode
168     language: str
169     source_text: str
170     references: list[str]
171     parent_table: list[list[str]] | None = None
172     request: str
173     generated_text: str
174     backend: GenerationBackend
175     run_id: str | None = None
176     pipeline_result_path: Path | None = None
177     writer_mode: str | None = None
178     release_status: str | None = None
179     approved_for_release: bool | None = None
180     primary_evaluation_eligible: bool | None = None
181     primary_evaluation_reason: str | None = None
182     repair_rounds_used: int | None = None
183     audit_support_rate: float | None = None
184     support_sentence_count: int | None = None
185     mapped_support_sentence_count: int | None = None
186     fact_id_reference_count: int | None = None
187     evidence_id_reference_count: int | None = None
188     invalid_fact_id_count: int | None = None
189     invalid_evidence_id_count: int | None = None
190     elapsed_seconds: float | None = None
191     input_tokens: int | None = None
192     output_tokens: int | None = None
193     total_tokens: int | None = None
194     estimated_cost_gbp: float | None = None
195     metadata: dict[str, Any] = Field(default_factory=dict)
196     error: str | None = None
197
198
199 class MetricObservation(StrictModel):
200     generation_id: str
201     dataset_id: str
202     example_id: str
203     variant_id: str
204     repetition: int
205     metric_family: str
206     metric_name: str
207     status: MetricStatus
208     score: float | None = None
209     higher_is_better: bool = True
210     duration_seconds: float | None = None
211     details: dict[str, Any] = Field(default_factory=dict)
212     error: str | None = None
213
214
215 class DeepEvalObservation(StrictModel):
216     generation_id: str
217     dataset_id: str
218     example_id: str
219     variant_id: str
220     repetition: int
221     metric_name: str
222     judge_model: str
223     judge_repetition: int
224     status: MetricStatus
225     score: float | None = None
226     reason: str | None = None
227     threshold: float | None = None
228     success: bool | None = None
229     duration_seconds: float | None = None
230     estimated_cost_gbp: float | None = None
231     error: str | None = None
232
233
234 class LLMJudgeErrorAnnotation(StrictModel):
235     error_span: str
236     category: Literal[
237         "NAME",
238         "NUMBER",
239         "WORD",
240

```



```

241     "CONTEXT",
242     "NOT_CHECKABLE",
243     "OTHER",
244     "OMISSION",
245     "TASK/FORMAT",
246 ]
247 correction_or_explanation: str
248
249
250 class LLMJudgeAnnotationPayload(StrictModel):
251     errors: list[LLMJudgeErrorAnnotation]
252
253
254 class LLMJudgeAnnotationRecord(StrictModel):
255     generation_id: str
256     dataset_id: str
257     example_id: str
258     variant_id: str
259     repetition: int
260     judge_provider: Literal["openai"] = "openai"
261     judge_model: str
262     judge_repetition: int
263     status: MetricStatus
264     errors: list[LLMJudgeErrorAnnotation] = Field(default_factory=list)
265     error_count: int = 0
266     duration_seconds: float | None = None
267     input_tokens: int | None = None
268     output_tokens: int | None = None
269     total_tokens: int | None = None
270     error: str | None = None
271
272
273 class HumanEvaluationPair(StrictModel):
274     pair_id: str
275     dataset_id: str
276     example_id: str
277     source_text: str
278     references: list[str]
279     output_a: str
280     output_b: str
281     hidden_variant_a: str
282     hidden_variant_b: str
283     order_seed: int
284
285
286 class HumanJudgement(StrictModel):
287     pair_id: str
288     evaluator_id: str
289     factual_correctness_a: int = Field(ge=1, le=5)
290     factual_correctness_b: int = Field(ge=1, le=5)
291     coverage_a: int = Field(ge=1, le=5)
292     coverage_b: int = Field(ge=1, le=5)
293     coherence_a: int = Field(ge=1, le=5)
294     coherence_b: int = Field(ge=1, le=5)
295     usefulness_a: int = Field(ge=1, le=5)
296     usefulness_b: int = Field(ge=1, le=5)
297     preference: Literal["A", "B", "tie"]
298     comments: str = ""
299
300
301 class ReferenceMetricConfig(StrictModel):
302     enabled_metrics: list[str] = Field(
303         default_factory=lambda: [
304             "bleu",
305             "chrF",
306             "ter",
307             "rouge1",
308             "rouge2",
309             "rougeL",
310             "rougeLsum",
311             "meteor",
312             "bertscore",
313             "parent",
314             "hhem",
315             "alignscore",
316         ]
317     )
318     bertscore_model: str | None = "roberta-base"
319     bertscore_num_layers: int | None = Field(default=None, ge=1)
320     bertscore_batch_size: int = 8
321     bertscore_device: str | None = None
322     bertscore_rescale_with_baseline: bool = False
323     parent_n_jobs: int = 1
324     hf_local_files_only: bool = True
325     external_factuality_context: Literal["references", "source_text"] = "references"
326     external_context_max_characters: int = Field(default=50_000, ge=1_000)
327     hhem_model: str = "vectara/hallucination_evaluation_model"
328     hhem_foundation_model: str = "google/flan-t5-base"
329     hhem_threshold: float = Field(default=0.5, ge=0.0, le=1.0)
330     hhem_batch_size: int = Field(default=16, ge=1)
331     hhem_context_max_characters: int = Field(default=1_000, ge=250)

```



```

332     hmem_device: str | None = None
333     alignscore_python_executable: Path | None = None
334     alignscore_worker_path: Path | None = None
335     alignscore_model_size: Literal["base", "large"] = "base"
336     alignscore_device: str = "cpu"
337     alignscore_batch_size: int = Field(default=8, ge=1)
338     alignscore_threshold: float = Field(default=0.5, ge=0.0, le=1.0)
339     lowercase: bool = False
340
341
342     class DeepEvalConfig(StrictModel):
343         enabled: bool = True
344         judge_provider: Literal["default", "deepseek", "openai"] = "default"
345         judge_model: str = "gpt-4.1-mini"
346         judge_repetitions: int = Field(default=3, ge=1)
347         threshold: float = Field(default=0.5, ge=0.0, le=1.0)
348         max_source_characters: int = 50_000
349         run_summarization: bool = True
350         run_faithfulness: bool = True
351         run_factual_correctness: bool = True
352         run_reference_adequacy: bool = True
353         run_task_relevance: bool = True
354         run_coherence: bool = True
355         run_usefulness: bool = True
356
357
358     class ExperimentConfig(StrictModel):
359         experiment_id: str
360         prepared_examples_path: Path
361         generations_path: Path
362         result_directory: Path = Path("evaluation/results")
363         reference_metrics: ReferenceMetricConfig = Field(default_factory=ReferenceMetricConfig)
364         deepeval: DeepEvalConfig = Field(default_factory=DeepEvalConfig)
365         baseline_variant: str = "single_agent_baseline"
366         bootstrap_resamples: int = Field(default=5_000, ge=100)
367         confidence_level: float = Field(default=0.95, gt=0.0, lt=1.0)
368         random_seed: int = 42

```

Listing 38: P38: table2text_pydanticai/src/table2text/evaluation/notebook.py

```

1  """Notebook-friendly wrappers around the evaluation framework."""
2
3  from __future__ import annotations
4
5  import json
6  import os
7  from pathlib import Path
8  from typing import Any
9
10 import pandas as pd
11
12 from .cli import write_default_metrics, write_default_variants
13 from table2text.config import load_env_files
14 from .datasets import (
15     load_dataset_configs,
16     prepare_datasets,
17     read_examples,
18     save_default_dataset_configs,
19 )
20 from .diagnostics import write_diagnostics
21 from .deepeval_metrics import evaluate_deepeval
22 from .generation import generate_all_async, load_variants, read_generations
23 from .llm_judge_annotations import (
24     OpenAIIJudgeAnnotationConfig,
25     annotate_generations_with_openai_judge,
26 )
27 from .models import ExperimentConfig
28 from .reference_metrics import evaluate_reference_metrics
29 from .statistics import write_analysis
30
31
32 def default_paths(project_dir: Path) -> dict[str, Path]:
33     project_dir = Path(project_dir)
34     return {
35         "dataset_config": project_dir / "evaluation/config/datasets.json",
36         "variant_config": project_dir / "evaluation/config/variants.json",
37         "metric_config": project_dir / "evaluation/config/metrics.json",
38         "prepared_examples": project_dir / "evaluation/prepared/all_examples.jsonl",
39         "generations": project_dir / "evaluation/generations/generations.jsonl",
40         "run_root": project_dir / "evaluation/generations/runs",
41         "reference_metrics": project_dir / "evaluation/results/reference_metrics.jsonl",
42         "deepeval_metrics": project_dir / "evaluation/results/deepeval_metrics.jsonl",
43         "llm_judge_annotations": project_dir
44         / "evaluation/results/llm_judge_annotations.jsonl",
45         "diagnostics": project_dir / "evaluation/results/diagnostics.csv",
46         "analysis": project_dir / "evaluation/results/analysis",
47     }
48
49

```

```

50 def load_project_env(project_dir: Path) -> None:
51     project_dir = Path(project_dir)
52     load_env_files(
53         [
54             project_dir.parent / ".env",
55             project_dir / ".env",
56             project_dir.parent / ".env.local",
57             project_dir / ".env.local",
58         ]
59     )
60
61
62 def init_notebook_evaluation(project_dir: Path) -> dict[str, Path]:
63     paths = default_paths(project_dir)
64     save_default_dataset_configs(paths["dataset_config"])
65     write_default_variants(paths["variant_config"])
66     write_default_metrics(paths["metric_config"])
67     for key in ("prepared_examples", "generations", "reference_metrics", "diagnostics"):
68         paths[key].parent.mkdir(parents=True, exist_ok=True)
69     paths["analysis"].mkdir(parents=True, exist_ok=True)
70     return paths
71
72
73 def prepare_examples_for_notebook(
74     project_dir: Path,
75     *,
76     dataset_config_path: Path | None = None,
77     output_directory: Path | None = None,
78     skip_unavailable: bool = True,
79 ) -> pd.DataFrame:
80     paths = default_paths(project_dir)
81     statuses = prepare_datasets(
82         load_dataset_configs(dataset_config_path or paths["dataset_config"]),
83         output_directory or paths["prepared_examples"].parent,
84         skip_unavailable=skip_unavailable,
85     )
86     return pd.DataFrame([status.model_dump(mode="json") for status in statuses])
87
88
89 async def generate_reports_for_notebook(
90     project_dir: Path,
91     *,
92     examples_path: Path | None = None,
93     variants_path: Path | None = None,
94     output_path: Path | None = None,
95     run_root: Path | None = None,
96     resume: bool = True,
97 ) -> pd.DataFrame:
98     paths = default_paths(project_dir)
99     records = await generate_all_async(
100         read_examples(examples_path or paths["prepared_examples"]),
101         load_variants(variants_path or paths["variant_config"]),
102         output_path or paths["generations"],
103         run_root or paths["run_root"],
104         resume=resume,
105     )
106     return pd.DataFrame([record.model_dump(mode="json") for record in records])
107
108
109 def score_reference_metrics_for_notebook(
110     project_dir: Path,
111     *,
112     generations_path: Path | None = None,
113     metric_config_path: Path | None = None,
114     output_path: Path | None = None,
115     include_ineligible: bool = False,
116 ) -> pd.DataFrame:
117     paths = default_paths(project_dir)
118     experiment = ExperimentConfig.model_validate(
119         json.loads((metric_config_path or paths["metric_config"]).read_text(encoding="utf-8"))
120     )
121     observations = evaluate_reference_metrics(
122         read_generations(generations_path or paths["generations"]),
123         experiment.reference_metrics,
124         output_path or paths["reference_metrics"],
125         include_ineligible=include_ineligible,
126     )
127     return pd.DataFrame([item.model_dump(mode="json") for item in observations])
128
129
130 def score_deepeval_for_notebook(
131     project_dir: Path,
132     *,
133     generations_path: Path | None = None,
134     metric_config_path: Path | None = None,
135     output_path: Path | None = None,
136     resume: bool = True,
137 ) -> pd.DataFrame:
138     load_project_env(project_dir)
139     paths = default_paths(project_dir)
140     experiment = ExperimentConfig.model_validate(

```

```

141     json.loads((metric_config_path or paths["metric_config"]).read_text(encoding="utf-8"))
142 )
143 observations = evaluate_deepeval(
144     read_generations(generations_path or paths["generations"]),
145     experiment.deepeval,
146     output_path or paths["deepeval_metrics"],
147     resume=resume,
148 )
149 return pd.DataFrame([item.model_dump(mode="json") for item in observations])
150
151
152 def score_openai_judge_for_notebook(
153     project_dir: Path,
154     *,
155     generations_path: Path | None = None,
156     metric_config_path: Path | None = None,
157     output_path: Path | None = None,
158     judge_model: str = "gpt-5.6-sol",
159     judge_repetitions: int = 1,
160     max_source_characters: int | None = None,
161     run_summarization: bool | None = None,
162     run_faithfulness: bool | None = None,
163     run_factual_correctness: bool | None = None,
164     run_reference_adequacy: bool | None = None,
165     run_task_relevance: bool | None = None,
166     run_coherence: bool | None = None,
167     run_usefulness: bool | None = None,
168     resume: bool = True,
169 ) -> pd.DataFrame:
170     """Run the configured DeepEval judge metrics with an OpenAI judge model.
171
172     This is a notebook convenience wrapper around the same evaluator used by
173     ``score_deepeval_for_notebook``. It keeps the metric toggles from the
174     selected metrics config unless an override is supplied here.
175     """
176
177     load_project_env(project_dir)
178     paths = default_paths(project_dir)
179     experiment = ExperimentConfig.model_validate(
180         json.loads((metric_config_path or paths["metric_config"]).read_text(encoding="utf-8"))
181     )
182     updates: dict[str, Any] = {
183         "enabled": True,
184         "judge_provider": "openai",
185         "judge_model": judge_model,
186         "judge_repetitions": judge_repetitions,
187     }
188     optional_updates = {
189         "max_source_characters": max_source_characters,
190         "run_summarization": run_summarization,
191         "run_faithfulness": run_faithfulness,
192         "run_factual_correctness": run_factual_correctness,
193         "run_reference_adequacy": run_reference_adequacy,
194         "run_task_relevance": run_task_relevance,
195         "run_coherence": run_coherence,
196         "run_usefulness": run_usefulness,
197     }
198     updates |= {
199         key: value
200         for key, value in optional_updates.items()
201         if value is not None
202     }
203     config = experiment.deepeval.model_copy(update=updates)
204     observations = evaluate_deepeval(
205         read_generations(generations_path or paths["generations"]),
206         config,
207         output_path
208         or (paths["deepeval_metrics"].parent / "openai_judge_metrics.jsonl"),
209         resume=resume,
210     )
211     return pd.DataFrame([item.model_dump(mode="json") for item in observations])
212
213
214 def annotate_with_openai_judge_for_notebook(
215     project_dir: Path,
216     *,
217     generations_path: Path | None = None,
218     output_path: Path | None = None,
219     judge_model: str | None = None,
220     judge_repetitions: int = 1,
221     reasoning_effort: str | None = None,
222     max_source_characters: int = 50_000,
223     max_output_tokens: int = 2_500,
224     include_references: bool = False,
225     include_system_identity: bool = False,
226     include_metric_scores: bool = False,
227     resume: bool = True,
228 ) -> pd.DataFrame:
229     """Run the OpenAI LLM judge as a structured error annotator.
230
231     Unlike the scalar DeepEval wrapper, this sends each generated output

```

```

232 independently and returns span-level error annotations using the configured
233 taxonomy. Human references and metric scores are excluded by default.
234 """
235
236 load_project_env(project_dir)
237 paths = default_paths(project_dir)
238 config = OpenAIIJudgeAnnotationConfig(
239     judge_model=(
240         judge_model
241         or os.getenv("T2T_OPENAI_JUDGE_ANNOTATION_MODEL")
242         or os.getenv("T2T_DEEPEVAL_JUDGE_MODEL")
243         or "gpt-5.6-sol"
244     ),
245     judge_repetitions=judge_repetitions,
246     reasoning_effort=(
247         reasoning_effort
248         or os.getenv("T2T_OPENAI_JUDGE_REASONING_EFFORT")
249         or "high"
250     ),
251     max_source_characters=max_source_characters,
252     max_output_tokens=max_output_tokens,
253     include_references=include_references,
254     include_system_identity=include_system_identity,
255     include_metric_scores=include_metric_scores,
256 )
257 annotations = annotate_generations_with_openai_judge(
258     read_generations(generations_path or paths["generations"]),
259     config,
260     output_path or paths["llm_judge_annotations"],
261     resume=resume,
262 )
263 return pd.DataFrame([item.model_dump(mode="json") for item in annotations])
264
265
266 def diagnostics_for_notebook(
267     project_dir: Path,
268     *,
269     generations_path: Path | None = None,
270     output_path: Path | None = None,
271 ) -> pd.DataFrame:
272     paths = default_paths(project_dir)
273     return write_diagnostics(
274         generations_path or paths["generations"],
275         output_path or paths["diagnostics"],
276     )
277
278
279 def aggregate_for_notebook(
280     project_dir: Path,
281     *,
282     metric_config_path: Path | None = None,
283     reference_metrics_path: Path | None = None,
284     deepeval_metrics_path: Path | None = None,
285     diagnostics_path: Path | None = None,
286     output_directory: Path | None = None,
287 ) -> dict[str, Path]:
288     paths = default_paths(project_dir)
289     experiment = ExperimentConfig.model_validate(
290         json.loads((metric_config_path or paths["metric_config"]).read_text(encoding="utf-8"))
291     )
292     write_analysis(
293         reference_observations_path=reference_metrics_path or paths["reference_metrics"],
294         deepeval_observations_path=deepeval_metrics_path or paths["deepeval_metrics"],
295         diagnostics_path=diagnostics_path or paths["diagnostics"],
296         output_directory=output_directory or paths["analysis"],
297         baseline_variant=experiment.baseline_variant,
298         bootstrap_resamples=experiment.bootstrap_resamples,
299         confidence_level=experiment.confidence_level,
300         seed=experiment.random_seed,
301     )
302     analysis_dir = output_directory or paths["analysis"]
303     return {
304         "all_metric_scores": analysis_dir / "all_metric_scores.csv",
305         "dataset_variant_summary": analysis_dir / "dataset_variant_summary.csv",
306         "macro_summary": analysis_dir / "macro_summary.csv",
307         "paired_bootstrap_comparisons": analysis_dir / "paired_bootstrap_comparisons.csv",
308         "metric_correlation_matrix": analysis_dir / "metric_correlation_matrix.csv",
309         "metric_correlations_long": analysis_dir / "metric_correlations_long.csv",
310         "cost_summary": analysis_dir / "cost_summary.csv",
311     }
312
313
314 def read_jsonl_as_frame(path: Path) -> pd.DataFrame:
315     rows: list[dict[str, Any]] = []
316     for line in Path(path).read_text(encoding="utf-8").splitlines():
317         if line.strip():
318             rows.append(json.loads(line))
319     return pd.DataFrame(rows)

```

Listing 39: P39: table2text_pydanticai/src/table2text/evaluation/reference_metrics.py

```

1  """Compute lexical, semantic, and source-grounded evaluation metrics."""
2
3  from __future__ import annotations
4
5  import json
6  import re
7  import time
8  import os
9  import gc
10 import tempfile
11 from contextlib import contextmanager
12 from collections import defaultdict
13 from pathlib import Path
14 from typing import Any, Callable, Mapping
15
16 import numpy as np
17
18 from .datasets import write_jsonl
19 from .alignscore_client import AlignScoreClient
20 from .external_factuality import ExternalFactualityResult, HHEMEvaluator
21 from .models import (
22     GenerationRecord,
23     MetricObservation,
24     MetricStatus,
25     ReferenceMetricConfig,
26     TaskFamily,
27 )
28
29
30 MARKDOWN_HEADING = re.compile(r"^\s{0,3}#{1,6}\s+", re.MULTILINE)
31 MARKDOWN_LINK = re.compile(r"\[([^\]]+)\]\([^\)]+\)")
32 MARKDOWN_DECORATION = re.compile(r"[*~]")
33 MARKDOWN_UNDERSCORE_EMPHASIS = re.compile(
34     r"(?![A-Za-z0-9])_([^\n]+)(?![A-Za-z0-9])"
35 )
36 WHITESPACE = re.compile(r"\s+")
37 DEFAULT_ALIGNSCORE_WORKER = Path(__file__).resolve().parents[3] / "scripts/alignscore_worker.py"
38
39
40 def default_alignscore_python_executable() -> Path | None:
41     project_root = DEFAULT_ALIGNSCORE_WORKER.parent.parent
42     candidates = [
43         project_root / ".venv-alignscore/bin/python",
44         project_root / ".venv-alignscore/Scripts/python.exe",
45     ]
46     for candidate in candidates:
47         if candidate.exists():
48             return candidate
49     return None
50
51
52 def plain_text(value: str) -> str:
53     value = MARKDOWN_LINK.sub(r"\1", value)
54     value = MARKDOWN_HEADING.sub("", value)
55     value = MARKDOWN_UNDERSCORE_EMPHASIS.sub(r"\1", value)
56     value = MARKDOWN_DECORATION.sub("", value)
57     return WHITESPACE.sub(" ", value).strip()
58
59
60 def valid_generation(
61     record: GenerationRecord,
62     *,
63     include_ineligible: bool = False,
64 ) -> bool:
65     return (
66         record.error is None
67         and bool(record.generated_text.strip())
68         and bool(record.references)
69         and (
70             include_ineligible
71             or record.primary_evaluation_eligible is not False
72         )
73     )
74
75
76 def ineligible_generation_reason(
77     record: GenerationRecord,
78     *,
79     include_ineligible: bool = False,
80 ) -> str | None:
81     if record.error is not None:
82         return f"Generation failed: {record.error}"
83     if not record.generated_text.strip():
84         return "Generated text is empty."
85     if not record.references:
86         return "No reference outputs are available."
87     if record.primary_evaluation_eligible is False and not include_ineligible:
88         return (
89             record.primary_evaluation_reason

```

```

90         or "The generation was marked ineligible for primary evaluation."
91     )
92     return None
93
94
95 def observation(
96     record: GenerationRecord,
97     *,
98     family: str,
99     name: str,
100     status: MetricStatus,
101     score: float | None = None,
102     higher_is_better: bool = True,
103     duration: float | None = None,
104     details: dict[str, Any] | None = None,
105     error: str | None = None,
106 ) -> MetricObservation:
107     return MetricObservation(
108         generation_id=record.generation_id,
109         dataset_id=record.dataset_id,
110         example_id=record.example_id,
111         variant_id=record.variant_id,
112         repetition=record.repetition,
113         metric_family=family,
114         metric_name=name,
115         status=status,
116         score=score,
117         higher_is_better=higher_is_better,
118         duration_seconds=duration,
119         details=details or {},
120         error=error,
121     )
122
123
124 def score_over_references(
125     candidate: str,
126     references: list[str],
127     function: Callable[[str, str], float],
128 ) -> tuple[float, int]:
129     scores = [float(function(candidate, reference)) for reference in references]
130     best_index = int(np.argmax(scores))
131     return scores[best_index], best_index
132
133
134 def _tokens(text: str) -> list[str]:
135     return re.findall(r"\w+", text.casefold())
136
137
138 def _token_f1(candidate: str, reference: str) -> float:
139     candidate_tokens = _tokens(candidate)
140     reference_tokens = _tokens(reference)
141     if not candidate_tokens and not reference_tokens:
142         return 1.0
143     if not candidate_tokens or not reference_tokens:
144         return 0.0
145     overlap = 0
146     reference_counts: dict[str, int] = defaultdict(int)
147     for token in reference_tokens:
148         reference_counts[token] += 1
149     for token in candidate_tokens:
150         if reference_counts[token] > 0:
151             overlap += 1
152             reference_counts[token] -= 1
153     precision = overlap / len(candidate_tokens)
154     recall = overlap / len(reference_tokens)
155     if precision + recall == 0:
156         return 0.0
157     return 2 * precision * recall / (precision + recall)
158
159
160 def _fallback_overlap(candidate: str, references: list[str]) -> tuple[float, int]:
161     return score_over_references(candidate, references, _token_f1)
162
163
164 @contextmanager
165 def huggingface_offline(enabled: bool):
166     keys = ("HF_HUB_OFFLINE", "TRANSFORMERS_OFFLINE", "HF_MODULES_CACHE", "MPLCONFIGDIR")
167     previous = {key: os.environ.get(key) for key in keys}
168     if enabled:
169         os.environ["HF_HUB_OFFLINE"] = "1"
170         os.environ["TRANSFORMERS_OFFLINE"] = "1"
171     if previous["HF_MODULES_CACHE"] is None:
172         modules_cache = Path(tempfile.gettempdir()) / "table2text_hf_modules"
173         modules_cache.mkdir(parents=True, exist_ok=True)
174         os.environ["HF_MODULES_CACHE"] = str(modules_cache)
175     if previous["MPLCONFIGDIR"] is None:
176         matplotlib_cache = Path(tempfile.gettempdir()) / "table2text_matplotlib"
177         matplotlib_cache.mkdir(parents=True, exist_ok=True)
178         os.environ["MPLCONFIGDIR"] = str(matplotlib_cache)
179     try:
180         yield

```

```

181     finally:
182         for key, value in previous.items():
183             if value is None:
184                 os.environ.pop(key, None)
185             else:
186                 os.environ[key] = value
187
188
189 def metric_enabled(config: ReferenceMetricConfig, *names: str) -> bool:
190     enabled = {name.casefold() for name in config.enabled_metrics}
191     return any(name.casefold() in enabled for name in names)
192
193
194 def local_huggingface_snapshot(model_name_or_path: str) -> str:
195     path = Path(model_name_or_path)
196     if path.exists():
197         return str(path)
198
199     from huggingface_hub import snapshot_download
200
201     return snapshot_download(
202         repo_id=model_name_or_path,
203         local_files_only=True,
204     )
205
206
207 def _primitive(value: Any) -> bool:
208     return value is None or isinstance(value, (str, int, float, bool))
209
210
211 def _clean_label(value: str) -> str:
212     value = re.sub(r"[_./]+", " ", value)
213     value = re.sub(r"\s+", " ", value)
214     return value.strip()
215
216
217 def _primitive_items(mapping: Mapping[str, Any]) -> list[tuple[str, Any]]:
218     return [
219         (_clean_label(str(key)), value)
220         for key, value in mapping.items()
221         if _primitive(value)
222         and str(value).strip()
223         and str(value).strip().lower() not in {"none", "null"}
224     ]
225
226
227 def _format_items(
228     items: list[tuple[str, Any]],
229     *,
230     skip: set[str] | None = None,
231     maximum: int = 16,
232 ) -> str:
233     skip = {item.casefold() for item in (skip or set())}
234     parts = [
235         f"{key} {value}"
236         for key, value in items
237         if key.casefold() not in skip
238     ]
239     return ", ".join(parts[:maximum])
240
241
242 def _format_number(value: float) -> str:
243     return str(int(value)) if value.is_integer() else str(value)
244
245
246 def _team_score(payload: Mapping[str, Any]) -> Any | None:
247     line_score = payload.get("line_score")
248     if not isinstance(line_score, Mapping):
249         return None
250     game = line_score.get("game")
251     if not isinstance(game, Mapping):
252         return None
253     return game.get("PTS") or game.get("points") or game.get("score")
254
255
256 def _event_participants(payload: Mapping[str, Any]) -> list[tuple[str, Mapping[str, Any]]]:
257     for key in ("teams", "participants", "sides", "competitors"):
258         value = payload.get(key)
259         if isinstance(value, Mapping):
260             participants = [
261                 (str(role), record)
262                 for role, record in value.items()
263                 if isinstance(record, Mapping)
264             ]
265             if participants:
266                 return participants
267     return []
268
269
270 def _named_records(value: Any) -> list[Mapping[str, Any]]:
271     records: list[Mapping[str, Any]] = []

```



```

272     if isinstance(value, Mapping):
273         if "name" in value and any(
274             isinstance(item, (int, float, str))
275             and str(item).strip()
276             for key, item in value.items()
277             if key != "name"
278         ):
279             records.append(value)
280             for item in value.values():
281                 records.extend(_named_records(item))
282     elif isinstance(value, list):
283         for item in value:
284             records.extend(_named_records(item))
285     return records
286
287
288 def _event_source_context_from_json(payload: Any) -> str | None:
289     if not isinstance(payload, Mapping):
290         return None
291
292     lines: list[str] = []
293     game = payload.get("game")
294     if isinstance(game, Mapping):
295         context = _format_items(
296             _primitive_items(game),
297             maximum=20,
298         )
299         if context:
300             lines.append(f"Event context: {context}.")
301
302     participants = _event_participants(payload)
303     participant_scores: list[tuple[str, str, float]] = []
304     for role, participant in participants:
305         name = (
306             participant.get("place")
307             or participant.get("name")
308             or participant.get("team")
309             or role
310         )
311         score = _team_score(participant)
312         if score is not None:
313             try:
314                 participant_scores.append((role, str(name), float(score)))
315             except (TypeError, ValueError):
316                 pass
317     totals = []
318     line_score = participant.get("line_score")
319     if isinstance(line_score, Mapping):
320         game_totals = line_score.get("game")
321         if isinstance(game_totals, Mapping):
322             totals = _primitive_items(game_totals)
323     if totals:
324         lines.append(
325             f"{role} participant {name} game totals: "
326             f"{_format_items(totals, maximum=18)}."
327         )
328     for segment_name, segment in (
329         line_score.items()
330         if isinstance(line_score, Mapping)
331         else []
332     ):
333         if not isinstance(segment, Mapping) or segment_name == "game":
334             continue
335         points = (
336             segment.get("PTS")
337             or segment.get("points")
338             or segment.get("score")
339         )
340         if points is not None:
341             lines.append(
342                 f"{role} participant {name} {segment_name} points {points}."
343             )
344
345     if len(participant_scores) >= 2:
346         ordered = sorted(
347             participant_scores,
348             key=lambda item: item[2],
349             reverse=True,
350         )
351         winner = ordered[0]
352         loser = ordered[-1]
353         margin = winner[2] - loser[2]
354         if margin.is_integer():
355             margin_text = str(int(margin))
356         else:
357             margin_text = str(margin)
358     lines.insert(
359         0,
360         (
361             f"Event result: {winner[1]} scored {_format_number(winner[2])} "
362             f"and {loser[1]} scored {_format_number(loser[2])}; "

```



```

363         f"the margin was {margin_text}."
364     ),
365 )
366
367 for record in _named_records(payload):
368     name = record.get("name")
369     if not name:
370         continue
371     values = _format_items(
372         _primitive_items(record),
373         skip={"name", "first name", "last name"},
374         maximum=22,
375     )
376     if values:
377         lines.append(f"Record for {name}: {values}.")
378
379 if not lines:
380     return None
381 return "\n".join(dict.fromkeys(lines))
382
383
384 def normalized_event_source_context(record: GenerationRecord) -> str | None:
385     if record.task_family not in {
386         TaskFamily.EVENT_REPORT,
387         TaskFamily.CROSS_LINGUAL_EVENT_REPORT,
388     }:
389         return None
390     try:
391         payload = json.loads(record.source_text)
392     except json.JSONDecodeError:
393         return None
394     return _event_source_context_from_json(payload)
395
396
397 def factuality_context(record: GenerationRecord, config: ReferenceMetricConfig) -> str:
398     if config.external_factuality_context == "references" and record.references:
399         source = "\n".join(
400             reference.strip()
401             for reference in record.references
402             if reference.strip()
403         )
404     else:
405         source = (
406             normalized_event_source_context(record)
407             if config.external_factuality_context == "source_text"
408             else None
409         ) or record.source_text.strip()
410     limit = config.external_context_max_characters
411     if len(source) <= limit:
412         return source
413     return source[:limit] + "\n\n[Source truncated by reference metric configuration.]"
414
415
416 def metric_status(value: str) -> MetricStatus:
417     try:
418         return MetricStatus(value)
419     except ValueError:
420         return MetricStatus.ERROR
421
422
423 def hhem_observations(
424     record: GenerationRecord,
425     result: ExternalFactualityResult,
426     *,
427     duration: float,
428 ) -> list[MetricObservation]:
429     status = metric_status(result.status)
430     details = result.details | {
431         "metric_group": result.metric_name,
432         "threshold": result.threshold,
433     }
434     outputs = [
435         (
436             f"{result.metric_name}_mean_support",
437             result.overall_score,
438             True,
439         ),
440         (
441             f"{result.metric_name}_min_sentence_support",
442             result.minimum_sentence_score,
443             True,
444         ),
445         (
446             f"{result.metric_name}_unsupported_sentence_rate",
447             result.unsupported_sentence_rate,
448             False,
449         ),
450     ]
451     return [
452         observation(
453             record,

```

```

454         family="external_factuality",
455         name=name,
456         status=status,
457         score=score if status == MetricStatus.SCORED else None,
458         higher_is_better=higher_is_better,
459         duration=duration,
460         details=details | {"sentence_scores": result.sentence_scores},
461         error=result.error,
462     )
463     for name, score, higher_is_better in outputs
464 ]
465
466 def alignscore_observation(
467     record: GenerationRecord,
468     result: ExternalFactualityResult,
469     *,
470     duration: float,
471 ) -> MetricObservation:
472     return observation(
473         record,
474         family="external_factuality",
475         name=result.metric_name,
476         status=metric_status(result.status),
477         score=result.overall_score if result.status == "scored" else None,
478         higher_is_better=True,
479         duration=duration,
480         details=result.details | {"threshold": result.threshold},
481         error=result.error,
482     )
483
484
485 def score_lexical_record(
486     record: GenerationRecord,
487     config: ReferenceMetricConfig,
488 ) -> list[MetricObservation]:
489     results: list[MetricObservation] = []
490     candidate = plain_text(record.generated_text)
491     references = [plain_text(reference) for reference in record.references]
492     if config.lowercase:
493         candidate = candidate.casefold()
494         references = [reference.casefold() for reference in references]
495     enabled = set(config.enabled_metrics)
496
497     try:
498         import sacrebleu
499     except ImportError:
500         sacrebleu = None
501
502     for name in ("bleu", "chrF", "ter"):
503         if name not in enabled:
504             continue
505         started = time.perf_counter()
506         if sacrebleu is None:
507             fallback, best_index = _fallback_overlap(candidate, references)
508             score = (1.0 - fallback) if name == "ter" else fallback
509             results.append(
510                 observation(
511                     record,
512                     family="lexical",
513                     name=name,
514                     status=MetricStatus.SCORED,
515                     score=score,
516                     higher_is_better=(name != "ter"),
517                     duration=time.perf_counter() - started,
518                     details={
519                         "fallback": "token_f1",
520                         "selected_reference_index": best_index,
521                     },
522                 )
523             )
524         continue
525     try:
526         if name == "bleu":
527             result = sacrebleu.sentence_bleu(
528                 candidate,
529                 references,
530                 lowercase=config.lowercase,
531                 smooth_method="exp",
532             )
533         elif name == "chrF":
534             result = sacrebleu.sentence_chrf(candidate, references, word_order=2)
535         else:
536             result = sacrebleu.sentence_ter(candidate, references)
537     results.append(
538         observation(
539             record,
540             family="lexical",
541             name=name,
542             status=MetricStatus.SCORED,
543             score=result.score / 100.0,
544

```

```

545         higher_is_better=(name != "ter"),
546         duration=time.perf_counter() - started,
547         details={"reference_count": len(references), "raw_percent": result.score},
548     )
549 )
550 except Exception as exc:
551     results.append(
552         observation(
553             record,
554             family="lexical",
555             name=name,
556             status=MetricStatus.ERROR,
557             higher_is_better=(name != "ter"),
558             duration=time.perf_counter() - started,
559             error=f"{type(exc).__name__}: {exc}",
560         )
561     )
562
563 rouge_names = {"rouge1", "rouge2", "rougeL", "rougeLsum"} & enabled
564 if rouge_names:
565     started = time.perf_counter()
566     try:
567         from rouge_score import rouge_scorer
568
569         scorer = rouge_scorer.RougeScorer(sorted(rouge_names), use_stemmer=True)
570         for name in sorted(rouge_names):
571             score, best_index = score_over_references(
572                 candidate,
573                 references,
574                 lambda prediction, reference: scorer.score(reference, prediction)[name].fmeasure,
575             )
576             results.append(
577                 observation(
578                     record,
579                     family="lexical",
580                     name=name,
581                     status=MetricStatus.SCORED,
582                     score=score,
583                     duration=time.perf_counter() - started,
584                     details={
585                         "reference_count": len(references),
586                         "selected_reference_index": best_index,
587                         "aggregation": "maximum_over_references",
588                     },
589                 )
590             )
591     except ImportError:
592         for name in sorted(rouge_names):
593             score, best_index = _fallback_overlap(candidate, references)
594             results.append(
595                 observation(
596                     record,
597                     family="lexical",
598                     name=name,
599                     status=MetricStatus.SCORED,
600                     score=score,
601                     duration=time.perf_counter() - started,
602                     details={
603                         "fallback": "token_f1",
604                         "selected_reference_index": best_index,
605                     },
606                 )
607             )
608     except Exception as exc:
609         for name in sorted(rouge_names):
610             results.append(
611                 observation(
612                     record,
613                     family="lexical",
614                     name=name,
615                     status=MetricStatus.ERROR,
616                     duration=time.perf_counter() - started,
617                     error=f"{type(exc).__name__}: {exc}",
618                 )
619             )
620
621 if "meteor" in enabled:
622     started = time.perf_counter()
623     try:
624         import nltk
625         from nltk.translate.meteor_score import meteor_score
626
627     try:
628         score, best_index = score_over_references(
629             candidate,
630             references,
631             lambda prediction, reference: meteor_score(
632                 [reference.split()],
633                 prediction.split(),
634             ),
635         )

```

```

636         except LookupError:
637             nltk.download("wordnet", quiet=True)
638             nltk.download("omw-1.4", quiet=True)
639             score, best_index = score_over_references(
640                 candidate,
641                 references,
642                 lambda prediction, reference: meteor_score(
643                     [reference.split()],
644                     [prediction.split()],
645                 ),
646             )
647             results.append(
648                 observation(
649                     record,
650                     family="lexical_semantic",
651                     name="meteor",
652                     status=MetricStatus.SCORED,
653                     score=score,
654                     duration=time.perf_counter() - started,
655                     details={
656                         "reference_count": len(references),
657                         "selected_reference_index": best_index,
658                         "aggregation": "maximum_over_references",
659                     },
660                 )
661             )
662         except ImportError:
663             results.append(
664                 observation(
665                     record,
666                     family="lexical_semantic",
667                     name="meteor",
668                     status=MetricStatus.UNAVAILABLE,
669                     error="nltk is not installed.",
670                 )
671             )
672         except Exception as exc:
673             results.append(
674                 observation(
675                     record,
676                     family="lexical_semantic",
677                     name="meteor",
678                     status=MetricStatus.ERROR,
679                     duration=time.perf_counter() - started,
680                     error=f"{type(exc).__name__}: {exc}",
681                 )
682             )
683
684     return results
685
686
687 def score_bertscore(
688     records: list[GenerationRecord],
689     config: ReferenceMetricConfig,
690 ) -> list[MetricObservation]:
691     if "bertscore" not in config.enabled_metrics:
692         return []
693     try:
694         with huggingface_offline(config.hf_local_files_only):
695             from bert_score import score as bert_score
696     except ImportError:
697         return [
698             observation(
699                 record,
700                 family="semantic",
701                 name="bertscore_f1",
702                 status=MetricStatus.UNAVAILABLE,
703                 error="bert-score is not installed.",
704             )
705             for record in records
706         ]
707
708     observations: list[MetricObservation] = []
709     grouped: dict[str, list[GenerationRecord]] = defaultdict(list)
710     for record in records:
711         grouped[record.language].append(record)
712
713     for language, language_records in grouped.items():
714         candidates: list[str] = []
715         references: list[str] = []
716         ownership: list[tuple[GenerationRecord, int]] = []
717         for record in language_records:
718             candidate = plain_text(record.generated_text)
719             for reference_index, reference in enumerate(record.references):
720                 candidates.append(candidate)
721                 references.append(plain_text(reference))
722                 ownership.append((record, reference_index))
723         if not candidates:
724             continue
725         started = time.perf_counter()
726         try:

```

```

727     kwargs: dict[str, Any] = {
728         "cands": candidates,
729         "refs": references,
730         "batch_size": config.bertscore_batch_size,
731         "verbose": False,
732     }
733     if config.bertscore_model:
734         model_name = config.bertscore_model
735         model_type = config.bertscore_model
736         if config.hf_local_files_only:
737             model_type = local_huggingface_snapshot(model_type)
738         kwargs["model_type"] = model_type
739         if config.bertscore_num_layers is not None:
740             kwargs["num_layers"] = config.bertscore_num_layers
741         elif model_type != model_name:
742             from bert_score.utils import model2layers
743
744             if model_name in model2layers:
745                 kwargs["num_layers"] = model2layers[model_name]
746             kwargs["rescale_with_baseline"] = False
747         else:
748             kwargs["lang"] = language.split("-")[0]
749             kwargs["rescale_with_baseline"] = config.bertscore_rescale_with_baseline
750     if config.bertscore_device:
751         kwargs["device"] = config.bertscore_device
752     with huggingface_offline(config.hf_local_files_only):
753         _, _, f1 = bert_score(**kwargs)
754     values_by_generation: dict[str, list[tuple[int, float]]] = defaultdict(list)
755     for (record, reference_index), value in zip(ownership, f1.tolist(), strict=True):
756         values_by_generation[record.generation_id].append((reference_index, float(value)))
757     total_duration = time.perf_counter() - started
758     duration_each = total_duration / max(len(language_records), 1)
759     for record in language_records:
760         best_index, best_value = max(
761             values_by_generation[record.generation_id],
762             key=lambda item: item[1],
763         )
764         observations.append(
765             observation(
766                 record,
767                 family="semantic",
768                 name="bertscore_f1",
769                 status=MetricStatus.SCORED,
770                 score=best_value,
771                 duration=duration_each,
772                 details={
773                     "language": language,
774                     "model": config.bertscore_model or "bert-score default",
775                     "selected_reference_index": best_index,
776                     "aggregation": "maximum_over_references",
777                 },
778             )
779         )
780     except Exception as exc:
781         duration = time.perf_counter() - started
782         for record in language_records:
783             observations.append(
784                 observation(
785                     record,
786                     family="semantic",
787                     name="bertscore_f1",
788                     status=MetricStatus.ERROR,
789                     duration=duration,
790                     error=f"{type(exc).__name__}: {exc}",
791                 )
792             )
793     return observations
794
795 def score_parent_record(
796     record: GenerationRecord,
797     config: ReferenceMetricConfig,
798 ) -> list[MetricObservation]:
799     if "parent" not in config.enabled_metrics:
800         return []
801     metric_names = ["parent_precision", "parent_recall", "parent_f1"]
802     if record.parent_table is None:
803         return [
804             observation(
805                 record,
806                 family="table_aware",
807                 name=name,
808                 status=MetricStatus.SKIPPED,
809                 details={
810                     "reason": "The dataset adapter did not expose a PARENT-compatible table."
811                 },
812             )
813         ]
814     for name in metric_names:
815         try:
816             from parent import parent

```

```

818 except ImportError:
819     return [
820         observation(
821             record,
822             family="table_aware",
823             name=name,
824             status=MetricStatus.UNAVAILABLE,
825             error="Install the optional KaijuML PARENT package.",
826         )
827         for name in metric_names
828     ]
829 candidate = plain_text(record.generated_text).split()
830 references = [plain_text(reference).split() for reference in record.references]
831 table = [[str(cell) for cell in table_row] for table_row in record.parent_table]
832 started = time.perf_counter()
833 try:
834     precision, recall, f1 = parent(
835         [candidate],
836         [references],
837         [table],
838         avg_results=True,
839         n_jobs=config.parent_n_jobs,
840         use_tqdm=False,
841     )
842     duration = time.perf_counter() - started
843     return [
844         observation(
845             record,
846             family="table_aware",
847             name="parent_precision",
848             status=MetricStatus.SCORED,
849             score=float(precision),
850             duration=duration,
851         ),
852         observation(
853             record,
854             family="table_aware",
855             name="parent_recall",
856             status=MetricStatus.SCORED,
857             score=float(recall),
858             duration=duration,
859         ),
860         observation(
861             record,
862             family="table_aware",
863             name="parent_f1",
864             status=MetricStatus.SCORED,
865             score=float(f1),
866             duration=duration,
867         ),
868     ]
869 except Exception as exc:
870     duration = time.perf_counter() - started
871     return [
872         observation(
873             record,
874             family="table_aware",
875             name=name,
876             status=MetricStatus.ERROR,
877             duration=duration,
878             error=f"{type(exc).__name__}: {exc}",
879         )
880         for name in metric_names
881     ]
882
883 def score_hhem_records(
884     records: list[GenerationRecord],
885     config: ReferenceMetricConfig,
886 ) -> list[MetricObservation]:
887     if not metric_enabled(config, "hhem", "hhem_2_1_open"):
888         return []
889
890     evaluator = HHEMEvaluator(
891         model_name=config.hhem_model,
892         foundation_model_name=config.hhem_foundation_model,
893         threshold=config.hhem_threshold,
894         batch_size=config.hhem_batch_size,
895         device=config.hhem_device,
896         local_files_only=config.hf_local_files_only,
897         max_context_characters=config.hhem_context_max_characters,
898     )
899     observations: list[MetricObservation] = []
900     for record in records:
901         started = time.perf_counter()
902         result = evaluator.evaluate(
903             context=factuality_context(record, config),
904             generated_text=plain_text(record.generated_text),
905         )
906         observations.extend(
907             hhem_observations(

```

```

909         record,
910         result,
911         duration=time.perf_counter() - started,
912     )
913 )
914 del evaluator
915 gc.collect()
916 return observations
917
918
919 def unavailable_alignscore_observation(
920     record: GenerationRecord,
921     config: ReferenceMetricConfig,
922     *,
923     reason: str,
924 ) -> MetricObservation:
925     return observation(
926         record,
927         family="external_factuality",
928         name=f"alignscore_{config.alignscore_model_size}",
929         status=MetricStatus.UNAVAILABLE,
930         details={
931             "threshold": config.alignscore_threshold,
932             "model_size": config.alignscore_model_size,
933         },
934         error=reason,
935     )
936
937
938 def score_alignscore_records(
939     records: list[GenerationRecord],
940     config: ReferenceMetricConfig,
941 ) -> list[MetricObservation]:
942     if not metric_enabled(config, "alignscore"):
943         return []
944     if not records:
945         return []
946
947     if config.alignscore_python_executable is None:
948         discovered_executable = default_alignscore_python_executable()
949     else:
950         discovered_executable = config.alignscore_python_executable
951
952     if discovered_executable is None:
953         reason = (
954             "AlignScore requires a separate worker Python executable. "
955             "Set reference_metrics.alignscore_python_executable in metrics.json "
956             "or create .venv-alignscore in the project root."
957         )
958         return [
959             unavailable_alignscore_observation(record, config, reason=reason)
960             for record in records
961         ]
962
963     worker_path = config.alignscore_worker_path or DEFAULT_ALIGNSCORE_WORKER
964     if not worker_path.exists():
965         reason = f"AlignScore worker script was not found at {worker_path}."
966         return [
967             unavailable_alignscore_observation(record, config, reason=reason)
968             for record in records
969         ]
970
971     try:
972         client = AlignScoreClient(
973             python_executable=discovered_executable,
974             worker_path=worker_path,
975             model_size=config.alignscore_model_size,
976             device=config.alignscore_device,
977             batch_size=config.alignscore_batch_size,
978             threshold=config.alignscore_threshold,
979             local_files_only=config.hf_local_files_only,
980         )
981     except Exception as exc:
982         reason = f"{type(exc).__name__}: {exc}"
983         return [
984             unavailable_alignscore_observation(record, config, reason=reason)
985             for record in records
986         ]
987
988     observations: list[MetricObservation] = []
989     try:
990         for record in records:
991             started = time.perf_counter()
992             try:
993                 result = client.evaluate(
994                     context=factuality_context(record, config),
995                     generated_text=plain_text(record.generated_text),
996                 )
997             except Exception as exc:
998                 result = ExternalFactualityResult(
999                     metric_name=f"alignscore_{config.alignscore_model_size}",

```

```

1000         status="error",
1001         threshold=config.alignscore_threshold,
1002         error=f"{type(exc).__name__}: {exc}",
1003     )
1004     observations.append(
1005         alignscore_observation(
1006             record,
1007             result,
1008             duration=time.perf_counter() - started,
1009         )
1010     )
1011 finally:
1012     try:
1013         client.close()
1014     except Exception:
1015         pass
1016 return observations
1017
1018 def corpus_sacrebleu_observations(
1019     records: list[GenerationRecord],
1020     config: ReferenceMetricConfig,
1021 ) -> list[MetricObservation]:
1022     enabled = set(config.enabled_metrics)
1023     relevant = {"bleu", "chrF", "ter"} & enabled
1024     if not relevant or not records:
1025         return []
1026     try:
1027         import sacrebleu
1028     except ImportError:
1029         return []
1030
1031 grouped: dict[tuple[str, str, int], list[GenerationRecord]] = defaultdict(list)
1032 for record in records:
1033     grouped[(record.dataset_id, record.variant_id, record.repetition)].append(record)
1034 results: list[MetricObservation] = []
1035 for (dataset_id, variant_id, repetition), group in grouped.items():
1036     ordered = sorted(group, key=lambda item: item.example_id)
1037     candidates = [plain_text(item.generated_text) for item in ordered]
1038     maximum_reference_count = max(len(item.references) for item in ordered)
1039     reference_streams = [
1040         [
1041             plain_text(item.references[reference_index])
1042             if reference_index < len(item.references)
1043             else ""
1044             for item in ordered
1045         ]
1046         for reference_index in range(maximum_reference_count)
1047     ]
1048 representative = ordered[0]
1049 if "bleu" in relevant:
1050     result = sacrebleu.corpus_bleu(
1051         candidates,
1052         reference_streams,
1053         lowercase=config.lowercase,
1054     )
1055     results.append(
1056         observation(
1057             representative,
1058             family="lexical_corpus",
1059             name="corpus_bleu",
1060             status=MetricStatus.SCORED,
1061             score=result.score / 100.0,
1062             details={
1063                 "dataset_id": dataset_id,
1064                 "variant_id": variant_id,
1065                 "repetition": repetition,
1066                 "example_count": len(ordered),
1067                 "reference_stream_count": maximum_reference_count,
1068             },
1069         )
1070     )
1071 if "chrF" in relevant:
1072     result = sacrebleu.corpus_chrf(candidates, reference_streams, word_order=2)
1073     results.append(
1074         observation(
1075             representative,
1076             family="lexical_corpus",
1077             name="corpus_chrf",
1078             status=MetricStatus.SCORED,
1079             score=result.score / 100.0,
1080             details={"example_count": len(ordered)},
1081         )
1082     )
1083 if "ter" in relevant:
1084     result = sacrebleu.corpus_ter(candidates, reference_streams)
1085     results.append(
1086         observation(
1087             representative,
1088             family="lexical_corpus",
1089             name="corpus_ter",

```



```

1091         status=MetricStatus.SCORED,
1092         score=result.score / 100.0,
1093         higher_is_better=False,
1094         details={"example_count": len(ordered)},
1095     )
1096 )
1097 return results
1098
1099
1100 def evaluate_reference_metrics(
1101     records: list[GenerationRecord],
1102     config: ReferenceMetricConfig,
1103     output_path: Path,
1104     *,
1105     include_ineligible: bool = False,
1106 ) -> list[MetricObservation]:
1107     eligible: list[GenerationRecord] = []
1108     observations: list[MetricObservation] = []
1109     for record in records:
1110         reason = ineligible_generation_reason(
1111             record,
1112             include_ineligible=include_ineligible,
1113         )
1114         if reason is None:
1115             eligible.append(record)
1116             continue
1117         observations.append(
1118             observation(
1119                 record,
1120                 family="reference",
1121                 name="reference_metrics",
1122                 status=MetricStatus.SKIPPED,
1123                 details={"reason": reason},
1124                 error=reason,
1125             )
1126         )
1127     for record in eligible:
1128         observations.extend(score_lexical_record(record, config))
1129         observations.extend(score_parent_record(record, config))
1130         observations.extend(score_hhem_records(eligible, config))
1131         observations.extend(score_alignscore_records(eligible, config))
1132         observations.extend(score_bertscore(eligible, config))
1133         observations.extend(corpus_sacrebleu_observations(eligible, config))
1134     write_jsonl(output_path, observations)
1135     return observations

```

Listing 40: P40: table2text_pydanticai/src/table2text/evaluation/statistics.py

```

1  """Aggregate metric observations and estimate uncertainty and comparisons."""
2
3  from __future__ import annotations
4
5  import json
6  from pathlib import Path
7  from typing import Any, Iterable
8
9  import numpy as np
10 import pandas as pd
11
12 from .models import DeepEvalObservation, MetricObservation, MetricStatus
13
14
15 def read_metric_observations(path: Path) -> list[MetricObservation]:
16     return [
17         MetricObservation.model_validate(json.loads(line))
18         for line in path.read_text(encoding="utf-8").splitlines()
19         if line.strip()
20     ]
21
22
23 def read_deepeval_observations(path: Path) -> list[DeepEvalObservation]:
24     return [
25         DeepEvalObservation.model_validate(json.loads(line))
26         for line in path.read_text(encoding="utf-8").splitlines()
27         if line.strip()
28     ]
29
30
31 def observations_to_frame(
32     observations: Iterable[MetricObservation | DeepEvalObservation],
33 ) -> pd.DataFrame:
34     rows: list[dict[str, Any]] = []
35     for item in observations:
36         if item.status != MetricStatus.SCORED:
37             continue
38         rows.append(
39             {
40                 "generation_id": item.generation_id,
41                 "dataset_id": item.dataset_id,

```

```

42         "example_id": item.example_id,
43         "variant_id": item.variant_id,
44         "repetition": item.repetition,
45         "metric_name": item.metric_name,
46         "score": item.score,
47         "duration_seconds": item.duration_seconds,
48     }
49 )
50 return pd.DataFrame(rows)
51
52
53 def collapse_judge_repetitions(frame: pd.DataFrame) -> pd.DataFrame:
54     if frame.empty:
55         return frame
56     return (
57         frame.groupby(
58             [
59                 "generation_id",
60                 "dataset_id",
61                 "example_id",
62                 "variant_id",
63                 "repetition",
64                 "metric_name",
65             ],
66             as_index=False,
67         )
68         .agg(
69             score=("score", "mean"),
70             judge_score_std=("score", "std"),
71             duration_seconds=("duration_seconds", "sum"),
72         )
73     )
74
75
76 def descriptive_summary(frame: pd.DataFrame) -> pd.DataFrame:
77     if frame.empty:
78         return pd.DataFrame()
79     return (
80         frame.groupby(["dataset_id", "variant_id", "metric_name"], as_index=False)
81         .agg(
82             count=("score", "count"),
83             mean=("score", "mean"),
84             standard_deviation=("score", "std"),
85             median=("score", "median"),
86             minimum=("score", "min"),
87             maximum=("score", "max"),
88         )
89     )
90
91
92 def macro_dataset_summary(dataset_summary: pd.DataFrame) -> pd.DataFrame:
93     if dataset_summary.empty:
94         return pd.DataFrame()
95     return (
96         dataset_summary.groupby(["variant_id", "metric_name"], as_index=False)
97         .agg(
98             dataset_count=("dataset_id", "nunique"),
99             macro_mean=("mean", "mean"),
100             macro_standard_deviation=("mean", "std"),
101         )
102     )
103
104
105 def percentile_interval(values: np.ndarray, confidence_level: float) -> tuple[float, float]:
106     alpha = 1.0 - confidence_level
107     return (
108         float(np.quantile(values, alpha / 2)),
109         float(np.quantile(values, 1 - alpha / 2)),
110     )
111
112
113 def paired_bootstrap(
114     baseline: np.ndarray,
115     candidate: np.ndarray,
116     *,
117     resamples: int,
118     confidence_level: float,
119     seed: int,
120 ) -> dict[str, float]:
121     if len(baseline) != len(candidate):
122         raise ValueError("Paired arrays must have equal lengths.")
123     if len(baseline) == 0:
124         raise ValueError("Paired arrays must not be empty.")
125     random_generator = np.random.default_rng(seed)
126     indexes = random_generator.integers(0, len(baseline), size=(resamples, len(baseline)))
127     differences = candidate[indexes].mean(axis=1) - baseline[indexes].mean(axis=1)
128     lower, upper = percentile_interval(differences, confidence_level)
129     observed = float(candidate.mean() - baseline.mean())
130     probability_candidate_better = float(np.mean(differences > 0))
131     return {
132         "mean_difference": observed,

```

```

133     "confidence_lower": lower,
134     "confidence_upper": upper,
135     "probability_candidate_better": probability_candidate_better,
136 }
137
138
139 def paired_variant_comparisons(
140     frame: pd.DataFrame,
141     *,
142     baseline_variant: str,
143     resamples: int,
144     confidence_level: float,
145     seed: int,
146 ) -> pd.DataFrame:
147     rows: list[dict[str, Any]] = []
148     if frame.empty:
149         return pd.DataFrame()
150     for (dataset_id, metric_name), group in frame.groupby(["dataset_id", "metric_name"]):
151         baseline = group[group["variant_id"] == baseline_variant][
152             ["example_id", "repetition", "score"]
153         ].rename(columns={"score": "baseline_score"})
154         if baseline.empty:
155             continue
156         for variant_id in sorted(set(group["variant_id"]) - {baseline_variant}):
157             candidate = group[group["variant_id"] == variant_id][
158                 ["example_id", "repetition", "score"]
159             ].rename(columns={"score": "candidate_score"})
160             paired = baseline.merge(candidate, on=["example_id", "repetition"], how="inner").dropna()
161             if paired.empty:
162                 continue
163             result = paired_bootstrap(
164                 paired["baseline_score"].to_numpy(dtype=float),
165                 paired["candidate_score"].to_numpy(dtype=float),
166                 resamples=resamples,
167                 confidence_level=confidence_level,
168                 seed=seed,
169             )
170             rows.append(
171                 {
172                     "dataset_id": dataset_id,
173                     "metric_name": metric_name,
174                     "baseline_variant": baseline_variant,
175                     "candidate_variant": variant_id,
176                     "paired_count": len(paired),
177                     **result,
178                 }
179             )
180     return pd.DataFrame(rows)
181
182
183 def metric_correlation_matrix(frame: pd.DataFrame) -> pd.DataFrame:
184     if frame.empty:
185         return pd.DataFrame()
186     pivot = frame.pivot_table(
187         index="generation_id",
188         columns="metric_name",
189         values="score",
190         aggfunc="mean",
191     )
192     if pivot.shape[1] < 2:
193         return pd.DataFrame()
194     return pivot.corr(method="spearmanr")
195
196
197 def pairwise_metric_correlations(frame: pd.DataFrame) -> pd.DataFrame:
198     if frame.empty:
199         return pd.DataFrame()
200     try:
201         from scipy.stats import spearmanr
202     except ImportError:
203         spearmanr = None
204     pivot = frame.pivot_table(
205         index="generation_id",
206         columns="metric_name",
207         values="score",
208         aggfunc="mean",
209     )
210     rows: list[dict[str, Any]] = []
211     columns = list(pivot.columns)
212     for first_index, first in enumerate(columns):
213         for second_index, second in enumerate(columns[first_index + 1 :]):
214             paired = pivot[[first, second]].dropna()
215             if len(paired) < 3 or spearmanr is None:
216                 coefficient = None
217                 p_value = None
218             else:
219                 result = spearmanr(paired[first], paired[second])
220                 coefficient = float(result.statistic)
221                 p_value = float(result.pvalue)
222             rows.append(
223                 {

```

```

224         "metric_a": first,
225         "metric_b": second,
226         "paired_count": len(paired),
227         "spearman_r": coefficient,
228         "p_value": p_value,
229     }
230 )
231 return pd.DataFrame(rows)
232
233
234 def cost_summary(
235     metric_frame: pd.DataFrame,
236     diagnostics: pd.DataFrame | None = None,
237 ) -> pd.DataFrame:
238     rows: list[dict[str, Any]] = []
239     if not metric_frame.empty:
240         for metric_name, group in metric_frame.groupby("metric_name"):
241             rows.append(
242                 {
243                     "category": "metric",
244                     "name": metric_name,
245                     "count": len(group),
246                     "total_duration_seconds": group["duration_seconds"].fillna(0).sum(),
247                     "mean_duration_seconds": group["duration_seconds"].mean(),
248                     "total_cost_gbp": None,
249                 }
250             )
251     if diagnostics is not None and not diagnostics.empty:
252         for variant_id, group in diagnostics.groupby("variant_id"):
253             rows.append(
254                 {
255                     "category": "generation",
256                     "name": variant_id,
257                     "count": len(group),
258                     "total_duration_seconds": group["elapsed_seconds"].fillna(0).sum(),
259                     "mean_duration_seconds": group["elapsed_seconds"].mean(),
260                     "total_cost_gbp": group["estimated_cost_gbp"].fillna(0).sum(),
261                 }
262             )
263     return pd.DataFrame(rows)
264
265
266 def write_analysis(
267     *,
268     reference_observations_path: Path,
269     deepeval_observations_path: Path | None,
270     diagnostics_path: Path | None,
271     output_directory: Path,
272     baseline_variant: str,
273     bootstrap_resamples: int,
274     confidence_level: float,
275     seed: int,
276 ) -> None:
277     reference = observations_to_frame(read_metric_observations(reference_observations_path))
278     frames = [reference]
279     if deepeval_observations_path is not None and deepeval_observations_path.exists():
280         judge = observations_to_frame(read_deepeval_observations(deepeval_observations_path))
281         frames.append(collapse_judge_repetitions(judge))
282     combined = pd.concat(frames, ignore_index=True)
283     output_directory.mkdir(parents=True, exist_ok=True)
284     combined.to_csv(output_directory / "all_metric_scores.csv", index=False)
285     dataset_summary = descriptive_summary(combined)
286     dataset_summary.to_csv(output_directory / "dataset_variant_summary.csv", index=False)
287     macro = macro_dataset_summary(dataset_summary)
288     macro.to_csv(output_directory / "macro_summary.csv", index=False)
289     comparisons = paired_variant_comparisons(
290         combined,
291         baseline_variant=baseline_variant,
292         resamples=bootstrap_resamples,
293         confidence_level=confidence_level,
294         seed=seed,
295     )
296     comparisons.to_csv(output_directory / "paired_bootstrap_comparisons.csv", index=False)
297     metric_correlation_matrix(combined).to_csv(output_directory / "metric_correlation_matrix.csv")
298     pairwise_metric_correlations(combined).to_csv(
299         output_directory / "metric_correlations_long.csv",
300         index=False,
301     )
302     diagnostics = (
303         pd.read_csv(diagnostics_path)
304         if diagnostics_path is not None and diagnostics_path.exists()
305         else None
306     )
307     cost_summary(combined, diagnostics).to_csv(output_directory / "cost_summary.csv", index=False)

```

Listing 41: P41: table2text_pydanticai/src/table2text/evaluation_backends.py

```

1 """Generation backends used by the evaluation framework and direct baselines."""
2

```

```

3 from __future__ import annotations
4
5 import json
6 import os
7 from typing import Any
8
9 from table2text.config import load_env_files
10 from table2text.evaluation.models import BenchmarkExample, VariantConfig
11
12
13 def _first_env(*names: str) -> str | None:
14     for name in names:
15         value = os.getenv(name)
16         if value is not None and value.strip():
17             return value.strip()
18     return None
19
20
21 def _variant_or_env(
22     variant: VariantConfig,
23     setting_name: str,
24     env_name: str,
25     default: Any,
26 ) -> Any:
27     if setting_name in variant.settings_overrides:
28         return variant.settings_overrides[setting_name]
29     value = _first_env(env_name)
30     return default if value is None else value
31
32
33 def _normalise_deepseek_model_name(model_name: str) -> str:
34     if model_name.startswith("deepseek:"):
35         return model_name.split(":", 1)[1]
36     return model_name
37
38
39 def _int_setting(value: Any) -> int:
40     if isinstance(value, int):
41         return value
42     return int(str(value).replace("_", ""))
43
44
45 def _float_setting(value: Any) -> float:
46     if isinstance(value, float):
47         return value
48     return float(str(value))
49
50
51 def _source_text(example: BenchmarkExample, max_characters: int) -> str:
52     source = example.source_text.strip()
53     if not source:
54         source = json.dumps(
55             example.source_payload,
56             ensure_ascii=False,
57             indent=2,
58             sort_keys=True,
59         )
60     if len(source) <= max_characters:
61         return source
62     return source[:max_characters] + "\n\n[Source truncated by raw baseline configuration.]"
63
64
65 def _readable_label(value: Any) -> str:
66     raw = getattr(value, "value", value)
67     return str(raw).replace("_", " ")
68
69
70 def build_single_agent_prompt(
71     example: BenchmarkExample,
72     *,
73     max_source_characters: int = 100_000,
74     prompt_style: str = "structured",
75 ) -> list[dict[str, str]]:
76     source = _source_text(example, max_source_characters)
77     system_prompt = (
78         "You are a raw single-LLM data-to-text baseline. Generate the requested "
79         "output directly from the supplied source data. Use only the source data "
80         "and the user request. Do not use outside knowledge. Do not invent "
81         "numbers, entities, chronology, causal explanations, or background. "
82         "Do not mention hidden references, evaluation, prompts, or uncertainty "
83         "unless the source itself makes the requested output impossible."
84     )
85     normalized_style = prompt_style.strip().lower()
86     if normalized_style == "generic":
87         user_prompt = (
88             f"Request:\n{example.request}\n\n"
89             "Source data:\n"
90             f"{source}\n\n"
91             "Write the final answer only."
92         )
93     elif normalized_style == "structured":

```

```

94     user_prompt = (
95         f"Task type: {_readable_label(example.task_family)}\n"
96         f"Expected form: {_readable_label(example.output_mode)}\n"
97         f"Language: {example.language}\n\n"
98         f"Request:\n{example.request}\n\n"
99         "Source data:\n"
100         f"{source}\n\n"
101         "Write the final answer only."
102     )
103 else:
104     raise ValueError(
105         "raw_baseline_prompt_style must be 'structured' or 'generic'."
106     )
107 return [
108     {"role": "system", "content": system_prompt},
109     {"role": "user", "content": user_prompt},
110 ]
111
112
113 def single_agent_baseline(
114     *,
115     example: BenchmarkExample,
116     variant: VariantConfig,
117     repetition: int,
118     seed: int,
119 ) -> dict[str, Any]:
120     del repetition, seed
121
122     load_env_files()
123
124     model_name = str(
125         _variant_or_env(
126             variant,
127             "raw_baseline_model",
128             "T2T_RAW_BASELINE_MODEL",
129             "deepseek-v4-pro",
130         )
131     )
132     model_name = _normalise_deepseek_model_name(model_name)
133     base_url = str(
134         _variant_or_env(
135             variant,
136             "raw_baseline_base_url",
137             "DEEPSEEK_BASE_URL",
138             "https://api.deepseek.com",
139         )
140     )
141     max_source_characters = _int_setting(
142         _variant_or_env(
143             variant,
144             "raw_baseline_max_source_characters",
145             "T2T_RAW_BASELINE_MAX_SOURCE_CHARACTERS",
146             100_000,
147         )
148     )
149     max_output_tokens = _int_setting(
150         _variant_or_env(
151             variant,
152             "raw_baseline_max_output_tokens",
153             "T2T_RAW_BASELINE_MAX_OUTPUT_TOKENS",
154             1_500,
155         )
156     )
157     temperature = _float_setting(
158         _variant_or_env(
159             variant,
160             "raw_baseline_temperature",
161             "T2T_RAW_BASELINE_TEMPERATURE",
162             0.2,
163         )
164     )
165     prompt_style = str(
166         _variant_or_env(
167             variant,
168             "raw_baseline_prompt_style",
169             "T2T_RAW_BASELINE_PROMPT_STYLE",
170             "structured",
171         )
172     )
173
174     api_key = _first_env("DEEPSEEK_API_KEY")
175     if api_key is None:
176         raise RuntimeError(
177             "DEEPSEEK_API_KEY is required to run the raw DeepSeek baseline."
178         )
179
180     try:
181         from openai import OpenAI
182     except ImportError as exc:
183         raise RuntimeError(
184             "The openai package is required to run the raw DeepSeek baseline."

```

```

185         ) from exc
186
187     messages = build_single_agent_prompt(
188         example,
189         max_source_characters=max_source_characters,
190         prompt_style=prompt_style,
191     )
192     client = OpenAI(api_key=api_key, base_url=base_url)
193     response = client.chat.completions.create(
194         model=model_name,
195         messages=messages,
196         temperature=temperature,
197         max_tokens=max_output_tokens,
198     )
199     text = response.choices[0].message.content or ""
200     usage = getattr(response, "usage", None)
201     usage_details = (
202         {
203             "prompt_tokens": getattr(usage, "prompt_tokens", None),
204             "completion_tokens": getattr(usage, "completion_tokens", None),
205             "total_tokens": getattr(usage, "total_tokens", None),
206         }
207         if usage is not None
208         else {}
209     )
210     return {
211         "generated_text": text.strip(),
212         "baseline_type": "raw_single_llm",
213         "provider": "deepseek",
214         "model": model_name,
215         "base_url": base_url,
216         "max_source_characters": max_source_characters,
217         "max_output_tokens": max_output_tokens,
218         "temperature": temperature,
219         "prompt_style": prompt_style,
220         **usage_details,
221     }

```

Listing 42: P42: table2text_pydanticai/src/table2text/narrative.py

```

1  """Select and order verified content for genre-appropriate narrative plans."""
2
3  from __future__ import annotations
4
5  import re
6  from collections import defaultdict
7  from typing import Any
8
9  from .schemas import (
10     EvidenceCapability,
11     EvidenceItem,
12     NarrativePlan,
13     NarrativeSlot,
14     NarrativeSlotPlan,
15     RealisationPolicy,
16     RecommendedUse,
17     ReportGenre,
18     VerifiedFact,
19     VerifiedInsight,
20     WriterEvidencePack,
21 )
22
23
24  EVENT_GENRES = {
25     ReportGenre.EVENT_REPORT,
26     ReportGenre.SPORTS_GAME_REPORT,
27 }
28
29  EVENT_SLOT_BY_EVIDENCE_TYPE = {
30     "event_outcome": NarrativeSlot.OPENING_RESULT,
31     "event_context": NarrativeSlot.EVENT_CONTEXT,
32     "event_status": NarrativeSlot.EVENT_CONTEXT,
33     "participant_record_context": NarrativeSlot.EVENT_CONTEXT,
34     "score_progression": NarrativeSlot.SCORE_PROGRESSION,
35     "event_sequence": NarrativeSlot.EVENT_SEQUENCE,
36     "entity_ranking": NarrativeSlot.LEADING_PERFORMANCES,
37     "entity_performance": NarrativeSlot.LEADING_PERFORMANCES,
38     "participant_comparison": NarrativeSlot.PARTICIPANT_CONTRASTS,
39     "event_contrast": NarrativeSlot.PARTICIPANT_CONTRASTS,
40     "group_comparison": NarrativeSlot.PARTICIPANT_CONTRASTS,
41 }
42
43  SLOT_PURPOSES = {
44     NarrativeSlot.OPENING_RESULT: (
45         "Open with the supplied event result, score/outcome and margin when "
46         "those values are verified."
47     ),
48     NarrativeSlot.EVENT_CONTEXT: (
49         "Attach compact supported context such as date, venue, event status "

```

```

50         "or participant record context to the opening rather than making a "
51         "dataset overview."
52     ),
53     NarrativeSlot.SCORE_PROGRESSION: (
54         "Use supplied segment-level progression to show how the result is "
55         "situated across the event."
56     ),
57     NarrativeSlot.EVENT_SEQUENCE: (
58         "Use supported ordered score-changing or event-sequence evidence for "
59         "natural chronology without inventing momentum or causes."
60     ),
61     NarrativeSlot.LEADING_PERFORMANCES: (
62         "Select the strongest supported entity performances and rankings, "
63         "combining related measures for the same entity where support allows."
64     ),
65     NarrativeSlot.PARTICIPANT_CONTRASTS: (
66         "Relate major participant-level contrasts with bounded connective "
67         "language such as while, compared with or despite."
68     ),
69     NarrativeSlot.SECONDARY_DETAILS: (
70         "Use additional supported details only when they add distinct value "
71         "after the result, sequence, leading performances and contrasts."
72     ),
73     NarrativeSlot.CLOSING_SCOPE: (
74         "Keep the scope bounded to the supplied event when a visible caveat "
75         "is required by the report contract."
76     ),
77 }
78
79 LOW_PRIORITY_METRIC_PATTERN = re.compile(
80     r"\b("
81     r"attempt(?:ed|s)?|minutes?|personal fouls?|turnovers?|"
82     r"field[- ]?goal attempts?|field[- ]?goal percentage|"
83     r"three[- ]?point attempts?|three[- ]?point percentage|"
84     r"free[- ]?throw attempts?|free[- ]?throw percentage|at bats?|"
85     r"plate appearances?|batters faced|pitch(?:es)?|strikes?|"
86     r"putouts?|walks?|left on base|game number|game score|"
87     r"season total|recorded steps?|capacity|attendance"
88     r")\b",
89     re.IGNORECASE,
90 )
91
92 CORE_ENTITY_METRIC_PATTERN = re.compile(
93     r"\b(?:assists?|blocks?|doubles?|goals?|hits?|home runs?|points?|"
94     r"rebounds?|runs(?: batted in)?|saves?|scor(?:e|ed|ing)|steals?|"
95     r"strikeouts?|triples?)\b",
96     re.IGNORECASE,
97 )
98
99 def _evidence_lookup(pack: WriterEvidencePack) -> dict[str, EvidenceItem]:
100     return {
101         item.evidence_id: item
102         for item in pack.evidence_ledger.items
103         if item.eligible_for_writer
104     }
105
106
107 def _fact_evidence(
108     fact: VerifiedFact,
109     evidence_by_id: dict[str, EvidenceItem],
110 ) -> list[EvidenceItem]:
111     return [
112         evidence_by_id[evidence_id]
113         for evidence_id in fact.evidence_ids
114         if evidence_id in evidence_by_id
115     ]
116
117
118 def _slot_for_fact(
119     fact: VerifiedFact,
120     evidence_by_id: dict[str, EvidenceItem],
121 ) -> NarrativeSlot | None:
122     evidence_items = _fact_evidence(fact, evidence_by_id)
123     if not evidence_items:
124         return None
125
126     ranked_slots = [
127         EVENT_SLOT_BY_EVIDENCE_TYPE.get(item.evidence_type)
128         for item in evidence_items
129     ]
130     ranked_slots = [slot for slot in ranked_slots if slot is not None]
131     if not ranked_slots:
132         if EvidenceCapability.RANKING in fact.source_capabilities:
133             return NarrativeSlot.LEADING_PERFORMANCES
134         if EvidenceCapability.GROUP_COMPARISON in fact.source_capabilities:
135             return NarrativeSlot.PARTICIPANT_CONTRASTS
136         return None
137
138     priority = {
139         NarrativeSlot.OPENING_RESULT: 0,
140         NarrativeSlot.SCORE_PROGRESSION: 1,

```



```

141     NarrativeSlot.EVENT_SEQUENCE: 2,
142     NarrativeSlot.LEADING_PERFORMANCES: 3,
143     NarrativeSlot.PARTICIPANT_CONTRASTS: 4,
144     NarrativeSlot.EVENT_CONTEXT: 5,
145 }
146 return min(ranked_slots, key=lambda slot: priority.get(slot, 100))
147
148
149 def _is_low_priority_event_fact(
150     fact: VerifiedFact,
151     evidence_items: list[EvidenceItem],
152 ) -> bool:
153     if fact.recommended_use == RecommendedUse.OMIT_UNLESS_REQUESTED:
154         return True
155     if fact.recommended_use in {
156         RecommendedUse.HEADLINE,
157         RecommendedUse.MAIN_FINDING,
158     }:
159         return False
160     if fact.salience < 0.75 or fact.user_relevance < 0.75:
161         return True
162
163     text = " ".join(
164         [
165             fact.fact_summary,
166             *[
167                 item.finding
168                 for item in evidence_items
169             ],
170             *[
171                 item.strength_label
172                 for item in evidence_items
173             ],
174         ]
175     )
176     if (
177         any(item.evidence_type == "entity_performance" for item in evidence_items)
178         and CORE_ENTITY_METRIC_PATTERN.search(text)
179     ):
180         return False
181     return bool(LOW_PRIORITY_METRIC_PATTERN.search(text))
182
183
184 def _insight_ids_for_facts(
185     insights: list[VerifiedInsight],
186     fact_ids: set[str],
187 ) -> list[str]:
188     return [
189         insight.insight_id
190         for insight in insights
191         if fact_ids.intersection(insight.source_fact_ids)
192     ]
193
194
195 def _slot_plan(
196     *,
197     slot: NarrativeSlot,
198     fact_ids: list[str],
199     insight_ids: list[str],
200     priority: int,
201     minimum_items: int,
202     paragraph_hint: int,
203     connective_hint: str | None = None,
204     maximum_items: int | None = None,
205     visible_caveat_allowed: bool = True,
206 ) -> NarrativeSlotPlan:
207     return NarrativeSlotPlan(
208         slot=slot,
209         purpose=SLOT_PURPOSES[slot],
210         fact_ids=list(dict.fromkeys(fact_ids)),
211         insight_ids=list(dict.fromkeys(insight_ids)),
212         priority=priority,
213         minimum_items=minimum_items,
214         maximum_items=maximum_items,
215         paragraph_hint=paragraph_hint,
216         connective_hint=connective_hint,
217         visible_caveat_allowed=visible_caveat_allowed,
218     )
219
220
221 def build_event_narrative_plan(
222     pack: WriterEvidencePack,
223     content_requirements: dict[str, Any] | None = None,
224 ) -> NarrativePlan:
225     if pack.report_specification.genre not in EVENT_GENRES:
226         return NarrativePlan()
227
228     requirements = content_requirements or {}
229     realisation_policy = requirements.get(
230         "realisation_policy",
231         pack.report_specification.realisation_policy.value,

```

```

232 )
233 event_recap_style = bool(
234     realisation_policy == RealisationPolicy.EVENT_RECAP_STYLE.value
235 )
236 reference_recap_style = bool(
237     requirements.get("reference_recap_style")
238     or pack.report_specification.focus_scope == "reference_recap"
239 )
240
241 evidence_by_id = _evidence_lookup(pack)
242 all_facts = [
243     *pack.priority_facts,
244     *pack.supporting_facts,
245     *pack.limitation_facts,
246 ]
247 all_insights = [
248     *pack.priority_verified_insights,
249     *pack.supporting_verified_insights,
250 ]
251
252 fact_ids_by_slot: dict[NarrativeSlot, list[str]] = defaultdict(list)
253 low_priority_fact_ids: list[str] = []
254
255 for fact in all_facts:
256     evidence_items = _fact_evidence(fact, evidence_by_id)
257     slot = _slot_for_fact(fact, evidence_by_id)
258     if slot is None:
259         continue
260     if _is_low_priority_event_fact(fact, evidence_items):
261         low_priority_fact_ids.append(fact.fact_id)
262         if slot == NarrativeSlot.LEADING_PERFORMANCES:
263             fact_ids_by_slot[NarrativeSlot.SECONDARY_DETAILS].append(
264                 fact.fact_id
265             )
266         continue
267     fact_ids_by_slot[slot].append(fact.fact_id)
268
269 planned_slots: list[NarrativeSlotPlan] = []
270 slot_order = [
271     (
272         NarrativeSlot.OPENING_RESULT,
273         1,
274         1,
275         1,
276         None,
277         None,
278     ),
279     (
280         NarrativeSlot.EVENT_CONTEXT,
281         2,
282         1,
283         1,
284         "Attach context to the result sentence or opening paragraph.",
285         None,
286     ),
287     (
288         NarrativeSlot.SCORE_PROGRESSION,
289         3,
290         1,
291         2,
292         "Use progression as the bridge from result to event story.",
293         None,
294     ),
295     (
296         NarrativeSlot.EVENT_SEQUENCE,
297         4,
298         1,
299         2,
300         "Narrate only verified score-changing or ordered event steps.",
301         None,
302     ),
303     (
304         NarrativeSlot.LEADING_PERFORMANCES,
305         5,
306         2,
307         3,
308         "Group related achievements by entity instead of listing metrics.",
309         None,
310     ),
311     (
312         NarrativeSlot.PARTICIPANT_CONTRASTS,
313         6,
314         1,
315         4,
316         "Use bounded contrastive connectors; avoid causal wording.",
317         None,
318     ),
319     (
320         NarrativeSlot.SECONDARY_DETAILS,
321         7,
322         0,

```

```

323         4,
324         "Use only if it adds distinct report value after higher slots.",
325         None,
326     ),
327 ]
328
329 for slot, priority, minimum, paragraph, connective, maximum in slot_order:
330     fact_ids = fact_ids_by_slot.get(slot, [])
331     if not fact_ids and minimum > 0:
332         continue
333     insight_ids = _insight_ids_for_facts(all_insights, set(fact_ids))
334     planned_slots.append(
335         _slot_plan(
336             slot=slot,
337             fact_ids=fact_ids,
338             insight_ids=insight_ids,
339             priority=priority,
340             minimum_items=min(minimum, len(fact_ids)),
341             maximum_items=maximum,
342             paragraph_hint=paragraph,
343             connective_hint=connective,
344         )
345     )
346
347 visible_scope_required = (
348     requirements
349     .get("narrative_requirements", {})
350     .get("minimum_scope_limitations", 1)
351 ) > 0
352 if visible_scope_required:
353     planned_slots.append(
354         _slot_plan(
355             slot=NarrativeSlot.CLOSING_SCOPE,
356             fact_ids=[],
357             insight_ids=[],
358             priority=8,
359             minimum_items=1,
360             paragraph_hint=4,
361             connective_hint=(
362                 "Use one concise event-scoped caveat; do not add a "
363                 "generic methodology section."
364             ),
365         )
366     )
367
368 if not planned_slots:
369     return NarrativePlan()
370
371 return NarrativePlan(
372     applies=True,
373     style=(
374         "reference_recap"
375         if reference_recap_style
376         else "event_recap"
377         if event_recap_style
378         else "structured_event_report"
379     ),
380     allow_headings=not reference_recap_style,
381     target_paragraphs=4 if (reference_recap_style or event_recap_style) else 5,
382     slots=planned_slots,
383     low_priority_fact_ids=list(dict.fromkeys(low_priority_fact_ids)),
384     prohibited_narrative_moves=[
385         "Do not add unsupported causes, momentum, dominance, historical "
386         "significance, comeback labels or broader trends.",
387         "Do not discuss row counts, missingness, constant columns, "
388         "correlation, regression, modelling or feature removal unless "
389         "the user explicitly requested dataset analysis.",
390         "Do not satisfy sequence coverage by saying the sequence was not "
391         "analysed when verified sequence evidence is available.",
392     ],
393     closing_policy=(
394         "Reference-style recaps should not add a visible generic "
395         "limitations paragraph. Add a final bounded sentence only when "
396         "needed to prevent a misleading unsupported inference."
397         if reference_recap_style
398         else (
399             "End with a concise event-scoped limitation that does not "
400             "displace the event recap."
401         )
402     ),
403 )

```

Listing 43: P43: table2text_pydtantical/src/table2text/schemas.py

```

1 """Strict Pydantic schemas shared across generation, verification, and audit."""
2
3 from __future__ import annotations
4
5 from enum import Enum

```

```

6 from typing import Any, Literal
7
8 from pydantic import BaseModel, ConfigDict, Field
9
10
11 class StrictModel(BaseModel):
12     model_config = ConfigDict(extra="forbid")
13
14
15 class AnalysisRoute(str, Enum):
16     DESCRIPTIVE = "descriptive"
17     ASSOCIATION_COMPARISON = "association_comparison"
18     PREDICTIVE = "predictive"
19     FORECASTING = "forecasting"
20     CAUSAL_FEASIBILITY = "causal_feasibility"
21
22
23 class ClaimPermission(str, Enum):
24     DESCRIPTIVE = "descriptive_claims_allowed"
25     COMPARATIVE = "comparative_claims_allowed"
26     ASSOCIATIONAL = "associational_claims_allowed"
27     PREDICTIVE = "predictive_claims_allowed_after_validation"
28     FORECAST = "forecast_claims_allowed_after_validation"
29     CAUSAL = "causal_claims_allowed_only_with_verified_design"
30     INSUFFICIENCY = "insufficiency_claims_allowed"
31     METHODOLOGICAL = "methodological_interpretation_allowed"
32
33
34 class InterpretationLevel(str, Enum):
35     FINDING = "finding"
36     BOUNDED_INSIGHT = "bounded_insight"
37     HYPOTHESIS = "hypothesis"
38
39
40 class ReportGenre(str, Enum):
41     DATA_SCIENCE_REPORT = "data_science_report"
42     DATASET_OVERVIEW = "dataset_overview"
43     EVENT_REPORT = "event_report"
44     # Retained for compatibility with existing notebook artifacts.
45     SPORTS_GAME_REPORT = "sports_game_report"
46
47
48 class CommunicationTask(str, Enum):
49     DATA_SCIENCE_REPORT = "data_science_report"
50     DATASET_OVERVIEW = "dataset_overview"
51     EVENT_REPORT = "event_report"
52     FOCUSED_TABLE_DESCRIPTION = "focused_table_description"
53     TABLE_ENTAILMENT = "table_entailment"
54     TABLE_QUESTION_ANSWERING = "table_question_answering"
55     ATTRIBUTE_VERBALISATION = "attribute_verbalisation"
56     TRIPLE_VERBALISATION = "triple_verbalisation"
57     CUSTOM = "custom"
58
59
60 class TaskFamilyHint(str, Enum):
61     DATA_SCIENCE_REPORT = "data_science_report"
62     DATASET_OVERVIEW = "dataset_overview"
63     EVENT_REPORT = "event_report"
64     HIGHLIGHTED_TABLE_DESCRIPTION = "highlighted_table_description"
65     LOGICAL_TABLE_STATEMENT = "logical_table_statement"
66     TABLE_QUESTION_ANSWERING = "table_question_answering"
67     ATTRIBUTE_VERBALISATION = "attribute_verbalisation"
68     TRIPLE_VERBALISATION = "triple_verbalisation"
69     CUSTOM = "custom"
70
71
72 class OutputForm(str, Enum):
73     ONE_SENTENCE = "one_sentence"
74     DIRECT_ANSWER = "direct_answer"
75     SHORT_TEXT = "short_text"
76     PARAGRAPH = "paragraph"
77     MULTI_PARAGRAPH_REPORT = "multi_paragraph_report"
78
79
80 class ReportPerspective(str, Enum):
81     NEUTRAL = "neutral"
82     SUBJECT_CENTRED = "subject_centred"
83
84
85 class RealisationPolicy(str, Enum):
86     STRICT_SOURCE_SURFACE = "strict_source_surface"
87     NATURAL_REFERENCE_STYLE = "natural_reference_style"
88     EVENT_RECAP_STYLE = "event_recap_style"
89     CONCISE_TABLE_PROPOSITION = "concise_table_proposition"
90
91
92 class NarrativeSlot(str, Enum):
93     OPENING_RESULT = "opening_result"
94     EVENT_CONTEXT = "event_context"
95     SCORE_PROGRESSION = "score_progression"
96     EVENT_SEQUENCE = "event_sequence"

```

```

97 LEADING_PERFORMANCES = "leading_performances"
98 PARTICIPANT_CONTRASTS = "participant_contrasts"
99 SECONDARY_DETAILS = "secondary_details"
100 CLOSING_SCOPE = "closing_scope"
101
102
103 class ReportSelectionSource(str, Enum):
104     EXPLICIT_USER_REQUEST = "explicit_user_request"
105     EXPERIMENT_CONFIGURATION = "experiment_configuration"
106     STRUCTURED_INFERENCE = "structured_inference"
107     FALLBACK = "fallback"
108
109
110 class TaskContractDecision(StrictModel):
111     task_family: TaskFamilyHint
112     report_genre: ReportGenre
113     communication_task: CommunicationTask
114     output_form: OutputForm
115     focus_scope: str | None = None
116
117     selection_source: ReportSelectionSource
118     confidence: float = Field(ge=0.0, le=1.0)
119     evidence: list[str] = Field(default_factory=list)
120     unresolved_ambiguities: list[str] = Field(default_factory=list)
121
122
123 class InputShape(str, Enum):
124     FLAT_TABLE = "flat_table"
125     NESTED_RECORD = "nested_record"
126     ENTITY_COLLECTION = "entity_collection"
127     EVENT_RECORD = "event_record"
128     TIME_SERIES = "time_series"
129     INPUT_REFERENCE_PAIRS = "input_reference_pairs"
130     AMBIGUOUS = "ambiguous"
131
132
133 class SemanticRole(str, Enum):
134     IDENTIFIER = "identifier"
135     CONTEXT = "context"
136     TIME = "time"
137     LOCATION = "location"
138     STATUS = "status"
139     PARTICIPANT_IDENTIFIER = "participant_identifier"
140     ENTITY_IDENTIFIER = "entity_identifier"
141     OUTCOME_MEASURE = "outcome_measure"
142     PERFORMANCE_MEASURE = "performance_measure"
143     MEASURE = "measure"
144     CATEGORY = "category"
145     METADATA = "metadata"
146
147
148 class AnalyticalFunction(str, Enum):
149     OUTCOME = "outcome"
150     OUTCOME_COMPONENT = "outcome_component"
151     PERFORMANCE = "performance"
152     PARTICIPATION = "participation"
153     CONTEXT = "context"
154
155
156 class SemanticLevel(str, Enum):
157     DATASET = "dataset"
158     EVENT = "event"
159     PARTICIPANT = "participant"
160     ENTITY = "entity"
161     OBSERVATION = "observation"
162
163
164 class EvidenceOperation(str, Enum):
165     RETRIEVE = "retrieve"
166     COMPARE = "compare"
167     RANK = "rank"
168
169
170 class InputRepresentationStatus(str, Enum):
171     VALID = "valid"
172     VALID_WITH_WARNINGS = "valid_with_warnings"
173     AMBIGUOUS = "ambiguous"
174     INVALID = "invalid"
175
176
177 class EvidenceCapability(str, Enum):
178     DATASET_PROFILE = "dataset_profile"
179     FOCUSED_TABLE_REGION = "focused_table_region"
180     STRUCTURED_RECORD_VERBALISATION = "structured_record_verbalisation"
181     MISSINGNESS = "missingness"
182     DUPLICATES = "duplicates"
183     DISTRIBUTION_SUMMARY = "distribution_summary"
184     ASSOCIATION = "association"
185     GROUP_COMPARISON = "group_comparison"
186     RANKING = "ranking"
187     EXTREMA = "extrema"

```

```

188     TEMPORAL_CHANGE = "temporal_change"
189     EVENT_OUTCOME = "event_outcome"
190     ENTITY_PERFORMANCE = "entity_performance"
191     ANOMALY_DETECTION = "anomaly_detection"
192
193
194     class InsightType(str, Enum):
195         DOMINANT_PATTERN = "dominant_pattern"
196         CONTRAST = "contrast"
197         REDUNDANCY = "redundancy"
198         ANOMALY = "anomaly"
199         OUTCOME_ASSOCIATION = "outcome_association"
200         TRADE_OFF = "trade_off"
201         DATA_QUALITY_IMPLICATION = "data_quality_implication"
202         NARRATIVE_SUMMARY = "narrative_summary"
203
204
205     class InsightContribution(str, Enum):
206         ANALYTICAL_IMPLICATION = "analytical_implication"
207         EVENT_SYNTHESIS = "event_synthesis"
208         DESCRIPTIVE_SYNTHESIS = "descriptive_synthesis"
209
210
211     class InsightVerificationStatus(str, Enum):
212         VERIFIED = "verified"
213         VERIFIED_WITH_CAVEAT = "verified_with_caveat"
214         HYPOTHESIS_ONLY = "hypothesis_only"
215         REJECTED = "rejected"
216
217
218     class AuditMode(str, Enum):
219         INTERNAL = "internal_evidence_fidelity"
220         EXTERNAL = "external_truth_mode"
221         ANNOTATION_ONLY = "annotation_only"
222
223
224     class AuditDecision(str, Enum):
225         PASS = "pass"
226         REVISE = "revise"
227         BLOCK = "block"
228
229
230     class ReleaseStatus(str, Enum):
231         APPROVED = "approved"
232         APPROVED_WITH_WARNINGS = "approved_with_warnings"
233         HUMAN_REVIEW_REQUIRED = "human_review_required"
234
235
236     class ReviewDecision(str, Enum):
237         APPROVE = "approve"
238         CAUTION = "caution"
239         REJECT = "reject"
240
241
242     class VerificationMethod(str, Enum):
243         LLM_VERIFIED = "llm_verified"
244         DETERMINISTIC_EVIDENCE_RECOVERY = (
245             "deterministic_evidence_recovery"
246         )
247
248
249     class ErrorType(str, Enum):
250         INCORRECT_NAMED_ENTITY = "incorrect_named_entity"
251         INCORRECT_NUMBER = "incorrect_number"
252         INCORRECT_WORD = "incorrect_word"
253         CONTEXT_ERROR = "context_error"
254         SUPPORT_MAPPING_ERROR = "support_mapping_error"
255         NOT_CHECKABLE = "not_checkable"
256         OTHER = "other"
257
258
259     class Severity(str, Enum):
260         LOW = "low"
261         MEDIUM = "medium"
262         HIGH = "high"
263         CRITICAL = "critical"
264
265
266     class RecommendedUse(str, Enum):
267         HEADLINE = "headline"
268         MAIN_FINDING = "main_finding"
269         SUPPORTING_DETAIL = "supporting_detail"
270         LIMITATION = "limitation"
271         OMIT_UNLESS_REQUESTED = "omit_unless_requested"
272
273
274     class ZeroRisk(str, Enum):
275         NONE = "none"
276         LIKELY_VALID = "likely_valid_zero"
277         CONTEXT_DEPENDENT = "context_dependent_zero"
278         UNUSUAL = "unusual_zero"

```

```

279     POSSIBLE_SENTINEL = "possible_sentinel_zero"
280
281
282 class ReportComponent(str, Enum):
283     DATASET_OVERVIEW = "dataset_overview"
284     DATA_QUALITY = "data_quality"
285     STRONGEST_RELATIONSHIPS = "strongest_relationships"
286     MODELLING_VALIDATION = "modelling_validation"
287     LIMITATIONS_NEXT_STEPS = "limitations_next_steps"
288
289
290 class TargetStatus(str, Enum):
291     USER_SELECTED = "user_selected"
292     METADATA_CONFIRMED = "metadata_confirmed"
293     EXPERIMENTAL_CANDIDATE = "experimental_candidate"
294     UNCONFIRMED = "unconfirmed"
295
296
297 class ValidationStrategy(str, Enum):
298     NONE = "none"
299     RANDOM_HOLDOUT = "random_holdout"
300     STRATIFIED_HOLDOUT = "stratified_holdout"
301     CHRONOLOGICAL_HOLDOUT = "chronological_holdout"
302     ROLLING_ORIGIN = "rolling_origin"
303
304
305 class SupportType(str, Enum):
306     DIRECT = "direct"
307     PARAPHRASE = "paraphrase"
308     MULTI_FACT_SYNTHESIS = "multi_fact_synthesis"
309     NON_FACTUAL = "non_factual_transition"
310
311
312 class RepairStrategy(str, Enum):
313     MINIMAL_CORRECTION = "minimal_correction"
314     EVIDENCE_REWRITE = "evidence_constrained_rewrite"
315     HEDGED_REWRITE = "hedged_rewrite"
316     DELETE = "delete_sentence"
317
318
319 class QualityStatus(str, Enum):
320     PASS = "pass"
321     WARNING = "warning"
322     REVISE = "revise"
323
324
325 class QualityIssueType(str, Enum):
326     MISSING_REQUIRED_COMPONENT = "missing_required_component"
327     LEDGER_STYLE_RENDERING = "ledger_style_rendering"
328     REPETITIVE_CAVEAT = "repetitive_caveat"
329     GENERIC_OPENING = "generic_opening"
330     WEAK_FINDING_SELECTION = "weak_finding_selection"
331     ROUTE_DOMINANCE = "route_dominance"
332     UNSUPPORTED_METHOD_INTERPRETATION = "unsupported_method_interpretation"
333
334
335 class EvaluationFieldPolicy(StrictModel):
336     operational_input_paths: list[str] = Field(default_factory=list)
337     held_out_reference_paths: list[str] = Field(default_factory=list)
338     metadata_paths: list[str] = Field(default_factory=list)
339
340
341 class InputStructureProfile(StrictModel):
342     shape: InputShape
343     representation_status: InputRepresentationStatus
344
345     source_paths: list[str] = Field(default_factory=list)
346     row_semantics: str | None = None
347     entity_levels: list[str] = Field(default_factory=list)
348     nested_paths: list[str] = Field(default_factory=list)
349
350     probable_input_fields: list[str] = Field(default_factory=list)
351     probable_reference_fields: list[str] = Field(default_factory=list)
352     probable_metadata_fields: list[str] = Field(default_factory=list)
353
354     heterogeneous_rows_detected: bool = False
355     sparse_flattening_detected: bool = False
356     ambiguity_notes: list[str] = Field(default_factory=list)
357
358     confidence: float = Field(ge=0.0, le=1.0)
359
360
361 class StructuralField(StrictModel):
362     table_name: str
363     path_pattern: str
364     value_types: list[str] = Field(default_factory=list)
365     sample_values: list[str] = Field(default_factory=list)
366     occurrence_count: int = Field(default=1, ge=1)
367
368
369 class SemanticBinding(StrictModel):

```

```

370     binding_id: str
371     table_name: str
372     label: str
373     role: SemanticRole
374     level: SemanticLevel
375     path_pattern: str
376     description: str
377     confidence: float = Field(ge=0.0, le=1.0)
378     evidence_basis: str
379     unit: str | None = None
380     analytical_function: AnalyticalFunction | None = None
381
382
383 class InputSemanticMap(StrictModel):
384     input_shape: InputShape
385     record_description: str
386     bindings: list[SemanticBinding] = Field(default_factory=list)
387     recommended_report_genre: ReportGenre | None = None
388     recommended_communication_task: CommunicationTask | None = None
389     recommended_output_form: OutputForm | None = None
390     recommended_focus_scope: str | None = None
391     report_rationale: str = ""
392     confidence: float = Field(ge=0.0, le=1.0)
393     unresolved_ambiguities: list[str] = Field(default_factory=list)
394
395
396 class CapabilityDefinition(StrictModel):
397     capability: EvidenceCapability
398     supported_input_shapes: list[InputShape]
399
400     requires_numeric_fields: bool = False
401     requires_time_field: bool = False
402     requires_entity_fields: bool = False
403     requires_event_participants: bool = False
404     requires_outcome_field: bool = False
405
406     minimum_observations: int | None = None
407     output_evidence_types: list[str] = Field(default_factory=list)
408
409
410 class GenreQualityAssessment(StrictModel):
411     status: QualityStatus
412     genre: ReportGenre
413
414     required_slots: list[str] = Field(default_factory=list)
415     supported_slots: list[str] = Field(default_factory=list)
416     covered_slots: list[str] = Field(default_factory=list)
417     missing_supported_slots: list[str] = Field(default_factory=list)
418
419     findings: list[str] = Field(default_factory=list)
420     recommendations: list[str] = Field(default_factory=list)
421
422
423 class ColumnProfile(StrictModel):
424     name: str
425     dtype: str
426     semantic_type: str
427
428     missing_count: int
429     missing_rate: float
430     unique_count: int
431
432     sample_values: list[str] = Field(default_factory=list)
433     numeric_summary: dict[str, float | int] = Field(default_factory=dict)
434
435     datetime_parse_rate: float = 0.0
436     candidate_key: bool = False
437     structured_values: bool = False
438
439     constant: bool = False
440     near_constant: bool = False
441     dominant_value_rate: float = 0.0
442
443     suspicious_zero_values: bool = False
444     possible_sentinel_values: bool = False
445     zero_risk: ZeroRisk = ZeroRisk.NONE
446     zero_risk_reason: str | None = None
447
448     quality_warnings: list[str] = Field(default_factory=list)
449
450
451 class TableProfile(StrictModel):
452     table_name: str
453     source_path: str
454
455     row_count: int
456     column_count: int
457     duplicate_row_count: int
458
459     candidate_keys: list[str] = Field(default_factory=list)
460     columns: list[ColumnProfile] = Field(default_factory=list)

```



```

461     warnings: list[str] = Field(default_factory=list)
462
463
464 class DataProfile(StrictModel):
465     fingerprint: str
466     source_paths: list[str]
467     tables: list[TableProfile]
468
469
470 class ColumnMeaning(StrictModel):
471     table_name: str
472     column_name: str
473     inferred_role: str
474     interpretation: str
475     evidence_basis: str
476     confidence: float = Field(ge=0.0, le=1.0)
477     caveat: str | None = None
478
479
480 class ColumnRisk(StrictModel):
481     table_name: str
482     column_name: str
483     risk_type: str
484     explanation: str
485     analytical_consequence: str
486     confidence: float = Field(ge=0.0, le=1.0)
487
488
489 class TableUnderstanding(StrictModel):
490     table_name: str
491     unit_of_observation: str
492     summary: str
493
494     likely_keys: list[str] = Field(default_factory=list)
495     column_meanings: list[ColumnMeaning] = Field(default_factory=list)
496     column_risks: list[ColumnRisk] = Field(default_factory=list)
497
498     quality_findings: list[str] = Field(default_factory=list)
499     usability_notes: list[str] = Field(default_factory=list)
500
501
502 class DataUnderstanding(StrictModel):
503     profile_fingerprint: str
504     dataset_summary: str
505     tables: list[TableUnderstanding]
506
507     cross_table_notes: list[str] = Field(default_factory=list)
508     supported_routes: list[AnalysisRoute] = Field(default_factory=list)
509     uncertain_routes: list[str] = Field(default_factory=list)
510     global_caveats: list[str] = Field(default_factory=list)
511     semantic_map: InputSemanticMap | None = None
512     inferred_task_contract: TaskContractDecision | None = None
513
514
515 class ReportSpecification(StrictModel):
516     intended_audience: str = "A reader seeking a data-science interpretation."
517     report_purpose: str
518
519     genre: ReportGenre = ReportGenre.DATA_SCIENCE_REPORT
520     communication_task: CommunicationTask = CommunicationTask.DATA_SCIENCE_REPORT
521     output_form: OutputForm = OutputForm.MULTI_PARAGRAPH_REPORT
522     focus_scope: str | None = None
523     allow_headings: bool = True
524     max_sentences: int | None = Field(default=None, ge=1)
525     max_paragraphs: int | None = Field(default=None, ge=1)
526     require_complete_sentence: bool = True
527     audience: str = "general analytical reader"
528     perspective: ReportPerspective = ReportPerspective.NEUTRAL
529     realisation_policy: RealisationPolicy = (
530         RealisationPolicy.STRICT_SOURCE_SURFACE
531     )
532     communication_goal: str = (
533         "Summarise the strongest supported findings."
534     )
535
536     target_length_words: int = Field(ge=1, le=2_500)
537     maximum_length_words: int | None = Field(
538         default=None,
539         ge=1,
540         le=2_500,
541     )
542     maximum_main_findings: int | None = Field(default=None, ge=2)
543     maximum_supporting_facts: int | None = Field(default=None, ge=2)
544
545     preferred_sections: list[str] = Field(default_factory=list)
546     required_components: list[ReportComponent] = Field(default_factory=list)
547     required_content_slots: list[str] = Field(default_factory=list)
548     optional_content_slots: list[str] = Field(default_factory=list)
549     prohibited_claim_types: list[str] = Field(default_factory=list)
550
551     selection_source: ReportSelectionSource = ReportSelectionSource.FALLBACK

```

```

552     selection_confidence: float = Field(default=1.0, ge=0.0, le=1.0)
553     unresolved_ambiguities: list[str] = Field(default_factory=list)
554
555     include_negative_findings: bool = True
556     include_methodological_details: bool = True
557
558     prioritisation_rule: str
559
560
561 class NarrativeSlotPlan(StrictModel):
562     slot: NarrativeSlot
563     purpose: str
564     fact_ids: list[str] = Field(default_factory=list)
565     insight_ids: list[str] = Field(default_factory=list)
566     priority: int = Field(ge=1, le=10)
567     minimum_items: int = Field(default=0, ge=0)
568     maximum_items: int | None = Field(default=None, ge=1)
569     paragraph_hint: int = Field(ge=1)
570     connective_hint: str | None = None
571     include_when_supported: bool = True
572     visible_caveat_allowed: bool = True
573
574
575 class NarrativePlan(StrictModel):
576     applies: bool = False
577     style: Literal[
578         "event_recap",
579         "reference_recap",
580         "structured_event_report",
581     ] = "structured_event_report"
582     allow_headings: bool = True
583     target_paragraphs: int = Field(default=4, ge=1)
584     slots: list[NarrativeSlotPlan] = Field(default_factory=list)
585     low_priority_fact_ids: list[str] = Field(default_factory=list)
586     prohibited_narrative_moves: list[str] = Field(default_factory=list)
587     closing_policy: str = ""
588
589
590 class InvestigationTask(StrictModel):
591     task_id: str
592     question: str
593
594     route: AnalysisRoute
595     priority: int = Field(ge=1, le=5)
596
597     table_name: str
598     columns: list[str] = Field(default_factory=list)
599     capability: EvidenceCapability | None = None
600     input_fields: list[str] = Field(default_factory=list)
601     entity_scope: list[str] = Field(default_factory=list)
602     expected_evidence_types: list[str] = Field(default_factory=list)
603
604     target_column: str | None = None
605     target_status: TargetStatus = TargetStatus.UNCONFIRMED
606     prediction_definition: str | None = None
607
608     time_column: str | None = None
609     validation_strategy: ValidationStrategy = ValidationStrategy.NONE
610
611     exposure_column: str | None = None
612     outcome_column: str | None = None
613     confounder_columns: list[str] = Field(default_factory=list)
614
615     required_evidence: list[str] = Field(default_factory=list)
616     claim_permissions: list[ClaimPermission]
617     answerability_note: str
618
619
620 class InsightObjective(StrictModel):
621     objective_id: str
622     question: str
623     preferred_insight_types: list[InsightType] = Field(
624         default_factory=list
625     )
626     relevant_task_ids: list[str] = Field(default_factory=list)
627     priority: str = "main"
628
629
630 class EvidenceQuery(StrictModel):
631     query_id: str
632     task_id: str
633     operation: EvidenceOperation
634     capability: EvidenceCapability
635     evidence_type: str
636
637     semantic_label: str
638     question: str
639     table_name: str
640     semantic_level: SemanticLevel
641
642     value_binding_ids: list[str] = Field(

```

```

643         default_factory=list,
644         description=(
645             "Exact SemanticBinding.binding_id values only; never field paths "
646             "or semantic labels."
647         ),
648     )
649     entity_binding_id: str | None = Field(
650         default=None,
651         description=(
652             "One exact SemanticBinding.binding_id for the entity being compared "
653             "or ranked; never a field path."
654         ),
655     )
656     group_binding_id: str | None = Field(
657         default=None,
658         description=(
659             "One exact SemanticBinding.binding_id for an optional parent group; "
660             "never a field path."
661         ),
662     )
663     context_binding_ids: list[str] = Field(
664         default_factory=list,
665         description=(
666             "Exact SemanticBinding.binding_id values for optional context only; "
667             "never field paths."
668         ),
669     )
670
671     limit: int = Field(default=3, ge=1)
672     descending: bool = True
673
674     recommended_use: RecommendedUse = RecommendedUse.MAIN_FINDING
675     user_relevance: float = Field(default=0.8, ge=0.0, le=1.0)
676     salience: float = Field(default=0.8, ge=0.0, le=1.0)
677
678
679 class ExecutionPlan(StrictModel):
680     objective: str
681     tasks: list[InvestigationTask]
682     route_order: list[AnalysisRoute]
683
684     report_specification: ReportSpecification
685     audit_mode: AuditMode
686
687     insight_objectives: list[InsightObjective] = Field(
688         default_factory=list
689     )
690     evidence_queries: list[EvidenceQuery] = Field(default_factory=list)
691
692     available_capabilities: list[EvidenceCapability] = Field(
693         default_factory=list
694     )
695     selected_capabilities: list[EvidenceCapability] = Field(
696         default_factory=list
697     )
698
699     revision_limit: int = Field(ge=0, le=3)
700     maximum_facts: int | None = Field(default=None, ge=1)
701
702     frozen: bool = True
703     rationale: str
704
705
706 class AnalyticalRecommendation(StrictModel):
707     recommendation_id: str
708     action: str
709     recommendation_type: Literal[
710         "data_cleaning",
711         "methodological_check",
712         "additional_analysis",
713         "validation",
714         "reporting",
715     ]
716     priority: Literal["high", "medium", "low"]
717     justification: str
718     affected_analyses: list[str] = Field(default_factory=list)
719     consequence_if_ignored: str = (
720         "The related analysis may be less reliable or harder to interpret."
721     )
722     confidence: float = Field(default=0.75, ge=0.0, le=1.0)
723
724
725 class EvidenceItem(StrictModel):
726     evidence_id: str
727     route: AnalysisRoute
728     task_ids: list[str]
729
730     capability: EvidenceCapability = EvidenceCapability.DATASET_PROFILE
731     evidence_type: str = "generic_finding"
732     source_paths: list[str] = Field(default_factory=list)
733     entity_scope: list[str] = Field(default_factory=list)

```

```

734 semantic_level: SemanticLevel = SemanticLevel.DATASET
735 semantic_binding_ids: list[str] = Field(default_factory=list)
736 analytical_function: AnalyticalFunction | None = None
737 query_id: str | None = None
738
739 finding: str
740 metrics: dict[str, Any] = Field(default_factory=dict)
741
742 source_tables: list[str] = Field(default_factory=list)
743 source_columns: list[str] = Field(default_factory=list)
744
745 method: str
746 validation_strategy: ValidationStrategy = ValidationStrategy.NONE
747
748 practical_interpretation: str
749 strength_label: str
750
751 limitations: list[str] = Field(default_factory=list)
752 prohibited_interpretations: list[str] = Field(default_factory=list)
753 recommendations: list[AnalyticalRecommendation] = Field(default_factory=list)
754
755 claim_permissions: list[ClaimPermission]
756
757 factual_confidence: float = Field(ge=0.0, le=1.0)
758 methodological_strength: float = Field(ge=0.0, le=1.0)
759 user_relevance: float = Field(ge=0.0, le=1.0)
760 salience: float = Field(ge=0.0, le=1.0)
761
762 recommended_use: RecommendedUse
763 eligible_for_writer: bool = True
764 exclusion_reason: str | None = None
765
766 class EvidenceLedger(StrictModel):
767     fingerprint: str
768     items: list[EvidenceItem]
769     execution_notes: list[str] = Field(default_factory=list)
770
771
772 class FactCandidate(StrictModel):
773     candidate_id: str
774     fact_summary: str
775
776     evidence_ids: list[str]
777     claim_permissions: list[ClaimPermission]
778
779     allowed_interpretations: list[str] = Field(default_factory=list)
780     prohibited_interpretations: list[str] = Field(default_factory=list)
781     required_caveats: list[str] = Field(default_factory=list)
782
783     factual_confidence: float = Field(ge=0.0, le=1.0)
784     methodological_strength: float = Field(ge=0.0, le=1.0)
785     user_relevance: float = Field(ge=0.0, le=1.0)
786     salience: float = Field(ge=0.0, le=1.0)
787
788     recommended_use: RecommendedUse
789     eligible_for_writer: bool = True
790
791
792 class FactCandidateSet(StrictModel):
793     candidates: list[FactCandidate]
794     synthesis_notes: list[str] = Field(default_factory=list)
795
796
797 class FactReview(StrictModel):
798     candidate_id: str
799     decision: ReviewDecision
800     rationale: str
801     required_caveats: list[str] = Field(default_factory=list)
802     prohibited_interpretations: list[str] = Field(default_factory=list)
803
804
805 class VerificationResult(StrictModel):
806     reviews: list[FactReview]
807     overall_notes: list[str] = Field(default_factory=list)
808
809
810 class VerifiedFact(StrictModel):
811     fact_id: str
812     source_candidate_id: str
813
814     verification_method: VerificationMethod = (
815         VerificationMethod.LLM_VERIFIED
816     )
817
818     fact_summary: str
819     evidence_ids: list[str]
820     source_capabilities: list[EvidenceCapability] = Field(
821         default_factory=list
822     )
823
824

```

```

825     structured_values: dict[str, Any] = Field(default_factory=dict)
826     entities: list[str] = Field(default_factory=list)
827
828     claim_permissions: list[ClaimPermission]
829     allowed_interpretations: list[str] = Field(default_factory=list)
830     prohibited_interpretations: list[str] = Field(default_factory=list)
831     required_caveats: list[str] = Field(default_factory=list)
832
833     factual_confidence: float = Field(ge=0.0, le=1.0)
834     methodological_strength: float = Field(ge=0.0, le=1.0)
835     user_relevance: float = Field(ge=0.0, le=1.0)
836     salience: float = Field(ge=0.0, le=1.0)
837
838     recommended_use: RecommendedUse
839
840
841     class RejectedFact(StrictModel):
842         source_candidate_id: str
843         fact_summary: str
844         reason: str
845
846
847     class FactLedger(StrictModel):
848         writer_ready_facts: list[VerifiedFact]
849         rejected_facts: list[RejectedFact] = Field(default_factory=list)
850         verifier_notes: list[str] = Field(default_factory=list)
851
852         deterministically_recovered_fact_ids: list[str] = Field(
853             default_factory=list
854         )
855         coverage_recovery_notes: list[str] = Field(
856             default_factory=list
857         )
858
859     class InsightCandidate(StrictModel):
860         insight_id: str
861
862         statement: str
863         insight_type: InsightType
864         interpretation_level: InterpretationLevel
865         contribution: InsightContribution = (
866             InsightContribution.ANALYTICAL_IMPLICATION
867         )
868
869         source_fact_ids: list[str] = Field(default_factory=list)
870         source_evidence_ids: list[str] = Field(default_factory=list)
871
872         why_it_matters: str | None = Field(
873             default=None,
874             description=(
875                 "For analytical reports, the evidence-bounded implication. For "
876                 "event and descriptive synthesis this may be omitted because the "
877                 "supported relationship itself can provide the reader value."
878             ),
879         )
880         supporting_summary: str
881
882         alternative_explanations: list[str] = Field(default_factory=list)
883         limitations: list[str] = Field(default_factory=list)
884
885         claim_permissions: list[ClaimPermission] = Field(default_factory=list)
886
887         confidence: float = Field(ge=0.0, le=1.0)
888         salience: float = Field(ge=0.0, le=1.0)
889
890         suitable_for_main_report: bool = True
891
892
893     class InsightCandidateSet(StrictModel):
894         candidates: list[InsightCandidate] = Field(default_factory=list)
895         synthesis_notes: list[str] = Field(default_factory=list)
896
897
898     class InsightVerificationRecord(StrictModel):
899         insight_id: str
900         status: InsightVerificationStatus
901
902         verified_statement: str | None = None
903
904         verified_source_fact_ids: list[str] = Field(default_factory=list)
905         verified_source_evidence_ids: list[str] = Field(default_factory=list)
906
907         confidence: float = Field(ge=0.0, le=1.0)
908         salience: float = Field(ge=0.0, le=1.0)
909
910         adds_bounded_synthesis: bool
911         analytical_implication_supported: bool
912         contains_hypothesis: bool
913
914         limitations: list[str] = Field(default_factory=list)

```

```

916     verification_notes: list[str] = Field(default_factory=list)
917
918
919 class InsightVerificationResult(StrictModel):
920     records: list[InsightVerificationRecord] = Field(default_factory=list)
921     verifier_notes: list[str] = Field(default_factory=list)
922
923
924 class VerifiedInsight(StrictModel):
925     insight_id: str
926
927     statement: str
928     insight_type: InsightType
929     interpretation_level: InterpretationLevel
930     contribution: InsightContribution = (
931         InsightContribution.ANALYTICAL_IMPLICATION
932     )
933
934     source_fact_ids: list[str] = Field(default_factory=list)
935     source_evidence_ids: list[str] = Field(default_factory=list)
936     source_capabilities: list[EvidenceCapability] = Field(
937         default_factory=list
938     )
939
940     why_it_matters: str | None = Field(
941         default=None,
942         description=(
943             "The verified analytical implication when the contribution "
944             "requires one; event and descriptive synthesis may omit it."
945         ),
946     )
947     limitations: list[str] = Field(default_factory=list)
948
949     claim_permissions: list[ClaimPermission] = Field(default_factory=list)
950
951     confidence: float = Field(ge=0.0, le=1.0)
952     salience: float = Field(ge=0.0, le=1.0)
953
954     verification_status: InsightVerificationStatus
955
956
957 class InsightRejection(StrictModel):
958     insight_id: str
959     candidate: InsightCandidate
960     reasons: list[str] = Field(default_factory=list)
961
962
963 class InsightVerificationFailure(StrictModel):
964     insight_id: str
965     candidate: InsightCandidate
966     reasons: list[str] = Field(default_factory=list)
967
968
969 class InsightLedger(StrictModel):
970     verified_insights: list[VerifiedInsight] = Field(default_factory=list)
971     hypothesis_only_insights: list[VerifiedInsight] = Field(
972         default_factory=list
973     )
974     rejected_insights: list[InsightRejection] = Field(default_factory=list)
975     unverified_insights: list[InsightVerificationFailure] = Field(
976         default_factory=list
977     )
978     verifier_notes: list[str] = Field(default_factory=list)
979     synthesis_enabled: bool = True
980     fallback_reason: str | None = None
981
982
983 class WriterEvidencePack(StrictModel):
984     user_request: str
985     report_specification: ReportSpecification
986
987     dataset_understanding: DataUnderstanding
988     input_structure: InputStructureProfile | None = None
989     available_capabilities: list[EvidenceCapability] = Field(
990         default_factory=list
991     )
992
993     priority_facts: list[VerifiedFact]
994     supporting_facts: list[VerifiedFact]
995     limitation_facts: list[VerifiedFact]
996
997     evidence_ledger: EvidenceLedger
998
999     insight_ledger: InsightLedger = Field(default_factory=InsightLedger)
1000     priority_verified_insights: list[VerifiedInsight] = Field(
1001         default_factory=list
1002     )
1003     supporting_verified_insights: list[VerifiedInsight] = Field(
1004         default_factory=list
1005     )
1006

```

```

1007     analytical_recommendations: list[AnalyticalRecommendation] = Field(
1008         default_factory=list
1009     )
1010     reader_facing_limitations: list[str] = Field(default_factory=list)
1011     internal_prohibited_interpretations: list[str] = Field(default_factory=list)
1012
1013
1014 class ProfileSupportRecord(StrictModel):
1015     support_id: str
1016     fact_kind: str
1017
1018     table_name: str
1019     column_name: str | None = None
1020
1021     statement: str
1022     structured_values: dict[str, Any] = Field(default_factory=dict)
1023     entities: list[str] = Field(default_factory=list)
1024
1025     claim_permissions: list[ClaimPermission] = Field(
1026         default_factory=lambda: [
1027             ClaimPermission.DESCRPTIVE
1028         ]
1029     )
1030
1031     provenance: str
1032
1033
1034 class ReportComponentAssessment(StrictModel):
1035     component: ReportComponent
1036     covered: bool
1037     supporting_fact_ids: list[str] = Field(default_factory=list)
1038     explanation: str
1039
1040
1041 class SentenceSupport(StrictModel):
1042     sentence_id: str
1043     sentence_text: str
1044
1045     fact_ids: list[str] = Field(default_factory=list)
1046     evidence_ids: list[str] = Field(default_factory=list)
1047     profile_support_ids: list[str] = Field(default_factory=list)
1048     insight_ids: list[str] = Field(default_factory=list)
1049
1050     interpretation_level: InterpretationLevel = InterpretationLevel.FINDING
1051
1052     support_type: SupportType
1053
1054
1055 class WriterSentenceDraft(StrictModel):
1056     text: str = Field(min_length=1)
1057
1058     fact_ids: list[str] = Field(default_factory=list)
1059
1060     insight_ids: list[str] = Field(default_factory=list)
1061     interpretation_level: InterpretationLevel = InterpretationLevel.FINDING
1062
1063     support_type: SupportType
1064
1065
1066 class WriterSectionDraft(StrictModel):
1067     heading: str = Field(min_length=1)
1068
1069     sentences: list[WriterSentenceDraft] = Field(default_factory=list)
1070
1071
1072 class WriterAgentDraft(StrictModel):
1073     title: str = Field(min_length=1)
1074     title_fact_ids: list[str] = Field(default_factory=list)
1075
1076     sections: list[WriterSectionDraft] = Field(default_factory=list)
1077
1078     writer_notes: list[str] = Field(default_factory=list)
1079
1080
1081 class WriterOutput(StrictModel):
1082     title: str
1083     markdown: str
1084     title_fact_ids: list[str] = Field(default_factory=list)
1085
1086     sentence_support: list[SentenceSupport]
1087
1088     selected_fact_ids: list[str] = Field(default_factory=list)
1089     omitted_fact_ids: list[str] = Field(default_factory=list)
1090
1091     writer_notes: list[str] = Field(default_factory=list)
1092
1093     writer_mode: Literal[
1094         "llm_writer",
1095         "deterministic_fallback",
1096         "deterministic_short_form_writer",
1097         "auditor_repaired",

```



```

1098     ] = "llm_writer"
1099
1100     eligible_for_primary_evaluation: bool = True
1101     quality_revision_round: int = Field(default=0, ge=0, le=1)
1102     quality_revision_summary: str | None = None
1103
1104
1105 class ExternalFact(StrictModel):
1106     fact_id: str
1107     fact_text: str
1108     entities: list[str] = Field(default_factory=list)
1109     numbers: list[float] = Field(default_factory=list)
1110     validity: str = "current"
1111
1112
1113 class ExternalTruthSource(StrictModel):
1114     source_id: str
1115     source_name: str
1116     source_type: str
1117     trust_level: str
1118
1119     source_uri: str | None = None
1120     retrieved_at: str | None = None
1121
1122     scope: list[str] = Field(default_factory=list)
1123     facts: list[ExternalFact] = Field(default_factory=list)
1124
1125
1126 class AuditAnnotation(StrictModel):
1127     annotation_id: str
1128
1129     sentence: str
1130     text_span: str
1131
1132     error_type: ErrorType
1133     subtype: str
1134     severity: Severity
1135
1136     explanation: str
1137     correction_goal: str
1138
1139     fact_ids: list[str] = Field(default_factory=list)
1140     evidence_ids: list[str] = Field(default_factory=list)
1141     external_fact_ids: list[str] = Field(default_factory=list)
1142     profile_support_ids: list[str] = Field(default_factory=list)
1143     insight_ids: list[str] = Field(default_factory=list)
1144
1145     confidence: float = Field(ge=0.0, le=1.0)
1146
1147
1148 class RepairCandidate(StrictModel):
1149     repair_id: str
1150     replacement_text: str
1151
1152     strategy: RepairStrategy
1153
1154     supporting_fact_ids: list[str] = Field(default_factory=list)
1155     supporting_evidence_ids: list[str] = Field(default_factory=list)
1156     supporting_insight_ids: list[str] = Field(default_factory=list)
1157
1158     factual_support_score: float = Field(ge=0.0, le=1.0)
1159     meaning_preservation_score: float = Field(ge=0.0, le=1.0)
1160     readability_score: float = Field(ge=0.0, le=1.0)
1161     residual_hallucination_risk: float = Field(ge=0.0, le=1.0)
1162
1163
1164 class SentenceRepair(StrictModel):
1165     sentence_id: str
1166     original_sentence: str
1167     annotation_ids: list[str]
1168
1169     candidates: list[RepairCandidate]
1170     preferred_repair_id: str | None = None
1171     selection_reason: str
1172
1173
1174 class ReportQualityAssessment(StrictModel):
1175     status: QualityStatus
1176
1177     request_responsiveness: float = Field(ge=0.0, le=1.0)
1178     finding_selection: float = Field(ge=0.0, le=1.0)
1179     coherence: float = Field(ge=0.0, le=1.0)
1180     concision: float = Field(ge=0.0, le=1.0)
1181     caveat_integration: float = Field(ge=0.0, le=1.0)
1182     data_science_interpretation: float = Field(ge=0.0, le=1.0)
1183
1184     findings: list[str] = Field(default_factory=list)
1185     recommendations: list[str] = Field(default_factory=list)
1186
1187
1188 class AuditRepairProposal(StrictModel):

```



```

1189     annotations: list[AuditAnnotation] = Field(default_factory=list)
1190     repairs: list[SentenceRepair] = Field(default_factory=list)
1191
1192     recommended_decision: AuditDecision
1193     residual_risk: str
1194
1195     quality_assessment: ReportQualityAssessment
1196     revision_instructions: list[str] = Field(default_factory=list)
1197
1198
1199 class ReportPatch(StrictModel):
1200     sentence_id: str
1201     original_text: str
1202     replacement_text: str
1203     operation: Literal["replace", "delete"]
1204     selected_repair_id: str
1205
1206
1207 class SupportMapPatch(StrictModel):
1208     sentence_id: str
1209     sentence_text: str
1210     added_profile_support_ids: list[str]
1211     reason: str
1212
1213
1214 class AuditReport(StrictModel):
1215     mode: AuditMode
1216     decision: AuditDecision
1217     release_status: ReleaseStatus
1218
1219     annotations: list[AuditAnnotation] = Field(default_factory=list)
1220     applied_patches: list[ReportPatch] = Field(default_factory=list)
1221     support_map_patches: list[SupportMapPatch] = Field(default_factory=list)
1222
1223     factual_sentence_count: int
1224     supported_sentence_count: int
1225     support_rate: float = Field(ge=0.0, le=1.0)
1226
1227     residual_risk: str
1228     revision_instructions: list[str] = Field(default_factory=list)
1229     quality_assessment: ReportQualityAssessment
1230     component_assessments: list[ReportComponentAssessment] = Field(default_factory=list)
1231     methodological_warnings: list[str] = Field(default_factory=list)
1232
1233     revision_round: int = 0
1234
1235
1236 class RunManifest(StrictModel):
1237     run_id: str
1238     created_at: str
1239
1240     input_paths: list[str]
1241     request: str
1242     fingerprint: str
1243
1244     use_llm: bool
1245     audit_mode: AuditMode
1246
1247     models: dict[str, str]
1248     input_representation_status: InputRepresentationStatus = (
1249         InputRepresentationStatus.VALID
1250     )
1251     report_genre: ReportGenre = ReportGenre.DATA_SCIENCE_REPORT
1252     communication_task: CommunicationTask = CommunicationTask.DATA_SCIENCE_REPORT
1253     output_form: OutputForm = OutputForm.MULTI_PARAGRAPH_REPORT
1254     focus_scope: str | None = None
1255     task_contract: TaskContractDecision | None = None
1256
1257
1258 class PipelineResult(StrictModel):
1259     run_id: str
1260
1261     inferred_task_contract: TaskContractDecision | None = None
1262     resolved_task_contract: TaskContractDecision | None = None
1263     task_contract_agreement: dict[str, Any] = Field(default_factory=dict)
1264     profile: DataProfile
1265     input_structure: InputStructureProfile | None = None
1266     structural_catalog: list[StructuralField] = Field(default_factory=list)
1267     evaluation_field_policy: EvaluationFieldPolicy = Field(
1268         default_factory=EvaluationFieldPolicy
1269     )
1270     understanding: DataUnderstanding
1271     execution_plan: ExecutionPlan
1272
1273     evidence_ledger: EvidenceLedger
1274     fact_candidates: FactCandidateSet
1275     verification: VerificationResult
1276     fact_ledger: FactLedger
1277     writer_evidence_pack: WriterEvidencePack
1278
1279     raw_writer_output: WriterOutput

```

```

1280     quality_revised_writer_output: WriterOutput | None = None
1281     final_writer_output: WriterOutput
1282
1283     initial_audit: AuditReport
1284     final_audit: AuditReport
1285
1286     repair_rounds_used: int
1287     release_status: ReleaseStatus
1288     approved_for_release: bool
1289     primary_evaluation_eligible: bool = True
1290     primary_evaluation_reason: str | None = None
1291     genre_quality_assessment: GenreQualityAssessment | None = None
1292
1293     insight_ledger: InsightLedger = Field(default_factory=InsightLedger)
1294
1295
1296 # Compatibility aliases for notebooks using the original names.
1297 ClaimCandidate = FactCandidate
1298 ClaimCandidateSet = FactCandidateSet
1299 ClaimReview = FactReview
1300 VerifiedClaim = VerifiedFact
1301 RejectedClaim = RejectedFact
1302 ClaimLedger = FactLedger
1303 ReportDraft = WriterOutput

```

Listing 44: P44: table2text_pydanticai/src/table2text/structure.py

```

1  """Interpret input shape, nested records, field roles, and reference isolation."""
2
3  from __future__ import annotations
4
5  import copy
6  import re
7  from collections.abc import Mapping, Sequence
8  from pathlib import Path
9  from typing import Any
10
11  from .schemas import (
12      EvaluationFieldPolicy,
13      InputRepresentationStatus,
14      InputShape,
15      InputStructureProfile,
16      StructuralField,
17  )
18
19
20  PARTICIPANT_CONTAINER_NAMES = {
21      "competitors",
22      "entities",
23      "participants",
24      "sides",
25      "teams",
26  }
27  IDENTITY_FIELD_NAMES = {
28      "display_name",
29      "entity_name",
30      "full_name",
31      "name",
32      "participant_name",
33      "team_name",
34  }
35  OUTCOME_FIELD_NAMES = {
36      "final_score",
37      "points",
38      "pts",
39      "result",
40      "score",
41      "team_runs",
42      "total",
43  }
44  PARTICIPANT_LINE_SUFFIXES = {
45      "line",
46      "record",
47      "summary",
48  }
49  PARTICIPANT_LINE_METRIC_NAMES = {
50      "final_score",
51      "points",
52      "pts",
53      "score",
54      "team_runs",
55      "team_points",
56      "total",
57  }
58
59
60  def normalise_key(value: str) -> str:
61      return re.sub(r"^[a-z0-9]+", "_", value.casefold()).strip("_")
62

```

```

63
64 def _looks_numeric(value: Any) -> bool:
65     if isinstance(value, bool) or value is None:
66         return False
67     if isinstance(value, (int, float)):
68         return True
69     if isinstance(value, str):
70         return bool(
71             re.fullmatch(
72                 r"[+-]?(\d+(\.\d*)?|\.\d+)",
73                 value.strip().replace(",", ""))
74         )
75     )
76     return False
77
78
79 def _mapping_looks_like_collection(value: Any) -> bool:
80     if not isinstance(value, Mapping) or len(value) < 2:
81         return False
82
83     children = [child for child in value.values() if isinstance(child, Mapping)]
84     return len(children) >= 2 and len(children) == len(value)
85
86
87 def _contains_identity_or_metrics(value: Mapping[str, Any]) -> bool:
88     keys = {normalise_key(str(key)) for key in value}
89     if keys & IDENTITY_FIELD_NAMES:
90         return True
91
92     stack: List[tuple[Any, int]] = [(value, 0)]
93     while stack:
94         current, depth = stack.pop()
95         if depth > 4 or not isinstance(current, Mapping):
96             continue
97         for key, child in current.items():
98             key_name = normalise_key(str(key))
99             if key_name in OUTCOME_FIELD_NAMES and _looks_numeric(child):
100                 return True
101             if isinstance(child, Mapping):
102                 stack.append((child, depth + 1))
103     return False
104
105
106 def find_participant_container(
107     payload: Any,
108 ) -> tuple[str, Mapping[str, Any]] | None:
109     stack: List[tuple[str, Any, int]] = [("", payload, 0)]
110     fallback: tuple[str, Mapping[str, Any]] | None = None
111
112     while stack:
113         path, current, depth = stack.pop()
114         if depth > 4 or not isinstance(current, Mapping):
115             continue
116
117         for key, child in current.items():
118             child_path = f"{path}.{key}" if path else str(key)
119             if isinstance(child, Mapping):
120                 if _mapping_looks_like_collection(child) and all(
121                     _contains_identity_or_metrics(item)
122                     for item in child.values()
123                     if isinstance(item, Mapping)
124                 ):
125                     candidate = (child_path, child)
126                     if normalise_key(str(key)) in PARTICIPANT_CONTAINER_NAMES:
127                         return candidate
128                     fallback = fallback or candidate
129                 stack.append((child_path, child, depth + 1))
130
131     return fallback
132
133
134 def _participant_line_records(payload: Any) -> List[tuple[str, Mapping[str, Any]]]:
135     if not isinstance(payload, Mapping):
136         return []
137
138     records: List[tuple[str, Mapping[str, Any]]] = []
139     for key, child in payload.items():
140         if not isinstance(child, Mapping):
141             continue
142         normalised_key = normalise_key(str(key))
143         key_parts = normalised_key.split(".")
144         if len(key_parts) < 2 or key_parts[-1] not in PARTICIPANT_LINE_SUFFIXES:
145             continue
146         child_keys = {normalise_key(str(child_key)) for child_key in child}
147         has_result = "result" in child_keys
148         has_score = bool(child_keys & PARTICIPANT_LINE_METRIC_NAMES)
149         has_identity = bool(child_keys & IDENTITY_FIELD_NAMES)
150         if has_result and (has_score or has_identity):
151             records.append((str(key), child))
152
153     return records

```

```

154
155
156 def has_paired_participant_line_records(payload: Any) -> bool:
157     records = _participant_line_records(payload)
158     if len(records) < 2:
159         return False
160     prefixes = {
161         normalise_key(key).rsplit("_", 1)[0]
162         for key, _ in records
163         if "_" in normalise_key(key)
164     }
165     return len(prefixes) >= 2
166
167
168 def nested_paths(payload: Any, *, limit: int = 240) -> list[str]:
169     paths: list[str] = []
170
171     def visit(value: Any, prefix: str, depth: int) -> None:
172         if len(paths) >= limit or depth > 6:
173             return
174
175         if isinstance(value, Mapping):
176             items = list(value.items())
177             if (
178                 len(items) > 8
179                 and all(isinstance(child, Mapping) for _, child in items)
180             ):
181                 wildcard = f"{prefix}.*" if prefix else "*"
182                 paths.append(wildcard)
183                 visit(items[0][1], wildcard, depth + 1)
184                 return
185
186             for key, child in items:
187                 path = f"{prefix}.{key}" if prefix else str(key)
188                 paths.append(path)
189                 visit(child, path, depth + 1)
190             elif isinstance(value, list) and value:
191                 path = f"{prefix}[]"
192                 paths.append(path)
193                 visit(value[0], path, depth + 1)
194
195     visit(payload, "", 0)
196     return list(dict.fromkeys(paths))[:limit]
197
198
199 def _key_overlap(rows: list[Mapping[str, Any]]) -> float:
200     if len(rows) < 2:
201         return 1.0
202
203     overlaps: list[float] = []
204     for left, right in zip(rows, rows[1:]):
205         left_keys = set(left)
206         right_keys = set(right)
207         union = left_keys | right_keys
208         overlaps.append(len(left_keys & right_keys) / max(len(union), 1))
209     return sum(overlaps) / len(overlaps)
210
211
212 def _homogeneous_record_mapping(value: Mapping[str, Any]) -> bool:
213     if len(value) < 2 or not all(isinstance(child, Mapping) for child in value.values()):
214         return False
215
216     children = [dict(child) for child in value.values()]
217     return _key_overlap(children) >= 0.6
218
219
220 def build_structural_catalog(
221     structured_inputs: Mapping[str, Any],
222     *,
223     maximum_fields: int = 500,
224     maximum_samples: int = 3,
225 ) -> list[StructuralField]:
226     """Build a compact, value-bearing schema from sanitized structured input."""
227
228     entries: dict[tuple[str, str], dict[str, Any]] = {}
229
230     def value_type(value: Any) -> str:
231         if value is None:
232             return "null"
233         if isinstance(value, bool):
234             return "boolean"
235         if isinstance(value, int):
236             return "integer"
237         if isinstance(value, float):
238             return "number"
239         if isinstance(value, str):
240             return "string"
241         return type(value).__name__
242
243     def sample_value(value: Any) -> str:
244         text = str(value)

```

```

245     return text if len(text) <= 120 else text[:117] + "...
246
247 def add_leaf(table_name: str, path: str, value: Any) -> None:
248     key = (table_name, path)
249     entry = entries.setdefault(
250         key,
251         {
252             "types": set(),
253             "samples": [],
254             "count": 0,
255         },
256     )
257     entry["types"].add(value_type(value))
258     entry["count"] += 1
259     sample = sample_value(value)
260     if sample not in entry["samples"] and len(entry["samples"]) < maximum_samples:
261         entry["samples"].append(sample)
262
263 def visit(table_name: str, value: Any, prefix: str, depth: int) -> None:
264     if depth > 12 or len(entries) >= maximum_fields:
265         return
266
267     if isinstance(value, Mapping):
268         if _homogeneous_record_mapping(value):
269             wildcard = f"{prefix}.*" if prefix else "*"
270             for child in list(value.values())[:50]:
271                 visit(table_name, child, wildcard, depth + 1)
272             return
273
274         for raw_key, child in value.items():
275             child_path = f"{prefix}.{raw_key}" if prefix else str(raw_key)
276             visit(table_name, child, child_path, depth + 1)
277         return
278
279     if isinstance(value, list):
280         wildcard = f"{prefix}.*" if prefix else "*"
281         for child in value[:50]:
282             visit(table_name, child, wildcard, depth + 1)
283         if not value:
284             add_leaf(table_name, prefix, value)
285         return
286
287     add_leaf(table_name, prefix, value)
288
289 for table_name, payload in structured_inputs.items():
290     visit(str(table_name), payload, "", 0)
291
292 return [
293     StructuralField(
294         table_name=table_name,
295         path_pattern=path,
296         value_types=sorted(entry["types"]),
297         sample_values=entry["samples"],
298         occurrence_count=entry["count"],
299     )
300     for (table_name, path), entry in sorted(entries.items())
301 ][:maximum_fields]
302
303
304 def _probable_reference_paths(payload: Any, event_like: bool) -> list[str]:
305     if not event_like or not isinstance(payload, Mapping):
306         return []
307
308     return [
309         str(key)
310         for key, value in payload.items()
311         if isinstance(value, str) and len(value.strip()) >= 500
312     ]
313
314
315 def _probable_metadata_paths(payload: Any) -> list[str]:
316     if not isinstance(payload, Mapping):
317         return []
318
319     metadata: list[str] = []
320     for key, value in payload.items():
321         name = normalise_key(str(key))
322         if name.endswith("_id") or name in {"id", "source", "source_url", "url"}:
323             if not isinstance(value, (Mapping, list)):
324                 metadata.append(str(key))
325     return metadata
326
327
328 def _sparse_flattening_risk(payload: Any) -> bool:
329     if not isinstance(payload, Mapping):
330         return False
331
332     mappings = [value for value in payload.values() if isinstance(value, Mapping)]
333     scalars = [
334         value
335         for value in payload.values()

```

```

336     if not isinstance(value, (Mapping, list))
337 ]
338 if len(mappings) < 2 or not scalars:
339     return False
340
341 nested_key_sets = [set(value) for value in mappings if value]
342 if len(nested_key_sets) < 2:
343     return False
344 union = set().union(*nested_key_sets)
345 intersection = set.intersection(*nested_key_sets)
346 return len(union) >= 4 and len(intersection) / max(len(union), 1) < 0.5
347
348
349 def _shape_for_payload(payload: Any) -> tuple[InputShape, str | None, list[str]]:
350     participant_container = find_participant_container(payload)
351     if participant_container is not None and isinstance(payload, Mapping):
352         return InputShape.EVENT_RECORD, "one event", ["event", "participant", "entity"]
353
354     if has_paired_participant_line_records(payload):
355         return InputShape.EVENT_RECORD, "one event", ["event", "participant", "entity"]
356
357     if isinstance(payload, list):
358         rows = [item for item in payload if isinstance(item, Mapping)]
359         if len(rows) == len(payload):
360             nested = any(
361                 any(isinstance(value, (Mapping, list)) for value in row.values())
362                 for row in rows
363             )
364             if nested:
365                 return InputShape.ENTITY_COLLECTION, "one entity per record", ["entity"]
366             return InputShape.FLAT_TABLE, "one observation per row", []
367
368     if isinstance(payload, Mapping):
369         values = list(payload.values())
370         if values and all(isinstance(value, list) for value in values):
371             lengths = {len(value) for value in values}
372             if len(lengths) == 1 and not all(
373                 not value or isinstance(value[0], Mapping)
374                 for value in values
375             ):
376                 return InputShape.FLAT_TABLE, "one observation per row", []
377             if all(
378                 not value or isinstance(value[0], Mapping)
379                 for value in values
380             ):
381                 return (
382                     InputShape.ENTITY_COLLECTION,
383                     "one entity per nested record",
384                     ["entity"],
385                 )
386             if any(isinstance(value, (Mapping, list)) for value in payload.values()):
387                 return InputShape.NESTED_RECORD, "one nested record", []
388             return InputShape.NESTED_RECORD, "one record", []
389
390     return InputShape.AMBIGUOUS, None, []
391
392
393 def _path_parts(path: str) -> list[str]:
394     if path.strip() in {"", "$"}:
395         return []
396     return [part for part in path.split(".") if part]
397
398
399 def get_path(payload: Any, path: str) -> tuple[bool, Any]:
400     if path.strip() in {"", "$"}:
401         return True, payload
402     current = payload
403     for part in _path_parts(path):
404         if not isinstance(current, Mapping) or part not in current:
405             return False, None
406         current = current[part]
407     return True, current
408
409
410 def set_path(target: dict[str, Any], path: str, value: Any) -> None:
411     parts = _path_parts(path)
412     if not parts:
413         if isinstance(value, Mapping):
414             target.clear()
415             target.update(copy.deepcopy(dict(value)))
416         return
417     current = target
418     for part in parts[:-1]:
419         child = current.get(part)
420         if not isinstance(child, dict):
421             child = {}
422         current[part] = child
423     current = child
424     current[parts[-1]] = copy.deepcopy(value)
425
426

```

```

427 def remove_path(payload: Any, path: str) -> None:
428     if not isinstance(payload, dict):
429         return
430     parts = _path_parts(path)
431     if not parts:
432         payload.clear()
433         return
434     current: Any = payload
435     for part in parts[:-1]:
436         if not isinstance(current, dict) or part not in current:
437             return
438         current = current[part]
439     if isinstance(current, dict):
440         current.pop(parts[-1], None)
441
442
443 def apply_field_policy(
444     payload: Any,
445     policy: EvaluationFieldPolicy,
446 ) -> Any:
447     if not isinstance(payload, Mapping):
448         return copy.deepcopy(payload)
449
450     if policy.operational_input_paths:
451         if any(path.strip() in {"", "$"} for path in policy.operational_input_paths):
452             operational = copy.deepcopy(dict(payload))
453         else:
454             operational: dict[str, Any] = {}
455             for path in policy.operational_input_paths:
456                 found, value = get_path(payload, path)
457                 if found:
458                     set_path(opperational, path, value)
459         else:
460             operational = copy.deepcopy(dict(payload))
461
462     for path in [
463         *policy.held_out_reference_paths,
464         *policy.metadata_paths,
465     ]:
466         remove_path(opperational, path)
467     return operational
468
469
470 def inspect_and_filter_payload(
471     *,
472     payload: Any,
473     source_path: Path,
474     field_policy: EvaluationFieldPolicy | None,
475 ) -> tuple[Any, InputStructureProfile, EvaluationFieldPolicy]:
476     supplied_policy = field_policy or EvaluationFieldPolicy()
477     overlap = set(supplied_policy.operational_input_paths) & set(
478         supplied_policy.held_out_reference_paths
479     )
480     if overlap:
481         raise ValueError(
482             "Operational and held-out paths overlap: " + ", ".join(sorted(overlap))
483         )
484
485     shape, row_semantics, entity_levels = _shape_for_payload(payload)
486     participant_container = find_participant_container(payload)
487     probable_references = _probable_reference_paths(
488         payload,
489         participant_container is not None,
490     )
491     probable_metadata = _probable_metadata_paths(payload)
492     ambiguity_notes: list[str] = []
493
494     undeclared_references = sorted(
495         set(probable_references)
496         - set(supplied_policy.held_out_reference_paths)
497     )
498     if undeclared_references:
499         ambiguity_notes.append(
500             "Long narrative fields paired with structured event data were "
501             "quarantined as probable evaluation references; declare them "
502             "explicitly for primary evaluation: "
503             + ", ".join(undeclared_references)
504         )
505
506     effective_policy = EvaluationFieldPolicy(
507         operational_input_paths=supplied_policy.operational_input_paths,
508         held_out_reference_paths=list(
509             dict.fromkeys(
510                 [
511                     *supplied_policy.held_out_reference_paths,
512                     *undeclared_references,
513                 ]
514             )
515         ),
516         metadata_paths=supplied_policy.metadata_paths,
517     )

```



```

518     operational = apply_field_policy(payload, effective_policy)
519
520     missing_declared_paths = [
521         path
522         for path in [
523             *supplied_policy.operational_input_paths,
524             *supplied_policy.held_out_reference_paths,
525             *supplied_policy.metadata_paths,
526         ]
527         if not get_path(payload, path)[0]
528     ]
529     if missing_declared_paths:
530         ambiguity_notes.append(
531             "Declared field paths were absent: "
532             + ", ".join(sorted(missing_declared_paths))
533         )
534
535     if shape == InputShape.AMBIGUOUS:
536         status = InputRepresentationStatus.INVALID
537         confidence = 0.2
538     elif undeclared_references:
539         status = InputRepresentationStatus.AMBIGUOUS
540         confidence = 0.75
541     elif missing_declared_paths:
542         status = InputRepresentationStatus.VALID_WITH_WARNINGS
543         confidence = 0.8
544     else:
545         status = InputRepresentationStatus.VALID
546         confidence = 0.98 if shape == InputShape.EVENT_RECORD else 0.95
547
548     heterogeneous = False
549     if isinstance(payload, list):
550         rows = [item for item in payload if isinstance(item, Mapping)]
551         heterogeneous = bool(rows and _key_overlap(rows) < 0.6)
552
553     top_level_fields = list(payload) if isinstance(payload, Mapping) else []
554     probable_inputs = [
555         str(field)
556         for field in top_level_fields
557         if str(field) not in effective_policy.held_out_reference_paths
558         and str(field) not in effective_policy.metadata_paths
559     ]
560
561     profile = InputStructureProfile(
562         shape=shape,
563         representation_status=status,
564         source_paths=[str(source_path)],
565         row_semantics=row_semantics,
566         entity_levels=entity_levels,
567         nested_paths=nested_paths(operational),
568         probable_input_fields=probable_inputs,
569         probable_reference_fields=probable_references,
570         probable_metadata_fields=probable_metadata,
571         heterogeneous_rows_detected=heterogeneous,
572         sparse_flattening_detected=sparse_flattening_risk(payload),
573         ambiguity_notes=ambiguity_notes,
574         confidence=confidence,
575     )
576     return operational, profile, effective_policy
577
578
579 def combine_structure_profiles(
580     profiles: Sequence[InputStructureProfile],
581 ) -> InputStructureProfile:
582     if not profiles:
583         return InputStructureProfile(
584             shape=InputShape.AMBIGUOUS,
585             representation_status=InputRepresentationStatus.INVALID,
586             row_semantics=None,
587             ambiguity_notes=["No input structure could be inspected."],
588             confidence=0.0,
589         )
590     if len(profiles) == 1:
591         return profiles[0]
592
593     shapes = {profile.shape for profile in profiles}
594     status_order = {
595         InputRepresentationStatus.VALID: 0,
596         InputRepresentationStatus.VALID_WITH_WARNINGS: 1,
597         InputRepresentationStatus.AMBIGUOUS: 2,
598         InputRepresentationStatus.INVALID: 3,
599     }
600     worst = max(
601         (profile.representation_status for profile in profiles),
602         key=status_order.__getitem__,
603     )
604     combined_shape = shapes.pop() if len(shapes) == 1 else InputShape.AMBIGUOUS
605     if combined_shape == InputShape.AMBIGUOUS and worst != InputRepresentationStatus.INVALID:
606         worst = InputRepresentationStatus.AMBIGUOUS
607
608     return InputStructureProfile(

```



```

609     shape=combined_shape,
610     representation_status=worst,
611     source_paths=[path for profile in profiles for path in profile.source_paths],
612     row_semantics=(
613         profiles[0].row_semantics
614         if len({profile.row_semantics for profile in profiles}) == 1
615         else "multiple input representations"
616     ),
617     entity_levels=list(
618         dict.fromkeys(level for profile in profiles for level in profile.entity_levels)
619     ),
620     nested_paths=list(
621         dict.fromkeys(path for profile in profiles for path in profile.nested_paths)
622     ),
623     probable_input_fields=list(
624         dict.fromkeys(
625             field for profile in profiles for field in profile.probable_input_fields
626         )
627     ),
628     probable_reference_fields=list(
629         dict.fromkeys(
630             field
631             for profile in profiles
632             for field in profile.probable_reference_fields
633         )
634     ),
635     probable_metadata_fields=list(
636         dict.fromkeys(
637             field
638             for profile in profiles
639             for field in profile.probable_metadata_fields
640         )
641     ),
642     heterogeneous_rows_detected=any(
643         profile.heterogeneous_rows_detected for profile in profiles
644     ),
645     sparse_flattening_detected=any(
646         profile.sparse_flattening_detected for profile in profiles
647     ),
648     ambiguity_notes=[
649         note for profile in profiles for note in profile.ambiguity_notes
650     ],
651     confidence=min(profile.confidence for profile in profiles),
652 )

```

Listing 45: P45: table2text_pydanticai/src/table2text/task_contracts.py

```

1  """Infer bounded communication contracts from requests and structured inputs."""
2
3  from __future__ import annotations
4
5  import re
6  from collections.abc import Mapping
7  from typing import Any
8
9  from .schemas import (
10     CommunicationTask,
11     InputSemanticMap,
12     InputShape,
13     InputStructureProfile,
14     OutputForm,
15     ReportGenre,
16     ReportSelectionSource,
17     TaskContractDecision,
18     TaskFamilyHint,
19 )
20
21 INFERENCE_CONFIDENCE_THRESHOLD = 0.75
22
23
24 def _normalise_key(value: Any) -> str:
25     return re.sub(r"^[a-z0-9]+", "_", str(value).casefold()).strip("_")
26
27
28 def _source_payloads(structured_inputs: Mapping[str, Any]) -> list[Any]:
29     payloads: list[Any] = []
30     for payload in structured_inputs.values():
31         if (
32             isinstance(payload, Mapping)
33             and payload.get("__table2text_benchmark_example__")
34             and "source_payload" in payload
35         ):
36             payloads.append(payload.get("source_payload"))
37         else:
38             payloads.append(payload)
39     return payloads
40
41
42

```

```

43 def _walk_mappings(value: Any, *, depth: int = 0):
44     if depth > 8:
45         return
46     if isinstance(value, Mapping):
47         yield value
48         for child in value.values():
49             yield from _walk_mappings(child, depth=depth + 1)
50     elif isinstance(value, list):
51         for child in value[:50]:
52             yield from _walk_mappings(child, depth=depth + 1)
53
54
55 def _contains_nonempty_field(payloads: list[Any], field_name: str) -> bool:
56     target = _normalise_key(field_name)
57     return any(
58         _normalise_key(key) == target
59         and value is not None
60         and value != ""
61         and value != []
62         and value != {}
63         for payload in payloads
64         for mapping in _walk_mappings(payload)
65         for key, value in mapping.items()
66     )
67
68
69 def _has_table(payloads: list[Any]) -> bool:
70     return any(
71         _normalise_key(key) in {"table", "table_array"}
72         and isinstance(value, list)
73         and bool(value)
74         for payload in payloads
75         for mapping in _walk_mappings(payload)
76         for key, value in mapping.items()
77     )
78
79
80 def _has_highlighted_region(payloads: list[Any]) -> bool:
81     has_highlights = any(
82         _normalise_key(key) in {"highlighted_cells", "highlighted_cell_ids"}
83         and isinstance(value, list)
84         and bool(value)
85         for payload in payloads
86         for mapping in _walk_mappings(payload)
87         for key, value in mapping.items()
88     )
89     return has_highlights and _has_table(payloads)
90
91
92 def _has_attribute_record(payloads: list[Any]) -> bool:
93     if _contains_nonempty_field(payloads, "meaning_representation"):
94         return True
95     return any(
96         {"attribute_name", "attribute_value"}.issubset(
97             {_normalise_key(key) for key in mapping}
98         )
99         for payload in payloads
100         for mapping in _walk_mappings(payload)
101     )
102
103
104 def _has_triple_record(payloads: list[Any]) -> bool:
105     if _contains_nonempty_field(payloads, "triples") or _contains_nonempty_field(
106         payloads, "tripleset"
107     ):
108         return True
109     triple_key_sets = (
110         {"subject", "relation", "object"},
111         {"subject", "predicate", "object"},
112         {"head", "relation", "tail"},
113     )
114     return any(
115         any(required.issubset({_normalise_key(key) for key in mapping}) for required in triple_key_sets)
116         for payload in payloads
117         for mapping in _walk_mappings(payload)
118     )
119
120
121 def _has_table_question(payloads: list[Any]) -> bool:
122     return _has_table(payloads) and _contains_nonempty_field(payloads, "question")
123
124
125 def _requested_output_form(request: str) -> OutputForm | None:
126     if re.search(r"\b(exactly|only) one (?:concise )?sentence\b", request, re.I):
127         return OutputForm.ONE_SENTENCE
128     if re.search(r"\bone sentence\b", request, re.I):
129         return OutputForm.ONE_SENTENCE
130     if re.search(r"\b(direct answer|answer (?:the )?question)\b", request, re.I):
131         return OutputForm.DIRECT_ANSWER
132     if re.search(r"\bone or two (?:fluent )?sentences\b|\bshort text\b", request, re.I):
133         return OutputForm.SHORT_TEXT

```

```

134     if re.search(r"\b(?:single )?paragraph\b", request, re.I):
135         return OutputForm.PARAGRAPH
136     if re.search(r"\b(multi[- ]paragraph|detailed report|full report)\b", request, re.I):
137         return OutputForm.MULTI_PARAGRAPH_REPORT
138     return None
139
140
141 def _explicit_task(request: str) -> CommunicationTask | None:
142     if re.search(r"\b(highlighted|selected|focused) (?:table)?(?:cell|cells|region)\b", request, re.I):
143         return CommunicationTask.FOCUSED_TABLE_DESCRIPTION
144     if re.search(r"\b(attributes?|meaning representation)\b", request, re.I):
145         return CommunicationTask.ATTRIBUTE_VERBALISATION
146     if re.search(r"\b(triples?|subject[- ]predicate[- ]object)\b", request, re.I):
147         return CommunicationTask.TRIPLE_VERBALISATION
148     if re.search(r"\b(event|game|match) (?:report|recap|summary)\b", request, re.I):
149         return CommunicationTask.EVENT_REPORT
150     if re.search(r"\b(question answering|answer (?:the )?question)\b", request, re.I):
151         return CommunicationTask.TABLE_QUESTION_ANSWERING
152     if re.search(r"\b(table entailment|entailed by the table)\b", request, re.I):
153         return CommunicationTask.TABLE_ENTAILMENT
154     if re.search(r"\bdataset overview\b", request, re.I):
155         return CommunicationTask.DATASET_OVERVIEW
156     if re.search(r"\b(data[- ]science report|statistical analysis)\b", request, re.I):
157         return CommunicationTask.DATA_SCIENCE_REPORT
158     return None
159
160
161 def _task_family(communication_task: CommunicationTask) -> TaskFamilyHint:
162     mapping = {
163         CommunicationTask.DATA_SCIENCE_REPORT: TaskFamilyHint.DATA_SCIENCE_REPORT,
164         CommunicationTask.DATASET_OVERVIEW: TaskFamilyHint.DATASET_OVERVIEW,
165         CommunicationTask.EVENT_REPORT: TaskFamilyHint.EVENT_REPORT,
166         CommunicationTask.FOCUSED_TABLE_DESCRIPTION: (
167             TaskFamilyHint.HIGHLIGHTED_TABLE_DESCRIPTION
168         ),
169         CommunicationTask.TABLE_ENTAILMENT: TaskFamilyHint.LOGICAL_TABLE_STATEMENT,
170         CommunicationTask.TABLE_QUESTION_ANSWERING: (
171             TaskFamilyHint.TABLE_QUESTION_ANSWERING
172         ),
173         CommunicationTask.ATTRIBUTE_VERBALISATION: (
174             TaskFamilyHint.ATTRIBUTE_VERBALISATION
175         ),
176         CommunicationTask.TRIPLE_VERBALISATION: (
177             TaskFamilyHint.TRIPLE_VERBALISATION
178         ),
179     }
180     return mapping.get(communication_task, TaskFamilyHint.CUSTOM)
181
182
183 def _defaults_for_task(
184     communication_task: CommunicationTask,
185 ) -> tuple[ReportGenre, OutputForm, str | None]:
186     if communication_task == CommunicationTask.EVENT_REPORT:
187         return (
188             ReportGenre.EVENT_REPORT,
189             OutputForm.MULTI_PARAGRAPH_REPORT,
190             "event_recap",
191         )
192     if communication_task == CommunicationTask.FOCUSED_TABLE_DESCRIPTION:
193         return (
194             ReportGenre.DATASET_OVERVIEW,
195             OutputForm.ONE_SENTENCE,
196             "highlighted_cells",
197         )
198     if communication_task in {
199         CommunicationTask.ATTRIBUTE_VERBALISATION,
200         CommunicationTask.TRIPLE_VERBALISATION,
201     }:
202         return ReportGenre.DATASET_OVERVIEW, OutputForm.SHORT_TEXT, None
203     if communication_task == CommunicationTask.TABLE_QUESTION_ANSWERING:
204         return ReportGenre.DATASET_OVERVIEW, OutputForm.DIRECT_ANSWER, None
205     if communication_task == CommunicationTask.TABLE_ENTAILMENT:
206         return ReportGenre.DATASET_OVERVIEW, OutputForm.ONE_SENTENCE, None
207     if communication_task == CommunicationTask.DATASET_OVERVIEW:
208         return (
209             ReportGenre.DATASET_OVERVIEW,
210             OutputForm.MULTI_PARAGRAPH_REPORT,
211             None,
212         )
213     return (
214         ReportGenre.DATA_SCIENCE_REPORT,
215         OutputForm.MULTI_PARAGRAPH_REPORT,
216         None,
217     )
218
219
220 def infer_task_contract(
221     *,
222     request: str,
223     structured_inputs: Mapping[str, Any],
224     input_structure: InputStructureProfile | None,

```

```

225     semantic_map: InputSemanticMap | None = None,
226 ) -> TaskContractDecision:
227     """Infer a communication contract without dataset IDs or reference text."""
228
229     payloads = _source_payloads(structured_inputs)
230     explicit_task = _explicit_task(request)
231     evidence: list[str] = []
232     ambiguities: list[str] = []
233
234     if explicit_task is not None:
235         communication_task = explicit_task
236         selection_source = ReportSelectionSource.EXPLICIT_USER_REQUEST
237         confidence = 1.0
238         evidence.append("The user request explicitly names the communication task.")
239     elif _has_highlighted_region(payloads):
240         communication_task = CommunicationTask.FOCUSED_TABLE_DESCRIPTION
241         selection_source = ReportSelectionSource.STRUCTURED_INFERENCE
242         confidence = 0.98
243         evidence.append("The operational source contains a table and a non-empty highlighted region.")
244     elif _has_attribute_record(payloads):
245         communication_task = CommunicationTask.ATTRIBUTE_VERBALISATION
246         selection_source = ReportSelectionSource.STRUCTURED_INFERENCE
247         confidence = 0.97
248         evidence.append("The operational source contains an attribute-oriented meaning representation.")
249     elif _has_triple_record(payloads):
250         communication_task = CommunicationTask.TRIPLE_VERBALISATION
251         selection_source = ReportSelectionSource.STRUCTURED_INFERENCE
252         confidence = 0.97
253         evidence.append("The operational source contains subject-relation-object records.")
254     elif _has_table_question(payloads):
255         communication_task = CommunicationTask.TABLE_QUESTION_ANSWERING
256         selection_source = ReportSelectionSource.STRUCTURED_INFERENCE
257         confidence = 0.95
258         evidence.append("The operational source pairs a table with a question.")
259     elif (
260         input_structure is not None
261         and input_structure.shape == InputShape.EVENT_RECORD
262         and input_structure.confidence >= 0.7
263     ):
264         communication_task = CommunicationTask.EVENT_REPORT
265         selection_source = ReportSelectionSource.STRUCTURED_INFERENCE
266         confidence = input_structure.confidence
267         evidence.append("The input-structure profile identifies one bounded event record.")
268     elif (
269         semantic_map is not None
270         and semantic_map.confidence >= 0.7
271         and semantic_map.recommended_communication_task is not None
272     ):
273         communication_task = semantic_map.recommended_communication_task
274         selection_source = ReportSelectionSource.STRUCTURED_INFERENCE
275         confidence = semantic_map.confidence
276         evidence.append("The validated semantic map recommends the communication task.")
277     elif re.search(
278         r"\b(understand|explore|strongest findings|key findings|report (?:(its |the )?findings))\b",
279         request,
280         re.I,
281     ):
282         communication_task = CommunicationTask.DATA_SCIENCE_REPORT
283         selection_source = ReportSelectionSource.FALLBACK
284         confidence = 0.8
285         evidence.append("The generic request asks for analytical findings.")
286     else:
287         communication_task = CommunicationTask.DATASET_OVERVIEW
288         selection_source = ReportSelectionSource.FALLBACK
289         confidence = 0.65
290         evidence.append("No specialised communication task was established.")
291         ambiguities.append("The intended communication task is not explicit in the request or structure.")
292
293     report_genre, default_output, focus_scope = _defaults_for_task(
294         communication_task
295     )
296     if (
297         semantic_map is not None
298         and semantic_map.confidence >= INFERENCE_CONFIDENCE_THRESHOLD
299     ):
300         if semantic_map.recommended_report_genre is not None:
301             report_genre = semantic_map.recommended_report_genre
302         if semantic_map.recommended_output_form is not None:
303             default_output = semantic_map.recommended_output_form
304         if semantic_map.recommended_focus_scope is not None:
305             focus_scope = semantic_map.recommended_focus_scope
306
307     requested_output = _requested_output_form(request)
308     output_form = requested_output or default_output
309     if requested_output is not None:
310         evidence.append("The user request explicitly constrains the output form.")
311
312     if (
313         communication_task == CommunicationTask.FOCUSED_TABLE_DESCRIPTION
314         and not _has_highlighted_region(payloads)
315     ):

```

```

316         ambiguities.append("A focused-table description was requested, but no highlighted region was detected.")
317         confidence = min(confidence, 0.6)
318     if (
319         communication_task == CommunicationTask.EVENT_REPORT
320         and (input_structure is None or input_structure.shape != InputShape.EVENT_RECORD)
321     ):
322         ambiguities.append("An event report was requested, but the structure was not confidently classified as an event.")
323     confidence = min(confidence, 0.7)
324
325     return TaskContractDecision(
326         task_family=_task_family(communication_task),
327         report_genre=report_genre,
328         communication_task=communication_task,
329         output_form=output_form,
330         focus_scope=focus_scope,
331         selection_source=selection_source,
332         confidence=confidence,
333         evidence=evidence,
334         unresolved_ambiguities=ambiguities,
335     )
336
337 def resolve_task_contract(
338     *,
339     inferred: TaskContractDecision,
340     selected_genre: ReportGenre,
341     genre_source: ReportSelectionSource,
342     genre_confidence: float,
343     configured_communication_task: CommunicationTask | None,
344     configured_output_form: OutputForm | None,
345     configured_focus_scope: str | None,
346 ) -> TaskContractDecision:
347     """Merge explicit/configured fields over a validated inferred contract."""
348
349     communication_task = (
350         configured_communication_task or inferred.communication_task
351     )
352     _, default_output, default_focus = _defaults_for_task(communication_task)
353     output_form = configured_output_form or (
354         inferred.output_form
355         if configured_communication_task is None
356         else default_output
357     )
358     focus_scope = configured_focus_scope
359     if focus_scope is None:
360         if configured_communication_task is None:
361             focus_scope = inferred.focus_scope
362         elif communication_task == CommunicationTask.EVENT_REPORT:
363             focus_scope = default_focus
364
365     configured = any(
366         value is not None
367         for value in (
368             configured_communication_task,
369             configured_output_form,
370             configured_focus_scope,
371         )
372     )
373     source = (
374         ReportSelectionSource.EXPERIMENT_CONFIGURATION
375         if configured and genre_source != ReportSelectionSource.EXPLICIT_USER_REQUEST
376         else genre_source
377     )
378     confidence = (
379         1.0
380         if configured_communication_task is not None
381         and configured_output_form is not None
382         else min(inferred.confidence, genre_confidence)
383     )
384     evidence = list(inferred.evidence)
385     if configured:
386         evidence.append("Configured task-contract fields take precedence over inferred values.")
387
388     return TaskContractDecision(
389         task_family=_task_family(communication_task),
390         report_genre=selected_genre,
391         communication_task=communication_task,
392         output_form=output_form,
393         focus_scope=focus_scope,
394         selection_source=source,
395         confidence=confidence,
396         evidence=list(dict.fromkeys(evidence)),
397         unresolved_ambiguities=inferred.unresolved_ambiguities,
398     )
399
400 def task_contract_agreement(
401     inferred: TaskContractDecision,
402     resolved: TaskContractDecision,
403 ) -> dict[str, Any]:

```

```

406     fields = {
407         "task_family": inferred.task_family == resolved.task_family,
408         "report_genre": inferred.report_genre == resolved.report_genre,
409         "communication_task": (
410             inferred.communication_task == resolved.communication_task
411         ),
412         "output_form": inferred.output_form == resolved.output_form,
413         "focus_scope": inferred.focus_scope == resolved.focus_scope,
414     }
415     return {
416         "fields": fields,
417         "exact_match": all(fields.values()),
418         "inferred_confidence": inferred.confidence,
419         "inferred_source": inferred.selection_source.value,
420         "resolved_source": resolved.selection_source.value,
421     }

```

Listing 46: P46: table2text_pydanticai/src/table2text/workflow.py

```

1  """Orchestrate the complete evidence-to-report workflow and persist artifacts."""
2
3  from __future__ import annotations
4
5  import asyncio
6  import json
7  import re
8  from datetime import datetime, timezone
9  from pathlib import Path
10 from typing import Any, Callable
11
12 from pydantic_ai import UsageLimits
13
14 from .agents import (
15     AgentDependencies,
16     build_auditor_agent,
17     build_data_understanding_agent,
18     build_evidence_agent,
19     build_insight_synthesis_agent,
20     build_insight_verifier_agent,
21     build_orchestrator_agent,
22     build_verifier_agent,
23     build_writer_agent,
24     empty_insight_ledger,
25     event_report_requested,
26     fallback_execution_plan,
27     fallback_understanding,
28     materialise_insight_ledger,
29     validate_insight_verification,
30 )
31 from .analytics import execute_plan
32 from .capabilities import (
33     available_capabilities,
34     normalise_event_evidence_queries,
35 )
36 from .audit import (
37     accept_writer_quality_revision,
38     apply_repair_proposal,
39     apply_support_map_patches,
40     assess_report_component_coverage,
41     assess_genre_quality,
42     augment_fact_ledger_for_report_coverage,
43     assess_report_components,
44     build_profile_support_registry,
45     build_writer_content_requirements,
46     build_writer_evidence_pack,
47     compact_json,
48     decide_release_status,
49     deterministic_audit,
50     deterministic_fact_candidate_scaffold,
51     empty_fact_candidate_enrichment,
52     fallback_audit_proposal,
53     fallback_fact_candidates,
54     fallback_verification,
55     fallback_writer,
56     finalise_fact_ledger,
57     json_safe,
58     materialise_writer_output,
59     merge_fact_candidate_scaffold,
60     merge_audit_proposal,
61     normalise_strength_label,
62     repair_spurious_missing_evidence_rejections,
63     scope_fact_ledger_for_genre,
64     validate_writer_output,
65 )
66 from .config import Settings
67 from .data import load_data, profile_data
68 from .narrative import build_event_narrative_plan
69 from .structure import build_structural_catalog
70 from .task_contracts import (

```

```

71     INFERENCE_CONFIDENCE_THRESHOLD,
72     infer_task_contract,
73     resolve_task_contract,
74     task_contract_agreement,
75 )
76 from .schemas import (
77     AuditDecision,
78     AuditMode,
79     AuditRepairProposal,
80     AuditReport,
81     AnalysisRoute,
82     ClaimPermission,
83     CommunicationTask,
84     DataUnderstanding,
85     EvaluationFieldPolicy,
86     EvidenceCapability,
87     ExecutionPlan,
88     ExternalTruthSource,
89     FactCandidateSet,
90     FactLedger,
91     InputSemanticMap,
92     InsightCandidateSet,
93     InsightLedger,
94     InsightObjective,
95     InsightType,
96     InsightVerificationResult,
97     InvestigationTask,
98     NarrativePlan,
99     PipelineResult,
100    ProfileSupportRecord,
101    QualityStatus,
102    ReportComponent,
103    ReportGenre,
104    ReportSelectionSource,
105    RealisationPolicy,
106    InputShape,
107    InputRepresentationStatus,
108    OutputForm,
109    ReleaseStatus,
110    RunManifest,
111    TaskContractDecision,
112    VerificationResult,
113    VerifiedFact,
114    WriterAgentDraft,
115    WriterEvidencePack,
116    WriterOutput,
117 )
118
119
120 def infer_required_report_components(
121     request: str,
122 ) -> list[ReportComponent]:
123     normalised = request.lower()
124     general_understanding_request = bool(
125         re.search(
126             r"\b("
127             r"understand|"
128             r"overview|"
129             r"summarise|summarize|"
130             r"report findings|"
131             r"strongest findings|"
132             r"key findings|"
133             r"explore"
134             r")\b",
135             normalised,
136         )
137     )
138
139     if general_understanding_request:
140         return [
141             ReportComponent.DATASET_OVERVIEW,
142             ReportComponent.DATA_QUALITY,
143             ReportComponent.STRONGEST_RELATIONSHIPS,
144             ReportComponent.LIMITATIONS_NEXT_STEPS,
145         ]
146
147     return []
148
149
150 EVENT_GENRES = {
151     ReportGenre.EVENT_REPORT,
152     ReportGenre.SPORTS_GAME_REPORT,
153 }
154 EVENT_CAPABILITIES = {
155     EvidenceCapability.EVENT_OUTCOME,
156     EvidenceCapability.ENTITY_PERFORMANCE,
157     EvidenceCapability.RANKING,
158     EvidenceCapability.GROUP_COMPARISON,
159 }
160
161

```



```

162 def build_orchestrator_prompt_context(
163     *,
164     understanding: DataUnderstanding,
165     input_structure: Any | None,
166     structural_catalog: list[Any],
167 ) -> dict[str, Any]:
168     """Expose semantic IDs to the planner without competing raw paths."""
169
170     understanding_payload = understanding.model_dump(mode="json")
171     semantic_map = understanding.semantic_map
172     if semantic_map is None:
173         return {
174             "understanding": understanding_payload,
175             "input_structure": input_structure,
176             "structural_catalog": structural_catalog,
177             "semantic_binding_catalog": [],
178         }
179
180     understanding_payload.pop("semantic_map", None)
181     structure_payload = (
182         input_structure.model_dump(mode="json")
183         if hasattr(input_structure, "model_dump")
184         else input_structure
185     )
186     if isinstance(structure_payload, dict):
187         structure_payload = {
188             **structure_payload,
189             "nested_paths": [],
190         }
191
192     return {
193         "understanding": understanding_payload,
194         "input_structure": structure_payload,
195         "structural_catalog": [],
196         "semantic_binding_catalog": [
197             {
198                 "binding_id": binding.binding_id,
199                 "table_name": binding.table_name,
200                 "label": binding.label,
201                 "role": binding.role.value,
202                 "level": binding.level.value,
203                 "analytical_function": (
204                     binding.analytical_function.value
205                     if binding.analytical_function is not None
206                     else None
207                 ),
208                 "unit": binding.unit,
209             }
210             for binding in semantic_map.bindings
211         ],
212     }
213
214
215 def resolve_report_genre(
216     *,
217     request: str,
218     planned_genre: ReportGenre,
219     configured_genre: ReportGenre | None,
220     input_structure: Any | None = None,
221     semantic_map: InputSemanticMap | None = None,
222     inferred_contract: TaskContractDecision | None = None,
223 ) -> tuple[ReportGenre, ReportSelectionSource, float]:
224     if event_report_requested(request):
225         return (
226             ReportGenre.EVENT_REPORT,
227             ReportSelectionSource.EXPLICIT_USER_REQUEST,
228             1.0,
229         )
230     if re.search(r"\bdata[- ]science report\b", request, re.IGNORECASE):
231         return (
232             ReportGenre.DATA_SCIENCE_REPORT,
233             ReportSelectionSource.EXPLICIT_USER_REQUEST,
234             1.0,
235         )
236     if re.search(r"\bdataset overview\b", request, re.IGNORECASE):
237         return (
238             ReportGenre.DATASET_OVERVIEW,
239             ReportSelectionSource.EXPLICIT_USER_REQUEST,
240             1.0,
241         )
242     if configured_genre is not None:
243         return (
244             configured_genre,
245             ReportSelectionSource.EXPERIMENT_CONFIGURATION,
246             1.0,
247         )
248
249     if (
250         inferred_contract is not None
251         and inferred_contract.confidence >= INFERENCE_CONFIDENCE_THRESHOLD
252     ):

```



```

253     return (
254         inferred_contract.report_genre,
255         inferred_contract.selection_source,
256         inferred_contract.confidence,
257     )
258
259     semantic_event = bool(
260         semantic_map is not None
261         and semantic_map.confidence >= 0.7
262         and (
263             semantic_map.input_shape == InputShape.EVENT_RECORD
264             or semantic_map.recommended_report_genre in EVENT_GENRES
265         )
266     )
267     structural_event = bool(
268         input_structure is not None
269         and input_structure.shape == InputShape.EVENT_RECORD
270         and input_structure.confidence >= 0.7
271     )
272     if semantic_event or structural_event:
273         return (
274             ReportGenre.EVENT_REPORT,
275             ReportSelectionSource.STRUCTURED_INFERENCE,
276             (
277                 semantic_map.confidence
278                 if semantic_event and semantic_map is not None
279                 else input_structure.confidence
280             ),
281         )
282
283     if re.search(
284         r"\b(understand|explore|strongest findings|key findings|"
285         r"report(?:its |the )?findings)\b",
286         request,
287         re.IGNORECASE,
288     ):
289         return (
290             ReportGenre.DATA_SCIENCE_REPORT,
291             ReportSelectionSource.FALLBACK,
292             0.8,
293         )
294
295     if planned_genre in EVENT_GENRES:
296         return (
297             ReportGenre.EVENT_REPORT,
298             ReportSelectionSource.STRUCTURED_INFERENCE,
299             0.85,
300         )
301     return (
302         planned_genre,
303         ReportSelectionSource.STRUCTURED_INFERENCE,
304         0.85,
305     )
306
307 def report_contract_fields(
308     genre: ReportGenre,
309 ) -> dict[str, Any]:
310     if genre in EVENT_GENRES:
311         return {
312             "communication_goal": (
313                 "Communicate the verified event result, leading performances "
314                 "major participant contrasts and any supplied event-context "
315                 "or score-progression evidence."
316             ),
317             "preferred_sections": [
318                 "Event overview",
319                 "Score progression",
320                 "Key performances",
321                 "Participant contrasts",
322                 "Scope limitations",
323             ],
324             "required_components": [],
325             "required_content_slots": [
326                 "event_result",
327                 "event_context",
328                 "participant_record_context",
329                 "event_status",
330                 "score_progression",
331                 "event_sequence",
332                 "leading_performance",
333                 "main_contrast",
334                 "scope_limitations",
335             ],
336             "optional_content_slots": [
337                 "secondary_performance",
338             ],
339             "prohibited_claim_types": [
340                 "unsupported_chronology",
341                 "unsupported_milestone",
342                 "unsupported_historical_significance",
343         }

```

```

344         "unsupported_causality",
345     ],
346     "include_negative_findings": False,
347     "include_methodological_details": False,
348 }
349
350 if genre == ReportGenre.DATASET_OVERVIEW:
351     return {
352         "required_content_slots": ["dataset_scope"],
353         "optional_content_slots": ["material_data_quality_issue"],
354         "prohibited_claim_types": ["unsupported_causality"],
355     }
356
357     return {
358         "required_content_slots": [
359             "dataset_scope",
360             "material_data_quality_issue",
361             "strongest_analytical_finding",
362             "limitation",
363         ],
364         "optional_content_slots": [],
365         "prohibited_claim_types": ["unsupported_causality"],
366     }
367
368
369 def task_contract_fields(
370     *,
371     genre: ReportGenre,
372     communication_task: CommunicationTask | None,
373     output_form: OutputForm | None,
374     focus_scope: str | None,
375 ) -> dict[str, Any]:
376     if communication_task is None:
377         if genre in EVENT_GENRES:
378             communication_task = CommunicationTask.EVENT_REPORT
379         elif genre == ReportGenre.DATASET_OVERVIEW:
380             communication_task = CommunicationTask.DATASET_OVERVIEW
381         else:
382             communication_task = CommunicationTask.DATA_SCIENCE_REPORT
383
384     if output_form is None:
385         output_form = OutputForm.MULTI_PARAGRAPH_REPORT
386
387     if communication_task == CommunicationTask.FOCUSED_TABLE_DESCRIPTION:
388         return {
389             "communication_task": communication_task,
390             "output_form": output_form,
391             "focus_scope": focus_scope or "focused_table_region",
392             "allow_headings": False,
393             "max_sentences": 1
394             if output_form == OutputForm.ONE_SENTENCE
395             else None,
396             "max_paragraphs": 1,
397             "require_complete_sentence": True,
398             "realisation_policy": RealisationPolicy.CONCISE_TABLE_PROPOSITION,
399             "communication_goal": (
400                 "Express the concise table-local relation conveyed by the "
401                 "selected cell or focused table region, using conservative "
402                 "cell-context wording only when the relation is ambiguous."
403             ),
404             "target_length_words": 40,
405             "maximum_length_words": 80,
406             "preferred_sections": [],
407             "required_components": [],
408             "required_content_slots": [
409                 "focused_table_region",
410             ],
411             "optional_content_slots": [
412                 "focused_table_relation",
413                 "focused_cell_context",
414             ],
415             "prohibited_claim_types": [
416                 "dataset_overview",
417                 "data_quality",
418                 "missingness",
419                 "correlation",
420                 "modelling",
421                 "unrelated_table_cells",
422                 "markdown_headings",
423             ],
424             "include_negative_findings": False,
425             "include_methodological_details": False,
426             "prioritisation_rule": (
427                 "Prefer a verified focused-table relation insight when the "
428                 "supplied page, section, row, header and source-text context "
429                 "support it. When multiple highlighted numeric values share "
430                 "the same header, prefer a scoped lower/higher contrast "
431                 "among the highlighted values over a table-wide rank claim; "
432                 "when a highlighted group label is paired with a highlighted "
433                 "record-like summary value, prefer the natural record "
434                 "proposition over a mechanical cell description; when a "

```

```

435         "highlighted text cell appears under a meaningful column in "
436         "a list page, prefer the natural list-page proposition over "
437         "unrelated row details; when highlighted record groups are "
438         "supplied, preserve all grouped highlighted records that "
439         "contribute to the focused proposition; "
440         "otherwise give the narrow selected-cell description."
441     ),
442 }
443
444 if communication_task in {
445     CommunicationTask.ATTRIBUTE_VERBALISATION,
446     CommunicationTask.TRIPLE_VERBALISATION,
447 }:
448     task_label = (
449         "attributes"
450         if communication_task == CommunicationTask.ATTRIBUTE_VERBALISATION
451         else "triples"
452     )
453     return {
454         "communication_task": communication_task,
455         "output_form": output_form,
456         "focus_scope": focus_scope or "structured_record",
457         "allow_headings": False,
458         "max_sentences": 1
459         if output_form == OutputForm.ONE_SENTENCE
460         else 2,
461         "max_paragraphs": 1,
462         "require_complete_sentence": True,
463         "realisation_policy": RealisationPolicy.NATURAL_REFERENCE_STYLE,
464         "communication_goal": (
465             f"Express all and only the supplied {task_label} as concise, "
466             "fluent natural language without adding unsupported details."
467         ),
468         "target_length_words": 35,
469         "maximum_length_words": 90,
470         "preferred_sections": [],
471         "required_components": [],
472         "required_content_slots": [
473             "structured_record_verbalisation",
474         ],
475         "optional_content_slots": [],
476         "prohibited_claim_types": [
477             "dataset_overview",
478             "data_quality",
479             "missingness",
480             "correlation",
481             "modelling",
482             "unsupported_attribute",
483             "unsupported_entity",
484             "markdown_headings",
485         ],
486         "include_negative_findings": False,
487         "include_methodological_details": False,
488         "prioritisation_rule": (
489             "Use every supplied attribute or triple that contributes to "
490             "the requested short verbalisation. Prefer natural phrasing "
491             "over mechanical key/value listing, but do not introduce "
492             "facts absent from the structured record. For triple "
493             "verbalisation, preserve the source relation order where "
494             "natural and use compact predicate-preserving wording; do "
495             "not add interpretive paraphrases or formatting changes that "
496             "are not required by grammar. Preserve source digits, "
497             "decimals and units exactly."
498         ),
499     )
500
501 if (
502     communication_task == CommunicationTask.EVENT_REPORT
503     and focus_scope in {"event_recap", "reference_recap"}
504 ):
505     reference_recap_style = focus_scope == "reference_recap"
506     return {
507         "communication_task": communication_task,
508         "output_form": output_form,
509         "focus_scope": focus_scope,
510         "allow_headings": not reference_recap_style,
511         "max_sentences": None,
512         "max_paragraphs": None,
513         "require_complete_sentence": True,
514         "realisation_policy": RealisationPolicy.EVENT_RECAP_STYLE,
515         "communication_goal": (
516             "Write a reference-style event recap from verified source "
517             "evidence: lead with the result, then integrate supplied "
518             "context, score progression, leading performances, major "
519             "participant contrasts and follow-up context when verified."
520         )
521         if reference_recap_style
522         else (
523             "Write a coherent event recap from verified source "
524             "evidence: lead with the result, then integrate supplied "
525

```

```

526         "context, score progression, leading performances and "
527         "major participant contrasts. Keep the report event-first, "
528         "not a dataset profile."
529     )
530 ),
531 "preferred_sections": [],
532 if reference_recap_style
533 else [
534     "Event overview",
535     "Score progression",
536     "Key performances",
537     "Participant contrasts",
538 ],
539 "required_components": [],
540 "required_content_slots": [
541     "event_result",
542     "event_context",
543     "participant_record_context",
544     "event_status",
545     "score_progression",
546     "event_sequence",
547     "leading_performance",
548     "main_contrast",
549 ],
550 "optional_content_slots": [
551     "secondary_performance",
552 ],
553 "prohibited_claim_types": [
554     "markdown_headings",
555     "dataset_overview",
556     "data_quality",
557     "missingness",
558     "correlation",
559     "modelling",
560     "unsupported_chronology",
561     "unsupported_milestone",
562     "unsupported_historical_significance",
563     "unsupported_causality",
564 ],
565 "include_negative_findings": False,
566 "include_methodological_details": False,
567 "prioritisation_rule": (
568     (
569         "Optimise for a fluent event recap: include the most "
570         "salient verified result, context, progression, "
571         "performances and contrasts. Keep factual caveats "
572         "internal unless needed to avoid a misleading claim. Do "
573         "not add a generic limitations section for reference-style "
574         "event recaps."
575     )
576     if reference_recap_style
577     else (
578         "Optimise for a fluent event recap: include the most "
579         "salient verified result, context, progression, "
580         "performances and contrasts before secondary rankings. "
581         "Keep any visible caveat concise and event-scoped."
582     )
583 ),
584 }
585
586 return {
587     "communication_task": communication_task,
588     "output_form": output_form,
589     "focus_scope": focus_scope,
590     "realisation_policy": RealisationPolicy.STRICT_SOURCE_SURFACE,
591 }
592
593
594 def infer_event_focus_scope(
595     *,
596     selected_genre: ReportGenre,
597     selection_source: ReportSelectionSource,
598     configured_communication_task: CommunicationTask | None,
599     configured_output_form: OutputForm | None,
600     configured_focus_scope: str | None,
601     input_structure: Any | None,
602     semantic_map: InputSemanticMap | None,
603 ) -> str | None:
604     if configured_focus_scope is not None:
605         return configured_focus_scope
606     if selected_genre not in EVENT_GENRES:
607         return None
608     if configured_communication_task is not None or configured_output_form is not None:
609         return None
610     if selection_source != ReportSelectionSource.STRUCTURED_INFERENCE:
611         return None
612
613     semantic_event = bool(
614         semantic_map is not None
615         and semantic_map.confidence >= 0.7
616         and (

```

```

617         semantic_map.input_shape == InputShape.EVENT_RECORD
618         or semantic_map.recommended_report_genre in EVENT_GENRES
619     )
620 )
621 structural_event = bool(
622     input_structure is not None
623     and input_structure.shape == InputShape.EVENT_RECORD
624     and input_structure.confidence >= 0.7
625 )
626 if semantic_event or structural_event:
627     return "event_recap"
628 return None
629
630
631 SHORT_FORM_COMMUNICATION_TASKS = {
632     CommunicationTask.FOCUSED_TABLE_DESCRIPTION,
633     CommunicationTask.ATTRIBUTE_VERBALISATION,
634     CommunicationTask.TRIPLE_VERBALISATION,
635 }
636
637
638 def is_short_form_realisation_task(
639     communication_task: CommunicationTask | None,
640     output_form: OutputForm | None,
641 ) -> bool:
642     return bool(
643         communication_task in SHORT_FORM_COMMUNICATION_TASKS
644         or output_form
645         in {
646             OutputForm.ONE_SENTENCE,
647             OutputForm.SHORT_TEXT,
648             OutputForm.DIRECT_ANSWER,
649         }
650         and communication_task
651         in SHORT_FORM_COMMUNICATION_TASKS
652     )
653
654
655 def add_focused_table_capability_task(
656     *,
657     plan: ExecutionPlan,
658     profile: Any,
659     capabilities: list[EvidenceCapability],
660     enable_insight_synthesis: bool,
661 ) -> ExecutionPlan:
662     if (
663         plan.report_specification.communication_task
664         != CommunicationTask.FOCUSED_TABLE_DESCRIPTION
665     ):
666         return plan
667
668     if EvidenceCapability.FOCUSED_TABLE_REGION not in capabilities:
669         return plan
670
671     table_name = next(
672         (
673             table.table_name
674             for table in profile.tables
675             if any(column.name == "is_highlighted" for column in table.columns)
676         ),
677         profile.tables[0].table_name if profile.tables else "",
678     )
679     if not table_name:
680         return plan
681
682     columns = [
683         "cell_value",
684         "is_highlighted",
685         "row_index",
686         "column_index",
687         "page_title",
688         "section_title",
689         "is_header",
690     ]
691     task = InvestigationTask(
692         task_id="TASK_FOCUSED_TABLE_REGION",
693         question=(
694             "Which selected table cell or focused region should be described, "
695             "and what local row, header and table context supports it?"
696         ),
697         route=AnalysisRoute.DESCRPTIVE,
698         priority=5,
699         table_name=table_name,
700         columns=columns,
701         capability=EvidenceCapability.FOCUSED_TABLE_REGION,
702         input_fields=columns,
703         expected_evidence_types=[
704             "focused_table_region",
705             "focused_cell_context",
706         ],
707         required_evidence=[

```

```

708         "selected cell value",
709         "row context",
710         "header or table context when available",
711     ],
712     claim_permissions=[ClaimPermission.DESCRPTIVE],
713     answerability_note=(
714         "The output should describe the focused table region only; broad "
715         "dataset profiling and unrelated analytical routes are out of scope."
716     ),
717 )
718 insight_objectives = []
719 if enable_insight_synthesis:
720     insight_objectives = [
721         InsightObjective(
722             objective_id="INSIGHT_FOCUSED_TABLE_RELATION",
723             question=(
724                 "What concise table-local proposition is expressed by "
725                 "the highlighted cell or focused table region, using "
726                 "only supplied page, section, row, header, highlighted "
727                 "cell and source-text context, including scoped "
728                 "lower/higher contrast among highlighted values when "
729                 "supported and record-style propositions when a "
730                 "highlighted group label is paired with a highlighted "
731                 "record-like value, or list-page propositions when a "
732                 "highlighted text cell is selected from a list table?"
733             ),
734             preferred_insight_types=[
735                 InsightType.NARRATIVE_SUMMARY,
736             ],
737             relevant_task_ids=[
738                 task.task_id,
739             ],
740             priority="main",
741         )
742     ]
743
744 return plan.model_copy(
745     update={
746         "tasks": [task],
747         "route_order": [AnalysisRoute.DESCRPTIVE],
748         "selected_capabilities": [
749             EvidenceCapability.FOCUSED_TABLE_REGION,
750         ],
751         "evidence_queries": [],
752         "insight_objectives": insight_objectives,
753         "rationale": (
754             "The controller selected a focused-table contract, so the "
755             "plan is restricted to selected-cell evidence and local "
756             "table-context relation synthesis."
757         ),
758     }
759 )
760
761 def add_structured_record_capability_task(
762     *,
763     plan: ExecutionPlan,
764     profile: Any,
765     capabilities: list[EvidenceCapability],
766     enable_insight_synthesis: bool,
767 ) -> ExecutionPlan:
768     if plan.report_specification.communication_task not in {
769         CommunicationTask.ATTRIBUTE_VERBALISATION,
770         CommunicationTask.TRIPLE_VERBALISATION,
771     }:
772         return plan
773
774     if (
775         EvidenceCapability.STRUCTURED_RECORD_VERBALISATION
776         not in capabilities
777     ):
778         return plan
779
780     table_name = next(
781         (
782             table.table_name
783             for table in profile.tables
784             if {
785                 "attribute_name",
786                 "attribute_value",
787             }.issubset({column.name for column in table.columns})
788             or {
789                 "subject",
790                 "relation",
791                 "object",
792             }.issubset({column.name for column in table.columns})
793         ),
794         profile.tables[0].table_name if profile.tables else "",
795     )
796     if not table_name:
797         return plan

```

```

799
800 columns = [
801     "attribute_name",
802     "attribute_value",
803     "subject",
804     "relation",
805     "object",
806     "source_text",
807 ]
808 task = InvestigationTask(
809     task_id="TASK_STRUCTURED_RECORD_VERBALISATION",
810     question=(
811         "Which supplied attributes or triples must be verbalised, and "
812         "what exact entities, relations and values do they contain?"
813     ),
814     route=AnalysisRoute.DESCRPTIVE,
815     priority=5,
816     table_name=table_name,
817     columns=columns,
818     capability=EvidenceCapability.STRUCTURED_RECORD_VERBALISATION,
819     input_fields=columns,
820     expected_evidence_types=[
821         "attribute_record",
822         "triple_record",
823         "structured_record",
824     ],
825     required_evidence=[
826         "supplied attributes or triples",
827         "entity, relation and value strings",
828     ],
829     claim_permissions=[ClaimPermission.DESCRPTIVE],
830     answerability_note=(
831         "The output should verbalise the supplied structured record only; "
832         "broad dataset profiling and analytical routes are out of scope."
833     ),
834 )
835 insight_objectives = []
836 if enable_insight_synthesis:
837     insight_objectives = [
838         InsightObjective(
839             objective_id="INSIGHT_STRUCTURED_RECORD_VERBALISATION",
840             question=(
841                 "What concise natural-language proposition is expressed "
842                 "by the supplied attributes or triples, using only the "
843                 "given entity, relation and value strings?"
844             ),
845             preferred_insight_types=[
846                 InsightType.NARRATIVE_SUMMARY,
847             ],
848             relevant_task_ids=[
849                 task.task_id,
850             ],
851             priority="main",
852         )
853     ]
854
855 return plan.model_copy(
856     update={
857         "tasks": [task],
858         "route_order": [AnalysisRoute.DESCRPTIVE],
859         "selected_capabilities": [
860             EvidenceCapability.STRUCTURED_RECORD_VERBALISATION,
861         ],
862         "evidence_queries": [],
863         "insight_objectives": insight_objectives,
864         "rationale": (
865             "The controller selected a structured-record verbalisation "
866             "contract, so the plan is restricted to supplied attribute "
867             "or triple evidence."
868         ),
869     }
870 )
871
872 def add_event_capability_tasks(
873     *,
874     plan: ExecutionPlan,
875     request: str,
876     profile: Any,
877     audit_mode: AuditMode,
878     settings: Settings,
879     input_structure: Any,
880     capabilities: list[EvidenceCapability],
881     genre: ReportGenre,
882 ) -> ExecutionPlan:
883     if genre not in EVENT_GENRES:
884         return plan
885
886     event_fallback = fallback_execution_plan(
887         request,
888         profile,

```

```

890     audit_mode,
891     settings,
892     input_structure=input_structure,
893     available_capabilities=capabilities,
894     report_genre_override=genre,
895 )
896 existing_capabilities = {
897     task.capability
898     for task in plan.tasks
899     if task.capability is not None
900 }
901 additional_tasks = [
902     task.model_copy(
903         update={
904             "task_id": (
905                 "TASK_CAPABILITY_"
906                 + task.capability.value.upper()
907             )
908         }
909     )
910     for task in event_fallback.tasks
911     if task.capability in EVENT_CAPABILITIES
912     and task.capability not in existing_capabilities
913 ]
914 tasks = [*plan.tasks, *additional_tasks]
915 route_order = list(
916     dict.fromkeys(
917         [
918             *plan.route_order,
919             *[
920                 task.route
921                 for task in additional_tasks
922             ],
923         ]
924     )
925 )
926 return plan.model_copy(
927     update={
928         "tasks": tasks,
929         "route_order": route_order,
930     }
931 )
932
933 def should_use_deterministic_event_plan(
934     *,
935     controller_genre: ReportGenre,
936     input_structure: Any,
937     semantic_map: InputSemanticMap | None,
938     capabilities: list[EvidenceCapability],
939 ) -> bool:
940     if controller_genre not in EVENT_GENRES:
941         return False
942     if input_structure is None:
943         return False
944     if input_structure.shape != InputShape.EVENT_RECORD:
945         return False
946     if input_structure.representation_status not in {
947         InputRepresentationStatus.VALID,
948         InputRepresentationStatus.VALID_WITH_WARNINGS,
949     }:
950         return False
951     if getattr(input_structure, "confidence", 0.0) < 0.7:
952         return False
953     if semantic_map is None or not semantic_map.bindings:
954         return False
955     return bool(EVENT_CAPABILITIES & set(capabilities))
956
957
958 def exception_cause_chain(
959     error: BaseException,
960 ) -> list[str]:
961     chain: list[str] = []
962     current: BaseException | None = error
963     seen: set[int] = set()
964
965     while (
966         current is not None
967         and id(current) not in seen
968     ):
969         seen.add(id(current))
970
971         message = getattr(
972             current,
973             "message",
974             str(current),
975         )
976
977         chain.append(
978             f"{type(current).__name__}: "
979             f"{message}"
980

```



```

981     )
982
983     current = current.__cause__
984
985     return chain
986
987
988 def _compact_writer_structured_value(
989     value: Any,
990     *,
991     item_limit: int | None,
992 ) -> Any:
993     if isinstance(value, list):
994         if item_limit is None:
995             return [
996                 _compact_writer_structured_value(
997                     item,
998                     item_limit=item_limit,
999                 )
1000                 for item in value
1001             ]
1002
1003         omitted_count = max(0, len(value) - item_limit)
1004         compacted = [
1005             _compact_writer_structured_value(
1006                 item,
1007                 item_limit=item_limit,
1008             )
1009             for item in value[:item_limit]
1010         ]
1011         if omitted_count:
1012             compacted.append(
1013                 {
1014                     "omitted_record_count": omitted_count,
1015                     "omission_reason": (
1016                         "Writer prompt compaction retained the highest-priority "
1017                         "records for this report-sized payload."
1018                     ),
1019                 }
1020             )
1021         return compacted
1022
1023     if isinstance(value, dict):
1024         return {
1025             key: _compact_writer_structured_value(
1026                 nested_value,
1027                 item_limit=item_limit,
1028             )
1029             for key, nested_value in value.items()
1030         }
1031
1032     return value
1033
1034
1035 def _compact_writer_fact(
1036     fact: VerifiedFact,
1037     *,
1038     item_limit: int | None,
1039 ) -> dict[str, Any]:
1040     payload = fact.model_dump(mode="json")
1041     payload["structured_values"] = _compact_writer_structured_value(
1042         payload.get("structured_values", {}),
1043         item_limit=item_limit,
1044     )
1045     entities = payload.get("entities")
1046     if isinstance(entities, list) and item_limit is not None:
1047         entity_limit = max(8, item_limit * 3)
1048         omitted_count = max(0, len(entities) - entity_limit)
1049         payload["entities"] = entities[:entity_limit]
1050         if omitted_count:
1051             payload["entities"].append(
1052                 f"... {omitted_count} lower-priority entities omitted "
1053                 "from the writer prompt"
1054             )
1055     return payload
1056
1057
1058 def _compact_short_form_structured_values(
1059     values: Any,
1060 ) -> Any:
1061     if isinstance(values, dict):
1062         keep_keys = {
1063             "description_proposition",
1064             "records",
1065             "attributes",
1066             "highlighted_values",
1067             "header_context",
1068             "page_title",
1069             "section_title",
1070             "focused_record_relation",
1071             "focused_list_relation",

```

```

1072         "highlighted_role_value_pairs",
1073     }
1074     if all(isinstance(item, dict) for item in values.values()):
1075         return {
1076             evidence_id: {
1077                 key: nested_value
1078                 for key, nested_value in item.items()
1079                 if key in keep_keys
1080                 and nested_value not in (None, [], {}, "")
1081             }
1082             for evidence_id, item in values.items()
1083             if isinstance(item, dict)
1084         }
1085     return {
1086         key: _compact_short_form_structured_values(nested_value)
1087         for key, nested_value in values.items()
1088         if key in keep_keys
1089         or key.startswith("EVD_")
1090     }
1091     if isinstance(values, list):
1092         return [
1093             _compact_short_form_structured_values(item)
1094             for item in values
1095         ]
1096     return values
1097
1098
1099 def _short_form_writer_fact(
1100     fact: VerifiedFact,
1101 ) -> dict[str, Any]:
1102     payload = fact.model_dump(mode="json")
1103     return {
1104         "fact_id": payload.get("fact_id"),
1105         "fact_summary": payload.get("fact_summary"),
1106         "evidence_ids": payload.get("evidence_ids", []),
1107         "source_capabilities": payload.get("source_capabilities", []),
1108         "structured_values": _compact_short_form_structured_values(
1109             payload.get("structured_values", {})
1110         ),
1111         "entities": payload.get("entities", []),
1112         "claim_permissions": payload.get("claim_permissions", []),
1113     }
1114
1115
1116 def _writer_structured_item_limit(
1117     pack: WriterEvidencePack,
1118 ) -> int | None:
1119     maximum_words = pack.report_specification.maximum_length_words
1120     if maximum_words is None:
1121         return None
1122
1123     event_report = pack.report_specification.genre in EVENT_GENRES
1124     if event_report:
1125         return max(12, min(60, maximum_words // 25))
1126
1127     return max(3, min(10, maximum_words // 100))
1128
1129
1130 def _event_report_writing_guidance(
1131     content_requirements: dict[str, Any] | None,
1132 ) -> dict[str, Any]:
1133     units = (
1134         content_requirements or {}
1135     ).get("units", [])
1136     supported_slots = [
1137         unit.get("unit_id")
1138         for unit in units
1139         if isinstance(unit, dict)
1140         and unit.get("candidate_fact_ids")
1141     ]
1142     realisation_policy = (
1143         content_requirements or {}
1144     ).get("realisation_policy", RealisationPolicy.STRICT_SOURCE_SURFACE.value)
1145     reference_recap_style = bool(
1146         (content_requirements or {}).get("reference_recap_style")
1147     )
1148     event_recap_style = bool(
1149         (content_requirements or {}).get("event_recap_style")
1150     )
1151
1152     return {
1153         "realisation_policy": realisation_policy,
1154         "event_recap_style": event_recap_style,
1155         "reference_recap_style": reference_recap_style,
1156         "style": (
1157             (
1158                 "Write a reference-style event recap from verified evidence "
1159                 "as flowing prose without visible headings."
1160             )
1161             if reference_recap_style
1162             else (

```

```

1163         "Write a coherent event recap from verified evidence, not a "
1164         "flat-table profile or a mechanical ranking dump."
1165     )
1166 ),
1167 "content_priority_order": [
1168     "event_result",
1169     "event_context",
1170     "participant_record_context",
1171     "event_status",
1172     "score_progression",
1173     "event_sequence",
1174     "leading_performance",
1175     "main_contrast",
1176     "secondary_performance",
1177     "scope_limitations",
1178 ],
1179 "supported_content_slots": supported_slots,
1180 "opening_policy": (
1181     "Lead with the supported result or outcome when it exists. Add "
1182     "date, venue/location, status and participant record context in "
1183     "the opening only when those facts are verified."
1184 ),
1185 "narration_policy": (
1186     "Use verified sequence and score-progression facts to describe "
1187     "what happened in natural order. Use connective wording only "
1188     "for directly supported contrasts. Do not invent causes, "
1189     "momentum, runs, streaks, dominance, historical significance or "
1190     "chronology absent from verified facts."
1191 ),
1192 "selection_policy": (
1193     "There is no fixed number of findings. Cover required supported "
1194     "slots first, then add distinct high-salience verified facts and "
1195     "insights that improve the recap. Omit low-value mechanical "
1196     "rankings unless they clarify a stronger event point or the user "
1197     "requested exhaustive detail."
1198 ),
1199 "surface_policy": (
1200     "For reference-style recaps, do not use Markdown headings or a "
1201     "generic limitations paragraph. Keep caveats internal unless a "
1202     "visible caveat is necessary to prevent an unsupported inference."
1203     if reference_recap_style
1204     else (
1205         "Use the requested report structure while keeping event "
1206         "scope limitations concise and event-specific."
1207     )
1208 ),
1209 "avoid": [
1210     "wrapper row or column counts",
1211     "constant-column analysis",
1212     "missingness discussion",
1213     "correlation or regression language",
1214     "statistical-power or predictive-modelling discussion",
1215     "unsupported explanation of why the result occurred",
1216 ],
1217 },
1218
1219
1220 def build_compact_writer_payload(
1221     pack: WriterEvidencePack,
1222     allow_hypotheses_in_report: bool = False,
1223     content_requirements: dict[str, Any] | None = None,
1224     narrative_plan: NarrativePlan | None = None,
1225 ) -> dict[str, Any]:
1226     facts_by_id = {
1227         fact.fact_id: fact
1228         for fact in [
1229             *pack.priority_facts,
1230             *pack.supporting_facts,
1231             *pack.limitation_facts,
1232         ]
1233     }
1234     evidence_by_id = {
1235         item.evidence_id: item
1236         for item in pack.evidence_ledger.items
1237     }
1238     strength_labels_by_fact_id = {
1239         fact_id: list(
1240             dict.fromkeys(
1241                 normalise_strength_label(
1242                     evidence_by_id[evidence_id].strength_label
1243                 )
1244                 for evidence_id in fact.evidence_ids
1245                 if evidence_id in evidence_by_id
1246             )
1247         )
1248         for fact_id, fact in facts_by_id.items()
1249     }
1250     structured_item_limit = _writer_structured_item_limit(pack)
1251     event_report = pack.report_specification.genre in EVENT_GENRES
1252     short_form = (
1253         pack.report_specification.communication_task

```

```

1254     in SHORT_FORM_COMMUNICATION_TASKS
1255 )
1256
1257 if short_form:
1258     requirements = content_requirements or {}
1259     realisation_policy = requirements.get(
1260         "realisation_policy",
1261         pack.report_specification.realisation_policy.value,
1262     )
1263     may_normalise_identifiers = bool(
1264         requirements.get(
1265             "style_rewrite_permissions",
1266             {},
1267         ).get("may_normalise_identifier_separators")
1268     )
1269     return {
1270         "user_request": pack.user_request,
1271         "report_specification": {
1272             "genre": pack.report_specification.genre.value,
1273             "communication_task": (
1274                 pack.report_specification.communication_task.value
1275             ),
1276             "output_form": pack.report_specification.output_form.value,
1277             "focus_scope": pack.report_specification.focus_scope,
1278             "realisation_policy": realisation_policy,
1279             "allow_headings": pack.report_specification.allow_headings,
1280             "max_sentences": pack.report_specification.max_sentences,
1281             "max_paragraphs": pack.report_specification.max_paragraphs,
1282             "require_complete_sentence": (
1283                 pack.report_specification.require_complete_sentence
1284             ),
1285             "communication_goal": (
1286                 pack.report_specification.communication_goal
1287             ),
1288             "prohibited_claim_types": (
1289                 pack.report_specification.prohibited_claim_types
1290             ),
1291             "prioritisation_rule": (
1292                 pack.report_specification.prioritisation_rule
1293             ),
1294         },
1295         "content_requirements": requirements,
1296         "priority_facts": [
1297             _short_form_writer_fact(fact)
1298             for fact in pack.priority_facts
1299         ],
1300         "supporting_facts": [
1301             _short_form_writer_fact(fact)
1302             for fact in pack.supporting_facts
1303         ],
1304         "priority_verified_insights": pack.priority_verified_insights,
1305         "supporting_verified_insights": pack.supporting_verified_insights,
1306         "internal_prohibited_interpretations": (
1307             pack.internal_prohibited_interpretations
1308         ),
1309         "surface_form_policy": (
1310             "Preserve digits, percentages, decimals and units exactly. "
1311             "You may normalize harmless identifier separators such as "
1312             "underscores to spaces when this improves natural reference "
1313             "style and the entity remains recognisable."
1314             if may_normalise_identifiers
1315             else (
1316                 "Preserve source identifiers, digits, percentages, "
1317                 "decimals, units and compact alphanumeric forms exactly "
1318                 "unless only minor grammar is needed."
1319             )
1320         ),
1321     }
1322
1323 payload = {
1324     "user_request": pack.user_request,
1325     "report_specification": (
1326         pack.report_specification
1327     ),
1328     "genre": pack.report_specification.genre,
1329     "audience": pack.report_specification.audience,
1330     "perspective": pack.report_specification.perspective,
1331     "communication_goal": (
1332         pack.report_specification.communication_goal
1333     ),
1334     "input_structure": pack.input_structure,
1335     "available_capabilities": pack.available_capabilities,
1336     "priority_verified_insights": (
1337         pack.priority_verified_insights
1338     ),
1339     "supporting_verified_insights": (
1340         pack.supporting_verified_insights
1341     ),
1342     "hypothesis_only_insights": (
1343         pack.insight_ledger.hypothesis_only_insights
1344         if allow_hypotheses_in_report

```

```

1345         else []
1346     ),
1347     "priority_facts": (
1348         [
1349             _compact_writer_fact(
1350                 fact,
1351                 item_limit=structured_item_limit,
1352             )
1353             for fact in pack.priority_facts
1354         ]
1355     ),
1356     "supporting_facts": (
1357         [
1358             _compact_writer_fact(
1359                 fact,
1360                 item_limit=structured_item_limit,
1361             )
1362             for fact in pack.supporting_facts
1363         ]
1364     ),
1365     "limitation_facts": (
1366         [
1367             _compact_writer_fact(
1368                 fact,
1369                 item_limit=structured_item_limit,
1370             )
1371             for fact in pack.limitation_facts
1372         ]
1373     ),
1374     "verified_strength_labels_by_fact_id": (
1375         strength_labels_by_fact_id
1376     ),
1377     "analytical_recommendations": (
1378         pack.analytical_recommendations
1379     ),
1380     "reader_facing_limitations": (
1381         pack.reader_facing_limitations
1382     ),
1383     "content_requirements": (
1384         content_requirements or {}
1385     ),
1386     "structured_value_compaction": (
1387         {
1388             "item_limit": structured_item_limit,
1389             "policy": (
1390                 "No list item limit is applied because no maximum word "
1391                 "ceiling is configured."
1392                 if structured_item_limit is None
1393                 else (
1394                     "Structured list values are compacted to fit the "
1395                     "configured maximum word ceiling."
1396                 )
1397             ),
1398         }
1399     ),
1400     "internal_prohibited_interpretations": (
1401         pack
1402         .internal_prohibited_interpretations
1403     ),
1404 }
1405 if event_report:
1406     payload["event_report_writing_guidance"] = (
1407         _event_report_writing_guidance(
1408             content_requirements,
1409         )
1410     )
1411     plan = narrative_plan or build_event_narrative_plan(
1412         pack,
1413         content_requirements,
1414     )
1415     if plan.applies:
1416         payload["narrative_plan"] = plan
1417     return payload
1418
1419 def build_compact_insight_payload(
1420     *,
1421     request: str,
1422     plan: ExecutionPlan,
1423     fact_ledger: FactLedger,
1424     evidence_ledger: Any,
1425     settings: Settings,
1426 ) -> dict[str, Any]:
1427     referenced_evidence_ids = {
1428         evidence_id
1429         for fact in fact_ledger.writer_ready_facts
1430         for evidence_id in fact.evidence_ids
1431     }
1432     referenced_evidence = [
1433         item
1434         for item in evidence_ledger.items

```

```

1436     if item.evidence_id in referenced_evidence_ids
1437 ]
1438
1439 return {
1440     "user_request": request,
1441     "report_specification": plan.report_specification,
1442     "frozen_insight_objectives": plan.insight_objectives,
1443     "writer_ready_verified_facts": fact_ledger.writer_ready_facts,
1444     "referenced_deterministic_evidence": referenced_evidence,
1445     "analytical_recommendations": [
1446         recommendation
1447         for item in referenced_evidence
1448         for recommendation in item.recommendations
1449     ],
1450     "prohibited_interpretations": list(
1451         dict.fromkeys(
1452             interpretation
1453             for fact in fact_ledger.writer_ready_facts
1454             for interpretation in fact.prohibited_interpretations
1455         )
1456     ),
1457     "selection_policy": {
1458         "candidate_count": "uncapped",
1459         "verified_insight_count": "uncapped",
1460         "report_word_ceiling": (
1461             plan.report_specification.maximum_length_words
1462         ),
1463         "min_facts_per_bounded_insight": (
1464             settings.min_facts_per_bounded_insight
1465         ),
1466         "min_insight_confidence": settings.min_insight_confidence,
1467         "min_insight_salience": settings.min_insight_salience,
1468         "allow_hypotheses_in_report": (
1469             settings.allow_hypotheses_in_report
1470         ),
1471         "focused_table_semantic_inference": (
1472             {
1473                 "expected_interpretation_level": "bounded_insight",
1474                 "expected_contribution": "descriptive_synthesis",
1475                 "instruction": (
1476                     "Infer a concise table-local relation from "
1477                     "highlighted cells, row context, headers, "
1478                     "page/section titles and source text only. Prefer the "
1479                     "relation conveyed by the table context over a "
1480                     "mechanical cell-location description when support is "
1481                     "clear. Do not use held-out references or outside "
1482                     "knowledge."
1483                 ),
1484                 "short_form_selection": (
1485                     "For one-sentence focused-table tasks, centre the "
1486                     "candidate on highlighted role/value pairs and the "
1487                     "most specific supported primary subject candidate. "
1488                     "Treat non-highlighted same-row values as context; "
1489                     "omit them unless they are required to disambiguate "
1490                     "the highlighted relation. Do not combine subject "
1491                     "candidates with co-entities into a joint subject "
1492                     "unless the supplied evidence explicitly represents "
1493                     "a combined entity. Use supplied highlighted-measure "
1494                     "comparisons when they support concise outcome-like "
1495                     "wording."
1496                 ),
1497                 "safe_fallback": (
1498                     "Use conservative selected-cell description if the "
1499                     "relation is ambiguous."
1500                 ),
1501             }
1502         ),
1503         if plan.report_specification.communication_task
1504         == CommunicationTask.FOCUSED_TABLE_DESCRIPTION
1505         else None
1506     ),
1507 },
1508 }
1509
1510 def build_writer_quality_revision_prompt(
1511     *,
1512     writer_pack: WriterEvidencePack,
1513     current_output: WriterOutput,
1514     missing_components: list[ReportComponent],
1515     quality_findings: list[str],
1516     content_requirements: dict[str, Any],
1517     settings: Settings,
1518 ) -> str:
1519     used_fact_ids = set(
1520         current_output.selected_fact_ids
1521     )
1522
1523     unused_priority_facts = [
1524         fact
1525         for fact in writer_pack.priority_facts
1526         if fact.fact_id not in used_fact_ids

```

```

1527     ]
1528     used_insight_ids = {
1529         insight_id
1530         for support in current_output.sentence_support
1531         for insight_id in support.insight_ids
1532     }
1533     unused_verified_insights = [
1534         insight
1535         for insight in [
1536             *writer_pack.priority_verified_insights,
1537             *writer_pack.supporting_verified_insights,
1538         ]
1539         if insight.insight_id not in used_insight_ids
1540     ]
1541
1542     current_word_count = len(
1543         re.findall(
1544             r"\b[\w'-]+\b",
1545             current_output.markdown,
1546         )
1547     )
1548
1549     target_words = (
1550         writer_pack.report_specification
1551         .target_length_words
1552     )
1553     maximum_words = writer_pack.report_specification.maximum_length_words
1554
1555     minimum_words = min(
1556         target_words,
1557         max(
1558             settings.minimum_report_word_floor,
1559             int(
1560                 target_words
1561                 * settings.minimum_report_word_ratio
1562             ),
1563         len(
1564             writer_pack
1565             .report_specification
1566             .required_components
1567         )
1568         * 45,
1569     ),
1570 )
1571     event_report = (
1572         writer_pack.report_specification.genre
1573         in {
1574             ReportGenre.EVENT_REPORT,
1575             ReportGenre.SPORTS_GAME_REPORT,
1576         }
1577     )
1578     writing_goal = (
1579         "event-report writing"
1580         if event_report
1581         else "data-science writing"
1582     )
1583     component_guidance = (
1584         "required event result, context, performance, contrast, and scope slots"
1585         if event_report
1586         else "required dataset overview, quality, relationship, and limitation components"
1587     )
1588
1589     return (
1590         "Revise the complete report once for task fulfilment and natural "
1591         f"{writing_goal} before factual audit.\n\n"
1592         "This is a Writer quality revision, not a factual repair.\n\n"
1593         "Do not merely rephrase the existing short report.\n"
1594         "Use unused verified facts and insights to cover missing sections and "
1595         "add non-duplicative synthesis.\n"
1596         + (
1597             f"Do not exceed {maximum_words} words.\n"
1598             if maximum_words is not None
1599             else (
1600                 "No explicit maximum word count is configured; use the "
1601                 "target length as guidance while prioritising supported "
1602                 "coverage and concision.\n"
1603             )
1604         )
1605         + "Do not invent calculations or facts.\n"
1606         "Do not calculate statistics.\n"
1607         "Do not introduce new numbers, entities, categories, metadata, "
1608         "causal claims, prediction claims, forecast claims, or deployment "
1609         "claims.\n"
1610         "Do not expose internal control fields such as Finding:, Strength:, "
1611         "Important Note:, Interpretation Notes:, Recommended Use:, or Global "
1612         "Prohibited Interpretations.\n"
1613         f"Use {writing_goal} and consolidate shared caveats.\n"
1614         "Prefer strong and moderate evidence over small effects.\n"
1615         "Preserve each supplied qualitative strength classification exactly "
1616         "and consistently.\n"
1617         "Do not turn a possible explanation into an ordinary next step; it is "

```

```

1618 "a hypothesis and must follow the configured hypothesis policy.\n"
1619 "Return structured sections and sentences only. Do not return "
1620 "Markdown or a separate support map; the controller will create both "
1621 "deterministically.\n"
1622 "Every factual sentence must list its supporting fact IDs.\n\n"
1623 f"Current word count: {current_word_count}\n"
1624 f"Minimum useful word count: {minimum_words}\n"
1625 + (
1626     f"Maximum word count: {maximum_words}\n"
1627     if maximum_words is not None
1628     else "Maximum word count: not configured\n"
1629 )
1630 + f"Available priority facts: {len(writer_pack.priority_facts)}\n"
1631 f"Unused priority facts: {len(unused_priority_facts)}\n\n"
1632 f"Unused verified insights: {len(unused_verified_insights)}\n\n"
1633 "Controller-enforced content requirements:\n"
1634 + compact_json(content_requirements)
1635 + "\n\n"
1636 "Missing components:\n"
1637 + (
1638     "\n".join(
1639         f"- {component.value}"
1640         for component in missing_components
1641     )
1642     if missing_components
1643     else "- None"
1644 )
1645 + "\n\nQuality findings:\n"
1646 + (
1647     "\n".join(
1648         f"- {finding}"
1649         for finding in quality_findings
1650     )
1651     if quality_findings
1652     else "- None"
1653 )
1654 + "\n\nUnused verified priority facts:\n"
1655 + f"\nUse verified support to cover the {component_guidance}.\n"
1656 + compact_json(unused_priority_facts)
1657 + "\n\nUnused verified insights:\n"
1658 + compact_json(unused_verified_insights)
1659 + "\n\nCompact Writer evidence pack:\n"
1660 + compact_json(
1661     build_compact_writer_payload(
1662         writer_pack,
1663         settings.allow_hypotheses_in_report,
1664         content_requirements,
1665     )
1666 )
1667 + "\n\nCurrent Writer output:\n"
1668 + compact_json(current_output)
1669 )
1670
1671 class ArtifactStore:
1672     def __init__(self, base_directory: Path, run_id: str):
1673         self.run_directory = base_directory / run_id
1674         self.run_directory.mkdir(parents=True, exist_ok=True)
1675         self.trace_path = self.run_directory / "trace.jsonl"
1676
1677     @staticmethod
1678     def create_run_id(fingerprint: str) -> str:
1679         timestamp = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%S")
1680         return f"{timestamp}_{fingerprint[:10]}"
1681
1682     def save_json(self, filename: str, value: Any) -> Path:
1683         path = self.run_directory / filename
1684         path.write_text(
1685             json.dumps(
1686                 json_safe(value),
1687                 indent=2,
1688                 ensure_ascii=False,
1689             ),
1690             encoding="utf-8",
1691         )
1692         return path
1693
1694     def save_text(self, filename: str, text: str) -> Path:
1695         path = self.run_directory / filename
1696         path.write_text(text, encoding="utf-8")
1697         return path
1698
1699     def trace(
1700         self,
1701         stage: str,
1702         status: str,
1703         details: dict[str, Any] | None = None,
1704     ) -> None:
1705         event = {
1706             "timestamp": datetime.now(timezone.utc).isoformat(),
1707             "stage": stage,
1708             "status": status,

```



```

1709         "details": json_safe(details or {}),
1710     }
1711
1712     with self.trace_path.open("a", encoding="utf-8") as handle:
1713         handle.write(
1714             json.dumps(event, ensure_ascii=False) + "\n"
1715         )
1716
1717
1718 class Table2TextWorkflow:
1719     def __init__(self, settings: Settings | None = None):
1720         self.settings = settings or Settings.from_env()
1721         if not isinstance(self.settings.output_dir, Path):
1722             self.settings = self.settings.__class__(
1723                 **{
1724                     **self.settings.__dict__,
1725                     "output_dir": Path(self.settings.output_dir),
1726                 }
1727             )
1728
1729         self.data_understanding_agent = None
1730         self.orchestrator_agent = None
1731         self.evidence_agent = None
1732         self.verifier_agent = None
1733         self.evidence_insight_synthesis_agent = None
1734         self.verifier_insight_verification_agent = None
1735         self.writer_agent = None
1736         self.auditor_agent = None
1737
1738         if self.settings.use_llm:
1739             self.data_understanding_agent = build_data_understanding_agent(
1740                 self.settings
1741             )
1742             self.orchestrator_agent = build_orchestrator_agent(self.settings)
1743             self.evidence_agent = build_evidence_agent(self.settings)
1744             self.verifier_agent = build_verifier_agent(self.settings)
1745             self.evidence_insight_synthesis_agent = (
1746                 build_insight_synthesis_agent(self.settings)
1747             )
1748             self.verifier_insight_verification_agent = (
1749                 build_insight_verifier_agent(self.settings)
1750             )
1751             self.writer_agent = build_writer_agent(self.settings)
1752             self.auditor_agent = build_auditor_agent(self.settings)
1753
1754     def usage_limits(self) -> UsageLimits:
1755         return UsageLimits(
1756             request_limit=self.settings.max_agent_requests,
1757             total_tokens_limit=self.settings.max_total_tokens,
1758         )
1759
1760     async def run_agent_or_fallback(
1761         self,
1762         *,
1763         stage: str,
1764         agent: Any,
1765         prompt: str,
1766         dependencies: AgentDependencies,
1767         fallback: Callable[[], Any],
1768         store: ArtifactStore,
1769     ) -> Any:
1770         if not self.settings.use_llm:
1771             store.trace(
1772                 stage,
1773                 "fallback",
1774                 {"reason": "LLM execution disabled"},
1775             )
1776             return fallback()
1777
1778         try:
1779             result = await agent.run(
1780                 prompt,
1781                 deps=dependencies,
1782                 usage_limits=self.usage_limits(),
1783             )
1784
1785             usage = getattr(result, "usage", None)
1786
1787             store.trace(
1788                 stage,
1789                 "completed",
1790                 {"usage": str(usage)},
1791             )
1792
1793             return result.output
1794
1795         except Exception as error:
1796             store.trace(
1797                 stage,
1798                 "fallback",
1799                 {

```

```

1800         "reason": (
1801             f"{type(error).__name__}: {error}"
1802         ),
1803         "cause_chain": (
1804             exception_cause_chain(
1805                 error
1806             )
1807         ),
1808     },
1809 )
1810     return fallback()
1811
1812 async def run_optional_insight_agent(
1813     self,
1814     *,
1815     stage: str,
1816     agent: Any,
1817     prompt: str,
1818     dependencies: AgentDependencies,
1819     store: ArtifactStore,
1820 ) -> tuple[Any | None, str | None]:
1821     if not self.settings.use_llm or agent is None:
1822         reason = "LLM execution disabled"
1823         store.trace(
1824             stage,
1825             "skipped",
1826             {"reason": reason},
1827         )
1828         return None, reason
1829
1830     try:
1831         result = await agent.run(
1832             prompt,
1833             deps=dependencies,
1834             usage_limits=self.usage_limits(),
1835         )
1836         usage = getattr(result, "usage", None)
1837         store.trace(
1838             stage,
1839             "completed",
1840             {"usage": str(usage)},
1841         )
1842         return result.output, None
1843     except Exception as error:
1844         reason = f"{type(error).__name__}: {error}"
1845         store.trace(
1846             stage,
1847             "fallback",
1848             {
1849                 "reason": reason,
1850                 "cause_chain": exception_cause_chain(error),
1851             },
1852         )
1853         return None, reason
1854
1855 async def audit_once(
1856     self,
1857     *,
1858     run_id: str,
1859     writer_output: WriterOutput,
1860     fact_ledger: Any,
1861     evidence_ledger: Any,
1862     insight_ledger: InsightLedger,
1863     profile_support_records: list[
1864         ProfileSupportRecord
1865     ],
1866     plan: ExecutionPlan,
1867     audit_mode: AuditMode,
1868     external_truth_sources: list[ExternalTruthSource],
1869     revision_round: int,
1870     store: ArtifactStore,
1871     stage_name: str,
1872 ) -> tuple[AuditReport, AuditRepairProposal, WriterOutput]:
1873     deterministic_pre_patch = deterministic_audit(
1874         writer_output=writer_output,
1875         fact_ledger=fact_ledger,
1876         evidence=evidence_ledger,
1877         mode=audit_mode,
1878         external_sources=external_truth_sources,
1879         revision_round=revision_round,
1880         report_specification=plan.report_specification,
1881         settings=self.settings,
1882         profile_support_records=profile_support_records,
1883         insight_ledger=insight_ledger,
1884     )
1885
1886     if revision_round == 0:
1887         pre_patch_audit_name = (
1888             "10_initial_audit_pre_profile_patch.json"
1889         )
1890         support_patch_name = (

```

```

1891         "10_initial_support_map_patches.json"
1892     )
1893     profile_patched_name = (
1894         "10_initial_profile_patched_output.json"
1895     )
1896 else:
1897     pre_patch_audit_name = (
1898         "14_post_repair_audit_pre_profile_patch"
1899         f"_round_{revision_round}.json"
1900     )
1901     support_patch_name = (
1902         "14_post_repair_support_map_patches"
1903         f"_round_{revision_round}.json"
1904     )
1905     profile_patched_name = (
1906         "14_post_repair_profile_patched_output"
1907         f"_round_{revision_round}.json"
1908     )
1909
1910 store.save_json(
1911     pre_patch_audit_name,
1912     deterministic_pre_patch,
1913 )
1914 store.save_json(
1915     support_patch_name,
1916     deterministic_pre_patch.support_map_patches,
1917 )
1918
1919 profile_patched_output = writer_output
1920
1921 if deterministic_pre_patch.support_map_patches:
1922     profile_patched_output = apply_support_map_patches(
1923         writer_output,
1924         deterministic_pre_patch.support_map_patches,
1925         {
1926             record.support_id
1927             for record in profile_support_records
1928         },
1929     )
1930
1931 store.save_json(
1932     profile_patched_name,
1933     profile_patched_output,
1934 )
1935
1936 deterministic = deterministic_audit(
1937     writer_output=profile_patched_output,
1938     fact_ledger=fact_ledger,
1939     evidence=evidence_ledger,
1940     mode=audit_mode,
1941     external_sources=external_truth_sources,
1942     revision_round=revision_round,
1943     report_specification=plan.report_specification,
1944     settings=self.settings,
1945     profile_support_records=profile_support_records,
1946     insight_ledger=insight_ledger,
1947 ).model_copy(
1948     update={
1949         "support_map_patches": (
1950             deterministic_pre_patch
1951             .support_map_patches
1952         )
1953     }
1954 )
1955
1956 deterministic_annotation_ids = [
1957     annotation.annotation_id
1958     for annotation in deterministic.annotations
1959 ]
1960 deterministic_serious_annotation_ids = [
1961     annotation.annotation_id
1962     for annotation in deterministic.annotations
1963     if annotation.severity.value
1964     in {"high", "critical"}
1965 ]
1966 short_form_output = (
1967     plan.report_specification.communication_task
1968     in SHORT_FORM_COMMUNICATION_TASKS
1969 )
1970
1971 if (
1972     short_form_output
1973     and deterministic.decision == AuditDecision.PASS
1974     and not deterministic_serious_annotation_ids
1975 ):
1976     proposal = fallback_audit_proposal(deterministic)
1977     store.trace(
1978         stage_name,
1979         "skipped",
1980         {
1981             "reason": (

```

```

1982         "Short-form output passed deterministic audit; "
1983         "LLM repair audit was skipped to avoid a large "
1984         "one-sentence audit prompt."
1985     ),
1986     "communication_task": (
1987         plan.report_specification.communication_task.value
1988     ),
1989     "deterministic_annotation_count": len(
1990         deterministic.annotations
1991     ),
1992 },
1993 )
1994 return (
1995     merge_audit_proposal(deterministic, proposal),
1996     proposal,
1997     profile_patched_output,
1998 )
1999
2000 prompt = (
2001     "Audit this report independently and propose targeted repairs "
2002     "for high-confidence factual errors.\n\n"
2003     "User objective:\n"
2004     + plan.objective
2005     + "\n\nReport specification:\n"
2006     + compact_json(plan.report_specification)
2007     + "\n\nWriter output:\n"
2008     + compact_json(profile_patched_output)
2009     + "\n\nVerified fact ledger:\n"
2010     + compact_json(fact_ledger)
2011     + "\n\nEvidence ledger:\n"
2012     + compact_json(evidence_ledger)
2013     + "\n\nVerified insight ledger:\n"
2014     + compact_json(insight_ledger)
2015     + "\n\nDeterministic profile support registry:\n"
2016     + compact_json(profile_support_records)
2017     + "\n\nDeterministic pre-audit:\n"
2018     + compact_json(deterministic)
2019     + "\n\nExternal truth sources:\n"
2020     + compact_json(external_truth_sources)
2021     + "\n\nGenerate no more than "
2022     + str(self.settings.repair_candidates_per_sentence)
2023     + " repair candidates for each flagged sentence."
2024 )
2025
2026 proposal = await self.run_agent_or_fallback(
2027     stage=stage_name,
2028     agent=self.auditor_agent,
2029     prompt=prompt,
2030     dependencies=AgentDependencies(
2031         run_id=run_id,
2032         payload={
2033             "report_text": profile_patched_output.markdown,
2034             "fact_ledger": fact_ledger.model_dump(mode="json"),
2035             "evidence_ledger": evidence_ledger.model_dump(
2036                 mode="json"
2037             ),
2038             "insight_ledger": insight_ledger.model_dump(
2039                 mode="json"
2040             ),
2041             "valid_fact_ids": [
2042                 fact.fact_id
2043                 for fact in fact_ledger.writer_ready_facts
2044             ],
2045             "valid_evidence_ids": [
2046                 item.evidence_id
2047                 for item in evidence_ledger.items
2048             ],
2049             "valid_profile_support_ids": [
2050                 record.support_id
2051                 for record in profile_support_records
2052             ],
2053             "valid_insight_ids": [
2054                 insight.insight_id
2055                 for insight in insight_ledger.verified_insights
2056             ],
2057             "verified_main_insight_ids": [
2058                 insight.insight_id
2059                 for insight in insight_ledger.verified_insights
2060             ],
2061             "hypothesis_only_insight_ids": [
2062                 insight.insight_id
2063                 for insight in (
2064                     insight_ledger.hypothesis_only_insights
2065                 )
2066             ],
2067             "insight_statements": {
2068                 insight.insight_id: insight.statement
2069                 for insight in [
2070                     *insight_ledger.verified_insights,
2071                     *insight_ledger.hypothesis_only_insights,
2072                 ]

```

```

2073         },
2074         "insight_source_fact_ids": {
2075             insight.insight_id: insight.source_fact_ids
2076             for insight in [
2077                 *insight_ledger.verified_insights,
2078                 *insight_ledger.hypothesis_only_insights,
2079             ]
2080         },
2081         "insight_source_evidence_ids": {
2082             insight.insight_id: insight.source_evidence_ids
2083             for insight in [
2084                 *insight_ledger.verified_insights,
2085                 *insight_ledger.hypothesis_only_insights,
2086             ]
2087         },
2088         "sentence_insight_ids": {
2089             support.sentence_text: support.insight_ids
2090             for support in profile_patched_output.sentence_support
2091         },
2092         "allow_hypotheses_in_report": (
2093             self.settings.allow_hypotheses_in_report
2094         ),
2095         "report_genre": plan.report_specification.genre.value,
2096         "report_perspective": (
2097             plan.report_specification.perspective.value
2098         ),
2099         "deterministic_annotation_ids": (
2100             deterministic_annotation_ids
2101         ),
2102         "deterministic_serious_annotation_ids": (
2103             deterministic_serious_annotation_ids
2104         ),
2105         "deterministic_annotation_sentences": [
2106             annotation.sentence
2107             for annotation in deterministic.annotations
2108         ],
2109     },
2110 ),
2111 fallback=lambda: fallback_audit_proposal(
2112     deterministic
2113 ),
2114 store=store,
2115 )
2116
2117 proposal = AuditRepairProposal.model_validate(proposal)
2118 merged = merge_audit_proposal(deterministic, proposal)
2119
2120 return merged, proposal, profile_patched_output
2121
2122 async def run(
2123     self,
2124     inputs: List[str | Path],
2125     request: str,
2126     *,
2127     audit_mode: AuditMode = AuditMode.INTERNAL,
2128     external_truth_sources: List[ExternalTruthSource] | None = None,
2129     evaluation_field_policy: EvaluationFieldPolicy | None = None,
2130     report_genre: ReportGenre | None = None,
2131     communication_task: CommunicationTask | None = None,
2132     output_form: OutputForm | None = None,
2133     focus_scope: str | None = None,
2134 ) -> PipelineResult:
2135     external_truth_sources = external_truth_sources or []
2136
2137     data_bundle = load_data(
2138         inputs,
2139         evaluation_field_policy=evaluation_field_policy,
2140     )
2141     input_structure = data_bundle.input_structure
2142     capabilities = available_capabilities(data_bundle)
2143     structural_catalog = build_structural_catalog(data_bundle.structured_inputs)
2144     preliminary_inferred_contract = infer_task_contract(
2145         request=request,
2146         structured_inputs=data_bundle.structured_inputs,
2147         input_structure=input_structure,
2148     )
2149     representation_eligible = bool(
2150         input_structure
2151         and input_structure.representation_status
2152         in {
2153             InputRepresentationStatus.VALID,
2154             InputRepresentationStatus.VALID_WITH_WARNINGS,
2155         }
2156     )
2157     profile = profile_data(data_bundle)
2158     profile_support_records = (
2159         build_profile_support_registry(
2160             profile
2161         )
2162     )
2163

```

```

2164     run_id = ArtifactStore.create_run_id(
2165         data_bundle.fingerprint
2166     )
2167     store = ArtifactStore(
2168         self.settings.output_dir,
2169         run_id,
2170     )
2171
2172     store.save_json("00_input_structure.json", input_structure)
2173     store.save_json(
2174         "00_evaluation_field_policy.json",
2175         data_bundle.evaluation_field_policy,
2176     )
2177     store.save_json("00_available_capabilities.json", capabilities)
2178     store.save_json(
2179         "00_structural_catalog.json",
2180         structural_catalog,
2181     )
2182
2183     models = {
2184         role: self.settings.model_for(role)
2185         for role in [
2186             "data_understanding",
2187             "orchestrator",
2188             "evidence",
2189             "verifier",
2190             "writer",
2191             "auditor",
2192         ]
2193     }
2194
2195     (
2196         initial_manifest_genre,
2197         initial_genre_source,
2198         initial_genre_confidence,
2199     ) = resolve_report_genre(
2200         request=request,
2201         planned_genre=preliminary_inferred_contract.report_genre,
2202         configured_genre=report_genre,
2203         input_structure=input_structure,
2204         inferred_contract=preliminary_inferred_contract,
2205     )
2206     preliminary_resolved_contract = resolve_task_contract(
2207         inferred=preliminary_inferred_contract,
2208         selected_genre=initial_manifest_genre,
2209         genre_source=initial_genre_source,
2210         genre_confidence=initial_genre_confidence,
2211         configured_communication_task=communication_task,
2212         configured_output_form=output_form,
2213         configured_focus_scope=focus_scope,
2214     )
2215     initial_manifest_task = preliminary_resolved_contract.communication_task
2216     short_form_realisation_task = is_short_form_realisation_task(
2217         initial_manifest_task,
2218         preliminary_resolved_contract.output_form,
2219     ) and not self.settings.force_llm_short_form_writer
2220     manifest = RunManifest(
2221         run_id=run_id,
2222         created_at=datetime.now(timezone.utc).isoformat(),
2223         input_paths=[
2224             str(path)
2225             for path in data_bundle.source_paths
2226         ],
2227         request=request,
2228         fingerprint=data_bundle.fingerprint,
2229         use_llm=self.settings.use_llm,
2230         audit_mode=audit_mode,
2231         models=models,
2232         input_representation_status=(
2233             input_structure.representation_status
2234             if input_structure is not None
2235             else InputRepresentationStatus.INVALID
2236         ),
2237         report_genre=initial_manifest_genre,
2238         communication_task=initial_manifest_task,
2239         output_form=preliminary_resolved_contract.output_form,
2240         focus_scope=preliminary_resolved_contract.focus_scope,
2241         task_contract=preliminary_resolved_contract,
2242     )
2243
2244     store.save_json("00_manifest.json", manifest)
2245     store.save_json(
2246         "00_preliminary_inferred_task_contract.json",
2247         preliminary_inferred_contract,
2248     )
2249     store.save_json("01_profile.json", profile)
2250     store.save_json(
2251         "02_profile_support_registry.json",
2252         profile_support_records,
2253     )
2254

```

```

2255     table_names = [
2256         table.table_name
2257         for table in profile.tables
2258     ]
2259     columns = {
2260         table.table_name: [
2261             column.name
2262             for column in table.columns
2263         ]
2264         for table in profile.tables
2265     }
2266
2267     if short_form_realisation_task:
2268         store.trace(
2269             "data_understanding",
2270             "skipped",
2271             {
2272                 "reason": (
2273                     "Short-form verbalisation uses deterministic input "
2274                     "structure and capability extraction instead of the "
2275                     "general analytical Data Understanding agent."
2276                 ),
2277                 "communication_task": initial_manifest_task.value,
2278             },
2279         )
2280         understanding = fallback_understanding(profile)
2281     else:
2282         understanding = await self.run_agent_or_fallback(
2283             stage="data_understanding",
2284             agent=self.data_understanding_agent,
2285             prompt=(
2286                 "Create a data understanding and analytical-risk report.\n\n"
2287                 "User request:\n"
2288                 + request
2289                 + "\n\n"
2290                 "Input structure:\n"
2291                 + compact_json(input_structure)
2292                 + "\n\nSanitized structural field catalog:\n"
2293                 + compact_json(structural_catalog)
2294                 + "\n\nSanitized data profile:\n"
2295                 + compact_json(profile)
2296             ),
2297             dependencies=AgentDependencies(
2298                 run_id=run_id,
2299                 payload={
2300                     "fingerprint": profile.fingerprint,
2301                     "table_names": table_names,
2302                     "columns": columns,
2303                     "input_structure": (
2304                         input_structure.model_dump(mode="json")
2305                         if input_structure is not None
2306                         else None
2307                     ),
2308                     "structural_catalog": [
2309                         field.model_dump(mode="json") for field in structural_catalog
2310                     ],
2311                     "semantic_map_required": bool(structural_catalog),
2312                     "user_request": request,
2313                 },
2314             ),
2315             fallback=lambda: fallback_understanding(profile),
2316             store=store,
2317         )
2318     understanding = DataUnderstanding.model_validate(understanding)
2319     semantic_map = understanding.semantic_map
2320     inferred_task_contract = infer_task_contract(
2321         request=request,
2322         structured_inputs=data_bundle.structured_inputs,
2323         input_structure=input_structure,
2324         semantic_map=semantic_map,
2325     )
2326     understanding = understanding.model_copy(
2327         update={"inferred_task_contract": inferred_task_contract}
2328     )
2329     store.save_json("02_understanding.json", understanding)
2330     store.save_json("02_semantic_map.json", semantic_map)
2331     store.save_json(
2332         "02_inferred_task_contract.json",
2333         inferred_task_contract,
2334     )
2335     capabilities = available_capabilities(
2336         data_bundle,
2337         semantic_map,
2338     )
2339     store.save_json(
2340         "02_available_capabilities.json",
2341         capabilities,
2342     )
2343     inferred_genre = (
2344         semantic_map.recommended_report_genre
2345         if semantic_map is not None and semantic_map.recommended_report_genre is not None

```

```

2346         else ReportGenre.DATA_SCIENCE_REPORT
2347     )
2348     (
2349         controller_genre,
2350         controller_genre_source,
2351         controller_genre_confidence,
2352     ) = resolve_report_genre(
2353         request=request,
2354         planned_genre=inferred_genre,
2355         configured_genre=report_genre,
2356         input_structure=input_structure,
2357         semantic_map=semantic_map,
2358         inferred_contract=inferred_task_contract,
2359     )
2360     resolved_task_contract = resolve_task_contract(
2361         inferred=inferred_task_contract,
2362         selected_genre=controller_genre,
2363         genre_source=controller_genre_source,
2364         genre_confidence=controller_genre_confidence,
2365         configured_communication_task=communication_task,
2366         configured_output_form=output_form,
2367         configured_focus_scope=focus_scope,
2368     )
2369     store.save_json("02_resolved_task_contract.json", resolved_task_contract)
2370     store.save_json(
2371         "02_task_contract_agreement.json",
2372         task_contract_agreement(
2373             inferred_task_contract,
2374             resolved_task_contract,
2375         ),
2376     )
2377     short_form_realisation_task = is_short_form_realisation_task(
2378         resolved_task_contract.communication_task,
2379         resolved_task_contract.output_form,
2380     ) and not self.settings.force_llm_short_form_writer
2381     controller_task_contract = task_contract_fields(
2382         genre=controller_genre,
2383         communication_task=resolved_task_contract.communication_task,
2384         output_form=resolved_task_contract.output_form,
2385         focus_scope=resolved_task_contract.focus_scope,
2386     )
2387     manifest = manifest.model_copy(
2388         update={
2389             "report_genre": resolved_task_contract.report_genre,
2390             "communication_task": resolved_task_contract.communication_task,
2391             "output_form": resolved_task_contract.output_form,
2392             "focus_scope": resolved_task_contract.focus_scope,
2393             "task_contract": resolved_task_contract,
2394         }
2395     )
2396     store.save_json("00_manifest.json", manifest)
2397     planner_context = build_orchestrator_prompt_context(
2398         understanding=understanding,
2399         input_structure=input_structure,
2400         structural_catalog=structural_catalog,
2401     )
2402
2403     deterministic_event_plan = should_use_deterministic_event_plan(
2404         controller_genre=controller_genre,
2405         input_structure=input_structure,
2406         semantic_map=semantic_map,
2407         capabilities=capabilities,
2408     )
2409     if short_form_realisation_task:
2410         store.trace(
2411             "orchestration_and_planning",
2412             "skipped",
2413             {
2414                 "reason": (
2415                     "Short-form verbalisation uses the controller "
2416                     "contract and deterministic capability task instead "
2417                     "of the general LLM Orchestrator."
2418                 ),
2419                 "selected_report_genre": controller_genre.value,
2420                 "communication_task": (
2421                     resolved_task_contract.communication_task.value
2422                 ),
2423                 "available_capabilities": [
2424                     capability.value for capability in capabilities
2425                 ],
2426             },
2427         )
2428     plan = fallback_execution_plan(
2429         request,
2430         profile,
2431         audit_mode,
2432         self.settings,
2433         input_structure=input_structure,
2434         available_capabilities=capabilities,
2435         report_genre_override=controller_genre,
2436     )

```



```

2437     elif deterministic_event_plan:
2438         store.trace(
2439             "orchestration_and_planning",
2440             "skipped",
2441             {
2442                 "reason": (
2443                     "High-confidence event structure uses the generic "
2444                     "deterministic capability plan instead of the LLM "
2445                     "Orchestrator retry path."
2446                 ),
2447                 "selected_report_genre": controller_genre.value,
2448                 "input_shape": input_structure.shape.value,
2449                 "available_event_capabilities": [
2450                     capability.value
2451                     for capability in capabilities
2452                     if capability in EVENT_CAPABILITIES
2453                 ],
2454             },
2455         )
2456         plan = fallback_execution_plan(
2457             request,
2458             profile,
2459             audit_mode,
2460             self.settings,
2461             input_structure=input_structure,
2462             available_capabilities=capabilities,
2463             report_genre_override=controller_genre,
2464         )
2465     else:
2466         plan = await self.run_agent_or_fallback(
2467             stage="orchestration_and_planning",
2468             agent=self.orchestrator_agent,
2469             prompt=(
2470                 "User objective:\n"
2471                 + request
2472                 + "\n\nData profile:\n"
2473                 + compact_json(profile)
2474                 + "\n\nData understanding:\n"
2475                 + compact_json(planner_context["understanding"])
2476                 + "\n\nInput structure:\n"
2477                 + compact_json(planner_context["input_structure"])
2478                 + "\n\nSanitized structural field catalog:\n"
2479                 + compact_json(planner_context["structural_catalog"])
2480                 + "\n\nID-only semantic binding catalogue:\n"
2481                 + compact_json(planner_context["semantic_binding_catalog"])
2482                 + "\n\nUse only `binding_id` values from that catalogue in all "
2483                 + "evidence-query binding fields. Raw paths are deliberately "
2484                 + "unavailable for semantic query planning.\n"
2485                 + "\n\nAvailable evidence capabilities:\n"
2486                 + compact_json(capabilities)
2487                 + "\n\nController-selected report genre:\n"
2488                 + controller_genre.value
2489                 + "\n\nController-selected task/output contract:\n"
2490                 + compact_json(controller_task_contract)
2491                 + "\n\nConfigured report genre override:\n"
2492                 + (report_genre.value if report_genre else "none")
2493                 + "\n\nAudit mode:\n"
2494                 + audit_mode.value
2495             ),
2496             dependencies=AgentDependencies(
2497                 run_id=run_id,
2498                 payload={
2499                     "table_names": table_names,
2500                     "columns": columns,
2501                     "user_request": request,
2502                     "allow_experimental_targets": (
2503                         self.settings.allow_experimental_targets
2504                     ),
2505                     "available_capabilities": [
2506                         capability.value
2507                         for capability in capabilities
2508                     ],
2509                     "event_genre_allowed": (
2510                         controller_genre in EVENT_GENRES
2511                     ),
2512                     "selected_report_genre": controller_genre.value,
2513                     "selected_task_contract": controller_task_contract,
2514                     "semantic_map": (
2515                         semantic_map.model_dump(mode="json")
2516                         if semantic_map is not None
2517                         else None
2518                     ),
2519                     "structural_catalog": [
2520                         field.model_dump(mode="json")
2521                         for field in structural_catalog
2522                     ],
2523                     "enable_insight_synthesis": (
2524                         self.settings.enable_insight_synthesis
2525                     ),
2526                 },
2527             ),

```

```

2528         fallback=lambda: fallback_execution_plan(
2529             request,
2530             profile,
2531             audit_mode,
2532             self.settings,
2533             input_structure=input_structure,
2534             available_capabilities=capabilities,
2535             report_genre_override=controller_genre,
2536         ),
2537         store=store,
2538     )
2539
2540 plan = ExecutionPlan.model_validate(plan)
2541 (
2542     selected_genre,
2543     selection_source,
2544     selection_confidence,
2545 ) = resolve_report_genre(
2546     request=request,
2547     planned_genre=plan.report_specification.genre,
2548     configured_genre=report_genre,
2549     input_structure=input_structure,
2550     semantic_map=semantic_map,
2551     inferred_contract=inferred_task_contract,
2552 )
2553 final_task_contract = resolve_task_contract(
2554     inferred=inferred_task_contract,
2555     selected_genre=selected_genre,
2556     genre_source=selection_source,
2557     genre_confidence=selection_confidence,
2558     configured_communication_task=communication_task,
2559     configured_output_form=output_form,
2560     configured_focus_scope=focus_scope,
2561 )
2562 contract_fields = {
2563     **report_contract_fields(selected_genre),
2564     **task_contract_fields(
2565         genre=selected_genre,
2566         communication_task=final_task_contract.communication_task,
2567         output_form=final_task_contract.output_form,
2568         focus_scope=final_task_contract.focus_scope,
2569     ),
2570 }
2571 required_components = (
2572     contract_fields.get("required_components", [])
2573     if (
2574         selected_genre in EVENT_GENRES
2575         or contract_fields.get("communication_task")
2576         in {
2577             CommunicationTask.FOCUSED_TABLE_DESCRIPTION,
2578             CommunicationTask.ATTRIBUTE_VERBALISATION,
2579             CommunicationTask.TRIPLE_VERBALISATION,
2580         }
2581     )
2582     else infer_required_report_components(request)
2583 )
2584 report_specification = plan.report_specification.model_copy(
2585     update={
2586         "report_purpose": request,
2587         "genre": selected_genre,
2588         "selection_source": final_task_contract.selection_source,
2589         "selection_confidence": final_task_contract.confidence,
2590         "unresolved_ambiguities": (
2591             final_task_contract.unresolved_ambiguities
2592         ),
2593         "target_length_words": (
2594             plan.report_specification.target_length_words
2595         ),
2596         "maximum_length_words": (
2597             self.settings.writer_max_words
2598         ),
2599         "maximum_main_findings": (
2600             self.settings.writer_max_main_findings
2601         ),
2602         "maximum_supporting_facts": (
2603             self.settings.writer_supporting_fact_limit
2604         ),
2605         "required_components": list(
2606             dict.fromkeys(
2607                 [
2608                     *plan.report_specification.required_components,
2609                     *required_components,
2610                 ]
2611             )
2612         ),
2613         **contract_fields,
2614     }
2615 )
2616 resolved_task_contract = final_task_contract
2617 store.save_json("02_resolved_task_contract.json", resolved_task_contract)
2618 store.save_json(

```

```

2619         "02_task_contract_agreement.json",
2620         task_contract_agreement(
2621             inferred_task_contract,
2622             resolved_task_contract,
2623         ),
2624     )
2625     selected_capabilities = [
2626         capability
2627         for capability in capabilities
2628         if (
2629             (
2630                 report_specification.communication_task
2631                 == CommunicationTask.FOCUSED_TABLE_DESCRIPTION
2632                 and capability
2633                 == EvidenceCapability.FOCUSED_TABLE_REGION
2634             )
2635             or (
2636                 report_specification.communication_task
2637                 != CommunicationTask.FOCUSED_TABLE_DESCRIPTION
2638                 and (
2639                     selected_genre not in EVENT_GENRES
2640                     or capability
2641                     in {
2642                         EvidenceCapability.DATASET_PROFILE,
2643                         *EVENT_CAPABILITIES,
2644                     }
2645                 )
2646             )
2647         ]
2648     ]
2649     plan = plan.model_copy(
2650         update={
2651             "objective": request,
2652             "report_specification": report_specification,
2653             "available_capabilities": capabilities,
2654             "selected_capabilities": selected_capabilities,
2655             "audit_mode": audit_mode,
2656             "revision_limit": min(
2657                 plan.revision_limit,
2658                 self.settings.max_revision_rounds,
2659             ),
2660             "maximum_facts": None,
2661             "insight_objectives": (
2662                 plan.insight_objectives
2663                 if self.settings.enable_insight_synthesis
2664                 else []
2665             ),
2666             "frozen": True,
2667         }
2668     )
2669     plan = add_event_capability_tasks(
2670         plan=plan,
2671         request=request,
2672         profile=profile,
2673         audit_mode=audit_mode,
2674         settings=self.settings,
2675         input_structure=input_structure,
2676         capabilities=capabilities,
2677         genre=selected_genre,
2678     )
2679     plan = add_focused_table_capability_task(
2680         plan=plan,
2681         profile=profile,
2682         capabilities=capabilities,
2683         enable_insight_synthesis=self.settings.enable_insight_synthesis,
2684     )
2685     plan = add_structured_record_capability_task(
2686         plan=plan,
2687         profile=profile,
2688         capabilities=capabilities,
2689         enable_insight_synthesis=self.settings.enable_insight_synthesis,
2690     )
2691     final_manifest = manifest.model_copy(
2692         update={
2693             "report_genre": plan.report_specification.genre,
2694             "communication_task": plan.report_specification.communication_task,
2695             "output_form": plan.report_specification.output_form,
2696             "focus_scope": plan.report_specification.focus_scope,
2697             "task_contract": resolved_task_contract,
2698         }
2699     )
2700     store.save_json("00_manifest.json", final_manifest)
2701     if (
2702         selected_genre in EVENT_GENRES
2703         and semantic_map is not None
2704         and semantic_map.bindings
2705     ):
2706         plan = plan.model_copy(
2707             update={
2708                 "evidence_queries": normalise_event_evidence_queries(
2709                     queries=plan.evidence_queries,

```

```

2710         semantic_map=semantic_map,
2711         tasks=plan.tasks,
2712         available_capabilities=set(capabilities),
2713         request=request,
2714         structural_catalog=structural_catalog,
2715     )
2716 }
2717 )
2718 manifest = manifest.model_copy(
2719     update={
2720         "report_genre": selected_genre,
2721         "communication_task": (
2722             plan.report_specification.communication_task
2723         ),
2724         "output_form": plan.report_specification.output_form,
2725         "focus_scope": plan.report_specification.focus_scope,
2726         "task_contract": resolved_task_contract,
2727     }
2728 )
2729 store.save_json("00_manifest.json", manifest)
2730 store.save_json("03_execution_plan.json", plan)
2731 store.save_json(
2732     "03_evidence_queries.json",
2733     plan.evidence_queries,
2734 )
2735 store.save_json(
2736     "03_insight_objectives.json",
2737     plan.insight_objectives,
2738 )
2739
2740 evidence_ledger = execute_plan(
2741     data_bundle,
2742     plan,
2743     self.settings,
2744     semantic_map,
2745 )
2746 store.save_json("04_evidence_ledger.json", evidence_ledger)
2747
2748 fact_candidate_scaffold = deterministic_fact_candidate_scaffold(
2749     evidence_ledger,
2750     maximum_facts=None,
2751 )
2752 store.save_json(
2753     "05_fact_candidates_scaffold.json",
2754     fact_candidate_scaffold,
2755 )
2756
2757 if short_form_realisation_task:
2758     store.trace(
2759         "evidence_synthesis",
2760         "skipped",
2761         {
2762             "reason": (
2763                 "Short-form verbalisation uses deterministic atomic "
2764                 "fact candidates from direct evidence extraction."
2765             )
2766         },
2767     )
2768 fact_candidate_enrichment = empty_fact_candidate_enrichment()
2769 else:
2770     fact_candidate_enrichment = await self.run_agent_or_fallback(
2771         stage="evidence_synthesis",
2772         agent=self.evidence_agent,
2773         prompt=(
2774             "Review this deterministic fact-candidate scaffold and "
2775             "return only fact candidates that materially improve, "
2776             "correct, combine, or prioritise the scaffold while staying "
2777             "strictly grounded in the evidence. You do not need to cover "
2778             "every evidence item; deterministic scaffold coverage is "
2779             "already preserved.\n\nEvidence ledger:\n"
2780             + compact_json(evidence_ledger)
2781             + "\n\nDeterministic scaffold:\n"
2782             + compact_json(fact_candidate_scaffold)
2783             + "\n\nEnrichment policy:\n"
2784             + "Return a concise set of higher-quality candidates only. "
2785             + "Single-evidence candidates may improve ordinary scaffold "
2786             + "candidates, but concrete event-sequence scaffold facts are "
2787             + "preserved. Multi-evidence candidates may add bounded "
2788             + "synthesis. Do not drop evidence coverage; the controller "
2789             + "will merge your valid candidates with the deterministic "
2790             + "scaffold."
2791         ),
2792         dependencies=AgentDependencies(
2793             run_id=run_id,
2794             payload={
2795                 "evidence_ledger": evidence_ledger.model_dump(mode="json")
2796             },
2797         ),
2798         fallback=empty_fact_candidate_enrichment,
2799         store=store,
2800     )

```

```

2801 fact_candidate_enrichment = FactCandidateSet.model_validate(
2802     fact_candidate_enrichment
2803 )
2804 store.save_json(
2805     "05_fact_candidates_enrichment.json",
2806     fact_candidate_enrichment,
2807 )
2808
2809 fact_candidates = merge_fact_candidate_scaffold(
2810     scaffold=fact_candidate_scaffold,
2811     enrichment=fact_candidate_enrichment,
2812     evidence=evidence_ledger,
2813 )
2814 store.save_json("05_fact_candidates.json", fact_candidates)
2815
2816 if short_form_realisation_task:
2817     store.trace(
2818         "fact_verification",
2819         "skipped",
2820         {
2821             "reason": (
2822                 "Short-form facts come from deterministic direct "
2823                 "extraction and are verified by deterministic "
2824                 "fallback review."
2825             )
2826         },
2827     )
2828     verification = fallback_verification(fact_candidates)
2829 else:
2830     verification = await self.run_agent_or_fallback(
2831         stage="fact_verification",
2832         agent=self.verifier_agent,
2833         prompt=(
2834             "Verify every fact candidate against the evidence.\n\n"
2835             "Candidates:\n"
2836             + compact_json(fact_candidates)
2837             + "\n\nEvidence:\n"
2838             + compact_json(evidence_ledger)
2839         ),
2840         dependencies=AgentDependencies(
2841             run_id=run_id,
2842             payload={
2843                 "fact_candidates": fact_candidates.model_dump(mode="json")
2844             },
2845         ),
2846         fallback=lambda: fallback_verification(
2847             fact_candidates
2848         ),
2849         store=store,
2850     )
2851     verification = VerificationResult.model_validate(verification)
2852     raw_verification = verification
2853     verification = repair_spurious_missing_evidence_rejections(
2854         candidate_set=fact_candidates,
2855         verification=verification,
2856         evidence=evidence_ledger,
2857     )
2858     if (
2859         verification.model_dump(mode="json")
2860         != raw_verification.model_dump(mode="json")
2861     ):
2862         store.save_json("06_verification_raw.json", raw_verification)
2863     store.save_json("06_verification.json", verification)
2864
2865     fact_ledger = finalise_fact_ledger(
2866         fact_candidates,
2867         verification,
2868         evidence_ledger,
2869     )
2870     if not fact_ledger.writer_ready_facts:
2871         store.trace(
2872             "fact_ledger_finalisation",
2873             "recovery",
2874             {
2875                 "reason": "Verifier rejected every fact candidate.",
2876             },
2877         )
2878         fact_candidates = fallback_fact_candidates(
2879             evidence_ledger,
2880             plan.maximum_facts,
2881         )
2882         verification = fallback_verification(fact_candidates)
2883         fact_ledger = finalise_fact_ledger(
2884             fact_candidates,
2885             verification,
2886             evidence_ledger,
2887         )
2888         store.save_json("05_fact_candidates_recovered.json", fact_candidates)
2889         store.save_json("06_verification_recovered.json", verification)
2890         store.save_json("07_fact_ledger_recovered.json", fact_ledger)
2891     store.save_json(

```

```

2892         "07_fact_ledger_pre_coverage_recovery.json",
2893         fact_ledger,
2894     )
2895
2896     fact_count_before_recovery = len(
2897         fact_ledger.writer_ready_facts
2898     )
2899
2900     fact_ledger = (
2901         augment_fact_ledger_for_report_coverage(
2902             fact_ledger=fact_ledger,
2903             evidence=evidence_ledger,
2904             required_components=(
2905                 plan.report_specification
2906                 .required_components
2907             ),
2908             required_content_slots=(
2909                 plan.report_specification
2910                 .required_content_slots
2911             ),
2912             settings=self.settings,
2913         )
2914     )
2915
2916     store.trace(
2917         "fact_ledger_coverage_recovery",
2918         "completed",
2919         {
2920             "facts_before": (
2921                 fact_count_before_recovery
2922             ),
2923             "facts_after": len(
2924                 fact_ledger
2925                 .writer_ready_facts
2926             ),
2927             "recovered_fact_ids": (
2928                 fact_ledger
2929                 .deterministically_recovered_fact_ids
2930             ),
2931             "notes": (
2932                 fact_ledger
2933                 .coverage_recovery_notes
2934             ),
2935         },
2936     )
2937
2938     store.save_json(
2939         "07_fact_ledger.json",
2940         fact_ledger,
2941     )
2942     genre_scoped_fact_ledger = scope_fact_ledger_for_genre(
2943         fact_ledger,
2944         evidence_ledger,
2945         plan.report_specification.genre,
2946     )
2947     if (
2948         plan.report_specification.communication_task
2949         in {
2950             CommunicationTask.FOCUSED_TABLE_DESCRIPTION,
2951             CommunicationTask.ATTRIBUTE_VERBALISATION,
2952             CommunicationTask.TRIPLE_VERBALISATION,
2953         }
2954     ):
2955         allowed_capability = (
2956             EvidenceCapability.FOCUSED_TABLE_REGION
2957             if (
2958                 plan.report_specification.communication_task
2959                 == CommunicationTask.FOCUSED_TABLE_DESCRIPTION
2960             )
2961             else EvidenceCapability.STRUCTURED_RECORD_VERBALISATION
2962         )
2963         evidence_by_id = {
2964             item.evidence_id: item
2965             for item in evidence_ledger.items
2966         }
2967         genre_scoped_fact_ledger = (
2968             genre_scoped_fact_ledger.model_copy(
2969                 update={
2970                     "writer_ready_facts": [
2971                         fact
2972                         for fact
2973                         in genre_scoped_fact_ledger.writer_ready_facts
2974                         if allowed_capability in fact.source_capabilities
2975                         or any(
2976                             evidence_by_id[evidence_id].capability
2977                             == allowed_capability
2978                             for evidence_id in fact.evidence_ids
2979                             if evidence_id in evidence_by_id
2980                         )
2981                     ]
2982                 }

```

```

2983         )
2984     )
2985
2986     insight_candidates = InsightCandidateSet()
2987     insight_verification = InsightVerificationResult()
2988
2989     if short_form_realisation_task:
2990         insight_ledger = empty_insight_ledger(
2991             synthesis_enabled=self.settings.enable_insight_synthesis,
2992             fallback_reason=(
2993                 "Short-form verbalisation skips bounded insight "
2994                 "synthesis; direct record evidence is sufficient for "
2995                 "the requested output form."
2996             ),
2997         )
2998         store.trace(
2999             "evidence.insight_synthesis",
3000             "skipped",
3001             {"reason": insight_ledger.fallback_reason},
3002         )
3003         store.trace(
3004             "verifier.insight_verification",
3005             "skipped",
3006             {"reason": insight_ledger.fallback_reason},
3007         )
3008     elif not self.settings.enable_insight_synthesis:
3009         insight_ledger = empty_insight_ledger(
3010             synthesis_enabled=False,
3011             fallback_reason=(
3012                 "Insight synthesis disabled by configuration."
3013             ),
3014         )
3015         store.trace(
3016             "evidence.insight_synthesis",
3017             "skipped",
3018             {"reason": insight_ledger.fallback_reason},
3019         )
3020         store.trace(
3021             "verifier.insight_verification",
3022             "skipped",
3023             {"reason": insight_ledger.fallback_reason},
3024         )
3025     elif not self.settings.use_llm:
3026         insight_ledger = empty_insight_ledger(
3027             synthesis_enabled=True,
3028             fallback_reason=(
3029                 "LLM execution disabled; the workflow continued through "
3030                 "the existing fact-led Writer path."
3031             ),
3032         )
3033         store.trace(
3034             "evidence.insight_synthesis",
3035             "skipped",
3036             {"reason": "LLM execution disabled"},
3037         )
3038         store.trace(
3039             "verifier.insight_verification",
3040             "skipped",
3041             {"reason": "LLM execution disabled"},
3042         )
3043     else:
3044         insight_payload = build_compact_insight_payload(
3045             request=request,
3046             plan=plan,
3047             fact_ledger=genre_scoped_fact_ledger,
3048             evidence_ledger=evidence_ledger,
3049             settings=self.settings,
3050         )
3051         raw_insight_candidates, synthesis_error = (
3052             await self.run_optional_insight_agent(
3053                 stage="evidence.insight_synthesis",
3054                 agent=self.evidence_insight_synthesis_agent,
3055                 prompt=(
3056                     "Perform the Evidence Analyst's second bounded "
3057                     "synthesis pass. Use only this compact package and "
3058                     "return structured insight candidates.\n\n"
3059                     + compact_json(insight_payload)
3060                 ),
3061                 dependencies=AgentDependencies(
3062                     run_id=run_id,
3063                     payload={
3064                         "fact_ledger": (
3065                             genre_scoped_fact_ledger.model_dump(
3066                                 mode="json"
3067                             )
3068                         ),
3069                         "evidence_ledger": evidence_ledger.model_dump(
3070                             mode="json"
3071                         ),
3072                     },
3073                 ),

```

```

3074         store=store,
3075     )
3076 )
3077
3078 if synthesis_error is not None:
3079     insight_ledger = empty_insight_ledger(
3080         synthesis_enabled=True,
3081         fallback_reason=(
3082             "Evidence Analyst second-pass insight synthesis "
3083             f"failed: {synthesis_error}"
3084         ),
3085     )
3086 else:
3087     try:
3088         insight_candidates = InsightCandidateSet.model_validate(
3089             raw_insight_candidates
3090         )
3091     except Exception as error:
3092         insight_ledger = empty_insight_ledger(
3093             synthesis_enabled=True,
3094             fallback_reason=(
3095                 "Evidence Analyst second-pass output remained "
3096                 "invalid: "
3097                 f"{type(error).__name__}: {error}"
3098             ),
3099         )
3100     else:
3101         if not insight_candidates.candidates:
3102             insight_ledger = empty_insight_ledger(
3103                 synthesis_enabled=True,
3104                 fallback_reason=(
3105                     "Evidence Analyst second pass returned no "
3106                     "bounded insight candidates."
3107                 ),
3108             )
3109         else:
3110             referenced_evidence_ids = {
3111                 evidence_id
3112                 for candidate in insight_candidates.candidates
3113                 for evidence_id in candidate.source_evidence_ids
3114             }
3115             referenced_evidence_ids.update(
3116                 evidence_id
3117                 for fact in genre_scoped_fact_ledger.writer_ready_facts
3118                 if fact.fact_id
3119                 in {
3120                     fact_id
3121                     for candidate in insight_candidates.candidates
3122                     for fact_id in candidate.source_fact_ids
3123                 }
3124                 for evidence_id in fact.evidence_ids
3125             )
3126             verifier_evidence = [
3127                 item
3128                 for item in evidence_ledger.items
3129                 if item.evidence_id
3130                 in referenced_evidence_ids
3131             ]
3132             raw_insight_verification, verifier_error = (
3133                 await self.run_optional_insight_agent(
3134                     stage="verifier.insight_verification",
3135                     agent=(
3136                         self.verifier_insight_verification_agent
3137                     ),
3138                     prompt=(
3139                         "Perform the Fact Verifier's second-pass "
3140                         "review of every bounded insight candidate."
3141                         "\n\nCandidates:\n"
3142                         + compact_json(insight_candidates)
3143                         + "\n\nWriter-ready facts:\n"
3144                         + compact_json(
3145                             genre_scoped_fact_ledger
3146                             .writer_ready_facts
3147                         )
3148                         + "\n\nReferenced deterministic evidence:\n"
3149                         + compact_json(verifier_evidence)
3150                         + "\n\nReport specification:\n"
3151                         + compact_json(
3152                             plan.report_specification
3153                         )
3154                     ),
3155                     dependencies=AgentDependencies(
3156                         run_id=run_id,
3157                         payload={
3158                             "insight_candidates": (
3159                                 insight_candidates.model_dump(
3160                                     mode="json"
3161                                 )
3162                             ),
3163                             "fact_ledger": (
3164                                 genre_scoped_fact_ledger

```



```

3165         .model_dump(
3166             mode="json"
3167         )
3168     ),
3169     "evidence_ledger": (
3170         evidence_ledger.model_dump(
3171             mode="json"
3172         )
3173     ),
3174 },
3175 ),
3176     store=store,
3177 )
3178 )
3179
3180 verification_notes: list[str] = []
3181 valid_records = []
3182 unresolved_candidates = list(
3183     insight_candidates.candidates
3184 )
3185
3186 if verifier_error is not None:
3187     verification_notes.append(
3188         "Batch insight verification failed: "
3189         + verifier_error
3190     )
3191 else:
3192     try:
3193         batch_verification = (
3194             InsightVerificationResult.model_validate(
3195                 raw_insight_verification
3196             )
3197         )
3198     except Exception as error:
3199         verification_notes.append(
3200             "Batch insight verification could not be "
3201             "parsed: "
3202             f"{type(error).__name__}: {error}"
3203         )
3204     else:
3205         batch_errors = validate_insight_verification(
3206             batch_verification,
3207             insight_candidates,
3208             genre_scoped_fact_ledger,
3209             evidence_ledger,
3210             self.settings,
3211             plan.report_specification.genre,
3212         )
3213         verification_notes.extend(batch_errors)
3214         unresolved_ids = {
3215             candidate.insight_id
3216             for candidate
3217             in insight_candidates.candidates
3218             if any(
3219                 candidate.insight_id in error
3220                 for error in batch_errors
3221             )
3222             or sum(
3223                 record.insight_id
3224                 == candidate.insight_id
3225                 for record
3226                 in batch_verification.records
3227             )
3228             != 1
3229         }
3230         valid_records = [
3231             record
3232             for record in batch_verification.records
3233             if record.insight_id
3234             not in unresolved_ids
3235             and record.insight_id
3236             in {
3237                 candidate.insight_id
3238                 for candidate
3239                 in insight_candidates.candidates
3240             }
3241         ]
3242         unresolved_candidates = [
3243             candidate
3244             for candidate
3245             in insight_candidates.candidates
3246             if candidate.insight_id in unresolved_ids
3247         ]
3248         verification_notes.extend(
3249             batch_verification.verifier_notes
3250         )
3251
3252 for retry_index, candidate in enumerate(
3253     unresolved_candidates,
3254     start=1,
3255 ):

```

```

3256 candidate_set = InsightCandidateSet(
3257     candidates=[candidate]
3258 )
3259 candidate_fact_ids = set(
3260     candidate.source_fact_ids
3261 )
3262 candidate_facts = [
3263     fact
3264     for fact
3265     in genre_scoped_fact_ledger.writer_ready_facts
3266     if fact.fact_id in candidate_fact_ids
3267 ]
3268 candidate_evidence_ids = {
3269     evidence_id
3270     for fact in candidate_facts
3271     for evidence_id in fact.evidence_ids
3272 }
3273 candidate_evidence_ids.update(
3274     candidate.source_evidence_ids
3275 )
3276 candidate_evidence = [
3277     item
3278     for item in evidence_ledger.items
3279     if item.evidence_id
3280     in candidate_evidence_ids
3281 ]
3282 retry_output, retry_error = (
3283     await self.run_optional_insight_agent(
3284         stage=(
3285             "verifier.insight_verification.retry."
3286             f"{retry_index:03d}"
3287         ),
3288         agent=(
3289             self
3290             .verifier_insight_verification_agent
3291         ),
3292         prompt=(
3293             "Review this one bounded insight "
3294             "candidate independently. Return "
3295             "exactly one verification record."
3296             "\n\nCandidate:\n"
3297             + compact_json(candidate_set)
3298             + "\n\nWriter-ready facts:\n"
3299             + compact_json(candidate_facts)
3300             + "\n\nReferenced deterministic "
3301             "evidence:\n"
3302             + compact_json(candidate_evidence)
3303             + "\n\nReport specification:\n"
3304             + compact_json(
3305                 plan.report_specification
3306             )
3307         ),
3308         dependencies=AgentDependencies(
3309             run_id=run_id,
3310             payload={
3311                 "insight_candidates": (
3312                     candidate_set.model_dump(
3313                         mode="json"
3314                     )
3315                 ),
3316                 "fact_ledger": (
3317                     genre_scoped_fact_ledger
3318                     .model_dump(mode="json")
3319                 ),
3320                 "evidence_ledger": (
3321                     evidence_ledger.model_dump(
3322                         mode="json"
3323                     )
3324                 ),
3325             },
3326         ),
3327         store=store,
3328     )
3329 )
3330 if retry_error is not None:
3331     verification_notes.append(
3332         f"{candidate.insight_id} verification "
3333         f"retry failed: {retry_error}"
3334     )
3335     continue
3336
3337 try:
3338     retry_verification = (
3339         InsightVerificationResult.model_validate(
3340             retry_output
3341         )
3342     )
3343 except Exception as error:
3344     verification_notes.append(
3345         f"{candidate.insight_id} verification "
3346         "retry could not be parsed: "

```

```

3347         f"{type(error).__name__}: {error}"
3348     )
3349     continue
3350
3351     retry_errors = validate_insight_verification(
3352         retry_verification,
3353         candidate_set,
3354         genre_scoped_fact_ledger,
3355         evidence_ledger,
3356         self.settings,
3357         plan.report_specification.genre,
3358     )
3359     matching_records = [
3360         record
3361         for record
3362         in retry_verification.records
3363         if record.insight_id
3364         == candidate.insight_id
3365     ]
3366     if retry_errors or len(matching_records) != 1:
3367         verification_notes.extend(retry_errors)
3368         if len(matching_records) != 1:
3369             verification_notes.append(
3370                 f"{candidate.insight_id} verification "
3371                 "retry did not return exactly one "
3372                 "matching record."
3373             )
3374         continue
3375
3376     valid_records.append(matching_records[0])
3377     verification_notes = [
3378         note
3379         for note in verification_notes
3380         if candidate.insight_id not in note
3381     ]
3382     verification_notes.append(
3383         f"{candidate.insight_id} was recovered by "
3384         "individual verification."
3385     )
3386     verification_notes.extend(
3387         retry_verification.verifier_notes
3388     )
3389
3390     if len(valid_records) == len(
3391         insight_candidates.candidates
3392     ):
3393         verification_notes = [
3394             note
3395             for note in verification_notes
3396             if not note.startswith(
3397                 "Batch insight verification"
3398             )
3399         ]
3400
3401     insight_verification = InsightVerificationResult(
3402         records=valid_records,
3403         verifier_notes=list(
3404             dict.fromkeys(verification_notes)
3405         ),
3406     )
3407     try:
3408         insight_ledger = materialise_insight_ledger(
3409             candidates=insight_candidates,
3410             verification=insight_verification,
3411             fact_ledger=genre_scoped_fact_ledger,
3412             evidence_ledger=evidence_ledger,
3413             settings=self.settings,
3414             report_genre=(
3415                 plan.report_specification.genre
3416             ),
3417         )
3418         if (
3419             not insight_ledger.verified_insights
3420             and insight_ledger.unverified_insights
3421             and not insight_ledger.rejected_insights
3422         ):
3423             insight_ledger = insight_ledger.model_copy(
3424                 update={
3425                     "fallback_reason": (
3426                         "No insight candidate received a "
3427                         "usable verifier review; the "
3428                         "Writer continued with verified "
3429                         "facts."
3430                     )
3431                 }
3432             )
3433     except Exception as error:
3434         insight_ledger = empty_insight_ledger(
3435             synthesis_enabled=True,
3436             fallback_reason=(
3437                 "Deterministic Insight Ledger "

```

```

3438         "materialisation failed: "
3439         f"{type(error).__name__}: {error}"
3440     ),
3441 )
3442
3443 store.save_json(
3444     "07_insight_candidates.json",
3445     insight_candidates,
3446 )
3447 store.save_json(
3448     "07_insight_verification.json",
3449     insight_verification,
3450 )
3451 store.save_json(
3452     "07_insight_ledger.json",
3453     insight_ledger,
3454 )
3455 store.trace(
3456     "insight_ledger",
3457     "completed" if insight_ledger.fallback_reason is None else "fallback",
3458     {
3459         "synthesis_enabled": insight_ledger.synthesis_enabled,
3460         "verified_insight_count": len(
3461             insight_ledger.verified_insights
3462         ),
3463         "hypothesis_only_count": len(
3464             insight_ledger.hypothesis_only_insights
3465         ),
3466         "rejected_count": len(
3467             insight_ledger.rejected_insights
3468         ),
3469         "unverified_count": len(
3470             insight_ledger.unverified_insights
3471         ),
3472         "fallback_reason": insight_ledger.fallback_reason,
3473     },
3474 )
3475
3476 writer_pack = build_writer_evidence_pack(
3477     request=request,
3478     understanding=understanding,
3479     plan=plan,
3480     evidence=evidence_ledger,
3481     fact_ledger=fact_ledger,
3482     settings=self.settings,
3483     insight_ledger=insight_ledger,
3484     input_structure=input_structure,
3485     available_capabilities=capabilities,
3486 )
3487 store.save_json("08_writer_evidence_pack.json", writer_pack)
3488 writer_visible_fact_ledger = FactLedger(
3489     writer_ready_facts=[
3490         *writer_pack.priority_facts,
3491         *writer_pack.supporting_facts,
3492         *writer_pack.limitation_facts,
3493     ],
3494     rejected_facts=genre_scoped_fact_ledger.rejected_facts,
3495     verifier_notes=genre_scoped_fact_ledger.verifier_notes,
3496     deterministically_recovered_fact_ids=(
3497         genre_scoped_fact_ledger
3498         .deterministically_recovered_fact_ids
3499     ),
3500     coverage_recovery_notes=(
3501         genre_scoped_fact_ledger.coverage_recovery_notes
3502     ),
3503 )
3504 writer_content_requirements = build_writer_content_requirements(
3505     report_specification=plan.report_specification,
3506     fact_ledger=writer_visible_fact_ledger,
3507     evidence=evidence_ledger,
3508     insight_ledger=writer_pack.insight_ledger,
3509     settings=self.settings,
3510 )
3511 store.save_json(
3512     "08_writer_content_requirements.json",
3513     writer_content_requirements,
3514 )
3515 narrative_plan = build_event_narrative_plan(
3516     writer_pack,
3517     writer_content_requirements,
3518 )
3519 store.save_json(
3520     "08_narrative_plan.json",
3521     narrative_plan,
3522 )
3523
3524 writer_prompt = (
3525     "Write the final report for the selected report contract from the "
3526     "compact verified-fact package below.\n\n"
3527     "The `content_requirements` field is a controller-enforced "
3528     "coverage checklist. Use the required supported items and meet "

```

```

3529         "the minimum useful word count when it is enforced.\n\n"
3530         "When `event_report_writing_guidance` is present, follow it as "
3531         "the task style contract: write a coherent event recap, lead with "
3532         "the supported result, integrate supported sequence/progression "
3533         "and performances, and avoid flat-table profiling or mechanical "
3534         "ranking dumps.\n\n"
3535         "When `narrative_plan` is present, use it to decide ordering, "
3536         "paragraph grouping and salience. Cover higher-priority narrative "
3537         "slots first, use low-priority fact IDs only when they add "
3538         "distinct value, and keep the prose flowing instead of listing "
3539         "every available ranking.\n\n"
3540         "When `realisation_policy` or `style_rewrite_permissions` are "
3541         "present, use them only to improve phrasing, ordering, compression "
3542         "and harmless surface formatting. They do not authorise new facts, "
3543         "new numbers, new entities, unsupported chronology, or unsupported "
3544         "explanations.\n\n"
3545         "Return structured sections and sentences. Do not return "
3546         "a Markdown field or construct a separate support map; the "
3547         "controller will create both deterministically.\n\n"
3548         + compact_json(
3549             build_compact_writer_payload(
3550                 writer_pack,
3551                 self.settings.allow_hypotheses_in_report,
3552                 writer_content_requirements,
3553                 narrative_plan,
3554             )
3555         )
3556     )
3557
3558     writer_material_available = bool(
3559         writer_pack.priority_facts
3560         or writer_pack.supporting_facts
3561         or writer_pack.limitation_facts
3562         or writer_pack.priority_verified_insights
3563         or writer_pack.supporting_verified_insights
3564     )
3565     if writer_material_available:
3566         writer_draft_or_fallback = await self.run_agent_or_fallback(
3567             stage="natural_writer",
3568             agent=self.writer_agent,
3569             prompt=writer_prompt,
3570             dependencies=AgentDependencies(
3571                 run_id=run_id,
3572                 payload={
3573                     "fact_ledger": genre_scoped_fact_ledger.model_dump(mode="json"),
3574                     "insight_ledger": writer_pack.insight_ledger.model_dump(
3575                         mode="json"
3576                     ),
3577                     "allow_hypotheses_in_report": (
3578                         self.settings.allow_hypotheses_in_report
3579                     ),
3580                     "report_genre": plan.report_specification.genre.value,
3581                     "report_perspective": (
3582                         plan.report_specification.perspective.value
3583                     ),
3584                     "maximum_length_words": (
3585                         plan.report_specification.maximum_length_words
3586                     ),
3587                     "writer_content_requirements": (
3588                         writer_content_requirements
3589                     ),
3590                     "narrative_plan": narrative_plan.model_dump(
3591                         mode="json"
3592                     ),
3593                 },
3594             ),
3595             fallback=lambda: fallback_writer(writer_pack),
3596             store=store,
3597         )
3598     else:
3599         writer_draft_or_fallback = fallback_writer(writer_pack)
3600         store.trace(
3601             "natural_writer",
3602             "skipped",
3603             {
3604                 "reason": "No verified genre-scoped facts or insights.",
3605                 "fallback": "deterministic_writer",
3606             },
3607         )
3608
3609     if isinstance(
3610         writer_draft_or_fallback,
3611         WriterOutput,
3612     ):
3613         raw_writer_output = (
3614             writer_draft_or_fallback
3615         )
3616     else:
3617         writer_draft = (
3618             WriterAgentDraft.model_validate(
3619                 writer_draft_or_fallback

```

```

3620     )
3621     )
3622     store.save_json(
3623         "09_writer_structured_draft.json",
3624         writer_draft,
3625     )
3626
3627     try:
3628         raw_writer_output = materialise_writer_output(
3629             writer_draft,
3630             genre_scoped_fact_ledger,
3631             insight_ledger=writer_pack.insight_ledger,
3632             allow_hypotheses_in_report=(
3633                 self.settings.allow_hypotheses_in_report
3634             ),
3635             content_requirements=writer_content_requirements,
3636             writer_mode="llm_writer",
3637             eligible_for_primary_evaluation=representation_eligible,
3638         )
3639     except ValueError as error:
3640         store.save_text(
3641             "09_writer_materialisation_error.txt",
3642             str(error),
3643         )
3644         store.trace(
3645             "natural_writer_materialisation",
3646             "fallback",
3647             {
3648                 "reason": f"ValueError: {error}",
3649                 "fallback": "deterministic_writer",
3650             },
3651         )
3652         raw_writer_output = fallback_writer(
3653             writer_pack
3654         )
3655
3656     if not representation_eligible:
3657         raw_writer_output = raw_writer_output.model_copy(
3658             update={"eligible_for_primary_evaluation": False}
3659         )
3660
3661     store.save_json(
3662         "09_writer_raw_output.json",
3663         raw_writer_output,
3664     )
3665     store.save_text(
3666         "09_writer_raw_report.md",
3667         raw_writer_output.markdown,
3668     )
3669     store.save_json(
3670         "09_writer_support_map.json",
3671         raw_writer_output.sentence_support,
3672     )
3673
3674     component_assessments = assess_report_component_coverage(
3675         writer_output=raw_writer_output,
3676         fact_ledger=fact_ledger,
3677         evidence=evidence_ledger,
3678         required_components=plan.report_specification.required_components,
3679     )
3680     missing_components = [
3681         assessment.component
3682         for assessment in component_assessments
3683         if not assessment.covered
3684     ]
3685     store.save_json(
3686         "09_writer_component_coverage.json",
3687         component_assessments,
3688     )
3689     initial_genre_quality = assess_genre_quality(
3690         raw_writer_output,
3691         plan.report_specification,
3692         evidence_ledger,
3693     )
3694     store.save_json(
3695         "09_writer_genre_quality.json",
3696         initial_genre_quality,
3697     )
3698
3699     initial_quality_audit = deterministic_audit(
3700         writer_output=raw_writer_output,
3701         fact_ledger=fact_ledger,
3702         evidence=evidence_ledger,
3703         mode=audit_mode,
3704         external_sources=external_truth_sources,
3705         revision_round=0,
3706         report_specification=plan.report_specification,
3707         settings=self.settings,
3708         profile_support_records=profile_support_records,
3709         insight_ledger=insight_ledger,
3710     )

```

```

3711     store.save_json("10_initial_writer_quality.json", initial_quality_audit)
3712
3713     writer_output_for_audit = raw_writer_output
3714     quality_revised_writer_output: WriterOutput | None = None
3715     short_form_output = (
3716         plan.report_specification.communication_task
3717         in SHORT_FORM_COMMUNICATION_TASKS
3718     )
3719     needs_quality_revision = (
3720         not short_form_output
3721         and (
3722             bool(missing_components)
3723             or initial_quality_audit.quality_assessment.status
3724             != QualityStatus.PASS
3725             or initial_genre_quality.status == QualityStatus.REVISE
3726         )
3727     )
3728
3729     if (
3730         needs_quality_revision
3731         and writer_material_available
3732         and self.settings.use_llm
3733         and self.writer_agent is not None
3734         and self.settings.writer_quality_revision_rounds > 0
3735     ):
3736         revised_draft_or_fallback = await self.run_agent_or_fallback(
3737             stage="writer_quality_revision",
3738             agent=self.writer_agent,
3739             prompt=build_writer_quality_revision_prompt(
3740                 writer_pack=writer_pack,
3741                 current_output=raw_writer_output,
3742                 missing_components=missing_components,
3743                 quality_findings=(
3744                     [
3745                         *initial_quality_audit.quality_assessment.findings,
3746                         *initial_genre_quality.findings,
3747                     ]
3748                 ),
3749                 content_requirements=(
3750                     writer_content_requirements
3751                 ),
3752                 settings=self.settings,
3753             ),
3754             dependencies=AgentDependencies(
3755                 run_id=run_id,
3756                 payload={
3757                     "fact_ledger": (
3758                         genre_scoped_fact_ledger.model_dump(
3759                             mode="json"
3760                         )
3761                     ),
3762                     "insight_ledger": (
3763                         writer_pack.insight_ledger.model_dump(
3764                             mode="json"
3765                         )
3766                     ),
3767                     "allow_hypotheses_in_report": (
3768                         self.settings.allow_hypotheses_in_report
3769                     ),
3770                     "report_genre": (
3771                         plan.report_specification.genre.value
3772                     ),
3773                     "report_perspective": (
3774                         plan.report_specification.perspective.value
3775                     ),
3776                     "maximum_length_words": (
3777                         plan.report_specification.maximum_length_words
3778                     ),
3779                     "writer_content_requirements": (
3780                         writer_content_requirements
3781                     ),
3782                 },
3783             ),
3784             fallback=lambda: raw_writer_output,
3785             store=store,
3786         )
3787
3788     revision_materialisation_error: str | None = None
3789     if isinstance(
3790         revised_draft_or_fallback,
3791         WriterOutput,
3792     ):
3793         revision_candidate = (
3794             revised_draft_or_fallback
3795             .model_copy(
3796                 update={
3797                     "quality_revision_round": 1,
3798                     "quality_revision_summary": (
3799                         "Bounded whole-report "
3800                         "quality-revision candidate."
3801                     ),
3802                 },

```

```

3802         }
3803     )
3804 )
3805 else:
3806     revised_writer_draft = (
3807         WriterAgentDraft.model_validate(
3808             revised_draft_or_fallback
3809         )
3810     )
3811     store.save_json(
3812         "10_writer_quality_revision_draft.json",
3813         revised_writer_draft,
3814     )
3815     try:
3816         revision_candidate = materialise_writer_output(
3817             revised_writer_draft,
3818             genre_scoped_fact_ledger,
3819             insight_ledger=writer_pack.insight_ledger,
3820             allow_hypotheses_in_report=(
3821                 self.settings.allow_hypotheses_in_report
3822             ),
3823             content_requirements=writer_content_requirements,
3824             writer_mode="llm_writer",
3825             eligible_for_primary_evaluation=representation_eligible,
3826             quality_revision_round=1,
3827             quality_revision_summary=(
3828                 "Bounded whole-report "
3829                 "quality-revision candidate."
3830             ),
3831         )
3832     except ValueError as error:
3833         revision_materialisation_error = str(error)
3834         store.save_text(
3835             "10_writer_quality_revision_materialisation_error.txt",
3836             str(error),
3837         )
3838         store.trace(
3839             "writer_quality_revision_materialisation",
3840             "rejected",
3841             {
3842                 "reason": f"ValueError: {error}",
3843                 "fallback": "pre_revision_writer_output",
3844             },
3845         )
3846         revision_candidate = raw_writer_output
3847
3848     revision_candidate = revision_candidate.model_copy(
3849         update={
3850             "quality_revision_round": 1,
3851             "quality_revision_summary": (
3852                 "Bounded whole-report "
3853                 "quality-revision candidate."
3854             ),
3855         }
3856     )
3857
3858     store.save_json(
3859         "10_writer_quality_revision_candidate.json",
3860         revision_candidate,
3861     )
3862     store.save_text(
3863         "10_writer_quality_revision_candidate.md",
3864         revision_candidate.markdown,
3865     )
3866
3867     revision_validation_errors = (
3868         validate_writer_output(
3869             revision_candidate,
3870             fact_ledger,
3871             insight_ledger,
3872             self.settings.allow_hypotheses_in_report,
3873         )
3874     )
3875     if revision_materialisation_error is not None:
3876         revision_validation_errors.append(
3877             "Writer quality revision failed materialisation: "
3878             + revision_materialisation_error
3879         )
3880
3881     revised_quality_audit = (
3882         deterministic_audit(
3883             writer_output=revision_candidate,
3884             fact_ledger=fact_ledger,
3885             evidence=evidence_ledger,
3886             mode=audit_mode,
3887             external_sources=(
3888                 external_truth_sources
3889             ),
3890             revision_round=0,
3891             report_specification=(
3892                 plan.report_specification

```



```

3893         ),
3894         settings=self.settings,
3895         profile_support_records=(
3896             profile_support_records
3897         ),
3898         insight_ledger=insight_ledger,
3899     )
3900 )
3901
3902 revision_accepted, revision_reasons = (
3903     accept_writer_quality_revision(
3904         before=raw_writer_output,
3905         after=revision_candidate,
3906         before_audit=(
3907             initial_quality_audit
3908         ),
3909         after_audit=(
3910             revised_quality_audit
3911         ),
3912         validation_errors=(
3913             revision_validation_errors
3914         ),
3915         report_specification=(
3916             plan.report_specification
3917         ),
3918         settings=self.settings,
3919     )
3920 )
3921
3922 store.save_json(
3923     "10_writer_quality_revision_assessment.json",
3924     {
3925         "attempted": True,
3926         "accepted": revision_accepted,
3927         "reasons": revision_reasons,
3928         "before_component_assessments": (
3929             initial_quality_audit
3930             .component_assessments
3931         ),
3932         "after_component_assessments": (
3933             revised_quality_audit
3934             .component_assessments
3935         ),
3936         "before_quality": (
3937             initial_quality_audit
3938             .quality_assessment
3939         ),
3940         "after_quality": (
3941             revised_quality_audit
3942             .quality_assessment
3943         ),
3944         "validation_errors": (
3945             revision_validation_errors
3946         ),
3947     },
3948 )
3949
3950 if revision_accepted:
3951     quality_revised_writer_output = (
3952         revision_candidate.model_copy(
3953             update={
3954                 "quality_revision_summary": (
3955                     "One bounded Writer "
3956                     "quality revision was "
3957                     "accepted before factual "
3958                     "auditing."
3959                 ),
3960             }
3961         )
3962     )
3963
3964     writer_output_for_audit = (
3965         quality_revised_writer_output
3966     )
3967
3968     store.save_json(
3969         "10_writer_quality_revision.json",
3970         writer_output_for_audit,
3971     )
3972     store.save_text(
3973         "10_writer_quality_revision.md",
3974         writer_output_for_audit.markdown,
3975     )
3976     store.save_json(
3977         "10_writer_quality_revision_component_coverage.json",
3978         revised_quality_audit.component_assessments,
3979     )
3980 else:
3981     store.trace(
3982         "writer_quality_revision",
3983         "rejected",

```

```

3984         {
3985             "reasons": (
3986                 revision_reasons
3987             )
3988         },
3989     )
3990
3991     (
3992         initial_audit,
3993         proposal,
3994         writer_output_for_audit,
3995     ) = await self.audit_once(
3996         run_id=run_id,
3997         writer_output=writer_output_for_audit,
3998         fact_ledger=fact_ledger,
3999         evidence_ledger=evidence_ledger,
4000         insight_ledger=insight_ledger,
4001         profile_support_records=profile_support_records,
4002         plan=plan,
4003         audit_mode=audit_mode,
4004         external_truth_sources=external_truth_sources,
4005         revision_round=0,
4006         store=store,
4007         stage_name="initial_audit_and_repair",
4008     )
4009
4010     store.save_json("10_initial_audit.json", initial_audit)
4011     store.save_json(
4012         "10_initial_quality_assessment.json",
4013         initial_audit.quality_assessment,
4014     )
4015     store.save_json("11_repair_candidates_round_0.json", proposal)
4016
4017     current_output = writer_output_for_audit
4018     current_audit = initial_audit
4019     repair_rounds = 0
4020     all_patches = []
4021
4022     while (
4023         current_audit.decision == AuditDecision.REVISE
4024         and repair_rounds < plan.revision_limit
4025     ):
4026         repaired_output, patches = apply_repair_proposal(
4027             current_output,
4028             proposal,
4029             fact_ledger,
4030             evidence_ledger,
4031             insight_ledger,
4032             self.settings.allow_hypotheses_in_report,
4033         )
4034
4035         if not patches:
4036             release_status = decide_release_status(
4037                 annotations=current_audit.annotations,
4038                 quality=current_audit.quality_assessment,
4039                 methodological_warnings=current_audit.methodological_warnings,
4040                 repair_budget_exhausted=True,
4041                 audit_mode=audit_mode,
4042             )
4043             current_audit = current_audit.model_copy(
4044                 update={
4045                     "decision": (
4046                         AuditDecision.BLOCK
4047                         if release_status == ReleaseStatus.HUMAN_REVIEW_REQUIRED
4048                         else AuditDecision.PASS
4049                     ),
4050                     "release_status": release_status,
4051                     "residual_risk": (
4052                         current_audit.residual_risk
4053                         + " No deterministic-valid repair candidate was available."
4054                     ),
4055                 }
4056             )
4057             break
4058
4059         repair_rounds += 1
4060         all_patches.extend(patches)
4061         current_output = repaired_output
4062
4063         store.save_json(
4064             f"12_selected_patches_round_{repair_rounds}.json",
4065             patches,
4066         )
4067         store.save_text(
4068             f"13_repaired_report_round_{repair_rounds}.md",
4069             current_output.markdown,
4070         )
4071         store.save_json(
4072             f"13_repaired_output_round_{repair_rounds}.json",
4073             current_output,
4074         )

```

```

4075         (
4076             current_audit,
4077             proposal,
4078             current_output,
4079         ) = await self.audit_once(
4080             run_id=run_id,
4081             writer_output=current_output,
4082             fact_ledger=fact_ledger,
4083             evidence_ledger=evidence_ledger,
4084             insight_ledger=insight_ledger,
4085             profile_support_records=profile_support_records,
4086             plan=plan,
4087             audit_mode=audit_mode,
4088             external_truth_sources=external_truth_sources,
4089             revision_round=repair_rounds,
4090             store=store,
4091             stage_name=f"post_repair_audit_round_{repair_rounds}",
4092         )
4093
4094     current_audit = current_audit.model_copy(
4095         update={"applied_patches": all_patches}
4096     )
4097
4098     store.save_json(
4099         f"14_post_repair_audit_round_{repair_rounds}.json",
4100         current_audit,
4101     )
4102     store.save_json(
4103         f"14_repair_candidates_round_{repair_rounds}.json",
4104         proposal,
4105     )
4106
4107     repair_budget_exhausted = current_audit.decision == AuditDecision.REVISE
4108
4109     if repair_budget_exhausted:
4110         release_status = decide_release_status(
4111             annotations=current_audit.annotations,
4112             quality=current_audit.quality_assessment,
4113             methodological_warnings=current_audit.methodological_warnings,
4114             repair_budget_exhausted=True,
4115             audit_mode=audit_mode,
4116         )
4117         current_audit = current_audit.model_copy(
4118             update={
4119                 "decision": (
4120                     AuditDecision.BLOCK
4121                     if release_status == ReleaseStatus.HUMAN_REVIEW_REQUIRED
4122                     else AuditDecision.PASS
4123                 ),
4124                 "release_status": release_status,
4125                 "residual_risk": (
4126                     current_audit.residual_risk
4127                     + " The bounded repair budget was exhausted."
4128                 ),
4129                 "applied_patches": all_patches,
4130             }
4131         )
4132
4133     final_audit = current_audit
4134     store.save_json(
4135         "14_final_component_coverage.json",
4136         assess_report_components(
4137             current_output,
4138             fact_ledger,
4139             evidence_ledger,
4140             plan.report_specification.required_components,
4141         ),
4142     )
4143
4144     genre_quality = assess_genre_quality(
4145         current_output,
4146         plan.report_specification,
4147         evidence_ledger,
4148     )
4149     store.save_json(
4150         "14_final_genre_quality.json",
4151         genre_quality,
4152     )
4153
4154     factual_release_status = decide_release_status(
4155         annotations=final_audit.annotations,
4156         quality=final_audit.quality_assessment,
4157         methodological_warnings=final_audit.methodological_warnings,
4158         repair_budget_exhausted=repair_budget_exhausted,
4159         audit_mode=audit_mode,
4160     )
4161
4162     if (
4163         factual_release_status
4164         == ReleaseStatus.HUMAN_REVIEW_REQUIRED
4165     ):

```

```

4166         final_decision = AuditDecision.BLOCK
4167     else:
4168         final_decision = AuditDecision.PASS
4169
4170     final_audit = final_audit.model_copy(
4171         update={
4172             "decision": final_decision,
4173             "release_status": factual_release_status,
4174         }
4175     )
4176
4177     release_status = factual_release_status
4178     if (
4179         not representation_eligible
4180         or genre_quality.status == QualityStatus.REVISE
4181     ):
4182         release_status = ReleaseStatus.HUMAN_REVIEW_REQUIRED
4183
4184     if not representation_eligible:
4185         current_output = current_output.model_copy(
4186             update={"eligible_for_primary_evaluation": False}
4187         )
4188
4189     approved = representation_eligible and genre_quality.status != (
4190         QualityStatus.REVISE
4191     ) and release_status in {
4192         ReleaseStatus.APPROVED,
4193         ReleaseStatus.APPROVED_WITH_WARNINGS,
4194     }
4195
4196     if representation_eligible:
4197         primary_evaluation_reason = None
4198     elif input_structure is None:
4199         primary_evaluation_reason = "input_structure_unavailable"
4200     else:
4201         primary_evaluation_reason = (
4202             "input_representation_"
4203             + input_structure.representation_status.value
4204         )
4205
4206     result = PipelineResult(
4207         run_id=run_id,
4208         inferred_task_contract=inferred_task_contract,
4209         resolved_task_contract=resolved_task_contract,
4210         task_contract_agreement=task_contract_agreement(
4211             inferred_task_contract,
4212             resolved_task_contract,
4213         ),
4214         profile=profile,
4215         input_structure=input_structure,
4216         structural_catalog=structural_catalog,
4217         evaluation_field_policy=data_bundle.evaluation_field_policy,
4218         understanding=understanding,
4219         execution_plan=plan,
4220         evidence_ledger=evidence_ledger,
4221         fact_candidates=fact_candidates,
4222         verification=verification,
4223         fact_ledger=fact_ledger,
4224         writer_evidence_pack=writer_pack,
4225         raw_writer_output=raw_writer_output,
4226         quality_revised_writer_output=quality_revised_writer_output,
4227         final_writer_output=current_output,
4228         initial_audit=initial_audit,
4229         final_audit=final_audit,
4230         repair_rounds_used=repair_rounds,
4231         release_status=release_status,
4232         approved_for_release=approved,
4233         primary_evaluation_eligible=representation_eligible,
4234         primary_evaluation_reason=primary_evaluation_reason,
4235         genre_quality_assessment=genre_quality,
4236         insight_ledger=insight_ledger,
4237     )
4238
4239     store.save_json("final_result.json", result)
4240
4241     if release_status == ReleaseStatus.APPROVED:
4242         header = "<!-- APPROVED BY AUDITOR -->\n\n"
4243     elif release_status == ReleaseStatus.APPROVED_WITH_WARNINGS:
4244         header = "<!-- APPROVED WITH RESIDUAL WARNINGS -->\n\n"
4245     else:
4246         header = "<!-- HUMAN REVIEW REQUIRED -->\n\n"
4247
4248     store.save_text(
4249         "final_report.md",
4250         header + current_output.markdown,
4251     )
4252
4253     store.trace(
4254         "workflow",
4255         "completed",
4256         {

```

```

4257         "release_status": release_status.value,
4258         "repair_rounds": repair_rounds,
4259         "writer_mode": current_output.writer_mode,
4260         "raw_writer_mode": raw_writer_output.writer_mode,
4261         "verified_insight_count": len(
4262             insight_ledger.verified_insights
4263         ),
4264         "insight_fallback_reason": (
4265             insight_ledger.fallback_reason
4266         ),
4267         "input_representation_status": (
4268             input_structure.representation_status.value
4269             if input_structure is not None
4270             else "invalid"
4271         ),
4272         "genre_quality_status": genre_quality.status.value,
4273         "primary_evaluation_eligible": representation_eligible,
4274     },
4275 )
4276
4277     return result
4278
4279 def run_sync(
4280     self,
4281     inputs: List[str | Path],
4282     request: str,
4283     *,
4284     audit_mode: AuditMode = AuditMode.INTERNAL,
4285     external_truth_sources: List[ExternalTruthSource] | None = None,
4286     evaluation_field_policy: EvaluationFieldPolicy | None = None,
4287     report_genre: ReportGenre | None = None,
4288     communication_task: CommunicationTask | None = None,
4289     output_form: OutputForm | None = None,
4290     focus_scope: str | None = None,
4291 ) -> PipelineResult:
4292     return asyncio.run(
4293         self.run(
4294             inputs,
4295             request,
4296             audit_mode=audit_mode,
4297             external_truth_sources=external_truth_sources,
4298             evaluation_field_policy=evaluation_field_policy,
4299             report_genre=report_genre,
4300             communication_task=communication_task,
4301             output_form=output_form,
4302             focus_scope=focus_scope,
4303         )
4304     )

```

2 Test Program Listings

Automated test programs for the main system and the isolated LLM-only experiment.

Listing 47: T01: experiments/llm_only_pipeline/tests/test_workflow.py

```

1 from __future__ import annotations
2
3 from table2text.evaluation.models import BenchmarkExample, OutputMode, TaskFamily
4
5 from table2text_llm_only.schemas import AgentCallTrace, UsageSummary
6 from table2text_llm_only.workflow import LLMOnlyWorkflow
7
8
9 def example() -> BenchmarkExample:
10     return BenchmarkExample(
11         dataset_id="demo_e2e",
12         example_id="demo-1",
13         task_family=TaskFamily.ATTRIBUTE_VERBALISATION,
14         output_mode=OutputMode.SHORT_TEXT,
15         language="en",
16         source_payload={"name": "The Eagle", "food": "French", "area": "riverside"},
17         source_text="name[The Eagle], food[French], area[riverside]",
18         references=["The Eagle is a French riverside restaurant."],
19         request="Express all and only the supplied attributes.",
20         parent_table=[["name", "The Eagle"], ["food", "French"], ["area", "riverside"]],
21         metadata={"normalizer": "e2e"},
22         source_sha256="demo-source",
23         reference_sha256="demo-reference",
24     )
25
26
27 class FakeClient:
28     config = type("Config", (), {"model": "fake-llm", "base_url": "offline"})()
29
30     def json_call(self, *, stage: str, system: str, user: str):
31         del system, user
32         claim = {
33             "claim_id": "C1",
34             "claim": "The Eagle serves French food in the riverside area.",
35             "claim_type": "attribute",
36             "source_refs": ["source_payload.name", "source_payload.food", "source_payload.area"],
37             "copied_values": ["The Eagle", "French", "riverside"],
38             "calculation_or_reasoning": "Direct attribute verbalisation.",
39             "confidence": 0.99,
40         }
41         payloads = {
42             "source_interpreter": {
43                 "task_understanding": "Verbalise the supplied restaurant attributes.",
44                 "source_units": "One meaning representation.",
45                 "important_entities": ["The Eagle"],
46                 "important_fields": ["name", "food", "area"],
47                 "allowed_claim_types": ["attribute verbalisation"],
48                 "risks": ["Do not add absent attributes."],
49             },
50             "llm_analysis": {"claims": [claim], "analysis_notes": []},
51             "claim_critic": {
52                 "reviews": [
53                     {
54                         "claim_id": "C1",
55                         "status": "supported",
56                         "reason": "All values occur in the source.",
57                         "corrected_claim": None,
58                     }
59                 ],
60                 "global_warnings": [],
61             },
62             "claim_adjudicator": {
63                 "accepted_claims": [claim],
64                 "rejected_claims": [],
65                 "warnings": [],
66             },
67             "writer": {
68                 "title": None,
69                 "sentences": [{"text": claim["claim"], "claim_ids": ["C1"]}],
70             },
71             "output_auditor": {
72                 "decision": "pass",
73                 "support_rate": 1.0,
74                 "findings": [],
75                 "notes": [],
76             },
77         }
78         trace = AgentCallTrace(
79             stage=stage,
80             model="fake-llm",
81             elapsed_seconds=0.0,
82             usage=UsageSummary(),
83         )

```

```

84         return payloads[stage], trace
85
86
87 def test_source_packet_holds_out_references():
88     workflow = LLMOnlyWorkflow(client=FakeClient())
89
90     packet = workflow._source_packet(example())
91
92     assert "references" not in packet
93     assert "reference_sha256" not in packet
94     assert packet["source_payload"]["name"] == "The Eagle"
95
96
97 def test_offline_workflow_preserves_supported_claim_path():
98     result = LLMOnlyWorkflow(client=FakeClient()).run(example())
99
100    assert result.generated_text == "The Eagle serves French food in the riverside area."
101    assert result.final_audit.decision == "pass"
102    assert [trace.stage for trace in result.traces] == [
103        "source_interpreter",
104        "llm_analysis",
105        "claim_critic",
106        "claim_adjudicator",
107        "writer",
108        "output_auditor",
109    ]
110
111
112 def test_evidence_gate_rejects_uncited_accepted_claim():
113     workflow = LLMOnlyWorkflow(client=FakeClient())
114     result = workflow.run(example())
115     claim = result.adjudicated_claims.accepted_claims[0].model_copy(
116         update={"source_refs": []}
117     )
118     adjudicated = result.adjudicated_claims.model_copy(
119         update={"accepted_claims": [claim]}
120     )
121
122     gated = workflow.enforce_accepted_claim_evidence(adjudicated)
123
124     assert gated.accepted_claims == []
125     assert gated.rejected_claims[0].claim_id == "C1"

```

Listing 48: T02: table2text_pydanticai/tests/test_reference_evaluation.py

```

1  from __future__ import annotations
2
3  import json
4  from pathlib import Path
5
6  import pandas as pd
7  import pytest
8
9  from table2text import Settings, Table2TextWorkflow
10 from table2text.agents import materialise_insight_ledger
11 from table2text.data import load_data
12 from table2text.evaluation.datasets import merge_examples, normalise_row
13 from table2text.evaluation.diagnostics import number_diagnostics
14 from table2text.evaluation.generation import (
15     focus_scope_for_task,
16     materialise_input,
17     workflow_contract_for_variant,
18 )
19 from table2text.evaluation.deepeval_metrics import input_for_judge
20 from table2text.evaluation.human_evaluation import make_blinded_pairs
21 from table2text.evaluation.llm_judge_annotations import (
22     OpenAIIJudgeAnnotationConfig,
23     build_annotation_judge_input,
24 )
25 from table2text.evaluation.models import (
26     BenchmarkExample,
27     DatasetConfig,
28     DatasetSource,
29     DeepEvalConfig,
30     GenerationBackend,
31     GenerationRecord,
32     OutputMode,
33     ReferenceMetricConfig,
34     TaskFamily,
35     VariantConfig,
36 )
37 from table2text.evaluation_backends import build_single_agent_prompt
38 from table2text.schemas import (
39     AuditMode,
40     ClaimPermission,
41     CommunicationTask,
42     EvidenceCapability,
43     InsightCandidate,
44     InsightCandidateSet,

```

```

45     InsightContribution,
46     InsightType,
47     InsightVerificationRecord,
48     InsightVerificationResult,
49     InsightVerificationStatus,
50     InputRepresentationStatus,
51     InputShape,
52     InputStructureProfile,
53     InterpretationLevel,
54     OutputForm as WorkflowOutputForm,
55     ReportGenre,
56     ReportSelectionSource,
57 )
58 from table2text.evaluation.notebook import (
59     generate_reports_for_notebook,
60     init_notebook_evaluation,
61 )
62 from table2text.evaluation import reference_metrics as reference_metrics_module
63 from table2text.evaluation.external_factuality import ExternalFactualityResult
64 from table2text.evaluation.reference_metrics import (
65     evaluate_reference_metrics,
66     factuality_context,
67     normalized_event_source_context,
68     plain_text,
69 )
70 from table2text.workflow import task_contract_fields
71 from table2text.task_contracts import infer_task_contract, resolve_task_contract
72 from table2text.evaluation.statistics import paired_bootstrap
73
74
75 def e2e_config() -> DatasetConfig:
76     return DatasetConfig(
77         dataset_id="e2e",
78         source=DatasetSource.HUGGINGFACE,
79         hub_id="fixture",
80         normalizer="e2e",
81         task_family=TaskFamily.ATTRIBUTE_VERBALISATION,
82         output_mode=OutputMode.SHORT_TEXT,
83         language="en",
84         sample_size=None,
85         reference_fields=["references", "target"],
86         id_fields=["gem_parent_id", "gem_id"],
87     )
88
89
90 def webnlg_config() -> DatasetConfig:
91     return DatasetConfig(
92         dataset_id="web_nlg",
93         source=DatasetSource.HUGGINGFACE,
94         hub_id="fixture",
95         normalizer="webnlg",
96         task_family=TaskFamily.TRIPLE_VERBALISATION,
97         output_mode=OutputMode.SHORT_TEXT,
98         language="en",
99         sample_size=None,
100         reference_fields=["target"],
101         id_fields=["gem_id"],
102     )
103
104
105 def dart_config() -> DatasetConfig:
106     return DatasetConfig(
107         dataset_id="dart",
108         source=DatasetSource.HUGGINGFACE,
109         hub_id="fixture",
110         normalizer="dart",
111         task_family=TaskFamily.TRIPLE_VERBALISATION,
112         output_mode=OutputMode.SHORT_TEXT,
113         language="en",
114         sample_size=None,
115         reference_fields=["target"],
116         id_fields=["gem_id"],
117     )
118
119
120 def test_e2e_normalisation_excludes_reference():
121     row = {
122         "gem_id": "e2e-test-1",
123         "gem_parent_id": "e2e-parent-1",
124         "meaning_representation": "name[The Eagle], area[riverside]",
125         "target": "The Eagle is in the riverside area.",
126         "references": ["The Eagle is by the river."],
127     }
128
129     example = normalise_row(row, e2e_config(), 0)
130
131     assert example.source_payload == {
132         "meaning_representation": "name[The Eagle], area[riverside]"
133     }
134     assert "The Eagle is by the river." not in example.source_text
135     assert len(example.references) == 2

```



```

136     assert example.parent_table == [{"name", "The Eagle"}, {"area", "riverside"}]
137
138
139 def test_single_agent_baseline_prompt_excludes_references():
140     example = BenchmarkExample(
141         dataset_id="sportsett_basketball",
142         example_id="fixture-1",
143         task_family=TaskFamily.EVENT_REPORT,
144         output_mode=OutputMode.MULTI_PARAGRAPH_REPORT,
145         language="en",
146         source_payload={"winner": "Home", "score": "10-8"},
147         source_text='{"winner": "Home", "score": "10-8"}',
148         references=["Home won by two points."],
149         request="Write a short event report.",
150         source_sha256="source",
151         reference_sha256="reference",
152     )
153
154     messages = build_single_agent_prompt(example)
155     prompt_text = "\n".join(message["content"] for message in messages)
156
157     assert '{"winner": "Home", "score": "10-8"}' in prompt_text
158     assert "Write a short event report." in prompt_text
159     assert "Home won by two points." not in prompt_text
160     assert "Dataset ID" not in prompt_text
161     assert "Example ID" not in prompt_text
162     assert "sportsett_basketball" not in prompt_text
163     assert "fixture-1" not in prompt_text
164     assert "Task type: event report" in prompt_text
165     assert "Expected form: multi paragraph report" in prompt_text
166
167
168 def test_single_agent_generic_prompt_excludes_task_metadata():
169     example = BenchmarkExample(
170         dataset_id="sportsett_basketball",
171         example_id="fixture-1",
172         task_family=TaskFamily.EVENT_REPORT,
173         output_mode=OutputMode.MULTI_PARAGRAPH_REPORT,
174         language="en",
175         source_payload={"winner": "Home", "score": "10-8"},
176         source_text='{"winner": "Home", "score": "10-8"}',
177         references=["Home won by two points."],
178         request="Understand the supplied data and report its strongest supported findings.",
179         source_sha256="source",
180         reference_sha256="reference",
181     )
182
183     messages = build_single_agent_prompt(example, prompt_style="generic")
184     prompt_text = "\n".join(message["content"] for message in messages)
185
186     assert '{"winner": "Home", "score": "10-8"}' in prompt_text
187     assert "Understand the supplied data" in prompt_text
188     assert "Home won by two points." not in prompt_text
189     assert "Task type:" not in prompt_text
190     assert "Expected form:" not in prompt_text
191     assert "Language:" not in prompt_text
192     assert "sportsett_basketball" not in prompt_text
193     assert "fixture-1" not in prompt_text
194
195
196 def test_event_benchmark_uses_reference_recap_focus_scope():
197     assert focus_scope_for_task(TaskFamily.EVENT_REPORT) == "reference_recap"
198     assert (
199         focus_scope_for_task(TaskFamily.CROSS_LINGUAL_EVENT_REPORT)
200         == "reference_recap"
201     )
202
203
204 def test_reference_recap_contract_removes_visible_report_scaffolding():
205     contract = task_contract_fields(
206         genre=ReportGenre.EVENT_REPORT,
207         communication_task=CommunicationTask.EVENT_REPORT,
208         output_form=WorkflowOutputForm.MULTI_PARAGRAPH_REPORT,
209         focus_scope="reference_recap",
210     )
211
212     assert contract["allow_headings"] is False
213     assert contract["preferred_sections"] == []
214     assert "scope_limitations" not in contract["required_content_slots"]
215     assert "markdown_headings" in contract["prohibited_claim_types"]
216
217
218 def test_event_source_context_normalizes_structured_source_without_reference():
219     source = {
220         "game": {
221             "stadium": "Wells Fargo Center",
222             "dayname": "Sunday",
223         },
224         "teams": {
225             "home": {
226                 "name": "76ers",

```

```

227         "place": "Philadelphia",
228         "line_score": {
229             "Q1": {"PTS": "26"},
230             "game": {"PTS": "103", "TREB": "44"},
231         },
232         "box_score": [
233             {"name": "J.J. Redick", "PTS": "24", "FGM": "9"},
234         ],
235     },
236     "vis": {
237         "name": "Grizzlies",
238         "place": "Memphis",
239         "line_score": {
240             "Q1": {"PTS": "25"},
241             "game": {"PTS": "95", "TREB": "35"},
242         },
243         "box_score": [
244             {"name": "Mike Conley", "PTS": "21", "AST": "5"},
245         ],
246     },
247 },
248 }
249 record = GenerationRecord(
250     generation_id="g1",
251     dataset_id="event_fixture",
252     example_id="1",
253     variant_id="full_system",
254     repetition=0,
255     seed=42,
256     task_family=TaskFamily.EVENT_REPORT,
257     output_mode=OutputMode.MULTI_PARAGRAPH_REPORT,
258     language="en",
259     source_text=json.dumps(source),
260     references=["A held-out human reference that must not be copied."],
261     parent_table=None,
262     request="Write an event report.",
263     generated_text="Philadelphia defeated Memphis 103-95.",
264     backend=GenerationBackend.TABLE2TEXT,
265 )
266
267 context = normalized_event_source_context(record)
268
269 assert context is not None
270 assert "Event result: Philadelphia scored 103 and Memphis scored 95" in context
271 assert "Wells Fargo Center" in context
272 assert "Record for J.J. Redick" in context
273 assert "held-out human reference" not in context
274
275 config = ReferenceMetricConfig(
276     enabled_metrics=["hmem"],
277     external_factuality_context="source_text",
278 )
279 assert factuality_context(record, config) == context
280
281
282 def test_attribute_verbalisation_materialises_structured_record(
283     tmp_path: Path,
284 ):
285     row = {
286         "gem_id": "e2e-test-1",
287         "meaning_representation": {
288             "name[Clowns], eatType[pub], customer rating[5 out of 5], "
289             "near[Crowne Plaza Hotel]"
290         },
291         "target": {
292             "The pub Clowns is near Crowne Plaza Hotel and has a customer "
293             "rating of 5 out of 5."
294         },
295     }
296     example = normalise_row(row, e2e_config(), 0)
297     input_path = materialise_input(example, tmp_path / "inputs")
298     bundle = load_data([input_path])
299     table = next(iter(bundle.tables.values()))
300
301     assert {"attribute_name", "attribute_value"}.issubset(table.columns)
302     assert table["attribute_name"].tolist() == [
303         "name",
304         "eatType",
305         "customer rating",
306         "near",
307     ]
308     assert table["attribute_value"].tolist() == [
309         "Clowns",
310         "pub",
311         "5 out of 5",
312         "Crowne Plaza Hotel",
313     ]
314
315
316 def test_generic_contract_inference_uses_source_structure_not_benchmark_labels():
317     input_structure = InputStructureProfile(

```

```

318     shape=InputShape.NESTED_RECORD,
319     representation_status=InputRepresentationStatus.VALID,
320     row_semantics="one record",
321     confidence=0.95,
322 )
323 decision = infer_task_contract(
324     request=(
325         "Understand the supplied data and report its strongest supported "
326         "findings."
327     ),
328     structured_inputs={
329         "input": {
330             "__table2text_benchmark_example__": True,
331             "dataset_id": "misleading_event_name",
332             "task_family": "event_report",
333             "output_mode": "multi_paragraph_report",
334             "source_payload": {
335                 "meaning_representation": "name[Clowns], eatType[pub]",
336             },
337         },
338     },
339     input_structure=input_structure,
340 )
341
342 assert decision.communication_task == CommunicationTask.ATTRIBUTE_VERBALISATION
343 assert decision.output_form == WorkflowOutputForm.SHORT_TEXT
344 assert decision.report_genre == ReportGenre.DATASET_OVERVIEW
345 assert decision.selection_source == ReportSelectionSource.STRUCTURED_INFERENCE
346 assert decision.confidence == pytest.approx(0.97)
347
348
349 def test_inferred_evaluation_variant_uses_source_only_and_omits_contract(
350     tmp_path: Path,
351 ):
352     example = normalise_row(
353         {
354             "gem_id": "e2e-test-1",
355             "meaning_representation": "name[Clowns], eatType[pub]",
356             "target": "Clowns is a pub.",
357         },
358         e2e_config(),
359         0,
360     )
361     variant = VariantConfig(
362         variant_id="full_inferred_contract",
363         task_contract_mode="inferred",
364         request_override=(
365             "Understand the supplied data and report its strongest supported "
366             "findings."
367         ),
368     )
369     input_path = materialise_input(
370         example,
371         tmp_path,
372         source_only=True,
373     )
374     payload = json.loads(input_path.read_text(encoding="utf-8"))
375
376     assert payload == example.source_payload
377     assert "task_family" not in payload
378     assert all(
379         value is None
380         for value in workflow_contract_for_variant(example, variant).values()
381     )
382
383
384 def test_configured_contract_fields_override_inferred_values():
385     input_structure = InputStructureProfile(
386         shape=InputShape.NESTED_RECORD,
387         representation_status=InputRepresentationStatus.VALID,
388         row_semantics="one record",
389         confidence=0.95,
390     )
391     inferred = infer_task_contract(
392         request="Understand the supplied data.",
393         structured_inputs={
394             "input": {
395                 "table": [{"Name", "Value"}, ["A", "1"]],
396                 "highlighted_cells": [[1, 1]],
397             },
398         },
399         input_structure=input_structure,
400     )
401     resolved = resolve_task_contract(
402         inferred=inferred,
403         selected_genre=ReportGenre.DATASET_OVERVIEW,
404         genre_source=ReportSelectionSource.EXPERIMENT_CONFIGURATION,
405         genre_confidence=1.0,
406         configured_communication_task=CommunicationTask.TRIPLE_VERBALISATION,
407         configured_output_form=WorkflowOutputForm.SHORT_TEXT,
408         configured_focus_scope=None,

```

```

409 )
410
411 assert resolved.communication_task == CommunicationTask.TRIPLE_VERBALISATION
412 assert resolved.output_form == WorkflowOutputForm.SHORT_TEXT
413 assert resolved.focus_scope is None
414 assert resolved.confidence == 1.0
415
416
417 @pytest.mark.parametrize(
418     ("payload", "expected_task", "expected_form", "expected_text"),
419     [
420         (
421             {
422                 "meaning_representation": (
423                     "name[Clowns], eatType[pub], customer rating[5 out of 5], "
424                     "near[Crowne Plaza Hotel]"
425                 )
426             },
427             CommunicationTask.ATTRIBUTE_VERBALISATION,
428             WorkflowOutputForm.SHORT_TEXT,
429             "Clowns",
430         ),
431         (
432             {
433                 "triples": [
434                     ["ALCO_RS-3", "engine", "Four-stroke_engine"],
435                     ["ALCO_RS-3", "cylinderCount", "12"],
436                 ]
437             },
438             CommunicationTask.TRIPLE_VERBALISATION,
439             WorkflowOutputForm.SHORT_TEXT,
440             "ALCO_RS-3",
441         ),
442         (
443             {
444                 "table_page_title": "Election",
445                 "table_section_title": "Results",
446                 "table": [
447                     [
448                         {"value": "Candidate", "is_header": True},
449                         {"value": "Vote share", "is_header": True},
450                     ],
451                     [
452                         {"value": "Ma Ying-jeou", "is_header": False},
453                         {"value": "58.45%", "is_header": False},
454                     ],
455                 ],
456                 "highlighted_cells": [[1, 1]],
457             },
458             CommunicationTask.FOCUSED_TABLE_DESCRIPTION,
459             WorkflowOutputForm.ONE_SENTENCE,
460             "58.45%",
461         ),
462     ],
463 )
464 def test_generic_workflow_infers_short_form_contract_from_source_only(
465     tmp_path: Path,
466     payload: dict,
467     expected_task: CommunicationTask,
468     expected_form: WorkflowOutputForm,
469     expected_text: str,
470 ):
471     input_path = tmp_path / f"{expected_task.value}.json"
472     input_path.write_text(json.dumps(payload), encoding="utf-8")
473     workflow = Table2TextWorkflow(
474         Settings(use_llm=False, output_dir=tmp_path / "runs")
475     )
476
477     result = workflow.run_sync(
478         inputs=[input_path],
479         request=(
480             "Understand the supplied data and report its strongest supported "
481             "findings."
482         ),
483         audit_mode=AuditMode.INTERNAL,
484     )
485
486     specification = result.execution_plan.report_specification
487     assert specification.communication_task == expected_task
488     assert specification.output_form == expected_form
489     assert specification.selection_source == ReportSelectionSource.STRUCTURED_INFERENCE
490     assert expected_text in plain_text(result.final_writer_output.markdown)
491
492
493 def test_attribute_verbalisation_workflow_avoids_dataset_profile(
494     tmp_path: Path,
495 ):
496     row = {
497         "gem_id": "e2e-test-1",
498         "meaning_representation": (
499             "name[Clowns], eatType[pub], customer rating[5 out of 5], "

```

```

500         "near[Crowne Plaza Hotel]"
501     ),
502     "target": (
503         "The pub Clowns is near Crowne Plaza Hotel and has a customer "
504         "rating of 5 out of 5."
505     ),
506 )
507 example = normalise_row(row, e2e_config(), 0)
508 input_path = materialise_input(example, tmp_path / "inputs")
509
510 workflow = Table2TextWorkflow(
511     Settings(
512         use_llm=False,
513         output_dir=tmp_path / "runs",
514     )
515 )
516 result = workflow.run_sync(
517     inputs=[input_path],
518     request=example.request,
519     audit_mode=AuditMode.INTERNAL,
520     report_genre=ReportGenre.DATASET_OVERVIEW,
521     communication_task=CommunicationTask.ATTRIBUTE_VERBALISATION,
522     output_form=WorkflowOutputForm.SHORT_TEXT,
523 )
524 report = plain_text(result.final_writer_output.markdown)
525
526 assert "Clowns" in report
527 assert "pub" in report
528 assert "Crowne Plaza Hotel" in report
529 assert "5 out of 5" in report
530 assert "Dataset overview" not in report
531 assert "1 rows" not in report
532 assert "meaning_representation" not in report
533 assert result.final_writer_output.writer_mode == (
534     "deterministic_short_form_writer"
535 )
536 assert result.primary_evaluation_eligible
537 assert (
538     EvidenceCapability.STRUCTURED_RECORD_VERBALISATION
539     in result.execution_plan.selected_capabilities
540 )
541
542
543 def test_webnlg_serialized_triples_use_structured_verbalisation(
544     tmp_path: Path,
545 ):
546     example = normalise_row(
547         {
548             "gem_id": "webnlg-test-1",
549             "input": [
550                 ["ALCO_RS-3", "engine", "Four-stroke engine"],
551                 ["ALCO_RS-3", "cylinderCount", "12"],
552                 ["ALCO_RS-3", "length", "17068.8 (millimetres)"],
553             ],
554             "target": (
555                 "The ALCO RS-3 has a four-stroke engine, 12 cylinders, "
556                 "and a length of 17068.8 millimetres."
557             ),
558         },
559         webnlg_config(),
560         0,
561     )
562     input_path = materialise_input(example, tmp_path / "inputs")
563
564     workflow = Table2TextWorkflow(
565         Settings(use_llm=False, output_dir=tmp_path / "runs")
566     )
567     result = workflow.run_sync(
568         inputs=[input_path],
569         request=example.request,
570         audit_mode=AuditMode.INTERNAL,
571         report_genre=ReportGenre.DATASET_OVERVIEW,
572         communication_task=CommunicationTask.TRIPLE_VERBALISATION,
573         output_form=WorkflowOutputForm.SHORT_TEXT,
574     )
575     report = plain_text(result.final_writer_output.markdown)
576
577     assert "ALCO RS-3" in report
578     assert "12 cylinders" in report
579     assert "17068.8" in report
580     assert "millimetres" in report
581     assert "seventeen" not in report.casefold()
582     assert result.final_writer_output.writer_mode == (
583         "deterministic_short_form_writer"
584     )
585     assert result.primary_evaluation_eligible
586     assert (
587         EvidenceCapability.STRUCTURED_RECORD_VERBALISATION
588         in result.execution_plan.selected_capabilities
589     )
590

```

```

591
592 def test_dart_rank_triple_uses_ordinal_phrasing(tmp_path: Path):
593     example = normalise_row(
594         {
595             "gem_id": "dart-test-1",
596             "tripleset": [
597                 ["Place A", "RANK", "11"],
598                 ["Place A", "TOTAL", "211.5"],
599             ],
600             "target": "Place A ranks 11th and has a total of 211.5.",
601         },
602         dart_config(),
603         0,
604     )
605     input_path = materialise_input(example, tmp_path / "inputs")
606
607     workflow = Table2TextWorkflow(
608         Settings(use_llm=False, output_dir=tmp_path / "runs")
609     )
610     result = workflow.run_sync(
611         inputs=[input_path],
612         request=example.request,
613         audit_mode=AuditMode.INTERNAL,
614         report_genre=ReportGenre.DATASET_OVERVIEW,
615         communication_task=CommunicationTask.TRIPLE_VERBALISATION,
616         output_form=WorkflowOutputForm.SHORT_TEXT,
617     )
618     report = plain_text(result.final_writer_output.markdown)
619
620     assert "ranks 11th" in report
621     assert "total of 211.5" in report
622     assert "eleventh" not in report.casefold()
623     assert result.final_writer_output.writer_mode == (
624         "deterministic_short_form_writer"
625     )
626     assert result.primary_evaluation_eligible
627
628 def test_multiple_reference_rows_are_merged():
629     config = e2e_config()
630     first = normalise_row(
631         {
632             "gem_parent_id": "shared",
633             "meaning_representation": "name[A]",
634             "target": "A is a venue.",
635             "references": [],
636         },
637         config,
638         0,
639     )
640
641     second = normalise_row(
642         {
643             "gem_parent_id": "shared",
644             "meaning_representation": "name[A]",
645             "target": "The venue is named A.",
646             "references": [],
647         },
648         config,
649         1,
650     )
651
652     merged = merge_examples([first, second])
653
654     assert len(merged) == 1
655     assert merged[0].references == ["A is a venue.", "The venue is named A."]
656
657
658 def test_highlighted_table_examples_materialise_as_cell_table(tmp_path: Path):
659     config = DatasetConfig(
660         dataset_id="totto",
661         source=DatasetSource.HUGGINGFACE,
662         hub_id="fixture",
663         normalizer="totto",
664         task_family=TaskFamily.HIGHLIGHTED_TABLE_DESCRIPTION,
665         output_mode=OutputMode.ONE_SENTENCE,
666         reference_fields=["target"],
667     )
668     example = normalise_row(
669         {
670             "table_page_title": "Election",
671             "table_section_title": "Results",
672             "table": [
673                 [
674                     {"value": "Candidate", "is_header": True},
675                     {"value": "Vote share", "is_header": True},
676                 ],
677                 [
678                     {"value": "Ma Ying-jeou", "is_header": False},
679                     {"value": "58.45%", "is_header": False},
680                 ],
681             ],
682         },

```

```

682         "highlighted_cells": [[1, 1]],
683         "target": "Ma Ying-jeou received 58.45% of the vote.",
684     },
685     config,
686     0,
687 )
688
689 input_path = materialise_input(example, tmp_path)
690 bundle = load_data([input_path])
691 frame = next(iter(bundle.tables.values()))
692
693 assert frame.shape[0] == 4
694 assert "cell_value" in frame.columns
695 assert "is_highlighted" in frame.columns
696 highlighted = frame[frame["is_highlighted"]]
697 assert highlighted["cell_value"].tolist() == ["58.45%"]
698 assert highlighted["request"].iloc[0] == example.request
699
700
701 def test_highlighted_table_task_uses_focused_one_sentence_contract(tmp_path: Path):
702     config = DatasetConfig(
703         dataset_id="totto",
704         source=DatasetSource.HUGGINGFACE,
705         hub_id="fixture",
706         normalizer="totto",
707         task_family=TaskFamily.HIGHLIGHTED_TABLE_DESCRIPTION,
708         output_mode=OutputMode.ONE_SENTENCE,
709         reference_fields=["target"],
710     )
711     example = normalise_row(
712         {
713             "table_page_title": "Election",
714             "table_section_title": "Results",
715             "table": [
716                 [
717                     {"value": "Candidate", "is_header": True},
718                     {"value": "Vote share", "is_header": True},
719                 ],
720                 [
721                     {"value": "Ma Ying-jeou", "is_header": False},
722                     {"value": "58.45%", "is_header": False},
723                 ],
724             ],
725             "highlighted_cells": [[1, 1]],
726             "target": "Ma Ying-jeou received 58.45% of the vote.",
727         },
728         config,
729         0,
730     )
731     input_path = materialise_input(example, tmp_path / "inputs")
732
733     workflow = Table2TextWorkflow(
734         Settings(
735             use_llm=False,
736             output_dir=tmp_path / "runs",
737         )
738     )
739     result = workflow.run_sync(
740         inputs=[input_path],
741         request=example.request,
742         audit_mode=AuditMode.INTERNAL,
743         report_genre=ReportGenre.DATASET_OVERVIEW,
744         communication_task=CommunicationTask.FOCUSED_TABLE_DESCRIPTION,
745         output_form=WorkflowOutputForm.ONE_SENTENCE,
746         focus_scope="highlighted_cells",
747     )
748
749     report = result.final_writer_output.markdown.strip()
750
751     assert "#" not in report
752     assert "58.45%" in report
753     assert "Ma Ying-jeou" in report
754     assert "rows" not in report.lower()
755     assert "columns" not in report.lower()
756     assert "correlation" not in report.lower()
757     assert result.execution_plan.selected_capabilities == [
758         EvidenceCapability.FOCUSED_TABLE_REGION
759     ]
760     assert len(result.execution_plan.insight_objectives) == 1
761     assert (
762         result.execution_plan.insight_objectives[0].objective_id
763         == "INSIGHT_FOCUSED_TABLE_RELATION"
764     )
765     assert (
766         "table-local proposition"
767         in result.execution_plan.insight_objectives[0].question
768     )
769     assert any(
770         item.capability == EvidenceCapability.FOCUSED_TABLE_REGION
771         for item in result.evidence_ledger.items
772     )

```

```

773
774
775 def test_highlighted_table_logical_row_context_uses_spans(tmp_path: Path):
776     config = DatasetConfig(
777         dataset_id="totto",
778         source=DatasetSource.HUGGINGFACE,
779         hub_id="fixture",
780         normalizer="totto",
781         task_family=TaskFamily.HIGHLIGHTED_TABLE_DESCRIPTION,
782         output_mode=OutputMode.ONE_SENTENCE,
783         reference_fields=["target"],
784     )
785     example = normalise_row(
786         {
787             "table_page_title": "Ma Ying-jeou",
788             "table_section_title": "Inauguration",
789             "table": [
790                 [
791                     {
792                         "value": "Party",
793                         "is_header": True,
794                         "row_span": 2,
795                         "column_span": 2,
796                     },
797                     {
798                         "value": "Candidate",
799                         "is_header": True,
800                         "row_span": 1,
801                         "column_span": 2,
802                     },
803                     {
804                         "value": "Votes",
805                         "is_header": True,
806                         "row_span": 2,
807                         "column_span": 1,
808                     },
809                     {
810                         "value": "Percentage",
811                         "is_header": True,
812                         "row_span": 2,
813                         "column_span": 2,
814                     },
815                 ],
816                 [
817                     {
818                         "value": "President",
819                         "is_header": False,
820                         "row_span": 1,
821                         "column_span": 1,
822                     },
823                     {
824                         "value": "Vice president",
825                         "is_header": False,
826                         "row_span": 1,
827                         "column_span": 1,
828                     },
829                 ],
830                 [
831                     {"value": "", "is_header": False},
832                     {"value": "-", "is_header": False},
833                     {"value": "-", "is_header": False},
834                     {"value": "Vincent Siew", "is_header": False},
835                     {"value": "7,659,014", "is_header": False},
836                     {"value": "58.45%", "is_header": False},
837                     {"value": "", "is_header": False},
838                 ],
839                 [
840                     {"value": "", "is_header": False},
841                     {"value": "-", "is_header": False},
842                     {"value": "Other candidate", "is_header": False},
843                     {"value": "Other running mate", "is_header": False},
844                     {"value": "5,444,949", "is_header": False},
845                     {"value": "41.55%", "is_header": False},
846                     {"value": "", "is_header": False},
847                 ],
848                 [
849                     {
850                         "value": "Total",
851                         "is_header": False,
852                         "column_span": 4,
853                     },
854                     {"value": "13,103,963", "is_header": False},
855                     {
856                         "value": "100.00%",
857                         "is_header": False,
858                         "column_span": 2,
859                     },
860                 ],
861             ],
862             "highlighted_cells": [[2, 5]],
863             "target": "Ma won the presidency by 58.45% of the vote.",

```



```

864     },
865     config,
866     0,
867 )
868 input_path = materialise_input(example, tmp_path / "inputs")
869 frame = next(iter(load_data([input_path]).tables.values()))
870
871 president_header = frame[
872     (frame["row_index"] == 1)
873     & (frame["cell_value"] == "President")
874 ].iloc[0]
875 vice_header = frame[
876     (frame["row_index"] == 1)
877     & (frame["cell_value"] == "Vice president")
878 ].iloc[0]
879 assert int(president_header["column_index"]) == 2
880 assert int(vice_header["column_index"]) == 3
881
882 workflow = Table2TextWorkflow(
883     Settings(
884         use_llm=False,
885         output_dir=tmp_path / "runs",
886     )
887 )
888 result = workflow.run_sync(
889     inputs=[input_path],
890     request=example.request,
891     audit_mode=AuditMode.INTERNAL,
892     report_genre=ReportGenre.DATASET_OVERVIEW,
893     communication_task=CommunicationTask.FOCUSED_TABLE_DESCRIPTION,
894     output_form=WorkflowOutputForm.ONE_SENTENCE,
895     focus_scope="highlighted_cells",
896 )
897 evidence = next(
898     item
899     for item in result.evidence_ledger.items
900     if item.capability == EvidenceCapability.FOCUSED_TABLE_REGION
901 )
902 metrics = evidence.metrics
903
904 def has_pair(
905     pairs: list[dict[str, object]],
906     headers: list[str],
907     value: str,
908 ) -> bool:
909     return any(
910         pair.get("headers") == headers
911         and pair.get("value") == value
912         for pair in pairs
913     )
914
915 assert has_pair(
916     metrics["highlighted_role_value_pairs"],
917     ["Percentage"],
918     "58.45%",
919 )
920 assert result.final_writer_output.writer_mode == (
921     "deterministic_short_form_writer"
922 )
923 assert result.primary_evaluation_eligible
924 assert has_pair(
925     metrics["logical_row_context"],
926     ["Candidate", "Vice president"],
927     "Vincent Siew",
928 )
929 assert has_pair(
930     metrics["logical_row_context"],
931     ["Votes"],
932     "7,659,014",
933 )
934 assert has_pair(
935     metrics["logical_row_placeholders"],
936     ["Candidate", "President"],
937     "-",
938 )
939 assert {
940     "value": "Ma Ying-jeou",
941     "related_headers": ["Candidate", "President"],
942     "basis": (
943         "page title with placeholder value under this logical header path "
944         "in the same row"
945     ),
946 } in metrics["page_title_subject_candidates"]
947 comparison = metrics["highlighted_measure_comparisons"][0]
948 assert comparison["highlighted_value"] == "58.45%"
949 assert comparison["headers"] == ["Percentage"]
950 assert comparison["rank_descending"] == 1
951 assert comparison["comparable_value_count"] == 2
952 assert comparison["is_highest_comparable_value"] is True
953 assert comparison["is_majority_percentage"] is True
954 assert comparison["excluded_aggregate_values"][0]["value"] == "100.00%"

```

```

955
956
957 def test_rectangular_table_does_not_use_previous_data_rows_as_headers(
958     tmp_path: Path,
959 ):
960     config = DatasetConfig(
961         dataset_id="totto",
962         source=DatasetSource.HUGGINGFACE,
963         hub_id="fixture",
964         normalizer="totto",
965         task_family=TaskFamily.HIGHLIGHTED_TABLE_DESCRIPTION,
966         output_mode=OutputMode.ONE_SENTENCE,
967         reference_fields=["target"],
968     )
969     example = normalise_row(
970         {
971             "table_page_title": "List of offices",
972             "table_section_title": "Office holders",
973             "table": [
974                 [
975                     {"value": "#", "is_header": True},
976                     {"value": "Name", "is_header": True},
977                     {"value": "Start", "is_header": True},
978                     {"value": "End", "is_header": True},
979                 ],
980                 [
981                     {"value": "1", "is_header": True},
982                     {"value": "Alice Example", "is_header": False},
983                     {"value": "1 January 2001", "is_header": False},
984                     {"value": "2 January 2002", "is_header": False},
985                 ],
986                 [
987                     {"value": "2", "is_header": True},
988                     {"value": "Bob Example", "is_header": False},
989                     {"value": "3 January 2003", "is_header": False},
990                     {"value": "4 January 2004", "is_header": False},
991                 ],
992             ],
993             "highlighted_cells": [[2, 1], [2, 2], [2, 3]],
994             "target": "Bob Example held the office from 2003 to 2004.",
995         },
996         config,
997         0,
998     )
999     input_path = materialise_input(example, tmp_path / "inputs")
1000
1001     workflow = Table2TextWorkflow(
1002         Settings(
1003             use_llm=False,
1004             output_dir=tmp_path / "runs",
1005         )
1006     )
1007     result = workflow.run_sync(
1008         inputs=[input_path],
1009         request=example.request,
1010         audit_mode=AuditMode.INTERNAL,
1011         report_genre=ReportGenre.DATASET_OVERVIEW,
1012         communication_task=CommunicationTask.FOCUSED_TABLE_DESCRIPTION,
1013         output_form=WorkflowOutputForm.ONE_SENTENCE,
1014         focus_scope="highlighted_cells",
1015     )
1016     evidence = next(
1017         item
1018         for item in result.evidence_ledger.items
1019         if item.capability == EvidenceCapability.FOCUSED_TABLE_REGION
1020     )
1021     highlighted_pairs = evidence.metrics["highlighted_role_value_pairs"]
1022
1023     assert evidence.metrics["page_title_subject_candidates"] == []
1024     assert evidence.metrics["concise_output_focus"][
1025         "primary_subject_candidates"
1026     ] == []
1027     assert result.fact_ledger.writer_ready_facts
1028     assert {
1029         "headers": ["Name"],
1030         "value": "Bob Example",
1031         "column_index": 1,
1032     } in highlighted_pairs
1028     assert {
1034         "headers": ["Start"],
1035         "value": "3 January 2003",
1036         "column_index": 2,
1037     } in highlighted_pairs
1038     assert {
1039         "headers": ["End"],
1040         "value": "4 January 2004",
1041         "column_index": 3,
1042     } in highlighted_pairs
1043     assert all(
1044         "Alice Example" not in pair["headers"]
1045         for pair in highlighted_pairs

```

```

1046     )
1047
1048
1049 def test_highlighted_table_numeric_header_supports_fact_summary(
1050     tmp_path: Path,
1051 ):
1052     config = DatasetConfig(
1053         dataset_id="totto",
1054         source=DatasetSource.HUGGINGFACE,
1055         hub_id="fixture",
1056         normalizer="totto",
1057         task_family=TaskFamily.HIGHLIGHTED_TABLE_DESCRIPTION,
1058         output_mode=OutputMode.ONE_SENTENCE,
1059         reference_fields=["target"],
1060     )
1061     example = normalise_row(
1062         {
1063             "table_page_title": "Tax table",
1064             "table_section_title": "Rates",
1065             "table": [
1066                 [
1067                     {"value": "Country", "is_header": True},
1068                     {
1069                         "value": "Corporate income tax rate (2016)",
1070                         "is_header": True,
1071                     },
1072                     {
1073                         "value": "Combined corporate tax rate (2016)",
1074                         "is_header": True,
1075                     },
1076                 ],
1077                 [
1078                     {"value": "France", "is_header": False},
1079                     {"value": "34.43%", "is_header": False},
1080                     {"value": "34.43%", "is_header": False},
1081                 ],
1082                 [
1083                     {"value": "Switzerland", "is_header": False},
1084                     {"value": "8.50%", "is_header": False},
1085                     {"value": "21.15%", "is_header": False},
1086                 ],
1087             ],
1088             "highlighted_cells": [[1, 0], [1, 1], [2, 0], [2, 1]],
1089             "target": (
1090                 "France had a 34.43% corporate income tax rate, while "
1091                 "Switzerland had an 8.50% rate."
1092             ),
1093         },
1094         config,
1095         0,
1096     )
1097     input_path = materialise_input(example, tmp_path / "inputs")
1098
1099     workflow = Table2TextWorkflow(
1100         Settings(
1101             use_llm=False,
1102             output_dir=tmp_path / "runs",
1103         )
1104     )
1105     result = workflow.run_sync(
1106         inputs=[input_path],
1107         request=example.request,
1108         audit_mode=AuditMode.INTERNAL,
1109         report_genre=ReportGenre.DATASET_OVERVIEW,
1110         communication_task=CommunicationTask.FOCUSED_TABLE_DESCRIPTION,
1111         output_form=WorkflowOutputForm.ONE_SENTENCE,
1112         focus_scope="highlighted_cells",
1113     )
1114
1115     assert result.fact_ledger.writer_ready_facts
1116     fact = result.fact_ledger.writer_ready_facts[0]
1117     assert "Corporate income tax rate (2016)" in fact.fact_summary
1118     assert "34.43%" in fact.fact_summary
1119     assert "8.50%" in fact.fact_summary
1120     assert "# Evidence-grounded data-science report" not in (
1121         result.final_writer_output.markdown.strip()
1122     )
1123
1124
1125 def test_irregular_highlighted_table_context_ignores_placeholders(tmp_path: Path):
1126     config = DatasetConfig(
1127         dataset_id="totto",
1128         source=DatasetSource.HUGGINGFACE,
1129         hub_id="fixture",
1130         normalizer="totto",
1131         task_family=TaskFamily.HIGHLIGHTED_TABLE_DESCRIPTION,
1132         output_mode=OutputMode.ONE_SENTENCE,
1133         reference_fields=["target"],
1134     )
1135     example = normalise_row(
1136         {

```

```

1137         "table_page_title": "Election page",
1138         "table_section_title": "Results",
1139         "table": [
1140             [
1141                 {"value": "Party", "is_header": True},
1142                 {"value": "Candidate", "is_header": True},
1143                 {"value": "Votes", "is_header": True},
1144                 {"value": "Percentage", "is_header": True},
1145             ],
1146             [
1147                 {"value": "President", "is_header": True},
1148                 {"value": "Vice president", "is_header": True},
1149             ],
1150             [
1151                 {"value": "", "is_header": False},
1152                 {"value": "-", "is_header": False},
1153                 {"value": "-", "is_header": False},
1154                 {"value": "Named candidate", "is_header": False},
1155                 {"value": "7,659,014", "is_header": False},
1156                 {"value": "58.45%", "is_header": False},
1157                 {"value": "", "is_header": False},
1158             ],
1159         ],
1160         "highlighted_cells": [[2, 5]],
1161         "target": "The highlighted value is 58.45%.",
1162     },
1163     config,
1164     0,
1165 )
1166 input_path = materialise_input(example, tmp_path / "inputs")
1167
1168 workflow = Table2TextWorkflow(
1169     Settings(
1170         use_llm=False,
1171         output_dir=tmp_path / "runs",
1172     )
1173 )
1174 result = workflow.run_sync(
1175     inputs=[input_path],
1176     request=example.request,
1177     audit_mode=AuditMode.INTERNAL,
1178     report_genre=ReportGenre.DATASET_OVERVIEW,
1179     communication_task=CommunicationTask.FOCUSED_TABLE_DESCRIPTION,
1180     output_form=WorkflowOutputForm.ONE_SENTENCE,
1181     focus_scope="highlighted_cells",
1182 )
1183
1184 evidence = next(
1185     item
1186     for item in result.evidence_ledger.items
1187     if item.capability == EvidenceCapability.FOCUSED_TABLE_REGION
1188 )
1189 report = result.final_writer_output.markdown.strip()
1190
1191 assert evidence.metrics["row_context"] == [
1192     "Named candidate",
1193     "7,659,014",
1194 ]
1195 assert evidence.metrics["header_context"][0] == "Percentage"
1196 assert "58.45%" in report
1197 assert "Named candidate" in report
1198 assert "Percentage" in report
1199 assert " for Party" not in report
1200 assert not report.startswith("-")
1201
1202
1203 def test_focused_table_relation_insight_can_be_verified_without_implication(
1204     tmp_path: Path,
1205 ):
1206     config = DatasetConfig(
1207         dataset_id="totto",
1208         source=DatasetSource.HUGGINGFACE,
1209         hub_id="fixture",
1210         normalizer="totto",
1211         task_family=TaskFamily.HIGHLIGHTED_TABLE_DESCRIPTION,
1212         output_mode=OutputMode.ONE_SENTENCE,
1213         reference_fields=["target"],
1214     )
1215     example = normalise_row(
1216         {
1217             "table_page_title": "Election page",
1218             "table_section_title": "Results",
1219             "table": [
1220                 [
1221                     {"value": "Candidate", "is_header": True},
1222                     {"value": "Vote share", "is_header": True},
1223                 ],
1224                 [
1225                     {"value": "Candidate A", "is_header": False},
1226                     {"value": "58.45%", "is_header": False},
1227                 ],
1228             ],
1229         }

```

```

1228         ],
1229         "highlighted_cells": [[1, 1]],
1230         "target": "Candidate A had 58.45% of the vote.",
1231     },
1232     config,
1233     0,
1234 )
1235 input_path = materialise_input(example, tmp_path / "inputs")
1236 workflow = Table2TextWorkflow(
1237     Settings(
1238         use_llm=False,
1239         output_dir=tmp_path / "runs",
1240     )
1241 )
1242 result = workflow.run_sync(
1243     inputs=[input_path],
1244     request=example.request,
1245     audit_mode=AuditMode.INTERNAL,
1246     report_genre=ReportGenre.DATASET_OVERVIEW,
1247     communication_task=CommunicationTask.FOCUSED_TABLE_DESCRIPTION,
1248     output_form=WorkflowOutputForm.ONE_SENTENCE,
1249     focus_scope="highlighted_cells",
1250 )
1251 fact_ids = [
1252     fact.fact_id
1253     for fact in result.fact_ledger.writer_ready_facts
1254     if EvidenceCapability.FOCUSED_TABLE_REGION in fact.source_capabilities
1255 ]
1256 evidence_ids = [
1257     item.evidence_id
1258     for item in result.evidence_ledger.items
1259     if item.capability == EvidenceCapability.FOCUSED_TABLE_REGION
1260 ]
1261
1262 ledger = materialise_insight_ledger(
1263     candidates=InsightCandidateSet(
1264         candidates=[
1265             InsightCandidate(
1266                 insight_id="INSIGHT_FOCUSED_001",
1267                 statement=(
1268                     "Candidate A had 58.45% of the vote in the "
1269                     "Election page table."
1270                 ),
1271                 insight_type=InsightType.NARRATIVE_SUMMARY,
1272                 interpretation_level=InterpretationLevel.FINDING,
1273                 contribution=InsightContribution.DESCRPTIVE_SYNTHESIS,
1274                 source_fact_ids=fact_ids,
1275                 source_evidence_ids=evidence_ids,
1276                 supporting_summary=(
1277                     "The highlighted focused-table evidence supplies the "
1278                     "value, row context, header and table context."
1279                 ),
1280                 claim_permissions=[ClaimPermission.DESCRPTIVE],
1281                 confidence=0.95,
1282                 salience=0.95,
1283             )
1284         ],
1285     ),
1286     verification=InsightVerificationResult(
1287         records=[
1288             InsightVerificationRecord(
1289                 insight_id="INSIGHT_FOCUSED_001",
1290                 status=InsightVerificationStatus.VERIFIED,
1291                 confidence=0.95,
1292                 salience=0.95,
1293                 adds_bounded_synthesis=False,
1294                 analytical_implication_supported=False,
1295                 contains_hypothesis=False,
1296             )
1297         ],
1298     ),
1299     fact_ledger=result.fact_ledger,
1300     evidence_ledger=result.evidence_ledger,
1301     settings=Settings(),
1302     report_genre=ReportGenre.DATASET_OVERVIEW,
1303 )
1304
1305 assert [insight.insight_id for insight in ledger.verified_insights] == [
1306     "INSIGHT_FOCUSED_001"
1307 ]
1308 assert (
1309     ledger.verified_insights[0].interpretation_level
1310     == InterpretationLevel.BOUNDED_INSIGHT
1311 )
1312
1313
1314 def test_focused_table_highlighted_set_contrast_is_scoped_locally(
1315     tmp_path: Path,
1316 ):
1317     config = DatasetConfig(
1318         dataset_id="totto",

```

```

1319     source=DatasetSource.HUGGINGFACE,
1320     hub_id="fixture",
1321     normalizer="totto",
1322     task_family=TaskFamily.HIGHLIGHTED_TABLE_DESCRIPTION,
1323     output_mode=OutputMode.ONE_SENTENCE,
1324     reference_fields=["target"],
1325 )
1326 example = normalise_row(
1327     {
1328         "table_page_title": "Corporate tax",
1329         "table_section_title": "International corporate tax rates",
1330         "table": [
1331             [
1332                 {"value": "Country", "is_header": True},
1333                 {
1334                     "value": "Corporate income tax rate (2016)",
1335                     "is_header": True,
1336                 },
1337             ],
1338             [
1339                 {"value": "France", "is_header": False},
1340                 {"value": "34.43%", "is_header": False},
1341             ],
1342             [
1343                 {"value": "Switzerland", "is_header": False},
1344                 {"value": "8.50%", "is_header": False},
1345             ],
1346             [
1347                 {"value": "United States", "is_header": False},
1348                 {"value": "35.00%", "is_header": False},
1349             ],
1350         ],
1351         "highlighted_cells": [[1, 0], [1, 1], [2, 0], [2, 1]],
1352         "target": (
1353             "Among the highlighted countries, Switzerland had the lower "
1354             "corporate tax rate and France had the higher rate."
1355         ),
1356     },
1357     config,
1358     0,
1359 )
1360 input_path = materialise_input(example, tmp_path / "inputs")
1361 workflow = Table2TextWorkflow(
1362     Settings(
1363         use_llm=False,
1364         output_dir=tmp_path / "runs",
1365     )
1366 )
1367 result = workflow.run_sync(
1368     inputs=[input_path],
1369     request=example.request,
1370     audit_mode=AuditMode.INTERNAL,
1371     report_genre=ReportGenre.DATASET_OVERVIEW,
1372     communication_task=CommunicationTask.FOCUSED_TABLE_DESCRIPTION,
1373     output_form=WorkflowOutputForm.ONE_SENTENCE,
1374     focus_scope="highlighted_cells",
1375 )
1376 evidence = next(
1377     item
1378     for item in result.evidence_ledger.items
1379     if item.capability == EvidenceCapability.FOCUSED_TABLE_REGION
1380 )
1381 contrast = evidence.metrics["highlighted_set_contrasts"][0]
1382
1383 assert contrast["scope"] == "highlighted_values_only"
1384 assert contrast["measure_headers"] == [
1385     "Corporate income tax rate (2016)"
1386 ]
1387 assert contrast["lower"]["subject"] == "Switzerland"
1388 assert contrast["lower"]["value"] == "8.50%"
1389 assert contrast["higher"]["subject"] == "France"
1390 assert contrast["higher"]["value"] == "34.43%"
1391 assert "Among the highlighted countries" in contrast["contrast_summary"]
1392
1393 report = plain_text(result.final_writer_output.markdown)
1394 assert "Switzerland" in report
1395 assert "lower" in report
1396 assert "France" in report
1397 assert "higher" in report
1398 assert "United States" not in report
1399
1400
1401 def test_focused_table_uses_raw_highlight_coordinates_with_colspan(
1402     tmp_path: Path,
1403 ):
1404     config = DatasetConfig(
1405         dataset_id="totto",
1406         source=DatasetSource.HUGGINGFACE,
1407         hub_id="fixture",
1408         normalizer="totto",
1409         task_family=TaskFamily.HIGHLIGHTED_TABLE_DESCRIPTION,

```

```

1410         output_mode=OutputMode.ONE_SENTENCE,
1411         reference_fields=["target"],
1412     )
1413     example = normalise_row(
1414     {
1415         "table_page_title": "Ernest Burton (American football)",
1416         "table_section_title": "Head coaching record",
1417         "table": [
1418             [
1419                 {"value": "Year", "is_header": True},
1420                 {"value": "Team", "is_header": True},
1421                 {"value": "Overall", "is_header": True},
1422                 {"value": "Conference", "is_header": True},
1423                 {"value": "Standing", "is_header": True},
1424                 {"value": "Bowl/playoffs", "is_header": True},
1425             ],
1426             [
1427                 {
1428                     "value": (
1429                         "Maine Black Bears "
1430                         "(Maine Intercollegiate Athletic Association) "
1431                         "(1900)"
1432                     ),
1433                     "is_header": False,
1434                     "column_span": 9,
1435                 },
1436             ],
1437             [
1438                 {"value": "1900", "is_header": False},
1439                 {"value": "Maine", "is_header": False},
1440                 {"value": "–44", "is_header": False},
1441                 {"value": "", "is_header": False},
1442                 {"value": "", "is_header": False},
1443                 {"value": "", "is_header": False},
1444             ],
1445             [
1446                 {
1447                     "value": "Maine:",
1448                     "is_header": False,
1449                     "column_span": 2,
1450                 },
1451                 {"value": "–44", "is_header": False},
1452                 {"value": "", "is_header": False},
1453                 {"value": "", "is_header": False, "column_span": 5},
1454             ],
1455             [
1456                 {
1457                     "value": "Total:",
1458                     "is_header": False,
1459                     "column_span": 2,
1460                 },
1461                 {"value": "–44", "is_header": False},
1462                 {"value": "", "is_header": False, "column_span": 7},
1463             ],
1464         ],
1465         "highlighted_cells": [[1, 0], [4, 1]],
1466         "target": (
1467             "C. Ernest Burton was the head coach of Maine's football "
1468             "team in 1900 and compiled a –44 record."
1469         ),
1470     },
1471     config,
1472     0,
1473 )
1474 input_path = materialise_input(example, tmp_path / "inputs")
1475 frame = next(iter(load_data([input_path]).tables.values()))
1476 highlighted_values = frame[
1477     frame["is_highlighted"].fillna(False).astype(bool)
1478 ][["cell_value"]].tolist()
1479
1480 assert highlighted_values == [
1481     "Maine Black Bears (Maine Intercollegiate Athletic Association) (1900)",
1482     "–44",
1483 ]
1484 assert "Total:" not in highlighted_values
1485
1486 workflow = Table2TextWorkflow(
1487     Settings(
1488         use_llm=False,
1489         output_dir=tmp_path / "runs",
1490     )
1491 )
1492 result = workflow.run_sync(
1493     inputs=[input_path],
1494     request=example.request,
1495     audit_mode=AuditMode.INTERNAL,
1496     report_genre=ReportGenre.DATASET_OVERVIEW,
1497     communication_task=CommunicationTask.FOCUSED_TABLE_DESCRIPTION,
1498     output_form=WorkflowOutputForm.ONE_SENTENCE,
1499     focus_scope="highlighted_cells",
1500 )

```

```

1501     evidence = next(
1502         item
1503         for item in result.evidence_ledger.items
1504         if item.capability == EvidenceCapability.FOCUSED_TABLE_REGION
1505     )
1506     relation = evidence.metrics["focused_record_relation"]
1507     report = plain_text(result.final_writer_output.markdown)
1508
1509     assert relation["compact_group_label"] == "Maine Black Bears"
1510     assert relation["year"] == "1900"
1511     assert relation["value"] == "-44"
1512     assert "Ernest Burton" in report
1513     assert "Maine Black Bears" in report
1514     assert "1900" in report
1515     assert "-44" in report
1516     assert "Total:" not in report
1517
1518
1519 def test_focused_table_list_page_uses_structural_header_not_previous_rows(
1520     tmp_path: Path,
1521 ):
1522     config = DatasetConfig(
1523         dataset_id="totto",
1524         source=DatasetSource.HUGGINGFACE,
1525         hub_id="fixture",
1526         normalizer="totto",
1527         task_family=TaskFamily.HIGHLIGHTED_TABLE_DESCRIPTION,
1528         output_mode=OutputMode.ONE_SENTENCE,
1529         reference_fields=["target"],
1530     )
1531     example = normalise_row(
1532         {
1533             "table_page_title": (
1534                 "List of NWA/WCW closed-circuit events and "
1535                 "pay-per-view events"
1536             ),
1537             "table_section_title": "1993",
1538             "table": [
1539                 [
1540                     {"value": "Date", "is_header": True},
1541                     {"value": "Event", "is_header": True},
1542                     {"value": "Venue", "is_header": True},
1543                     {"value": "Location", "is_header": True},
1544                     {"value": "Main event", "is_header": True},
1545                 ],
1546                 [
1547                     {"value": "February 21", "is_header": False},
1548                     {"value": "SuperBrawl III", "is_header": False},
1549                     {"value": "Asheville Civic Center", "is_header": False},
1550                     {
1551                         "value": "Asheville, North Carolina",
1552                         "is_header": False,
1553                     },
1554                     {"value": "Big Van Vader vs. Sting", "is_header": False},
1555                 ],
1556                 [
1557                     {"value": "December 27", "is_header": False},
1558                     {"value": "Starrcade", "is_header": False},
1559                     {"value": "Independence Arena", "is_header": False},
1560                     {
1561                         "value": "Charlotte, North Carolina",
1562                         "is_header": False,
1563                     },
1564                     {
1565                         "value": "Big Van Vader vs. Ric Flair",
1566                         "is_header": False,
1567                     },
1568                 ],
1569             ],
1570             "highlighted_cells": [[2, 1]],
1571             "target": (
1572                 "In 1993, Starrcade was a pay-per-view event by World "
1573                 "Championship Wrestling."
1574             ),
1575         },
1576         config,
1577         0,
1578     )
1579     input_path = materialise_input(example, tmp_path / "inputs")
1580
1581     workflow = Table2TextWorkflow(
1582         Settings(
1583             use_llm=False,
1584             output_dir=tmp_path / "runs",
1585         )
1586     )
1587     result = workflow.run_sync(
1588         inputs=[input_path],
1589         request=example.request,
1590         audit_mode=AuditMode.INTERNAL,
1591         report_genre=ReportGenre.DATASET_OVERVIEW,

```



```

1592         communication_task=CommunicationTask.FOCUSED_TABLE_DESCRIPTION,
1593         output_form=WorkflowOutputForm.ONE_SENTENCE,
1594         focus_scope="highlighted_cells",
1595     )
1596     evidence = next(
1597         item
1598         for item in result.evidence_ledger.items
1599         if item.capability == EvidenceCapability.FOCUSED_TABLE_REGION
1600     )
1601     highlighted_pairs = evidence.metrics["highlighted_role_value_pairs"]
1602     relation = evidence.metrics["focused_list_relation"]
1603     report = plain_text(result.final_writer_output.markdown)
1604
1605     assert highlighted_pairs == [
1606         {
1607             "headers": ["Event"],
1608             "value": "Starrcade",
1609             "column_index": 1,
1610         }
1611     ]
1612     assert relation["value"] == "Starrcade"
1613     assert relation["section_title"] == "1993"
1614     assert relation["member_category"] == "pay-per-view event"
1615     assert "Starrcade" in report
1616     assert "1993" in report
1617     assert "pay-per-view event" in report
1618     assert "SuperBrawl" not in report
1619     assert not report.endswith("vs.")
1620
1621
1622 def test_focused_table_preserves_grouped_highlighted_records_with_rowspans(
1623     tmp_path: Path,
1624 ):
1625     config = DatasetConfig(
1626         dataset_id="totto",
1627         source=DatasetSource.HUGGINGFACE,
1628         hub_id="fixture",
1629         normalizer="totto",
1630         task_family=TaskFamily.HIGHLIGHTED_TABLE_DESCRIPTION,
1631         output_mode=OutputMode.ONE_SENTENCE,
1632         reference_fields=["target"],
1633     )
1634     example = normalise_row(
1635         {
1636             "table_page_title": "Ryan Hart",
1637             "table_section_title": "Some Achievements",
1638             "table": [
1639                 [
1640                     {"value": "Tournament", "is_header": True},
1641                     {"value": "Game", "is_header": True},
1642                     {"value": "Place", "is_header": True},
1643                     {"value": "Note", "is_header": True},
1644                 ],
1645                 [
1646                     {
1647                         "value": "Hypespotting",
1648                         "is_header": False,
1649                         "row_span": 2,
1650                     },
1651                     {
1652                         "value": "Super Street Fighter IV: Arcade Edition",
1653                         "is_header": False,
1654                     },
1655                     {"value": "1st", "is_header": False},
1656                     {"value": "", "is_header": False},
1657                 ],
1658                 [
1659                     {
1660                         "value": "Street Fighter III: 3rd Strike",
1661                         "is_header": False,
1662                     },
1663                     {"value": "2nd", "is_header": False},
1664                     {"value": "", "is_header": False},
1665                 ],
1666                 [
1667                     {
1668                         "value": "Stunfest 2012",
1669                         "is_header": False,
1670                         "row_span": 2,
1671                     },
1672                     {
1673                         "value": "Super Street Fighter IV: Arcade Edition",
1674                         "is_header": False,
1675                     },
1676                     {"value": "4th", "is_header": False},
1677                     {"value": "", "is_header": False},
1678                 ],
1679                 [
1680                     {
1681                         "value": "Street Fighter X Tekken",
1682                         "is_header": False,

```

```

1683         },
1684         {"value": "1st", "is_header": False},
1685         {"value": "", "is_header": False},
1686     ],
1687     [
1688         {
1689             "value": "Cross Up Gamerbase #3",
1690             "is_header": False,
1691         },
1692         {
1693             "value": "Street Fighter X Tekken",
1694             "is_header": False,
1695         },
1696         {"value": "1st", "is_header": False},
1697         {"value": "", "is_header": False},
1698     ],
1699     [
1700         {
1701             "value": "Cross Up Gamerbase #2",
1702             "is_header": False,
1703         },
1704         {
1705             "value": "Street Fighter X Tekken",
1706             "is_header": False,
1707         },
1708         {"value": "1st", "is_header": False},
1709         {"value": "", "is_header": False},
1710     ],
1711     [
1712         {
1713             "value": "Cross Up Gamerbase",
1714             "is_header": False,
1715         },
1716         {
1717             "value": "Street Fighter X Tekken",
1718             "is_header": False,
1719         },
1720         {"value": "1st", "is_header": False},
1721         {"value": "", "is_header": False},
1722     ],
1723     "highlighted_cells": [
1724         [1, 0],
1725         [2, 0],
1726         [2, 1],
1727         [3, 0],
1728         [4, 0],
1729         [4, 1],
1730         [5, 0],
1731         [5, 2],
1732         [6, 0],
1733         [6, 2],
1734         [7, 0],
1735         [7, 2],
1736     ],
1737     ),
1738     "target": (
1739         "Hart placed 2nd in Street Fighter III: 3rd Strike at "
1740         "Hypespotting, placing 1st in Street Fighter X Tekken at "
1741         "Stunfest 2012, and won three times in a row in Cross Up "
1742         "tournament."
1743     ),
1744 },
1745 config,
1746 0,
1747 )
1748 input_path = materialise_input(example, tmp_path / "inputs")
1749
1750 workflow = Table2TextWorkflow(
1751     Settings(
1752         use_llm=False,
1753         output_dir=tmp_path / "runs",
1754     )
1755 )
1756 result = workflow.run_sync(
1757     inputs=[input_path],
1758     request=example.request,
1759     audit_mode=AuditMode.INTERNAL,
1760     report_genre=ReportGenre.DATASET_OVERVIEW,
1761     communication_task=CommunicationTask.FOCUSED_TABLE_DESCRIPTION,
1762     output_form=WorkflowOutputForm.ONE_SENTENCE,
1763     focus_scope="highlighted_cells",
1764 )
1765 evidence = next(
1766     item
1767     for item in result.evidence_ledger.items
1768     if item.capability == EvidenceCapability.FOCUSED_TABLE_REGION
1769 )
1770 summary = evidence.metrics["highlighted_record_group_summary"]
1771 records = evidence.metrics["highlighted_record_groups"]
1772 report = plain_text(result.final_writer_output.markdown)
1773

```

```

1774 assert len(records) == 5
1775 assert len(evidence.metrics["highlighted_role_value_pairs"]) == 9
1776 assert "Hypespotting" in summary
1777 assert "Stunfest 2012" in summary
1778 assert "Cross Up Gamerbase" in summary
1779 assert "three times in a row in Street Fighter X Tekken" in summary
1780 assert "Hypespotting" in report
1781 assert "Stunfest 2012" in report
1782 assert "Cross Up Gamerbase" in report
1783
1784
1785 def generation(*, variant: str, text: str) -> GenerationRecord:
1786     return GenerationRecord(
1787         generation_id=f"dataset__1__{variant}__r0__s42",
1788         dataset_id="dataset",
1789         example_id="1",
1790         variant_id=variant,
1791         repetition=0,
1792         seed=42,
1793         task_family=TaskFamily.ATTRIBUTE_VERBALISATION,
1794         output_mode=OutputMode.SHORT_TEXT,
1795         language="en",
1796         source_text="name[A], area[riverside]",
1797         references=["A is located by the riverside."],
1798         parent_table=[["name", "A"], ["area", "riverside"]],
1799         request="Write one sentence.",
1800         generated_text=text,
1801         backend=GenerationBackend.PRECOMPUTED,
1802         primary_evaluation_eligible=True,
1803     )
1804
1805
1806 def test_plain_text_removes_markdown_heading():
1807     assert plain_text("# Report\n\n**A result.**") == "Report A result."
1808
1809
1810 def test_deepeval_input_includes_focused_table_task_guidance():
1811     record = generation(
1812         variant="full",
1813         text="Ma Ying-jeou received 58.45% of the vote.",
1814     ).model_copy(
1815         update={
1816             "dataset_id": "totto",
1817             "task_family": TaskFamily.HIGHLIGHTED_TABLE_DESCRIPTION,
1818             "output_mode": OutputMode.ONE_SENTENCE,
1819             "source_text": (
1820                 "page_title: Ma Ying-jeou\n"
1821                 "section_title: Inauguration\n"
1822                 "row: Ma Ying-jeou | Vincent Siew | 7,659,014 | 58.45%"
1823             ),
1824             "request": "Describe the highlighted table region.",
1825         }
1826     )
1827
1828     judge_input = input_for_judge(record, DeepEvalConfig())
1829
1830     assert "focused-table task" in judge_input
1831     assert "table-local proposition" in judge_input
1832     assert "row co-entity" in judge_input
1833     assert "Ma Ying-jeou" in judge_input
1834
1835
1836 def test_openai_annotation_judge_input_uses_source_output_and_taxonomy_only():
1837     record = generation(
1838         variant="full",
1839         text="Ma Ying-jeou received 58.45% of the vote.",
1840     ).model_copy(
1841         update={
1842             "dataset_id": "totto",
1843             "task_family": TaskFamily.HIGHLIGHTED_TABLE_DESCRIPTION,
1844             "output_mode": OutputMode.ONE_SENTENCE,
1845             "source_text": (
1846                 "page_title: Ma Ying-jeou\n"
1847                 "section_title: Inauguration\n"
1848                 "row: Ma Ying-jeou | Vincent Siew | 7,659,014 | 58.45%"
1849             ),
1850             "request": "Describe the highlighted table region.",
1851             "references": ["Ma won the presidency by 58.45% of the vote."],
1852         }
1853     )
1854
1855     judge_input = build_annotation_judge_input(
1856         record,
1857         OpenAIJudgeAnnotationConfig(),
1858     )
1859
1860     assert "SOURCE DATA" in judge_input
1861     assert "GENERATED OUTPUT" in judge_input
1862     assert "ERROR TAXONOMY" in judge_input
1863     assert "NAME" in judge_input
1864     assert "NUMBER" in judge_input

```

```

1865     assert "NOT CHECKABLE" in judge_input
1866     assert "HUMAN REFERENCES" not in judge_input
1867     assert "Ma won the presidency" not in judge_input
1868
1869
1870 def test_reference_metrics_score_identical_text(tmp_path: Path):
1871     record = generation(variant="full", text="A is located by the riverside.")
1872
1873     observations = evaluate_reference_metrics(
1874         [record],
1875         ReferenceMetricConfig(enabled_metrics=["bleu", "rougeL", "chrF"]),
1876         tmp_path / "metrics.jsonl",
1877     )
1878     scored = {
1879         item.metric_name: item.score
1880         for item in observations
1881         if item.status.value == "scored"
1882     }
1883
1884     assert scored["rougeL"] == 1.0
1885     assert scored["chrF"] > 0.99
1886     assert scored["bleu"] > 0.99
1887
1888
1889 def test_reference_metrics_can_include_ineligible_generations_for_smoke_tests(
1890     tmp_path: Path,
1891 ):
1892     record = generation(
1893         variant="full",
1894         text="A is located by the riverside.",
1895     ).model_copy(
1896         update={
1897             "primary_evaluation_eligible": False,
1898             "primary_evaluation_reason": "deterministic fallback",
1899         }
1900     )
1901
1902     default_observations = evaluate_reference_metrics(
1903         [record],
1904         ReferenceMetricConfig(enabled_metrics=["rougeL"]),
1905         tmp_path / "default_metrics.jsonl",
1906     )
1907     assert default_observations[0].status.value == "skipped"
1908
1909     smoke_observations = evaluate_reference_metrics(
1910         [record],
1911         ReferenceMetricConfig(enabled_metrics=["rougeL"]),
1912         tmp_path / "smoke_metrics.jsonl",
1913         include_ineligible=True,
1914     )
1915     scored = [
1916         item
1917         for item in smoke_observations
1918         if item.metric_name == "rougeL"
1919         and item.status.value == "scored"
1920     ]
1921
1922     assert scored
1923     assert scored[0].score == 1.0
1924
1925
1926 def test_hhem_metric_records_sentence_level_support(
1927     tmp_path: Path,
1928     monkeypatch: pytest.MonkeyPatch,
1929 ):
1930     record = generation(
1931         variant="full",
1932         text="A is located by the riverside. It has outdoor seating.",
1933     )
1934
1935     class FakeHHEMEvaluator:
1936         def __init__(self, **_: object) -> None:
1937             pass
1938
1939         def evaluate(self, *, context: str, generated_text: str) -> ExternalFactualityResult:
1940             assert "A is located by the riverside" in context
1941             assert "outdoor seating" in generated_text
1942             return ExternalFactualityResult(
1943                 metric_name="hhem_2_1_open",
1944                 status="scored",
1945                 overall_score=0.6,
1946                 sentence_scores=[0.9, 0.3],
1947                 minimum_sentence_score=0.3,
1948                 unsupported_sentence_rate=0.5,
1949                 threshold=0.5,
1950                 details={"sentence_count": 2},
1951             )
1952
1953     monkeypatch.setattr(
1954         reference_metrics_module,
1955         "HHEMEvaluator",

```

```

1956         FakeHHEvaluator,
1957     )
1958
1959     observations = evaluate_reference_metrics(
1960         [record],
1961         ReferenceMetricConfig(enabled_metrics=["hhem"]),
1962         tmp_path / "hhem_metrics.jsonl",
1963     )
1964     scored = {
1965         item.metric_name: item
1966         for item in observations
1967         if item.status.value == "scored"
1968     }
1969
1970     assert scored["hhem_2_1_open_mean_support"].score == 0.6
1971     assert scored["hhem_2_1_open_min_sentence_support"].score == 0.3
1972     assert scored["hhem_2_1_open_unsupported_sentence_rate"].score == 0.5
1973     assert scored["hhem_2_1_open_unsupported_sentence_rate"].higher_is_better is False
1974     assert scored["hhem_2_1_open_mean_support"].details["sentence_scores"] == [0.9, 0.3]
1975
1976
1977 def test_alignscore_metric_is_unavailable_without_worker_python(
1978     tmp_path: Path,
1979     monkeypatch: pytest.MonkeyPatch,
1980 ):
1981     record = generation(variant="full", text="A is located by the riverside.")
1982     monkeypatch.setattr(
1983         reference_metrics_module,
1984         "default_alignscore_python_executable",
1985         lambda: None,
1986     )
1987
1988     observations = evaluate_reference_metrics(
1989         [record],
1990         ReferenceMetricConfig(enabled_metrics=["alignscore"]),
1991         tmp_path / "alignscore_metrics.jsonl",
1992     )
1993
1994     assert len(observations) == 1
1995     assert observations[0].metric_name == "alignscore_base"
1996     assert observations[0].status.value == "unavailable"
1997     assert "alignscore_python_executable" in str(observations[0].error)
1998
1999
2000 def test_numeric_diagnostics():
2001     result = number_diagnostics(
2002         "The result was 111-95.",
2003         "Home scored 111 and visitors scored 95.",
2004         ["The home team won 111 to 95."],
2005     )
2006
2007     assert result["generated_number_source_precision"] == 1.0
2008
2009
2010 def test_human_pair_order_is_deterministic():
2011     first = generation(variant="baseline", text="A is riverside.")
2012     second = generation(variant="full", text="A is located by the riverside.")
2013
2014     pairs_one = make_blinded_pairs([first, second], seed=42)
2015     pairs_two = make_blinded_pairs([first, second], seed=42)
2016
2017     assert pairs_one[0].model_dump() == pairs_two[0].model_dump()
2018
2019
2020 def test_paired_bootstrap_positive_difference():
2021     result = paired_bootstrap(
2022         baseline=pd.Series([0.1, 0.2, 0.3]).to_numpy(),
2023         candidate=pd.Series([0.3, 0.4, 0.5]).to_numpy(),
2024         resamples=1000,
2025         confidence_level=0.95,
2026         seed=42,
2027     )
2028
2029     assert result["mean_difference"] > 0
2030     assert result["probability_candidate_better"] > 0.95
2031
2032
2033 @pytest.mark.asyncio
2034 async def test_notebook_helpers_generate_from_precomputed_variant(tmp_path: Path):
2035     paths = init_notebook_evaluation(tmp_path)
2036     example = normalise_row(
2037         {
2038             "gem_parent_id": "notebook-example",
2039             "meaning_representation": "name[A]",
2040             "target": "A is a venue.",
2041         },
2042         e2e_config(),
2043         0,
2044     )
2045     examples_path = tmp_path / "examples.jsonl"
2046     precomputed_path = tmp_path / "precomputed.jsonl"

```

```

2047 variants_path = tmp_path / "variants.json"
2048 output_path = tmp_path / "generations.jsonl"
2049
2050 examples_path.write_text(
2051     json.dumps(example.model_dump(mode="json"), ensure_ascii=False) + "\n",
2052     encoding="utf-8",
2053 )
2054 precomputed = generation(variant="precomputed_full", text="A is a venue.").model_copy(
2055     update={
2056         "dataset_id": example.dataset_id,
2057         "example_id": example.example_id,
2058         "generation_id": (
2059             f"{example.dataset_id}__{example.example_id}__"
2060             "precomputed_full__r0__s42"
2061         ),
2062     }
2063 )
2064 precomputed_path.write_text(
2065     json.dumps(precomputed.model_dump(mode="json"), ensure_ascii=False) + "\n",
2066     encoding="utf-8",
2067 )
2068 variants_path.write_text(
2069     json.dumps(
2070         {
2071             "variants": [
2072                 {
2073                     "variant_id": "precomputed_full",
2074                     "backend": "precomputed",
2075                     "precomputed_path": str(precomputed_path),
2076                     "repetitions": 1,
2077                     "seeds": [42],
2078                 }
2079             ]
2080         },
2081         encoding="utf-8",
2082     )
2083 )
2084
2085 frame = await generate_reports_for_notebook(
2086     tmp_path,
2087     examples_path=examples_path,
2088     variants_path=variants_path,
2089     output_path=output_path,
2090     run_root=paths["run_root"],
2091 )
2092
2093 assert len(frame) == 1
2094 assert frame.loc[0, "generated_text"] == "A is a venue."

```

Listing 49: T03: table2text_pydanticai/tests/test_semantic_event_pipeline.py

```

1 from __future__ import annotations
2
3 import json
4
5 import pandas as pd
6 import pytest
7
8 from table2text.agents import validate_insight_candidates
9 from table2text.analytics import execute_plan
10 from table2text.audit import (
11     assess_genre_quality,
12     augment_fact_ledger_for_report_coverage,
13     build_writer_content_requirements,
14     content_requirement_errors,
15     deterministic_audit,
16     deterministic_fact_candidate_scaffold,
17     event_scope_limitation_present,
18     fact_support_numbers,
19     extract_number_tokens,
20     fallback_fact_candidates,
21     flatten_numbers,
22     event_fact_slot,
23     merge_fact_candidate_scaffold,
24     numbers_supported,
25     repair_spurious_missing_evidence_rejections,
26     scope_fact_ledger_for_genre,
27     select_event_priority_facts,
28     split_markdown_sentences,
29     validate_fact_candidates,
30     validate_writer_output,
31 )
32 from table2text.capabilities import (
33     available_capabilities,
34     build_event_evidence_queries,
35     event_capability_evidence,
36     normalise_event_evidence_queries,
37     normalise_semantic_map,
38     semantic_query_evidence,

```

```

39     validate_event_query_priorities,
40     validate_evidence_queries,
41     validate_semantic_map,
42 )
43 from table2text.config import Settings
44 from table2text.data import DataBundle, load_data
45 from table2text.narrative import build_event_narrative_plan
46 from table2text.schemas import (
47     AnalyticalFunction,
48     AnalysisRoute,
49     AuditMode,
50     ClaimPermission,
51     CommunicationTask,
52     DataUnderstanding,
53     EvaluationFieldPolicy,
54     EvidenceCapability,
55     EvidenceItem,
56     EvidenceLedger,
57     EvidenceOperation,
58     EvidenceQuery,
59     ExecutionPlan,
60     FactCandidate,
61     FactCandidateSet,
62     FactLedger,
63     FactReview,
64     InputRepresentationStatus,
65     InputSemanticMap,
66     InputShape,
67     InputStructureProfile,
68     InsightCandidate,
69     InsightCandidateSet,
70     InsightLedger,
71     InsightType,
72     InterpretationLevel,
73     InvestigationTask,
74     NarrativeSlot,
75     QualityStatus,
76     RealisationPolicy,
77     RecommendedUse,
78     ReportGenre,
79     ReportSelectionSource,
80     ReportSpecification,
81     ReviewDecision,
82     SemanticBinding,
83     SemanticLevel,
84     SemanticRole,
85     SentenceSupport,
86     StructuralField,
87     SupportType,
88     VerificationResult,
89     VerifiedFact,
90     WriterEvidencePack,
91     WriterOutput,
92     WriterAgentDraft,
93 )
94 from table2text.structure import build_structural_catalog
95 from table2text.task_contracts import infer_task_contract
96 from table2text.workflow import (
97     build_orchestrator_prompt_context,
98     infer_event_focus_scope,
99     resolve_report_genre,
100     task_contract_fields,
101 )
102
103
104 def renamed_event() -> dict:
105     return {
106         "occasion": {
107             "when": "2026-07-23",
108             "where": "Civic Hall",
109             "extra": False,
110         },
111         "sides": {
112             "north": {
113                 "label": "North",
114                 "tally": 12,
115                 "attempts": 17,
116                 "members": {
117                     "n1": {"label": "Nia", "alpha": 7},
118                     "n2": {"label": "Noor", "alpha": 4},
119                 },
120             },
121             "south": {
122                 "label": "South",
123                 "tally": 9,
124                 "attempts": 22,
125                 "members": {
126                     "s1": {"label": "Sol", "alpha": 6},
127                     "s2": {"label": "Sage", "alpha": 2},
128                 },
129             },
130         },
131     }

```

```

130     },
131 }
132
133
134 def semantic_map() -> InputSemanticMap:
135     def binding(
136         binding_id: str,
137         label: str,
138         role: SemanticRole,
139         level: SemanticLevel,
140         path: str,
141         analytical_function: AnalyticalFunction | None = None,
142     ) -> SemanticBinding:
143         return SemanticBinding(
144             binding_id=binding_id,
145             table_name="contest",
146             label=label,
147             role=role,
148             level=level,
149             path_pattern=path,
150             description=f"Semantic interpretation of {label}.",
151             confidence=0.98,
152             evidence_basis="Observed path and values in the sanitized catalog.",
153             analytical_function=analytical_function,
154         )
155
156     return InputSemanticMap(
157         input_shape=InputShape.EVENT_RECORD,
158         record_description="One event with two participants and nested entities.",
159         bindings=[
160             binding(
161                 "B_PARTICIPANT",
162                 "participant",
163                 SemanticRole.PARTICIPANT_IDENTIFIER,
164                 SemanticLevel.PARTICIPANT,
165                 "sides.*.label",
166             ),
167             binding(
168                 "B_OUTCOME",
169                 "event tally",
170                 SemanticRole.OUTCOME_MEASURE,
171                 SemanticLevel.PARTICIPANT,
172                 "sides.*.tally",
173                 AnalyticalFunction.OUTCOME,
174             ),
175             binding(
176                 "B_ATTEMPTS",
177                 "attempts",
178                 SemanticRole.MEASURE,
179                 SemanticLevel.PARTICIPANT,
180                 "sides.*.attempts",
181                 AnalyticalFunction.OUTCOME_COMPONENT,
182             ),
183             binding(
184                 "B_ENTITY",
185                 "member",
186                 SemanticRole.ENTITY_IDENTIFIER,
187                 SemanticLevel.ENTITY,
188                 "sides.*.members.*.label",
189             ),
190             binding(
191                 "B_ALPHA",
192                 "alpha performance",
193                 SemanticRole.PERFORMANCE_MEASURE,
194                 SemanticLevel.ENTITY,
195                 "sides.*.members.*.alpha",
196                 AnalyticalFunction.PERFORMANCE,
197             ),
198             binding(
199                 "B_TIME",
200                 "event date",
201                 SemanticRole.TIME,
202                 SemanticLevel.EVENT,
203                 "occasion.when",
204             ),
205             binding(
206                 "B_LOCATION",
207                 "event venue",
208                 SemanticRole.LOCATION,
209                 SemanticLevel.EVENT,
210                 "occasion.where",
211             ),
212             binding(
213                 "B_STATUS",
214                 "extra-period status",
215                 SemanticRole.STATUS,
216                 SemanticLevel.EVENT,
217                 "occasion.extra",
218             ),
219         ],
220         recommended_report_genre=ReportGenre.EVENT_REPORT,

```



```

221     report_rationale="The sanitized input describes one bounded event.",
222     confidence=0.98,
223 )
224
225
226 def semantic_queries() -> list[EvidenceQuery]:
227     common = {
228         "table_name": "contest",
229         "user_relevance": 0.95,
230         "salience": 0.95,
231     }
232     return [
233         EvidenceQuery(
234             query_id="QUERY_CONTEXT",
235             task_id="TASK_OUTCOME",
236             operation=EvidenceOperation.RETRIEVE,
237             capability=EvidenceCapability.EVENT_OUTCOME,
238             evidence_type="event_context",
239             semantic_label="event context",
240             question="What supplied context locates this event?",
241             semantic_level=SemanticLevel.EVENT,
242             value_binding_ids=["B_TIME", "B_LOCATION"],
243             recommended_use=RecommendedUse.HEADLINE,
244             **common,
245         ),
246         EvidenceQuery(
247             query_id="QUERY_STATUS",
248             task_id="TASK_OUTCOME",
249             operation=EvidenceOperation.RETRIEVE,
250             capability=EvidenceCapability.EVENT_OUTCOME,
251             evidence_type="event_status",
252             semantic_label="event status",
253             question="What status is recorded for this event?",
254             semantic_level=SemanticLevel.EVENT,
255             value_binding_ids=["B_STATUS"],
256             **common,
257         ),
258         EvidenceQuery(
259             query_id="QUERY_OUTCOME",
260             task_id="TASK_OUTCOME",
261             operation=EvidenceOperation.COMPARE,
262             capability=EvidenceCapability.EVENT_OUTCOME,
263             evidence_type="event_outcome",
264             semantic_label="event outcome",
265             question="How do the participant outcome measures compare?",
266             semantic_level=SemanticLevel.PARTICIPANT,
267             value_binding_ids=["B_OUTCOME"],
268             entity_binding_id="B_PARTICIPANT",
269             recommended_use=RecommendedUse.HEADLINE,
270             **common,
271         ),
272         EvidenceQuery(
273             query_id="QUERY_RANKING",
274             task_id="TASK_RANKING",
275             operation=EvidenceOperation.RANK,
276             capability=EvidenceCapability.RANKING,
277             evidence_type="entity_ranking",
278             semantic_label="alpha ranking",
279             question="Which entities have the highest alpha values?",
280             semantic_level=SemanticLevel.ENTITY,
281             value_binding_ids=["B_ALPHA"],
282             entity_binding_id="B_ENTITY",
283             group_binding_id="B_PARTICIPANT",
284             limit=3,
285             **common,
286         ),
287         EvidenceQuery(
288             query_id="QUERY_CONTRAST",
289             task_id="TASK_CONTRAST",
290             operation=EvidenceOperation.COMPARE,
291             capability=EvidenceCapability.GROUP_COMPARISON,
292             evidence_type="event_contrast",
293             semantic_label="attempt contrast",
294             question="How do participant attempts compare?",
295             semantic_level=SemanticLevel.PARTICIPANT,
296             value_binding_ids=["B_ATTEMPTS"],
297             entity_binding_id="B_PARTICIPANT",
298             **common,
299         ),
300     ]
301
302
303 def event_query_tasks() -> list[InvestigationTask]:
304     return [
305         InvestigationTask(
306             task_id="TASK_OUTCOME",
307             question="What is the verified event outcome and context?",
308             route=AnalysisRoute.DESCRPTIVE,
309             priority=5,
310             table_name="contest",
311             capability=EvidenceCapability.EVENT_OUTCOME,

```

```

312         expected_evidence_types=[
313             "event_context",
314             "event_status",
315             "event_outcome",
316         ],
317         required_evidence=[
318             "event_context",
319             "event_status",
320             "event_outcome",
321         ],
322         claim_permissions=[
323             ClaimPermission.DESCRPTIVE,
324             ClaimPermission.COMPARATIVE,
325         ],
326         answerability_note="Answerable from the structured event.",
327     ),
328     InvestigationTask(
329         task_id="TASK_RANKING",
330         question="Which entities lead recorded performance measures?",
331         route=AnalysisRoute.DESCRPTIVE,
332         priority=4,
333         table_name="contest",
334         capability=EvidenceCapability.RANKING,
335         expected_evidence_types=["entity_ranking"],
336         required_evidence=["entity_ranking"],
337         claim_permissions=[
338             ClaimPermission.DESCRPTIVE,
339             ClaimPermission.COMPARATIVE,
340         ],
341         answerability_note="Answerable from entity measures.",
342     ),
343     InvestigationTask(
344         task_id="TASK_CONTRAST",
345         question="How do participants compare on recorded measures?",
346         route=AnalysisRoute.ASSOCIATION_COMPARISON,
347         priority=4,
348         table_name="contest",
349         capability=EvidenceCapability.GROUP_COMPARISON,
350         expected_evidence_types=["participant_comparison"],
351         required_evidence=["participant_comparison"],
352         claim_permissions=[
353             ClaimPermission.DESCRPTIVE,
354             ClaimPermission.COMPARATIVE,
355         ],
356         answerability_note="Answerable from participant measures.",
357     ),
358 ]
359
360
361 def test_semantic_map_accepts_broad_event_binding_coverage():
362     compact_map = semantic_map()
363     bindings = [
364         binding.model_copy(
365             update={"binding_id": f"B_{index:02d}"},
366         )
367         for index, binding in enumerate(
368             (compact_map.bindings * 4)[:25],
369             start=1,
370         )
371     ]
372
373     broad_map = InputSemanticMap(
374         input_shape=compact_map.input_shape,
375         record_description=compact_map.record_description,
376         bindings=bindings,
377         recommended_report_genre=compact_map.recommended_report_genre,
378         report_rationale=compact_map.report_rationale,
379         confidence=compact_map.confidence,
380     )
381
382     assert len(broad_map.bindings) == 25
383
384
385 def test_orchestrator_context_exposes_binding_ids_without_raw_paths():
386     understanding = DataUnderstanding(
387         profile_fingerprint="fixture",
388         dataset_summary="One structured event.",
389         tables=[],
390         semantic_map=semantic_map(),
391     )
392     structure = event_structure().model_copy(
393         update={"nested_paths": ["sides.*.members.*.alpha"]}
394     )
395     context = build_orchestrator_prompt_context(
396         understanding=understanding,
397         input_structure=structure,
398         structural_catalog=build_structural_catalog(
399             {"contest": renamed_event()}
400         ),
401     )
402

```

```

403     assert "semantic_map" not in context["understanding"]
404     assert context["input_structure"]["nested_paths"] == []
405     assert context["structural_catalog"] == []
406     assert {
407         item["binding_id"]
408         for item in context["semantic_binding_catalog"]
409     } == {binding.binding_id for binding in semantic_map().bindings}
410     assert all(
411         "path_pattern" not in item
412         for item in context["semantic_binding_catalog"]
413     )
414     assert next(
415         item
416         for item in context["semantic_binding_catalog"]
417         if item["binding_id"] == "B_ALPHA"
418     )["analytical_function"] == "performance"
419
420 def test_event_semantic_map_reserves_substantive_entity_measures():
421     payload = renamed_event()
422     for side in payload["sides"].values():
423         for member in side["members"].values():
424             member["beta"] = 3
425             member["gamma"] = 2
426
427     errors = validate_semantic_map(
428         semantic_map(),
429         build_structural_catalog({"contest": payload}),
430         require_entity_measure_coverage=True,
431     )
432
433     assert any(
434         "reserve substantive entity-performance bindings"
435         in error
436         for error in errors
437     )
438
439
440 def paired_line_event() -> dict:
441     return {
442         "left_line": {
443             "result": "win",
444             "team_runs": "7",
445             "team_hits": "11",
446             "team_errors": "1",
447             "team_name": "Mets",
448         },
449         "right_line": {
450             "result": "loss",
451             "team_runs": "2",
452             "team_hits": "7",
453             "team_errors": "1",
454             "team_name": "D-backs",
455         },
456         "box_score": [
457             {
458                 "full_name": "Jose Reyes",
459                 "team": "Mets",
460                 "h": "4",
461                 "r": "3",
462                 "rbi": "0",
463                 "hr": "0",
464             },
465             {
466                 "full_name": "Ryan Church",
467                 "team": "Mets",
468                 "h": "2",
469                 "r": "2",
470                 "rbi": "3",
471                 "hr": "1",
472             },
473             {
474                 "full_name": "John Maine",
475                 "team": "Mets",
476                 "p_so": "6",
477                 "p_r": "2",
478             },
479             {
480                 "full_name": "Chris Young",
481                 "team": "D-backs",
482                 "h": "1",
483                 "r": "0",
484                 "rbi": "1",
485             },
486         ],
487         "day": "05_02_08",
488     }
489
490
491 def test_paired_participant_line_records_enable_event_capabilities(
492     tmp_path,
493 
```

```

494 ):
495     path = tmp_path / "paired_event.json"
496     path.write_text(json.dumps(paired_line_event()), encoding="utf-8")
497     bundle = load_data([path])
498     capabilities = available_capabilities(bundle)
499
500     assert bundle.input_structure.shape == InputShape.EVENT_RECORD
501     assert EvidenceCapability.EVENT_OUTCOME in capabilities
502     assert EvidenceCapability.RANKING in capabilities
503     assert EvidenceCapability.GROUP_COMPARISON in capabilities
504
505     evidence = event_capability_evidence(paired_line_event())
506     outcome = next(
507         item
508         for item in evidence
509         if item.evidence_type == "event_outcome"
510     )
511     rankings = [
512         item
513         for item in evidence
514         if item.evidence_type == "entity_ranking"
515     ]
516     contrasts = [
517         item
518         for item in evidence
519         if item.evidence_type == "participant_comparison"
520     ]
521
522     assert outcome.metrics["winner"] == "Mets"
523     assert outcome.metrics["loser"] == "D-backs"
524     assert outcome.metrics["winner_score"] == 7
525     assert any(
526         item.metrics["metric"] == "hits"
527         and item.metrics["ranking"][0]["entity"] == "Jose Reyes"
528         and item.metrics["ranking"][0]["value"] == 4
529         for item in rankings
530     )
531     assert any(item.metrics["metric"] == "hits" for item in contrasts)
532
533
534 def test_event_sequence_evidence_is_extracted_from_ordered_nested_records():
535     payload = {
536         "left_line": {
537             "result": "win",
538             "team_points": "9",
539             "team_name": "North",
540         },
541         "right_line": {
542             "result": "loss",
543             "team_points": "4",
544             "team_name": "South",
545         },
546         "timeline": [
547             {
548                 "period": 1,
549                 "actions": [
550                     {
551                         "actor": "North",
552                         "action": "score",
553                         "north_score": 2,
554                         "south_score": 0,
555                     },
556                     {
557                         "actor": "South",
558                         "action": "score",
559                         "north_score": 2,
560                         "south_score": 1,
561                     },
562                 ],
563             },
564             {
565                 "period": 2,
566                 "actions": [
567                     {
568                         "actor": "North",
569                         "action": "score",
570                         "north_score": 9,
571                         "south_score": 4,
572                     },
573                 ],
574             },
575         ],
576     }
577
578     evidence = event_capability_evidence(payload)
579     sequence = next(
580         item
581         for item in evidence
582         if item.evidence_type == "event_sequence"
583     )
584

```

```

585     assert sequence.metrics["step_count"] == 3
586     assert sequence.metrics["score_state_step_count"] == 3
587     assert "ordered event sequence" in sequence.finding
588     assert "score-state fields" in sequence.finding
589
590
591 def test_semantic_map_normalisation_repairs_obvious_event_function_mismatches():
592     catalog = [
593         StructuralField(
594             table_name="event",
595             path_pattern="left.name",
596             value_types=["string"],
597         ),
598         StructuralField(
599             table_name="event",
600             path_pattern="left.score",
601             value_types=["integer"],
602         ),
603         StructuralField(
604             table_name="event",
605             path_pattern="event.date",
606             value_types=["string"],
607         ),
608     ]
609     semantic_map = InputSemanticMap(
610         input_shape=InputShape.EVENT_RECORD,
611         record_description="One event.",
612         confidence=0.9,
613         bindings=[
614             SemanticBinding(
615                 binding_id="b_name",
616                 table_name="event",
617                 label="Participant name",
618                 role=SemanticRole.PARTICIPANT_IDENTIFIER,
619                 level=SemanticLevel.PARTICIPANT,
620                 path_pattern="left.name",
621                 description="Participant identity.",
622                 confidence=0.9,
623                 evidence_basis="field name",
624                 analytical_function=AnalyticalFunction.PERFORMANCE,
625             ),
626             SemanticBinding(
627                 binding_id="b_score",
628                 table_name="event",
629                 label="Score",
630                 role=SemanticRole.OUTCOME_MEASURE,
631                 level=SemanticLevel.PARTICIPANT,
632                 path_pattern="left.score",
633                 description="Recorded outcome value.",
634                 confidence=0.9,
635                 evidence_basis="field name",
636                 analytical_function=AnalyticalFunction.PERFORMANCE,
637             ),
638             SemanticBinding(
639                 binding_id="b_date",
640                 table_name="event",
641                 label="Date",
642                 role=SemanticRole.TIME,
643                 level=SemanticLevel.EVENT,
644                 path_pattern="event.date",
645                 description="Event time context.",
646                 confidence=0.9,
647                 evidence_basis="field name",
648                 analytical_function=AnalyticalFunction.OUTCOME_COMPONENT,
649             ),
650         ],
651     )
652
653     assert validate_semantic_map(semantic_map, catalog)
654
655     repaired = normalise_semantic_map(semantic_map)
656
657     assert repaired is not None
658     assert not validate_semantic_map(repaired, catalog)
659     repaired_by_id = {
660         binding.binding_id: binding
661         for binding in repaired.bindings
662     }
663     assert repaired_by_id["b_name"].analytical_function is None
664     assert (
665         repaired_by_id["b_score"].analytical_function
666         == AnalyticalFunction.OUTCOME
667     )
668     assert repaired_by_id["b_date"].analytical_function is None
669
670
671 def test_event_query_validation_accepts_numeric_like_string_samples():
672     catalog = [
673         StructuralField(
674             table_name="event",
675             path_pattern="entities[].name",

```

```

676         value_types=["string"],
677         sample_values=["Alpha", "Beta"],
678     ),
679     StructuralField(
680         table_name="event",
681         path_pattern="entities[].metric",
682         value_types=["string"],
683         sample_values=["N/A", "4", "2"],
684     ),
685 ]
686 semantic_map = InputSemanticMap(
687     input_shape=InputShape.EVENT_RECORD,
688     record_description="One event.",
689     confidence=0.9,
690     bindings=[
691         SemanticBinding(
692             binding_id="b_entity",
693             table_name="event",
694             label="Entity name",
695             role=SemanticRole.ENTITY_IDENTIFIER,
696             level=SemanticLevel.ENTITY,
697             path_pattern="entities[].name",
698             description="Entity identity.",
699             confidence=0.9,
700             evidence_basis="field name",
701         ),
702         SemanticBinding(
703             binding_id="b_metric",
704             table_name="event",
705             label="Entity metric",
706             role=SemanticRole.PERFORMANCE_MEASURE,
707             level=SemanticLevel.ENTITY,
708             path_pattern="entities[].metric",
709             description="Numeric-like metric stored as strings.",
710             confidence=0.9,
711             evidence_basis="sample values",
712             analytical_function=AnalyticalFunction.PERFORMANCE,
713         ),
714     ],
715 )
716 query = EvidenceQuery(
717     query_id="QUERY_METRIC",
718     task_id="TASK_EVENT",
719     operation=EvidenceOperation.RANK,
720     capability=EvidenceCapability.RANKING,
721     evidence_type="entity_ranking",
722     semantic_label="entity metric",
723     question="Which entities have the highest recorded values?",
724     semantic_level=SemanticLevel.ENTITY,
725     table_name="event",
726     value_binding_ids=["b_metric"],
727     entity_binding_id="b_entity",
728 )
729
730 errors = validate_evidence_queries(
731     [query],
732     semantic_map,
733     catalog,
734     task_ids={"TASK_EVENT"},
735     available={EvidenceCapability.RANKING},
736     task_capabilities={"TASK_EVENT": EvidenceCapability.RANKING},
737 )
738
739 assert not any("non-numeric" in error for error in errors)
740
741
742 def test_event_query_completion_recovers_repeated_actor_performance_fields():
743     catalog = [
744         StructuralField(
745             table_name="event",
746             path_pattern="actors.*.name",
747             value_types=["string"],
748             sample_values=["Ada", "Bo"],
749         ),
750         StructuralField(
751             table_name="event",
752             path_pattern="actors.*.side",
753             value_types=["string"],
754             sample_values=["North", "South"],
755         ),
756         StructuralField(
757             table_name="event",
758             path_pattern="actors.*.points",
759             value_types=["string"],
760             sample_values=["9", "4"],
761         ),
762         StructuralField(
763             table_name="event",
764             path_pattern="actors.*.assists",
765             value_types=["string"],
766             sample_values=["3", "7"],

```

```

767     ),
768 ]
769 semantic_map = InputSemanticMap(
770     input_shape=InputShape.EVENT_RECORD,
771     record_description="One event with repeated actor records.",
772     confidence=0.9,
773     bindings=[
774         SemanticBinding(
775             binding_id="b_actor",
776             table_name="event",
777             label="Actor name",
778             role=SemanticRole.PARTICIPANT_IDENTIFIER,
779             level=SemanticLevel.PARTICIPANT,
780             path_pattern="actors.*.name",
781             description="Actor identity inside a repeated record.",
782             confidence=0.9,
783             evidence_basis="field name",
784         ),
785         SemanticBinding(
786             binding_id="b_side",
787             table_name="event",
788             label="Actor side",
789             role=SemanticRole.ENTITY_IDENTIFIER,
790             level=SemanticLevel.PARTICIPANT,
791             path_pattern="actors.*.side",
792             description="Actor affiliation inside the repeated record.",
793             confidence=0.9,
794             evidence_basis="field name",
795         ),
796         SemanticBinding(
797             binding_id="b_points",
798             table_name="event",
799             label="Points",
800             role=SemanticRole.PERFORMANCE_MEASURE,
801             level=SemanticLevel.PARTICIPANT,
802             path_pattern="actors.*.points",
803             description="Actor performance value.",
804             confidence=0.9,
805             evidence_basis="numeric-like values",
806             analytical_function=AnalyticalFunction.PERFORMANCE,
807         ),
808         SemanticBinding(
809             binding_id="b_assists",
810             table_name="event",
811             label="Assists",
812             role=SemanticRole.PERFORMANCE_MEASURE,
813             level=SemanticLevel.PARTICIPANT,
814             path_pattern="actors.*.assists",
815             description="Actor performance value.",
816             confidence=0.9,
817             evidence_basis="numeric-like values",
818             analytical_function=AnalyticalFunction.PERFORMANCE,
819         ),
820     ],
821 )
822 tasks = [
823     InvestigationTask(
824         task_id="TASK_RANKING",
825         question="Which actors have the highest recorded performances?",
826         route=AnalysisRoute.DESRIPTIVE,
827         priority=4,
828         table_name="event",
829         capability=EvidenceCapability.RANKING,
830         expected_evidence_types=["entity_ranking"],
831         required_evidence=["entity_ranking"],
832         claim_permissions=[
833             ClaimPermission.DESRIPTIVE,
834             ClaimPermission.COMPARATIVE,
835         ],
836         answerability_note="Answerable from repeated actor records.",
837     )
838 ]
839
840 repaired = normalise_semantic_map(semantic_map)
841 queries = normalise_event_evidence_queries(
842     queries=[],
843     semantic_map=repaired,
844     tasks=tasks,
845     available_capabilities={EvidenceCapability.RANKING},
846     request="Write an event report.",
847 )
848
849 by_value = {
850     tuple(query.value_binding_ids): query
851     for query in queries
852     if query.evidence_type == "entity_ranking"
853 }
854
855 assert repaired is not None
856 assert next(
857     binding

```

```

858     for binding in repaired.bindings
859     if binding.binding_id == "b_actor"
860 ).level == SemanticLevel.ENTITY
861 assert by_value[("b_points",)].entity_binding_id == "b_actor"
862 assert by_value[("b_points",)].group_binding_id == "b_side"
863 assert by_value[("b_assists",)].entity_binding_id == "b_actor"
864 assert not validate_evidence_queries(
865     queries,
866     repaired,
867     catalog,
868     task_ids={"TASK_RANKING"},
869     available={EvidenceCapability.RANKING},
870     task_capabilities={"TASK_RANKING": EvidenceCapability.RANKING},
871 )
872
873
874 def test_event_query_normalisation_prunes_unexecutable_rankings():
875     catalog = [
876         StructuralField(
877             table_name="event",
878             path_pattern="teams.*.name",
879             value_types=["string"],
880             sample_values=["North", "South"],
881         ),
882         StructuralField(
883             table_name="event",
884             path_pattern="teams.*.score",
885             value_types=["integer"],
886             sample_values=["9", "4"],
887         ),
888         StructuralField(
889             table_name="event",
890             path_pattern="players.*.name",
891             value_types=["string"],
892             sample_values=["Ada", "Bo"],
893         ),
894         StructuralField(
895             table_name="event",
896             path_pattern="players.*.metric",
897             value_types=["string"],
898             sample_values=["N/A", "-"],
899         ),
900         StructuralField(
901             table_name="event",
902             path_pattern="players.*.points",
903             value_types=["string"],
904             sample_values=["9", "4"],
905         ),
906     ]
907     semantic_map = InputSemanticMap(
908         input_shape=InputShape.EVENT_RECORD,
909         record_description="One event with team and player records.",
910         confidence=0.9,
911         bindings=[
912             SemanticBinding(
913                 binding_id="team_name",
914                 table_name="event",
915                 label="Team name",
916                 role=SemanticRole.PARTICIPANT_IDENTIFIER,
917                 level=SemanticLevel.PARTICIPANT,
918                 path_pattern="teams.*.name",
919                 description="Team identity.",
920                 confidence=0.9,
921                 evidence_basis="field name",
922             ),
923             SemanticBinding(
924                 binding_id="team_score",
925                 table_name="event",
926                 label="Team score",
927                 role=SemanticRole.OUTCOME_MEASURE,
928                 level=SemanticLevel.PARTICIPANT,
929                 path_pattern="teams.*.score",
930                 description="Team score.",
931                 confidence=0.9,
932                 evidence_basis="field name",
933                 analytical_function=AnalyticalFunction.OUTCOME,
934             ),
935             SemanticBinding(
936                 binding_id="player_name",
937                 table_name="event",
938                 label="Player name",
939                 role=SemanticRole.ENTITY_IDENTIFIER,
940                 level=SemanticLevel.ENTITY,
941                 path_pattern="players.*.name",
942                 description="Player identity.",
943                 confidence=0.9,
944                 evidence_basis="field name",
945             ),
946             SemanticBinding(
947                 binding_id="player_metric",
948                 table_name="event",

```



```

949         label="Unavailable player metric",
950         role=SemanticRole.PERFORMANCE_MEASURE,
951         level=SemanticLevel.ENTITY,
952         path_pattern="players.*.metric",
953         description="A metric with no numeric support.",
954         confidence=0.9,
955         evidence_basis="field name",
956         analytical_function=AnalyticalFunction.PERFORMANCE,
957     ),
958     SemanticBinding(
959         binding_id="player_points",
960         table_name="event",
961         label="Player points",
962         role=SemanticRole.PERFORMANCE_MEASURE,
963         level=SemanticLevel.ENTITY,
964         path_pattern="players.*.points",
965         description="A numeric player performance value.",
966         confidence=0.9,
967         evidence_basis="field name",
968         analytical_function=AnalyticalFunction.PERFORMANCE,
969     ),
970 ],
971 )
972 tasks = [
973     InvestigationTask(
974         task_id="TASK_RANKING",
975         question="Rank player performances.",
976         route=AnalysisRoute.DESRIPTIVE,
977         priority=4,
978         table_name="event",
979         capability=EvidenceCapability.RANKING,
980         expected_evidence_types=["entity_ranking"],
981         required_evidence=["entity_ranking"],
982         claim_permissions=[ClaimPermission.DESRIPTIVE],
983         answerability_note="Only executable if a numeric measure exists.",
984     ),
985     InvestigationTask(
986         task_id="TASK_EVENT",
987         question="Report the result.",
988         route=AnalysisRoute.DESRIPTIVE,
989         priority=5,
990         table_name="event",
991         capability=EvidenceCapability.EVENT_OUTCOME,
992         expected_evidence_types=["event_outcome"],
993         required_evidence=["event_outcome"],
994         claim_permissions=[ClaimPermission.DESRIPTIVE],
995         answerability_note="Supported by team score.",
996     ),
997 ]
998 bad_queries = [
999     EvidenceQuery(
1000         query_id="BAD_ENTITY_RANKING",
1001         task_id="TASK_RANKING",
1002         operation=EvidenceOperation.RANK,
1003         capability=EvidenceCapability.RANKING,
1004         evidence_type="entity_ranking",
1005         semantic_label="player unavailable metric",
1006         question="Which players rank highest?",
1007         semantic_level=SemanticLevel.ENTITY,
1008         table_name="event",
1009         value_binding_ids=["player_metric"],
1010         entity_binding_id="player_name",
1011     ),
1012     EvidenceQuery(
1013         query_id="BAD_TEAM_RANKING",
1014         task_id="TASK_RANKING",
1015         operation=EvidenceOperation.RANK,
1016         capability=EvidenceCapability.RANKING,
1017         evidence_type="entity_ranking",
1018         semantic_label="team score as entity ranking",
1019         question="Which entities rank highest?",
1020         semantic_level=SemanticLevel.ENTITY,
1021         table_name="event",
1022         value_binding_ids=["team_score"],
1023         entity_binding_id="team_name",
1024     ),
1025     EvidenceQuery(
1026         query_id="GOOD_ENTITY_RANKING",
1027         task_id="TASK_RANKING",
1028         operation=EvidenceOperation.RANK,
1029         capability=EvidenceCapability.RANKING,
1030         evidence_type="entity_ranking",
1031         semantic_label="player points",
1032         question="Which players rank highest by points?",
1033         semantic_level=SemanticLevel.ENTITY,
1034         table_name="event",
1035         value_binding_ids=["player_points"],
1036         entity_binding_id="player_name",
1037     ),
1038 ]
1039

```

```

1040 queries = normalise_event_evidence_queries(
1041     queries=bad_queries,
1042     semantic_map=semantic_map,
1043     tasks=tasks,
1044     available_capabilities={
1045         EvidenceCapability.EVENT_OUTCOME,
1046         EvidenceCapability.RANKING,
1047     },
1048     request="Write an event report.",
1049     structural_catalog=catalog,
1050 )
1051
1052 assert all(
1053     query.query_id not in {"BAD_ENTITY_RANKING", "BAD_TEAM_RANKING"}
1054     for query in queries
1055 )
1056 assert any(query.query_id == "GOOD_ENTITY_RANKING" for query in queries)
1057 assert not validate_evidence_queries(
1058     queries,
1059     semantic_map,
1060     catalog,
1061     task_ids={"TASK_RANKING", "TASK_EVENT"},
1062     available={
1063         EvidenceCapability.EVENT_OUTCOME,
1064         EvidenceCapability.RANKING,
1065     },
1066     task_capabilities={
1067         "TASK_RANKING": EvidenceCapability.RANKING,
1068         "TASK_EVENT": EvidenceCapability.EVENT_OUTCOME,
1069     },
1070 )
1071
1072
1073 def test_semantic_rankings_skip_zero_only_performance_leaders():
1074     payload = {
1075         "players": [
1076             {"name": "Ada", "metric": "0"},
1077             {"name": "Bo", "metric": "0"},
1078         ]
1079     }
1080     semantic_map = InputSemanticMap(
1081         input_shape=InputShape.EVENT_RECORD,
1082         record_description="One event with player records.",
1083         confidence=0.9,
1084         bindings=[
1085             SemanticBinding(
1086                 binding_id="player_name",
1087                 table_name="event",
1088                 label="Player name",
1089                 role=SemanticRole.ENTITY_IDENTIFIER,
1090                 level=SemanticLevel.ENTITY,
1091                 path_pattern="players.*.name",
1092                 description="Player identity.",
1093                 confidence=0.9,
1094                 evidence_basis="field name",
1095             ),
1096             SemanticBinding(
1097                 binding_id="player_metric",
1098                 table_name="event",
1099                 label="Player performance metric",
1100                 role=SemanticRole.PERFORMANCE_MEASURE,
1101                 level=SemanticLevel.ENTITY,
1102                 path_pattern="players.*.metric",
1103                 description="Player performance value.",
1104                 confidence=0.9,
1105                 evidence_basis="field name",
1106                 analytical_function=AnalyticalFunction.PERFORMANCE,
1107             ),
1108         ],
1109     )
1110     query = EvidenceQuery(
1111         query_id="QUERY_ZERO_ONLY",
1112         task_id="TASK_RANKING",
1113         operation=EvidenceOperation.RANK,
1114         capability=EvidenceCapability.RANKING,
1115         evidence_type="entity_ranking",
1116         semantic_label="zero-only performance",
1117         question="Which players rank highest?",
1118         semantic_level=SemanticLevel.ENTITY,
1119         table_name="event",
1120         value_binding_ids=["player_metric"],
1121         entity_binding_id="player_name",
1122     )
1123
1124     evidence = semantic_query_evidence(
1125         table_name="event",
1126         payload=payload,
1127         semantic_map=semantic_map,
1128         queries=[query],
1129     )
1130

```

```

1131     assert evidence == []
1132
1133
1134 def test_semantic_event_outcome_pairs_parallel_participant_fields():
1135     payload = {
1136         "home_name": "Astros",
1137         "vis_name": "Rangers",
1138         "home_line": {"team_runs": "4"},
1139         "vis_line": {"team_runs": "3"},
1140     }
1141     catalog = [
1142         StructuralField(
1143             table_name="event",
1144             path_pattern="home_name",
1145             value_types=["string"],
1146             sample_values=["Astros"],
1147         ),
1148         StructuralField(
1149             table_name="event",
1150             path_pattern="vis_name",
1151             value_types=["string"],
1152             sample_values=["Rangers"],
1153         ),
1154         StructuralField(
1155             table_name="event",
1156             path_pattern="home_line.team_runs",
1157             value_types=["string"],
1158             sample_values=["4"],
1159         ),
1160         StructuralField(
1161             table_name="event",
1162             path_pattern="vis_line.team_runs",
1163             value_types=["string"],
1164             sample_values=["3"],
1165         ),
1166     ]
1167     semantic_map = InputSemanticMap(
1168         input_shape=InputShape.EVENT_RECORD,
1169         record_description="One event with parallel participant lines.",
1170         confidence=0.9,
1171         bindings=[
1172             SemanticBinding(
1173                 binding_id="home_name",
1174                 table_name="event",
1175                 label="Home team name",
1176                 role=SemanticRole.PARTICIPANT_IDENTIFIER,
1177                 level=SemanticLevel.PARTICIPANT,
1178                 path_pattern="home_name",
1179                 description="Home participant identity.",
1180                 confidence=0.9,
1181                 evidence_basis="field name",
1182             ),
1183             SemanticBinding(
1184                 binding_id="vis_name",
1185                 table_name="event",
1186                 label="Visiting team name",
1187                 role=SemanticRole.PARTICIPANT_IDENTIFIER,
1188                 level=SemanticLevel.PARTICIPANT,
1189                 path_pattern="vis_name",
1190                 description="Visiting participant identity.",
1191                 confidence=0.9,
1192                 evidence_basis="field name",
1193             ),
1194             SemanticBinding(
1195                 binding_id="home_runs",
1196                 table_name="event",
1197                 label="Home team total runs",
1198                 role=SemanticRole.OUTCOME_MEASURE,
1199                 level=SemanticLevel.PARTICIPANT,
1200                 path_pattern="home_line.team_runs",
1201                 description="Home participant outcome value.",
1202                 confidence=0.9,
1203                 evidence_basis="field name",
1204                 analytical_function=AnalyticalFunction.OUTCOME,
1205             ),
1206             SemanticBinding(
1207                 binding_id="vis_runs",
1208                 table_name="event",
1209                 label="Visiting team total runs",
1210                 role=SemanticRole.OUTCOME_MEASURE,
1211                 level=SemanticLevel.PARTICIPANT,
1212                 path_pattern="vis_line.team_runs",
1213                 description="Visiting participant outcome value.",
1214                 confidence=0.9,
1215                 evidence_basis="field name",
1216                 analytical_function=AnalyticalFunction.OUTCOME,
1217             ),
1218         ],
1219     )
1220     tasks = [
1221         InvestigationTask(

```

```

1222         task_id="TASK_EVENT",
1223         question="Report the event outcome.",
1224         route=AnalysisRoute.DESCRPTIVE,
1225         priority=5,
1226         table_name="event",
1227         capability=EvidenceCapability.EVENT_OUTCOME,
1228         expected_evidence_types=["event_outcome"],
1229         required_evidence=["event_outcome"],
1230         claim_permissions=[
1231             ClaimPermission.DESCRPTIVE,
1232             ClaimPermission.COMPARATIVE,
1233         ],
1234         answerability_note="Supported by parallel participant lines.",
1235     )
1236 ]
1237
1238 queries = normalise_event_evidence_queries(
1239     queries=[],
1240     semantic_map=semantic_map,
1241     tasks=tasks,
1242     available_capabilities={EvidenceCapability.EVENT_OUTCOME},
1243     request="Write an event report.",
1244     structural_catalog=catalog,
1245 )
1246 outcome_query = next(
1247     query for query in queries if query.evidence_type == "event_outcome"
1248 )
1249
1250 assert set(outcome_query.value_binding_ids) == {
1251     "home_runs",
1252     "vis_runs",
1253 }
1254 assert outcome_query.group_binding_id is None
1255 assert not validate_evidence_queries(
1256     queries,
1257     semantic_map,
1258     catalog,
1259     task_ids={"TASK_EVENT"},
1260     available={EvidenceCapability.EVENT_OUTCOME},
1261     task_capabilities={"TASK_EVENT": EvidenceCapability.EVENT_OUTCOME},
1262 )
1263
1264 evidence = semantic_query_evidence(
1265     table_name="event",
1266     payload=payload,
1267     semantic_map=semantic_map,
1268     queries=queries,
1269 )
1270 outcome = next(
1271     item for item in evidence if item.evidence_type == "event_outcome"
1272 )
1273
1274 assert outcome.metrics["difference"] == 1
1275 assert {
1276     (record["entity"], record["value"])
1277     for record in outcome.metrics["records"]
1278 } == {("Astros", 4.0), ("Rangers", 3.0)}
1279
1280 def test_semantic_event_comparisons_prefer_game_totals_over_segments():
1281     payload = {
1282         "teams": {
1283             "home": {
1284                 "name": "Bucks",
1285                 "line_score": {
1286                     "game": {"PTS": "114"},
1287                     "Q1": {"PTS": "30"},
1288                     "Q2": {"PTS": "31"},
1289                 },
1290             },
1291             "vis": {
1292                 "name": "Suns",
1293                 "line_score": {
1294                     "game": {"PTS": "116"},
1295                     "Q1": {"PTS": "34"},
1296                     "Q2": {"PTS": "30"},
1297                 },
1298             },
1299         },
1300     }
1301 }
1302 semantic_map = InputSemanticMap(
1303     input_shape=InputShape.EVENT_RECORD,
1304     record_description="One event with game and segment scores.",
1305     confidence=0.95,
1306     bindings=[
1307         SemanticBinding(
1308             binding_id="team_name",
1309             table_name="event",
1310             label="Team name",
1311             role=SemanticRole.PARTICIPANT_IDENTIFIER,
1312             level=SemanticLevel.PARTICIPANT,

```

```

1313         path_pattern="teams.*.name",
1314         description="Participant identity.",
1315         confidence=0.95,
1316         evidence_basis="field name",
1317     ),
1318     SemanticBinding(
1319         binding_id="team_points",
1320         table_name="event",
1321         label="Team points",
1322         role=SemanticRole.OUTCOME_MEASURE,
1323         level=SemanticLevel.PARTICIPANT,
1324         path_pattern="teams.*.line_score.*.PTS",
1325         description="Participant score values at multiple levels.",
1326         confidence=0.95,
1327         evidence_basis="field name",
1328         analytical_function=AnalyticalFunction.OUTCOME,
1329     ),
1330 ],
1331 )
1332 query = EvidenceQuery(
1333     query_id="QUERY_EVENT_OUTCOME",
1334     task_id="TASK_EVENT",
1335     operation=EvidenceOperation.COMPARE,
1336     capability=EvidenceCapability.EVENT_OUTCOME,
1337     evidence_type="event_outcome",
1338     semantic_label="event outcome",
1339     question="How do participant scores compare?",
1340     semantic_level=SemanticLevel.PARTICIPANT,
1341     table_name="event",
1342     value_binding_ids=["team_points"],
1343     entity_binding_id="team_name",
1344 )
1345
1346 evidence = semantic_query_evidence(
1347     table_name="event",
1348     payload=payload,
1349     semantic_map=semantic_map,
1350     queries=[query],
1351 )
1352 outcome = next(
1353     item for item in evidence if item.evidence_type == "event_outcome"
1354 )
1355
1356 assert outcome.metrics["record_level_filter"] == "aggregate_event_level"
1357 assert outcome.metrics["difference"] == 2
1358 assert {
1359     (record["entity"], record["value"], record["source_path"])
1360     for record in outcome.metrics["records"]
1361 } == {
1362     ("Bucks", 114.0, "teams.home.line_score.game.PTS"),
1363     ("Suns", 116.0, "teams.vis.line_score.game.PTS"),
1364 }
1365
1366
1367 def test_event_capability_evidence_adds_record_context_and_score_progression():
1368     payload = {
1369         "teams": {
1370             "home": {
1371                 "name": "Bucks",
1372                 "place": "Milwaukee",
1373                 "wins": "13",
1374                 "losses": "5",
1375                 "line_score": {
1376                     "game": {"PTS": "114"},
1377                     "Q1": {"PTS": "30"},
1378                     "Q2": {"PTS": "31"},
1379                     "Q3": {"PTS": "29"},
1380                     "Q4": {"PTS": "24"},
1381                 },
1382                 "box_score": [
1383                     {"name": "Home Star", "PTS": "35"},
1384                 ],
1385             },
1386             "vis": {
1387                 "name": "Suns",
1388                 "place": "Phoenix",
1389                 "wins": "4",
1390                 "losses": "14",
1391                 "line_score": {
1392                     "game": {"PTS": "116"},
1393                     "Q1": {"PTS": "34"},
1394                     "Q2": {"PTS": "30"},
1395                     "Q3": {"PTS": "27"},
1396                     "Q4": {"PTS": "25"},
1397                 },
1398                 "box_score": [
1399                     {"name": "Vis Star", "PTS": "29"},
1400                 ],
1401             },
1402         }
1403     }

```

```

1404
1405 evidence = event_capability_evidence(payload)
1406 by_type = {item.evidence_type: item for item in evidence}
1407
1408 assert "participant_record_context" in by_type
1409 assert "Milwaukee Bucks entered with 13 wins and 5 losses" in (
1410     by_type["participant_record_context"].finding
1411 )
1412 assert "score_progression" in by_type
1413 progression = by_type["score_progression"].metrics["segments"]
1414 after_q2 = next(item for item in progression if item["segment"] == "Q2")
1415 assert after_q2["cumulative_values"] == {
1416     "Milwaukee Bucks": 61.0,
1417     "Phoenix Suns": 64.0,
1418 }
1419 assert after_q2["leader"] == "Phoenix Suns"
1420
1421
1422 def test_event_capability_evidence_adds_rich_entity_performances_and_adjacent_context():
1423     payload = {
1424         "teams": {
1425             "home": {
1426                 "name": "North",
1427                 "wins": "8",
1428                 "losses": "2",
1429                 "next_game": {
1430                     "opponent_name": "East",
1431                     "dayname": "Friday",
1432                     "day": "9",
1433                     "month": "May",
1434                     "year": "2026",
1435                     "stadium": "Civic Arena",
1436                     "city": "Metro",
1437                     "is_home": "true",
1438                 },
1439                 "line_score": {"game": {"PTS": "90"}},
1440                 "box_score": [
1441                     {
1442                         "name": "North Guard",
1443                         "PTS": "28",
1444                         "TREB": "6",
1445                         "AST": "7",
1446                         "STL": "2",
1447                         "BLK": "1",
1448                         "FGA": "19",
1449                     },
1450                     {
1451                         "name": "North Forward",
1452                         "PTS": "18",
1453                         "TREB": "12",
1454                         "AST": "3",
1455                     },
1456                 ],
1457             },
1458             "vis": {
1459                 "name": "South",
1460                 "wins": "5",
1461                 "losses": "5",
1462                 "next_game": {
1463                     "opponent_name": "West",
1464                     "dayname": "Saturday",
1465                     "day": "10",
1466                     "month": "May",
1467                     "year": "2026",
1468                     "stadium": "Harbor Court",
1469                     "city": "Coast",
1470                     "is_home": "false",
1471                 },
1472                 "line_score": {"game": {"PTS": "84"}},
1473                 "box_score": [
1474                     {
1475                         "name": "South Wing",
1476                         "PTS": "24",
1477                         "TREB": "8",
1478                         "AST": "5",
1479                     },
1480                     {
1481                         "name": "South Center",
1482                         "PTS": "15",
1483                         "TREB": "14",
1484                         "BLK": "4",
1485                     },
1486                 ],
1487             },
1488         },
1489     }
1490
1491 evidence = event_capability_evidence(payload)
1492 performance_findings = [
1493     item.finding
1494     for item in evidence

```

```

1495         if item.evidence_type == "entity_performance"
1496     ]
1497     adjacent_context = next(
1498         item
1499         for item in evidence
1500         if item.evidence_type == "participant_record_context"
1501         and item.metrics.get("context_kind") == "adjacent_event"
1502     )
1503
1504     assert any(
1505         "North Guard recorded 28 points, 6 rebounds, 7 assists, "
1506         "2 steals, 1 block"
1507         in finding
1508         for finding in performance_findings
1509     )
1510     assert any("South Wing recorded 24 points" in finding for finding in performance_findings)
1511     assert any(
1512         "South Center recorded 15 points, 14 rebounds, 4 blocks" in finding
1513         for finding in performance_findings
1514     )
1515     assert "North next game at home against East" in adjacent_context.finding
1516     assert "South next game away against West" in adjacent_context.finding
1517
1518 def test_mixed_side_line_outcome_query_is_split_by_measure_family():
1519     def binding(
1520         binding_id: str,
1521         label: str,
1522         path: str,
1523     ) -> SemanticBinding:
1524         return SemanticBinding(
1525             binding_id=binding_id,
1526             table_name="event",
1527             label=label,
1528             role=SemanticRole.OUTCOME_MEASURE,
1529             level=SemanticLevel.ENTITY,
1530             path_pattern=path,
1531             description="Side-specific team line field.",
1532             confidence=0.9,
1533             evidence_basis="field name",
1534             analytical_function=AnalyticalFunction.OUTCOME,
1535         )
1536
1537     semantic_map = InputSemanticMap(
1538         input_shape=InputShape.EVENT_RECORD,
1539         record_description="One event with side-specific line fields.",
1540         confidence=0.9,
1541         bindings=[
1542             SemanticBinding(
1543                 binding_id="home_name",
1544                 table_name="event",
1545                 label="Home team",
1546                 role=SemanticRole.ENTITY_IDENTIFIER,
1547                 level=SemanticLevel.ENTITY,
1548                 path_pattern="home_name",
1549                 description="Home-side participant.",
1550                 confidence=0.9,
1551                 evidence_basis="field name",
1552             ),
1553             SemanticBinding(
1554                 binding_id="vis_name",
1555                 table_name="event",
1556                 label="Visiting team",
1557                 role=SemanticRole.ENTITY_IDENTIFIER,
1558                 level=SemanticLevel.ENTITY,
1559                 path_pattern="vis_name",
1560                 description="Visiting-side participant.",
1561                 confidence=0.9,
1562                 evidence_basis="field name",
1563             ),
1564             binding("home_runs", "Home team runs", "home_line.team_runs"),
1565             binding("vis_runs", "Visiting team runs", "vis_line.team_runs"),
1566             binding("home_hits", "Home team hits", "home_line.team_hits"),
1567             binding("vis_hits", "Visiting team hits", "vis_line.team_hits"),
1568             binding("home_errors", "Home team errors", "home_line.team_errors"),
1569             binding("vis_errors", "Visiting team errors", "vis_line.team_errors"),
1570         ],
1571     )
1572
1573     catalog = [
1574         StructuralField(
1575             table_name="event",
1576             path_pattern=path,
1577             value_types=["integer"],
1578             sample_values=["1"],
1579         )
1580         for path in [
1581             "home_line.team_runs",
1582             "vis_line.team_runs",
1583             "home_line.team_hits",
1584             "vis_line.team_hits",
1585             "home_line.team_errors",

```

```

1586         "vis_line.team_errors",
1587     ]
1588 ]
1589 tasks = [
1590     InvestigationTask(
1591         task_id="TASK_EVENT",
1592         question="Report the event outcome.",
1593         route=AnalysisRoute.DESRIPTIVE,
1594         priority=5,
1595         table_name="event",
1596         capability=EvidenceCapability.EVENT_OUTCOME,
1597         expected_evidence_types=["event_outcome"],
1598         required_evidence=["event_outcome"],
1599         claim_permissions=[ClaimPermission.DESRIPTIVE],
1600         answerability_note="Supported.",
1601     ),
1602     InvestigationTask(
1603         task_id="TASK_CONTRAST",
1604         question="Compare participant line components.",
1605         route=AnalysisRoute.DESRIPTIVE,
1606         priority=4,
1607         table_name="event",
1608         capability=EvidenceCapability.GROUP_COMPARISON,
1609         expected_evidence_types=["participant_comparison"],
1610         required_evidence=["participant_comparison"],
1611         claim_permissions=[ClaimPermission.COMPARATIVE],
1612         answerability_note="Supported.",
1613     ),
1614 ]
1615 bad_query = EvidenceQuery(
1616     query_id="BAD_MIXED_OUTCOME",
1617     task_id="TASK_EVENT",
1618     operation=EvidenceOperation.COMPARE,
1619     capability=EvidenceCapability.EVENT_OUTCOME,
1620     evidence_type="event_outcome",
1621     semantic_label="team outcome line",
1622     question="How many runs, hits, and errors did each team have?",
1623     semantic_level=SemanticLevel.PARTICIPANT,
1624     table_name="event",
1625     value_binding_ids=[
1626         "home_runs",
1627         "vis_runs",
1628         "home_hits",
1629         "vis_hits",
1630         "home_errors",
1631         "vis_errors",
1632     ],
1633     entity_binding_id="home_name",
1634 )
1635
1636 queries = normalise_event_evidence_queries(
1637     queries=[bad_query],
1638     semantic_map=semantic_map,
1639     tasks=tasks,
1640     available_capabilities={
1641         EvidenceCapability.EVENT_OUTCOME,
1642         EvidenceCapability.GROUP_COMPARISON,
1643     },
1644     request="Write a coherent event report.",
1645     structural_catalog=catalog,
1646 )
1647
1648 assert "BAD_MIXED_OUTCOME" not in {query.query_id for query in queries}
1649 outcome = next(
1650     query for query in queries if query.evidence_type == "event_outcome"
1651 )
1652 contrasts = [
1653     query
1654     for query in queries
1655     if query.evidence_type == "participant_comparison"
1656 ]
1657
1658 assert set(outcome.value_binding_ids) == {"home_runs", "vis_runs"}
1659 assert any(
1660     set(query.value_binding_ids) == {"home_hits", "vis_hits"}
1661     for query in contrasts
1662 )
1663
1664
1665 def test_event_sequence_highlights_score_changing_records():
1666     payload = {
1667         "home_name": "North",
1668         "away_name": "South",
1669         "timeline": [
1670             {
1671                 "period": "1",
1672                 "home_score": "0",
1673                 "away_score": "1",
1674                 "points": "1",
1675                 "actor": "Sol",
1676                 "event": "opening score",

```



```

1677         },
1678         {
1679             "period": "2",
1680             "home_score": "2",
1681             "away_score": "1",
1682             "points": "2",
1683             "actor": "Nia",
1684             "event": "late score",
1685         },
1686     ],
1687 }
1688
1689 evidence = event_capability_evidence(payload)
1690 sequence = next(
1691     item
1692     for item in evidence
1693     if item.evidence_type == "event_sequence"
1694     and item.strength_label == "event_sequence_highlight"
1695 )
1696
1697 assert sequence.recommended_use == RecommendedUse.MAIN_FINDING
1698 assert sequence.metrics["highlight_count"] == 2
1699 assert "Sol recorded opening score" in sequence.finding
1700 assert "Nia recorded late score" in sequence.finding
1701 assert "South led 1-0" in sequence.finding
1702 assert "North led 2-1" in sequence.finding
1703
1704
1705 def test_event_sequence_metrics_retain_all_score_changing_records():
1706     timeline = []
1707     north_score = 0
1708     south_score = 0
1709     for index in range(8):
1710         if index % 2 == 0:
1711             north_score += 1
1712             actor = f"North scorer {index}"
1713             event = "score"
1714         else:
1715             south_score += 2
1716             actor = f"South scorer {index}"
1717             event = "response"
1718         timeline.append(
1719             {
1720                 "period": str(index + 1),
1721                 "home_score": str(north_score),
1722                 "away_score": str(south_score),
1723                 "points": "1" if index % 2 == 0 else "2",
1724                 "actor": actor,
1725                 "event": event,
1726             }
1727         )
1728
1729 evidence = event_capability_evidence(
1730     {
1731         "home_name": "North",
1732         "away_name": "South",
1733         "timeline": timeline,
1734     }
1735 )
1736 sequence = next(
1737     item
1738     for item in evidence
1739     if item.evidence_type == "event_sequence"
1740     and item.strength_label == "event_sequence_highlight"
1741 )
1742
1743 assert sequence.metrics["highlight_count"] == 8
1744 assert len(sequence.metrics["highlights"]) == 8
1745 assert len(sequence.metrics["summary_highlights"]) == 6
1746 assert sequence.metrics["omitted_highlight_count"] == 2
1747 assert "2 later score-changing steps are omitted" in sequence.finding
1748
1749
1750 def test_semantic_entity_ranking_uses_row_local_affiliation_context():
1751     payload = {
1752         "home_name": "Reds",
1753         "box_score": [
1754             {"full_name": "Carlos Delgado", "team": "Mets", "rbi": "4"},
1755             {"full_name": "Joey Votto", "team": "Reds", "rbi": "3"},
1756         ],
1757     }
1758
1759     def binding(
1760         binding_id: str,
1761         label: str,
1762         role: SemanticRole,
1763         level: SemanticLevel,
1764         path: str,
1765         analytical_function: AnalyticalFunction | None = None,
1766     ) -> SemanticBinding:
1767         return SemanticBinding(

```

```

1768         binding_id=binding_id,
1769         table_name="game",
1770         label=label,
1771         role=role,
1772         level=level,
1773         path_pattern=path,
1774         description=f"Semantic interpretation of {label}.",
1775         confidence=1.0,
1776         evidence_basis="Fixture.",
1777         analytical_function=analytical_function,
1778     )
1779
1780     semantic = InputSemanticMap(
1781         input_shape=InputShape.EVENT_RECORD,
1782         record_description="One game with player rows.",
1783         bindings=[
1784             binding(
1785                 "B_HOME",
1786                 "Home team name",
1787                 SemanticRole.PARTICIPANT_IDENTIFIER,
1788                 SemanticLevel.PARTICIPANT,
1789                 "home_name",
1790             ),
1791             binding(
1792                 "B_PLAYER",
1793                 "Player full name",
1794                 SemanticRole.ENTITY_IDENTIFIER,
1795                 SemanticLevel.ENTITY,
1796                 "box_score. *.full_name",
1797             ),
1798             binding(
1799                 "B_TEAM",
1800                 "Player team",
1801                 SemanticRole.CONTEXT,
1802                 SemanticLevel.PARTICIPANT,
1803                 "box_score. *.team",
1804             ),
1805             binding(
1806                 "B_RBI",
1807                 "Runs batted in",
1808                 SemanticRole.PERFORMANCE_MEASURE,
1809                 SemanticLevel.ENTITY,
1810                 "box_score. *.rbi",
1811                 AnalyticalFunction.PERFORMANCE,
1812             ),
1813         ],
1814         recommended_report_genre=ReportGenre.EVENT_REPORT,
1815         report_rationale="Fixture.",
1816         confidence=1.0,
1817     )
1818
1819     row_local_query = EvidenceQuery(
1820         query_id="Q_RBI",
1821         task_id="TASK_EVENT",
1822         operation=EvidenceOperation.RANK,
1823         capability=EvidenceCapability.RANKING,
1824         evidence_type="entity_ranking",
1825         semantic_label="entity ranking for RBI",
1826         question="Which players had the most RBI?",
1827         table_name="game",
1828         semantic_level=SemanticLevel.ENTITY,
1829         value_binding_ids=["B_RBI"],
1830         entity_binding_id="B_PLAYER",
1831         group_binding_id="B_TEAM",
1832         recommended_use=RecommendedUse.MAIN_FINDING,
1833         user_relevance=1.0,
1834         salience=1.0,
1835     )
1836     evidence = semantic_query_evidence(
1837         table_name="game",
1838         payload=payload,
1839         semantic_map=semantic,
1840         queries=[row_local_query],
1841     )
1842     ranking = evidence[0].metrics["ranking"]
1843
1844     assert ranking[0]["entity"] == "Carlos Delgado"
1845     assert ranking[0]["group"] == "Mets"
1846     assert ranking[1]["group"] == "Reds"
1847
1848     global_group_query = row_local_query.model_copy(
1849         update={"group_binding_id": "B_HOME"}
1850     )
1851     global_group_evidence = semantic_query_evidence(
1852         table_name="game",
1853         payload=payload,
1854         semantic_map=semantic,
1855         queries=[global_group_query],
1856     )
1857     global_ranking = global_group_evidence[0].metrics["ranking"]
1858

```

```

1859     assert global_ranking[0]["group"] is None
1860     assert global_ranking[1]["group"] is None
1861
1862
1863 def test_segment_rankings_do_not_satisfy_leading_performance_slot():
1864     segment = evidence_item(
1865         evidence_id="EVID_SEGMENT",
1866         capability=EvidenceCapability.RANKING,
1867         evidence_type="entity_ranking",
1868         semantic_level=SemanticLevel.ENTITY,
1869         analytical_function=AnalyticalFunction.PERFORMANCE,
1870     ).model_copy(
1871         update={
1872             "metrics": {
1873                 "semantic_label": "period scoring ranking",
1874                 "question": "Which periods had the most scores?",
1875             },
1876             "source_paths": ["rounds.1.score"],
1877         }
1878     )
1879     actor = evidence_item(
1880         evidence_id="EVID_ACTOR",
1881         capability=EvidenceCapability.RANKING,
1882         evidence_type="entity_ranking",
1883         semantic_level=SemanticLevel.ENTITY,
1884         analytical_function=AnalyticalFunction.PERFORMANCE,
1885     ).model_copy(
1886         update={
1887             "metrics": {
1888                 "semantic_label": "actor points ranking",
1889                 "question": "Which actors had the most points?",
1890             },
1891             "source_paths": ["actors.0.points"],
1892         }
1893     )
1894     lookup = {
1895         segment.evidence_id: segment,
1896         actor.evidence_id: actor,
1897     }
1898
1899     assert (
1900         event_fact_slot(
1901             verified_fact(fact_id="FACT_SEGMENT", evidence=segment),
1902             lookup,
1903         )
1904         == "event_sequence"
1905     )
1906     assert (
1907         event_fact_slot(
1908             verified_fact(fact_id="FACT_ACTOR", evidence=actor),
1909             lookup,
1910         )
1911         == "leading_performance"
1912     )
1913
1914
1915 def test_event_quality_gate_rejects_absent_sequence_claim_when_sequence_supported():
1916     event = evidence_item(
1917         evidence_id="EVID_EVENT",
1918         capability=EvidenceCapability.EVENT_OUTCOME,
1919         evidence_type="event_outcome",
1920         semantic_level=SemanticLevel.EVENT,
1921     )
1922     sequence = evidence_item(
1923         evidence_id="EVID_SEQUENCE",
1924         capability=EvidenceCapability.EVENT_OUTCOME,
1925         evidence_type="event_sequence",
1926         semantic_level=SemanticLevel.EVENT,
1927     )
1928     output = WriterOutput(
1929         title="Event report",
1930         markdown=(
1931             "# Event report\n\n"
1932             "North defeated South 9-4.\n"
1933             "The supplied data does not capture event dynamics or scoring progression.\n"
1934         ),
1935         sentence_support=[
1936             SentenceSupport(
1937                 sentence_id="SENT_0001",
1938                 sentence_text="North defeated South 9-4.",
1939                 fact_ids=["FACT_EVENT"],
1940                 evidence_ids=[event.evidence_id],
1941                 support_type=SupportType.DIRECT,
1942             )
1943         ],
1944     )
1945
1946     assessment = assess_genre_quality(
1947         output,
1948         event_report_specification(),
1949         EvidenceLedger(fingerprint="fixture", items=[event, sequence]),

```

```

1950     )
1951
1952     assert assessment.status == QualityStatus.REVISE
1953     assert any("event-sequence evidence" in finding for finding in assessment.findings)
1954
1955
1956 def test_event_sequence_slot_recovers_writer_ready_fact_when_llm_omits_it():
1957     event = evidence_item(
1958         evidence_id="EVID_EVENT",
1959         capability=EvidenceCapability.EVENT_OUTCOME,
1960         evidence_type="event_outcome",
1961         semantic_level=SemanticLevel.EVENT,
1962     )
1963     sequence = evidence_item(
1964         evidence_id="EVID_SEQUENCE",
1965         capability=EvidenceCapability.EVENT_OUTCOME,
1966         evidence_type="event_sequence",
1967         semantic_level=SemanticLevel.EVENT,
1968     ).model_copy(
1969         update={
1970             "finding": (
1971                 "The recorded event sequence includes two score-changing "
1972                 "steps from the supplied play records."
1973             ),
1974             "strength_label": "event_sequence_highlight",
1975         }
1976     )
1977     ledger = FactLedger(
1978         writer_ready_facts=[
1979             verified_fact(fact_id="FACT_EVENT", evidence=event)
1980         ]
1981     )
1982
1983     recovered = augment_fact_ledger_for_report_coverage(
1984         fact_ledger=ledger,
1985         evidence=EvidenceLedger(
1986             fingerprint="fixture",
1987             items=[event, sequence],
1988         ),
1989         required_components=[],
1990         required_content_slots=["event_result", "event_sequence"],
1991         settings=Settings(),
1992     )
1993
1994     sequence_facts = [
1995         fact
1996         for fact in recovered.writer_ready_facts
1997         if sequence.evidence_id in fact.evidence_ids
1998     ]
1999
2000     assert len(sequence_facts) == 1
2001     assert sequence_facts[0].fact_summary == sequence.finding
2002     assert (
2003         sequence_facts[0].verification_method
2004         == "deterministic_evidence_recovery"
2005     )
2006
2007
2008 def test_required_event_contrast_recovers_when_existing_fact_is_omitted():
2009     event = evidence_item(
2010         evidence_id="EVID_EVENT",
2011         capability=EvidenceCapability.EVENT_OUTCOME,
2012         evidence_type="event_outcome",
2013         semantic_level=SemanticLevel.EVENT,
2014     )
2015     contrast = evidence_item(
2016         evidence_id="EVID_CONTRAST",
2017         capability=EvidenceCapability.GROUP_COMPARISON,
2018         evidence_type="participant_comparison",
2019         semantic_level=SemanticLevel.EVENT,
2020         analytical_function=AnalyticalFunction.OUTCOME_COMPONENT,
2021     ).model_copy(
2022         update={
2023             "query_id": "QUERY_CONTRAST",
2024             "finding": (
2025                 "Validated semantic query result for participant contrast."
2026             ),
2027             "metrics": {
2028                 "records": [
2029                     {
2030                         "entity": "North",
2031                         "value": 12,
2032                         "measure": "goals",
2033                     },
2034                     {
2035                         "entity": "South",
2036                         "value": 9,
2037                         "measure": "goals",
2038                     },
2039                 ],
2040                 "difference": 3,

```

```

2041     },
2042     }
2043 )
2044 omitted_contrast_fact = verified_fact(
2045     fact_id="FACT_BAD_CONTRAST",
2046     evidence=contrast,
2047 ).model_copy(
2048     update={
2049         "fact_summary": (
2050             "Semantic query result is incomplete in the provided ledger."
2051         ),
2052         "recommended_use": RecommendedUse.OMIT_UNLESS_REQUESTED,
2053     }
2054 )
2055 ledger = FactLedger(
2056     writer_ready_facts=[
2057         verified_fact(fact_id="FACT_EVENT", evidence=event),
2058         omitted_contrast_fact,
2059     ]
2060 )
2061
2062 recovered = augment_fact_ledger_for_report_coverage(
2063     fact_ledger=ledger,
2064     evidence=EvidenceLedger(
2065         fingerprint="fixture",
2066         items=[event, contrast],
2067     ),
2068     required_components=[],
2069     required_content_slots=["event_result", "main_contrast"],
2070     settings=Settings(),
2071 )
2072
2073 recovered_contrasts = [
2074     fact
2075     for fact in recovered.writer_ready_facts
2076     if (
2077         contrast.evidence_id in fact.evidence_ids
2078         and fact.fact_id != omitted_contrast_fact.fact_id
2079     )
2080 ]
2081
2082 assert len(recovered_contrasts) == 1
2083 assert "North recorded 12" in recovered_contrasts[0].fact_summary
2084 assert recovered_contrasts[0].recommended_use == contrast.recommended_use
2085
2086 def test_event_scope_caveat_does_not_trigger_causal_overclaim():
2087     event = evidence_item(
2088         evidence_id="EVID_EVENT",
2089         capability=EvidenceCapability.EVENT_OUTCOME,
2090         evidence_type="event_outcome",
2091         semantic_level=SemanticLevel.EVENT,
2092     )
2093
2094     sentence = (
2095         "This report describes only the supplied game; observed performances "
2096         "do not explain why the result occurred."
2097     )
2098
2099     output = WriterOutput(
2100         title="Event report",
2101         markdown=sentence + "\n",
2102         sentence_support=[
2103             SentenceSupport(
2104                 sentence_id="SENT_0001",
2105                 sentence_text=sentence,
2106                 fact_ids=[],
2107                 evidence_ids=[],
2108                 support_type=SupportType.NON_FACTUAL,
2109             )
2110         ],
2111     )
2112
2113     audit = deterministic_audit(
2114         writer_output=output,
2115         fact_ledger=FactLedger(writer_ready_facts=[]),
2116         evidence=EvidenceLedger(fingerprint="fixture", items=[event]),
2117         mode=AuditMode.INTERNAL,
2118         external_sources=[],
2119         revision_round=0,
2120         report_specification=event_report_specification(),
2121         settings=Settings(),
2122     )
2123
2124     assert not any(
2125         annotation.subtype == "causal_overclaim"
2126         for annotation in audit.annotations
2127     )
2128
2129 def test_event_scope_limitation_accepts_supplied_record_and_causality_caveats():
2130     output = WriterOutput(
2131         title="Event report",

```

```

2132         markdown=(
2133             "Sequence highlights describe recorded score-changing steps only "
2134             "and do not establish causality or momentum.\n"
2135             "The result is limited to values present in the supplied record.\n"
2136         ),
2137         sentence_support=[],
2138     )
2139
2140     assert event_scope_limitation_present(output)
2141
2142
2143 def test_fact_summary_numbers_support_recovered_sequence_sentence():
2144     sequence = evidence_item(
2145         evidence_id="EVID_SEQUENCE",
2146         capability=EvidenceCapability.EVENT_OUTCOME,
2147         evidence_type="event_sequence",
2148         semantic_level=SemanticLevel.EVENT,
2149     ).model_copy(
2150         update={
2151             "metrics": {
2152                 "highlights": [
2153                     {
2154                         "event_text": (
2155                             "inning 9, segment top: Ramon Vazquez "
2156                             "recorded Home Run"
2157                         ),
2158                         "score_phrase": "Houston led 4-3",
2159                         "left_value": 4,
2160                         "right_value": 3,
2161                     }
2162                 ]
2163             }
2164         }
2165     )
2166     fact = verified_fact(
2167         fact_id="FACT_SEQUENCE",
2168         evidence=sequence,
2169     ).model_copy(
2170         update={
2171             "fact_summary": (
2172                 "The recorded score-changing sequence includes: inning 9, "
2173                 "segment top: Ramon Vazquez recorded Home Run, after which "
2174                 "Houston led 4-3."
2175             )
2176         }
2177     )
2178
2179     support_numbers = fact_support_numbers(
2180         fact,
2181         EvidenceLedger(fingerprint="fixture", items=[sequence]),
2182     )
2183
2184     assert numbers_supported(fact.fact_summary, support_numbers)
2185
2186
2187 def test_event_content_requirements_ignore_omit_unless_requested_candidates():
2188     result = evidence_item(
2189         evidence_id="EVID_RESULT",
2190         capability=EvidenceCapability.EVENT_OUTCOME,
2191         evidence_type="event_outcome",
2192         semantic_level=SemanticLevel.EVENT,
2193     )
2194     contrast = evidence_item(
2195         evidence_id="EVID_CONTRAST",
2196         capability=EvidenceCapability.GROUP_COMPARISON,
2197         evidence_type="participant_comparison",
2198         semantic_level=SemanticLevel.EVENT,
2199         analytical_function=AnalyticalFunction.OUTCOME_COMPONENT,
2200     )
2201     omitted = verified_fact(
2202         fact_id="FACT_OMITTED_CONTRAST",
2203         evidence=contrast,
2204     ).model_copy(
2205         update={"recommended_use": RecommendedUse.OMIT_UNLESS_REQUESTED}
2206     )
2207     usable = verified_fact(
2208         fact_id="FACT_USABLE_CONTRAST",
2209         evidence=contrast,
2210     )
2211     requirements = build_writer_content_requirements(
2212         report_specification=event_report_specification().model_copy(
2213             update={
2214                 "required_content_slots": [
2215                     "event_result",
2216                     "main_contrast",
2217                 ]
2218             }
2219         ),
2220         fact_ledger=FactLedger(
2221             writer_ready_facts=[
2222                 verified_fact(fact_id="FACT_RESULT", evidence=result),

```

```

2223         omitted,
2224         usable,
2225     ]
2226 ),
2227 evidence=EvidenceLedger(
2228     fingerprint="fixture",
2229     items=[result, contrast],
2230 ),
2231 insight_ledger=InsightLedger(),
2232 settings=Settings(),
2233 )
2234 contrast_unit = next(
2235     unit
2236     for unit in requirements["units"]
2237     if unit["unit_id"] == "main_contrast"
2238 )
2239
2240 assert contrast_unit["candidate_fact_ids"] == [usable.fact_id]
2241 assert contrast_unit["minimum_items"] == 1
2242
2243 def test_writer_validation_counts_title_fact_ids_as_selected():
2244     title_evidence = evidence_item(
2245         evidence_id="EVID_TITLE",
2246         capability=EvidenceCapability.EVENT_OUTCOME,
2247         evidence_type="event_outcome",
2248         semantic_level=SemanticLevel.EVENT,
2249     )
2250     sentence_evidence = evidence_item(
2251         evidence_id="EVID_SENTENCE",
2252         capability=EvidenceCapability.EVENT_OUTCOME,
2253         evidence_type="event_outcome",
2254         semantic_level=SemanticLevel.EVENT,
2255     )
2256     title_fact = verified_fact(
2257         fact_id="FACT_TITLE",
2258         evidence=title_evidence,
2259     )
2260     sentence_fact = verified_fact(
2261         fact_id="FACT_SENTENCE",
2262         evidence=sentence_evidence,
2263     )
2264     output = WriterOutput(
2265         title="Supported event report",
2266         title_fact_ids=[title_fact.fact_id],
2267         markdown="North defeated South 12-9.\n",
2268         sentence_support=[
2269             SentenceSupport(
2270                 sentence_id="SENT_0001",
2271                 sentence_text="North defeated South 12-9.",
2272                 fact_ids=[sentence_fact.fact_id],
2273                 evidence_ids=[sentence_evidence.evidence_id],
2274                 support_type=SupportType.DIRECT,
2275             )
2276         ],
2277         selected_fact_ids=[
2278             title_fact.fact_id,
2279             sentence_fact.fact_id,
2280         ],
2281     )
2282     errors = validate_writer_output(
2283         output,
2284         FactLedger(
2285             writer_ready_facts=[
2286                 title_fact,
2287                 sentence_fact,
2288             ]
2289         ),
2290         allow_hypotheses_in_report=False,
2291     )
2292     assert "selected_fact_ids must match" not in "\n".join(errors)
2293
2294 def test_event_score_tie_claim_requires_sequence_score_state_support():
2295     performance = evidence_item(
2296         evidence_id="EVID_PERFORMANCE",
2297         capability=EvidenceCapability.RANKING,
2298         evidence_type="entity_ranking",
2299         semantic_level=SemanticLevel.ENTITY,
2300     )
2301     sequence = evidence_item(
2302         evidence_id="EVID_SEQUENCE",
2303         capability=EvidenceCapability.EVENT_OUTCOME,
2304         evidence_type="event_sequence",
2305         semantic_level=SemanticLevel.EVENT,
2306     ).model_copy(
2307         update={
2308             "metrics": {
2309                 "highlights": [

```

```

2314         {
2315             "event_text": (
2316                 "inning 9, segment top: Ramon Vazquez "
2317                 "recorded Home Run"
2318             ),
2319             "score_phrase": "Houston led 4-3",
2320             "left_value": 4,
2321             "right_value": 3,
2322         }
2323     ]
2324 }
2325
2326 )
2327 fact = verified_fact(fact_id="FACT_PERFORMANCE", evidence=performance)
2328 sentence = "Ramon Vazquez hit a game-tying home run."
2329 output = WriterOutput(
2330     title="Event report",
2331     markdown=sentence + "\n",
2332     sentence_support=[
2333         SentenceSupport(
2334             sentence_id="SENT_0001",
2335             sentence_text=sentence,
2336             fact_ids=[fact.fact_id],
2337             evidence_ids=[performance.evidence_id],
2338             support_type=SupportType.DIRECT,
2339         )
2340     ],
2341 )
2342
2343 audit = deterministic_audit(
2344     writer_output=output,
2345     fact_ledger=FactLedger(writer_ready_facts=[fact]),
2346     evidence=EvidenceLedger(
2347         fingerprint="fixture",
2348         items=[performance, sequence],
2349     ),
2350     mode=AuditMode.INTERNAL,
2351     external_sources=[],
2352     revision_round=0,
2353     report_specification=event_report_specification(),
2354     settings=Settings(),
2355 )
2356
2357 assert any(
2358     annotation.subtype == "unsupported_event_score_state"
2359     for annotation in audit.annotations
2360 )
2361
2362
2363 def test_event_score_tie_claim_accepts_matching_sequence_highlight():
2364     sequence = evidence_item(
2365         evidence_id="EVID_SEQUENCE",
2366         capability=EvidenceCapability.EVENT_OUTCOME,
2367         evidence_type="event_sequence",
2368         semantic_level=SemanticLevel.EVENT,
2369     ).model_copy(
2370         update={
2371             "metrics": {
2372                 "highlights": [
2373                     {
2374                         "event_text": (
2375                             "inning 2, segment bottom: Ty Wigginton "
2376                             "recorded Single"
2377                         ),
2378                         "score_phrase": "the score was level at 1-1",
2379                         "left_value": 1,
2380                         "right_value": 1,
2381                     }
2382                 ]
2383             }
2384         )
2385     )
2386 fact = verified_fact(fact_id="FACT_SEQUENCE", evidence=sequence)
2387 sentence = "Ty Wigginton singled to tie the game."
2388 output = WriterOutput(
2389     title="Event report",
2390     markdown=sentence + "\n",
2391     sentence_support=[
2392         SentenceSupport(
2393             sentence_id="SENT_0001",
2394             sentence_text=sentence,
2395             fact_ids=[fact.fact_id],
2396             evidence_ids=[sequence.evidence_id],
2397             support_type=SupportType.DIRECT,
2398         )
2399     ],
2400 )
2401
2402 audit = deterministic_audit(
2403     writer_output=output,
2404     fact_ledger=FactLedger(writer_ready_facts=[fact]),

```



```

2405     evidence=EvidenceLedger(fingerprint="fixture", items=[sequence]),
2406     mode=AuditMode.INTERNAL,
2407     external_sources=[],
2408     revision_round=0,
2409     report_specification=event_report_specification(),
2410     settings=Settings(),
2411 )
2412
2413 assert not any(
2414     annotation.subtype == "unsupported_event_score_state"
2415     for annotation in audit.annotations
2416 )
2417
2418
2419 def test_event_score_tie_guardrail_allows_negated_tie_wording():
2420     sequence = evidence_item(
2421         evidence_id="EVID_SEQUENCE",
2422         capability=EvidenceCapability.EVENT_OUTCOME,
2423         evidence_type="event_sequence",
2424         semantic_level=SemanticLevel.EVENT,
2425     ).model_copy(
2426         update={
2427             "metrics": {
2428                 "highlights": [
2429                     {
2430                         "event_text": (
2431                             "inning 8, segment bottom: Chris Dickerson "
2432                             "recorded Sac Fly"
2433                         ),
2434                         "score_phrase": "NY Mets led 9-7",
2435                         "left_value": 7,
2436                         "right_value": 9,
2437                     }
2438                 ]
2439             }
2440         )
2441     )
2442     fact = verified_fact(fact_id="FACT_SEQUENCE", evidence=sequence)
2443     sentence = (
2444         "The Reds scored late on a Chris Dickerson sacrifice fly, but "
2445         "could not tie the game."
2446     )
2447     output = WriterOutput(
2448         title="Event report",
2449         markdown=sentence + "\n",
2450         sentence_support=[
2451             SentenceSupport(
2452                 sentence_id="SENT_0001",
2453                 sentence_text=sentence,
2454                 fact_ids=[fact.fact_id],
2455                 evidence_ids=[sequence.evidence_id],
2456                 support_type=SupportType.DIRECT,
2457             )
2458         ],
2459     )
2460
2461     audit = deterministic_audit(
2462         writer_output=output,
2463         fact_ledger=FactLedger(writer_ready_facts=[fact]),
2464         evidence=EvidenceLedger(fingerprint="fixture", items=[sequence]),
2465         mode=AuditMode.INTERNAL,
2466         external_sources=[],
2467         revision_round=0,
2468         report_specification=event_report_specification(),
2469         settings=Settings(),
2470     )
2471
2472     assert not any(
2473         annotation.subtype == "unsupported_event_score_state"
2474         for annotation in audit.annotations
2475     )
2476
2477
2478 def test_event_query_priorities_reject_participation_substitution():
2479     base = semantic_map()
2480     alpha = next(
2481         binding
2482         for binding in base.bindings
2483         if binding.binding_id == "B_ALPHA"
2484     )
2485     enriched = base.model_copy(
2486         update={
2487             "bindings": [
2488                 *base.bindings,
2489                 alpha.model_copy(
2490                     update={
2491                         "binding_id": "B_BETA",
2492                         "label": "beta performance",
2493                         "path_pattern": "sides.*members.*beta",
2494                     }
2495                 ),
2496             ]
2497         },

```

```

2496         alpha.model_copy(
2497             update={
2498                 "binding_id": "B_GAMMA",
2499                 "label": "gamma performance",
2500                 "path_pattern": "sides.*members.*gamma",
2501             }
2502         ),
2503         alpha.model_copy(
2504             update={
2505                 "binding_id": "B_DURATION",
2506                 "label": "participation duration",
2507                 "path_pattern": (
2508                     "sides.*members.*duration"
2509                 ),
2510                 "analytical_function": (
2511                     AnalyticalFunction.PARTICIPATION
2512                 ),
2513             }
2514         ),
2515     ]
2516 }
2517 )
2518 duration_query = semantic_queries()[3].model_copy(
2519     update={
2520         "query_id": "QUERY_DURATION",
2521         "semantic_label": "participation duration ranking",
2522         "question": (
2523             "Which entities recorded the greatest duration?"
2524         ),
2525         "value_binding_ids": ["B_DURATION"],
2526     }
2527 )
2528
2529 errors = validate_event_query_priorities(
2530     [semantic_queries()[3], duration_query],
2531     enriched,
2532     "Understand the event and report its strongest findings.",
2533 )
2534
2535 assert any(
2536     "distinct substantive entity-performance" in error
2537     for error in errors
2538 )
2539
2540
2541 def test_writer_draft_accepts_broad_support_id_sequences():
2542     draft = WriterAgentDraft(
2543         title="Supported title",
2544         title_fact_ids=[f"FACT_{index:04d}" for index in range(25)],
2545     )
2546
2547     assert len(draft.title_fact_ids) == 25
2548
2549
2550 def test_numeric_string_evidence_supports_rendered_dates_and_identifiers():
2551     support_numbers = flatten_numbers(
2552         {
2553             "values": ["4885", "2017", "11", "09"],
2554             "non_numeric": "Capital One Arena",
2555         }
2556     )
2557
2558     assert support_numbers == [4885.0, 2017.0, 11.0, 9.0]
2559     assert numbers_supported(
2560         "Game 4885 took place on 2017-11-09.",
2561         support_numbers,
2562     )
2563     date_support_numbers = [
2564         number
2565         for _, number in extract_number_tokens(
2566             "Game date is 04_08_09."
2567         )
2568     ]
2569
2570     assert numbers_supported(
2571         "The event was played on April 8, 2009.",
2572         date_support_numbers,
2573     )
2574
2575
2576 def test_number_extraction_ignores_digits_embedded_in_entity_names():
2577     sentence = (
2578         "Philadelphia 76ers defeated Memphis Grizzlies 103-95, "
2579         "a margin of 8."
2580     )
2581
2582     assert extract_number_tokens(sentence) == [
2583         ("103", 103.0),
2584         ("95", 95.0),
2585         ("8", 8.0),
2586     ]

```

```

2587     assert numbers_supported(
2588         sentence,
2589         [103.0, 95.0, 8.0],
2590     )
2591
2592
2593 def test_sentence_splitter_keeps_player_initials_with_name():
2594     markdown = (
2595         "J.J. Redick recorded 24 points for Philadelphia 76ers. "
2596         "Jimmy Butler recorded 21."
2597     )
2598
2599     assert split_markdown_sentences(markdown) == [
2600         "J.J. Redick recorded 24 points for Philadelphia 76ers.",
2601         "Jimmy Butler recorded 21.",
2602     ]
2603
2604
2605 def test_sentence_splitter_keeps_versus_abbreviation_with_names():
2606     markdown = (
2607         "Starrcade featured Big Van Vader vs. Ric Flair. "
2608         "The event took place in 1993."
2609     )
2610
2611     assert split_markdown_sentences(markdown) == [
2612         "Starrcade featured Big Van Vader vs. Ric Flair.",
2613         "The event took place in 1993.",
2614     ]
2615
2616
2617 def event_structure() -> InputStructureProfile:
2618     return InputStructureProfile(
2619         shape=InputShape.EVENT_RECORD,
2620         representation_status=InputRepresentationStatus.VALID,
2621         row_semantics="one event",
2622         confidence=0.98,
2623     )
2624
2625
2626 def event_report_specification() -> ReportSpecification:
2627     return ReportSpecification(
2628         report_purpose="Describe the event.",
2629         genre=ReportGenre.EVENT_REPORT,
2630         communication_goal="Communicate the verified event evidence.",
2631         target_length_words=250,
2632         maximum_main_findings=5,
2633         required_components=[],
2634         required_content_slots=["event_result"],
2635         prohibited_claim_types=["unsupported_causality"],
2636         selection_source=ReportSelectionSource.STRUCTURED_INFERENCE,
2637         prioritisation_rule="Prefer salient event evidence.",
2638     )
2639
2640
2641 def evidence_item(
2642     *,
2643     evidence_id: str,
2644     capability: EvidenceCapability,
2645     evidence_type: str,
2646     semantic_level: SemanticLevel,
2647     analytical_function: AnalyticalFunction | None = None,
2648 ) -> EvidenceItem:
2649     return EvidenceItem(
2650         evidence_id=evidence_id,
2651         route=AnalysisRoute.DESCRPTIVE,
2652         task_ids=["TASK_EVENT"],
2653         capability=capability,
2654         evidence_type=evidence_type,
2655         semantic_level=semantic_level,
2656         analytical_function=analytical_function,
2657         finding="Supported evidence item.",
2658         metrics={"value": 12},
2659         source_tables=["contest"],
2660         method="Validated test evidence.",
2661         practical_interpretation="Direct descriptive evidence.",
2662         strength_label="direct",
2663         claim_permissions=[ClaimPermission.DESCRPTIVE],
2664         factual_confidence=1.0,
2665         methodological_strength=1.0,
2666         user_relevance=1.0,
2667         salience=1.0,
2668         recommended_use=RecommendedUse.MAIN_FINDING,
2669     )
2670
2671
2672 def verified_fact(
2673     *,
2674     fact_id: str,
2675     evidence: EvidenceItem,
2676 ) -> VerifiedFact:
2677     return VerifiedFact(

```

```

2678         fact_id=fact_id,
2679         source_candidate_id=f"CAN_{fact_id}",
2680         fact_summary="A directly supported fact.",
2681         evidence_ids=[evidence.evidence_id],
2682         source_capabilities=[evidence.capability],
2683         structured_values={evidence.evidence_id: evidence.metrics},
2684         entities=["contest"],
2685         claim_permissions=[ClaimPermission.DESRIPTIVE],
2686         factual_confidence=1.0,
2687         methodological_strength=1.0,
2688         user_relevance=1.0,
2689         salience=1.0,
2690         recommended_use=RecommendedUse.MAIN_FINDING,
2691     )
2692
2693
2694 def test_event_narrative_plan_orders_reference_recap_slots():
2695     result = evidence_item(
2696         evidence_id="EVD_RESULT",
2697         capability=EvidenceCapability.EVENT_OUTCOME,
2698         evidence_type="event_outcome",
2699         semantic_level=SemanticLevel.EVENT,
2700     )
2701     sequence = evidence_item(
2702         evidence_id="EVD_SEQUENCE",
2703         capability=EvidenceCapability.EVENT_OUTCOME,
2704         evidence_type="event_sequence",
2705         semantic_level=SemanticLevel.EVENT,
2706     )
2707     ranking = evidence_item(
2708         evidence_id="EVD_RANKING",
2709         capability=EvidenceCapability.RANKING,
2710         evidence_type="entity_ranking",
2711         semantic_level=SemanticLevel.ENTITY,
2712     )
2713     contrast = evidence_item(
2714         evidence_id="EVD_CONTRAST",
2715         capability=EvidenceCapability.GROUP_COMPARISON,
2716         evidence_type="participant_comparison",
2717         semantic_level=SemanticLevel.EVENT,
2718     )
2719     spec = event_report_specification().model_copy(
2720         update={
2721             "focus_scope": "reference_recap",
2722             "realisation_policy": RealisationPolicy.EVENT_RECAP_STYLE,
2723             "required_content_slots": [
2724                 "event_result",
2725                 "event_sequence",
2726                 "leading_performance",
2727                 "main_contrast",
2728             ],
2729         }
2730     )
2731     facts = [
2732         verified_fact(fact_id="FACT_RESULT", evidence=result),
2733         verified_fact(fact_id="FACT_SEQUENCE", evidence=sequence),
2734         verified_fact(fact_id="FACT_RANKING", evidence=ranking),
2735         verified_fact(fact_id="FACT_CONTRAST", evidence=contrast),
2736     ]
2737     pack = WriterEvidencePack(
2738         user_request="Write a recap.",
2739         report_specification=spec,
2740         dataset_understanding=DataUnderstanding(
2741             profile_fingerprint="fixture",
2742             dataset_summary="Synthetic event.",
2743             tables=[],
2744         ),
2745         input_structure=event_structure(),
2746         available_capabilities=[
2747             EvidenceCapability.EVENT_OUTCOME,
2748             EvidenceCapability.RANKING,
2749             EvidenceCapability.GROUP_COMPARISON,
2750         ],
2751         priority_facts=facts,
2752         supporting_facts=[],
2753         limitation_facts=[],
2754         evidence_ledger=EvidenceLedger(
2755             fingerprint="fixture",
2756             items=[result, sequence, ranking, contrast],
2757         ),
2758     )
2759
2760     plan = build_event_narrative_plan(
2761         pack,
2762         {
2763             "realisation_policy": RealisationPolicy.EVENT_RECAP_STYLE.value,
2764             "reference_recap_style": True,
2765             "narrative_requirements": {
2766                 "minimum_scope_limitations": 0,
2767             },
2768         },

```

```

2769 )
2770
2771 assert plan.applies
2772 assert plan.style == "reference_recap"
2773 assert not plan.allow_headings
2774 assert [
2775     slot.slot
2776     for slot in plan.slots[:4]
2777 ] == [
2778     NarrativeSlot.OPENING_RESULT,
2779     NarrativeSlot.EVENT_SEQUENCE,
2780     NarrativeSlot.LEADING_PERFORMANCES,
2781     NarrativeSlot.PARTICIPANT_CONTRASTS,
2782 ]
2783 assert NarrativeSlot.CLOSING_SCOPE not in {
2784     slot.slot
2785     for slot in plan.slots
2786 }
2787
2788
2789 def test_event_narrative_plan_distinguishes_generic_event_recap_style():
2790     result = evidence_item(
2791         evidence_id="EVD_RESULT",
2792         capability=EvidenceCapability.EVENT_OUTCOME,
2793         evidence_type="event_outcome",
2794         semantic_level=SemanticLevel.EVENT,
2795     )
2796     contrast = evidence_item(
2797         evidence_id="EVD_CONTRAST",
2798         capability=EvidenceCapability.GROUP_COMPARISON,
2799         evidence_type="participant_comparison",
2800         semantic_level=SemanticLevel.EVENT,
2801     )
2802     spec = event_report_specification().model_copy(
2803         update={
2804             "focus_scope": "event_recap",
2805             "realisation_policy": RealisationPolicy.EVENT_RECAP_STYLE,
2806             "required_content_slots": [
2807                 "event_result",
2808                 "main_contrast",
2809             ],
2810         }
2811     )
2812     facts = [
2813         verified_fact(fact_id="FACT_RESULT", evidence=result),
2814         verified_fact(fact_id="FACT_CONTRAST", evidence=contrast),
2815     ]
2816     pack = WriterEvidencePack(
2817         user_request="Understand the data and report its strongest findings.",
2818         report_specification=spec,
2819         dataset_understanding=DataUnderstanding(
2820             profile_fingerprint="fixture",
2821             dataset_summary="Synthetic event.",
2822             tables=[],
2823         ),
2824         input_structure=event_structure(),
2825         available_capabilities=[
2826             EvidenceCapability.EVENT_OUTCOME,
2827             EvidenceCapability.GROUP_COMPARISON,
2828         ],
2829         priority_facts=facts,
2830         supporting_facts=[],
2831         limitation_facts=[],
2832         evidence_ledger=EvidenceLedger(
2833             fingerprint="fixture",
2834             items=[result, contrast],
2835         ),
2836     )
2837
2838     plan = build_event_narrative_plan(
2839         pack,
2840         {
2841             "realisation_policy": RealisationPolicy.EVENT_RECAP_STYLE.value,
2842             "event_recap_style": True,
2843             "reference_recap_style": False,
2844             "narrative_requirements": {
2845                 "minimum_scope_limitations": 1,
2846             },
2847         },
2848     )
2849
2850 assert plan.applies
2851 assert plan.style == "event_recap"
2852 assert plan.allow_headings
2853 assert NarrativeSlot.CLOSING_SCOPE in {
2854     slot.slot
2855     for slot in plan.slots
2856 }
2857
2858
2859 def test_reference_recap_quality_gate_does_not_require_visible_limitation():

```

```

2860     result = evidence_item(
2861         evidence_id="EVD_RESULT",
2862         capability=EvidenceCapability.EVENT_OUTCOME,
2863         evidence_type="event_outcome",
2864         semantic_level=SemanticLevel.EVENT,
2865     )
2866     ranking = evidence_item(
2867         evidence_id="EVD_RANKING",
2868         capability=EvidenceCapability.RANKING,
2869         evidence_type="entity_ranking",
2870         semantic_level=SemanticLevel.ENTITY,
2871     )
2872     contrast = evidence_item(
2873         evidence_id="EVD_CONTRAST",
2874         capability=EvidenceCapability.GROUP_COMPARISON,
2875         evidence_type="participant_comparison",
2876         semantic_level=SemanticLevel.EVENT,
2877     )
2878     output = WriterOutput(
2879         title="Event recap",
2880         markdown=(
2881             "Alpha defeated Beta 90-80 while Nia led all players with 20 "
2882             "points. Alpha recorded more field goals than Beta."
2883         ),
2884         sentence_support=[
2885             SentenceSupport(
2886                 sentence_id="SENT_0001",
2887                 sentence_text=(
2888                     "Alpha defeated Beta 90-80 while Nia led all players "
2889                     "with 20 points."
2890                 ),
2891                 evidence_ids=[
2892                     "EVD_RESULT",
2893                     "EVD_RANKING",
2894                 ],
2895                 support_type=SupportType.MULTI_FACT_SYNTHESIS,
2896             ),
2897             SentenceSupport(
2898                 sentence_id="SENT_0002",
2899                 sentence_text="Alpha recorded more field goals than Beta.",
2900                 evidence_ids=[
2901                     "EVD_CONTRAST",
2902                     "EVD_RESULT",
2903                 ],
2904                 support_type=SupportType.MULTI_FACT_SYNTHESIS,
2905             ),
2906         ],
2907     )
2908     spec = event_report_specification().model_copy(
2909         update={
2910             "focus_scope": "reference_recap",
2911             "realisation_policy": RealisationPolicy.EVENT_RECAP_STYLE,
2912             "required_content_slots": [
2913                 "event_result",
2914                 "leading_performance",
2915                 "main_contrast",
2916             ],
2917         }
2918     )
2919     assessment = assess_genre_quality(
2920         output,
2921         spec,
2922         EvidenceLedger(
2923             fingerprint="fixture",
2924             items=[result, ranking, contrast],
2925         ),
2926     )
2927
2928     assert assessment.status == QualityStatus.PASS
2929     assert "scope_limitations" not in assessment.supported_slots
2930
2931
2932 def test_structural_catalog_uses_wildcards_and_excludes_held_out_reference(
2933     tmp_path,
2934 ):
2935     sentinel = "SECRET HELD OUT REFERENCE " * 50
2936     path = tmp_path / "contest.json"
2937     path.write_text(
2938         json.dumps({**renamed_event(), "reference": sentinel}),
2939         encoding="utf-8",
2940     )
2941     bundle = load_data(
2942         [path],
2943         evaluation_field_policy=EvaluationFieldPolicy(
2944             operational_input_paths=["occasion", "sides"],
2945             held_out_reference_paths=["reference"],
2946         ),
2947     )
2948
2949     catalog = build_structural_catalog(bundle.structured_inputs)

```

```

2951     serialized = json.dumps([field.model_dump(mode="json") for field in catalog])
2952     paths = {field.path_pattern for field in catalog}
2953
2954     assert "sides.*.members.*.alpha" in paths
2955     assert "sides.*.tally" in paths
2956     assert sentinel.strip() not in serialized
2957     assert "reference" not in serialized
2958
2959
2960 def test_generic_semantic_queries_execute_renamed_event_without_authored_claims():
2961     catalog = build_structural_catalog({"contest": renamed_event()})
2962     queries = semantic_queries()
2963     available = {
2964         EvidenceCapability.DATASET_PROFILE,
2965         EvidenceCapability.EVENT_OUTCOME,
2966         EvidenceCapability.RANKING,
2967         EvidenceCapability.GROUP_COMPARISON,
2968     }
2969
2970     assert not validate_evidence_queries(
2971         queries,
2972         semantic_map(),
2973         catalog,
2974         task_ids={"TASK_OUTCOME", "TASK_RANKING", "TASK_CONTRAST"},
2975         available=available,
2976         task_capabilities={
2977             "TASK_OUTCOME": EvidenceCapability.EVENT_OUTCOME,
2978             "TASK_RANKING": EvidenceCapability.RANKING,
2979             "TASK_CONTRAST": EvidenceCapability.GROUP_COMPARISON,
2980         },
2981     )
2982
2983     results = semantic_query_evidence(
2984         table_name="contest",
2985         payload=renamed_event(),
2986         semantic_map=semantic_map(),
2987         queries=queries,
2988     )
2989     by_type = {item.evidence_type: item for item in results}
2990
2991     outcome = by_type["event_outcome"]
2992     assert outcome.metrics["records"][0]["entity"] == "North"
2993     assert outcome.metrics["records"][0]["value"] == 12
2994     assert outcome.metrics["records"][1]["entity"] == "South"
2995     assert outcome.metrics["difference"] == 3
2996     assert "winner" not in outcome.metrics
2997     assert "defeated" not in outcome.finding.lower()
2998
2999     ranking = by_type["entity_ranking"].metrics["ranking"]
3000     assert [(item["entity"], item["group"], item["value"]) for item in ranking] == [
3001         ("Nia", "North", 7.0),
3002         ("Sol", "South", 6.0),
3003         ("Noor", "North", 4.0),
3004     ]
3005     context_values = by_type["event_context"].metrics["values"]
3006     assert {item["value"] for item in context_values} == {
3007         "2026-07-23",
3008         "Civic Hall",
3009     }
3010
3011
3012 def test_event_fallback_evidence_emits_generic_context():
3013     results = event_capability_evidence(renamed_event())
3014     by_type = {item.evidence_type: item for item in results}
3015
3016     assert "event_context" in by_type
3017     context_values = by_type["event_context"].metrics["values"]
3018     assert {item["value"] for item in context_values} == {
3019         "2026-07-23",
3020         "Civic Hall",
3021     }
3022     assert by_type["event_context"].capability == (
3023         EvidenceCapability.DATASET_PROFILE
3024     )
3025     assert "supplied event context" in by_type["event_context"].finding
3026
3027
3028 def test_event_fallback_query_builder_creates_executable_queries():
3029     catalog = build_structural_catalog({"contest": renamed_event()})
3030     available = {
3031         EvidenceCapability.DATASET_PROFILE,
3032         EvidenceCapability.EVENT_OUTCOME,
3033         EvidenceCapability.RANKING,
3034         EvidenceCapability.GROUP_COMPARISON,
3035     }
3036     queries = build_event_evidence_queries(
3037         semantic_map=semantic_map(),
3038         tasks=event_query_tasks(),
3039         available_capabilities=available,
3040         request="Understand the event and report its strongest findings.",
3041     )

```

```

3042
3043     assert {
3044         query.evidence_type
3045     } >= {
3046         "event_context",
3047         "event_status",
3048         "event_outcome",
3049         "entity_ranking",
3050         "participant_comparison",
3051     }
3052
3053     assert not validate_evidence_queries(
3054         queries,
3055         semantic_map(),
3056         catalog,
3057         task_ids={
3058             task.task_id
3059             for task in event_query_tasks()
3060         },
3061         available=available,
3062         task_capabilities={
3063             task.task_id: task.capability
3064             for task in event_query_tasks()
3065         },
3066     )
3067
3068     results = semantic_query_evidence(
3069         table_name="contest",
3070         payload=renamed_event(),
3071         semantic_map=semantic_map(),
3072         queries=queries,
3073     )
3074
3075     assert results
3076     assert {
3077         item.evidence_type
3078     } >= {
3079         "event_context",
3080         "event_status",
3081         "event_outcome",
3082         "entity_ranking",
3083         "participant_comparison",
3084     }
3085 }
3086
3087 def test_event_query_normaliser_keeps_broad_participant_comparisons():
3088     compact_map = semantic_map()
3089     bindings = [
3090         *compact_map.bindings,
3091         *[
3092             compact_map.bindings[4].model_copy(
3093                 update={
3094                     "binding_id": f"B_COMPONENT_{index:02d}",
3095                     "label": f"participant component {index}",
3096                     "path_pattern": f"sides.*component_{index}",
3097                     "analytical_function": (
3098                         AnalyticalFunction.OUTCOME_COMPONENT
3099                     ),
3100                 },
3101             )
3102             for index in range(6)
3103         ],
3104     ]
3105
3106     enriched = compact_map.model_copy(
3107         update={"bindings": bindings}
3108     )
3109     queries = [
3110         semantic_queries()[-1].model_copy(
3111             update={
3112                 "query_id": f"QUERY_CONTRAST_{index:02d}",
3113                 "value_binding_ids": [
3114                     f"B_COMPONENT_{index:02d}"
3115                 ],
3116             }
3117         )
3118         for index in range(6)
3119     ]
3120
3121     normalised = normalise_event_evidence_queries(
3122         queries=queries,
3123         semantic_map=enriched,
3124         tasks=event_query_tasks(),
3125         available_capabilities={
3126             EvidenceCapability.DATASET_PROFILE,
3127             EvidenceCapability.EVENT_OUTCOME,
3128             EvidenceCapability.RANKING,
3129             EvidenceCapability.GROUP_COMPARISON,
3130         },
3131         request="Understand the event and report its strongest findings.",
3132     )

```



```

3133
3134 comparison_count = sum(
3135     query.evidence_type
3136     in {"participant_comparison", "event_contrast"}
3137     for query in normalised
3138 )
3139
3140 assert comparison_count >= 6
3141 assert {
3142     f"B_COMPONENT_{index:02d}"
3143     for index in range(6)
3144 }.issubset(
3145     {
3146         binding_id
3147         for query in normalised
3148         for binding_id in query.value_binding_ids
3149     }
3150 )
3151
3152
3153 def test_semantic_query_validator_rejects_authored_operation_mismatch():
3154     bad_query = semantic_queries()[2].model_copy(update={"operation": EvidenceOperation.RETRIEVE})
3155
3156     errors = validate_evidence_queries(
3157         [bad_query],
3158         semantic_map(),
3159         build_structural_catalog({"contest": renamed_event()}),
3160         task_ids={"TASK_OUTCOME"},
3161         available={EvidenceCapability.EVENT_OUTCOME},
3162         task_capabilities={
3163             "TASK_OUTCOME": EvidenceCapability.EVENT_OUTCOME,
3164         },
3165     )
3166
3167     assert any("must use operation 'compare'" in error for error in errors)
3168
3169
3170 def test_generic_request_uses_semantically_inferred_event_genre():
3171     genre, source, confidence = resolve_report_genre(
3172         request="Understand the dataset and report its strongest findings.",
3173         planned_genre=ReportGenre.DATA_SCIENCE_REPORT,
3174         configured_genre=None,
3175         semantic_map=semantic_map(),
3176     )
3177
3178     assert genre == ReportGenre.EVENT_REPORT
3179     assert source == ReportSelectionSource.STRUCTURED_INFERENCE
3180     assert confidence == 0.98
3181
3182     explicit_genre, explicit_source, _ = resolve_report_genre(
3183         request="Write a data-science report.",
3184         planned_genre=ReportGenre.EVENT_REPORT,
3185         configured_genre=None,
3186         semantic_map=semantic_map(),
3187     )
3188     assert explicit_genre == ReportGenre.DATA_SCIENCE_REPORT
3189     assert explicit_source == ReportSelectionSource.EXPLICIT_USER_REQUEST
3190
3191
3192 def test_generic_event_inference_uses_event_recap_contract():
3193     inferred = infer_task_contract(
3194         request="Understand the supplied data and report its strongest findings.",
3195         structured_inputs={"contest": renamed_event()},
3196         input_structure=event_structure(),
3197         semantic_map=semantic_map(),
3198     )
3199     assert inferred.communication_task == CommunicationTask.EVENT_REPORT
3200     assert inferred.output_form.value == "multi_paragraph_report"
3201     assert inferred.focus_scope == "event_recap"
3202     assert inferred.selection_source == ReportSelectionSource.STRUCTURED_INFERENCE
3203
3204     genre, source, _ = resolve_report_genre(
3205         request="Understand the supplied data and report its strongest findings.",
3206         planned_genre=ReportGenre.DATA_SCIENCE_REPORT,
3207         configured_genre=None,
3208         input_structure=event_structure(),
3209         semantic_map=semantic_map(),
3210     )
3211     focus_scope = infer_event_focus_scope(
3212         selected_genre=genre,
3213         selection_source=source,
3214         configured_communication_task=None,
3215         configured_output_form=None,
3216         configured_focus_scope=None,
3217         input_structure=event_structure(),
3218         semantic_map=semantic_map(),
3219     )
3220     contract = task_contract_fields(
3221         genre=genre,
3222         communication_task=None,
3223         output_form=None,

```

```

3224         focus_scope=focus_scope,
3225     )
3226
3227     assert focus_scope == "event_recap"
3228     assert contract["communication_task"].value == "event_report"
3229     assert contract["realisation_policy"] == RealisationPolicy.EVENT_RECAP_STYLE
3230     assert contract["allow_headings"] is True
3231     assert "main_contrast" in contract["required_content_slots"]
3232
3233
3234 def test_explicit_or_configured_event_contract_does_not_infer_event_recap_scope():
3235     assert (
3236         infer_event_focus_scope(
3237             selected_genre=ReportGenre.EVENT_REPORT,
3238             selection_source=ReportSelectionSource.EXPLICIT_USER_REQUEST,
3239             configured_communication_task=None,
3240             configured_output_form=None,
3241             configured_focus_scope=None,
3242             input_structure=event_structure(),
3243             semantic_map=semantic_map(),
3244         )
3245         is None
3246     )
3247     assert (
3248         infer_event_focus_scope(
3249             selected_genre=ReportGenre.EVENT_REPORT,
3250             selection_source=ReportSelectionSource.STRUCTURED_INFERENCE,
3251             configured_communication_task=CommunicationTask.EVENT_REPORT,
3252             configured_output_form=None,
3253             configured_focus_scope=None,
3254             input_structure=event_structure(),
3255             semantic_map=semantic_map(),
3256         )
3257         is None
3258     )
3259
3260
3261 def test_event_capabilities_require_event_semantics_within_one_table():
3262     collection_map = semantic_map().model_copy(
3263         update={"input_shape": InputShape.ENTITY_COLLECTION}
3264     )
3265     bundle = DataBundle(
3266         tables={},
3267         source_paths=[],
3268         fingerprint="fixture",
3269         input_structure=InputStructureProfile(
3270             shape=InputShape.ENTITY_COLLECTION,
3271             representation_status=InputRepresentationStatus.VALID,
3272             confidence=0.95,
3273         ),
3274     )
3275
3276     capabilities = available_capabilities(bundle, collection_map)
3277
3278     assert EvidenceCapability.RANKING in capabilities
3279     assert EvidenceCapability.EVENT_OUTCOME not in capabilities
3280     assert EvidenceCapability.ENTITY_PERFORMANCE not in capabilities
3281
3282
3283 def test_event_writer_scope_excludes_flat_wrapper_profile_facts():
3284     event = evidence_item(
3285         evidence_id="EVID_EVENT",
3286         capability=EvidenceCapability.EVENT_OUTCOME,
3287         evidence_type="event_outcome",
3288         semantic_level=SemanticLevel.EVENT,
3289     )
3290     wrapper = evidence_item(
3291         evidence_id="EVID_WRAPPER",
3292         capability=EvidenceCapability.DATASET_PROFILE,
3293         evidence_type="dataset_overview",
3294         semantic_level=SemanticLevel.DATASET,
3295     )
3296     ledger = EvidenceLedger(fingerprint="fixture", items=[event, wrapper])
3297     facts = FactLedger(
3298         writer_ready_facts=[
3299             verified_fact(fact_id="FACT_EVENT", evidence=event),
3300             verified_fact(fact_id="FACT_WRAPPER", evidence=wrapper),
3301         ]
3302     )
3303
3304     scoped = scope_fact_ledger_for_genre(
3305         facts,
3306         ledger,
3307         ReportGenre.EVENT_REPORT,
3308     )
3309
3310     assert [fact.fact_id for fact in scoped.writer_ready_facts] == ["FACT_EVENT"]
3311
3312
3313 def test_event_fact_selection_keeps_broad_verified_event_coverage():
3314     items = [

```

```

3315     evidence_item(
3316         evidence_id="EVID_RESULT",
3317         capability=EvidenceCapability.EVENT_OUTCOME,
3318         evidence_type="event_outcome",
3319         semantic_level=SemanticLevel.PARTICIPANT,
3320         analytical_function=AnalyticalFunction.OUTCOME,
3321     ),
3322     *[
3323         evidence_item(
3324             evidence_id=f"EVID_PERFORMANCE_{index}",
3325             capability=EvidenceCapability.RANKING,
3326             evidence_type="entity_ranking",
3327             semantic_level=SemanticLevel.ENTITY,
3328             analytical_function=AnalyticalFunction.PERFORMANCE,
3329         )
3330         for index in range(1, 4)
3331     ],
3332     evidence_item(
3333         evidence_id="EVID_PARTICIPATION",
3334         capability=EvidenceCapability.RANKING,
3335         evidence_type="entity_ranking",
3336         semantic_level=SemanticLevel.ENTITY,
3337         analytical_function=AnalyticalFunction.PARTICIPATION,
3338     ),
3339     evidence_item(
3340         evidence_id="EVID_GENERAL_CONTRAST",
3341         capability=EvidenceCapability.GROUP_COMPARISON,
3342         evidence_type="event_contrast",
3343         semantic_level=SemanticLevel.PARTICIPANT,
3344         analytical_function=AnalyticalFunction.PERFORMANCE,
3345     ),
3346     *[
3347         evidence_item(
3348             evidence_id=f"EVID_CONTRAST_{index}",
3349             capability=EvidenceCapability.GROUP_COMPARISON,
3350             evidence_type="event_contrast",
3351             semantic_level=SemanticLevel.PARTICIPANT,
3352             analytical_function=(
3353                 AnalyticalFunction.OUTCOME_COMPONENT
3354             ),
3355         )
3356         for index in range(1, 4)
3357     ],
3358 ]
3359 ledger = EvidenceLedger(
3360     fingerprint="fixture",
3361     items=items,
3362 )
3363 facts = [
3364     verified_fact(
3365         fact_id=f"FACT_{item.evidence_id}",
3366         evidence=item,
3367     )
3368     for item in items
3369 ]
3370
3371 priority, supporting = select_event_priority_facts(
3372     facts=facts,
3373     evidence=ledger,
3374     settings=Settings(),
3375     request=(
3376         "Understand the event and report its strongest findings."
3377     ),
3378 )
3379 selected_evidence_ids = {
3380     evidence_id
3381     for fact in [*priority, *supporting]
3382     for evidence_id in fact.evidence_ids
3383 }
3384 priority_evidence_ids = {
3385     evidence_id
3386     for fact in priority
3387     for evidence_id in fact.evidence_ids
3388 }
3389
3390 assert "EVID_RESULT" in selected_evidence_ids
3391 assert {
3392     "EVID_PERFORMANCE_1",
3393     "EVID_PERFORMANCE_2",
3394     "EVID_PERFORMANCE_3",
3395 }.issubset(selected_evidence_ids)
3396 assert len(
3397     {
3398         evidence_id
3399         for evidence_id in priority_evidence_ids
3400         if evidence_id.startswith("EVID_CONTRAST_")
3401     }
3402 ) == 3
3403 assert "EVID_GENERAL_CONTRAST" in priority_evidence_ids
3404 assert "EVID_PARTICIPATION" not in selected_evidence_ids
3405

```

```

3406
3407 def test_event_priority_prefers_actionable_sequence_over_structural_sequence():
3408     result = evidence_item(
3409         evidence_id="EVID_RESULT",
3410         capability=EvidenceCapability.EVENT_OUTCOME,
3411         evidence_type="event_outcome",
3412         semantic_level=SemanticLevel.PARTICIPANT,
3413     )
3414     structural_sequence = evidence_item(
3415         evidence_id="EVID_STRUCTURAL_SEQUENCE",
3416         capability=EvidenceCapability.EVENT_OUTCOME,
3417         evidence_type="event_sequence",
3418         semantic_level=SemanticLevel.EVENT,
3419     ).model_copy(
3420         update={
3421             "strength_label": "event_sequence",
3422             "metrics": {"step_count": 120},
3423         }
3424     )
3425     actionable_sequence = evidence_item(
3426         evidence_id="EVID_ACTIONABLE_SEQUENCE",
3427         capability=EvidenceCapability.EVENT_OUTCOME,
3428         evidence_type="event_sequence",
3429         semantic_level=SemanticLevel.EVENT,
3430     ).model_copy(
3431         update={
3432             "strength_label": "event_sequence_highlight",
3433             "metrics": {
3434                 "sequence_type": "score_changing_sequence",
3435                 "highlights": [
3436                     {
3437                         "event_text": "period 1: North recorded action A",
3438                         "score_phrase": "North led 2-0",
3439                         "score_delta": 2,
3440                         "left_value": 2,
3441                         "right_value": 0,
3442                     },
3443                     {
3444                         "event_text": "period 2: South recorded action B",
3445                         "score_phrase": "South led 3-2",
3446                         "score_delta": 3,
3447                         "left_value": 2,
3448                         "right_value": 3,
3449                     },
3450                 ],
3451             },
3452         }
3453     )
3454     ledger = EvidenceLedger(
3455         fingerprint="fixture",
3456         items=[result, structural_sequence, actionable_sequence],
3457     )
3458     facts = [
3459         verified_fact(fact_id="FACT_RESULT", evidence=result),
3460         verified_fact(
3461             fact_id="FACT_STRUCTURAL_SEQUENCE",
3462             evidence=structural_sequence,
3463         ),
3464         verified_fact(
3465             fact_id="FACT_ACTIONABLE_SEQUENCE",
3466             evidence=actionable_sequence,
3467         ),
3468     ]
3469
3470     priority, _ = select_event_priority_facts(
3471         facts=facts,
3472         evidence=ledger,
3473         settings=Settings(),
3474         request="Write an event report.",
3475     )
3476
3477     priority_ids = {fact.fact_id for fact in priority}
3478     assert "FACT_ACTIONABLE_SEQUENCE" in priority_ids
3479     assert "FACT_STRUCTURAL_SEQUENCE" not in priority_ids
3480
3481
3482 def test_event_content_requirements_use_actionable_sequence_candidates():
3483     structural_sequence = evidence_item(
3484         evidence_id="EVID_STRUCTURAL_SEQUENCE",
3485         capability=EvidenceCapability.EVENT_OUTCOME,
3486         evidence_type="event_sequence",
3487         semantic_level=SemanticLevel.EVENT,
3488     ).model_copy(
3489         update={
3490             "strength_label": "event_sequence",
3491             "metrics": {"step_count": 120},
3492         }
3493     )
3494     actionable_sequence = evidence_item(
3495         evidence_id="EVID_ACTIONABLE_SEQUENCE",
3496         capability=EvidenceCapability.EVENT_OUTCOME,

```

```

3497     evidence_type="event_sequence",
3498     semantic_level=SemanticLevel.EVENT,
3499 ).model_copy(
3500     update={
3501         "strength_label": "event_sequence_highlight",
3502         "metrics": {
3503             "sequence_type": "score_changing_sequence",
3504             "highlights": [
3505                 {
3506                     "event_text": "period 1: North recorded action A",
3507                     "score_phrase": "North led 2-0",
3508                     "score_delta": 2,
3509                 },
3510             ],
3511         },
3512     }
3513 )
3514 second_actionable_sequence = actionable_sequence.model_copy(
3515     update={
3516         "evidence_id": "EVID_SECOND_ACTIONABLE_SEQUENCE",
3517         "metrics": {
3518             "sequence_type": "score_changing_sequence",
3519             "highlights": [
3520                 {
3521                     "event_text": "period 2: South recorded action B",
3522                     "score_phrase": "South led 3-2",
3523                     "score_delta": 3,
3524                 },
3525             ],
3526         },
3527     }
3528 )
3529 spec = event_report_specification().model_copy(
3530     update={
3531         "required_content_slots": ["event_sequence"],
3532         "optional_content_slots": [],
3533     }
3534 )
3535 ledger = EvidenceLedger(
3536     fingerprint="fixture",
3537     items=[
3538         structural_sequence,
3539         actionable_sequence,
3540         second_actionable_sequence,
3541     ],
3542 )
3543 facts = [
3544     verified_fact(
3545         fact_id="FACT_STRUCTURAL_SEQUENCE",
3546         evidence=structural_sequence,
3547     ),
3548     verified_fact(
3549         fact_id="FACT_ACTIONABLE_SEQUENCE",
3550         evidence=actionable_sequence,
3551     ),
3552     verified_fact(
3553         fact_id="FACT_SECOND_ACTIONABLE_SEQUENCE",
3554         evidence=second_actionable_sequence,
3555     ),
3556 ]
3557
3558 requirements = build_writer_content_requirements(
3559     report_specification=spec,
3560     fact_ledger=FactLedger(writer_ready_facts=facts),
3561     evidence=ledger,
3562     insight_ledger=InsightLedger(synthesis_enabled=False),
3563     settings=Settings(),
3564 )
3565
3566 sequence_unit = next(
3567     unit
3568     for unit in requirements["units"]
3569     if unit["unit_id"] == "event_sequence"
3570 )
3571 assert sequence_unit["candidate_fact_ids"] == [
3572     "FACT_ACTIONABLE_SEQUENCE",
3573     "FACT_SECOND_ACTIONABLE_SEQUENCE",
3574 ]
3575 assert sequence_unit["minimum_items"] == 1
3576
3577
3578 def test_event_content_requirements_prioritise_performance_before_sequence():
3579     result = evidence_item(
3580         evidence_id="EVID_RESULT",
3581         capability=EvidenceCapability.EVENT_OUTCOME,
3582         evidence_type="event_outcome",
3583         semantic_level=SemanticLevel.EVENT,
3584     )
3585     sequence = evidence_item(
3586         evidence_id="EVID_SEQUENCE",
3587         capability=EvidenceCapability.EVENT_OUTCOME,

```

```

3588     evidence_type="event_sequence",
3589     semantic_level=SemanticLevel.EVENT,
3590 ).model_copy(
3591     update={
3592         "strength_label": "event_sequence_highlight",
3593         "metrics": {
3594             "sequence_type": "score_changing_sequence",
3595             "highlights": [
3596                 {
3597                     "event_text": "period 1: North recorded action A",
3598                     "score_phrase": "North led 2-0",
3599                     "score_delta": 2,
3600                 }
3601             ],
3602         },
3603     }
3604 )
3605 ranking = evidence_item(
3606     evidence_id="EVID_RANKING",
3607     capability=EvidenceCapability.RANKING,
3608     evidence_type="entity_ranking",
3609     semantic_level=SemanticLevel.ENTITY,
3610     analytical_function=AnalyticalFunction.PERFORMANCE,
3611 )
3612 contrast = evidence_item(
3613     evidence_id="EVID_CONTRAST",
3614     capability=EvidenceCapability.GROUP_COMPARISON,
3615     evidence_type="event_contrast",
3616     semantic_level=SemanticLevel.PARTICIPANT,
3617     analytical_function=AnalyticalFunction.OUTCOME_COMPONENT,
3618 )
3619 spec = event_report_specification().model_copy(
3620     update={
3621         "required_content_slots": [
3622             "event_result",
3623             "event_sequence",
3624             "leading_performance",
3625             "main_contrast",
3626         ],
3627         "optional_content_slots": [],
3628     }
3629 )
3630
3631 requirements = build_writer_content_requirements(
3632     report_specification=spec,
3633     fact_ledger=FactLedger(
3634         writer_ready_facts=[
3635             verified_fact(fact_id="FACT_RESULT", evidence=result),
3636             verified_fact(fact_id="FACT_SEQUENCE", evidence=sequence),
3637             verified_fact(fact_id="FACT_RANKING", evidence=ranking),
3638             verified_fact(fact_id="FACT_CONTRAST", evidence=contrast),
3639         ]
3640     ),
3641     evidence=EvidenceLedger(
3642         fingerprint="fixture",
3643         items=[result, sequence, ranking, contrast],
3644     ),
3645     insight_ledger=InsightLedger(synthesis_enabled=False),
3646     settings=Settings(),
3647 )
3648
3649 unit_ids = [unit["unit_id"] for unit in requirements["units"]]
3650
3651 assert unit_ids == [
3652     "event_result",
3653     "leading_performance",
3654     "main_contrast",
3655     "event_sequence",
3656 ]
3657
3658
3659 def test_content_requirement_counts_insight_source_fact_coverage():
3660     requirements = {
3661         "units": [
3662             {
3663                 "unit_id": "event_sequence",
3664                 "description": "Use sequence facts.",
3665                 "minimum_items": 2,
3666                 "candidate_fact_ids": ["FACT_A", "FACT_B"],
3667                 "candidate_insight_ids": ["INS_SEQUENCE"],
3668                 "candidate_insight_fact_ids": {
3669                     "INS_SEQUENCE": ["FACT_A", "FACT_B"]
3670                 },
3671             }
3672         ]
3673     }
3674
3675     errors = content_requirement_errors(
3676         used_fact_ids=set(),
3677         used_insight_ids={"INS_SEQUENCE"},
3678         word_count=120,

```

```

3679         requirements=requirements,
3680     )
3681
3682     assert errors == []
3683
3684
3685 def test_event_content_requirements_are_quality_not_fatal_for_writer_validation():
3686     result = evidence_item(
3687         evidence_id="EVID_RESULT",
3688         capability=EvidenceCapability.EVENT_OUTCOME,
3689         evidence_type="event_outcome",
3690         semantic_level=SemanticLevel.EVENT,
3691     )
3692     contrast = evidence_item(
3693         evidence_id="EVID_CONTRAST",
3694         capability=EvidenceCapability.GROUP_COMPARISON,
3695         evidence_type="event_contrast",
3696         semantic_level=SemanticLevel.PARTICIPANT,
3697         analytical_function=AnalyticalFunction.OUTCOME_COMPONENT,
3698     )
3699     spec = event_report_specification().model_copy(
3700         update={
3701             "focus_scope": "event_recap",
3702             "realisation_policy": RealisationPolicy.EVENT_RECAP_STYLE,
3703             "required_content_slots": [
3704                 "event_result",
3705                 "main_contrast",
3706             ],
3707             "optional_content_slots": [],
3708         }
3709     )
3710     requirements = build_writer_content_requirements(
3711         report_specification=spec,
3712         fact_ledger=FactLedger(
3713             writer_ready_facts=[
3714                 verified_fact(fact_id="FACT_RESULT", evidence=result),
3715                 verified_fact(fact_id="FACT_CONTRAST", evidence=contrast),
3716             ]
3717         ),
3718         evidence=EvidenceLedger(
3719             fingerprint="fixture",
3720             items=[result, contrast],
3721         ),
3722         insight_ledger=InsightLedger(synthesis_enabled=False),
3723         settings=Settings(),
3724     )
3725
3726     fatal_errors = content_requirement_errors(
3727         used_fact_ids={"FACT_RESULT"},
3728         used_insight_ids=set(),
3729         word_count=80,
3730         requirements=requirements,
3731     )
3732     quality_errors = content_requirement_errors(
3733         used_fact_ids={"FACT_RESULT"},
3734         used_insight_ids=set(),
3735         word_count=80,
3736         requirements=requirements,
3737         respect_validation_severity=False,
3738     )
3739
3740     assert requirements["content_unit_validation"] == "quality"
3741     assert fatal_errors == []
3742     assert any("main_contrast" in error for error in quality_errors)
3743
3744
3745 def test_event_quality_gate_requires_actionable_sequence_coverage():
3746     structural_sequence = evidence_item(
3747         evidence_id="EVID_STRUCTURAL_SEQUENCE",
3748         capability=EvidenceCapability.EVENT_OUTCOME,
3749         evidence_type="event_sequence",
3750         semantic_level=SemanticLevel.EVENT,
3751     ).model_copy(
3752         update={
3753             "strength_label": "event_sequence",
3754             "metrics": {"step_count": 120},
3755         }
3756     )
3757     actionable_sequence = evidence_item(
3758         evidence_id="EVID_ACTIONABLE_SEQUENCE",
3759         capability=EvidenceCapability.EVENT_OUTCOME,
3760         evidence_type="event_sequence",
3761         semantic_level=SemanticLevel.EVENT,
3762     ).model_copy(
3763         update={
3764             "strength_label": "event_sequence_highlight",
3765             "metrics": {
3766                 "sequence_type": "score_changing_sequence",
3767                 "highlights": [
3768                     {
3769                         "event_text": "period 1: North recorded action A",

```



```

3770         "score_phrase": "North led 2-0",
3771         "score_delta": 2,
3772     },
3773 ],
3774 },
3775 )
3776
3777 spec = event_report_specification().model_copy(
3778     update={
3779         "required_content_slots": ["event_sequence"],
3780         "optional_content_slots": [],
3781     }
3782 )
3783
3784 output = WriterOutput(
3785     title="Event report",
3786     markdown=(
3787         "# Event report\n\n"
3788         "The event structure includes an ordered sequence, but detailed "
3789         "chronology was not analyzed in this report.\n"
3790     ),
3791     sentence_support=[
3792         SentenceSupport(
3793             sentence_id="SENT_0001",
3794             sentence_text=(
3795                 "The event structure includes an ordered sequence, but "
3796                 "detailed chronology was not analyzed in this report."
3797             ),
3798             fact_ids=["FACT_STRUCTURAL_SEQUENCE"],
3799             evidence_ids=[structural_sequence.evidence_id],
3800             support_type=SupportType.DIRECT,
3801         )
3802     ],
3803 )
3804
3805 assessment = assess_genre_quality(
3806     output,
3807     spec,
3808     EvidenceLedger(
3809         fingerprint="fixture",
3810         items=[structural_sequence, actionable_sequence],
3811     ),
3812 )
3813
3814 assert assessment.status == QualityStatus.REVISE
3815 assert "event_sequence" in assessment.missing_supported_slots
3816 assert any(
3817     "omits or disclaims event-sequence narration" in finding
3818     for finding in assessment.findings
3819 )
3820
3821 def test_insight_rejects_unsupported_completeness_and_duplicate_claims():
3822     event = evidence_item(
3823         evidence_id="EVID_EVENT_CONTEXT",
3824         capability=EvidenceCapability.EVENT_OUTCOME,
3825         evidence_type="event_context",
3826         semantic_level=SemanticLevel.EVENT,
3827     )
3828
3829     fact = verified_fact(
3830         fact_id="FACT_EVENT_CONTEXT",
3831         evidence=event,
3832     )
3833
3834     candidate = InsightCandidate(
3835         insight_id="INS_EVENT_QUALITY",
3836         statement=(
3837             "The event record contains no missing data or duplicate rows."
3838         ),
3839         insight_type=InsightType.NARRATIVE_SUMMARY,
3840         interpretation_level=InterpretationLevel.BOUNDED_INSIGHT,
3841         source_fact_ids=[fact.fact_id],
3842         source_evidence_ids=[event.evidence_id],
3843         why_it_matters=(
3844             "Every recorded field would therefore be available for "
3845             "interpretation."
3846         ),
3847         supporting_summary="One event-context fact was supplied.",
3848         claim_permissions=[ClaimPermission.DESCRPTIVE],
3849         confidence=0.9,
3850         salience=0.8,
3851     )
3852
3853     errors = validate_insight_candidates(
3854         InsightCandidateSet(candidates=[candidate]),
3855         FactLedger(writer_ready_facts=[fact]),
3856         EvidenceLedger(fingerprint="fixture", items=[event]),
3857         Settings(),
3858     )
3859
3860     assert any(
3861         "without missingness evidence" in error
3862         for error in errors
    
```



```

3861     )
3862     assert any(
3863         "without duplicate-row evidence" in error
3864         for error in errors
3865     )
3866
3867 @pytest.mark.parametrize(
3868     "boilerplate",
3869     [
3870         "Statistical modeling is not possible because the wrapper has one row.",
3871         (
3872             "Observed associations are descriptive. "
3873             "Group comparisons are unadjusted."
3874         ),
3875     ],
3876 )
3877 )
3878 def test_event_quality_gate_rejects_flat_modelling_discussion(boilerplate):
3879     event = evidence_item(
3880         evidence_id="EVID_EVENT",
3881         capability=EvidenceCapability.EVENT_OUTCOME,
3882         evidence_type="event_outcome",
3883         semantic_level=SemanticLevel.EVENT,
3884     )
3885     output = WriterOutput(
3886         title="Event report",
3887         markdown=(
3888             f"# Event report\n\nNorth recorded 12. {boilerplate}\n"
3889         ),
3890         sentence_support=[
3891             SentenceSupport(
3892                 sentence_id="SENT_0001",
3893                 sentence_text="North recorded 12.",
3894                 fact_ids=["FACT_EVENT"],
3895                 evidence_ids=[event.evidence_id],
3896                 support_type=SupportType.DIRECT,
3897             )
3898         ],
3899     )
3900
3901     assessment = assess_genre_quality(
3902         output,
3903         event_report_specification(),
3904         EvidenceLedger(fingerprint="fixture", items=[event]),
3905     )
3906
3907     assert assessment.status == QualityStatus.REVISE
3908     assert any("flat-table profiling or modelling" in finding for finding in assessment.findings)
3909
3910
3911 def test_event_quality_gate_rejects_participation_substitution():
3912     performance = evidence_item(
3913         evidence_id="EVID_PERFORMANCE",
3914         capability=EvidenceCapability.RANKING,
3915         evidence_type="entity_ranking",
3916         semantic_level=SemanticLevel.ENTITY,
3917         analytical_function=AnalyticalFunction.PERFORMANCE,
3918     )
3919     participation = evidence_item(
3920         evidence_id="EVID_PARTICIPATION",
3921         capability=EvidenceCapability.RANKING,
3922         evidence_type="entity_ranking",
3923         semantic_level=SemanticLevel.ENTITY,
3924         analytical_function=AnalyticalFunction.PARTICIPATION,
3925     )
3926     output = WriterOutput(
3927         title="Event report",
3928         markdown=(
3929             "# Event report\n\n"
3930             "One entity recorded the longest duration.\n"
3931         ),
3932         sentence_support=[
3933             SentenceSupport(
3934                 sentence_id="SENT_0001",
3935                 sentence_text=(
3936                     "One entity recorded the longest duration."
3937                 ),
3938                 fact_ids=["FACT_PARTICIPATION"],
3939                 evidence_ids=[participation.evidence_id],
3940                 support_type=SupportType.DIRECT,
3941             )
3942         ],
3943     )
3944
3945     assessment = assess_genre_quality(
3946         output,
3947         event_report_specification(),
3948         EvidenceLedger(
3949             fingerprint="fixture",
3950             items=[performance, participation],
3951         ),

```

```

3952 )
3953
3954 assert assessment.status == QualityStatus.REVISE
3955 assert any(
3956     "uses participation evidence" in finding
3957     for finding in assessment.findings
3958 )
3959
3960
3961 def test_factual_title_must_map_every_named_entity_to_its_facts():
3962     event = evidence_item(
3963         evidence_id="EVID_EVENT",
3964         capability=EvidenceCapability.EVENT_OUTCOME,
3965         evidence_type="event_outcome",
3966         semantic_level=SemanticLevel.EVENT,
3967     )
3968     alpha_fact = verified_fact(
3969         fact_id="FACT_ALPHA",
3970         evidence=event,
3971     ).model_copy(update={"entities": ["Alpha"]})
3972     beta_fact = verified_fact(
3973         fact_id="FACT_BETA",
3974         evidence=event,
3975     ).model_copy(update={"entities": ["Beta"]})
3976     output = WriterOutput(
3977         title="Alpha defeats Beta",
3978         title_fact_ids=[alpha_fact.fact_id],
3979         markdown=(
3980             "# Alpha defeats Beta\n\n## Event overview\n\n"
3981             "Alpha has a supported event fact.\n"
3982         ),
3983         sentence_support=[
3984             SentenceSupport(
3985                 sentence_id="SENT_0001",
3986                 sentence_text="Alpha has a supported event fact.",
3987                 fact_ids=[alpha_fact.fact_id],
3988                 evidence_ids=[event.evidence_id],
3989                 support_type=SupportType.DIRECT,
3990             )
3991         ],
3992     )
3993
3994     errors = validate_writer_output(
3995         output,
3996         FactLedger(writer_ready_facts=[alpha_fact, beta_fact]),
3997     )
3998
3999     assert any(
4000         "entities unsupported by its facts" in error
4001         and "Beta" in error
4002         for error in errors
4003     )
4004
4005
4006 def test_factual_title_numbers_can_be_supported_by_fact_summary():
4007     event = evidence_item(
4008         evidence_id="EVID_EVENT",
4009         capability=EvidenceCapability.EVENT_OUTCOME,
4010         evidence_type="event_outcome",
4011         semantic_level=SemanticLevel.EVENT,
4012     )
4013     score_fact = verified_fact(
4014         fact_id="FACT_SCORE",
4015         evidence=event,
4016     ).model_copy(
4017         update={
4018             "fact_summary": (
4019                 "Mets recorded 9 for visiting runs, while Reds recorded 7 "
4020                 "for home runs."
4021             ),
4022             "structured_values": {},
4023             "entities": ["Mets", "Reds"],
4024         }
4025     )
4026     output = WriterOutput(
4027         title="Mets Defeat Reds 9-7",
4028         title_fact_ids=["FACT_SCORE"],
4029         markdown=(
4030             "# Mets Defeat Reds 9-7\n\n"
4031             "Mets recorded 9 runs while Reds recorded 7.\n"
4032         ),
4033         sentence_support=[
4034             SentenceSupport(
4035                 sentence_id="SENT_0001",
4036                 sentence_text=(
4037                     "Mets recorded 9 runs while Reds recorded 7."
4038                 ),
4039                 fact_ids=["FACT_SCORE"],
4040                 evidence_ids=[event.evidence_id],
4041                 support_type=SupportType.DIRECT,
4042             )

```

```

4043     ],
4044     selected_fact_ids=["FACT_SCORE"],
4045 )
4046
4047 assert not validate_writer_output(
4048     output,
4049     FactLedger(writer_ready_facts=[score_fact]),
4050 )
4051
4052
4053 def test_fact_candidate_rejects_driven_by_without_causal_permission():
4054     event = evidence_item(
4055         evidence_id="EVID_EVENT",
4056         capability=EvidenceCapability.EVENT_OUTCOME,
4057         evidence_type="event_outcome",
4058         semantic_level=SemanticLevel.EVENT,
4059     )
4060     candidate = FactCandidate(
4061         candidate_id="CAN_0001",
4062         fact_summary="The result was driven by the recorded value of 12.",
4063         evidence_ids=[event.evidence_id],
4064         claim_permissions=[ClaimPermission.DESCRPTIVE],
4065         factual_confidence=1.0,
4066         methodological_strength=1.0,
4067         user_relevance=1.0,
4068         salience=1.0,
4069         recommended_use=RecommendedUse.MAIN_FINDING,
4070     )
4071
4072     errors = validate_fact_candidates(
4073         FactCandidateSet(candidates=[candidate]),
4074         EvidenceLedger(fingerprint="fixture", items=[event]),
4075     )
4076
4077     assert any("unsupported causal wording" in error for error in errors)
4078
4079
4080 def test_deterministic_fact_fallback_recovers_semantic_query_evidence():
4081     query_evidence = evidence_item(
4082         evidence_id="EVID_QUERY",
4083         capability=EvidenceCapability.EVENT_OUTCOME,
4084         evidence_type="event_outcome",
4085         semantic_level=SemanticLevel.PARTICIPANT,
4086     ).model_copy(
4087         update={
4088             "query_id": "QUERY_OUTCOME",
4089             "finding": "Validated semantic query result for `event outcome`.",
4090         }
4091     )
4092
4093     candidates = fallback_fact_candidates(
4094         EvidenceLedger(fingerprint="fixture", items=[query_evidence]),
4095         maximum_facts=10,
4096     )
4097
4098     assert len(candidates.candidates) == 1
4099     assert candidates.candidates[0].evidence_ids == ["EVID_QUERY"]
4100
4101
4102 def test_deterministic_fact_fallback_summarises_event_outcome_records():
4103     query_evidence = evidence_item(
4104         evidence_id="EVID_OUTCOME",
4105         capability=EvidenceCapability.EVENT_OUTCOME,
4106         evidence_type="event_outcome",
4107         semantic_level=SemanticLevel.PARTICIPANT,
4108         analytical_function=AnalyticalFunction.OUTCOME,
4109     ).model_copy(
4110         update={
4111             "query_id": "QUERY_OUTCOME",
4112             "metrics": {
4113                 "records": [
4114                     {
4115                         "entity": "Astros",
4116                         "value": 4.0,
4117                         "measure": "Home team total runs",
4118                     },
4119                     {
4120                         "entity": "Rangers",
4121                         "value": 3.0,
4122                         "measure": "Visiting team total runs",
4123                     },
4124                 ]
4125             },
4126         }
4127     )
4128
4129     candidates = fallback_fact_candidates(
4130         EvidenceLedger(fingerprint="fixture", items=[query_evidence]),
4131         maximum_facts=10,
4132     )
4133

```

```

4134     assert len(candidates.candidates) == 1
4135     assert "Astros recorded 4" in candidates.candidates[0].fact_summary
4136     assert "Rangers recorded 3" in candidates.candidates[0].fact_summary
4137
4138
4139 def test_fact_candidate_scaffold_merge_preserves_unenriched_evidence():
4140     first = evidence_item(
4141         evidence_id="EVID_FIRST",
4142         capability=EvidenceCapability.EVENT_OUTCOME,
4143         evidence_type="event_outcome",
4144         semantic_level=SemanticLevel.PARTICIPANT,
4145     ).model_copy(update={"finding": "First supported fact."})
4146     second = evidence_item(
4147         evidence_id="EVID_SECOND",
4148         capability=EvidenceCapability.RANKING,
4149         evidence_type="entity_ranking",
4150         semantic_level=SemanticLevel.ENTITY,
4151         analytical_function=AnalyticalFunction.PERFORMANCE,
4152     ).model_copy(update={"finding": "Second supported fact."})
4153     ledger = EvidenceLedger(fingerprint="fixture", items=[first, second])
4154     scaffold = deterministic_fact_candidate_scaffold(
4155         ledger,
4156         maximum_facts=None,
4157     )
4158     enrichment = FactCandidateSet(
4159         candidates=[
4160             FactCandidate(
4161                 candidate_id="CAN_ENRICHED",
4162                 fact_summary="First supported fact.",
4163                 evidence_ids=["EVID_FIRST"],
4164                 claim_permissions=[ClaimPermission.DESSCRIPTIVE],
4165                 factual_confidence=1.0,
4166                 methodological_strength=1.0,
4167                 user_relevance=1.0,
4168                 salience=1.0,
4169                 recommended_use=RecommendedUse.MAIN_FINDING,
4170             )
4171         ],
4172         synthesis_notes=["LLM enriched one candidate."],
4173     )
4174
4175     merged = merge_fact_candidate_scaffold(
4176         scaffold=scaffold,
4177         enrichment=enrichment,
4178         evidence=ledger,
4179     )
4180     covered = {
4181         evidence_id
4182         for candidate in merged.candidates
4183         for evidence_id in candidate.evidence_ids
4184     }
4185
4186     assert covered == {"EVID_FIRST", "EVID_SECOND"}
4187     assert [candidate.candidate_id for candidate in merged.candidates] == [
4188         "CAN_0001",
4189         "CAN_0002",
4190     ]
4191
4192
4193 def test_fact_candidate_scaffold_expands_event_sequence_highlights():
4194     sequence = evidence_item(
4195         evidence_id="EVID_SEQUENCE",
4196         capability=EvidenceCapability.EVENT_OUTCOME,
4197         evidence_type="event_sequence",
4198         semantic_level=SemanticLevel.EVENT,
4199     ).model_copy(
4200         update={
4201             "strength_label": "event_sequence_highlight",
4202             "metrics": {
4203                 "sequence_type": "score_changing_sequence",
4204                 "highlights": [
4205                     {
4206                         "event_text": "period 1: North recorded action A",
4207                         "score_phrase": "North led 2-0",
4208                         "score_delta": 2,
4209                         "left_value": 2,
4210                         "right_value": 0,
4211                         "sequence_roles": ["opening_score"],
4212                     },
4213                     {
4214                         "event_text": "period 2: South recorded action B",
4215                         "score_phrase": "South led 3-2",
4216                         "score_delta": 3,
4217                         "left_value": 2,
4218                         "right_value": 3,
4219                         "sequence_roles": [
4220                             "lead_change",
4221                             "largest_score_change",
4222                         ],
4223                     },
4224                 ],
4225             },
4226         },
4227     )

```

```

4225         },
4226     }
4227 )
4228
4229 scaffold = deterministic_fact_candidate_scaffold(
4230     EvidenceLedger(fingerprint="fixture", items=[sequence]),
4231     maximum_facts=None,
4232 )
4233
4234 assert len(scaffold.candidates) == 2
4235 summaries = [candidate.fact_summary for candidate in scaffold.candidates]
4236 assert "period 1: North recorded action A" in summaries[0]
4237 assert "period 2: South recorded action B" in summaries[1]
4238 assert "lead change" not in summaries[1]
4239
4240
4241 def test_scaffold_merge_preserves_multiple_candidates_for_one_sequence_evidence():
4242     sequence = evidence_item(
4243         evidence_id="EVID_SEQUENCE",
4244         capability=EvidenceCapability.EVENT_OUTCOME,
4245         evidence_type="event_sequence",
4246         semantic_level=SemanticLevel.EVENT,
4247     ).model_copy(
4248         update={
4249             "strength_label": "event_sequence_highlight",
4250             "metrics": {
4251                 "sequence_type": "score_changing_sequence",
4252                 "highlights": [
4253                     {
4254                         "event_text": "period 1: North recorded action A",
4255                         "score_phrase": "North led 2-0",
4256                         "score_delta": 2,
4257                         "left_value": 2,
4258                         "right_value": 0,
4259                     },
4260                     {
4261                         "event_text": "period 2: South recorded action B",
4262                         "score_phrase": "South led 3-2",
4263                         "score_delta": 3,
4264                         "left_value": 2,
4265                         "right_value": 3,
4266                     },
4267                 ],
4268             },
4269         }
4270     )
4271     ledger = EvidenceLedger(fingerprint="fixture", items=[sequence])
4272     scaffold = deterministic_fact_candidate_scaffold(
4273         ledger,
4274         maximum_facts=None,
4275     )
4276     enrichment = FactCandidateSet(
4277         candidates=[
4278             FactCandidate(
4279                 candidate_id="CAN_ENRICHED",
4280                 fact_summary=(
4281                     "The supplied sequence contains supported score-changing "
4282                     "steps."
4283                 ),
4284                 evidence_ids=["EVID_SEQUENCE"],
4285                 claim_permissions=[
4286                     ClaimPermission.DESRIPTIVE,
4287                 ],
4288                 factual_confidence=1.0,
4289                 methodological_strength=1.0,
4290                 user_relevance=1.0,
4291                 salience=1.0,
4292                 recommended_use=RecommendedUse.MAIN_FINDING,
4293             )
4294         ],
4295         synthesis_notes=["LLM enriched one sequence candidate."],
4296     )
4297
4298     merged = merge_fact_candidate_scaffold(
4299         scaffold=scaffold,
4300         enrichment=enrichment,
4301         evidence=ledger,
4302     )
4303
4304     assert len(merged.candidates) == 3
4305     assert any(
4306         "period 1: North recorded action A" in candidate.fact_summary
4307         for candidate in merged.candidates
4308     )
4309     assert any(
4310         "period 2: South recorded action B" in candidate.fact_summary
4311         for candidate in merged.candidates
4312     )
4313     assert any(
4314         "supported score-changing steps" in candidate.fact_summary
4315         for candidate in merged.candidates

```

```

4316     )
4317
4318
4319 def test_spurious_missing_evidence_rejection_is_repaired():
4320     evidence = evidence_item(
4321         evidence_id="EVID_SEQUENCE",
4322         capability=EvidenceCapability.EVENT_OUTCOME,
4323         evidence_type="event_sequence",
4324         semantic_level=SemanticLevel.EVENT,
4325     ).model_copy(
4326         update={"limitations": ["Sequence evidence is descriptive only."]}
4327     )
4328     candidate = deterministic_fact_candidate_scaffold(
4329         EvidenceLedger(fingerprint="fixture", items=[evidence]),
4330         maximum_facts=None,
4331     ).candidates[0]
4332     verification = VerificationResult(
4333         reviews=[
4334             FactReview(
4335                 candidate_id=candidate.candidate_id,
4336                 decision=ReviewDecision.REJECT,
4337                 rationale=(
4338                     "Cited evidence EVID_SEQUENCE not present in the "
4339                     "evidence list."
4340                 ),
4341             ),
4342         ]
4343     )
4344
4345     repaired = repair_spurious_missing_evidence_rejections(
4346         candidate_set=FactCandidateSet(candidates=[candidate]),
4347         verification=verification,
4348         evidence=EvidenceLedger(fingerprint="fixture", items=[evidence]),
4349     )
4350
4351     assert repaired.reviews[0].decision == ReviewDecision.CAUTION
4352     assert any(
4353         "Repaired spurious missing-evidence rejection" in note
4354         for note in repaired.overall_notes
4355     )
4356
4357 def test_event_insight_rejects_report_omission_meta_commentary():
4358     sequence = evidence_item(
4359         evidence_id="EVID_SEQUENCE",
4360         capability=EvidenceCapability.EVENT_OUTCOME,
4361         evidence_type="event_sequence",
4362         semantic_level=SemanticLevel.EVENT,
4363     )
4364
4365     fact = verified_fact(fact_id="FACT_SEQUENCE", evidence=sequence)
4366     candidates = InsightCandidateSet(
4367         candidates=[
4368             InsightCandidate(
4369                 insight_id="INS_META",
4370                 statement=(
4371                     "The game data includes recorded play-by-play, but "
4372                     "detailed event chronology is not analyzed in this report."
4373                 ),
4374                 insight_type=InsightType.NARRATIVE_SUMMARY,
4375                 interpretation_level=InterpretationLevel.BOUNDED_INSIGHT,
4376                 source_fact_ids=[fact.fact_id],
4377                 source_evidence_ids=[sequence.evidence_id],
4378                 supporting_summary=(
4379                     "The source fact says event-sequence evidence exists."
4380                 ),
4381                 claim_permissions=[ClaimPermission.DESRIPTIVE],
4382                 confidence=1.0,
4383                 salience=0.8,
4384                 suitable_for_main_report=True,
4385             ),
4386         ]
4387     )
4388
4389     errors = validate_insight_candidates(
4390         candidates,
4391         FactLedger(writer_ready_facts=[fact]),
4392         EvidenceLedger(fingerprint="fixture", items=[sequence]),
4393         Settings(),
4394         report_genre=ReportGenre.EVENT_REPORT,
4395     )
4396
4397     assert any(
4398         "describes report omissions" in error
4399         for error in errors
4400     )
4401
4402 def test_execute_plan_propagates_semantic_query_provenance():
4403     payload = renamed_event()
4404     bundle = DataBundle(
4405         tables={"contest": pd.DataFrame([{"event": payload}])},

```

```

4407     source_paths=[],
4408     fingerprint="fixture",
4409     structured_inputs={"contest": payload},
4410     input_structure=event_structure(),
4411 )
4412
4413 def task(
4414     task_id: str,
4415     capability: EvidenceCapability,
4416     route: AnalysisRoute,
4417 ) -> InvestigationTask:
4418     return InvestigationTask(
4419         task_id=task_id,
4420         question=f"What can {capability.value} establish?",
4421         route=route,
4422         priority=5,
4423         table_name="contest",
4424         capability=capability,
4425         expected_evidence_types=[],
4426         required_evidence=[],
4427         claim_permissions=[
4428             ClaimPermission.DESCRPTIVE,
4429             ClaimPermission.COMPARATIVE,
4430         ],
4431         answerability_note="Use the validated semantic query plan.",
4432     )
4433
4434 plan = ExecutionPlan(
4435     objective="Describe the event.",
4436     tasks=[
4437         task(
4438             "TASK_OUTCOME",
4439             EvidenceCapability.EVENT_OUTCOME,
4440             AnalysisRoute.DESCRPTIVE,
4441         ),
4442         task(
4443             "TASK_RANKING",
4444             EvidenceCapability.RANKING,
4445             AnalysisRoute.DESCRPTIVE,
4446         ),
4447         task(
4448             "TASK_CONTRAST",
4449             EvidenceCapability.GROUP_COMPARISON,
4450             AnalysisRoute.ASSOCIATION_COMPARISON,
4451         ),
4452     ],
4453     route_order=[
4454         AnalysisRoute.DESCRPTIVE,
4455         AnalysisRoute.ASSOCIATION_COMPARISON,
4456     ],
4457     report_specification=event_report_specification(),
4458     audit_mode=AuditMode.INTERNAL,
4459     evidence_queries=semantic_queries(),
4460     available_capabilities=[
4461         EvidenceCapability.DATASET_PROFILE,
4462         EvidenceCapability.EVENT_OUTCOME,
4463         EvidenceCapability.RANKING,
4464         EvidenceCapability.GROUP_COMPARISON,
4465     ],
4466     selected_capabilities=[
4467         EvidenceCapability.DATASET_PROFILE,
4468         EvidenceCapability.EVENT_OUTCOME,
4469         EvidenceCapability.RANKING,
4470         EvidenceCapability.GROUP_COMPARISON,
4471     ],
4472     revision_limit=0,
4473     maximum_facts=10,
4474     rationale="Frozen semantic event plan.",
4475 )
4476
4477 evidence = execute_plan(
4478     bundle,
4479     plan,
4480     Settings(),
4481     semantic_map(),
4482 )
4483 query_items = [item for item in evidence.items if item.query_id]
4484 overview = next(
4485     item
4486     for item in evidence.items
4487     if item.evidence_type == "event_record_overview"
4488 )
4489
4490 assert len(query_items) == len(semantic_queries())
4491 assert all(
4492     item.method == "Validated generic semantic-query execution."
4493     for item in query_items
4494 )
4495 assert all(item.semantic_binding_ids for item in query_items)
4496 assert not overview.eligible_for_writer
4497

```

```

4498
4499 def test_semantic_map_without_queries_does_not_use_legacy_alias_extraction():
4500     payload = renamed_event()
4501     structure = event_structure()
4502     bundle = DataBundle(
4503         tables={"contest": pd.DataFrame([{"event": payload}])},
4504         source_paths=[],
4505         fingerprint="fixture",
4506         structured_inputs={"contest": payload},
4507         input_structure=structure,
4508     )
4509     task = InvestigationTask(
4510         task_id="TASK_EVENT",
4511         question="What is the event outcome?",
4512         route=AnalysisRoute.DESCRPTIVE,
4513         priority=5,
4514         table_name="contest",
4515         capability=EvidenceCapability.EVENT_OUTCOME,
4516         expected_evidence_types=["event_outcome"],
4517         required_evidence=["event_outcome"],
4518         claim_permissions=[
4519             ClaimPermission.DESCRPTIVE,
4520             ClaimPermission.COMPARATIVE,
4521         ],
4522         answerability_note="Use the semantic query plan.",
4523     )
4524     plan = ExecutionPlan(
4525         objective="Describe the event.",
4526         tasks=[task],
4527         route_order=[AnalysisRoute.DESCRPTIVE],
4528         report_specification=event_report_specification(),
4529         audit_mode=AuditMode.INTERNAL,
4530         available_capabilities=[EvidenceCapability.EVENT_OUTCOME],
4531         selected_capabilities=[EvidenceCapability.EVENT_OUTCOME],
4532         revision_limit=0,
4533         maximum_facts=10,
4534         rationale="Frozen semantic event plan.",
4535     )
4536
4537     evidence = execute_plan(
4538         bundle,
4539         plan,
4540         Settings(),
4541         semantic_map(),
4542     )
4543
4544     assert not evidence.items
4545     assert any(
4546         "Legacy field-alias extraction was not used" in note for note in evidence.execution_notes
4547     )

```

Listing 50: T04: table2text_pydanticai/tests/test_smoke.py

```

1  from __future__ import annotations
2
3  import json
4  from dataclasses import replace
5  from types import MethodType
6
7  import numpy as np
8  import pandas as pd
9  import pytest
10
11  from table2text import Settings, Table2TextWorkflow
12  from table2text.analytics import execute_plan
13  from table2text.audit import (
14      apply_repair_proposal,
15      apply_support_map_patches,
16      assess_genre_quality,
17      build_profile_support_registry,
18      build_writer_content_requirements,
19      build_writer_evidence_pack,
20      content_requirement_errors,
21      decide_release_status,
22      deterministic_audit,
23      fallback_writer,
24      materialise_writer_output,
25      merge_quality_assessments,
26      merge_audit_proposal,
27      select_priority_verified_insights,
28      sentence_support_narrative_stats,
29      split_markdown_sentences,
30      validate_repair_candidate,
31      validate_writer_output,
32  )
33  from table2text.capabilities import available_capabilities
34  from table2text.workflow import (
35      build_compact_writer_payload,
36      resolve_report_genre,

```



```

37     should_use_deterministic_event_plan,
38 )
39 from table2text.data import load_data, profile_data
40 from table2text.schemas import (
41     AnalysisRoute,
42     AnalyticalRecommendation,
43     AuditAnnotation,
44     AuditDecision,
45     AuditMode,
46     AuditRepairProposal,
47     AuditReport,
48     ClaimPermission,
49     ColumnProfile,
50     DataUnderstanding,
51     DataProfile,
52     ErrorType,
53     EvaluationFieldPolicy,
54     EvidenceCapability,
55     EvidenceItem,
56     EvidenceLedger,
57     ExecutionPlan,
58     FactLedger,
59     InsightCandidate,
60     InsightCandidateSet,
61     InsightContribution,
62     InsightLedger,
63     InsightVerificationStatus,
64     InsightRejection,
65     InsightType,
66     InsightVerificationRecord,
67     InsightVerificationResult,
68     InvestigationTask,
69     InputRepresentationStatus,
70     InputSemanticMap,
71     InputStructureProfile,
72     InputShape,
73     InterpretationLevel,
74     QualityStatus,
75     RecommendedUse,
76     ReleaseStatus,
77     RepairCandidate,
78     RepairStrategy,
79     ReportQualityAssessment,
80     ReportComponent,
81     ReportGenre,
82     ReportPerspective,
83     ReportSpecification,
84     SentenceRepair,
85     SentenceSupport,
86     Severity,
87     SupportType,
88     TableProfile,
89     TableUnderstanding,
90     TargetStatus,
91     ValidationStrategy,
92     VerifiedFact,
93     VerifiedInsight,
94     WriterEvidencePack,
95     WriterAgentDraft,
96     WriterOutput,
97     WriterSectionDraft,
98     WriterSentenceDraft,
99     ZeroRisk,
100 )
101 from table2text.agents import (
102     empty_insight_ledger,
103     fallback_execution_plan,
104     materialise_insight_ledger,
105     recover_missing_writer_insight_ids,
106     validate_insight_candidates,
107     validate_insight_verification,
108     valid_quality_finding,
109     writer_sentence_grounding_errors,
110 )
111
112
113 def make_passing_audit_report() -> AuditReport:
114     return AuditReport(
115         mode=AuditMode.INTERNAL,
116         decision=AuditDecision.PASS,
117         release_status=ReleaseStatus.APPROVED,
118         annotations=[],
119         applied_patches=[],
120         factual_sentence_count=1,
121         supported_sentence_count=1,
122         support_rate=1.0,
123         residual_risk="No high-confidence factual issue detected.",
124         revision_instructions=[],
125         quality_assessment=ReportQualityAssessment(
126             status=QualityStatus.PASS,
127             request_responsiveness=1.0,

```

```

128         finding_selection=1.0,
129         coherence=1.0,
130         concision=1.0,
131         caveat_integration=1.0,
132         data_science_interpretation=1.0,
133     ),
134 )
135
136
137 def test_profile_detects_constant_and_suspicious_zero(tmp_path):
138     path = tmp_path / "quality.csv"
139
140     frame = pd.DataFrame(
141         {
142             "constant": [0] * 200,
143             "pressure_like": [0] * 3 + list(np.linspace(990, 1030, 197)),
144             "temperature": np.linspace(-10, 30, 200),
145         }
146     )
147     frame.to_csv(path, index=False)
148
149     profile = profile_data(load_data([path]))
150     table = profile.tables[0]
151
152     constant = next(
153         column
154         for column in table.columns
155         if column.name == "constant"
156     )
157     pressure = next(
158         column
159         for column in table.columns
160         if column.name == "pressure_like"
161     )
162
163     assert constant.constant
164     assert pressure.suspicious_zero_values
165
166
167 def test_constant_outcome_not_group_compared(tmp_path):
168     path = tmp_path / "groups.csv"
169
170     pd.DataFrame(
171         {
172             "group": ["rain", "snow"] * 100,
173             "constant": [0] * 200,
174             "variable": np.arange(200),
175         }
176     ).to_csv(path, index=False)
177
178     bundle = load_data([path])
179     profile = profile_data(bundle)
180
181     plan = fallback_execution_plan(
182         "Describe the strongest relationships.",
183         profile,
184         AuditMode.INTERNAL,
185         Settings(),
186     )
187
188     evidence = execute_plan(
189         bundle,
190         plan,
191         Settings(),
192     )
193
194     constant_comparisons = [
195         item
196         for item in evidence.items
197         if item.route == AnalysisRoute.ASSOCIATION_COMPARISON
198         and "constant" in item.source_columns
199     ]
200
201     assert not constant_comparisons
202
203
204 def test_weak_correlations_are_filtered(tmp_path):
205     rng = np.random.default_rng(42)
206     path = tmp_path / "weak.csv"
207
208     pd.DataFrame(
209         {
210             "x": rng.normal(size=1_000),
211             "y": rng.normal(size=1_000),
212             "z": rng.normal(size=1_000),
213         }
214     ).to_csv(path, index=False)
215
216     settings = replace(
217         Settings(),
218         min_abs_correlation=0.20,

```

```

219     )
220
221     bundle = load_data([path])
222     profile = profile_data(bundle)
223
224     plan = fallback_execution_plan(
225         "Report the strongest associations.",
226         profile,
227         AuditMode.INTERNAL,
228         settings,
229     )
230
231     evidence = execute_plan(
232         bundle,
233         plan,
234         settings,
235     )
236
237     correlations = [
238         item.metrics["pearson_r"]
239         for item in evidence.items
240         if "pearson_r" in item.metrics
241     ]
242
243     assert all(abs(value) >= 0.20 for value in correlations)
244
245
246 def test_tabular_relationship_evidence_retains_capability_provenance(
247     tmp_path,
248 ):
249     path = tmp_path / "relationships.csv"
250     values = np.arange(200, dtype=float)
251     pd.DataFrame(
252         {
253             "x": values,
254             "y": values * 2,
255             "group": ["a"] * 100 + ["b"] * 100,
256         }
257     ).to_csv(path, index=False)
258     bundle = load_data([path])
259     plan = fallback_execution_plan(
260         "Report the strongest relationships.",
261         profile_data(bundle),
262         AuditMode.INTERNAL,
263         Settings(),
264     )
265
266     evidence = execute_plan(bundle, plan, Settings())
267
268     correlation = next(
269         item
270         for item in evidence.items
271         if "pearson_r" in item.metrics
272     )
273     group_comparison = next(
274         item
275         for item in evidence.items
276         if "group_counts" in item.metrics
277     )
278     assert correlation.capability == EvidenceCapability.ASSOCIATION
279     assert group_comparison.capability == (
280         EvidenceCapability.GROUP_COMPARISON
281     )
282
283 def test_final_approved_status_cannot_have_block_decision():
284     release_status = (
285         ReleaseStatus.APPROVED_WITH_WARNINGS
286     )
287
288     final_decision = (
289         AuditDecision.BLOCK
290         if release_status
291         == ReleaseStatus.HUMAN_REVIEW_REQUIRED
292         else AuditDecision.PASS
293     )
294
295     assert final_decision == AuditDecision.PASS
296
297 def test_semantic_block_without_serious_annotations_is_advisory():
298     deterministic = make_passing_audit_report()
299
300     proposal = AuditRepairProposal(
301         annotations=[],
302         repairs=[],
303         recommended_decision=AuditDecision.BLOCK,
304         residual_risk=(
305             "The model requested blocking without "
306             "supporting serious annotations."
307         ),
308         quality_assessment=(
309             deterministic.quality_assessment

```

```

310     ),
311 )
312
313 merged = merge_audit_proposal(
314     deterministic,
315     proposal,
316 )
317
318 assert merged.decision == AuditDecision.PASS
319 assert merged.release_status in {
320     ReleaseStatus.APPROVED,
321     ReleaseStatus.APPROVED_WITH_WARNINGS,
322 }
323
324 def test_generic_request_does_not_invent_prediction_target(tmp_path):
325     path = tmp_path / "weather.csv"
326
327     pd.DataFrame(
328         {
329             "Formatted Date": pd.date_range(
330                 "2020-01-01",
331                 periods=300,
332                 freq="h",
333             ),
334             "Temperature (C)": np.sin(np.arange(300) / 24),
335             "Humidity": np.linspace(0.3, 0.9, 300),
336         }
337     ).to_csv(path, index=False)
338
339     profile = profile_data(load_data([path]))
340
341     plan = fallback_execution_plan(
342         "Understand the dataset and report its strongest findings.",
343         profile,
344         AuditMode.INTERNAL,
345         Settings(),
346     )
347
348     assert not any(
349         task.route == AnalysisRoute.PREDICTIVE
350         for task in plan.tasks
351     )
352
353 def test_explicit_prediction_uses_selected_target(tmp_path):
354     path = tmp_path / "model.csv"
355
356     dates = pd.date_range(
357         "2020-01-01",
358         periods=500,
359         freq="h",
360     )
361
362     temperature = np.sin(np.arange(500) / 24)
363
364     pd.DataFrame(
365         {
366             "Formatted Date": dates,
367             "Temperature": temperature,
368             "Apparent Temperature": temperature + 0.001,
369             "Humidity": np.linspace(0.2, 0.9, 500),
370         }
371     ).to_csv(path, index=False)
372
373     settings = Settings()
374     bundle = load_data([path])
375     profile = profile_data(bundle)
376
377     plan = fallback_execution_plan(
378         "Predict Temperature from the available variables.",
379         profile,
380         AuditMode.INTERNAL,
381         settings,
382     )
383
384     predictive_task = next(
385         task
386         for task in plan.tasks
387         if task.route == AnalysisRoute.PREDICTIVE
388     )
389
390     assert predictive_task.target_column == "Temperature"
391     assert predictive_task.target_status == TargetStatus.USER_SELECTED
392     assert predictive_task.validation_strategy == (
393         ValidationStrategy.CHRONOLOGICAL_HOLDOUT
394     )
395
396     evidence = execute_plan(bundle, plan, settings)
397
398     predictive_evidence = next(
399         item
400

```

```

401     for item in evidence.items
402     if item.route == AnalysisRoute.PREDICTIVE
403     )
404
405     excluded = predictive_evidence.metrics.get(
406         "features_excluded",
407         [],
408     )
409
410     assert any(
411         item.get("risk_type") == "target_proxy"
412         for item in excluded
413     )
414
415
416 def test_hourly_forecast_uses_longer_rolling_windows(tmp_path):
417     path = tmp_path / "forecast.csv"
418     length = 5_000
419
420     pd.DataFrame(
421         {
422             "time": pd.date_range(
423                 "2020-01-01",
424                 periods=length,
425                 freq="h",
426             ),
427             "target": (
428                 10
429                 + np.sin(np.arange(length) * 2 * np.pi / 24)
430             ),
431         }
432     ).to_csv(path, index=False)
433
434     settings = Settings()
435     bundle = load_data([path])
436     profile = profile_data(bundle)
437
438     plan = fallback_execution_plan(
439         "Forecast target.",
440         profile,
441         AuditMode.INTERNAL,
442         settings,
443     )
444
445     evidence = execute_plan(bundle, plan, settings)
446
447     forecast = next(
448         item
449         for item in evidence.items
450         if item.route == AnalysisRoute.FORECASTING
451     )
452
453     assert forecast.validation_strategy == ValidationStrategy.ROLLING_ORIGIN
454     assert forecast.metrics["test_window_points"] >= 168
455     assert forecast.metrics["fold_count"] >= 1
456     assert any(
457         name.startswith("seasonal_naive_")
458         for name in forecast.metrics["mean_mae"]
459     )
460
461
462 def make_fact_fixture() -> tuple[FactLedger, EvidenceLedger]:
463     evidence = EvidenceLedger(
464         fingerprint="test",
465         items=[
466             EvidenceItem(
467                 evidence_id="EVD_0001",
468                 route=AnalysisRoute.DESRIPTIVE,
469                 task_ids=["TASK_001"],
470                 finding="The table contains 96,453 rows.",
471                 metrics={"row_count": 96_453},
472                 source_tables=["weather"],
473                 source_columns=[],
474                 method="Direct row count.",
475                 validation_strategy=ValidationStrategy.NONE,
476                 practical_interpretation="The dataset is large.",
477                 strength_label="dataset_overview",
478                 limitations=[],
479                 prohibited_interpretations=[],
480                 recommendations=[],
481                 claim_permissions=[ClaimPermission.DESRIPTIVE],
482                 factual_confidence=1.0,
483                 methodological_strength=1.0,
484                 user_relevance=1.0,
485                 salience=1.0,
486                 recommended_use=RecommendedUse.HEADLINE,
487             )
488         ],
489     )
490
491     ledger = FactLedger(

```

```

492     writer_ready_facts=[
493         VerifiedFact(
494             fact_id="FACT_0001",
495             source_candidate_id="CAN_0001",
496             fact_summary="The table contains 96,453 rows.",
497             evidence_ids=["EVD_0001"],
498             structured_values={
499                 "EVD_0001": {"row_count": 96_453}
500             },
501             entities=["weather"],
502             claim_permissions=[ClaimPermission.DESCRPTIVE],
503             factual_confidence=1.0,
504             methodological_strength=1.0,
505             user_relevance=1.0,
506             salience=1.0,
507             recommended_use=RecommendedUse.HEADLINE,
508         )
509     ]
510 )
511
512 return ledger, evidence
513
514
515 def test_approximate_number_is_allowed():
516     ledger, evidence = make_fact_fixture()
517
518     sentence = "The dataset contains more than 96,000 observations."
519
520     writer = WriterOutput(
521         title="Test",
522         markdown=f"# Test\n\n{sentence}\n",
523         sentence_support=[
524             SentenceSupport(
525                 sentence_id="SENT_0001",
526                 sentence_text=sentence,
527                 fact_ids=["FACT_0001"],
528                 evidence_ids=["EVD_0001"],
529                 support_type=SupportType.PARAPHRASE,
530             )
531         ],
532         selected_fact_ids=["FACT_0001"],
533     )
534
535     plan_spec = ReportSpecification(
536         report_purpose="Test",
537         target_length_words=300,
538         maximum_main_findings=5,
539         prioritisation_rule="Test",
540     )
541
542     audit = deterministic_audit(
543         writer,
544         ledger,
545         evidence,
546         AuditMode.INTERNAL,
547         [],
548         0,
549         plan_spec,
550     )
551
552     assert audit.decision == AuditDecision.PASS
553
554
555 def _supporting_fact_budget_fixture(
556     count: int,
557 ) -> tuple[FactLedger, EvidenceLedger, list[str], list[str]]:
558     evidence_items: list[EvidenceItem] = []
559     facts: list[VerifiedFact] = []
560
561     for index in range(count):
562         evidence_id = f"EVD_BUDGET_{index:04d}"
563         fact_id = f"FACT_BUDGET_{index:04d}"
564         evidence_items.append(
565             EvidenceItem(
566                 evidence_id=evidence_id,
567                 route=AnalysisRoute.DESCRPTIVE,
568                 task_ids=["TASK_BUDGET"],
569                 capability=EvidenceCapability.DATASET_PROFILE,
570                 evidence_type="supporting_context",
571                 finding="Verified source evidence supports the summary.",
572                 method="Synthetic regression fixture.",
573                 practical_interpretation=(
574                     "The evidence can be used as support for reader-facing prose."
575                 ),
576                 strength_label="direct",
577                 claim_permissions=[ClaimPermission.DESCRPTIVE],
578                 factual_confidence=1.0,
579                 methodological_strength=1.0,
580                 user_relevance=1.0,
581                 salience=1.0,
582                 recommended_use=RecommendedUse.SUPPORTING_DETAIL,

```

```

583     )
584     )
585     facts.append(
586         VerifiedFact(
587             fact_id=fact_id,
588             source_candidate_id=f"CAN_{fact_id}",
589             fact_summary=(
590                 "Verified source evidence supports the summary."
591             ),
592             evidence_ids=[evidence_id],
593             source_capabilities=[
594                 EvidenceCapability.DATASET_PROFILE,
595             ],
596             claim_permissions=[ClaimPermission.DESRIPTIVE],
597             factual_confidence=1.0,
598             methodological_strength=1.0,
599             user_relevance=1.0,
600             salience=1.0,
601             recommended_use=RecommendedUse.SUPPORTING_DETAIL,
602         )
603     )
604
605     return (
606         FactLedger(writer_ready_facts=facts),
607         EvidenceLedger(
608             fingerprint="support-budget",
609             items=evidence_items,
610         ),
611         [fact.fact_id for fact in facts],
612         [item.evidence_id for item in evidence_items],
613     )
614
615
616 def test_report_facts_are_uncapped_by_default():
617     ledger, evidence, fact_ids, evidence_ids = (
618         _supporting_fact_budget_fixture(12)
619     )
620     sentence = (
621         "Verified source evidence supports a focused reader summary with "
622         "enough detail to connect context, evidence, and limitations clearly"
623     )
624     sentence_count = 9
625     markdown = "# Supported Summary\n\n" + " ".join(
626         f"{sentence}."
627         for _ in range(sentence_count)
628     )
629     support = [
630         SentenceSupport(
631             sentence_id=f"SENT_BUDGET_{index:04d}",
632             sentence_text=sentence + ".",
633             fact_ids=fact_ids,
634             evidence_ids=evidence_ids,
635             support_type=SupportType.PARAPHRASE,
636         )
637         for index in range(sentence_count)
638     ]
639     output = WriterOutput(
640         title="Supported Summary",
641         markdown=markdown,
642         sentence_support=support,
643         selected_fact_ids=fact_ids,
644     )
645     spec = ReportSpecification(
646         report_purpose="Test uncapped fact selection.",
647         target_length_words=150,
648         prioritisation_rule="Use supported facts within the word ceiling.",
649     )
650
651     audit = deterministic_audit(
652         output,
653         ledger,
654         evidence,
655         AuditMode.INTERNAL,
656         [],
657         0,
658         spec,
659         Settings(),
660     )
661
662     assert not any(
663         "fact budget" in finding
664         for finding in audit.quality_assessment.findings
665     )
666
667
668 def test_report_word_ceiling_is_enforced():
669     ledger, evidence, fact_ids, evidence_ids = (
670         _supporting_fact_budget_fixture(2)
671     )
672     sentence = (
673         "Verified source evidence supports a focused reader summary with "

```

```

674         "enough detail to connect context and evidence clearly."
675     )
676     markdown = "# Supported Summary\n\n" + " ".join(
677         f"{sentence}."
678         for _ in range(15)
679     )
680     output = WriterOutput(
681         title="Supported Summary",
682         markdown=markdown,
683         sentence_support=[
684             SentenceSupport(
685                 sentence_id=f"SENT_WORDS_{index:04d}",
686                 sentence_text=sentence + ".",
687                 fact_ids=fact_ids,
688                 evidence_ids=evidence_ids,
689                 support_type=SupportType.PARAPHRASE,
690             )
691             for index in range(15)
692         ],
693         selected_fact_ids=fact_ids,
694     )
695     spec = ReportSpecification(
696         report_purpose="Test report word ceiling.",
697         target_length_words=150,
698         maximum_length_words=150,
699         prioritisation_rule="Use supported facts within the word ceiling.",
700     )
701
702     audit = deterministic_audit(
703         output,
704         ledger,
705         evidence,
706         AuditMode.INTERNAL,
707         [],
708         0,
709         spec,
710         Settings(),
711     )
712
713     assert any(
714         "word ceiling" in finding
715         for finding in audit.quality_assessment.findings
716     )
717
718 def test_target_length_is_not_an_implicit_word_ceiling():
719     ledger, evidence, fact_ids, evidence_ids = (
720         _supporting_fact_budget_fixture(2)
721     )
722     sentence = (
723         "Verified source evidence supports a focused reader summary with "
724         "enough detail to connect context and evidence clearly."
725     )
726     markdown = "# Supported Summary\n\n" + " ".join(
727         f"{sentence}."
728         for _ in range(15)
729     )
730
731     output = WriterOutput(
732         title="Supported Summary",
733         markdown=markdown,
734         sentence_support=[
735             SentenceSupport(
736                 sentence_id=f"SENT_WORDS_{index:04d}",
737                 sentence_text=sentence + ".",
738                 fact_ids=fact_ids,
739                 evidence_ids=evidence_ids,
740                 support_type=SupportType.PARAPHRASE,
741             )
742             for index in range(15)
743         ],
744         selected_fact_ids=fact_ids,
745     )
746     spec = ReportSpecification(
747         report_purpose="Test report target guidance.",
748         target_length_words=150,
749         prioritisation_rule="Use supported facts with concise prose.",
750     )
751
752     audit = deterministic_audit(
753         output,
754         ledger,
755         evidence,
756         AuditMode.INTERNAL,
757         [],
758         0,
759         spec,
760         Settings(),
761     )
762
763     assert not any(
764         "word ceiling" in finding

```



```

765         for finding in audit.quality_assessment.findings
766     )
767
768
769 def test_wrong_number_triggers_repair():
770     ledger, evidence = make_fact_fixture()
771
772     wrong_sentence = "The dataset contains 12 observations."
773
774     writer = WriterOutput(
775         title="Test",
776         markdown=f"# Test\n\n{wrong_sentence}\n",
777         sentence_support=[
778             SentenceSupport(
779                 sentence_id="SENT_0001",
780                 sentence_text=wrong_sentence,
781                 fact_ids=["FACT_0001"],
782                 evidence_ids=["EVD_0001"],
783                 support_type=SupportType.PARAPHRASE,
784             )
785         ],
786         selected_fact_ids=["FACT_0001"],
787     )
788
789     replacement = "The dataset contains 96,453 observations."
790
791     candidate = RepairCandidate(
792         repair_id="REP_001",
793         replacement_text=replacement,
794         strategy=RepairStrategy.MINIMAL_CORRECTION,
795         supporting_fact_ids=["FACT_0001"],
796         supporting_evidence_ids=["EVD_0001"],
797         factual_support_score=1.0,
798         meaning_preservation_score=1.0,
799         readability_score=1.0,
800         residual_hallucination_risk=0.0,
801     )
802
803     assert not validate_repair_candidate(
804         candidate,
805         ledger,
806         evidence,
807     )
808
809     proposal = AuditRepairProposal(
810         annotations=[],
811         repairs=[
812             SentenceRepair(
813                 sentence_id="SENT_0001",
814                 original_sentence=wrong_sentence,
815                 annotation_ids=[],
816                 candidates=[candidate],
817                 preferred_repair_id="REP_001",
818                 selection_reason="Correct the unsupported number.",
819             )
820         ],
821         recommended_decision=AuditDecision.REVISE,
822         residual_risk="Repair required.",
823         quality_assessment=ReportQualityAssessment(
824             status=QualityStatus.PASS,
825             request_responsiveness=1.0,
826             finding_selection=1.0,
827             coherence=1.0,
828             concision=1.0,
829             caveat_integration=1.0,
830             data_science_interpretation=1.0,
831         ),
832     )
833
834     repaired, patches = apply_repair_proposal(
835         writer,
836         proposal,
837         ledger,
838         evidence,
839     )
840
841     assert patches
842     assert replacement in repaired.markdown
843     assert wrong_sentence not in repaired.markdown
844
845
846 def test_full_workflow_without_llm(tmp_path):
847     path = tmp_path / "example.csv"
848
849     frame = pd.DataFrame(
850         {
851             "category": ["a", "b"] * 100,
852             "value": np.arange(200),
853             "constant": [0] * 200,
854         }
855     )

```

```

856 frame.to_csv(path, index=False)
857
858
859 settings = replace(
860     Settings(),
861     use_llm=False,
862     output_dir=tmp_path / "runs",
863     max_revision_rounds=1,
864 )
865
866 workflow = Table2TextWorkflow(settings)
867 assert workflow.evidence_insight_synthesis_agent is None
868 assert workflow.verifier_insight_verification_agent is None
869
870 result = workflow.run_sync(
871     inputs=[path],
872     request=(
873         "Understand the dataset and report its strongest supported findings."
874     ),
875     audit_mode=AuditMode.INTERNAL,
876 )
877
878 assert result.evidence_ledger.items
879 assert result.fact_ledger.writer_ready_facts
880 assert not result.insight_ledger.verified_insights
881 assert "LLM execution disabled" in (
882     result.insight_ledger.fallback_reason or ""
883 )
884 assert result.raw_writer_output.writer_mode == "deterministic_fallback"
885
886 run_directory = settings.output_dir / result.run_id
887
888 assert (run_directory / "09_writer_raw_report.md").exists()
889 assert (run_directory / "final_report.md").exists()
890 assert (run_directory / "final_result.json").exists()
891 assert (
892     run_directory
893     / "02_profile_support_registry.json"
894 ).exists()
895 assert (run_directory / "03_insight_objectives.json").exists()
896 assert (run_directory / "07_insight_candidates.json").exists()
897 assert (run_directory / "07_insight_verification.json").exists()
898 assert (run_directory / "07_insight_ledger.json").exists()
899 assert "insight_ledger" in result.model_dump(mode="json")
900
901 assert result.release_status in {
902     ReleaseStatus.APPROVED,
903     ReleaseStatus.APPROVED_WITH_WARNINGS,
904     ReleaseStatus.HUMAN_REVIEW_REQUIRED,
905 }
906
907
908 def test_zero_wind_bearing_is_likely_valid(tmp_path):
909     path = tmp_path / "bearing.csv"
910     pd.DataFrame(
911         {
912             "Wind Bearing (degrees)": [0, 10, 90, 180, 270] * 20,
913         }
914     ).to_csv(path, index=False)
915
916     profile = profile_data(load_data([path]))
917     column = profile.tables[0].columns[0]
918
919     assert column.zero_risk == ZeroRisk.LIKELY_VALID
920     assert not column.suspicious_zero_values
921
922
923 def test_zero_visibility_is_context_dependent(tmp_path):
924     path = tmp_path / "visibility.csv"
925     pd.DataFrame(
926         {
927             "Visibility (km)": [0, 1, 5, 10, 16] * 20,
928         }
929     ).to_csv(path, index=False)
930
931     profile = profile_data(load_data([path]))
932     column = profile.tables[0].columns[0]
933
934     assert column.zero_risk == ZeroRisk.CONTEXT_DEPENDENT
935     assert not column.suspicious_zero_values
936
937
938 def test_zero_pressure_is_possible_sentinel(tmp_path):
939     path = tmp_path / "pressure.csv"
940     pd.DataFrame(
941         {
942             "Pressure (millibars)": [0] * 2 + list(np.linspace(990, 1030, 198)),
943         }
944     ).to_csv(path, index=False)
945
946     profile = profile_data(load_data([path]))

```

```

947     column = profile.tables[0].columns[0]
948
949     assert column.zero_risk == ZeroRisk.POSSIBLE_SENTINEL
950     assert column.suspicious_zero_values
951
952
953 def test_generic_dataset_report_requires_overview(tmp_path):
954     path = tmp_path / "overview.csv"
955     pd.DataFrame(
956         {
957             "group": ["a", "b"] * 60,
958             "value": np.arange(120),
959         }
960     ).to_csv(path, index=False)
961
962     profile = profile_data(load_data([path]))
963     plan = fallback_execution_plan(
964         "Understand the dataset and report the strongest findings.",
965         profile,
966         AuditMode.INTERNAL,
967         Settings(),
968     )
969
970     required = set(plan.report_specification.required_components)
971     assert ReportComponent.DATASET_OVERVIEW in required
972     assert ReportComponent.DATA_QUALITY in required
973     assert ReportComponent.STRONGEST_RELATIONSHIPS in required
974
975
976 def test_writer_output_rejects_internal_guardrail_leakage():
977     ledger, _ = make_fact_fixture()
978     markdown = """
979     ## Global Prohibited Interpretations
980     - Do not say group membership caused the difference.
981     """
982     output = WriterOutput(
983         title="Leak",
984         markdown=markdown,
985         sentence_support=[],
986         selected_fact_ids=[],
987     )
988
989     assert validate_writer_output(output, ledger)
990
991
992 def test_writer_output_rejects_ledger_field_rendering():
993     ledger, _ = make_fact_fixture()
994     output = WriterOutput(
995         title="Leak",
996         markdown="Finding: Rain is warmer.\n\nImportant Note: Do not say causal.\n",
997         sentence_support=[],
998         selected_fact_ids=[],
999     )
1000
1001     assert validate_writer_output(output, ledger)
1002
1003
1004 def test_writer_output_accepts_natural_effect_interpretation():
1005     ledger, _ = make_fact_fixture()
1006     sentence = (
1007         "Rain observations were on average 17.3°C warmer than snow "
1008         "observations, representing a large difference."
1009     )
1010     fact = ledger.writer_ready_facts[0].model_copy(
1011         update={
1012             "fact_summary": sentence,
1013             "structured_values": {"EVD_0001": {"difference": 17.3}},
1014             "entities": ["Rain", "snow"],
1015         }
1016     )
1017     ledger = FactLedger(writer_ready_facts=[fact])
1018     output = WriterOutput(
1019         title="Natural",
1020         markdown=f"## Natural\n\n{sentence}\n",
1021         sentence_support=[
1022             SentenceSupport(
1023                 sentence_id="SENT_0001",
1024                 sentence_text=sentence,
1025                 fact_ids=["FACT_0001"],
1026                 evidence_ids=["EVD_0001"],
1027                 support_type=SupportType.PARAPHRASE,
1028             )
1029         ],
1030         selected_fact_ids=["FACT_0001"],
1031     )
1032
1033     assert not validate_writer_output(output, ledger)
1034
1035
1036 def test_materialise_writer_output_splits_multi_sentence_draft_text():
1037     ledger, _ = make_fact_fixture()

```

```

1038     draft = WriterAgentDraft(
1039         title="Weather summary",
1040         sections=[
1041             WriterSectionDraft(
1042                 heading="Dataset overview",
1043                 sentences=[
1044                     WriterSentenceDraft(
1045                         text=(
1046                             "The table contains 96,453 rows. "
1047                             "The dataset is large."
1048                         ),
1049                         fact_ids=["FACT_0001"],
1050                         support_type=SupportType.PARAPHRASE,
1051                     )
1052                 ],
1053             )
1054         ],
1055     )
1056
1057     output = materialise_writer_output(draft, ledger)
1058
1059     assert [
1060         support.sentence_text
1061         for support in output.sentence_support
1062     ] == [
1063         "The table contains 96,453 rows.",
1064         "The dataset is large.",
1065     ]
1066     assert output.selected_fact_ids == ["FACT_0001"]
1067     assert not validate_writer_output(output, ledger)
1068
1069
1070 def test_quality_warning_results_in_approved_with_warnings():
1071     quality = ReportQualityAssessment(
1072         status=QualityStatus.WARNING,
1073         request_responsiveness=0.8,
1074         finding_selection=0.8,
1075         coherence=0.9,
1076         concision=0.9,
1077         caveat_integration=0.8,
1078         data_science_interpretation=0.9,
1079         findings=["The report omits a requested overview."],
1080     )
1081
1082     status = decide_release_status(
1083         annotations=[],
1084         quality=quality,
1085         methodological_warnings=[],
1086         repair_budget_exhausted=False,
1087         audit_mode=AuditMode.INTERNAL,
1088     )
1089
1090     assert status == ReleaseStatus.APPROVED_WITH_WARNINGS
1091
1092
1093 def test_missing_required_component_is_quality_warning_not_human_review():
1094     ledger, evidence = make_fact_fixture()
1095     sentence = "The table contains 96,453 rows."
1096     writer = WriterOutput(
1097         title="Overview only",
1098         markdown=f"# Overview only\n\n{sentence}\n",
1099         sentence_support=[
1100             SentenceSupport(
1101                 sentence_id="SENT_0001",
1102                 sentence_text=sentence,
1103                 fact_ids=["FACT_0001"],
1104                 evidence_ids=["EVD_0001"],
1105                 support_type=SupportType.PARAPHRASE,
1106             )
1107         ],
1108         selected_fact_ids=["FACT_0001"],
1109     )
1110     spec = ReportSpecification(
1111         report_purpose="Describe the data.",
1112         target_length_words=300,
1113         maximum_main_findings=5,
1114         prioritisation_rule="Cover required components.",
1115         required_components=[
1116             ReportComponent.DATASET_OVERVIEW,
1117             ReportComponent.DATA_QUALITY,
1118         ],
1119     )
1120
1121     audit = deterministic_audit(
1122         writer,
1123         ledger,
1124         evidence,
1125         AuditMode.INTERNAL,
1126         [],
1127         0,
1128         spec,

```

```

1129     Settings(),
1130 )
1131
1132 assert audit.quality_assessment.status == QualityStatus.REVISE
1133 assert audit.release_status == ReleaseStatus.APPROVED_WITH_WARNINGS
1134
1135
1136 def test_imbalance_bias_wording_is_methodological_warning():
1137     evidence = EvidenceLedger(
1138         fingerprint="test",
1139         items=[
1140             EvidenceItem(
1141                 evidence_id="EVD_0001",
1142                 route=AnalysisRoute.ASSOCIATION_COMPARISON,
1143                 task_ids=["TASK_001"],
1144                 finding="Rain and snow groups have different mean temperatures.",
1145                 metrics={"rain_mean": 12.38, "snow_mean": -4.95},
1146                 source_tables=["weather"],
1147                 source_columns=["Temperature (°C)", "Precip Type"],
1148                 method="Unadjusted group comparison.",
1149                 validation_strategy=ValidationStrategy.NONE,
1150                 practical_interpretation=(
1151                     "The groups differ descriptively, and unequal group sizes "
1152                     "may affect precision and stability."
1153                 ),
1154                 strength_label="large_group_difference",
1155                 limitations=[],
1156                 prohibited_interpretations=[],
1157                 recommendations=[],
1158                 claim_permissions=[
1159                     ClaimPermission.DESRIPTIVE,
1160                     ClaimPermission.COMPARATIVE,
1161                 ],
1162                 factual_confidence=1.0,
1163                 methodological_strength=0.9,
1164                 user_relevance=1.0,
1165                 salience=1.0,
1166                 recommended_use=RecommendedUse.HEADLINE,
1167             ),
1168         ],
1169     )
1170     ledger = FactLedger(
1171         writer_ready_facts=[
1172             VerifiedFact(
1173                 fact_id="FACT_0001",
1174                 source_candidate_id="CAN_0001",
1175                 fact_summary=(
1176                     "Rain and snow groups have different mean temperatures."
1177                 ),
1178                 evidence_ids=["EVD_0001"],
1179                 structured_values={
1180                     "EVD_0001": {"rain_mean": 12.38, "snow_mean": -4.95}
1181                 },
1182                 entities=["Temperature (°C)", "Precip Type", "rain", "snow"],
1183                 claim_permissions=[
1184                     ClaimPermission.DESRIPTIVE,
1185                     ClaimPermission.COMPARATIVE,
1186                 ],
1187                 factual_confidence=1.0,
1188                 methodological_strength=0.9,
1189                 user_relevance=1.0,
1190                 salience=1.0,
1191                 recommended_use=RecommendedUse.HEADLINE,
1192             )
1193         ]
1194     )
1195     sentence = "The imbalanced group sizes may bias the observed means."
1196     writer = WriterOutput(
1197         title="Groups",
1198         markdown=f"# Groups\n\n{sentence}\n",
1199         sentence_support=[
1200             SentenceSupport(
1201                 sentence_id="SENT_0001",
1202                 sentence_text=sentence,
1203                 fact_ids=["FACT_0001"],
1204                 evidence_ids=["EVD_0001"],
1205                 support_type=SupportType.PARAPHRASE,
1206             )
1207         ],
1208         selected_fact_ids=["FACT_0001"],
1209     )
1210     spec = ReportSpecification(
1211         report_purpose="Describe group differences.",
1212         target_length_words=200,
1213         maximum_main_findings=3,
1214         prioritisation_rule="Use supported comparisons.",
1215     )
1216
1217     audit = deterministic_audit(
1218         writer,
1219         ledger,

```

```

1220     evidence,
1221     AuditMode.INTERNAL,
1222     [],
1223     0,
1224     spec,
1225     Settings(),
1226 )
1227
1228 assert any(
1229     annotation.subtype == "unsupported_methodological_interpretation"
1230     and annotation.severity == Severity.MEDIUM
1231     for annotation in audit.annotations
1232 )
1233 assert audit.release_status == ReleaseStatus.APPROVED_WITH_WARNINGS
1234
1235
1236 def test_precision_stability_imbalance_wording_is_allowed():
1237     ledger, evidence = make_fact_fixture()
1238     sentence = (
1239         "Unequal group sizes may affect precision and stability of the "
1240         "observed means."
1241     )
1242     fact = ledger.writer_ready_facts[0].model_copy(
1243         update={
1244             "fact_summary": sentence,
1245             "claim_permissions": [
1246                 ClaimPermission.DESCRPTIVE,
1247                 ClaimPermission.COMPARATIVE,
1248             ],
1249         }
1250     )
1251     ledger = FactLedger(writer_ready_facts=[fact])
1252     writer = WriterOutput(
1253         title="Groups",
1254         markdown=f"# Groups\n\n{sentence}\n",
1255         sentence_support=[
1256             SentenceSupport(
1257                 sentence_id="SENT_0001",
1258                 sentence_text=sentence,
1259                 fact_ids=["FACT_0001"],
1260                 evidence_ids=["EVD_0001"],
1261                 support_type=SupportType.PARAPHRASE,
1262             )
1263         ],
1264         selected_fact_ids=["FACT_0001"],
1265     )
1266     spec = ReportSpecification(
1267         report_purpose="Describe group differences.",
1268         target_length_words=200,
1269         maximum_main_findings=3,
1270         prioritisation_rule="Use supported comparisons.",
1271     )
1272
1273     audit = deterministic_audit(
1274         writer,
1275         ledger,
1276         evidence,
1277         AuditMode.INTERNAL,
1278         [],
1279         0,
1280         spec,
1281         Settings(),
1282     )
1283
1284     assert not [
1285         annotation
1286         for annotation in audit.annotations
1287         if annotation.subtype == "unsupported_methodological_interpretation"
1288     ]
1289
1290
1291 def test_unresolved_high_factual_error_requires_human_review():
1292     quality = ReportQualityAssessment(
1293         status=QualityStatus.PASS,
1294         request_responsiveness=1.0,
1295         finding_selection=1.0,
1296         coherence=1.0,
1297         concision=1.0,
1298         caveat_integration=1.0,
1299         data_science_interpretation=1.0,
1300     )
1301     status = decide_release_status(
1302         annotations=[
1303             AuditAnnotation(
1304                 annotation_id="ANN_0001",
1305                 sentence="The table has 12 rows.",
1306                 text_span="12",
1307                 error_type=ErrorType.INCORRECT_NUMBER,
1308                 subtype="unsupported_number",
1309                 severity=Severity.HIGH,
1310                 explanation="Wrong number.",

```

```

1311         correction_goal="Use the supported number.",
1312         confidence=0.95,
1313     )
1314 ],
1315     quality=quality,
1316     methodological_warnings=[],
1317     repair_budget_exhausted=True,
1318     audit_mode=AuditMode.INTERNAL,
1319 )
1320
1321 assert status == ReleaseStatus.HUMAN_REVIEW_REQUIRED
1322
1323
1324 def test_targeted_repair_preserves_unflagged_sentences():
1325     ledger, evidence = make_fact_fixture()
1326     bad_sentence = "The dataset contains 12 observations."
1327     good_sentence = "The table contains 96,453 rows."
1328     writer = WriterOutput(
1329         title="Test",
1330         markdown=f"# Test\n\n{bad_sentence}\n\n{good_sentence}\n",
1331         sentence_support=[
1332             SentenceSupport(
1333                 sentence_id="SENT_0001",
1334                 sentence_text=bad_sentence,
1335                 fact_ids=["FACT_0001"],
1336                 evidence_ids=["EVD_0001"],
1337                 support_type=SupportType.PARAPHRASE,
1338             ),
1339             SentenceSupport(
1340                 sentence_id="SENT_0002",
1341                 sentence_text=good_sentence,
1342                 fact_ids=["FACT_0001"],
1343                 evidence_ids=["EVD_0001"],
1344                 support_type=SupportType.DIRECT,
1345             ),
1346         ],
1347         selected_fact_ids=["FACT_0001"],
1348     )
1349     proposal = AuditRepairProposal(
1350         annotations=[],
1351         repairs=[
1352             SentenceRepair(
1353                 sentence_id="SENT_0001",
1354                 original_sentence=bad_sentence,
1355                 annotation_ids=[],
1356                 candidates=[
1357                     RepairCandidate(
1358                         repair_id="REP_001",
1359                         replacement_text="The dataset contains 96,453 observations.",
1360                         strategy=RepairStrategy.MINIMAL_CORRECTION,
1361                         supporting_fact_ids=["FACT_0001"],
1362                         supporting_evidence_ids=["EVD_0001"],
1363                         factual_support_score=1.0,
1364                         meaning_preservation_score=1.0,
1365                         readability_score=1.0,
1366                         residual_hallucination_risk=0.0,
1367                     )
1368                 ],
1369                 preferred_repair_id="REP_001",
1370                 selection_reason="Correct the number.",
1371             )
1372         ],
1373         recommended_decision=AuditDecision.REVISE,
1374         residual_risk="Repair required.",
1375         quality_assessment=ReportQualityAssessment(
1376             status=QualityStatus.PASS,
1377             request_responsiveness=1.0,
1378             finding_selection=1.0,
1379             coherence=1.0,
1380             concision=1.0,
1381             caveat_integration=1.0,
1382             data_science_interpretation=1.0,
1383         ),
1384     )
1385
1386     repaired, _ = apply_repair_proposal(writer, proposal, ledger, evidence)
1387
1388     assert good_sentence in repaired.markdown
1389
1390
1391 def test_deterministic_writer_fallback_is_not_primary_evaluation():
1392     ledger, evidence = make_fact_fixture()
1393     understanding = DataUnderstanding(
1394         profile_fingerprint="test",
1395         dataset_summary="Test.",
1396         tables=[],
1397     )
1398     plan = ExecutionPlan(
1399         objective="Describe the data.",
1400         tasks=[],
1401         route_order=[],

```

```

1402     report_specification=ReportSpecification(
1403         report_purpose="Describe the data.",
1404         target_length_words=300,
1405         maximum_main_findings=5,
1406         prioritisation_rule="Use verified facts.",
1407         required_components=[ReportComponent.DATASET_OVERVIEW],
1408     ),
1409     audit_mode=AuditMode.INTERNAL,
1410     revision_limit=1,
1411     maximum_facts=10,
1412     rationale="Test plan.",
1413 )
1414 pack = build_writer_evidence_pack(
1415     request="Describe the data.",
1416     understanding=understanding,
1417     plan=plan,
1418     evidence=evidence,
1419     fact_ledger=ledger,
1420     settings=Settings(),
1421 )
1422
1423 output = fallback_writer(pack)
1424
1425 assert output.writer_mode == "deterministic_fallback"
1426 assert not output.eligible_for_primary_evaluation
1427
1428 def test_event_writer_content_requirements_are_controller_enforced():
1429     def item(
1430         evidence_id: str,
1431         evidence_type: str,
1432         capability: EvidenceCapability,
1433         *,
1434         route: AnalysisRoute = AnalysisRoute.DESCRPTIVE,
1435     ) -> EvidenceItem:
1436         return EvidenceItem(
1437             evidence_id=evidence_id,
1438             route=route,
1439             task_ids=["TASK_EVENT"],
1440             capability=capability,
1441             evidence_type=evidence_type,
1442             finding=f"Supported {evidence_type}.",
1443             metrics={"value": evidence_id},
1444             source_tables=["event"],
1445             source_columns=[evidence_type],
1446             method="Validated event evidence.",
1447             practical_interpretation="Direct event evidence.",
1448             strength_label=evidence_type,
1449             claim_permissions=[
1450                 ClaimPermission.DESCRPTIVE,
1451                 ClaimPermission.COMPARATIVE,
1452             ],
1453             factual_confidence=1.0,
1454             methodological_strength=1.0,
1455             user_relevance=0.9,
1456             salience=0.9,
1457             recommended_use=RecommendedUse.MAIN_FINDING,
1458         )
1459
1460 evidence = EvidenceLedger(
1461     fingerprint="event-content",
1462     items=[
1463         item(
1464             "EVD_RESULT",
1465             "event_outcome",
1466             EvidenceCapability.EVENT_OUTCOME,
1467         ),
1468         item(
1469             "EVD_CONTEXT",
1470             "event_context",
1471             EvidenceCapability.DATASET_PROFILE,
1472         ),
1473         item(
1474             "EVD_RANKING_1",
1475             "entity_ranking",
1476             EvidenceCapability.RANKING,
1477         ),
1478         item(
1479             "EVD_RANKING_2",
1480             "entity_ranking",
1481             EvidenceCapability.RANKING,
1482         ),
1483         item(
1484             "EVD_CONTRAST_1",
1485             "participant_comparison",
1486             EvidenceCapability.GROUP_COMPARISON,
1487             route=AnalysisRoute.ASSOCIATION_COMPARISON,
1488         ),
1489         item(
1490             "EVD_CONTRAST_2",
1491             "participant_comparison",

```



```

1493         EvidenceCapability.GROUP_COMPARISON,
1494         route=AnalysisRoute.ASSOCIATION_COMPARISON,
1495     ),
1496 ],
1497 )
1498 facts = [
1499     VerifiedFact(
1500         fact_id=f"FACT_{index:04d}",
1501         source_candidate_id=f"CAN_{index:04d}",
1502         fact_summary=evidence_item.finding,
1503         evidence_ids=[evidence_item.evidence_id],
1504         source_capabilities=[evidence_item.capability],
1505         structured_values={
1506             evidence_item.evidence_id: evidence_item.metrics,
1507         },
1508         entities=["event"],
1509         claim_permissions=evidence_item.claim_permissions,
1510         allowed_interpretations=[
1511             evidence_item.practical_interpretation,
1512         ],
1513         factual_confidence=1.0,
1514         methodological_strength=1.0,
1515         user_relevance=0.9,
1516         salience=0.9,
1517         recommended_use=RecommendedUse.MAIN_FINDING,
1518     )
1519     for index, evidence_item in enumerate(evidence.items, start=1)
1520 ]
1521 ledger = FactLedger(writer_ready_facts=facts)
1522 insight = VerifiedInsight(
1523     insight_id="INS_EVENT_SYNTHESIS",
1524     statement=(
1525         "The supported result, rankings and participant contrasts form "
1526         "a bounded event narrative."
1527     ),
1528     insight_type=InsightType.NARRATIVE_SUMMARY,
1529     interpretation_level=InterpretationLevel.BOUNDED_INSIGHT,
1530     source_fact_ids=[fact.fact_id for fact in facts],
1531     source_evidence_ids=[
1532         item.evidence_id
1533         for item in evidence.items
1534     ],
1535     source_capabilities=[
1536         EvidenceCapability.EVENT_OUTCOME,
1537         EvidenceCapability.RANKING,
1538         EvidenceCapability.GROUP_COMPARISON,
1539     ],
1540     why_it_matters=(
1541         "It connects the event result with rankings and contrasts."
1542     ),
1543     claim_permissions=[
1544         ClaimPermission.DESCRPTIVE,
1545         ClaimPermission.COMPARATIVE,
1546     ],
1547     confidence=0.95,
1548     salience=0.95,
1549     verification_status=InsightVerificationStatus.VERIFIED,
1550 )
1551 spec = ReportSpecification(
1552     report_purpose="Write an event report.",
1553     genre=ReportGenre.EVENT_REPORT,
1554     target_length_words=500,
1555     required_components=[],
1556     required_content_slots=[
1557         "event_result",
1558         "event_context",
1559         "leading_performance",
1560         "main_contrast",
1561         "scope_limitations",
1562     ],
1563     optional_content_slots=[],
1564     prioritisation_rule="Use supported event material.",
1565 )
1566 requirements = build_writer_content_requirements(
1567     report_specification=spec,
1568     fact_ledger=ledger,
1569     evidence=evidence,
1570     insight_ledger=InsightLedger(
1571         verified_insights=[insight],
1572     ),
1573     settings=Settings(),
1574 )
1575
1576 assert requirements["enforce_minimum_words"]
1577 assert requirements["minimum_word_count"] == 225
1578 assert requirements["word_count_validation"] == "quality"
1579 assert requirements["narrative_validation"] == "quality"
1580 assert requirements["content_unit_validation"] == "quality"
1581 assert requirements["narrative_requirements"] == {
1582     "enforce": True,
1583     "minimum_synthesis_sentences": 2,

```

```

1584         "minimum_insight_sentences": 1,
1585         "minimum_connective_sentences": 1,
1586         "minimum_scope_limitations": 1,
1587     }
1588
1589     incomplete_errors = content_requirement_errors(
1590         used_fact_ids={
1591             "FACT_0001",
1592             "FACT_0002",
1593             "FACT_0003",
1594             "FACT_0005",
1595         },
1596         used_insight_ids=set(),
1597         word_count=130,
1598         requirements=requirements,
1599         narrative_stats={
1600             "synthesis_sentences": 0,
1601             "insight_sentences": 0,
1602             "connective_sentences": 0,
1603             "scope_limitation_sentences": 0,
1604         },
1605     )
1606
1607     quality_errors = content_requirement_errors(
1608         used_fact_ids={
1609             "FACT_0001",
1610             "FACT_0002",
1611             "FACT_0003",
1612             "FACT_0005",
1613         },
1614         used_insight_ids=set(),
1615         word_count=130,
1616         requirements=requirements,
1617         narrative_stats={
1618             "synthesis_sentences": 0,
1619             "insight_sentences": 0,
1620             "connective_sentences": 0,
1621             "scope_limitation_sentences": 0,
1622         },
1623         respect_validation_severity=False,
1624     )
1625
1626     assert incomplete_errors == []
1627     assert any("at least 225 words" in error for error in quality_errors)
1628     assert any("leading performance" in error for error in quality_errors)
1629     assert any("main contrast" in error for error in quality_errors)
1630     assert any("multi-fact" in error for error in quality_errors)
1631     assert any("verified insight-backed" in error for error in quality_errors)
1632     assert any("event-scoped limitation" in error for error in quality_errors)
1633
1634     narrative_support = [
1635         SentenceSupport(
1636             sentence_id="SENT_SYNTHESIS_1",
1637             sentence_text=(
1638                 "The result and context frame the supplied event while "
1639                 "rankings identify the leading participants."
1640             ),
1641             fact_ids=["FACT_0001", "FACT_0002", "FACT_0003"],
1642             evidence_ids=["EVD_RESULT", "EVD_CONTEXT", "EVD_RANKING_1"],
1643             insight_ids=["INS_EVENT_SYNTHESIS"],
1644             interpretation_level=InterpretationLevel.BOUNDED_INSIGHT,
1645             support_type=SupportType.MULTI_FACT_SYNTHESIS,
1646         ),
1647         SentenceSupport(
1648             sentence_id="SENT_SYNTHESIS_2",
1649             sentence_text=(
1650                 "Participant contrasts describe the supplied event only "
1651                 "and do not establish why the result occurred."
1652             ),
1653             fact_ids=["FACT_0005", "FACT_0006"],
1654             evidence_ids=["EVD_CONTRAST_1", "EVD_CONTRAST_2"],
1655             support_type=SupportType.MULTI_FACT_SYNTHESIS,
1656         ),
1657     ]
1658     assert not content_requirement_errors(
1659         used_fact_ids={fact.fact_id for fact in facts},
1660         used_insight_ids={insight.insight_id},
1661         word_count=230,
1662         requirements=requirements,
1663         narrative_stats=sentence_support_narrative_stats(
1664             narrative_support
1665         ),
1666     )
1667
1668
1669     def test_writer_materialisation_maps_each_compound_sentence_fragment():
1670         ledger, _ = make_fact_fixture()
1671         first = "The table contains 96,453 rows."
1672         second = "The dataset contains more than 96,000 observations."
1673         draft = WriterAgentDraft(
1674             title="Supported report",

```

```

1675     sections=[
1676         WriterSectionDraft(
1677             heading="Overview",
1678             sentences=[
1679                 WriterSentenceDraft(
1680                     text=f"{{first}} {{second}}",
1681                     fact_ids=["FACT_0001"],
1682                     support_type=SupportType.PARAPHRASE,
1683                 )
1684             ],
1685         )
1686     ],
1687 )
1688
1689 output = materialise_writer_output(draft, ledger)
1690
1691 assert [
1692     support.sentence_text
1693     for support in output.sentence_support
1694 ] == [first, second]
1695
1696
1697 def test_fallback_writer_maps_each_compound_limitation_fragment():
1698     ledger, evidence = make_fact_fixture()
1699     fact = ledger.writer_ready_facts[0].model_copy(
1700         update={
1701             "claim_permissions": [
1702                 ClaimPermission.ASSOCIATIONAL,
1703             ]
1704         }
1705     )
1706     understanding = DataUnderstanding(
1707         profile_fingerprint="test",
1708         dataset_summary="Test.",
1709         tables=[],
1710     )
1711     plan = ExecutionPlan(
1712         objective="Describe the data.",
1713         tasks=[],
1714         route_order=[],
1715         report_specification=ReportSpecification(
1716             report_purpose="Describe the data.",
1717             target_length_words=300,
1718             maximum_main_findings=5,
1719             prioritisation_rule="Use verified facts.",
1720             required_components=[ReportComponent.DATASET_OVERVIEW],
1721         ),
1722         audit_mode=AuditMode.INTERNAL,
1723         revision_limit=1,
1724         maximum_facts=10,
1725         rationale="Test plan.",
1726     )
1727     pack = build_writer_evidence_pack(
1728         request="Describe the data.",
1729         understanding=understanding,
1730         plan=plan,
1731         evidence=evidence,
1732         fact_ledger=FactLedger(writer_ready_facts=[fact]),
1733         settings=Settings(),
1734     )
1735     limitation = (
1736         "Observed associations are descriptive. "
1737         "They are not evidence of causal effects."
1738     )
1739     pack = pack.model_copy(
1740         update={
1741             "priority_facts": [fact],
1742             "reader_facing_limitations": [limitation],
1743         }
1744     )
1745
1746     output = fallback_writer(pack)
1747     mapped_sentences = {
1748         support.sentence_text
1749         for support in output.sentence_support
1750     }
1751
1752     assert set(split_markdown_sentences(limitation)).issubset(
1753         mapped_sentences
1754     )
1755
1756
1757 def test_repair_rejects_identical_replacement_text():
1758     ledger, evidence = make_fact_fixture()
1759     sentence = "The table contains 96,453 rows."
1760     candidate = RepairCandidate(
1761         repair_id="REP_NOOP",
1762         replacement_text=sentence,
1763         strategy=RepairStrategy.MINIMAL_CORRECTION,
1764         supporting_fact_ids=["FACT_0001"],
1765         supporting_evidence_ids=["EVD_0001"],

```

```

1766         factual_support_score=1.0,
1767         meaning_preservation_score=1.0,
1768         readability_score=1.0,
1769         residual_hallucination_risk=0.0,
1770     )
1771
1772     errors = validate_repair_candidate(
1773         candidate,
1774         ledger,
1775         evidence,
1776         original_text=sentence,
1777     )
1778
1779     assert errors == [
1780         "The replacement is identical to the original sentence."
1781     ]
1782
1783
1784 def test_repair_maps_every_fragment_of_compound_replacement():
1785     ledger, evidence = make_fact_fixture()
1786     original = "The dataset contains 12 observations."
1787     first = "The table contains 96,453 rows."
1788     second = "The dataset contains more than 96,000 observations."
1789     writer = WriterOutput(
1790         title="Test",
1791         markdown=f"# Test\n\n{original}\n",
1792         sentence_support=[
1793             SentenceSupport(
1794                 sentence_id="SENT_0001",
1795                 sentence_text=original,
1796                 fact_ids=["FACT_0001"],
1797                 evidence_ids=["EVD_0001"],
1798                 support_type=SupportType.PARAPHRASE,
1799             )
1800         ],
1801         selected_fact_ids=["FACT_0001"],
1802     )
1803     candidate = RepairCandidate(
1804         repair_id="REP_COMPOUND",
1805         replacement_text=f"{first} {second}",
1806         strategy=RepairStrategy.MINIMAL_CORRECTION,
1807         supporting_fact_ids=["FACT_0001"],
1808         supporting_evidence_ids=["EVD_0001"],
1809         factual_support_score=1.0,
1810         meaning_preservation_score=1.0,
1811         readability_score=1.0,
1812         residual_hallucination_risk=0.0,
1813     )
1814     proposal = AuditRepairProposal(
1815         repairs=[
1816             SentenceRepair(
1817                 sentence_id="SENT_0001",
1818                 original_sentence=original,
1819                 annotation_ids=[],
1820                 candidates=[candidate],
1821                 preferred_repair_id=candidate.repair_id,
1822                 selection_reason="Correct and clarify the supported count.",
1823             )
1824         ],
1825         recommended_decision=AuditDecision.REVISE,
1826         residual_risk="Repair required.",
1827         quality_assessment=(
1828             make_passing_audit_report().quality_assessment
1829         ),
1830     )
1831
1832     repaired, patches = apply_repair_proposal(
1833         writer,
1834         proposal,
1835         ledger,
1836         evidence,
1837     )
1838
1839     assert len(patches) == 1
1840     assert [
1841         support.sentence_text
1842         for support in repaired.sentence_support
1843     ] == [first, second]
1844     assert len(
1845         {
1846             support.sentence_id
1847             for support in repaired.sentence_support
1848         }
1849     ) == 2
1850
1851
1852 def test_writer_recovers_one_unambiguous_missing_insight_id():
1853     insight = VerifiedInsight(
1854         insight_id="INS_RECOVER",
1855         statement=(
1856             "The verified findings form one bounded pattern."

```

```

1857         ),
1858         insight_type=InsightType.NARRATIVE_SUMMARY,
1859         interpretation_level=InterpretationLevel.BOUNDED_INSIGHT,
1860         source_fact_ids=["FACT_0001"],
1861         source_evidence_ids=["EVD_0001"],
1862         why_it_matters="It provides a bounded interpretation.",
1863         claim_permissions=[ClaimPermission.DESCRPTIVE],
1864         confidence=0.95,
1865         salience=0.90,
1866         verification_status=InsightVerificationStatus.VERIFIED,
1867     )
1868     draft = WriterAgentDraft(
1869         title="Insight report",
1870         sections=[
1871             WriterSectionDraft(
1872                 heading="Finding",
1873                 sentences=[
1874                     WriterSentenceDraft(
1875                         text=insight.statement,
1876                         fact_ids=["FACT_0001"],
1877                         interpretation_level=(
1878                             InterpretationLevel.BOUNDED_INSIGHT
1879                         ),
1880                         support_type=SupportType.MULTI_FACT_SYNTHESIS,
1881                     )
1882                 ],
1883             )
1884         ],
1885     )
1886     recovered = recover_missing_writer_insight_ids(
1887         draft,
1888         {insight.insight_id: insight},
1889     )
1890
1891     assert recovered.sections[0].sentences[0].insight_ids == [
1892         insight.insight_id
1893     ]
1894
1895
1896
1897 # =====
1898 # REPORT-COVERAGE REGRESSION TESTS
1899 # =====
1900
1901
1902 def _coverage_evidence_item(
1903     *,
1904     evidence_id,
1905     finding,
1906     route,
1907     metrics,
1908     strength_label,
1909     recommended_use,
1910     permissions,
1911     relevance=0.95,
1912     salience=0.95,
1913 ):
1914     return EvidenceItem(
1915         evidence_id=evidence_id,
1916         route=route,
1917         task_ids=["TASK_COVERAGE"],
1918         finding=finding,
1919         metrics=metrics,
1920         source_tables=["weather"],
1921         source_columns=list(
1922             metrics.get(
1923                 "source_columns",
1924                 [],
1925             )
1926         ),
1927         method="Deterministic test evidence.",
1928         validation_strategy=ValidationStrategy.NONE,
1929         practical_interpretation=finding,
1930         strength_label=strength_label,
1931         limitations=[],
1932         prohibited_interpretations=[],
1933         recommendations=[],
1934         claim_permissions=permissions,
1935         factual_confidence=1.0,
1936         methodological_strength=0.95,
1937         user_relevance=relevance,
1938         salience=salience,
1939         recommended_use=recommended_use,
1940         eligible_for_writer=True,
1941     )
1942
1943
1944 def _coverage_fact(
1945     item,
1946     fact_id,
1947 ):
```

```

1948     return VerifiedFact(
1949         fact_id=fact_id,
1950         source_candidate_id=(
1951             f"CAN_{fact_id}"
1952         ),
1953         fact_summary=item.finding,
1954         evidence_ids=[item.evidence_id],
1955         structured_values={
1956             item.evidence_id: item.metrics
1957         },
1958         entities=[
1959             "weather",
1960             *item.source_columns,
1961         ],
1962         claim_permissions=(
1963             item.claim_permissions
1964         ),
1965         factual_confidence=(
1966             item.factual_confidence
1967         ),
1968         methodological_strength=(
1969             item.methodological_strength
1970         ),
1971         user_relevance=item.user_relevance,
1972         salience=item.salience,
1973         recommended_use=item.recommended_use,
1974     )
1975
1976
1977 def _coverage_fixture():
1978     overview = _coverage_evidence_item(
1979         evidence_id="EVD_COV_001",
1980         finding=(
1981             "Table `weather` contains 96,453 "
1982             "rows and 12 columns."
1983         ),
1984         route=AnalysisRoute.DESRIPTIVE,
1985         metrics={
1986             "row_count": 96_453,
1987             "column_count": 12,
1988         },
1989         strength_label="dataset_overview",
1990         recommended_use=RecommendedUse.HEADLINE,
1991         permissions=[
1992             ClaimPermission.DESRIPTIVE
1993         ],
1994     )
1995
1996     quality = _coverage_evidence_item(
1997         evidence_id="EVD_COV_002",
1998         finding=(
1999             "`Loud Cover` is constant at `0` "
2000             "across all observations."
2001         ),
2002         route=AnalysisRoute.DESRIPTIVE,
2003         metrics={
2004             "constant": True,
2005             "constant_value": 0,
2006         },
2007         strength_label="constant_column",
2008         recommended_use=(
2009             RecommendedUse.MAIN_FINDING
2010         ),
2011         permissions=[
2012             ClaimPermission.DESRIPTIVE,
2013             ClaimPermission.METHODOLOGICAL,
2014         ],
2015     )
2016
2017     correlation = _coverage_evidence_item(
2018         evidence_id="EVD_COV_003",
2019         finding=(
2020             "Temperature (C)` and "
2021             "`Apparent Temperature (C)` have "
2022             "a Pearson correlation of 0.9926."
2023         ),
2024         route=(
2025             AnalysisRoute
2026             .ASSOCIATION_COMPARISON
2027         ),
2028         metrics={
2029             "pearson_r": 0.9926,
2030             "complete_pairs": 96_453,
2031         },
2032         strength_label=(
2033             "very_strong_association"
2034         ),
2035         recommended_use=(
2036             RecommendedUse.MAIN_FINDING
2037         ),
2038         permissions=[

```

```

2039         ClaimPermission.ASSOCIATIONAL,
2040         ClaimPermission.METHODOLOGICAL,
2041     ],
2042 )
2043
2044 large_group = _coverage_evidence_item(
2045     evidence_id="EVD_COV_004",
2046     finding=(
2047         "Rain observations have a mean "
2048         "temperature of 12.36 compared with "
2049         "-4.97 for snow, a difference of "
2050         "17.33."
2051     ),
2052     route=(
2053         AnalysisRoute
2054         .ASSOCIATION_COMPARISON
2055     ),
2056     metrics={
2057         "highest_group": {
2058             "group": "rain",
2059             "mean": 12.36,
2060         },
2061         "lowest_group": {
2062             "group": "snow",
2063             "mean": -4.97,
2064         },
2065         "difference": 17.33,
2066         "standardised_difference": 1.0,
2067     },
2068     strength_label=(
2069         "large_group_difference"
2070     ),
2071     recommended_use=(
2072         RecommendedUse.MAIN_FINDING
2073     ),
2074     permissions=[
2075         ClaimPermission.COMPARATIVE,
2076         ClaimPermission.METHODOLOGICAL,
2077     ],
2078 )
2079
2080 small_group = _coverage_evidence_item(
2081     evidence_id="EVD_COV_005",
2082     finding=(
2083         "Rain observations have a mean wind "
2084         "speed of 10.97 compared with 9.482 "
2085         "for snow, a difference of 1.489."
2086     ),
2087     route=(
2088         AnalysisRoute
2089         .ASSOCIATION_COMPARISON
2090     ),
2091     metrics={
2092         "highest_group": {
2093             "group": "rain",
2094             "mean": 10.97,
2095         },
2096         "lowest_group": {
2097             "group": "snow",
2098             "mean": 9.482,
2099         },
2100         "difference": 1.489,
2101         "standardised_difference": 0.22,
2102     },
2103     strength_label=(
2104         "small_group_difference"
2105     ),
2106     recommended_use=(
2107         RecommendedUse.MAIN_FINDING
2108     ),
2109     permissions=[
2110         ClaimPermission.COMPARATIVE,
2111         ClaimPermission.METHODOLOGICAL,
2112     ],
2113     relevance=0.80,
2114     salience=0.75,
2115 )
2116
2117 evidence = EvidenceLedger(
2118     fingerprint="coverage-test",
2119     items=[
2120         overview,
2121         quality,
2122         correlation,
2123         large_group,
2124         small_group,
2125     ],
2126 )
2127
2128 facts = {
2129     item.evidence_id: _coverage_fact(

```

```

2130         item,
2131         f"FACT_COV_{index:03d}",
2132     )
2133     for index, item in enumerate(
2134         evidence.items,
2135         start=1,
2136     )
2137 }
2138
2139 return evidence, facts
2140
2141
2142 def test_report_coverage_recovery_regression():
2143     from table2text.audit import (
2144         augment_fact_ledger_for_report_coverage,
2145     )
2146     from table2text.schemas import (
2147         VerificationMethod,
2148     )
2149
2150     evidence, facts = _coverage_fixture()
2151
2152     thin_ledger = FactLedger(
2153         writer_ready_facts=[
2154             facts["EVD_COV_005"]
2155         ]
2156     )
2157
2158     recovered = (
2159         augment_fact_ledger_for_report_coverage(
2160             fact_ledger=thin_ledger,
2161             evidence=evidence,
2162             required_components=[
2163                 ReportComponent.DATASET_OVERVIEW,
2164                 ReportComponent.DATA_QUALITY,
2165                 ReportComponent.STRONGEST_RELATIONSHIPS,
2166                 ReportComponent.LIMITATIONS_NEXT_STEPS,
2167             ],
2168             settings=Settings(),
2169         )
2170     )
2171
2172     assert (
2173         len(recovered.writer_ready_facts)
2174         > len(thin_ledger.writer_ready_facts)
2175     )
2176
2177     assert (
2178         recovered
2179         .deterministically_recovered_fact_ids
2180     )
2181
2182     recovered_facts = [
2183         fact
2184         for fact in recovered.writer_ready_facts
2185         if fact.fact_id
2186         in recovered
2187         .deterministically_recovered_fact_ids
2188     ]
2189
2190     assert recovered_facts
2191
2192     assert all(
2193         fact.verification_method
2194         == VerificationMethod
2195         .DETERMINISTIC_EVIDENCE_RECOVERY
2196         for fact in recovered_facts
2197     )
2198
2199     represented = {
2200         evidence_id
2201         for fact in recovered.writer_ready_facts
2202         for evidence_id in fact.evidence_ids
2203     }
2204
2205     assert "EVD_COV_001" in represented
2206     assert "EVD_COV_002" in represented
2207     assert "EVD_COV_003" in represented
2208     assert "EVD_COV_004" in represented
2209
2210
2211 def test_priority_selection_never_refills_with_small_effect():
2212     from table2text.audit import (
2213         select_balanced_priority_facts,
2214     )
2215
2216     evidence, facts = _coverage_fixture()
2217
2218     ledger = FactLedger(
2219         writer_ready_facts=list(
2220             facts.values()

```



```

2221     )
2222 )
2223
2224 selected = (
2225     select_balanced_priority_facts(
2226         facts=ledger.writer_ready_facts,
2227         evidence=evidence,
2228         required_components=[
2229             ReportComponent.DATASET_OVERVIEW,
2230             ReportComponent.DATA_QUALITY,
2231             ReportComponent.STRONGEST_RELATIONSHIPS,
2232         ],
2233         settings=Settings(),
2234     )
2235 )
2236
2237 selected_evidence_ids = {
2238     evidence_id
2239     for fact in selected
2240     for evidence_id in fact.evidence_ids
2241 }
2242
2243 assert "EVD_COV_001" in selected_evidence_ids
2244 assert "EVD_COV_002" in selected_evidence_ids
2245 assert "EVD_COV_003" in selected_evidence_ids
2246 assert "EVD_COV_004" in selected_evidence_ids
2247 assert "EVD_COV_005" not in selected_evidence_ids
2248
2249
2250 def test_minimum_report_words_never_exceeds_target():
2251     from table2text.audit import (
2252         minimum_useful_report_words,
2253     )
2254
2255     minimum = minimum_useful_report_words(
2256         target_words=150,
2257         required_component_count=4,
2258         settings=Settings(),
2259     )
2260
2261     assert minimum <= 150
2262     assert minimum > 0
2263
2264
2265 def test_recovered_balanced_fallback_is_not_two_sentence_report():
2266     from table2text.audit import (
2267         augment_fact_ledger_for_report_coverage,
2268     )
2269
2270     evidence, facts = _coverage_fixture()
2271
2272     thin_ledger = FactLedger(
2273         writer_ready_facts=[
2274             facts["EVD_COV_005"]
2275         ]
2276     )
2277
2278     ledger = (
2279         augment_fact_ledger_for_report_coverage(
2280             fact_ledger=thin_ledger,
2281             evidence=evidence,
2282             required_components=[
2283                 ReportComponent.DATASET_OVERVIEW,
2284                 ReportComponent.DATA_QUALITY,
2285                 ReportComponent.STRONGEST_RELATIONSHIPS,
2286                 ReportComponent.LIMITATIONS_NEXT_STEPS,
2287             ],
2288             settings=Settings(),
2289         )
2290     )
2291
2292     understanding = DataUnderstanding(
2293         profile_fingerprint="coverage-test",
2294         dataset_summary=(
2295             "Weather observations."
2296         ),
2297         tables=[],
2298     )
2299
2300     plan = ExecutionPlan(
2301         objective=(
2302             "Understand the weather dataset and "
2303             "report its strongest findings."
2304         ),
2305         tasks=[],
2306         route_order=[],
2307         report_specification=ReportSpecification(
2308             report_purpose=(
2309                 "Understand the weather dataset."
2310             ),
2311             target_length_words=300,

```

```

2312         maximum_main_findings=8,
2313         prioritisation_rule=(
2314             "Cover required components using "
2315             "the strongest evidence."
2316         ),
2317         required_components=[
2318             ReportComponent.DATASET_OVERVIEW,
2319             ReportComponent.DATA_QUALITY,
2320             ReportComponent.STRONGEST_RELATIONSHIPS,
2321             ReportComponent.LIMITATIONS_NEXT_STEPS,
2322         ],
2323     ),
2324     audit_mode=AuditMode.INTERNAL,
2325     revision_limit=1,
2326     maximum_facts=20,
2327     rationale="Regression test.",
2328 )
2329
2330 pack = build_writer_evidence_pack(
2331     request=plan.objective,
2332     understanding=understanding,
2333     plan=plan,
2334     evidence=evidence,
2335     fact_ledger=ledger,
2336     settings=Settings(),
2337 )
2338
2339 output = fallback_writer(pack)
2340
2341 assert "## Dataset overview" in output.markdown
2342 assert "## Data quality" in output.markdown
2343 assert (
2344     "## Strongest observed relationships"
2345     in output.markdown
2346 )
2347 assert "1.489" not in output.markdown
2348 assert len(output.sentence_support) >= 4
2349 assert (
2350     output.writer_mode
2351     == "deterministic_fallback"
2352 )
2353 assert not output.eligible_for_primary_evaluation
2354
2355 # =====
2356 # PROFILE-SUPPORT AND AUDIT-AUTHORITY REGRESSION TESTS
2357 # =====
2358
2359
2360 def _profile_authority_fixture() -> DataProfile:
2361     return DataProfile(
2362         fingerprint="profile-authority",
2363         source_paths=["memory.csv"],
2364         tables=[
2365             TableProfile(
2366                 table_name="weather",
2367                 source_path="memory.csv",
2368                 row_count=4,
2369                 column_count=4,
2370                 duplicate_row_count=1,
2371                 candidate_keys=["Timestamp"],
2372                 columns=[
2373                     ColumnProfile(
2374                         name="Timestamp",
2375                         dtype="object",
2376                         semantic_type="datetime",
2377                         missing_count=0,
2378                         missing_rate=0.0,
2379                         unique_count=4,
2380                         sample_values=[
2381                             "2020-01-01 00:00",
2382                             "2020-01-01 01:00",
2383                         ],
2384                         datetime_parse_rate=1.0,
2385                         candidate_key=True,
2386                     ),
2387                     ColumnProfile(
2388                         name="Constant",
2389                         dtype="int64",
2390                         semantic_type="numeric",
2391                         missing_count=0,
2392                         missing_rate=0.0,
2393                         unique_count=1,
2394                         sample_values=["0"],
2395                         numeric_summary={
2396                             "count": 4,
2397                             "mean": 0.0,
2398                             "minimum": 0.0,
2399                             "maximum": 0.0,
2400                         },
2401                         constant=True,
2402

```

```

2403         ),
2404         ColumnProfile(
2405             name="Pressure",
2406             dtype="float64",
2407             semantic_type="numeric",
2408             missing_count=0,
2409             missing_rate=0.0,
2410             unique_count=3,
2411             numeric_summary={
2412                 "count": 4,
2413                 "mean": 750.0,
2414                 "median": 1000.0,
2415                 "minimum": 0.0,
2416                 "maximum": 1010.0,
2417                 "zero_count": 1,
2418                 "zero_rate": 0.25,
2419             },
2420             suspicious_zero_values=True,
2421             possible_sentinel_values=True,
2422             zero_risk=ZeroRisk.POSSIBLE_SENTINEL,
2423             zero_risk_reason=(
2424                 "Zero is separated from positive pressure values."
2425             ),
2426         ),
2427         ColumnProfile(
2428             name="Precip Type",
2429             dtype="object",
2430             semantic_type="categorical",
2431             missing_count=1,
2432             missing_rate=0.25,
2433             unique_count=2,
2434         ),
2435     ],
2436 ),
2437 ],
2438 )
2439
2440 def _basic_spec() -> ReportSpecification:
2441     return ReportSpecification(
2442         report_purpose="Understand the dataset.",
2443         target_length_words=300,
2444         maximum_main_findings=5,
2445         prioritisation_rule="Use supported facts.",
2446     )
2447
2448
2449 def _audit_sentence(
2450     sentence: str,
2451     *,
2452     support: SentenceSupport | None = None,
2453     profile_records=None,
2454     ledger: FactLedger | None = None,
2455     evidence: EvidenceLedger | None = None,
2456 ):
2457     ledger = ledger or make_fact_fixture()[0]
2458     evidence = evidence or make_fact_fixture()[1]
2459     support = support or SentenceSupport(
2460         sentence_id="SENT_0001",
2461         sentence_text=sentence,
2462         fact_ids=["FACT_0001"],
2463         evidence_ids=["EVD_0001"],
2464         support_type=SupportType.PARAPHRASE,
2465     )
2466
2467     output = WriterOutput(
2468         title="Audit",
2469         markdown=f"# Audit\n\n{sentence}\n",
2470         sentence_support=[support],
2471         selected_fact_ids=support.fact_ids,
2472     )
2473     return deterministic_audit(
2474         output,
2475         ledger,
2476         evidence,
2477         AuditMode.INTERNAL,
2478         [],
2479         0,
2480         _basic_spec(),
2481         Settings(),
2482         profile_support_records=profile_records or [],
2483     ), output
2484
2485
2486 def test_profile_support_registry_contains_structural_records():
2487     records = build_profile_support_registry(
2488         _profile_authority_fixture()
2489     )
2490     by_kind = {record.fact_kind for record in records}
2491
2492     assert "table_dimensions" in by_kind
2493     assert "duplicate_rows" in by_kind

```

```

2494     assert "constant_column" in by_kind
2495     assert "column_missingness" in by_kind
2496     assert "numeric_summary" in by_kind
2497     assert "zero_diagnostic" in by_kind
2498     assert "datetime_presence" in by_kind
2499     assert "candidate_key" in by_kind
2500
2501
2502 def test_execute_plan_emits_duplicate_row_evidence(tmp_path):
2503     path = tmp_path / "dups.csv"
2504     pd.DataFrame(
2505         {
2506             "a": [1, 1, 2],
2507             "b": ["x", "x", "y"],
2508         }
2509     ).to_csv(path, index=False)
2510     bundle = load_data([path])
2511     plan = ExecutionPlan(
2512         objective="Describe duplicates.",
2513         tasks=[
2514             InvestigationTask(
2515                 task_id="TASK_DUP",
2516                 question="Check duplicate rows.",
2517                 route=AnalysisRoute.DESRIPTIVE,
2518                 priority=1,
2519                 table_name="dups",
2520                 claim_permissions=[
2521                     ClaimPermission.DESRIPTIVE
2522                 ],
2523                 answerability_note="Deterministic profile.",
2524             )
2525         ],
2526         route_order=[AnalysisRoute.DESRIPTIVE],
2527         report_specification=_basic_spec(),
2528         audit_mode=AuditMode.INTERNAL,
2529         revision_limit=1,
2530         maximum_facts=20,
2531         rationale="Test.",
2532     )
2533
2534     evidence = execute_plan(bundle, plan, Settings())
2535     duplicate_items = [
2536         item
2537         for item in evidence.items
2538         if item.strength_label == "duplicate_rows"
2539     ]
2540
2541     assert len(duplicate_items) == 1
2542     assert (
2543         duplicate_items[0]
2544         .metrics["duplicate_row_count"]
2545         == 1
2546     )
2547
2548
2549 def test_profile_supported_unmapped_number_creates_hidden_patch():
2550     records = build_profile_support_registry(
2551         _profile_authority_fixture()
2552     )
2553     sentence = "The dataset contains 1 exact duplicate row."
2554     audit, output = _audit_sentence(
2555         sentence,
2556         profile_records=records,
2557     )
2558
2559     assert any(
2560         annotation.error_type
2561         == ErrorType.SUPPORT_MAPPING_ERROR
2562         and annotation.severity == Severity.MEDIUM
2563         for annotation in audit.annotations
2564     )
2565     assert not any(
2566         annotation.subtype == "unsupported_number"
2567         and annotation.severity == Severity.HIGH
2568         for annotation in audit.annotations
2569     )
2570     assert audit.support_map_patches
2571
2572     patched = apply_support_map_patches(
2573         output,
2574         audit.support_map_patches,
2575         {record.support_id for record in records},
2576     )
2577
2578     assert patched.markdown == output.markdown
2579     assert (
2580         patched.sentence_support[0].sentence_text
2581         == sentence
2582     )
2583     assert patched.sentence_support[0].profile_support_ids
2584

```

```

2585     post_audit = deterministic_audit(
2586         patched,
2587         make_fact_fixture()[0],
2588         make_fact_fixture()[1],
2589         AuditMode.INTERNAL,
2590         [],
2591         0,
2592         _basic_spec(),
2593         Settings(),
2594         profile_support_records=records,
2595     )
2596
2597     assert not any(
2598         annotation.error_type
2599         == ErrorType.SUPPORT_MAPPING_ERROR
2600         for annotation in post_audit.annotations
2601     )
2602
2603
2604 def test_wrong_profile_number_remains_high_error():
2605     records = build_profile_support_registry(
2606         _profile_authority_fixture()
2607     )
2608     audit, _ = _audit_sentence(
2609         "The dataset contains 2 exact duplicate rows.",
2610         profile_records=records,
2611     )
2612
2613     assert not audit.support_map_patches
2614     assert any(
2615         annotation.subtype == "unsupported_number"
2616         and annotation.severity == Severity.HIGH
2617         for annotation in audit.annotations
2618     )
2619
2620
2621 def test_data_understanding_is_not_factual_authority_for_metadata():
2622     audit, _ = _audit_sentence(
2623         "The dataset contains hourly observations at a specific location."
2624     )
2625
2626     subtypes = {
2627         annotation.subtype
2628         for annotation in audit.annotations
2629     }
2630     assert "unsupported_temporal_cadence" in subtypes
2631     assert "unsupported_location_metadata" in subtypes
2632
2633
2634 def test_datetime_parse_rate_does_not_prove_hourly_cadence():
2635     records = build_profile_support_registry(
2636         _profile_authority_fixture()
2637     )
2638     audit, _ = _audit_sentence(
2639         "The dataset contains hourly observations.",
2640         profile_records=records,
2641     )
2642
2643     assert any(
2644         annotation.subtype
2645         == "unsupported_temporal_cadence"
2646         for annotation in audit.annotations
2647     )
2648
2649
2650 def test_wording_guardrails_for_constant_zero_missing_duplicate_and_pearson():
2651     examples = {
2652         "The Constant column provides no analytical value.": (
2653             "overbroad_constant_interpretation"
2654         ),
2655         "Pressure zeros likely represents encoded missingness.": (
2656             "overconfident_zero_interpretation"
2657         ),
2658         "The missingness is unlikely to cause major issues.": (
2659             "unsupported_missingness_impact"
2660         ),
2661         "Duplicate rows should likely be removed.": (
2662             "unsupported_duplicate_removal"
2663         ),
2664         "Pearson correlation may be influenced by non-linear patterns.": (
2665             "imprecise_pearson_limitation"
2666         ),
2667     }
2668
2669     for sentence, subtype in examples.items():
2670         audit, _ = _audit_sentence(sentence)
2671         assert any(
2672             annotation.subtype == subtype
2673             for annotation in audit.annotations
2674         )
2675

```

```

2676
2677 def test_safe_profile_supported_wording_is_allowed_after_hidden_patch():
2678     records = build_profile_support_registry(
2679         _profile_authority_fixture()
2680     )
2681     constant_id = next(
2682         record.support_id
2683         for record in records
2684         if record.fact_kind == "constant_column"
2685     )
2686     sentence = (
2687         "The Constant column contains no observed variation for analyses "
2688         "that depend on variation."
2689     )
2690     support = SentenceSupport(
2691         sentence_id="SENT_0001",
2692         sentence_text=sentence,
2693         fact_ids=[],
2694         evidence_ids=[],
2695         profile_support_ids=[constant_id],
2696         support_type=SupportType.PARAPHRASE,
2697     )
2698     audit, _ = _audit_sentence(
2699         sentence,
2700         support=support,
2701         profile_records=records,
2702     )
2703
2704     assert not audit.annotations
2705
2706
2707 def test_unsupported_and_supported_future_recommendations():
2708     unsupported, _ = _audit_sentence(
2709         "Future work should explore temporal trends."
2710     )
2711     assert any(
2712         annotation.subtype
2713         == "unsupported_analytical_recommendation"
2714         for annotation in unsupported.annotations
2715     )
2716
2717     evidence = EvidenceLedger(
2718         fingerprint="recommendation",
2719         items=[
2720             EvidenceItem(
2721                 evidence_id="EVD_REC",
2722                 route=AnalysisRoute.DESCRPTIVE,
2723                 task_ids=["TASK_REC"],
2724                 finding="The table contains 4 rows.",
2725                 metrics={"row_count": 4},
2726                 source_tables=["weather"],
2727                 source_columns=[],
2728                 method="Direct count.",
2729                 validation_strategy=ValidationStrategy.NONE,
2730                 practical_interpretation="Small table.",
2731                 strength_label="dataset_overview",
2732                 limitations=[],
2733                 prohibited_interpretations=[],
2734                 recommendations=[
2735                     AnalyticalRecommendation(
2736                         recommendation_id="REC_TEMPORAL",
2737                         action=(
2738                             "Future work should explore temporal trends."
2739                         ),
2740                         recommendation_type="additional_analysis",
2741                         priority="low",
2742                         justification=(
2743                             "The source includes a timestamp field."
2744                         ),
2745                     ),
2746                 ],
2747                 claim_permissions=[
2748                     ClaimPermission.DESCRPTIVE
2749                 ],
2750                 factual_confidence=1.0,
2751                 methodological_strength=1.0,
2752                 user_relevance=0.8,
2753                 salience=0.7,
2754                 recommended_use=RecommendedUse.SUPPORTING_DETAIL,
2755             ),
2756         ],
2757     )
2758     ledger = FactLedger(
2759         writer_ready_facts=[
2760             VerifiedFact(
2761                 fact_id="FACT_REC",
2762                 source_candidate_id="CAN_REC",
2763                 fact_summary="The table contains 4 rows.",
2764                 evidence_ids=["EVD_REC"],
2765                 structured_values={
2766                     "EVD_REC": {"row_count": 4}

```

```

2767         },
2768         entities=["weather"],
2769         claim_permissions=[
2770             ClaimPermission.DESCRPTIVE
2771         ],
2772         factual_confidence=1.0,
2773         methodological_strength=1.0,
2774         user_relevance=0.8,
2775         salience=0.7,
2776         recommended_use=RecommendedUse.SUPPORTING_DETAIL,
2777     )
2778 ]
2779 )
2780 sentence = "Future work should explore temporal trends."
2781 support = SentenceSupport(
2782     sentence_id="SENT_0001",
2783     sentence_text=sentence,
2784     fact_ids=["FACT_REC"],
2785     evidence_ids=["EVD_REC"],
2786     support_type=SupportType.PARAPHRASE,
2787 )
2788 supported, _ = _audit_sentence(
2789     sentence,
2790     support=support,
2791     ledger=ledger,
2792     evidence=evidence,
2793 )
2794
2795 assert not any(
2796     annotation.subtype
2797     == "unsupported_analytical_recommendation"
2798     for annotation in supported.annotations
2799 )
2800
2801 def test_quality_finding_validation_and_conservative_merge():
2802     assert not valid_quality_finding(
2803         "The dataset contains 24 duplicate rows."
2804     )
2805     assert valid_quality_finding(
2806         "The report recommends duplicate removal without sufficient justification."
2807     )
2808
2809 deterministic = ReportQualityAssessment(
2810     status=QualityStatus.REVISE,
2811     request_responsiveness=0.9,
2812     finding_selection=0.8,
2813     coherence=0.7,
2814     concision=0.8,
2815     caveat_integration=0.9,
2816     data_science_interpretation=0.8,
2817     findings=["The report omits a required limitation."],
2818     recommendations=["Add the missing limitation."],
2819 )
2820
2821 semantic = ReportQualityAssessment(
2822     status=QualityStatus.PASS,
2823     request_responsiveness=1.0,
2824     finding_selection=0.6,
2825     coherence=0.9,
2826     concision=0.9,
2827     caveat_integration=1.0,
2828     data_science_interpretation=0.9,
2829     findings=["The report repeats closely related findings."],
2830     recommendations=["Consolidate repeated findings."],
2831 )
2832
2833 merged = merge_quality_assessments(
2834     deterministic,
2835     semantic,
2836 )
2837
2838 assert merged.status == QualityStatus.REVISE
2839 assert "The report omits a required limitation." in merged.findings
2840 assert "The report repeats closely related findings." in merged.findings
2841 assert merged.finding_selection == 0.6
2842
2843 def test_writer_payload_removes_data_understanding_factual_prose():
2844     ledger, evidence = make_fact_fixture()
2845     understanding = DataUnderstanding(
2846         profile_fingerprint="payload",
2847         dataset_summary=(
2848             "The data are hourly and collected at a specific location."
2849         ),
2850     ),
2851     tables=[
2852         TableUnderstanding(
2853             table_name="weather",
2854             unit_of_observation="hourly weather station measurement",
2855             summary="Location-specific hourly weather station data.",
2856         )
2857     ],

```

```

2858 )
2859 plan = ExecutionPlan(
2860     objective="Understand the data.",
2861     tasks=[],
2862     route_order=[],
2863     report_specification=_basic_spec(),
2864     audit_mode=AuditMode.INTERNAL,
2865     revision_limit=1,
2866     maximum_facts=20,
2867     rationale="Test.",
2868 )
2869 pack = build_writer_evidence_pack(
2870     request=plan.objective,
2871     understanding=understanding,
2872     plan=plan,
2873     evidence=evidence,
2874     fact_ledger=ledger,
2875     settings=Settings(),
2876 )
2877
2878 payload = build_compact_writer_payload(pack)
2879
2880 assert "dataset_summary" not in payload
2881 assert "table_context" not in payload
2882 assert "semantic_map" not in payload
2883 assert payload["priority_facts"]
2884 assert "analytical_recommendations" in payload
2885
2886
2887 # =====
2888 # VERIFIED BOUNDED INSIGHT SYNTHESIS TESTS
2889 # =====
2890
2891
2892 def _insight_fixture() -> tuple[FactLedger, EvidenceLedger]:
2893     evidence = EvidenceLedger(
2894         fingerprint="insight-test",
2895         items=[
2896             EvidenceItem(
2897                 evidence_id="EVD_INS_001",
2898                 route=AnalysisRoute.ASSOCIATION_COMPARISON,
2899                 task_ids=["TASK_INS"],
2900                 finding=(
2901                     "`Humidity` and `Temperature` have a negative Pearson "
2902                     "correlation."
2903                 ),
2904                 metrics={"pearson_r": -0.63},
2905                 source_tables=["weather"],
2906                 source_columns=["Humidity", "Temperature"],
2907                 method="Deterministic Pearson correlation.",
2908                 practical_interpretation=(
2909                     "Higher humidity is associated with lower temperature "
2910                     "in this dataset."
2911                 ),
2912                 strength_label="strong_association",
2913                 claim_permissions=[ClaimPermission.ASSOCIATIONAL],
2914                 factual_confidence=1.0,
2915                 methodological_strength=0.9,
2916                 user_relevance=0.9,
2917                 salience=0.9,
2918                 recommended_use=RecommendedUse.MAIN_FINDING,
2919             ),
2920             EvidenceItem(
2921                 evidence_id="EVD_INS_002",
2922                 route=AnalysisRoute.ASSOCIATION_COMPARISON,
2923                 task_ids=["TASK_INS"],
2924                 finding=(
2925                     "`Humidity` and `Temperature` also have a negative rank "
2926                     "association."
2927                 ),
2928                 metrics={"spearman_r": -0.61},
2929                 source_tables=["weather"],
2930                 source_columns=["Humidity", "Temperature"],
2931                 method="Deterministic rank association.",
2932                 practical_interpretation=(
2933                     "The inverse association is present under a rank-based "
2934                     "summary as well."
2935                 ),
2936                 strength_label="strong_association",
2937                 claim_permissions=[ClaimPermission.ASSOCIATIONAL],
2938                 factual_confidence=1.0,
2939                 methodological_strength=0.9,
2940                 user_relevance=0.85,
2941                 salience=0.85,
2942                 recommended_use=RecommendedUse.MAIN_FINDING,
2943             ),
2944         ],
2945     )
2946     facts = [
2947         VerifiedFact(
2948             fact_id=f"FACT_INS_{index:03d}",

```



```

2949         source_candidate_id=f"CAN_INS_{index:03d}",
2950         fact_summary=item.finding,
2951         evidence_ids=[item.evidence_id],
2952         structured_values={item.evidence_id: item.metrics},
2953         entities=[
2954             "weather",
2955             "Humidity",
2956             "Temperature",
2957         ],
2958         claim_permissions=item.claim_permissions,
2959         allowed_interpretations=[item.practical_interpretation],
2960         factual_confidence=1.0,
2961         methodological_strength=0.9,
2962         user_relevance=item.user_relevance,
2963         salience=item.salience,
2964         recommended_use=RecommendedUse.MAIN_FINDING,
2965     )
2966     for index, item in enumerate(evidence.items, start=1)
2967 ]
2968 return FactLedger(writer_ready_facts=facts), evidence
2969
2970
2971 def _insight_candidate(
2972     *,
2973     insight_id: str = "INSIGHT_001",
2974     statement: str = (
2975         "Pearson and rank-based summaries both show an inverse association "
2976         "between `Humidity` and `Temperature` in this dataset."
2977     ),
2978     insight_type: InsightType = InsightType.OUTCOME_ASSOCIATION,
2979     interpretation_level: InterpretationLevel = (
2980         InterpretationLevel.BOUNDED_INSIGHT
2981     ),
2982     source_fact_ids: list[str] | None = None,
2983     source_evidence_ids: list[str] | None = None,
2984     suitable_for_main_report: bool = True,
2985     confidence: float = 0.9,
2986     salience: float = 0.9,
2987 ) -> InsightCandidate:
2988     return InsightCandidate(
2989         insight_id=insight_id,
2990         statement=statement,
2991         insight_type=insight_type,
2992         interpretation_level=interpretation_level,
2993         source_fact_ids=(
2994             source_fact_ids
2995             if source_fact_ids is not None
2996             else ["FACT_INS_001", "FACT_INS_002"]
2997         ),
2998         source_evidence_ids=(
2999             source_evidence_ids
3000             if source_evidence_ids is not None
3001             else ["EVD_INS_001", "EVD_INS_002"]
3002         ),
3003         why_it_matters=(
3004             "Agreement across both summaries makes the direction less "
3005             "dependent on a single association measure."
3006         ),
3007         supporting_summary="Two verified association summaries agree.",
3008         limitations=["The association is descriptive, not causal."],
3009         claim_permissions=[ClaimPermission.ASSOCIATIONAL],
3010         confidence=confidence,
3011         salience=salience,
3012         suitable_for_main_report=suitable_for_main_report,
3013     )
3014
3015
3016 def _verified_insight(
3017     candidate: InsightCandidate | None = None,
3018     *,
3019     status: InsightVerificationStatus = InsightVerificationStatus.VERIFIED,
3020 ) -> VerifiedInsight:
3021     candidate = candidate or _insight_candidate()
3022     return VerifiedInsight(
3023         insight_id=candidate.insight_id,
3024         statement=candidate.statement,
3025         insight_type=candidate.insight_type,
3026         interpretation_level=candidate.interpretation_level,
3027         source_fact_ids=candidate.source_fact_ids,
3028         source_evidence_ids=candidate.source_evidence_ids,
3029         why_it_matters=candidate.why_it_matters,
3030         limitations=candidate.limitations,
3031         claim_permissions=candidate.claim_permissions,
3032         confidence=candidate.confidence,
3033         salience=candidate.salience,
3034         verification_status=status,
3035     )
3036
3037
3038 def test_insight_schema_defaults_and_report_controls():
3039     plan = ExecutionPlan(

```

```

3040         objective="Describe the data.",
3041         tasks=[],
3042         route_order=[],
3043         report_specification=_basic_spec(),
3044         audit_mode=AuditMode.INTERNAL,
3045         revision_limit=1,
3046         maximum_facts=10,
3047         rationale="Compatibility fixture.",
3048     )
3049     sentence = WriterSentenceDraft(
3050         text="A direct finding.",
3051         support_type=SupportType.DIRECT,
3052     )
3053
3054     assert plan.insight_objectives == []
3055     assert sentence.insight_ids == []
3056     assert sentence.interpretation_level == InterpretationLevel.FINDING
3057     assert plan.report_specification.genre == ReportGenre.DATA_SCIENCE_REPORT
3058     assert plan.report_specification.perspective == ReportPerspective.NEUTRAL
3059     assert InsightLedger().verified_insights == []
3060
3061
3062 def test_insight_configuration_defaults_and_validation():
3063     settings = Settings()
3064
3065     assert settings.enable_insight_synthesis
3066     assert settings.max_insight_candidates is None
3067     assert settings.max_verified_main_insights is None
3068     assert settings.writer_max_main_findings is None
3069     assert settings.writer_priority_fact_limit is None
3070     assert settings.writer_supporting_fact_limit is None
3071
3072     with pytest.raises(ValueError):
3073         replace(settings, max_insight_candidates=0)
3074     with pytest.raises(ValueError):
3075         replace(
3076             settings,
3077             max_insight_candidates=6,
3078             max_verified_main_insights=7,
3079         )
3080     with pytest.raises(ValueError):
3081         replace(settings, min_insight_confidence=1.1)
3082     with pytest.raises(ValueError):
3083         replace(settings, min_insight_salience=-0.1)
3084     with pytest.raises(ValueError):
3085         replace(settings, min_facts_per_bounded_insight=0)
3086
3087
3088 def test_verified_insight_selection_is_uncapped_by_default():
3089     insights = [
3090         _verified_insight(
3091             _insight_candidate(
3092                 insight_id=f"INSIGHT_UNCAPPED_{index:02d}",
3093                 salience=0.95 - index * 0.01,
3094             )
3095         )
3096         for index in range(7)
3097     ]
3098
3099     priority, supporting = select_priority_verified_insights(
3100         insight_ledger=InsightLedger(verified_insights=insights),
3101         maximum=None,
3102     )
3103
3104     assert len(priority) == 7
3105     assert supporting == []
3106
3107
3108 def test_event_writer_payload_preserves_structured_values_without_word_ceiling():
3109     evidence = EvidenceLedger(
3110         fingerprint="event-payload",
3111         items=[
3112             EvidenceItem(
3113                 evidence_id="EVD_SEQUENCE",
3114                 route=AnalysisRoute.DESCRPTIVE,
3115                 task_ids=["TASK_EVENT"],
3116                 capability=EvidenceCapability.RANKING,
3117                 evidence_type="event_sequence",
3118                 finding="Supported ordered event sequence.",
3119                 metrics={},
3120                 source_tables=["event"],
3121                 source_columns=["sequence"],
3122                 method="Sequence extraction.",
3123                 practical_interpretation="Use the ordered sequence.",
3124                 strength_label="event_sequence",
3125                 claim_permissions=[ClaimPermission.DESCRPTIVE],
3126                 factual_confidence=1.0,
3127                 methodological_strength=1.0,
3128                 user_relevance=0.9,
3129                 salience=0.9,
3130                 recommended_use=RecommendedUse.MAIN_FINDING,

```

```

3131     )
3132     ],
3133 )
3134 fact = VerifiedFact(
3135     fact_id="FACT_SEQUENCE",
3136     source_candidate_id="CAN_SEQUENCE",
3137     fact_summary="The event has an ordered sequence.",
3138     evidence_ids=["EVD_SEQUENCE"],
3139     source_capabilities=[EvidenceCapability.RANKING],
3140     structured_values={
3141         "EVD_SEQUENCE": {
3142             "steps": [
3143                 {"step": index}
3144                 for index in range(20)
3145             ]
3146         }
3147     },
3148     entities=["event"],
3149     claim_permissions=[ClaimPermission.DESRIPTIVE],
3150     factual_confidence=1.0,
3151     methodological_strength=1.0,
3152     user_relevance=0.9,
3153     salience=0.9,
3154     recommended_use=RecommendedUse.MAIN_FINDING,
3155 )
3156 pack = WriterEvidencePack(
3157     user_request="Write an event report.",
3158     report_specification=ReportSpecification(
3159         report_purpose="Write an event report.",
3160         genre=ReportGenre.EVENT_REPORT,
3161         target_length_words=650,
3162         maximum_length_words=None,
3163         prioritisation_rule="Use supported event evidence.",
3164     ),
3165     dataset_understanding=DataUnderstanding(
3166         profile_fingerprint="event-payload",
3167         dataset_summary="event",
3168         tables=[],
3169     ),
3170     priority_facts=[fact],
3171     supporting_facts=[],
3172     limitation_facts=[],
3173     evidence_ledger=evidence,
3174 )
3175
3176 payload = build_compact_writer_payload(pack)
3177
3178 steps = payload["priority_facts"][0]["structured_values"]["EVD_SEQUENCE"]["steps"]
3179 assert len(steps) == 20
3180 assert "omitted_record_count" not in steps[-1]
3181 assert payload["structured_value_compaction"]["item_limit"] is None
3182 assert "event_report_writing_guidance" in payload
3183
3184
3185 def test_event_writer_payload_compacts_structured_values_with_word_ceiling():
3186     evidence = EvidenceLedger(
3187         fingerprint="event-payload-capped",
3188         items=[
3189             EvidenceItem(
3190                 evidence_id="EVD_SEQUENCE",
3191                 route=AnalysisRoute.DESRIPTIVE,
3192                 task_ids=["TASK_EVENT"],
3193                 capability=EvidenceCapability.RANKING,
3194                 evidence_type="event_sequence",
3195                 finding="Supported ordered event sequence.",
3196                 metrics={},
3197                 source_tables=["event"],
3198                 source_columns=["sequence"],
3199                 method="Sequence extraction.",
3200                 practical_interpretation="Use the ordered sequence.",
3201                 strength_label="event_sequence",
3202                 claim_permissions=[ClaimPermission.DESRIPTIVE],
3203                 factual_confidence=1.0,
3204                 methodological_strength=1.0,
3205                 user_relevance=0.9,
3206                 salience=0.9,
3207                 recommended_use=RecommendedUse.MAIN_FINDING,
3208             )
3209         ],
3210     )
3211     fact = VerifiedFact(
3212         fact_id="FACT_SEQUENCE",
3213         source_candidate_id="CAN_SEQUENCE",
3214         fact_summary="The event has an ordered sequence.",
3215         evidence_ids=["EVD_SEQUENCE"],
3216         source_capabilities=[EvidenceCapability.RANKING],
3217         structured_values={
3218             "EVD_SEQUENCE": {
3219                 "steps": [
3220                     {"step": index}
3221                     for index in range(20)

```

```

3222     ]
3223     }
3224 },
3225 entities=["event"],
3226 claim_permissions=[ClaimPermission.DESCRPTIVE],
3227 factual_confidence=1.0,
3228 methodological_strength=1.0,
3229 user_relevance=0.9,
3230 salience=0.9,
3231 recommended_use=RecommendedUse.MAIN_FINDING,
3232 )
3233 pack = WriterEvidencePack(
3234     user_request="Write an event report.",
3235     report_specification=ReportSpecification(
3236         report_purpose="Write an event report.",
3237         genre=ReportGenre.EVENT_REPORT,
3238         target_length_words=650,
3239         maximum_length_words=300,
3240         prioritisation_rule="Use supported event evidence.",
3241     ),
3242     dataset_understanding=DataUnderstanding(
3243         profile_fingerprint="event-payload-capped",
3244         dataset_summary="event",
3245         tables=[],
3246     ),
3247     priority_facts=[fact],
3248     supporting_facts=[],
3249     limitation_facts=[],
3250     evidence_ledger=evidence,
3251 )
3252
3253 payload = build_compact_writer_payload(pack)
3254
3255 steps = payload["priority_facts"][0]["structured_values"]["EVD_SEQUENCE"]["steps"]
3256 assert payload["structured_value_compaction"]["item_limit"] == 12
3257 assert steps[-1]["omitted_record_count"] == 8
3258
3259 def test_event_writer_pack_priority_selection_is_uncapped_by_default():
3260     def event_item(
3261         index: int,
3262         evidence_type: str,
3263         capability: EvidenceCapability,
3264     ) -> EvidenceItem:
3265         return EvidenceItem(
3266             evidence_id=f"EVD_{index:04d}",
3267             route=AnalysisRoute.DESCRPTIVE,
3268             task_ids=["TASK_EVENT"],
3269             capability=capability,
3270             evidence_type=evidence_type,
3271             finding=f"Supported event fact {index}.",
3272             metrics={"index": index},
3273             source_tables=["event"],
3274             source_columns=[evidence_type],
3275             method="Event evidence extraction.",
3276             practical_interpretation="Use the supported event fact.",
3277             strength_label=evidence_type,
3278             claim_permissions=[
3279                 ClaimPermission.DESCRPTIVE,
3280                 ClaimPermission.COMPARATIVE,
3281             ],
3282             factual_confidence=1.0,
3283             methodological_strength=1.0,
3284             user_relevance=0.9,
3285             salience=0.9,
3286             recommended_use=RecommendedUse.MAIN_FINDING,
3287         )
3288
3289 evidence = EvidenceLedger(
3290     fingerprint="event-uncapped",
3291     items=[
3292         event_item(
3293             1,
3294             "event_outcome",
3295             EvidenceCapability.EVENT_OUTCOME,
3296         ),
3297         *[
3298             event_item(
3299                 index,
3300                 "participant_comparison",
3301                 EvidenceCapability.GROUP_COMPARISON,
3302             )
3303             for index in range(2, 6)
3304         ],
3305         *[
3306             event_item(
3307                 index,
3308                 "entity_ranking",
3309                 EvidenceCapability.RANKING,
3310             )
3311             for index in range(6, 10)
3312

```

```

3313         ],
3314     ],
3315 )
3316 facts = [
3317     VerifiedFact(
3318         fact_id=f"FACT_{index:04d}",
3319         source_candidate_id=f"CAN_{index:04d}",
3320         fact_summary=item.finding,
3321         evidence_ids=[item.evidence_id],
3322         source_capabilities=[item.capability],
3323         structured_values={item.evidence_id: item.metrics},
3324         entities=["event"],
3325         claim_permissions=item.claim_permissions,
3326         factual_confidence=1.0,
3327         methodological_strength=1.0,
3328         user_relevance=0.9,
3329         salience=0.9,
3330         recommended_use=RecommendedUse.MAIN_FINDING,
3331     )
3332     for index, item in enumerate(evidence.items, start=1)
3333 ]
3334 plan = ExecutionPlan(
3335     objective="Write an event report.",
3336     tasks=[],
3337     route_order=[AnalysisRoute.DESCRPTIVE],
3338     report_specification=ReportSpecification(
3339         report_purpose="Write an event report.",
3340         genre=ReportGenre.EVENT_REPORT,
3341         target_length_words=650,
3342         maximum_length_words=None,
3343         prioritisation_rule="Use supported event material.",
3344     ),
3345     audit_mode=AuditMode.INTERNAL,
3346     revision_limit=1,
3347     rationale="test",
3348 )
3349
3350 pack = build_writer_evidence_pack(
3351     request="Write an event report.",
3352     understanding=DataUnderstanding(
3353         profile_fingerprint="event-uncapped",
3354         dataset_summary="event",
3355         tables=[],
3356     ),
3357     plan=plan,
3358     evidence=evidence,
3359     fact_ledger=FactLedger(writer_ready_facts=facts),
3360     settings=Settings(),
3361 )
3362
3363 priority_evidence_types = [
3364     evidence.items[int(fact.evidence_ids[0].split("_")[1]) - 1].evidence_type
3365     for fact in pack.priority_facts
3366 ]
3367 assert priority_evidence_types.count("participant_comparison") == 4
3368 assert priority_evidence_types.count("entity_ranking") == 4
3369
3370 def test_high_confidence_event_report_uses_deterministic_plan():
3371     input_structure = InputStructureProfile(
3372         shape=InputShape.EVENT_RECORD,
3373         representation_status=InputRepresentationStatus.VALID,
3374         row_semantics="one event",
3375         entity_levels=["event", "participant", "entity"],
3376         confidence=0.95,
3377     )
3378
3379     semantic_map = InputSemanticMap.model_validate(
3380         {
3381             "input_shape": "event_record",
3382             "record_description": "A generic event record.",
3383             "recommended_report_genre": "event_report",
3384             "confidence": 0.95,
3385             "bindings": [
3386                 {
3387                     "binding_id": "b01",
3388                     "table_name": "event",
3389                     "label": "Participant",
3390                     "role": "participant_identifier",
3391                     "level": "participant",
3392                     "path_pattern": "participants.*name",
3393                     "description": "Participant name.",
3394                     "confidence": 0.95,
3395                     "evidence_basis": "structure",
3396                 },
3397             ],
3398         }
3399     )
3400
3401     assert should_use_deterministic_event_plan(
3402         controller_genre=ReportGenre.EVENT_REPORT,
3403         input_structure=input_structure,

```

```

3404         semantic_map=semantic_map,
3405         capabilities=[
3406             EvidenceCapability.EVENT_OUTCOME,
3407             EvidenceCapability.RANKING,
3408         ],
3409     )
3410     assert not should_use_deterministic_event_plan(
3411         controller_genre=ReportGenre.DATA_SCIENCE_REPORT,
3412         input_structure=input_structure,
3413         semantic_map=semantic_map,
3414         capabilities=[
3415             EvidenceCapability.EVENT_OUTCOME,
3416             EvidenceCapability.RANKING,
3417         ],
3418     )
3419     assert not should_use_deterministic_event_plan(
3420         controller_genre=ReportGenre.EVENT_REPORT,
3421         input_structure=input_structure.model_copy(
3422             update={"confidence": 0.4}
3423         ),
3424         semantic_map=semantic_map,
3425         capabilities=[
3426             EvidenceCapability.EVENT_OUTCOME,
3427         ],
3428     )
3429
3430 def test_fallback_plan_freezes_questions_and_genre_defaults():
3431     profile = _profile_authority_fixture()
3432     generic = fallback_execution_plan(
3433         "Understand the dataset.",
3434         profile,
3435         AuditMode.INTERNAL,
3436         Settings(),
3437     )
3438
3439     sports = fallback_execution_plan(
3440         "Write a game report.",
3441         profile,
3442         AuditMode.INTERNAL,
3443         Settings(),
3444     )
3445
3446     assert generic.insight_objectives
3447     assert all(
3448         objective.question.endswith("?")
3449         for objective in generic.insight_objectives
3450     )
3451     assert all(
3452         not any(character.isdigit() for character in objective.question)
3453         for objective in generic.insight_objectives
3454     )
3455     assert generic.report_specification.genre == ReportGenre.DATA_SCIENCE_REPORT
3456     assert sports.report_specification.genre == ReportGenre.EVENT_REPORT
3457
3458 def test_valid_bounded_insight_and_safe_association_are_accepted():
3459     ledger, evidence = _insight_fixture()
3460     candidate = _insight_candidate()
3461     overlap = _insight_candidate(
3462         insight_id="INSIGHT_OVERLAP",
3463         statement=(
3464             "The two measures contain highly overlapping information in "
3465             "this dataset."
3466         ),
3467     ),
3468     insight_type=InsightType.REDUNDANCY,
3469 )
3470
3471     assert not validate_insight_candidates(
3472         InsightCandidateSet(candidates=[candidate]),
3473         ledger,
3474         evidence,
3475         Settings(),
3476     )
3477     assert not validate_insight_candidates(
3478         InsightCandidateSet(candidates=[overlap]),
3479         ledger,
3480         evidence,
3481         Settings(),
3482     )
3483
3484 def test_insight_candidate_rejects_unknown_fact_and_number():
3485     ledger, evidence = _insight_fixture()
3486     unknown = _insight_candidate(
3487         source_fact_ids=["FACT_UNKNOWN", "FACT_INS_002"]
3488     )
3489     numbered = _insight_candidate(
3490         statement=(
3491             "Higher `Humidity` is associated with a 99-point reduction in "
3492             "`Temperature`."
3493         )
3494     )

```

```

3495     )
3496     unknown_entity = _insight_candidate(
3497         statement=(
3498             "`Pressure` contains highly overlapping information with "
3499             "`Temperature` in this dataset."
3500         )
3501     )
3502
3503     unknown_errors = validate_insight_candidates(
3504         InsightCandidateSet(candidates=[unknown]),
3505         ledger,
3506         evidence,
3507         Settings(),
3508     )
3509
3510     number_errors = validate_insight_candidates(
3511         InsightCandidateSet(candidates=[numbered]),
3512         ledger,
3513         evidence,
3514         Settings(),
3515     )
3516
3517     entity_errors = validate_insight_candidates(
3518         InsightCandidateSet(candidates=[unknown_entity]),
3519         ledger,
3520         evidence,
3521         Settings(),
3522     )
3523
3524     assert any("unknown fact" in error for error in unknown_errors)
3525     assert any("unsupported numbers" in error for error in number_errors)
3526     assert any("unsupported table or column" in error for error in entity_errors)
3527
3528 def test_insight_candidate_requires_grounded_non_hypothetical_implication():
3529     ledger, evidence = _insight_fixture()
3530     restatement = _insight_candidate().model_copy(
3531         update={
3532             "why_it_matters": (
3533                 ledger.writer_ready_facts[0].allowed_interpretations[0]
3534             )
3535         }
3536     )
3537     hidden_hypothesis = _insight_candidate().model_copy(
3538         update={
3539             "why_it_matters": (
3540                 "The pattern may reflect a data artifact in the source."
3541             )
3542         }
3543     )
3544     unsupported_number = _insight_candidate().model_copy(
3545         update={
3546             "why_it_matters": (
3547                 "The implication applies to 99 analytical settings."
3548             )
3549         }
3550     )
3551
3552     restatement_errors = validate_insight_candidates(
3553         InsightCandidateSet(candidates=[restatement]),
3554         ledger,
3555         evidence,
3556         Settings(),
3557     )
3558
3559     hypothesis_errors = validate_insight_candidates(
3560         InsightCandidateSet(candidates=[hidden_hypothesis]),
3561         ledger,
3562         evidence,
3563         Settings(),
3564     )
3565
3566     number_errors = validate_insight_candidates(
3567         InsightCandidateSet(candidates=[unsupported_number]),
3568         ledger,
3569         evidence,
3570         Settings(),
3571     )
3572
3573     assert any("restates a source finding" in error for error in restatement_errors)
3574     assert any("explanatory hypothesis" in error for error in hypothesis_errors)
3575     assert any("why_it_matters" in error for error in number_errors)
3576
3577 def test_single_fact_pseudo_insight_rejected_but_anomaly_allowed():
3578     ledger, evidence = _insight_fixture()
3579     pseudo = _insight_candidate(
3580         insight_type=InsightType.CONTRAST,
3581         source_fact_ids=["FACT_INS_001"],
3582         source_evidence_ids=["EVD_INS_001"],
3583     )
3584     anomaly = _insight_candidate(
3585         insight_id="INSIGHT_ANOMALY",
3586         statement=(
3587             "The association in `Humidity` and `Temperature` is an anomaly "

```



```

3586         "requiring further review in this dataset."
3587     ),
3588     insight_type=InsightType.ANOMALY,
3589     source_fact_ids=["FACT_INS_001"],
3590     source_evidence_ids=["EVD_INS_001"],
3591 )
3592
3593 assert any(
3594     "single-fact pseudo-insight" in error
3595     for error in validate_insight_candidates(
3596         InsightCandidateSet(candidates=[pseudo]),
3597         ledger,
3598         evidence,
3599         Settings(),
3600     )
3601 )
3602 assert not validate_insight_candidates(
3603     InsightCandidateSet(candidates=[anomaly]),
3604     ledger,
3605     evidence,
3606     Settings(),
3607 )
3608
3609 def test_single_fact_event_ranking_can_be_event_synthesis():
3610     evidence = EvidenceLedger(
3611         fingerprint="event-insight-test",
3612         items=[
3613             EvidenceItem(
3614                 evidence_id="EVD_EVENT_RANKING",
3615                 route=AnalysisRoute.DESRIPTIVE,
3616                 task_ids=["TASK_EVENT"],
3617                 capability=EvidenceCapability.RANKING,
3618                 evidence_type="entity_ranking",
3619                 finding=(
3620                     "The leading recorded assists performances were: "
3621                     "Ben Simmons recorded 6."
3622                 ),
3623                 metrics={
3624                     "metric": "assists",
3625                     "ranking": [
3626                         {
3627                             "rank": 1,
3628                             "entity": "Ben Simmons",
3629                             "value": 6,
3630                         }
3631                     ],
3632                 },
3633                 source_tables=["event"],
3634                 source_columns=["assists"],
3635                 method="Validated structured-event extraction.",
3636                 practical_interpretation=(
3637                     "This ranks entities by the observed event metric."
3638                 ),
3639                 strength_label="entity_ranking",
3640                 claim_permissions=[
3641                     ClaimPermission.DESRIPTIVE,
3642                     ClaimPermission.COMPARATIVE,
3643                 ],
3644                 factual_confidence=1.0,
3645                 methodological_strength=1.0,
3646                 user_relevance=0.9,
3647                 salience=0.9,
3648                 recommended_use=RecommendedUse.MAIN_FINDING,
3649             ),
3650         ],
3651     )
3652
3653 ledger = FactLedger(
3654     writer_ready_facts=[
3655         VerifiedFact(
3656             fact_id="FACT_EVENT_RANKING",
3657             source_candidate_id="CAN_EVENT_RANKING",
3658             fact_summary=evidence.items[0].finding,
3659             evidence_ids=["EVD_EVENT_RANKING"],
3660             structured_values={
3661                 "EVD_EVENT_RANKING": evidence.items[0].metrics,
3662             },
3663             entities=["event", "assists", "Ben Simmons"],
3664             claim_permissions=[
3665                 ClaimPermission.DESRIPTIVE,
3666                 ClaimPermission.COMPARATIVE,
3667             ],
3668             allowed_interpretations=[
3669                 evidence.items[0].practical_interpretation,
3670             ],
3671             factual_confidence=1.0,
3672             methodological_strength=1.0,
3673             user_relevance=0.9,
3674             salience=0.9,
3675             recommended_use=RecommendedUse.MAIN_FINDING,
3676         )

```



```

3677     ]
3678   )
3679   candidate = InsightCandidate(
3680     insight_id="INS_EVENT_RANKING",
3681     statement="Ben Simmons led the recorded assists ranking with 6.",
3682     insight_type=InsightType.DOMINANT_PATTERN,
3683     interpretation_level=InterpretationLevel.BOUNDED_INSIGHT,
3684     contribution=InsightContribution.EVENT_SYNTHESIS,
3685     source_fact_ids=["FACT_EVENT_RANKING"],
3686     source_evidence_ids=["EVD_EVENT_RANKING"],
3687     why_it_matters=None,
3688     supporting_summary="The ranking fact records Ben Simmons at 6 assists.",
3689     alternative_explanations=[],
3690     limitations=[],
3691     claim_permissions=[
3692       ClaimPermission.DESRIPTIVE,
3693       ClaimPermission.COMPARATIVE,
3694     ],
3695     confidence=1.0,
3696     salience=0.9,
3697     suitable_for_main_report=True,
3698   )
3699   candidate_set = InsightCandidateSet(candidates=[candidate])
3700   verification = InsightVerificationResult(
3701     records=[
3702       InsightVerificationRecord(
3703         insight_id="INS_EVENT_RANKING",
3704         status=InsightVerificationStatus.VERIFIED,
3705         confidence=1.0,
3706         salience=0.9,
3707         adds_bounded_synthesis=False,
3708         analytical_implication_supported=False,
3709         contains_hypothesis=False,
3710       )
3711     ]
3712   )
3713
3714   assert not validate_insight_candidates(
3715     candidate_set,
3716     ledger,
3717     evidence,
3718     Settings(),
3719     ReportGenre.EVENT_REPORT,
3720   )
3721   assert any(
3722     "single-fact pseudo-insight" in error
3723     for error in validate_insight_candidates(
3724       candidate_set,
3725       ledger,
3726       evidence,
3727       Settings(),
3728       ReportGenre.DATA_SCIENCE_REPORT,
3729     )
3730   )
3731   assert not validate_insight_verification(
3732     verification,
3733     candidate_set,
3734     ledger,
3735     evidence,
3736     Settings(),
3737     ReportGenre.EVENT_REPORT,
3738   )
3739
3740
3741   def test_causal_escalation_is_rejected():
3742     ledger, evidence = _insight_fixture()
3743     causal = _insight_candidate(
3744       statement="Humidity` causes lower `Temperature`."
3745     )
3746
3747     assert any(
3748       "causal wording" in error
3749       for error in validate_insight_candidates(
3750         InsightCandidateSet(candidates=[causal]),
3751         ledger,
3752         evidence,
3753         Settings(),
3754       )
3755     )
3756
3757
3758   def test_predictive_and_forecast_escalation_are_rejected():
3759     ledger, evidence = _insight_fixture()
3760     predictive = _insight_candidate(
3761       statement="Humidity` predicts `Temperature`."
3762     )
3763     forecast = _insight_candidate(
3764       statement="Humidity` forecasts future `Temperature` values."
3765     )
3766
3767     predictive_errors = validate_insight_candidates(

```

```

3768         InsightCandidateSet(candidates=[predictive]),
3769         ledger,
3770         evidence,
3771         Settings(),
3772     )
3773     forecast_errors = validate_insight_candidates(
3774         InsightCandidateSet(candidates=[forecast]),
3775         ledger,
3776         evidence,
3777         Settings(),
3778     )
3779
3780     assert any("predictive wording" in error for error in predictive_errors)
3781     assert any("forecast wording" in error for error in forecast_errors)
3782
3783
3784 def test_insight_verifier_must_confirm_synthesis_and_implication():
3785     ledger, evidence = _insight_fixture()
3786     candidate = _insight_candidate()
3787     candidates = InsightCandidateSet(candidates=[candidate])
3788     restatement_review = InsightVerificationResult(
3789         records=[
3790             InsightVerificationRecord(
3791                 insight_id=candidate.insight_id,
3792                 status=InsightVerificationStatus.VERIFIED,
3793                 confidence=0.9,
3794                 salience=0.9,
3795                 adds_bounded_synthesis=False,
3796                 analytical_implication_supported=False,
3797                 contains_hypothesis=False,
3798             )
3799         ]
3800     )
3801     valid_review = InsightVerificationResult(
3802         records=[
3803             InsightVerificationRecord(
3804                 insight_id=candidate.insight_id,
3805                 status=InsightVerificationStatus.VERIFIED,
3806                 confidence=0.9,
3807                 salience=0.9,
3808                 adds_bounded_synthesis=True,
3809                 analytical_implication_supported=True,
3810                 contains_hypothesis=False,
3811             )
3812         ]
3813     )
3814
3815     errors = validate_insight_verification(
3816         restatement_review,
3817         candidates,
3818         ledger,
3819         evidence,
3820         Settings(),
3821     )
3822
3823     assert any("direct-finding restatement" in error for error in errors)
3824     assert any("supported analytical implication" in error for error in errors)
3825     assert not validate_insight_verification(
3826         valid_review,
3827         candidates,
3828         ledger,
3829         evidence,
3830         Settings(),
3831     )
3832
3833
3834 def test_event_insight_does_not_require_analytical_implication():
3835     ledger, evidence = _insight_fixture()
3836     candidate = _insight_candidate().model_copy(
3837         update={
3838             "contribution": InsightContribution.EVENT_SYNTHESIS,
3839             "why_it_matters": None,
3840         }
3841     )
3842     candidates = InsightCandidateSet(candidates=[candidate])
3843     verification = InsightVerificationResult(
3844         records=[
3845             InsightVerificationRecord(
3846                 insight_id=candidate.insight_id,
3847                 status=InsightVerificationStatus.VERIFIED,
3848                 confidence=0.9,
3849                 salience=0.9,
3850                 adds_bounded_synthesis=True,
3851                 analytical_implication_supported=False,
3852                 contains_hypothesis=False,
3853             )
3854         ]
3855     )
3856
3857     analytical_errors = validate_insight_candidates(
3858         candidates,

```

```

3859         ledger,
3860         evidence,
3861         Settings(),
3862         ReportGenre.DATA_SCIENCE_REPORT,
3863     )
3864     event_errors = validate_insight_candidates(
3865         candidates,
3866         ledger,
3867         evidence,
3868         Settings(),
3869         ReportGenre.EVENT_REPORT,
3870     )
3871
3872     assert any(
3873         "lacks an analytical implication" in error
3874         for error in analytical_errors
3875     )
3876     assert not event_errors
3877     assert not validate_insight_verification(
3878         verification,
3879         candidates,
3880         ledger,
3881         evidence,
3882         Settings(),
3883         ReportGenre.EVENT_REPORT,
3884     )
3885
3886     result = materialise_insight_ledger(
3887         candidates=candidates,
3888         verification=verification,
3889         fact_ledger=ledger,
3890         evidence_ledger=evidence,
3891         settings=Settings(),
3892         report_genre=ReportGenre.EVENT_REPORT,
3893     )
3894
3895     assert [item.insight_id for item in result.verified_insights] == [
3896         candidate.insight_id
3897     ]
3898     assert result.verified_insights[0].contribution == (
3899         InsightContribution.EVENT_SYNTHESIS
3900     )
3901     assert result.verified_insights[0].why_it_matters is None
3902
3903
3904 def test_invalid_verifier_record_does_not_drop_valid_insight():
3905     ledger, evidence = _insight_fixture()
3906     valid_candidate = _insight_candidate(
3907         insight_id="INSIGHT_VALID",
3908     )
3909     unresolved_candidate = _insight_candidate(
3910         insight_id="INSIGHT_UNRESOLVED",
3911         statement=(
3912             "The two verified summaries provide a second distinct bounded "
3913             "view of the association in this dataset."
3914         ),
3915         insight_type=InsightType.REDUNDANCY,
3916     )
3917     candidates = InsightCandidateSet(
3918         candidates=[valid_candidate, unresolved_candidate]
3919     )
3920     verification = InsightVerificationResult(
3921         records=[
3922             InsightVerificationRecord(
3923                 insight_id=valid_candidate.insight_id,
3924                 status=InsightVerificationStatus.VERIFIED,
3925                 confidence=0.9,
3926                 salience=0.9,
3927                 adds_bounded_synthesis=True,
3928                 analytical_implication_supported=True,
3929                 contains_hypothesis=False,
3930             ),
3931             InsightVerificationRecord(
3932                 insight_id=unresolved_candidate.insight_id,
3933                 status=InsightVerificationStatus.VERIFIED,
3934                 confidence=0.9,
3935                 salience=0.8,
3936                 adds_bounded_synthesis=False,
3937                 analytical_implication_supported=False,
3938                 contains_hypothesis=False,
3939             ),
3940         ]
3941     )
3942
3943     result = materialise_insight_ledger(
3944         candidates=candidates,
3945         verification=verification,
3946         fact_ledger=ledger,
3947         evidence_ledger=evidence,
3948         settings=Settings(),
3949     )

```

```

3950
3951     assert [item.insight_id for item in result.verified_insights] == [
3952         valid_candidate.insight_id
3953     ]
3954     assert [item.insight_id for item in result.unverified_insights] == [
3955         unresolved_candidate.insight_id
3956     ]
3957     assert not result.rejected_insights
3958
3959
3960 def test_hypotheses_remain_separate_under_both_policies():
3961     ledger, evidence = _insight_fixture()
3962     candidate = _insight_candidate(
3963         insight_id="INSIGHT_HYP",
3964         statement=(
3965             "Hypothesis: the observed association may reflect an unmeasured "
3966             "process."
3967         ),
3968         interpretation_level=InterpretationLevel.HYPOTHESIS,
3969         suitable_for_main_report=False,
3970     )
3971     candidates = InsightCandidateSet(candidates=[candidate])
3972     verification = InsightVerificationResult(
3973         records=[
3974             InsightVerificationRecord(
3975                 insight_id="INSIGHT_HYP",
3976                 status=InsightVerificationStatus.HYPOTHESIS_ONLY,
3977                 confidence=0.8,
3978                 salience=0.7,
3979                 adds_bounded_synthesis=False,
3980                 analytical_implication_supported=False,
3981                 contains_hypothesis=True,
3982             )
3983         ]
3984     )
3985
3986     for allow in [False, True]:
3987         settings = replace(
3988             Settings(),
3989             allow_hypotheses_in_report=allow,
3990         )
3991         assert not validate_insight_candidates(
3992             candidates,
3993             ledger,
3994             evidence,
3995             settings,
3996         )
3997         result = materialise_insight_ledger(
3998             candidates=candidates,
3999             verification=verification,
4000             fact_ledger=ledger,
4001             evidence_ledger=evidence,
4002             settings=settings,
4003         )
4004         assert not result.verified_insights
4005         assert [
4006             insight.insight_id
4007             for insight in result.hypothesis_only_insights
4008         ] == ["INSIGHT_HYP"]
4009
4010
4011 def test_insight_ledger_materialisation_retains_all_eligible_insights():
4012     ledger, evidence = _insight_fixture()
4013     first = _insight_candidate(
4014         insight_id="INSIGHT_FIRST",
4015         salience=0.95,
4016     )
4017     second = _insight_candidate(
4018         insight_id="INSIGHT_SECOND",
4019         statement=(
4020             "The two verified association summaries provide overlapping "
4021             "directional information in this dataset."
4022         ),
4023         insight_type=InsightType.REDUNDANCY,
4024         salience=0.8,
4025     )
4026     rejected = _insight_candidate(
4027         insight_id="INSIGHT_REJECTED",
4028         statement=(
4029             "The verified findings form a bounded narrative summary for this "
4030             "dataset."
4031         ),
4032         insight_type=InsightType.NARRATIVE_SUMMARY,
4033         salience=0.7,
4034     )
4035     candidates = InsightCandidateSet(
4036         candidates=[first, second, rejected]
4037     )
4038     verification = InsightVerificationResult(
4039         records=[
4040             InsightVerificationRecord(

```

```

4041         insight_id="INSIGHT_FIRST",
4042         status=InsightVerificationStatus.VERIFIED_WITH_CAVEAT,
4043         verified_statement=(
4044             first.statement
4045             + " This remains a descriptive association."
4046         ),
4047         confidence=0.9,
4048         salience=0.95,
4049         adds_bounded_synthesis=True,
4050         analytical_implication_supported=True,
4051         contains_hypothesis=False,
4052         limitations=["No causal conclusion is supported."],
4053     ),
4054     InsightVerificationRecord(
4055         insight_id="INSIGHT_SECOND",
4056         status=InsightVerificationStatus.VERIFIED,
4057         confidence=0.85,
4058         salience=0.8,
4059         adds_bounded_synthesis=True,
4060         analytical_implication_supported=True,
4061         contains_hypothesis=False,
4062     ),
4063     InsightVerificationRecord(
4064         insight_id="INSIGHT_REJECTED",
4065         status=InsightVerificationStatus.REJECTED,
4066         confidence=0.9,
4067         salience=0.7,
4068         adds_bounded_synthesis=False,
4069         analytical_implication_supported=False,
4070         contains_hypothesis=False,
4071         verification_notes=["The synthesis is not sufficiently useful."],
4072     ),
4073 ]
4074 )
4075 settings = replace(
4076     Settings(),
4077     max_verified_main_insights=1,
4078 )
4079
4080 result = materialise_insight_ledger(
4081     candidates=candidates,
4082     verification=verification,
4083     fact_ledger=ledger,
4084     evidence_ledger=evidence,
4085     settings=settings,
4086 )
4087
4088 assert [
4089     insight.insight_id
4090     for insight in result.verified_insights
4091 ] == ["INSIGHT_FIRST", "INSIGHT_SECOND"]
4092 assert result.verified_insights[0].verification_status == (
4093     InsightVerificationStatus.VERIFIED_WITH_CAVEAT
4094 )
4095 assert "descriptive association" in result.verified_insights[0].statement
4096 assert {
4097     rejection.insight_id
4098     for rejection in result.rejected_insights
4099 } == {"INSIGHT_REJECTED"}
4100
4101
4102 def test_writer_payload_contains_only_writer_eligible_insights():
4103     ledger, evidence = _insight_fixture()
4104     main_candidate = _insight_candidate()
4105     hypothesis_candidate = _insight_candidate(
4106         insight_id="INSIGHT_HYP",
4107         statement="Hypothesis: another process may contribute.",
4108         interpretation_level=InterpretationLevel.HYPOTHESIS,
4109         suitable_for_main_report=False,
4110     )
4111     insight_ledger = InsightLedger(
4112         verified_insights=[_verified_insight(main_candidate)],
4113         hypothesis_only_insights=[
4114             _verified_insight(
4115                 hypothesis_candidate,
4116                 status=InsightVerificationStatus.HYPOTHESIS_ONLY,
4117             )
4118         ],
4119         rejected_insights=[
4120             InsightRejection(
4121                 insight_id="INSIGHT_BAD",
4122                 candidate=_insight_candidate(insight_id="INSIGHT_BAD"),
4123                 reasons=["Rejected."],
4124             )
4125         ],
4126     )
4127     understanding = DataUnderstanding(
4128         profile_fingerprint="insight-test",
4129         dataset_summary="Unverified prose must remain private.",
4130         tables=[],
4131     )

```

```

4132     plan = ExecutionPlan(
4133         objective="Describe the strongest findings.",
4134         tasks=[],
4135         route_order=[],
4136         report_specification=_basic_spec(),
4137         audit_mode=AuditMode.INTERNAL,
4138         revision_limit=1,
4139         maximum_facts=10,
4140         rationale="Test.",
4141     )
4142     pack = build_writer_evidence_pack(
4143         request=plan.objective,
4144         understanding=understanding,
4145         plan=plan,
4146         evidence=evidence,
4147         fact_ledger=ledger,
4148         settings=Settings(),
4149         insight_ledger=insight_ledger,
4150     )
4151     payload = build_compact_writer_payload(pack)
4152
4153     assert payload["priority_verified_insights"]
4154     assert payload["priority_verified_insights"][0].source_fact_ids
4155     assert payload["priority_facts"]
4156     assert payload["genre"] == ReportGenre.DATA_SCIENCE_REPORT
4157     assert payload["perspective"] == ReportPerspective.NEUTRAL
4158     assert payload["communication_goal"]
4159     assert payload["hypothesis_only_insights"] == []
4160     assert payload["verified_strength_labels_by_fact_id"][
4161         "FACT_INS_001"
4162     ] == ["strong_association"]
4163     assert "INSIGHT_BAD" not in str(payload)
4164     assert "Unverified prose" not in str(payload)
4165
4166 def test_writer_materialisation_expands_insight_provenance_without_text_change():
4167     ledger, _ = _insight_fixture()
4168     insight = _verified_insight()
4169     insight_ledger = InsightLedger(verified_insights=[insight])
4170     draft = WriterAgentDraft(
4171         title="Bounded report",
4172         sections=[
4173             WriterSectionDraft(
4174                 heading="Main insight",
4175                 sentences=[
4176                     WriterSentenceDraft(
4177                         text=insight.statement,
4178                         insight_ids=[insight.insight_id],
4179                         interpretation_level=(
4180                             InterpretationLevel.BOUNDED_INSIGHT
4181                         ),
4182                         support_type=SupportType.MULTI_FACT_SYNTHESIS,
4183                     ),
4184                 ],
4185             ),
4186         ],
4187     )
4188
4189     output = materialise_writer_output(
4190         draft,
4191         ledger,
4192         insight_ledger=insight_ledger,
4193     )
4194     support = output.sentence_support[0]
4195
4196     assert insight.statement in output.markdown
4197     assert support.insight_ids == [insight.insight_id]
4198     assert support.interpretation_level == InterpretationLevel.BOUNDED_INSIGHT
4199     assert set(support.fact_ids) == set(insight.source_fact_ids)
4200     assert set(support.evidence_ids) == set(insight.source_evidence_ids)
4201
4202 def test_writer_hypothesis_requires_enabled_separate_section():
4203     ledger, _ = _insight_fixture()
4204     candidate = _insight_candidate(
4205         insight_id="INSIGHT_HYP_WRITER",
4206         statement="Hypothesis: an unmeasured process may contribute.",
4207         interpretation_level=InterpretationLevel.HYPOTHESIS,
4208         suitable_for_main_report=False,
4209     )
4210     hypothesis = _verified_insight(
4211         candidate,
4212         status=InsightVerificationStatus.HYPOTHESIS_ONLY,
4213     )
4214     insight_ledger = InsightLedger(
4215         hypothesis_only_insights=[hypothesis]
4216     )
4217     draft = WriterAgentDraft(
4218         title="Further investigation",
4219         sections=[
4220             WriterSectionDraft(

```

```

4223         heading="Questions for Further Investigation",
4224         sentences=[
4225             WriterSentenceDraft(
4226                 text=hypothesis.statement,
4227                 insight_ids=[hypothesis.insight_id],
4228                 interpretation_level=InterpretationLevel.HYPOTHESIS,
4229                 support_type=SupportType.MULTI_FACT_SYNTHESIS,
4230             ),
4231         ],
4232     ),
4233 ],
4234 )
4235
4236 with pytest.raises(ValueError, match="disabled"):
4237     materialise_writer_output(
4238         draft,
4239         ledger,
4240         insight_ledger=insight_ledger,
4241     )
4242
4243 output = materialise_writer_output(
4244     draft,
4245     ledger,
4246     insight_ledger=insight_ledger,
4247     allow_hypotheses_in_report=True,
4248 )
4249 assert output.sentence_support[0].interpretation_level == (
4250     InterpretationLevel.HYPOTHESIS
4251 )
4252
4253 def test_writer_rejects_explanatory_hypothesis_disguised_as_next_step():
4254     ledger, _ = _insight_fixture()
4255     sentence = (
4256         "Further analysis could explore whether the association reflects a "
4257         "data artifact."
4258     )
4259     draft = WriterAgentDraft(
4260         title="Unsupported explanation",
4261         sections=[
4262             WriterSectionDraft(
4263                 heading="Limitations and next steps",
4264                 sentences=[
4265                     WriterSentenceDraft(
4266                         text=sentence,
4267                         fact_ids=["FACT_INS_001", "FACT_INS_002"],
4268                         support_type=SupportType.PARAPHRASE,
4269                     ),
4270                 ],
4271             ),
4272         ],
4273     )
4274
4275 with pytest.raises(ValueError, match="possible explanation"):
4276     materialise_writer_output(
4277         draft,
4278         ledger,
4279         insight_ledger=InsightLedger(),
4280     )
4281
4282 def test_writer_validation_rejects_unknown_or_missing_insight_mapping():
4283     ledger, _ = _insight_fixture()
4284     insight = _verified_insight()
4285     insight_ledger = InsightLedger(verified_insights=[insight])
4286
4287 with pytest.raises(ValueError, match="unknown insight"):
4288     materialise_writer_output(
4289         WriterAgentDraft(
4290             title="Unknown",
4291             sections=[
4292                 WriterSectionDraft(
4293                     heading="Main",
4294                     sentences=[
4295                         WriterSentenceDraft(
4296                             text=insight.statement,
4297                             insight_ids=["INSIGHT_UNKNOWN"],
4298                             interpretation_level=(
4299                                 InterpretationLevel.BOUNDED_INSIGHT
4300                             ),
4301                             support_type=SupportType.MULTI_FACT_SYNTHESIS,
4302                         ),
4303                     ],
4304                 ),
4305             ],
4306         ),
4307         ledger,
4308         insight_ledger=insight_ledger,
4309     )
4310
4311 sentence = insight.statement
4312
4313

```

```

4314     invalid = WriterOutput(
4315         title="Missing mapping",
4316         markdown=f"# Missing mapping\n\n{sentence}\n",
4317         sentence_support=[
4318             SentenceSupport(
4319                 sentence_id="SENT_0001",
4320                 sentence_text=sentence,
4321                 fact_ids=insight.source_fact_ids,
4322                 evidence_ids=insight.source_evidence_ids,
4323                 interpretation_level=InterpretationLevel.BOUNDED_INSIGHT,
4324                 support_type=SupportType.MULTI_FACT_SYNTHESIS,
4325             )
4326         ],
4327         selected_fact_ids=insight.source_fact_ids,
4328     )
4329
4330     assert any(
4331         "bounded insight" in error
4332         for error in validate_writer_output(
4333             invalid,
4334             ledger,
4335             insight_ledger,
4336         )
4337     )
4338
4339
4340 def test_direct_finding_remains_valid_without_insight_id():
4341     ledger, _ = _insight_fixture()
4342     sentence = ledger.writer_ready_facts[0].fact_summary
4343     output = WriterOutput(
4344         title="Finding",
4345         markdown=f"# Finding\n\n{sentence}\n",
4346         sentence_support=[
4347             SentenceSupport(
4348                 sentence_id="SENT_0001",
4349                 sentence_text=sentence,
4350                 fact_ids=["FACT_INS_001"],
4351                 evidence_ids=["EVD_INS_001"],
4352                 support_type=SupportType.DIRECT,
4353             )
4354         ],
4355         selected_fact_ids=["FACT_INS_001"],
4356     )
4357
4358     assert not validate_writer_output(output, ledger, InsightLedger())
4359
4360
4361 def test_writer_entity_grounding_is_case_insensitive_and_diagnostic():
4362     ledger, _ = _insight_fixture()
4363     fact_lookup = {
4364         fact.fact_id: fact
4365         for fact in ledger.writer_ready_facts
4366     }
4367     case_variant = WriterSentenceDraft(
4368         text=(
4369             "The `humidity` and `temperature` fields are associated in this "
4370             "dataset."
4371         ),
4372         fact_ids=["FACT_INS_001"],
4373         support_type=SupportType.PARAPHRASE,
4374     )
4375
4376     assert not writer_sentence_grounding_errors(
4377         sentence=case_variant,
4378         fact_lookup=fact_lookup,
4379         insight_lookup={},
4380         sentence_label="Sentence 1.1",
4381     )
4382
4383     unsupported = WriterSentenceDraft(
4384         text="The `Pressure` and `Wind` fields are associated.",
4385         fact_ids=["FACT_INS_001"],
4386         support_type=SupportType.PARAPHRASE,
4387     )
4388     early_errors = writer_sentence_grounding_errors(
4389         sentence=unsupported,
4390         fact_lookup=fact_lookup,
4391         insight_lookup={},
4392         sentence_label="Sentence 1.2",
4393     )
4394
4395     assert early_errors == [
4396         "Sentence 1.2 contains unsupported entities ['Pressure', 'Wind']; "
4397         "mapped fact IDs: ['FACT_INS_001'].",
4398     ]
4399
4400     unsupported_number = case_variant.model_copy(
4401         update={
4402             "text": (
4403                 "The `humidity` and `temperature` fields have a correlation "
4404                 "of 99."

```



```

4405         )
4406     }
4407 )
4408 assert any(
4409     "number unsupported" in error
4410     for error in writer_sentence_grounding_errors(
4411         sentence=unsupported_number,
4412         fact_lookup=fact_lookup,
4413         insight_lookup={},
4414         sentence_label="Sentence 1.3",
4415     )
4416 )
4417
4418 sentence = unsupported.text
4419 output = WriterOutput(
4420     title="Unsupported entities",
4421     markdown=f"# Unsupported entities\n\n{sentence}\n",
4422     sentence_support=[
4423         SentenceSupport(
4424             sentence_id="SENT_0001",
4425             sentence_text=sentence,
4426             fact_ids=["FACT_INS_001"],
4427             evidence_ids=["EVD_INS_001"],
4428             support_type=SupportType.PARAPHRASE,
4429         )
4430     ],
4431     selected_fact_ids=["FACT_INS_001"],
4432 )
4433 late_entity_errors = [
4434     error
4435     for error in validate_writer_output(
4436         output,
4437         ledger,
4438         InsightLedger(),
4439     )
4440     if "unsupported entities" in error
4441 ]
4442
4443 assert late_entity_errors == [
4444     "SENT_0001 contains unsupported entities ['Pressure', 'Wind']; "
4445     "mapped fact IDs: ['FACT_INS_001']."
4446 ]
4447
4448 def _audit_verified_insight_sentence(
4449     sentence: str,
4450     *,
4451     insight: VerifiedInsight | None = None,
4452 ) -> AuditReport:
4453     ledger, evidence = _insight_fixture()
4454     insight = insight or _verified_insight()
4455     output = WriterOutput(
4456         title="Insight audit",
4457         markdown=f"# Insight audit\n\n## Main insight\n\n{sentence}\n",
4458         sentence_support=[
4459             SentenceSupport(
4460                 sentence_id="SENT_0001",
4461                 sentence_text=sentence,
4462                 fact_ids=insight.source_fact_ids,
4463                 evidence_ids=insight.source_evidence_ids,
4464                 insight_ids=[insight.insight_id],
4465                 interpretation_level=InterpretationLevel.BOUNDED_INSIGHT,
4466                 support_type=SupportType.MULTI_FACT_SYNTHESIS,
4467             )
4468         ],
4469         selected_fact_ids=insight.source_fact_ids,
4470     )
4471
4472     return deterministic_audit(
4473         output,
4474         ledger,
4475         evidence,
4476         AuditMode.INTERNAL,
4477         [],
4478         0,
4479         _basic_spec(),
4480         Settings(),
4481         insight_ledger=InsightLedger(
4482             verified_insights=[insight]
4483         ),
4484     )
4485
4486 def test_verified_insight_sentence_is_not_treated_as_hallucination():
4487     audit = _audit_verified_insight_sentence(
4488         _verified_insight().statement
4489     )
4490
4491     assert not [
4492         annotation
4493         for annotation in audit.annotations
4494         if annotation.subtype

```

```

4496         in {
4497             "unsupported_insight",
4498             "insight_exceeds_verified_wording",
4499         }
4500     ]
4501     assert not any(
4502         "lists findings without relating" in finding
4503         for finding in audit.quality_assessment.findings
4504     )
4505
4506
4507 def test_quality_warns_when_available_insights_are_not_used():
4508     ledger, evidence = _insight_fixture()
4509     insight = _verified_insight()
4510     sentence = ledger.writer_ready_facts[0].fact_summary
4511     output = WriterOutput(
4512         title="Fact list",
4513         markdown=f"# Fact list\n\n{sentence}\n",
4514         sentence_support=[
4515             SentenceSupport(
4516                 sentence_id="SENT_0001",
4517                 sentence_text=sentence,
4518                 fact_ids=["FACT_INS_001"],
4519                 evidence_ids=["EVD_INS_001"],
4520                 support_type=SupportType.DIRECT,
4521             )
4522         ],
4523         selected_fact_ids=["FACT_INS_001"],
4524     )
4525     audit = deterministic_audit(
4526         output,
4527         ledger,
4528         evidence,
4529         AuditMode.INTERNAL,
4530         [],
4531         0,
4532         _basic_spec(),
4533         Settings(),
4534         insight_ledger=InsightLedger(
4535             verified_insights=[insight]
4536         ),
4537     )
4538
4539     assert any(
4540         "lists findings without relating" in finding
4541         for finding in audit.quality_assessment.findings
4542     )
4543
4544
4545 def test_quality_requires_verified_analytical_implication():
4546     insight = _verified_insight()
4547     restatement_only = _audit_verified_insight_sentence(
4548         insight.statement,
4549         insight=insight,
4550     )
4551
4552     assert any(
4553         "does not explain its supported analytical implication" in finding
4554         for finding in restatement_only.quality_assessment.findings
4555     )
4556
4557     ledger, evidence = _insight_fixture()
4558     markdown = (
4559         "# Insight audit\n\n## Main insight\n\n"
4560         f"{insight.statement}\n\n{insight.why_it_matters}\n"
4561     )
4562     output = WriterOutput(
4563         title="Insight audit",
4564         markdown=markdown,
4565         sentence_support=[
4566             SentenceSupport(
4567                 sentence_id="SENT_0001",
4568                 sentence_text=insight.statement,
4569                 fact_ids=insight.source_fact_ids,
4570                 evidence_ids=insight.source_evidence_ids,
4571                 insight_ids=[insight.insight_id],
4572                 interpretation_level=InterpretationLevel.BOUNDED_INSIGHT,
4573                 support_type=SupportType.MULTI_FACT_SYNTHESIS,
4574             ),
4575             SentenceSupport(
4576                 sentence_id="SENT_0002",
4577                 sentence_text=insight.why_it_matters,
4578                 fact_ids=insight.source_fact_ids,
4579                 evidence_ids=insight.source_evidence_ids,
4580                 insight_ids=[insight.insight_id],
4581                 interpretation_level=InterpretationLevel.BOUNDED_INSIGHT,
4582                 support_type=SupportType.MULTI_FACT_SYNTHESIS,
4583             ),
4584         ],
4585         selected_fact_ids=insight.source_fact_ids,
4586     )

```

```

4587     with_implication = deterministic_audit(
4588         output,
4589         ledger,
4590         evidence,
4591         AuditMode.INTERNAL,
4592         [],
4593         0,
4594         _basic_spec(),
4595         Settings(),
4596         insight_ledger=InsightLedger(verified_insights=[insight]),
4597     )
4598
4599     assert not any(
4600         "does not explain its supported analytical implication" in finding
4601         for finding in with_implication.quality_assessment.findings
4602     )
4603
4604
4605 def test_audit_flags_strength_classification_inconsistency():
4606     ledger, evidence = _insight_fixture()
4607     sentence = "The two measures have moderate correlations."
4608     output = WriterOutput(
4609         title="Strength mismatch",
4610         markdown=f"# Strength mismatch\n\n{sentence}\n",
4611         sentence_support=[
4612             SentenceSupport(
4613                 sentence_id="SENT_0001",
4614                 sentence_text=sentence,
4615                 fact_ids=["FACT_INS_001", "FACT_INS_002"],
4616                 evidence_ids=["EVD_INS_001", "EVD_INS_002"],
4617                 support_type=SupportType.PARAPHRASE,
4618             )
4619         ],
4620         selected_fact_ids=["FACT_INS_001", "FACT_INS_002"],
4621     )
4622     audit = deterministic_audit(
4623         output,
4624         ledger,
4625         evidence,
4626         AuditMode.INTERNAL,
4627         [],
4628         0,
4629         _basic_spec(),
4630         Settings(),
4631     )
4632
4633     assert any(
4634         annotation.subtype == "inconsistent_strength_label"
4635         for annotation in audit.annotations
4636     )
4637
4638
4639 def test_insight_wording_escalation_is_flagged():
4640     audit = _audit_verified_insight_sentence(
4641         "One measure is completely redundant and should always be removed."
4642     )
4643
4644     assert any(
4645         annotation.subtype == "insight_exceeds_verified_wording"
4646         for annotation in audit.annotations
4647     )
4648
4649
4650 def test_unlabelled_hypothesis_is_flagged():
4651     ledger, evidence = _insight_fixture()
4652     sentence = (
4653         "The dataset pattern may reflect lower temperature because of an "
4654         "unmeasured process."
4655     )
4656     output = WriterOutput(
4657         title="Hypothesis",
4658         markdown=f"# Hypothesis\n\n{sentence}\n",
4659         sentence_support=[
4660             SentenceSupport(
4661                 sentence_id="SENT_0001",
4662                 sentence_text=sentence,
4663                 fact_ids=["FACT_INS_001", "FACT_INS_002"],
4664                 evidence_ids=["EVD_INS_001", "EVD_INS_002"],
4665                 support_type=SupportType.PARAPHRASE,
4666             )
4667         ],
4668         selected_fact_ids=["FACT_INS_001", "FACT_INS_002"],
4669     )
4670     audit = deterministic_audit(
4671         output,
4672         ledger,
4673         evidence,
4674         AuditMode.INTERNAL,
4675         [],
4676         0,
4677         _basic_spec(),

```

```

4678         Settings(),
4679     )
4680
4681     assert any(
4682         annotation.subtype == "unlabelled_hypothesis"
4683         for annotation in audit.annotations
4684     )
4685
4686
4687 def test_unsupported_sports_chronology_is_flagged():
4688     ledger, evidence = _insight_fixture()
4689     sentence = "Player X led a dramatic comeback."
4690     fact = ledger.writer_ready_facts[0].model_copy(
4691         update={
4692             "fact_summary": "Player X scored for Team A.",
4693             "entities": ["Player X", "Team A"],
4694         }
4695     )
4696     ledger = FactLedger(writer_ready_facts=[fact])
4697     output = WriterOutput(
4698         title="Game",
4699         markdown=f"# Game\n\n{sentence}\n",
4700         sentence_support=[
4701             SentenceSupport(
4702                 sentence_id="SENT_0001",
4703                 sentence_text=sentence,
4704                 fact_ids=[fact.fact_id],
4705                 evidence_ids=fact.evidence_ids,
4706                 support_type=SupportType.PARAPHRASE,
4707             )
4708         ],
4709         selected_fact_ids=[fact.fact_id],
4710     )
4711     audit = deterministic_audit(
4712         output,
4713         ledger,
4714         evidence,
4715         AuditMode.INTERNAL,
4716         [],
4717         0,
4718         _basic_spec(),
4719         Settings(),
4720     )
4721
4722     assert any(
4723         annotation.subtype == "unsupported_sports_narrative"
4724         for annotation in audit.annotations
4725     )
4726
4727
4728 def test_safe_sports_synthesis_requires_no_chronology():
4729     statements = [
4730         "Team A won the game.",
4731         "Player X and Player Y shared the Team A scoring lead.",
4732         "Team A recorded more rebounds.",
4733         "Team A recorded fewer turnovers.",
4734     ]
4735     evidence_items = [
4736         EvidenceItem(
4737             evidence_id=f"EVD_GAME_{index:03d}",
4738             route=AnalysisRoute.DESCRPTIVE,
4739             task_ids=["TASK_GAME"],
4740             finding=statement,
4741             metrics={},
4742             source_tables=["game"],
4743             source_columns=["Team", "Player", "Points", "Rebounds", "Turnovers"],
4744             method="Direct deterministic game summary.",
4745             practical_interpretation=statement,
4746             strength_label="game_fact",
4747             claim_permissions=[ClaimPermission.DESCRPTIVE],
4748             factual_confidence=1.0,
4749             methodological_strength=1.0,
4750             user_relevance=1.0,
4751             salience=1.0,
4752             recommended_use=RecommendedUse.MAIN_FINDING,
4753         )
4754         for index, statement in enumerate(statements, start=1)
4755     ]
4756     evidence = EvidenceLedger(
4757         fingerprint="game",
4758         items=evidence_items,
4759     )
4760     facts = [
4761         VerifiedFact(
4762             fact_id=f"FACT_GAME_{index:03d}",
4763             source_candidate_id=f"CAN_GAME_{index:03d}",
4764             fact_summary=item.finding,
4765             evidence_ids=[item.evidence_id],
4766             entities=[
4767                 "game",
4768                 "Team A",

```

```

4769         "Player X",
4770         "Player Y",
4771         "Team",
4772         "Player",
4773         "Points",
4774         "Rebounds",
4775         "Turnovers",
4776     ],
4777     claim_permissions=[ClaimPermission.DESCRPTIVE],
4778     factual_confidence=1.0,
4779     methodological_strength=1.0,
4780     user_relevance=1.0,
4781     salience=1.0,
4782     recommended_use=RecommendedUse.MAIN_FINDING,
4783 )
4784     for index, item in enumerate(evidence_items, start=1)
4785 ]
4786 ledger = FactLedger(writer_ready_facts=facts)
4787 sentence = (
4788     "Team A combined a shared scoring lead with advantages in rebounds "
4789     "and turnovers."
4790 )
4791 insight = VerifiedInsight(
4792     insight_id="INSIGHT_GAME",
4793     statement=sentence,
4794     insight_type=InsightType.NARRATIVE_SUMMARY,
4795     interpretation_level=InterpretationLevel.BOUNDED_INSIGHT,
4796     source_fact_ids=[fact.fact_id for fact in facts],
4797     source_evidence_ids=[item.evidence_id for item in evidence_items],
4798     why_it_matters="It provides a bounded game narrative.",
4799     claim_permissions=[ClaimPermission.DESCRPTIVE],
4800     confidence=0.95,
4801     salience=0.95,
4802     verification_status=InsightVerificationStatus.VERIFIED,
4803 )
4804 output = WriterOutput(
4805     title="Game report",
4806     markdown=f"# Game report\n\n## Game narrative\n\n{sentence}\n",
4807     sentence_support=[
4808         SentenceSupport(
4809             sentence_id="SENT_0001",
4810             sentence_text=sentence,
4811             fact_ids=insight.source_fact_ids,
4812             evidence_ids=insight.source_evidence_ids,
4813             insight_ids=[insight.insight_id],
4814             interpretation_level=InterpretationLevel.BOUNDED_INSIGHT,
4815             support_type=SupportType.MULTI_FACT_SYNTHESIS,
4816         )
4817     ],
4818     selected_fact_ids=insight.source_fact_ids,
4819 )
4820 spec = _basic_spec().model_copy(
4821     update={"genre": ReportGenre.SPORTS_GAME_REPORT}
4822 )
4823 audit = deterministic_audit(
4824     output,
4825     ledger,
4826     evidence,
4827     AuditMode.INTERNAL,
4828     [],
4829     0,
4830     spec,
4831     Settings(),
4832     insight_ledger=InsightLedger(
4833         verified_insights=[insight]
4834     ),
4835 )
4836
4837 assert not audit.annotations
4838
4839 def test_insight_feature_flag_ablation_keeps_fact_writer_path(tmp_path):
4840     path = tmp_path / "ablation.csv"
4841     pd.DataFrame(
4842         {
4843             "group": ["a", "b"] * 60,
4844             "value": np.arange(120),
4845         }
4846     ).to_csv(path, index=False)
4847     settings = replace(
4848         Settings(),
4849         use_llm=False,
4850         enable_insight_synthesis=False,
4851         output_dir=tmp_path / "runs",
4852     )
4853
4854     result = Table2TextWorkflow(settings).run_sync(
4855         [path],
4856         "Understand the dataset.",
4857     )
4858
4859

```

```

4860     assert not result.insight_ledger.synthesis_enabled
4861     assert "disabled by configuration" in (
4862         result.insight_ledger.fallback_reason or ""
4863     )
4864     assert result.raw_writer_output.writer_mode == "deterministic_fallback"
4865     assert (
4866         settings.output_dir
4867         / result.run_id
4868         / "07_insight_ledger.json"
4869     ).exists()
4870
4871
4872 def test_insight_stage_failure_continues_without_changing_llm_writer_mode(
4873     tmp_path,
4874 ):
4875     path = tmp_path / "failure.csv"
4876     pd.DataFrame(
4877         {
4878             "group": ["a", "b"] * 60,
4879             "value": np.arange(120),
4880         }
4881     ).to_csv(path, index=False)
4882     settings = replace(
4883         Settings(),
4884         use_llm=True,
4885         output_dir=tmp_path / "runs",
4886         writer_quality_revision_rounds=0,
4887     )
4888     workflow = Table2TextWorkflow(settings)
4889
4890     async def deterministic_regular_fallback(
4891         self,
4892         *,
4893         stage,
4894         dependencies,
4895         fallback,
4896         **_,
4897     ):
4898         if stage == "natural_writer":
4899             ledger = FactLedger.model_validate(
4900                 dependencies.payload["fact_ledger"]
4901             )
4902             fact = ledger.writer_ready_facts[0]
4903             return WriterAgentDraft(
4904                 title="Supported report",
4905                 sections=[
4906                     WriterSectionDraft(
4907                         heading="Dataset overview",
4908                         sentences=[
4909                             WriterSentenceDraft(
4910                                 text=fact.fact_summary,
4911                                 fact_ids=[fact.fact_id],
4912                                 support_type=SupportType.DIRECT,
4913                             )
4914                         ],
4915                     )
4916                 ],
4917             )
4918             return fallback()
4919
4920     async def failed_optional_stage(self, **_):
4921         return None, "simulated insight-stage failure"
4922
4923     workflow.run_agent_or_fallback = MethodType(
4924         deterministic_regular_fallback,
4925         workflow,
4926     )
4927     workflow.run_optional_insight_agent = MethodType(
4928         failed_optional_stage,
4929         workflow,
4930     )
4931
4932     result = workflow.run_sync(
4933         [path],
4934         "Understand the dataset.",
4935     )
4936
4937     assert "simulated insight-stage failure" in (
4938         result.insight_ledger.fallback_reason or ""
4939     )
4940     assert result.raw_writer_output.writer_mode == "llm_writer"
4941     assert result.release_status in {
4942         ReleaseStatus.APPROVED,
4943         ReleaseStatus.APPROVED_WITH_WARNINGS,
4944         ReleaseStatus.HUMAN_REVIEW_REQUIRED,
4945     }
4946     assert result.model_dump_json()
4947
4948
4949 def test_late_writer_materialisation_failure_uses_deterministic_fallback(
4950     tmp_path,

```

```

4951 monkeypatch,
4952 ):
4953     path = tmp_path / "materialisation.csv"
4954     pd.DataFrame(
4955         {
4956             "group": ["a", "b"] * 60,
4957             "value": np.arange(120),
4958         }
4959     ).to_csv(path, index=False)
4960     settings = replace(
4961         Settings(),
4962         use_llm=True,
4963         output_dir=tmp_path / "runs",
4964         writer_quality_revision_rounds=0,
4965     )
4966     workflow = Table2TextWorkflow(settings)
4967
4968     async def deterministic_regular_fallback(
4969         self,
4970         *,
4971         stage,
4972         dependencies,
4973         fallback,
4974         **_,
4975     ):
4976         if stage == "natural_writer":
4977             ledger = FactLedger.model_validate(
4978                 dependencies.payload["fact_ledger"]
4979             )
4980             fact = ledger.writer_ready_facts[0]
4981             return WriterAgentDraft(
4982                 title="Materialisation candidate",
4983                 sections=[
4984                     WriterSectionDraft(
4985                         heading="Dataset overview",
4986                         sentences=[
4987                             WriterSentenceDraft(
4988                                 text=fact.fact_summary,
4989                                 fact_ids=[fact.fact_id],
4990                                 support_type=SupportType.DIRECT,
4991                             )
4992                         ],
4993                     )
4994                 ],
4995             )
4996             return fallback()
4997
4998     async def failed_optional_stage(self, **_):
4999         return None, "simulated insight-stage failure"
5000
5001     def failed_materialisation(*_, **_):
5002         raise ValueError("simulated late entity mismatch")
5003
5004     workflow.run_agent_or_fallback = MethodType(
5005         deterministic_regular_fallback,
5006         workflow,
5007     )
5008     workflow.run_optional_insight_agent = MethodType(
5009         failed_optional_stage,
5010         workflow,
5011     )
5012     monkeypatch.setattr(
5013         "table2text.workflow.materialise_writer_output",
5014         failed_materialisation,
5015     )
5016
5017     result = workflow.run_sync(
5018         [path],
5019         "Understand the dataset.",
5020     )
5021     run_directory = settings.output_dir / result.run_id
5022
5023     assert result.raw_writer_output.writer_mode == "deterministic_fallback"
5024     assert (run_directory / "09_writer_structured_draft.json").exists()
5025     error_path = run_directory / "09_writer_materialisation_error.txt"
5026     assert error_path.exists()
5027     assert "simulated late entity mismatch" in error_path.read_text()
5028
5029
5030 def test_empty_insight_ledger_fallback_is_explicit():
5031     ledger = empty_insight_ledger(
5032         synthesis_enabled=True,
5033         fallback_reason="request budget exhausted",
5034     )
5035
5036     assert ledger.synthesis_enabled
5037     assert not ledger.verified_insights
5038     assert ledger.fallback_reason == "request budget exhausted"
5039
5040
5041 def _nested_event_fixture(reference_text: str) -> dict:

```

```

5042     return {
5043         "event_id": "EVENT-001",
5044         "date": {"year": 2026, "month": 7, "day": 23},
5045         "venue": {"city": "Example City", "name": "Example Arena"},
5046         "overtime": False,
5047         "participants": {
5048             "home": {
5049                 "name": "Alpha",
5050                 "statistics": {
5051                     "team": {
5052                         "game": {
5053                             "points": 90,
5054                             "rebounds": 40,
5055                             "assists": 20,
5056                         }
5057                     },
5058                     "entities": {
5059                         "alpha_one": {
5060                             "name": "Alex One",
5061                             "points": 25,
5062                             "rebounds": 8,
5063                             "assists": 6,
5064                         },
5065                         "alpha_two": {
5066                             "name": "Alex Two",
5067                             "points": 18,
5068                             "rebounds": 5,
5069                             "assists": 4,
5070                         },
5071                     },
5072                 },
5073             },
5074             "visitor": {
5075                 "name": "Beta",
5076                 "statistics": {
5077                     "team": {
5078                         "game": {
5079                             "points": 80,
5080                             "rebounds": 35,
5081                             "assists": 17,
5082                         }
5083                     },
5084                     "entities": {
5085                         "beta_one": {
5086                             "name": "Blair One",
5087                             "points": 22,
5088                             "rebounds": 9,
5089                             "assists": 3,
5090                         },
5091                     },
5092                 },
5093             },
5094         },
5095         "reference_text": reference_text,
5096     }
5097
5098
5099 def _event_field_policy() -> EvaluationFieldPolicy:
5100     return EvaluationFieldPolicy(
5101         operational_input_paths=[
5102             "event_id",
5103             "date",
5104             "venue",
5105             "overtime",
5106             "participants",
5107         ],
5108         held_out_reference_paths=["reference_text"],
5109     )
5110
5111
5112 def _write_nested_event(tmp_path, reference_text: str):
5113     path = tmp_path / "nested_event.json"
5114     path.write_text(
5115         json.dumps(_nested_event_fixture(reference_text)),
5116         encoding="utf-8",
5117     )
5118     return path
5119
5120
5121 def _stringify_numbers(value):
5122     if isinstance(value, bool) or value is None:
5123         return value
5124     if isinstance(value, int | float):
5125         return str(value)
5126     if isinstance(value, dict):
5127         return {key: _stringify_numbers(item) for key, item in value.items()}
5128     if isinstance(value, list):
5129         return [_stringify_numbers(item) for item in value]
5130     return value
5131
5132

```



```

5133 def test_nested_event_is_one_record_and_explicit_reference_is_held_out(
5134     tmp_path,
5135 ):
5136     reference_text = "REFERENCE SENTINEL " * 80
5137     path = _write_nested_event(tmp_path, reference_text)
5138
5139     bundle = load_data(
5140         [path],
5141         evaluation_field_policy=_event_field_policy(),
5142     )
5143     frame = next(iter(bundle.tables.values()))
5144     profile = profile_data(bundle)
5145
5146     assert len(frame) == 1
5147     assert bundle.input_structure is not None
5148     assert bundle.input_structure.shape == InputShape.EVENT_RECORD
5149     assert bundle.input_structure.row_semantics == "one event"
5150     assert bundle.input_structure.representation_status == (
5151         InputRepresentationStatus.VALID
5152     )
5153     assert "reference_text" not in frame.columns
5154     assert "reference_text" not in bundle.input_structure.nested_paths
5155     assert reference_text.strip() not in json.dumps(
5156         {
5157             "profile": profile.model_dump(mode="json"),
5158             "structure": bundle.input_structure.model_dump(mode="json"),
5159             "operational": bundle.structured_inputs,
5160         },
5161         default=str,
5162     )
5163
5164 def test_root_evaluation_policy_keeps_full_operational_payload(tmp_path):
5165     reference_text = "REFERENCE SENTINEL " * 80
5166     path = _write_nested_event(tmp_path, reference_text)
5167
5168     bundle = load_data(
5169         [path],
5170         evaluation_field_policy=EvaluationFieldPolicy(
5171             operational_input_paths=["$"],
5172             held_out_reference_paths=["reference_text"],
5173         ),
5174     )
5175     frame = next(iter(bundle.tables.values()))
5176     profile = profile_data(bundle)
5177
5178     assert len(frame) == 1
5179     assert set(frame.columns) == {
5180         "event_id",
5181         "date",
5182         "venue",
5183         "overtime",
5184         "participants",
5185     }
5186     assert "reference_text" not in frame.columns
5187     assert "participants" in bundle.structured_inputs["nested_event"]
5188     assert profile.tables[0].column_count == 5
5189
5190 def test_event_capabilities_accept_numeric_strings(tmp_path):
5191     path = tmp_path / "numeric_string_event.json"
5192     payload = _stringify_numbers(_nested_event_fixture("HELD OUT " * 100))
5193     for participant in payload["participants"].values():
5194         entities = participant["statistics"]["entities"]
5195         participant["statistics"]["entities"] = list(entities.values())
5196     path.write_text(
5197         json.dumps(payload),
5198         encoding="utf-8",
5199     )
5200
5201     bundle = load_data(
5202         [path],
5203         evaluation_field_policy=EvaluationFieldPolicy(
5204             operational_input_paths=["$"],
5205             held_out_reference_paths=["reference_text"],
5206         ),
5207     )
5208     capabilities = available_capabilities(bundle)
5209     profile = profile_data(bundle)
5210     plan = fallback_execution_plan(
5211         "Write a neutral event report.",
5212         profile,
5213         AuditMode.INTERNAL,
5214         Settings(),
5215         input_structure=bundle.input_structure,
5216         available_capabilities=capabilities,
5217     )
5218     evidence = execute_plan(bundle, plan, Settings())
5219
5220     assert {
5221         EvidenceCapability.EVENT_OUTCOME,
5222         EvidenceCapability.ENTITY_PERFORMANCE,

```

```

5224         EvidenceCapability.RANKING,
5225         EvidenceCapability.GROUP_COMPARISON,
5226     }.issubset(capabilities)
5227     outcome = next(
5228         item
5229         for item in evidence.items
5230         if item.evidence_type == "event_outcome"
5231     )
5232     assert outcome.metrics["winner_score"] == 90
5233     assert outcome.metrics["loser_score"] == 80
5234
5235
5236 def test_nested_entity_collection_retains_core_tabular_capabilities(
5237     tmp_path,
5238 ):
5239     path = tmp_path / "entities.json"
5240     path.write_text(
5241         json.dumps(
5242             [
5243                 {
5244                     "name": f"entity-{index}",
5245                     "value": index,
5246                     "attributes": {"group": index % 2},
5247                     "tags": ["example"],
5248                 }
5249                 for index in range(30)
5250             ]
5251         ),
5252         encoding="utf-8",
5253     )
5254
5255     bundle = load_data([path])
5256     capabilities = available_capabilities(bundle)
5257
5258     assert bundle.input_structure is not None
5259     assert bundle.input_structure.shape == InputShape.ENTITY_COLLECTION
5260     assert len(next(iter(bundle.tables.values()))) == 30
5261     assert {
5262         EvidenceCapability.MISSINGNESS,
5263         EvidenceCapability.DUPPLICATES,
5264         EvidenceCapability.DISTRIBUTION_SUMMARY,
5265         EvidenceCapability.ASSOCIATION,
5266         EvidenceCapability.GROUP_COMPARISON,
5267     }.issubset(capabilities)
5268
5269
5270 def test_undeclared_event_reference_is_quarantined_and_ineligible(
5271     tmp_path,
5272 ):
5273     reference_text = "UNDECLARED REFERENCE SENTINEL " * 60
5274     path = _write_nested_event(tmp_path, reference_text)
5275     bundle = load_data([path])
5276
5277     assert len(next(iter(bundle.tables.values()))) == 1
5278     assert bundle.input_structure is not None
5279     assert bundle.input_structure.sparse_flattening_detected
5280     assert bundle.input_structure.representation_status == (
5281         InputRepresentationStatus.AMBIGUOUS
5282     )
5283     assert bundle.evaluation_field_policy.held_out_reference_paths == [
5284         "reference_text"
5285     ]
5286
5287     settings = replace(
5288         Settings(),
5289         use_llm=False,
5290         enable_insight_synthesis=False,
5291         output_dir=tmp_path / "runs_ambiguous",
5292     )
5293     result = Table2TextWorkflow(settings).run_sync(
5294         [path],
5295         "Understand the dataset and report its strongest findings.",
5296     )
5297
5298     assert not result.primary_evaluation_eligible
5299     assert result.primary_evaluation_reason == (
5300         "input_representation_ambiguous"
5301     )
5302     assert result.release_status == ReleaseStatus.HUMAN_REVIEW_REQUIRED
5303     assert reference_text.strip() not in result.final_writer_output.markdown
5304
5305
5306 def test_generic_event_capabilities_extract_outcome_rankings_and_paths(
5307     tmp_path,
5308 ):
5309     path = _write_nested_event(tmp_path, "HELD OUT " * 100)
5310     bundle = load_data(
5311         [path],
5312         evaluation_field_policy=_event_field_policy(),
5313     )
5314     capabilities = available_capabilities(bundle)

```

```

5315 profile = profile_data(bundle)
5316 plan = fallback_execution_plan(
5317     "Write a neutral event report.",
5318     profile,
5319     AuditMode.INTERNAL,
5320     Settings(),
5321     input_structure=bundle.input_structure,
5322     available_capabilities=capabilities,
5323 )
5324 evidence = execute_plan(bundle, plan, Settings())
5325
5326 assert {
5327     EvidenceCapability.EVENT_OUTCOME,
5328     EvidenceCapability.ENTITY_PERFORMANCE,
5329     EvidenceCapability.RANKING,
5330     EvidenceCapability.GROUP_COMPARISON,
5331 }.issubset(capabilities)
5332
5333 outcome = next(
5334     item
5335     for item in evidence.items
5336     if item.evidence_type == "event_outcome"
5337 )
5338 assert outcome.metrics["winner"] == "Alpha"
5339 assert outcome.metrics["loser"] == "Beta"
5340 assert outcome.metrics["winner_score"] == 90
5341 assert outcome.metrics["loser_score"] == 80
5342 assert outcome.metrics["margin"] == 10
5343 assert {
5344     "participants.home.name",
5345     "participants.home.statistics.team.game.points",
5346     "participants.visitor.name",
5347     "participants.visitor.statistics.team.game.points",
5348 }.issubset(outcome.source_paths)
5349
5350 points_ranking = next(
5351     item
5352     for item in evidence.items
5353     if item.evidence_type == "entity_ranking"
5354     and item.metrics["metric"] == "points"
5355 )
5356 assert points_ranking.metrics["ranking"][0]["entity"] == "Alex One"
5357 assert points_ranking.metrics["ranking"][0]["value"] == 25
5358 assert "participants.home.statistics.entities.alpha_one.name" in (
5359     points_ranking.source_paths
5360 )
5361
5362
5363 def test_event_capability_selection_and_report_contract_are_bounded(
5364     tmp_path,
5365 ):
5366     path = _write_nested_event(tmp_path, "HELD OUT " * 100)
5367     bundle = load_data(
5368         [path],
5369         evaluation_field_policy=_event_field_policy(),
5370     )
5371     profile = profile_data(bundle)
5372     restricted_capabilities = [
5373         EvidenceCapability.DATASET_PROFILE,
5374         EvidenceCapability.EVENT_OUTCOME,
5375     ]
5376
5377     generic = fallback_execution_plan(
5378         "Understand the dataset and report its strongest findings.",
5379         profile,
5380         AuditMode.INTERNAL,
5381         Settings(),
5382         input_structure=bundle.input_structure,
5383         available_capabilities=restricted_capabilities,
5384     )
5385     event = fallback_execution_plan(
5386         "Write a neutral event report.",
5387         profile,
5388         AuditMode.INTERNAL,
5389         Settings(),
5390         input_structure=bundle.input_structure,
5391         available_capabilities=restricted_capabilities,
5392     )
5393
5394     assert generic.report_specification.genre == ReportGenre.EVENT_REPORT
5395     assert event.report_specification.genre == ReportGenre.EVENT_REPORT
5396     assert "event_result" in event.report_specification.required_content_slots
5397     assert all(
5398         task.capability is None
5399         or task.capability in restricted_capabilities
5400         for task in event.tasks
5401     )
5402     assert EvidenceCapability.RANKING not in event.selected_capabilities
5403
5404     resolved_genre, _, _ = resolve_report_genre(
5405         request="Understand the dataset and report its strongest findings.",

```

```

5406         planned_genre=ReportGenre.DATASET_OVERVIEW,
5407         configured_genre=None,
5408         input_structure=bundle.input_structure,
5409     )
5410     assert resolved_genre == ReportGenre.EVENT_REPORT
5411
5412
5413 def test_genre_quality_revises_event_report_that_omits_supported_result(
5414     tmp_path,
5415 ):
5416     path = _write_nested_event(tmp_path, "HELD OUT " * 100)
5417     bundle = load_data(
5418         [path],
5419         evaluation_field_policy=_event_field_policy(),
5420     )
5421     capabilities = available_capabilities(bundle)
5422     plan = fallback_execution_plan(
5423         "Write a neutral event report.",
5424         profile_data(bundle),
5425         AuditMode.INTERNAL,
5426         Settings(),
5427         input_structure=bundle.input_structure,
5428         available_capabilities=capabilities,
5429     )
5430     evidence = execute_plan(bundle, plan, Settings())
5431     ranking = next(
5432         item
5433         for item in evidence.items
5434         if item.evidence_type == "entity_ranking"
5435     )
5436     output = WriterOutput(
5437         title="Incomplete event report",
5438         markdown=f"# Incomplete event report\n\n{ranking.finding}\n",
5439         sentence_support=[
5440             SentenceSupport(
5441                 sentence_id="SENT_0001",
5442                 sentence_text=ranking.finding,
5443                 evidence_ids=[ranking.evidence_id],
5444                 support_type=SupportType.DIRECT,
5445             )
5446         ],
5447     )
5448
5449     assessment = assess_genre_quality(
5450         output,
5451         plan.report_specification,
5452         evidence,
5453     )
5454
5455     assert assessment.status == QualityStatus.REVISE
5456     assert "event_result" in assessment.missing_supported_slots
5457
5458
5459 def test_genre_quality_revises_event_report_without_narration():
5460     def event_item(
5461         evidence_id: str,
5462         evidence_type: str,
5463         capability: EvidenceCapability,
5464     ) -> EvidenceItem:
5465         return EvidenceItem(
5466             evidence_id=evidence_id,
5467             route=AnalysisRoute.DESCRPTIVE,
5468             task_ids=["TASK_EVENT"],
5469             capability=capability,
5470             evidence_type=evidence_type,
5471             finding=f"Supported {evidence_type}.",
5472             metrics={"value": evidence_id},
5473             source_tables=["event"],
5474             source_columns=[evidence_type],
5475             method="Validated event evidence.",
5476             practical_interpretation="Direct event evidence.",
5477             strength_label=evidence_type,
5478             claim_permissions=[
5479                 ClaimPermission.DESCRPTIVE,
5480                 ClaimPermission.COMPARATIVE,
5481             ],
5482             factual_confidence=1.0,
5483             methodological_strength=1.0,
5484             user_relevance=0.9,
5485             salience=0.9,
5486             recommended_use=RecommendedUse.MAIN_FINDING,
5487         )
5488
5489     evidence = EvidenceLedger(
5490         fingerprint="event-narrative",
5491         items=[
5492             event_item(
5493                 "EVD_RESULT",
5494                 "event_outcome",
5495                 EvidenceCapability.EVENT_OUTCOME,
5496             ),

```

```

5497         event_item(
5498             "EVD_RANKING",
5499             "entity_ranking",
5500             EvidenceCapability.RANKING,
5501         ),
5502         event_item(
5503             "EVD_CONTRAST",
5504             "participant_comparison",
5505             EvidenceCapability.GROUP_COMPARISON,
5506         ),
5507     ],
5508 )
5509 output = WriterOutput(
5510     title="Event report",
5511     markdown=(
5512         "# Event report\n\n"
5513         "Alpha defeated Beta 90-80.\n"
5514         "Nia led all entities with 20.\n"
5515         "Alpha recorded more attempts.\n"
5516     ),
5517     sentence_support=[
5518         SentenceSupport(
5519             sentence_id="SENT_0001",
5520             sentence_text="Alpha defeated Beta 90-80.",
5521             evidence_ids=["EVD_RESULT"],
5522             support_type=SupportType.DIRECT,
5523         ),
5524         SentenceSupport(
5525             sentence_id="SENT_0002",
5526             sentence_text="Nia led all entities with 20.",
5527             evidence_ids=["EVD_RANKING"],
5528             support_type=SupportType.DIRECT,
5529         ),
5530         SentenceSupport(
5531             sentence_id="SENT_0003",
5532             sentence_text="Alpha recorded more attempts.",
5533             evidence_ids=["EVD_CONTRAST"],
5534             support_type=SupportType.DIRECT,
5535         ),
5536     ],
5537 )
5538 spec = ReportSpecification(
5539     report_purpose="Write an event report.",
5540     genre=ReportGenre.EVENT_REPORT,
5541     target_length_words=250,
5542     required_components=[],
5543     required_content_slots=[
5544         "event_result",
5545         "leading_performance",
5546         "main_contrast",
5547         "scope_limitations",
5548     ],
5549     optional_content_slots=[],
5550     prioritisation_rule="Use supported event material.",
5551 )
5552
5553 assessment = assess_genre_quality(
5554     output,
5555     spec,
5556     evidence,
5557 )
5558
5559 assert assessment.status == QualityStatus.REVISE
5560 assert any(
5561     "without enough multi-fact" in finding
5562     for finding in assessment.findings
5563 )
5564 assert any(
5565     "narrative comparison" in finding
5566     for finding in assessment.findings
5567 )
5568 assert any(
5569     "event-scoped limitation" in finding
5570     for finding in assessment.findings
5571 )
5572
5573 def test_event_reference_never_reaches_operational_prompts_or_report(
5574     tmp_path,
5575 ):
5576     reference_text = "SECRET REFERENCE PROSE " * 70
5577     path = _write_nested_event(tmp_path, reference_text)
5578     settings = replace(
5579         Settings(),
5580         use_llm=True,
5581         enable_insight_synthesis=False,
5582         writer_quality_revision_rounds=0,
5583         output_dir=tmp_path / "runs_explicit",
5584     )
5585     workflow = Table2TextWorkflow(settings)
5586     operational_payloads: List[str] = []

```

```

5588
5589 async def capture_and_fallback(
5590     self,
5591     *,
5592     prompt,
5593     dependencies,
5594     fallback,
5595     **_,
5596 ):
5597     operational_payloads.append(str(prompt))
5598     operational_payloads.append(
5599         json.dumps(
5600             dependencies.payload,
5601             default=str,
5602             sort_keys=True,
5603         )
5604     )
5605     return fallback()
5606
5607 workflow.run_agent_or_fallback = MethodType(
5608     capture_and_fallback,
5609     workflow,
5610 )
5611 result = workflow.run_sync(
5612     [path],
5613     "Write a neutral event report.",
5614     evaluation_field_policy=_event_field_policy(),
5615     report_genre=ReportGenre.EVENT_REPORT,
5616 )
5617
5618 operational_text = "\n".join(operational_payloads)
5619 assert reference_text.strip() not in operational_text
5620 assert reference_text.strip() not in result.final_writer_output.markdown
5621 assert result.primary_evaluation_eligible
5622 assert result.execution_plan.report_specification.genre == (
5623     ReportGenre.EVENT_REPORT
5624 )
5625 assert result.genre_quality_assessment is not None
5626 assert result.genre_quality_assessment.status == QualityStatus.PASS
5627 assert result.genre_quality_assessment.missing_supported_slots == []
5628 assert "Alpha defeated Beta 90-80" in result.final_writer_output.markdown
5629 assert "comeback" not in result.final_writer_output.markdown.lower()

```

3 Configuration Listings

Reproducibility configuration. Environment examples contain placeholders only; no secrets are included.

Listing 51: C01: experiments/llm_only_pipeline/.env.example

```

1 # The experiment loads the project-level environment before making API calls.
2 DEEPSEEK_API_KEY=
3 T2T_LLM_ONLY_MODEL=deepseek-v4-flash
4 T2T_LLM_ONLY_BASE_URL=https://api.deepseek.com
5 T2T_LLM_ONLY_MAX_OUTPUT_TOKENS=6000
6 T2T_LLM_ONLY_TEMPERATURE=0.1
7 T2T_LLM_ONLY_ARTIFACT_DIR=experiments/llm_only_pipeline/artifacts/runs

```

Listing 52: C02: experiments/llm_only_pipeline/config/variants.json

```

1 {
2   "variants": [
3     {
4       "variant_id": "llm_only_flash",
5       "enabled": true,
6       "backend": "callable",
7       "description": "LLM-only multi-agent workflow using DeepSeek V4 Flash.",
8       "settings_overrides": {
9         "llm_only_model": "deepseek-v4-flash",
10        "llm_only_max_source_characters": 100000,
11        "llm_only_max_source_payload_characters": 5000,
12        "llm_only_max_output_tokens": 6000,
13        "llm_only_temperature": 0.1,
14        "llm_only_max_claims": 18,
15        "llm_only_repair_rounds": 1,
16        "llm_only_artifact_dir": "experiments/llm_only_pipeline/artifacts/runs/flash"
17      },
18      "callable_path": "table2text_llm_only.backend.llm_only_multi_agent",
19      "command": [],
20      "precomputed_path": null,
21      "repetitions": 1,
22      "seeds": [42]
23    },
24    {
25      "variant_id": "llm_only_pro",
26      "enabled": false,
27      "backend": "callable",
28      "description": "Optional model-strength comparison using DeepSeek V4 Pro.",
29      "settings_overrides": {
30        "llm_only_model": "deepseek-v4-pro",
31        "llm_only_max_source_characters": 100000,
32        "llm_only_max_source_payload_characters": 5000,
33        "llm_only_max_output_tokens": 6000,
34        "llm_only_temperature": 0.1,
35        "llm_only_max_claims": 18,
36        "llm_only_repair_rounds": 1,
37        "llm_only_artifact_dir": "experiments/llm_only_pipeline/artifacts/runs/pro"
38      },
39      "callable_path": "table2text_llm_only.backend.llm_only_multi_agent",
40      "command": [],
41      "precomputed_path": null,
42      "repetitions": 1,
43      "seeds": [42]
44    }
45  ]
46 }

```

Listing 53: C03: experiments/llm_only_pipeline/pyproject.toml

```

1 [build-system]
2 requires = ["hatchling"]
3 build-backend = "hatchling.build"
4
5 [project]
6 name = "table2text-llm-only-experiment"
7 version = "0.1.0"
8 description = "Isolated LLM-only multi-agent comparison for the MSc Table2Text project."
9 readme = "README.md"
10 requires-python = ">=3.11"
11 dependencies = [
12     "openai>=1.0,<2.0",
13     "pydantic>=2.10,<3.0",
14 ]
15
16 [project.optional-dependencies]
17 dev = [
18     "pytest>=8.0,<9.0",
19     "ruff>=0.8,<1.0",
20 ]

```

```

21
22 [tool.hatch.build.targets.wheel]
23 packages = ["src/table2text_llm_only"]
24
25 [tool.pytest.ini_options]
26 testpaths = ["tests"]
27 pythonpath = ["src", "../table2text_pydanticai/src"]
28 addopts = "-q"
29
30 [tool.ruff]
31 line-length = 100
32 target-version = "py311"

```

Listing 54: C04: table2text_pydanticai/.env.example

```

1 # =====
2 # GENERAL
3 # =====
4
5 T2T_USE_LLM=true
6 T2T_OUTPUT_DIR=runs
7
8 T2T_MAX_REVISION_ROUNDS=2
9 T2T_MAX_AGENT_REQUESTS=4
10 T2T_MAX_TOTAL_TOKENS=24000
11
12 T2T_RANDOM_SEED=42
13 T2T_MAX_ANALYSIS_ROWS=50000
14 T2T_STRUCTURED_OUTPUT_MODE=native
15
16 # =====
17 # ANALYTICAL QUALITY
18 # =====
19
20 # Correlations use the full dataset below this limit.
21 T2T_FULL_DATA_CORRELATION_LIMIT=250000
22
23 # Weak relationships below this magnitude are not promoted.
24 T2T_MIN_ABS_CORRELATION=0.20
25 T2T_MAX_CORRELATION_FINDINGS=6
26 T2T_MAX_GROUP_FINDINGS=8
27
28 # Numeric feature/target correlations above this threshold
29 # are treated as possible target proxies.
30 T2T_TARGET_PROXY_CORRELATION=0.98
31
32 # Experimental target selection is disabled by default.
33 # Enable only for explicit modelling experiments.
34 T2T_ALLOW_EXPERIMENTAL_TARGETS=false
35
36 # Forecast evaluation.
37 T2T_FORECAST_FOLDS=3
38 T2T_MAX_FORECAST_TEST_POINTS=2000
39
40 # =====
41 # REPORT WRITING
42 # =====
43
44 T2T_WRITER_TARGET_WORDS=650
45 # Optional hard ceiling. Leave blank to use the target as the ceiling.
46 T2T_WRITER_MAX_WORDS=
47 T2T_REPAIR_CANDIDATES_PER_SENTENCE=3
48 # For model-comparison experiments, force short-form benchmark tasks through
49 # the LLM writer instead of the deterministic short-form verbaliser.
50 T2T_FORCE_LLM_SHORT_FORM_WRITER=false
51
52 # =====
53 # VERIFIED BOUNDED INSIGHT SYNTHESIS
54 # =====
55
56 T2T_ENABLE_INSIGHT_SYNTHESIS=true
57 T2T_MIN_INSIGHT_CONFIDENCE=0.75
58 T2T_MIN_INSIGHT_SALIENCE=0.65
59 T2T_MIN_FACTS_PER_BOUNDED_INSIGHT=2
60 T2T_ALLOW_HYPOTHESES_IN_REPORT=false
61
62 # =====
63 # WRITER QUALITY AND FACT SELECTION
64 # =====
65
66 # At most one bounded whole-report Writer quality revision.
67 T2T_WRITER_QUALITY_REVISION_ROUNDS=1
68
69 # Minimum useful report-length diagnostics.
70 T2T_MINIMUM_REPORT_WORD_RATIO=0.45
71 T2T_MINIMUM_REPORT_WORD_FLOOR=160
72
73 # Repeated causal/confounding caveats beyond this count are a quality issue.
74 T2T_MAXIMUM_REPEATED_CAVEAT_MENTIONS=4

```



```

75
76 T2T_MINIMUM_MAIN_FINDING_SCORE=0.65
77 T2T_MINIMUM_MAIN_EFFECT_STRENGTH=moderate
78
79 # Zero values above this risk threshold may be treated as unusual.
80 T2T_ZERO_UNUSUAL_RATE_THRESHOLD=0.05
81
82 # =====
83 # OLLAMA
84 # =====
85
86 OLLAMA_BASE_URL=http://localhost:11434/v1
87
88 # =====
89 # MODEL ROUTING
90 # =====
91
92 T2T_MODEL_DATA_UNDERSTANDING=ollama:gemma3:4b
93 T2T_MODEL_ORCHESTRATOR=ollama:gemma3:12b
94 T2T_MODEL_EVIDENCE=ollama:gemma3:12b
95 T2T_MODEL_VERIFIER=ollama:gemma3:12b
96 T2T_MODEL_WRITER=ollama:gemma3:12b
97 T2T_MODEL_AUDITOR=ollama:gemma3:12b
98
99 # To use DeepSeek for all LLM-backed agents, copy this file to .env,
100 # paste your key below, switch structured output to prompted, and uncomment
101 # the model routing block.
102 # DEEPSEEK_API_KEY=
103 # T2T_STRUCTURED_OUTPUT_MODE=prompted
104 # T2T_MODEL_DATA_UNDERSTANDING=deepseek:deepseek-v4-flash
105 # T2T_MODEL_ORCHESTRATOR=deepseek:deepseek-v4-flash
106 # T2T_MODEL_EVIDENCE=deepseek:deepseek-v4-flash
107 # T2T_MODEL_VERIFIER=deepseek:deepseek-v4-flash
108 # T2T_MODEL_WRITER=deepseek:deepseek-v4-pro
109 # T2T_MODEL_AUDITOR=deepseek:deepseek-v4-pro
110
111 # DeepEval judge configuration. This reuses DEEPSEEK_API_KEY above.
112 # T2T_DEEPEVAL_JUDGE_PROVIDER=deepseek
113 # T2T_DEEPEVAL_JUDGE_MODEL=deepseek-chat
114 # T2T_DEEPEVAL_JUDGE_REPETITIONS=1
115 # DEEPEVAL_PER_ATTEMPT_TIMEOUT_SECONDS_OVERRIDE=None
116
117 # OpenAI judge configuration for DeepEval/LLM-as-judge evaluation.
118 # Copy this file to .env, paste your OpenAI key, then select the model
119 # exposed on your account. If the exact model ID differs, update the value.
120 # OPENAI_API_KEY=
121 # T2T_DEEPEVAL_JUDGE_PROVIDER=openai
122 # T2T_DEEPEVAL_JUDGE_MODEL=gpt-5.6-sol
123 # T2T_DEEPEVAL_JUDGE_REPETITIONS=1
124
125 # OpenAI structured error-annotation judge. This is the recommended path
126 # for comparing LLM judging against human error annotations.
127 # T2T_OPENAI_JUDGE_ANNOTATION_MODEL=gpt-5.6-sol
128 # T2T_OPENAI_JUDGE_REASONING_EFFORT=high
129
130 # Raw single-LLM evaluation baseline. This bypasses the multi-agent pipeline
131 # and sends the benchmark request plus source data directly to DeepSeek.
132 # T2T_RAW_BASELINE_MODEL=deepseek-v4-pro
133 # T2T_RAW_BASELINE_MAX_SOURCE_CHARACTERS=100000
134 # T2T_RAW_BASELINE_MAX_OUTPUT_TOKENS=1500
135 # T2T_RAW_BASELINE_TEMPERATURE=0.2
136
137 # =====
138 # DETERMINISTIC FACT-COVERAGE RECOVERY
139 # =====
140
141 # Recover safe writer-ready facts directly from trusted deterministic
142 # evidence when the LLM evidence/verifier stages leave the ledger too thin.
143 T2T_MINIMUM_WRITER_READY_FACT_COUNT=6
144
145 T2T_MINIMUM_OVERVIEW_FACT_COUNT=1
146 T2T_MINIMUM_DATA_QUALITY_FACT_COUNT=1
147 T2T_MINIMUM_RELATIONSHIP_FACT_COUNT=2
148
149 T2T_MAXIMUM_RECOVERED_OVERVIEW_FACTS=2
150 T2T_MAXIMUM_RECOVERED_DATA_QUALITY_FACTS=3
151 T2T_MAXIMUM_RECOVERED_CORRELATION_FACTS=3
152 T2T_MAXIMUM_RECOVERED_GROUP_COMPARISON_FACTS=2
153 T2T_MAXIMUM_RECOVERED_MODELLING_FACTS=1

```

Listing 55: C05: table2text_pydanticai/evaluation/config/datasets.json

```

1 {
2   "datasets": [
3     {
4       "dataset_id": "sportsett_basketball",
5       "enabled": true,
6       "source": "huggingface",
7       "hub_id": "GEM/sportsett_basketball",

```

```

8      "config_name": null,
9      "revision": null,
10     "split": "test",
11     "trust_remote_code": false,
12     "local_path": null,
13     "normalizer": "sportsett",
14     "task_family": "event_report",
15     "output_mode": "multi_paragraph_report",
16     "language": "en",
17     "sample_size": 30,
18     "seed": 42,
19     "source_fields": [],
20     "reference_fields": [
21         "references",
22         "target",
23         "summaries"
24     ],
25     "id_fields": [
26         "sportsett_id",
27         "gem_parent_id",
28         "gem_id"
29     ],
30     "group_fields": [],
31     "metadata_fields": [
32         "sportsett_id"
33     ],
34     "options": {}
35 },
36 {
37     "dataset_id": "totto",
38     "enabled": true,
39     "source": "huggingface",
40     "hub_id": "GEM/totto",
41     "config_name": null,
42     "revision": null,
43     "split": "validation",
44     "trust_remote_code": false,
45     "local_path": null,
46     "normalizer": "totto",
47     "task_family": "highlighted_table_description",
48     "output_mode": "one_sentence",
49     "language": "en",
50     "sample_size": 30,
51     "seed": 42,
52     "source_fields": [],
53     "reference_fields": [
54         "references",
55         "target"
56     ],
57     "id_fields": [
58         "gem_parent_id",
59         "gem_id",
60         "example_id",
61         "id"
62     ],
63     "group_fields": [],
64     "metadata_fields": [
65         "overlap_subset",
66         "table_page_title",
67         "table_section_title"
68     ],
69     "options": {}
70 },
71 {
72     "dataset_id": "e2e_nlg",
73     "enabled": true,
74     "source": "huggingface",
75     "hub_id": "GEM/e2e_nlg",
76     "config_name": null,
77     "revision": null,
78     "split": "test",
79     "trust_remote_code": false,
80     "local_path": null,
81     "normalizer": "e2e",
82     "task_family": "attribute_verbalisation",
83     "output_mode": "short_text",
84     "language": "en",
85     "sample_size": 30,
86     "seed": 42,
87     "source_fields": [],
88     "reference_fields": [
89         "references",
90         "target"
91     ],
92     "id_fields": [
93         "gem_parent_id",
94         "gem_id",
95         "example_id",
96         "id"
97     ],
98     "group_fields": [],

```

```

99     "metadata_fields": [],
100     "options": {}
101 },
102 {
103     "dataset_id": "web_nlg",
104     "enabled": true,
105     "source": "huggingface",
106     "hub_id": "GEM/web_nlg",
107     "config_name": "en",
108     "revision": null,
109     "split": "test",
110     "trust_remote_code": false,
111     "local_path": null,
112     "normalizer": "webnlg",
113     "task_family": "triple_verbalisation",
114     "output_mode": "short_text",
115     "language": "en",
116     "sample_size": 30,
117     "seed": 42,
118     "source_fields": [],
119     "reference_fields": [
120         "references",
121         "target"
122     ],
123     "id_fields": [
124         "gem_parent_id",
125         "gem_id",
126         "example_id",
127         "id"
128     ],
129     "group_fields": [],
130     "metadata_fields": [
131         "category"
132     ],
133     "options": {}
134 },
135 {
136     "dataset_id": "dart",
137     "enabled": true,
138     "source": "huggingface",
139     "hub_id": "GEM/dart",
140     "config_name": null,
141     "revision": null,
142     "split": "test",
143     "trust_remote_code": false,
144     "local_path": null,
145     "normalizer": "dart",
146     "task_family": "triple_verbalisation",
147     "output_mode": "short_text",
148     "language": "en",
149     "sample_size": 30,
150     "seed": 42,
151     "source_fields": [],
152     "reference_fields": [
153         "references",
154         "target"
155     ],
156     "id_fields": [
157         "gem_parent_id",
158         "gem_id",
159         "example_id",
160         "id"
161     ],
162     "group_fields": [],
163     "metadata_fields": [
164         "target_sources",
165         "subtree_was_extended"
166     ],
167     "options": {}
168 },
169 {
170     "dataset_id": "logicnlg",
171     "enabled": true,
172     "source": "huggingface",
173     "hub_id": "kasnerz/logicnlg",
174     "config_name": null,
175     "revision": null,
176     "split": "test",
177     "trust_remote_code": false,
178     "local_path": null,
179     "normalizer": "logicnlg",
180     "task_family": "logical_table_statement",
181     "output_mode": "one_sentence",
182     "language": "en",
183     "sample_size": 30,
184     "seed": 42,
185     "source_fields": [],
186     "reference_fields": [
187         "ref",
188         "references",
189         "target"

```

```

190     ],
191     "id_fields": [
192         "table_id",
193         "id"
194     ],
195     "group_fields": [],
196     "metadata_fields": [
197         "title",
198         "template"
199     ],
200     "options": {}
201 },
202 {
203     "dataset_id": "fetaqa",
204     "enabled": true,
205     "source": "huggingface",
206     "hub_id": "table-benchmark/fetaqa",
207     "config_name": null,
208     "revision": null,
209     "split": "test",
210     "trust_remote_code": false,
211     "local_path": null,
212     "normalizer": "fetaqa",
213     "task_family": "table_question_answering",
214     "output_mode": "direct_answer",
215     "language": "en",
216     "sample_size": 30,
217     "seed": 42,
218     "source_fields": [],
219     "reference_fields": [
220         "answer",
221         "references",
222         "target"
223     ],
224     "id_fields": [
225         "feta_id",
226         "id"
227     ],
228     "group_fields": [],
229     "metadata_fields": [
230         "table_page_title",
231         "table_section_title"
232     ],
233     "options": {}
234 },
235 {
236     "dataset_id": "viggo",
237     "enabled": true,
238     "source": "huggingface",
239     "hub_id": "GEM/viggo",
240     "config_name": null,
241     "revision": null,
242     "split": "test",
243     "trust_remote_code": false,
244     "local_path": null,
245     "normalizer": "viggo",
246     "task_family": "attribute_verbalisation",
247     "output_mode": "short_text",
248     "language": "en",
249     "sample_size": 30,
250     "seed": 42,
251     "source_fields": [],
252     "reference_fields": [
253         "references",
254         "target"
255     ],
256     "id_fields": [
257         "gem_parent_id",
258         "gem_id",
259         "example_id",
260         "id"
261     ],
262     "group_fields": [],
263     "metadata_fields": [],
264     "options": {}
265 },
266 {
267     "dataset_id": "mlb_data_to_text",
268     "enabled": true,
269     "source": "huggingface",
270     "hub_id": "GEM/mlb_data_to_text",
271     "config_name": null,
272     "revision": null,
273     "split": "test",
274     "trust_remote_code": true,
275     "local_path": null,
276     "normalizer": "mlb",
277     "task_family": "event_report",
278     "output_mode": "multi_paragraph_report",
279     "language": "en",
280     "sample_size": 20,

```

```

281     "seed": 42,
282     "source_fields": [],
283     "reference_fields": [
284         "references",
285         "target",
286         "summary_eval",
287         "summary"
288     ],
289     "id_fields": [
290         "gem_parent_id",
291         "gem_id",
292         "game_id",
293         "id"
294     ],
295     "group_fields": [],
296     "metadata_fields": [],
297     "options": {}
298 },
299 {
300     "dataset_id": "conversational_weather",
301     "enabled": true,
302     "source": "huggingface",
303     "hub_id": "GEM/conversational_weather",
304     "config_name": null,
305     "revision": null,
306     "split": "test",
307     "trust_remote_code": true,
308     "local_path": null,
309     "normalizer": "generic",
310     "task_family": "weather_response",
311     "output_mode": "short_text",
312     "language": "en",
313     "sample_size": 30,
314     "seed": 42,
315     "source_fields": [
316         "dialogue_act",
317         "weather_data",
318         "input",
319         "linearized_input"
320     ],
321     "reference_fields": [
322         "references",
323         "target",
324         "response"
325     ],
326     "id_fields": [
327         "gem_parent_id",
328         "gem_id",
329         "example_id",
330         "id"
331     ],
332     "group_fields": [],
333     "metadata_fields": [],
334     "options": {}
335 },
336 {
337     "dataset_id": "turku_hockey",
338     "enabled": true,
339     "source": "huggingface",
340     "hub_id": "GEM/turku_hockey_data2text",
341     "config_name": null,
342     "revision": null,
343     "split": "test",
344     "trust_remote_code": true,
345     "local_path": null,
346     "normalizer": "turku_hockey",
347     "task_family": "event_report",
348     "output_mode": "paragraph",
349     "language": "fi",
350     "sample_size": 20,
351     "seed": 42,
352     "source_fields": [],
353     "reference_fields": [
354         "references",
355         "target",
356         "news_article"
357     ],
358     "id_fields": [
359         "gem_parent_id",
360         "gem_id",
361         "example_id",
362         "id"
363     ],
364     "group_fields": [],
365     "metadata_fields": [],
366     "options": {}
367 },
368 {
369     "dataset_id": "rotowire_english_german",
370     "enabled": true,
371     "source": "huggingface",

```

```

372     "hub_id": "GEM/RotoWire_English-German",
373     "config_name": null,
374     "revision": null,
375     "split": "test",
376     "trust_remote_code": true,
377     "local_path": null,
378     "normalizer": "rotowire_en_de",
379     "task_family": "cross_lingual_event_report",
380     "output_mode": "multi_paragraph_report",
381     "language": "de",
382     "sample_size": 20,
383     "seed": 42,
384     "source_fields": [],
385     "reference_fields": [
386         "references",
387         "target",
388         "summary_de"
389     ],
390     "id_fields": [
391         "gem_parent_id",
392         "gem_id",
393         "example_id",
394         "id"
395     ],
396     "group_fields": [],
397     "metadata_fields": [],
398     "options": {}
399 },
400 {
401     "dataset_id": "rotowire_fg",
402     "enabled": false,
403     "source": "local",
404     "hub_id": null,
405     "config_name": null,
406     "revision": null,
407     "split": "test",
408     "trust_remote_code": false,
409     "local_path": "evaluation/local_datasets/rotowire_fg",
410     "normalizer": "generic",
411     "task_family": "event_report",
412     "output_mode": "multi_paragraph_report",
413     "language": "en",
414     "sample_size": 30,
415     "seed": 42,
416     "source_fields": [
417         "table",
418         "metadata",
419         "input",
420         "box_score",
421         "line_score"
422     ],
423     "reference_fields": [
424         "summary",
425         "reference",
426         "references",
427         "target",
428         "explanation"
429     ],
430     "id_fields": [
431         "game_id",
432         "id",
433         "table_id"
434     ],
435     "group_fields": [],
436     "metadata_fields": [],
437     "options": {}
438 },
439 {
440     "dataset_id": "rotowire_original",
441     "enabled": false,
442     "source": "local",
443     "hub_id": null,
444     "config_name": null,
445     "revision": null,
446     "split": "test",
447     "trust_remote_code": false,
448     "local_path": "evaluation/local_datasets/rotowire_original",
449     "normalizer": "generic",
450     "task_family": "event_report",
451     "output_mode": "multi_paragraph_report",
452     "language": "en",
453     "sample_size": 30,
454     "seed": 42,
455     "source_fields": [
456         "table",
457         "metadata",
458         "input",
459         "box_score",
460         "line_score"
461     ],
462     "reference_fields": [

```

```

463     "summary",
464     "reference",
465     "references",
466     "target",
467     "explanation"
468 ],
469 "id_fields": [
470     "game_id",
471     "id",
472     "table_id"
473 ],
474 "group_fields": [],
475 "metadata_fields": [],
476 "options": {}
477 },
478 {
479     "dataset_id": "wikitablet",
480     "enabled": false,
481     "source": "local",
482     "hub_id": null,
483     "config_name": null,
484     "revision": null,
485     "split": "test",
486     "trust_remote_code": false,
487     "local_path": "evaluation/local_datasets/wikitablet",
488     "normalizer": "generic",
489     "task_family": "long_form_table_report",
490     "output_mode": "paragraph",
491     "language": "en",
492     "sample_size": 30,
493     "seed": 42,
494     "source_fields": [
495         "table",
496         "metadata",
497         "input",
498         "box_score",
499         "line_score"
500 ],
501     "reference_fields": [
502         "summary",
503         "reference",
504         "references",
505         "target",
506         "explanation"
507 ],
508     "id_fields": [
509         "game_id",
510         "id",
511         "table_id"
512 ],
513     "group_fields": [],
514     "metadata_fields": [],
515     "options": {}
516 },
517 {
518     "dataset_id": "takg",
519     "enabled": false,
520     "source": "local",
521     "hub_id": null,
522     "config_name": null,
523     "revision": null,
524     "split": "test",
525     "trust_remote_code": false,
526     "local_path": "evaluation/local_datasets/takg",
527     "normalizer": "generic",
528     "task_family": "long_form_table_report",
529     "output_mode": "paragraph",
530     "language": "en",
531     "sample_size": 30,
532     "seed": 42,
533     "source_fields": [
534         "table",
535         "metadata",
536         "input",
537         "box_score",
538         "line_score"
539 ],
540     "reference_fields": [
541         "summary",
542         "reference",
543         "references",
544         "target",
545         "explanation"
546 ],
547     "id_fields": [
548         "game_id",
549         "id",
550         "table_id"
551 ],
552     "group_fields": [],
553     "metadata_fields": [],

```

```

554     "options": {}
555 },
556 {
557     "dataset_id": "ml_performance_explanations",
558     "enabled": false,
559     "source": "local",
560     "hub_id": null,
561     "config_name": null,
562     "revision": null,
563     "split": "test",
564     "trust_remote_code": false,
565     "local_path": "evaluation/local_datasets/ml_performance_explanations",
566     "normalizer": "generic",
567     "task_family": "analytical_explanation",
568     "output_mode": "paragraph",
569     "language": "en",
570     "sample_size": 30,
571     "seed": 42,
572     "source_fields": [
573         "table",
574         "metadata",
575         "input",
576         "box_score",
577         "line_score"
578     ],
579     "reference_fields": [
580         "summary",
581         "reference",
582         "references",
583         "target",
584         "explanation"
585     ],
586     "id_fields": [
587         "game_id",
588         "id",
589         "table_id"
590     ],
591     "group_fields": [],
592     "metadata_fields": [],
593     "options": {}
594 }
595 ]
596 }

```

Listing 56: C06: table2text_pydanticai/evaluation/config/metrics.json

```

1  {
2  "experiment_id": "table2text_reference_evaluation_v1",
3  "prepared_examples_path": "evaluation/prepared/all_examples.jsonl",
4  "generations_path": "evaluation/generations/generations.jsonl",
5  "result_directory": "evaluation/results",
6  "baseline_variant": "single_agent_baseline",
7  "bootstrap_resamples": 5000,
8  "confidence_level": 0.95,
9  "random_seed": 42,
10 "reference_metrics": {
11     "enabled_metrics": [
12         "bleu",
13         "chrF",
14         "ter",
15         "rouge1",
16         "rouge2",
17         "rougeL",
18         "rougeLsum",
19         "meteor",
20         "bertscore",
21         "parent",
22         "hhem",
23         "alignscore"
24     ],
25     "bertscore_model": "roberta-base",
26     "bertscore_num_layers": null,
27     "bertscore_batch_size": 8,
28     "bertscore_device": null,
29     "bertscore_rescale_with_baseline": false,
30     "parent_n_jobs": 1,
31     "hf_local_files_only": true,
32     "external_factuality_context": "references",
33     "external_context_max_characters": 50000,
34     "hhem_model": "vectara/hallucination_evaluation_model",
35     "hhem_foundation_model": "google/flan-t5-base",
36     "hhem_threshold": 0.5,
37     "hhem_batch_size": 16,
38     "hhem_context_max_characters": 1000,
39     "hhem_device": null,
40     "alignscore_python_executable": null,
41     "alignscore_worker_path": null,
42     "alignscore_model_size": "base",
43     "alignscore_device": "cpu",

```



```

44     "alignscore_batch_size": 8,
45     "alignscore_threshold": 0.5,
46     "lowercase": false
47 },
48 "deepeval": {
49     "enabled": true,
50     "judge_provider": "deepseek",
51     "judge_model": "deepseek-chat",
52     "judge_repetitions": 3,
53     "threshold": 0.5,
54     "max_source_characters": 50000,
55     "run_summarization": true,
56     "run_faithfulness": true,
57     "run_factual_correctness": true,
58     "run_reference_adequacy": true,
59     "run_task_relevance": true,
60     "run_coherence": true,
61     "run_usefulness": true
62 }
63 }

```

Listing 57: C07: table2text_pydanticai/evaluation/config/metrics_reference_similarity.json

```

1  {
2  "experiment_id": "table2text_reference_evaluation_v1",
3  "prepared_examples_path": "evaluation/prepared/all_examples.jsonl",
4  "generations_path": "evaluation/generations/generations.jsonl",
5  "result_directory": "evaluation/results",
6  "baseline_variant": "single_agent_baseline",
7  "bootstrap_resamples": 5000,
8  "confidence_level": 0.95,
9  "random_seed": 42,
10 "reference_metrics": {
11     "enabled_metrics": [
12         "bleu",
13         "chrf",
14         "ter",
15         "rouge1",
16         "rouge2",
17         "rougeL",
18         "rougeLsum",
19         "meteor",
20         "bertscore",
21         "parent"
22     ],
23     "bertscore_model": "roberta-base",
24     "bertscore_num_layers": null,
25     "bertscore_batch_size": 8,
26     "bertscore_device": null,
27     "bertscore_rescale_with_baseline": false,
28     "parent_n_jobs": 1,
29     "hf_local_files_only": true,
30     "external_factuality_context": "references",
31     "external_context_max_characters": 50000,
32     "hhem_model": "vectara/hallucination_evaluation_model",
33     "hhem_foundation_model": "google/flan-t5-base",
34     "hhem_threshold": 0.5,
35     "hhem_batch_size": 16,
36     "hhem_context_max_characters": 1000,
37     "hhem_device": null,
38     "alignscore_python_executable": null,
39     "alignscore_worker_path": null,
40     "alignscore_model_size": "base",
41     "alignscore_device": "cpu",
42     "alignscore_batch_size": 8,
43     "alignscore_threshold": 0.5,
44     "lowercase": false
45 },
46 "deepeval": {
47     "enabled": true,
48     "judge_provider": "deepseek",
49     "judge_model": "deepseek-chat",
50     "judge_repetitions": 3,
51     "threshold": 0.5,
52     "max_source_characters": 50000,
53     "run_summarization": true,
54     "run_faithfulness": true,
55     "run_factual_correctness": true,
56     "run_reference_adequacy": true,
57     "run_task_relevance": true,
58     "run_coherence": true,
59     "run_usefulness": true
60 }
61 }

```

Listing 58: C08: table2text_pydanticai/evaluation/config/metrics_source_grounded.json

```

1 {
2   "experiment_id": "table2text_reference_evaluation_v1",
3   "prepared_examples_path": "evaluation/prepared/all_examples.jsonl",
4   "generations_path": "evaluation/generations/generations.jsonl",
5   "result_directory": "evaluation/results",
6   "baseline_variant": "single_agent_baseline",
7   "bootstrap_resamples": 5000,
8   "confidence_level": 0.95,
9   "random_seed": 42,
10  "reference_metrics": {
11    "enabled_metrics": [
12      "hhem",
13      "alignscore"
14    ],
15    "bertscore_model": "roberta-base",
16    "bertscore_num_layers": null,
17    "bertscore_batch_size": 8,
18    "bertscore_device": null,
19    "bertscore_rescale_with_baseline": false,
20    "parent_n_jobs": 1,
21    "hf_local_files_only": true,
22    "external_factuality_context": "source_text",
23    "external_context_max_characters": 100000,
24    "hhem_model": "vectara/hallucination_evaluation_model",
25    "hhem_foundation_model": "google/flan-t5-base",
26    "hhem_threshold": 0.5,
27    "hhem_batch_size": 16,
28    "hhem_context_max_characters": 1500,
29    "hhem_device": null,
30    "alignscore_python_executable": null,
31    "alignscore_worker_path": null,
32    "alignscore_model_size": "base",
33    "alignscore_device": "cpu",
34    "alignscore_batch_size": 8,
35    "alignscore_threshold": 0.5,
36    "lowercase": false
37  },
38  "deepeval": {
39    "enabled": true,
40    "judge_provider": "deepseek",
41    "judge_model": "deepseek-chat",
42    "judge_repetitions": 3,
43    "threshold": 0.5,
44    "max_source_characters": 50000,
45    "run_summarization": false,
46    "run_faithfulness": true,
47    "run_factual_correctness": true,
48    "run_reference_adequacy": false,
49    "run_task_relevance": true,
50    "run_coherence": true,
51    "run_usefulness": true
52  }
53 }

```

Listing 59: C09: table2text_pydanticai/evaluation/config/variants.json

```

1 {
2   "variants": [
3     {
4       "variant_id": "single_agent_baseline",
5       "enabled": false,
6       "backend": "callable",
7       "description": "Configure callable_path with a direct single-agent generation function.",
8       "settings_overrides": {},
9       "callable_path": "table2text.evaluation_backends.single_agent_baseline",
10      "command": [],
11      "precomputed_path": null,
12      "repetitions": 1,
13      "seeds": [
14        42
15      ]
16    },
17    {
18      "variant_id": "full_system",
19      "enabled": true,
20      "backend": "table2text",
21      "description": "Current full multi-agent system.",
22      "settings_overrides": {},
23      "callable_path": null,
24      "command": [],
25      "precomputed_path": null,
26      "repetitions": 1,
27      "seeds": [
28        42
29      ]
30    }
31  ]
32 }

```

```

32     "variant_id": "full_inferred_contract",
33     "enabled": false,
34     "backend": "table2text",
35     "description": "Full multi-agent system with a generic request and a task contract inferred only from operational
36     source structure.",
37     "settings_overrides": {},
38     "task_contract_mode": "inferred",
39     "request_override": "Understand the supplied data and report its strongest supported findings.",
40     "callable_path": null,
41     "command": [],
42     "precomputed_path": null,
43     "repetitions": 1,
44     "seeds": [
45         42
46     ]
47 },
48 {
49     "variant_id": "insight_synthesis_off",
50     "enabled": true,
51     "backend": "table2text",
52     "description": "Full system with insight synthesis disabled.",
53     "settings_overrides": {
54         "enable_insight_synthesis": false
55     },
56     "callable_path": null,
57     "command": [],
58     "precomputed_path": null,
59     "repetitions": 1,
60     "seeds": [
61         42
62     ]
63 },
64 {
65     "variant_id": "verifier_off",
66     "enabled": false,
67     "backend": "precomputed",
68     "description": "Enable after the operational pipeline exposes a verifier feature flag, or provide precomputed
69     outputs.",
70     "settings_overrides": {},
71     "callable_path": null,
72     "command": [],
73     "precomputed_path": "evaluation/generations/verifier_off.jsonl",
74     "repetitions": 1,
75     "seeds": [
76         42
77     ]
78 },
79 {
80     "variant_id": "auditor_off",
81     "enabled": false,
82     "backend": "precomputed",
83     "description": "Enable after generating outputs with the final Auditor and repair stages disabled.",
84     "settings_overrides": {},
85     "callable_path": null,
86     "command": [],
87     "precomputed_path": "evaluation/generations/auditor_off.jsonl",
88     "repetitions": 1,
89     "seeds": [
90         42
91     ]
92 },
93 {
94     "variant_id": "auditor_detection_only",
95     "enabled": false,
96     "backend": "precomputed",
97     "description": "Auditor detects issues but does not repair them.",
98     "settings_overrides": {},
99     "callable_path": null,
100    "command": [],
101    "precomputed_path": "evaluation/generations/auditor_detection_only.jsonl",
102    "repetitions": 1,
103    "seeds": [
104        42
105    ]
106 }

```

Listing 60: C10: table2text_pydanticai/evaluation/config/variants_ablation.json

```

1 {
2   "variants": [
3     {
4       "variant_id": "raw_deepseek_v4_flash",
5       "enabled": true,
6       "backend": "callable",
7       "description": "Raw single-LLM DeepSeek Flash baseline from source data only.",
8       "settings_overrides": {

```

```

9      "raw_baseline_model": "deepseek-v4-flash",
10      "raw_baseline_max_source_characters": 100000,
11      "raw_baseline_max_output_tokens": 3000,
12      "raw_baseline_temperature": 0.2
13    },
14    "callable_path": "table2text.evaluation_backends.single_agent_baseline",
15    "command": [],
16    "precomputed_path": null,
17    "repetitions": 1,
18    "seeds": [
19      42
20    ]
21  },
22  {
23    "variant_id": "full_system",
24    "enabled": true,
25    "backend": "table2text",
26    "description": "Current full multi-agent system.",
27    "settings_overrides": {},
28    "callable_path": null,
29    "command": [],
30    "precomputed_path": null,
31    "repetitions": 1,
32    "seeds": [
33      42
34    ]
35  },
36  {
37    "variant_id": "no_insight_synthesis",
38    "enabled": true,
39    "backend": "table2text",
40    "description": "Full system with the insight synthesis and insight-verification stages disabled.",
41    "settings_overrides": {
42      "enable_insight_synthesis": false
43    },
44    "callable_path": null,
45    "command": [],
46    "precomputed_path": null,
47    "repetitions": 1,
48    "seeds": [
49      42
50    ]
51  },
52  {
53    "variant_id": "no_writer_quality_revision",
54    "enabled": true,
55    "backend": "table2text",
56    "description": "Full system with the pre-audit writer quality revision pass disabled.",
57    "settings_overrides": {
58      "writer_quality_revision_rounds": 0
59    },
60    "callable_path": null,
61    "command": [],
62    "precomputed_path": null,
63    "repetitions": 1,
64    "seeds": [
65      42
66    ]
67  },
68  {
69    "variant_id": "no_audit_repair_rounds",
70    "enabled": true,
71    "backend": "table2text",
72    "description": "Full system with post-audit repair rounds disabled; deterministic audit still runs.",
73    "settings_overrides": {
74      "max_revision_rounds": 0
75    },
76    "callable_path": null,
77    "command": [],
78    "precomputed_path": null,
79    "repetitions": 1,
80    "seeds": [
81      42
82    ]
83  }
84 ]
85 }

```

Listing 61: C11: table2text_pydanticai/evaluation/config/variants_raw_deepseek_v4_flash.json

```

1 {
2   "variants": [
3     {
4       "variant_id": "raw_deepseek_v4_flash",
5       "enabled": true,
6       "backend": "callable",
7       "description": "Raw single-LLM DeepSeek baseline from source data only.",
8       "settings_overrides": {

```

```

9      "raw_baseline_model": "deepseek-v4-flash",
10      "raw_baseline_max_source_characters": 100000,
11      "raw_baseline_max_output_tokens": 3000,
12      "raw_baseline_temperature": 0.2
13    },
14    "callable_path": "table2text.evaluation_backends.single_agent_baseline",
15    "command": [],
16    "precomputed_path": null,
17    "repetitions": 1,
18    "seeds": [
19      42
20    ]
21  }
22 ]
23 }

```

Listing 62: C12: table2text_pydanticai/evaluation/config/variants_raw_deepseek_v4_pro.json

```

1 {
2   "variants": [
3     {
4       "variant_id": "raw_deepseek_v4_pro",
5       "enabled": true,
6       "backend": "callable",
7       "description": "Raw single-LLM DeepSeek baseline from source data only.",
8       "settings_overrides": {
9         "raw_baseline_model": "deepseek-v4-flash",
10        "raw_baseline_max_source_characters": 100000,
11        "raw_baseline_max_output_tokens": 3000,
12        "raw_baseline_temperature": 0.2
13      },
14      "callable_path": "table2text.evaluation_backends.single_agent_baseline",
15      "command": [],
16      "precomputed_path": null,
17      "repetitions": 1,
18      "seeds": [
19        42
20      ]
21    }
22  ]
23 }

```

Listing 63: C13: table2text_pydanticai/evaluation/protected_holdout_full_system/config/protected_full_system_flash.json

```

1 {
2   "variants": [
3     {
4       "variant_id": "full_system",
5       "enabled": true,
6       "backend": "table2text",
7       "description": "Frozen protected-holdout Full-System run with all six roles on DeepSeek V4 Flash.",
8       "settings_overrides": {
9         "use_llm": true,
10        "max_revision_rounds": 2,
11        "max_agent_requests": 8,
12        "max_total_tokens": 300000,
13        "max_analysis_rows": 50000,
14        "structured_output_mode": "prompted",
15        "full_data_correlation_limit": 250000,
16        "min_abs_correlation": 0.2,
17        "max_correlation_findings": 6,
18        "max_group_findings": 8,
19        "target_proxy_correlation": 0.98,
20        "allow_experimental_targets": false,
21        "forecast_folds": 3,
22        "max_forecast_test_points": 2000,
23        "writer_target_words": 650,
24        "writer_max_words": null,
25        "writer_max_main_findings": null,
26        "repair_candidates_per_sentence": 3,
27        "writer_quality_revision_rounds": 1,
28        "writer_priority_fact_limit": null,
29        "writer_supporting_fact_limit": null,
30        "force_llm_short_form_writer": false,
31        "enable_insight_synthesis": true,
32        "max_insight_candidates": null,
33        "max_verified_main_insights": null,
34        "min_insight_confidence": 0.75,
35        "min_insight_salience": 0.65,
36        "min_facts_per_bounded_insight": 2,
37        "allow_hypotheses_in_report": false,
38        "minimum_writer_ready_fact_count": 6,
39        "minimum_overview_fact_count": 1,
40        "minimum_data_quality_fact_count": 1,
41        "minimum_relationship_fact_count": 2,
42        "maximum_recovered_overview_facts": 2,

```

```

43     "maximum_recovered_data_quality_facts": 3,
44     "maximum_recovered_correlation_facts": 3,
45     "maximum_recovered_group_comparison_facts": 2,
46     "maximum_recovered_modelling_facts": 1,
47     "minimum_main_finding_score": 0.65,
48     "minimum_main_effect_strength": "moderate",
49     "minimum_report_word_ratio": 0.45,
50     "minimum_report_word_floor": 160,
51     "maximum_repeated_caveat_mentions": 4,
52     "zero_unusual_rate_threshold": 0.05,
53     "ollama_base_url": "http://localhost:11434/v1",
54     "data_understanding_model": "deepseek:deepseek-v4-flash",
55     "orchestrator_model": "deepseek:deepseek-v4-flash",
56     "evidence_model": "deepseek:deepseek-v4-flash",
57     "verifier_model": "deepseek:deepseek-v4-flash",
58     "writer_model": "deepseek:deepseek-v4-flash",
59     "auditor_model": "deepseek:deepseek-v4-flash"
60 },
61 "task_contract_mode": "explicit",
62 "request_override": null,
63 "callable_path": null,
64 "command": [],
65 "precomputed_path": null,
66 "repetitions": 1,
67 "seeds": [
68     42
69 ]
70 }
71 ]
72 }

```

Listing 64: C14: table2text_pydanticai/pyproject.toml

```

1  [build-system]
2  requires = ["hatchling"]
3  build-backend = "hatchling.build"
4
5  [project]
6  name = "table2text-pydanticai"
7  version = "0.1.0"
8  description = "A PydanticAI multi-agent Table2Text system for reducing hallucinations."
9  readme = "README.md"
10 requires-python = ">=3.11"
11
12 dependencies = [
13     "pydantic>=2.10,<3.0",
14     "pydantic-ai-slim[openai]>=1.0,<2.0",
15     "pandas>=2.2,<3.0",
16     "numpy>=1.26,<3.0",
17     "scikit-learn>=1.5,<2.0",
18     "openpyxl>=3.1,<4.0",
19     "pyarrow>=16.0",
20 ]
21
22 [project.optional-dependencies]
23 dev = [
24     "pytest>=8.0,<9.0",
25     "pytest-asyncio>=0.24,<1.0",
26     "ruff>=0.8,<1.0",
27 ]
28
29 evaluation = [
30     "datasets>=3.2,<4.0",
31     "huggingface-hub>=0.27,<1.0",
32     "sacrebleu>=2.4,<3.0",
33     "rouge-score>=0.1.2,<0.2",
34     "nltk>=3.9,<4.0",
35     "bert-score>=0.3.13,<0.4",
36     "transformers>=4.46,<5.0",
37     "torch>=2.3,<3.0",
38     "sentencepiece>=0.2,<0.3",
39     "scipy>=1.13,<2.0",
40     "tabulate>=0.9,<1.0",
41 ]
42
43 evaluation-ragas = [
44     "ragas>=0.3.5,<1.0",
45 ]
46
47 evaluation-deepeval = [
48     "deepeval",
49 ]
50
51 evaluation-hhem = [
52     "transformers>=4.46,<5.0",
53     "torch>=2.3,<3.0",
54 ]
55
56 evaluation-opik = [

```

```
57     "opik",
58 ]
59
60 evaluation-summac = [
61     "summac",
62 ]
63
64 evaluation-qafacteval = [
65     "qafacteval",
66 ]
67
68 evaluation-cleanlab = [
69     "cleanlab-tlm",
70 ]
71
72 evaluation-patronus = [
73     "patronus",
74 ]
75
76 evaluation-phoenix = [
77     "arize-phoenix-evals>=3",
78 ]
79
80 evaluation-alignscore = [
81     "huggingface-hub>=0.27,<1.0",
82     "spacy>=3.7,<4.0",
83 ]
84
85 [project.scripts]
86 table2text = "table2text.cli:main"
87 table2text-evaluate = "table2text.evaluation.cli:main"
88
89 [tool.hatch.build.targets.wheel]
90 packages = ["src/table2text"]
91
92 [tool.pytest.ini_options]
93 testpaths = ["tests"]
94 pythonpath = ["src"]
95 addopts = "-q"
96
97 [tool.ruff]
98 line-length = 100
99 target-version = "py311"
```

4 Protected Data Listings

All 25 protected-holdout inputs, grouped across five datasets and formatted as readable JSON. Where the benchmark stores both a structured payload and a duplicate serialized source-text field, this appendix displays the complete structured payload once.

Listing 65: D01: submission/program_listings/build/generated/data/dart__dart-test-1791.json

```

1 {
2   "dataset_id": "dart",
3   "example_id": "dart-test-1791",
4   "language": "en",
5   "output_mode": "short_text",
6   "parent_table": [
7     [
8       "Elliot See",
9       "ALMA_MATER",
10      "University of Texas at Austin"
11    ],
12    [
13      "Elliot See",
14      "DEATH_PLACE",
15      "St. Louis"
16    ],
17    [
18      "Elliot See",
19      "WAS_SELECTED_BY_NASA",
20      "1962"
21    ]
22  ],
23  "reference_sha256": "af602b3a77c5ee0df21a0b25b33a21eac28b98a21cd8922ee4fd59c0c260ee96",
24  "references": [
25    "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
26  ],
27  "request": "Express all and only the supplied triples as short, coherent natural language. Do not add unsupported facts.",
28  "source_payload": {
29    "category": null,
30    "triples": [
31      [
32        "Elliot See",
33        "ALMA_MATER",
34        "University of Texas at Austin"
35      ],
36      [
37        "Elliot See",
38        "DEATH_PLACE",
39        "St. Louis"
40      ],
41      [
42        "Elliot See",
43        "WAS_SELECTED_BY_NASA",
44        "1962"
45      ]
46    ]
47  },
48  "source_sha256": "20a5eccd67eb73bbf6f37ec51941e57ad416c4faa691d2cc277daf07d08e88d4",
49  "task_family": "triple_verbalisation"
50 }

```

Listing 66: D02: submission/program_listings/build/generated/data/dart__dart-test-1805.json

```

1 {
2   "dataset_id": "dart",
3   "example_id": "dart-test-1805",
4   "language": "en",
5   "output_mode": "short_text",
6   "parent_table": [
7     [
8       "A Severed Wasp",
9       "NUMBER_OF_PAGES",
10      "\"388\""
11    ],
12    [
13      "A Severed Wasp",
14      "OCLC_NUMBER",
15      "8805735"
16    ],
17    [
18      "A Severed Wasp",
19      "MEDIA_TYPE",
20      "Hardcover"
21    ]
22  ],
23  "reference_sha256": "3a25a85473ab0357b86dd5806e5dce9a5073145299aceb30558b56d66fa6dfe6",
24  "references": [
25    "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
26  ],

```



```

27 "request": "Express all and only the supplied triples as short, coherent natural language. Do not add unsupported
28 facts.",
29 "source_payload": {
30   "category": null,
31   "triples": [
32     [
33       "A Severed Wasp",
34       "NUMBER_OF_PAGES",
35       "\"388\""
36     ],
37     [
38       "A Severed Wasp",
39       "OCLC_NUMBER",
40       "8805735"
41     ],
42     [
43       "A Severed Wasp",
44       "MEDIA_TYPE",
45       "Hardcover"
46     ]
47   ],
48   "source_sha256": "6b2447749b4569aa9ccd293353536c1e5de45907abab23a3d17be3202be8f9f9",
49   "task_family": "triple_verbalisation"
50 }

```

Listing 67: D03: submission/program_listings/build/generated/data/dart__dart-test-1828.json

```

1 {
2   "dataset_id": "dart",
3   "example_id": "dart-test-1828",
4   "language": "en",
5   "output_mode": "short_text",
6   "parent_table": [
7     [
8       "A.C. Lumezzane",
9       "FULL_NAME",
10      "\"Associazione Calcio Lumezzane SpA\""
11     ],
12     [
13       "A.C. Lumezzane",
14       "LEAGUE",
15       "\"Lega Pro/A\""
16     ],
17     [
18       "A.C. Lumezzane",
19       "NUMBER_OF_MEMBERS",
20       "4150"
21     ]
22   ],
23   "reference_sha256": "0d539c5af7762ac1859ecb654efbff4c76292ab5629b138b455834c2a2d98c92",
24   "references": [
25     "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
26   ],
27   "request": "Express all and only the supplied triples as short, coherent natural language. Do not add unsupported
28 facts.",
29   "source_payload": {
30     "category": null,
31     "triples": [
32       [
33         "A.C. Lumezzane",
34         "FULL_NAME",
35         "\"Associazione Calcio Lumezzane SpA\""
36       ],
37       [
38         "A.C. Lumezzane",
39         "LEAGUE",
40         "\"Lega Pro/A\""
41       ],
42       [
43         "A.C. Lumezzane",
44         "NUMBER_OF_MEMBERS",
45         "4150"
46       ]
47     ],
48     "source_sha256": "943f0cf7a385a91bb9ae4340fc1c3f852658d553173f9aacb7b19e5c8036e1c0",
49     "task_family": "triple_verbalisation"
50 }

```

Listing 68: D04: submission/program_listings/build/generated/data/dart__dart-test-2278.json

```

1 {
2   "dataset_id": "dart",
3   "example_id": "dart-test-2278",
4   "language": "en",

```

```

5  "output_mode": "short_text",
6  "parent_table": [
7    [
8      "Al Asad Airbase",
9      "OPERATING_ORGANISATION",
10     "United States Air Force"
11   ],
12   [
13     "United States Air Force",
14     "ATTACK_AIRCRAFT",
15     "Lockheed AC-130"
16   ],
17   [
18     "United States Air Force",
19     "TRANSPORT_AIRCRAFT",
20     "Boeing C-17 Globemaster III"
21   ],
22   [
23     "United States Air Force",
24     "AIRCRAFT_FIGHTER",
25     "General Dynamics F-16 Fighting Falcon"
26   ],
27   [
28     "United States Air Force",
29     "BATTLES",
30     "1986 United States bombing of Libya"
31   ]
32 ],
33 "reference_sha256": "dce109bd22e76fd1ea233f5c335019f0b17608be71f544d9f143331402310cf6",
34 "references": [
35   "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
36 ],
37 "request": "Express all and only the supplied triples as short, coherent natural language. Do not add unsupported
   facts.",
38 "source_payload": {
39   "category": null,
40   "triples": [
41     [
42       "Al Asad Airbase",
43       "OPERATING_ORGANISATION",
44       "United States Air Force"
45     ],
46     [
47       "United States Air Force",
48       "ATTACK_AIRCRAFT",
49       "Lockheed AC-130"
50     ],
51     [
52       "United States Air Force",
53       "TRANSPORT_AIRCRAFT",
54       "Boeing C-17 Globemaster III"
55     ],
56     [
57       "United States Air Force",
58       "AIRCRAFT_FIGHTER",
59       "General Dynamics F-16 Fighting Falcon"
60     ],
61     [
62       "United States Air Force",
63       "BATTLES",
64       "1986 United States bombing of Libya"
65     ]
66   ]
67 },
68 "source_sha256": "83a08c4f27a66f1a6306ae6360a6ed977011c4ca6ded4a7b1df2f31b72c49479",
69 "task_family": "triple_verbalisation"
70 }

```

Listing 69: D05: submission/program_listings/build/generated/data/dart__dart-test-4597.json

```

1  {
2    "dataset_id": "dart",
3    "example_id": "dart-test-4597",
4    "language": "en",
5    "output_mode": "short_text",
6    "parent_table": [
7      [
8        "The Vaults",
9        "eatType",
10       "pub"
11     ],
12     [
13       "The Vaults",
14       "food",
15       "Italian"
16     ],
17     [
18       "The Vaults",
19       "priceRange",

```

```

20     "moderate"
21   ],
22   [
23     "The Vaults",
24     "customer rating",
25     "average"
26   ],
27   [
28     "The Vaults",
29     "area",
30     "riverside"
31   ],
32   [
33     "The Vaults",
34     "familyFriendly",
35     "yes"
36   ],
37   [
38     "The Vaults",
39     "near",
40     "Rainbow Vegetarian Café"
41   ]
42 ],
43 "reference_sha256": "4bd00c10ae67936e233cbc1a60c47d4265f1871969876cbfb8a66c8fbcba1669",
44 "references": [
45   "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
46 ],
47 "request": "Express all and only the supplied triples as short, coherent natural language. Do not add unsupported
48   facts.",
49 "source_payload": {
50   "category": null,
51   "triples": [
52     [
53       "The Vaults",
54       "eatType",
55       "pub"
56     ],
57     [
58       "The Vaults",
59       "food",
60       "Italian"
61     ],
62     [
63       "The Vaults",
64       "priceRange",
65       "moderate"
66     ],
67     [
68       "The Vaults",
69       "customer rating",
70       "average"
71     ],
72     [
73       "The Vaults",
74       "area",
75       "riverside"
76     ],
77     [
78       "The Vaults",
79       "familyFriendly",
80       "yes"
81     ],
82     [
83       "The Vaults",
84       "near",
85       "Rainbow Vegetarian Café"
86     ]
87   ],
88   "source_sha256": "652f9936ba6424bb0f260c603db3501d3047181a928e2fdb4eec3ca81f2829e5",
89   "task_family": "triple_verbalisation"
90 }

```

Listing 70: D06: submission/program_listings/build/generated/data/e2e_nlg__e2e_nlg-test-1330.json

```

1  {
2    "dataset_id": "e2e_nlg",
3    "example_id": "e2e_nlg-test-1330",
4    "language": "en",
5    "output_mode": "short_text",
6    "parent_table": [
7      [
8        "name",
9        "The Vaults"
10     ],
11     [
12       "eatType",
13       "pub"
14     ],

```

```

15  [
16    "food",
17    "Italian"
18  ],
19  [
20    "customer rating",
21    "3 out of 5"
22  ],
23  [
24    "area",
25    "city centre"
26  ],
27  [
28    "familyFriendly",
29    "yes"
30  ],
31  [
32    "near",
33    "Rainbow Vegetarian Café"
34  ]
35 ],
36 "reference_sha256": "bf634ba957a0a8228c44057960a6512c941e7510853f54028f6e436b4c50b29f",
37 "references": [
38   "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
39 ],
40 "request": "Express all and only the supplied attributes in one or two fluent sentences. Do not add headings or
41 unsupported details.",
42 "source_payload": {
43   "meaning_representation": "name[The Vaults], eatType[pub], food[Italian], customer rating[3 out of 5], area[city
44     centre], familyFriendly[yes], near[Rainbow Vegetarian Café]"
45 },
46 "source_sha256": "9e8fb57080429c1e645ec58eeb3803675a8038d04638e3801c08fece45ea4cfc",
47 "task_family": "attribute_verbalisation"
48 }

```

Listing 71: D07: submission/program_listings/build/generated/data/e2e_nlg__e2e_nlg-test-209.json

```

1  {
2    "dataset_id": "e2e_nlg",
3    "example_id": "e2e_nlg-test-209",
4    "language": "en",
5    "output_mode": "short_text",
6    "parent_table": [
7      [
8        "name",
9        "The Cricketers"
10     ],
11     [
12       "eatType",
13       "coffee shop"
14     ],
15     [
16       "familyFriendly",
17       "yes"
18     ],
19     [
20       "near",
21       "Café Sicilia"
22     ]
23   ],
24   "reference_sha256": "f5b104892617f5bd1d9bf95d2252a6ff0fea0dd279e574255231c301381504c4",
25   "references": [
26     "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
27   ],
28   "request": "Express all and only the supplied attributes in one or two fluent sentences. Do not add headings or
29 unsupported details.",
30   "source_payload": {
31     "meaning_representation": "name[The Cricketers], eatType[coffee shop], familyFriendly[yes], near[Café Sicilia]"
32   },
33   "source_sha256": "7de0ca441f2fb833ff0197f110c66c54c545fc71f7264a1a1c4dfb337b21c277",
34   "task_family": "attribute_verbalisation"
35 }

```

Listing 72: D08: submission/program_listings/build/generated/data/e2e_nlg__e2e_nlg-test-447.json

```

1  {
2    "dataset_id": "e2e_nlg",
3    "example_id": "e2e_nlg-test-447",
4    "language": "en",
5    "output_mode": "short_text",
6    "parent_table": [
7      [
8        "name",
9        "The Mill"
10     ],
11     [

```

```

12     "eatType",
13     "pub"
14   ],
15   [
16     "food",
17     "English"
18   ],
19   [
20     "familyFriendly",
21     "no"
22   ],
23   [
24     "near",
25     "Raja Indian Cuisine"
26   ]
27 ],
28 "reference_sha256": "94a4d50214686760602a9c1c5c903df9ffc684adbed2fc0b1d14c37cdbb91e06",
29 "references": [
30   "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
31 ],
32 "request": "Express all and only the supplied attributes in one or two fluent sentences. Do not add headings or
33 unsupported details.",
34 "source_payload": {
35   "meaning_representation": "name[The Mill], eatType[pub], food[English], familyFriendly[no], near[Raja Indian
36   Cuisine]"
37 },
38 "source_sha256": "ee20f1488f496eb0fa489b07f5fb0d6f77c740eb5148166dae377bca9b9ff4ca",
39 "task_family": "attribute_verbalisation"
40 }

```

Listing 73: D09: submission/program_listings/build/generated/data/e2e_nlg__e2e_nlg-test-476.json

```

1 {
2   "dataset_id": "e2e_nlg",
3   "example_id": "e2e_nlg-test-476",
4   "language": "en",
5   "output_mode": "short_text",
6   "parent_table": [
7     [
8       "name",
9       "The Mill"
10    ],
11    [
12      "eatType",
13      "pub"
14    ],
15    [
16      "food",
17      "Fast food"
18    ],
19    [
20      "customer rating",
21      "3 out of 5"
22    ],
23    [
24      "area",
25      "riverside"
26    ],
27    [
28      "familyFriendly",
29      "yes"
30    ],
31    [
32      "near",
33      "Café Rouge"
34    ]
35  ],
36  "reference_sha256": "6d7229dffa4ca4a672bc73b14e4bd031d67f9a879f4a2cfeee3dddadb677541b",
37  "references": [
38    "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
39  ],
40  "request": "Express all and only the supplied attributes in one or two fluent sentences. Do not add headings or
41 unsupported details.",
42  "source_payload": {
43    "meaning_representation": "name[The Mill], eatType[pub], food[Fast food], customer rating[3 out of 5], area[
44    riverside], familyFriendly[yes], near[Café Rouge]"
45  },
46  "source_sha256": "dc16cc8d4dfa17c95af826014892da10e7adc35b3dfec6455e3e14a60dd75116",
47  "task_family": "attribute_verbalisation"
48 }

```

Listing 74: D10: submission/program_listings/build/generated/data/e2e_nlg__e2e_nlg-test-864.json

```

1 {
2   "dataset_id": "e2e_nlg",
3   "example_id": "e2e_nlg-test-864",

```

```

4  "language": "en",
5  "output_mode": "short_text",
6  "parent_table": [
7    [
8      "name",
9      "The Phoenix"
10   ],
11   [
12     "eatType",
13     "pub"
14   ],
15   [
16     "near",
17     "Crowne Plaza Hotel"
18   ]
19 ],
20 "reference_sha256": "5ef157458d3427cf0014a21463a855d8194f2045332afb43eded6fd3cb096645",
21 "references": [
22   "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
23 ],
24 "request": "Express all and only the supplied attributes in one or two fluent sentences. Do not add headings or
25             unsupported details.",
26 "source_payload": {
27   "meaning_representation": "name[The Phoenix], eatType[pub], near[Crowne Plaza Hotel]"
28 },
29 "source_sha256": "772650e2992ed07ecfb68cae7171298479d3d2d9bd6216b563c4fdc4e4f5c712",
30 "task_family": "attribute_verbalisation"
31 }

```

Listing 75: D11: submission/program_listings/build/generated/data/sportsett_basketball__5130.json

```

1  {
2    "dataset_id": "sportsett_basketball",
3    "example_id": "5130",
4    "language": "en",
5    "output_mode": "multi_paragraph_report",
6    "parent_table": null,
7    "reference_sha256": "2c74673beb1cb0b6c4b079762408b342913c08ad6f7271c4c7229717fcc6eb27",
8    "references": [
9      "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
10   ],
11   "request": "Write a coherent game report from the supplied structured game data. Lead with the result, select the
12               most important performances and contrasts, and do not invent information.",
13   "source_payload": {
14     "game": {
15       "attendance": "16600",
16       "capacity": "19000",
17       "city": "Los Angeles",
18       "day": "5",
19       "dayname": "Monday",
20       "game_id": "5130",
21       "month": "November",
22       "season": "2018",
23       "stadium": "Staples Center",
24       "state": "California",
25       "year": "2018"
26     },
27     "teams": {
28       "home": {
29         "box_score": [
30           {
31             "+/-": "7",
32             "AST": "4",
33             "BLK": "0",
34             "DOUBLE": "none",
35             "DREB": "4",
36             "FG3A": "6",
37             "FG3M": "2",
38             "FG3_PCT": "33",
39             "FGA": "10",
40             "FGM": "4",
41             "FG_PCT": "40",
42             "FTA": "0",
43             "FTM": "0",
44             "FT_PCT": "0",
45             "MIN": "38",
46             "OREB": "2",
47             "PF": "5",
48             "PTS": "10",
49             "STL": "0",
50             "TOV": "0",
51             "TREB": "6",
52             "first_name": "Patrick",
53             "last_name": "Beverley",
54             "name": "Patrick Beverley",
55             "starter": "True"
56           },
57           {
38             "+/-": "7",

```

```

"AST": "2",
"BLK": "0",
"DOUBLE": "double",
"DREB": "9",
"FG3A": "7",
"FG3M": "4",
"FG3_PCT": "57",
"FGA": "16",
"FGM": "8",
"FG_PCT": "50",
"FTA": "3",
"FTM": "2",
"FT_PCT": "67",
"MIN": "34",
"OREB": "1",
"PF": "2",
"PTS": "22",
"STL": "1",
"TOV": "1",
"TREB": "10",
"first_name": "Tobias",
"last_name": "Harris",
"name": "Tobias Harris",
"starter": "True"
},
{
  "+/-": "0",
  "AST": "3",
  "BLK": "0",
  "DOUBLE": "none",
  "DREB": "4",
  "FG3A": "6",
  "FG3M": "2",
  "FG3_PCT": "33",
  "FGA": "16",
  "FGM": "7",
  "FG_PCT": "44",
  "FTA": "6",
  "FTM": "6",
  "FT_PCT": "100",
  "MIN": "31",
  "OREB": "0",
  "PF": "5",
  "PTS": "22",
  "STL": "0",
  "TOV": "2",
  "TREB": "4",
  "first_name": "Danilo",
  "last_name": "Gallinari",
  "name": "Danilo Gallinari",
  "starter": "True"
},
{
  "+/-": "7",
  "AST": "5",
  "BLK": "1",
  "DOUBLE": "none",
  "DREB": "1",
  "FG3A": "1",
  "FG3M": "0",
  "FG3_PCT": "0",
  "FGA": "6",
  "FGM": "2",
  "FG_PCT": "33",
  "FTA": "2",
  "FTM": "2",
  "FT_PCT": "100",
  "MIN": "24",
  "OREB": "2",
  "PF": "2",
  "PTS": "6",
  "STL": "1",
  "TOV": "2",
  "TREB": "3",
  "first_name": "Shai",
  "last_name": "Gilgeous-Alexander",
  "name": "Shai Gilgeous-Alexander",
  "starter": "True"
},
{
  "+/-": "2",
  "AST": "2",
  "BLK": "2",
  "DOUBLE": "none",
  "DREB": "5",
  "FG3A": "0",
  "FG3M": "0",
  "FG3_PCT": "0",
  "FGA": "8",
  "FGM": "3",
  "FG_PCT": "38",

```

```

149         "FTA": "4",
150         "FTM": "4",
151         "FT_PCT": "100",
152         "MIN": "17",
153         "OREB": "4",
154         "PF": "3",
155         "PTS": "10",
156         "STL": "0",
157         "TOV": "1",
158         "TREB": "9",
159         "first_name": "Boban",
160         "last_name": "ćMarjanovi",
161         "name": "Boban ćMarjanovi",
162         "starter": "True"
163     },
164     {
165         "+/-": "9",
166         "AST": "2",
167         "BLK": "0",
168         "DOUBLE": "none",
169         "DREB": "3",
170         "FG3A": "0",
171         "FG3M": "0",
172         "FG3_PCT": "0",
173         "FGA": "7",
174         "FGM": "6",
175         "FG_PCT": "86",
176         "FTA": "2",
177         "FTM": "1",
178         "FT_PCT": "50",
179         "MIN": "30",
180         "OREB": "4",
181         "PF": "1",
182         "PTS": "13",
183         "STL": "3",
184         "TOV": "4",
185         "TREB": "7",
186         "first_name": "Montrezl",
187         "last_name": "Harrell",
188         "name": "Montrezl Harrell",
189         "starter": "False"
190     },
191     {
192         "+/-": "8",
193         "AST": "6",
194         "BLK": "1",
195         "DOUBLE": "none",
196         "DREB": "0",
197         "FG3A": "4",
198         "FG3M": "1",
199         "FG3_PCT": "25",
200         "FGA": "17",
201         "FGM": "7",
202         "FG_PCT": "41",
203         "FTA": "5",
204         "FTM": "5",
205         "FT_PCT": "100",
206         "MIN": "27",
207         "OREB": "0",
208         "PF": "1",
209         "PTS": "20",
210         "STL": "0",
211         "TOV": "1",
212         "TREB": "0",
213         "first_name": "Lou",
214         "last_name": "Williams",
215         "name": "Lou Williams",
216         "starter": "False"
217     },
218     {
219         "+/-": "11",
220         "AST": "2",
221         "BLK": "0",
222         "DOUBLE": "none",
223         "DREB": "4",
224         "FG3A": "3",
225         "FG3M": "2",
226         "FG3_PCT": "67",
227         "FGA": "4",
228         "FGM": "3",
229         "FG_PCT": "75",
230         "FTA": "0",
231         "FTM": "0",
232         "FT_PCT": "0",
233         "MIN": "16",
234         "OREB": "0",
235         "PF": "3",
236         "PTS": "8",
237         "STL": "0",
238         "TOV": "0",
239         "TREB": "4",

```



```

240     "first_name": "Mike",
241     "last_name": "Scott",
242     "name": "Mike Scott",
243     "starter": "False"
244 },
245 {
246     "+/-": "3",
247     "AST": "4",
248     "BLK": "0",
249     "DOUBLE": "none",
250     "DREB": "0",
251     "FG3A": "1",
252     "FG3M": "1",
253     "FG3_PCT": "100",
254     "FGA": "1",
255     "FGM": "1",
256     "FG_PCT": "100",
257     "FTA": "0",
258     "FTM": "0",
259     "FT_PCT": "0",
260     "MIN": "11",
261     "OREB": "0",
262     "PF": "3",
263     "PTS": "3",
264     "STL": "0",
265     "TOV": "1",
266     "TREB": "0",
267     "first_name": "šMilo",
268     "last_name": "ćTeodosi",
269     "name": "šMilo ćTeodosi",
270     "starter": "False"
271 },
272 {
273     "+/-": "1",
274     "AST": "0",
275     "BLK": "0",
276     "DOUBLE": "none",
277     "DREB": "0",
278     "FG3A": "3",
279     "FG3M": "2",
280     "FG3_PCT": "67",
281     "FGA": "3",
282     "FGM": "2",
283     "FG_PCT": "67",
284     "FTA": "0",
285     "FTM": "0",
286     "FT_PCT": "0",
287     "MIN": "6",
288     "OREB": "0",
289     "PF": "2",
290     "PTS": "6",
291     "STL": "0",
292     "TOV": "0",
293     "TREB": "0",
294     "first_name": "Jerome",
295     "last_name": "Robinson",
296     "name": "Jerome Robinson",
297     "starter": "False"
298 },
299 {
300     "+/-": "0",
301     "AST": "0",
302     "BLK": "0",
303     "DOUBLE": "none",
304     "DREB": "0",
305     "FG3A": "0",
306     "FG3M": "0",
307     "FG3_PCT": "0",
308     "FGA": "0",
309     "FGM": "0",
310     "FG_PCT": "0",
311     "FTA": "0",
312     "FTM": "0",
313     "FT_PCT": "0",
314     "MIN": "0",
315     "OREB": "0",
316     "PF": "0",
317     "PTS": "0",
318     "STL": "0",
319     "TOV": "0",
320     "TREB": "0",
321     "first_name": "Marcin",
322     "last_name": "Gortat",
323     "name": "Marcin Gortat",
324     "starter": "False"
325 },
326 {
327     "+/-": "0",
328     "AST": "0",
329     "BLK": "0",
330     "DOUBLE": "none",

```

```

    "DREB": "0",
    "FG3A": "0",
    "FG3M": "0",
    "FG3_PCT": "0",
    "FGA": "0",
    "FGM": "0",
    "FG_PCT": "0",
    "FTA": "0",
    "FTM": "0",
    "FT_PCT": "0",
    "MIN": "0",
    "OREB": "0",
    "PF": "0",
    "PTS": "0",
    "STL": "0",
    "TOV": "0",
    "TREB": "0",
    "first_name": "Sindarius",
    "last_name": "Thornwell",
    "name": "Sindarius Thornwell",
    "starter": "False"
  },
  {
    "+/-": "0",
    "AST": "0",
    "BLK": "0",
    "DOUBLE": "none",
    "DREB": "0",
    "FG3A": "0",
    "FG3M": "0",
    "FG3_PCT": "0",
    "FGA": "0",
    "FGM": "0",
    "FG_PCT": "0",
    "FTA": "0",
    "FTM": "0",
    "FT_PCT": "0",
    "MIN": "0",
    "OREB": "0",
    "PF": "0",
    "PTS": "0",
    "STL": "0",
    "TOV": "0",
    "TREB": "0",
    "first_name": "Tyrone",
    "last_name": "Wallace",
    "name": "Tyrone Wallace",
    "starter": "False"
  }
],
"conference": "Western Conference",
"conference_standing": 5,
"division": "Pacific",
"game_number": "10",
"line_score": {
  "H1": {
    "AST": "109",
    "BLK": "20",
    "DREB": "69",
    "FG3A": "711",
    "FG3M": "45",
    "FG3_PCT": "6",
    "FGA": "2423",
    "FGM": "1212",
    "FG_PCT": "50",
    "FTA": "34",
    "FTM": "33",
    "FT_PCT": "97",
    "MIN": "6060",
    "OREB": "24",
    "PTS": "3132",
    "STL": "01",
    "TOV": "05",
    "TREB": "93"
  },
  "H2": {
    "AST": "56",
    "BLK": "20",
    "DREB": "87",
    "FG3A": "67",
    "FG3M": "32",
    "FG3_PCT": "48",
    "FGA": "1923",
    "FGM": "910",
    "FG_PCT": "47",
    "FTA": "87",
    "FTM": "86",
    "FT_PCT": "99",
    "MIN": "6060",
    "OREB": "34",
    "PTS": "2928",

```

```

422     "STL": "04",
423     "TOV": "43",
424     "TREB": "121"
425 },
426 "0T": {
427     "AST": "0",
428     "BLK": "0",
429     "DREB": "0",
430     "FG3A": "0",
431     "FG3M": "0",
432     "FG3_PCT": "0",
433     "FGA": "0",
434     "FGM": "0",
435     "FG_PCT": "0",
436     "FTA": "0",
437     "FTM": "0",
438     "FT_PCT": "0",
439     "MIN": "0",
440     "OREB": "0",
441     "PTS": "0",
442     "STL": "0",
443     "TOV": "0",
444     "TREB": "0"
445 },
446 "Q1": {
447     "AST": "10",
448     "BLK": "2",
449     "DREB": "6",
450     "FG3A": "7",
451     "FG3M": "4",
452     "FG3_PCT": "57",
453     "FGA": "24",
454     "FGM": "12",
455     "FG_PCT": "50",
456     "FTA": "3",
457     "FTM": "3",
458     "FT_PCT": "100",
459     "MIN": "60",
460     "OREB": "2",
461     "PTS": "31",
462     "STL": "0",
463     "TOV": "0",
464     "TREB": "8"
465 },
466 "Q2": {
467     "AST": "9",
468     "BLK": "0",
469     "DREB": "9",
470     "FG3A": "11",
471     "FG3M": "5",
472     "FG3_PCT": "45",
473     "FGA": "23",
474     "FGM": "12",
475     "FG_PCT": "52",
476     "FTA": "4",
477     "FTM": "3",
478     "FT_PCT": "75",
479     "MIN": "60",
480     "OREB": "4",
481     "PTS": "32",
482     "STL": "1",
483     "TOV": "5",
484     "TREB": "13"
485 },
486 "Q3": {
487     "AST": "5",
488     "BLK": "2",
489     "DREB": "8",
490     "FG3A": "6",
491     "FG3M": "3",
492     "FG3_PCT": "50",
493     "FGA": "19",
494     "FGM": "9",
495     "FG_PCT": "47",
496     "FTA": "8",
497     "FTM": "8",
498     "FT_PCT": "100",
499     "MIN": "60",
500     "OREB": "3",
501     "PTS": "29",
502     "STL": "0",
503     "TOV": "4",
504     "TREB": "11"
505 },
506 "Q4": {
507     "AST": "6",
508     "BLK": "0",
509     "DREB": "7",
510     "FG3A": "7",
511     "FG3M": "2",
512     "FG3_PCT": "29",

```

```

513         "FGA": "23",
514         "FGM": "10",
515         "FG_PCT": "43",
516         "FTA": "7",
517         "FTM": "6",
518         "FT_PCT": "86",
519         "MIN": "60",
520         "OREB": "4",
521         "PTS": "28",
522         "STL": "4",
523         "TOV": "3",
524         "TREB": "11"
525     },
526     "game": {
527         "AST": "30",
528         "BLK": "4",
529         "DREB": "30",
530         "FG3A": "31",
531         "FG3M": "14",
532         "FG3_PCT": "45",
533         "FGA": "88",
534         "FGM": "43",
535         "FG_PCT": "49",
536         "FTA": "22",
537         "FTM": "20",
538         "FT_PCT": "91",
539         "MIN": "4",
540         "OREB": "13",
541         "PF": "27",
542         "PTS": "120",
543         "STL": "5",
544         "TOV": "12",
545         "TREB": "43"
546     }
547 },
548 "losses": "4",
549 "name": "Clippers",
550 "next_game": {
551     "city": "Portland",
552     "day": "8",
553     "dayname": "Thursday",
554     "is_home": "False",
555     "month": "November",
556     "opponent_name": "Trail Blazers",
557     "opponent_place": "Portland",
558     "stadium": "Moda Center",
559     "year": "2018"
560 },
561 "next_game_id": "6035",
562 "place": "Los Angeles",
563 "previous_game_id": "5499",
564 "wins": "6"
565 },
566 "vis": {
567     "box_score": [
568         {
569             "+/-": "-11",
570             "AST": "5",
571             "BLK": "1",
572             "DOUBLE": "none",
573             "DREB": "4",
574             "FG3A": "2",
575             "FG3M": "0",
576             "FG3_PCT": "0",
577             "FGA": "13",
578             "FGM": "6",
579             "FG_PCT": "46",
580             "FTA": "11",
581             "FTM": "8",
582             "FT_PCT": "73",
583             "MIN": "39",
584             "OREB": "0",
585             "PF": "4",
586             "PTS": "20",
587             "STL": "1",
588             "TOV": "2",
589             "TREB": "4",
590             "first_name": "Jimmy",
591             "last_name": "Butler",
592             "name": "Jimmy Butler",
593             "starter": "True"
594         },
595         {
596             "+/-": "-5",
597             "AST": "2",
598             "BLK": "4",
599             "DOUBLE": "double",
600             "DREB": "9",
601             "FG3A": "3",
602             "FG3M": "1",
603             "FG3_PCT": "33",

```

```

604         "FGA": "13",
605         "FGM": "8",
606         "FG_PCT": "62",
607         "FTA": "3",
608         "FTM": "3",
609         "FT_PCT": "100",
610         "MIN": "35",
611         "OREB": "3",
612         "PF": "3",
613         "PTS": "20",
614         "STL": "3",
615         "TOV": "3",
616         "TREB": "12",
617         "first_name": "Karl-Anthony",
618         "last_name": "Towns",
619         "name": "Karl-Anthony Towns",
620         "starter": "True"
621     },
622     {
623         "+/-": "-8",
624         "AST": "4",
625         "BLK": "0",
626         "DOUBLE": "none",
627         "DREB": "2",
628         "FG3A": "3",
629         "FG3M": "1",
630         "FG3_PCT": "33",
631         "FGA": "20",
632         "FGM": "8",
633         "FG_PCT": "40",
634         "FTA": "4",
635         "FTM": "4",
636         "FT_PCT": "100",
637         "MIN": "34",
638         "OREB": "1",
639         "PF": "2",
640         "PTS": "21",
641         "STL": "0",
642         "TOV": "2",
643         "TREB": "3",
644         "first_name": "Derrick",
645         "last_name": "Rose",
646         "name": "Derrick Rose",
647         "starter": "True"
648     },
649     {
650         "+/-": "-8",
651         "AST": "2",
652         "BLK": "2",
653         "DOUBLE": "none",
654         "DREB": "6",
655         "FG3A": "7",
656         "FG3M": "2",
657         "FG3_PCT": "29",
658         "FGA": "16",
659         "FGM": "4",
660         "FG_PCT": "25",
661         "FTA": "4",
662         "FTM": "3",
663         "FT_PCT": "75",
664         "MIN": "34",
665         "OREB": "0",
666         "PF": "3",
667         "PTS": "13",
668         "STL": "1",
669         "TOV": "2",
670         "TREB": "6",
671         "first_name": "Andrew",
672         "last_name": "Wiggins",
673         "name": "Andrew Wiggins",
674         "starter": "True"
675     },
676     {
677         "+/-": "-7",
678         "AST": "1",
679         "BLK": "1",
680         "DOUBLE": "none",
681         "DREB": "6",
682         "FG3A": "0",
683         "FG3M": "0",
684         "FG3_PCT": "0",
685         "FGA": "9",
686         "FGM": "6",
687         "FG_PCT": "67",
688         "FTA": "4",
689         "FTM": "3",
690         "FT_PCT": "75",
691         "MIN": "32",
692         "OREB": "3",
693         "PF": "6",
694         "PTS": "15",

```

```

695     "STL": "2",
696     "TOV": "0",
697     "TREB": "9",
698     "first_name": "Taj",
699     "last_name": "Gibson",
700     "name": "Taj Gibson",
701     "starter": "True"
702 },
703 {
704     "+/-": "-1",
705     "AST": "2",
706     "BLK": "0",
707     "DOUBLE": "none",
708     "DREB": "1",
709     "FG3A": "3",
710     "FG3M": "1",
711     "FG3_PCT": "33",
712     "FGA": "6",
713     "FGM": "3",
714     "FG_PCT": "50",
715     "FTA": "0",
716     "FTM": "0",
717     "FT_PCT": "0",
718     "MIN": "19",
719     "OREB": "0",
720     "PF": "4",
721     "PTS": "7",
722     "STL": "0",
723     "TOV": "1",
724     "TREB": "1",
725     "first_name": "Josh",
726     "last_name": "Okogie",
727     "name": "Josh Okogie",
728     "starter": "False"
729 },
730 {
731     "+/-": "-4",
732     "AST": "0",
733     "BLK": "0",
734     "DOUBLE": "none",
735     "DREB": "1",
736     "FG3A": "0",
737     "FG3M": "0",
738     "FG3_PCT": "0",
739     "FGA": "0",
740     "FGM": "0",
741     "FG_PCT": "0",
742     "FTA": "2",
743     "FTM": "1",
744     "FT_PCT": "50",
745     "MIN": "15",
746     "OREB": "0",
747     "PF": "1",
748     "PTS": "1",
749     "STL": "0",
750     "TOV": "0",
751     "TREB": "1",
752     "first_name": "Anthony",
753     "last_name": "Tolliver",
754     "name": "Anthony Tolliver",
755     "starter": "False"
756 },
757 {
758     "+/-": "-5",
759     "AST": "3",
760     "BLK": "0",
761     "DOUBLE": "none",
762     "DREB": "1",
763     "FG3A": "3",
764     "FG3M": "0",
765     "FG3_PCT": "0",
766     "FGA": "5",
767     "FGM": "2",
768     "FG_PCT": "40",
769     "FTA": "0",
770     "FTM": "0",
771     "FT_PCT": "0",
772     "MIN": "15",
773     "OREB": "0",
774     "PF": "0",
775     "PTS": "4",
776     "STL": "1",
777     "TOV": "1",
778     "TREB": "1",
779     "first_name": "Tyus",
780     "last_name": "Jones",
781     "name": "Tyus Jones",
782     "starter": "False"
783 },
784 {
785     "+/-": "-6",

```

```

786     "AST": "1",
787     "BLK": "0",
788     "DOUBLE": "none",
789     "DREB": "0",
790     "FG3A": "0",
791     "FG3M": "0",
792     "FG3_PCT": "0",
793     "FGA": "3",
794     "FGM": "3",
795     "FG_PCT": "100",
796     "FTA": "2",
797     "FTM": "2",
798     "FT_PCT": "100",
799     "MIN": "12",
800     "OREB": "1",
801     "PF": "0",
802     "PTS": "8",
803     "STL": "1",
804     "TOV": "0",
805     "TREB": "1",
806     "first_name": "Gorgui",
807     "last_name": "Dieng",
808     "name": "Gorgui Dieng",
809     "starter": "False"
810 },
811 {
812     "+/-": "0",
813     "AST": "0",
814     "BLK": "0",
815     "DOUBLE": "none",
816     "DREB": "0",
817     "FG3A": "0",
818     "FG3M": "0",
819     "FG3_PCT": "0",
820     "FGA": "0",
821     "FGM": "0",
822     "FG_PCT": "0",
823     "FTA": "0",
824     "FTM": "0",
825     "FT_PCT": "0",
826     "MIN": "0",
827     "OREB": "0",
828     "PF": "0",
829     "PTS": "0",
830     "STL": "0",
831     "TOV": "0",
832     "TREB": "0",
833     "first_name": "Luol",
834     "last_name": "Deng",
835     "name": "Luol Deng",
836     "starter": "False"
837 },
838 {
839     "+/-": "0",
840     "AST": "0",
841     "BLK": "0",
842     "DOUBLE": "none",
843     "DREB": "0",
844     "FG3A": "0",
845     "FG3M": "0",
846     "FG3_PCT": "0",
847     "FGA": "0",
848     "FGM": "0",
849     "FG_PCT": "0",
850     "FTA": "0",
851     "FTM": "0",
852     "FT_PCT": "0",
853     "MIN": "0",
854     "OREB": "0",
855     "PF": "0",
856     "PTS": "0",
857     "STL": "0",
858     "TOV": "0",
859     "TREB": "0",
860     "first_name": "Keita",
861     "last_name": "Bates-Diop",
862     "name": "Keita Bates-Diop",
863     "starter": "False"
864 },
865 {
866     "+/-": "0",
867     "AST": "0",
868     "BLK": "0",
869     "DOUBLE": "none",
870     "DREB": "0",
871     "FG3A": "0",
872     "FG3M": "0",
873     "FG3_PCT": "0",
874     "FGA": "0",
875     "FGM": "0",
876     "FG_PCT": "0",

```

```

877     "FTA": "0",
878     "FTM": "0",
879     "FT_PCT": "0",
880     "MIN": "0",
881     "OREB": "0",
882     "PF": "0",
883     "PTS": "0",
884     "STL": "0",
885     "TOV": "0",
886     "TREB": "0",
887     "first_name": "C.J.",
888     "last_name": "Williams",
889     "name": "C.J. Williams",
890     "starter": "False"
891 },
892 {
893     "+/-": "0",
894     "AST": "0",
895     "BLK": "0",
896     "DOUBLE": "none",
897     "DREB": "0",
898     "FG3A": "0",
899     "FG3M": "0",
900     "FG3_PCT": "0",
901     "FGA": "0",
902     "FGM": "0",
903     "FG_PCT": "0",
904     "FTA": "0",
905     "FTM": "0",
906     "FT_PCT": "0",
907     "MIN": "0",
908     "OREB": "0",
909     "PF": "0",
910     "PTS": "0",
911     "STL": "0",
912     "TOV": "0",
913     "TREB": "0",
914     "first_name": "James",
915     "last_name": "Nunnally",
916     "name": "James Nunnally",
917     "starter": "False"
918 }
919 ],
920 "conference": "Western Conference",
921 "conference_standing": 13,
922 "division": "Northwest",
923 "game_number": "11",
924 "line_score": {
925     "H1": {
926         "AST": "58",
927         "BLK": "23",
928         "DREB": "107",
929         "FG3A": "65",
930         "FG3M": "21",
931         "FG3_PCT": "32",
932         "FGA": "2520",
933         "FGM": "1310",
934         "FG_PCT": "52",
935         "FTA": "65",
936         "FTM": "55",
937         "FT_PCT": "85",
938         "MIN": "6060",
939         "OREB": "41",
940         "PTS": "3326",
941         "STL": "03",
942         "TOV": "03",
943         "TREB": "148"
944     },
945     "H2": {
946         "AST": "52",
947         "BLK": "30",
948         "DREB": "67",
949         "FG3A": "46",
950         "FG3M": "20",
951         "FG3_PCT": "43",
952         "FGA": "2317",
953         "FGM": "107",
954         "FG_PCT": "5",
955         "FTA": "415",
956         "FTM": "311",
957         "FT_PCT": "75",
958         "MIN": "6060",
959         "OREB": "30",
960         "PTS": "2525",
961         "STL": "42",
962         "TOV": "35",
963         "TREB": "97"
964     },
965     "OT": {
966         "AST": "0",
967         "BLK": "0",

```



```
968         "DREB": "0",
969         "FG3A": "0",
970         "FG3M": "0",
971         "FG3_PCT": "0",
972         "FGA": "0",
973         "FGM": "0",
974         "FG_PCT": "0",
975         "FTA": "0",
976         "FTM": "0",
977         "FT_PCT": "0",
978         "MIN": "0",
979         "OREB": "0",
980         "PTS": "0",
981         "STL": "0",
982         "TOV": "0",
983         "TREB": "0"
984     },
985     "Q1": {
986         "AST": "5",
987         "BLK": "2",
988         "DREB": "10",
989         "FG3A": "6",
990         "FG3M": "2",
991         "FG3_PCT": "33",
992         "FGA": "25",
993         "FGM": "13",
994         "FG_PCT": "52",
995         "FTA": "6",
996         "FTM": "5",
997         "FT_PCT": "83",
998         "MIN": "60",
999         "OREB": "4",
1000         "PTS": "33",
1001         "STL": "0",
1002         "TOV": "0",
1003         "TREB": "14"
1004     },
1005     "Q2": {
1006         "AST": "8",
1007         "BLK": "3",
1008         "DREB": "7",
1009         "FG3A": "5",
1010         "FG3M": "1",
1011         "FG3_PCT": "20",
1012         "FGA": "20",
1013         "FGM": "10",
1014         "FG_PCT": "50",
1015         "FTA": "5",
1016         "FTM": "5",
1017         "FT_PCT": "100",
1018         "MIN": "60",
1019         "OREB": "1",
1020         "PTS": "26",
1021         "STL": "3",
1022         "TOV": "3",
1023         "TREB": "8"
1024     },
1025     "Q3": {
1026         "AST": "5",
1027         "BLK": "3",
1028         "DREB": "6",
1029         "FG3A": "4",
1030         "FG3M": "2",
1031         "FG3_PCT": "50",
1032         "FGA": "23",
1033         "FGM": "10",
1034         "FG_PCT": "43",
1035         "FTA": "4",
1036         "FTM": "3",
1037         "FT_PCT": "75",
1038         "MIN": "60",
1039         "OREB": "3",
1040         "PTS": "25",
1041         "STL": "4",
1042         "TOV": "3",
1043         "TREB": "9"
1044     },
1045     "Q4": {
1046         "AST": "2",
1047         "BLK": "0",
1048         "DREB": "7",
1049         "FG3A": "6",
1050         "FG3M": "0",
1051         "FG3_PCT": "0",
1052         "FGA": "17",
1053         "FGM": "7",
1054         "FG_PCT": "41",
1055         "FTA": "15",
1056         "FTM": "11",
1057         "FT_PCT": "73",
1058         "MIN": "60",
```

```

1059         "OREB": "0",
1060         "PTS": "25",
1061         "STL": "2",
1062         "TOV": "5",
1063         "TREB": "7"
1064     },
1065     "game": {
1066         "AST": "20",
1067         "BLK": "8",
1068         "DREB": "30",
1069         "FG3A": "21",
1070         "FG3M": "5",
1071         "FG3_PCT": "24",
1072         "FGA": "85",
1073         "FGM": "40",
1074         "FG_PCT": "47",
1075         "FTA": "30",
1076         "FTM": "24",
1077         "FT_PCT": "80",
1078         "MIN": "4",
1079         "OREB": "8",
1080         "PF": "23",
1081         "PTS": "109",
1082         "STL": "9",
1083         "TOV": "11",
1084         "TREB": "38"
1085     }
1086 },
1087 "losses": "7",
1088 "name": "Timberwolves",
1089 "next_game": {
1090     "city": "Los Angeles",
1091     "day": "7",
1092     "dayname": "Wednesday",
1093     "is_home": "False",
1094     "month": "November",
1095     "opponent_name": "Lakers",
1096     "opponent_place": "Los Angeles",
1097     "stadium": "Staples Center",
1098     "year": "2018"
1099 },
1100 "next_game_id": "5459",
1101 "place": "Minnesota",
1102 "previous_game_id": "6033",
1103 "wins": "4"
1104 }
1105 }
1106 },
1107 "source_sha256": "432bc48818396a9345adffd433b22b1710c709c30e18a4f6935c3464b65c8e2f",
1108 "task_family": "event_report"
1109 }

```

Listing 76: D12: submission/program_listings/build/generated/data/sportsett_basketball__5372.json

```

1  {
2  "dataset_id": "sportsett_basketball",
3  "example_id": "5372",
4  "language": "en",
5  "output_mode": "multi_paragraph_report",
6  "parent_table": null,
7  "reference_sha256": "087382c8eb93adfbe153771f92bcac63ce11fe957c30e23620526c9e93103113",
8  "references": [
9      "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
10 ],
11 "request": "Write a coherent game report from the supplied structured game data. Lead with the result, select the
12     most important performances and contrasts, and do not invent information.",
13 "source_payload": {
14     "game": {
15         "attendance": "17600",
16         "capacity": "17600",
17         "city": "Sacramento",
18         "day": "17",
19         "dayname": "Wednesday",
20         "game_id": "5372",
21         "month": "October",
22         "season": "2018",
23         "stadium": "Golden 1 Center",
24         "state": "California",
25         "year": "2018"
26     },
27     "teams": {
28         "home": {
29             "box_score": [
30                 {
31                     "+/-": "16",
32                     "AST": "4",
33                     "BLK": "2",
34                     "DOUBLE": "none",

```

```

    "DREB": "5",
    "FG3A": "0",
    "FG3M": "0",
    "FG3_PCT": "0",
    "FGA": "15",
    "FGM": "10",
    "FG_PCT": "67",
    "FTA": "6",
    "FTM": "3",
    "FT_PCT": "50",
    "MIN": "38",
    "OREB": "2",
    "PF": "2",
    "PTS": "23",
    "STL": "1",
    "TOV": "0",
    "TREB": "7",
    "first_name": "Willie",
    "last_name": "Cauley-Stein",
    "name": "Willie Cauley-Stein",
    "starter": "True"
  },
  {
    "+/-": "19",
    "AST": "7",
    "BLK": "0",
    "DOUBLE": "none",
    "DREB": "4",
    "FG3A": "2",
    "FG3M": "0",
    "FG3_PCT": "0",
    "FGA": "16",
    "FGM": "8",
    "FG_PCT": "50",
    "FTA": "7",
    "FTM": "5",
    "FT_PCT": "71",
    "MIN": "37",
    "OREB": "0",
    "PF": "5",
    "PTS": "21",
    "STL": "3",
    "TOV": "3",
    "TREB": "4",
    "first_name": "De'Aaron",
    "last_name": "Fox",
    "name": "De'Aaron Fox",
    "starter": "True"
  },
  {
    "+/-": "9",
    "AST": "1",
    "BLK": "1",
    "DOUBLE": "none",
    "DREB": "5",
    "FG3A": "4",
    "FG3M": "1",
    "FG3_PCT": "25",
    "FGA": "18",
    "FGM": "9",
    "FG_PCT": "50",
    "FTA": "1",
    "FTM": "0",
    "FT_PCT": "0",
    "MIN": "31",
    "OREB": "1",
    "PF": "6",
    "PTS": "19",
    "STL": "1",
    "TOV": "4",
    "TREB": "6",
    "first_name": "Buddy",
    "last_name": "Hield",
    "name": "Buddy Hield",
    "starter": "True"
  },
  {
    "+/-": "10",
    "AST": "1",
    "BLK": "0",
    "DOUBLE": "none",
    "DREB": "1",
    "FG3A": "3",
    "FG3M": "2",
    "FG3_PCT": "67",
    "FGA": "7",
    "FGM": "3",
    "FG_PCT": "43",
    "FTA": "4",
    "FTM": "4",
    "FT_PCT": "100",

```

```

125     "MIN": "30",
126     "OREB": "0",
127     "PF": "2",
128     "PTS": "12",
129     "STL": "0",
130     "TOV": "1",
131     "TREB": "1",
132     "first_name": "Yogi",
133     "last_name": "Ferrell",
134     "name": "Yogi Ferrell",
135     "starter": "True"
136 },
137 {
138     "+/-": "12",
139     "AST": "2",
140     "BLK": "0",
141     "DOUBLE": "none",
142     "DREB": "7",
143     "FG3A": "2",
144     "FG3M": "2",
145     "FG3_PCT": "100",
146     "FGA": "12",
147     "FGM": "8",
148     "FG_PCT": "67",
149     "FTA": "0",
150     "FTM": "0",
151     "FT_PCT": "0",
152     "MIN": "27",
153     "OREB": "1",
154     "PF": "2",
155     "PTS": "18",
156     "STL": "1",
157     "TOV": "0",
158     "TREB": "8",
159     "first_name": "Nemanja",
160     "last_name": "Bjelica",
161     "name": "Nemanja Bjelica",
162     "starter": "True"
163 },
164 {
165     "+/-": "-19",
166     "AST": "1",
167     "BLK": "0",
168     "DOUBLE": "none",
169     "DREB": "2",
170     "FG3A": "4",
171     "FG3M": "0",
172     "FG3_PCT": "0",
173     "FGA": "10",
174     "FGM": "4",
175     "FG_PCT": "40",
176     "FTA": "0",
177     "FTM": "0",
178     "FT_PCT": "0",
179     "MIN": "30",
180     "OREB": "0",
181     "PF": "2",
182     "PTS": "8",
183     "STL": "2",
184     "TOV": "0",
185     "TREB": "2",
186     "first_name": "Justin",
187     "last_name": "Jackson",
188     "name": "Justin Jackson",
189     "starter": "False"
190 },
191 {
192     "+/-": "-29",
193     "AST": "0",
194     "BLK": "0",
195     "DOUBLE": "none",
196     "DREB": "3",
197     "FG3A": "3",
198     "FG3M": "1",
199     "FG3_PCT": "33",
200     "FGA": "7",
201     "FGM": "2",
202     "FG_PCT": "29",
203     "FTA": "0",
204     "FTM": "0",
205     "FT_PCT": "0",
206     "MIN": "17",
207     "OREB": "0",
208     "PF": "2",
209     "PTS": "5",
210     "STL": "0",
211     "TOV": "0",
212     "TREB": "3",
213     "first_name": "Iman",
214     "last_name": "Shumpert",
215     "name": "Iman Shumpert",

```

```

216     "starter": "False"
217 },
218 {
219     "+/-": "-14",
220     "AST": "0",
221     "BLK": "0",
222     "DOUBLE": "none",
223     "DREB": "4",
224     "FG3A": "0",
225     "FG3M": "0",
226     "FG3_PCT": "0",
227     "FGA": "6",
228     "FGM": "3",
229     "FG_PCT": "50",
230     "FTA": "0",
231     "FTM": "0",
232     "FT_PCT": "0",
233     "MIN": "12",
234     "OREB": "1",
235     "PF": "2",
236     "PTS": "6",
237     "STL": "0",
238     "TOV": "0",
239     "TREB": "5",
240     "first_name": "Marvin",
241     "last_name": "Bagley",
242     "name": "Marvin Bagley",
243     "starter": "False"
244 },
245 {
246     "+/-": "-22",
247     "AST": "1",
248     "BLK": "0",
249     "DOUBLE": "none",
250     "DREB": "1",
251     "FG3A": "0",
252     "FG3M": "0",
253     "FG3_PCT": "0",
254     "FGA": "2",
255     "FGM": "1",
256     "FG_PCT": "50",
257     "FTA": "0",
258     "FTM": "0",
259     "FT_PCT": "0",
260     "MIN": "9",
261     "OREB": "0",
262     "PF": "4",
263     "PTS": "2",
264     "STL": "0",
265     "TOV": "1",
266     "TREB": "1",
267     "first_name": "Harry",
268     "last_name": "Giles",
269     "name": "Harry Giles",
270     "starter": "False"
271 },
272 {
273     "+/-": "-12",
274     "AST": "0",
275     "BLK": "0",
276     "DOUBLE": "none",
277     "DREB": "0",
278     "FG3A": "1",
279     "FG3M": "1",
280     "FG3_PCT": "100",
281     "FGA": "2",
282     "FGM": "1",
283     "FG_PCT": "50",
284     "FTA": "0",
285     "FTM": "0",
286     "FT_PCT": "0",
287     "MIN": "4",
288     "OREB": "0",
289     "PF": "0",
290     "PTS": "3",
291     "STL": "0",
292     "TOV": "0",
293     "TREB": "0",
294     "first_name": "Frank",
295     "last_name": "Mason",
296     "name": "Frank Mason",
297     "starter": "False"
298 },
299 {
300     "+/-": "0",
301     "AST": "0",
302     "BLK": "0",
303     "DOUBLE": "none",
304     "DREB": "0",
305     "FG3A": "0",
306     "FG3M": "0",

```

```

"FG3_PCT": "0",
"FGA": "0",
"FGM": "0",
"FG_PCT": "0",
"FTA": "0",
"FTM": "0",
"FT_PCT": "0",
"MIN": "0",
"OREB": "0",
"PF": "0",
"PTS": "0",
"STL": "0",
"TOV": "0",
"TREB": "0",
"first_name": "Ben",
"last_name": "McLemore",
"name": "Ben McLemore",
"starter": "False"
},
{
  "+/-": "0",
  "AST": "0",
  "BLK": "0",
  "DOUBLE": "none",
  "DREB": "0",
  "FG3A": "0",
  "FG3M": "0",
  "FG3_PCT": "0",
  "FGA": "0",
  "FGM": "0",
  "FG_PCT": "0",
  "FTA": "0",
  "FTM": "0",
  "FT_PCT": "0",
  "MIN": "0",
  "OREB": "0",
  "PF": "0",
  "PTS": "0",
  "STL": "0",
  "TOV": "0",
  "TREB": "0",
  "first_name": "Skal",
  "last_name": "Labissière",
  "name": "Skal Labissière",
  "starter": "False"
},
{
  "+/-": "0",
  "AST": "0",
  "BLK": "0",
  "DOUBLE": "none",
  "DREB": "0",
  "FG3A": "0",
  "FG3M": "0",
  "FG3_PCT": "0",
  "FGA": "0",
  "FGM": "0",
  "FG_PCT": "0",
  "FTA": "0",
  "FTM": "0",
  "FT_PCT": "0",
  "MIN": "0",
  "OREB": "0",
  "PF": "0",
  "PTS": "0",
  "STL": "0",
  "TOV": "0",
  "TREB": "0",
  "first_name": "Wenyen",
  "last_name": "Gabriel",
  "name": "Wenyen Gabriel",
  "starter": "False"
}
},
"conference": "Western Conference",
"conference_standing": 13,
"division": "Pacific",
"game_number": "1",
"line_score": {
  "H1": {
    "AST": "42",
    "BLK": "00",
    "DREB": "810",
    "FG3A": "34",
    "FG3M": "30",
    "FG3_PCT": "88",
    "FGA": "2423",
    "FGM": "159",
    "FG_PCT": "7",
    "FTA": "34",
    "FTM": "13",

```

```
398         "FT_PCT": "38",
399         "MIN": "6060",
400         "OREB": "10",
401         "PTS": "3421",
402         "STL": "21",
403         "TOV": "23",
404         "TREB": "820"
405     },
406     "H2": {
407         "AST": "74",
408         "BLK": "21",
409         "DREB": "68",
410         "FG3A": "75",
411         "FG3M": "31",
412         "FG3_PCT": "41",
413         "FGA": "2127",
414         "FGM": "1312",
415         "FG_PCT": "62",
416         "FTA": "56",
417         "FTM": "35",
418         "FT_PCT": "62",
419         "MIN": "6060",
420         "OREB": "13",
421         "PTS": "3230",
422         "STL": "14",
423         "TOV": "22",
424         "TREB": "81"
425     },
426     "OT": {
427         "AST": "0",
428         "BLK": "0",
429         "DREB": "0",
430         "FG3A": "0",
431         "FG3M": "0",
432         "FG3_PCT": "0",
433         "FGA": "0",
434         "FGM": "0",
435         "FG_PCT": "0",
436         "FTA": "0",
437         "FTM": "0",
438         "FT_PCT": "0",
439         "MIN": "0",
440         "OREB": "0",
441         "PTS": "0",
442         "STL": "0",
443         "TOV": "0",
444         "TREB": "0"
445     },
446     "Q1": {
447         "AST": "4",
448         "BLK": "0",
449         "DREB": "8",
450         "FG3A": "3",
451         "FG3M": "3",
452         "FG3_PCT": "100",
453         "FGA": "24",
454         "FGM": "15",
455         "FG_PCT": "62",
456         "FTA": "3",
457         "FTM": "1",
458         "FT_PCT": "33",
459         "MIN": "60",
460         "OREB": "1",
461         "PTS": "34",
462         "STL": "2",
463         "TOV": "2",
464         "TREB": "9"
465     },
466     "Q2": {
467         "AST": "2",
468         "BLK": "0",
469         "DREB": "10",
470         "FG3A": "4",
471         "FG3M": "0",
472         "FG3_PCT": "0",
473         "FGA": "23",
474         "FGM": "9",
475         "FG_PCT": "39",
476         "FTA": "4",
477         "FTM": "3",
478         "FT_PCT": "75",
479         "MIN": "60",
480         "OREB": "0",
481         "PTS": "21",
482         "STL": "1",
483         "TOV": "3",
484         "TREB": "10"
485     },
486     "Q3": {
487         "AST": "7",
488         "BLK": "2",
```

```

489         "DREB": "6",
490         "FG3A": "7",
491         "FG3M": "3",
492         "FG3_PCT": "43",
493         "FGA": "21",
494         "FGM": "13",
495         "FG_PCT": "62",
496         "FTA": "5",
497         "FTM": "3",
498         "FT_PCT": "60",
499         "MIN": "60",
500         "OREB": "1",
501         "PTS": "32",
502         "STL": "1",
503         "TOV": "2",
504         "TREB": "7"
505     },
506     "Q4": {
507         "AST": "4",
508         "BLK": "1",
509         "DREB": "8",
510         "FG3A": "5",
511         "FG3M": "1",
512         "FG3_PCT": "20",
513         "FGA": "27",
514         "FGM": "12",
515         "FG_PCT": "44",
516         "FTA": "6",
517         "FTM": "5",
518         "FT_PCT": "83",
519         "MIN": "60",
520         "OREB": "3",
521         "PTS": "30",
522         "STL": "4",
523         "TOV": "2",
524         "TREB": "11"
525     },
526     "game": {
527         "AST": "17",
528         "BLK": "3",
529         "DREB": "32",
530         "FG3A": "19",
531         "FG3M": "7",
532         "FG3_PCT": "37",
533         "FGA": "95",
534         "FGM": "49",
535         "FG_PCT": "52",
536         "FTA": "18",
537         "FTM": "12",
538         "FT_PCT": "67",
539         "MIN": "4",
540         "OREB": "5",
541         "PF": "27",
542         "PTS": "117",
543         "STL": "8",
544         "TOV": "9",
545         "TREB": "37"
546     }
547 },
548 "losses": "1",
549 "name": "Kings",
550 "next_game": {
551     "city": "New Orleans",
552     "day": "19",
553     "dayname": "Friday",
554     "is_home": "False",
555     "month": "October",
556     "opponent_name": "Pelicans",
557     "opponent_place": "New Orleans",
558     "stadium": "Smoothie King Center",
559     "year": "2018"
560 },
561 "next_game_id": "5700",
562 "place": "Sacramento",
563 "previous_game_id": "null",
564 "wins": "0"
565 },
566 "vis": {
567     "box_score": [
568         {
569             "+/-": "-9",
570             "AST": "0",
571             "BLK": "3",
572             "DOUBLE": "double",
573             "DREB": "12",
574             "FG3A": "0",
575             "FG3M": "0",
576             "FG3_PCT": "0",
577             "FGA": "9",
578             "FGM": "7",
579             "FG_PCT": "78",

```



```

580         "FTA": "5",
581         "FTM": "5",
582         "FT_PCT": "100",
583         "MIN": "37",
584         "OREB": "3",
585         "PF": "3",
586         "PTS": "19",
587         "STL": "0",
588         "TOV": "2",
589         "TREB": "15",
590         "first_name": "Rudy",
591         "last_name": "Gobert",
592         "name": "Rudy Gobert",
593         "starter": "True"
594     },
595     {
596         "+/-": "-12",
597         "AST": "2",
598         "BLK": "0",
599         "DOUBLE": "none",
600         "DREB": "3",
601         "FG3A": "10",
602         "FG3M": "3",
603         "FG3_PCT": "30",
604         "FGA": "21",
605         "FGM": "8",
606         "FG_PCT": "38",
607         "FTA": "6",
608         "FTM": "5",
609         "FT_PCT": "83",
610         "MIN": "36",
611         "OREB": "0",
612         "PF": "5",
613         "PTS": "24",
614         "STL": "2",
615         "TOV": "4",
616         "TREB": "3",
617         "first_name": "Donovan",
618         "last_name": "Mitchell",
619         "name": "Donovan Mitchell",
620         "starter": "True"
621     },
622     {
623         "+/-": "-7",
624         "AST": "6",
625         "BLK": "0",
626         "DOUBLE": "none",
627         "DREB": "1",
628         "FG3A": "6",
629         "FG3M": "4",
630         "FG3_PCT": "67",
631         "FGA": "12",
632         "FGM": "9",
633         "FG_PCT": "75",
634         "FTA": "0",
635         "FTM": "0",
636         "FT_PCT": "0",
637         "MIN": "35",
638         "OREB": "0",
639         "PF": "3",
640         "PTS": "22",
641         "STL": "4",
642         "TOV": "5",
643         "TREB": "1",
644         "first_name": "Joe",
645         "last_name": "Ingles",
646         "name": "Joe Ingles",
647         "starter": "True"
648     },
649     {
650         "+/-": "7",
651         "AST": "1",
652         "BLK": "1",
653         "DOUBLE": "none",
654         "DREB": "8",
655         "FG3A": "0",
656         "FG3M": "0",
657         "FG3_PCT": "0",
658         "FGA": "8",
659         "FGM": "7",
660         "FG_PCT": "88",
661         "FTA": "8",
662         "FTM": "4",
663         "FT_PCT": "50",
664         "MIN": "23",
665         "OREB": "1",
666         "PF": "1",
667         "PTS": "18",
668         "STL": "0",
669         "TOV": "2",
670         "TREB": "9",

```

```
671         "first_name": "Derrick",
672         "last_name": "Favors",
673         "name": "Derrick Favors",
674         "starter": "True"
675     },
676     {
677         "+/-": "-17",
678         "AST": "4",
679         "BLK": "0",
680         "DOUBLE": "none",
681         "DREB": "2",
682         "FG3A": "0",
683         "FG3M": "0",
684         "FG3_PCT": "0",
685         "FGA": "4",
686         "FGM": "0",
687         "FG_PCT": "0",
688         "FTA": "1",
689         "FTM": "1",
690         "FT_PCT": "100",
691         "MIN": "21",
692         "OREB": "0",
693         "PF": "3",
694         "PTS": "1",
695         "STL": "1",
696         "TOV": "2",
697         "TREB": "2",
698         "first_name": "Ricky",
699         "last_name": "Rubio",
700         "name": "Ricky Rubio",
701         "starter": "True"
702     },
703     {
704         "+/-": "17",
705         "AST": "0",
706         "BLK": "0",
707         "DOUBLE": "none",
708         "DREB": "5",
709         "FG3A": "4",
710         "FG3M": "2",
711         "FG3_PCT": "50",
712         "FGA": "5",
713         "FGM": "2",
714         "FG_PCT": "40",
715         "FTA": "9",
716         "FTM": "7",
717         "FT_PCT": "78",
718         "MIN": "33",
719         "OREB": "0",
720         "PF": "2",
721         "PTS": "13",
722         "STL": "0",
723         "TOV": "0",
724         "TREB": "5",
725         "first_name": "Jae",
726         "last_name": "Crowder",
727         "name": "Jae Crowder",
728         "starter": "False"
729     },
730     {
731         "+/-": "22",
732         "AST": "4",
733         "BLK": "0",
734         "DOUBLE": "none",
735         "DREB": "3",
736         "FG3A": "3",
737         "FG3M": "1",
738         "FG3_PCT": "33",
739         "FGA": "9",
740         "FGM": "4",
741         "FG_PCT": "44",
742         "FTA": "7",
743         "FTM": "4",
744         "FT_PCT": "57",
745         "MIN": "26",
746         "OREB": "1",
747         "PF": "1",
748         "PTS": "13",
749         "STL": "0",
750         "TOV": "1",
751         "TREB": "4",
752         "first_name": "Dante",
753         "last_name": "Exum",
754         "name": "Dante Exum",
755         "starter": "False"
756     },
757     {
758         "+/-": "25",
759         "AST": "4",
760         "BLK": "0",
761         "DOUBLE": "none",
```

```

762         "DREB": "3",
763         "FG3A": "3",
764         "FG3M": "3",
765         "FG3_PCT": "100",
766         "FGA": "7",
767         "FGM": "4",
768         "FG_PCT": "57",
769         "FTA": "2",
770         "FTM": "2",
771         "FT_PCT": "100",
772         "MIN": "17",
773         "OREB": "0",
774         "PF": "1",
775         "PTS": "13",
776         "STL": "1",
777         "TOV": "1",
778         "TREB": "3",
779         "first_name": "Alec",
780         "last_name": "Burks",
781         "name": "Alec Burks",
782         "starter": "False"
783     },
784     {
785         "+/-": "6",
786         "AST": "0",
787         "BLK": "0",
788         "DOUBLE": "none",
789         "DREB": "2",
790         "FG3A": "1",
791         "FG3M": "0",
792         "FG3_PCT": "0",
793         "FGA": "4",
794         "FGM": "0",
795         "FG_PCT": "0",
796         "FTA": "0",
797         "FTM": "0",
798         "FT_PCT": "0",
799         "MIN": "6",
800         "OREB": "0",
801         "PF": "0",
802         "PTS": "0",
803         "STL": "0",
804         "TOV": "0",
805         "TREB": "2",
806         "first_name": "Royce",
807         "last_name": "O'Neale",
808         "name": "Royce O'Neale",
809         "starter": "False"
810     },
811     {
812         "+/-": "-2",
813         "AST": "0",
814         "BLK": "0",
815         "DOUBLE": "none",
816         "DREB": "0",
817         "FG3A": "0",
818         "FG3M": "0",
819         "FG3_PCT": "0",
820         "FGA": "0",
821         "FGM": "0",
822         "FG_PCT": "0",
823         "FTA": "0",
824         "FTM": "0",
825         "FT_PCT": "0",
826         "MIN": "1",
827         "OREB": "0",
828         "PF": "0",
829         "PTS": "0",
830         "STL": "0",
831         "TOV": "0",
832         "TREB": "0",
833         "first_name": "Georges",
834         "last_name": "Niang",
835         "name": "Georges Niang",
836         "starter": "False"
837     },
838     {
839         "+/-": "0",
840         "AST": "0",
841         "BLK": "0",
842         "DOUBLE": "none",
843         "DREB": "0",
844         "FG3A": "0",
845         "FG3M": "0",
846         "FG3_PCT": "0",
847         "FGA": "0",
848         "FGM": "0",
849         "FG_PCT": "0",
850         "FTA": "0",
851         "FTM": "0",
852         "FT_PCT": "0",

```

```

      "MIN": "0",
      "OREB": "0",
      "PF": "0",
      "PTS": "0",
      "STL": "0",
      "TOV": "0",
      "TREB": "0",
      "first_name": "Ekpe",
      "last_name": "Udoh",
      "name": "Ekpe Udoh",
      "starter": "False"
    },
    {
      "+/-": "0",
      "AST": "0",
      "BLK": "0",
      "DOUBLE": "none",
      "DREB": "0",
      "FG3A": "0",
      "FG3M": "0",
      "FG3_PCT": "0",
      "FGA": "0",
      "FGM": "0",
      "FG_PCT": "0",
      "FTA": "0",
      "FTM": "0",
      "FT_PCT": "0",
      "MIN": "0",
      "OREB": "0",
      "PF": "0",
      "PTS": "0",
      "STL": "0",
      "TOV": "0",
      "TREB": "0",
      "first_name": "Thabo",
      "last_name": "Sefolosha",
      "name": "Thabo Sefolosha",
      "starter": "False"
    },
    {
      "+/-": "0",
      "AST": "0",
      "BLK": "0",
      "DOUBLE": "none",
      "DREB": "0",
      "FG3A": "0",
      "FG3M": "0",
      "FG3_PCT": "0",
      "FGA": "0",
      "FGM": "0",
      "FG_PCT": "0",
      "FTA": "0",
      "FTM": "0",
      "FT_PCT": "0",
      "MIN": "0",
      "OREB": "0",
      "PF": "0",
      "PTS": "0",
      "STL": "0",
      "TOV": "0",
      "TREB": "0",
      "first_name": "Grayson",
      "last_name": "Allen",
      "name": "Grayson Allen",
      "starter": "False"
    }
  ],
  "conference": "Western Conference",
  "conference_standing": 6,
  "division": "Northwest",
  "game_number": "1",
  "line_score": {
    "H1": {
      "AST": "55",
      "BLK": "21",
      "DREB": "914",
      "FG3A": "610",
      "FG3M": "45",
      "FG3_PCT": "7",
      "FGA": "1926",
      "FGM": "1013",
      "FG_PCT": "53",
      "FTA": "88",
      "FTM": "67",
      "FT_PCT": "76",
      "MIN": "6060",
      "OREB": "11",
      "PTS": "3038",
      "STL": "22",
      "TOV": "51",
      "TREB": "925"
    }
  }

```

```

},
"H2": {
  "AST": "56",
  "BLK": "01",
  "DREB": "79",
  "FG3A": "29",
  "FG3M": "13",
  "FG3_PCT": "45",
  "FGA": "1618",
  "FGM": "99",
  "FG_PCT": "6",
  "FTA": "1012",
  "FTM": "69",
  "FT_PCT": "7",
  "MIN": "6060",
  "OREB": "21",
  "PTS": "2530",
  "STL": "22",
  "TOV": "56",
  "TREB": "100"
},
"0T": {
  "AST": "0",
  "BLK": "0",
  "DREB": "0",
  "FG3A": "0",
  "FG3M": "0",
  "FG3_PCT": "0",
  "FGA": "0",
  "FGM": "0",
  "FG_PCT": "0",
  "FTA": "0",
  "FTM": "0",
  "FT_PCT": "0",
  "MIN": "0",
  "OREB": "0",
  "PTS": "0",
  "STL": "0",
  "TOV": "0",
  "TREB": "0"
},
"Q1": {
  "AST": "5",
  "BLK": "2",
  "DREB": "9",
  "FG3A": "6",
  "FG3M": "4",
  "FG3_PCT": "67",
  "FGA": "19",
  "FGM": "10",
  "FG_PCT": "53",
  "FTA": "8",
  "FTM": "6",
  "FT_PCT": "75",
  "MIN": "60",
  "OREB": "1",
  "PTS": "30",
  "STL": "2",
  "TOV": "5",
  "TREB": "10"
},
"Q2": {
  "AST": "5",
  "BLK": "1",
  "DREB": "14",
  "FG3A": "10",
  "FG3M": "5",
  "FG3_PCT": "50",
  "FGA": "26",
  "FGM": "13",
  "FG_PCT": "50",
  "FTA": "8",
  "FTM": "7",
  "FT_PCT": "88",
  "MIN": "60",
  "OREB": "1",
  "PTS": "38",
  "STL": "2",
  "TOV": "1",
  "TREB": "15"
},
"Q3": {
  "AST": "5",
  "BLK": "0",
  "DREB": "7",
  "FG3A": "2",
  "FG3M": "1",
  "FG3_PCT": "50",
  "FGA": "16",
  "FGM": "9",
  "FG_PCT": "56",

```

```

1035     "FTA": "10",
1036     "FTM": "6",
1037     "FT_PCT": "60",
1038     "MIN": "60",
1039     "OREB": "2",
1040     "PTS": "25",
1041     "STL": "2",
1042     "TOV": "5",
1043     "TREB": "9"
1044 },
1045 "Q4": {
1046     "AST": "6",
1047     "BLK": "1",
1048     "DREB": "9",
1049     "FG3A": "9",
1050     "FG3M": "3",
1051     "FG3_PCT": "33",
1052     "FGA": "18",
1053     "FGM": "9",
1054     "FG_PCT": "50",
1055     "FTA": "12",
1056     "FTM": "9",
1057     "FT_PCT": "75",
1058     "MIN": "60",
1059     "OREB": "1",
1060     "PTS": "30",
1061     "STL": "2",
1062     "TOV": "6",
1063     "TREB": "10"
1064 },
1065 "game": {
1066     "AST": "21",
1067     "BLK": "4",
1068     "DREB": "39",
1069     "FG3A": "27",
1070     "FG3M": "13",
1071     "FG3_PCT": "48",
1072     "FGA": "79",
1073     "FGM": "41",
1074     "FG_PCT": "52",
1075     "FTA": "38",
1076     "FTM": "28",
1077     "FT_PCT": "74",
1078     "MIN": "4",
1079     "OREB": "5",
1080     "PF": "19",
1081     "PTS": "123",
1082     "STL": "8",
1083     "TOV": "17",
1084     "TREB": "44"
1085 }
1086 },
1087 "losses": "0",
1088 "name": "Jazz",
1089 "next_game": {
1090     "city": "Salt Lake City",
1091     "day": "19",
1092     "dayname": "Friday",
1093     "is_home": "True",
1094     "month": "October",
1095     "opponent_name": "Warriors",
1096     "opponent_place": "Golden State",
1097     "stadium": "Vivint Smart Home Arena",
1098     "year": "2018"
1099 },
1100 "next_game_id": "5331",
1101 "place": "Utah",
1102 "previous_game_id": "null",
1103 "wins": "1"
1104 }
1105 }
1106 },
1107 "source_sha256": "d0793e0e6623f60983f0cc365fa6269099446399d87b82f687382512d9bde29a",
1108 "task_family": "event_report"
1109 }

```

Listing 77: D13: submission/program_listings/build/generated/data/sportsett_basketball__5786.json

```

1 {
2   "dataset_id": "sportsett_basketball",
3   "example_id": "5786",
4   "language": "en",
5   "output_mode": "multi_paragraph_report",
6   "parent_table": null,
7   "reference_sha256": "bb95fec76d8be49f9311a03c0fbf3396fdab22b489d16729eec104139375f72e",
8   "references": [
9     "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
10  ],
11  "request": "Write a coherent game report from the supplied structured game data. Lead with the result, select the

```

```

12     most important performances and contrasts, and do not invent information.",
13     "source_payload": {
14         "game": {
15             "attendance": "19800",
16             "capacity": "19800",
17             "city": "Toronto",
18             "day": "26",
19             "dayname": "Friday",
20             "game_id": "5786",
21             "month": "October",
22             "season": "2018",
23             "stadium": "Scotiabank Arena",
24             "state": "Ontario",
25             "year": "2018"
26         },
27         "teams": {
28             "home": {
29                 "box_score": [
30                     {
31                         "+/-": "11",
32                         "AST": "3",
33                         "BLK": "0",
34                         "DOUBLE": "none",
35                         "DREB": "6",
36                         "FG3A": "2",
37                         "FG3M": "0",
38                         "FG3_PCT": "0",
39                         "FGA": "8",
40                         "FGM": "4",
41                         "FG_PCT": "50",
42                         "FTA": "2",
43                         "FTM": "2",
44                         "FT_PCT": "100",
45                         "MIN": "37",
46                         "OREB": "1",
47                         "PF": "3",
48                         "PTS": "10",
49                         "STL": "1",
50                         "TOV": "3",
51                         "TREB": "7",
52                         "first_name": "Pascal",
53                         "last_name": "Siakam",
54                         "name": "Pascal Siakam",
55                         "starter": "True"
56                     },
57                     {
58                         "+/-": "6",
59                         "AST": "12",
60                         "BLK": "1",
61                         "DOUBLE": "double",
62                         "DREB": "4",
63                         "FG3A": "6",
64                         "FG3M": "3",
65                         "FG3_PCT": "50",
66                         "FGA": "14",
67                         "FGM": "8",
68                         "FG_PCT": "57",
69                         "FTA": "2",
70                         "FTM": "1",
71                         "FT_PCT": "50",
72                         "MIN": "35",
73                         "OREB": "0",
74                         "PF": "2",
75                         "PTS": "20",
76                         "STL": "0",
77                         "TOV": "0",
78                         "TREB": "4",
79                         "first_name": "Kyle",
80                         "last_name": "Lowry",
81                         "name": "Kyle Lowry",
82                         "starter": "True"
83                     },
84                     {
85                         "+/-": "5",
86                         "AST": "5",
87                         "BLK": "1",
88                         "DOUBLE": "none",
89                         "DREB": "8",
90                         "FG3A": "3",
91                         "FG3M": "1",
92                         "FG3_PCT": "33",
93                         "FGA": "16",
94                         "FGM": "7",
95                         "FG_PCT": "44",
96                         "FTA": "6",
97                         "FTM": "6",
98                         "FT_PCT": "100",
99                         "MIN": "34",
100                        "OREB": "1",
101                        "PF": "0",

```

```

102     "STL": "3",
103     "TOV": "0",
104     "TREB": "9",
105     "first_name": "Kawhi",
106     "last_name": "Leonard",
107     "name": "Kawhi Leonard",
108     "starter": "True"
109 },
110 {
111     "+/-": "4",
112     "AST": "0",
113     "BLK": "1",
114     "DOUBLE": "none",
115     "DREB": "6",
116     "FG3A": "7",
117     "FG3M": "4",
118     "FG3_PCT": "57",
119     "FGA": "8",
120     "FGM": "4",
121     "FG_PCT": "50",
122     "FTA": "3",
123     "FTM": "3",
124     "FT_PCT": "100",
125     "MIN": "30",
126     "OREB": "2",
127     "PF": "3",
128     "PTS": "15",
129     "STL": "1",
130     "TOV": "0",
131     "TREB": "8",
132     "first_name": "Danny",
133     "last_name": "Green",
134     "name": "Danny Green",
135     "starter": "True"
136 },
137 {
138     "+/-": "12",
139     "AST": "1",
140     "BLK": "0",
141     "DOUBLE": "none",
142     "DREB": "3",
143     "FG3A": "1",
144     "FG3M": "0",
145     "FG3_PCT": "0",
146     "FGA": "16",
147     "FGM": "7",
148     "FG_PCT": "44",
149     "FTA": "4",
150     "FTM": "3",
151     "FT_PCT": "75",
152     "MIN": "22",
153     "OREB": "5",
154     "PF": "4",
155     "PTS": "17",
156     "STL": "1",
157     "TOV": "1",
158     "TREB": "8",
159     "first_name": "Jonas",
160     "last_name": "ČūValaninas",
161     "name": "Jonas čūValaninas",
162     "starter": "True"
163 },
164 {
165     "+/-": "-3",
166     "AST": "1",
167     "BLK": "2",
168     "DOUBLE": "none",
169     "DREB": "6",
170     "FG3A": "3",
171     "FG3M": "0",
172     "FG3_PCT": "0",
173     "FGA": "13",
174     "FGM": "5",
175     "FG_PCT": "38",
176     "FTA": "1",
177     "FTM": "1",
178     "FT_PCT": "100",
179     "MIN": "25",
180     "OREB": "2",
181     "PF": "2",
182     "PTS": "11",
183     "STL": "0",
184     "TOV": "5",
185     "TREB": "8",
186     "first_name": "Serge",
187     "last_name": "Ibaka",
188     "name": "Serge Ibaka",
189     "starter": "False"
190 },
191 {
192     "+/-": "8",

```



```
193     "AST": "0",
194     "BLK": "0",
195     "DOUBLE": "none",
196     "DREB": "2",
197     "FG3A": "0",
198     "FG3M": "0",
199     "FG3_PCT": "0",
200     "FGA": "1",
201     "FGM": "1",
202     "FG_PCT": "100",
203     "FTA": "0",
204     "FTM": "0",
205     "FT_PCT": "0",
206     "MIN": "19",
207     "OREB": "0",
208     "PF": "1",
209     "PTS": "2",
210     "STL": "1",
211     "TOV": "1",
212     "TREB": "2",
213     "first_name": "Norman",
214     "last_name": "Powell",
215     "name": "Norman Powell",
216     "starter": "False"
217 },
218 {
219     "+/-": "5",
220     "AST": "1",
221     "BLK": "0",
222     "DOUBLE": "none",
223     "DREB": "1",
224     "FG3A": "2",
225     "FG3M": "1",
226     "FG3_PCT": "50",
227     "FGA": "8",
228     "FGM": "4",
229     "FG_PCT": "50",
230     "FTA": "0",
231     "FTM": "0",
232     "FT_PCT": "0",
233     "MIN": "15",
234     "OREB": "0",
235     "PF": "2",
236     "PTS": "9",
237     "STL": "2",
238     "TOV": "1",
239     "TREB": "1",
240     "first_name": "Lorenzo",
241     "last_name": "Brown",
242     "name": "Lorenzo Brown",
243     "starter": "False"
244 },
245 {
246     "+/-": "-5",
247     "AST": "0",
248     "BLK": "0",
249     "DOUBLE": "none",
250     "DREB": "2",
251     "FG3A": "2",
252     "FG3M": "2",
253     "FG3_PCT": "100",
254     "FGA": "6",
255     "FGM": "4",
256     "FG_PCT": "67",
257     "FTA": "0",
258     "FTM": "0",
259     "FT_PCT": "0",
260     "MIN": "11",
261     "OREB": "0",
262     "PF": "4",
263     "PTS": "10",
264     "STL": "1",
265     "TOV": "1",
266     "TREB": "2",
267     "first_name": "C.J.",
268     "last_name": "Miles",
269     "name": "C.J. Miles",
270     "starter": "False"
271 },
272 {
273     "+/-": "2",
274     "AST": "0",
275     "BLK": "0",
276     "DOUBLE": "none",
277     "DREB": "0",
278     "FG3A": "1",
279     "FG3M": "0",
280     "FG3_PCT": "0",
281     "FGA": "1",
282     "FGM": "0",
283     "FG_PCT": "0",
```

```

    "FTA": "2",
    "FTM": "1",
    "FT_PCT": "50",
    "MIN": "8",
    "OREB": "1",
    "PF": "1",
    "PTS": "1",
    "STL": "0",
    "TOV": "0",
    "TREB": "1",
    "first_name": "Malachi",
    "last_name": "Richardson",
    "name": "Malachi Richardson",
    "starter": "False"
  },
  {
    "+/-": "0",
    "AST": "0",
    "BLK": "0",
    "DOUBLE": "none",
    "DREB": "0",
    "FG3A": "0",
    "FG3M": "0",
    "FG3_PCT": "0",
    "FGA": "0",
    "FGM": "0",
    "FG_PCT": "0",
    "FTA": "0",
    "FTM": "0",
    "FT_PCT": "0",
    "MIN": "0",
    "OREB": "0",
    "PF": "0",
    "PTS": "0",
    "STL": "0",
    "TOV": "0",
    "TREB": "0",
    "first_name": "Greg",
    "last_name": "Monroe",
    "name": "Greg Monroe",
    "starter": "False"
  }
],
"conference": "Eastern Conference",
"conference_standing": 1,
"division": "Atlantic",
"game_number": "6",
"line_score": {
  "H1": {
    "AST": "106",
    "BLK": "21",
    "DREB": "127",
    "FG3A": "116",
    "FG3M": "44",
    "FG3_PCT": "38",
    "FGA": "2423",
    "FGM": "1312",
    "FG_PCT": "54",
    "FTA": "102",
    "FTM": "92",
    "FT_PCT": "90",
    "MIN": "6060",
    "OREB": "42",
    "PTS": "3930",
    "STL": "22",
    "TOV": "23",
    "TREB": "169"
  },
  "H2": {
    "AST": "25",
    "BLK": "11",
    "DREB": "811",
    "FG3A": "37",
    "FG3M": "21",
    "FG3_PCT": "57",
    "FGA": "1925",
    "FGM": "910",
    "FG_PCT": "47",
    "FTA": "44",
    "FTM": "33",
    "FT_PCT": "75",
    "MIN": "6060",
    "OREB": "15",
    "PTS": "2324",
    "STL": "06",
    "TOV": "43",
    "TREB": "826"
  },
  "OT": {
    "AST": "0",
    "BLK": "0",

```

```
375     "DREB": "0",
376     "FG3A": "0",
377     "FG3M": "0",
378     "FG3_PCT": "0",
379     "FGA": "0",
380     "FGM": "0",
381     "FG_PCT": "0",
382     "FTA": "0",
383     "FTM": "0",
384     "FT_PCT": "0",
385     "MIN": "0",
386     "OREB": "0",
387     "PTS": "0",
388     "STL": "0",
389     "TOV": "0",
390     "TREB": "0"
391 },
392 "Q1": {
393     "AST": "10",
394     "BLK": "2",
395     "DREB": "12",
396     "FG3A": "11",
397     "FG3M": "4",
398     "FG3_PCT": "36",
399     "FGA": "24",
400     "FGM": "13",
401     "FG_PCT": "54",
402     "FTA": "10",
403     "FTM": "9",
404     "FT_PCT": "90",
405     "MIN": "60",
406     "OREB": "4",
407     "PTS": "39",
408     "STL": "2",
409     "TOV": "2",
410     "TREB": "16"
411 },
412 "Q2": {
413     "AST": "6",
414     "BLK": "1",
415     "DREB": "7",
416     "FG3A": "6",
417     "FG3M": "4",
418     "FG3_PCT": "67",
419     "FGA": "23",
420     "FGM": "12",
421     "FG_PCT": "52",
422     "FTA": "2",
423     "FTM": "2",
424     "FT_PCT": "100",
425     "MIN": "60",
426     "OREB": "2",
427     "PTS": "30",
428     "STL": "2",
429     "TOV": "3",
430     "TREB": "9"
431 },
432 "Q3": {
433     "AST": "2",
434     "BLK": "1",
435     "DREB": "8",
436     "FG3A": "3",
437     "FG3M": "2",
438     "FG3_PCT": "67",
439     "FGA": "19",
440     "FGM": "9",
441     "FG_PCT": "47",
442     "FTA": "4",
443     "FTM": "3",
444     "FT_PCT": "75",
445     "MIN": "60",
446     "OREB": "1",
447     "PTS": "23",
448     "STL": "0",
449     "TOV": "4",
450     "TREB": "9"
451 },
452 "Q4": {
453     "AST": "5",
454     "BLK": "1",
455     "DREB": "11",
456     "FG3A": "7",
457     "FG3M": "1",
458     "FG3_PCT": "14",
459     "FGA": "25",
460     "FGM": "10",
461     "FG_PCT": "40",
462     "FTA": "4",
463     "FTM": "3",
464     "FT_PCT": "75",
465     "MIN": "60",
```

```

466         "OREB": "5",
467         "PTS": "24",
468         "STL": "6",
469         "TOV": "3",
470         "TREB": "16"
471     },
472     "game": {
473         "AST": "23",
474         "BLK": "5",
475         "DREB": "38",
476         "FG3A": "27",
477         "FG3M": "11",
478         "FG3_PCT": "41",
479         "FGA": "91",
480         "FGM": "44",
481         "FG_PCT": "48",
482         "FTA": "20",
483         "FTM": "17",
484         "FT_PCT": "85",
485         "MIN": "4",
486         "OREB": "12",
487         "PF": "22",
488         "PTS": "116",
489         "STL": "10",
490         "TOV": "12",
491         "TREB": "50"
492     }
493 },
494 "losses": "0",
495 "name": "Raptors",
496 "next_game": {
497     "city": "Milwaukee",
498     "day": "29",
499     "dayname": "Monday",
500     "is_home": "False",
501     "month": "October",
502     "opponent_name": "Bucks",
503     "opponent_place": "Milwaukee",
504     "stadium": "Fiserv Forum",
505     "year": "2018"
506 },
507 "next_game_id": "4966",
508 "place": "Toronto",
509 "previous_game_id": "5785",
510 "wins": "6"
511 },
512 "vis": {
513     "box_score": [
514         {
515             "+/-": "0",
516             "AST": "4",
517             "BLK": "0",
518             "DOUBLE": "none",
519             "DREB": "4",
520             "FG3A": "6",
521             "FG3M": "4",
522             "FG3_PCT": "67",
523             "FGA": "14",
524             "FGM": "7",
525             "FG_PCT": "50",
526             "FTA": "4",
527             "FTM": "4",
528             "FT_PCT": "100",
529             "MIN": "35",
530             "OREB": "1",
531             "PF": "1",
532             "PTS": "22",
533             "STL": "0",
534             "TOV": "3",
535             "TREB": "5",
536             "first_name": "Luka",
537             "last_name": "čćDoni",
538             "name": "Luka čćDoni",
539             "starter": "True"
540         },
541         {
542             "+/-": "0",
543             "AST": "5",
544             "BLK": "1",
545             "DOUBLE": "double",
546             "DREB": "10",
547             "FG3A": "0",
548             "FG3M": "0",
549             "FG3_PCT": "0",
550             "FGA": "10",
551             "FGM": "5",
552             "FG_PCT": "50",
553             "FTA": "9",
554             "FTM": "8",
555             "FT_PCT": "89",
556             "MIN": "33",

```

```

557         "OREB": "5",
558         "PF": "1",
559         "PTS": "18",
560         "STL": "0",
561         "TOV": "2",
562         "TREB": "15",
563         "first_name": "DeAndre",
564         "last_name": "Jordan",
565         "name": "DeAndre Jordan",
566         "starter": "True"
567     },
568     {
569         "+/-": "-8",
570         "AST": "1",
571         "BLK": "0",
572         "DOUBLE": "none",
573         "DREB": "0",
574         "FG3A": "8",
575         "FG3M": "3",
576         "FG3_PCT": "38",
577         "FGA": "15",
578         "FGM": "9",
579         "FG_PCT": "60",
580         "FTA": "0",
581         "FTM": "0",
582         "FT_PCT": "0",
583         "MIN": "32",
584         "OREB": "0",
585         "PF": "4",
586         "PTS": "21",
587         "STL": "0",
588         "TOV": "2",
589         "TREB": "0",
590         "first_name": "Wesley",
591         "last_name": "Matthews",
592         "name": "Wesley Matthews",
593         "starter": "True"
594     },
595     {
596         "+/-": "-3",
597         "AST": "4",
598         "BLK": "0",
599         "DOUBLE": "none",
600         "DREB": "3",
601         "FG3A": "4",
602         "FG3M": "2",
603         "FG3_PCT": "50",
604         "FGA": "11",
605         "FGM": "3",
606         "FG_PCT": "27",
607         "FTA": "0",
608         "FTM": "0",
609         "FT_PCT": "0",
610         "MIN": "29",
611         "OREB": "0",
612         "PF": "2",
613         "PTS": "8",
614         "STL": "1",
615         "TOV": "2",
616         "TREB": "3",
617         "first_name": "Jalen",
618         "last_name": "Brunson",
619         "name": "Jalen Brunson",
620         "starter": "True"
621     },
622     {
623         "+/-": "-24",
624         "AST": "3",
625         "BLK": "1",
626         "DOUBLE": "none",
627         "DREB": "5",
628         "FG3A": "5",
629         "FG3M": "1",
630         "FG3_PCT": "20",
631         "FGA": "17",
632         "FGM": "5",
633         "FG_PCT": "29",
634         "FTA": "5",
635         "FTM": "3",
636         "FT_PCT": "60",
637         "MIN": "28",
638         "OREB": "1",
639         "PF": "2",
640         "PTS": "14",
641         "STL": "0",
642         "TOV": "1",
643         "TREB": "6",
644         "first_name": "Harrison",
645         "last_name": "Barnes",
646         "name": "Harrison Barnes",
647         "starter": "True"

```

```
648 },
649 {
650   "+/-": "-1",
651   "AST": "1",
652   "BLK": "4",
653   "DOUBLE": "none",
654   "DREB": "7",
655   "FG3A": "4",
656   "FG3M": "1",
657   "FG3_PCT": "25",
658   "FGA": "6",
659   "FGM": "2",
660   "FG_PCT": "33",
661   "FTA": "4",
662   "FTM": "3",
663   "FT_PCT": "75",
664   "MIN": "30",
665   "OREB": "1",
666   "PF": "0",
667   "PTS": "8",
668   "STL": "2",
669   "TOV": "0",
670   "TREB": "8",
671   "first_name": "Maxi",
672   "last_name": "Kleber",
673   "name": "Maxi Kleber",
674   "starter": "False"
675 },
676 {
677   "+/-": "2",
678   "AST": "1",
679   "BLK": "0",
680   "DOUBLE": "none",
681   "DREB": "2",
682   "FG3A": "2",
683   "FG3M": "1",
684   "FG3_PCT": "50",
685   "FGA": "6",
686   "FGM": "2",
687   "FG_PCT": "33",
688   "FTA": "0",
689   "FTM": "0",
690   "FT_PCT": "0",
691   "MIN": "19",
692   "OREB": "1",
693   "PF": "4",
694   "PTS": "5",
695   "STL": "1",
696   "TOV": "0",
697   "TREB": "3",
698   "first_name": "Dorian",
699   "last_name": "Finney-Smith",
700   "name": "Dorian Finney-Smith",
701   "starter": "False"
702 },
703 {
704   "+/-": "-5",
705   "AST": "3",
706   "BLK": "0",
707   "DOUBLE": "none",
708   "DREB": "1",
709   "FG3A": "1",
710   "FG3M": "0",
711   "FG3_PCT": "0",
712   "FGA": "11",
713   "FGM": "3",
714   "FG_PCT": "27",
715   "FTA": "2",
716   "FTM": "1",
717   "FT_PCT": "50",
718   "MIN": "15",
719   "OREB": "1",
720   "PF": "0",
721   "PTS": "7",
722   "STL": "0",
723   "TOV": "2",
724   "TREB": "2",
725   "first_name": "J.J.",
726   "last_name": "Barea",
727   "name": "J.J. Barea",
728   "starter": "False"
729 },
730 {
731   "+/-": "-9",
732   "AST": "0",
733   "BLK": "0",
734   "DOUBLE": "none",
735   "DREB": "2",
736   "FG3A": "0",
737   "FG3M": "0",
738   "FG3_PCT": "0",
```

```

739         "FGA": "2",
740         "FGM": "2",
741         "FG_PCT": "100",
742         "FTA": "0",
743         "FTM": "0",
744         "FT_PCT": "0",
745         "MIN": "14",
746         "OREB": "0",
747         "PF": "3",
748         "PTS": "4",
749         "STL": "2",
750         "TOV": "2",
751         "TREB": "2",
752         "first_name": "Dwight",
753         "last_name": "Powell",
754         "name": "Dwight Powell",
755         "starter": "False"
756     },
757     {
758         "+/-": "3",
759         "AST": "0",
760         "BLK": "0",
761         "DOUBLE": "none",
762         "DREB": "0",
763         "FG3A": "0",
764         "FG3M": "0",
765         "FG3_PCT": "0",
766         "FGA": "0",
767         "FGM": "0",
768         "FG_PCT": "0",
769         "FTA": "0",
770         "FTM": "0",
771         "FT_PCT": "0",
772         "MIN": "0",
773         "OREB": "0",
774         "PF": "0",
775         "PTS": "0",
776         "STL": "0",
777         "TOV": "0",
778         "TREB": "0",
779         "first_name": "Daryl",
780         "last_name": "Macon",
781         "name": "Daryl Macon",
782         "starter": "False"
783     },
784     {
785         "+/-": "0",
786         "AST": "0",
787         "BLK": "0",
788         "DOUBLE": "none",
789         "DREB": "0",
790         "FG3A": "0",
791         "FG3M": "0",
792         "FG3_PCT": "0",
793         "FGA": "0",
794         "FGM": "0",
795         "FG_PCT": "0",
796         "FTA": "0",
797         "FTM": "0",
798         "FT_PCT": "0",
799         "MIN": "0",
800         "OREB": "0",
801         "PF": "0",
802         "PTS": "0",
803         "STL": "0",
804         "TOV": "0",
805         "TREB": "0",
806         "first_name": "Dennis",
807         "last_name": "Smith",
808         "name": "Dennis Smith",
809         "starter": "False"
810     },
811     {
812         "+/-": "0",
813         "AST": "0",
814         "BLK": "0",
815         "DOUBLE": "none",
816         "DREB": "0",
817         "FG3A": "0",
818         "FG3M": "0",
819         "FG3_PCT": "0",
820         "FGA": "0",
821         "FGM": "0",
822         "FG_PCT": "0",
823         "FTA": "0",
824         "FTM": "0",
825         "FT_PCT": "0",
826         "MIN": "0",
827         "OREB": "0",
828         "PF": "0",
829         "PTS": "0",

```

```

830     "STL": "0",
831     "TOV": "0",
832     "TREB": "0",
833     "first_name": "Salah",
834     "last_name": "Mejri",
835     "name": "Salah Mejri",
836     "starter": "False"
837 },
838 {
839     "+/-": "0",
840     "AST": "0",
841     "BLK": "0",
842     "DOUBLE": "none",
843     "DREB": "0",
844     "FG3A": "0",
845     "FG3M": "0",
846     "FG3_PCT": "0",
847     "FGA": "0",
848     "FGM": "0",
849     "FG_PCT": "0",
850     "FTA": "0",
851     "FTM": "0",
852     "FT_PCT": "0",
853     "MIN": "0",
854     "OREB": "0",
855     "PF": "0",
856     "PTS": "0",
857     "STL": "0",
858     "TOV": "0",
859     "TREB": "0",
860     "first_name": "Ryan",
861     "last_name": "Broekhoff",
862     "name": "Ryan Broekhoff",
863     "starter": "False"
864 }
865 ],
866 "conference": "Western Conference",
867 "conference_standing": 10,
868 "division": "Southwest",
869 "game_number": "5",
870 "line_score": {
871     "H1": {
872         "AST": "39",
873         "BLK": "03",
874         "DREB": "88",
875         "FG3A": "97",
876         "FG3M": "24",
877         "FG3_PCT": "25",
878         "FGA": "2323",
879         "FGM": "913",
880         "FG_PCT": "39",
881         "FTA": "74",
882         "FTM": "64",
883         "FT_PCT": "86",
884         "MIN": "6060",
885         "OREB": "02",
886         "PTS": "2634",
887         "STL": "12",
888         "TOV": "23",
889         "TREB": "90"
890     },
891     "H2": {
892         "AST": "55",
893         "BLK": "21",
894         "DREB": "99",
895         "FG3A": "86",
896         "FG3M": "42",
897         "FG3_PCT": "49",
898         "FGA": "2224",
899         "FGM": "97",
900         "FG_PCT": "4",
901         "FTA": "103",
902         "FTM": "72",
903         "FT_PCT": "70",
904         "MIN": "6060",
905         "OREB": "35",
906         "PTS": "2918",
907         "STL": "12",
908         "TOV": "27",
909         "TREB": "134"
910     },
911     "OT": {
912         "AST": "0",
913         "BLK": "0",
914         "DREB": "0",
915         "FG3A": "0",
916         "FG3M": "0",
917         "FG3_PCT": "0",
918         "FGA": "0",
919         "FGM": "0",
920         "FG_PCT": "0",

```



```

921     "FTA": "0",
922     "FTM": "0",
923     "FT_PCT": "0",
924     "MIN": "0",
925     "OREB": "0",
926     "PTS": "0",
927     "STL": "0",
928     "TOV": "0",
929     "TREB": "0"
930 },
931 "Q1": {
932     "AST": "3",
933     "BLK": "0",
934     "DREB": "8",
935     "FG3A": "9",
936     "FG3M": "2",
937     "FG3_PCT": "22",
938     "FGA": "23",
939     "FGM": "9",
940     "FG_PCT": "39",
941     "FTA": "7",
942     "FTM": "6",
943     "FT_PCT": "86",
944     "MIN": "60",
945     "OREB": "0",
946     "PTS": "26",
947     "STL": "1",
948     "TOV": "2",
949     "TREB": "8"
950 },
951 "Q2": {
952     "AST": "9",
953     "BLK": "3",
954     "DREB": "8",
955     "FG3A": "7",
956     "FG3M": "4",
957     "FG3_PCT": "57",
958     "FGA": "23",
959     "FGM": "13",
960     "FG_PCT": "57",
961     "FTA": "4",
962     "FTM": "4",
963     "FT_PCT": "100",
964     "MIN": "60",
965     "OREB": "2",
966     "PTS": "34",
967     "STL": "2",
968     "TOV": "3",
969     "TREB": "10"
970 },
971 "Q3": {
972     "AST": "5",
973     "BLK": "2",
974     "DREB": "9",
975     "FG3A": "8",
976     "FG3M": "4",
977     "FG3_PCT": "50",
978     "FGA": "22",
979     "FGM": "9",
980     "FG_PCT": "41",
981     "FTA": "10",
982     "FTM": "7",
983     "FT_PCT": "70",
984     "MIN": "60",
985     "OREB": "3",
986     "PTS": "29",
987     "STL": "1",
988     "TOV": "2",
989     "TREB": "12"
990 },
991 "Q4": {
992     "AST": "5",
993     "BLK": "1",
994     "DREB": "9",
995     "FG3A": "6",
996     "FG3M": "2",
997     "FG3_PCT": "33",
998     "FGA": "24",
999     "FGM": "7",
1000     "FG_PCT": "29",
1001     "FTA": "3",
1002     "FTM": "2",
1003     "FT_PCT": "67",
1004     "MIN": "60",
1005     "OREB": "5",
1006     "PTS": "18",
1007     "STL": "2",
1008     "TOV": "7",
1009     "TREB": "14"
1010 },
1011 "game": {

```

```

1012         "AST": "22",
1013         "BLK": "6",
1014         "DREB": "34",
1015         "FG3A": "30",
1016         "FG3M": "12",
1017         "FG3_PCT": "40",
1018         "FGA": "92",
1019         "FGM": "38",
1020         "FG_PCT": "41",
1021         "FTA": "24",
1022         "FTM": "19",
1023         "FT_PCT": "79",
1024         "MIN": "4",
1025         "OREB": "10",
1026         "PF": "17",
1027         "PTS": "107",
1028         "STL": "6",
1029         "TOV": "14",
1030         "TREB": "44"
1031     }
1032 },
1033 "losses": "3",
1034 "name": "Mavericks",
1035 "next_game": {
1036     "city": "Dallas",
1037     "day": "28",
1038     "dayname": "Sunday",
1039     "is_home": "True",
1040     "month": "October",
1041     "opponent_name": "Jazz",
1042     "opponent_place": "Utah",
1043     "stadium": "American Airlines Center",
1044     "year": "2018"
1045 },
1046 "next_game_id": "5538",
1047 "place": "Dallas",
1048 "previous_game_id": "5208",
1049 "wins": "2"
1050 }
1051 }
1052 },
1053 "source_sha256": "2d380f4702e071a20f97f90c6d2771c5357144f5f890c974fe06cab042aaf70d",
1054 "task_family": "event_report"
1055 }

```

Listing 78: D14: submission/program_listings/build/generated/data/sportsett_basketball__5955.json

```

1  {
2  "dataset_id": "sportsett_basketball",
3  "example_id": "5955",
4  "language": "en",
5  "output_mode": "multi_paragraph_report",
6  "parent_table": null,
7  "reference_sha256": "833109f0de34b3c6c88341b8443b6db76e62efa6cd0eacea3e3a9b99a3995d5f",
8  "references": [
9      "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
10 ],
11 "request": "Write a coherent game report from the supplied structured game data. Lead with the result, select the
    most important performances and contrasts, and do not invent information.",
12 "source_payload": {
13     "game": {
14         "attendance": "18200",
15         "capacity": "18200",
16         "city": "Oklahoma City",
17         "day": "24",
18         "dayname": "Saturday",
19         "game_id": "5955",
20         "month": "November",
21         "season": "2018",
22         "stadium": "Chesapeake Energy Arena",
23         "state": "Oklahoma",
24         "year": "2018"
25     },
26     "teams": {
27         "home": {
28             "box_score": [
29                 {
30                     "+/-": "-1",
31                     "AST": "3",
32                     "BLK": "3",
33                     "DOUBLE": "double",
34                     "DREB": "7",
35                     "FG3A": "6",
36                     "FG3M": "3",
37                     "FG3_PCT": "50",
38                     "FGA": "21",
39                     "FGM": "8",
40                     "FG_PCT": "38",
41                     "FTA": "7",

```

```

    "FTM": "5",
    "FT_PCT": "71",
    "MIN": "39",
    "OREB": "4",
    "PF": "5",
    "PTS": "24",
    "STL": "2",
    "TOV": "1",
    "TREB": "11",
    "first_name": "Paul",
    "last_name": "George",
    "name": "Paul George",
    "starter": "True"
  },
  {
    "+/-": "-3",
    "AST": "12",
    "BLK": "0",
    "DOUBLE": "triple",
    "DREB": "7",
    "FG3A": "12",
    "FG3M": "1",
    "FG3_PCT": "8",
    "FGA": "23",
    "FGM": "6",
    "FG_PCT": "26",
    "FTA": "7",
    "FTM": "3",
    "FT_PCT": "43",
    "MIN": "39",
    "OREB": "3",
    "PF": "5",
    "PTS": "16",
    "STL": "1",
    "TOV": "4",
    "TREB": "10",
    "first_name": "Russell",
    "last_name": "Westbrook",
    "name": "Russell Westbrook",
    "starter": "True"
  },
  {
    "+/-": "-5",
    "AST": "0",
    "BLK": "0",
    "DOUBLE": "double",
    "DREB": "7",
    "FG3A": "0",
    "FG3M": "0",
    "FG3_PCT": "0",
    "FGA": "15",
    "FGM": "6",
    "FG_PCT": "40",
    "FTA": "1",
    "FTM": "0",
    "FT_PCT": "0",
    "MIN": "37",
    "OREB": "7",
    "PF": "3",
    "PTS": "12",
    "STL": "1",
    "TOV": "2",
    "TREB": "14",
    "first_name": "Steven",
    "last_name": "Adams",
    "name": "Steven Adams",
    "starter": "True"
  },
  {
    "+/-": "2",
    "AST": "2",
    "BLK": "1",
    "DOUBLE": "none",
    "DREB": "2",
    "FG3A": "5",
    "FG3M": "2",
    "FG3_PCT": "40",
    "FGA": "11",
    "FGM": "5",
    "FG_PCT": "45",
    "FTA": "2",
    "FTM": "1",
    "FT_PCT": "50",
    "MIN": "31",
    "OREB": "0",
    "PF": "5",
    "PTS": "13",
    "STL": "0",
    "TOV": "0",
    "TREB": "2",
    "first_name": "Jerami",

```

```

133     "last_name": "Grant",
134     "name": "Jerami Grant",
135     "starter": "True"
136 },
137 {
138     "+/-": "-10",
139     "AST": "1",
140     "BLK": "1",
141     "DOUBLE": "none",
142     "DREB": "4",
143     "FG3A": "4",
144     "FG3M": "1",
145     "FG3_PCT": "25",
146     "FGA": "6",
147     "FGM": "1",
148     "FG_PCT": "17",
149     "FTA": "0",
150     "FTM": "0",
151     "FT_PCT": "0",
152     "MIN": "20",
153     "OREB": "1",
154     "PF": "1",
155     "PTS": "3",
156     "STL": "1",
157     "TOV": "1",
158     "TREB": "5",
159     "first_name": "Timoth  ",
160     "last_name": "Luwawu-Cabarrot",
161     "name": "Timoth   Luwawu-Cabarrot",
162     "starter": "True"
163 },
164 {
165     "+/-": "4",
166     "AST": "2",
167     "BLK": "0",
168     "DOUBLE": "none",
169     "DREB": "2",
170     "FG3A": "7",
171     "FG3M": "2",
172     "FG3_PCT": "29",
173     "FGA": "14",
174     "FGM": "5",
175     "FG_PCT": "36",
176     "FTA": "6",
177     "FTM": "6",
178     "FT_PCT": "100",
179     "MIN": "29",
180     "OREB": "0",
181     "PF": "0",
182     "PTS": "18",
183     "STL": "1",
184     "TOV": "2",
185     "TREB": "2",
186     "first_name": "Dennis",
187     "last_name": "Schr  der",
188     "name": "Dennis Schr  der",
189     "starter": "False"
190 },
191 {
192     "+/-": "-1",
193     "AST": "0",
194     "BLK": "0",
195     "DOUBLE": "none",
196     "DREB": "2",
197     "FG3A": "1",
198     "FG3M": "0",
199     "FG3_PCT": "0",
200     "FGA": "6",
201     "FGM": "4",
202     "FG_PCT": "67",
203     "FTA": "0",
204     "FTM": "0",
205     "FT_PCT": "0",
206     "MIN": "13",
207     "OREB": "0",
208     "PF": "2",
209     "PTS": "8",
210     "STL": "1",
211     "TOV": "0",
212     "TREB": "2",
213     "first_name": "Deonte",
214     "last_name": "Burton",
215     "name": "Deonte Burton",
216     "starter": "False"
217 },
218 {
219     "+/-": "-2",
220     "AST": "1",
221     "BLK": "0",
222     "DOUBLE": "none",
223     "DREB": "2",

```

```

224         "FG3A": "0",
225         "FG3M": "0",
226         "FG3_PCT": "0",
227         "FGA": "2",
228         "FGM": "2",
229         "FG_PCT": "100",
230         "FTA": "0",
231         "FTM": "0",
232         "FT_PCT": "0",
233         "MIN": "10",
234         "OREB": "3",
235         "PF": "2",
236         "PTS": "4",
237         "STL": "1",
238         "TOV": "0",
239         "TREB": "5",
240         "first_name": "Nerlens",
241         "last_name": "Noel",
242         "name": "Nerlens Noel",
243         "starter": "False"
244     },
245     {
246         "+/-": "-10",
247         "AST": "0",
248         "BLK": "0",
249         "DOUBLE": "none",
250         "DREB": "1",
251         "FG3A": "2",
252         "FG3M": "0",
253         "FG3_PCT": "0",
254         "FGA": "3",
255         "FGM": "0",
256         "FG_PCT": "0",
257         "FTA": "0",
258         "FTM": "0",
259         "FT_PCT": "0",
260         "MIN": "9",
261         "OREB": "1",
262         "PF": "0",
263         "PTS": "0",
264         "STL": "1",
265         "TOV": "0",
266         "TREB": "2",
267         "first_name": "Patrick",
268         "last_name": "Patterson",
269         "name": "Patrick Patterson",
270         "starter": "False"
271     },
272     {
273         "+/-": "-9",
274         "AST": "1",
275         "BLK": "0",
276         "DOUBLE": "none",
277         "DREB": "1",
278         "FG3A": "2",
279         "FG3M": "0",
280         "FG3_PCT": "0",
281         "FGA": "2",
282         "FGM": "0",
283         "FG_PCT": "0",
284         "FTA": "0",
285         "FTM": "0",
286         "FT_PCT": "0",
287         "MIN": "7",
288         "OREB": "0",
289         "PF": "1",
290         "PTS": "0",
291         "STL": "0",
292         "TOV": "0",
293         "TREB": "1",
294         "first_name": "Álex",
295         "last_name": "Abrines",
296         "name": "Álex Abrines",
297         "starter": "False"
298     },
299     {
300         "+/-": "0",
301         "AST": "0",
302         "BLK": "0",
303         "DOUBLE": "none",
304         "DREB": "0",
305         "FG3A": "0",
306         "FG3M": "0",
307         "FG3_PCT": "0",
308         "FGA": "0",
309         "FGM": "0",
310         "FG_PCT": "0",
311         "FTA": "0",
312         "FTM": "0",
313         "FT_PCT": "0",
314         "MIN": "0",

```

```
"OREB": "0",
"PF": "0",
"PTS": "0",
"STL": "0",
"TOV": "0",
"TREB": "0",
"first_name": "Raymond",
"last_name": "Felton",
"name": "Raymond Felton",
"starter": "False"
},
{
  "+/-": "0",
  "AST": "0",
  "BLK": "0",
  "DOUBLE": "none",
  "DREB": "0",
  "FG3A": "0",
  "FG3M": "0",
  "FG3_PCT": "0",
  "FGA": "0",
  "FGM": "0",
  "FG_PCT": "0",
  "FTA": "0",
  "FTM": "0",
  "FT_PCT": "0",
  "MIN": "0",
  "OREB": "0",
  "PF": "0",
  "PTS": "0",
  "STL": "0",
  "TOV": "0",
  "TREB": "0",
  "first_name": "Abdel",
  "last_name": "Nader",
  "name": "Abdel Nader",
  "starter": "False"
}
],
"conference": "Western Conference",
"conference_standing": 5,
"division": "Northwest",
"game_number": "19",
"line_score": {
  "H1": {
    "AST": "43",
    "BLK": "02",
    "DREB": "78",
    "FG3A": "136",
    "FG3M": "20",
    "FG3_PCT": "15",
    "FGA": "2523",
    "FGM": "86",
    "FG_PCT": "3",
    "FTA": "79",
    "FTM": "57",
    "FT_PCT": "72",
    "MIN": "6060",
    "OREB": "63",
    "PTS": "2319",
    "STL": "32",
    "TOV": "36",
    "TREB": "141"
  },
  "H2": {
    "AST": "411",
    "BLK": "21",
    "DREB": "1010",
    "FG3A": "812",
    "FG3M": "34",
    "FG3_PCT": "4",
    "FGA": "2827",
    "FGM": "1013",
    "FG_PCT": "36",
    "FTA": "52",
    "FTM": "12",
    "FT_PCT": "23",
    "MIN": "6060",
    "OREB": "73",
    "PTS": "2432",
    "STL": "22",
    "TOV": "10",
    "TREB": "1083"
  },
  "OT": {
    "AST": "0",
    "BLK": "0",
    "DREB": "0",
    "FG3A": "0",
    "FG3M": "0",
    "FG3_PCT": "0",
```

```
406         "FGA": "0",
407         "FGM": "0",
408         "FG_PCT": "0",
409         "FTA": "0",
410         "FTM": "0",
411         "FT_PCT": "0",
412         "MIN": "0",
413         "OREB": "0",
414         "PTS": "0",
415         "STL": "0",
416         "TOV": "0",
417         "TREB": "0"
418     },
419     "Q1": {
420         "AST": "4",
421         "BLK": "0",
422         "DREB": "7",
423         "FG3A": "13",
424         "FG3M": "2",
425         "FG3_PCT": "15",
426         "FGA": "25",
427         "FGM": "8",
428         "FG_PCT": "32",
429         "FTA": "7",
430         "FTM": "5",
431         "FT_PCT": "71",
432         "MIN": "60",
433         "OREB": "6",
434         "PTS": "23",
435         "STL": "3",
436         "TOV": "3",
437         "TREB": "13"
438     },
439     "Q2": {
440         "AST": "3",
441         "BLK": "2",
442         "DREB": "8",
443         "FG3A": "6",
444         "FG3M": "0",
445         "FG3_PCT": "0",
446         "FGA": "23",
447         "FGM": "6",
448         "FG_PCT": "26",
449         "FTA": "9",
450         "FTM": "7",
451         "FT_PCT": "78",
452         "MIN": "60",
453         "OREB": "3",
454         "PTS": "19",
455         "STL": "2",
456         "TOV": "6",
457         "TREB": "11"
458     },
459     "Q3": {
460         "AST": "4",
461         "BLK": "2",
462         "DREB": "10",
463         "FG3A": "8",
464         "FG3M": "3",
465         "FG3_PCT": "38",
466         "FGA": "28",
467         "FGM": "10",
468         "FG_PCT": "36",
469         "FTA": "5",
470         "FTM": "1",
471         "FT_PCT": "20",
472         "MIN": "60",
473         "OREB": "7",
474         "PTS": "24",
475         "STL": "2",
476         "TOV": "1",
477         "TREB": "17"
478     },
479     "Q4": {
480         "AST": "11",
481         "BLK": "1",
482         "DREB": "10",
483         "FG3A": "12",
484         "FG3M": "4",
485         "FG3_PCT": "33",
486         "FGA": "27",
487         "FGM": "13",
488         "FG_PCT": "48",
489         "FTA": "2",
490         "FTM": "2",
491         "FT_PCT": "100",
492         "MIN": "60",
493         "OREB": "3",
494         "PTS": "32",
495         "STL": "2",
496         "TOV": "0",
```

```

    "TREB": "13"
  },
  "game": {
    "AST": "22",
    "BLK": "5",
    "DREB": "35",
    "FG3A": "39",
    "FG3M": "9",
    "FG3_PCT": "23",
    "FGA": "103",
    "FGM": "37",
    "FG_PCT": "36",
    "FTA": "23",
    "FTM": "15",
    "FT_PCT": "65",
    "MIN": "4",
    "OREB": "19",
    "PF": "24",
    "PTS": "98",
    "STL": "9",
    "TOV": "10",
    "TREB": "54"
  }
},
"losses": "7",
"name": "Thunder",
"next_game": {
  "city": "Oklahoma City",
  "day": "28",
  "dayname": "Wednesday",
  "is_home": "True",
  "month": "November",
  "opponent_name": "Cavaliers",
  "opponent_place": "Cleveland",
  "stadium": "Chesapeake Energy Arena",
  "year": "2018"
},
"next_game_id": "5956",
"place": "Oklahoma City",
"previous_game_id": "5954",
"wins": "12"
},
"vis": {
  "box_score": [
    {
      "+/-": "4",
      "AST": "8",
      "BLK": "0",
      "DOUBLE": "none",
      "DREB": "6",
      "FG3A": "6",
      "FG3M": "2",
      "FG3_PCT": "33",
      "FGA": "23",
      "FGM": "9",
      "FG_PCT": "39",
      "FTA": "2",
      "FTM": "2",
      "FT_PCT": "100",
      "MIN": "39",
      "OREB": "2",
      "PF": "1",
      "PTS": "22",
      "STL": "0",
      "TOV": "3",
      "TREB": "8",
      "first_name": "Jamal",
      "last_name": "Murray",
      "name": "Jamal Murray",
      "starter": "True"
    }
  ],
  {
    "+/-": "-2",
    "AST": "1",
    "BLK": "2",
    "DOUBLE": "none",
    "DREB": "4",
    "FG3A": "2",
    "FG3M": "0",
    "FG3_PCT": "0",
    "FGA": "8",
    "FGM": "4",
    "FG_PCT": "50",
    "FTA": "1",
    "FTM": "0",
    "FT_PCT": "0",
    "MIN": "36",
    "OREB": "6",
    "PF": "2",
    "PTS": "8",
    "STL": "1",

```



```

588     "TOV": "2",
589     "TREB": "10",
590     "first_name": "Torrey",
591     "last_name": "Craig",
592     "name": "Torrey Craig",
593     "starter": "True"
594 },
595 {
596     "+/-": "4",
597     "AST": "0",
598     "BLK": "1",
599     "DOUBLE": "none",
600     "DREB": "7",
601     "FG3A": "7",
602     "FG3M": "3",
603     "FG3_PCT": "43",
604     "FGA": "12",
605     "FGM": "4",
606     "FG_PCT": "33",
607     "FTA": "4",
608     "FTM": "4",
609     "FT_PCT": "100",
610     "MIN": "35",
611     "OREB": "1",
612     "PF": "2",
613     "PTS": "15",
614     "STL": "2",
615     "TOV": "1",
616     "TREB": "8",
617     "first_name": "Juan",
618     "last_name": "Hernangómez",
619     "name": "Juan Hernangómez",
620     "starter": "True"
621 },
622 {
623     "+/-": "2",
624     "AST": "3",
625     "BLK": "1",
626     "DOUBLE": "none",
627     "DREB": "6",
628     "FG3A": "3",
629     "FG3M": "2",
630     "FG3_PCT": "67",
631     "FGA": "7",
632     "FGM": "3",
633     "FG_PCT": "43",
634     "FTA": "2",
635     "FTM": "0",
636     "FT_PCT": "0",
637     "MIN": "28",
638     "OREB": "0",
639     "PF": "3",
640     "PTS": "8",
641     "STL": "1",
642     "TOV": "1",
643     "TREB": "6",
644     "first_name": "Paul",
645     "last_name": "Millsap",
646     "name": "Paul Millsap",
647     "starter": "True"
648 },
649 {
650     "+/-": "15",
651     "AST": "5",
652     "BLK": "1",
653     "DOUBLE": "none",
654     "DREB": "4",
655     "FG3A": "7",
656     "FG3M": "0",
657     "FG3_PCT": "0",
658     "FGA": "20",
659     "FGM": "6",
660     "FG_PCT": "30",
661     "FTA": "5",
662     "FTM": "4",
663     "FT_PCT": "80",
664     "MIN": "27",
665     "OREB": "2",
666     "PF": "4",
667     "PTS": "16",
668     "STL": "0",
669     "TOV": "4",
670     "TREB": "6",
671     "first_name": "Nikola",
672     "last_name": "ĆJoki",
673     "name": "Nikola ĆJoki",
674     "starter": "True"
675 },
676 {
677     "+/-": "-8",
678     "AST": "2",

```

```

679         "BLK": "4",
680         "DOUBLE": "none",
681         "DREB": "4",
682         "FG3A": "0",
683         "FG3M": "0",
684         "FG3_PCT": "0",
685         "FGA": "8",
686         "FGM": "4",
687         "FG_PCT": "50",
688         "FTA": "6",
689         "FTM": "3",
690         "FT_PCT": "50",
691         "MIN": "20",
692         "OREB": "3",
693         "PF": "4",
694         "PTS": "11",
695         "STL": "0",
696         "TOV": "2",
697         "TREB": "7",
698         "first_name": "Mason",
699         "last_name": "Plumlee",
700         "name": "Mason Plumlee",
701         "starter": "False"
702     },
703     {
704         "+/-": "7",
705         "AST": "6",
706         "BLK": "0",
707         "DOUBLE": "none",
708         "DREB": "2",
709         "FG3A": "0",
710         "FG3M": "0",
711         "FG3_PCT": "0",
712         "FGA": "5",
713         "FGM": "1",
714         "FG_PCT": "20",
715         "FTA": "2",
716         "FTM": "2",
717         "FT_PCT": "100",
718         "MIN": "20",
719         "OREB": "0",
720         "PF": "1",
721         "PTS": "4",
722         "STL": "2",
723         "TOV": "0",
724         "TREB": "2",
725         "first_name": "Monte",
726         "last_name": "Morris",
727         "name": "Monte Morris",
728         "starter": "False"
729     },
730     {
731         "+/-": "5",
732         "AST": "1",
733         "BLK": "0",
734         "DOUBLE": "none",
735         "DREB": "3",
736         "FG3A": "2",
737         "FG3M": "2",
738         "FG3_PCT": "100",
739         "FGA": "6",
740         "FGM": "6",
741         "FG_PCT": "100",
742         "FTA": "2",
743         "FTM": "2",
744         "FT_PCT": "100",
745         "MIN": "19",
746         "OREB": "1",
747         "PF": "1",
748         "PTS": "16",
749         "STL": "0",
750         "TOV": "1",
751         "TREB": "4",
752         "first_name": "Trey",
753         "last_name": "Lyles",
754         "name": "Trey Lyles",
755         "starter": "False"
756     },
757     {
758         "+/-": "8",
759         "AST": "0",
760         "BLK": "0",
761         "DOUBLE": "none",
762         "DREB": "3",
763         "FG3A": "3",
764         "FG3M": "1",
765         "FG3_PCT": "33",
766         "FGA": "6",
767         "FGM": "2",
768         "FG_PCT": "33",
769         "FTA": "0",

```

```

770         "FTM": "0",
771         "FT_PCT": "0",
772         "MIN": "12",
773         "OREB": "0",
774         "PF": "2",
775         "PTS": "5",
776         "STL": "0",
777         "TOV": "0",
778         "TREB": "3",
779         "first_name": "Malik",
780         "last_name": "Beasley",
781         "name": "Malik Beasley",
782         "starter": "False"
783     },
784     {
785         "+/-": "0",
786         "AST": "0",
787         "BLK": "0",
788         "DOUBLE": "none",
789         "DREB": "0",
790         "FG3A": "0",
791         "FG3M": "0",
792         "FG3_PCT": "0",
793         "FGA": "0",
794         "FGM": "0",
795         "FG_PCT": "0",
796         "FTA": "0",
797         "FTM": "0",
798         "FT_PCT": "0",
799         "MIN": "0",
800         "OREB": "0",
801         "PF": "0",
802         "PTS": "0",
803         "STL": "0",
804         "TOV": "0",
805         "TREB": "0",
806         "first_name": "DeVaughn",
807         "last_name": "Akoon-Purcell",
808         "name": "DeVaughn Akoon-Purcell",
809         "starter": "False"
810     },
811     {
812         "+/-": "0",
813         "AST": "0",
814         "BLK": "0",
815         "DOUBLE": "none",
816         "DREB": "0",
817         "FG3A": "0",
818         "FG3M": "0",
819         "FG3_PCT": "0",
820         "FGA": "0",
821         "FGM": "0",
822         "FG_PCT": "0",
823         "FTA": "0",
824         "FTM": "0",
825         "FT_PCT": "0",
826         "MIN": "0",
827         "OREB": "0",
828         "PF": "0",
829         "PTS": "0",
830         "STL": "0",
831         "TOV": "0",
832         "TREB": "0",
833         "first_name": "Gary",
834         "last_name": "Harris",
835         "name": "Gary Harris",
836         "starter": "False"
837     },
838     {
839         "+/-": "0",
840         "AST": "0",
841         "BLK": "0",
842         "DOUBLE": "none",
843         "DREB": "0",
844         "FG3A": "0",
845         "FG3M": "0",
846         "FG3_PCT": "0",
847         "FGA": "0",
848         "FGM": "0",
849         "FG_PCT": "0",
850         "FTA": "0",
851         "FTM": "0",
852         "FT_PCT": "0",
853         "MIN": "0",
854         "OREB": "0",
855         "PF": "0",
856         "PTS": "0",
857         "STL": "0",
858         "TOV": "0",
859         "TREB": "0",
860         "first_name": "Tyler",

```

```
861     "last_name": "Lydon",
862     "name": "Tyler Lydon",
863     "starter": "False"
864   }
865 ],
866 "conference": "Western Conference",
867 "conference_standing": 4,
868 "division": "Northwest",
869 "game_number": "20",
870 "line_score": {
871   "H1": {
872     "AST": "97",
873     "BLK": "02",
874     "DREB": "912",
875     "FG3A": "98",
876     "FG3M": "52",
877     "FG3_PCT": "53",
878     "FGA": "2327",
879     "FGM": "1312",
880     "FG_PCT": "56",
881     "FTA": "46",
882     "FTM": "24",
883     "FT_PCT": "52",
884     "MIN": "6060",
885     "OREB": "25",
886     "PTS": "3330",
887     "STL": "24",
888     "TOV": "36",
889     "TREB": "937"
890   },
891   "H2": {
892     "AST": "55",
893     "BLK": "52",
894     "DREB": "99",
895     "FG3A": "94",
896     "FG3M": "21",
897     "FG3_PCT": "22",
898     "FGA": "2223",
899     "FGM": "59",
900     "FG_PCT": "3",
901     "FTA": "410",
902     "FTM": "47",
903     "FT_PCT": "11",
904     "MIN": "6060",
905     "OREB": "44",
906     "PTS": "1626",
907     "STL": "00",
908     "TOV": "32",
909     "TREB": "143"
910   },
911   "OT": {
912     "AST": "0",
913     "BLK": "0",
914     "DREB": "0",
915     "FG3A": "0",
916     "FG3M": "0",
917     "FG3_PCT": "0",
918     "FGA": "0",
919     "FGM": "0",
920     "FG_PCT": "0",
921     "FTA": "0",
922     "FTM": "0",
923     "FT_PCT": "0",
924     "MIN": "0",
925     "OREB": "0",
926     "PTS": "0",
927     "STL": "0",
928     "TOV": "0",
929     "TREB": "0"
930   },
931   "Q1": {
932     "AST": "9",
933     "BLK": "0",
934     "DREB": "9",
935     "FG3A": "9",
936     "FG3M": "5",
937     "FG3_PCT": "56",
938     "FGA": "23",
939     "FGM": "13",
940     "FG_PCT": "57",
941     "FTA": "4",
942     "FTM": "2",
943     "FT_PCT": "50",
944     "MIN": "60",
945     "OREB": "2",
946     "PTS": "33",
947     "STL": "2",
948     "TOV": "3",
949     "TREB": "11"
950   },
951   "Q2": {
```

```

    "AST": "7",
    "BLK": "2",
    "DREB": "12",
    "FG3A": "8",
    "FG3M": "2",
    "FG3_PCT": "25",
    "FGA": "27",
    "FGM": "12",
    "FG_PCT": "44",
    "FTA": "6",
    "FTM": "4",
    "FT_PCT": "67",
    "MIN": "60",
    "OREB": "5",
    "PTS": "30",
    "STL": "4",
    "TOV": "6",
    "TREB": "17"
  },
  "Q3": {
    "AST": "5",
    "BLK": "5",
    "DREB": "9",
    "FG3A": "9",
    "FG3M": "2",
    "FG3_PCT": "22",
    "FGA": "22",
    "FGM": "5",
    "FG_PCT": "23",
    "FTA": "4",
    "FTM": "4",
    "FT_PCT": "100",
    "MIN": "60",
    "OREB": "4",
    "PTS": "16",
    "STL": "0",
    "TOV": "3",
    "TREB": "13"
  },
  "Q4": {
    "AST": "5",
    "BLK": "2",
    "DREB": "9",
    "FG3A": "4",
    "FG3M": "1",
    "FG3_PCT": "25",
    "FGA": "23",
    "FGM": "9",
    "FG_PCT": "39",
    "FTA": "10",
    "FTM": "7",
    "FT_PCT": "70",
    "MIN": "60",
    "OREB": "4",
    "PTS": "26",
    "STL": "0",
    "TOV": "2",
    "TREB": "13"
  },
  "game": {
    "AST": "26",
    "BLK": "9",
    "DREB": "39",
    "FG3A": "30",
    "FG3M": "10",
    "FG3_PCT": "33",
    "FGA": "95",
    "FGM": "39",
    "FG_PCT": "41",
    "FTA": "24",
    "FTM": "17",
    "FT_PCT": "71",
    "MIN": "4",
    "OREB": "15",
    "PF": "20",
    "PTS": "105",
    "STL": "6",
    "TOV": "14",
    "TREB": "54"
  }
},
"losses": "7",
"name": "Nuggets",
"next_game": {
  "city": "Denver",
  "day": "27",
  "dayname": "Tuesday",
  "is_home": "True",
  "month": "November",
  "opponent_name": "Lakers",
  "opponent_place": "Los Angeles",

```

```

1043     "stadium": "Pepsi Center",
1044     "year": "2018"
1045   },
1046   "next_game_id": "5629",
1047   "place": "Denver",
1048   "previous_game_id": "5628",
1049   "wins": "13"
1050 }
1051 }
1052 },
1053 "source_sha256": "86491264c5dd17cd78b7120bb3f179565618a1c2778e388ea25b41e288ef6199",
1054 "task_family": "event_report"
1055 }

```

Listing 79: D15: submission/program_listings/build/generated/data/sportsett_basketball__6127.json

```

1 {
2   "dataset_id": "sportsett_basketball",
3   "example_id": "6127",
4   "language": "en",
5   "output_mode": "multi_paragraph_report",
6   "parent_table": null,
7   "reference_sha256": "b35653aa64d112e412a5fd2d9f6320a37fe2b9791b2854ce11248a82c31ef183",
8   "references": [
9     "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
10  ],
11  "request": "Write a coherent game report from the supplied structured game data. Lead with the result, select the
12    most important performances and contrasts, and do not invent information.",
13  "source_payload": {
14    "game": {
15      "attendance": "15300",
16      "capacity": "20400",
17      "city": "Washington",
18      "day": "2",
19      "dayname": "Wednesday",
20      "game_id": "6127",
21      "month": "January",
22      "season": "2018",
23      "stadium": "Capital One Arena",
24      "state": "Washington",
25      "year": "2019"
26    },
27    "teams": {
28      "home": {
29        "box_score": [
30          {
31            "+/-": "20",
32            "AST": "1",
33            "BLK": "2",
34            "DOUBLE": "double",
35            "DREB": "14",
36            "FG3A": "1",
37            "FG3M": "0",
38            "FG3_PCT": "0",
39            "FGA": "7",
40            "FGM": "5",
41            "FG_PCT": "71",
42            "FTA": "6",
43            "FTM": "6",
44            "FT_PCT": "100",
45            "MIN": "39",
46            "OREB": "1",
47            "PF": "3",
48            "PTS": "16",
49            "STL": "2",
50            "TOV": "1",
51            "TREB": "15",
52            "first_name": "Thomas",
53            "last_name": "Bryant",
54            "name": "Thomas Bryant",
55            "starter": "True"
56          },
57          {
58            "+/-": "11",
59            "AST": "4",
60            "BLK": "0",
61            "DOUBLE": "none",
62            "DREB": "4",
63            "FG3A": "5",
64            "FG3M": "1",
65            "FG3_PCT": "20",
66            "FGA": "15",
67            "FGM": "5",
68            "FG_PCT": "33",
69            "FTA": "3",
70            "FTM": "1",
71            "FT_PCT": "33",
72            "MIN": "39",
73            "OREB": "1",

```

```
"PF": "1",
"PTS": "12",
"STL": "1",
"TOV": "1",
"TREB": "5",
"first_name": "Trevor",
"last_name": "Ariza",
"name": "Trevor Ariza",
"starter": "True"
},
{
  "+/-": "13",
  "AST": "7",
  "BLK": "0",
  "DOUBLE": "double",
  "DREB": "7",
  "FG3A": "2",
  "FG3M": "1",
  "FG3_PCT": "50",
  "FGA": "10",
  "FGM": "5",
  "FG_PCT": "50",
  "FTA": "6",
  "FTM": "3",
  "FT_PCT": "50",
  "MIN": "36",
  "OREB": "4",
  "PF": "1",
  "PTS": "14",
  "STL": "2",
  "TOV": "1",
  "TREB": "11",
  "first_name": "šTomá",
  "last_name": "Satoranský",
  "name": "šTomá Satoranský",
  "starter": "True"
},
{
  "+/-": "15",
  "AST": "6",
  "BLK": "0",
  "DOUBLE": "none",
  "DREB": "6",
  "FG3A": "8",
  "FG3M": "4",
  "FG3_PCT": "50",
  "FGA": "13",
  "FGM": "7",
  "FG_PCT": "54",
  "FTA": "4",
  "FTM": "4",
  "FT_PCT": "100",
  "MIN": "36",
  "OREB": "0",
  "PF": "4",
  "PTS": "22",
  "STL": "0",
  "TOV": "1",
  "TREB": "6",
  "first_name": "Jeff",
  "last_name": "Green",
  "name": "Jeff Green",
  "starter": "True"
},
{
  "+/-": "22",
  "AST": "6",
  "BLK": "0",
  "DOUBLE": "none",
  "DREB": "4",
  "FG3A": "7",
  "FG3M": "3",
  "FG3_PCT": "43",
  "FGA": "20",
  "FGM": "9",
  "FG_PCT": "45",
  "FTA": "3",
  "FTM": "3",
  "FT_PCT": "100",
  "MIN": "36",
  "OREB": "0",
  "PF": "5",
  "PTS": "24",
  "STL": "2",
  "TOV": "3",
  "TREB": "4",
  "first_name": "Bradley",
  "last_name": "Beal",
  "name": "Bradley Beal",
  "starter": "True"
},
```

```

164     {
165         "+/-": "-3",
166         "AST": "1",
167         "BLK": "0",
168         "DOUBLE": "none",
169         "DREB": "1",
170         "FG3A": "2",
171         "FG3M": "1",
172         "FG3_PCT": "50",
173         "FGA": "6",
174         "FGM": "3",
175         "FG_PCT": "50",
176         "FTA": "1",
177         "FTM": "0",
178         "FT_PCT": "0",
179         "MIN": "17",
180         "OREB": "2",
181         "PF": "3",
182         "PTS": "7",
183         "STL": "3",
184         "TOV": "2",
185         "TREB": "3",
186         "first_name": "Chasson",
187         "last_name": "Randle",
188         "name": "Chasson Randle",
189         "starter": "False"
190     },
191     {
192         "+/-": "-1",
193         "AST": "1",
194         "BLK": "0",
195         "DOUBLE": "none",
196         "DREB": "0",
197         "FG3A": "0",
198         "FG3M": "0",
199         "FG3_PCT": "0",
200         "FGA": "10",
201         "FGM": "4",
202         "FG_PCT": "40",
203         "FTA": "0",
204         "FTM": "0",
205         "FT_PCT": "0",
206         "MIN": "15",
207         "OREB": "2",
208         "PF": "2",
209         "PTS": "8",
210         "STL": "0",
211         "TOV": "0",
212         "TREB": "2",
213         "first_name": "Sam",
214         "last_name": "Dekker",
215         "name": "Sam Dekker",
216         "starter": "False"
217     },
218     {
219         "+/-": "3",
220         "AST": "2",
221         "BLK": "0",
222         "DOUBLE": "none",
223         "DREB": "1",
224         "FG3A": "3",
225         "FG3M": "1",
226         "FG3_PCT": "33",
227         "FGA": "8",
228         "FGM": "4",
229         "FG_PCT": "50",
230         "FTA": "0",
231         "FTM": "0",
232         "FT_PCT": "0",
233         "MIN": "13",
234         "OREB": "0",
235         "PF": "1",
236         "PTS": "9",
237         "STL": "0",
238         "TOV": "1",
239         "TREB": "1",
240         "first_name": "Otto",
241         "last_name": "Porter",
242         "name": "Otto Porter",
243         "starter": "False"
244     },
245     {
246         "+/-": "0",
247         "AST": "1",
248         "BLK": "0",
249         "DOUBLE": "none",
250         "DREB": "1",
251         "FG3A": "1",
252         "FG3M": "0",
253         "FG3_PCT": "0",
254         "FGA": "3",

```



```

    "FGM": "1",
    "FG_PCT": "33",
    "FTA": "0",
    "FTM": "0",
    "FT_PCT": "0",
    "MIN": "6",
    "OREB": "0",
    "PF": "0",
    "PTS": "2",
    "STL": "0",
    "TOV": "0",
    "TREB": "1",
    "first_name": "Troy",
    "last_name": "Brown",
    "name": "Troy Brown",
    "starter": "False"
  },
  {
    "+/-": "0",
    "AST": "0",
    "BLK": "0",
    "DOUBLE": "none",
    "DREB": "0",
    "FG3A": "0",
    "FG3M": "0",
    "FG3_PCT": "0",
    "FGA": "0",
    "FGM": "0",
    "FG_PCT": "0",
    "FTA": "0",
    "FTM": "0",
    "FT_PCT": "0",
    "MIN": "0",
    "OREB": "0",
    "PF": "0",
    "PTS": "0",
    "STL": "0",
    "TOV": "0",
    "TREB": "0",
    "first_name": "Ron",
    "last_name": "Baker",
    "name": "Ron Baker",
    "starter": "False"
  },
  {
    "+/-": "0",
    "AST": "0",
    "BLK": "0",
    "DOUBLE": "none",
    "DREB": "0",
    "FG3A": "0",
    "FG3M": "0",
    "FG3_PCT": "0",
    "FGA": "0",
    "FGM": "0",
    "FG_PCT": "0",
    "FTA": "0",
    "FTM": "0",
    "FT_PCT": "0",
    "MIN": "0",
    "OREB": "0",
    "PF": "0",
    "PTS": "0",
    "STL": "0",
    "TOV": "0",
    "TREB": "0",
    "first_name": "Ian",
    "last_name": "Mahinmi",
    "name": "Ian Mahinmi",
    "starter": "False"
  }
],
"conference": "Eastern Conference",
"conference_standing": 11,
"division": "Southeast",
"game_number": "38",
"line_score": {
  "H1": {
    "AST": "88",
    "BLK": "01",
    "DREB": "912",
    "FG3A": "86",
    "FG3M": "41",
    "FG3_PCT": "48",
    "FGA": "2126",
    "FGM": "1212",
    "FG_PCT": "57",
    "FTA": "106",
    "FTM": "74",
    "FT_PCT": "70",
    "MIN": "6060",

```

```
346         "OREB": "32",
347         "PTS": "3529",
348         "STL": "23",
349         "TOV": "12",
350         "TREB": "944"
351     },
352     "H2": {
353         "AST": "67",
354         "BLK": "01",
355         "DREB": "611",
356         "FG3A": "87",
357         "FG3M": "33",
358         "FG3_PCT": "38",
359         "FGA": "2025",
360         "FGM": "811",
361         "FG_PCT": "40",
362         "FTA": "52",
363         "FTM": "51",
364         "FT_PCT": "98",
365         "MIN": "6060",
366         "OREB": "23",
367         "PTS": "2426",
368         "STL": "14",
369         "TOV": "61",
370         "TREB": "634"
371     },
372     "OT": {
373         "AST": "0",
374         "BLK": "0",
375         "DREB": "0",
376         "FG3A": "0",
377         "FG3M": "0",
378         "FG3_PCT": "0",
379         "FGA": "0",
380         "FGM": "0",
381         "FG_PCT": "0",
382         "FTA": "0",
383         "FTM": "0",
384         "FT_PCT": "0",
385         "MIN": "0",
386         "OREB": "0",
387         "PTS": "0",
388         "STL": "0",
389         "TOV": "0",
390         "TREB": "0"
391     },
392     "Q1": {
393         "AST": "8",
394         "BLK": "0",
395         "DREB": "9",
396         "FG3A": "8",
397         "FG3M": "4",
398         "FG3_PCT": "50",
399         "FGA": "21",
400         "FGM": "12",
401         "FG_PCT": "57",
402         "FTA": "10",
403         "FTM": "7",
404         "FT_PCT": "70",
405         "MIN": "60",
406         "OREB": "3",
407         "PTS": "35",
408         "STL": "2",
409         "TOV": "1",
410         "TREB": "12"
411     },
412     "Q2": {
413         "AST": "8",
414         "BLK": "1",
415         "DREB": "12",
416         "FG3A": "6",
417         "FG3M": "1",
418         "FG3_PCT": "17",
419         "FGA": "26",
420         "FGM": "12",
421         "FG_PCT": "46",
422         "FTA": "6",
423         "FTM": "4",
424         "FT_PCT": "67",
425         "MIN": "60",
426         "OREB": "2",
427         "PTS": "29",
428         "STL": "3",
429         "TOV": "2",
430         "TREB": "14"
431     },
432     "Q3": {
433         "AST": "6",
434         "BLK": "0",
435         "DREB": "6",
436         "FG3A": "8",
```

```

437         "FG3M": "3",
438         "FG3_PCT": "38",
439         "FGA": "20",
440         "FGM": "8",
441         "FG_PCT": "40",
442         "FTA": "5",
443         "FTM": "5",
444         "FT_PCT": "100",
445         "MIN": "60",
446         "OREB": "2",
447         "PTS": "24",
448         "STL": "1",
449         "TOV": "6",
450         "TREB": "8"
451     },
452     "Q4": {
453         "AST": "7",
454         "BLK": "1",
455         "DREB": "11",
456         "FG3A": "7",
457         "FG3M": "3",
458         "FG3_PCT": "43",
459         "FGA": "25",
460         "FGM": "11",
461         "FG_PCT": "44",
462         "FTA": "2",
463         "FTM": "1",
464         "FT_PCT": "50",
465         "MIN": "60",
466         "OREB": "3",
467         "PTS": "26",
468         "STL": "4",
469         "TOV": "1",
470         "TREB": "14"
471     },
472     "game": {
473         "AST": "29",
474         "BLK": "2",
475         "DREB": "38",
476         "FG3A": "29",
477         "FG3M": "11",
478         "FG3_PCT": "38",
479         "FGA": "92",
480         "FGM": "43",
481         "FG_PCT": "47",
482         "FTA": "23",
483         "FTM": "17",
484         "FT_PCT": "74",
485         "MIN": "4",
486         "OREB": "10",
487         "PF": "20",
488         "PTS": "114",
489         "STL": "10",
490         "TOV": "10",
491         "TREB": "48"
492     }
493 },
494 "losses": "23",
495 "name": "Wizards",
496 "next_game": {
497     "city": "Miami",
498     "day": "4",
499     "dayname": "Friday",
500     "is_home": "False",
501     "month": "January",
502     "opponent_name": "Heat",
503     "opponent_place": "Miami",
504     "stadium": "American Airlines Arena",
505     "year": "2019"
506 },
507 "next_game_id": "5268",
508 "place": "Washington",
509 "previous_game_id": "6126",
510 "wins": "15"
511 },
512 "vis": {
513     "box_score": [
514         {
515             "+/-": "-11",
516             "AST": "5",
517             "BLK": "1",
518             "DOUBLE": "none",
519             "DREB": "1",
520             "FG3A": "7",
521             "FG3M": "1",
522             "FG3_PCT": "14",
523             "FGA": "14",
524             "FGM": "5",
525             "FG_PCT": "36",
526             "FTA": "1",
527             "FTM": "1",

```

```

528         "FT_PCT": "100",
529         "MIN": "44",
530         "OREB": "0",
531         "PF": "2",
532         "PTS": "12",
533         "STL": "0",
534         "TOV": "2",
535         "TREB": "1",
536         "first_name": "Kevin",
537         "last_name": "Huerter",
538         "name": "Kevin Huerter",
539         "starter": "True"
540     },
541     {
542         "+/-": "-19",
543         "AST": "9",
544         "BLK": "1",
545         "DOUBLE": "none",
546         "DREB": "3",
547         "FG3A": "2",
548         "FG3M": "1",
549         "FG3_PCT": "50",
550         "FGA": "8",
551         "FGM": "2",
552         "FG_PCT": "25",
553         "FTA": "0",
554         "FTM": "0",
555         "FT_PCT": "0",
556         "MIN": "27",
557         "OREB": "0",
558         "PF": "2",
559         "PTS": "5",
560         "STL": "0",
561         "TOV": "4",
562         "TREB": "3",
563         "first_name": "Trae",
564         "last_name": "Young",
565         "name": "Trae Young",
566         "starter": "True"
567     },
568     {
569         "+/-": "-12",
570         "AST": "1",
571         "BLK": "1",
572         "DOUBLE": "none",
573         "DREB": "6",
574         "FG3A": "1",
575         "FG3M": "0",
576         "FG3_PCT": "0",
577         "FGA": "8",
578         "FGM": "3",
579         "FG_PCT": "38",
580         "FTA": "0",
581         "FTM": "0",
582         "FT_PCT": "0",
583         "MIN": "27",
584         "OREB": "3",
585         "PF": "3",
586         "PTS": "6",
587         "STL": "1",
588         "TOV": "1",
589         "TREB": "9",
590         "first_name": "Dewayne",
591         "last_name": "Dedmon",
592         "name": "Dewayne Dedmon",
593         "starter": "True"
594     },
595     {
596         "+/-": "-7",
597         "AST": "3",
598         "BLK": "0",
599         "DOUBLE": "none",
600         "DREB": "7",
601         "FG3A": "5",
602         "FG3M": "4",
603         "FG3_PCT": "80",
604         "FGA": "14",
605         "FGM": "8",
606         "FG_PCT": "57",
607         "FTA": "2",
608         "FTM": "1",
609         "FT_PCT": "50",
610         "MIN": "26",
611         "OREB": "1",
612         "PF": "4",
613         "PTS": "21",
614         "STL": "0",
615         "TOV": "2",
616         "TREB": "8",
617         "first_name": "John",
618         "last_name": "Collins",

```

```

619         "name": "John Collins",
620         "starter": "True"
621     },
622     {
623         "+/-": "-8",
624         "AST": "1",
625         "BLK": "0",
626         "DOUBLE": "none",
627         "DREB": "5",
628         "FG3A": "3",
629         "FG3M": "1",
630         "FG3_PCT": "33",
631         "FGA": "8",
632         "FGM": "3",
633         "FG_PCT": "38",
634         "FTA": "2",
635         "FTM": "1",
636         "FT_PCT": "50",
637         "MIN": "20",
638         "OREB": "2",
639         "PF": "1",
640         "PTS": "8",
641         "STL": "0",
642         "TOV": "0",
643         "TREB": "7",
644         "first_name": "Daniel",
645         "last_name": "Hamilton",
646         "name": "Daniel Hamilton",
647         "starter": "True"
648     },
649     {
650         "+/-": "-6",
651         "AST": "0",
652         "BLK": "3",
653         "DOUBLE": "double",
654         "DREB": "5",
655         "FG3A": "3",
656         "FG3M": "1",
657         "FG3_PCT": "33",
658         "FGA": "19",
659         "FGM": "11",
660         "FG_PCT": "58",
661         "FTA": "2",
662         "FTM": "1",
663         "FT_PCT": "50",
664         "MIN": "27",
665         "OREB": "6",
666         "PF": "3",
667         "PTS": "24",
668         "STL": "0",
669         "TOV": "1",
670         "TREB": "11",
671         "first_name": "Alex",
672         "last_name": "Len",
673         "name": "Alex Len",
674         "starter": "False"
675     },
676     {
677         "+/-": "-8",
678         "AST": "0",
679         "BLK": "0",
680         "DOUBLE": "none",
681         "DREB": "6",
682         "FG3A": "1",
683         "FG3M": "0",
684         "FG3_PCT": "0",
685         "FGA": "9",
686         "FGM": "5",
687         "FG_PCT": "56",
688         "FTA": "2",
689         "FTM": "1",
690         "FT_PCT": "50",
691         "MIN": "25",
692         "OREB": "1",
693         "PF": "3",
694         "PTS": "11",
695         "STL": "1",
696         "TOV": "1",
697         "TREB": "7",
698         "first_name": "DeAndre'",
699         "last_name": "Bembry",
700         "name": "DeAndre' Bembry",
701         "starter": "False"
702     },
703     {
704         "+/-": "5",
705         "AST": "5",
706         "BLK": "0",
707         "DOUBLE": "none",
708         "DREB": "2",
709         "FG3A": "4",

```

```
710     "FG3M": "1",
711     "FG3_PCT": "25",
712     "FGA": "7",
713     "FGM": "2",
714     "FG_PCT": "29",
715     "FTA": "4",
716     "FTM": "3",
717     "FT_PCT": "75",
718     "MIN": "19",
719     "OREB": "0",
720     "PF": "0",
721     "PTS": "8",
722     "STL": "3",
723     "TOV": "2",
724     "TREB": "2",
725     "first_name": "Jeremy",
726     "last_name": "Lin",
727     "name": "Jeremy Lin",
728     "starter": "False"
729 },
730 {
731     "+/-": "-5",
732     "AST": "2",
733     "BLK": "0",
734     "DOUBLE": "none",
735     "DREB": "2",
736     "FG3A": "4",
737     "FG3M": "1",
738     "FG3_PCT": "25",
739     "FGA": "6",
740     "FGM": "1",
741     "FG_PCT": "17",
742     "FTA": "0",
743     "FTM": "0",
744     "FT_PCT": "0",
745     "MIN": "13",
746     "OREB": "0",
747     "PF": "0",
748     "PTS": "3",
749     "STL": "0",
750     "TOV": "0",
751     "TREB": "2",
752     "first_name": "Vince",
753     "last_name": "Carter",
754     "name": "Vince Carter",
755     "starter": "False"
756 },
757 {
758     "+/-": "-7",
759     "AST": "0",
760     "BLK": "0",
761     "DOUBLE": "none",
762     "DREB": "0",
763     "FG3A": "1",
764     "FG3M": "0",
765     "FG3_PCT": "0",
766     "FGA": "2",
767     "FGM": "0",
768     "FG_PCT": "0",
769     "FTA": "0",
770     "FTM": "0",
771     "FT_PCT": "0",
772     "MIN": "5",
773     "OREB": "0",
774     "PF": "1",
775     "PTS": "0",
776     "STL": "0",
777     "TOV": "1",
778     "TREB": "0",
779     "first_name": "Justin",
780     "last_name": "Anderson",
781     "name": "Justin Anderson",
782     "starter": "False"
783 },
784 {
785     "+/-": "-2",
786     "AST": "0",
787     "BLK": "0",
788     "DOUBLE": "none",
789     "DREB": "0",
790     "FG3A": "0",
791     "FG3M": "0",
792     "FG3_PCT": "0",
793     "FGA": "0",
794     "FGM": "0",
795     "FG_PCT": "0",
796     "FTA": "0",
797     "FTM": "0",
798     "FT_PCT": "0",
799     "MIN": "0",
800     "OREB": "0",
```

```

      "PF": "0",
      "PTS": "0",
      "STL": "0",
      "TOV": "0",
      "TREB": "0",
      "first_name": "Tyler",
      "last_name": "Dorsey",
      "name": "Tyler Dorsey",
      "starter": "False"
    },
    {
      "+/-": "0",
      "AST": "0",
      "BLK": "0",
      "DOUBLE": "none",
      "DREB": "0",
      "FG3A": "0",
      "FG3M": "0",
      "FG3_PCT": "0",
      "FGA": "0",
      "FGM": "0",
      "FG_PCT": "0",
      "FTA": "0",
      "FTM": "0",
      "FT_PCT": "0",
      "MIN": "0",
      "OREB": "0",
      "PF": "0",
      "PTS": "0",
      "STL": "0",
      "TOV": "0",
      "TREB": "0",
      "first_name": "Miles",
      "last_name": "Plumlee",
      "name": "Miles Plumlee",
      "starter": "False"
    }
  ],
  "conference": "Eastern Conference",
  "conference_standing": 12,
  "division": "Southeast",
  "game_number": "37",
  "line_score": {
    "H1": {
      "AST": "98",
      "BLK": "02",
      "DREB": "712",
      "FG3A": "1111",
      "FG3M": "44",
      "FG3_PCT": "4",
      "FGA": "2326",
      "FGM": "1210",
      "FG_PCT": "52",
      "FTA": "20",
      "FTM": "10",
      "FT_PCT": "50",
      "MIN": "6060",
      "OREB": "22",
      "PTS": "2924",
      "STL": "11",
      "TOV": "25",
      "TREB": "734"
    },
    "H2": {
      "AST": "72",
      "BLK": "13",
      "DREB": "810",
      "FG3A": "63",
      "FG3M": "20",
      "FG3_PCT": "32",
      "FGA": "2521",
      "FGM": "126",
      "FG_PCT": "5",
      "FTA": "83",
      "FTM": "52",
      "FT_PCT": "63",
      "MIN": "6060",
      "OREB": "63",
      "PTS": "3114",
      "STL": "30",
      "TOV": "34",
      "TREB": "873"
    },
    "OT": {
      "AST": "0",
      "BLK": "0",
      "DREB": "0",
      "FG3A": "0",
      "FG3M": "0",
      "FG3_PCT": "0",
      "FGA": "0",

```

```

892         "FGM": "0",
893         "FG_PCT": "0",
894         "FTA": "0",
895         "FTM": "0",
896         "FT_PCT": "0",
897         "MIN": "0",
898         "OREB": "0",
899         "PTS": "0",
900         "STL": "0",
901         "TOV": "0",
902         "TREB": "0"
903     },
904     "Q1": {
905         "AST": "9",
906         "BLK": "0",
907         "DREB": "7",
908         "FG3A": "11",
909         "FG3M": "4",
910         "FG3_PCT": "36",
911         "FGA": "23",
912         "FGM": "12",
913         "FG_PCT": "52",
914         "FTA": "2",
915         "FTM": "1",
916         "FT_PCT": "50",
917         "MIN": "60",
918         "OREB": "2",
919         "PTS": "29",
920         "STL": "1",
921         "TOV": "2",
922         "TREB": "9"
923     },
924     "Q2": {
925         "AST": "8",
926         "BLK": "2",
927         "DREB": "12",
928         "FG3A": "11",
929         "FG3M": "4",
930         "FG3_PCT": "36",
931         "FGA": "26",
932         "FGM": "10",
933         "FG_PCT": "38",
934         "FTA": "0",
935         "FTM": "0",
936         "FT_PCT": "0",
937         "MIN": "60",
938         "OREB": "2",
939         "PTS": "24",
940         "STL": "1",
941         "TOV": "5",
942         "TREB": "14"
943     },
944     "Q3": {
945         "AST": "7",
946         "BLK": "1",
947         "DREB": "8",
948         "FG3A": "6",
949         "FG3M": "2",
950         "FG3_PCT": "33",
951         "FGA": "25",
952         "FGM": "12",
953         "FG_PCT": "48",
954         "FTA": "8",
955         "FTM": "5",
956         "FT_PCT": "62",
957         "MIN": "60",
958         "OREB": "6",
959         "PTS": "31",
960         "STL": "3",
961         "TOV": "3",
962         "TREB": "14"
963     },
964     "Q4": {
965         "AST": "2",
966         "BLK": "3",
967         "DREB": "10",
968         "FG3A": "3",
969         "FG3M": "0",
970         "FG3_PCT": "0",
971         "FGA": "21",
972         "FGM": "6",
973         "FG_PCT": "29",
974         "FTA": "3",
975         "FTM": "2",
976         "FT_PCT": "67",
977         "MIN": "60",
978         "OREB": "3",
979         "PTS": "14",
980         "STL": "0",
981         "TOV": "4",
982         "TREB": "13"

```



```

983     },
984     "game": {
985         "AST": "26",
986         "BLK": "6",
987         "DREB": "37",
988         "FG3A": "31",
989         "FG3M": "10",
990         "FG3_PCT": "32",
991         "FGA": "95",
992         "FGM": "40",
993         "FG_PCT": "42",
994         "FTA": "13",
995         "FTM": "8",
996         "FT_PCT": "62",
997         "MIN": "4",
998         "OREB": "13",
999         "PF": "19",
1000        "PTS": "98",
1001        "STL": "5",
1002        "TOV": "14",
1003        "TREB": "50"
1004    }
1005 },
1006 "losses": "26",
1007 "name": "Hawks",
1008 "next_game": {
1009     "city": "Milwaukee",
1010     "day": "4",
1011     "dayname": "Friday",
1012     "is_home": "False",
1013     "month": "January",
1014     "opponent_name": "Bucks",
1015     "opponent_place": "Milwaukee",
1016     "stadium": "Fiserv Forum",
1017     "year": "2019"
1018 },
1019 "next_game_id": "4982",
1020 "place": "Atlanta",
1021 "previous_game_id": "5677",
1022 "wins": "11"
1023 }
1024 }
1025 },
1026 "source_sha256": "532b36909f92f9f2d8e8c7faf6b1f5643fde5ecac64a97df32804fc816cd53df",
1027 "task_family": "event_report"
1028 }

```

Listing 80: D16: submission/program_listings/build/generated/data/totto__totto-validation-1828.json

```

1  {
2  "dataset_id": "totto",
3  "example_id": "totto-validation-1828",
4  "language": "en",
5  "output_mode": "one_sentence",
6  "parent_table": [
7      [
8          "Club",
9          "Season",
10         "Division",
11         "League",
12         "Cup",
13         "Total"
14     ],
15     [
16         "Apps",
17         "Goals",
18         "Apps",
19         "Goals",
20         "Apps",
21         "Goals"
22     ],
23     [
24         "2004",
25         "Fredrikstad",
26         "Tippeligaen",
27         "1",
28         "0",
29         "0",
30         "0",
31         "1",
32         "0"
33     ],
34     [
35         "2005",
36         "0",
37         "0",
38         "0",
39         "0",
40         "0",

```

```
41     "0"
42 ],
43 [
44     "2006",
45     "4",
46     "0",
47     "0",
48     "0",
49     "3",
50     "0"
51 ],
52 [
53     "2007",
54     "4",
55     "0",
56     "1",
57     "0",
58     "5",
59     "0"
60 ],
61 [
62     "2008",
63     "11",
64     "0",
65     "1",
66     "0",
67     "12",
68     "0"
69 ],
70 [
71     "2009",
72     "15",
73     "0",
74     "2",
75     "0",
76     "17",
77     "0"
78 ],
79 [
80     "2010",
81     "Adeccoligaen",
82     "18",
83     "0",
84     "2",
85     "0",
86     "20",
87     "0"
88 ],
89 [
90     "2011",
91     "Tippeligaen",
92     "22",
93     "0",
94     "1",
95     "0",
96     "23",
97     "0"
98 ],
99 [
100     "2012",
101     "Syrianska",
102     "Allsvenskan",
103     "8",
104     "0",
105     "0",
106     "0",
107     "8",
108     "0"
109 ],
110 [
111     "2012",
112     "Lillestrøm",
113     "Tippeligaen",
114     "2",
115     "0",
116     "0",
117     "0",
118     "2",
119     "0"
120 ],
121 [
122     "2013",
123     "Aalesund",
124     "0",
125     "0",
126     "2",
127     "0",
128     "2",
129     "0"
130 ],
131 [
```

```

132     "2014",
133     "Bodø/Glimt",
134     "14",
135     "0",
136     "4",
137     "0",
138     "18",
139     "0"
140 ],
141 [
142     "2015",
143     "9",
144     "0",
145     "1",
146     "0",
147     "10",
148     "0"
149 ],
150 [
151     "Career Total",
152     "108",
153     "0",
154     "14",
155     "0",
156     "122",
157     "0"
158 ]
159 ],
160 "reference_sha256": "13f4d6ba14f2d8846ae5d47990ba0abea068972dc688c623d49d9dd967b01b34",
161 "references": [
162     "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
163 ],
164 "request": "Write exactly one concise sentence describing the highlighted table cells. Do not discuss unrelated cells
            and do not add headings.",
165 "source_payload": {
166     "highlighted_cells": [
167         [
168             10,
169             0
170         ],
171         [
172             10,
173             1
174         ],
175         [
176             10,
177             2
178         ]
179     ],
180     "table": [
181         [
182             {
183                 "column_span": 1,
184                 "is_header": true,
185                 "row_span": 2,
186                 "value": "Club"
187             },
188             {
189                 "column_span": 1,
190                 "is_header": true,
191                 "row_span": 2,
192                 "value": "Season"
193             },
194             {
195                 "column_span": 1,
196                 "is_header": true,
197                 "row_span": 2,
198                 "value": "Division"
199             },
200             {
201                 "column_span": 2,
202                 "is_header": true,
203                 "row_span": 1,
204                 "value": "League"
205             },
206             {
207                 "column_span": 2,
208                 "is_header": true,
209                 "row_span": 1,
210                 "value": "Cup"
211             },
212             {
213                 "column_span": 2,
214                 "is_header": true,
215                 "row_span": 1,
216                 "value": "Total"
217             }
218         ],
219         [
220             {
221                 "column_span": 1,

```

```

222     "is_header": true,
223     "row_span": 1,
224     "value": "Apps"
225 },
226 {
227     "column_span": 1,
228     "is_header": true,
229     "row_span": 1,
230     "value": "Goals"
231 },
232 {
233     "column_span": 1,
234     "is_header": true,
235     "row_span": 1,
236     "value": "Apps"
237 },
238 {
239     "column_span": 1,
240     "is_header": true,
241     "row_span": 1,
242     "value": "Goals"
243 },
244 {
245     "column_span": 1,
246     "is_header": true,
247     "row_span": 1,
248     "value": "Apps"
249 },
250 {
251     "column_span": 1,
252     "is_header": true,
253     "row_span": 1,
254     "value": "Goals"
255 }
256 ],
257 [
258     {
259         "column_span": 1,
260         "is_header": false,
261         "row_span": 1,
262         "value": "2004"
263     },
264     {
265         "column_span": 1,
266         "is_header": false,
267         "row_span": 8,
268         "value": "Fredrikstad"
269     },
270     {
271         "column_span": 1,
272         "is_header": false,
273         "row_span": 6,
274         "value": "Tippeligaen"
275     },
276     {
277         "column_span": 1,
278         "is_header": false,
279         "row_span": 1,
280         "value": "1"
281     },
282     {
283         "column_span": 1,
284         "is_header": false,
285         "row_span": 1,
286         "value": "0"
287     },
288     {
289         "column_span": 1,
290         "is_header": false,
291         "row_span": 1,
292         "value": "0"
293     },
294     {
295         "column_span": 1,
296         "is_header": false,
297         "row_span": 1,
298         "value": "0"
299     },
300     {
301         "column_span": 1,
302         "is_header": false,
303         "row_span": 1,
304         "value": "1"
305     },
306     {
307         "column_span": 1,
308         "is_header": false,
309         "row_span": 1,
310         "value": "0"
311     }
312 ],

```

```

313 [
314   {
315     "column_span": 1,
316     "is_header": false,
317     "row_span": 1,
318     "value": "2005"
319   },
320   {
321     "column_span": 1,
322     "is_header": false,
323     "row_span": 1,
324     "value": "0"
325   },
326   {
327     "column_span": 1,
328     "is_header": false,
329     "row_span": 1,
330     "value": "0"
331   },
332   {
333     "column_span": 1,
334     "is_header": false,
335     "row_span": 1,
336     "value": "0"
337   },
338   {
339     "column_span": 1,
340     "is_header": false,
341     "row_span": 1,
342     "value": "0"
343   },
344   {
345     "column_span": 1,
346     "is_header": false,
347     "row_span": 1,
348     "value": "0"
349   },
350   {
351     "column_span": 1,
352     "is_header": false,
353     "row_span": 1,
354     "value": "0"
355   }
356 ],
357 [
358   {
359     "column_span": 1,
360     "is_header": false,
361     "row_span": 1,
362     "value": "2006"
363   },
364   {
365     "column_span": 1,
366     "is_header": false,
367     "row_span": 1,
368     "value": "4"
369   },
370   {
371     "column_span": 1,
372     "is_header": false,
373     "row_span": 1,
374     "value": "0"
375   },
376   {
377     "column_span": 1,
378     "is_header": false,
379     "row_span": 1,
380     "value": "0"
381   },
382   {
383     "column_span": 1,
384     "is_header": false,
385     "row_span": 1,
386     "value": "0"
387   },
388   {
389     "column_span": 1,
390     "is_header": false,
391     "row_span": 1,
392     "value": "3"
393   },
394   {
395     "column_span": 1,
396     "is_header": false,
397     "row_span": 1,
398     "value": "0"
399   }
400 ],
401 [
402   {
403     "column_span": 1,

```

```

404         "is_header": false,
405         "row_span": 1,
406         "value": "2007"
407     },
408     {
409         "column_span": 1,
410         "is_header": false,
411         "row_span": 1,
412         "value": "4"
413     },
414     {
415         "column_span": 1,
416         "is_header": false,
417         "row_span": 1,
418         "value": "0"
419     },
420     {
421         "column_span": 1,
422         "is_header": false,
423         "row_span": 1,
424         "value": "1"
425     },
426     {
427         "column_span": 1,
428         "is_header": false,
429         "row_span": 1,
430         "value": "0"
431     },
432     {
433         "column_span": 1,
434         "is_header": false,
435         "row_span": 1,
436         "value": "5"
437     },
438     {
439         "column_span": 1,
440         "is_header": false,
441         "row_span": 1,
442         "value": "0"
443     }
444 ],
445 [
446     {
447         "column_span": 1,
448         "is_header": false,
449         "row_span": 1,
450         "value": "2008"
451     },
452     {
453         "column_span": 1,
454         "is_header": false,
455         "row_span": 1,
456         "value": "11"
457     },
458     {
459         "column_span": 1,
460         "is_header": false,
461         "row_span": 1,
462         "value": "0"
463     },
464     {
465         "column_span": 1,
466         "is_header": false,
467         "row_span": 1,
468         "value": "1"
469     },
470     {
471         "column_span": 1,
472         "is_header": false,
473         "row_span": 1,
474         "value": "0"
475     },
476     {
477         "column_span": 1,
478         "is_header": false,
479         "row_span": 1,
480         "value": "12"
481     },
482     {
483         "column_span": 1,
484         "is_header": false,
485         "row_span": 1,
486         "value": "0"
487     }
488 ],
489 [
490     {
491         "column_span": 1,
492         "is_header": false,
493         "row_span": 1,
494         "value": "2009"

```

```

495     },
496     {
497         "column_span": 1,
498         "is_header": false,
499         "row_span": 1,
500         "value": "15"
501     },
502     {
503         "column_span": 1,
504         "is_header": false,
505         "row_span": 1,
506         "value": "0"
507     },
508     {
509         "column_span": 1,
510         "is_header": false,
511         "row_span": 1,
512         "value": "2"
513     },
514     {
515         "column_span": 1,
516         "is_header": false,
517         "row_span": 1,
518         "value": "0"
519     },
520     {
521         "column_span": 1,
522         "is_header": false,
523         "row_span": 1,
524         "value": "17"
525     },
526     {
527         "column_span": 1,
528         "is_header": false,
529         "row_span": 1,
530         "value": "0"
531     }
532 ],
533 [
534     {
535         "column_span": 1,
536         "is_header": false,
537         "row_span": 1,
538         "value": "2010"
539     },
540     {
541         "column_span": 1,
542         "is_header": false,
543         "row_span": 1,
544         "value": "Adeccoligaen"
545     },
546     {
547         "column_span": 1,
548         "is_header": false,
549         "row_span": 1,
550         "value": "18"
551     },
552     {
553         "column_span": 1,
554         "is_header": false,
555         "row_span": 1,
556         "value": "0"
557     },
558     {
559         "column_span": 1,
560         "is_header": false,
561         "row_span": 1,
562         "value": "2"
563     },
564     {
565         "column_span": 1,
566         "is_header": false,
567         "row_span": 1,
568         "value": "0"
569     },
570     {
571         "column_span": 1,
572         "is_header": false,
573         "row_span": 1,
574         "value": "20"
575     },
576     {
577         "column_span": 1,
578         "is_header": false,
579         "row_span": 1,
580         "value": "0"
581     }
582 ],
583 [
584     {
585         "column_span": 1,

```

```

586         "is_header": false,
587         "row_span": 1,
588         "value": "2011"
589     },
590     {
591         "column_span": 1,
592         "is_header": false,
593         "row_span": 1,
594         "value": "Tippeligaen"
595     },
596     {
597         "column_span": 1,
598         "is_header": false,
599         "row_span": 1,
600         "value": "22"
601     },
602     {
603         "column_span": 1,
604         "is_header": false,
605         "row_span": 1,
606         "value": "0"
607     },
608     {
609         "column_span": 1,
610         "is_header": false,
611         "row_span": 1,
612         "value": "1"
613     },
614     {
615         "column_span": 1,
616         "is_header": false,
617         "row_span": 1,
618         "value": "0"
619     },
620     {
621         "column_span": 1,
622         "is_header": false,
623         "row_span": 1,
624         "value": "23"
625     },
626     {
627         "column_span": 1,
628         "is_header": false,
629         "row_span": 1,
630         "value": "0"
631     }
632 ],
633 [
634     {
635         "column_span": 1,
636         "is_header": false,
637         "row_span": 1,
638         "value": "2012"
639     },
640     {
641         "column_span": 1,
642         "is_header": false,
643         "row_span": 1,
644         "value": "Syrianska"
645     },
646     {
647         "column_span": 1,
648         "is_header": false,
649         "row_span": 1,
650         "value": "Allsvenskan"
651     },
652     {
653         "column_span": 1,
654         "is_header": false,
655         "row_span": 1,
656         "value": "8"
657     },
658     {
659         "column_span": 1,
660         "is_header": false,
661         "row_span": 1,
662         "value": "0"
663     },
664     {
665         "column_span": 1,
666         "is_header": false,
667         "row_span": 1,
668         "value": "0"
669     },
670     {
671         "column_span": 1,
672         "is_header": false,
673         "row_span": 1,
674         "value": "0"
675     },
676     {

```



```

677         "column_span": 1,
678         "is_header": false,
679         "row_span": 1,
680         "value": "8"
681     },
682     {
683         "column_span": 1,
684         "is_header": false,
685         "row_span": 1,
686         "value": "0"
687     }
688 ],
689 [
690     {
691         "column_span": 1,
692         "is_header": false,
693         "row_span": 1,
694         "value": "2012"
695     },
696     {
697         "column_span": 1,
698         "is_header": false,
699         "row_span": 1,
700         "value": "Lillestrøm"
701     },
702     {
703         "column_span": 1,
704         "is_header": false,
705         "row_span": 4,
706         "value": "Tippeligaen"
707     },
708     {
709         "column_span": 1,
710         "is_header": false,
711         "row_span": 1,
712         "value": "2"
713     },
714     {
715         "column_span": 1,
716         "is_header": false,
717         "row_span": 1,
718         "value": "0"
719     },
720     {
721         "column_span": 1,
722         "is_header": false,
723         "row_span": 1,
724         "value": "0"
725     },
726     {
727         "column_span": 1,
728         "is_header": false,
729         "row_span": 1,
730         "value": "0"
731     },
732     {
733         "column_span": 1,
734         "is_header": false,
735         "row_span": 1,
736         "value": "2"
737     },
738     {
739         "column_span": 1,
740         "is_header": false,
741         "row_span": 1,
742         "value": "0"
743     }
744 ],
745 [
746     {
747         "column_span": 1,
748         "is_header": false,
749         "row_span": 1,
750         "value": "2013"
751     },
752     {
753         "column_span": 1,
754         "is_header": false,
755         "row_span": 1,
756         "value": "Aalesund"
757     },
758     {
759         "column_span": 1,
760         "is_header": false,
761         "row_span": 1,
762         "value": "0"
763     },
764     {
765         "column_span": 1,
766         "is_header": false,
767         "row_span": 1,

```

```

768     "value": "0"
769   },
770   {
771     "column_span": 1,
772     "is_header": false,
773     "row_span": 1,
774     "value": "2"
775   },
776   {
777     "column_span": 1,
778     "is_header": false,
779     "row_span": 1,
780     "value": "0"
781   },
782   {
783     "column_span": 1,
784     "is_header": false,
785     "row_span": 1,
786     "value": "2"
787   },
788   {
789     "column_span": 1,
790     "is_header": false,
791     "row_span": 1,
792     "value": "0"
793   }
794 ],
795 [
796   {
797     "column_span": 1,
798     "is_header": false,
799     "row_span": 1,
800     "value": "2014"
801   },
802   {
803     "column_span": 1,
804     "is_header": false,
805     "row_span": 2,
806     "value": "Bodø/Glimt"
807   },
808   {
809     "column_span": 1,
810     "is_header": false,
811     "row_span": 1,
812     "value": "14"
813   },
814   {
815     "column_span": 1,
816     "is_header": false,
817     "row_span": 1,
818     "value": "0"
819   },
820   {
821     "column_span": 1,
822     "is_header": false,
823     "row_span": 1,
824     "value": "4"
825   },
826   {
827     "column_span": 1,
828     "is_header": false,
829     "row_span": 1,
830     "value": "0"
831   },
832   {
833     "column_span": 1,
834     "is_header": false,
835     "row_span": 1,
836     "value": "18"
837   },
838   {
839     "column_span": 1,
840     "is_header": false,
841     "row_span": 1,
842     "value": "0"
843   }
844 ],
845 [
846   {
847     "column_span": 1,
848     "is_header": false,
849     "row_span": 1,
850     "value": "2015"
851   },
852   {
853     "column_span": 1,
854     "is_header": false,
855     "row_span": 1,
856     "value": "9"
857   },
858   {

```

```

859     "column_span": 1,
860     "is_header": false,
861     "row_span": 1,
862     "value": "0"
863   },
864   {
865     "column_span": 1,
866     "is_header": false,
867     "row_span": 1,
868     "value": "1"
869   },
870   {
871     "column_span": 1,
872     "is_header": false,
873     "row_span": 1,
874     "value": "0"
875   },
876   {
877     "column_span": 1,
878     "is_header": false,
879     "row_span": 1,
880     "value": "10"
881   },
882   {
883     "column_span": 1,
884     "is_header": false,
885     "row_span": 1,
886     "value": "0"
887   }
888 ],
889 [
890   {
891     "column_span": 3,
892     "is_header": true,
893     "row_span": 1,
894     "value": "Career Total"
895   },
896   {
897     "column_span": 1,
898     "is_header": true,
899     "row_span": 1,
900     "value": "108"
901   },
902   {
903     "column_span": 1,
904     "is_header": true,
905     "row_span": 1,
906     "value": "0"
907   },
908   {
909     "column_span": 1,
910     "is_header": true,
911     "row_span": 1,
912     "value": "14"
913   },
914   {
915     "column_span": 1,
916     "is_header": true,
917     "row_span": 1,
918     "value": "0"
919   },
920   {
921     "column_span": 1,
922     "is_header": true,
923     "row_span": 1,
924     "value": "122"
925   },
926   {
927     "column_span": 1,
928     "is_header": true,
929     "row_span": 1,
930     "value": "0"
931   }
932 ]
933 ],
934 "table_page_title": "Lasse Staw",
935 "table_section_text": "",
936 "table_section_title": "Career statistics"
937 },
938 "source_sha256": "4f01b0f2d1a2e944dc9c88cbc4ea7035f7f57569055cd98fd963a9f717fd3a40",
939 "task_family": "highlighted_table_description"
940 }

```

Listing 81: D17: submission/program_listings/build/generated/data/totto__totto-validation-4467.json

```

1 {
2   "dataset_id": "totto",
3   "example_id": "totto-validation-4467",

```

```

4  "language": "en",
5  "output_mode": "one_sentence",
6  "parent_table": [
7    [
8      "World record",
9      "Galina Chistyakova (URS)",
10     "7.52",
11     "Leningrad, Soviet Union",
12     "11 June 1988"
13   ],
14   [
15     "Championship record",
16     "Jackie Joyner-Kersey (USA)",
17     "7.36",
18     "Rome, Italy",
19     "3 September 1987"
20   ],
21   [
22     "World leading",
23     "Tianna Bartoletta (USA)",
24     "7.12",
25     "Eugene, United States",
26     "27 June 2015"
27   ],
28   [
29     "African record",
30     "Chioma Ajunwa (NGR)",
31     "7.12",
32     "Atlanta, GA, United States",
33     "2 August 1996"
34   ],
35   [
36     "Asian record",
37     "Yao Weili (CHN)",
38     "7.01",
39     "Jinan, People's Republic of China",
40     "5 June 1993"
41   ],
42   [
43     "North, Central American and Caribbean record",
44     "Jackie Joyner-Kersey (USA)",
45     "7.49",
46     "New York City, United States",
47     "22 May 1994"
48   ],
49   [
50     "Sestriere, Italy",
51     "31 July 1994"
52   ],
53   [
54     "South American record",
55     "Maurren Higa Maggi (BRA)",
56     "7.26A",
57     "Bogotá, Colombia",
58     "26 June 1999"
59   ],
60   [
61     "European record",
62     "Galina Chistyakova (URS)",
63     "7.52",
64     "Leningrad, Soviet Union",
65     "11 June 1988"
66   ],
67   [
68     "Oceanian record",
69     "Bronwyn Thompson (AUS)",
70     "7.00",
71     "Melbourne, Australia",
72     "7 March 2002"
73   ],
74   [
75     "The following records were established during the competition:"
76   ],
77   [
78     "World Leading",
79     "Tianna Bartoletta (USA)",
80     "7.14",
81     "Beijing, China",
82     "28 August 2015"
83   ]
84 ],
85 "reference_sha256": "b41d8e37a0c759b92fd4173207ba1c235bdd583b30f969d564167dfe6e23b13d",
86 "references": [
87   "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
88 ],
89 "request": "Write exactly one concise sentence describing the highlighted table cells. Do not discuss unrelated cells
90            and do not add headings.",
91 "source_payload": {
92   "highlighted_cells": [
93     11,

```

```

94         0
95     ],
96     [
97         11,
98         1
99     ],
100    [
101        11,
102        2
103    ]
104 ],
105 "table": [
106     [
107         {
108             "column_span": 1,
109             "is_header": false,
110             "row_span": 1,
111             "value": "World record"
112         },
113         {
114             "column_span": 1,
115             "is_header": false,
116             "row_span": 1,
117             "value": "Galina Chistyakova (URS)"
118         },
119         {
120             "column_span": 1,
121             "is_header": false,
122             "row_span": 1,
123             "value": "7.52"
124         },
125         {
126             "column_span": 1,
127             "is_header": false,
128             "row_span": 1,
129             "value": "Leningrad, Soviet Union"
130         },
131         {
132             "column_span": 1,
133             "is_header": false,
134             "row_span": 1,
135             "value": "11 June 1988"
136         }
137     ],
138     [
139         {
140             "column_span": 1,
141             "is_header": false,
142             "row_span": 1,
143             "value": "Championship record"
144         },
145         {
146             "column_span": 1,
147             "is_header": false,
148             "row_span": 1,
149             "value": "Jackie Joyner-Kersey (USA)"
150         },
151         {
152             "column_span": 1,
153             "is_header": false,
154             "row_span": 1,
155             "value": "7.36"
156         },
157         {
158             "column_span": 1,
159             "is_header": false,
160             "row_span": 1,
161             "value": "Rome, Italy"
162         },
163         {
164             "column_span": 1,
165             "is_header": false,
166             "row_span": 1,
167             "value": "3 September 1987"
168         }
169     ],
170     [
171         {
172             "column_span": 1,
173             "is_header": false,
174             "row_span": 1,
175             "value": "World leading"
176         },
177         {
178             "column_span": 1,
179             "is_header": false,
180             "row_span": 1,
181             "value": "Tianna Bartoletta (USA)"
182         },
183         {
184             "column_span": 1,

```

```

185     "is_header": false,
186     "row_span": 1,
187     "value": "7.12"
188 },
189 {
190     "column_span": 1,
191     "is_header": false,
192     "row_span": 1,
193     "value": "Eugene, United States"
194 },
195 {
196     "column_span": 1,
197     "is_header": false,
198     "row_span": 1,
199     "value": "27 June 2015"
200 }
201 ],
202 [
203     {
204         "column_span": 1,
205         "is_header": false,
206         "row_span": 1,
207         "value": "African record"
208     },
209     {
210         "column_span": 1,
211         "is_header": false,
212         "row_span": 1,
213         "value": "Chioma Ajunwa (NGR)"
214     },
215     {
216         "column_span": 1,
217         "is_header": false,
218         "row_span": 1,
219         "value": "7.12"
220     },
221     {
222         "column_span": 1,
223         "is_header": false,
224         "row_span": 1,
225         "value": "Atlanta, GA, United States"
226     },
227     {
228         "column_span": 1,
229         "is_header": false,
230         "row_span": 1,
231         "value": "2 August 1996"
232     }
233 ],
234 [
235     {
236         "column_span": 1,
237         "is_header": false,
238         "row_span": 1,
239         "value": "Asian record"
240     },
241     {
242         "column_span": 1,
243         "is_header": false,
244         "row_span": 1,
245         "value": "Yao Weili (CHN)"
246     },
247     {
248         "column_span": 1,
249         "is_header": false,
250         "row_span": 1,
251         "value": "7.01"
252     },
253     {
254         "column_span": 1,
255         "is_header": false,
256         "row_span": 1,
257         "value": "Jinan, People's Republic of China"
258     },
259     {
260         "column_span": 1,
261         "is_header": false,
262         "row_span": 1,
263         "value": "5 June 1993"
264     }
265 ],
266 [
267     {
268         "column_span": 1,
269         "is_header": false,
270         "row_span": 2,
271         "value": "North, Central American and Caribbean record"
272     },
273     {
274         "column_span": 1,
275         "is_header": false,

```

```

276     "row_span": 2,
277     "value": "Jackie Joyner-Kersey (USA)"
278   },
279   {
280     "column_span": 1,
281     "is_header": false,
282     "row_span": 2,
283     "value": "7.49"
284   },
285   {
286     "column_span": 1,
287     "is_header": false,
288     "row_span": 1,
289     "value": "New York City, United States"
290   },
291   {
292     "column_span": 1,
293     "is_header": false,
294     "row_span": 1,
295     "value": "22 May 1994"
296   }
297 ],
298 [
299   {
300     "column_span": 1,
301     "is_header": false,
302     "row_span": 1,
303     "value": "Sestriere, Italy"
304   },
305   {
306     "column_span": 1,
307     "is_header": false,
308     "row_span": 1,
309     "value": "31 July 1994"
310   }
311 ],
312 [
313   {
314     "column_span": 1,
315     "is_header": false,
316     "row_span": 1,
317     "value": "South American record"
318   },
319   {
320     "column_span": 1,
321     "is_header": false,
322     "row_span": 1,
323     "value": "Maurren Higa Maggi (BRA)"
324   },
325   {
326     "column_span": 1,
327     "is_header": false,
328     "row_span": 1,
329     "value": "7.26A"
330   },
331   {
332     "column_span": 1,
333     "is_header": false,
334     "row_span": 1,
335     "value": "Bogotá, Colombia"
336   },
337   {
338     "column_span": 1,
339     "is_header": false,
340     "row_span": 1,
341     "value": "26 June 1999"
342   }
343 ],
344 [
345   {
346     "column_span": 1,
347     "is_header": false,
348     "row_span": 1,
349     "value": "European record"
350   },
351   {
352     "column_span": 1,
353     "is_header": false,
354     "row_span": 1,
355     "value": "Galina Chistyakova (URS)"
356   },
357   {
358     "column_span": 1,
359     "is_header": false,
360     "row_span": 1,
361     "value": "7.52"
362   },
363   {
364     "column_span": 1,
365     "is_header": false,
366     "row_span": 1,

```

```

367     "value": "Leningrad, Soviet Union"
368   },
369   {
370     "column_span": 1,
371     "is_header": false,
372     "row_span": 1,
373     "value": "11 June 1988"
374   }
375 ],
376 [
377   {
378     "column_span": 1,
379     "is_header": false,
380     "row_span": 1,
381     "value": "Oceanian record"
382   },
383   {
384     "column_span": 1,
385     "is_header": false,
386     "row_span": 1,
387     "value": "Bronwyn Thompson (AUS)"
388   },
389   {
390     "column_span": 1,
391     "is_header": false,
392     "row_span": 1,
393     "value": "7.00"
394   },
395   {
396     "column_span": 1,
397     "is_header": false,
398     "row_span": 1,
399     "value": "Melbourne, Australia"
400   },
401   {
402     "column_span": 1,
403     "is_header": false,
404     "row_span": 1,
405     "value": "7 March 2002"
406   }
407 ],
408 [
409   {
410     "column_span": 5,
411     "is_header": true,
412     "row_span": 1,
413     "value": "The following records were established during the competition:"
414   }
415 ],
416 [
417   {
418     "column_span": 1,
419     "is_header": false,
420     "row_span": 1,
421     "value": "World Leading"
422   },
423   {
424     "column_span": 1,
425     "is_header": false,
426     "row_span": 1,
427     "value": "Tianna Bartoletta (USA)"
428   },
429   {
430     "column_span": 1,
431     "is_header": false,
432     "row_span": 1,
433     "value": "7.14"
434   },
435   {
436     "column_span": 1,
437     "is_header": false,
438     "row_span": 1,
439     "value": "Beijing, China"
440   },
441   {
442     "column_span": 1,
443     "is_header": false,
444     "row_span": 1,
445     "value": "28 August 2015"
446   }
447 ]
448 ],
449 "table_page_title": "2015 World Championships in Athletics – Women's long jump",
450 "table_section_text": "Prior to the competition, the established records were as follows.",
451 "table_section_title": "Records"
452 },
453 "source_sha256": "aa436856c0ebc8aea944693e765472828cbdc6aafb670970a5c15a03f8df6d1d",
454 "task_family": "highlighted_table_description"
455 }

```


Listing 82: D18: submission/program_listings/build/generated/data/totto__totto-validation-6067.json

```

1 {
2   "dataset_id": "totto",
3   "example_id": "totto-validation-6067",
4   "language": "en",
5   "output_mode": "one_sentence",
6   "parent_table": [
7     [
8       "Year",
9       "Peruvian-born population",
10      "Other data"
11    ],
12    [
13      "2001",
14      "",
15      "26,831"
16    ],
17    [
18      "2006",
19      "",
20      "66.506"
21    ],
22    [
23      "2007",
24      "",
25      "70.755"
26    ],
27    [
28      "2008",
29      "",
30      "77.629"
31    ],
32    [
33      "2009",
34      "",
35      "87.747"
36    ],
37    [
38      "2010",
39      "225,795",
40      "98.603"
41    ],
42    [
43      "2011",
44      "246,908",
45      ""
46    ],
47    [
48      "2012",
49      "",
50      ""
51    ],
52    [
53      "2013",
54      "",
55      ""
56    ]
57  ],
58   "reference_sha256": "bddc524b00580d101af2646170f1309589439cffb40ef1626fc3a61fb14e3f83",
59   "references": [
60     "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
61   ],
62   "request": "Write exactly one concise sentence describing the highlighted table cells. Do not discuss unrelated cells and do not add headings.",
63   "source_payload": {
64     "highlighted_cells": [
65       [
66         7,
67         0
68       ],
69       [
70         7,
71         1
72       ]
73     ],
74     "table": [
75       [
76         {
77           "column_span": 1,
78           "is_header": true,
79           "row_span": 1,
80           "value": "Year"
81         },
82         {
83           "column_span": 1,
84           "is_header": true,
85           "row_span": 1,
86           "value": "Peruvian-born population"
87         },
88         {

```

```

89         "column_span": 1,
90         "is_header": true,
91         "row_span": 1,
92         "value": "Other data"
93     },
94 ],
95 [
96     {
97         "column_span": 1,
98         "is_header": false,
99         "row_span": 1,
100        "value": "2001"
101    },
102    {
103        "column_span": 1,
104        "is_header": false,
105        "row_span": 1,
106        "value": ""
107    },
108    {
109        "column_span": 1,
110        "is_header": false,
111        "row_span": 1,
112        "value": "26,831"
113    }
114 ],
115 [
116     {
117         "column_span": 1,
118         "is_header": false,
119         "row_span": 1,
120         "value": "2006"
121     },
122     {
123         "column_span": 1,
124         "is_header": false,
125         "row_span": 1,
126         "value": ""
127     },
128     {
129         "column_span": 1,
130         "is_header": false,
131         "row_span": 1,
132         "value": "66.506"
133     }
134 ],
135 [
136     {
137         "column_span": 1,
138         "is_header": false,
139         "row_span": 1,
140         "value": "2007"
141     },
142     {
143         "column_span": 1,
144         "is_header": false,
145         "row_span": 1,
146         "value": ""
147     },
148     {
149         "column_span": 1,
150         "is_header": false,
151         "row_span": 1,
152         "value": "70.755"
153     }
154 ],
155 [
156     {
157         "column_span": 1,
158         "is_header": false,
159         "row_span": 1,
160         "value": "2008"
161     },
162     {
163         "column_span": 1,
164         "is_header": false,
165         "row_span": 1,
166         "value": ""
167     },
168     {
169         "column_span": 1,
170         "is_header": false,
171         "row_span": 1,
172         "value": "77.629"
173     }
174 ],
175 [
176     {
177         "column_span": 1,
178         "is_header": false,
179         "row_span": 1,

```

```

180     "value": "2009"
181   },
182   {
183     "column_span": 1,
184     "is_header": false,
185     "row_span": 1,
186     "value": ""
187   },
188   {
189     "column_span": 1,
190     "is_header": false,
191     "row_span": 1,
192     "value": "87.747"
193   }
194 ],
195 [
196   {
197     "column_span": 1,
198     "is_header": false,
199     "row_span": 1,
200     "value": "2010"
201   },
202   {
203     "column_span": 1,
204     "is_header": false,
205     "row_span": 1,
206     "value": "225,795"
207   },
208   {
209     "column_span": 1,
210     "is_header": false,
211     "row_span": 1,
212     "value": "98.603"
213   }
214 ],
215 [
216   {
217     "column_span": 1,
218     "is_header": false,
219     "row_span": 1,
220     "value": "2011"
221   },
222   {
223     "column_span": 1,
224     "is_header": false,
225     "row_span": 1,
226     "value": "246,908"
227   },
228   {
229     "column_span": 1,
230     "is_header": false,
231     "row_span": 1,
232     "value": ""
233   }
234 ],
235 [
236   {
237     "column_span": 1,
238     "is_header": false,
239     "row_span": 1,
240     "value": "2012"
241   },
242   {
243     "column_span": 1,
244     "is_header": false,
245     "row_span": 1,
246     "value": ""
247   },
248   {
249     "column_span": 1,
250     "is_header": false,
251     "row_span": 1,
252     "value": ""
253   }
254 ],
255 [
256   {
257     "column_span": 1,
258     "is_header": false,
259     "row_span": 1,
260     "value": "2013"
261   },
262   {
263     "column_span": 1,
264     "is_header": false,
265     "row_span": 1,
266     "value": ""
267   },
268   {
269     "column_span": 1,
270     "is_header": false,

```

```

271         "row_span": 1,
272         "value": ""
273     }
274 ]
275 ],
276 "table_page_title": "Peruvians in Italy",
277 "table_section_text": "",
278 "table_section_title": "History"
279 },
280 "source_sha256": "93ea1c14d25c908009aadf5d17515d948625a4673384762f1e058b9319db1c05",
281 "task_family": "highlighted_table_description"
282 }

```

Listing 83: D19: submission/program_listings/build/generated/data/totto__totto-validation-839.json

```

1 {
2   "dataset_id": "totto",
3   "example_id": "totto-validation-839",
4   "language": "en",
5   "output_mode": "one_sentence",
6   "parent_table": [
7     [
8       "Year",
9       "Team",
10      "Overall",
11      "Conference",
12      "Standing",
13      "Bowl/playoffs"
14    ],
15    [
16      "Maine Black Bears (Maine Intercollegiate Athletic Association) (1900)"
17    ],
18    [
19      "1900",
20      "Maine",
21      "-44",
22      "",
23      "",
24      ""
25    ],
26    [
27      "Maine:",
28      "-44",
29      "",
30      ""
31    ],
32    [
33      "Total:",
34      "-44",
35      ""
36    ]
37  ],
38  "reference_sha256": "4bc3fecf5a648f242388bf1ed91ce31c75f2b90e15263b2e563f0f18efb5cbf1",
39  "references": [
40    "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
41  ],
42  "request": "Write exactly one concise sentence describing the highlighted table cells. Do not discuss unrelated cells and do not add headings.",
43  "source_payload": {
44    "highlighted_cells": [
45      [
46        1,
47        0
48      ],
49      [
50        4,
51        1
52      ]
53    ],
54    "table": [
55      [
56        {
57          "column_span": 1,
58          "is_header": true,
59          "row_span": 1,
60          "value": "Year"
61        },
62        {
63          "column_span": 1,
64          "is_header": true,
65          "row_span": 1,
66          "value": "Team"
67        },
68        {
69          "column_span": 1,
70          "is_header": true,
71          "row_span": 1,
72          "value": "Overall"
73        }

```

```

74     {
75         "column_span": 1,
76         "is_header": true,
77         "row_span": 1,
78         "value": "Conference"
79     },
80     {
81         "column_span": 1,
82         "is_header": true,
83         "row_span": 1,
84         "value": "Standing"
85     },
86     {
87         "column_span": 1,
88         "is_header": true,
89         "row_span": 1,
90         "value": "Bowl/playoffs"
91     }
92 ],
93 [
94     {
95         "column_span": 9,
96         "is_header": false,
97         "row_span": 1,
98         "value": "Maine Black Bears (Maine Intercollegiate Athletic Association) (1900)"
99     }
100 ],
101 [
102     {
103         "column_span": 1,
104         "is_header": false,
105         "row_span": 1,
106         "value": "1900"
107     },
108     {
109         "column_span": 1,
110         "is_header": false,
111         "row_span": 1,
112         "value": "Maine"
113     },
114     {
115         "column_span": 1,
116         "is_header": false,
117         "row_span": 1,
118         "value": "-44"
119     },
120     {
121         "column_span": 1,
122         "is_header": false,
123         "row_span": 1,
124         "value": ""
125     },
126     {
127         "column_span": 1,
128         "is_header": false,
129         "row_span": 1,
130         "value": ""
131     },
132     {
133         "column_span": 1,
134         "is_header": false,
135         "row_span": 1,
136         "value": ""
137     }
138 ],
139 [
140     {
141         "column_span": 2,
142         "is_header": false,
143         "row_span": 1,
144         "value": "Maine:"
145     },
146     {
147         "column_span": 1,
148         "is_header": false,
149         "row_span": 1,
150         "value": "-44"
151     },
152     {
153         "column_span": 1,
154         "is_header": false,
155         "row_span": 1,
156         "value": ""
157     },
158     {
159         "column_span": 5,
160         "is_header": false,
161         "row_span": 1,
162         "value": ""
163     }
164 ],

```

```

165     [
166     {
167         "column_span": 2,
168         "is_header": false,
169         "row_span": 1,
170         "value": "Total:"
171     },
172     {
173         "column_span": 1,
174         "is_header": false,
175         "row_span": 1,
176         "value": "-44"
177     },
178     {
179         "column_span": 7,
180         "is_header": false,
181         "row_span": 1,
182         "value": ""
183     }
184     ]
185 },
186 "table_page_title": "Ernest Burton (American football)",
187 "table_section_text": "",
188 "table_section_title": "Head coaching record"
189 },
190 "source_sha256": "09a9a840e0ecbe6e8cc23300cef1fc925e8577e9d8c77bcd996240dff713676",
191 "task_family": "highlighted_table_description"
192 }

```

Listing 84: D20: submission/program_listings/build/generated/data/totto__totto-validation-912.json

```

1  {
2  "dataset_id": "totto",
3  "example_id": "totto-validation-912",
4  "language": "en",
5  "output_mode": "one_sentence",
6  "parent_table": [
7      [
8          "Year",
9          "Team",
10         "GP",
11         "Carries",
12         "Yards",
13         "TDs",
14         "Avg",
15         "Long",
16         "",
17         "Rec",
18         "Yards",
19         "TDs"
20     ],
21     [
22         "2013",
23         "MTL",
24         "12",
25         "55",
26         "342",
27         "3",
28         "6.2",
29         "20",
30         "14",
31         "156",
32         "0"
33     ],
34     [
35         "2014",
36         "MTL",
37         "12",
38         "96",
39         "500",
40         "1",
41         "5.2",
42         "24",
43         "15",
44         "190",
45         "0"
46     ],
47     [
48         "2015",
49         "MTL",
50         "15",
51         "180",
52         "1,059",
53         "5",
54         "5.9",
55         "54",
56         "43",
57         "334",
58         "2"

```

```

59     ],
60     [
61         "2016",
62         "MTL",
63         "7",
64         "74",
65         "412",
66         "0",
67         "5.6",
68         "27",
69         "27",
70         "206",
71         "0"
72     ],
73     [
74         "2017",
75         "MTL",
76         "14",
77         "152",
78         "843",
79         "5",
80         "5.5",
81         "43",
82         "44",
83         "312",
84         "1"
85     ],
86     [
87         "2018",
88         "MTL",
89         "9",
90         "86",
91         "417",
92         "1",
93         "4.8",
94         "44",
95         "30",
96         "309",
97         "0"
98     ],
99     [
100        "BC",
101        "4",
102        "55",
103        "268",
104        "2",
105        "4.9",
106        "31",
107        "5",
108        "32",
109        "0"
110    ],
111    [
112        "Total",
113        "73",
114        "698",
115        "3,841",
116        "17",
117        "5.5",
118        "54",
119        "178",
120        "1,539",
121        "3"
122    ]
123 ],
124 "reference_sha256": "85e3d857aa14ca068b9257722fc7a85a211536410554c00103c0b93772d8aaaa",
125 "references": [
126     "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
127 ],
128 "request": "Write exactly one concise sentence describing the highlighted table cells. Do not discuss unrelated cells
and do not add headings.",
129 "source_payload": {
130     "highlighted_cells": [
131         [
132             4,
133             0
134         ],
135         [
136             4,
137             2
138         ],
139         [
140             4,
141             3
142         ],
143         [
144             4,
145             4
146         ],
147         [
148             4,

```

```

149         6
150     ]
151 ],
152 "table": [
153     [
154         {
155             "column_span": 1,
156             "is_header": true,
157             "row_span": 1,
158             "value": "Year"
159         },
160         {
161             "column_span": 1,
162             "is_header": true,
163             "row_span": 1,
164             "value": "Team"
165         },
166         {
167             "column_span": 1,
168             "is_header": true,
169             "row_span": 1,
170             "value": "GP"
171         },
172         {
173             "column_span": 1,
174             "is_header": true,
175             "row_span": 1,
176             "value": "Carries"
177         },
178         {
179             "column_span": 1,
180             "is_header": true,
181             "row_span": 1,
182             "value": "Yards"
183         },
184         {
185             "column_span": 1,
186             "is_header": true,
187             "row_span": 1,
188             "value": "TDs"
189         },
190         {
191             "column_span": 1,
192             "is_header": true,
193             "row_span": 1,
194             "value": "Avg"
195         },
196         {
197             "column_span": 1,
198             "is_header": true,
199             "row_span": 1,
200             "value": "Long"
201         },
202         {
203             "column_span": 1,
204             "is_header": true,
205             "row_span": 9,
206             "value": ""
207         },
208         {
209             "column_span": 1,
210             "is_header": true,
211             "row_span": 1,
212             "value": "Rec"
213         },
214         {
215             "column_span": 1,
216             "is_header": true,
217             "row_span": 1,
218             "value": "Yards"
219         },
220         {
221             "column_span": 1,
222             "is_header": true,
223             "row_span": 1,
224             "value": "TDs"
225         }
226     ],
227     [
228         {
229             "column_span": 1,
230             "is_header": false,
231             "row_span": 1,
232             "value": "2013"
233         },
234         {
235             "column_span": 1,
236             "is_header": false,
237             "row_span": 1,
238             "value": "MTL"
239         }

```



```

240     {
241         "column_span": 1,
242         "is_header": false,
243         "row_span": 1,
244         "value": "12"
245     },
246     {
247         "column_span": 1,
248         "is_header": false,
249         "row_span": 1,
250         "value": "55"
251     },
252     {
253         "column_span": 1,
254         "is_header": false,
255         "row_span": 1,
256         "value": "342"
257     },
258     {
259         "column_span": 1,
260         "is_header": false,
261         "row_span": 1,
262         "value": "3"
263     },
264     {
265         "column_span": 1,
266         "is_header": false,
267         "row_span": 1,
268         "value": "6.2"
269     },
270     {
271         "column_span": 1,
272         "is_header": false,
273         "row_span": 1,
274         "value": "20"
275     },
276     {
277         "column_span": 1,
278         "is_header": false,
279         "row_span": 1,
280         "value": "14"
281     },
282     {
283         "column_span": 1,
284         "is_header": false,
285         "row_span": 1,
286         "value": "156"
287     },
288     {
289         "column_span": 1,
290         "is_header": false,
291         "row_span": 1,
292         "value": "0"
293     }
294 ],
295 [
296     {
297         "column_span": 1,
298         "is_header": false,
299         "row_span": 1,
300         "value": "2014"
301     },
302     {
303         "column_span": 1,
304         "is_header": false,
305         "row_span": 1,
306         "value": "MTL"
307     },
308     {
309         "column_span": 1,
310         "is_header": false,
311         "row_span": 1,
312         "value": "12"
313     },
314     {
315         "column_span": 1,
316         "is_header": false,
317         "row_span": 1,
318         "value": "96"
319     },
320     {
321         "column_span": 1,
322         "is_header": false,
323         "row_span": 1,
324         "value": "500"
325     },
326     {
327         "column_span": 1,
328         "is_header": false,
329         "row_span": 1,
330         "value": "1"

```

```

331     },
332     {
333         "column_span": 1,
334         "is_header": false,
335         "row_span": 1,
336         "value": "5.2"
337     },
338     {
339         "column_span": 1,
340         "is_header": false,
341         "row_span": 1,
342         "value": "24"
343     },
344     {
345         "column_span": 1,
346         "is_header": false,
347         "row_span": 1,
348         "value": "15"
349     },
350     {
351         "column_span": 1,
352         "is_header": false,
353         "row_span": 1,
354         "value": "190"
355     },
356     {
357         "column_span": 1,
358         "is_header": false,
359         "row_span": 1,
360         "value": "0"
361     }
362 ],
363 [
364     {
365         "column_span": 1,
366         "is_header": false,
367         "row_span": 1,
368         "value": "2015"
369     },
370     {
371         "column_span": 1,
372         "is_header": false,
373         "row_span": 1,
374         "value": "MTL"
375     },
376     {
377         "column_span": 1,
378         "is_header": false,
379         "row_span": 1,
380         "value": "15"
381     },
382     {
383         "column_span": 1,
384         "is_header": false,
385         "row_span": 1,
386         "value": "180"
387     },
388     {
389         "column_span": 1,
390         "is_header": false,
391         "row_span": 1,
392         "value": "1,059"
393     },
394     {
395         "column_span": 1,
396         "is_header": false,
397         "row_span": 1,
398         "value": "5"
399     },
400     {
401         "column_span": 1,
402         "is_header": false,
403         "row_span": 1,
404         "value": "5.9"
405     },
406     {
407         "column_span": 1,
408         "is_header": false,
409         "row_span": 1,
410         "value": "54"
411     },
412     {
413         "column_span": 1,
414         "is_header": false,
415         "row_span": 1,
416         "value": "43"
417     },
418     {
419         "column_span": 1,
420         "is_header": false,
421         "row_span": 1,

```

```

422     "value": "334"
423   },
424   {
425     "column_span": 1,
426     "is_header": false,
427     "row_span": 1,
428     "value": "2"
429   }
430 ],
431 [
432   {
433     "column_span": 1,
434     "is_header": false,
435     "row_span": 1,
436     "value": "2016"
437   },
438   {
439     "column_span": 1,
440     "is_header": false,
441     "row_span": 1,
442     "value": "MTL"
443   },
444   {
445     "column_span": 1,
446     "is_header": false,
447     "row_span": 1,
448     "value": "7"
449   },
450   {
451     "column_span": 1,
452     "is_header": false,
453     "row_span": 1,
454     "value": "74"
455   },
456   {
457     "column_span": 1,
458     "is_header": false,
459     "row_span": 1,
460     "value": "412"
461   },
462   {
463     "column_span": 1,
464     "is_header": false,
465     "row_span": 1,
466     "value": "0"
467   },
468   {
469     "column_span": 1,
470     "is_header": false,
471     "row_span": 1,
472     "value": "5.6"
473   },
474   {
475     "column_span": 1,
476     "is_header": false,
477     "row_span": 1,
478     "value": "27"
479   },
480   {
481     "column_span": 1,
482     "is_header": false,
483     "row_span": 1,
484     "value": "27"
485   },
486   {
487     "column_span": 1,
488     "is_header": false,
489     "row_span": 1,
490     "value": "206"
491   },
492   {
493     "column_span": 1,
494     "is_header": false,
495     "row_span": 1,
496     "value": "0"
497   }
498 ],
499 [
500   {
501     "column_span": 1,
502     "is_header": false,
503     "row_span": 1,
504     "value": "2017"
505   },
506   {
507     "column_span": 1,
508     "is_header": false,
509     "row_span": 1,
510     "value": "MTL"
511   },
512   {

```

```

513         "column_span": 1,
514         "is_header": false,
515         "row_span": 1,
516         "value": "14"
517     },
518     {
519         "column_span": 1,
520         "is_header": false,
521         "row_span": 1,
522         "value": "152"
523     },
524     {
525         "column_span": 1,
526         "is_header": false,
527         "row_span": 1,
528         "value": "843"
529     },
530     {
531         "column_span": 1,
532         "is_header": false,
533         "row_span": 1,
534         "value": "5"
535     },
536     {
537         "column_span": 1,
538         "is_header": false,
539         "row_span": 1,
540         "value": "5.5"
541     },
542     {
543         "column_span": 1,
544         "is_header": false,
545         "row_span": 1,
546         "value": "43"
547     },
548     {
549         "column_span": 1,
550         "is_header": false,
551         "row_span": 1,
552         "value": "44"
553     },
554     {
555         "column_span": 1,
556         "is_header": false,
557         "row_span": 1,
558         "value": "312"
559     },
560     {
561         "column_span": 1,
562         "is_header": false,
563         "row_span": 1,
564         "value": "1"
565     }
566 ],
567 [
568     {
569         "column_span": 1,
570         "is_header": false,
571         "row_span": 2,
572         "value": "2018"
573     },
574     {
575         "column_span": 1,
576         "is_header": false,
577         "row_span": 1,
578         "value": "MTL"
579     },
580     {
581         "column_span": 1,
582         "is_header": false,
583         "row_span": 1,
584         "value": "9"
585     },
586     {
587         "column_span": 1,
588         "is_header": false,
589         "row_span": 1,
590         "value": "86"
591     },
592     {
593         "column_span": 1,
594         "is_header": false,
595         "row_span": 1,
596         "value": "417"
597     },
598     {
599         "column_span": 1,
600         "is_header": false,
601         "row_span": 1,
602         "value": "1"
603     },

```

```

604     {
605         "column_span": 1,
606         "is_header": false,
607         "row_span": 1,
608         "value": "4.8"
609     },
610     {
611         "column_span": 1,
612         "is_header": false,
613         "row_span": 1,
614         "value": "44"
615     },
616     {
617         "column_span": 1,
618         "is_header": false,
619         "row_span": 1,
620         "value": "30"
621     },
622     {
623         "column_span": 1,
624         "is_header": false,
625         "row_span": 1,
626         "value": "309"
627     },
628     {
629         "column_span": 1,
630         "is_header": false,
631         "row_span": 1,
632         "value": "0"
633     }
634 ],
635 [
636     {
637         "column_span": 1,
638         "is_header": false,
639         "row_span": 1,
640         "value": "BC"
641     },
642     {
643         "column_span": 1,
644         "is_header": false,
645         "row_span": 1,
646         "value": "4"
647     },
648     {
649         "column_span": 1,
650         "is_header": false,
651         "row_span": 1,
652         "value": "55"
653     },
654     {
655         "column_span": 1,
656         "is_header": false,
657         "row_span": 1,
658         "value": "268"
659     },
660     {
661         "column_span": 1,
662         "is_header": false,
663         "row_span": 1,
664         "value": "2"
665     },
666     {
667         "column_span": 1,
668         "is_header": false,
669         "row_span": 1,
670         "value": "4.9"
671     },
672     {
673         "column_span": 1,
674         "is_header": false,
675         "row_span": 1,
676         "value": "31"
677     },
678     {
679         "column_span": 1,
680         "is_header": false,
681         "row_span": 1,
682         "value": "5"
683     },
684     {
685         "column_span": 1,
686         "is_header": false,
687         "row_span": 1,
688         "value": "32"
689     },
690     {
691         "column_span": 1,
692         "is_header": false,
693         "row_span": 1,
694         "value": "0"

```

```

695     }
696   ],
697   [
698     {
699       "column_span": 2,
700       "is_header": false,
701       "row_span": 1,
702       "value": "Total"
703     },
704     {
705       "column_span": 1,
706       "is_header": false,
707       "row_span": 1,
708       "value": "73"
709     },
710     {
711       "column_span": 1,
712       "is_header": false,
713       "row_span": 1,
714       "value": "698"
715     },
716     {
717       "column_span": 1,
718       "is_header": false,
719       "row_span": 1,
720       "value": "3,841"
721     },
722     {
723       "column_span": 1,
724       "is_header": false,
725       "row_span": 1,
726       "value": "17"
727     },
728     {
729       "column_span": 1,
730       "is_header": false,
731       "row_span": 1,
732       "value": "5.5"
733     },
734     {
735       "column_span": 1,
736       "is_header": false,
737       "row_span": 1,
738       "value": "54"
739     },
740     {
741       "column_span": 1,
742       "is_header": false,
743       "row_span": 1,
744       "value": "178"
745     },
746     {
747       "column_span": 1,
748       "is_header": false,
749       "row_span": 1,
750       "value": "1,539"
751     },
752     {
753       "column_span": 1,
754       "is_header": false,
755       "row_span": 1,
756       "value": "3"
757     }
758   ]
759 ],
760 "table_page_title": "Tyrell Sutton",
761 "table_section_text": "",
762 "table_section_title": "CFL Statistics"
763 },
764 "source_sha256": "02076943868e187e53172594cfe40e876163ca4a1385a814147a0a1655d4b5aa",
765 "task_family": "highlighted_table_description"
766 }

```

Listing 85: D21: submission/program_listings/build/generated/data/web_nlg__web_nlg_en-test-1209.json

```

1 {
2   "dataset_id": "web_nlg",
3   "example_id": "web_nlg_en-test-1209",
4   "language": "en",
5   "output_mode": "short_text",
6   "parent_table": null,
7   "reference_sha256": "2c8538c423f63fce8e472bfb22f4b2c28bc8595bb9c2016f9d0728cde2b8995e",
8   "references": [
9     "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
10  ],
11  "request": "Express all and only the supplied triples as short, coherent natural language. Do not add unsupported facts.",
12  "source_payload": {

```

```

13   "category": "Athlete",
14   "triples": [
15     "Piotr_Hallmann | birthPlace | Gdynia,_Poland",
16     "Gdynia,_Poland | timeZone | Central_European_Time",
17     "Gdynia,_Poland | timeZone | Central_European_Summer_Time",
18     "Piotr_Hallmann | height | 175.26"
19   ]
20 },
21 "source_sha256": "a413a081e355b67874e04f93ab59ee7b33bd20c885e0574a724090f7fe1a163b",
22 "task_family": "triple_verbalisation"
23 }

```

Listing 86: D22: submission/program_listings/build/generated/data/web_nlg__web_nlg_en-test-1330.json

```

1 {
2   "dataset_id": "web_nlg",
3   "example_id": "web_nlg_en-test-1330",
4   "language": "en",
5   "output_mode": "short_text",
6   "parent_table": null,
7   "reference_sha256": "df9c74f2b730dd5fa43fcdd32b773b9f320e7a007f1424599601776a78857ab8",
8   "references": [
9     "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
10  ],
11  "request": "Express all and only the supplied triples as short, coherent natural language. Do not add unsupported facts.",
12  "source_payload": {
13    "category": "Scientist",
14    "triples": [
15      "Brandon_Carter | doctoralAdvisor | Dennis_William_Sciama",
16      "Brandon_Carter | birthPlace | England",
17      "Brandon_Carter | birthDate | 1942-01-01",
18      "Brandon_Carter | almaMater | University_of_Cambridge",
19      "Brandon_Carter | knownFor | Doomsday_argument"
20    ]
21  },
22  "source_sha256": "0dc9960da718f111cf6eb450519a016a22ee640015ee8bec715c72ba8d05a6b8",
23  "task_family": "triple_verbalisation"
24 }

```

Listing 87: D23: submission/program_listings/build/generated/data/web_nlg__web_nlg_en-test-1466.json

```

1 {
2   "dataset_id": "web_nlg",
3   "example_id": "web_nlg_en-test-1466",
4   "language": "en",
5   "output_mode": "short_text",
6   "parent_table": null,
7   "reference_sha256": "0944315b3912e63a7949969411805989726f2e28d27211c704460c9fd1606042",
8   "references": [
9     "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
10  ],
11  "request": "Express all and only the supplied triples as short, coherent natural language. Do not add unsupported facts.",
12  "source_payload": {
13    "category": "City",
14    "triples": [
15      "Ciudad_Ayala | timeZone | Pacific_Daylight_Time"
16    ]
17  },
18  "source_sha256": "ceda5e34141ed6d3857c4ea8e2e05f21e7098cf4f2c6fe0eb35471edeeaf60ee",
19  "task_family": "triple_verbalisation"
20 }

```

Listing 88: D24: submission/program_listings/build/generated/data/web_nlg__web_nlg_en-test-859.json

```

1 {
2   "dataset_id": "web_nlg",
3   "example_id": "web_nlg_en-test-859",
4   "language": "en",
5   "output_mode": "short_text",
6   "parent_table": null,
7   "reference_sha256": "8d32ebce4aaf2ab161d1495559f07b3f74fa982abc751bf28c8bb264c8d0c9aa",
8   "references": [
9     "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
10  ],
11  "request": "Express all and only the supplied triples as short, coherent natural language. Do not add unsupported facts.",
12  "source_payload": {
13    "category": "MeanOfTransportation",
14    "triples": [
15      "Pontiac_Rageous | assembly | Detroit"
16    ]
17  },

```

```

18 "source_sha256": "850b090ceaabc5f7faf4782b25926cf343176e102230332065c2f88daff7e34",
19 "task_family": "triple_verbalisation"
20 }

```

Listing 89: D25: submission/program_listings/build/generated/data/web_nlg__web_nlg_en-test-864.json

```

1 {
2   "dataset_id": "web_nlg",
3   "example_id": "web_nlg_en-test-864",
4   "language": "en",
5   "output_mode": "short_text",
6   "parent_table": null,
7   "reference_sha256": "94f9b0173d9ce29a5afa00e4d7523b95eb7a3b8e9981398b9cd1b2f0c2a36b74",
8   "references": [
9     "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
10  ],
11  "request": "Express all and only the supplied triples as short, coherent natural language. Do not add unsupported facts.",
12  "source_payload": {
13    "category": "Athlete",
14    "triples": [
15      "Akeem_Ayers | debutTeam | Tennessee_Titans",
16      "Akeem_Ayers | activeYearsStartYear | 2011"
17    ]
18  },
19  "source_sha256": "3d589eb09ad48805c2de71a9c53f355e9c0dd090d7ad7330a531db2dd1163f66",
20  "task_family": "triple_verbalisation"
21 }

```


5 Evaluation Result Listings

Paired Full System/Baseline outputs and the principal protected-holdout result tables.

Listing 90: R01: submission/program_listings/build/generated/protected_holdout_25_paired_outputs.json

```

1  [
2  {
3    "baseline": {
4      "generated_text": "Elliot See attended the University of Texas at Austin, was selected by NASA in 1962, and died
5      in St. Louis.",
6      "primary_evaluation_eligible": true,
7      "release_status": null,
8      "writer_mode": null
9    },
10   "dataset_id": "dart",
11   "example_id": "dart-test-1791",
12   "full_system": {
13     "generated_text": "Elliot See attended the University of Texas at Austin, died in St.\nLouis, and was selected by
14     NASA in 1962.\n",
15     "primary_evaluation_eligible": true,
16     "release_status": "approved_with_warnings",
17     "writer_mode": "llm_writer"
18   },
19   "references": [
20     "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
21   ],
22   "request": "Understand the supplied data and report its strongest supported findings.",
23   "task_family": "triple_verbalisation"
24 },
25 {
26   "baseline": {
27     "generated_text": "The supplied data supports three factual findings about the book *A Severed Wasp*: it has 388
28     pages, its OCLC number is 8805735, and its media type is hardcover.",
29     "primary_evaluation_eligible": true,
30     "release_status": null,
31     "writer_mode": null
32   },
33   "dataset_id": "dart",
34   "example_id": "dart-test-1805",
35   "full_system": {
36     "generated_text": "A Severed Wasp has 388 pages, OCLC number 8805735, and a hardcover media type.\n",
37     "primary_evaluation_eligible": true,
38     "release_status": "approved_with_warnings",
39     "writer_mode": "llm_writer"
40   },
41   "references": [
42     "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
43   ],
44   "request": "Understand the supplied data and report its strongest supported findings.",
45   "task_family": "triple_verbalisation"
46 },
47 {
48   "baseline": {
49     "generated_text": "The strongest supported findings are that A.C. Lumezzane's full name is \"Associazione Calcio
50     Lumezzane SpA,\" it competes in the \"Lega Pro/A\" league, and it has 4,150 members.",
51     "primary_evaluation_eligible": true,
52     "release_status": null,
53     "writer_mode": null
54   },
55   "dataset_id": "dart",
56   "example_id": "dart-test-1828",
57   "full_system": {
58     "generated_text": "A.C. Lumezzane, whose full name is Associazione Calcio Lumezzane SpA, competes in Lega Pro/A
59     and has 4150 members.\n",
60     "primary_evaluation_eligible": true,
61     "release_status": "approved_with_warnings",
62     "writer_mode": "llm_writer"
63   },
64   "references": [
65     "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
66   ],
67   "request": "Understand the supplied data and report its strongest supported findings.",
68   "task_family": "triple_verbalisation"
69 },
70 {
71   "baseline": {
72     "generated_text": "Al Asad Airbase is operated by the United States Air Force (USAF). The USAF is associated with
73     specific aircraft types: the Lockheed AC-130 as its attack aircraft, the Boeing C-17 Globemaster III as its
74     transport aircraft, and the General Dynamics F-16 Fighting Falcon as its fighter aircraft. The data also links the
75     USAF to the 1986 United States bombing of Libya.",
76     "primary_evaluation_eligible": true,
77     "release_status": null,
78     "writer_mode": null
79   },
80   "dataset_id": "dart",
81   "example_id": "dart-test-2278",
82   "full_system": {
83     "generated_text": "Al Asad Airbase is operated by the United States Air Force, whose aircraft include the

```

```

76     Lockheed AC-130 attack aircraft, Boeing C-17 Globemaster III transport aircraft, and General Dynamics F-16 Fighting
77     Falcon fighter aircraft, and whose battles include the 1986 United States bombing of Libya.\n",
78     "primary_evaluation_eligible": true,
79     "release_status": "approved",
80     "writer_mode": "llm_writer"
81 },
82 "references": [
83     "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
84 ],
85 "request": "Understand the supplied data and report its strongest supported findings.",
86 "task_family": "triple_verbalisation"
87 },
88 {
89     "baseline": {
90         "generated_text": "The Vaults is a family-friendly pub with moderate prices, serving Italian food in a riverside
91         area. It has an average customer rating and is located near Rainbow Vegetarian Café.",
92         "primary_evaluation_eligible": true,
93         "release_status": null,
94         "writer_mode": null
95     },
96     "dataset_id": "dart",
97     "example_id": "dart-test-4597",
98     "full_system": {
99         "generated_text": "The Vaults is a pub serving Italian food at moderate prices with an average customer rating,
100         is located in the riverside area, is family friendly, and is near Rainbow Vegetarian Café.\n",
101         "primary_evaluation_eligible": true,
102         "release_status": "approved_with_warnings",
103         "writer_mode": "llm_writer"
104     },
105     "references": [
106         "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
107     ],
108     "request": "Understand the supplied data and report its strongest supported findings.",
109     "task_family": "triple_verbalisation"
110 },
111 {
112     "baseline": {
113         "generated_text": "The Vaults is a city-centre pub serving Italian food, with a customer rating of 3 out of 5. It
114         is family friendly and located near Rainbow Vegetarian Café.",
115         "primary_evaluation_eligible": true,
116         "release_status": null,
117         "writer_mode": null
118     },
119     "dataset_id": "e2e_nlg",
120     "example_id": "e2e_nlg-test-1330",
121     "full_system": {
122         "generated_text": "The Vaults is a family-friendly pub in the city centre serving Italian food, with a customer
123         rating of 3 out of 5, near Rainbow Vegetarian Café.\n",
124         "primary_evaluation_eligible": true,
125         "release_status": "approved_with_warnings",
126         "writer_mode": "llm_writer"
127     },
128     "references": [
129         "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
130     ],
131     "request": "Understand the supplied data and report its strongest supported findings.",
132     "task_family": "attribute_verbalisation"
133 },
134 {
135     "baseline": {
136         "generated_text": "The Cricketers is a family-friendly coffee shop located near Café Sicilia.",
137         "primary_evaluation_eligible": true,
138         "release_status": null,
139         "writer_mode": null
140     },
141     "dataset_id": "e2e_nlg",
142     "example_id": "e2e_nlg-test-209",
143     "full_system": {
144         "generated_text": "The Cricketers is a family-friendly coffee shop near Café Sicilia.\n",
145         "primary_evaluation_eligible": true,
146         "release_status": "approved_with_warnings",
147         "writer_mode": "llm_writer"
148     },
149     "references": [
150         "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
151     ],
152     "request": "Understand the supplied data and report its strongest supported findings.",
153     "task_family": "attribute_verbalisation"
154 },
155 {
156     "baseline": {
157         "generated_text": "The Mill is a pub that serves English food, is not family-friendly, and is located near Raja
158         Indian Cuisine.",
159         "primary_evaluation_eligible": true,
160         "release_status": null,
161         "writer_mode": null
162     },
163     "dataset_id": "e2e_nlg",
164     "example_id": "e2e_nlg-test-447",
165     "full_system": {
166         "generated_text": "The Mill is a pub that serves English food, is not family friendly, and is near Raja Indian

```

```

160     Cuisine.\n",
161     "primary_evaluation_eligible": true,
162     "release_status": "approved_with_warnings",
163     "writer_mode": "llm_writer"
164 },
165 "references": [
166     "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
167 ],
168 "request": "Understand the supplied data and report its strongest supported findings.",
169 "task_family": "attribute_verbalisation"
170 },
171 {
172     "baseline": {
173         "generated_text": "The Mill is a riverside pub that serves fast food, has a customer rating of 3 out of 5, is
174         family-friendly, and is located near Café Rouge.",
175         "primary_evaluation_eligible": true,
176         "release_status": null,
177         "writer_mode": null
178     },
179     "dataset_id": "e2e_nlg",
180     "example_id": "e2e_nlg-test-476",
181     "full_system": {
182         "generated_text": "The Mill is a riverside pub that serves fast food, has a customer rating of 3 out of 5, is
183         family-friendly, and is near Café Rouge.\n",
184         "primary_evaluation_eligible": true,
185         "release_status": "approved_with_warnings",
186         "writer_mode": "llm_writer"
187     },
188     "references": [
189         "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
190     ],
191     "request": "Understand the supplied data and report its strongest supported findings.",
192     "task_family": "attribute_verbalisation"
193 },
194 {
195     "baseline": {
196         "generated_text": "The Phoenix is a pub located near the Crowne Plaza Hotel.",
197         "primary_evaluation_eligible": true,
198         "release_status": null,
199         "writer_mode": null
200     },
201     "dataset_id": "e2e_nlg",
202     "example_id": "e2e_nlg-test-864",
203     "full_system": {
204         "generated_text": "The Phoenix is a pub near Crowne Plaza Hotel.\n",
205         "primary_evaluation_eligible": true,
206         "release_status": "approved_with_warnings",
207         "writer_mode": "llm_writer"
208     },
209     "references": [
210         "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
211     ],
212     "request": "Understand the supplied data and report its strongest supported findings.",
213     "task_family": "attribute_verbalisation"
214 },
215 {
216     "baseline": {
217         "generated_text": "The Los Angeles Clippers defeated the Minnesota Timberwolves -120109 at Staples Center on
218         November 5, 2018, in front of 16,600 fans. The Clippers shot 49% from the field, 45% on three-pointers (14 makes),
219         and 91% on free throws, while the Timberwolves shot 47% from the field, 24% on three-pointers (5 makes), and 80% on
220         free throws. Tobias Harris and Danilo Gallinari each scored 22 points for the Clippers, with Harris adding a
221         double-double (10 rebounds), and Lou Williams added 20 points off the bench. For the Timberwolves, Derrick Rose
222         scored 21 points, while Jimmy Butler and Karl-Anthony Towns each scored 20 points, with Towns posting a double-
223         double (12 rebounds). The Clippers improved to -64, while the Timberwolves fell to -47.",
224         "primary_evaluation_eligible": true,
225         "release_status": null,
226         "writer_mode": null
227     },
228     "dataset_id": "sportsett_basketball",
229     "example_id": "5130",
230     "full_system": {
231         "generated_text": "The Los Angeles Clippers defeated the Minnesota Timberwolves 120-109 on Monday, November 5,
232         2018, at Staples Center.\nLos Angeles entered with 6 wins and 4 losses, while Minnesota entered with 4 wins and 7
233         losses.\nBoth teams were next scheduled away from home: Los Angeles at the Trail Blazers on Thursday, November 8,
234         and Minnesota at the Lakers on Wednesday, November 7.\nFollowing a 33-31 Timberwolves lead after the first quarter,
235         the Clippers led 63-59 after the second quarter and 92-84 after the third before closing out the 120-109 victory.\n
236         Los Angeles outscored Minnesota 32-26 in the second quarter, 29-25 in the third and 28-25 in the fourth.\nTobias
237         Harris and Danilo Gallinari each scored a game-high 22 points, while Derrick Rose led Minnesota with 21 and Lou
238         Williams, Jimmy Butler and Karl-Anthony Towns each had 20.\nHarris tied for the game lead with 8 field goals and
239         collected 10 rebounds; he and Towns each pulled down a game-high 9 defensive rebounds.\nTowns added a game-high 12
240         rebounds, a game-high 4 blocks and 3 steals alongside his 20 points.\nBoban ĆMarjanovi and Montrezl Harrell each
241         grabbed a game-high 4 offensive rebounds, with ĆMarjanovi also recording 2 blocks.\nTaj Gibson added 15 points and
242         9 rebounds for Minnesota.\nThe Clippers also controlled playmaking, finishing with 30 team assists to Minnesota's
243         20; Lou Williams led all players with 6 assists, while Shai Gilgeous-Alexander and Jimmy Butler each had 5.\nLos
244         Angeles won the rebounding comparison 43-38, including a 13-8 edge on the offensive glass, while the teams matched
245         at 30 defensive rebounds each.\nThe Clippers made 14 three-pointers to Minnesota's 5, while the Timberwolves held a
246         24-20 advantage at the free-throw line.\nLos Angeles finished with 43 field goals to Minnesota's 40, while
247         Minnesota recorded more steals (9-5) and blocks (8-4).\n",
248         "primary_evaluation_eligible": true,
249         "release_status": "approved",
250         "writer_mode": "llm_writer"
251     }
252 }

```

```

226     },
227     "references": [
228         "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
229     ],
230     "request": "Understand the supplied data and report its strongest supported findings.",
231     "task_family": "event_report"
232 },
233 {
234     "baseline": {
235         "generated_text": "The visiting Utah Jazz defeated the Sacramento Kings –123117 on October 17, 2018, at a sold-
out Golden 1 Center (attendance 17,600, full capacity). Utah outscored Sacramento in three-point shooting (13 made
threes on 48% shooting vs. 7 on 37%), free throws (28/38 vs. 12/18), rebounds –(4437), and assists –(2117).
Sacramento committed fewer turnovers (9 vs. 17) and had more field-goal attempts (95 vs. 79), but the Jazz's
efficiency from deep and the foul line proved decisive. Donovan Mitchell led all scorers with 24 points, Joe Ingles
added 22, Rudy Gobert posted a double-double (19 points, 15 rebounds), and Derrick Favors scored 18. For the Kings
, Willie Cauley–Stein scored 23, De'Aaron Fox added 21, Buddy Hield scored 19, and Nemanja Bjelica contributed 18."
,
236         "primary_evaluation_eligible": true,
237         "release_status": null,
238         "writer_mode": null
239     },
240     "dataset_id": "sportsett_basketball",
241     "example_id": "5372",
242     "full_system": {
243         "generated_text": "The Utah Jazz defeated the Sacramento Kings 123–117 at Golden 1 Center on Wednesday, October
17, 2018.\nThe contest was the opening game of the 2018 campaign for both teams – and Sacramento's home opener –
with the Jazz improving to 1–0 and the Kings falling to 0–1.\nUtah also held the better conference standing,
arriving in sixth place compared with the Kings' 13th.\nSacramento jumped out to a 34–30 lead after the first
quarter, but Utah answered with a 38–21 second quarter – the Jazz's highest-scoring period – to lead 68–55 at
halftime.\nThe Kings responded with a 32–25 third quarter to trim the advantage to 93–87, and the fourth quarter
was level at 30–30 before Utah closed out the 123–117 win.\nDonovan Mitchell led all scorers with 24 points, while
Joe Ingles added 22 points, six assists and a game-high four steals for the Jazz.\nRudy Gobert anchored the
interior with a game-high 15 rebounds (12 defensive), three blocks and 19 points, and Derrick Favors contributed 18
points and nine rebounds.\nSacramento was paced by Willie Cauley–Stein's 23 points on a game-high 10 made field
goals, while De'Aaron Fox contributed 21 points, a game-high seven assists and three steals.\nBuddy Hield (19
points) and Nemanja Bjelica (18 points and eight rebounds) also reached double figures for the Kings.\nIngles' nine
made field goals were tied for the second-most in the game.\nThe teams produced contrasting shot profiles: while
Sacramento out-shot Utah from the field (49 made field goals to 41), the Jazz held a clear edge in three-pointers
made (13 to 7) and a 28–to–12 advantage in made free throws.\nUtah also held the advantage in total rebounds
(44–37), defensive rebounds (39–32), assists (21–17) and blocks (4–3), while the teams finished tied in offensive
rebounds (5) and steals (8).\nBoth teams return to action on Friday, October 19, 2018, when the Jazz host the
Warriors at Vivint Smart Home Arena and the Kings visit the New Orleans Pelicans at Smoothie King Center.\n",
244         "primary_evaluation_eligible": true,
245         "release_status": "approved",
246         "writer_mode": "auditor_repaired"
247     },
248     "references": [
249         "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
250     ],
251     "request": "Understand the supplied data and report its strongest supported findings.",
252     "task_family": "event_report"
253 },
254 {
255     "baseline": {
256         "generated_text": "– The Toronto Raptors defeated the Dallas Mavericks –116107 at Scotiabank Arena on October 26,
2018, in front of a sold-out crowd of 19,800.\n– The Raptors improved to –60 and held the No. 1 spot in the
Eastern Conference; the Mavericks fell to –23 and stood 10th in the Western Conference.\n– Toronto shot 48% from
the field, 41% on three-pointers, and 85% on free throws; Dallas shot 41% from the field, 40% on threes, and 79% on
free throws.\n– The Raptors had seven players score in double figures, led by Kawhi Leonard (21 points, 9 rebounds
, 5 assists, 3 steals), Kyle Lowry (20 points, 12 assists, double-double), and Jonas čValaninas (17 points, 8
rebounds).\n– The Mavericks were led by Luka čDoni (22 points, 5 rebounds, 4 assists) and Wesley Matthews (21
points); DeAndre Jordan posted a double-double with 18 points and 15 rebounds.\n– Toronto won the first and fourth
quarters, while Dallas won the second and third quarters, but 'Torontos 13-point first-quarter margin helped secure
the 9-point victory.",
257         "primary_evaluation_eligible": true,
258         "release_status": null,
259         "writer_mode": null
260     },
261     "dataset_id": "sportsett_basketball",
262     "example_id": "5786",
263     "full_system": {
264         "generated_text": "The Toronto Raptors defeated the Dallas Mavericks 116–107 on Friday, October 26, 2018, at
Scotiabank Arena in the 2018 season.\nToronto entered with a 6–0 record and first place in its conference standings
, while Dallas arrived at 2–3.\nIt was Toronto's sixth game of the season and Dallas's fifth.\nToronto led after
every quarter: 39–26 after the first, 69–60 at halftime, 92–89 after the third, and 116–107 at the end.\nDallas
outscored Toronto in the middle quarters (34–30 in the second and 29–23 in the third), but the Raptors' 39–26 start
and 24–18 finish kept them in front.\nLuka čDoni led all scorers with 22 points, while Kawhi Leonard and Wesley
Matthews tied for second with 21 apiece.\nLeonard complemented his scoring with 9 rebounds, 5 assists, 3 steals and
a block, while Kyle Lowry added 20 points and a game-high 12 assists.\nDeAndre Jordan paced Dallas with 15
rebounds, including 10 defensive rebounds, and added 18 points and 5 assists; he and Jonas čValaninas each
collected 5 offensive rebounds.\nWesley Matthews made a game-high 9 field goals, Maxi Kleber led all players with 4
blocks, and Serge Ibaka added 11 points and 8 rebounds for Toronto.\nThe Raptors held the shooting and rebounding
edges, making more field goals (44–38) and grabbing more total rebounds (50–44), although the Mavericks attempted
more field goals (92–91).\nDallas had the edge from three-point range, making 12 three-pointers to Toronto's 11,
with čDoni and Danny Green each hitting 4.\nToronto also recorded more defensive rebounds (38–34), offensive
rebounds (12–10), steals (10–6) and assists (23–22), while Dallas led in blocks (6–5) and free throws made (19–17)
.\nBoth teams had immediate follow-up fixtures: Toronto travelled to Milwaukee to face the Bucks on Monday, October
29, while Dallas hosted the Jazz on Sunday, October 28.\n",
265         "primary_evaluation_eligible": true,
266         "release_status": "approved",
267         "writer_mode": "llm_writer"

```

```

268     },
269     "references": [
270         "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
271     ],
272     "request": "Understand the supplied data and report its strongest supported findings.",
273     "task_family": "event_report"
274 },
275 {
276     "baseline": {
277         "generated_text": "The Denver Nuggets defeated the Oklahoma City Thunder –10598 on Saturday, November 24, 2018,
at Chesapeake Energy Arena in Oklahoma City. The game was a sellout (18,200 attendance).\n\nStrongest supported
findings:\n– Denver won despite Oklahoma City outscoring them –3226 in the fourth quarter; the Nuggets led after Q1
–(3323), Q2 –(6342), and Q3 –(7966).\n– The Nuggets shot better overall: 41% FG, 33% 3PT, and 71% FT compared to
the 'Thunders 36% FG, 23% 3PT, and 65% FT.\n– Denver had 9 blocks to Oklahoma 'Citys 5, and 26 assists to the
'Thunders 22, while both teams totaled 54 rebounds.\n– For the Thunder, Paul George scored 24 points, Russell
Westbrook recorded a triple-double (16 points, 10 rebounds, 12 assists), and Steven Adams had a double-double (12
points, 14 rebounds).\n– For the Nuggets, Jamal Murray scored 22 points, Trey Lyles scored 16 points on 6-of-6
shooting, Nikola ĆJoki had 16 points and 5 assists, and Juan Hernangómez added 15 points.\n– At the time, the
Nuggets were –137 and the Thunder –127, with Denver holding the higher Western Conference standing (4th vs. 5th).",
278         "primary_evaluation_eligible": true,
279         "release_status": null,
280         "writer_mode": null
281     },
282     "dataset_id": "sportsett_basketball",
283     "example_id": "5955",
284     "full_system": {
285         "generated_text": "Denver Nuggets defeated Oklahoma City Thunder 105–98 at Chesapeake Energy Arena on Saturday,
November 24, 2018.\nDenver (13–7) and Oklahoma City (12–7) each entered the game with seven losses, with the
Thunder playing at home in their own arena.\nDenver led at the end of every quarter (33–23 after the first, 63–42
at halftime, 79–66 after the third) before closing out a 105–98 win.\nOklahoma City outscored Denver 24–16 in the
third quarter and 32–26 in the fourth, narrowing the margin without taking the lead at a recorded checkpoint.\nPaul
George led all scorers with 24 points and added 11 rebounds, two steals and three blocks, while Jamal Murray
topped Denver with 22 points and nine made field goals.\nRussell Westbrook finished with 16 points and a game-high
12 assists, and Steven Adams grabbed a game-high 14 rebounds with seven offensive boards.\nOklahoma City attempted
more field goals and three-pointers than Denver (103 vs 95 field goals; 39 vs 30 three-pointers), yet Denver made
more of both (39 vs 37 field goals; 10 vs 9 three-pointers).\nDenver also held a 26–22 edge in team assists and a
9–5 advantage in blocks, while Oklahoma City led 19–15 in offensive rebounds and 9–6 in steals.\nThe result covers
only the supplied game record and does not establish why either team won.\n",
286         "primary_evaluation_eligible": true,
287         "release_status": "approved_with_warnings",
288         "writer_mode": "auditor_repaired"
289     },
290     "references": [
291         "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
292     ],
293     "request": "Understand the supplied data and report its strongest supported findings.",
294     "task_family": "event_report"
295 },
296 {
297     "baseline": {
298         "generated_text": "The strongest supported finding is that the Washington Wizards defeated the Atlanta Hawks –
11498 on Wednesday, January 2, 2019, at Capital One Arena in Washington.\n\nThe Wizards led after every quarter
except the third, outscoring the Hawks –3529 in Q1, –2924 in Q2, and –2614 in Q4. Atlanta won Q3 –3124, but
Washington closed the game strongly.\n– Bradley Beal led all scorers with 24 points, and Jeff Green added 22 points
. Thomas Bryant (16), ŠTomá Satoranský (14), and Trevor Ariza (12) joined them, giving the Wizards five starters in
double figures.\n– Alex Len led the Hawks with 24 points off the bench, while John Collins scored 21. No other
Atlanta player reached 15 points.\n– Washington shot better overall: 47% from the field (43/92) vs. Atlanta's 42%
(40/95), and 74% from the free-throw line vs. Atlanta's 62%.\n– The Wizards committed fewer turnovers (10 vs. 14)
and recorded more assists (29 vs. 26) and steals (10 vs. 5). The Hawks had more rebounds (50 vs. 48) and blocks (6
vs. 2).\n– Attendance was 15,300, below the arena capacity of 20,400.\n– Entering the game, the Wizards were –1523
and the Hawks were –1126.",
299         "primary_evaluation_eligible": true,
300         "release_status": null,
301         "writer_mode": null
302     },
303     "dataset_id": "sportsett_basketball",
304     "example_id": "6127",
305     "full_system": {
306         "generated_text": "The Washington Wizards defeated the Atlanta Hawks 114–98 at Capital One Arena on Wednesday,
January 2, 2019, in a 2018–season matchup.\nWashington entered with a 15–23 record in its 38th game, while Atlanta
arrived at 11–26 in its 37th game.\nWashington led at the end of every quarter, ahead 35–29 after the first, 64–53
at halftime, and 88–84 after three.\n\nThe Hawks' only quarter-winning margin was a 31–24 third period, while the
Wizards closed with a 26–14 fourth quarter to seal the result.\nBradley Beal and Alex Len tied for game-high
scoring honours with 24 points each; Jeff Green added 22 for the Wizards, and John Collins paced Atlanta with 21.\n
len also led the game with 11 field goals made and 3 blocks, grabbed 11 rebounds and a game-high 6 offensive
rebounds, while Thomas Bryant pulled down a game-high 15 rebounds (14 defensive) and Trae Young dished out a game-
high 9 assists.\nŠTomá Satoranský supported Washington with 11 rebounds and 7 assists, while Kevin Huerter and
Jeremy Lin each dished out 5 assists for Atlanta.\nChasson Randle and Jeremy Lin each recorded a game-high 3 steals
, and Jeff Green and John Collins each made a game-high 4 three-pointers.\n\nThe largest team-level gap in the
supplied statistics came at the free-throw line, where the Wizards finished 17 of 23 and the Hawks 8 of 13.\n
Washington also made more field goals than Atlanta (43–40) despite fewer attempts (92–95), and held a 10–5 edge in
steals while committing fewer turnovers (10–14).\nAtlanta, meanwhile, controlled the glass, outrebounding
Washington 50–48, and blocked more shots (6–2).\nWashington held a 38–37 edge in defensive rebounds, while the
Hawks led 13–10 on the offensive glass.\n\nThe Wizards also recorded a narrow 29–26 advantage in assists.\nBoth teams
return to action on Friday, January 4, 2019, with Washington visiting the Miami Heat and Atlanta travelling to the
Milwaukee Bucks.\n",
307         "primary_evaluation_eligible": true,
308         "release_status": "approved",
309         "writer_mode": "llm_writer"
310     },
311     "references": [

```



```

312     "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
313   },
314   "request": "Understand the supplied data and report its strongest supported findings.",
315   "task_family": "event_report"
316 },
317 {
318   "baseline": {
319     "generated_text": "Lasse 'Staws career statistics show that he never scored a goal in any recorded competition:
across all clubs and seasons he made 108 league appearances, 14 cup appearances, and 122 total appearances, all
with 0 goals. His most productive season by appearances was 2011 with Fredrikstad in Tippeligaen (22 league apps, 1
cup app, 23 total). He played for Fredrikstad -(20042011), Syrianska (2012), Lillestrøm (2012), Aalesund (2013),
and Bodø/Glimt -(20142015). His highest league-appearance totals were 22 in 2011 and 18 in 2010.",
320     "primary_evaluation_eligible": true,
321     "release_status": null,
322     "writer_mode": null
323   },
324   "dataset_id": "totto",
325   "example_id": "totto-validation-1828",
326   "full_system": {
327     "generated_text": "The selected table value is 2012, Syrianska, Allsvenskan under the Career Total header in the
row containing 8 and 0, within Career statistics / Lasse Staw.\n",
328     "primary_evaluation_eligible": true,
329     "release_status": "approved_with_warnings",
330     "writer_mode": "deterministic_short_form_writer"
331   },
332   "references": [
333     "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
334   ],
335   "request": "Understand the supplied data and report its strongest supported findings.",
336   "task_family": "highlighted_table_description"
337 },
338 {
339   "baseline": {
340     "generated_text": "Based on the supplied data, the strongest supported finding is that Tianna Bartoletta (USA)
established a new world leading mark of 7.14 meters in the women's long jump at the 2015 World Championships in
Beijing, China, on 28 August 2015. This improved upon her previous world leading mark of 7.12 meters set on 27 June
2015 in Eugene, United States.",
341     "primary_evaluation_eligible": true,
342     "release_status": null,
343     "writer_mode": null
344   },
345   "dataset_id": "totto",
346   "example_id": "totto-validation-4467",
347   "full_system": {
348     "generated_text": "During the competition, Tianna Bartoletta (USA) set a world-leading mark of 7.14 in the women'
s long jump.\n",
349     "primary_evaluation_eligible": true,
350     "release_status": "approved_with_warnings",
351     "writer_mode": "llm_writer"
352   },
353   "references": [
354     "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
355   ],
356   "request": "Understand the supplied data and report its strongest supported findings.",
357   "task_family": "highlighted_table_description"
358 },
359 {
360   "baseline": {
361     "generated_text": "The strongest supported findings are: (1) the only highlighted figure is the 2011 Peruvian-
born population of 246,908, the largest number shown in the table; and (2) the "other "data column rises steadily
from 26,831 in 2001 to 98,603 in 2010, with no figures provided for 2012 or 2013.",
362     "primary_evaluation_eligible": true,
363     "release_status": null,
364     "writer_mode": null
365   },
366   "dataset_id": "totto",
367   "example_id": "totto-validation-6067",
368   "full_system": {
369     "generated_text": "In 2011, the Peruvian-born population in Italy was 246,908.\n",
370     "primary_evaluation_eligible": true,
371     "release_status": "approved_with_warnings",
372     "writer_mode": "llm_writer"
373   },
374   "references": [
375     "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
376   ],
377   "request": "Understand the supplied data and report its strongest supported findings.",
378   "task_family": "highlighted_table_description"
379 },
380 {
381   "baseline": {
382     "generated_text": "Ernest Burton coached the Maine Black Bears football team in 1900, and the team finished with
an overall record of -44. No conference record, standing, or bowl/playoff information is provided.",
383     "primary_evaluation_eligible": true,
384     "release_status": null,
385     "writer_mode": null
386   },
387   "dataset_id": "totto",
388   "example_id": "totto-validation-839",
389   "full_system": {
390     "generated_text": "Ernest Burton's head coaching record shows Maine Black Bears in 1900 with an overall record of

```

```

391     -44.\n",
392     "primary_evaluation_eligible": true,
393     "release_status": "approved_with_warnings",
394     "writer_mode": "llm_writer"
395 },
396 "references": [
397     "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
398 ],
399 "request": "Understand the supplied data and report its strongest supported findings.",
400 "task_family": "highlighted_table_description"
401 },
402 {
403     "baseline": {
404         "generated_text": "Tyrell 'Suttons strongest season was 2015: he set career highs in rushing yards (1,059), rushing touchdowns (5), carries (180), receptions (43), and receiving yards (334), while averaging 5.9 yards per carry with a long run of 54 yards. Across his CFL career, he totaled 3,841 rushing yards on 698 carries (5.5 average) with 17 rushing touchdowns, plus 178 receptions for 1,539 yards and 3 receiving touchdowns in 73 games. His 2016 season was highlighted but was shortened to 7 games, producing 412 rushing yards on 74 carries (5.6 average) with 0 touchdowns.",
405         "primary_evaluation_eligible": true,
406         "release_status": null,
407         "writer_mode": null
408     },
409     "dataset_id": "totto",
410     "example_id": "totto-validation-912",
411     "full_system": {
412         "generated_text": "In 2016, Tyrell Sutton played 7 games, with 74 carries for 412 yards and an average of 5.6 yards per carry.\n",
413         "primary_evaluation_eligible": true,
414         "release_status": "approved_with_warnings",
415         "writer_mode": "llm_writer"
416     },
417     "references": [
418         "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
419     ],
420     "request": "Understand the supplied data and report its strongest supported findings.",
421     "task_family": "highlighted_table_description"
422 },
423 {
424     "baseline": {
425         "generated_text": "Piotr Hallmann was born in Gdynia, Poland. Gdynia uses Central European Time and Central European Summer Time. Piotr Hallmann's height is listed as 175.26.",
426         "primary_evaluation_eligible": true,
427         "release_status": null,
428         "writer_mode": null
429     },
430     "dataset_id": "web_nlg",
431     "example_id": "web_nlg_en-test-1209",
432     "full_system": {
433         "generated_text": "Piotr Hallmann was born in Gdynia, Poland, which uses Central European Time and Central European Summer Time, and has a height of 175.26.\n",
434         "primary_evaluation_eligible": true,
435         "release_status": "approved_with_warnings",
436         "writer_mode": "llm_writer"
437     },
438     "references": [
439         "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
440     ],
441     "request": "Understand the supplied data and report its strongest supported findings.",
442     "task_family": "triple_verbalisation"
443 },
444 {
445     "baseline": {
446         "generated_text": "Brandon Carter was born in England in 1942, studied at the University of Cambridge, had Dennis William Sciana as his doctoral advisor, and is known for the Doomsday argument.",
447         "primary_evaluation_eligible": true,
448         "release_status": null,
449         "writer_mode": null
450     },
451     "dataset_id": "web_nlg",
452     "example_id": "web_nlg_en-test-1330",
453     "full_system": {
454         "generated_text": "Brandon Carter's doctoral advisor was Dennis William Sciana; he was born in England on 1942-01-01, studied at the University of Cambridge, and is known for the Doomsday argument.\n",
455         "primary_evaluation_eligible": true,
456         "release_status": "approved_with_warnings",
457         "writer_mode": "llm_writer"
458     },
459     "references": [
460         "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
461     ],
462     "request": "Understand the supplied data and report its strongest supported findings.",
463     "task_family": "triple_verbalisation"
464 },
465 {
466     "baseline": {
467         "generated_text": "The data shows that Ciudad_Ayala is in the Pacific Daylight Time time zone.",
468         "primary_evaluation_eligible": true,
469         "release_status": null,
470         "writer_mode": null
471     },
472     "dataset_id": "web_nlg",
473     "example_id": "web_nlg_en-test-1330",
474     "full_system": {
475         "generated_text": "The data shows that Ciudad_Ayala is in the Pacific Daylight Time time zone.\n",
476         "primary_evaluation_eligible": true,
477         "release_status": "approved_with_warnings",
478         "writer_mode": "llm_writer"
479     },
480     "references": [
481         "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
482     ],
483     "request": "Understand the supplied data and report its strongest supported findings.",
484     "task_family": "triple_verbalisation"
485 },
486 {
487     "baseline": {
488         "generated_text": "The data shows that Ciudad_Ayala is in the Pacific Daylight Time time zone.",
489         "primary_evaluation_eligible": true,
490         "release_status": null,
491         "writer_mode": null
492     },
493     "dataset_id": "web_nlg",
494     "example_id": "web_nlg_en-test-1330",
495     "full_system": {
496         "generated_text": "The data shows that Ciudad_Ayala is in the Pacific Daylight Time time zone.\n",
497         "primary_evaluation_eligible": true,
498         "release_status": "approved_with_warnings",
499         "writer_mode": "llm_writer"
500     },
501     "references": [
502         "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
503     ],
504     "request": "Understand the supplied data and report its strongest supported findings.",
505     "task_family": "triple_verbalisation"
506 },
507 {
508     "baseline": {
509         "generated_text": "The data shows that Ciudad_Ayala is in the Pacific Daylight Time time zone.",
510         "primary_evaluation_eligible": true,
511         "release_status": null,
512         "writer_mode": null
513     },
514     "dataset_id": "web_nlg",
515     "example_id": "web_nlg_en-test-1330",
516     "full_system": {
517         "generated_text": "The data shows that Ciudad_Ayala is in the Pacific Daylight Time time zone.\n",
518         "primary_evaluation_eligible": true,
519         "release_status": "approved_with_warnings",
520         "writer_mode": "llm_writer"
521     },
522     "references": [
523         "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
524     ],
525     "request": "Understand the supplied data and report its strongest supported findings.",
526     "task_family": "triple_verbalisation"
527 },
528 {
529     "baseline": {
530         "generated_text": "The data shows that Ciudad_Ayala is in the Pacific Daylight Time time zone.",
531         "primary_evaluation_eligible": true,
532         "release_status": null,
533         "writer_mode": null
534     },
535     "dataset_id": "web_nlg",
536     "example_id": "web_nlg_en-test-1330",
537     "full_system": {
538         "generated_text": "The data shows that Ciudad_Ayala is in the Pacific Daylight Time time zone.\n",
539         "primary_evaluation_eligible": true,
540         "release_status": "approved_with_warnings",
541         "writer_mode": "llm_writer"
542     },
543     "references": [
544         "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
545     ],
546     "request": "Understand the supplied data and report its strongest supported findings.",
547     "task_family": "triple_verbalisation"
548 },
549 {
550     "baseline": {
551         "generated_text": "The data shows that Ciudad_Ayala is in the Pacific Daylight Time time zone.",
552         "primary_evaluation_eligible": true,
553         "release_status": null,
554         "writer_mode": null
555     },
556     "dataset_id": "web_nlg",
557     "example_id": "web_nlg_en-test-1330",
558     "full_system": {
559         "generated_text": "The data shows that Ciudad_Ayala is in the Pacific Daylight Time time zone.\n",
560         "primary_evaluation_eligible": true,
561         "release_status": "approved_with_warnings",
562         "writer_mode": "llm_writer"
563     },
564     "references": [
565         "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
566     ],
567     "request": "Understand the supplied data and report its strongest supported findings.",
568     "task_family": "triple_verbalisation"
569 },
570 {
571     "baseline": {
572         "generated_text": "The data shows that Ciudad_Ayala is in the Pacific Daylight Time time zone.",
573         "primary_evaluation_eligible": true,
574         "release_status": null,
575         "writer_mode": null
576     },
577     "dataset_id": "web_nlg",
578     "example_id": "web_nlg_en-test-1330",
579     "full_system": {
580         "generated_text": "The data shows that Ciudad_Ayala is in the Pacific Daylight Time time zone.\n",
581         "primary_evaluation_eligible": true,
582         "release_status": "approved_with_warnings",
583         "writer_mode": "llm_writer"
584     },
585     "references": [
586         "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
587     ],
588     "request": "Understand the supplied data and report its strongest supported findings.",
589     "task_family": "triple_verbalisation"
590 },
591 {
592     "baseline": {
593         "generated_text": "The data shows that Ciudad_Ayala is in the Pacific Daylight Time time zone.",
594         "primary_evaluation_eligible": true,
595         "release_status": null,
596         "writer_mode": null
597     },
598     "dataset_id": "web_nlg",
599     "example_id": "web_nlg_en-test-1330",
600     "full_system": {
601         "generated_text": "The data shows that Ciudad_Ayala is in the Pacific Daylight Time time zone.\n",
602         "primary_evaluation_eligible": true,
603         "release_status": "approved_with_warnings",
604         "writer_mode": "llm_writer"
605     },
606     "references": [
607         "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
608     ],
609     "request": "Understand the supplied data and report its strongest supported findings.",
610     "task_family": "triple_verbalisation"
611 },
612 {
613     "baseline": {
614         "generated_text": "The data shows that Ciudad_Ayala is in the Pacific Daylight Time time zone.",
615         "primary_evaluation_eligible": true,
616         "release_status": null,
617         "writer_mode": null
618     },
619     "dataset_id": "web_nlg",
620     "example_id": "web_nlg_en-test-1330",
621     "full_system": {
622         "generated_text": "The data shows that Ciudad_Ayala is in the Pacific Daylight Time time zone.\n",
623         "primary_evaluation_eligible": true,
624         "release_status": "approved_with_warnings",
625         "writer_mode": "llm_writer"
626     },
627     "references": [
628         "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
629     ],
630     "request": "Understand the supplied data and report its strongest supported findings.",
631     "task_family": "triple_verbalisation"
632 },
633 {
634     "baseline": {
635         "generated_text": "The data shows that Ciudad_Ayala is in the Pacific Daylight Time time zone.",
636         "primary_evaluation_eligible": true,
637         "release_status": null,
638         "writer_mode": null
639     },
640     "dataset_id": "web_nlg",
641     "example_id": "web_nlg_en-test-1330",
642     "full_system": {
643         "generated_text": "The data shows that Ciudad_Ayala is in the Pacific Daylight Time time zone.\n",
644         "primary_evaluation_eligible": true,
645         "release_status": "approved_with_warnings",
646         "writer_mode": "llm_writer"
647     },
648     "references": [
649         "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
650     ],
651     "request": "Understand the supplied data and report its strongest supported findings.",
652     "task_family": "triple_verbalisation"
653 },
654 {
655     "baseline": {
656         "generated_text": "The data shows that Ciudad_Ayala is in the Pacific Daylight Time time zone.",
657         "primary_evaluation_eligible": true,
658         "release_status": null,
659         "writer_mode": null
660     },
661     "dataset_id": "web_nlg",
662     "example_id": "web_nlg_en-test-1330",
663     "full_system": {
664         "generated_text": "The data shows that Ciudad_Ayala is in the Pacific Daylight Time time zone.\n",
665         "primary_evaluation_eligible": true,
666         "release_status": "approved_with_warnings",
667         "writer_mode": "llm_writer"
668     },
669     "references": [
670         "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
671     ],
672     "request": "Understand the supplied data and report its strongest supported findings.",
673     "task_family": "triple_verbalisation"
674 },
675 {
676     "baseline": {
677         "generated_text
```

```

471 "dataset_id": "web_nlg",
472 "example_id": "web_nlg_en-test-1466",
473 "full_system": {
474   "generated_text": "Ciudad Ayala is in the Pacific Daylight Time time zone.\n",
475   "primary_evaluation_eligible": true,
476   "release_status": "approved_with_warnings",
477   "writer_mode": "llm_writer"
478 },
479 "references": [
480   "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
481 ],
482 "request": "Understand the supplied data and report its strongest supported findings.",
483 "task_family": "triple_verbalisation"
484 },
485 {
486   "baseline": {
487     "generated_text": "Pontiac Rageous was assembled in Detroit.",
488     "primary_evaluation_eligible": true,
489     "release_status": null,
490     "writer_mode": null
491   },
492   "dataset_id": "web_nlg",
493   "example_id": "web_nlg_en-test-859",
494   "full_system": {
495     "generated_text": "Pontiac Rageous was assembled in Detroit.\n",
496     "primary_evaluation_eligible": true,
497     "release_status": "approved_with_warnings",
498     "writer_mode": "llm_writer"
499   },
500   "references": [
501     "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
502   ],
503   "request": "Understand the supplied data and report its strongest supported findings.",
504   "task_family": "triple_verbalisation"
505 },
506 {
507   "baseline": {
508     "generated_text": "Akeem Ayers debuted with the Tennessee Titans, and his active years began in 2011.",
509     "primary_evaluation_eligible": true,
510     "release_status": null,
511     "writer_mode": null
512   },
513   "dataset_id": "web_nlg",
514   "example_id": "web_nlg_en-test-864",
515   "full_system": {
516     "generated_text": "Akeem Ayers debuted for the Tennessee Titans, and his active years started in 2011.\n",
517     "primary_evaluation_eligible": true,
518     "release_status": "approved_with_warnings",
519     "writer_mode": "llm_writer"
520   },
521   "references": [
522     "<HELD_OUT_REFERENCE_NOT_AVAILABLE_DURING_GENERATION>"
523   ],
524   "request": "Understand the supplied data and report its strongest supported findings.",
525   "task_family": "triple_verbalisation"
526 }
527 ]

```

Listing 91: R02: submission/program_listings/build/generated/results/protected_generation_summary.csv

```

1 dataset_id,example_id,protected_unseen,selection_timestamp,run_id,generation_id,execution_outcome,error,start_time,
  end_time,elapsed_seconds,task_family,request,output_mode,language,report_genre,communication_task,focus_scope,
  generation_seed,configuration_sha256,implementation_sha256,reference_available_to_generator,source_sha256,
  reference_sha256,materialized_model_input_sha256,model_input_matches_reference_isolation_audit,pipeline_result_path
  ,run_directory,final_report_path,initial_writer_mode,final_writer_mode,used_deterministic_fallback,
  final_generation_path,repair_rounds_used,audit_decision,release_status,approved_for_release,
  primary_evaluation_eligible,native_support_rate,factual_sentence_count,supported_sentence_count,
  unsupported_factual_sentence_count,sentence_support_mapping_count,output_word_count,output_sentence_count,
  evidence_item_count,fact_candidate_count,verified_fact_count,rejected_fact_count,insight_candidate_count,
  verified_insight_count,rejected_insight_count,unverified_insight_count,hypothesis_only_insight_count,
  top_level_attempt_count,trace_retry_event_count,fallback_event_count,non_success_trace_event_count,
  provider_reported_input_tokens,provider_reported_output_tokens,provider_reported_total_tokens,
  provider_reported_cache_read_tokens,provider_reported_requests,monetary_cost,monetary_cost_source,artifact_count,
  artifact_index_sha256
2 e2e_nlg,e2e_nlg-test-1330,True,2026-08-21T00:28:22.944030+00:00,20260821T002823Z_9b23675387,e2e_nlg__e2e_nlg-test-1330
  _full_system_r0_s42,success,,2026-08-21T00:28:23.173886+00:00,2026-08-21T00
  :28:48.830828+00:00,25.767118207877502,attribute_verbalisation,Express all and only the supplied attributes in one
  or two fluent sentences. Do not add headings or unsupported details.,short_text,en,dataset_overview,
  attribute_verbalisation,structured_record,42,1fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85,
  f525ca0c932819f533eab9229752a055a8e9c1acef8dea420bfa7e1e8b6b91fb,False,9
  e8fb57080429c1e645ec58eeb3803675a8038d04638e3801c08fece45ea4cfc,
  bf634ba957a0a8228c44057960a6512c941e7510853f54028f6e436b4c50b29f,
  a644bff015ac6f07fcc632deef47cfd6616dbac3c83921fc362ae4f144d5b3,True,evaluation/protected_holdout_full_system/
  generations/runs/e2e_nlg__e2e_nlg-test-1330/full_system/e2e_nlg/20260821T002823Z_9b23675387/pipeline_result.json,
  evaluation/protected_holdout_full_system/generations/runs/e2e_nlg__e2e_nlg-test-1330/full_system/e2e_nlg/20260821
  T002823Z_9b23675387,evaluation/protected_holdout_full_system/generations/runs/e2e_nlg__e2e_nlg-test-1330/
  full_system/e2e_nlg/20260821T002823Z_9b23675387/final_report.md,llm_writer,llm_writer,False,normal_llm_writer,0,
  pass,approved_with_warnings,True,True,1.0,1,1,0,1,26,1,1,1,1,0,0,0,0,0,0,1,1,5627,2151,7778,0,1,,not directly
  available,49,d080e9cd366d4492be990901fe2ecc29486a182a28c07b7056b37eafab6fce72

```


- 3 web_nlg,web_nlg_en-test-1209,True,2026-08-21T00:28:22.944030+00:00,20260821T002848Z_bled943cf4,web_nlg__web_nlg_en-test-1209__full_system_r0__s42,success,,2026-08-21T00:28:48.934552+00:00,2026-08-21T00:29:01.588825+00:00,12.709543416975066,triple_verbalisation,"Express all and only the supplied triples as short, coherent natural language. Do not add unsupported facts.",short_text,en,dataset_overview,triple_verbalisation,structured_record,42,1fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85,f525ca0c932819f533eab9229752a055a8e9c1acef8dea420bfa7e1e8b6b91fb,False,a413a081e355b67874e04f93ab59ee7b33bd20c885e0574a724090f7fe1a163b,2c8538c423f63f3fce8e472bf22f4b2c28bc8595bb9c2016f9d0728cde2b8995e,fb51a14bb56c48d5e823d84b458c2a3c981cd82e3e62ee1d85f7090099365713,True,evaluation/protected_holdout_full_system/generations/runs/web_nlg__web_nlg_en-test-1209/full_system/web_nlg/20260821T002848Z_bled943cf4/pipeline_result.json,evaluation/protected_holdout_full_system/generations/runs/web_nlg__web_nlg_en-test-1209/full_system/web_nlg/20260821T002848Z_bled943cf4,evaluation/protected_holdout_full_system/generations/runs/web_nlg__web_nlg_en-test-1209/full_system/web_nlg/20260821T002848Z_bled943cf4/final_report.md,llm_writer,llm_writer,False,normal_llm_writer,0,pass,approved_with_warnings,True,True,1.0,1,1,0,1,24,1,1,1,0,0,0,0,0,1,0,1,1,5608,1198,6806,0,1,,not directly available,49,bd10e5712860e8cebe57a4fa1182d48abfe286ed056acabc01b571988dece53d
- 4 dart,dart-test-1791,True,2026-08-21T00:28:22.944030+00:00,20260821T002901Z_5a8c24f4e7,dart__dart-test-1791__full_system_r0__s42,success,,2026-08-21T00:29:01.711158+00:00,2026-08-21T00:29:11.418030+00:00,9.758939750026911,triple_verbalisation,"Express all and only the supplied triples as short, coherent natural language. Do not add unsupported facts.",short_text,en,dataset_overview,triple_verbalisation,structured_record,42,1fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85,f525ca0c932819f533eab9229752a055a8e9c1acef8dea420bfa7e1e8b6b91fb,False,20a5eccd67eb73bbf6f37ec51941e57ad416c4faa691d2cc277daf07d08e88d4,af602b3a77c5ee0df21a0b25b33a21eac28b98a21cd8922ee4fd59c0c260ee96,f0e41d4a3d15e1e3caf1149b225e110e8008c53596a1750ece87ddb72a923d09,True,evaluation/protected_holdout_full_system/generations/runs/dart__dart-test-1791/full_system/dart/20260821T002901Z_5a8c24f4e7/pipeline_result.json,evaluation/protected_holdout_full_system/generations/runs/dart__dart-test-1791/full_system/dart/20260821T002901Z_5a8c24f4e7/final_report.md,llm_writer,llm_writer,False,normal_llm_writer,0,pass,approved_with_warnings,True,True,1.0,1,1,0,2,20,2,1,1,0,0,0,0,0,1,0,1,1,5513,777,6290,4096,1,,not directly available,49,fb24dd10421aea0783f5a804ade4d2e884443e7f4accc3381d34fc400512df39
- 5 tutto,totto-validation-1828,True,2026-08-21T00:28:22.944030+00:00,20260821T002911Z_531f6f3190,totto__totto-validation-1828__full_system_r0__s42,success,,2026-08-21T00:29:11.549449+00:00,2026-08-21T00:29:34.656463+00:00,23.17030820879154,highlighted_table_description,Write exactly one concise sentence describing the highlighted table cells. Do not discuss unrelated cells and do not add headings.,one_sentence,en,dataset_overview,focused_table_description,highlighted_cells,42,1fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85,f525ca0c932819f533eab9229752a055a8e9c1acef8dea420bfa7e1e8b6b91fb,False,4f01b0f2d1a2e944dc9c88cbc4ea7035f7f57569055cd98fd963a9f717fd3a40,13f4d6ba14f2d8846ae5d47990ba0abea068972dc688c623d49d9dd967b01b34,9d73b0046588b89d78ab7497565c684dd8e4cdc95d78728b9ae683a648af80f6,True,evaluation/protected_holdout_full_system/generations/runs/totto__totto-validation-1828/full_system/totto/20260821T002911Z_531f6f3190/pipeline_result.json,evaluation/protected_holdout_full_system/generations/runs/totto__totto-validation-1828/full_system/totto/20260821T002911Z_531f6f3190,evaluation/protected_holdout_full_system/generations/runs/totto__totto-validation-1828/full_system/totto/20260821T002911Z_531f6f3190/final_report.md,deterministic_short_form_writer,deterministic_short_form_writer,True,deterministic_fallback,0,pass,approved_with_warnings,True,True,1.0,1,1,0,1,25,1,1,1,0,0,0,0,0,1,0,2,2,6663,2138,8801,4096,1,,not directly available,50,03556a05249ef0cd2272881091a3ec47a7b090d5c351c4f6878ae26702dd0b4
- 6 sportsett_basketball_5130,True,2026-08-21T00:28:22.944030+00:00,20260821T002934Z_9f590b408d,sportsett_basketball_5130__full_system_r0__s42,success,,2026-08-21T00:29:34.810759+00:00,2026-08-21T00:42:17.898690+00:00,763.1489245828707,event_report,"Write a coherent game report from the supplied structured game data. Lead with the result, select the most important performances and contrasts, and do not invent information.",multi_paragraph_report,en,event_report,event_report,reference_recap,42,1fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85,f525ca0c932819f533eab9229752a055a8e9c1acef8dea420bfa7e1e8b6b91fb,False,432bc48818396a9345adffd433b22b1710c709c30e18a4f6935c3464b65c8e2f,2c74673beb1cb0b6c4b079762408b342913c08ad6f7271c4c7229717fcc6eb27,26ca385127f08fb26ab04479148d8fe74c388b4be05214273e832577d44adc71,True,evaluation/protected_holdout_full_system/generations/runs/sportsett_basketball_5130/full_system/sportsett_basketball/20260821T002934Z_9f590b408d/pipeline_result.json,evaluation/protected_holdout_full_system/generations/runs/sportsett_basketball_5130/full_system/sportsett_basketball/20260821T002934Z_9f590b408d,evaluation/protected_holdout_full_system/generations/runs/sportsett_basketball_5130/full_system/sportsett_basketball/20260821T002934Z_9f590b408d/final_report.md,deterministic_fallback,llm_writer,True,deterministic_fallback,0,pass,approved,True,True,1.0,14,14,0,14,299,14,41,41,41,0,8,5,3,0,0,1,3,1,1,408173,106077,514250,2560,10,,not directly available,55,6e951b00266a12c391f8e75b043db771def5202c10ec0f14632080d4f7c16d87
- 7 e2e_nlg,e2e_nlg-test-209,True,2026-08-21T00:28:22.944030+00:00,20260821T004218Z_ffab5b6ba8,e2e_nlg__e2e_nlg-test-209__full_system_r0__s42,success,,2026-08-21T00:42:18.102324+00:00,2026-08-21T00:42:23.992816+00:00,5.955769082996994,attribute_verbalisation,Express all and only the supplied attributes in one or two fluent sentences. Do not add headings or unsupported details.,short_text,en,dataset_overview,attribute_verbalisation,structured_record,42,1fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85,f525ca0c932819f533eab9229752a055a8e9c1acef8dea420bfa7e1e8b6b91fb,False,7de0ca441f2fb833ff0f7f110c66c54c545fc71f7264a1alc4dfb337b21c277,f5b104892617f5bd1d9bf95d2252a6ff0fea0dd279e574255231c301381504c4,fd2c12ff2916d87343e03ebc9ebab88dc4f5644e729fffb08fcd2b3556e51472,True,evaluation/protected_holdout_full_system/generations/runs/e2e_nlg__e2e_nlg-test-209/full_system/e2e_nlg/20260821T004218Z_ffab5b6ba8/pipeline_result.json,evaluation/protected_holdout_full_system/generations/runs/e2e_nlg__e2e_nlg-test-209/full_system/e2e_nlg/20260821T004218Z_ffab5b6ba8,evaluation/protected_holdout_full_system/generations/runs/e2e_nlg__e2e_nlg-test-209/full_system/e2e_nlg/20260821T004218Z_ffab5b6ba8/final_report.md,llm_writer,llm_writer,False,normal_llm_writer,0,pass,approved_with_warnings,True,True,1.0,0,0,0,1,10,1,1,1,1,0,0,0,0,0,1,0,1,1,5471,600,6071,4096,1,,not directly available,49,9eb52065e0303437cac418f5c5c22f565ec94882b69c9c88c3c989708fa1d803
- 8 web_nlg,web_nlg_en-test-1330,True,2026-08-21T00:28:22.944030+00:00,20260821T004224Z_ac74aa80e2,web_nlg__web_nlg_en-test-1330__full_system_r0__s42,success,,2026-08-21T00:42:24.139140+00:00,2026-08-21T00:42:33.394108+00:00,9.308612042106688,triple_verbalisation,"Express all and only the supplied triples as short, coherent natural language. Do not add unsupported facts.",short_text,en,dataset_overview,triple_verbalisation,structured_record,42,1fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85,f525ca0c932819f533eab9229752a055a8e9c1acef8dea420bfa7e1e8b6b91fb,False,0dc9960da718f111cf6eb450519a016a22ee640015ee8bec715c72ba8d05a6b8,df9c74fb2b730dd343fcd32b773b9f320e7a007f1424599601776a78857ab8,1aeaecb39121f7a760d220a0c868a17b756a9f1911fd4c015b43fbb0205beaea,True,evaluation/protected_holdout_full_system/generations/runs/web_nlg__web_nlg_en-test-1330/full_system/web_nlg/20260821T004224Z_ac74aa80e2/pipeline_result.json,evaluation/protected_holdout_full_system/generations/runs/web_nlg__web_nlg_en-test-1330/full_system/web_nlg/20260821T004224Z_ac74aa80e2,evaluation/protected_holdout_full_system/generations/runs/web_nlg__web_nlg_en-test

- 1330/full_system/web_nlg/20260821T004224Z_ac74aa80e2/final_report.md, llm_writer, llm_writer, False, normal_llm_writer, 0, pass, approved_with_warnings, True, True, 1.0, 1, 1, 0, 1, 28, 1, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 11526, 887, 12413, 10496, 2, not directly available, 49, f470a6bdf1fd66b7e09e2929116e9b92258efeb407f73d7e21f6ee99fa1b12c5
- 9 dart, dart-test-1805, True, 2026-08-21T00:28:22.944030+00:00, 20260821T004233Z_19351712cf, dart__dart-test-1805__full_system__r0__s42, success,, 2026-08-21T00:42:33.540914+00:00, 2026-08-21T00:42:45.879636+00:00, 12.389780750032514, triple_verbalisation, "Express all and only the supplied triples as short, coherent natural language. Do not add unsupported facts.", short_text, en, dataset_overview, triple_verbalisation, structured_record, 42, 1fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85, f525ca0c932819f533eab9229752a055a8e9c1acef8dea420bfa7e1e8b6b91fb, False, 6, b2447749b4569aa9ccdd293353536c1e5de45907abab23a3d17be3202be8f9f9, 3, a25a85473ab0357b86dd5806e5dce9a5073145299aceb30558b56d66fa6df6e, 5, ec666ff500091f5de279a217bc369d3115678a2e547652691ac30175ed31a60, True, evaluation/protected_holdout_full_system/generations/runs/dart__dart-test-1805/full_system/dart/20260821T004233Z_19351712cf/pipeline_result.json, evaluation/protected_holdout_full_system/generations/runs/dart__dart-test-1805/full_system/dart/20260821T004233Z_19351712cf, evaluation/protected_holdout_full_system/generations/runs/dart__dart-test-1805/full_system/dart/20260821T004233Z_19351712cf/final_report.md, llm_writer, llm_writer, False, normal_llm_writer, 0, pass, approved_with_warnings, True, True, 1.0, 1, 1, 0, 1, 14, 1, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 5502, 1353, 6855, 4864, 1, not directly available, 49, d3b7136aff702c97912f246daf8c110210c8162c05b7b1b08639bcbf1223733e8
- 10 tutto, tutto-validation-4467, True, 2026-08-21T00:28:22.944030+00:00, 20260821T004246Z_3312e5d979, tutto__tutto-validation-4467__full_system__r0__s42, success,, 2026-08-21T00:42:46.114225+00:00, 2026-08-21T00:42:56.985513+00:00, 10.96569841587916, highlighted_table_description, Write exactly one concise sentence describing the highlighted table cells. Do not discuss unrelated cells and do not add headings., one_sentence, en, dataset_overview, focused_table_description, highlighted_cells, 42, 1, fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85, f525ca0c932819f533eab9229752a055a8e9c1acef8dea420bfa7e1e8b6b91fb, False, aa436856c0ebc8aea944693e765472828cbdc6aafb670970a5c15a03f8df6d1d, b41d8e37a0c759b92fd4173207ba1c235bdd583b30f969d564167dfe6e23b13d, 6879, c3566a759745dc6ce587a348cc9f3c41e5c39b51aa34a53fbfb9e922cd5, True, evaluation/protected_holdout_full_system/generations/runs/tutto__tutto-validation-4467/full_system/tutto/20260821T004246Z_3312e5d979/pipeline_result.json, evaluation/protected_holdout_full_system/generations/runs/tutto__tutto-validation-4467/full_system/tutto/20260821T004246Z_3312e5d979, evaluation/protected_holdout_full_system/generations/runs/tutto__tutto-validation-4467/full_system/tutto/20260821T004246Z_3312e5d979/final_report.md, llm_writer, llm_writer, False, normal_llm_writer, 0, pass, approved_with_warnings, True, True, 1.0, 1, 1, 0, 1, 18, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 13956, 959, 14915, 11008, 2, not directly available, 49, 02bd8fe32860faf1f2a08100141a4923769bcbca93075339bf121057cc7a30ea
- 11 sportsett_basketball, 5372, True, 2026-08-21T00:28:22.944030+00:00, 20260821T004257Z_df24b18c95, sportsett_basketball__5372__full_system__r0__s42, success,, 2026-08-21T00:42:57.205878+00:00, 2026-08-21T01:06:15.146550+00:00, 1398.0258121669758, event_report, "Write a coherent game report from the supplied structured game data. Lead with the result, select the most important performances and contrasts, and do not invent information.", multi_paragraph_report, en, event_report, event_report, reference_recap, 42, 1, fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85, f525ca0c932819f533eab9229752a055a8e9c1acef8dea420bfa7e1e8b6b91fb, False, d0793e0e6623f60983f0cc365fa6269099446399d87b82f687382512d9bde29a, 087382, c8eb93adfbe153771f92bcbac63ce11fe957c30e23620526c9e93103113, e25f8f441a3038f80ebc2b7463486988f9514dd2a04eab41dd528c2d7ae9387a, True, evaluation/protected_holdout_full_system/generations/runs/sportsett_basketball__5372/full_system/sportsett_basketball/20260821T004257Z_df24b18c95/pipeline_result.json, evaluation/protected_holdout_full_system/generations/runs/sportsett_basketball__5372/full_system/sportsett_basketball/20260821T004257Z_df24b18c95, evaluation/protected_holdout_full_system/generations/runs/sportsett_basketball__5372/full_system/sportsett_basketball/20260821T004257Z_df24b18c95/final_report.md, llm_writer, auditor_repaired, False, auditor_repaired, 1, pass, approved, True, True, 1.0, 12, 12, 0, 13, 330, 13, 41, 40, 40, 0, 7, 3, 4, 0, 0, 1, 4, 0, 0, 650702, 162470, 813172, 119552, 14, not directly available, 58, c276a781479b201b6f27976f1a1ab03b4b5bdabbe76c63025f05fce37f5d92bf
- 12 e2e_nlg, e2e_nlg-test-447, True, 2026-08-21T00:28:22.944030+00:00, 20260821T010615Z_718ad58c86, e2e_nlg__e2e_nlg-test-447__full_system__r0__s42, success,, 2026-08-21T01:06:15.464293+00:00, 2026-08-21T01:06:25.779949+00:00, 10.39158095815219, attribute_verbalisation, Express all and only the supplied attributes in one or two fluent sentences. Do not add headings or unsupported details., short_text, en, dataset_overview, attribute_verbalisation, structured_record, 42, 1fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85, f525ca0c932819f533eab9229752a055a8e9c1acef8dea420bfa7e1e8b6b91fb, False, ee20f1488f496eb0fa489b07f5fb0d6f77c740eb5148166dae377bca9b9ff4ca, 94, a4d50214686760602a9c1c5c903df9ff6c84adbed2fc0b1d14c37cdabb91e06, 453836, dd9c3bb836f0358a4971e161bbb91d7f19013dc833a5ed9f797c76896a, True, evaluation/protected_holdout_full_system/generations/runs/e2e_nlg__e2e_nlg-test-447/full_system/e2e_nlg/20260821T010615Z_718ad58c86/pipeline_result.json, evaluation/protected_holdout_full_system/generations/runs/e2e_nlg__e2e_nlg-test-447/full_system/e2e_nlg/20260821T010615Z_718ad58c86, evaluation/protected_holdout_full_system/generations/runs/e2e_nlg__e2e_nlg-test-447/full_system/e2e_nlg/20260821T010615Z_718ad58c86/final_report.md, llm_writer, llm_writer, False, normal_llm_writer, 0, pass, approved_with_warnings, True, True, 1.0, 0, 0, 0, 1, 19, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 5502, 1004, 6506, 4864, 1, not directly available, 49, fe52795cc2714dc6d84a99d1ad67496e3c3845b8030bcbdb5bf5a1ccdf40b6579
- 13 web_nlg, web_nlg_en-test-1466, True, 2026-08-21T00:28:22.944030+00:00, 20260821T010626Z_5c736a6ff8, web_nlg__web_nlg_en-test-1466__full_system__r0__s42, success,, 2026-08-21T01:06:26.064023+00:00, 2026-08-21T01:06:32.732803+00:00, 6.723016832955182, triple_verbalisation, "Express all and only the supplied triples as short, coherent natural language. Do not add unsupported facts.", short_text, en, dataset_overview, triple_verbalisation, structured_record, 42, 1fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85, f525ca0c932819f533eab9229752a055a8e9c1acef8dea420bfa7e1e8b6b91fb, False, ceda5e34141ed6d3857c4ea8e2e05f21e7098cf4f2c6fe0eb35471edeeaf60ee, 0944315, b3912e63a7949969411805989726f2e28d27211c704460c9fd1606042, 34, ea60f546eabda7e8cdd2500c91cd0562b7f558f2c4eda41f35c82a21509573, True, evaluation/protected_holdout_full_system/generations/runs/web_nlg__web_nlg_en-test-1466/full_system/web_nlg/20260821T010626Z_5c736a6ff8/pipeline_result.json, evaluation/protected_holdout_full_system/generations/runs/web_nlg__web_nlg_en-test-1466/full_system/web_nlg/20260821T010626Z_5c736a6ff8, evaluation/protected_holdout_full_system/generations/runs/web_nlg__web_nlg_en-test-1466/full_system/web_nlg/20260821T010626Z_5c736a6ff8/final_report.md, llm_writer, llm_writer, False, normal_llm_writer, 0, pass, approved_with_warnings, True, True, 1.0, 0, 0, 0, 1, 10, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 5334, 611, 5945, 4864, 1, not directly available, 49, f36949c97a76bdf7b4a8afd2bf3c776adb1b181c531f96fb7b9215343a7085b7
- 14 dart, dart-test-1828, True, 2026-08-21T00:28:22.944030+00:00, 20260821T010632Z_beec1e5b68, dart__dart-test-1828__full_system__r0__s42, success,, 2026-08-21T01:06:32.994970+00:00, 2026-08-21T01:06:42.879699+00:00, 9.937432208098471, triple_verbalisation, "Express all and only the supplied triples as short, coherent natural language. Do not add unsupported facts.", short_text, en, dataset_overview, triple_verbalisation, structured_record, 42, 1fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85, f525ca0c932819f533eab9229752a055a8e9c1acef8dea420bfa7e1e8b6b91fb, False, 943, f0cf7a385a91bb9ae4340fc1c3f852658d553173f9aacb7b19e5c8036e1c0, 0, d539c5af7762ac1859ecb654efbf4c76292ab5629b138b455834c2a2d98c92, 1861, c00c7ed7dba3202cf5b41e0ff7502cf79aa46694f3db43db0425cbe547, True, evaluation/protected_holdout_full_system/generations/runs/dart__dart-test-1828/full_system/dart/20260821T010632Z_beec1e5b68/pipeline_result.json, evaluation/

- protected_holdout_full_system/generations/runs/dart__dart-test-1828/full_system/dart/20260821T010632Z_beec1e5b68, evaluation/protected_holdout_full_system/generations/runs/dart__dart-test-1828/full_system/dart/20260821T010632Z_beec1e5b68/final_report.md, llm_writer, llm_writer, False, normal_llm_writer, 0, pass, approved_with_warnings, True, True, 1.0, 1.1, 1.0, 1.2, 2.1, 1.1, 0.0, 0.0, 0.0, 0.1, 0.1, 1.5572, 1065, 6637, 4864, 1, not directly available, 49, 7
- 15 tutto, tutto-validation-6067, True, 2026-08-21T00:28:22.944030+00:00, 20260821T010643Z_3e17181bc2, tutto__tutto-validation-6067__full_system__r0__s42, success,, 2026-08-21T01:06:43.164947+00:00, 2026-08-21T01:06:50.670410+00:00, 7.5639510420151055, highlighted_table_description, Write exactly one concise sentence describing the highlighted table cells. Do not discuss unrelated cells and do not add headings., one_sentence, en, dataset_overview, focused_table_description, highlighted_cells, 42, 1
- fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85, f525ca0c932819f533eab9229752a055a8e9c1acef8dea420bfa7e1e8b6b91fb, False, 93
- ea1c14d25c908009aadf5d17515d948625a4673384762f1e058b9319db1c05, bddc524b00580d101af2646170f1309589439cfbf40ef1626fc3a61fb14e3f83, 28660718
- b3c71e9ba23bbe8273647fa06a098917d4784aef024bc506f098713, True, evaluation/protected_holdout_full_system/generations/runs/totto__tutto-validation-6067/full_system/totto/20260821T010643Z_3e17181bc2/pipeline_result.json, evaluation/protected_holdout_full_system/generations/runs/totto__tutto-validation-6067/full_system/totto/20260821T010643Z_3e17181bc2, evaluation/protected_holdout_full_system/generations/runs/totto__tutto-validation-6067/full_system/totto/20260821T010643Z_3e17181bc2/final_report.md, llm_writer, llm_writer, False, normal_llm_writer, 0, pass, approved_with_warnings, True, True, 1.0, 1.1, 0.1, 1.0, 1.1, 1.1, 1.0, 0.0, 0.0, 0.0, 1.0, 1.1, 6311, 638, 6949, 4992, 1, not directly available, 49, b4615997df80ad39a3883c36a5fcb8dd0cac9a166e5022af6b471b4d1e67c02
- 16 sportsett_basketball_5786, True, 2026-08-21T00:28:22.944030+00:00, 20260821T010650Z_c25deef725, sportsett_basketball_5786__full_system__r0__s42, success,, 2026-08-21T01:06:50.947868+00:00, 2026-08-21T01:20:51.227147+00:00, 840.341477500042, event_report, "Write a coherent game report from the supplied structured game data. Lead with the result, select the most important performances and contrasts, and do not invent information.", multi_paragraph_report, en, event_report, event_report, reference_recap, 42, 1
- fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85, f525ca0c932819f533eab9229752a055a8e9c1acef8dea420bfa7e1e8b6b91fb, False, 2
- d380f4702e071a20f97f90c6d2771c5357144f5f890c974fe06cab042aaf70d, bb95fec76d8be49f9311a03c0fbf3396fdbab22b489d16729eec104139375f72e, 8
- fca2894a7dc7912df896d0cf1f9a596634a84007d9806bfa7a446c0908c27d, True, evaluation/protected_holdout_full_system/generations/runs/sportsett_basketball_5786/full_system/sportsett_basketball/20260821T010650Z_c25deef725/pipeline_result.json, evaluation/protected_holdout_full_system/generations/runs/sportsett_basketball_5786/full_system/sportsett_basketball/20260821T010650Z_c25deef725, evaluation/protected_holdout_full_system/generations/runs/sportsett_basketball_5786/full_system/sportsett_basketball/20260821T010650Z_c25deef725/final_report.md, deterministic_fallback, llm_writer, True, deterministic_fallback, 0, pass, approved, True, True
- 1, 0.13, 13, 0, 13, 298, 13, 42, 43, 42, 1, 8, 5, 3, 0, 0, 1, 3, 1, 1, 408293, 106141, 514434, 89728, 10, not directly available, 56, e038d8abc883e64db09516bb430570c7d32ee48bf681164263c9f46aba75a04
- 17 e2e_nlg, e2e_nlg-test-476, True, 2026-08-21T00:28:22.944030+00:00, 20260821T012051Z_0fe9ddab53, e2e_nlg__e2e_nlg-test-476__full_system__r0__s42, success,, 2026-08-21T01:20:51.558176+00:00, 2026-08-21T01:20:58.003751+00:00, 6.506523834075779, attribute_verbalisation, Express all and only the supplied attributes in one or two fluent sentences. Do not add headings or unsupported details., short_text, en, dataset_overview, attribute_verbalisation, structured_record, 42, 1fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85, f525ca0c932819f533eab9229752a055a8e9c1acef8dea420bfa7e1e8b6b91fb, False, dc16cc8d4dfa17c95af826014892da10e7adc35b3dfec6455e3e14a60dd75116, 6
- d7229df4a4ca4a627bc73b14e4bd031d67f9a879f4a2cfeee3ddadb677541b, d515c0a2e0ed478856ed2addb4dd3a7e91f720d799e5d5b2e0ec059969c559d, True, evaluation/protected_holdout_full_system/generations/runs/e2e_nlg__e2e_nlg-test-476/full_system/e2e_nlg/20260821T012051Z_0fe9ddab53/pipeline_result.json, evaluation/protected_holdout_full_system/generations/runs/e2e_nlg__e2e_nlg-test-476/full_system/e2e_nlg/20260821T012051Z_0fe9ddab53, evaluation/protected_holdout_full_system/generations/runs/e2e_nlg__e2e_nlg-test-476/full_system/e2e_nlg/20260821T012051Z_0fe9ddab53/final_report.md, llm_writer, llm_writer, False, normal_llm_writer, 0, pass, approved_with_warnings, True, True, 1.0, 1.1, 0.1, 1.2, 1.1, 1.0, 0.0, 0.0, 0.1, 0.1, 1.5618, 608, 6226, 4864, 1, not directly available, 49, 0e3dade35d586087d1b68d14b20b3b0ae4dac90189d9298bc2d88a3eb2e6a42
- 18 web_nlg, web_nlg_en-test-859, True, 2026-08-21T00:28:22.944030+00:00, 20260821T012058Z_5d63e851c3, web_nlg__web_nlg_en-test-859__full_system__r0__s42, success,, 2026-08-21T01:20:58.210855+00:00, 2026-08-21T01:21:15.662071+00:00, 17.502832167083398, triple_verbalisation, "Express all and only the supplied triples as short, coherent natural language. Do not add unsupported facts.", short_text, en, dataset_overview, triple_verbalisation, structured_record, 42, 1fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85, f525ca0c932819f533eab9229752a055a8e9c1acef8dea420bfa7e1e8b6b91fb, False, 850
- b090ceaabc5f7faf4782b25926cf343176e102230332065c2f88daff7e3a, 8
- d32ebce4aaf2ab161d1495559f07b3f74fa982abc751baf28c8bb264c8d0c9aa, d17ed3c212b73db0e553a3791c3db0bae4f138d35e8f3c8a31d47d48139290bab, True, evaluation/protected_holdout_full_system/generations/runs/web_nlg__web_nlg_en-test-859/full_system/web_nlg/20260821T012058Z_5d63e851c3/pipeline_result.json, evaluation/protected_holdout_full_system/generations/runs/web_nlg__web_nlg_en-test-859/full_system/web_nlg/20260821T012058Z_5d63e851c3, evaluation/protected_holdout_full_system/generations/runs/web_nlg__web_nlg_en-test-859/full_system/web_nlg/20260821T012058Z_5d63e851c3/final_report.md, llm_writer, llm_writer, False, normal_llm_writer, 0, pass, approved_with_warnings, True, True, 1.0, 0.0, 0.0, 1.6, 1.1, 1.0, 0.0, 0.0, 0.0, 1.0, 1.1, 10789, 1631, 12420, 10112, 2, not directly available, 49, 9b5971936ff583df6f8f59c2f0e4ef6ede2d8760fd442081797f744dfe4c3f4c
- 19 dart, dart-test-2278, True, 2026-08-21T00:28:22.944030+00:00, 20260821T012116Z_5b71a80fdb, dart__dart-test-2278__full_system__r0__s42, success,, 2026-08-21T01:21:16.010470+00:00, 2026-08-21T01:21:29.983110+00:00, 14.031965209171176, triple_verbalisation, "Express all and only the supplied triples as short, coherent natural language. Do not add unsupported facts.", short_text, en, dataset_overview, triple_verbalisation, structured_record, 42, 1fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85, f525ca0c932819f533eab9229752a055a8e9c1acef8dea420bfa7e1e8b6b91fb, False, 83
- a08c4f27a66f1a6306ae6360a6ed977011c4ca6ded4a7b1df2f31b72c49479, dce109bd22e76fd1ea233f5c335019f0b17608be71f544d9f143331402310cf6, cd61af4769422d6f75aeb8afad43812438605a4beb5761a7266edc0f59e12194, True, evaluation/protected_holdout_full_system/generations/runs/dart__dart-test-2278/full_system/dart/20260821T012116Z_5b71a80fdb/pipeline_result.json, evaluation/protected_holdout_full_system/generations/runs/dart__dart-test-2278/full_system/dart/20260821T012116Z_5b71a80fdb, evaluation/protected_holdout_full_system/generations/runs/dart__dart-test-2278/full_system/dart/20260821T012116Z_5b71a80fdb/final_report.md, llm_writer, llm_writer, False, normal_llm_writer, 0, pass, approved, True, True
- 1, 0.1, 1.0, 1.44, 1.1, 1.1, 1.0, 0.0, 0.0, 0.0, 1.0, 1.1, 5770, 1427, 7197, 4864, 1, not directly available, 49, 75
- cb22c0d60a2645b58e7287f0ef26d8fc2de8eb90e3f4bd2ea5d3aee9c87703
- 20 tutto, tutto-validation-839, True, 2026-08-21T00:28:22.944030+00:00, 20260821T012130Z_163d7186d4, tutto__tutto-validation-839__full_system__r0__s42, success,, 2026-08-21T01:21:30.319944+00:00, 2026-08-21T01:21:45.702150+00:00, 15.441314999945462, highlighted_table_description, Write exactly one concise sentence describing the highlighted table cells. Do not discuss unrelated cells and do not add headings., one_sentence, en, dataset_overview, focused_table_description, highlighted_cells, 42, 1
- fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85, f525ca0c932819f533eab9229752a055a8e9c1acef8dea420bfa7e1e8b6b91fb, False, 09
- a9a840e0ecbe6e8cc23300cef1fc925e8577e9d8c77bcd996240dff713676, 4

- bc3fecf5a648f242388bf1ed91ce31c75f2b90e15263b2e563f0f18efb5cbf1,
c80c2c21b0a275a0439f6e804f6792e735c5f35eeb4ca1bd409615e069c3ae9f,True,evaluation/protected_holdout_full_system/
generations/runs/totto__totto-validation-839/full_system/totto/20260821T012130Z_163d7186d4/pipeline_result.json,
evaluation/protected_holdout_full_system/generations/runs/totto__totto-validation-839/full_system/totto/20260821
T012130Z_163d7186d4,evaluation/protected_holdout_full_system/generations/runs/totto__totto-validation-839/
full_system/totto/20260821T012130Z_163d7186d4/final_report.md,llm_writer,False,normalllm_writer,0,pass,
approved_with_warnings,True,True,1.0,1,1,0,1,18,1,1,1,1,0,0,0,0,0,0,1,0,1,1,6577,1436,8013,4992,1,,not directly
available,49,a91727ee1be71d80ca7b5c882b18a689f46dfee7c53211456dec313a5a0d5331
- 21 sportsett_basketball,5955,True,2026-08-21T00:28:22.944030+00:00,20260821T012146Z_70aaa903bc,
sportsett_basketball__5955__full_system__r0__s42,success,,2026-08-21T01:21:46.080685+00:00,2026-08-21T01
:37:38.580465+00:00,952.5461365829688,event_report,"Write a coherent game report from the supplied structured game
data. Lead with the result, select the most important performances and contrasts, and do not invent information.",
multi_paragraph_report,en,event_report,event_report,reference_recap,42,1
fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85,
f525ca0c932819f533eab9229752a055a8e9c1acef8dea420bfa7e1e8b6b91fb,False,86491264
c5dd17cd78b7120bb3f179565618a1c2778e388ea25b41e288ef6199,833109
f0de34b3c6c88341b8443b6db76e62efaf6cd0eacea3e3a9b99a3995d5f,752
a2496373eed12fd9993082feb1bf5a42d2871cce123d5266d07ed96e72646,True,evaluation/protected_holdout_full_system/
generations/runs/sportsett_basketball__5955/full_system/sportsett_basketball/20260821T012146Z_70aaa903bc/
pipeline_result.json,evaluation/protected_holdout_full_system/generations/runs/sportsett_basketball__5955/
full_system/sportsett_basketball/20260821T012146Z_70aaa903bc,evaluation/protected_holdout_full_system/generations/
runs/sportsett_basketball__5955/full_system/sportsett_basketball/20260821T012146Z_70aaa903bc/final_report.md,
deterministic_fallback,auditor_repaired,True,auditor_repaired,1,pass,approved_with_warnings,True,True
1.0,9,9,0,9,219,9,42,42,42,0,8,3,5,0,1,5,1,1,558467,125567,684034,48640,15,,not directly available,63,
cb803c3c293bcfbf7614818f4445843af10310369917c7b5738f9201c810ef6f
- 22 e2e_nlg,e2e_nlg-test-864,True,2026-08-21T00:28:22.944030+00:00,20260821T013739Z_5acf94b523,e2e_nlg__e2e_nlg-test-864
__full_system__r0__s42,success,,2026-08-21T01:37:39.187295+00:00,2026-08-21T01
:38:07.159833+00:00,28.031933709047735,attribute_verbalisation,Express all and only the supplied attributes in one
or two fluent sentences. Do not add headings or unsupported details.,short_text,en,dataset_overview,
attribute_verbalisation,structured_record,42,1fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85,
f525ca0c932819f533eab9229752a055a8e9c1acef8dea420bfa7e1e8b6b91fb,False,772650
e2992ed07ecfb68cae7171298479d3d2d9bd6216b563c4fddc4e4f5c712,5
ef157458d3427cf0014a21463a855d8194f2045332af643deded6fd3cb096645,
e0e90b1c004440b8f04d49acee89229015ddc5b85e0b5a8af5d474a4993191,True,evaluation/protected_holdout_full_system/
generations/runs/e2e_nlg__e2e_nlg-test-864/full_system/e2e_nlg/20260821T013739Z_5acf94b523/pipeline_result.json,
evaluation/protected_holdout_full_system/generations/runs/e2e_nlg__e2e_nlg-test-864/full_system/e2e_nlg/20260821
T013739Z_5acf94b523,evaluation/protected_holdout_full_system/generations/runs/e2e_nlg__e2e_nlg-test-864/full_system
/e2e_nlg/20260821T013739Z_5acf94b523/final_report.md,llm_writer,llm_writer,False,normalllm_writer,0,pass,
approved_with_warnings,True,True,1.0,0,0,0,1,9,1,1,1,1,0,0,0,0,0,0,1,0,1,1,10874,2595,13469,10240,2,,not directly
available,49,03d12c065e9eb6b1f9eb8b3a8a099023191fc2377b70b5b115aaf7dccc4e7f29
- 23 web_nlg,web_nlg_en-test-864,True,2026-08-21T00:28:22.944030+00:00,20260821T013807Z_bf2039a3c8,web_nlg__web_nlg_en-test
-864__full_system__r0__s42,success,,2026-08-21T01:38:07.544245+00:00,2026-08-21T01
:38:17.097550+00:00,9.602543875109404,triple_verbalisation,"Express all and only the supplied triples as short,
coherent natural language. Do not add unsupported facts.",short_text,en,dataset_overview,triple_verbalisation,
structured_record,42,1fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85,
f525ca0c932819f533eab9229752a055a8e9c1acef8dea420bfa7e1e8b6b91fb,False,3
d589eb09ad48805c2de71a9c53f355e9c0dd090d7ad7330a531db2dd1163f66,94
f9b0173d9ce29a5afa00e4d7523b95eb7a3b8e9981398b9cd1b2f0c2a36b74,157
b12b40509cd022398b69a491f40f38f8e89e6b9d8f52bf17ab1c4f27c2258,True,evaluation/protected_holdout_full_system/
generations/runs/web_nlg__web_nlg_en-test-864/full_system/web_nlg/20260821T013807Z_bf2039a3c8/pipeline_result.json,
evaluation/protected_holdout_full_system/generations/runs/web_nlg__web_nlg_en-test-864/full_system/web_nlg/20260821
T013807Z_bf2039a3c8,evaluation/protected_holdout_full_system/generations/runs/web_nlg__web_nlg_en-test-864/
full_system/web_nlg/20260821T013807Z_bf2039a3c8/final_report.md,llm_writer,llm_writer,False,normalllm_writer,0,
pass,approved_with_warnings,True,True,1.0,1,1,0,1,14,1,1,1,1,0,0,0,0,0,0,1,0,1,1,5410,1005,6415,4864,1,,not
directly available,49,67f1dea33001839dc492290fe53932ad5e76ccacd06446b0227fb7232da93972
- 24 dart,dart-test-4597,True,2026-08-21T00:28:22.944030+00:00,20260821T013817Z_645510c269,dart__dart-test-4597
__full_system__r0__s42,success,,2026-08-21T01:38:17.352361+00:00,2026-08-21T01
:38:28.727024+00:00,11.425348166842014,triple_verbalisation,"Express all and only the supplied triples as short,
coherent natural language. Do not add unsupported facts.",short_text,en,dataset_overview,triple_verbalisation,
structured_record,42,1fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85,
f525ca0c932819f533eab9229752a055a8e9c1acef8dea420bfa7e1e8b6b91fb,False,652
f9936ba6424bb0f260c03db3501d3047181a928e2fdb4eecc3ca81f2829e5,4
bd00c10ae67936e233cbc1a60c47d4265f1871969876cbfb8a66c8fbcba1669,50
ef55e205bea5237082f68f2ab4cbb1aae5b920a6540948611473c2ffe33516,True,evaluation/protected_holdout_full_system/
generations/runs/dart__dart-test-4597/full_system/dart/20260821T013817Z_645510c269/pipeline_result.json,evaluation/
protected_holdout_full_system/generations/runs/dart__dart-test-4597/full_system/dart/20260821T013817Z_645510c269/
evaluation/protected_holdout_full_system/generations/runs/dart__dart-test-4597/full_system/dart/20260821
T013817Z_645510c269/final_report.md,llm_writer,llm_writer,False,normalllm_writer,0,pass,approved_with_warnings,
True,True,1.0,0,0,0,0,1,31,1,1,1,1,0,0,0,0,0,0,1,0,1,1,5734,1421,7155,4864,1,,not directly available,49,
f80a59623b576277139effd4f65b16d5f3bf8a5fb12786196f2f64a405c25fffb
- 25 totto,totto-validation-912,True,2026-08-21T00:28:22.944030+00:00,20260821T013829Z_1e2ad3ba70,totto__totto-validation
-912__full_system__r0__s42,success,,2026-08-21T01:38:29.002196+00:00,2026-08-21T01
:38:38.216563+00:00,9.277694500051439,highlighted_table_description,Write exactly one concise sentence describing
the highlighted table cells. Do not discuss unrelated cells and do not add headings.,one_sentence,en,
dataset_overview,focused_table_description,highlighted_cells,42,1
fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85,
f525ca0c932819f533eab9229752a055a8e9c1acef8dea420bfa7e1e8b6b91fb,False,02076943868
e187e53172594cfe40e876163ca4a1385a814147a0a1655d4b5aa,85
e3d857aa14ca068b9257722fc7a85a211536410554c00103c0b93772d8aaae,
bc8a912666dc4bbdba9a149dd67cf0d6b6bb0d8919ea34aad32f844943bee21e,True,evaluation/protected_holdout_full_system/
generations/runs/totto__totto-validation-912/full_system/totto/20260821T013829Z_1e2ad3ba70/pipeline_result.json,
evaluation/protected_holdout_full_system/generations/runs/totto__totto-validation-912/full_system/totto/20260821
T013829Z_1e2ad3ba70,evaluation/protected_holdout_full_system/generations/runs/totto__totto-validation-912/
full_system/totto/20260821T013829Z_1e2ad3ba70/final_report.md,llm_writer,llm_writer,False,normalllm_writer,0,pass,
approved_with_warnings,True,True,1.0,1,1,0,1,22,1,1,1,1,0,0,0,0,0,0,1,0,1,1,6831,872,7703,4992,1,,not directly
available,49,695a606e7b43318595e8a6d854d337a1ca6f3e361a5613f4dccea40958357e7bf
- 26 sportsett_basketball,6127,True,2026-08-21T00:28:22.944030+00:00,20260821T013838Z_4f66014f7c,
sportsett_basketball__6127__full_system__r0__s42,success,,2026-08-21T01:38:38.490633+00:00,2026-08-21T01
:58:41.876236+00:00,1293.414530124981,event_report,"Write a coherent game report from the supplied structured game
data. Lead with the result, select the most important performances and contrasts, and do not invent information.",
multi_paragraph_report,en,event_report,event_report,reference_recap,42,1

```

fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85,
f525ca0c932819f533eab9229752a055a8e9c1acef8dea420bfa7e1e8b6b91fb,False,532
b36909f92f9f2d8e8c7fafa6b1f5643fde5ecac64a97df32804fc816cd53df,
b35653aa64d112e412a5fd2d9f6320a37fe2b9791b2854ce11248a82c31ef183,9
b5e74df71dd5da8c8e17caae0d89f352c3ae0ae65a4faefe9ca5ba70dbabbd,True,evaluation/protected_holdout_full_system/
generations/runs/sportsett_basketball_6127/full_system/sportsett_basketball/20260821T013838Z_4f66014f7c/
pipeline_result.json,evaluation/protected_holdout_full_system/generations/runs/sportsett_basketball_6127/
full_system/sportsett_basketball/20260821T013838Z_4f66014f7c,evaluation/protected_holdout_full_system/generations/
runs/sportsett_basketball_6127/full_system/sportsett_basketball/20260821T013838Z_4f66014f7c/final_report.md,
llm_writer,llm_writer,False,normal_llm_writer,0,pass,approved,True,True
,1.0,14,14,0,14,313,14,42,43,39,4,12,6,6,0,0,1,7,1,1,476312,133199,609511,46464,16,,not directly available,56,4
e52886db4001892fda49f27a464ce8062696e23b19ae3406911d42b0d992fc0

```

Listing 92: R03: submission/program_listings/build/generated/results/automatic_metrics_overall.csv

```

1 metric_name,Baseline,Full System,Full System improvement,preferred_condition
2 alignscore_base,0.5819312291126698,0.7291240739822388,0.14719284486956896,Full System
3 bertscore_f1,0.9088549995422364,0.9238852119445801,0.015030212402343701,Full System
4 bleu,0.24549424509986253,0.3125988501666169,0.06710460506675439,Full System
5 chrfr,0.4931009729268074,0.5517936556761668,0.058692682749359404,Full System
6 corpus_bleu,0.24320799626716397,0.3096713058809419,0.06646330961377792,Full System
7 corpus_chrf,0.48639181546986715,0.5407746706144317,0.05438285514456459,Full System
8 corpus_ter,1.9789142595211664,1.0245850177733535,-0.9543292417478129,Baseline
9 hhem_2_1_open_mean_support,0.49525193874393075,0.5367802866666223,0.0415283479226915,Full System
10 hhem_2_1_open_min_sentence_support,0.4211749585159123,0.526073329616338,0.10489837110042571,Full System
11 hhem_2_1_open_unsupported_sentence_rate,0.4,0.32,-0.08000000000000002,Baseline
12 meteor,0.41942823166838233,0.4595027942196693,0.04007456255128694,Full System
13 rouge1,0.5846557532675175,0.6647926993129255,0.080136946045408,Full System
14 rouge2,0.36973732835228357,0.41875719295821434,0.049019864605930774,Full System
15 rougeL,0.4606315347852144,0.4966731008768414,0.03604156609162701,Full System
16 rougeLsum,0.4606315347852144,0.4966731008768414,0.03604156609162701,Full System
17 ter,1.6357413130978977,0.7742425313614247,0.861498781736473,Full System

```

Listing 93: R04: submission/program_listings/build/generated/results/automatic_metrics_by_dataset.csv

```

1 dataset_id,metric_name,Baseline,Full System,Full System improvement
2 dart,alignscore_base,0.6908375659491867,0.8290527999401093,0.13821523399092261
3 dart,bertscore_f1,0.9470098137855529,0.9560114026069642,0.009001588821411222
4 dart,bleu,0.3533700215990173,0.47138731931106437,0.11801729771204705
5 dart,chrfr,0.6522585358192777,0.6798641688444466,0.02760563302516883
6 dart,corpus_bleu,0.36574877627684826,0.5040107922092354,0.13826201593238713
7 dart,corpus_chrf,0.6597653030530851,0.6846408293517797,0.024875526298694606
8 dart,corpus_ter,0.8073654390934845,0.6288951841359773,-0.17847025495750712
9 dart,hhem_2_1_open_mean_support,0.6490371704101563,0.7715212345123291,0.12248406410217283
10 dart,hhem_2_1_open_min_sentence_support,0.5416393756866456,0.7451884150505066,0.20354903936386104
11 dart,hhem_2_1_open_unsupported_sentence_rate,0.2,0.0,-0.2
12 dart,meteor,0.5591384358399113,0.6066814729081225,0.04754303706821117
13 dart,rouge1,0.7442516664509224,0.8133335762615577,0.06908190981063522
14 dart,rouge2,0.49147136934559377,0.573049973049973,0.0815786037043792
15 dart,rougeL,0.5994329257487152,0.6200064690854468,0.02057354333673156
16 dart,rougeLsum,0.5994329257487152,0.6200064690854468,0.02057354333673156
17 dart,ter,0.7869125948815883,0.5838654094087711,0.2030471854728172
18 e2e_nlg,alignscore_base,0.9550493478775024,0.9693797349929809,0.014330387115478516
19 e2e_nlg,bertscore_f1,0.9382282733917237,0.9373874425888061,-0.0008408308029175249
20 e2e_nlg,bleu,0.2745132200579291,0.2821091374348484,0.007595917376919281
21 e2e_nlg,chrfr,0.5380970060819097,0.5082978890449223,-0.029799117036987344
22 e2e_nlg,corpus_bleu,0.22491520398514198,0.2807654052324925,0.05585020124735052
23 e2e_nlg,corpus_chrf,0.5069638642256684,0.4847648191349233,-0.022199045090745106
24 e2e_nlg,corpus_ter,1.5916230366492146,1.5287958115183247,-0.0628272251308899
25 e2e_nlg,hhem_2_1_open_mean_support,0.5598814368247986,0.5511923044919967,-0.008689132332801885
26 e2e_nlg,hhem_2_1_open_min_sentence_support,0.5556727886199951,0.5511923044919967,-0.0044804841279983965
27 e2e_nlg,hhem_2_1_open_unsupported_sentence_rate,0.2,0.2,0.0
28 e2e_nlg,meteor,0.4475771859972627,0.4284464459128129,-0.019130740084449815
29 e2e_nlg,rouge1,0.7060159402744148,0.6873541592696963,-0.018661781004718492
30 e2e_nlg,rouge2,0.46286846147627136,0.4588278388278388,-0.004040622648432579
31 e2e_nlg,rougeL,0.5050898432521983,0.4852176002591039,-0.019872242993094424
32 e2e_nlg,rougeLsum,0.5050898432521983,0.4852176002591039,-0.019872242993094424
33 e2e_nlg,ter,0.6318221143582656,0.6327119920000899,-0.0008898776418242438
34 sportsett_basketball,alignscore_base,0.10906427055597305,0.12013751864433289,0.011073248088359841
35 sportsett_basketball,bertscore_f1,0.8388357639312745,0.8489838004112243,0.010148036479949885
36 sportsett_basketball,bleu,0.06123871170267736,0.09550992130783834,0.034271209605160974
37 sportsett_basketball,chrfr,0.2992520927382952,0.41507691642545474,0.11582482368715957
38 sportsett_basketball,corpus_bleu,0.06262677819865266,0.0999171924054129,0.03729041420676024
39 sportsett_basketball,corpus_chrf,0.29967779895554136,0.41597362325698134,0.11629582430143998
40 sportsett_basketball,corpus_ter,0.8146564885496184,0.8598473282442748,0.045190839694656426
41 sportsett_basketball,hhem_2_1_open_mean_support,0.06605279411396218,0.03570042370195399,-0.030352370412008195
42 sportsett_basketball,hhem_2_1_open_min_sentence_support,0.011042013857513666,0.00849845791235566,-0.002543555945158005
43 sportsett_basketball,hhem_2_1_open_unsupported_sentence_rate,1.0,1.0,0.0
44 sportsett_basketball,meteor,0.18380804179153848,0.26058726010088706,0.07677921830934858
45 sportsett_basketball,rouge1,0.431365543243332,0.4881240831195474,0.056787528795214204
46 sportsett_basketball,rouge2,0.16665888915127952,0.15896239825952568,-0.007696490891753838
47 sportsett_basketball,rougeL,0.2399532171341694,0.22963888686319542,-0.010314330270973976
48 sportsett_basketball,rougeLsum,0.2399532171341694,0.22963888686319542,-0.010314330270973976
49 sportsett_basketball,ter,0.813929763736955,0.8607708738868218,-0.046841110149866805
50 tutto,alignscore_base,0.2693070024251938,0.7752031445503235,0.5058961421251297
51 tutto,bertscore_f1,0.8678574204444885,0.9163552403450013,0.04849781990051272

```

```

52 tutto,bleu,0.041421712898675675,0.12635862040491172,0.08493690750623605
53 tutto,chrfr,0.2714968383745506,0.41953248665973064,0.14803564828518007
54 tutto,corpus_bleu,0.034155668705103975,0.11198198300101653,0.07782631429591255
55 tutto,corpus_chrf,0.26711536010070275,0.4178816583224375,0.15076629822173476
56 tutto,corpus_ter,6.172661870503597,1.5971223021582732,-4.575539568345324
57 tutto,hhem_2_1_open_mean_support,0.38965561942507826,0.5829517483711243,0.19329612894604603
58 tutto,hhem_2_1_open_min_sentence_support,0.20553822591900825,0.5829517483711243,0.377413522452116
59 tutto,hhem_2_1_open_unsupported_sentence_rate,0.6,0.4,-0.19999999999999996
60 tutto,meteor,0.27269354560836084,0.3123041704106459,0.039610624802285055
61 tutto,rouge1,0.26243556475855406,0.5183150183150184,0.2558794535564643
62 tutto,rouge2,0.11488686776982789,0.2486810802600276,0.13379421249019974
63 tutto,rougeL,0.22211170040132977,0.4332600732600733,0.21114837285874355
64 tutto,rougeLsum,0.22211170040132977,0.4332600732600733,0.21114837285874355
65 tutto,ter,5.428787190551896,1.362582330229389,4.066204860322507
66 web_nlg,alignscore_base,0.8853979587554932,0.9518471717834472,0.06644921302795403
67 web_nlg,bertscore_f1,0.9523437261581421,0.9606881737709045,0.008344447612762429
68 web_nlg,bleu,0.49692755924101323,0.587629252374422,0.09070169313340876
69 web_nlg,chrfr,0.7044003916200039,0.7361968174062797,0.03179642578627584
70 web_nlg,corpus_bleu,0.528593554170073,0.5516811565565523,0.023087602386479222
71 web_nlg,corpus_chrf,0.698436751014338,0.700612423006037,0.002175671991698991
72 web_nlg,corpus_ter,0.5082644628099174,0.5082644628099174,0.0
73 web_nlg,hhem_2_1_open_mean_support,0.8116326729456583,0.7425357222557067,-0.06909695068995159
74 web_nlg,hhem_2_1_open_min_sentence_support,0.7919823884963989,0.7425357222557067,-0.04944666624069216
75 web_nlg,hhem_2_1_open_unsupported_sentence_rate,0.0,0.0,0.0
76 web_nlg,meteor,0.6339239491048383,0.6894946217658778,0.055570672661039544
77 web_nlg,rouge1,0.7792390405293631,0.8168366595988079,0.03759761906944481
78 web_nlg,rouge2,0.6128010540184453,0.6542646743937066,0.0414636203752613
79 web_nlg,rougeL,0.7365699873896595,0.7152424749163879,-0.02132751247327158
80 web_nlg,rougeLsum,0.7365699873896595,0.7152424749163879,-0.02132751247327158
81 web_nlg,ter,0.5172549019607843,0.43128205128205127,0.085972850678733

```

Listing 94: R05: submission/program_listings/build/generated/results/gpt56_annotation_summary.csv

```

1 dataset_id,example_id,condition,judge_model,error_count,duration_seconds,input_tokens,output_tokens,total_tokens
2 tutto,tutto-validation-912,Baseline,gpt-5.6-sol,2,14.502956165932119,1186,781,1967
3 e2e_nlg,e2e_nlg-test-209,Full System,gpt-5.6-sol,0,2.7142687500454485,753,15,768
4 e2e_nlg,e2e_nlg-test-476,Baseline,gpt-5.6-sol,0,1.5289726660121232,790,15,805
5 dart,dart-test-1828,Baseline,gpt-5.6-sol,0,3.168028000043705,816,93,909
6 tutto,tutto-validation-839,Baseline,gpt-5.6-sol,1,5.556205875007436,845,295,1140
7 dart,dart-test-1791,Baseline,gpt-5.6-sol,0,2.2904556661378592,779,76,855
8 web_nlg,web_nlg_en-test-1466,Baseline,gpt-5.6-sol,0,1.553344374988228,743,15,758
9 sportsett_basketball,6127,Baseline,gpt-5.6-sol,3,14.955795833142474,8721,981,9702
10 tutto,tutto-validation-4467,Baseline,gpt-5.6-sol,1,6.058824875159189,1160,357,1517
11 e2e_nlg,e2e_nlg-test-1330,Full System,gpt-5.6-sol,0,2.510921541135758,790,15,805
12 tutto,tutto-validation-839,Full System,gpt-5.6-sol,1,4.919411958195269,827,227,1054
13 dart,dart-test-4597,Baseline,gpt-5.6-sol,0,1.2095647908281535,805,15,820
14 sportsett_basketball,5786,Baseline,gpt-5.6-sol,1,6.230140540981665,8903,377,9280
15 sportsett_basketball,5372,Full System,gpt-5.6-sol,2,16.708003415958956,9531,1084,10615
16 dart,dart-test-1791,Full System,gpt-5.6-sol,0,1.2078252090141177,779,15,794
17 sportsett_basketball,5130,Full System,gpt-5.6-sol,2,16.768215583171695,9515,1023,10538
18 tutto,tutto-validation-1828,Full System,gpt-5.6-sol,2,8.205335082951933,1143,571,1714
19 web_nlg,web_nlg_en-test-864,Baseline,gpt-5.6-sol,0,1.1868233329150826,761,15,776
20 web_nlg,web_nlg_en-test-1466,Full System,gpt-5.6-sol,0,1.0690114998724312,738,15,753
21 dart,dart-test-2278,Baseline,gpt-5.6-sol,0,2.7176588331349194,886,73,959
22 e2e_nlg,e2e_nlg-test-864,Full System,gpt-5.6-sol,0,1.2211443341802806,744,15,759
23 tutto,tutto-validation-6067,Full System,gpt-5.6-sol,0,1.2960889160167426,846,15,861
24 tutto,tutto-validation-1828,Baseline,gpt-5.6-sol,3,7.602055750088766,1250,460,1710
25 web_nlg,web_nlg_en-test-1330,Baseline,gpt-5.6-sol,1,3.8522649579681456,820,130,950
26 e2e_nlg,e2e_nlg-test-447,Full System,gpt-5.6-sol,0,1.4490189589560032,763,15,778
27 sportsett_basketball,5786,Full System,gpt-5.6-sol,2,9.494036082876846,9092,654,9746
28 dart,dart-test-1805,Full System,gpt-5.6-sol,0,1.8678115841466933,771,15,786
29 dart,dart-test-1805,Baseline,gpt-5.6-sol,0,1.2919981661252677,786,15,801
30 sportsett_basketball,5955,Full System,gpt-5.6-sol,3,21.554561167024076,8966,1425,10391
31 dart,dart-test-2278,Full System,gpt-5.6-sol,0,2.0519464160315692,864,68,932
32 tutto,tutto-validation-4467,Full System,gpt-5.6-sol,0,1.1871975830290467,1103,15,1118
33 sportsett_basketball,5372,Baseline,gpt-5.6-sol,1,5.524871040834114,9265,305,9570
34 web_nlg,web_nlg_en-test-1209,Baseline,gpt-5.6-sol,0,1.5396389169618487,815,55,870
35 dart,dart-test-4597,Full System,gpt-5.6-sol,0,2.429595791036263,807,15,822
36 web_nlg,web_nlg_en-test-864,Full System,gpt-5.6-sol,0,1.7385758748278022,761,15,776
37 e2e_nlg,e2e_nlg-test-864,Baseline,gpt-5.6-sol,0,1.668377875117585,746,15,761
38 web_nlg,web_nlg_en-test-1330,Full System,gpt-5.6-sol,0,1.797253540949896,824,52,876
39 e2e_nlg,e2e_nlg-test-209,Baseline,gpt-5.6-sol,0,1.6008541251067072,754,15,769
40 sportsett_basketball,5130,Baseline,gpt-5.6-sol,1,4.513940084027126,9256,209,9465
41 e2e_nlg,e2e_nlg-test-1330,Baseline,gpt-5.6-sol,0,1.126990829750896,793,15,808
42 e2e_nlg,e2e_nlg-test-447,Baseline,gpt-5.6-sol,0,1.7653185420203954,764,15,779
43 e2e_nlg,e2e_nlg-test-476,Full System,gpt-5.6-sol,0,1.135826291050762,789,15,804
44 sportsett_basketball,5955,Baseline,gpt-5.6-sol,1,4.975983374984935,8960,307,9267
45 web_nlg,web_nlg_en-test-859,Full System,gpt-5.6-sol,0,1.2205187501385808,730,15,745
46 web_nlg,web_nlg_en-test-859,Baseline,gpt-5.6-sol,0,2.1456217078957707,730,15,745
47 tutto,tutto-validation-912,Full System,gpt-5.6-sol,0,1.3600447908975184,1075,15,1090
48 tutto,tutto-validation-6067,Baseline,gpt-5.6-sol,1,8.782929416978732,908,537,1445
49 web_nlg,web_nlg_en-test-1209,Full System,gpt-5.6-sol,0,2.3566573751159012,809,100,909
50 dart,dart-test-1828,Full System,gpt-5.6-sol,0,1.0564299169927835,802,15,817
51 sportsett_basketball,6127,Full System,gpt-5.6-sol,4,24.131696166004986,8868,1719,10587

```


Listing 95: R06: submission/program_listings/build/generated/results/gpt56_category_counts.csv

```

1 dataset_id,example_id,condition,category,count
2 totto,totto-validation-912,Baseline,NAME,0
3 totto,totto-validation-912,Baseline,NUMBER,0
4 totto,totto-validation-912,Baseline,WORD,0
5 totto,totto-validation-912,Baseline,CONTEXT,1
6 totto,totto-validation-912,Baseline,NOT_CHECKABLE,0
7 totto,totto-validation-912,Baseline,OTHER,0
8 totto,totto-validation-912,Baseline,OMISSION,0
9 totto,totto-validation-912,Baseline,TASK/FORMAT,1
10 e2e_nlg,e2e_nlg-test-209,Full System,NAME,0
11 e2e_nlg,e2e_nlg-test-209,Full System,NUMBER,0
12 e2e_nlg,e2e_nlg-test-209,Full System,WORD,0
13 e2e_nlg,e2e_nlg-test-209,Full System,CONTEXT,0
14 e2e_nlg,e2e_nlg-test-209,Full System,NOT_CHECKABLE,0
15 e2e_nlg,e2e_nlg-test-209,Full System,OTHER,0
16 e2e_nlg,e2e_nlg-test-209,Full System,OMISSION,0
17 e2e_nlg,e2e_nlg-test-209,Full System,TASK/FORMAT,0
18 e2e_nlg,e2e_nlg-test-476,Baseline,NAME,0
19 e2e_nlg,e2e_nlg-test-476,Baseline,NUMBER,0
20 e2e_nlg,e2e_nlg-test-476,Baseline,WORD,0
21 e2e_nlg,e2e_nlg-test-476,Baseline,CONTEXT,0
22 e2e_nlg,e2e_nlg-test-476,Baseline,NOT_CHECKABLE,0
23 e2e_nlg,e2e_nlg-test-476,Baseline,OTHER,0
24 e2e_nlg,e2e_nlg-test-476,Baseline,OMISSION,0
25 e2e_nlg,e2e_nlg-test-476,Baseline,TASK/FORMAT,0
26 dart,dart-test-1828,Baseline,NAME,0
27 dart,dart-test-1828,Baseline,NUMBER,0
28 dart,dart-test-1828,Baseline,WORD,0
29 dart,dart-test-1828,Baseline,CONTEXT,0
30 dart,dart-test-1828,Baseline,NOT_CHECKABLE,0
31 dart,dart-test-1828,Baseline,OTHER,0
32 dart,dart-test-1828,Baseline,OMISSION,0
33 dart,dart-test-1828,Baseline,TASK/FORMAT,0
34 totto,totto-validation-839,Baseline,NAME,0
35 totto,totto-validation-839,Baseline,NUMBER,0
36 totto,totto-validation-839,Baseline,WORD,0
37 totto,totto-validation-839,Baseline,CONTEXT,0
38 totto,totto-validation-839,Baseline,NOT_CHECKABLE,0
39 totto,totto-validation-839,Baseline,OTHER,0
40 totto,totto-validation-839,Baseline,OMISSION,0
41 totto,totto-validation-839,Baseline,TASK/FORMAT,1
42 dart,dart-test-1791,Baseline,NAME,0
43 dart,dart-test-1791,Baseline,NUMBER,0
44 dart,dart-test-1791,Baseline,WORD,0
45 dart,dart-test-1791,Baseline,CONTEXT,0
46 dart,dart-test-1791,Baseline,NOT_CHECKABLE,0
47 dart,dart-test-1791,Baseline,OTHER,0
48 dart,dart-test-1791,Baseline,OMISSION,0
49 dart,dart-test-1791,Baseline,TASK/FORMAT,0
50 web_nlg,web_nlg_en-test-1466,Baseline,NAME,0
51 web_nlg,web_nlg_en-test-1466,Baseline,NUMBER,0
52 web_nlg,web_nlg_en-test-1466,Baseline,WORD,0
53 web_nlg,web_nlg_en-test-1466,Baseline,CONTEXT,0
54 web_nlg,web_nlg_en-test-1466,Baseline,NOT_CHECKABLE,0
55 web_nlg,web_nlg_en-test-1466,Baseline,OTHER,0
56 web_nlg,web_nlg_en-test-1466,Baseline,OMISSION,0
57 web_nlg,web_nlg_en-test-1466,Baseline,TASK/FORMAT,0
58 sportsett_basketball,6127,Baseline,NAME,0
59 sportsett_basketball,6127,Baseline,NUMBER,0
60 sportsett_basketball,6127,Baseline,WORD,1
61 sportsett_basketball,6127,Baseline,CONTEXT,1
62 sportsett_basketball,6127,Baseline,NOT_CHECKABLE,0
63 sportsett_basketball,6127,Baseline,OTHER,0
64 sportsett_basketball,6127,Baseline,OMISSION,0
65 sportsett_basketball,6127,Baseline,TASK/FORMAT,1
66 totto,totto-validation-4467,Baseline,NAME,0
67 totto,totto-validation-4467,Baseline,NUMBER,0
68 totto,totto-validation-4467,Baseline,WORD,0
69 totto,totto-validation-4467,Baseline,CONTEXT,0
70 totto,totto-validation-4467,Baseline,NOT_CHECKABLE,0
71 totto,totto-validation-4467,Baseline,OTHER,0
72 totto,totto-validation-4467,Baseline,OMISSION,0
73 totto,totto-validation-4467,Baseline,TASK/FORMAT,1
74 e2e_nlg,e2e_nlg-test-1330,Full System,NAME,0
75 e2e_nlg,e2e_nlg-test-1330,Full System,NUMBER,0
76 e2e_nlg,e2e_nlg-test-1330,Full System,WORD,0
77 e2e_nlg,e2e_nlg-test-1330,Full System,CONTEXT,0
78 e2e_nlg,e2e_nlg-test-1330,Full System,NOT_CHECKABLE,0
79 e2e_nlg,e2e_nlg-test-1330,Full System,OTHER,0
80 e2e_nlg,e2e_nlg-test-1330,Full System,OMISSION,0
81 e2e_nlg,e2e_nlg-test-1330,Full System,TASK/FORMAT,0
82 totto,totto-validation-839,Full System,NAME,0
83 totto,totto-validation-839,Full System,NUMBER,0
84 totto,totto-validation-839,Full System,WORD,0
85 totto,totto-validation-839,Full System,CONTEXT,0
86 totto,totto-validation-839,Full System,NOT_CHECKABLE,0
87 totto,totto-validation-839,Full System,OTHER,0
88 totto,totto-validation-839,Full System,OMISSION,1
89 totto,totto-validation-839,Full System,TASK/FORMAT,0

```

```
90 dart,dart-test-4597,Baseline,NAME,0
91 dart,dart-test-4597,Baseline,NUMBER,0
92 dart,dart-test-4597,Baseline,WORD,0
93 dart,dart-test-4597,Baseline,CONTEXT,0
94 dart,dart-test-4597,Baseline,NOT CHECKABLE,0
95 dart,dart-test-4597,Baseline,OTHER,0
96 dart,dart-test-4597,Baseline,OMISSION,0
97 dart,dart-test-4597,Baseline,TASK/FORMAT,0
98 sportsett_basketball,5786,Baseline,NAME,0
99 sportsett_basketball,5786,Baseline,NUMBER,0
100 sportsett_basketball,5786,Baseline,WORD,0
101 sportsett_basketball,5786,Baseline,CONTEXT,0
102 sportsett_basketball,5786,Baseline,NOT CHECKABLE,0
103 sportsett_basketball,5786,Baseline,OTHER,0
104 sportsett_basketball,5786,Baseline,OMISSION,0
105 sportsett_basketball,5786,Baseline,TASK/FORMAT,1
106 sportsett_basketball,5372,Full System,NAME,0
107 sportsett_basketball,5372,Full System,NUMBER,0
108 sportsett_basketball,5372,Full System,WORD,0
109 sportsett_basketball,5372,Full System,CONTEXT,1
110 sportsett_basketball,5372,Full System,NOT CHECKABLE,0
111 sportsett_basketball,5372,Full System,OTHER,0
112 sportsett_basketball,5372,Full System,OMISSION,0
113 sportsett_basketball,5372,Full System,TASK/FORMAT,1
114 dart,dart-test-1791,Full System,NAME,0
115 dart,dart-test-1791,Full System,NUMBER,0
116 dart,dart-test-1791,Full System,WORD,0
117 dart,dart-test-1791,Full System,CONTEXT,0
118 dart,dart-test-1791,Full System,NOT CHECKABLE,0
119 dart,dart-test-1791,Full System,OTHER,0
120 dart,dart-test-1791,Full System,OMISSION,0
121 dart,dart-test-1791,Full System,TASK/FORMAT,0
122 sportsett_basketball,5130,Full System,NAME,0
123 sportsett_basketball,5130,Full System,NUMBER,0
124 sportsett_basketball,5130,Full System,WORD,0
125 sportsett_basketball,5130,Full System,CONTEXT,1
126 sportsett_basketball,5130,Full System,NOT CHECKABLE,0
127 sportsett_basketball,5130,Full System,OTHER,0
128 sportsett_basketball,5130,Full System,OMISSION,0
129 sportsett_basketball,5130,Full System,TASK/FORMAT,1
130 tutto,tutto-validation-1828,Full System,NAME,0
131 tutto,tutto-validation-1828,Full System,NUMBER,0
132 tutto,tutto-validation-1828,Full System,WORD,1
133 tutto,tutto-validation-1828,Full System,CONTEXT,0
134 tutto,tutto-validation-1828,Full System,NOT CHECKABLE,0
135 tutto,tutto-validation-1828,Full System,OTHER,0
136 tutto,tutto-validation-1828,Full System,OMISSION,0
137 tutto,tutto-validation-1828,Full System,TASK/FORMAT,1
138 web_nlg,web_nlg_en-test-864,Baseline,NAME,0
139 web_nlg,web_nlg_en-test-864,Baseline,NUMBER,0
140 web_nlg,web_nlg_en-test-864,Baseline,WORD,0
141 web_nlg,web_nlg_en-test-864,Baseline,CONTEXT,0
142 web_nlg,web_nlg_en-test-864,Baseline,NOT CHECKABLE,0
143 web_nlg,web_nlg_en-test-864,Baseline,OTHER,0
144 web_nlg,web_nlg_en-test-864,Baseline,OMISSION,0
145 web_nlg,web_nlg_en-test-864,Baseline,TASK/FORMAT,0
146 web_nlg,web_nlg_en-test-1466,Full System,NAME,0
147 web_nlg,web_nlg_en-test-1466,Full System,NUMBER,0
148 web_nlg,web_nlg_en-test-1466,Full System,WORD,0
149 web_nlg,web_nlg_en-test-1466,Full System,CONTEXT,0
150 web_nlg,web_nlg_en-test-1466,Full System,NOT CHECKABLE,0
151 web_nlg,web_nlg_en-test-1466,Full System,OTHER,0
152 web_nlg,web_nlg_en-test-1466,Full System,OMISSION,0
153 web_nlg,web_nlg_en-test-1466,Full System,TASK/FORMAT,0
154 dart,dart-test-2278,Baseline,NAME,0
155 dart,dart-test-2278,Baseline,NUMBER,0
156 dart,dart-test-2278,Baseline,WORD,0
157 dart,dart-test-2278,Baseline,CONTEXT,0
158 dart,dart-test-2278,Baseline,NOT CHECKABLE,0
159 dart,dart-test-2278,Baseline,OTHER,0
160 dart,dart-test-2278,Baseline,OMISSION,0
161 dart,dart-test-2278,Baseline,TASK/FORMAT,0
162 e2e_nlg,e2e_nlg-test-864,Full System,NAME,0
163 e2e_nlg,e2e_nlg-test-864,Full System,NUMBER,0
164 e2e_nlg,e2e_nlg-test-864,Full System,WORD,0
165 e2e_nlg,e2e_nlg-test-864,Full System,CONTEXT,0
166 e2e_nlg,e2e_nlg-test-864,Full System,NOT CHECKABLE,0
167 e2e_nlg,e2e_nlg-test-864,Full System,OTHER,0
168 e2e_nlg,e2e_nlg-test-864,Full System,OMISSION,0
169 e2e_nlg,e2e_nlg-test-864,Full System,TASK/FORMAT,0
170 tutto,tutto-validation-6067,Full System,NAME,0
171 tutto,tutto-validation-6067,Full System,NUMBER,0
172 tutto,tutto-validation-6067,Full System,WORD,0
173 tutto,tutto-validation-6067,Full System,CONTEXT,0
174 tutto,tutto-validation-6067,Full System,NOT CHECKABLE,0
175 tutto,tutto-validation-6067,Full System,OTHER,0
176 tutto,tutto-validation-6067,Full System,OMISSION,0
177 tutto,tutto-validation-6067,Full System,TASK/FORMAT,0
178 tutto,tutto-validation-1828,Baseline,NAME,0
179 tutto,tutto-validation-1828,Baseline,NUMBER,0
180 tutto,tutto-validation-1828,Baseline,WORD,0
```


181 tutto,totto-validation-1828,Baseline,CONTEXT,0
182 tutto,totto-validation-1828,Baseline,NOT CHECKABLE,0
183 tutto,totto-validation-1828,Baseline,OTHER,0
184 tutto,totto-validation-1828,Baseline,OMISSION,1
185 tutto,totto-validation-1828,Baseline,TASK/FORMAT,2
186 web_nlg,web_nlg_en-test-1330,Baseline,NAME,0
187 web_nlg,web_nlg_en-test-1330,Baseline,NUMBER,1
188 web_nlg,web_nlg_en-test-1330,Baseline,WORD,0
189 web_nlg,web_nlg_en-test-1330,Baseline,CONTEXT,0
190 web_nlg,web_nlg_en-test-1330,Baseline,NOT CHECKABLE,0
191 web_nlg,web_nlg_en-test-1330,Baseline,OTHER,0
192 web_nlg,web_nlg_en-test-1330,Baseline,OMISSION,0
193 web_nlg,web_nlg_en-test-1330,Baseline,TASK/FORMAT,0
194 e2e_nlg,e2e_nlg-test-447,Full System,NAME,0
195 e2e_nlg,e2e_nlg-test-447,Full System,NUMBER,0
196 e2e_nlg,e2e_nlg-test-447,Full System,WORD,0
197 e2e_nlg,e2e_nlg-test-447,Full System,CONTEXT,0
198 e2e_nlg,e2e_nlg-test-447,Full System,NOT CHECKABLE,0
199 e2e_nlg,e2e_nlg-test-447,Full System,OTHER,0
200 e2e_nlg,e2e_nlg-test-447,Full System,OMISSION,0
201 e2e_nlg,e2e_nlg-test-447,Full System,TASK/FORMAT,0
202 sportsett_basketball,5786,Full System,NAME,0
203 sportsett_basketball,5786,Full System,NUMBER,0
204 sportsett_basketball,5786,Full System,WORD,0
205 sportsett_basketball,5786,Full System,CONTEXT,1
206 sportsett_basketball,5786,Full System,NOT CHECKABLE,0
207 sportsett_basketball,5786,Full System,OTHER,0
208 sportsett_basketball,5786,Full System,OMISSION,0
209 sportsett_basketball,5786,Full System,TASK/FORMAT,1
210 dart,dart-test-1805,Full System,NAME,0
211 dart,dart-test-1805,Full System,NUMBER,0
212 dart,dart-test-1805,Full System,WORD,0
213 dart,dart-test-1805,Full System,CONTEXT,0
214 dart,dart-test-1805,Full System,NOT CHECKABLE,0
215 dart,dart-test-1805,Full System,OTHER,0
216 dart,dart-test-1805,Full System,OMISSION,0
217 dart,dart-test-1805,Full System,TASK/FORMAT,0
218 dart,dart-test-1805,Baseline,NAME,0
219 dart,dart-test-1805,Baseline,NUMBER,0
220 dart,dart-test-1805,Baseline,WORD,0
221 dart,dart-test-1805,Baseline,CONTEXT,0
222 dart,dart-test-1805,Baseline,NOT CHECKABLE,0
223 dart,dart-test-1805,Baseline,OTHER,0
224 dart,dart-test-1805,Baseline,OMISSION,0
225 dart,dart-test-1805,Baseline,TASK/FORMAT,0
226 sportsett_basketball,5955,Full System,NAME,0
227 sportsett_basketball,5955,Full System,NUMBER,0
228 sportsett_basketball,5955,Full System,WORD,0
229 sportsett_basketball,5955,Full System,CONTEXT,1
230 sportsett_basketball,5955,Full System,NOT CHECKABLE,0
231 sportsett_basketball,5955,Full System,OTHER,0
232 sportsett_basketball,5955,Full System,OMISSION,1
233 sportsett_basketball,5955,Full System,TASK/FORMAT,1
234 dart,dart-test-2278,Full System,NAME,0
235 dart,dart-test-2278,Full System,NUMBER,0
236 dart,dart-test-2278,Full System,WORD,0
237 dart,dart-test-2278,Full System,CONTEXT,0
238 dart,dart-test-2278,Full System,NOT CHECKABLE,0
239 dart,dart-test-2278,Full System,OTHER,0
240 dart,dart-test-2278,Full System,OMISSION,0
241 dart,dart-test-2278,Full System,TASK/FORMAT,0
242 tutto,totto-validation-4467,Full System,NAME,0
243 tutto,totto-validation-4467,Full System,NUMBER,0
244 tutto,totto-validation-4467,Full System,WORD,0
245 tutto,totto-validation-4467,Full System,CONTEXT,0
246 tutto,totto-validation-4467,Full System,NOT CHECKABLE,0
247 tutto,totto-validation-4467,Full System,OTHER,0
248 tutto,totto-validation-4467,Full System,OMISSION,0
249 tutto,totto-validation-4467,Full System,TASK/FORMAT,0
250 sportsett_basketball,5372,Baseline,NAME,0
251 sportsett_basketball,5372,Baseline,NUMBER,0
252 sportsett_basketball,5372,Baseline,WORD,0
253 sportsett_basketball,5372,Baseline,CONTEXT,0
254 sportsett_basketball,5372,Baseline,NOT CHECKABLE,0
255 sportsett_basketball,5372,Baseline,OTHER,0
256 sportsett_basketball,5372,Baseline,OMISSION,0
257 sportsett_basketball,5372,Baseline,TASK/FORMAT,1
258 web_nlg,web_nlg_en-test-1209,Baseline,NAME,0
259 web_nlg,web_nlg_en-test-1209,Baseline,NUMBER,0
260 web_nlg,web_nlg_en-test-1209,Baseline,WORD,0
261 web_nlg,web_nlg_en-test-1209,Baseline,CONTEXT,0
262 web_nlg,web_nlg_en-test-1209,Baseline,NOT CHECKABLE,0
263 web_nlg,web_nlg_en-test-1209,Baseline,OTHER,0
264 web_nlg,web_nlg_en-test-1209,Baseline,OMISSION,0
265 web_nlg,web_nlg_en-test-1209,Baseline,TASK/FORMAT,0
266 dart,dart-test-4597,Full System,NAME,0
267 dart,dart-test-4597,Full System,NUMBER,0
268 dart,dart-test-4597,Full System,WORD,0
269 dart,dart-test-4597,Full System,CONTEXT,0
270 dart,dart-test-4597,Full System,NOT CHECKABLE,0
271 dart,dart-test-4597,Full System,OTHER,0

```

272 dart,dart-test-4597,Full System,OMISSION,0
273 dart,dart-test-4597,Full System,TASK/FORMAT,0
274 web_nlg,web_nlg_en-test-864,Full System,NAME,0
275 web_nlg,web_nlg_en-test-864,Full System,NUMBER,0
276 web_nlg,web_nlg_en-test-864,Full System,WORD,0
277 web_nlg,web_nlg_en-test-864,Full System,CONTEXT,0
278 web_nlg,web_nlg_en-test-864,Full System,NOT CHECKABLE,0
279 web_nlg,web_nlg_en-test-864,Full System,OTHER,0
280 web_nlg,web_nlg_en-test-864,Full System,OMISSION,0
281 web_nlg,web_nlg_en-test-864,Full System,TASK/FORMAT,0
282 e2e_nlg,e2e_nlg-test-864,Baseline,NAME,0
283 e2e_nlg,e2e_nlg-test-864,Baseline,NUMBER,0
284 e2e_nlg,e2e_nlg-test-864,Baseline,WORD,0
285 e2e_nlg,e2e_nlg-test-864,Baseline,CONTEXT,0
286 e2e_nlg,e2e_nlg-test-864,Baseline,NOT CHECKABLE,0
287 e2e_nlg,e2e_nlg-test-864,Baseline,OTHER,0
288 e2e_nlg,e2e_nlg-test-864,Baseline,OMISSION,0
289 e2e_nlg,e2e_nlg-test-864,Baseline,TASK/FORMAT,0
290 web_nlg,web_nlg_en-test-1330,Full System,NAME,0
291 web_nlg,web_nlg_en-test-1330,Full System,NUMBER,0
292 web_nlg,web_nlg_en-test-1330,Full System,WORD,0
293 web_nlg,web_nlg_en-test-1330,Full System,CONTEXT,0
294 web_nlg,web_nlg_en-test-1330,Full System,NOT CHECKABLE,0
295 web_nlg,web_nlg_en-test-1330,Full System,OTHER,0
296 web_nlg,web_nlg_en-test-1330,Full System,OMISSION,0
297 web_nlg,web_nlg_en-test-1330,Full System,TASK/FORMAT,0
298 e2e_nlg,e2e_nlg-test-209,Baseline,NAME,0
299 e2e_nlg,e2e_nlg-test-209,Baseline,NUMBER,0
300 e2e_nlg,e2e_nlg-test-209,Baseline,WORD,0
301 e2e_nlg,e2e_nlg-test-209,Baseline,CONTEXT,0
302 e2e_nlg,e2e_nlg-test-209,Baseline,NOT CHECKABLE,0
303 e2e_nlg,e2e_nlg-test-209,Baseline,OTHER,0
304 e2e_nlg,e2e_nlg-test-209,Baseline,OMISSION,0
305 e2e_nlg,e2e_nlg-test-209,Baseline,TASK/FORMAT,0
306 sportsett_basketball,5130,Baseline,NAME,0
307 sportsett_basketball,5130,Baseline,NUMBER,0
308 sportsett_basketball,5130,Baseline,WORD,0
309 sportsett_basketball,5130,Baseline,CONTEXT,0
310 sportsett_basketball,5130,Baseline,NOT CHECKABLE,0
311 sportsett_basketball,5130,Baseline,OTHER,0
312 sportsett_basketball,5130,Baseline,OMISSION,0
313 sportsett_basketball,5130,Baseline,TASK/FORMAT,1
314 e2e_nlg,e2e_nlg-test-1330,Baseline,NAME,0
315 e2e_nlg,e2e_nlg-test-1330,Baseline,NUMBER,0
316 e2e_nlg,e2e_nlg-test-1330,Baseline,WORD,0
317 e2e_nlg,e2e_nlg-test-1330,Baseline,CONTEXT,0
318 e2e_nlg,e2e_nlg-test-1330,Baseline,NOT CHECKABLE,0
319 e2e_nlg,e2e_nlg-test-1330,Baseline,OTHER,0
320 e2e_nlg,e2e_nlg-test-1330,Baseline,OMISSION,0
321 e2e_nlg,e2e_nlg-test-1330,Baseline,TASK/FORMAT,0
322 e2e_nlg,e2e_nlg-test-447,Baseline,NAME,0
323 e2e_nlg,e2e_nlg-test-447,Baseline,NUMBER,0
324 e2e_nlg,e2e_nlg-test-447,Baseline,WORD,0
325 e2e_nlg,e2e_nlg-test-447,Baseline,CONTEXT,0
326 e2e_nlg,e2e_nlg-test-447,Baseline,NOT CHECKABLE,0
327 e2e_nlg,e2e_nlg-test-447,Baseline,OTHER,0
328 e2e_nlg,e2e_nlg-test-447,Baseline,OMISSION,0
329 e2e_nlg,e2e_nlg-test-447,Baseline,TASK/FORMAT,0
330 e2e_nlg,e2e_nlg-test-476,Full System,NAME,0
331 e2e_nlg,e2e_nlg-test-476,Full System,NUMBER,0
332 e2e_nlg,e2e_nlg-test-476,Full System,WORD,0
333 e2e_nlg,e2e_nlg-test-476,Full System,CONTEXT,0
334 e2e_nlg,e2e_nlg-test-476,Full System,NOT CHECKABLE,0
335 e2e_nlg,e2e_nlg-test-476,Full System,OTHER,0
336 e2e_nlg,e2e_nlg-test-476,Full System,OMISSION,0
337 e2e_nlg,e2e_nlg-test-476,Full System,TASK/FORMAT,0
338 sportsett_basketball,5955,Baseline,NAME,0
339 sportsett_basketball,5955,Baseline,NUMBER,0
340 sportsett_basketball,5955,Baseline,WORD,0
341 sportsett_basketball,5955,Baseline,CONTEXT,0
342 sportsett_basketball,5955,Baseline,NOT CHECKABLE,0
343 sportsett_basketball,5955,Baseline,OTHER,0
344 sportsett_basketball,5955,Baseline,OMISSION,0
345 sportsett_basketball,5955,Baseline,TASK/FORMAT,1
346 web_nlg,web_nlg_en-test-859,Full System,NAME,0
347 web_nlg,web_nlg_en-test-859,Full System,NUMBER,0
348 web_nlg,web_nlg_en-test-859,Full System,WORD,0
349 web_nlg,web_nlg_en-test-859,Full System,CONTEXT,0
350 web_nlg,web_nlg_en-test-859,Full System,NOT CHECKABLE,0
351 web_nlg,web_nlg_en-test-859,Full System,OTHER,0
352 web_nlg,web_nlg_en-test-859,Full System,OMISSION,0
353 web_nlg,web_nlg_en-test-859,Full System,TASK/FORMAT,0
354 web_nlg,web_nlg_en-test-859,Baseline,NAME,0
355 web_nlg,web_nlg_en-test-859,Baseline,NUMBER,0
356 web_nlg,web_nlg_en-test-859,Baseline,WORD,0
357 web_nlg,web_nlg_en-test-859,Baseline,CONTEXT,0
358 web_nlg,web_nlg_en-test-859,Baseline,NOT CHECKABLE,0
359 web_nlg,web_nlg_en-test-859,Baseline,OTHER,0
360 web_nlg,web_nlg_en-test-859,Baseline,OMISSION,0
361 web_nlg,web_nlg_en-test-859,Baseline,TASK/FORMAT,0
362 tutto,tutto-validation-912,Full System,NAME,0

```

```
363 tutto,tutto-validation-912,Full System,NUMBER,0
364 tutto,tutto-validation-912,Full System,WORD,0
365 tutto,tutto-validation-912,Full System,CONTEXT,0
366 tutto,tutto-validation-912,Full System,NOT CHECKABLE,0
367 tutto,tutto-validation-912,Full System,OTHER,0
368 tutto,tutto-validation-912,Full System,OMISSION,0
369 tutto,tutto-validation-912,Full System,TASK/FORMAT,0
370 tutto,tutto-validation-6067,Baseline,NAME,0
371 tutto,tutto-validation-6067,Baseline,NUMBER,0
372 tutto,tutto-validation-6067,Baseline,WORD,0
373 tutto,tutto-validation-6067,Baseline,CONTEXT,0
374 tutto,tutto-validation-6067,Baseline,NOT CHECKABLE,0
375 tutto,tutto-validation-6067,Baseline,OTHER,0
376 tutto,tutto-validation-6067,Baseline,OMISSION,0
377 tutto,tutto-validation-6067,Baseline,TASK/FORMAT,1
378 web_nlg,web_nlg_en-test-1209,Full System,NAME,0
379 web_nlg,web_nlg_en-test-1209,Full System,NUMBER,0
380 web_nlg,web_nlg_en-test-1209,Full System,WORD,0
381 web_nlg,web_nlg_en-test-1209,Full System,CONTEXT,0
382 web_nlg,web_nlg_en-test-1209,Full System,NOT CHECKABLE,0
383 web_nlg,web_nlg_en-test-1209,Full System,OTHER,0
384 web_nlg,web_nlg_en-test-1209,Full System,OMISSION,0
385 web_nlg,web_nlg_en-test-1209,Full System,TASK/FORMAT,0
386 dart,dart-test-1828,Full System,NAME,0
387 dart,dart-test-1828,Full System,NUMBER,0
388 dart,dart-test-1828,Full System,WORD,0
389 dart,dart-test-1828,Full System,CONTEXT,0
390 dart,dart-test-1828,Full System,NOT CHECKABLE,0
391 dart,dart-test-1828,Full System,OTHER,0
392 dart,dart-test-1828,Full System,OMISSION,0
393 dart,dart-test-1828,Full System,TASK/FORMAT,0
394 sportsett_basketball,6127,Full System,NAME,0
395 sportsett_basketball,6127,Full System,NUMBER,0
396 sportsett_basketball,6127,Full System,WORD,1
397 sportsett_basketball,6127,Full System,CONTEXT,2
398 sportsett_basketball,6127,Full System,NOT CHECKABLE,0
399 sportsett_basketball,6127,Full System,OTHER,0
400 sportsett_basketball,6127,Full System,OMISSION,0
401 sportsett_basketball,6127,Full System,TASK/FORMAT,1
```

6 Listing Manifest

This machine-readable manifest records the category, path, line count and SHA-256 digest of every included listing.

Listing 96: M01: Program-listing inclusion manifest

```

1 [
2   {
3     "category": "program",
4     "lines": 6,
5     "path": "experiments/llm_only_pipeline/src/table2text_llm_only/__init__.py",
6     "sha256": "3cbc85a9423931bcc1fd9d9269e1d6616a7b7ad0adc46c6001fc3a65624a1a2a"
7   },
8   {
9     "category": "program",
10    "lines": 66,
11    "path": "experiments/llm_only_pipeline/src/table2text_llm_only/backend.py",
12    "sha256": "c810106b84aa9d948fa2dfd08a92e95db0df2e7a51b050c038f2e6bf15949edf"
13  },
14  {
15    "category": "program",
16    "lines": 216,
17    "path": "experiments/llm_only_pipeline/src/table2text_llm_only/client.py",
18    "sha256": "ca613fc355af4491b702d8e2aeb93e5367448cfae2fdcfc4539d870a6bd267a"
19  },
20  {
21    "category": "program",
22    "lines": 270,
23    "path": "experiments/llm_only_pipeline/src/table2text_llm_only/schemas.py",
24    "sha256": "07cde88256b20381f53a8dcd2f9cd23ed8865cc1e8dc501e6cf29eb11d26b6f0"
25  },
26  {
27    "category": "program",
28    "lines": 644,
29    "path": "experiments/llm_only_pipeline/src/table2text_llm_only/workflow.py",
30    "sha256": "c0101870206667cfff06ad72faa8723381f7a07295eaff20af1454cca4ff8ef2"
31  },
32  {
33    "category": "program",
34    "lines": 554,
35    "path": "submission/program_listings/build_program_listings.py",
36    "sha256": "035fd29d6b71fc02360afefe1829eb0fcc7ec726f08622213e45c17ba00ed283"
37  },
38  {
39    "category": "program",
40    "lines": 342,
41    "path": "submission/software_archive/build_software_submission.py",
42    "sha256": "0b4ff388d67fa33e70090d8670f83d4378d41ca090bf81a0bd7705e995247050"
43  },
44  {
45    "category": "program",
46    "lines": 12,
47    "path": "submission/software_archive/examples/run_deterministic_demo.sh",
48    "sha256": "b9fbc51fa272db7c786dd488a1db82528ef7738c9af7f607cb0d20c0697b7fcd"
49  },
50  {
51    "category": "program",
52    "lines": 799,
53    "path": "table2text_pydanticai/evaluation/notebooks/build_dissertation_visual_evaluations.py",
54    "sha256": "b47313b09f62481fb34d7fb05c4297235a59babc08c6cf54ce3752a9ae26ea9d"
55  },
56  {
57    "category": "program",
58    "lines": 92,
59    "path": "table2text_pydanticai/evaluation/notebooks/build_protected_holdout_notebook.py",
60    "sha256": "f0ae56419ef12e8395b58bdfc5f21d8c6e4445cb331b94f5b992f97152ea2431"
61  },
62  {
63    "category": "program",
64    "lines": 1738,
65    "path": "table2text_pydanticai/evaluation/notebooks/protected_holdout_full_system_evaluation.py",
66    "sha256": "d1c90873e30a6e203470d0bd0f188f9fe2eba52cd13e9ee6ebcc40a035137c82"
67  },
68  {
69    "category": "program",
70    "lines": 1213,
71    "path": "table2text_pydanticai/evaluation/notebooks/task_aware_direct_baseline_evaluation.py",
72    "sha256": "a91a7d88ba11933f92d85eab1930f998a55522c45f957f449ff482a7f5b2ee3"
73  },
74  {
75    "category": "program",
76    "lines": 493,
77    "path": "table2text_pydanticai/evaluation/scripts/annotations/build_interactive_gpt56_annotation_artifacts.py",
78    "sha256": "fc75987bfe9d703e72a62c06c8919dd13614159880a32d1a98854dda9f73d323"
79  },
80  {
81    "category": "program",
82    "lines": 1226,
83    "path": "table2text_pydanticai/evaluation/scripts/build_protected_holdout_results_bank.py",

```

```

84 "sha256": "3d44f5c17d80b7798356b426c818d9146865b1156392538a397758c5d03e5495"
85 },
86 {
87   "category": "program",
88   "lines": 345,
89   "path": "table2text_pydanticai/evaluation/scripts/figures/build_evaluation_design_flow.py",
90   "sha256": "f79034e386fa373016c8516b6638a46e3215e7ca4c50b55d56c97c5785a6f103"
91 },
92 {
93   "category": "program",
94   "lines": 350,
95   "path": "table2text_pydanticai/evaluation/scripts/figures/build_gpt56_sol_annotation_figure.py",
96   "sha256": "5514fa7ea58122536215eb7ffdb26bbf1d69414eb9674d6892280ce71c77df31"
97 },
98 {
99   "category": "program",
100   "lines": 951,
101   "path": "table2text_pydanticai/evaluation/scripts/reports/build_task_aware_complete_evaluation_dossier.py",
102   "sha256": "2bec2271c079dc4339aaff66dd0d49cf23ffbc6a74c1f4eb0bd9bfbba6b3d8be"
103 },
104 {
105   "category": "program",
106   "lines": 8,
107   "path": "table2text_pydanticai/src/table2text/__init__.py",
108   "sha256": "2a6b8db0813173a6a11611995fad0ca35185f0b2225355d5366804527ed6716d"
109 },
110 {
111   "category": "program",
112   "lines": 7,
113   "path": "table2text_pydanticai/src/table2text/__main__.py",
114   "sha256": "a9abe0c45c748c6f8e6f00669b1160829f0ef06f247eee4466e9d9c5eda919bb"
115 },
116 {
117   "category": "program",
118   "lines": 4719,
119   "path": "table2text_pydanticai/src/table2text/agents.py",
120   "sha256": "b81f8ea1fcbc41c8a408a96907159632f3040d1f09782aa0b40c403e9a3055b4"
121 },
122 {
123   "category": "program",
124   "lines": 4980,
125   "path": "table2text_pydanticai/src/table2text/analytics.py",
126   "sha256": "56bf8ec43497f5191d12b1f9d31e95ad6461cdfad1612739f8ef945ee0d143d9"
127 },
128 {
129   "category": "program",
130   "lines": 9892,
131   "path": "table2text_pydanticai/src/table2text/audit.py",
132   "sha256": "dbc996d9478611b8871615b3320dd4cc8f94d3243e4f088913077a56d22446c1"
133 },
134 {
135   "category": "program",
136   "lines": 5503,
137   "path": "table2text_pydanticai/src/table2text/capabilities.py",
138   "sha256": "5dca7ff823a880283705c719dbd5442a8169ac53a925128f1e50f42891df311c"
139 },
140 {
141   "category": "program",
142   "lines": 180,
143   "path": "table2text_pydanticai/src/table2text/cli.py",
144   "sha256": "13f570f80f14eab42ce81e8f58a227553a9cc062287da56cd7793077129f8579"
145 },
146 {
147   "category": "program",
148   "lines": 432,
149   "path": "table2text_pydanticai/src/table2text/config.py",
150   "sha256": "e40bc641d03913c55a27fdd53909b4efc15f641a9157476a155e724a9f28ffc6"
151 },
152 {
153   "category": "program",
154   "lines": 967,
155   "path": "table2text_pydanticai/src/table2text/data.py",
156   "sha256": "ca2b7f91c4b693ddb475202fd7820ad34c65cc489a66bdcd4e64add5b74e4f77"
157 },
158 {
159   "category": "program",
160   "lines": 60,
161   "path": "table2text_pydanticai/src/table2text/evaluation/__init__.py",
162   "sha256": "bbb5ee5c1f516cf6df02d2725a1a63f30bec283e187847413e78984fb62ae478"
163 },
164 {
165   "category": "program",
166   "lines": 147,
167   "path": "table2text_pydanticai/src/table2text/evaluation/alignscore_client.py",
168   "sha256": "b19f0dcec1f803a51f8086a397131ddab01ea4058028387ec408be69b0727db7"
169 },
170 {
171   "category": "program",
172   "lines": 361,
173   "path": "table2text_pydanticai/src/table2text/evaluation/cli.py",
174   "sha256": "05111037ca611797995b637e4701920a213f77318e55cdeed8e3e0d29dd1e5ac"

```

```

175 },
176 {
177     "category": "program",
178     "lines": 871,
179     "path": "table2text_pydanticai/src/table2text/evaluation/datasets.py",
180     "sha256": "eca0cb2dc17b17bba89d743aa7cda8b2e2aa4d19440080b7e83a373887c23c00"
181 },
182 {
183     "category": "program",
184     "lines": 478,
185     "path": "table2text_pydanticai/src/table2text/evaluation/deepeval_metrics.py",
186     "sha256": "c825f9c6964634d51adaab0beb8c724e9eb5a3a677bdac5aaef06f644cacea3b"
187 },
188 {
189     "category": "program",
190     "lines": 117,
191     "path": "table2text_pydanticai/src/table2text/evaluation/diagnostics.py",
192     "sha256": "37c90a1f5beebf2d5ae5b6c06b275646e44a81978b7f5c7fe5b86c55f20cdf4"
193 },
194 {
195     "category": "program",
196     "lines": 298,
197     "path": "table2text_pydanticai/src/table2text/evaluation/external_factuality.py",
198     "sha256": "dbbd5c1875f53248ab12d090537ef8c61abda97fc59eb2cbfdbbb762203f02bb"
199 },
200 {
201     "category": "program",
202     "lines": 773,
203     "path": "table2text_pydanticai/src/table2text/evaluation/generation.py",
204     "sha256": "cd3e7a2d4c04cb1bca2d97ad36a6a4ae279aa95f3b621ed64b7caa6c9009ce40"
205 },
206 {
207     "category": "program",
208     "lines": 254,
209     "path": "table2text_pydanticai/src/table2text/evaluation/human_evaluation.py",
210     "sha256": "733269e61e8651b99f42d04bd0625f4facf8abb5eb7e4e9711c6a22d5005d5b5"
211 },
212 {
213     "category": "program",
214     "lines": 305,
215     "path": "table2text_pydanticai/src/table2text/evaluation/llm_judge_annotations.py",
216     "sha256": "a51fcb529210af7dfc815dc1a0483e2a1024c8f9f3cb0493753faac2109de0f3"
217 },
218 {
219     "category": "program",
220     "lines": 368,
221     "path": "table2text_pydanticai/src/table2text/evaluation/models.py",
222     "sha256": "0c5087b3d145a7a26a9b8b8c61116ef9adc000b3d8906a6fac2ffbe061fe623d"
223 },
224 {
225     "category": "program",
226     "lines": 319,
227     "path": "table2text_pydanticai/src/table2text/evaluation/notebook.py",
228     "sha256": "b4f80a7f23ade4cf0fab51251be72921083af03a2b9761ea4b0ac926dd1758ea"
229 },
230 {
231     "category": "program",
232     "lines": 1135,
233     "path": "table2text_pydanticai/src/table2text/evaluation/reference_metrics.py",
234     "sha256": "504b928d9821e0849212bd0081cf40f1e7826d534a11e9eae464b7ca00d66d44"
235 },
236 {
237     "category": "program",
238     "lines": 307,
239     "path": "table2text_pydanticai/src/table2text/evaluation/statistics.py",
240     "sha256": "86c476dd3c9898482800c9b8333ed7bb37164e76eb135afec86a3b9ca8a303ea"
241 },
242 {
243     "category": "program",
244     "lines": 221,
245     "path": "table2text_pydanticai/src/table2text/evaluation_backends.py",
246     "sha256": "3dec2e68aad35f109fbbf6d56e0c6c7f68bc00c427a41d31130fab2175e7042c"
247 },
248 {
249     "category": "program",
250     "lines": 403,
251     "path": "table2text_pydanticai/src/table2text/narrative.py",
252     "sha256": "b7dd6f00cd29f74accf4c4271d2cb05aa23f18b284778eb6da5a83562aa1edf2"
253 },
254 {
255     "category": "program",
256     "lines": 1303,
257     "path": "table2text_pydanticai/src/table2text/schemas.py",
258     "sha256": "fa248c3463fd70b63b99b5bce2e3a61c41bb63d26d7d7e1ed577c4ffbf07eb76"
259 },
260 {
261     "category": "program",
262     "lines": 652,
263     "path": "table2text_pydanticai/src/table2text/structure.py",
264     "sha256": "498e104a7e7a289e11c96eb7a2a7a77c9f2f3ae16ecdb5d63cefc1dd96b42518"
265 },

```



```

266 {
267   "category": "program",
268   "lines": 421,
269   "path": "table2text_pydanticai/src/table2text/task_contracts.py",
270   "sha256": "9b69069e3b197a006b04bbb36020b83464c3833aa66e36018dfbeb2262d62944"
271 },
272 {
273   "category": "program",
274   "lines": 4304,
275   "path": "table2text_pydanticai/src/table2text/workflow.py",
276   "sha256": "4a9c3ac367293b91629999ff2b67734d24c17d916f9f64ef13ee75119775f2c9"
277 },
278 {
279   "category": "test",
280   "lines": 125,
281   "path": "experiments/llm_only_pipeline/tests/test_workflow.py",
282   "sha256": "93f2208e0f50dc7ef7d0a6a5e0812e6ee5d90636c964375d808898ec087f5894"
283 },
284 {
285   "category": "test",
286   "lines": 2094,
287   "path": "table2text_pydanticai/tests/test_reference_evaluation.py",
288   "sha256": "c20cbd4d8c7ab7382da4a08e5a801e35033ee3a2e048a1be3184dac75fe4229b"
289 },
290 {
291   "category": "test",
292   "lines": 4547,
293   "path": "table2text_pydanticai/tests/test_semantic_event_pipeline.py",
294   "sha256": "6ad8e66bcd265504ccd4843012002b963a2d653baf974bebe379b74837f115a5"
295 },
296 {
297   "category": "test",
298   "lines": 5629,
299   "path": "table2text_pydanticai/tests/test_smoke.py",
300   "sha256": "d500518b21038a58a3ad3d833ecf80e3f903d1ddeddfdbd3b00d75d846f384d2"
301 },
302 {
303   "category": "configuration",
304   "lines": 7,
305   "path": "experiments/llm_only_pipeline/.env.example",
306   "sha256": "e6c29db78326a06ade7342df611d2f047475a59d383c1bf847a0ce6174a046ca"
307 },
308 {
309   "category": "configuration",
310   "lines": 46,
311   "path": "experiments/llm_only_pipeline/config/variants.json",
312   "sha256": "2bfacea3de17449593e14b598ecb0a037a8f77e7a4125444eac259a71c282b73"
313 },
314 {
315   "category": "configuration",
316   "lines": 32,
317   "path": "experiments/llm_only_pipeline/pyproject.toml",
318   "sha256": "b841917576264f59f2161c11ec885a3bbc5a5d3d3c81e4fe66c5450bdc399639"
319 },
320 {
321   "category": "configuration",
322   "lines": 153,
323   "path": "table2text_pydanticai/.env.example",
324   "sha256": "2051a4e9097f677370cfdc93855d409dacf1418b4b29058d3683f3dc80b488b9"
325 },
326 {
327   "category": "configuration",
328   "lines": 596,
329   "path": "table2text_pydanticai/evaluation/config/datasets.json",
330   "sha256": "d486459481ae41ffc3252034f56fbeb9aeb67e176148f58e5dbd5d1f0c572d0"
331 },
332 {
333   "category": "configuration",
334   "lines": 63,
335   "path": "table2text_pydanticai/evaluation/config/metrics.json",
336   "sha256": "81694267ef8eb23e520832742bdf68b3e053a46156ff3f6c882a6ebd0b545a28"
337 },
338 {
339   "category": "configuration",
340   "lines": 61,
341   "path": "table2text_pydanticai/evaluation/config/metrics_reference_similarity.json",
342   "sha256": "fdfbfff7924b32e0a0d40309587a5e0975149dcbb1afbb1afe73e9cd394a3f76e"
343 },
344 {
345   "category": "configuration",
346   "lines": 53,
347   "path": "table2text_pydanticai/evaluation/config/metrics_source_grounded.json",
348   "sha256": "4eea544e205ffb5e3b889df98171e02cec692e087c9299f5205ea633a8a4cc1e"
349 },
350 {
351   "category": "configuration",
352   "lines": 106,
353   "path": "table2text_pydanticai/evaluation/config/variants.json",
354   "sha256": "1ddb805f58231b64e5d5bed1951e63fef4bdaa0612264669ef6bd5c15407e7e5"
355 },
356 {

```

```

357     "category": "configuration",
358     "lines": 85,
359     "path": "table2text_pydanticai/evaluation/config/variants_ablation.json",
360     "sha256": "aa76cebe8834755cf1ea3088ea8f9b940412861c133983dec820d8bee201c931"
361 },
362 {
363     "category": "configuration",
364     "lines": 23,
365     "path": "table2text_pydanticai/evaluation/config/variants_raw_deepseek_v4_flash.json",
366     "sha256": "fdd7d214b32cfb742985fa183c492cc7abe85f0a2fae32c379583bc951a14377"
367 },
368 {
369     "category": "configuration",
370     "lines": 23,
371     "path": "table2text_pydanticai/evaluation/config/variants_raw_deepseek_v4_pro.json",
372     "sha256": "dd70c7446a39a66e1c8fd9b80831b01ad97437098e59c16dfe2522e80315a6c7"
373 },
374 {
375     "category": "configuration",
376     "lines": 72,
377     "path": "table2text_pydanticai/evaluation/protected_holdout_full_system/config/protected_full_system_flash.json",
378     "sha256": "1fe2242bf777f26ada83114301c7c1fc11b6c25f613fa9d42e0b9484e4555d85"
379 },
380 {
381     "category": "configuration",
382     "lines": 99,
383     "path": "table2text_pydanticai/pyproject.toml",
384     "sha256": "88be1e2bb349accf96bc6e57f18e958e12af6022670518da7286d453d758444f"
385 },
386 {
387     "category": "data",
388     "lines": 50,
389     "path": "submission/program_listings/build/generated/data/dart__dart-test-1791.json",
390     "sha256": "a39424dae1fe234a9c3a518493fb5a49f36918764b81838ae39b1582c587c0ee"
391 },
392 {
393     "category": "data",
394     "lines": 50,
395     "path": "submission/program_listings/build/generated/data/dart__dart-test-1805.json",
396     "sha256": "5c4ffbe23d22a09ebf3a31f4951e6bfa1bfbf0cea4daf346befb6794a39f3fa6"
397 },
398 {
399     "category": "data",
400     "lines": 50,
401     "path": "submission/program_listings/build/generated/data/dart__dart-test-1828.json",
402     "sha256": "91c585fb8457a6b39ec00dbbde8a3d1d32e14c2fe54f7925512ef618a0bb1197"
403 },
404 {
405     "category": "data",
406     "lines": 70,
407     "path": "submission/program_listings/build/generated/data/dart__dart-test-2278.json",
408     "sha256": "700a6c218e74e182fd97cd66fda2791ead3c698ad4b4c9688730335915aabea6"
409 },
410 {
411     "category": "data",
412     "lines": 90,
413     "path": "submission/program_listings/build/generated/data/dart__dart-test-4597.json",
414     "sha256": "5bde46ff72a9ec1888b55ec4497bd19e9717cd961e0275f7ca270f5c35ea1069"
415 },
416 {
417     "category": "data",
418     "lines": 46,
419     "path": "submission/program_listings/build/generated/data/e2e_nlg__e2e_nlg-test-1330.json",
420     "sha256": "dff2645f87fc35c291df3b5f0f47bdf9270cba50e965a0f3d902560ed28b6fc5"
421 },
422 {
423     "category": "data",
424     "lines": 34,
425     "path": "submission/program_listings/build/generated/data/e2e_nlg__e2e_nlg-test-209.json",
426     "sha256": "b1e80fa82f6422088ff91ffdeee2e3093446e8a593a17e9c12426c0dd42d2b69"
427 },
428 {
429     "category": "data",
430     "lines": 38,
431     "path": "submission/program_listings/build/generated/data/e2e_nlg__e2e_nlg-test-447.json",
432     "sha256": "db17d78d0315372710b01a297090fc2c32da0f74cbb5d66e11e85b2a071b1bbf"
433 },
434 {
435     "category": "data",
436     "lines": 46,
437     "path": "submission/program_listings/build/generated/data/e2e_nlg__e2e_nlg-test-476.json",
438     "sha256": "9e59f97b7a56ea4e284a4e54d26c58dd0301d0d1df45d6c511c33fa4dab6b5a5"
439 },
440 {
441     "category": "data",
442     "lines": 30,
443     "path": "submission/program_listings/build/generated/data/e2e_nlg__e2e_nlg-test-864.json",
444     "sha256": "2fe2d14d5d2905fe50f6ebdc86861d4f5f48217160a0c4eb601d36769687c9cd"
445 },
446 {
447     "category": "data",

```



```

448     "lines": 1109,
449     "path": "submission/program_listings/build/generated/data/sportsett_basketball__5130.json",
450     "sha256": "0811ed5a23b45145be2c522abdd67936b6c588973c8925be103f79663eaa0992"
451 },
452 {
453     "category": "data",
454     "lines": 1109,
455     "path": "submission/program_listings/build/generated/data/sportsett_basketball__5372.json",
456     "sha256": "52ee950819131c32bf1a5d664925f923f8e0f8dc8dbc1a4b2f6332f9772e4a4b"
457 },
458 {
459     "category": "data",
460     "lines": 1055,
461     "path": "submission/program_listings/build/generated/data/sportsett_basketball__5786.json",
462     "sha256": "6471eb2a87114849e62f41513d4976399a24a6def9418f3f1dc9ee2c8007fddc"
463 },
464 {
465     "category": "data",
466     "lines": 1055,
467     "path": "submission/program_listings/build/generated/data/sportsett_basketball__5955.json",
468     "sha256": "ac9b26faecdc39704a43bfe8a3d3c04edbd829f95acf1e54a61eed6e1ecbe6ce"
469 },
470 {
471     "category": "data",
472     "lines": 1028,
473     "path": "submission/program_listings/build/generated/data/sportsett_basketball__6127.json",
474     "sha256": "6d51a4d0529786a7c7678b10139bfd073e7e23965dc29f24a2363998b4063506"
475 },
476 {
477     "category": "data",
478     "lines": 940,
479     "path": "submission/program_listings/build/generated/data/totto__totto-validation-1828.json",
480     "sha256": "ed8d13556a4d001ce13780a6e77823982c700381618dd046c92fb5be16a9aec3"
481 },
482 {
483     "category": "data",
484     "lines": 455,
485     "path": "submission/program_listings/build/generated/data/totto__totto-validation-4467.json",
486     "sha256": "15fb49776ce53d18b3623ae0d4c637bacf00976c31a351038cc71044b2faa890"
487 },
488 {
489     "category": "data",
490     "lines": 282,
491     "path": "submission/program_listings/build/generated/data/totto__totto-validation-6067.json",
492     "sha256": "5477fe250112015bfb8c94585781589956bac8e668f89ca9c0bb94bac914113c"
493 },
494 {
495     "category": "data",
496     "lines": 192,
497     "path": "submission/program_listings/build/generated/data/totto__totto-validation-839.json",
498     "sha256": "d85d3ea8e3b48460e6315401e02184f7e44d091d01d0df9117f6a40e635fb21d"
499 },
500 {
501     "category": "data",
502     "lines": 766,
503     "path": "submission/program_listings/build/generated/data/totto__totto-validation-912.json",
504     "sha256": "d2e30cc093dea4e5b08d32c98d0602fb50f6911fba24f378ef615e48a2251243"
505 },
506 {
507     "category": "data",
508     "lines": 23,
509     "path": "submission/program_listings/build/generated/data/web_nlg__web_nlg_en-test-1209.json",
510     "sha256": "b3e8b46368029776e74ce99b02fc6a3be418b428b4344a0b46c37c44ed0d3415"
511 },
512 {
513     "category": "data",
514     "lines": 24,
515     "path": "submission/program_listings/build/generated/data/web_nlg__web_nlg_en-test-1330.json",
516     "sha256": "1d7d613add9dd9a2d3532e864f70716c9b709808735d0be76f7e468753598b82"
517 },
518 {
519     "category": "data",
520     "lines": 20,
521     "path": "submission/program_listings/build/generated/data/web_nlg__web_nlg_en-test-1466.json",
522     "sha256": "3f8a0517834612e255ec91b414b8cd9121f2382b5b59619bb4b89c561477532b"
523 },
524 {
525     "category": "data",
526     "lines": 20,
527     "path": "submission/program_listings/build/generated/data/web_nlg__web_nlg_en-test-859.json",
528     "sha256": "1506613d3b74d83287d22e131023ccbf35278008509f959e0ad5584e8362625"
529 },
530 {
531     "category": "data",
532     "lines": 21,
533     "path": "submission/program_listings/build/generated/data/web_nlg__web_nlg_en-test-864.json",
534     "sha256": "e89e0c56711d0e9672247f978e84f4a2177ebf4d1b9437f20482ba87b69528df"
535 },
536 {
537     "category": "result",
538     "lines": 527,

```

```
539   "path": "submission/program_listings/build/generated/protected_holdout_25_paired_outputs.json",
540   "sha256": "7a06366cae53d0e194471420115ffa342fa4ba3a865896a25eeb94dbd8cd0dbd"
541 },
542 {
543   "category": "result",
544   "lines": 26,
545   "path": "submission/program_listings/build/generated/results/protected_generation_summary.csv",
546   "sha256": "854e5d0a8611ca7854a1e3068fd79cc7f2c1bd18550a5cc8b0751f9cbb85eb13"
547 },
548 {
549   "category": "result",
550   "lines": 17,
551   "path": "submission/program_listings/build/generated/results/automatic_metrics_overall.csv",
552   "sha256": "8d8fab4a8c40024ea6686bf646244e8374bc3becece4989fe610d17c74f7aa38"
553 },
554 {
555   "category": "result",
556   "lines": 81,
557   "path": "submission/program_listings/build/generated/results/automatic_metrics_by_dataset.csv",
558   "sha256": "f97673dfee6b3a30f55f47b6d82f7e952070b0c8c5f4386dc01a12b8504ca0c1"
559 },
560 {
561   "category": "result",
562   "lines": 51,
563   "path": "submission/program_listings/build/generated/results/gpt56_annotation_summary.csv",
564   "sha256": "cc2d74a71e706b50e0d92fccbc775bc5b29f6a8dde6f4f3a4381f7d8774f0c41"
565 },
566 {
567   "category": "result",
568   "lines": 401,
569   "path": "submission/program_listings/build/generated/results/gpt56_category_counts.csv",
570   "sha256": "a83bf116a3b41d91fb0caf829e471b0ab9d711e7ffcc1ad7fab190e346890a64"
571 }
572 ]
```